

# Edeviser Complete Codebase Volume 03

Generated: 8/1/2026 | Author: Edeviser AI Reviewer

FILE: src/components/shared/ThemeToggle.tsx

TYPE: .tsx | LINES: 1-51 | SHA256: 68254fbfb026e8dcedd14f215a4fa322b4aba002caa54ac41d12be4111bf7576

```
1 | import { useTheme } from "@hooks/useTheme";
2 | import { Button } from "@components/ui/button";
3 | import { Sun, Moon, Monitor } from "lucide-react";
4 | import { useTranslation } from "react-i18next";
5 |
6 | /**
7 |  * Tri-state theme toggle (light / dark / system).
8 |  * Backed by useTheme hook which syncs to profiles.theme_preference.
9 |  *
10 |  * Design: ADR-01
11 |  * Requirements: 2.10
12 |  *
13 |  * @example
14 |  * <ThemeToggle />
15 |  */
16 | const ThemeToggle = () => {
17 |   const { t } = useTranslation("common");
18 |   const { themeMode, setThemeMode } = useTheme();
19 |
20 |   const themes = [
21 |     { mode: "light" as const, icon: Sun, label: t("theme.light") },
22 |     { mode: "dark" as const, icon: Moon, label: t("theme.dark") },
23 |     { mode: "system" as const, icon: Monitor, label: t("theme.system") },
24 |   ];
25 |
26 |   return (
27 |     <div className="flex items-center gap-1 p-1 bg-slate-100 rounded-lg">
28 |       {themes.map(({ mode, icon: Icon, label }) => (
29 |         <Button
30 |           key={mode}
31 |           variant={themeMode === mode ? "default" : "ghost"}
32 |           size="sm"
33 |           onClick={() => setThemeMode(mode)}
34 |           title={label}
35 |           aria-label={label}
36 |           aria-pressed={themeMode === mode}
37 |           className={`p-2 h-8 w-8 ${
38 |             themeMode === mode
39 |               ? "bg-white text-blue-600 shadow-sm"
40 |               : "text-gray-600 hover:text-gray-900"
41 |           }}
42 |         >
43 |           <Icon className="h-4 w-4" />
44 |         </Button>
45 |       ))}
46 |     </div>
47 |   );
48 | };
49 |
50 | export default ThemeToggle;
51 |
```

FILE: src/components/shared/TimetableGrid.tsx

TYPE: .tsx | LINES: 1-70 | SHA256: 716b8232af050b3a3c705169e2a887ed46911cac18660f331f02fbe438951f99

```
1 | // =====
2 | // TimetableGrid - Weekly timetable grid
3 | // =====
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | interface TimetableEntry {
8 |   id: string;
9 |   dayOfWeek: string;
10 |   startTime: string;
11 |   endTime: string;
12 |   courseName: string;
13 |   room: string;
14 |   color?: string;
15 | }
16 |
17 | interface TimetableGridProps {
18 |   entries: TimetableEntry[];
19 |   className?: string;
20 | }
21 |
22 | const DAYS = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"];
23 |
24 | const DAY_COLORS = [
25 |   "bg-blue-50 border-blue-200 text-blue-700",
26 |   "bg-green-50 border-green-200 text-green-700",
27 |   "bg-purple-50 border-purple-200 text-purple-700",
28 |   "bg-amber-50 border-amber-200 text-amber-700",
29 |   "bg-teal-50 border-teal-200 text-teal-700",
30 | ];
31 |
32 | const TimetableGrid = ({ entries, className }: TimetableGridProps) => (
33 |   <div className={cn("grid grid-cols-5 gap-2", className)}>
34 |     {DAYS.map((day, dayIdx) => (
35 |       <div key={day} className="space-y-2">
36 |         <h4 className="text-xs font-bold tracking-widest uppercase text-gray-500 text-center py-2">
37 |           {day.slice(0, 3)}
38 |         </h4>
```

```

39 | {entries
40 |   .filter((e) => e.dayOfWeek === day)
41 |   .sort((a, b) => a.startTime.localeCompare(b.startTime))
42 |   .map((entry) => (
43 |     <div
44 |       key={entry.id}
45 |       className={cn(
46 |         "rounded-lg border p-2 text-xs space-y-0.5",
47 |         entry.color ?? DAY_COLORS[dayIdx % DAY_COLORS.length]
48 |       )}
49 |     >
50 |       <p className="font-semibold truncate">{entry.courseName}</p>
51 |       <p className="opacity-70">
52 |         {entry.startTime} - {entry.endTime}
53 |       </p>
54 |       {entry.room} && <p className="opacity-60">{entry.room}</p>
55 |     </div>
56 |   )})
57 | {entries.filter((e) => e.dayOfWeek === day).length === 0 && (
58 |   <div className="rounded-lg border border-dashed border-gray-200 p-2 text-xs text-gray-300 text-center">
59 |     Free
60 |   </div>
61 | )})
62 | </div>
63 | )})
64 | </div>
65 | );
66 |
67 | export default TimetableGrid;
68 | export { DAYS };
69 | export type { TimetableGridProps, TimetableEntry };
70 |

```

FILE: src/components/shared/TodayTimeline.tsx

TYPE: .tsx | LINES: 1-334 | SHA256: 602c11f4f736d4016d69b35fbcffdb670745d026b04a4e600455602f5d715a5e

[illegible]

[illegible]

[illegible]

```

277 |     return items;
278 | }, [sessions, tasks, deadlines]);
279 |
280 | // Group items by time of day
281 | const grouped = useMemo(
282 |   () => groupByTimeOfDay(timelineItems),
283 |   [timelineItems]
284 | );
285 |
286 | const sectionOrder: Array<TimeOfDay | "todo"> = [
287 |   "morning",
288 |   "afternoon",
289 |   "evening",
290 |   "todo",
291 | ];
292 | const hasItems = sectionOrder.some((key) => grouped[key].length > 0);
293 |
294 | return (
295 |   <PCard className="overflow-hidden">
296 |     <div className="p-5 pb-4">
297 |       <SectionHeader icon={Clock} title="Today's Timeline" />
298 |     </div>
299 |     <div className="space-y-4 px-5 pb-5">
300 |       </* Timeline Sections */>
301 |       {hasItems ? (
302 |         sectionOrder.map((key) => (
303 |           <TimelineSection
304 |             key={key}
305 |             sectionKey={key}
306 |             items={grouped[key]}
307 |             onSessionStart={onSessionStart}
308 |             onSessionEdit={onSessionEdit}
309 |             onTaskToggle={onTaskToggle}
310 |             onTaskEdit={onTaskEdit}
311 |             onTaskDelete={onTaskDelete}
312 |           />
313 |         ))
314 |       ) : (
315 |         <div className="flex flex-col items-center justify-center py-8 text-center">
316 |           <div className="p-3 rounded-full bg-blue-50 mb-3">
317 |             <Clock className="h-8 w-8 text-blue-400" />
318 |           </div>
319 |           <p className="text-sm text-gray-500">
320 |             Nothing scheduled for today. Add a session or task to get started.
321 |           </p>
322 |         </div>
323 |       )}
324 |
325 |       </* Habit Status */>
326 |       <HabitStatusSection habits={habits} />
327 |     </div>
328 |   </PCard>
329 | );
330 | };
331 |
332 | export default TodayTimeline;
333 | export type { TodayTimelineProps };
334 |

```

FILE: [src/components/shared/ToAcceptanceDialog.tsx](#)

TYPE: .tsx | LINES: 1-117 | SHA256: 11711282928b97bc554ca47195a4614431871369a96271946f27045823608f19

```

1 | // Task 87.4: Terms of Service acceptance dialog
2 | // Shown when profile.tos_accepted_at is null - blocks usage until accepted
3 |
4 | import { useState } from "react";
5 | import { useTranslation } from "react-i18next";
6 | import {
7 |   Dialog,
8 |   DialogContent,
9 |   DialogHeader,
10 |   DialogTitle,
11 |   DialogDescription,
12 |   DialogFooter,
13 | } from "@components/ui/dialog";
14 | import { Button } from "@components/ui/button";
15 | import { Checkbox } from "@components/ui/checkbox";
16 | import { supabase } from "@lib/supabase";
17 | import { toast } from "sonner";
18 | import { Loader2 } from "lucide-react";
19 |
20 | interface ToAcceptanceDialogProps {
21 |   open: boolean;
22 |   userId: string;
23 |   onAccepted: () => void;
24 | }
25 |
26 | const ToAcceptanceDialog = ({
27 |   open,
28 |   userId,
29 |   onAccepted,
30 | }: ToAcceptanceDialogProps) => {
31 |   const { t } = useTranslation("common");
32 |   const [checked, setChecked] = useState(false);
33 |   const [isPending, setIsPending] = useState(false);
34 |
35 |   const handleAccept = async () => {
36 |     if (!checked) return;
37 |     setIsPending(true);
38 |
39 |     try {
40 |       const { error } = await supabase
41 |         .from("profiles")
42 |         .update({ tos_accepted_at: new Date().toISOString() })

```

```

43 |         .eq("id", userId);
44 |
45 |         if (error) {
46 |             toast.error(t("tos.acceptError"));
47 |             console.error("[ToSAcceptanceDialog]", error.message);
48 |             return;
49 |         }
50 |
51 |         toast.success(t("tos.acceptSuccess"));
52 |         onAccepted();
53 |     } catch (err) {
54 |         toast.error(t("tos.acceptError"));
55 |         console.error("[ToSAcceptanceDialog]", err);
56 |     } finally {
57 |         setIsPending(false);
58 |     }
59 | };
60 |
61 | return (
62 |     <Dialog open={open}>
63 |         <DialogContent showCloseButton={false} className="sm:max-w-md">
64 |             <DialogHeader>
65 |                 <DialogTitle>{t("tos.title")}</DialogTitle>
66 |                 <DialogDescription>{t("tos.description")}</DialogDescription>
67 |             </DialogHeader>
68 |
69 |             <div className="flex items-start gap-3 py-2">
70 |                 <Checkbox
71 |                     id="tos-checkbox"
72 |                     checked={checked}
73 |                     onCheckedChange={(v) => setChecked(v === true)}
74 |                 />
75 |                 <Label
76 |                     htmlFor="tos-checkbox"
77 |                     className="text-sm text-gray-700 leading-snug cursor-pointer"
78 |                 >
79 |                     {t("tos.checkboxLabel")}{ " " }
80 |                     <a
81 |                         href="/terms"
82 |                         target="_blank"
83 |                         rel="noopener noreferrer"
84 |                         className="text-blue-600 underline"
85 |                     >
86 |                         {t("tos.termsLink")}
87 |                     </a>{ " " }
88 |                     {t("tos.and")}{ " " }
89 |                     <a
90 |                         href="/privacy"
91 |                         target="_blank"
92 |                         rel="noopener noreferrer"
93 |                         className="text-blue-600 underline"
94 |                     >
95 |                         {t("tos.privacyLink")}
96 |                     </a>
97 |                 </Label>
98 |             </div>
99 |
100 |             <DialogFooter>
101 |                 <Button
102 |                     onClick={handleAccept}
103 |                     disabled={!checked || isPending}
104 |                     variant="tactile"
105 |                 >
106 |                     {isPending && <Loader2 className="h-4 w-4 animate-spin" />}
107 |                     {t("tos.acceptButton")}
108 |                 </Button>
109 |             </DialogFooter>
110 |         </DialogContent>
111 |     </Dialog>
112 | );
113 | };
114 | };
115 |
116 | export default ToSAcceptanceDialog;
117 |

```

## FILE: src/components/shared/TourRunner.tsx

TYPE: .tsx | LINES: 1-151 | SHA256: 5a144e85fd5df0f4beadeeca93800a049fb2e5a402c334170d037741ccc1cd5a

```

1 | // =====
2 | // TourRunner – the react-joyride render surface (Task 15.2, heavy-dep hygiene)
3 | // =====
4 | //
5 | // All `react-joyride` runtime usage (the <Joyride> component, CallBackProps,
6 | // STATUS) lives here so it can be code-split away from the always-mounted
7 | // GuidedTour. GuidedTour lazy-loads this component only when a tour actually
8 | // runs, so react-joyride is fetched on first tour use rather than on every
9 | // authenticated page load.
10 |
11 | import { useTranslation } from "react-i18next";
12 | import Joyride, { type CallBackProps, STATUS } from "react-joyride";
13 | import type { TourStep, TourCallbackData } from "@/hooks/useGuidedTour";
14 |
15 | interface TourRunnerProps {
16 |     steps: TourStep[];
17 |     run: boolean;
18 |     /** Forwarded to the useGuidedTour callback (completion/skip persistence). */
19 |     onCallback: (data: TourCallbackData) => void;
20 |     /** Invoked when the tour reaches a terminal (finished/skipped) state. */
21 |     onFinishOrSkip: () => void;
22 | }
23 |
24 | const TourRunner = ({
25 |     steps,

```

```

26 | run,
27 | onCallback,
28 | onFinishOrSkip,
29 | }: TourRunnerProps) => {
30 |   const { t } = useTranslation("common");
31 |
32 |   const handleJoyrideCallback = (data: CallBackProps) => {
33 |     const { status, action, index, type } = data;
34 |
35 |     // Map joyride callback to our hook's callback format
36 |     onCallback({
37 |       action,
38 |       index,
39 |       step: steps[index] ?? steps[0] ?? { target: "", content: "" },
40 |       type,
41 |       status: status as string,
42 |     });
43 |
44 |     // Stop tour on finish or skip
45 |     if (status === STATUS.FINISHED || status === STATUS.SKIPPED) {
46 |       onFinishOrSkip();
47 |     }
48 |   };
49 |
50 |   // We intentionally do NOT pass stepIndex here. The hook returns 0 as a
51 |   // placeholder, but passing it makes Joyride a controlled component that
52 |   // never advances on Next clicks. Letting Joyride manage stepIndex
53 |   // internally (uncontrolled) is what makes navigation work.
54 |   return (
55 |     <Joyride
56 |       steps={steps}
57 |       run={run}
58 |       callback={handleJoyrideCallback}
59 |       continuous
60 |       showSkipButton
61 |       showProgress
62 |       disableScrolling={false}
63 |       spotlightClicks={false}
64 |       styles={{
65 |         options: {
66 |           primaryColor: "#3b82f6",
67 |           zIndex: 10000,
68 |           arrowColor: "ffffff",
69 |           backgroundColor: "ffffff",
70 |           overlayColor: "rgba(15, 23, 42, 0.6)",
71 |           textColor: "#1e293b",
72 |           width: 400,
73 |         },
74 |         buttonNext: {
75 |           background:
76 |             "linear-gradient(93.65deg, #14B8A6 5.37%, #0382BD 78.89%)",
77 |           color: "ffffff",
78 |           borderRadius: "10px",
79 |           padding: "10px 20px",
80 |           fontSize: "14px",
81 |           fontWeight: "600",
82 |           border: "none",
83 |           boxShadow: "0 2px 8px rgba(20, 184, 166, 0.3)",
84 |           transition: "transform 100ms",
85 |         },
86 |         buttonBack: {
87 |           color: "#64748b",
88 |           marginInlineEnd: "8px",
89 |           fontSize: "14px",
90 |           fontWeight: "500",
91 |         },
92 |         buttonSkip: {
93 |           color: "#94a3b8",
94 |           fontSize: "13px",
95 |           fontWeight: "500",
96 |         },
97 |         buttonClose: {
98 |           width: "16px",
99 |           height: "16px",
100 |           color: "#94a3b8",
101 |         },
102 |         tooltip: {
103 |           borderRadius: "16px",
104 |           boxShadow:
105 |             "0 20px 50px -10px rgba(15, 23, 42, 0.25), 0 0 0 1px rgba(15, 23, 42, 0.05)",
106 |           padding: "24px",
107 |         },
108 |         tooltipContainer: {
109 |           textAlign: "start",
110 |         },
111 |         tooltipTitle: {
112 |           fontSize: "18px",
113 |           fontWeight: "700",
114 |           color: "#0f172a",
115 |           marginBottom: "8px",
116 |           letterSpacing: "-0.01em",
117 |         },
118 |         tooltipContent: {
119 |           fontSize: "14px",
120 |           color: "#475569",
121 |           lineHeight: "1.6",
122 |           padding: "8px 0 16px 0",
123 |         },
124 |         tooltipFooter: {
125 |           marginTop: "16px",
126 |           paddingTop: "16px",
127 |           borderTop: "1px solid #e2e8f0",
128 |         },
129 |         spotlight: {

```

```

130 |         borderRadius: "12px",
131 |         boxShadow:
132 |           "0 0 0 4px rgba(59, 130, 246, 0.15), 0 0 0 8px rgba(59, 130, 246, 0.08)",
133 |       },
134 |       beacon: {
135 |         // Hide default beacon – we use auto-launch
136 |       },
137 |     }
138 |   },
139 |   locale: {
140 |     back: t("tour.back", "Back"),
141 |     close: t("tour.close", "Close"),
142 |     last: t("tour.finish", "Finish"),
143 |     next: t("tour.next", "Next"),
144 |     open: t("tour.open", "Open the dialog"),
145 |     skip: t("tour.skip", "Skip tour"),
146 |   }
147 | }
148 | );
149 | };
150 | export default TourRunner;
151 |

```

## FILE: src/components/shared/TutorEntryButton.tsx

TYPE: .tsx | LINES: 1-74 | SHA256: fab564faf9da082223115aec22fa85d8d9edaab51763b7f7f825ee7a4f100227

```

1 | // =====
2 | // TutorEntryButton – Contextual "Ask Tutor" button for entry points
3 | // =====
4 |
5 | import { useNavigate } from "react-router-dom";
6 | import { Bot, MessageSquare } from "lucide-react";
7 | import { Button } from "@components/ui/button";
8 | import { cn } from "@lib/utils";
9 |
10 | interface TutorEntryButtonProps {
11 |   courseId?: string;
12 |   cloIds?: string[];
13 |   assignmentId?: string;
14 |   /** Compact mode shows just an icon button */
15 |   compact?: boolean;
16 |   /** Custom label text */
17 |   label?: string;
18 |   className?: string;
19 | }
20 |
21 | const TutorEntryButton = ({
22 |   courseId,
23 |   cloIds,
24 |   assignmentId,
25 |   compact = false,
26 |   label = "Ask Tutor",
27 |   className,
28 | }: TutorEntryButtonProps) => {
29 |   const navigate = useNavigate();
30 |
31 |   const handleClick = () => {
32 |     const params = new URLSearchParams();
33 |     if (courseId) params.set("courseId", courseId);
34 |     if (assignmentId) params.set("assignmentId", assignmentId);
35 |     if (cloIds && cloIds.length > 0) params.set("cloIds", cloIds.join(","));
36 |
37 |     const queryString = params.toString();
38 |     const path = `/student/tutor${queryString ? `? ${queryString}` : ""}`;
39 |     navigate(path);
40 |   };
41 |
42 |   if (compact) {
43 |     return (
44 |       <Button
45 |         variant="ghost"
46 |         size="icon-sm"
47 |         onClick={handleClick}
48 |         aria-label={label}
49 |         className={cn(
50 |           "text-teal-600 hover:text-teal-700 hover:bg-teal-50",
51 |           className
52 |         )}
53 |       >
54 |         <MessageSquare className="h-4 w-4" />
55 |       </Button>
56 |     );
57 |   }
58 |
59 |   return (
60 |     <Button
61 |       onClick={handleClick}
62 |       variant="tactile"
63 |       className={cn(className)}
64 |       aria-label={label}
65 |     >
66 |       <Bot className="h-4 w-4" />
67 |       {label}
68 |     </Button>
69 |   );
70 | };
71 |
72 | export default TutorEntryButton;
73 | export type { TutorEntryButtonProps };
74 |

```



# FILE: src/components/shared/UploadProgress.tsx

TYPE: .tsx | LINES: 1-96 | SHA256: e4583b1f542397527b35fdb27f999f40581e6a2828f35794b636ed2ab7619d66

```
1  // =====
2  // UploadProgress - File upload progress indicator
3  // =====
4  /* eslint-disable react-refresh/only-export-components */
5
6  import { CheckCircle, AlertCircle, X, RefreshCw, File } from "lucide-react";
7  import { Button } from "@components/ui/button";
8  import { cn } from "@lib/utls";
9
10 interface UploadProgressProps {
11   progress: number;
12   fileName: string;
13   fileSize: number;
14   status: "uploading" | "success" | "error";
15   onRetry?: () => void;
16   onCancel?: () => void;
17   className?: string;
18 }
19
20 function formatFileSize(bytes: number): string {
21   if (bytes < 1024) return `${bytes} B`;
22   if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
23   return `${(bytes / (1024 * 1024)).toFixed(1)} MB`;
24 }
25
26 const UploadProgress = ({
27   progress,
28   fileName,
29   fileSize,
30   status,
31   onRetry,
32   onCancel,
33   className,
34 }: UploadProgressProps) => {
35   const clamped = Math.max(0, Math.min(100, progress));
36
37   return (
38     <div
39       className={cn(
40         "rounded-xl border border-slate-200 bg-white p-4 space-y-2",
41         className
42       )}
43     >
44       <div className="flex items-center gap-3">
45         <File className="h-5 w-5 text-gray-400 shrink-0" />
46         <div className="flex-1 min-w-0">
47           <p className="text-sm font-medium truncate">{fileName}</p>
48           <p className="text-xs text-gray-400">{formatFileSize(fileSize)}</p>
49         </div>
50         {status === "uploading" && onCancel && (
51           <Button
52             variant="ghost"
53             size="sm"
54             onClick={onCancel}
55             className="h-8 w-8 p-0"
56             aria-label="Cancel upload"
57           >
58             <X className="h-4 w-4" />
59           </Button>
60         )}
61         {status === "success" && (
62           <CheckCircle className="h-5 w-5 text-green-500 shrink-0" />
63         )}
64         {status === "error" && (
65           <div className="flex items-center gap-1">
66             <AlertCircle className="h-5 w-5 text-red-500 shrink-0" />
67             {onRetry && (
68               <Button
69                 variant="ghost"
70                 size="sm"
71                 onClick={onRetry}
72                 className="h-8 w-8 p-0"
73                 aria-label="Retry upload"
74               >
75                 <RefreshCw className="h-4 w-4" />
76               </Button>
77             )}
78           </div>
69         )}
80       </div>
81       {status === "uploading" && (
82         <div className="h-1.5 w-full rounded-full bg-slate-100 overflow-hidden">
83           <div
84             className="h-full rounded-full bg-blue-500 transition-all duration-300"
85             style={{ width: `${clamped}%` }}
86           />
87         </div>
88       )}
89     </div>
90   );
91 };
92
93 export default UploadProgress;
94 export { formatFileSize };
95 export type { UploadProgressProps };
96
```

TYPE: .tsx | LINES: 1-567 | SHA256: c1941eb6387f1719397cbfafc184a5df952678d1f923948dc278cf714fc98872

```
// =====  
2 // WeeklyCalendarGrid - 7-column grid (Mon-Sun) with day headers, today  
3 // highlight, session/task/deadline/review cards per day; single-day view  
4 // on mobile with tab navigation  
5 //  
6 =====  
7 import { useState, useMemo } from "react";  
8 import { cn } from "@lib/utils";  
9 import { format, parseISO } from "date-fns";  
10 import { Button } from "@components/ui/button";  
11 import {  
12   ChevronLeft,  
13   ChevronRight,  
14   Play,  
15   SkipForward,  
16   CheckCircle2,  
17 } from "lucide-react";  
18 import StudySessionCard from "@components/shared/StudySessionCard";  
19 import PlannerTaskItem from "@components/shared/PlannerTaskItem";  
20 import DeadlineItem from "@components/shared/DeadlineItem";  
21 import SuggestedSessionItem from "@components/shared/SuggestedSessionItem";  
22 import { ReviewSessionBadge } from "@components/shared/ReviewSessionBadge";  
23 import { sortTasksByPriority } from "@lib/plannerUtils";  
24 import type {  
25   WeekDay,  
26   StudySession,  
27   PlannerTask,  
28   ReviewSchedule,  
29   SuggestedStudySession,  
30 } from "@types/planner";  
31  
32 interface WeeklyCalendarGridProps {  
33   weekData: WeekDay[];  
34   today: string;  
35   reviews?: ReviewSchedule[];  
36   onStartReview?: (review: ReviewSchedule) => void;  
37   onStopReview?: (reviewId: string) => void;  
38   onToggleTask?: (task: PlannerTask) => void;  
39   onDeleteTask?: (task: PlannerTask) => void;  
40   onClickDay?: (date: string) => void;  
41   onStartReview?: (review: ReviewSchedule) => void;  
42   onStopReview?: (reviewId: string) => void;  
43   onPlanSuggestion?: (suggestion: SuggestedStudySession) => void;  
44   readOnly?: boolean;  
45 }  
46  
47 const DAY_LABELS = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];  
48  
49 const formatDateHeader = (dateStr: string): string => {  
50   try {  
51     return format(parseISO(dateStr), "d");  
52   } catch {  
53     return dateStr;  
54   }  
55 };  
56  
57 const formatDateHeaderFull = (dateStr: string): string => {  
58   try {  
59     return format(parseISO(dateStr), "EEEE, MMM d");  
60   } catch {  
61     return dateStr;  
62   }  
63 };  
64  
65 /** Group reviews by reviewDate into a Map for O(1) lookup per day. */  
66 const groupReviewsByDate = (  
67   reviews: ReviewSchedule[],  
68 ): Map<string, ReviewSchedule[]> => {  
69   const map = new Map<string, ReviewSchedule[]>();  
70   for (const review of reviews) {  
71     const existing = map.get(review.reviewDate);  
72     if (existing) {  
73       existing.push(review);  
74     } else {  
75       map.set(review.reviewDate, [review]);  
76     }  
77   }  
78   return map;  
79 }  
80  
81 // % % % Review Item (shared between desktop & mobile) % % % % % % % % % % % % % % % % % % % % % % % % % %  
82  
83 interface ReviewItemProps {  
84   review: ReviewSchedule;  
85   onStartReview?: (review: ReviewSchedule) => void;  
86   onStopReview?: (reviewId: string) => void;  
87   readOnly?: boolean;  
88   compact?: boolean;  
89 }  
90  
91 const ReviewItem = ({  
92   review,  
93   onStartReview,  
94   onStopReview,  
95   readOnly = false,  
96   compact = false,  
97 }: ReviewItemProps) => {  
98   const isPending = review.status === "pending";  
99   const isCompleted = review.status === "completed";  
100   const isSkipped = review.status === "skipped";
```

[illegible]

```

206 |   onDayClick,
207 |   onStartReview,
208 |   onSkipReview,
209 |   onPlanSuggestion,
210 |   readOnly = false,
211 | }: DayColumnProps) => {
212 |   const sortedTasks = useMemo(
213 |     () => sortTasksByPriority(day.tasks),
214 |     [day.tasks]
215 |   );
216 |   const dayIndex = (() => {
217 |     try {
218 |       const d = parseISO(day.date);
219 |       const jsDay = d.getDay();
220 |       return jsDay === 0 ? 6 : jsDay - 1; // Mon=0 ... Sun=6
221 |     } catch {
222 |       return 0;
223 |     }
224 |   })();
225 |
226 |   return (
227 |     <div
228 |       className={cn(
229 |         "flex flex-col rounded-lg border p-2 min-h-[200px]",
230 |         day.isToday
231 |           ? "border-blue-400 bg-blue-50/30 ring-1 ring-blue-200"
232 |           : "border-gray-200 bg-white"
233 |       )}
234 |     >
235 |       {/* Day header */}
236 |       <button
237 |         type="button"
238 |         className={cn(
239 |           "mb-2 flex items-center justify-between rounded-md px-2 py-1 text-xs font-semibold transition-colors",
240 |           day.isToday
241 |             ? "bg-blue-600 text-white"
242 |             : "text-gray-600 hover:bg-gray-100"
243 |         )}
244 |         onClick={() => onDayClick?.(day.date)}
245 |         aria-label={`View ${formatDayHeaderFull(day.date)}`}
246 |       >
247 |         <span>{DAY_LABELS[dayIndex]}</span>
248 |         <span>{formatDayHeader(day.date)}</span>
249 |       </button>
250 |
251 |       {/* Items */}
252 |       <div className="flex-1 space-y-1.5 overflow-y-auto">
253 |         {/* Sessions */}
254 |         {day.sessions.map((session) => (
255 |           <StudySessionCard
256 |             key={session.id}
257 |             session={session}
258 |             onStart={readOnly ? undefined : onSessionStart}
259 |             onEdit={readOnly ? undefined : onSessionEdit}
260 |             compact
261 |           >
262 |         ))}
263 |
264 |         {/* Tasks (sorted by priority) */}
265 |         {sortedTasks.map((task) => (
266 |           <PlannerTaskItem
267 |             key={task.id}
268 |             task={task}
269 |             onToggle={readOnly ? undefined : onTaskToggle}
270 |             onEdit={readOnly ? undefined : onTaskEdit}
271 |             onDelete={readOnly ? undefined : onTaskDelete}
272 |             compact
273 |           >
274 |         ))}
275 |
276 |         {/* Reviews (after sessions/tasks, before deadlines) */}
277 |         {day.reviews.map((review) => (
278 |           <ReviewItem
279 |             key={review.id}
280 |             review={review}
281 |             onStartReview={onStartReview}
282 |             onSkipReview={onSkipReview}
283 |             readOnly={readOnly}
284 |             compact
285 |           >
286 |         ))}
287 |
288 |         {/* Deadlines */}
289 |         {day.deadlines.map((deadline) => (
290 |           <DeadlineItem key={deadline.id} deadline={deadline} compact />
291 |         ))}
292 |
293 |         {/* Suggested study session (empty days only) */}
294 |         {day.sessions.length === 0 &&
295 |           day.tasks.length === 0 &&
296 |           day.reviews.length === 0 &&
297 |           day.deadlines.length === 0 &&
298 |           (day.suggestions?.length ?? 0) > 0 &&
299 |           day.suggestions!.map((suggestion) => (
300 |             <SuggestedSessionItem
301 |               key={suggestion.id}
302 |               suggestion={suggestion}
303 |               onPlan={readOnly ? undefined : onPlanSuggestion}
304 |               compact
305 |             >
306 |           ))}
307 |
308 |         {/* Empty state – only when there is genuinely nothing to surface */}
309 |         {day.sessions.length === 0 &&

```

[illegible]

[illegible]

```

518 |   }: WeeklyCalendarGridProps) => {
519 |     const reviewsByDate = useMemo(() => groupReviewsByDate(reviews), [reviews]);
520 |
521 |     return (
522 |       <
523 |         { /* Desktop: 7-column grid */ }
524 |         <div className="hidden md:grid md:grid-cols-7 md:gap-2">
525 |           {weekData.map((day) => (
526 |             <DayColumn
527 |               key={day.date}
528 |               day={day}
529 |               dayReviews={reviewsByDate.get(day.date) ?? []}
530 |               onSessionStart={onSessionStart}
531 |               onSessionEdit={onSessionEdit}
532 |               onTaskToggle={onTaskToggle}
533 |               onTaskEdit={onTaskEdit}
534 |               onTaskDelete={onTaskDelete}
535 |               onDayClick={onDayClick}
536 |               onStartReview={onStartReview}
537 |               onSkipReview={onSkipReview}
538 |               onPlanSuggestion={onPlanSuggestion}
539 |               readOnly={readOnly}
540 |             </>
541 |           )}}
542 |         </div>
543 |
544 |         { /* Mobile: single-day view with tab navigation */ }
545 |         <div className="md:hidden">
546 |           <MobileDayView
547 |             weekData={weekData}
548 |             today={today}
549 |             reviewsByDate={reviewsByDate}
550 |             onSessionStart={onSessionStart}
551 |             onSessionEdit={onSessionEdit}
552 |             onTaskToggle={onTaskToggle}
553 |             onTaskEdit={onTaskEdit}
554 |             onTaskDelete={onTaskDelete}
555 |             onStartReview={onStartReview}
556 |             onSkipReview={onSkipReview}
557 |             onPlanSuggestion={onPlanSuggestion}
558 |             readOnly={readOnly}
559 |           </>
560 |         </div>
561 |       </>
562 |     );
563 |   };
564 |
565 |   export default WeeklyCalendarGrid;
566 |   export type { WeeklyCalendarGridProps };
567 |

```

## FILE: src/components/shared/WeeklyGoalPanel.tsx

TYPE: .tsx | LINES: 1-340 | SHA256: 2e0444332b84905afaeebff88f992aeadd5539d678b0376440df571fe4736bc1d

```

1 | // =====
2 | // WeeklyGoalPanel – Goal setting panel with up to 3 goal rows
3 | // (type select + target input), progress bars, edit/save toggle,
4 | // disabled for past weeks
5 | // =====
6 |
7 | import { useState, useCallback } from "react";
8 | import { Button } from "@components/ui/button";
9 | import { Input } from "@components/ui/input";
10 | import {
11 |   Select,
12 |   SelectContent,
13 |   SelectItem,
14 |   SelectTrigger,
15 |   SelectValue,
16 | } from "@components/ui/select";
17 | import { PCard, SectionHeader } from "@design-system";
18 | import { cn } from "@lib/utils";
19 | import type { GoalProgress, GoalType, WeeklyGoal } from "@types/planner";
20 | import type { CreateWeeklyGoalInput } from "@lib/schemas/planner";
21 | import {
22 |   Target,
23 |   Plus,
24 |   Pencil,
25 |   Save,
26 |   Trash2,
27 |   Loader2,
28 |   CheckCircle2,
29 |   Lightbulb,
30 | } from "lucide-react";
31 |
32 | interface ExampleGoalDisplay {
33 |   /** Localized goal-type label, e.g. "Study Hours". */
34 |   label: string;
35 |   /** Localized target text, e.g. "5 hours". */
36 |   targetText: string;
37 | }
38 |
39 | interface WeeklyGoalPanelProps {
40 |   goals: WeeklyGoal[];
41 |   progress: GoalProgress[];
42 |   weekStartDate: string;
43 |   onSave: (goals: CreateWeeklyGoalInput[]) => void;
44 |   isEditable?: boolean;
45 |   isPending?: boolean;
46 |   /**
47 |    * Example goals to display when the student has set none, guiding
48 |    * goal-setting (R19.4). Already localized by the consumer. When omitted, the
49 |    * panel falls back to its plain empty state.
50 |    */

```

```

51 | exampleGoals?: ExampleGoalDisplay[];
52 | /** Localized copy for the example-goals block. */
53 | exampleGoalsCopy?: { heading: string; hint: string; cta: string };
54 | }
55 |
56 | interface GoalDraft {
57 |   goalType: GoalType;
58 |   targetValue: number;
59 | }
60 |
61 | const GOAL_TYPE_LABELS: Record<GoalType, string> = {
62 |   study_hours: "Study Hours",
63 |   sessions_completed: "Sessions",
64 |   tasks_completed: "Tasks",
65 | };
66 |
67 | const GOAL_TYPE_UNITS: Record<GoalType, string> = {
68 |   study_hours: "hours",
69 |   sessions_completed: "sessions",
70 |   tasks_completed: "tasks",
71 | };
72 |
73 | const WeeklyGoalPanel = ({
74 |   goals,
75 |   progress,
76 |   weekStartDate,
77 |   onSave,
78 |   isEditable = true,
79 |   isPending = false,
80 |   exampleGoals,
81 |   exampleGoalsCopy,
82 | }: WeeklyGoalPanelProps) => {
83 |   const [isEditing, setIsEditing] = useState(false);
84 |   const [drafts, setDrafts] = useState<GoalDraft[]>([]);
85 |
86 |   const startEditing = useCallback(() => {
87 |     setDrafts(
88 |       goals.length > 0
89 |       ? goals.map((g) => ({
90 |         goalType: g.goalType,
91 |         targetValue: g.targetValue,
92 |       }))
93 |       : [{ goalType: "study_hours", targetValue: 5 }]
94 |     );
95 |     setIsEditing(true);
96 |   }, [goals]);
97 |
98 |   const handleSave = () => {
99 |     const validDrafts = drafts.filter((d) => d.targetValue > 0);
100 |     if (validDrafts.length === 0) return;
101 |
102 |     onSave(
103 |       validDrafts.map((d) => ({
104 |         weekStartDate,
105 |         goalType: d.goalType,
106 |         targetValue: d.targetValue,
107 |       }))
108 |     );
109 |     setIsEditing(false);
110 |   };
111 |
112 |   const addGoal = () => {
113 |     if (drafts.length >= 3) return;
114 |     const usedTypes = new Set(drafts.map((d) => d.goalType));
115 |     const availableType =
116 |       (
117 |         ["study_hours", "sessions_completed", "tasks_completed"] as GoalType[]
118 |       ).find((t) => !usedTypes.has(t)) ?? "study_hours";
119 |     setDrafts([...drafts, { goalType: availableType, targetValue: 3 }]);
120 |   };
121 |
122 |   const removeGoal = (index: number) => {
123 |     setDrafts(drafts.filter((_, i) => i !== index));
124 |   };
125 |
126 |   const updateDraft = (index: number, updates: Partial<GoalDraft>) => {
127 |     setDrafts(drafts.map((d, i) => (i === index ? { ...d, ...updates } : d)));
128 |   };
129 |
130 |   // The header renders as a white strip, so the header action buttons use a
131 |   // matching dark text color.
132 |   const headerActionClass = cn(
133 |     "h-7 gap-1 text-xs",
134 |     "text-gray-600 hover:bg-gray-100 hover:text-gray-900"
135 |   );
136 |
137 |   return (
138 |     <PCard className="overflow-hidden">
139 |       <div className="p-5 pb-4">
140 |         <SectionHeader
141 |           icon={Target}
142 |           title="Weekly Goals"
143 |           action={
144 |             isEditable && !isEditing ? (
145 |               <Button
146 |                 variant="ghost"
147 |                 size="sm"
148 |                 className={headerActionClass}
149 |                 onClick={startEditing}
150 |               />
151 |               <Pencil className="h-3.5 w-3.5" />
152 |               Edit
153 |             ) : isEditing ? (
154 |

```



```

155 |         <Button
156 |             variant="ghost"
157 |             size="sm"
158 |             className={headerActionClass}
159 |             onClick={handleSave}
160 |             disabled={isPending}
161 |         >
162 |             {isPending ? (
163 |                 <Loader2 className="h-3.5 w-3.5 animate-spin" />
164 |             ) : (
165 |                 <Save className="h-3.5 w-3.5" />
166 |             )}
167 |         </Button>
168 |     ) : null
169 | }
170 | />
171 | </div>
172 |
173 | <div className="space-y-4 px-5 pb-5">
174 |     {/* Editing mode */}
175 |     {isEditing ? (
176 |         <>
177 |             {drafts.map((draft, index) => (
178 |                 <div key={index} className="flex items-center gap-2">
179 |                     <Select
180 |                         value={draft.goalType}
181 |                         onChange={v =>
182 |                             updateDraft(index, { goalType: v as GoalType })
183 |                         }
184 |                     >
185 |                         <SelectTrigger className="w-40">
186 |                             <SelectValue />
187 |                         </SelectTrigger>
188 |                         <SelectContent>
189 |                             <SelectItem value="study_hours">Study Hours</SelectItem>
190 |                             <SelectItem value="sessions_completed">Sessions</SelectItem>
191 |                             <SelectItem value="tasks_completed">Tasks</SelectItem>
192 |                         </SelectContent>
193 |                     </Select>
194 |                     <Input
195 |                         type="number"
196 |                         min={1}
197 |                         value={draft.targetValue}
198 |                         onChange={e =>
199 |                             updateDraft(index, {
200 |                                 targetValue: Number(e.target.value) || 0,
201 |                             })
202 |                         }
203 |                     >
204 |                         <span className="text-xs text-gray-500">
205 |                             {GOAL_TYPE_UNITS[draft.goalType]}
206 |                         </span>
207 |                     <Button
208 |                         variant="ghost"
209 |                         size="sm"
210 |                         className="h-7 w-7 p-0 shrink-0"
211 |                         onClick={() => removeGoal(index)}
212 |                     >
213 |                         <Trash2 className="h-3.5 w-3.5 text-red-500" />
214 |                     </Button>
215 |                 </div>
216 |             ))}
217 |             {drafts.length < 3 && (
218 |                 <Button
219 |                     variant="outline"
220 |                     size="sm"
221 |                     className="h-8 gap-1 text-xs"
222 |                     onClick={addGoal}
223 |                 >
224 |                     <Plus className="h-3.5 w-3.5" />
225 |                     Add Goal
226 |                 </Button>
227 |             )}
228 |         </>
229 |     ) : goals.length === 0 ? (
230 |         {/* Empty state */}
231 |         exampleGoals && exampleGoals.length > 0 ? (
232 |             <div className="space-y-3">
233 |                 <div className="flex items-start gap-2 rounded-lg border border-dashed border-teal-300 bg-teal-5">
234 |                     <Lightbulb className="mt-0.5 h-4 w-4 shrink-0 text-teal-500" />
235 |                     <div className="min-w-0">
236 |                         <p className="text-sm font-semibold text-gray-800">
237 |                             {exampleGoalsCopy?.heading ?? "Goal ideas to get started"}
238 |                         </p>
239 |                         <p className="mt-0.5 text-xs text-gray-500">
240 |                             {exampleGoalsCopy?.hint ??
241 |                                 "Set a goal below to track your week."}
242 |                         </p>
243 |                     </div>
244 |                 </div>
245 |             </div>
246 |         ) : (
247 |             <ul className="space-y-2">
248 |                 {exampleGoals.map((example, index) => (
249 |                     <li
250 |                         key={index}
251 |                         className="flex items-center justify-between rounded-lg bg-slate-50 px-3 py-2"
252 |                     >
253 |                         <span className="text-sm font-medium text-gray-700">
254 |                             {example.label}
255 |                         </span>
256 |                         <span className="text-xs text-gray-500">
257 |                             {example.targetText}
258 |                         </span>

```

```

259 |         </span>
260 |       </li>
261 |     )}}
262 |   </ul>
263 |
264 |   {isEditable && (
265 |     <Button
266 |       variant="outline"
267 |       size="sm"
268 |       className="h-8 w-full gap-1 text-xs"
269 |       onClick={startEditing}
270 |     >
271 |       <Plus className="h-3.5 w-3.5" />
272 |       {exampleGoalsCopy?.cta ?? "Set Goals"}
273 |     </Button>
274 |   )}
275 | </div>
276 | ) : (
277 |   <div className="text-center py-4">
278 |     <Target className="mx-auto h-8 w-8 text-gray-300" />
279 |     <p className="mt-2 text-sm text-gray-500">
280 |       No goals set for this week
281 |     </p>
282 |     {isEditable && (
283 |       <Button
284 |         variant="outline"
285 |         size="sm"
286 |         className="mt-3 h-8 gap-1 text-xs"
287 |         onClick={startEditing}
288 |       >
289 |         <Plus className="h-3.5 w-3.5" />
290 |         Set Goals
291 |       </Button>
292 |     )}
293 |   </div>
294 | )
295 | ) : (
296 |   /* Display mode with progress */
297 |   progress.map((gp) => (
298 |     <div key={gp.goal.id} className="space-y-1.5">
299 |       <div className="flex items-center justify-between">
300 |         <span className="text-sm font-medium text-gray-700">
301 |           {GOAL_TYPE_LABELS[gp.goal.goalType]}
302 |         </span>
303 |         <div className="flex items-center gap-1.5">
304 |           <span className="text-xs text-gray-500">
305 |             {gp.goal.goalType === "study_hours"
306 |               ? `${gp.currentValue.toFixed(1)} / ${
307 |                 gp.goal.targetValue
308 |               }h`
309 |               : `${gp.currentValue} / ${gp.goal.targetValue}`}
310 |           </span>
311 |           {gp.isMet && (
312 |             <CheckCircle2 className="h-4 w-4 text-green-500" />
313 |           )}
314 |         </div>
315 |       </div>
316 |       /* Progress bar */
317 |       <div className="h-2 w-full overflow-hidden rounded-full bg-gray-100">
318 |         <div
319 |           className={cn(
320 |             "h-full rounded-full transition-all duration-500",
321 |             gp.isMet
322 |               ? "bg-green-500"
323 |               : gp.percentage >= 50
324 |                 ? "bg-blue-500"
325 |                 : "bg-amber-500"
326 |             )}
327 |           style={{ width: `${Math.min(gp.percentage, 100)}%` }}
328 |         />
329 |       </div>
330 |     </div>
331 |   ))
332 | )}
333 | </div>
334 | </PCard>
335 | );
336 | };
337 |
338 | export default WeeklyGoalPanel;
339 | export type { WeeklyGoalPanelProps, ExampleGoalDisplay };
340 |

```

## FILE: src/components/shared/WeeklyReflectionPanel.tsx

TYPE: .tsx | LINES: 1-172 | SHA256: f3ccb07cdca4cc5ca85acd54d2c0dd9b0fba74f65705af4d7ad034b057d760d1

```

1 | // =====
2 | // WeeklyReflectionPanel – Textarea with live word count (min 50 words),
3 | // save button, creates journal_entries record on save.
4 | // Supports reflection templates and streak indicator.
5 | // =====
6 |
7 | import { useCallback, useMemo } from "react";
8 | import { Card } from "@components/ui/card";
9 | import { Button } from "@components/ui/button";
10 | import { GradientCardHeader } from "@design-system";
11 | import { cn } from "@lib/utils";
12 | import { countWords } from "@lib/plannerUtils";
13 | import { BookOpen, Loader2, Save } from "lucide-react";
14 | import { Textarea } from "@components/ui/textarea";
15 | import { useReflectionTemplates } from "@hooks/useReflectionTemplates";
16 | import ReflectionTemplateSelector from "@components/shared/ReflectionTemplateSelector";
17 | import SimpleReflectionTemplate from "@components/shared/SimpleReflectionTemplate";
18 | import GibbsReflectionTemplate from "@components/shared/GibbsReflectionTemplate";

```

[illegible]

```

123 |         !hasContent
124 |         ? "text-gray-400"
125 |         : meetsMinimum
126 |         ? "text-green-600"
127 |         : "text-red-500"
128 |     ))
129 |     aria-live="polite"
130 |     data-testid="word-count-indicator"
131 | >
132 |     {wordCount} / {MIN_WORDS} words
133 |     {hasContent && !meetsMinimum && (
134 |       <span className="ms-1">
135 |         ({MIN_WORDS - wordCount} more needed)
136 |       </span>
137 |     )}
138 |     {meetsMinimum && <span className="ms-1">'</span>}
139 | </div>
140 |
141 | {/* Save Button */}
142 | <Button
143 |   type="button"
144 |   size="sm"
145 |   variant="tactile"
146 |   className="h-9 gap-1.5"
147 |   onClick={handleSave}
148 |   disabled={!meetsMinimum || isPending || disabled}
149 |   data-testid="save-reflection-button"
150 | >
151 |   {isPending ? (
152 |     <Loader2 className="h-3.5 w-3.5 animate-spin" />
153 |   ) : (
154 |     <Save className="h-3.5 w-3.5" />
155 |   )}
156 |   Save Reflection
157 | </Button>
158 | </div>
159 |
160 | {/* XP hint */}
161 | <p className="text-xs text-gray-400">
162 |   Saving your weekly reflection creates a journal entry and awards 20
163 |   XP. Quality reflections earn bonus XP.
164 | </p>
165 | </div>
166 | </Card>
167 | );
168 | };
169 |
170 | export default WeeklyReflectionPanel;
171 | export type { WeeklyReflectionPanelProps };
172 |

```

#### FILE: [src/components/shared/WelcomeHero.ts](#)

TYPE: .ts | LINES: 1-5 | SHA256: c503cadaeb76e4adbcbb8a7751a64bdd76c98c3ba274a15dde80da81f3555fcb

```

1 | // Re-export shim – implementation lives in the design system (single source).
2 | // Legacy direct importers keep working; deleted at P5 once none remain.
3 | export { default } from "@design-system/patterns/WelcomeHero";
4 | export type { WelcomeHeroProps } from "@design-system/patterns/WelcomeHero";
5 |

```

#### FILE: [src/components/shared/WelcomeHero.tsx](#)

TYPE: .tsx | LINES: 1-82 | SHA256: e20cf49a10576d5f7048bd8db1bbd9277705c55160fb4d7401c10cc2ed264c1a

```

1 | import { useTranslation } from "react-i18next";
2 | import { getDisplayFirstName } from "@lib/displayName";
3 | import type { ReactNode } from "react";
4 | import type { UserRole } from "@types/app";
5 | import { PCard } from "@design-system";
6 |
7 | export interface WelcomeHeroProps {
8 |   /** User's first name or full name */
9 |   name: string;
10 |   /** User's role (admin, coordinator, teacher, student, parent) */
11 |   userRole: UserRole;
12 |   /** Subtitle/motivational text */
13 |   subtitle: string;
14 |   /** Optional stats section to render on the right (e.g., XP/Level chips for Student) */
15 |   stats?: ReactNode;
16 | }
17 |
18 | /**
19 |  * Universal welcome hero card for all role dashboards.
20 |  * Renders a dark gradient hero with personalized greeting, role context, and optional stats.
21 |  *
22 |  * Design: ADR-07, §8.1
23 |  * Requirements: 2.21
24 |  *
25 |  * @example
26 |  * <WelcomeHero
27 |  *   name="John"
28 |  *   userRole="student"
29 |  *   subtitle="Keep up the momentum!"
30 |  *   stats={<div>XP: 1500</div>}
31 |  * />
32 | */
33 | const WelcomeHero = ({
34 |   name,
35 |   userRole: _userRole,
36 |   subtitle,
37 |   stats,
38 | }: WelcomeHeroProps) => {
39 |   const { t } = useTranslation("common");
40 |
41 |   // Determine greeting based on time of day

```

```

42 | const greeting = (() => {
43 |   const hour = new Date().getHours();
44 |   if (hour < 12) return t("greeting.morning", "Good morning");
45 |   if (hour < 17) return t("greeting.afternoon", "Good afternoon");
46 |   return t("greeting.evening", "Good evening");
47 | })();
48 |
49 | // Extract a display first name. Seeded names carry honorific titles
50 | // ("Mr. David Okonkwo", "Dr. Aisha Al-Mansoori"), so the shared helper skips
51 | // a leading honorific token to avoid greeting users as "Mr."/"Dr.". Empty
52 | // names fall back to the translated "there".
53 | const firstName = getDisplayName(name) ?? t("greeting.there", "there");
54 |
55 | return (
56 |   <PCard
57 |     role="region"
58 |     aria-label={` ${greeting}, ${firstName}`}
59 |     className="overflow-hidden text-white"
60 |     style={{
61 |       background:
62 |         "linear-gradient(135deg, #0f172a 0%, #1e3a8a 50%, #312e81 100%)",
63 |     }}
64 |     data-tour="welcome-hero"
65 |   >
66 |     <div className="p-6 flex items-center justify-between">
67 |       <div>
68 |         <h1 className="text-2xl font-bold tracking-tight">
69 |           {greeting}, {firstName} Ø=ÜK
70 |         </h1>
71 |         <p className="text-sm text-white/70 mt-1">{subtitle}</p>
72 |       </div>
73 |       {stats && (
74 |         <div className="hidden md:flex items-center gap-4">{stats}</div>
75 |       )}
76 |     </div>
77 |   </PCard>
78 | );
79 | };
80 |
81 | export default WelcomeHero;
82 |

```

## FILE: src/components/shared/WellnessGoalInput.tsx

TYPE: .tsx | LINES: 1-133 | SHA256: 3fa7fd23eafa9cbf28e2b09310407510139de4062a07e6e29775bad87b75a26d

```

1 | import { useState } from "react";
2 | import { Target, Check } from "lucide-react";
3 | import { Input } from "@components/ui/input";
4 | import { Button } from "@components/ui/button";
5 | import { Label } from "@components/ui/label";
6 | import { cn } from "@lib/utills";
7 | import { WELLNESS_UNITS } from "@lib/wellnessTips";
8 | import type { WellnessHabitType, WellnessTarget } from "@types/habits";
9 |
10 | const HABIT_LABELS: Record<WellnessHabitType, string> = {
11 |   meditation: "Meditation",
12 |   hydration: "Hydration",
13 |   exercise: "Exercise",
14 |   sleep: "Sleep",
15 | };
16 |
17 | export interface WellnessGoalInputProps {
18 |   habitType: WellnessHabitType;
19 |   target: WellnessTarget | null;
20 |   progress: number;
21 |   currentValue: number;
22 |   onSave: (target: WellnessTarget) => void;
23 | }
24 |
25 | const WellnessGoalInput = ({
26 |   habitType,
27 |   target,
28 |   progress,
29 |   currentValue,
30 |   onSave,
31 | }: WellnessGoalInputProps) => {
32 |   const unit = target?.unit ?? WELLNESS_UNITS[habitType];
33 |   const [editValue, setEditValue] = useState<string>({
34 |     target?.targetValue?.toString() ?? ""
35 |   });
36 |   const [isEditing, setIsEditing] = useState(!target);
37 |
38 |   const targetMet = progress >= 100;
39 |
40 |   const handleSave = () => {
41 |     const numVal = parseFloat(editValue);
42 |     if (isNaN(numVal) || numVal <= 0) return;
43 |     onSave({ habitType, targetValue: numVal, unit });
44 |     setIsEditing(false);
45 |   };
46 |
47 |   return (
48 |     <div
49 |       data-testid={`wellness-goal-${habitType}`}
50 |       className="space-y-2 rounded-xl border bg-white p-3 shadow-sm"
51 |     >
52 |       <div className="flex items-center justify-between">
53 |         <div className="flex items-center gap-2">
54 |           <Target className="h-4 w-4 text-gray-500" />
55 |           <Label className="text-sm font-medium">
56 |             {HABIT_LABELS[habitType]} Goal
57 |           </Label>
58 |         </div>
59 |

```

```

60 |         {targetMet && (
61 |             <div
62 |                 data-testid={`goal-met-${habitType}`}
63 |                 className="flex items-center gap-1 text-green-600"
64 |             >
65 |                 <Check className="h-4 w-4" />
66 |                 <span className="text-xs font-medium">Target met</span>
67 |             </div>
68 |         )}
69 |     </div>
70 |
71 |     {isEditing ? (
72 |         <div className="flex items-center gap-2">
73 |             <Input
74 |                 data-testid={`goal-input-${habitType}`}
75 |                 type="number"
76 |                 min={1}
77 |                 placeholder="Target"
78 |                 value={editValue}
79 |                 onChange={(e) => setEditValue(e.target.value)}
80 |                 className="w-24 h-8 text-xs"
81 |                 aria-label={` ${HABIT_LABELS[habitType]} target in ${unit}`}
82 |             />
83 |             <span className="text-xs text-gray-500">{unit}</span>
84 |             <Button
85 |                 data-testid={`goal-save-${habitType}`}
86 |                 variant="outline"
87 |                 size="sm"
88 |                 onClick={handleSave}
89 |                 className="h-8 text-xs"
90 |             >
91 |                 Save
92 |             </Button>
93 |         </div>
94 |     ) : (
95 |         <div className="flex items-center justify-between text-xs text-gray-600">
96 |             <span data-testid={`goal-progress-label-${habitType}`}>
97 |                 {currentValue}/{target?.targetValue ?? 0} {unit}
98 |             </span>
99 |             <Button
100 |                 data-testid={`goal-edit-${habitType}`}
101 |                 variant="ghost"
102 |                 size="sm"
103 |                 onClick={() => setIsEditing(true)}
104 |                 className="text-blue-600 hover:text-blue-800 text-xs font-medium h-auto p-0"
105 |             >
106 |                 Edit
107 |             </Button>
108 |         </div>
109 |
110 |         { /* Progress bar */ }
111 |         <div
112 |             data-testid={`goal-progress-bar-${habitType}`}
113 |             className="h-2 w-full rounded-full bg-gray-100 overflow-hidden"
114 |         >
115 |             <div
116 |                 className={cn(
117 |                     "h-full rounded-full transition-all duration-300",
118 |                     targetMet
119 |                     ? "bg-gradient-to-r from-green-400 to-green-500"
120 |                     : "bg-gradient-to-r from-teal-400 to-blue-500"
121 |                 )}
122 |                 style={{ width: `${Math.min(progress, 100)}%` }}
123 |             />
124 |         </div>
125 |     )}
126 | </div>
127 | )}
128 | </div>
129 | );
130 | };
131 |
132 | export default WellnessGoalInput;
133 |

```

FILE: [src/components/shared/WellnessHabitLogger.tsx](#)

TYPE: .tsx | LINES: 1-183 | SHA256: 54f2e76e7e8b1c088b92764b8593776b6465010cb14203c8427a67e9141d4c4b

```

1 | import { useState } from "react";
2 | import { Brain, Droplets, Dumbbell, Moon, Check } from "lucide-react";
3 | import { Switch } from "@components/ui/switch";
4 | import { Input } from "@components/ui/input";
5 | import { Label } from "@components/ui/label";
6 | import { cn } from "@lib/utils";
7 | import type { WellnessHabitType, WellnessHabitLog } from "@types/habits";
8 |
9 | export interface WellnessHabitLoggerProps {
10 |     enabledHabits: WellnessHabitType[];
11 |     todayLogs: WellnessHabitLog[];
12 |     onLog: (type: WellnessHabitType, value?: number) => void;
13 |     dailyProgress?: Record<
14 |         string,
15 |         { progress: number; logged: number; target: number; unit: string }
16 |     >;
17 | }
18 |
19 | const HABIT_CONFIG: Record<
20 |     WellnessHabitType,
21 |     { label: string; description: string; unit: string; icon: typeof Brain }
22 | > = {
23 |     meditation: {
24 |         label: "Meditation",
25 |         description: "5+ minute session",
26 |         unit: "minutes",

```

```

27 |     icon: Brain,
28 |   },
29 |   hydration: {
30 |     label: "Hydration",
31 |     description: "8 glasses of water",
32 |     unit: "glasses",
33 |     icon: Droplets,
34 |   },
35 |   exercise: {
36 |     label: "Exercise",
37 |     description: "30+ minute activity",
38 |     unit: "minutes",
39 |     icon: Dumbbell,
40 |   },
41 |   sleep: {
42 |     label: "Sleep",
43 |     description: "7+ hours logged",
44 |     unit: "hours",
45 |     icon: Moon,
46 |   },
47 | };
48 |
49 | const WellnessHabitLogger = ({
50 |   enabledHabits,
51 |   todayLogs,
52 |   onLog,
53 |   dailyProgress,
54 | }: WellnessHabitLoggerProps) => {
55 |   const [values, setValues] = useState<
56 |     Partial<Record<WellnessHabitType, string>>
57 |   >({});
58 |
59 |   const isLogged = (type: WellnessHabitType): boolean =>
60 |     todayLogs.some((log) => log.wellnessType === type);
61 |
62 |   const handleToggle = (type: WellnessHabitType) => {
63 |     if (isLogged(type)) return;
64 |     const raw = values[type];
65 |     const numericValue = raw ? parseFloat(raw) : undefined;
66 |     onLog(
67 |       type,
68 |       numericValue && !isNaN(numericValue) ? numericValue : undefined
69 |     );
70 |   };
71 |
72 |   const handleValueChange = (type: WellnessHabitType, val: string) => {
73 |     setValues((prev) => ({ ...prev, [type]: val }));
74 |   };
75 |
76 |   if (enabledHabits.length === 0) return null;
77 |
78 |   return (
79 |     <div data-testid="wellness-habit-logger" className="space-y-3">
80 |       <h3 className="text-sm font-bold tracking-wide uppercase text-gray-500">
81 |         Wellness
82 |       </h3>
83 |       <div className="space-y-2">
84 |         {enabledHabits.map((type) => {
85 |           const config = HABIT_CONFIG[type];
86 |           const icon = config.icon;
87 |           const logged = isLogged(type);
88 |           const prog = dailyProgress?.[type];
89 |
90 |           return (
91 |             <div
92 |               key={type}
93 |               data-testid={`wellness-habit-${type}`}
94 |               className={cn(
95 |                 "flex items-center gap-3 rounded-xl border bg-white p-3 shadow-sm transition-colors",
96 |                 logged && "bg-green-50 border-green-200"
97 |               )}
98 |             >
99 |               <div
100 |                 className={cn(
101 |                   "p-2 rounded-lg",
102 |                   logged ? "bg-green-100" : "bg-slate-50"
103 |                 )}
104 |               >
105 |                 <Icon
106 |                   className={cn(
107 |                     "h-4 w-4",
108 |                     logged ? "text-green-600" : "text-slate-500"
109 |                   )}
110 |                 />
111 |               </div>
112 |
113 |               <div className="flex-1 min-w-0">
114 |                 <Label className="text-sm font-medium">{config.label}</Label>
115 |                 <p className="text-xs text-gray-500">{config.description}</p>
116 |                 {prog && (
117 |                   <div className="mt-1">
118 |                     <div className="flex items-center justify-between text-xs text-gray-500 mb-0.5">
119 |                       <span data-testid={`wellness-progress-${type}`}>
120 |                         {prog.logged}/{prog.target} {prog.unit}
121 |                       </span>
122 |                       {prog.progress >= 100 && (
123 |                         <span className="text-green-600 font-medium text-xs">
124 |                           Target met
125 |                         </span>
126 |                       )}
127 |                     </div>
128 |                     <div className="h-1.5 w-full rounded-full bg-gray-100 overflow-hidden">
129 |                       <div
130 |                         data-testid={`wellness-progress-bar-${type}`}

```

```

131 |         className={cn(
132 |             "h-full rounded-full transition-all duration-300",
133 |             prog.progress >= 100
134 |             ? "bg-green-400"
135 |             : "bg-gradient-to-r from-teal-400 to-blue-500"
136 |         )}
137 |         style={{ width: `${Math.min(prog.progress, 100)}%` }}
138 |     />
139 | </div>
140 | </div>
141 | )}
142 | </div>
143 |
144 | <div className="flex items-center gap-2">
145 |   <Input
146 |     data-testid={`wellness-value-${type}`}
147 |     type="number"
148 |     min={0}
149 |     placeholder={config.unit}
150 |     value={values[type] ?? ""}
151 |     onChange={(e) => handleValueChange(type, e.target.value)}
152 |     disabled={logged}
153 |     className="w-20 h-8 text-xs"
154 |     aria-label={` ${config.label} value in ${config.unit}`}
155 |   />
156 |
157 |   {logged ? (
158 |     <div
159 |       data-testid={`wellness-check-${type}`}
160 |       className="flex items-center justify-center h-5 w-9"
161 |       aria-label={` ${config.label} completed`}
162 |     >
163 |       <Check className="h-5 w-5 text-green-600" />
164 |     </div>
165 |   ) : (
166 |     <Switch
167 |       data-testid={`wellness-toggle-${type}`}
168 |       checked={false}
169 |       onCheckedChange={() => handleToggle(type)}
170 |       aria-label={`Log ${config.label}`}
171 |     />
172 |   )}
173 | </div>
174 | </div>
175 |   );
176 | }
177 | </div>
178 | </div>
179 | );
180 | };
181 |
182 | export default WellnessHabitLogger;
183 |

```

**FILE:** [src/components/shared/WellnessReminderSettings.tsx](#)

TYPE: .tsx | LINES: 1-79 | SHA256: ac816f4de305c062967d988a3652e127ef302e8b80b62d73b451d05437dfffc3c

```

1 | import { Bell } from "lucide-react";
2 | import { Switch } from "@components/ui/switch";
3 | import { Input } from "@components/ui/input";
4 | import { Label } from "@components/ui/label";
5 | import { cn } from "@lib/utils";
6 | import type { WellnessHabitType, WellnessReminderConfig } from "@types/habits";
7 |
8 | const HABIT_LABELS: Record<WellnessHabitType, string> = {
9 |   meditation: "Meditation",
10 |   hydration: "Hydration",
11 |   exercise: "Exercise",
12 |   sleep: "Sleep",
13 | };
14 |
15 | export interface WellnessReminderSettingsProps {
16 |   reminders: WellnessReminderConfig[];
17 |   onToggle: (habitType: WellnessHabitType, enabled: boolean) => void;
18 |   onTimeChange: (habitType: WellnessHabitType, time: string) => void;
19 | }
20 |
21 | const WellnessReminderSettings = ({
22 |   reminders,
23 |   onToggle,
24 |   onTimeChange,
25 | }: WellnessReminderSettingsProps) => {
26 |   if (reminders.length === 0) return null;
27 |
28 |   return (
29 |     <div data-testid="wellness-reminder-settings" className="space-y-3">
30 |       <div className="flex items-center gap-2">
31 |         <Bell className="h-4 w-4 text-gray-500" />
32 |         <h4 className="text-sm font-bold tracking-wide uppercase text-gray-500">
33 |           Reminders
34 |         </h4>
35 |       </div>
36 |
37 |       <div className="space-y-2">
38 |         {reminders.map((reminder) => (
39 |           <div
40 |             key={reminder.habitType}
41 |             data-testid={`reminder-${reminder.habitType}`}
42 |             className="flex items-center gap-3 rounded-xl border bg-white p-3 shadow-sm"
43 |           >
44 |             <div className="flex-1 min-w-0">
45 |               <Label className="text-sm font-medium">
46 |                 {HABIT_LABELS[reminder.habitType]}
47 |               </Label>

```



```

48 |         </div>
49 |
50 |         <Input
51 |           data-testid={`reminder-time-${reminder.habitType}`}
52 |           type="time"
53 |           value={reminder.reminderTime ?? ""}
54 |           onChange={(e) => onTimeChange(reminder.habitType, e.target.value)}
55 |           disabled={!reminder.enabled}
56 |           className={cn(
57 |             "w-28 h-8 text-xs",
58 |             !reminder.enabled && "opacity-50"
59 |           )}
60 |           aria-label={`${HABIT_LABELS[reminder.habitType]} reminder time`}
61 |         />
62 |
63 |         <Switch
64 |           data-testid={`reminder-toggle-${reminder.habitType}`}
65 |           checked={reminder.enabled}
66 |           onCheckedChange={(checked) =>
67 |             onToggle(reminder.habitType, checked)
68 |           }
69 |           aria-label={`Enable ${HABIT_LABELS[reminder.habitType]} reminder`}
70 |         />
71 |       </div>
72 |     )}}
73 |   </div>
74 | </div>
75 | );
76 | };
77 |
78 | export default WellnessReminderSettings;
79 |

```

FILE: [src/components/shared/WellnessSettingsPanel.tsx](#)

TYPE: .tsx | LINES: 1-181 | SHA256: 82c5b8a62bb3485bb8eccf97d0ee567be7b8f5d4e1f6f898b0c9c237d034fa7b

```

1 | import { Brain, Droplets, Dumbbell, Moon, Eye } from "lucide-react";
2 | import { Switch } from "@components/ui/switch";
3 | import { Label } from "@components/ui/label";
4 | import { cn } from "@lib/utils";
5 | import WellnessReminderSettings from "@components/shared/WellnessReminderSettings";
6 | import WellnessGoalInput from "@components/shared/WellnessGoalInput";
7 | import type {
8 |   WellnessHabitType,
9 |   WellnessReminderConfig,
10 |   WellnessTarget,
11 | } from "@types/habits";
12 |
13 | export interface WellnessSettingsPanelProps {
14 |   enabledHabits: WellnessHabitType[];
15 |   parentVisibility: boolean;
16 |   onToggleHabit: (type: WellnessHabitType, enabled: boolean) => void;
17 |   onToggleParentVisibility: (visible: boolean) => void;
18 |   reminders?: WellnessReminderConfig[];
19 |   onReminderToggle?: (habitType: WellnessHabitType, enabled: boolean) => void;
20 |   onReminderTimeChange?: (habitType: WellnessHabitType, time: string) => void;
21 |   goals?: WellnessTarget[];
22 |   goalProgress?: Record<
23 |     string,
24 |     { progress: number; logged: number; target: number; unit: string }
25 |   >;
26 |   onGoalSave?: (target: WellnessTarget) => void;
27 | }
28 |
29 | const WELLNESS_HABITS: {
30 |   type: WellnessHabitType;
31 |   label: string;
32 |   description: string;
33 |   icon: typeof Brain;
34 | }[] = [
35 |   {
36 |     type: "meditation",
37 |     label: "Meditation",
38 |     description: "Track daily meditation sessions (5+ minutes)",
39 |     icon: Brain,
40 |   },
41 |   {
42 |     type: "hydration",
43 |     label: "Hydration",
44 |     description: "Track daily water intake (8 glasses)",
45 |     icon: Droplets,
46 |   },
47 |   {
48 |     type: "exercise",
49 |     label: "Exercise",
50 |     description: "Track daily physical activity (30+ minutes)",
51 |     icon: Dumbbell,
52 |   },
53 |   {
54 |     type: "sleep",
55 |     label: "Sleep",
56 |     description: "Track nightly sleep duration (7+ hours)",
57 |     icon: Moon,
58 |   },
59 | ];
60 |
61 | const WellnessSettingsPanel = ({
62 |   enabledHabits,
63 |   parentVisibility,
64 |   onToggleHabit,
65 |   onToggleParentVisibility,
66 |   reminders,
67 |   onReminderToggle,
68 |   onReminderTimeChange,

```

```

69 | goals,
70 | goalProgress,
71 | onGoalSave,
72 | }: WellnessSettingsPanelProps) => {
73 |   return (
74 |     <div data-testid="wellness-settings-panel" className="space-y-4">
75 |       <h3 className="text-sm font-bold tracking-wide uppercase text-gray-500">
76 |         Wellness Preferences
77 |       </h3>
78 |
79 |       <div className="space-y-2">
80 |         {WELLNESS_HABITS.map(({ type, label, description, icon: Icon }) => {
81 |           const enabled = enabledHabits.includes(type);
82 |           return (
83 |             <div
84 |               key={type}
85 |               data-testid={`wellness-setting-${type}`}
86 |               className={cn(
87 |                 "flex items-center gap-3 rounded-xl border bg-white p-3 shadow-sm transition-colors",
88 |                 enabled && "border-blue-200 bg-blue-50/30"
89 |               )}
90 |             >
91 |               <div
92 |                 className={cn(
93 |                   "p-2 rounded-lg",
94 |                   enabled ? "bg-blue-100" : "bg-slate-50"
95 |                 )}
96 |               >
97 |                 <Icon
98 |                   className={cn(
99 |                     "h-4 w-4",
100 |                     enabled ? "text-blue-600" : "text-slate-500"
101 |                   )}
102 |                 />
103 |               </div>
104 |
105 |               <div className="flex-1 min-w-0">
106 |                 <Label className="text-sm font-medium">{label}</Label>
107 |                 <p className="text-xs text-gray-500">{description}</p>
108 |               </div>
109 |
110 |               <Switch
111 |                 data-testid={`wellness-setting-toggle-${type}`}
112 |                 checked={enabled}
113 |                 onCheckedChange={(checked) => onToggleHabit(type, checked)}
114 |                 aria-label={`Enable ${label}`}
115 |               />
116 |             </div>
117 |           );
118 |         })}
119 |     </div>
120 |
121 |     <div
122 |       data-testid="wellness-parent-visibility"
123 |       className="flex items-center gap-3 rounded-xl border bg-white p-3 shadow-sm"
124 |     >
125 |       <div className="p-2 rounded-lg bg-slate-50">
126 |         <Eye className="h-4 w-4 text-slate-500" />
127 |       </div>
128 |       <div className="flex-1 min-w-0">
129 |         <Label className="text-sm font-medium">Parent Visibility</Label>
130 |         <p className="text-xs text-gray-500">
131 |           Allow parents to see wellness data
132 |         </p>
133 |       </div>
134 |       <Switch
135 |         data-testid="wellness-parent-visibility-toggle"
136 |         checked={parentVisibility}
137 |         onCheckedChange={onToggleParentVisibility}
138 |         aria-label="Allow parents to see wellness data"
139 |       />
140 |     </div>
141 |
142 |     {/* Reminders Section */}
143 |     {reminders &&
144 |       reminders.length > 0 &&
145 |       onReminderToggle &&
146 |       onReminderTimeChange && (
147 |         <WellnessReminderSettings
148 |           reminders={reminders}
149 |           onToggle={onReminderToggle}
150 |           onTimeChange={onReminderTimeChange}
151 |         />
152 |       )}
153 |
154 |     {/* Goals Section */}
155 |     {onGoalSave && enabledHabits.length > 0 && (
156 |       <div data-testid="wellness-goals-section" className="space-y-3">
157 |         <h4 className="text-sm font-bold tracking-wide uppercase text-gray-500">
158 |           Daily Goals
159 |         </h4>
160 |         {enabledHabits.map((ht) => {
161 |           const target = goals?.find((g) => g.habitType === ht) ?? null;
162 |           const prog = goalProgress?.[ht];
163 |           return (
164 |             <WellnessGoalInput
165 |               key={ht}
166 |               habitType={ht}
167 |               target={target}
168 |               progress={prog?.progress ?? 0}
169 |               currentValue={prog?.logged ?? 0}
170 |               onSave={onGoalSave}
171 |             />
172 |           );

```

```

173 |         }}}
174 |       </div>
175 |     )}
176 |   </div>
177 | );
178 | };
179 |
180 | export default WellnessSettingsPanel;
181 |

```

#### FILE: src/components/shared/WellnessTipCard.tsx

TYPE: .tsx | LINES: 1-70 | SHA256: dcd6310df46ca607a4c8e4751315e5f19b25d64cbd45900fd6d6bc6c26b8f464

```

1 | import { Lightbulb, ExternalLink, X } from "lucide-react";
2 | import { Button } from "@components/ui/button";
3 | import type { WellnessTip } from "@types/habits";
4 | import { PCard } from "@design-system";
5 |
6 | export interface WellnessTipCardProps {
7 |   tip: WellnessTip;
8 |   isOnboarding: boolean;
9 |   onDismiss?: () => void;
10 | }
11 |
12 | const WellnessTipCard = ({
13 |   tip,
14 |   isOnboarding,
15 |   onDismiss,
16 | }: WellnessTipCardProps) => {
17 |   return (
18 |     <PCard
19 |       data-testid="wellness-tip-card"
20 |       className="bg-amber-50 border-amber-200 p-3"
21 |     >
22 |       <div className="flex items-start gap-3">
23 |         <div className="p-1.5 rounded-lg bg-amber-100 shrink-0">
24 |           <Lightbulb className="h-4 w-4 text-amber-600" />
25 |         </div>
26 |
27 |         <div className="flex-1 min-w-0">
28 |           <p data-testid="wellness-tip-text" className="text-sm text-amber-900">
29 |             {tip.text}
30 |           </p>
31 |
32 |           {tip.resourceUrl && (
33 |             <Button
34 |               asChild
35 |               variant="ghost"
36 |               size="sm"
37 |               className="inline-flex items-center gap-1 text-xs text-amber-700 hover:text-amber-900 mt-1.5 fon
38 |             >
39 |               <a
40 |                 data-testid="wellness-tip-resource"
41 |                 href={tip.resourceUrl}
42 |                 target="_blank"
43 |                 rel="noopener noreferrer"
44 |               >
45 |                 {tip.resourceLabel ?? "Learn more"}
46 |                 <ExternalLink className="h-3 w-3" />
47 |               </a>
48 |             </Button>
49 |           )}
50 |         </div>
51 |
52 |         {isOnboarding && onDismiss && (
53 |           <Button
54 |             data-testid="wellness-tip-dismiss"
55 |             variant="ghost"
56 |             size="sm"
57 |             onClick={onDismiss}
58 |             className="h-6 w-6 p-0 shrink-0 text-amber-500 hover:text-amber-700 hover:bg-amber-100"
59 |             aria-label="Dismiss tip"
60 |           >
61 |             <X className="h-3.5 w-3.5" />
62 |           </Button>
63 |         )}
64 |       </div>
65 |     </PCard>
66 |   );
67 | };
68 |
69 | export default WellnessTipCard;
70 |

```

#### FILE: src/components/shared/WhyThisPopover.tsx

TYPE: .tsx | LINES: 1-70 | SHA256: 728ad4b06083eff382fcb635925e2506f69b914054c95662db2648cbf18263ef

```

1 | import { CircleHelp } from "lucide-react";
2 | import { useId } from "react";
3 | import { useTranslation } from "react-i18next";
4 | import { Button } from "@components/ui/button";
5 | import {
6 |   Popover,
7 |   PopoverContent,
8 |   PopoverTrigger,
9 | } from "@components/ui/popover";
10 | import { cn } from "@lib/utills";
11 |
12 | interface WhyThisPopoverProps {
13 |   title: string;
14 |   reasons: string[];
15 |   className?: string;
16 | }
17 |

```

```

18 | /**
19 |  * Reusable A2 explainability control for hero, rail, and AI-suggestion cards.
20 |  * The caller supplies only auditable, data-backed reasons for the card.
21 |  */
22 | const WhyThisPopover = ({ title, reasons, className }: WhyThisPopoverProps) => {
23 |   const { t } = useTranslation("common");
24 |   const titleId = useId();
25 |
26 |   return (
27 |     <Popover>
28 |       <PopoverTrigger asChild>
29 |         <Button
30 |           type="button"
31 |           variant="ghost"
32 |           size="sm"
33 |           className={cn(
34 |             "h-8 gap-1 px-2 text-xs text-muted-foreground hover:text-foreground",
35 |             className
36 |           )}
37 |           aria-label={t("header.whyThis")}
38 |         >
39 |           <CircleHelp className="size-3.5" aria-hidden="true" />
40 |           {t("header.whyThis")}
41 |         </Button>
42 |       </PopoverTrigger>
43 |       <PopoverContent
44 |         role="dialog"
45 |         aria-labelledby={titleId}
46 |         className="w-80 space-y-2"
47 |       >
48 |         <h3 id={titleId} className="text-sm font-bold text-foreground">
49 |           {title}
50 |         </h3>
51 |         <p className="text-xs text-muted-foreground">
52 |           {t("header.whyThisDescription")}
53 |         </p>
54 |         <ul className="space-y-1.5 text-xs text-muted-foreground">
55 |           {reasons.map((reason) => (
56 |             <li key={reason} className="flex gap-2">
57 |               <span className="text-primary" aria-hidden="true">
58 |                 •
59 |               </span>
60 |               <span>{reason}</span>
61 |             </li>
62 |           ))}
63 |         </ul>
64 |       </PopoverContent>
65 |     </Popover>
66 |   );
67 | };
68 |
69 | export default WhyThisPopover;
70 |

```

#### FILE: src/components/shared/XPAwardToast.tsx

TYPE: .tsx | LINES: 1-119 | SHA256: e8282ce0627afcb36e39303808fb4d949bd792583c25dc51286c8061625d9aed

```

1 | import { useEffect, useRef, useState, useCallback } from "react";
2 | import { motion, AnimatePresence, useReducedMotion } from "framer-motion";
3 | import { launchConfetti } from "@lib/confetti";
4 | import { Star, TrendingUp } from "lucide-react";
5 |
6 | interface XPAwardToastProps {
7 |   xpAmount: number;
8 |   source: string;
9 |   levelUp?: boolean;
10 |   newLevel?: number;
11 |   onComplete?: () => void;
12 | }
13 |
14 | const useCountUp = (target: number, duration: number = 600): number => {
15 |   const prefersReducedMotion = useReducedMotion();
16 |   const [count, setCount] = useState(prefersReducedMotion ? target : 0);
17 |
18 |   useEffect(() => {
19 |     if (prefersReducedMotion || target <= 0) return;
20 |
21 |     let rafId: number;
22 |     const startTime = performance.now();
23 |
24 |     const animate = (now: number) => {
25 |       const elapsed = now - startTime;
26 |       const progress = Math.min(elapsed / duration, 1);
27 |       // Ease-out cubic for a satisfying deceleration
28 |       const eased = 1 - Math.pow(1 - progress, 3);
29 |       setCount(Math.round(eased * target));
30 |
31 |       if (progress < 1) {
32 |         rafId = requestAnimationFrame(animate);
33 |       }
34 |     };
35 |
36 |     rafId = requestAnimationFrame(animate);
37 |     return () => cancelAnimationFrame(rafId);
38 |   }, [target, duration, prefersReducedMotion]);
39 |
40 |   return count;
41 | };
42 |
43 | const XPAwardToast = ({
44 |   xpAmount,
45 |   source,
46 |   levelUp = false,
47 |   newLevel,

```

```

48 |   onComplete,
49 | }: XPAwardToastProps) => {
50 |   const [visible, setVisible] = useState(true);
51 |   const prefersReducedMotion = useReducedMotion();
52 |   const confettiFired = useRef(false);
53 |   const count = useCountUp(xpAmount);
54 |
55 |   const fireConfetti = useCallback(() => {
56 |     if (confettiFired.current || prefersReducedMotion) return;
57 |     confettiFired.current = true;
58 |
59 |     void launchConfetti({
60 |       particleCount: 30,
61 |       spread: 60,
62 |       origin: { y: 0.8 },
63 |       colors: ["#f59e0b", "#eab308", "#fbbf24"],
64 |     });
65 |   }, [prefersReducedMotion]);
66 |
67 |   useEffect(() => {
68 |     fireConfetti();
69 |   }, [fireConfetti]);
70 |
71 |   useEffect(() => {
72 |     const timer = setTimeout(() => {
73 |       setVisible(false);
74 |     }, 3000);
75 |     return () => clearTimeout(timer);
76 |   }, []);
77 |
78 |   const handleExitComplete = () => {
79 |     onComplete?.();
80 |   };
81 |
82 |   return (
83 |     <AnimatePresence onExitComplete={handleExitComplete}>
84 |       {visible && (
85 |         <motion.div
86 |           initial={prefersReducedMotion ? undefined : { opacity: 0, y: 20 }}
87 |           animate={prefersReducedMotion ? undefined : { opacity: 1, y: 0 }}
88 |           exit={prefersReducedMotion ? { opacity: 0 } : { opacity: 0, y: -10 }}
89 |           transition={{ duration: 0.3, ease: "easeOut" }}
90 |           className="bg-white border-0 shadow-lg rounded-xl p-4 w-64"
91 |           role="status"
92 |           aria-live="polite"
93 |         >
94 |           <div className="flex items-center gap-3">
95 |             <div className="p-2 rounded-lg bg-amber-50">
96 |               <Star className="h-5 w-5 text-amber-500 animate-xp-pulse" />
97 |             </div>
98 |             <div className="flex-1 min-w-0">
99 |               <p className="text-2xl font-black text-amber-500">+{count} XP</p>
100 |               <p className="text-xs text-gray-500 truncate">{source}</p>
101 |             </div>
102 |           </div>
103 |
104 |           {levelUp && newLevel !== null && (
105 |             <div className="mt-3 flex items-center gap-2 rounded-lg bg-blue-50 px-3 py-2">
106 |               <TrendingUp className="h-4 w-4 text-blue-600" />
107 |               <span className="text-sm font-semibold text-blue-600">
108 |                 Level Up! !' Level {newLevel}
109 |               </span>
110 |             </div>
111 |           )}
112 |         </motion.div>
113 |       )}
114 |     </AnimatePresence>
115 |   );
116 | };
117 |
118 | export default XPAwardToast;
119 |

```

FILE: [src/components/shared/XPBalanceBadge.tsx](#)

TYPE: .tsx | LINES: 1-56 | SHA256: ef338fdbee055ec2eec3d139db25621433404e33a6aae69758f32866d9f8e5aa

```

1 | // =====
2 | // XPBalanceBadge – Persistent XP balance indicator for sidebar and marketplace
3 | // =====
4 |
5 | import { Coins, Loader2 } from "lucide-react";
6 | import { useAuth } from "@/hooks/useAuth";
7 | import { useXPBalance } from "@/hooks/useXPBalance";
8 | import { cn } from "@/lib/utlis";
9 |
10 | interface XPBalanceBadgeProps {
11 |   className?: string;
12 |   size?: "sm" | "md" | "lg";
13 | }
14 |
15 | const XPBalanceBadge = ({ className, size = "md" }: XPBalanceBadgeProps) => {
16 |   const { user } = useAuth();
17 |   const { data, isLoading } = useXPBalance(user?.id ?? "");
18 |
19 |   const sizeClasses = {
20 |     sm: "text-xs px-2 py-1 gap-1",
21 |     md: "text-sm px-3 py-1.5 gap-1.5",
22 |     lg: "text-base px-4 py-2 gap-2",
23 |   };
24 |
25 |   const iconSizes = {
26 |     sm: "h-3 w-3",
27 |     md: "h-4 w-4",
28 |     lg: "h-5 w-5",

```

```

29 | };
30 |
31 | return (
32 |   <div
33 |     title="Available XP – your spendable balance in the marketplace"
34 |     aria-label={`Available XP (spendable balance): ${
35 |       data?.balance ?? 0
36 |     }.toLocaleString()}`
37 |     className={cn(
38 |       "inline-flex items-center rounded-full bg-amber-50 border border-amber-200 font-semibold text-amber-70
39 |       sizeClasses[size],
40 |       className
41 |     )}
42 |   >
43 |     <Coins className={cn(iconSizes[size], "text-amber-500")} />
44 |     {isLoading ? (
45 |       <Loader2
46 |         className={cn(iconSizes[size], "animate-spin text-amber-400")}
47 |       />
48 |     ) : (
49 |       <span>{(data?.balance ?? 0).toLocaleString()} XP</span>
50 |     )}
51 |   </div>
52 | );
53 | };
54 |
55 | export default XPBalanceBadge;
56 |

```

FILE: [src/components/shared/XPTransactionRow.tsx](#)

TYPE: .tsx | LINES: 1-68 | SHA256: db7bc69d15b4b4faaec229fc710c5459621ec59effaded1b84921167821250d8

```

1 | // =====
2 | // XPTransactionRow – Single XP transaction row for history list
3 | // =====
4 |
5 | import { Star, TrendingDown } from "lucide-react";
6 | import { cn } from "@lib/Utils";
7 |
8 | interface XPTransactionRowProps {
9 |   sourceLabel: string;
10 |   xpAmount: number;
11 |   description?: string;
12 |   timestamp: string;
13 |   className?: string;
14 | }
15 |
16 | const XPTransactionRow = ({
17 |   sourceLabel,
18 |   xpAmount,
19 |   description,
20 |   timestamp,
21 |   className,
22 | }: XPTransactionRowProps) => {
23 |   const isPositive = xpAmount >= 0;
24 |
25 |   return (
26 |     <div
27 |       className={cn(
28 |         "flex items-center gap-3 py-3 border-b border-slate-100 last:border-0",
29 |         className
30 |       )}
31 |     >
32 |       <div
33 |         className={cn(
34 |           "p-2 rounded-lg",
35 |           isPositive ? "bg-amber-50" : "bg-red-50"
36 |         )}
37 |       >
38 |         {isPositive ? (
39 |           <Star className="h-4 w-4 text-amber-500" />
40 |         ) : (
41 |           <TrendingDown className="h-4 w-4 text-red-500" />
42 |         )}
43 |       </div>
44 |       <div className="flex-1 min-w-0">
45 |         <p className="text-sm font-medium truncate">{sourceLabel}</p>
46 |         {description && (
47 |           <p className="text-xs text-gray-500 truncate">{description}</p>
48 |         )}
49 |       </div>
50 |       <div className="text-end shrink-0">
51 |         <p
52 |           className={cn(
53 |             "text-sm font-bold tabular-nums",
54 |             isPositive ? "text-amber-600" : "text-red-600"
55 |           )}
56 |         >
57 |           {isPositive ? "+" : ""}
58 |           {xpAmount} XP
59 |         </p>
60 |         <p className="text-[10px] text-gray-400">{timestamp}</p>
61 |       </div>
62 |     </div>
63 |   );
64 | };
65 |
66 | export default XPTransactionRow;
67 | export type { XPTransactionRowProps };
68 |

```

## FILE: src/components/ui/alert.tsx

TYPE: .tsx | LINES: 1-62 | SHA256: 81b14e7d6fe2c185c3660e8d00271cbdde28fbb4553f2d9fa16c53153ae741e

```
1 | import * as React from "react";
2 | import { cva, type VariantProps } from "class-variance-authority";
3 | import { cn } from "@lib/utils";
4 |
5 | const alertVariants = cva(
6 |   "relative w-full rounded-lg border px-4 py-3 text-sm [&svg]:translate-y-[-3px] [&svg]:absolute [&svg]
7 |   {
8 |     variants: {
9 |       variant: {
10 |         // Use shadcn theme tokens so Alert adapts to dark mode once the
11 |         // theme variables flip. Hardcoded greys / reds were bypassing the
12 |         // design-system tokens and would stay light-mode only.
13 |         default: "bg-background text-foreground border-border",
14 |         destructive:
15 |           "border-destructive/50 bg-destructive/10 text-destructive [&svg]:text-destructive dark:border-destr
16 |       },
17 |     },
18 |     defaultVariants: {
19 |       variant: "default",
20 |     },
21 |   }
22 | );
23 |
24 | const Alert = React.forwardRef<
25 |   HTMLDivElement,
26 |   React.HTMLAttributes<HTMLDivElement> & VariantProps<typeof alertVariants>
27 | >(({ className, variant, ...props }, ref) => (
28 |   <div
29 |     ref={ref}
30 |     role="alert"
31 |     className={cn(alertVariants({ variant }), className)}
32 |     {...props}
33 |   />
34 | ));
35 | Alert.displayName = "Alert";
36 |
37 | const AlertTitle = React.forwardRef<
38 |   HTMLParagraphElement,
39 |   React.HTMLAttributes<HTMLHeadingElement>
40 | >(({ className, ...props }, ref) => (
41 |   <h5
42 |     ref={ref}
43 |     className={cn("mb-1 font-medium leading-none tracking-tight", className)}
44 |     {...props}
45 |   />
46 | ));
47 | AlertTitle.displayName = "AlertTitle";
48 |
49 | const AlertDescription = React.forwardRef<
50 |   HTMLParagraphElement,
51 |   React.HTMLAttributes<HTMLParagraphElement>
52 | >(({ className, ...props }, ref) => (
53 |   <div
54 |     ref={ref}
55 |     className={cn("text-sm [&p]:leading-relaxed", className)}
56 |     {...props}
57 |   />
58 | ));
59 | AlertDescription.displayName = "AlertDescription";
60 |
61 | export { Alert, AlertTitle, AlertDescription };
62 |
```

## FILE: src/components/ui/avatar.tsx

TYPE: .tsx | LINES: 1-49 | SHA256: 0a7a44cc0d16b33ebd22c92eb4e84fa565125e70e5fcb4eaf6fc2a872502cbcd

```
1 | import * as React from "react";
2 | import * as AvatarPrimitive from "@radix-ui/react-avatar";
3 |
4 | import { cn } from "@lib/utils";
5 |
6 | const Avatar = React.forwardRef<
7 |   React.ElementRef<typeof AvatarPrimitive.Root>,
8 |   React.ComponentPropsWithoutRef<typeof AvatarPrimitive.Root>
9 | >(({ className, ...props }, ref) => (
10 |   <AvatarPrimitive.Root
11 |     ref={ref}
12 |     className={cn(
13 |       "relative flex h-10 w-10 shrink-0 overflow-hidden rounded-full",
14 |       className
15 |     )}
16 |     {...props}
17 |   />
18 | ));
19 | Avatar.displayName = AvatarPrimitive.Root.displayName;
20 |
21 | const AvatarImage = React.forwardRef<
22 |   React.ElementRef<typeof AvatarPrimitive.Image>,
23 |   React.ComponentPropsWithoutRef<typeof AvatarPrimitive.Image>
24 | >(({ className, ...props }, ref) => (
25 |   <AvatarPrimitive.Image
26 |     ref={ref}
27 |     className={cn("aspect-square h-full w-full", className)}
28 |     {...props}
29 |   />
30 | ));
31 | AvatarImage.displayName = AvatarPrimitive.Image.displayName;
32 |
33 | const AvatarFallback = React.forwardRef<
34 |   React.ElementRef<typeof AvatarPrimitive.Fallback>,
```

```

35 |   React.ComponentPropsWithoutRef<typeof AvatarPrimitive.Fallback>
36 | >(({ className, ...props }, ref) => {
37 |   <AvatarPrimitive.Fallback
38 |     ref={ref}
39 |     className={cn(
40 |       "flex h-full w-full items-center justify-center rounded-full bg-muted",
41 |       className
42 |     )}
43 |     {...props}
44 |   />
45 | ));
46 | AvatarFallback.displayName = AvatarPrimitive.Fallback.displayName;
47 |
48 | export { Avatar, AvatarImage, AvatarFallback };
49 |

```

#### FILE: src/components/ui/badge.tsx

TYPE: .tsx | LINES: 1-49 | SHA256: 9f30390c27e0593a502dc79b31c2fc6d27a982ebd86958025d6d487c671380e5

```

1 | import * as React from "react";
2 | import { cva, type VariantProps } from "class-variance-authority";
3 | import { Slot } from "radix-ui";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | const badgeVariants = cva(
8 |   "inline-flex items-center justify-center rounded-full border border-transparent px-2 py-0.5 text-xs font-med
9 |   {
10 |     variants: {
11 |       variant: {
12 |         default: "bg-primary text-primary-foreground [&]:hover:bg-primary/90",
13 |         secondary:
14 |           "bg-secondary text-secondary-foreground [&]:hover:bg-secondary/90",
15 |         destructive:
16 |           "bg-destructive text-white [&]:hover:bg-destructive/90 focus-visible:ring-destructive/20 dark:focus
17 |         outline:
18 |           "border-border text-foreground [&]:hover:bg-accent [&]:hover:text-accent-foreground",
19 |         ghost: "[&]:hover:bg-accent [&]:hover:text-accent-foreground",
20 |         link: "text-primary underline-offset-4 [&]:hover:underline",
21 |       },
22 |     },
23 |     defaultVariants: {
24 |       variant: "default",
25 |     },
26 |   },
27 | );
28 |
29 | function Badge({
30 |   className,
31 |   variant = "default",
32 |   asChild = false,
33 |   ...props
34 | }: React.ComponentProps<"span"> &
35 |   VariantProps<typeof badgeVariants> & { asChild?: boolean }) {
36 |   const Comp = asChild ? Slot.Root : "span";
37 |
38 |   return (
39 |     <Comp
40 |       data-slot="badge"
41 |       data-variant={variant}
42 |       className={cn(badgeVariants({ variant }), className)}
43 |       {...props}
44 |     />
45 |   );
46 | }
47 |
48 | export { Badge, badgeVariants };
49 |

```

#### FILE: src/components/ui/button.tsx

TYPE: .tsx | LINES: 1-80 | SHA256: daacdda76eb6f1c177ee1e8a489620aa90cc2dee7adb6a14ef335b41fd67d6b9

```

1 | import * as React from "react";
2 | import { cva, type VariantProps } from "class-variance-authority";
3 | import { Slot } from "radix-ui";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | const buttonVariants = cva(
8 |   "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium transition-a
9 |   {
10 |     variants: {
11 |       variant: {
12 |         default: "bg-primary text-primary-foreground hover:bg-primary/90",
13 |         destructive:
14 |           "bg-destructive text-white hover:bg-destructive/90 focus-visible:ring-destructive/20 dark:focus-visi
15 |         outline:
16 |           "border bg-background shadow-xs hover:bg-accent hover:text-accent-foreground dark:bg-input/30 dark:b
17 |         secondary:
18 |           "bg-secondary text-secondary-foreground hover:bg-secondary/80",
19 |         ghost:
20 |           "hover:bg-accent hover:text-accent-foreground dark:hover:bg-accent/50",
21 |         link: "text-primary underline-offset-4 hover:underline",
22 |         // Tactile (pressable) brand CTA – Duolingo-style solid bottom edge that
23 |         // compresses on :active. Surface + press live in `btn-tactile`
24 |         // (src/index.css); token-bound and reduced-motion safe. Additive: does
25 |         // not affect any existing variant.
26 |         tactile: "btn-tactile rounded-xl text-white",
27 |       },
28 |     },
29 |     size: {
30 |       default: "h-9 px-4 py-2 has-[:>svg]:px-3",
31 |       xs: "h-6 gap-1 rounded-md px-2 text-xs has-[:>svg]:px-1.5 [&_svg:not([class*='size-'])]:size-3",
32 |       sm: "h-8 rounded-md gap-1.5 px-3 has-[:>svg]:px-2.5",
33 |       lg: "h-10 rounded-md px-6 has-[:>svg]:px-4",

```



```

33 |         icon: "size-9",
34 |         "icon-xs": "size-6 rounded-md [&_svg:not([class*='size-'])]:size-3",
35 |         "icon-sm": "size-8",
36 |         "icon-lg": "size-10",
37 |     },
38 | },
39 |     defaultVariants: {
40 |       variant: "default",
41 |       size: "default",
42 |     },
43 |   },
44 | );
45 |
46 | const Button = React.forwardRef<
47 |   HTMLButtonElement,
48 |   React.ComponentProps<"button"> &
49 |     VariantProps<typeof buttonVariants> & {
50 |       asChild?: boolean;
51 |     }
52 | > (
53 |   (
54 |     {
55 |       className,
56 |       variant = "default",
57 |       size = "default",
58 |       asChild = false,
59 |       ...props
60 |     },
61 |     ref
62 |   ) => {
63 |     const Comp = asChild ? Slot.Root : "button";
64 |
65 |     return (
66 |       <Comp
67 |         data-slot="button"
68 |         data-variant={variant}
69 |         data-size={size}
70 |         className={cn(buttonVariants({ variant, size, className })))}
71 |         ref={ref}
72 |         {...props}
73 |       />
74 |     );
75 |   }
76 | );
77 | Button.displayName = "Button";
78 |
79 | export { Button, buttonVariants };
80 |

```

## FILE: src/components/ui/card.tsx

TYPE: .tsx | LINES: 1-93 | SHA256: 1397e7d264d90162220ea7473b311e434651886267c71e7f58d4488be9d8ff39

```

1 | import * as React from "react";
2 |
3 | import { cn } from "@lib/utls";
4 |
5 | function Card({ className, ...props }: React.ComponentProps<"div">) {
6 |   return (
7 |     <div
8 |       data-slot="card"
9 |       className={cn(
10 |         "bg-card text-card-foreground flex flex-col gap-6 rounded-xl border py-6 shadow-sm",
11 |         className
12 |       )}
13 |       {...props}
14 |     />
15 |   );
16 | }
17 |
18 | function CardHeader({ className, ...props }: React.ComponentProps<"div">) {
19 |   return (
20 |     <div
21 |       data-slot="card-header"
22 |       className={cn(
23 |         "@container/card-header grid auto-rows-min grid-rows-[auto_auto] items-start gap-2 px-6 has-data-[slot"
24 |         className
25 |       )}
26 |       {...props}
27 |     />
28 |   );
29 | }
30 |
31 | function CardTitle({ className, ...props }: React.ComponentProps<"div">) {
32 |   return (
33 |     <div
34 |       data-slot="card-title"
35 |       className={cn("leading-none font-semibold", className)}
36 |       {...props}
37 |     />
38 |   );
39 | }
40 |
41 | function CardDescription({ className, ...props }: React.ComponentProps<"div">) {
42 |   return (
43 |     <div
44 |       data-slot="card-description"
45 |       className={cn("text-muted-foreground text-sm", className)}
46 |       {...props}
47 |     />
48 |   );
49 | }
50 |
51 | function CardAction({ className, ...props }: React.ComponentProps<"div">) {
52 |   return (

```

```

53 |     <div
54 |       data-slot="card-action"
55 |       className={cn(
56 |         "col-start-2 row-span-2 row-start-1 self-start justify-self-end",
57 |         className
58 |       )}
59 |       {...props}
60 |     />
61 |   );
62 | }
63 |
64 | function CardContent({ className, ...props }: React.ComponentProps<"div">) {
65 |   return (
66 |     <div
67 |       data-slot="card-content"
68 |       className={cn("px-6", className)}
69 |       {...props}
70 |     />
71 |   );
72 | }
73 |
74 | function CardFooter({ className, ...props }: React.ComponentProps<"div">) {
75 |   return (
76 |     <div
77 |       data-slot="card-footer"
78 |       className={cn("flex items-center px-6 [.border-t]:pt-6", className)}
79 |       {...props}
80 |     />
81 |   );
82 | }
83 |
84 | export {
85 |   Card,
86 |   CardHeader,
87 |   CardFooter,
88 |   CardTitle,
89 |   CardAction,
90 |   CardDescription,
91 |   CardContent,
92 | };
93 |

```

#### FILE: src/components/ui/checkbox.tsx

TYPE: .tsx | LINES: 1-31 | SHA256: b810ab2c528b9e23e29005c3281d43def213945eda11475c9c3ea667559e65b7

```

1 | import * as React from "react";
2 | import { CheckIcon } from "lucide-react";
3 | import { Checkbox as CheckboxPrimitive } from "radix-ui";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | function Checkbox({
8 |   className,
9 |   ...props
10 | }: React.ComponentProps<typeof CheckboxPrimitive.Root>) {
11 |   return (
12 |     <CheckboxPrimitive.Root
13 |       data-slot="checkbox"
14 |       className={cn(
15 |         "peer size-4 shrink-0 rounded-[4px] border border-input shadow-xs transition-shadow outline-none focus
16 |         className
17 |       )}
18 |       {...props}
19 |     >
20 |       <CheckboxPrimitive.Indicator
21 |         data-slot="checkbox-indicator"
22 |         className="grid place-content-center text-current transition-none"
23 |       >
24 |         <CheckIcon className="size-3.5" />
25 |       </CheckboxPrimitive.Indicator>
26 |     </CheckboxPrimitive.Root>
27 |   );
28 | }
29 |
30 | export { Checkbox };
31 |

```

#### FILE: src/components/ui/dialog.tsx

TYPE: .tsx | LINES: 1-159 | SHA256: 86c0958fd6cda5e8c806d35e31330a410e9bfc5dda874d114e0c7b64c9adec2c

```

1 | "use client";
2 |
3 | import * as React from "react";
4 | import { XIcon } from "lucide-react";
5 | import { Dialog as DialogPrimitive } from "radix-ui";
6 |
7 | import { cn } from "@lib/utils";
8 | import { Button } from "@components/ui/button";
9 |
10 | function Dialog({
11 |   ...props
12 | }: React.ComponentProps<typeof DialogPrimitive.Root>) {
13 |   return <DialogPrimitive.Root data-slot="dialog" {...props} />;
14 | }
15 |
16 | function DialogTrigger({
17 |   ...props
18 | }: React.ComponentProps<typeof DialogPrimitive.Trigger>) {
19 |   return <DialogPrimitive.Trigger data-slot="dialog-trigger" {...props} />;
20 | }
21 |
22 | function DialogPortal({
23 |   ...props
24 | }: React.ComponentProps<typeof DialogPrimitive.Portal>) {

```

```

25 |   return <DialogPrimitive.Portal data-slot="dialog-portal" {...props} />;
26 | }
27 |
28 | function DialogClose({
29 |   ...props
30 | }): React.ComponentProps<typeof DialogPrimitive.Close> {
31 |   return <DialogPrimitive.Close data-slot="dialog-close" {...props} />;
32 | }
33 |
34 | function DialogOverlay({
35 |   className,
36 |   ...props
37 | }): React.ComponentProps<typeof DialogPrimitive.Overlay> {
38 |   return (
39 |     <DialogPrimitive.Overlay
40 |       data-slot="dialog-overlay"
41 |       className={cn(
42 |         "data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[sta
43 |         className
44 |       )}
45 |       {...props}
46 |     />
47 |   );
48 | }
49 |
50 | function DialogContent({
51 |   className,
52 |   children,
53 |   showCloseButton = true,
54 |   ...props
55 | }): React.ComponentProps<typeof DialogPrimitive.Content> & {
56 |   showCloseButton?: boolean;
57 | }) {
58 |   return (
59 |     <DialogPortal data-slot="dialog-portal">
60 |       <DialogOverlay />
61 |       <DialogPrimitive.Content
62 |         data-slot="dialog-content"
63 |         className={cn(
64 |           "bg-background data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade
65 |         className
66 |       )}
67 |       {...props}
68 |     >
69 |       {children}
70 |       {showCloseButton && (
71 |         <DialogPrimitive.Close
72 |           data-slot="dialog-close"
73 |           className="ring-offset-background focus:ring-ring data-[state=open]:bg-accent data-[state=open]:te
74 |         >
75 |           <XIcon />
76 |           <span className="sr-only">Close</span>
77 |         </DialogPrimitive.Close>
78 |       )}
79 |     </DialogPrimitive.Content>
80 |   </DialogPortal>
81 | );
82 | }
83 |
84 | function DialogHeader({ className, ...props }: React.ComponentProps<"div">) {
85 |   return (
86 |     <div
87 |       data-slot="dialog-header"
88 |       className={cn("flex flex-col gap-2 text-center sm:text-left", className)}
89 |       {...props}
90 |     />
91 |   );
92 | }
93 |
94 | function DialogFooter({
95 |   className,
96 |   showCloseButton = false,
97 |   children,
98 |   ...props
99 | }): React.ComponentProps<"div"> & {
100 |   showCloseButton?: boolean;
101 | }) {
102 |   return (
103 |     <div
104 |       data-slot="dialog-footer"
105 |       className={cn(
106 |         "flex flex-col-reverse gap-2 sm:flex-row sm:justify-end",
107 |         className
108 |       )}
109 |       {...props}
110 |     >
111 |       {children}
112 |       {showCloseButton && (
113 |         <DialogPrimitive.Close asChild>
114 |           <Button variant="outline">Close</Button>
115 |         </DialogPrimitive.Close>
116 |       )}
117 |     </div>
118 |   );
119 | }
120 |
121 | function DialogTitle({
122 |   className,
123 |   ...props
124 | }): React.ComponentProps<typeof DialogPrimitive.Title> {
125 |   return (
126 |     <DialogPrimitive.Title
127 |       data-slot="dialog-title"
128 |       className={cn("text-lg leading-none font-semibold", className)}

```

```

129 |     {...props}
130 |   />
131 | );
132 | }
133 |
134 | function DialogDescription({
135 |   className,
136 |   ...props
137 | }: React.ComponentProps<typeof DialogPrimitive.Description>) {
138 |   return (
139 |     <DialogPrimitive.Description
140 |       data-slot="dialog-description"
141 |       className={cn("text-muted-foreground text-sm", className)}
142 |       {...props}
143 |     />
144 |   );
145 | }
146 |
147 | export {
148 |   Dialog,
149 |   DialogClose,
150 |   DialogContent,
151 |   DialogDescription,
152 |   DialogFooter,
153 |   DialogHeader,
154 |   DialogOverlay,
155 |   DialogPortal,
156 |   DialogTitle,
157 |   DialogTrigger,
158 | };
159 |

```

## FILE: src/components/ui/dropdown-menu.tsx

TYPE: .tsx | LINES: 1-258 | SHA256: c7c840aea78fde2c37797e0075f71d6c83a0cc06d6f588e97ee4b2006df55e22

```

1 | "use client";
2 |
3 | import * as React from "react";
4 | import { CheckIcon, ChevronRightIcon, CircleIcon } from "lucide-react";
5 | import { DropdownMenu as DropdownMenuPrimitive } from "radix-ui";
6 |
7 | import { cn } from "@lib/utills";
8 |
9 | function DropdownMenu({
10 |   ...props
11 | }: React.ComponentProps<typeof DropdownMenuPrimitive.Root>) {
12 |   return <DropdownMenuPrimitive.Root data-slot="dropdown-menu" {...props} />;
13 | }
14 |
15 | function DropdownMenuPortal({
16 |   ...props
17 | }: React.ComponentProps<typeof DropdownMenuPrimitive.Portal>) {
18 |   return (
19 |     <DropdownMenuPrimitive.Portal data-slot="dropdown-menu-portal" {...props} />
20 |   );
21 | }
22 |
23 | function DropdownMenuTrigger({
24 |   ...props
25 | }: React.ComponentProps<typeof DropdownMenuPrimitive.Trigger>) {
26 |   return (
27 |     <DropdownMenuPrimitive.Trigger
28 |       data-slot="dropdown-menu-trigger"
29 |       {...props}
30 |     />
31 |   );
32 | }
33 |
34 | function DropdownMenuContent({
35 |   className,
36 |   sideOffset = 4,
37 |   ...props
38 | }: React.ComponentProps<typeof DropdownMenuPrimitive.Content>) {
39 |   return (
40 |     <DropdownMenuPrimitive.Portal>
41 |       <DropdownMenuPrimitive.Content
42 |         data-slot="dropdown-menu-content"
43 |         sideOffset={sideOffset}
44 |         className={cn(
45 |           "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out dat
46 |           className
47 |         )}
48 |         {...props}
49 |       />
50 |     </DropdownMenuPrimitive.Portal>
51 |   );
52 | }
53 |
54 | function DropdownMenuGroup({
55 |   ...props
56 | }: React.ComponentProps<typeof DropdownMenuPrimitive.Group>) {
57 |   return (
58 |     <DropdownMenuPrimitive.Group data-slot="dropdown-menu-group" {...props} />
59 |   );
60 | }
61 |
62 | function DropdownMenuItem({
63 |   className,
64 |   inset,
65 |   variant = "default",
66 |   ...props
67 | }: React.ComponentProps<typeof DropdownMenuPrimitive.Item> & {
68 |   inset?: boolean;
69 |   variant?: "default" | "destructive";

```

```

70 | }) {
71 |   return (
72 |     <DropdownMenuPrimitive.Item
73 |       data-slot="dropdown-menu-item"
74 |       data-inset={inset}
75 |       data-variant={variant}
76 |       className={cn(
77 |         "focus:bg-accent focus:text-accent-foreground data-[variant=destructive]:text-destructive data-[varian
78 |         className
79 |       )}
80 |       {...props}
81 |     />
82 |   );
83 | }
84 |
85 | function DropdownMenuCheckboxItem({
86 |   className,
87 |   children,
88 |   checked,
89 |   ...props
90 | }: React.ComponentProps<typeof DropdownMenuPrimitive.CheckboxItem>) {
91 |   return (
92 |     <DropdownMenuPrimitive.CheckboxItem
93 |       data-slot="dropdown-menu-checkbox-item"
94 |       className={cn(
95 |         "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2 rounded-
96 |         className
97 |       )}
98 |       checked={checked}
99 |       {...props}
100 |     >
101 |       <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
102 |         <DropdownMenuPrimitive.ItemIndicator>
103 |           <CheckIcon className="size-4" />
104 |         </DropdownMenuPrimitive.ItemIndicator>
105 |       </span>
106 |       {children}
107 |     </DropdownMenuPrimitive.CheckboxItem>
108 |   );
109 | }
110 |
111 | function DropdownMenuRadioGroup({
112 |   ...props
113 | }: React.ComponentProps<typeof DropdownMenuPrimitive.RadioGroup>) {
114 |   return (
115 |     <DropdownMenuPrimitive.RadioGroup
116 |       data-slot="dropdown-menu-radio-group"
117 |       {...props}
118 |     />
119 |   );
120 | }
121 |
122 | function DropdownMenuRadioItem({
123 |   className,
124 |   children,
125 |   ...props
126 | }: React.ComponentProps<typeof DropdownMenuPrimitive.RadioItem>) {
127 |   return (
128 |     <DropdownMenuPrimitive.RadioItem
129 |       data-slot="dropdown-menu-radio-item"
130 |       className={cn(
131 |         "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2 rounded-
132 |         className
133 |       )}
134 |       {...props}
135 |     >
136 |       <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
137 |         <DropdownMenuPrimitive.ItemIndicator>
138 |           <CircleIcon className="size-2 fill-current" />
139 |         </DropdownMenuPrimitive.ItemIndicator>
140 |       </span>
141 |       {children}
142 |     </DropdownMenuPrimitive.RadioItem>
143 |   );
144 | }
145 |
146 | function DropdownMenuLabel({
147 |   className,
148 |   inset,
149 |   ...props
150 | }: React.ComponentProps<typeof DropdownMenuPrimitive.Label> & {
151 |   inset?: boolean;
152 | }) {
153 |   return (
154 |     <DropdownMenuPrimitive.Label
155 |       data-slot="dropdown-menu-label"
156 |       data-inset={inset}
157 |       className={cn(
158 |         "px-2 py-1.5 text-sm font-medium data-[inset]:pl-8",
159 |         className
160 |       )}
161 |       {...props}
162 |     />
163 |   );
164 | }
165 |
166 | function DropdownMenuSeparator({
167 |   className,
168 |   ...props
169 | }: React.ComponentProps<typeof DropdownMenuPrimitive.Separator>) {
170 |   return (
171 |     <DropdownMenuPrimitive.Separator
172 |       data-slot="dropdown-menu-separator"
173 |       className={cn("bg-border -mx-1 my-1 h-px", className)}

```

```

174 |     {...props}
175 |   />
176 | );
177 | }
178 |
179 | function DropdownMenuShortcut({
180 |   className,
181 |   ...props
182 | }: React.ComponentProps<"span">) {
183 |   return (
184 |     <span
185 |       data-slot="dropdown-menu-shortcut"
186 |       className={cn(
187 |         "text-muted-foreground ml-auto text-xs tracking-widest",
188 |         className
189 |       )}
190 |       {...props}
191 |     />
192 |   );
193 | }
194 |
195 | function DropdownMenuSub({
196 |   ...props
197 | }: React.ComponentProps<typeof DropdownMenuPrimitive.Sub>) {
198 |   return <DropdownMenuPrimitive.Sub data-slot="dropdown-menu-sub" {...props} />;
199 | }
200 |
201 | function DropdownMenuSubTrigger({
202 |   className,
203 |   inset,
204 |   children,
205 |   ...props
206 | }: React.ComponentProps<typeof DropdownMenuPrimitive.SubTrigger> & {
207 |   inset?: boolean;
208 | }) {
209 |   return (
210 |     <DropdownMenuPrimitive.SubTrigger
211 |       data-slot="dropdown-menu-sub-trigger"
212 |       data-inset={inset}
213 |       className={cn(
214 |         "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accen
215 |         className
216 |       )}
217 |       {...props}
218 |     >
219 |       {children}
220 |       <ChevronRightIcon className="ml-auto size-4" />
221 |     </DropdownMenuPrimitive.SubTrigger>
222 |   );
223 | }
224 |
225 | function DropdownMenuSubContent({
226 |   className,
227 |   ...props
228 | }: React.ComponentProps<typeof DropdownMenuPrimitive.SubContent>) {
229 |   return (
230 |     <DropdownMenuPrimitive.SubContent
231 |       data-slot="dropdown-menu-sub-content"
232 |       className={cn(
233 |         "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out data-
234 |         className
235 |       )}
236 |       {...props}
237 |     />
238 |   );
239 | }
240 |
241 | export {
242 |   DropdownMenu,
243 |   DropdownMenuPortal,
244 |   DropdownMenuTrigger,
245 |   DropdownMenuContent,
246 |   DropdownMenuGroup,
247 |   DropdownMenuLabel,
248 |   DropdownMenuItem,
249 |   DropdownMenuCheckboxItem,
250 |   DropdownMenuRadioGroup,
251 |   DropdownMenuRadioItem,
252 |   DropdownMenuSeparator,
253 |   DropdownMenuShortcut,
254 |   DropdownMenuSub,
255 |   DropdownMenuSubTrigger,
256 |   DropdownMenuSubContent,
257 | };
258 |

```

## FILE: src/components/ui/EdvToggle.tsx

TYPE: .tsx | LINES: 1-42 | SHA256: 7bfff406341418cd0b185c0a9cbd2268c1cfc5f30c477a69d80373760a9ec4b31

```

1 | import * as React from "react";
2 | import * as SwitchPrimitives from "@radix-ui/react-switch";
3 | import { cn } from "@lib/utls";
4 |
5 | export interface EdvToggleProps
6 |   extends React.ComponentPropsWithoutRef<typeof SwitchPrimitives.Root> {
7 |   noTick?: boolean;
8 | }
9 |
10 | /**
11 |  * Reusable prototype-exact toggle (.edv-toggle in shared.css).
12 |  * ON: Brand gradient / Teal track, knob moved right, visible white checkmark (' ).
13 |  * OFF: Muted gray track, knob left, no checkmark.
14 |  */
15 | export const EdvToggle = React.forwardRef<

```

```

16 | React.ComponentRef<typeof SwitchPrimitives.Root>,
17 | EdvToggleProps
18 | >(({ className, noTick = false, ...props }, ref) => (
19 |   <SwitchPrimitives.Root
20 |     className={cn(
21 |       "peer relative inline-flex h-[26px] w-[44px] shrink-0 cursor-pointer items-center rounded-full transio
22 |       className
23 |     )}
24 |     {...props}
25 |     ref={ref}
26 |   >
27 |     <SwitchPrimitives.Thumb
28 |       className={cn(
29 |         "pointer-events-none relative flex h-[20px] w-[20px] items-center justify-center rounded-full bg-white
30 |       )}
31 |   >
32 |     {!noTick && (
33 |       <span className="text-[11px] font-black text-teal-600 opacity-0 transition-opacity data-[state=checked]
34 |       |
35 |     </span>
36 |     )}
37 |   </SwitchPrimitives.Thumb>
38 | </SwitchPrimitives.Root>
39 | ));
40 |
41 | EdvToggle.displayName = "EdvToggle";
42 |

```

## FILE: src/components/ui/form.tsx

TYPE: .tsx | LINES: 1-167 | SHA256: 47b938e0d9cbef3ddfbfd2fa90c19decef2440d815a040f1dfd2469ce51a4a090

```

1 | import * as React from "react";
2 | import type { Label as LabelPrimitive } from "radix-ui";
3 | import { Slot } from "radix-ui";
4 | import {
5 |   Controller,
6 |   FormProvider,
7 |   useFormContext,
8 |   useFormState,
9 |   type ControllerProps,
10 |   type FieldPath,
11 |   type FieldValues,
12 | } from "react-hook-form";
13 |
14 | import { cn } from "@lib/utils";
15 | import { Label } from "@components/ui/label";
16 |
17 | const Form = FormProvider;
18 |
19 | type FormFieldContextValue<
20 |   TFieldValues extends FieldValues = FieldValues,
21 |   TName extends FieldPath<TFieldValues> = FieldPath<TFieldValues>
22 | > = {
23 |   name: TName;
24 | };
25 |
26 | const FormFieldContext = React.createContext<FormFieldContextValue>(
27 |   {} as FormFieldContextValue
28 | );
29 |
30 | const FormField = <
31 |   TFieldValues extends FieldValues = FieldValues,
32 |   TName extends FieldPath<TFieldValues> = FieldPath<TFieldValues>
33 | >({
34 |   ...props
35 | }): ControllerProps<TFieldValues, TName> => {
36 |   return (
37 |     <FormFieldContext.Provider value={{ name: props.name }}>
38 |       <Controller {...props} />
39 |     </FormFieldContext.Provider>
40 |   );
41 | };
42 |
43 | const useFormField = () => {
44 |   const fieldContext = React.useContext(FormFieldContext);
45 |   const itemContext = React.useContext(FormItemContext);
46 |   const { getFieldState } = useFormContext();
47 |   const formState = useFormState({ name: fieldContext.name });
48 |   const fieldState = getFieldState(fieldContext.name, formState);
49 |
50 |   if (!fieldContext) {
51 |     throw new Error("useFormField should be used within <FormField>");
52 |   }
53 |
54 |   const { id } = itemContext;
55 |
56 |   return {
57 |     id,
58 |     name: fieldContext.name,
59 |     formItemId: `${id}-form-item`,
60 |     formDescriptionId: `${id}-form-item-description`,
61 |     formMessageId: `${id}-form-item-message`,
62 |     ...fieldState,
63 |   };
64 | };
65 |
66 | type FormItemContextValue = {
67 |   id: string;
68 | };
69 |
70 | const FormItemContext = React.createContext<FormItemContextValue>(
71 |   {} as FormItemContextValue
72 | );
73 |

```

```

74 | function FormItem({ className, ...props }: React.ComponentProps<"div">) {
75 |   const id = React.useId();
76 |
77 |   return (
78 |     <FormItemContext.Provider value={{ id }}>
79 |       <div
80 |         data-slot="form-item"
81 |         className={cn("grid gap-2", className)}
82 |         {...props}
83 |       />
84 |     </FormItemContext.Provider>
85 |   );
86 | }
87 |
88 | function FormLabel({
89 |   className,
90 |   ...props
91 | }: React.ComponentProps<typeof LabelPrimitive.Root>) {
92 |   const { error, formItemId } = useFormField();
93 |
94 |   return (
95 |     <Label
96 |       data-slot="form-label"
97 |       data-error={!!error}
98 |       className={cn("data-[error=true]:text-destructive", className)}
99 |       htmlFor={formItemId}
100 |      {...props}
101 |    />
102 |   );
103 | }
104 |
105 | function FormControl({ ...props }: React.ComponentProps<typeof Slot.Root>) {
106 |   const { error, formItemId, formDescriptionId, formMessageId } =
107 |     useFormField();
108 |
109 |   return (
110 |     <Slot.Root
111 |       data-slot="form-control"
112 |       id={formItemId}
113 |       aria-describedby={
114 |         !error
115 |           ? `${formDescriptionId}`
116 |           : `${formDescriptionId} ${formMessageId}`
117 |       }
118 |       aria-invalid={!!error}
119 |       {...props}
120 |     />
121 |   );
122 | }
123 |
124 | function FormDescription({ className, ...props }: React.ComponentProps<"p">) {
125 |   const { formDescriptionId } = useFormField();
126 |
127 |   return (
128 |     <p
129 |       data-slot="form-description"
130 |       id={formDescriptionId}
131 |       className={cn("text-muted-foreground text-sm", className)}
132 |       {...props}
133 |     />
134 |   );
135 | }
136 |
137 | function FormMessage({ className, ...props }: React.ComponentProps<"p">) {
138 |   const { error, formMessageId } = useFormField();
139 |   const body = error ? String(error?.message ?? "") : props.children;
140 |
141 |   if (!body) {
142 |     return null;
143 |   }
144 |
145 |   return (
146 |     <p
147 |       data-slot="form-message"
148 |       id={formMessageId}
149 |       className={cn("text-destructive text-sm", className)}
150 |       {...props}
151 |     >
152 |       {body}
153 |     </p>
154 |   );
155 | }
156 |
157 | export {
158 |   useFormField,
159 |   Form,
160 |   FormItem,
161 |   FormLabel,
162 |   FormControl,
163 |   FormDescription,
164 |   FormMessage,
165 |   FormField,
166 | };
167 |

```

FILE: src/components/ui/input.tsx

TYPE: .tsx | LINES: 1-26 | SHA256: 5fcd5b7d7b66965a54af578246797ee4416c4a7de202e002af7492fe155b1954

```

1 | import * as React from "react";
2 |
3 | import { cn } from "@lib/utls";
4 |
5 | const Input = React.forwardRef<HTMLInputElement, React.ComponentProps<"input">>>(
6 |   ({ className, type, ...props }, ref) => {

```



```

7 |     return (
8 |       <input
9 |         type={type}
10 |         data-slot="input"
11 |         className={cn(
12 |           "file:text-foreground placeholder:text-muted-foreground selection:bg-primary selection:text-primary-
13 |             focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px]",
14 |           "aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40 aria-invalid:border-destruct
15 |             className
16 |         )}
17 |         ref={ref}
18 |         {...props}
19 |       />
20 |     );
21 |   }
22 | };
23 | Input.displayName = "Input";
24 |
25 | export { Input };
26 |

```

#### FILE: src/components/ui/label.tsx

TYPE: .tsx | LINES: 1-23 | SHA256: ab8ed276c7215eebed685029f4c991155994c756aea8de5f242261bd766b2f6c

```

1 | import * as React from "react";
2 | import { Label as LabelPrimitive } from "radix-ui";
3 |
4 | import { cn } from "@lib/utils";
5 |
6 | function Label({
7 |   className,
8 |   ...props
9 | }: React.ComponentProps<typeof LabelPrimitive.Root>) {
10 |   return (
11 |     <LabelPrimitive.Root
12 |       data-slot="label"
13 |       className={cn(
14 |         "flex items-center gap-2 text-sm leading-none font-medium text-gray-700 select-none group-data-[disabl
15 |         className
16 |       )}
17 |       {...props}
18 |     />
19 |   );
20 | }
21 |
22 | export { Label };
23 |

```

#### FILE: src/components/ui/popover.tsx

TYPE: .tsx | LINES: 1-30 | SHA256: 7a300fa2f5c1e3392fb8b6363e9ffbbbbb388c3a7a8090a8b73727725e7da9b4e

```

1 | import * as React from "react";
2 | import * as PopoverPrimitive from "@radix-ui/react-popover";
3 |
4 | import { cn } from "@lib/utils";
5 |
6 | const Popover = PopoverPrimitive.Root;
7 |
8 | const PopoverTrigger = PopoverPrimitive.Trigger;
9 |
10 | const PopoverContent = React.forwardRef<
11 |   React.ElementRef<typeof PopoverPrimitive.Content>,
12 |   React.ComponentPropsWithoutRef<typeof PopoverPrimitive.Content>
13 | >(({ className, align = "center", sideOffset = 4, ...props }, ref) => (
14 |   <PopoverPrimitive.Portal>
15 |     <PopoverPrimitive.Content
16 |       ref={ref}
17 |       align={align}
18 |       sideOffset={sideOffset}
19 |       className={cn(
20 |         "z-50 w-72 rounded-md border bg-popover p-4 text-popover-foreground shadow-md outline-none data-[state
21 |         className
22 |       )}
23 |       {...props}
24 |     />
25 |   </PopoverPrimitive.Portal>
26 | ));
27 | PopoverContent.displayName = PopoverPrimitive.Content.displayName;
28 |
29 | export { Popover, PopoverTrigger, PopoverContent };
30 |

```

#### FILE: src/components/ui/select.tsx

TYPE: .tsx | LINES: 1-189 | SHA256: a9ae4027c4eaafcac65e39824db2fc1fe618ba65535e3cb9f881072056540280

```

1 | import * as React from "react";
2 | import { CheckIcon, ChevronDownIcon, ChevronUpIcon } from "lucide-react";
3 | import { Select as SelectPrimitive } from "radix-ui";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | function Select({
8 |   ...props
9 | }: React.ComponentProps<typeof SelectPrimitive.Root>) {
10 |   return <SelectPrimitive.Root data-slot="select" {...props} />;
11 | }
12 |
13 | function SelectGroup({
14 |   ...props
15 | }: React.ComponentProps<typeof SelectPrimitive.Group>) {
16 |   return <SelectPrimitive.Group data-slot="select-group" {...props} />;
17 | }
18 |

```

```

19 | function SelectValue({
20 |   ...props
21 | }): React.ComponentProps<typeof SelectPrimitive.Value> {
22 |   return <SelectPrimitive.Value data-slot="select-value" {...props} />;
23 | }
24 |
25 | function SelectTrigger({
26 |   className,
27 |   size = "default",
28 |   children,
29 |   ...props
30 | }): React.ComponentProps<typeof SelectPrimitive.Trigger> & {
31 |   size?: "sm" | "default";
32 | }) {
33 |   return (
34 |     <SelectPrimitive.Trigger
35 |       data-slot="select-trigger"
36 |       data-size={size}
37 |       className={cn(
38 |         "border-input data-[placeholder]:text-muted-foreground [&_svg:not([class*='text-']):text-muted-foregr
39 |         className
40 |       )}
41 |       {...props}
42 |     >
43 |       {children}
44 |       <SelectPrimitive.Icon asChild>
45 |         <ChevronDownIcon className="size-4 opacity-50" />
46 |       </SelectPrimitive.Icon>
47 |     </SelectPrimitive.Trigger>
48 |   );
49 | }
50 |
51 | function SelectContent({
52 |   className,
53 |   children,
54 |   position = "item-aligned",
55 |   align = "center",
56 |   ...props
57 | }): React.ComponentProps<typeof SelectPrimitive.Content> {
58 |   return (
59 |     <SelectPrimitive.Portal>
60 |       <SelectPrimitive.Content
61 |         data-slot="select-content"
62 |         className={cn(
63 |           "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out dat
64 |           position === "popper" &&
65 |           "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1 data-[side=right]:translate-x-1
66 |           className
67 |         )}
68 |         position={position}
69 |         align={align}
70 |         {...props}
71 |       >
72 |         <SelectScrollUpButton />
73 |         <SelectPrimitive.Viewport
74 |           className={cn(
75 |             "p-1",
76 |             position === "popper" &&
77 |             "h-[var(--radix-select-trigger-height)] w-full min-w-[var(--radix-select-trigger-width)] scroll-
78 |           )}
79 |       >
80 |         {children}
81 |       </SelectPrimitive.Viewport>
82 |       <SelectScrollDownButton />
83 |     </SelectPrimitive.Content>
84 |   </SelectPrimitive.Portal>
85 | );
86 | }
87 |
88 | function SelectLabel({
89 |   className,
90 |   ...props
91 | }): React.ComponentProps<typeof SelectPrimitive.Label> {
92 |   return (
93 |     <SelectPrimitive.Label
94 |       data-slot="select-label"
95 |       className={cn("text-muted-foreground px-2 py-1.5 text-xs", className)}
96 |       {...props}
97 |     </>
98 |   );
99 | }
100 |
101 | function SelectItem({
102 |   className,
103 |   children,
104 |   ...props
105 | }): React.ComponentProps<typeof SelectPrimitive.Item> {
106 |   return (
107 |     <SelectPrimitive.Item
108 |       data-slot="select-item"
109 |       className={cn(
110 |         "focus:bg-accent focus:text-accent-foreground [&_svg:not([class*='text-']):text-muted-foreground rela
111 |       className
112 |     )}
113 |     {...props}
114 |   >
115 |     <span
116 |       data-slot="select-item-indicator"
117 |       className="absolute right-2 flex size-3.5 items-center justify-center"
118 |     >
119 |       <SelectPrimitive.ItemIndicator>
120 |         <CheckIcon className="size-4" />
121 |       </SelectPrimitive.ItemIndicator>
122 |     </span>

```

```

123 |         <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
124 |       </SelectPrimitive.Item>
125 |     );
126 |   }
127 |
128 |   function SelectSeparator({
129 |     className,
130 |     ...props
131 |   }: React.ComponentProps<typeof SelectPrimitive.Separator>) {
132 |     return (
133 |       <SelectPrimitive.Separator
134 |         data-slot="select-separator"
135 |         className={cn("bg-border pointer-events-none -mx-1 my-1 h-px", className)}
136 |         {...props}
137 |       >/>
138 |     );
139 |   }
140 |
141 |   function SelectScrollUpButton({
142 |     className,
143 |     ...props
144 |   }: React.ComponentProps<typeof SelectPrimitive.ScrollUpButton>) {
145 |     return (
146 |       <SelectPrimitive.ScrollUpButton
147 |         data-slot="select-scroll-up-button"
148 |         className={cn(
149 |           "flex cursor-default items-center justify-center py-1",
150 |           className
151 |         )}
152 |         {...props}
153 |       >
154 |         <ChevronUpIcon className="size-4" />
155 |       </SelectPrimitive.ScrollUpButton>
156 |     );
157 |   }
158 |
159 |   function SelectScrollDownButton({
160 |     className,
161 |     ...props
162 |   }: React.ComponentProps<typeof SelectPrimitive.ScrollDownButton>) {
163 |     return (
164 |       <SelectPrimitive.ScrollDownButton
165 |         data-slot="select-scroll-down-button"
166 |         className={cn(
167 |           "flex cursor-default items-center justify-center py-1",
168 |           className
169 |         )}
170 |         {...props}
171 |       >
172 |         <ChevronDownIcon className="size-4" />
173 |       </SelectPrimitive.ScrollDownButton>
174 |     );
175 |   }
176 |
177 |   export {
178 |     Select,
179 |     SelectContent,
180 |     SelectGroup,
181 |     SelectItem,
182 |     SelectLabel,
183 |     SelectScrollDownButton,
184 |     SelectScrollUpButton,
185 |     SelectSeparator,
186 |     SelectTrigger,
187 |     SelectValue,
188 |   };
189 |

```

## FILE: src/components/ui/separator.tsx

TYPE: .tsx | LINES: 1-29 | SHA256: c7d600dd7c8428a9c13f6d81f2597da5daaa1c34735d14285dc8bb3a0c71dfba

```

1 | "use client";
2 |
3 | import * as React from "react";
4 | import { Separator as SeparatorPrimitive } from "radix-ui";
5 |
6 | import { cn } from "@lib/utlils";
7 |
8 | function Separator({
9 |   className,
10 |   orientation = "horizontal",
11 |   decorative = true,
12 |   ...props
13 | }: React.ComponentProps<typeof SeparatorPrimitive.Root>) {
14 |   return (
15 |     <SeparatorPrimitive.Root
16 |       data-slot="separator"
17 |       decorative={decorative}
18 |       orientation={orientation}
19 |       className={cn(
20 |         "bg-border shrink-0 data-[orientation=horizontal]:h-px data-[orientation=horizontal]:w-full data-[orie
21 |         className
22 |       )}
23 |       {...props}
24 |     >/>
25 |   );
26 | }
27 |
28 | export { Separator };
29 |

```

# FILE: src/components/ui/sheet.tsx

TYPE: .tsx | LINES: 1-142 | SHA256: a8341d7d90e68f3cff54a7ea04c955570a3f0b08b5676e3bf531fe1ade54a85b

```
1 | import * as React from "react";
2 | import { XIcon } from "lucide-react";
3 | import { Dialog as SheetPrimitive } from "radix-ui";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | function Sheet({ ...props }: React.ComponentProps<typeof SheetPrimitive.Root>) {
8 |   return <SheetPrimitive.Root data-slot="sheet" {...props} />;
9 | }
10 |
11 | function SheetTrigger({
12 |   ...props
13 | }: React.ComponentProps<typeof SheetPrimitive.Trigger>) {
14 |   return <SheetPrimitive.Trigger data-slot="sheet-trigger" {...props} />;
15 | }
16 |
17 | function SheetClose({
18 |   ...props
19 | }: React.ComponentProps<typeof SheetPrimitive.Close>) {
20 |   return <SheetPrimitive.Close data-slot="sheet-close" {...props} />;
21 | }
22 |
23 | function SheetPortal({
24 |   ...props
25 | }: React.ComponentProps<typeof SheetPrimitive.Portal>) {
26 |   return <SheetPrimitive.Portal data-slot="sheet-portal" {...props} />;
27 | }
28 |
29 | function SheetOverlay({
30 |   className,
31 |   ...props
32 | }: React.ComponentProps<typeof SheetPrimitive.Overlay>) {
33 |   return (
34 |     <SheetPrimitive.Overlay
35 |       data-slot="sheet-overlay"
36 |       className={cn(
37 |         "data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[sta
38 |         className
39 |       )}
40 |       {...props}
41 |     />
42 |   );
43 | }
44 |
45 | function SheetContent({
46 |   className,
47 |   children,
48 |   side = "right",
49 |   showCloseButton = true,
50 |   ...props
51 | }: React.ComponentProps<typeof SheetPrimitive.Content> & {
52 |   side?: "top" | "right" | "bottom" | "left";
53 |   showCloseButton?: boolean;
54 | }) {
55 |   return (
56 |     <SheetPortal>
57 |       <SheetOverlay />
58 |       <SheetPrimitive.Content
59 |         data-slot="sheet-content"
60 |         className={cn(
61 |           "bg-background data-[state=open]:animate-in data-[state=closed]:animate-out fixed z-50 flex flex-col
62 |           side === "right" &&
63 |             "data-[state=closed]:slide-out-to-right data-[state=open]:slide-in-from-right inset-y-0 right-0 h-
64 |             side === "left" &&
65 |             "data-[state=closed]:slide-out-to-left data-[state=open]:slide-in-from-left inset-y-0 left-0 h-ful
66 |             side === "top" &&
67 |             "data-[state=closed]:slide-out-to-top data-[state=open]:slide-in-from-top inset-x-0 top-0 h-auto b
68 |             side === "bottom" &&
69 |             "data-[state=closed]:slide-out-to-bottom data-[state=open]:slide-in-from-bottom inset-x-0 bottom-0
70 |           className
71 |         )}
72 |         {...props}
73 |       >
74 |         {children}
75 |         {showCloseButton && (
76 |           <SheetPrimitive.Close className="ring-offset-background focus:ring-ring data-[state=open]:bg-seconda
77 |             <XIcon className="size-4" />
78 |             <span className="sr-only">Close</span>
79 |           </SheetPrimitive.Close>
80 |         )}
81 |       </SheetPrimitive.Content>
82 |     </SheetPortal>
83 |   );
84 | }
85 |
86 | function SheetHeader({ className, ...props }: React.ComponentProps<"div">) {
87 |   return (
88 |     <div
89 |       data-slot="sheet-header"
90 |       className={cn("flex flex-col gap-1.5 p-4", className)}
91 |       {...props}
92 |     />
93 |   );
94 | }
95 |
96 | function SheetFooter({ className, ...props }: React.ComponentProps<"div">) {
97 |   return (
98 |     <div
99 |       data-slot="sheet-footer"
100 |       className={cn("mt-auto flex flex-col gap-2 p-4", className)}
101 |       {...props}
```

```

102 |     />
103 |   );
104 | }
105 |
106 | function SheetTitle({
107 |   className,
108 |   ...props
109 | }: React.ComponentProps<typeof SheetPrimitive.Title>) {
110 |   return (
111 |     <SheetPrimitive.Title
112 |       data-slot="sheet-title"
113 |       className={cn("text-foreground font-semibold", className)}
114 |       {...props}
115 |     />
116 |   );
117 | }
118 |
119 | function SheetDescription({
120 |   className,
121 |   ...props
122 | }: React.ComponentProps<typeof SheetPrimitive.Description>) {
123 |   return (
124 |     <SheetPrimitive.Description
125 |       data-slot="sheet-description"
126 |       className={cn("text-muted-foreground text-sm", className)}
127 |       {...props}
128 |     />
129 |   );
130 | }
131 |
132 | export {
133 |   Sheet,
134 |   SheetTrigger,
135 |   SheetClose,
136 |   SheetContent,
137 |   SheetHeader,
138 |   SheetFooter,
139 |   SheetTitle,
140 |   SheetDescription,
141 | };
142 |

```

#### FILE: src/components/ui/sonner.tsx

TYPE: .tsx | LINES: 1-50 | SHA256: b9ae57ea5b977d20f9da00358f695594ed1773f0245e3b31728018ca9c214179

```

1 | import {
2 |   CircleCheckIcon,
3 |   InfoIcon,
4 |   Loader2Icon,
5 |   OctagonXIcon,
6 |   TriangleAlertIcon,
7 | } from "lucide-react";
8 | import { Toaster as Sonner, type ToasterProps } from "sonner";
9 |
10 | const Toaster = ({ ...props }: ToasterProps) => {
11 |   const { toastOptions, ...rest } = props;
12 |
13 |   return (
14 |     <Sonner
15 |       theme="system"
16 |       className="toaster group lg:ms-[132px]"
17 |       icons={{
18 |         success: <CircleCheckIcon className="size-4" />,
19 |         info: <InfoIcon className="size-4" />,
20 |         warning: <TriangleAlertIcon className="size-4" />,
21 |         error: <OctagonXIcon className="size-4" />,
22 |         loading: <Loader2Icon className="size-4 animate-spin" />,
23 |       }}
24 |       style={
25 |         {
26 |           "--normal-bg": "var(--popover)",
27 |           "--normal-text": "var(--popover-foreground)",
28 |           "--normal-border": "var(--border)",
29 |           "--border-radius": "12px",
30 |         } as React.CSSProperties
31 |       }
32 |       toastOptions={{
33 |         ...toastOptions,
34 |         classNames: {
35 |           toast:
36 |             "!border-transparent !bg-slate-900 !text-white !shadow-[0_12px_32px_rgba(0,0,0,0.28)]",
37 |           title: "!text-[13px] !font-semibold !text-white",
38 |           description: "!text-xs !text-slate-300",
39 |           actionButton: "!bg-white !text-slate-900",
40 |           cancelButton: "!bg-slate-700 !text-white",
41 |           ...toastOptions?.classNames,
42 |         },
43 |       }}
44 |       {...rest}
45 |     />
46 |   );
47 | };
48 |
49 | export { Toaster };
50 |

```

#### FILE: src/components/ui/switch.tsx

TYPE: .tsx | LINES: 1-28 | SHA256: d2390b8d38149f231b2011413e1e40a1f8e3acfc72060db25aa9316b1f219a7a

```

1 | import * as React from "react";
2 | import * as SwitchPrimitives from "@radix-ui/react-switch";
3 |
4 | import { cn } from "@lib/utills";
5 |

```

```

6 | const Switch = React.forwardRef<
7 |   React.ComponentRef<typeof SwitchPrimitives.Root>,
8 |   React.ComponentPropsWithoutRef<typeof SwitchPrimitives.Root>
9 | >(({ className, ...props }, ref) => (
10 |   <SwitchPrimitives.Root
11 |     className={cn(
12 |       "peer inline-flex h-5 w-9 shrink-0 cursor-pointer items-center rounded-full border-2 border-transparent
13 |       className
14 |     )}
15 |     {...props}
16 |     ref={ref}
17 |   >
18 |     <SwitchPrimitives.Thumb
19 |       className={cn(
20 |         "pointer-events-none block h-4 w-4 rounded-full bg-white shadow-lg ring-0 transition-transform data-[s
21 |         )}
22 |       />
23 |     </SwitchPrimitives.Root>
24 |   ));
25 | Switch.displayName = SwitchPrimitives.Root.displayName;
26 |
27 | export { Switch };
28 |

```

## FILE: src/components/ui/table.tsx

TYPE: .tsx | LINES: 1-115 | SHA256: 085566291cf03c9fea590aa9b77339feebf04cf8ce23494a3667f84bf4811e29

```

1 | import * as React from "react";
2 |
3 | import { cn } from "@lib/utlis";
4 |
5 | function Table({ className, ...props }: React.ComponentProps<"table">) {
6 |   return (
7 |     <div
8 |       data-slot="table-container"
9 |       className="relative w-full overflow-x-auto"
10 |     >
11 |       <table
12 |         data-slot="table"
13 |         className={cn("w-full caption-bottom text-sm", className)}
14 |         {...props}
15 |       />
16 |     </div>
17 |   );
18 | }
19 |
20 | function TableHeader({ className, ...props }: React.ComponentProps<"thead">) {
21 |   return (
22 |     <thead
23 |       data-slot="table-header"
24 |       className={cn("[&_tr]:border-b", className)}
25 |       {...props}
26 |     />
27 |   );
28 | }
29 |
30 | function TableBody({ className, ...props }: React.ComponentProps<"tbody">) {
31 |   return (
32 |     <tbody
33 |       data-slot="table-body"
34 |       className={cn("[&_tr:last-child]:border-0", className)}
35 |       {...props}
36 |     />
37 |   );
38 | }
39 |
40 | function TableFooter({ className, ...props }: React.ComponentProps<"tfoot">) {
41 |   return (
42 |     <tfoot
43 |       data-slot="table-footer"
44 |       className={cn(
45 |         "bg-muted/50 border-t font-medium [&>tr]:last:border-b-0",
46 |         className
47 |       )}
48 |       {...props}
49 |     />
50 |   );
51 | }
52 |
53 | function TableRow({ className, ...props }: React.ComponentProps<"tr">) {
54 |   return (
55 |     <tr
56 |       data-slot="table-row"
57 |       className={cn(
58 |         "hover:bg-muted/50 data-[state=selected]:bg-muted border-b transition-colors",
59 |         className
60 |       )}
61 |       {...props}
62 |     />
63 |   );
64 | }
65 |
66 | function TableHead({ className, ...props }: React.ComponentProps<"th">) {
67 |   return (
68 |     <th
69 |       data-slot="table-head"
70 |       className={cn(
71 |         "text-foreground h-10 px-2 text-left align-middle font-medium whitespace-nowrap [&:has([role=checkbox]
72 |         className
73 |       )}
74 |       {...props}
75 |     />
76 |   );
77 | }

```

```

78 |
79 | function TableCell({ className, ...props }: React.ComponentProps<"td">) {
80 |   return (
81 |     <td
82 |       data-slot="table-cell"
83 |       className={cn(
84 |         "p-2 align-middle whitespace-nowrap [&:has([role=checkbox])]:pr-0 [&[role=checkbox]]:translate-y-[2px",
85 |         className
86 |       )}
87 |       {...props}
88 |     />
89 |   );
90 | }
91 |
92 | function TableCaption({
93 |   className,
94 |   ...props
95 | }: React.ComponentProps<"caption">) {
96 |   return (
97 |     <caption
98 |       data-slot="table-caption"
99 |       className={cn("text-muted-foreground mt-4 text-sm", className)}
100 |       {...props}
101 |     />
102 |   );
103 | }
104 |
105 | export {
106 |   Table,
107 |   TableHeader,
108 |   TableBody,
109 |   TableFooter,
110 |   TableHead,
111 |   TableRow,
112 |   TableCell,
113 |   TableCaption,
114 | };
115 |

```

FILE: [src/components/ui/tabs.tsx](#)

TYPE: .tsx | LINES: 1-92 | SHA256: 8f837209f09b369d1b7f636b72916517e48f1404baa79ef45a8c62c887037f82

```

1 | "use client";
2 |
3 | import * as React from "react";
4 | import { cva, type VariantProps } from "class-variance-authority";
5 | import { Tabs as TabsPrimitive } from "radix-ui";
6 |
7 | import { cn } from "@lib/utlis";
8 |
9 | function Tabs({
10 |   className,
11 |   orientation = "horizontal",
12 |   ...props
13 | }: React.ComponentProps<typeof TabsPrimitive.Root>) {
14 |   return (
15 |     <TabsPrimitive.Root
16 |       data-slot="tabs"
17 |       data-orientation={orientation}
18 |       orientation={orientation}
19 |       className={cn(
20 |         "group/tabs flex gap-2 data-[orientation=horizontal]:flex-col",
21 |         className
22 |       )}
23 |       {...props}
24 |     />
25 |   );
26 | }
27 |
28 | const tabsListVariants = cva(
29 |   "rounded-lg p-[3px] group-data-[orientation=horizontal]/tabs:h-9 data-[variant=line]:rounded-none group/tabs",
30 |   {
31 |     variants: {
32 |       variant: {
33 |         default: "bg-muted",
34 |         line: "gap-1 bg-transparent",
35 |       },
36 |     },
37 |     defaultVariants: {
38 |       variant: "default",
39 |     },
40 |   },
41 | );
42 |
43 | function TabsList({
44 |   className,
45 |   variant = "default",
46 |   ...props
47 | }: React.ComponentProps<typeof TabsPrimitive.List> &
48 | VariantProps<typeof tabsListVariants>) {
49 |   return (
50 |     <TabsPrimitive.List
51 |       data-slot="tabs-list"
52 |       data-variant={variant}
53 |       className={cn(tabsListVariants({ variant }), className)}
54 |       {...props}
55 |     />
56 |   );
57 | }
58 |
59 | function TabsTrigger({
60 |   className,
61 |   ...props
62 | }: React.ComponentProps<typeof TabsPrimitive.Trigger>) {

```

```

63 |   return (
64 |     <TabsPrimitive.Trigger
65 |       data-slot="tabs-trigger"
66 |       className={cn(
67 |         "focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:outline-ring text-foreground/60 ho
68 |         "group-data-[variant=line]/tabs-list:bg-transparent group-data-[variant=line]/tabs-list:data-[state=ac
69 |         "data-[state=active]:bg-background dark:data-[state=active]:text-foreground dark:data-[state=active]:b
70 |         "after:bg-foreground after:absolute after:opacity-0 after:transition-opacity group-data-[orientation=h
71 |         className
72 |       )}
73 |     {...props}
74 |   ) />
75 | );
76 | }
77 |
78 | function TabsContent({
79 |   className,
80 |   ...props
81 | }: React.ComponentProps<typeof TabsPrimitive.Content>) {
82 |   return (
83 |     <TabsPrimitive.Content
84 |       data-slot="tabs-content"
85 |       className={cn("flex-1 outline-none", className)}
86 |       {...props}
87 |     ) />
88 |   );
89 | }
90 |
91 | export { Tabs, TabsList, TabsTrigger, TabsContent, tabsListVariants };
92 |

```

#### FILE: src/components/ui/textarea.tsx

TYPE: .tsx | LINES: 1-21 | SHA256: 6dd48083645afcc5c36a305c65f35aaffff904354c6998dabbb5928b3e6eda7b7

```

1 | import * as React from "react";
2 |
3 | import { cn } from "@lib/utills";
4 |
5 | function Textarea({ className, ...props }: React.ComponentProps<"textarea">) {
6 |   return (
7 |     <textarea
8 |       data-slot="textarea"
9 |       className={cn(
10 |         "placeholder:text-muted-foreground selection:bg-primary selection:text-primary-foreground dark:bg-inpu
11 |         "focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px]",
12 |         "aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructiv
13 |         className
14 |       )}
15 |       {...props}
16 |     ) />
17 |   );
18 | }
19 |
20 | export { Textarea };
21 |

```

#### FILE: src/design-system/index.ts

TYPE: .ts | LINES: 1-28 | SHA256: 57609105e07d272feedc6aeacef3ab154088cdda1b5403e4b6985ac9b130a451

```

1 | // =====
2 | // Design system – single import surface (prototype design language).
3 | //
4 | // Screens should build from HERE, not from the legacy `@/components/shared/*`
5 | // app components. This makes the eventual transformation to the prototype UI a
6 | // mechanical swap of imports rather than a rewrite.
7 | //
8 | //   import { PageHeader, SectionCard, KPICard, EMeter, Button, Card } from "@design-system";
9 | //
10 | // Layers:
11 | //   - primitives/ – Shadcn ui/* adopted as the L2 primitive layer.
12 | //   - patterns/ – prototype-faithful compositions (SectionHeader, KPICard,
13 | //     PCard, SectionCard, HeroCard, StatusDot, StatePanel, EMeter).
14 | //   - mascot/ – living character system (Foxi/Owlie/Pengu).
15 | //   - tokens.css – L1 foundations (imported at cutover; the live index.css
16 | //     already carries the canonical 93.65deg --brand-gradient).
17 | // =====
18 |
19 | export * from "../primitives";
20 | export * from "../patterns";
21 |
22 | // Mascot: pure catalog/resolvers + the two components.
23 | export * from "../mascot";
24 | export { default as MascotCharacter } from "../mascot/MascotCharacter";
25 | export type { MascotCharacterProps } from "../mascot/MascotCharacter";
26 | export { default as MascotCompanion } from "../mascot/MascotCompanion";
27 | export type { MascotCompanionProps } from "../mascot/MascotCompanion";
28 |

```

#### FILE: src/design-system/mascot/characters.ts

TYPE: .ts | LINES: 1-354 | SHA256: 258e6b9e17ecf3954581009ae9ed4976563d976575dd7250be1a2b36b3fd3d04

```

1 | // =====
2 | // Mascot character system – catalog + pure resolvers (L2)
3 | // =====
4 | //
5 | // Living mascots: Foxi (companion) · Owlie (AI tutor) · Pengu (habit buddy).
6 | // Ported 1:1 from the prototype's `EdvCharacter` system (prototype/shared.js)
7 | // and its assets (prototype/characters/*), which are copied into
8 | // ./assets/characters/* so `src/` never imports from `prototype/` (guardrail G.2).
9 | //
10 | // Every appearance maps to a real product / gamification state (greeting, XP,
11 | // level-up, streak, weak CLO, tutor, reflection). Emotion !' filename is the
12 | // authoritative map from the prototype; missing emotions fall back to `default`.

```



```

13 | //
14 | // This module is PURE (no JSX/DOM) so it is fully unit- + property-testable and
15 | // safe to re-export from the mascot barrel.
16 | // =====
17 |
18 | import type { MascotMomentId } from "@lib/mascotGuidance";
19 |
20 | /** The three living mascots. */
21 | export type CharacterId = "foxi" | "penguin" | "owl";
22 |
23 | /** Product role each character embodies (drives default selection). */
24 | export type CharacterRole = "companion" | "tutor" | "habit-buddy";
25 |
26 | /** Expressive emotions (superset across characters; each maps to a PNG pose). */
27 | export type CharacterEmotion =
28 |   | "default"
29 |   | "happy"
30 |   | "excited"
31 |   | "celebrating"
32 |   | "proud"
33 |   | "playful"
34 |   | "smart"
35 |   | "studious"
36 |   | "focused"
37 |   | "curious"
38 |   | "encouraging"
39 |   | "magical"
40 |   | "wise"
41 |   | "worried"
42 |   | "concerned"
43 |   | "nervous"
44 |   | "annoyed"
45 |   | "level-up"
46 |   | "victory"
47 |   | "thinking"
48 |   | "walking"
49 |   | "beacon"
50 |   | "signal";
51 |
52 | /** Render sizes (px) – ported from the prototype `chr-` scale. */
53 | export type CharactersSize = "xs" | "sm" | "md" | "lg" | "xl";
54 |
55 | /** Motion presets – ported from the prototype character keyframes. */
56 | export type CharacterAnimation =
57 |   | "none"
58 |   | "float"
59 |   | "in"
60 |   | "wave"
61 |   | "pop"
62 |   | "nudge";
63 |
64 | export const CHARACTER_IDS: readonly CharacterId[] = ["foxi", "penguin", "owl"];
65 |
66 | export const CHARACTER_NAMES: Record<CharacterId, string> = {
67 |   foxi: "Foxy",
68 |   penguin: "Pengu",
69 |   owl: "Owlie",
70 | };
71 |
72 | export const CHARACTER_ROLES: Record<CharacterId, CharacterRole> = {
73 |   foxi: "companion",
74 |   owl: "tutor",
75 |   penguin: "habit-buddy",
76 | };
77 |
78 | /** Exact prototype `chr-` pixel sizes. */
79 | export const SIZE_PX: Record<CharacterSize, number> = {
80 |   xs: 46,
81 |   sm: 64,
82 |   md: 104,
83 |   lg: 150,
84 |   xl: 200,
85 | };
86 |
87 | /**
88 |  * Authoritative emotion !' filename map (from prototype `EdvCharacter.ASSETS`),
89 |  * extended so every shipped pose is addressable. Missing emotions fall back to
90 |  * `default`.
91 |  */
92 | export const CHARACTER_ASSETS: Record<
93 |   CharacterId,
94 |   Partial<Record<CharacterEmotion, string>>
95 | > = {
96 |   foxi: {
97 |     default: "foxi-default.png",
98 |     happy: "foxi-smiling.png",
99 |     excited: "foxi-excited.png",
100 |     celebrating: "foxi-celebrating.png",
101 |     proud: "foxi-proud.png",
102 |     playful: "foxi-playful.png",
103 |     smart: "foxi-smart.png",
104 |     studious: "foxi-studious.png",
105 |     focused: "foxi-studious-2.png",
106 |     curious: "foxi-curious.png",
107 |     encouraging: "foxi-signal-beacon-3.png",
108 |     magical: "foxi-magical.png",
109 |     wise: "foxi-learning-portal.png",
110 |     worried: "foxi-worried.png",
111 |     concerned: "foxi-nervous.png",
112 |     nervous: "foxi-nervous.png",
113 |     annoyed: "foxi-annoyed.png",
114 |     "level-up": "foxi-level-up.png",
115 |     victory: "foxi-glowing-coin.png",
116 |     thinking: "foxi-studious.png",

```

[illegible]

```

221 | /** The learner just earned XP / a reward. */
222 | earnedXp?: boolean;
223 | }
224 |
225 | export interface MascotPick {
226 |   character: CharacterId;
227 |   emotion: CharacterEmotion;
228 | }
229 |
230 | /**
231 |  * Pick the mascot character + emotion for a gamification / product state.
232 |  * Ordered by signal strength (a level-up outranks a generic greeting).
233 |  */
234 | export function pickMascot(state: MascotState): MascotPick {
235 |   if (state.leveldUp) return { character: "foxi", emotion: "level-up" };
236 |   if (state.context === "celebration")
237 |     return { character: "foxi", emotion: "celebrating" };
238 |   if (state.earnedXp || state.context === "reward")
239 |     return { character: "foxi", emotion: "victory" };
240 |   if (state.context === "tutor") return { character: "owl", emotion: "wise" };
241 |   if (state.context === "needsAttention" || state.hasWeakClo)
242 |     return { character: "owl", emotion: "concerned" };
243 |   if (state.context === "habit")
244 |     return { character: "penguin", emotion: "happy" };
245 |   if (state.context === "reflection")
246 |     return { character: "penguin", emotion: "proud" };
247 |   if (state.context === "focus")
248 |     return { character: "foxi", emotion: "focused" };
249 |   if (state.context === "empty")
250 |     return { character: "foxi", emotion: "curious" };
251 |   if (typeof state.streakDays === "number" && state.streakDays >= 7)
252 |     return { character: "foxi", emotion: "proud" };
253 |   return { character: "foxi", emotion: "happy" };
254 | }
255 |
256 | /**
257 |  * Character + emotion for a coaching moment, integrating the existing
258 |  * `mascotGuidance` moment vocabulary (so the character imagery and the i18n
259 |  * speech copy stay aligned rather than forking).
260 |  */
261 | export function mascotForMoment(moment: MascotMomentId): MascotPick {
262 |   switch (moment) {
263 |     case "welcome":
264 |       return { character: "foxi", emotion: "happy" };
265 |     case "assessmentIntro":
266 |       return { character: "owl", emotion: "wise" };
267 |     case "emptyState":
268 |       return { character: "foxi", emotion: "curious" };
269 |     case "firstXp":
270 |       return { character: "foxi", emotion: "victory" };
271 |     case "firstEnrollment":
272 |       return { character: "foxi", emotion: "excited" };
273 |     case "password":
274 |       return { character: "owl", emotion: "wise" };
275 |   }
276 | }
277 |
278 | // % % % Marketplace companion skins (gamification economy) % % % % % % % % % % % % % % % %
279 |
280 | export type SkinRarity = "common" | "rare" | "epic" | "legendary";
281 |
282 | export interface CharacterSkin {
283 |   id: string;
284 |   character: CharacterId;
285 |   name: string;
286 |   /** Pose shown as the skin's representative art. */
287 |   emotion: CharacterEmotion;
288 |   rarity: SkinRarity;
289 |   /** Price in spendable XP (Edeviser has no separate currency). */
290 |   priceXp: number;
291 |   description: string;
292 | }
293 |
294 | /**
295 |  * Companion skins sold in the marketplace (mirrors the prototype marketplace
296 |  * data). Prices are spendable XP. Backend wiring happens at migration.
297 |  */
298 | export const CHARACTER_SKINS: readonly CharacterSkin[] = [
299 |   {
300 |     id: "foxi-classic",
301 |     character: "foxi",
302 |     name: "Foxy · Classic",
303 |     emotion: "default",
304 |     rarity: "common",
305 |     priceXp: 0,
306 |     description: "Default companion.",
307 |   },
308 |   {
309 |     id: "foxi-scholar",
310 |     character: "foxi",
311 |     name: "Foxy · Scholar",
312 |     emotion: "studious",
313 |     rarity: "rare",
314 |     priceXp: 450,
315 |     description: "Cap & glasses look.",
316 |   },
317 |   {
318 |     id: "foxi-magical",
319 |     character: "foxi",
320 |     name: "Foxy · Magical",
321 |     emotion: "magical",
322 |     rarity: "epic",
323 |     priceXp: 560,
324 |     description: "Sparkle particle trail.",

```

```

325 | },
326 | {
327 |   id: "foxi-champion",
328 |   character: "foxi",
329 |   name: "Foxy · Champion",
330 |   emotion: "celebrating",
331 |   rarity: "legendary",
332 |   priceXp: 1500,
333 |   description: "Confetti-ready celebration skin.",
334 | },
335 | {
336 |   id: "owl-night-sage",
337 |   character: "owl",
338 |   name: "Owl · Night Sage",
339 |   emotion: "wise",
340 |   rarity: "epic",
341 |   priceXp: 800,
342 |   description: "Swap companion entirely.",
343 | },
344 | {
345 |   id: "penguin-habit-buddy",
346 |   character: "penguin",
347 |   name: "Pengu · Habit Buddy",
348 |   emotion: "happy",
349 |   rarity: "epic",
350 |   priceXp: 800,
351 |   description: "Swap companion entirely.",
352 | },
353 | ];
354 |

```

#### FILE: [src/design-system/mascot/index.ts](#)

TYPE: .ts | LINES: 1-20 | SHA256: dd598d18d5b266fde31549ff7b1bab1e8a5eb7ccbf0d9904ac452cf508bb6eff

```

1 | /**
2 |  * L2 mascot / character system (shared).
3 |  *
4 |  * Living mascots – Foxi (companion) · Owlie (AI tutor) · Pengu (habit buddy) –
5 |  * ported from the prototype `EdvCharacter` system, with assets copied under
6 |  * `./assets` (src/ never imports from prototype/, guardrail G.2). Every appearance
7 |  * maps to a real product / gamification state; see ../PARITY.md §B.7.
8 |  *
9 |  * This barrel re-exports ONLY the pure catalog + resolvers (types, asset maps,
10 |  * `pickMascot`, `mascotForMoment`, `CHARACTER_SKINS`). The components are
11 |  * imported directly to avoid `react-refresh/only-export-components` warnings:
12 |  *   import MascotCharacter from "@design-system/mascot/MascotCharacter";
13 |  *   import MascotCompanion from "@design-system/mascot/MascotCompanion";
14 |  *
15 |  * Complements the existing `@components/shared/Mascot` (i18n coaching-text
16 |  * bubble): pair `mascotForMoment(moment)` here with that copy for a full
17 |  * character + speech coaching unit.
18 |  */
19 | export * from "../characters";
20 |

```

#### FILE: [src/design-system/mascot/mascot.css](#)

TYPE: .css | LINES: 1-203 | SHA256: 40a944d5f2ae515c2062d8dd150df39208dcf8d34a25010c927b77cc5ce2cc3c

```

1 | /* =====
2 |  Mascot character system styles – ported from prototype/shared.css `chr*`.
3 |  Namespaced `edv-mascot` to avoid collisions; physical left/right replaced
4 |  with logical properties (inset-inline / border-inline) for RTL. All decorative
5 |  motion is disabled under prefers-reduced-motion.
6 |  ===== */
7 |
8 | .edv-mascot {
9 |   display: inline-block;
10 |   object-fit: contain;
11 |   user-select: none;
12 |   -webkit-user-drag: none;
13 |   flex: 0 0 auto;
14 |   filter: drop-shadow(0 6px 12px rgba(15, 23, 42, 0.16));
15 | }
16 |
17 | /* % % Character motion (idle + entrance + reactions) % % */
18 | @keyframes edvMascotFloat {
19 |   0%,
20 |   100% {
21 |     transform: translateY(0);
22 |   }
23 |   50% {
24 |     transform: translateY(-6px);
25 |   }
26 | }
27 | @keyframes edvMascotBobIn {
28 |   0% {
29 |     opacity: 0;
30 |     transform: translateY(10px) scale(0.9);
31 |   }
32 |   60% {
33 |     opacity: 1;
34 |     transform: translateY(-2px) scale(1.03);
35 |   }
36 |   100% {
37 |     opacity: 1;
38 |     transform: translateY(0) scale(1);
39 |   }
40 | }
41 | @keyframes edvMascotWave {
42 |   0%,
43 |   100% {
44 |     transform: rotate(0);
45 |   }
46 |   25% {

```

```

47 |     transform: rotate(-7deg);
48 | }
49 | 75% {
50 |     transform: rotate(7deg);
51 | }
52 | }
53 | @keyframes edvMascotPop {
54 |     0% {
55 |         transform: scale(0) rotate(-10deg);
56 |     }
57 |     60% {
58 |         transform: scale(1.14) rotate(3deg);
59 |     }
60 |     100% {
61 |         transform: scale(1) rotate(0);
62 |     }
63 | }
64 | @keyframes edvMascotNudge {
65 |     0%,
66 |     100% {
67 |         transform: translateX(0);
68 |     }
69 |     50% {
70 |         transform: translateX(4px);
71 |     }
72 | }
73 | .edv-mascot-float {
74 |     animation: edvMascotFloat 3.2s ease-in-out infinite;
75 | }
76 | .edv-mascot-in {
77 |     animation: edvMascotBobIn 0.4s cubic-bezier(0.34, 1.56, 0.64, 1) both;
78 | }
79 | .edv-mascot-wave {
80 |     animation: edvMascotWave 0.5s ease-in-out 2;
81 | }
82 | .edv-mascot-pop {
83 |     animation: edvMascotPop 0.45s cubic-bezier(0.34, 1.56, 0.64, 1) both;
84 | }
85 | .edv-mascot-nudge {
86 |     animation: edvMascotNudge 1.4s ease-in-out infinite;
87 | }
88 |
89 | /* %% Companion row: character + bubble side by side %% */
90 | .edv-mascot-row {
91 |     display: flex;
92 |     align-items: flex-start;
93 |     gap: 10px;
94 | }
95 | .edv-mascot-row--center {
96 |     align-items: center;
97 | }
98 |
99 | /* %% Speech bubble %% */
100 | .edv-mascot-bubble {
101 |     position: relative;
102 |     background: #fff;
103 |     border: 1px solid #e2e8f0;
104 |     border-radius: 16px;
105 |     padding: 10px 14px;
106 |     box-shadow: 0 4px 14px rgba(16, 24, 40, 0.08);
107 |     max-width: 260px;
108 | }
109 | .edv-mascot-bubble__ttl {
110 |     font-size: 12px;
111 |     font-weight: 800;
112 |     color: #0f172a;
113 |     margin: 0 0 2px;
114 | }
115 | .edv-mascot-bubble__msg {
116 |     font-size: 12.5px;
117 |     line-height: 1.45;
118 |     color: #475569;
119 |     margin: 0;
120 | }
121 |
122 | /* tail: pointing to the inline-start (character on the leading side) */
123 | .edv-mascot-bubble--tail-l::before {
124 |     content: "";
125 |     position: absolute;
126 |     top: 16px;
127 |     inset-inline-start: -7px;
128 |     width: 12px;
129 |     height: 12px;
130 |     background: #fff;
131 |     border-inline-start: 1px solid #e2e8f0;
132 |     border-bottom: 1px solid #e2e8f0;
133 |     transform: rotate(45deg);
134 | }
135 | /* tail: pointing down (character above) */
136 | .edv-mascot-bubble--tail-b::before {
137 |     content: "";
138 |     position: absolute;
139 |     bottom: -7px;
140 |     inset-inline-start: 50%;
141 |     margin-inline-start: -6px;
142 |     width: 12px;
143 |     height: 12px;
144 |     background: #fff;
145 |     border-inline-end: 1px solid #e2e8f0;
146 |     border-bottom: 1px solid #e2e8f0;
147 |     transform: rotate(45deg);
148 | }
149 |
150 | /* tinted variants keep the tail colour in sync */

```

```

151 | .edv-mascot-bubble--teal {
152 |   background: #f0fdfa;
153 |   border-color: #99f6e4;
154 | }
155 | .edv-mascot-bubble--teal.edv-mascot-bubble--tail-l::before {
156 |   background: #f0fdfa;
157 |   border-color: #99f6e4;
158 | }
159 | .edv-mascot-bubble--amber {
160 |   background: #fffbeb;
161 |   border-color: #fde68a;
162 | }
163 | .edv-mascot-bubble--amber.edv-mascot-bubble--tail-l::before {
164 |   background: #fffbeb;
165 |   border-color: #fde68a;
166 | }
167 | .edv-mascot-bubble--blue {
168 |   background: #eff6ff;
169 |   border-color: #bdfbfe;
170 | }
171 | .edv-mascot-bubble--blue.edv-mascot-bubble--tail-l::before {
172 |   background: #eff6ff;
173 |   border-color: #bdfbfe;
174 | }
175 |
176 | /* on dark hero surfaces, the bubble flips to translucent light */
177 | .edv-mascot-onhero .edv-mascot-bubble {
178 |   background: rgba(255, 255, 255, 0.14);
179 |   border-color: rgba(255, 255, 255, 0.22);
180 |   box-shadow: none;
181 | }
182 | .edv-mascot-onhero .edv-mascot-bubble__ttl {
183 |   color: #fff;
184 | }
185 | .edv-mascot-onhero .edv-mascot-bubble__msg {
186 |   color: rgba(255, 255, 255, 0.82);
187 | }
188 | .edv-mascot-onhero .edv-mascot-bubble--tail-l::before {
189 |   background: rgba(255, 255, 255, 0.14);
190 |   border-color: rgba(255, 255, 255, 0.22);
191 | }
192 |
193 | /* % % Reduced motion: disable all character motion % % */
194 | @media (prefers-reduced-motion: reduce) {
195 |   .edv-mascot-float,
196 |   .edv-mascot-in,
197 |   .edv-mascot-wave,
198 |   .edv-mascot-pop,
199 |   .edv-mascot-nudge {
200 |     animation: none !important;
201 |   }
202 | }
203 |

```

## FILE: [src/design-system/mascot/MascotCharacter.tsx](#)

TYPE: .tsx | LINES: 1-81 | SHA256: 14d15deb2b9a4c40b16a0aab475419c382d6021e9b070f6edc1cf204b68b7667

```

1 | // =====
2 | // MascotCharacter – renders a living mascot pose (L2 pattern)
3 | // =====
4 | //
5 | // Faithful React port of the prototype `EdvCharacter.img()` / `chr` system.
6 | // Resolves a character + emotion to a bundled PNG, sizes it (46-200px), and
7 | // applies an optional motion preset. Non-draggable, reduced-motion safe, and
8 | // accessible (meaningful alt by default; `decorative` for purely ornamental use).
9 | // =====
10 |
11 | import './mascot.css';
12 | import { cn } from '@lib/utills';
13 | import {
14 |   characterAssetUrl,
15 |   characterName,
16 |   SIZE_PX,
17 |   type CharacterAnimation,
18 |   type CharacterEmotion,
19 |   type CharacterId,
20 |   type CharacterSize,
21 | } from './characters';
22 |
23 | const ANIMATION_CLASS: Record<CharacterAnimation, string> = {
24 |   none: '',
25 |   float: 'edv-mascot-float',
26 |   in: 'edv-mascot-in',
27 |   wave: 'edv-mascot-wave',
28 |   pop: 'edv-mascot-pop',
29 |   nudge: 'edv-mascot-nudge',
30 | };
31 |
32 | export interface MascotCharacterProps {
33 |   character: CharacterId;
34 |   /** Emotion / pose. Falls back to the character's default when unavailable. */
35 |   emotion?: CharacterEmotion;
36 |   /** Render size (prototype `chr` scale). Default `md`. */
37 |   size?: CharacterSize;
38 |   /** Motion preset. Default `in` (a subtle spring entrance). */
39 |   animation?: CharacterAnimation;
40 |   /** Override the accessible alt text. */
41 |   alt?: string;
42 |   /** Purely ornamental: renders empty alt + aria-hidden. */
43 |   decorative?: boolean;
44 |   /** Image loading strategy. Default `lazy`. */
45 |   loading?: 'eager' | 'lazy';
46 |   className?: string;
47 | }

```

```

48 |
49 | const MascotCharacter = ({
50 |   character,
51 |   emotion = "default",
52 |   size = "md",
53 |   animation = "in",
54 |   alt,
55 |   decorative = false,
56 |   loading = "lazy",
57 |   className,
58 | }: MascotCharacterProps) => {
59 |   const src = characterAssetUrl(character, emotion);
60 |   const px = SIZE_PX[size];
61 |   const computedAlt = decorative
62 |     ? ""
63 |     : alt ?? `${characterName(character)} looking ${emotion}`;
64 |
65 |   return (
66 |     <img
67 |       src={src}
68 |       alt={computedAlt}
69 |       aria-hidden={decorative || undefined}
70 |       width={px}
71 |       height={px}
72 |       draggable={false}
73 |       loading={loading}
74 |       className={cn("edv-mascot", ANIMATION_CLASS[animation], className)}
75 |       style={{ width: px, height: px }}
76 |     />
77 |   );
78 | };
79 |
80 | export default MascotCharacter;
81 |

```

## FILE: src/design-system/mascot/MascotCompanion.tsx

TYPE: .tsx | LINES: 1-85 | SHA256: b58a65064b60351e628aa3704dc52be6df494e745e0d999cd8c4f4f997d9d700

```

1 | // =====
2 | // MascotCompanion - character + speech bubble (L2 pattern)
3 | // =====
4 | //
5 | // Faithful React port of the prototype `EdvCharacter.companion()` - a mascot
6 | // pose beside a speech bubble. The signature gamified coaching surface used on
7 | // dashboards, the learning path, journal, and the tutor.
8 | //
9 | // Copy is passed in already-localized by the caller (keeps this presentational
10 | // and bilingual-ready). For moment-driven copy, pair `mascotForMoment()` with
11 | // the i18n text from the existing guidance layer.
12 | // =====
13 |
14 | import "./mascot.css";
15 | import { cn } from "@lib/utils";
16 | import MascotCharacter from "../MascotCharacter";
17 | import type {
18 |   CharacterEmotion,
19 |   CharacterId,
20 |   CharacterSize,
21 | } from "../characters";
22 |
23 | export interface MascotCompanionProps {
24 |   character: CharacterId;
25 |   emotion?: CharacterEmotion;
26 |   /** Optional bold headline inside the bubble. */
27 |   title?: string;
28 |   /** Bubble body copy (already localized). */
29 |   message: string;
30 |   size?: CharacterSize;
31 |   /** Bubble tint. Omit for the default white bubble. */
32 |   tint?: "teal" | "amber" | "blue";
33 |   /** Which way the bubble tail points: inline-start (`l`) or down (`b`). */
34 |   tail?: "l" | "b";
35 |   /** Center-align the character with the bubble. */
36 |   center?: boolean;
37 |   /** Idle float on the character (otherwise a spring entrance). */
38 |   float?: boolean;
39 |   /** Style the bubble for a dark hero surface (translucent light). */
40 |   onHero?: boolean;
41 |   className?: string;
42 | }
43 |
44 | const MascotCompanion = ({
45 |   character,
46 |   emotion = "happy",
47 |   title,
48 |   message,
49 |   size = "md",
50 |   tint,
51 |   tail = "l",
52 |   center = false,
53 |   float = false,
54 |   onHero = false,
55 |   className,
56 | }: MascotCompanionProps) => (
57 |   <div
58 |     className={cn(
59 |       "edv-mascot-row",
60 |       center && "edv-mascot-row--center",
61 |       onHero && "edv-mascot-onhero",
62 |       className
63 |     )}
64 |   >
65 |     <MascotCharacter
66 |       character={character}

```

```

67 |     emotion={emotion}
68 |     size={size}
69 |     animation={float ? "float" : "in"}
70 |   />
71 |   <div
72 |     className=cn(
73 |       "edv-mascot-bubble",
74 |       `edv-mascot-bubble--tail-${tail}`,
75 |       tint && `edv-mascot-bubble-${tint}`
76 |     )
77 |   >
78 |     {title && <p className="edv-mascot-bubble__ttl">{title}</p>}
79 |     <p className="edv-mascot-bubble__msg">{message}</p>
80 |   </div>
81 | </div>
82 | );
83 |
84 | export default MascotCompanion;
85 |

```

## FILE: src/design-system/PARITY.md

TYPE: .md | LINES: 1-419 | SHA256: 68fcb9bc697a58a16b5dadff7e907582dd9fa1cf299c1f74f44c015c841f82d2

```

1 | # PARITY.md – Prototype !' React fidelity contract (Path A)
2 |
3 | This file **pins fidelity before screens are built**. It is the single reference the
4 | rebuild targets so "make it look exactly like the prototype" is a fixed, reviewable
5 | spec instead of a moving one. It has three locked parts:
6 |
7 | - **$A – Class !' Component parity map:** every structural prototype CSS class !'
8 |   the one React primitive/pattern that reproduces it.
9 | - **$B – Emoji !' Lucide icon map:** every prototype emoji !' its `lucide-react`
10 |  replacement (emoji are OS-variant and never pixel-stable), with the categories
11 |  that must **not** be converted (content/gamification art).
12 | - **$C – Per-screen diff tolerances:** a tier per `visual/screen-map.ts` id, the
13 |   number that lands in that screen's `maxDiffRatio` when its parity check activates.
14 |
15 | Plus **$D** (demo-only artifacts we never port) and **$E** (fidelity caveats).
16 |
17 | > Source of truth = the prototype (`prototype/*.html`, `shared.css` "H64 KB / 312
18 | > classes, `shared.js`). Those files never ship (guardrail G.2); only their
19 | > **values and structure** are reproduced. Ground truth here was extracted directly
20 | > from those files, not assumed.
21 |
22 | > **Scope:** the prototype defines **light mode + LTR only**. Everything below is the
23 | > light/LTR contract. Dark mode and Arabic/RTL are **net-new** deliverables gated
24 | > separately ($E) – they cannot be diffed against a prototype that lacks them.
25 |
26 | ---
27 |
28 | ## How this plugs into the harness
29 |
30 | 1. Build the screen in the new design system using **$A** (primitives) + **$B**
31 |   (icons), reproducing the token values from `tokens.css`.
32 | 2. In `visual/screen-map.ts` set the screen's `appPath`, flip `rebuilt: true`, and
33 |   set `maxDiffRatio` to its **$C** tier value (Tier B = the 0.12 default, no
34 |   override needed).
35 | 3. `npm run test:visual:capture` (if the reference is stale) !' `npm run test:visual`.
36 |   Iterate until the diff is within tolerance at all four viewports.
37 |
38 | Primitive locations: Shadcn wrappers in `src/components/ui/*`; shared
39 | patterns/chrome in `src/components/shared/*`. Both are **reused** as the design
40 | system's L2 layer (rebuild `tasks.md` 0.3/0.4) – this map names the exact target.
41 |
42 | ---
43 |
44 | ## $A – Class !' Component parity map
45 |
46 | The prototype's structural classes and the React primitive that reproduces each.
47 | "Match" = same padding / radius / shadow / color / typography / motion, driven by
48 | `tokens.css`.
49 |
50 | > **Built (P0.4) – screens build from `@/design-system`.** The prototype design
51 | > system now ships these patterns as prototype-faithful components (import from
52 | > `@/design-system` or `@/design-system/patterns`): `PageHeader`, `PCard`,
53 | > `SectionCard`, `SectionHeader`, `KPICard`, `HeroCard`, `StatusDot`, `StatePanel`,
54 | > `EMeter`, plus `MascotCharacter` / `MascotCompanion`. **Primitives** are the Shadcn
55 | > `ui/*` re-exported via `@/design-system` (`Button` incl. `tactile`, `Card`,
56 | > `Badge`, `Input`, `Select`, `Dialog`, `Tabs`, `Switch`, ...). New screens import
57 | > from `@/design-system` – NOT `@/components/shared/*` (legacy). Reference
58 | > migration already on the design system: the student + parent **Fees** vertical.
59 | >
60 | > **Self-contained + reskinned (prototype-fidelity pass):** `MasteryRing`,
61 | > `WelcomeHero`, `SeverityIcon`, and `GradientCardHeader` are now **internalized**
62 | > into `src/design-system/patterns/` (the design system no longer depends on
63 | > `@/components/shared/*`; those legacy files were deleted). `PCard` / `SectionHeader` /
64 | > `KPICard` / `StatusDot` were reskinned to their **exact** `shared.css` values
65 | > (`p-card` 20px + `#eef2f6` + two-layer shadow + hover-lift; `sec-h` .chip
66 | > 26px/9px + teal halo + 13px/800 title; `kpi-ic` 38px/11px; `dot` 8px). Per the
67 | > strict `prototype-fidelity.md` rule, the raw `shared.css` values win over any
68 | > earlier deployed-flavored simplification.
69 | > The rows below remain the prototype-class !' component reference.
70 |
71 | ### A.1 Surfaces & layout
72 |
73 | | Prototype class | Role | React target | Notes |
74 | | --- | --- | --- | --- |
75 | | `p-card` | standard card | `ui/card` `Card` | base surface: white, `border-0`, `shadow-md`, `rounded-xl` |
76 | | `p-card` + gradient top | section card w/ header | `shared/GradientCardHeader` inside `Card` | header uses *
77 | | `sec-h` + .chip | section header + icon chip | `shared/SectionHeader` | .chip icon !' Lucide (see $B); t
78 | | `kpi`, `kpi-ic`, .stat-title, .stat-chip | KPI/metric tile | `shared/KPICard` | label `text-[10px]` font
79 | | `p-page-content` | main scroll region | role layout `<main>` in `src/app/` | `bg-slate-50` canvas so white ch
80 | | `card-select`, .opt | selectable option card | `Card` + `data-state` selected ring | |
81 |

```



```

82 | ### A.2 Chrome (the frame)
83 |
84 | | Prototype class | Role | React target | Notes |
85 | | --- | --- | --- | --- |
86 | | .app-header | top header | `shared/GlobalHeader` | delineated by surface + hairline border/shadow |
87 | | .bottom-bar | sidebar (misnamed) | `shared/Sidebar` | solid white, right border + soft shadow; `w-64` |
88 | | .side-link, .side-profile, .side-lvlbar, .side-ava, .side-name, .side-sep, .side-label | sideb
89 | | .side-upgrade, .sidebar-extra, .su-btn | upgrade card | `shared/Sidebar` (student-only slot) | brand b
90 | | .right-rail, .rail-row, .rail-btn, .rail-card, .rail-h, .rail-check | right insight rail | `shar
91 | | .top-search | # K search entry | `shared/SearchCommand` |
92 | | .cmdk, .cmdk-item, .cmdk-list, .cmdk-sec, .cmdk-back, .kbd | command palette | `shared/SearchCom
93 | | .hdr-profile, .hdr-brand, .hdr-right | header profile/brand | `shared/ProfileDropdown` + header slots
94 | | .tutor-fab | floating tutor button | `shared/TutorEntryButton` |
95 | | .notif-* | `item/panel/bell/badge/dot/backdrop` | notifications | `shared/NotificationBell` + `shared/Noti
96 | | .mode-laptop, .mode-mobile, .demo-*, .dock-* | device/demo toggle | **NOT PORTED** | demo aid ($D) -
97 |
98 | ### A.3 Buttons & interactive
99 |
100 | | Prototype class | Role | React target | Notes |
101 | | --- | --- | --- | --- |
102 | | .btn3d | (the primary tactile button, 40x uses) | button | `shared/PrimaryCTA` / tactile `Button` variant (
103 | | .b-soft, .b-ghost, .b-danger, .b-teal, .b-amber, .b-blue | button variants | `Button` variants (
104 | | .sm, .full | size/width mods | `Button` size + `w-full` |
105 | | .cta-brand/-indigo/-sky/-violet/-emerald/-sunset/-aurora | CTA color skins | `Button` gradient variants |
106 | | .tap-bounce, .pcard-tap, .hover-lift | press/hover motion | `active:scale-95` + `tokens` motion | hono
107 | | .edv-toggle + .etk + .on | on/off switch | `ui/switch` `Switch` | see $B.5 (state, not emoji); ON=bran
108 | | .tab-btn, .tab-ic, .tri-tab, .tri-tabs, .tier-seg | pill tabs | `ui/tabs` `Tabs` (pill style) | ac
109 |
110 | > **CTA gradient (decision):** the canonical primary-button fill is
111 | > `var(--brand-gradient)` (93.65deg). **New rebuild components (`features/**`
112 | > use it directly.** The legacy from-teal-500 to-blue-600 utility (~180 `src/`
113 | > usages, and the current steering design-system rule) is intentionally left
114 | > as-is – consolidating the CTA gradient is a **token-canonicalization (P0.2 /
115 | > cutover)** concern, not a per-component edit: changing one component would
116 | > desync it from ~180 others, and mass-replacing would churn legacy code that P5
117 | > deletes.
118 |
119 | ### A.4 Data, meters & status
120 |
121 | | Prototype class | Role | React target | Notes |
122 | | --- | --- | --- | --- |
123 | | .emeter | (+.strong/good/.attention/.critical) | semantic meter | `design-system/patterns/EMeter` | **bui
124 | | .emeter.pro | professional (institution) meter | `EMeter` .pro | flat, muted, inset; no gradient |
125 | | .progress-track, .progress-fill, .bar-fill, .lz-prog | progress bar | `AttainmentBar` / `ui` progres
126 | | .ring-mini | mini mastery ring | `shared/MasteryRing` (sm) | SVG ring |
127 | | .mini-bar, .spark | mini chart/sparkline | Recharts mini or `features` viz |
128 | | .pill, .pill-red/-amber/-green/-blue/-slate | status pill | `ui/badge` `Badge` variants |
129 | | .dot, .notif-dot | status dot | .dot span (semantic color) | 0=β&0=Y40=β&0=Y5&^ !' colored dot, not an icon ($B.
130 | | .trend, .trend-up/-down/-flat | trend indicator | Lucide `ArrowUpRight`/`ArrowDownRight`/`ArrowRight` +
131 | | .sev-, .sev-high/-med/-low/-brand/-info/-violet | severity | `shared/SeverityIcon` |
132 | | .i-blue/-green/-amber/-red/-violet/-teal | icon color token | `text-*` token classes |
133 | | .dtable | data table | `shared/DataTable` | sort/filter/paginate |
134 | | .wk-tile, .wk-grid, .wk-col, .wk-legend, .wk-today, .wk-empty | week/habit grid | `shared/HabitG
135 | | .hm-blue/-teal/-violet/-sunset | heatmap cells | `shared/HeatmapGrid` |
136 |
137 | ### A.5 Overlays, chat, forms, profile, celebration
138 |
139 | | Prototype class | Role | React target | Notes |
140 | | --- | --- | --- | --- |
141 | | .edv-modal | (+.back/-head/-body/-foot/-x) | modal | `ui/dialog` `Dialog` |
142 | | .edv-toast | toast | Sonner `toast()` (`ui/sonner`) |
143 | | confirm/delete modal | destructive confirm | `shared/ConfirmDialog` |
144 | | .bubble-ai, .bubble-user, .msg, .typing-dot | tutor chat bubbles | `shared/ChatMessage` | tutor conv
145 | | .chr, .chr-* | (float/in/wave/pop/nudge/xs..xl) | mascot/character | `design-system/mascot/MascotCharact
146 | | .chr-bubble | (+.tail-l/-b, .b-teal/-amber/-blue, .chr-onhero) | mascot speech bubble | `design-system
147 | | .chr-row | companion row | `MascotCompanion` | character + bubble layout |
148 | | .ef-field, .ef-row2, .ef-avatar-pick, .ava-cam, .ava-edit | form field/avatar | `ui/form` + `ui/in
149 | | .phdr, .phdr-me, .phdr-stat(-ic/-v/-l/-tx), .phdr-av, .phdr-name, .phdr-role, .phdr-contact,
150 | | .reveal, .flip, .front, .back, .reveal-rays | reward reveal | `shared/BadgeAwardModal` / `shared
151 | | .badge-pop, .xp-float, .streak-flame, .pop-in, .glow-pulse, .confetti-piece, .fade-in-up | cel
152 | | .tree-stage, .node-current, .camp, .marker, .milestone, .hero-arrow | learning-path journey | `f
153 |
154 | > **To add (cited by the L2 plan but not yet in `ui`):** `Tooltip`. There is no
155 | > `ui/tooltip.tsx` today – add a Radix tooltip primitive before screens that need
156 | > hover affordances, or use `ui/popover`.
157 |
158 | ---
159 |
160 | ## $B – Emoji !' Lucide icon map
161 |
162 | **Rule:** prototype emoji fall into two buckets.
163 |
164 | - **Chrome / iconography** (nav, section chips, buttons, status, form affordances)
165 |   !' replace with a `lucide-react` icon (imports below). Emoji render differently
166 |   per OS/browser and are never pixel-stable; Lucide is the pixel-stable equivalent.
167 | - **Content / gamification art** (marketplace items, companions, mascot,
168 |   celebration, role-picker avatars) !' **do NOT convert.** These are data/assets
169 |   ($B.7), not UI chrome.
170 |
171 | Icon sizing (design-system rule): nav `h-5 w-5`; in buttons `h-4 w-4` with `gap-2`.
172 | Always pair an icon with a label or an accessible name.
173 |
174 | ### B.1 Navigation icons (1:1 – must match `Sidebar`/`MobileTabBar` order)
175 |
176 | From the per-page nav arrays in the prototype (`shared.js`, page headers).
177 |
178 | | Emoji | Nav item | Lucide |
179 | | --- | --- | --- |
180 | | 0<β& | Home / Dashboard | `Home` |
181 | | 0=ÜÜ | Learn / Courses | `BookOpen` |
182 | | & | Learning Path | `Route` |
183 | | 0=ÜE | Progress | `TrendingUp` |
184 | | 0>Y | AI Tutor | `Bot` |
185 | | 0<β& | Leaderboard | `Trophy` |

```

```

186 | Ø=pİp | Marketplace | `ShoppingBag` |
187 | Ø=Ue | Team | Users |
188 | Ø=UÅ | Calendar | `Calendar` |
189 | ' b | Grading | `PenLine` |
190 | Ø>Yİ | Curriculum | `Dna` |
191 | Ø>YŊ | Ø<B" | Students | `GraduationCap` |
192 | Ø=YÅp | Matrix | `LayoutGrid` |
193 | Ø<B1 | Growth | `Sprout` |
194 | Ø=U- | Support / Discussions | `MessageCircle` |
195 | Ø=YÖp | Attendance | `CalendarCheck` |
196 | Ø=ÜB | Rubric Builder | `Ruler` |
197 | Ø=ÜØ | Course File | `BookMarked` |
198 | Ø=Üd | Me / Profile | `User` |
199 | &mp | Settings | `Settings` |
200 | Ø=Y | Notifications | `Bell` |
201 | Ø=Y | Sign in | `Login` |
202
203 |### B.2 Section-header chips (`.sec-h .chip`) & common UI glyphs
204
205 | Semantic mapping; where several emoji share a meaning they collapse to one icon.
206
207 | Emoji | Lucide | Emoji | Lucide |
208 | --- | --- | --- | --- |
209 | Ø<B | `Target` | Ø=ÜÉ | `BarChart3` |
210 | Ø=ÜÉ | `TrendingUp` | Ø=ÜÉ | `TrendingDown` |
211 | Ø>Yà | `Brain` | Ø>Y | `Bot` |
212 | & p | `AlertTriangle` | ' | `CheckCircle2` |
213 | 'L | `XCircle` | 'S | `HelpCircle` |
214 | Ø=Üj | `Lightbulb` | Ø=Üā Ø=Üā | `Megaphone` |
215 | Ø=Y | Ø=Y | `Search` | Ø=YüB | `Map` |
216 | Ø>Yİ | `Compass` | Ø=Y | `Link` |
217 | Ø=ÜÉ | `ClipboardList` | Ø=ÜY | `NotebookPen` |
218 | Ø=ÜÅ Ø=ÜÅ Ø=ÜÜ | `FileText` | Ø>Yp | `ReceiptText` |
219 | Ø=ÜÅ Ø=YÅp Ø=YÅp | `FolderOpen` | Ø=Üæ | `Package` |
220 | Ø=Y Ø=Y | `RefreshCw` | Ø=Y | `Plug` |
221 | Ø<Bē | `School` | Ø<BÜB | `Building2` |
222 | Ø<B" | `Palette` | Ø<B | `Globe` |
223 | &"p | `Swords` | Ø<B+p | `Tag` |
224 | Ø=ÜØ Ø=ÜØ Ø=ÜØ Ø=ÜØ Ø=ÜØ Ø=ÜØ | `BookOpen` | Ø<B' | `Backpack` |
225 | Ø=Üā | `Inbox` | Ø=Üā | `Upload` |
226 | + p | `Download` | Ø=Üİ | `Paperclip` |
227 | Ø=p« | `Ban` | Ø=Y Ø=Y | `Lock` |
228 | Ø=Y | `KeyRound` | Ø=pāB | `ShieldCheck` |
229 | Ø<Bšp Ø<Bšp | `SlidersHorizontal` | &mp | `Settings` |
230 | Ø=Y% | `Flame` | &i | `Zap` |
231 | '( Ø<B +P | `Sparkles` / `Star` | Ø<B% | `PartyPopper` |
232 | Ø<B" | `GraduationCap` | Ø=Üe | `Users` |
233 | ' b Ø=Üc | `Mail` | Ø=Üp | `Phone` |
234 | Ø=Üİ | `MapPin` | Ø=Ü³ | `CreditCard` |
235 | Ø=Ü% | `Save` | Ø=Ü» Ø=Y%b | `Monitor` |
236 | Ø=Üñ | `Smartphone` | Ø=ÜÅp | `Eye` |
237 | Ø=YŊp | `Trash2` | ' | `Plus` |
238 | ' p | `Pencil` / `PenLine` | Ø=ÜK | `Hand` |
239 | #d #ñp Ø=Yp Ø=YR | `Clock` / `Timer` | #ó | `Hourglass` |
240 | Ø>Yé | `FlaskConical` | Ø>Yé | `Puzzle` |
241 | Ø<B% | `Video` | Ø=Ü+ | `Camera` |
242 | Ø<B | `Moon` | & p | `Sun` |
243 | 'Dp | `Snowflake` | Ø>Y | `Handshake` |
244 | Ø=pē | `Rocket` | Ø=ÜQ | `Crown` |
245 | Ø=Ü- Ø=Ü- | `MessageCircle` | Ø>YØ | `HeartPulse` |
246 | Ø<B | `Waves` | Ø=Y' Ø=Y( | `Wrench` |
247 | Ø=Üā | `Dumbbell` | Ø=Yvp | `EyeOff` (privacy/anon) |
248 | Ø>pz | `Stethoscope` | Ø=Ü% | `Briefcase` |
249 | Ø<Y" | `Contact` | Ø=Y | `Bookmark` |
250 | Ø=Üİ | `Pin` | Ø<B1 | `Sprout` |
251
252 |### B.3 Medals & ranks
253
254 | Emoji | Lucide | Note |
255 | --- | --- | --- |
256 | Ø>YĠ | `Medal` (gold) | leaderboard/badges gold tier – color via token |
257 | Ø>YH | `Medal` (silver) | |
258 | Ø>YI | `Medal` (bronze) | |
259 | Ø<BĖ | `Trophy` | overall leaderboard |
260 | Ø<BÅ Ø<B-p | `Award` | earned badge |
261
262 |### B.4 Status dots & trend arrows (not glyph icons)
263
264 | Emoji | Target | Note |
265 | --- | --- | --- |
266 | Ø=Bā Ø=Y4 Ø=Bā Ø=Y5 &a | `.`dot` span, semantic color | on-track/critical/attention/monitor/neutral – a colored dot, *
267 | !- | `ArrowUpRight` (or `TrendingUp`) | rising |
268 | !~ | `ArrowDownRight` (or `TrendingDown`) | falling |
269 | 'İb | `ArrowRight` (or `Minus`) | flat/stagnant |
270 | !" | `ArrowLeftRight` | comparison |
271 | %İ %İp | `Play` (media) or `ChevronRight` (pager) | context-dependent |
272 | %A %Ap | `ChevronLeft` | pager |
273 | #Ø | `Pause` | timer |
274 | Ø=Y; Ø=Y: | `TriangleAlert` / `ArrowDown` `ArrowUp` | delta emphasis |
275
276 |### B.5 Toggle-state pairs !' `Switch` state (NOT icons)
277
278 | The prototype encodes on/off with an emoji pair as a second cue (good ally
279 | instinct). In React the `Switch` (`.edv-toggle`) conveys state via **knob position +
280 | a check mark**, so these pairs collapse into switch state – do not render the emoji:
281
282 | Ø=Y / Ø=Y' (notifications), Ø<B / & p or Ø<B / &a (theme/dark), `(/&a`, Ø<B / &a` (bilingual),
283 | Ø=ÜÉ/&a, Ø=ÜØ/&a, Ø=Y (contrast), Ø=Y / Ø=Y' (sound), Ø=ÜÅp / Ø=Yvp (leaderboard anonymity).
284
285 | Rule (from `prototype-backend-parity` R8 research): ON = brand accent + knob right +
286 | check; OFF = neutral gray + knob left; **never red for a plain off**, never
287 | color-only.
288
289 |### B.6 Bloom's Learning-Path camp metaphor (`path.html`)

```

```

290 |
291 | Decorative journey art tied to Bloom levels (which carry canonical colors from the
292 | design-system rule). Lucide is the fallback; the Learning Path may keep richer SVG /
293 | `prototype/characters` art where flat icons lose fidelity.
294 |
295 | | Emoji | Camp | Bloom level | Lucide fallback |
296 | | --- | --- | --- | --- |
297 | | 0<B0p | Base camp | Remember | `Tent` |
298 | | &u | Camp I | Understand | `Tent` (+ level color) |
299 | | &dp | Camp II | Apply | `Mountain` |
300 | | 0>Yx | Camp III | Analyze | `Footprints` |
301 | | 0<B0p | High camp | Evaluate | `MountainSnow` |
302 | | 0<BA | Summit | Create | `Flag` |
303 | | 0<B3 | Knowledge Tree view | - | `Network` / `GitBranch` |
304 | | 0=p0 | Next milestone | - | `Flag` |
305 |
306 | ### B.7 Content / gamification art – DO NOT convert
307 |
308 | These are **data or illustration**, not chrome. Keep them as the item's `icon`
309 | data field or port them as illustration assets from `prototype/characters` +
310 | `prototype/brand`. Swapping them for Lucide would strip product identity.
311 |
312 | | Category | Examples | Handling |
313 | | --- | --- | --- |
314 | | Marketplace companions | 0>Y% 0>YŠ 0=Ü | **built (P0.5)** `CHARACTER_SKINS` (design-system/mascot) – real charact
315 | | Marketplace cosmetics | 0=Y%p 0<BYp 0=UQ 0<B 0<B (theme) 'Dp (shield) | item `icon` data / `shared/CosmeticPreview`
316 | | Mascot / character | 0>YŠ Foxi · 0>Y% Owlie · 0=Ü' Pengu | **built (P0.5)** `design-system/mascot` – emotion! `pose
317 | | Celebration | 0<B% (confetti burst) | canvas-confetti, not an icon |
318 | | Role-picker avatars | 0>YŃ 0<Bè 0>YŃ 0<B“ 0=Üh 0=Üi 0=Üg 0<B0p | large avatar art in auth/role picker may stay illustration;
319 | | Mood pickers | 0=p 0=p 0=p 0=pB 0=p 0=p 0=p0 0=p | `features` mood component using Lucide face set `Angry`/`Frown`/`Me
320 |
321 | ---
322 |
323 | ## §C – Per-screen diff tolerances
324 |
325 | Cross-implementation parity (prototype HTML vs React) is never byte-identical
326 | (font hinting, emoji! `Lucide`, dynamic data), so each screen gets a **tolerance tier**.
327 | The tier value is what goes into that screen's `maxDiffRatio` in
328 | `visual/screen-map.ts` when its parity check activates.
329 |
330 | | Tier | maxDiffRatio | Applies to |
331 | | --- | --- | --- |
332 | | **A – tight** | `0.08` | text/form/settings/auth – few icons, little dynamic data |
333 | | **B – standard** | `0.12` (the `DEFAULT`) | typical dashboards, lists, tables, detail pages |
334 | | **C – loose** | `0.16` | chart/heatmap/ring-heavy, mascot/gamified, or emoji-dense |
335 |
336 | Guidance: start at the tier value; **tighten** once a real baseline is stable.
337 | Mobile (360) may diverge more than desktop – if only mobile exceeds tier, allow up
338 | to +0.02 on that viewport before loosening the whole screen. Tier B needs no
339 | override (it equals the default).
340 |
341 | Grouped by tier (every `screen-map.ts` id appears exactly once):
342 |
343 | **Tier A – `0.08` (7):** `auth-login`, `student-settings`, `student-profile`,
344 | `teacher-profile`, `parent-profile`, `coordinator-profile`, `admin-profile`.
345 |
346 | **Tier C – `0.16` (24):**
347 | - Student (12): `student-dashboard`, `student-path`, `student-tutor`,
348 | `student-progress`, `student-leaderboard`, `student-marketplace`, `student-team`,
349 | `student-calendar`, `student-wellness`, `student-focus`, `student-quests`,
350 | `student-badges`.
351 | - Teacher (3): `teacher-dashboard`, `teacher-gradebook`, `teacher-attendance`.
352 | - Parent (2): `parent-dashboard`, `parent-progress`.
353 | - Coordinator (3): `coordinator-dashboard`, `coordinator-curriculum`,
354 | `coordinator-teams`.
355 | - Admin (4): `admin-dashboard`, `admin-analytics`, `admin-marketplace`,
356 | `admin-badges`.
357 |
358 | **Tier B – `0.12` / default (33):**
359 | - Student (9): `student-lesson`, `student-review`, `student-journal`,
360 | `student-portfolio`, `student-courses`, `student-course-detail`,
361 | `student-assignment`, `student-transcript`, `student-learning-profile`.
362 | - Teacher (7): `teacher-grading`, `teacher-students`, `teacher-curriculum`,
363 | `teacher-rubrics`, `teacher-questions`, `teacher-materials`, `teacher-handoffs`.
364 | - Parent (1): `parent-support`.
365 | - Coordinator (5): `coordinator-outcomes`, `coordinator-accreditation`,
366 | `coordinator-cqi`, `coordinator-competencies`, `coordinator-course-file`.
367 | - Admin (6): `admin-users`, `admin-fees`, `admin-governance`, `admin-security`,
368 | `admin-structure`, `admin-import`.
369 | - Shared (5): `shared-announcements`, `shared-notifications`, `shared-discussions`,
370 | `shared-surveys`, `shared-fees`.
371 |
372 | Tier counts: **A** x7 . **C** x24 . **B** x33 = **64**.
373 |
374 | ---
375 |
376 | ## §D – Demo-only prototype artifacts (never ported)
377 |
378 | The prototype is a clickable demo; these exist only to simulate an app and must not
379 | leak into `src/` (rebuild guardrails G.2, R7):
380 |
381 | - **Device toggle** `.mode-laptop` / `.mode-mobile` and the demo dock `.demo-menu` /
382 | `.demo-dock` / `.demo-seg` / `.demo-btn` / `#dock-screens` !' replaced by **real CSS
383 | breakpoints** (360/768/1024/1440) and the sidebar! "bottom-tab swap at `lg`.
384 | - **CDN Tailwind** + **Google Fonts** <link> ** !' real Vite Tailwind v4 build; fonts
385 | self-hosted/bundled.
386 | - **shared.css** / `shared.js` ** !' never imported; their values live in `tokens.css`
387 | and their behavior in React components/hooks.
388 | - **Hardcoded demo data** and `toast()`/`confirm()` stubs !' replaced by real hooks
389 | (no new backend).
390 | - **Raw emoji as icons** !' Lucide per §B (except §B.7 content art).
391 | - **localStorage` role`/`edv-role` switching** !' real `useAuth` role + `RouteGuard`.
392 |
393 | ---

```

```

394 |
395 | ## SE – Fidelity caveats (what "pixel-perfect" does NOT cover)
396 |
397 | Pinned so scope stays honest (rebuild design.md §4 / risks R2):
398 |
399 | 1. **Light + LTR only.** The prototype has no dark mode and no RTL. Dark tokens and
400 |   Arabic/RTL are **net-new** design deliverables with their own review + gates;
401 |   they cannot be diffed against the prototype.
402 | 2. **Dynamic data differs.** Real hooks return different values than the mock's
403 |   hardcoded content; tolerances (§C) absorb this. Parity is about layout/treatment,
404 |   not identical text.
405 | 3. **Icons are equivalents, not identical.** Emoji! 'Lucide (§B) changes glyph shape by
406 |   design; this is the intended, pixel-stable outcome.
407 | 4. **Motion** is verified for correctness + `prefers-reduced-motion`, not diffed
408 |   (animations are frozen during capture).
409 | 5. **A11y** (axe/keyboard/contrast/44px targets) is a separate gate, not a visual
410 |   diff.
411 |
412 | ---
413 |
414 | ### Change log
415 |
416 | - **v1** – Initial contract. $A from `shared.css` (312 classes) + `src/components`
417 |   inventory; $B from a full emoji scan of `prototype/` (190 distinct glyphs); $C tiers
418 |   the 64 `screen-map.ts` ids. Adopted brand gradient `93.65deg` (see `tokens.css`).
419 |

```

## FILE: src/design-system/patterns/EMeter.tsx

TYPE: .tsx | LINES: 1-126 | SHA256: fabb47508cdfba7a5a39e4e5e66206491729eca39359edb5b02b1eb27b83fef7

```

1 | // =====
2 | // EMeter – semantic threshold meter (L2 pattern)
3 | // =====
4 | //
5 | // Pixel-faithful React reproduction of the prototype `.emeter` (prototype/
6 | // shared.css) – see src/design-system/PARITY.md §A.4. The fill color encodes
7 | // status against thresholds, like the native HTML <meter>; the default fill is
8 | // the 93.65deg brand gradient (student) or flat slate (`pro`, institution).
9 | //
10 | // This is DISTINCT from `@/components/shared/AttainmentBar`, which is the
11 | // attainment-LEVEL bar (Excellent/Satisfactory/Developing/Not-Yet, labeled,
12 | // solid colors, 85/70/50 thresholds). EMeter is the generic prototype meter:
13 | //   - default (no variant) !' positive-progress gradient fill (student)
14 | //   - strong/good/attention/critical !' status-by-color
15 | //   - pro !' flat, muted, inset professional variant (institution side)
16 | //
17 | // Reduced-motion safe; renders role="meter" with aria-valuenow/min/max.
18 | //
19 | // The pure helpers (clampPercent / emeterFillBackground) are exported alongside
20 | // the component for isolated unit + property testing – matching the established
21 | // AttainmentBar.tsx pattern, hence the same targeted lint exception.
22 | // =====
23 | /* eslint-disable react-refresh/only-export-components */
24 |
25 | import { cn } from "@lib/utills";
26 |
27 | /** Semantic status modifiers (fill color encodes status). */
28 | export type EMeterVariant = "strong" | "good" | "attention" | "critical";
29 |
30 | /**
31 |  * Exact fill backgrounds reproduced verbatim from prototype/shared.css `.emeter`.
32 |  * - `default` = student/brand-forward (base = 93.65deg brand gradient).
33 |  * - `pro` = institution flat/muted professional variant (`.emeter.pro`).
34 |  */
35 | export const EMETER_FILL = {
36 |   default: {
37 |     base: "var(--brand-gradient)",
38 |     strong: "#16a34a",
39 |     good: "#0d9488",
40 |     attention: "#f59e0b",
41 |     critical: "#ef4444",
42 |   },
43 |   pro: {
44 |     base: "#334155",
45 |     strong: "#15803d",
46 |     good: "#0f766e",
47 |     attention: "#b45309",
48 |     critical: "#b91c1c",
49 |   },
50 | } as const;
51 |
52 | /**
53 |  * Clamp a raw percentage into the meter's [0,100] domain. Non-finite inputs
54 |  * (NaN / ±Infinity) collapse to 0 – a meter cannot render an undefined or
55 |  * infinite width, and 0 is the safe, non-misleading default.
56 |  */
57 | export function clampPercent(value: number): number {
58 |   if (!Number.isFinite(value)) return 0;
59 |   if (value < 0) return 0;
60 |   if (value > 100) return 100;
61 |   return value;
62 | }
63 |
64 | /**
65 |  * Resolve the exact fill `background` for a variant (+ `pro`), matching the
66 |  * prototype `.emeter` rules. An omitted variant returns the default fill
67 |  * (brand gradient for student, flat slate for `pro`).
68 |  */
69 | export function emeterFillBackground(
70 |   variant?: EMeterVariant,
71 |   pro = false
72 | ): string {
73 |   const set = pro ? EMETER_FILL.pro : EMETER_FILL.default;
74 |   return variant ? set[variant] : set.base;

```

```

75 | }
76 |
77 | export interface EMeterProps {
78 |   /** Progress/attainment percentage; clamped to [0,100]. */
79 |   value: number;
80 |   /**
81 |    * Semantic status. Omit for the default positive-progress fill (student =
82 |    * brand gradient, institution = flat slate when `pro`).
83 |    */
84 |   variant?: EMeterVariant;
85 |   /** Institution/professional flat variant (`.emeter.pro`). */
86 |   pro?: boolean;
87 |   /** Accessible name for the meter (sets `aria-label`). */
88 |   label?: string;
89 |   className?: string;
90 | }
91 |
92 | const EMeter = ({
93 |   value,
94 |   variant,
95 |   pro = false,
96 |   label,
97 |   className,
98 | }: EMeterProps) => {
99 |   const pct = clampPercent(value);
100 |
101 |   return (
102 |     <div
103 |       role="meter"
104 |       aria-label={label}
105 |       aria-valuenow={Math.round(pct)}
106 |       aria-valuemin={0}
107 |       aria-valuemax={100}
108 |       className={cn(
109 |         "w-full overflow-hidden rounded-full bg-[#eef2f6]",
110 |         pro ? "h-2.5 shadow-[inset_0_1px_2px_rgba(15,23,42,0.06)]" : "h-2",
111 |         className
112 |       )}
113 |     >
114 |       <span
115 |         className="block h-full rounded-[inherit] transition-[width] duration-500 motion-reduce:transition-non
116 |         style={{
117 |           width: `${pct}%`,
118 |           background: emeterFillBackground(variant, pro),
119 |         }}
120 |       />
121 |     </div>
122 |   );
123 | };
124 |
125 | export default EMeter;
126 |

```

## FILE: src/design-system/patterns/GradientCardHeader.tsx

TYPE: .tsx | LINES: 1-52 | SHA256: 043c747e0b3a23f39e0e5a364d715bc55fc714d37b2db407ce5da0de3d20291a

```

1 | // =====
2 | // GradientCardHeader – section-card header strip (design system)
3 | // =====
4 | // A full-width card-top header: brand-gradient icon chip + title on a hairline
5 | // bordered strip (prototype `.pcard` gradient header). Distinct from
6 | // `SectionHeader` (the inline `.sec-h`). Internalized into the design system
7 | // (P0.4/$A) so screens no longer depend on the legacy shared component.
8 | // =====
9 |
10 | import type { LucideIcon } from "lucide-react";
11 | import type { ReactNode } from "react";
12 |
13 | import { cn } from "@lib/utills";
14 |
15 | export interface GradientCardHeaderProps {
16 |   icon?: LucideIcon;
17 |   title: string;
18 |   className?: string;
19 |   children?: ReactNode;
20 | }
21 |
22 | const GradientCardHeader = ({
23 |   icon: Icon,
24 |   title,
25 |   className,
26 |   children,
27 | }: GradientCardHeaderProps) => (
28 |   <div
29 |     className={cn(
30 |       "flex items-center gap-2 border-b border-[#eef2f6] px-6 py-4",
31 |       className
32 |     )}
33 |   >
34 |     {Icon && (
35 |       // Prototype `.sec-h .chip`: 26px gradient chip, 9px radius, teal halo.
36 |       <span
37 |         className="inline-flex size-[26px] shrink-0 items-center justify-center rounded-[9px] text-white shado
38 |         style={{ background: "var(--brand-gradient)" }}
39 |         aria-hidden="true"
40 |       >
41 |         <Icon className="h-3.5 w-3.5" />
42 |       </span>
43 |     )}
44 |     <h2 className="truncate text-[13px] font-extrabold tracking-[0.02em] text-slate-900">
45 |       {title}
46 |     </h2>
47 |     {children && <div className="ms-auto shrink-0">{children}</div>}
48 |   </div>

```

```

49 | });
50 |
51 | export default GradientCardHeader;
52 |

```

## FILE: src/design-system/patterns/HeroCard.tsx

TYPE: .tsx | LINES: 1-36 | SHA256: b04e1b7313050c9dddc2fd2dafef34b715e1b9777f2d5e59c2702c58e79f9e34e

```

1 | // =====
2 | // HeroCard – dark gradient hero surface (design system)
3 | // =====
4 | // The dashboard/hero card: slate!blue!indigo gradient with white text, matching
5 | // the prototype hero treatment. Content is supplied by the caller.
6 | // =====
7 |
8 | import type { ReactNode } from "react";
9 | import { cn } from "@lib/utils";
10 |
11 | const HERO_GRADIENT =
12 |   "linear-gradient(135deg, #0f172a 0%, #1e3a8a 50%, #312e81 100%)";
13 |
14 | export interface HeroCardProps {
15 |   children: ReactNode;
16 |   /** When set, exposes the card as a labeled region to assistive tech. */
17 |   ariaLabel?: string;
18 |   className?: string;
19 | }
20 |
21 | const HeroCard = ({ children, ariaLabel, className }: HeroCardProps) => (
22 |   <div
23 |     role={ariaLabel ? "region" : undefined}
24 |     aria-label={ariaLabel}
25 |     className={cn(
26 |       "overflow-hidden rounded-xl border-0 text-white shadow-lg",
27 |       className
28 |     )}
29 |     style={{ background: HERO_GRADIENT }}
30 |   >
31 |     {children}
32 |   </div>
33 | );
34 |
35 | export default HeroCard;
36 |

```

## FILE: src/design-system/patterns/HeroCarousel.tsx

TYPE: .tsx | LINES: 1-170 | SHA256: a1304aeada0a9815138aa091c6a3f7113d1d1bc3b877617342c2c443c32eeed

```

1 | // =====
2 | // HeroCarousel – prototype `hero-carousel` (shared.css + shared.js
3 | // `initHeroCarousel`). A multi-slide dashboard hero: dots + prev/next arrows +
4 | // touch swipe + auto-advance that pauses on hover/focus, reduced-motion-safe.
5 | //
6 | // Generic + presentational: the caller supplies fully-styled slides and the
7 | // root's look (rounding / gradient background / shadow) via `className`+`style`,
8 | // exactly like the prototype markup where the carousel root carries
9 | // `rounded-2xl text-white shadow-lg` + `background:var(--hero-gradient)`.
10 | //
11 | // Values are reproduced 1:1 from `prototype/shared.css`:
12 | //   .hero-slides transition .35s cubic-bezier(.2,.7,.2,1)
13 | //   .hero-dots button 16x4 radius2 bg rgba(255,255,255,.25); .on -> 22px #fff
14 | //   .hero-arrow 26x26 round bg rgba(255,255,255,.14) border rgba(255,255,255,.2)
15 | //   auto-advance 7000ms (shared.js), swipe threshold 40px
16 | // =====
17 |
18 | import { useCallback, useEffect, useRef, useState } from "react";
19 | import { useReducedMotion } from "framer-motion";
20 | import { useTranslation } from "react-i18next";
21 | import { Button } from "@components/ui/button";
22 | import { cn } from "@lib/utils";
23 |
24 | export interface HeroCarouselProps {
25 |   /** Fully-styled slide nodes. With a single slide, chrome (dots/arrows) is hidden. */
26 |   slides: React.ReactNode[];
27 |   /** Applied to the carousel root – carries the hero look (rounding/bg/shadow). */
28 |   className?: string;
29 |   style?: React.CSSProperties;
30 |   /** Auto-advance interval; matches the prototype's 7s. */
31 |   autoAdvanceMs?: number;
32 |   /** Accessible name for the carousel region. */
33 |   ariaLabel?: string;
34 | }
35 |
36 | /**
37 |  * Prototype-faithful hero carousel. Auto-advance is disabled under
38 |  * `prefers-reduced-motion` and while the user is hovering/focused within it.
39 |  */
40 | const HeroCarousel = ({
41 |   slides,
42 |   className,
43 |   style,
44 |   autoAdvanceMs = 7000,
45 |   ariaLabel = "Highlights",
46 | }: HeroCarouselProps) => {
47 |   const { t } = useTranslation("common");
48 |   const count = slides.length;
49 |   const [index, setIndex] = useState(0);
50 |   const [paused, setPaused] = useState(false);
51 |   const reduceMotion = useReducedMotion();
52 |   const touchStartX = useRef<number | null>(null);
53 |
54 |   const current = count > 0 ? index % count : 0;
55 |
56 |   const go = useCallback(

```

```

57 |     (n: number) => {
58 |       if (count === 0) return;
59 |       setIndex(((n % count) + count) % count);
60 |     },
61 |     [count]
62 |   );
63 |
64 |   // Auto-advance (functional update so the interval never needs re-creating on
65 |   // each tick). Pauses on hover/focus and honors reduced-motion.
66 |   useEffect(() => {
67 |     if (count < 2 || paused || reduceMotion) return;
68 |     const id = window.setInterval(
69 |       () => setIndex((i) => (i + 1) % count),
70 |       autoAdvanceMs
71 |     );
72 |     return () => window.clearInterval(id);
73 |   }, [count, paused, reduceMotion, autoAdvanceMs]);
74 |
75 |   const onTouchStart = (e: React.TouchEvent) => {
76 |     touchStartX.current = e.touches[0]?.clientX ?? null;
77 |   };
78 |   const onTouchEnd = (e: React.TouchEvent) => {
79 |     if (touchStartX.current == null) return;
80 |     const dx = (e.changedTouches[0]?.clientX ?? 0) - touchStartX.current;
81 |     if (Math.abs(dx) > 40) go(current + (dx < 0 ? 1 : -1));
82 |     touchStartX.current = null;
83 |   };
84 |
85 |   const multi = count > 1;
86 |
87 |   return (
88 |     <div
89 |       className={cn("relative overflow-hidden", className)}
90 |       style={style}
91 |       role="region"
92 |       aria-roledescription="carousel"
93 |       aria-label={ariaLabel}
94 |       onMouseEnter={() => setPaused(true)}
95 |       onMouseLeave={() => setPaused(false)}
96 |       onFocusCapture={() => setPaused(true)}
97 |       onBlurCapture={() => setPaused(false)}
98 |     >
99 |       {multi && (
100 |         <>
101 |           <Button
102 |             type="button"
103 |             variant="ghost"
104 |             size="icon-xs"
105 |             onClick={() => go(current - 1)}
106 |             aria-label={t("carousel.previous")}
107 |             className="absolute start-1.5 top-1/2 z-[5] !h-[26px] !w-[26px] -translate-y-1/2 rounded-full border
108 |           >
109 |             <span aria-hidden="true">&lsaquo;</span>
110 |           </Button>
111 |           <Button
112 |             type="button"
113 |             variant="ghost"
114 |             size="icon-xs"
115 |             onClick={() => go(current + 1)}
116 |             aria-label={t("carousel.next")}
117 |             className="absolute end-1.5 top-1/2 z-[5] !h-[26px] !w-[26px] -translate-y-1/2 rounded-full border
118 |           >
119 |             <span aria-hidden="true">&rsaquo;</span>
120 |           </Button>
121 |         </>
122 |       )}
123 |
124 |     <div
125 |       className="flex transition-transform duration-[350ms] ease-[cubic-bezier(0.2,0.7,0.2,1)] motion-reduce
126 |       style={{ transform: `translateX(-${current * 100}%)` }}
127 |       onTouchStart={onTouchStart}
128 |       onTouchEnd={onTouchEnd}
129 |     >
130 |       {/* Slides are a stable, caller-ordered list – index identity is correct. */}
131 |       {slides.map((slide, i) => (
132 |         <div
133 |           key={i}
134 |           className="min-w-0 flex-[0_0_100%]"
135 |           role="group"
136 |           aria-roledescription="slide"
137 |           aria-label={t("carousel.slideOf", {
138 |             current: i + 1,
139 |             total: count,
140 |           })}
141 |         >
142 |           {slide}
143 |         </div>
144 |       ))}
145 |     </div>
146 |
147 |     {multi && (
148 |       <div className="flex justify-center gap-[5px] pb-2.5 pt-2">
149 |         {slides.map((_, i) => (
150 |           <Button
151 |             key={i}
152 |             type="button"
153 |             variant="ghost"
154 |             onClick={() => go(i)}
155 |             aria-label={t("carousel.goTo", { number: i + 1 })}
156 |             aria-current={i === current}
157 |             className={cn(
158 |               "!h-1 min-h-0 rounded-sm p-0 transition-all duration-150 hover:bg-white/50",
159 |               i === current ? "!w-[22px] bg-white" : "!w-4 bg-white/25"
160 |             )}

```

```

161 |         />
162 |     ))})
163 | </div>
164 | })
165 | </div>
166 | );
167 | };
168 |
169 | export default HeroCarousel;
170 |

```

FILE: [src/design-system/patterns/index.ts](#)

TYPE: .ts | LINES: 1-61 | SHA256: 26731295e1336b81d4156e39fc37090cb00b26599a623e55fde77966e0905663

```

1 | // =====
2 | // L2 patterns – prototype design-system component surface.
3 | // Barrel so screens can `import { PageHeader, SectionCard, KPICard, ... } from
4 | // "@design-system/patterns"`. A `.ts` barrel (not a component module), so
5 | // react-refresh does not apply – safe to re-export components + types together.
6 | // =====
7 |
8 | export { default as PageHeader } from "./PageHeader";
9 | export type { PageHeaderProps } from "./PageHeader";
10 |
11 | export { default as PCard } from "./PCard";
12 | export type { PCardProps } from "./PCard";
13 |
14 | export { default as SectionHeader } from "./SectionHeader";
15 | export type { SectionHeaderProps } from "./SectionHeader";
16 |
17 | export { default as SectionCard } from "./SectionCard";
18 | export type { SectionCardProps } from "./SectionCard";
19 |
20 | export { default as KPICard } from "./KPICard";
21 | export type { KPICardProps } from "./KPICard";
22 |
23 | export { default as HeroCard } from "./HeroCard";
24 | export type { HeroCardProps } from "./HeroCard";
25 |
26 | export { default as HeroCarousel } from "./HeroCarousel";
27 | export type { HeroCarouselProps } from "./HeroCarousel";
28 |
29 | export { RailCard, RailHead, RailRow } from "./Rail";
30 | export type { RailCardProps, RailHeadProps, RailRowProps } from "./Rail";
31 |
32 | export { default as StatusDot } from "./StatusDot";
33 | export type { DotTone, StatusDotProps } from "./StatusDot";
34 |
35 | export { default as StatePanel } from "./StatePanel";
36 | export type { StatePanelProps } from "./StatePanel";
37 |
38 | export { default as Shimmer } from "./Shimmer";
39 | export type { ShimmerProps } from "./Shimmer";
40 |
41 | // Internalized patterns (PARITY.md SA) – the design system OWNS these now; the
42 | // legacy `@/components/shared/*` files are thin re-export shims that point here
43 | // (deleted at P5 once no legacy screen imports them directly).
44 | export { default as MasteryRing } from "./MasteryRing";
45 | export type { MasteryRingProps } from "./MasteryRing";
46 | export { default as WelcomeHero } from "./WelcomeHero";
47 | export type { WelcomeHeroProps } from "./WelcomeHero";
48 | export { SeverityIcon } from "./SeverityIcon";
49 | export type { SeverityIconProps } from "./SeverityIcon";
50 | export { default as GradientCardHeader } from "./GradientCardHeader";
51 | export type { GradientCardHeaderProps } from "./GradientCardHeader";
52 |
53 | export { default as EMeter } from "./EMeter";
54 | export {
55 |   clampPercent,
56 |   emeterFillBackground,
57 |   EMETER_FILL,
58 |   type EMeterVariant,
59 |   type EMeterProps,
60 | } from "./EMeter";
61 |

```

FILE: [src/design-system/patterns/KPICard.tsx](#)

TYPE: .tsx | LINES: 1-65 | SHA256: 6c709909df317cc07ecd57055b01bc71e087e07819a8590a4f06dc53427e5c43

```

1 | // =====
2 | // KPICard – prototype `.kpi` metric tile (design system)
3 | // =====
4 | // Metric card: tiny uppercase label + black value + icon in a soft rounded
5 | // surface with a group-hover scale. Matches the prototype KPI treatment.
6 | // =====
7 |
8 | import type { LucideIcon } from "lucide-react";
9 | import { cn } from "@lib/utils";
10 |
11 | export interface KPICardProps {
12 |   icon: LucideIcon;
13 |   label: string;
14 |   value: string | number;
15 |   iconBgClass?: string;
16 |   iconColorClass?: string;
17 |   /** Value color; defaults to deep brand blue (not near-black). */
18 |   valueClassName?: string;
19 |   className?: string;
20 | }
21 |
22 | const KPICard = ({
23 |   icon: Icon,
24 |   label,
25 |   value,

```



```

26 | iconBgClass = "bg-blue-50",
27 | iconColorClass = "text-blue-600",
28 | valueClassName = "text-sky-700",
29 | className,
30 |}: KPICardProps) => (
31 | <div
32 |   className={cn(
33 |     // Prototype `.pcard` surface holding a `.kpi` layout (values verbatim
34 |     // from shared.css: 20px radius, hairline border, two-layer depth + lift).
35 |     "rounded-[20px] border border-[#eef2f6] bg-white p-4 shadow-[0_1px_2px_rgba(16,24,40,0.04),0_10px_26px_r
36 |     className
37 |   )}
38 | >
39 |   <div className="flex items-start justify-between gap-2.5">
40 |     <div>
41 |       <p className="text-[10px] font-black uppercase tracking-widest text-gray-500">
42 |         {label}
43 |       </p>
44 |       <p className={cn("mt-1 text-2xl font-black", valueClassName)}>
45 |         {value}
46 |       </p>
47 |     </div>
48 |     {/* Prototype `.kpi-ic`: fixed 38px tile, 11px radius. */}
49 |     <div>
50 |       className={cn(
51 |         "flex size-[38px] shrink-0 items-center justify-center rounded-[11px]",
52 |         iconBgClass
53 |       )}
54 |     >
55 |       <Icon
56 |         className={cn("h-[18px] w-[18px]", iconColorClass)}
57 |         aria-hidden="true"
58 |       />
59 |     </div>
60 |   </div>
61 | </div>
62 | );
63 |
64 | export default KPICard;
65 |

```

## FILE: src/design-system/patterns/MasteryRing.tsx

TYPE: .tsx | LINES: 1-126 | SHA256: ea88f8520dc62f8c729101a3cb805ccd0a703d67b47c5dc5a4e03812cb8d0fcb

```

1 | // =====
2 | // MasteryRing – SVG progress/mastery ring (design system, prototype `.ring-mini`)
3 | // =====
4 | // Circular progress ring for mastery / attainment / completion. `tone="auto"`
5 | // colors by OBE attainment thresholds; `tone="brand"` uses the teal/blue brand
6 | // gradient (prototype ring fill). Dark-mode aware, reduced-motion safe.
7 | // Internalized into the design system (P0.4/$A) so screens no longer depend on
8 | // the legacy shared component.
9 | // =====
10 |
11 | import * as React from "react";
12 |
13 | import { cn } from "@lib/Utils";
14 |
15 | export interface MasteryRingProps {
16 |   /** Progress value, 0-100 (clamped and rounded). */
17 |   value: number;
18 |   /** Outer diameter in px. */
19 |   size?: number;
20 |   /** Ring stroke width in px. */
21 |   strokeWidth?: number;
22 |   /**
23 |    * "auto" (default) colors the ring by attainment threshold; "brand" uses the
24 |    * teal/blue brand gradient regardless of value.
25 |    */
26 |   tone?: "auto" | "brand";
27 |   /** Show the numeric percentage in the center. Ignored when `children` is set. */
28 |   showValue?: boolean;
29 |   /** Accessible label; defaults to "<value>%". */
30 |   label?: string;
31 |   /** Optional custom center content (overrides `showValue`). */
32 |   children?: React.ReactNode;
33 |   className?: string;
34 | }
35 |
36 | /** Maps a value to its attainment-level stroke color (domain thresholds). */
37 | const attainmentStroke = (v: number): string => {
38 |   if (v >= 85) return "#22c55e"; // success – Excellent
39 |   if (v >= 70) return "#3b82f6"; // brand blue – Satisfactory
40 |   if (v >= 50) return "#f59e0b"; // warning – Developing
41 |   return "#ef4444"; // destructive – Not Yet
42 | };
43 |
44 | const MasteryRing = ({
45 |   value,
46 |   size = 72,
47 |   strokeWidth = 8,
48 |   tone = "auto",
49 |   showValue = true,
50 |   label,
51 |   children,
52 |   className,
53 | }: MasteryRingProps) => {
54 |   const clamped = Math.max(0, Math.min(100, Math.round(value)));
55 |   const radius = (size - strokeWidth) / 2;
56 |   const circumference = 2 * Math.PI * radius;
57 |   const offset = circumference - (clamped / 100) * circumference;
58 |   const gradientId = React.useId();
59 |   const stroke =
60 |     tone === "brand" ? `url(#${gradientId})` : attainmentStroke(clamped);

```

```

61 |
62 | return (
63 |   <div
64 |     className={cn(
65 |       "relative inline-flex items-center justify-center",
66 |       className
67 |     )}
68 |     style={{ width: size, height: size }}
69 |   >
70 |     <svg
71 |       width={size}
72 |       height={size}
73 |       viewBox={`0 0 ${size} ${size}`}
74 |       role="img"
75 |       aria-label={label ?? `${clamped}%`}
76 |     >
77 |       {tone === "brand" && (
78 |         <defs>
79 |           <linearGradient id={gradientId} x1="0%" y1="0%" x2="100%" y2="0%">
80 |             <stop offset="0%" stopColor="#14b8a6" />
81 |             <stop offset="100%" stopColor="#0382bd" />
82 |           </linearGradient>
83 |         </defs>
84 |       )}
85 |       {/* Track */}
86 |       <circle
87 |         cx={size / 2}
88 |         cy={size / 2}
89 |         r={radius}
90 |         fill="none"
91 |         strokeWidth={strokeWidth}
92 |         className="stroke-slate-200 dark:stroke-slate-700"
93 |       />
94 |       {/* Progress */}
95 |       <circle
96 |         cx={size / 2}
97 |         cy={size / 2}
98 |         r={radius}
99 |         fill="none"
100 |         stroke={stroke}
101 |         strokeWidth={strokeWidth}
102 |         strokeLinecap="round"
103 |         strokeDasharray={circumference}
104 |         strokeDashoffset={offset}
105 |         transform={`rotate(-90 ${size / 2} ${size / 2})`}
106 |         className="transition-[stroke-dashoffset] duration-500 motion-reduce:transition-none"
107 |       />
108 |     </svg>
109 |     {children ? (
110 |       <div className="absolute inset-0 flex items-center justify-center">
111 |         {children}
112 |       </div>
113 |     ) : showValue ? (
114 |       <span
115 |         className="absolute inset-0 flex items-center justify-center text-sm font-bold tabular-nums text-for"
116 |         aria-hidden="true"
117 |       >
118 |         {clamped}%
119 |       </span>
120 |     ) : null}
121 |   </div>
122 | );
123 | };
124 |
125 | export default MasteryRing;
126 |

```

FILE: [src/design-system/patterns/PageHeader.tsx](#)

TYPE: .tsx | LINES: 1-28 | SHA256: 762f4ffa4b0b35a1f09aa15e49681a4eace6596e810b2e9a718f91124c1f4ab3

```

1 | // =====
2 | // PageHeader – page title row (prototype design system)
3 | // =====
4 | // The standard page heading: `text-2xl font-bold tracking-tight` with an
5 | // optional trailing action, matching the prototype's page titles.
6 | // =====
7 |
8 | import type { ReactNode } from "react";
9 | import { cn } from "@lib/utils";
10 |
11 | export interface PageHeaderProps {
12 |   title: string;
13 |   /** Optional trailing action (button/link), inline-end aligned. */
14 |   action?: ReactNode;
15 |   className?: string;
16 | }
17 |
18 | const PageHeader = ({ title, action, className }: PageHeaderProps) => (
19 |   <div className={cn("flex items-center justify-between gap-3", className)}>
20 |     <h1 className="text-2xl font-bold tracking-tight text-foreground">
21 |       {title}
22 |     </h1>
23 |     {action ? <div className="shrink-0">{action}</div> : null}
24 |   </div>
25 | );
26 |
27 | export default PageHeader;
28 |

```

## FILE: src/design-system/patterns/PCard.tsx

TYPE: .tsx | LINES: 1-33 | SHA256: a47b903e6aeaaa578330676074d3ea33777463b1c2675cba443b86678496a877

```
1 | // =====
2 | // PCard – prototype `p-card` surface (design system)
3 | // =====
4 | // The base card surface: white, borderless, soft shadow, rounded-xl. No padding
5 | // by default (compose with `p-4`/`p-6` or use SectionCard).
6 | // =====
7 |
8 | import type { ReactNode } from "react";
9 | import type { HTMLAttributes } from "react";
10 | import { cn } from "@lib/utils";
11 |
12 | export interface PCardProps extends HTMLAttributes<HTMLDivElement> {
13 |   children: ReactNode;
14 |   className?: string;
15 | }
16 |
17 | const PCard = ({ children, className, ...props }: PCardProps) => (
18 |   <div
19 |     data-slot="card"
20 |     className={cn(
21 |       // Prototype `p-card`: white, 1px hairline border, 20px radius, soft
22 |       // two-layer depth, and a hover-lift (values verbatim from shared.css).
23 |       "rounded-[20px] border border-[#eef2f6] bg-white shadow-[0_1px_2px_rgba(16,24,40,0.04),0_10px_26px_rgba(
24 |       className
25 |     )}
26 |     {...props}
27 |   >
28 |     {children}
29 |   </div>
30 | );
31 |
32 | export default PCard;
33 |
```

## FILE: src/design-system/patterns/Rail.tsx

TYPE: .tsx | LINES: 1-73 | SHA256: 16e02ef18ec639a814891df842583a62fde63e03c73d582e11781ed20f02dc77

```
1 | // =====
2 | // Rail primitives – prototype right-rail building blocks (shared.css
3 | // `rail-card` / `rail-h` / `rail-row`). Shared by every per-role dashboard
4 | // rail (student, teacher, parent, coordinator, admin) so the rail treatment is
5 | // defined once and stays 1:1 with the prototype. Pure presentational – no
6 | // data, no router – the feature rails compose these with real hooks.
7 | // =====
8 |
9 | import { cn } from "@lib/utils";
10 |
11 | export interface RailCardProps {
12 |   children: React.ReactNode;
13 |   className?: string;
14 |   style?: React.CSSProperties;
15 | }
16 |
17 | /** `rail-card` – white, 1px slate-200 border, radius-16, p-4, 14px bottom gap. */
18 | export const RailCard = ({ children, className, style }: RailCardProps) => (
19 |   <div
20 |     className={cn(
21 |       "mb-3.5 rounded-2xl border border-slate-200 bg-white p-4 dark:border-border dark:bg-card",
22 |       className
23 |     )}
24 |     style={style}
25 |   >
26 |     {children}
27 |   </div>
28 | );
29 |
30 | export interface RailHeadProps {
31 |   title: string;
32 |   /** Optional right-aligned note (`rail-r`), e.g. a count or "3d left". */
33 |   right?: string;
34 |   /** false when rendered on a dark card (white text) instead of a light one. */
35 |   onLight?: boolean;
36 | }
37 |
38 | /** `rail-h` – uppercase 11px/800 slate-400 header with an optional right note. */
39 | export const RailHead = ({ title, right, onLight = true }: RailHeadProps) => (
40 |   <div
41 |     className={cn(
42 |       "mb-2.5 flex items-center justify-between text-[11px] font-extrabold uppercase tracking-[0.08em]",
43 |       onLight ? "text-slate-400" : "text-white/60"
44 |     )}
45 |   >
46 |     <span>{title}</span>
47 |     {right ? (
48 |       <span className="text-[10px] font-bold normal-case tracking-normal text-slate-400">
49 |         {right}
50 |       </span>
51 |     ) : null}
52 |   </div>
53 | );
54 |
55 | export interface RailRowProps {
56 |   children: React.ReactNode;
57 |   className?: string;
58 |   style?: React.CSSProperties;
59 | }
60 |
61 | /** `rail-row` – 13px slate-700 row; bold values render 12px/800 slate-900. */
62 | export const RailRow = ({ children, className, style }: RailRowProps) => (
63 |   <div
```

```

64 |     className={cn(
65 |       "flex items-center gap-2.5 py-[5px] text-[13px] text-slate-700 dark:text-slate-300",
66 |       className
67 |     )}
68 |     style={style}
69 |   >
70 |     {children}
71 |   </div>
72 | );
73 |

```

#### FILE: src/design-system/patterns/SectionCard.tsx

TYPE: .tsx | LINES: 1-36 | SHA256: 73f446aedf23cfa2cd663b8f09fc482f11c8fc2710b8c2463bdd9949e4d2e838

```

1 | // =====
2 | // SectionCard – PCard + in-body SectionHeader (design system)
3 | // =====
4 | // The standard prototype section card: a white surface with a `sec-h` header
5 | // (gradient chip + title + optional action) and a content body.
6 | // =====
7 |
8 | import type { LucideIcon } from "lucide-react";
9 | import type { ReactNode } from "react";
10 | import { cn } from "@lib/Utils";
11 | import PCard from "../PCard";
12 | import SectionHeader from "../SectionHeader";
13 |
14 | export interface SectionCardProps {
15 |   icon?: LucideIcon;
16 |   title: string;
17 |   action?: ReactNode;
18 |   children: ReactNode;
19 |   className?: string;
20 | }
21 |
22 | const SectionCard = ({
23 |   icon,
24 |   title,
25 |   action,
26 |   children,
27 |   className,
28 | }: SectionCardProps) => (
29 |   <PCard className={cn("space-y-4 p-6", className)}>
30 |     <SectionHeader icon={icon} title={title} action={action} />
31 |     {children}
32 |   </PCard>
33 | );
34 |
35 | export default SectionCard;
36 |

```

#### FILE: src/design-system/patterns/SectionHeader.tsx

TYPE: .tsx | LINES: 1-54 | SHA256: 32a174a4965b41a8c7673094d6e1906f34ae8b2b148b928411236c2ea9c785b8

```

1 | // =====
2 | // SectionHeader – prototype `sec-h` header (design system)
3 | // =====
4 | // A brand-gradient icon chip + title (+ optional description/action). Use inside
5 | // a card body or standalone. RTL-safe (`ms-auto`), dark-mode aware.
6 | // =====
7 |
8 | import type { LucideIcon } from "lucide-react";
9 | import type { ReactNode } from "react";
10 | import { cn } from "@lib/Utils";
11 |
12 | export interface SectionHeaderProps {
13 |   /** Leading icon shown inside the brand-gradient chip. */
14 |   icon?: LucideIcon;
15 |   title: string;
16 |   description?: string;
17 |   /** Trailing action, inline-end aligned. */
18 |   action?: ReactNode;
19 |   as?: "h2" | "h3";
20 |   className?: string;
21 | }
22 |
23 | const SectionHeader = ({
24 |   icon: Icon,
25 |   title,
26 |   description,
27 |   action,
28 |   as: Heading = "h2",
29 |   className,
30 | }: SectionHeaderProps) => (
31 |   <div className={cn("flex items-center gap-2", className)}>
32 |     {Icon && (
33 |       <span
34 |         className="inline-flex size-[28px] shrink-0 items-center justify-center rounded-[9px] border border-sl
35 |         aria-hidden="true"
36 |       >
37 |         <Icon className="h-3.5 w-3.5" />
38 |       </span>
39 |     )}
40 |     <div className="min-w-0">
41 |       {/** Prototype `sec-h h2`: 13px / 800 / .02em / slate-900. */}
42 |       <Heading className="truncate text-[13px] font-extrabold tracking-[0.02em] text-slate-900">
43 |         {title}
44 |       </Heading>
45 |       {description && (
46 |         <p className="text-sm text-muted-foreground">{description}</p>
47 |       )}
48 |     </div>
49 |     {action && <div className="ms-auto shrink-0">{action}</div>}
50 |   </div>

```

```

51 | });
52 |
53 | export default SectionHeader;
54 |

```

## FILE: src/design-system/patterns/SeverityIcon.tsx

TYPE: .tsx | LINES: 1-82 | SHA256: c5c72be5696b253bb315e09a633ffc6dd1cce1be20a3a990adbccdedfe844c92

```

1 | // =====
2 | // SeverityIcon – status icon tile with a soft halo ring (design system)
3 | // =====
4 | // Prototype `sev-*`: a leading severity icon in a tinted tile. Status is
5 | // conveyed by color + icon; callers MUST pair it with a text label/badge nearby
6 | // (never color alone – WCAG 1.4.1 / R10.3). When `label` is provided the tile is
7 | // announced; otherwise it is decorative. Internalized into the design system
8 | // (P0.4/$A) so screens no longer depend on the legacy shared component.
9 | // =====
10 |
11 | import * as React from "react";
12 | import { cva, type VariantProps } from "class-variance-authority";
13 | import type { LucideIcon } from "lucide-react";
14 |
15 | import { cn } from "@lib/utils";
16 |
17 | const severityIconVariants = cva(
18 |   "inline-flex items-center justify-center rounded-xl ring-4 shrink-0 [>svg]:shrink-0",
19 |   {
20 |     variants: {
21 |       severity: {
22 |         high: "bg-red-50 text-red-600 ring-red-500/10 dark:bg-red-950/40 dark:text-red-400 dark:ring-red-400/10",
23 |         med: "bg-amber-50 text-amber-600 ring-amber-500/10 dark:bg-amber-950/40 dark:text-amber-400 dark:ring-amber-400/10",
24 |         low: "bg-green-50 text-green-600 ring-green-500/10 dark:bg-green-950/40 dark:text-green-400 dark:ring-green-400/10",
25 |         info: "bg-blue-50 text-blue-600 ring-blue-500/10 dark:bg-blue-950/40 dark:text-blue-400 dark:ring-blue-400/10",
26 |         brand: "bg-teal-50 text-teal-600 ring-teal-500/10 dark:bg-teal-950/40 dark:text-teal-400 dark:ring-teal-400/10",
27 |         neutral: "bg-slate-100 text-slate-600 ring-slate-400/10 dark:bg-slate-800 dark:text-slate-300 dark:ring-slate-400/10",
28 |       },
29 |       size: {
30 |         sm: "size-8 [>svg]:size-4",
31 |         md: "size-10 [>svg]:size-5",
32 |         lg: "size-12 [>svg]:size-6",
33 |       },
34 |     },
35 |     defaultVariants: {
36 |       severity: "brand",
37 |       size: "md",
38 |     },
39 |   },
40 | );
41 |
42 | export interface SeverityIconProps
43 |   extends React.ComponentProps<"span">,
44 |   VariantProps<typeof severityIconVariants> {
45 |   /** The Lucide icon to render inside the tile. */
46 |   icon: LucideIcon;
47 |   /**
48 |    * Accessible label. When provided, the tile is announced (role="img").
49 |    * When omitted, the tile is treated as decorative (aria-hidden) – in that
50 |    * case the surrounding UI MUST carry the status in text.
51 |    */
52 |   label?: string;
53 | }
54 |
55 | const SeverityIcon = ({
56 |   icon,
57 |   severity,
58 |   size,
59 |   label,
60 |   className,
61 |   ...props
62 | }: SeverityIconProps) => (
63 |   <span
64 |     {...props}
65 |     className={cn(severityIconVariants({ severity, size }), className)}
66 |     role={label ? "img" : undefined}
67 |     aria-label={label}
68 |     aria-hidden={label ? undefined : true}
69 |   >
70 |     <Icon />
71 |   </span>
72 | );
73 |
74 | SeverityIcon.displayName = "SeverityIcon";
75 |
76 |
77 | // `severityIconVariants` is intentionally NOT exported: this file is linted with
78 | // `react-refresh/only-export-components`, which forbids mixing a component export
79 | // with a constant export. If the variants are needed elsewhere, move them to a
80 | // sibling `*.variants.ts` file.
81 | export { SeverityIcon };
82 |

```

## FILE: src/design-system/patterns/Shimmer.tsx

TYPE: .tsx | LINES: 1-24 | SHA256: e92dcce7af934773a722fba911f3996d01d40c5295eb59ee040e92c01169864f

```

1 | // =====
2 | // Shimmer – skeleton loading block (design system)
3 | // =====
4 | // A sized placeholder for loading states. Uses the canonical `animate-shimmer`
5 | // gradient-sweep utility (defined in index.css) – the prototype's loading
6 | // treatment, and reduced-motion + dark-mode aware there. Compose several for
7 | // skeleton grids/lists.
8 | // =====
9 |
10 | import { cn } from "@lib/utils";

```

```

11 |
12 | export interface ShimmerProps {
13 |   className?: string;
14 | }
15 |
16 | const Shimmer = ({ className }: ShimmerProps) => (
17 |   <div
18 |     aria-hidden="true"
19 |     className={cn("animate-shimmer rounded-xl", className)}
20 |   />
21 | );
22 |
23 | export default Shimmer;
24 |

```

## FILE: src/design-system/patterns/StatePanel.tsx

TYPE: .tsx | LINES: 1-52 | SHA256: 23eff77358cdae3d8e7cae4b4a392ac271f0370785949a2b1289eab051b56c30

```

1 | // =====
2 | // StatePanel – loading / empty / error states (design system)
3 | // =====
4 | // A single primitive for the three common async states. Loading is a
5 | // reduced-motion-safe shimmer block; empty/error render inside a card surface.
6 | // =====
7 |
8 | import { cn } from "@lib/utils";
9 |
10 | export interface StatePanelProps {
11 |   variant: "loading" | "empty" | "error";
12 |   /** Message for empty/error variants. */
13 |   message?: string;
14 |   className?: string;
15 | }
16 |
17 | const StatePanel = ({ variant, message, className }: StatePanelProps) => {
18 |   if (variant === "loading") {
19 |     return (
20 |       <div
21 |         aria-hidden="true"
22 |         className={cn(
23 |           "h-40 animate-pulse rounded-xl bg-slate-100 motion-reduce:animate-none",
24 |           className
25 |         )}
26 |       />
27 |     );
28 |   }
29 |
30 |   if (variant === "error") {
31 |     return (
32 |       <div
33 |         className={cn("rounded-xl border-0 bg-white p-6 shadow-md", className)}
34 |       >
35 |         <p className="text-sm text-red-600" role="alert">
36 |           {message ?? "Something went wrong. Please try again."}
37 |         </p>
38 |       </div>
39 |     );
40 |   }
41 |
42 |   return (
43 |     <div
44 |       className={cn("rounded-xl border-0 bg-white p-6 shadow-md", className)}
45 |     >
46 |       <p className="text-sm text-gray-500">{message ?? "Nothing here yet."}</p>
47 |     </div>
48 |   );
49 | };
50 |
51 | export default StatePanel;
52 |

```

## FILE: src/design-system/patterns/StatusDot.tsx

TYPE: .tsx | LINES: 1-42 | SHA256: c693102b0a1db0df39a45f486a1ad494b50f49828d09225c04410f420386283f

```

1 | // =====
2 | // StatusDot – semantic status dot (design system)
3 | // =====
4 | // A small colored dot encoding status (PARITY.md §B.4). Decorative by default;
5 | // pass `label` to expose an accessible name.
6 | // =====
7 |
8 | import { cn } from "@lib/utils";
9 |
10 | export type DotTone = "success" | "warning" | "danger" | "info" | "neutral";
11 |
12 | const TONE_CLASS: Record<DotTone, string> = {
13 |   success: "bg-green-500",
14 |   warning: "bg-amber-500",
15 |   danger: "bg-red-500",
16 |   info: "bg-blue-500",
17 |   neutral: "bg-slate-300",
18 | };
19 |
20 | export interface StatusDotProps {
21 |   tone: DotTone;
22 |   /** Accessible label; when omitted the dot is decorative (aria-hidden). */
23 |   label?: string;
24 |   className?: string;
25 | }
26 |
27 | const StatusDot = ({ tone, label, className }: StatusDotProps) => (
28 |   <span
29 |     role={label ? "img" : undefined}
30 |     aria-label={label}

```

```

31 |     aria-hidden={label ? undefined : true}
32 |     className={cn(
33 |       // Prototype `.dot`: 8px round semantic status dot.
34 |       "inline-block h-2 w-2 shrink-0 rounded-full",
35 |       TONE_CLASS[tone],
36 |       className
37 |     )}
38 |   )}
39 | );
40 |
41 | export default StatusDot;
42 |

```

FILE: [src/design-system/patterns/WelcomeHero.tsx](#)

TYPE: .tsx | LINES: 1-83 | SHA256: 4226d5f727cd58ee7e7c637f477a5ccaa5e925abe071cf98bff3c031827dea4d

```

1 | // =====
2 | // WelcomeHero – dashboard greeting hero (design system)
3 | // =====
4 | // Universal welcome hero for all role dashboards: the prototype dark hero
5 | // gradient (135deg slate!blue!indigo, verbatim from shared.css `--hero-gradient`)
6 | // with a personalized greeting + subtitle + optional stats slot. Internalized
7 | // into the design system (P0.4/$A) so screens no longer depend on the legacy
8 | // shared component.
9 | // =====
10 |
11 | import { useTranslation } from "react-i18next";
12 | import type { ReactNode } from "react";
13 |
14 | import { Card } from "@components/ui/card";
15 | import { getDisplayFirstName } from "@lib/displayName";
16 | import type { UserRole } from "@types/app";
17 |
18 | export interface WelcomeHeroProps {
19 |   /** User's first name or full name */
20 |   name: string;
21 |   /** User's role (admin, coordinator, teacher, student, parent) */
22 |   userRole: UserRole;
23 |   /** Subtitle/motivational text */
24 |   subtitle: string;
25 |   /** Optional stats section to render on the right (e.g., XP/Level chips for Student) */
26 |   stats?: ReactNode;
27 | }
28 |
29 | /**
30 |  * Universal welcome hero card for all role dashboards. Renders the prototype's
31 |  * dark gradient hero with a personalized greeting, role context, and optional
32 |  * stats. Design: ADR-07 §8.1; Requirement 2.21.
33 |  */
34 | const WelcomeHero = ({
35 |   name,
36 |   userRole: _userRole,
37 |   subtitle,
38 |   stats,
39 | }: WelcomeHeroProps) => {
40 |   const { t } = useTranslation("common");
41 |
42 |   // Determine greeting based on time of day
43 |   const greeting = (() => {
44 |     const hour = new Date().getHours();
45 |     if (hour < 12) return t("greeting.morning", "Good morning");
46 |     if (hour < 17) return t("greeting.afternoon", "Good afternoon");
47 |     return t("greeting.evening", "Good evening");
48 |   })();
49 |
50 |   // Extract a display first name. Seeded names carry honorific titles
51 |   // ("Mr. David Okonkwo", "Dr. Aisha Al-Mansoori"), so the shared helper skips
52 |   // a leading honorific token to avoid greeting users as "Mr."/"Dr.". Empty
53 |   // names fall back to the translated "there".
54 |   const firstName = getDisplayFirstName(name) ?? t("greeting.there", "there");
55 |
56 |   return (
57 |     <Card
58 |       role="region"
59 |       aria-label={` ${greeting}, ${firstName} `}
60 |       className="border-0 shadow-lg rounded-xl overflow-hidden text-white"
61 |       style={{
62 |         background:
63 |           "linear-gradient(135deg, #0f172a 0%, #1e3a8a 50%, #312e81 100%)",
64 |       }}
65 |       data-tour="welcome-hero"
66 |     >
67 |       <div className="p-6 flex items-center justify-between">
68 |         <div>
69 |           <h1 className="text-2xl font-bold tracking-tight">
70 |             {greeting}, {firstName} 0=ÜK
71 |           </h1>
72 |           <p className="text-sm text-white/70 mt-1">{subtitle}</p>
73 |         </div>
74 |         {stats && (
75 |           <div className="hidden md:flex items-center gap-4">{stats}</div>
76 |         )}
77 |       </div>
78 |     </Card>
79 |   );
80 | };
81 |
82 | export default WelcomeHero;
83 |

```

## FILE: src/design-system/primitives/index.ts

TYPE: .ts | LINES: 1-30 | SHA256: c352aca529c15125c64843a562c64438734b4de303d37ff34b2b578676101166

```
1 // =====
2 // L2 primitives – the prototype design system adopts the existing Shadcn
3 // wrappers ( `src/components/ui/*` ) as its primitive layer (PARITY.md §A). This
4 // `*.ts` facade re-exports them so screens import primitives from a single
5 // design-system surface. `Button` carries the `tactile` variant that reproduces
6 // the prototype `btn3d`.
7 // =====
8
9 export * from "@components/ui/button";
10 export * from "@components/ui/card";
11 export * from "@components/ui/badge";
12 export * from "@components/ui/input";
13 export * from "@components/ui/textarea";
14 export * from "@components/ui/label";
15 export * from "@components/ui/select";
16 export * from "@components/ui/checkbox";
17 export * from "@components/ui/switch";
18 export * from "@components/ui/dialog";
19 export * from "@components/ui/sheet";
20 export * from "@components/ui/tabs";
21 export * from "@components/ui/popover";
22 export * from "@components/ui/separator";
23 export * from "@components/ui/alert";
24 export * from "@components/ui/avatar";
25 export * from "@components/ui/dropdown-menu";
26 export * from "@components/ui/table";
27
28 // Toast lives in the `sonner` library (ui/sonner is just the <Toaster/> host).
29 export { toast } from "sonner";
30
```

## FILE: src/design-system/README.md

TYPE: .md | LINES: 1-65 | SHA256: 9736a266f57fa594aeb5d0720dcafd32b39741af939e511f95203f2df7cee3f4

```
1 # Edeviser Design System (Path A rebuild)
2
3 The **single, shared** design system for the rebuilt UI. Extracted 1:1 from the
4 approved prototype ( `prototype/*.html` , `shared.css` ) and re-implemented as real
5 Tailwind v4 tokens + React/Shadcn primitives. The prototype HTML is the **visual
6 spec** only – it never ships.
7
8 > Full plan: `./kiro/specs/prototype-frontend-rebuild/` (requirements, design,
9 > tasks, missing-screens-catalog).
10
11 ## Architecture – layered, not per-role-forked
12
13 The system is **shared** ; only the *composition* layer is per-role. This gives
14 both easy global changes (edit tokens once) and easy role-specific changes (edit
15 one role folder) without drift.
16
17 | Layer | Location | Shared or per-role? |
18 |-----|-----|-----|
19 | **L1 Foundations** – tokens (color, gradient, type, radius, shadow, motion, dark, role-accents) | `design-sy
20 | **L2 Primitives** – Button, Card, Badge, Input, Table, Dialog, Tabs, etc. | `design-system/primitives/` | **
21 | **L3 Patterns** – MasteryRing, SectionHeader, KPICard, SeverityIcon, StatCard, chat bubbles, mascot | `desig
22 | **L3 Composition** – role dashboards, widgets, page layouts, flows | `src/features/{role}/...` | **Per-role (m
23
24 ### Why not a separate design system per role?
25
26 Forking tokens/primitives per role causes drift (the teal stops matching), 5x
27 maintenance, and broken dark mode. A **global change** (brand color, radius)
28 should be one edit in `tokens.css` ; a **role change** should be isolated to
29 `features/{role}/` ; a **role identity tweak** is a few scoped vars in the
30 `[data-role="..."]` block. That is the maintainable version of "separate files per
31 profile."
32
33 ## Folder layout (target)
34
35 ```
36 src/
37   design-system/
38     tokens.css      # L1 – canonical tokens (this slice) – 93.65deg gradient
39     themes/         # optional split of dark / role-accent scopes as it grows
40     primitives/     # L2 – Button, Card, Badge, Input, Table, Dialog, Tabs...
41     patterns/       # L2 – MasteryRing, SectionHeader, KPICard, SeverityIcon...
42     mascot/         # character system (assets from prototype/characters + brand)
43     index.ts        # barrel export
44     features/       # L3 – role-modular composition ("a folder per profile")
45       student/    teacher/  coordinator/  parent/  admin/
46     app/           # router, providers, role layouts
47     hooks/ lib/   providers/ types/      # KEPT AS-IS (backend/data layer, untouched)
48   ```
49
50 ## Hard rules
51
52 1. **Never** import `prototype/shared.css` , `prototype/shared.js` , or CDN Tailwind
53 into `src/` . Re-declare values as tokens here.
54 2. Components consume **existing hooks** ( `src/hooks/*` ) – never `supabase.*`
55 directly.
56 3. No arbitrary hex in components – reference tokens only.
57 4. Every primitive verified in **light + dark** and **LTR + RTL** (logical CSS
58 props) before use. Dark + RTL are **net-new** (the prototype defines neither).
59 5. Emoji icons in the prototype are placeholders !' use **Lucide** equivalents.
60
61 ## Status
62
63 `tokens.css` is **additive and not yet imported** – zero runtime impact until the
64 P0 wiring task in the spec. Nothing here affects the current app.
65
```



# FILE: src/design-system/tokens.css

TYPE: .css | LINES: 1-205 | SHA256: b636ed122c7545b50d37e77381d75f6381be4a7761d29f39b32093159413538a

```
1  /* =====
2  EDEVISER DESIGN SYSTEM – CANONICAL TOKENS (Path A: Prototype !' Production)
3  -----
4  SOURCE OF TRUTH for the rebuilt UI. Values extracted 1:1 from the approved
5  prototype (`prototype/shared.css` :root + design-system steering).
6
7  STATUS: CANONICAL / WIRED.
8  - This file is imported by the app shell entrypoint and provides the shared
9  prototype-derived tokens for the rebuilt UI.
10 - The legacy brand block in `src/index.css` still exists for backward
11 compatibility, but the shell-specific `--app-*` values now come from here.
12 - DO NOT import prototype/shared.css or CDN Tailwind into src/. This file
13 re-declares the prototype's *values* as real Tailwind v4 tokens.
14
15 FIDELITY DECISION (approved): the brand gradient uses the prototype's exact
16 angle 93.65deg (NOT the legacy 135deg). This is the pixel-match canonical.
17
18 SCOPE NOTE: the prototype is LIGHT-MODE + LTR only. The `.dark` values below
19 are NET-NEW (designed for parity, not copied from the prototype) and must pass
20 a dark-mode design review before cutover. RTL is handled via logical CSS
21 properties in components, not here.
22 ===== */
23
24 /* -----
25 1. Tailwind v4 @theme – exposes tokens as utilities (bg-brand, text-brand...)
26 ----- */
27 @theme {
28   --font-sans: "Noto Sans", ui-sans-serif, system-ui, sans-serif;
29
30   /* Brand palette (verbatim from prototype) */
31   --color-brand-teal: #14b8a6; /* --teal : gradient start / secondary accent */
32   --color-brand-blue: #0382bd; /* --blue : gradient end */
33   --color-brand-primary: #3b82f6; /* primary CTA / active / links */
34   --color-brand-primary-dark: #2563eb; /* hover */
35
36   /* Semantic */
37   --color-success: #22c55e;
38   --color-warning: #f59e0b;
39   --color-destructive: #ef4444;
40   --color-neutral: #64748b;
41   --color-info: #3b82f6;
42
43   /* Radii (prototype: cards 16px, tabs/links 12px, chips 999px, phone 44px) */
44   --radius-chip: 999px;
45   --radius-btn: 12px;
46   --radius-card: 16px;
47   --radius-panel: 16px;
48
49   /* Motion (prototype animation durations) */
50   --ease-spring: cubic-bezier(0.34, 1.56, 0.64, 1);
51   --dur-fast: 0.15s;
52   --dur-base: 0.35s;
53   --dur-celebrate: 0.6s;
54 }
55
56 /* -----
57 2. Brand / gradient tokens (raw CSS vars, referenced by components + @theme)
58 Gradients can't be `--color-*`; they live here as first-class tokens.
59 ----- */
60 :root {
61   /* === CORE BRAND (from prototype shared.css :root) === */
62   --brand-teal: #14b8a6;
63   --brand-blue: #0382bd;
64   --brand-primary: #3b82f6;
65   --brand-primary-dark: #2563eb;
66   --brand-secondary: #14b8a6;
67
68   /* === GRADIENTS (exact prototype values – 93.65deg adopted as canonical) === */
69   --gradient-start: #14b8a6;
70   --gradient-end: #0382bd;
71   --brand-gradient: linear-gradient(93.65deg, #14b8a6 5.37%, #0382bd 78.89%);
72   --hero-gradient: linear-gradient(
73     135deg,
74     #0f172a 0%,
75     #1e3a8a 50%,
76     #312e81 100%
77   );
78   --onboarding-bg: linear-gradient(
79     160deg,
80     #f0fdfa 0%,
81     #eff6ff 50%,
82     #eef2ff 100%
83   );
84   /* Auth background (preserved from production – an explicit invariant) */
85   --auth-bg: linear-gradient(to bottom right, #1e293b, #0f172a, #020617);
86
87   /* === SEMANTIC === */
88   --success: #22c55e;
89   --warning: #f59e0b;
90   --destructive: #ef4444;
91   --neutral: #64748b;
92   --info: #3b82f6;
93
94   /* === SURFACES / SLATE SCALE (from prototype) === */
95   --surface-bg: #f8faff; /* body background */
96   --surface-card: #ffffff;
97   --surface-subtle: #f1f5f9;
98   --surface-muted: #eef2f6;
99   --border-subtle: #e2e8f0;
100  --border-hair: #f1f5f9;
101 }
```

```

102 | --slate-900: #0f172a;
103 | --slate-800: #1e293b;
104 | --slate-700: #334155;
105 | --slate-600: #475569;
106 | --slate-500: #64748b;
107 | --slate-400: #94a3b8;
108 | --slate-300: #cbd5e1;
109 | --slate-200: #e2e8f0;
110 | --slate-100: #f1f5f9;
111 | --slate-50: #f8fafc;
112 |
113 | /* === ELEVATION (confirm .pcard against prototype at build time) === */
114 | --shadow-card: 0 1px 2px rgba(0, 0, 0, 0.04),
115 |   0 4px 12px rgba(15, 23, 42, 0.06);
116 | --shadow-card-hover: 0 8px 24px rgba(15, 23, 42, 0.1);
117 | --shadow-panel: 0 16px 48px rgba(0, 0, 0, 0.2);
118 | --tactile-edge: 0 3px 0 rgba(0, 0, 0, 0.12); /* .btn3d bottom edge */
119 |
120 | /* === CHROME DIMENSIONS (prototype laptop layout) === */
121 | --sidebar-w: 264px;
122 | --rail-w: 328px;
123 | --header-h: 56px;
124 | --feed-max: 1160px;
125 | --feed-max-wide: 1320px;
126 |
127 | /* === APP SHELL (P1.0 – shared responsive layout contract) ===
128 |    These are deliberately separate from the raw prototype measurements above:
129 |    the application shell uses one consistent grid across every role, while
130 |    individual prototype cards can still use their captured dimensions. */
131 | --app-header-h: 56px;
132 | --app-sidebar-w: 16.5rem;
133 | --app-rail-w: 20rem;
134 | /* The prototype feed grows with the viewport. 48rem was only correct around
135 |    a 1440px three-column viewport and left a large void on wide screens. */
136 | --app-content-max: 82.5rem;
137 | --app-gutter: 1.5rem;
138 | --app-gutter-mobile: 1rem;
139 |
140 | /* === DOMAIN: BLOOM'S TAXONOMY (design-system steering) === */
141 | --bloom-remember: #a855f7; /* purple-500 */
142 | --bloom-understand: #3b82f6; /* blue-500 */
143 | --bloom-apply: #22c55e; /* green-500 */
144 | --bloom-analyze: #eab308; /* yellow-500 */
145 | --bloom-evaluate: #f97316; /* orange-500 */
146 | --bloom-create: #ef4444; /* red-500 */
147 |
148 | /* === DOMAIN: OUTCOME TYPES === */
149 | --outcome-ilo: #dc2626; /* red family */
150 | --outcome-plo: #2563eb; /* blue family */
151 | --outcome-clo: #16a34a; /* green family */
152 |
153 | /* === DOMAIN: ATTAINMENT BANDS === */
154 | --attain-excellent: #16a34a; /* >=85% */
155 | --attain-satisfactory: #2563eb; /* 70-84% */
156 | --attain-developing: #ca8a04; /* 50-69% */
157 | --attain-notyet: #dc2626; /* <50% */
158 |
159 | /* === DOMAIN: GAMIFICATION === */
160 | --xp: #f59e0b; /* amber-500 */
161 | --streak-from: #ef4444; /* red-500 */
162 | --streak-to: #f97316; /* orange-500 */
163 | --rank-gold: #facc15;
164 | --rank-silver: #9ca3af;
165 | --rank-bronze: #d97706;
166 | }
167 |
168 | /* -----
169 |    3. DARK MODE – NET-NEW (not in prototype). Parity-oriented defaults; must pass
170 |    a dark-mode design review before cutover. Uses the existing `.dark` class.
171 |    ----- */
172 | .dark {
173 |   --surface-bg: #0f172a;
174 |   --surface-card: #1e293b;
175 |   --surface-subtle: #1e293b;
176 |   --surface-muted: #334155;
177 |   --border-subtle: rgba(255, 255, 255, 0.1);
178 |   --border-hair: rgba(255, 255, 255, 0.06);
179 |   --shadow-card: 0 1px 2px rgba(0, 0, 0, 0.4), 0 4px 12px rgba(0, 0, 0, 0.5);
180 |   --shadow-card-hover: 0 8px 24px rgba(0, 0, 0, 0.6);
181 |   /* Brand + gradients keep their hues in dark; components choose tint/opacity. */
182 | }
183 |
184 | /* -----
185 |    4. PER-ROLE ACCENT SCOPES (optional identity, layered ON TOP of shared tokens)
186 |    Change a role's flavor by editing ONLY its scope – never fork the system.
187 |    Applied via a wrapper attribute, e.g. <div data-role="student">.
188 |    Defaults below = no divergence (brand primary). Tune per design review.
189 |    ----- */
190 | [data-role="student"] {
191 |   --role-accent: var(--brand-teal);
192 | }
193 | [data-role="teacher"] {
194 |   --role-accent: var(--brand-primary);
195 | }
196 | [data-role="coordinator"] {
197 |   --role-accent: #7c3aed; /* placeholder identity – confirm in design review */
198 | }
199 | [data-role="parent"] {
200 |   --role-accent: #0d9488;
201 | }
202 | [data-role="admin"] {
203 |   --role-accent: var(--slate-700);
204 | }
205 |

```

# FILE: src/features/admin/dashboard/AdminDashboardRail.tsx

TYPE: .tsx | LINES: 1-183 | SHA256: c05a653920052be373a388ec41cc947230c348071fc57d4e395663e3446551ec

```
1  | // =====
2  | // AdminDashboardRail – the admin dashboard's right rail (prototype `railHTML`)
3  | // admin case in shared.js). Fixed, laptop-only (xl+):
4  | //   Institution · AI governance · Departments.
5  | //
6  | // Wired to the REAL hooks the admin dashboard already uses (cache hits; no faked
7  | // data R17; no backend change G.1):
8  | //   - useAdminDashboardAggregate !' users / active / programs / courses
9  | //   - useDepartmentAnalytics      !' departments by mastery
10 | //
11 | // FLAGGED GAPS (prototype shows them; no admin-scope hook – adapted or omitted):
12 | //   - "Weekly active %" and "Retention risk" counts have no source, so the
13 | //   - Institution card shows real totals (users / active / programs / courses).
14 | //   - "AI governance" auto-action counts have no source and there is no
15 | //   - /admin/governance route; the card states the real A2 autonomy ceiling as
16 | //   - static policy chrome (matching the dashboard footer), with no fake counts.
17 | // =====
18 |
19 | import { useMemo } from "react";
20 | import { useNavigate } from "react-router-dom";
21 | import { useTranslation } from "react-i18next";
22 |
23 | import { RailCard, RailHead, RailRow, Shimmer } from "@design-system";
24 | import { Button } from "@components/ui/button";
25 | import WhyThisPopover from "@components/shared/WhyThisPopover";
26 | import { cn } from "@lib/Utils";
27 | import { useAuth } from "@hooks/useAuth";
28 | import { useAdminDashboardAggregate } from "@hooks/useAdminDashboardAggregate";
29 | import { useDepartmentAnalytics } from "@hooks/useAdminDashboard";
30 | import { attainmentValueClass } from "@lib/attainmentTone";
31 | import { formatNumber } from "@lib/formatNumber";
32 |
33 | const RailLink = ({ to, label }: { to: string; label: string }) => {
34 |   const navigate = useNavigate();
35 |   return (
36 |     <Button
37 |       type="button"
38 |       variant="link"
39 |       size="sm"
40 |       onClick={() => navigate(to)}
41 |       className="mt-2 h-auto px-0 text-xs font-extrabold text-blue-600"
42 |     >
43 |       {label}
44 |     </Button>
45 |   );
46 | };
47 |
48 | const AdminDashboardRail = () => {
49 |   const { t } = useTranslation("admin");
50 |   const { institutionId } = useAuth();
51 |
52 |   const aggregate = useAdminDashboardAggregate(institutionId);
53 |   const departmentsQuery = useDepartmentAnalytics();
54 |
55 |   const kpis = aggregate.data;
56 |   const departments = useMemo(
57 |     () =>
58 |       (departmentsQuery.data ?? []).
59 |         .filter((d) => d.avg_plo_attainment > 0)
60 |         .sort((a, b) => b.avg_plo_attainment - a.avg_plo_attainment)
61 |         .slice(0, 3),
62 |     [departmentsQuery.data]
63 |   );
64 |
65 |   return (
66 |     <aside
67 |       aria-label={t("dashboard.rail.label", "Institution")}
68 |       className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto border-s border-border bg-white
69 |     >
70 |       { /* % % Institution (real totals) % % */ }
71 |       <RailCard>
72 |         <RailHead title={t("dashboard.rail.institution", "ðŸ’Š Institution")} />
73 |         <WhyThisPopover
74 |           title={t("dashboard.rail.institution", "ðŸ’Š Institution")}
75 |           reasons={[
76 |             t("common:header.whySignals.admin", {
77 |               users: formatNumber(kpis?.totalUsers ?? 0),
78 |               departments: departments.length,
79 |             }),
80 |           ]}
81 |         />
82 |         {aggregate.isPending ? (
83 |           <Shimmer className="h-20 rounded-lg" />
84 |         ) : (
85 |           <>
86 |             <RailRow>
87 |               <span className="min-w-0 flex-1">
88 |                 {t("dashboard.rail.users", "Users")}
89 |               </span>
90 |               <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
91 |                 {formatNumber(kpis?.totalUsers ?? 0)}
92 |               </b>
93 |             </RailRow>
94 |             <RailRow>
95 |               <span className="min-w-0 flex-1">
96 |                 {t("dashboard.rail.active", "Active accounts")}
97 |               </span>
98 |               <b className="text-[12px] font-extrabold text-green-600">
99 |                 {formatNumber(kpis?.activeUsers ?? 0)}
100 |               </b>
101 |             </RailRow>
```

```

102 |         <RailRow>
103 |             <span className="min-w-0 flex-1">
104 |                 {t("dashboard.rail.programs", "Programs")}
105 |             </span>
106 |             <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
107 |                 {formatNumber(kpis?.totalPrograms ?? 0)}
108 |             </b>
109 |         </RailRow>
110 |         <RailRow>
111 |             <span className="min-w-0 flex-1">
112 |                 {t("dashboard.rail.courses", "Courses")}
113 |             </span>
114 |             <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
115 |                 {formatNumber(kpis?.totalCourses ?? 0)}
116 |             </b>
117 |         </RailRow>
118 |         <RailLink>
119 |             to="/admin/reports"
120 |             label={t("dashboard.rail.analytics", "See analytics !")}
121 |         </>
122 |     </>
123 | }
124 | </RailCard>
125 |
126 | {/} % % AI governance (real A2 policy – no fabricated counts) % % */}
127 | <RailCard>
128 |     <RailHead title={t("dashboard.rail.governance", "ðŸ’Œ AI governance")} />
129 |     <RailRow>
130 |         <span className="min-w-0 flex-1">
131 |             {t("dashboard.rail.autonomyCeiling", "Autonomy ceiling")}
132 |         </span>
133 |         <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
134 |             {t("dashboard.rail.a2", "A2 · approve")}
135 |         </b>
136 |     </RailRow>
137 |     <p className="mt-1 text-[11px] text-slate-500">
138 |         {t(
139 |             "dashboard.rail.governanceNote",
140 |             "AI never acts on institution data without human approval."
141 |         )}
142 |     </p>
143 | </RailCard>
144 |
145 | {/} % % Departments by mastery (real) % % */}
146 | <RailCard>
147 |     <RailHead title={t("dashboard.rail.departments", "Departments")} />
148 |     {departmentsQuery.isPending ? (
149 |         <Shimmer className="h-16 rounded-lg" />
150 |     ) : departments.length > 0 ? (
151 |         <div className="space-y-0.5">
152 |             {departments.map((d) => (
153 |                 <RailRow key={d.department_id}>
154 |                     <span className="min-w-0 flex-1 truncate">
155 |                         {d.department_name}
156 |                     </span>
157 |                     <b
158 |                         className={cn(
159 |                             "text-[12px] font-extrabold",
160 |                             attainmentValueClass(d.avg_plo_attainment)
161 |                         )}
162 |                         >
163 |                             {d.avg_plo_attainment}%
164 |                     </b>
165 |                 </RailRow>
166 |             ))}
167 |             <RailLink>
168 |                 to="/admin/outcomes"
169 |                 label={t("dashboard.rail.allDepartments", "All departments !")}
170 |             </>
171 |         </div>
172 |     ) : (
173 |         <p className="text-xs text-slate-500">
174 |             {t("dashboard.rail.departmentsEmpty", "No attainment data yet.")}
175 |         </p>
176 |     )}
177 | </RailCard>
178 | </aside>
179 | );
180 | };
181 |
182 | export default AdminDashboardRail;
183 |

```

FILE: [src/features/admin/dashboard/AdminDashboardScreen.tsx](#)

TYPE: .tsx | LINES: 1-441 | SHA256: a7c0f0f5193f6b55a5dbc79d65c09d5aef16579c6781399f1422152a0058a5f8

```

1 | // =====
2 | // AdminDashboardScreen – prototype rebuild (prototype-frontend-rebuild P2.5)
3 | // =====
4 | //
5 | // Rebuilds `prototype/admin-dashboard.html` on `@/design-system` + tokens as a
6 | // SINGLE-COLUMN institution feed, wired to the REAL existing hooks only (no
7 | // faked data R17, no backend changes G.1):
8 | // - useAdminDashboardAggregate !' AdminKPIData (users / active / programs /
9 | //   courses / usersByRole)
10 | // - useDepartmentAnalytics !' "Departments by mastery" (real per-dept
11 | //   avg PLO attainment) + institution avg mastery
12 | //
13 | // DEFERRED / FLAGGED GAPS (prototype shows them; no admin-scope hook – adapted
14 | // to real signals or omitted, never fabricated):
15 | // - Hero carousel slides !' single primary slide (greeting + real chips).
16 | // - "Weekly active" login % and "Retention risk" count !' no such metric in
17 | //   AdminKPIData; the KPI row shows real Users / Active accounts / Avg mastery
18 | //   / Courses instead.

```

```

19 // - "Executive insight" is an AI-generated narrative in the mock !' replaced by
20 // a FACTUAL real-data institution summary (no AI-generated claim).
21 // - "AI Governance" metrics (214 suggestions/wk, 0 unapproved) have no hook
22 // and no `/admin/governance` route !' replaced by a real "Users by role"
23 // card + a static A2 autonomy footer (policy chrome, matching teacher/parent).
24 // =====
25
26 import { useMemo } from "react";
27 import { useNavigate } from "react-router-dom";
28 import { useTranslation } from "react-i18next";
29 import {
30   ArrowRight,
31   BookOpen,
32   Building2,
33   GraduationCap,
34   Lightbulb,
35   School,
36   ShieldCheck,
37   TrendingUp,
38   UserCheck,
39   Users,
40 } from "lucide-react";
41
42 import {
43   Badge,
44   Button,
45   HeroCarousel,
46   KPICard,
47   SectionHeader,
48   Shimmer,
49 } from "@design-system";
50 import { useAuth } from "@hooks/useAuth";
51 import { useAdminDashboardAggregate } from "@hooks/useAdminDashboardAggregate";
52 import { useDepartmentAnalytics } from "@hooks/useAdminDashboard";
53 import { attainmentValueClass } from "@lib/attainmentTone";
54 import { formatNumber } from "@lib/formatNumber";
55 import { cn } from "@lib/utills";
56
57 const BRAND_GRADIENT = "var(--brand-gradient)";
58 const HERO_GRADIENT = "var(--hero-gradient)";
59
60 /** Prototype `pcard` surface. */
61 const CARD =
62   "rounded-[20px] border border-[#eef2f6] bg-white shadow-[0_1px_2px_rgba(16,24,40,0.04),0_10px_26px_rgba(16,2
63
64 /** Outcome-type role badge colors (design-system domain coding). */
65 const ROLE_BADGE: Record<string, string> = {
66   admin: "bg-red-100 text-red-700 border-red-200",
67   coordinator: "bg-blue-100 text-blue-700 border-blue-200",
68   teacher: "bg-green-100 text-green-700 border-green-200",
69   student: "bg-amber-100 text-amber-700 border-amber-200",
70   parent: "bg-purple-100 text-purple-700 border-purple-200",
71 };
72 const ROLE_ORDER = ["admin", "coordinator", "teacher", "student", "parent"];
73
74 const AdminDashboardScreen = () => {
75   const { t } = useTranslation("admin");
76   const navigate = useNavigate();
77   const { institutionId } = useAuth();
78
79   const aggregate = useAdminDashboardAggregate(institutionId);
80   const kpis = aggregate.data;
81   const totalUsers = kpis?.totalUsers ?? 0;
82   const activeUsers = kpis?.activeUsers ?? 0;
83   const totalCourses = kpis?.totalCourses ?? 0;
84   const totalPrograms = kpis?.totalPrograms ?? 0;
85
86   const roleEntries = useMemo(() => {
87     const byRole = kpis?.usersByRole ?? {};
88     return Object.entries(byRole).sort(
89       (a, b) =>
90         (ROLE_ORDER.indexOf(a[0]) + 1 || 99) -
91         (ROLE_ORDER.indexOf(b[0]) + 1 || 99)
92     );
93   }, [kpis]);
94
95   const departmentsQuery = useDepartmentAnalytics();
96   const departments = useMemo(
97     () =>
98       (departmentsQuery.data ?? []).
99         .slice()
100         .sort((a, b) => b.avg_plo_attainment - a.avg_plo_attainment),
101     [departmentsQuery.data]
102   );
103   const measuredDepts = useMemo(
104     () => departments.filter((d) => d.avg_plo_attainment > 0),
105     [departments]
106   );
107   const avgMastery = useMemo(
108     () =>
109       measuredDepts.length > 0
110       ? Math.round(
111         measuredDepts.reduce((s, d) => s + d.avg_plo_attainment, 0) /
112         measuredDepts.length
113       )
114       : 0,
115     [measuredDepts]
116   );
117   const lowestDept =
118     measuredDepts.length > 0 ? measuredDepts[measuredDepts.length - 1] : null;
119
120   const kpiLoading = aggregate.isPending;
121
122   return (

```

```

123 <div className="w-full space-y-4">
124   { /* % % Institution carousel (overview + real executive watch item) % % */
125   <HeroCarousel
126     ariaLabel={t("dashboard.hero.carouselLabel", "Institution highlights")}
127     className="rounded-2xl text-white shadow-lg"
128     style={{ background: HERO_GRADIENT }}
129     slides={[
130       <div key="overview" className="min-h-[126px] p-5">
131         <div className="flex items-center gap-3">
132           <div className="flex h-11 w-11 shrink-0 items-center justify-center rounded-2xl border border-wh
133             <Building2 className="h-6 w-6" aria-hidden="true" />
134           </div>
135           <div className="min-w-0">
136             <h1 className="text-lg font-bold tracking-tight">
137               {t("dashboard.hero.title", "Your institution, this week")}
138             </h1>
139             <p className="text-[12px] text-white/75">
140               {t(
141                 "dashboard.hero.subtitle",
142                 "De-identified, institution-wide signals – no individual student data."
143               )}
144             </p>
145           </div>
146         </div>
147         <div className="mt-3 flex flex-wrap gap-2">
148           <Button
149             type="button"
150             variant="ghost"
151             onClick={() => navigate("/admin/users")}
152             className="h-auto rounded-full border border-white/20 bg-white/15 px-3 py-1.5 text-[12px] font
153           >
154             <Users className="h-3.5 w-3.5" aria-hidden="true" />
155             {t("dashboard.hero.users", {
156               defaultValue: "{{n}} users",
157               n: formatNumber(totalUsers),
158             })}
159           </Button>
160           <Button
161             type="button"
162             variant="ghost"
163             onClick={() => navigate("/admin/reports")}
164             className="h-auto rounded-full border border-white/20 bg-white/15 px-3 py-1.5 text-[12px] font
165           >
166             <UserCheck className="h-3.5 w-3.5" aria-hidden="true" />
167             {t("dashboard.hero.active", {
168               defaultValue: "{{n}} active",
169               n: formatNumber(activeUsers),
170             })}
171           </Button>
172           <Button
173             type="button"
174             variant="ghost"
175             onClick={() => navigate("/admin/programs")}
176             className="h-auto rounded-full border border-white/20 bg-white/15 px-3 py-1.5 text-[12px] font
177           >
178             <BookOpen className="h-3.5 w-3.5" aria-hidden="true" />
179             {t("dashboard.hero.programs", {
180               defaultValue: "{{n}} programs",
181               n: formatNumber(totalPrograms),
182             })}
183           </Button>
184         </div>
185       </div>,
186       ...(lowestDept
187         ? [
188           <div
189             key="watch-item"
190             className="flex min-h-[126px] items-center gap-4 p-5"
191           >
192             <div className="flex h-12 w-12 shrink-0 items-center justify-center rounded-2xl border borde
193               <Lightbulb className="h-6 w-6" aria-hidden="true" />
194             </div>
195             <div className="min-w-0 flex-1">
196               <p className="text-[10px] font-black uppercase tracking-widest text-amber-200">
197                 {t("dashboard.hero.watchEyebrow", "Executive watch item")}
198               </p>
199               <h2 className="mt-0.5 truncate text-lg font-bold">
200                 {lowestDept.department_name}
201               </h2>
202               <p className="mt-1 text-[12px] text-white/75">
203                 {t("dashboard.hero.watchBody", {
204                   defaultValue:
205                     "{{percent}}% average PLO attainment · lowest measured department",
206                   percent: Math.round(lowestDept.avg_plo_attainment),
207                 })}
208               </p>
209             </div>
210             <Button
211               type="button"
212               variant="ghost"
213               onClick={() => navigate("/admin/reports")}
214               className="shrink-0 rounded-xl border border-white/20 bg-white/15 px-3 text-xs font-bold t
215             >
216               {t("dashboard.hero.openAnalytics", "Review")}
217             <ArrowRight className="h-3.5 w-3.5" aria-hidden="true" />
218           </Button>
219         </div>,
220       ]
221     : []),
222   ]}
223 </div>
224
225 { /* % % KPI row (real aggregate metrics) % % */
226 {kpiLoading ? (

```

```

227 <div className="grid grid-cols-2 gap-3 md:grid-cols-4">
228   {Array.from({ length: 4 }).map((_, i) => (
229     <Shimmer key={i} className="h-[92px] rounded-[20px]" />
230   ))}
231 </div>
232 ) : (
233   <div className="grid grid-cols-2 gap-3 md:grid-cols-4">
234     <KPICard
235       icon={Users}
236       label={t("dashboard.totalUsers", "Users")}
237       value={formatNumber(totalUsers)}
238     />
239     <KPICard
240       icon={UserCheck}
241       label={t("dashboard.activeUsers", "Active users")}
242       value={formatNumber(activeUsers)}
243       iconBgClass="bg-green-50"
244       iconColorClass="text-green-600"
245     />
246     <KPICard
247       icon={TrendingUp}
248       label={t("dashboard.avgMastery", "Avg mastery")}
249       value={` ${avgMastery}% `}
250       valueClassName={attainmentValueClass(avgMastery)}
251     />
252     <KPICard
253       icon={GraduationCap}
254       label={t("dashboard.courses", "Courses")}
255       value={formatNumber(totalCourses)}
256     />
257   </div>
258 )}
259
260 {/} % % Institution insight (factual real-data summary) % % */}
261 <section className={cn(CARD, "p-5")} id="exec-sec">
262   <SectionHeader
263     icon={Lightbulb}
264     title={t("dashboard.insight.title", "Institution insight")}
265     className="mb-2"
266   />
267   {kpiLoading ? (
268     <Shimmer className="h-16 rounded-lg" />
269   ) : (
270     <div>
271       <p className="text-sm leading-relaxed text-gray-700">
272         {t("dashboard.insight.body", {
273           defaultValue:
274             "Your institution has {{users}} users ({{active}} active) across {{programs}} programs and {
275             users: formatNumber(totalUsers),
276             active: formatNumber(activeUsers),
277             programs: formatNumber(totalPrograms),
278             courses: formatNumber(totalCourses),
279           }})}
280         {lowestDept && (
281           <div>
282             {t("dashboard.insight.lowest", {
283               defaultValue:
284                 "{{dept}} currently has the lowest outcome attainment at {{pct}}% – a candidate for targ
285                 dept: lowestDept.department_name,
286                 pct: lowestDept.avg_plo_attainment,
287               }})}
288             </div>
289           )}
290         </p>
291       <div className="mt-3 flex flex-wrap gap-2">
292         <Button
293           variant="tactile"
294           onClick={() => navigate("/admin/reports")}
295         />
296         {t("dashboard.insight.analytics", "See analytics")}
297         <ArrowRight className="h-4 w-4" aria-hidden="true" />
298       </div>
299       <Button
300         variant="outline"
301         onClick={() => navigate("/admin/outcomes")}
302       />
303       {t("dashboard.insight.outcomes", "Outcomes")}
304     </div>
305   )}
306 </section>
307
308 {/} % % Departments by mastery + Users by role % % */}
309 <div className="grid grid-cols-1 gap-4 lg:grid-cols-2">
310   {/} Departments by mastery (real useDepartmentAnalytics) */}
311   <section className={cn(CARD, "p-4")}>
312     <SectionHeader
313       icon={School}
314       title={t("dashboard.departments.title", "Departments by mastery")}
315       action={
316         <button
317           type="button"
318           onClick={() => navigate("/admin/outcomes")}
319           className="text-xs font-bold text-sky-700 hover:underline"
320         />
321       }
322     />
323     {t("dashboard.departments.all", "All !")}
324   </section>
325   <div className="mb-3">
326     {departmentsQuery.isPending ? (
327       <div className="space-y-3">
328

```

```

331         {Array.from({ length: 3 }).map((_, i) => (
332             <Shimmer key={i} className="h-10 rounded-lg" />
333         ))}
334     </div>
335 ) : measuredDepts.length > 0 ? (
336     <div className="space-y-2.5">
337         {measuredDepts.slice(0, 6).map((d) => (
338             <div key={d.department_id}>
339                 <div className="mb-1 flex items-center justify-between">
340                     <span className="truncate text-sm font-medium text-gray-900">
341                         {d.department_name}
342                     </span>
343                     <b
344                         className={cn(
345                             "text-sm",
346                             attainmentValueClass(d.avg_plo_attainment)
347                         )}
348                     >
349                         {d.avg_plo_attainment}%
350                     </b>
351                 </div>
352                 <div className="h-2 overflow-hidden rounded-full bg-gray-100">
353                     <div
354                         className="h-full rounded-full bg-blue-500"
355                         style={{ width: `${d.avg_plo_attainment}%` }}
356                     />
357                 </div>
358             </div>
359         ))}
360     </div>
361 ) : (
362     <p className="py-6 text-center text-sm text-gray-500">
363         {t(
364             "dashboard.departments.empty",
365             "No department attainment data yet."
366         )}
367     </p>
368 )}
369 </section>
370
371 /* Users by role (real usersByRole breakdown) */
372 <section className={cn(CARD, "p-4")}>
373     <SectionHeader
374         icon={Users}
375         title={t("dashboard.usersByRole", "Users by role")}
376         action={
377             <button
378                 type="button"
379                 onClick={() => navigate("/admin/users")}
380                 className="text-xs font-bold text-sky-700 hover:underline"
381             >
382                 {t("dashboard.manage", "Manage !")}
383             </button>
384         }
385         className="mb-3"
386     />
387     {kpiLoading ? (
388         <div className="space-y-3">
389             {Array.from({ length: 5 }).map((_, i) => (
390                 <Shimmer key={i} className="h-8 rounded-lg" />
391             ))}
392         </div>
393     ) : roleEntries.length > 0 ? (
394         <div className="space-y-2.5">
395             {roleEntries.map(([role, count]) => (
396                 <div
397                     key={role}
398                     className="flex items-center justify-between gap-2"
399                 >
400                     <Badge variant="outline" className={ROLE_BADGE[role] ?? ""}>
401                         {role.charAt(0).toUpperCase() + role.slice(1)}
402                     </Badge>
403                     <span className="text-sm font-semibold text-gray-900">
404                         {formatNumber(count)}
405                     </span>
406                 </div>
407             ))}
408         </div>
409     ) : (
410         <p className="py-6 text-center text-sm text-gray-500">
411             {t("dashboard.noActiveUsers", "No active users yet.")}
412         </p>
413     )}
414 </section>
415 </div>
416
417 /* % % Autonomy footer (static policy chrome - A2) % % */
418 <div className="flex items-center justify-between gap-3 rounded-xl border border-gray-200 bg-white px-4"
419     <p className="flex items-center gap-2 text-xs text-gray-600">
420         <ShieldCheck
421             className="h-4 w-4 shrink-0 text-sky-600"
422             aria-hidden="true"
423         />
424         {t(
425             "dashboard.autonomy.note",
426             "AI autonomy ceiling: A2 – Suggest & approve. AI never acts on institution data without human appr
427         )}
428     </p>
429     <span
430         className="hidden shrink-0 rounded-lg px-2 py-1 text-[10px] font-bold text-white sm:inline"
431         style={{ background: BRAND_GRADIENT }}
432     >
433         {t("dashboard.autonomy.tag", "Governed")}
434     </span>

```



```

435 |         </div>
436 |     </div>
437 |     );
438 | };
439 |
440 | export default AdminDashboardScreen;
441 |

```

#### FILE: src/features/admin/index.ts

TYPE: .ts | LINES: 1-13 | SHA256: 35f31379d831058f2288c751691d18ae6a1422cb4da6eea48a0d1083f2ac215c

```

1 | /**
2 |  * L3 composition – Admin.
3 |  *
4 |  * Role pages, widgets, and flows composed from `@/design-system` primitives +
5 |  * patterns and bound to EXISTING hooks in `src/hooks` (no new backend). Staff
6 |  * treatment: professional/restrained.
7 |  *
8 |  * Populated across P2 (dashboard) and P3 (users/import/invite, structure, ILOs,
9 |  * reports, audit, marketplace mgmt, badges, fees, settings) + P3.6 net-new Admin
10 |  * Security console, per `missing-screens-catalog.md`. Empty until then (task 0.8).
11 |  */
12 | export {};
13 |

```

#### FILE: src/features/admin/security/AdminSecurityPage.tsx

TYPE: .tsx | LINES: 1-220 | SHA256: 425de20d9d3e46d5f6f3bfff24c30245151e57f2373a11a5a001c6791db556b8a

```

1 | // =====
2 | // AdminSecurityPage – security console (net-new screen, P3.6)
3 | // =====
4 | // Built from the prototype design system (`@/design-system`): PageHeader +
5 | // KPICard + SectionCard + StatusDot + StatePanel. Surfaces the login
6 | // rate-limiter's data (blocked IPs, lockouts, rate-limit events) via
7 | // `useAdminSecurity`. Status encoded with semantic dots (PARITY.md §B.4).
8 | // =====
9 |
10 | import { useTranslation } from "react-i18next";
11 | import { Activity, Ban, Lock, ShieldAlert } from "lucide-react";
12 |
13 | import {
14 |   KPICard,
15 |   PageHeader,
16 |   SectionCard,
17 |   StatePanel,
18 |   StatusDot,
19 |   type DotTone,
20 | } from "@/design-system";
21 | import { useAdminSecurity } from "@/hooks/useAdminSecurity";
22 | import {
23 |   isBlockActive,
24 |   loginLockStatus,
25 |   rateLimitSeverity,
26 |   type LockStatus,
27 |   type RateLimitSeverity,
28 | } from "../security";
29 |
30 | const fmt = (iso: string) => new Date(iso).toLocaleString();
31 |
32 | const LOCK_TONE: Record<LockStatus, DotTone> = {
33 |   locked: "danger",
34 |   "at-risk": "warning",
35 |   ok: "success",
36 | };
37 |
38 | const SEVERITY_TONE: Record<RateLimitSeverity, DotTone> = {
39 |   critical: "danger",
40 |   attention: "warning",
41 |   monitor: "info",
42 | };
43 |
44 | const AdminSecurityPage = () => {
45 |   const { t } = useTranslation("common");
46 |   const { data, isLoading, isError } = useAdminSecurity();
47 |
48 |   const header = <PageHeader title={t("security.title", "Security")} />;
49 |
50 |   if (isLoading) {
51 |     return (
52 |       <div className="space-y-6">
53 |         {header}
54 |         <StatePanel variant="loading" />
55 |       </div>
56 |     );
57 |   }
58 |
59 |   if (isError || !data) {
60 |     return (
61 |       <div className="space-y-6">
62 |         {header}
63 |         <StatePanel
64 |           variant="error"
65 |           message={t(
66 |             "security.error",
67 |             "Could not load security data. Please try again."
68 |           )}
69 |         />
70 |       </div>
71 |     );
72 |   }
73 |
74 |   const { blockedIps, lockedAccounts, rateLimitEvents } = data;
75 |   const activeBlocks = blockedIps.filter((b) =>
76 |     isBlockActive(b.blocked_until)

```

```

77 | ).length;
78 | const lockedCount = lockedAccounts.filter(
79 |   (a) => loginLockStatus(a.locked_until, a.attempt_count) === "locked"
80 | ).length;
81 |
82 | return (
83 |   <div className="space-y-6">
84 |     {header}
85 |
86 |     <div className="grid grid-cols-2 gap-4 md:grid-cols-3">
87 |       <KPICard
88 |         icon={Ban}
89 |         label={t("security.activeBlocks", "Active IP blocks")}
90 |         value={activeBlocks}
91 |         iconBgClass="bg-red-50"
92 |         iconColorClass="text-red-600"
93 |         valueClassName={activeBlocks > 0 ? "text-red-600" : "text-sky-700"}
94 |       />
95 |       <KPICard
96 |         icon={Lock}
97 |         label={t("security.lockedAccounts", "Locked accounts")}
98 |         value={lockedCount}
99 |         iconBgClass="bg-amber-50"
100 |         iconColorClass="text-amber-600"
101 |         valueClassName={lockedCount > 0 ? "text-amber-600" : "text-sky-700"}
102 |       />
103 |       <KPICard
104 |         icon={Activity}
105 |         label={t("security.recentEvents", "Recent events")}
106 |         value={rateLimitEvents.length}
107 |       />
108 |     </div>
109 |
110 |     <SectionCard icon={Ban} title={t("security.blockedIps", "Blocked IPs")}>
111 |       {blockedIps.length === 0 ? (
112 |         <p className="py-3 text-sm text-gray-500">
113 |           {t("security.noBlockedIps", "No blocked IPs.")}
114 |         </p>
115 |       ) : (
116 |         <ul className="divide-y divide-slate-100">
117 |           {blockedIps.map((b) => {
118 |             const active = isBlockActive(b.blocked_until);
119 |             return (
120 |               <li key={b.ip_address} className="flex items-center gap-3 py-3">
121 |                 <StatusDot tone={active ? "danger" : "neutral"} />
122 |                 <div className="min-w-0 flex-1">
123 |                   <p className="truncate text-sm font-bold text-gray-900">
124 |                     {b.ip_address}
125 |                   </p>
126 |                   <p className="truncate text-[11px] text-gray-500">
127 |                     {b.reason}
128 |                   </p>
129 |                 </div>
130 |                 <span className="shrink-0 text-xs font-medium text-gray-500">
131 |                   {active
132 |                     ? t("security.blockedUntil", "Until {{when}}", {
133 |                       when: fmt(b.blocked_until),
134 |                     })
135 |                     : t("security.expired", "Expired")}
136 |                 </span>
137 |               </li>
138 |             );
139 |           })}
140 |         </ul>
141 |       )}
142 |     </SectionCard>
143 |
144 |     <SectionCard
145 |       icon={Lock}
146 |       title={t("security.lockedAccountsTitle", "Login lockouts")}
147 |     >
148 |       {lockedAccounts.length === 0 ? (
149 |         <p className="py-3 text-sm text-gray-500">
150 |           {t("security.noLockouts", "No login lockouts.")}
151 |         </p>
152 |       ) : (
153 |         <ul className="divide-y divide-slate-100">
154 |           {lockedAccounts.map((a) => {
155 |             const status = loginLockStatus(a.locked_until, a.attempt_count);
156 |             return (
157 |               <li key={a.email} className="flex items-center gap-3 py-3">
158 |                 <StatusDot tone={LOCK_TONE[status]} />
159 |                 <div className="min-w-0 flex-1">
160 |                   <p className="truncate text-sm font-bold text-gray-900">
161 |                     {a.email}
162 |                   </p>
163 |                   <p className="text-[11px] text-gray-500">
164 |                     {t(
165 |                       "security.failedAttempts",
166 |                       "{{count}} failed attempts",
167 |                       {
168 |                         count: a.attempt_count,
169 |                       }
170 |                     )}
171 |                   </p>
172 |                 </div>
173 |                 <span className="shrink-0 text-xs font-semibold uppercase tracking-wide text-gray-500">
174 |                   {status}
175 |                 </span>
176 |               </li>
177 |             );
178 |           })}
179 |         </ul>
180 |       )}

```

```

181 |         </SectionCard>
182 |
183 |         <SectionCard
184 |             icon={ShieldAlert}
185 |             title={t("security.recentEventsTitle", "Recent rate-limit events")}
186 |         >
187 |             {rateLimitEvents.length === 0 ? (
188 |                 <p className="py-3 text-sm text-gray-500">
189 |                     {t("security.noEvents", "No recent events.")}
190 |                 </p>
191 |             ) : (
192 |                 <ul className="divide-y divide-slate-100">
193 |                     {rateLimitEvents.map((e) => {
194 |                         const severity = rateLimitSeverity(e.event_type);
195 |                         return (
196 |                             <li key={e.id} className="flex items-center gap-3 py-3">
197 |                                 <StatusDot tone={SEVERITY_TONE[severity]} />
198 |                                 <div className="min-w-0 flex-1">
199 |                                     <p className="truncate text-sm font-bold text-gray-900">
200 |                                         {e.event_type}
201 |                                     </p>
202 |                                     <p className="truncate text-[11px] text-gray-500">
203 |                                         {e.ip_address}
204 |                                     </p>
205 |                                 </div>
206 |                                 <span className="shrink-0 text-xs font-medium text-gray-500">
207 |                                     {fmt(e.occurred_at)}
208 |                                 </span>
209 |                             </li>
210 |                         );
211 |                     })}
212 |                 </ul>
213 |             )}
214 |         </SectionCard>
215 |     </div>
216 | );
217 | };
218 |
219 | export default AdminSecurityPage;
220 |

```

## FILE: src/features/admin/security/security.ts

TYPE: .ts | LINES: 1-62 | SHA256: 32f9cb340053ada05035653eb612c47bd6b583d94d45cc8d0cc9a193644eef66

```

1 | // =====
2 | // Admin Security console – pure classification logic
3 | // =====
4 | //
5 | // Presentation-layer helpers for the security console. Pure (no JSX / no
6 | // Supabase) so they are fully unit- + property-testable. They classify the raw
7 | // rows of `blocked_ips`, `login_attempts`, and `rate_limit_events` into the
8 | // status/severity the UI renders (status dots, §B.4 of PARITY.md).
9 | // =====
10 |
11 | export type RateLimitSeverity = "critical" | "attention" | "monitor";
12 | export type LockStatus = "locked" | "at-risk" | "ok";
13 |
14 | /**
15 |  * Failed-attempt count at or above which an account is surfaced as "at-risk"
16 |  * (a display heuristic for the console; the authoritative lockout decision is
17 |  * the `check-login-rate` edge function, not this UI).
18 |  */
19 | export const AT_RISK_ATTEMPTS = 3;
20 |
21 | /** True while a block is still in effect (its `blocked_until` is in the future). */
22 | export function isBlockActive(
23 |     blockedUntil: string,
24 |     now: number = Date.now()
25 | ): boolean {
26 |     const until = new Date(blockedUntil).getTime();
27 |     return Number.isFinite(until) && until > now;
28 | }
29 |
30 | /**
31 |  * Classify a login-attempt row: `locked` while `locked_until` is in the future,
32 |  * else `at-risk` once repeated failures reach the threshold, else `ok`.
33 |  */
34 | export function loginLockStatus(
35 |     lockedUntil: string | null,
36 |     attemptCount: number,
37 |     now: number = Date.now()
38 | ): LockStatus {
39 |     if (lockedUntil) {
40 |         const until = new Date(lockedUntil).getTime();
41 |         if (Number.isFinite(until) && until > now) return "locked";
42 |     }
43 |     return attemptCount >= AT_RISK_ATTEMPTS ? "at-risk" : "ok";
44 | }
45 |
46 | /**
47 |  * Map a rate-limit `event_type` to a display severity. Keyword-based so it
48 |  * degrades gracefully for event types the UI hasn't seen before (!' `monitor`).
49 |  */
50 | export function rateLimitSeverity(eventType: string): RateLimitSeverity {
51 |     const type = eventType.toLowerCase();
52 |     if (type.includes("block") || type.includes("lock")) return "critical";
53 |     if (
54 |         type.includes("limit") ||
55 |         type.includes("throttle") ||
56 |         type.includes("captcha")
57 |     ) {
58 |         return "attention";
59 |     }
60 |     return "monitor";

```

```
61 | }
62 |
```

FILE: src/features/coordinator/dashboard/CoordinatorDashboardRail.tsx

TYPE: .tsx | LINES: 1-179 | SHA256: b978dff1e0db69df007126193af48e6d96d955950def3fc33b51e69e649ac368

```
1  // =====
2  // CoordinatorDashboardRail – the coordinator dashboard's right rail (prototype
3  // `railHTML()` coordinator case in shared.js). Fixed, laptop-only (x!+):
4  // Attainment alerts · Curriculum gap · Accreditation.
5  //
6  // Wired to the REAL hooks the coordinator dashboard already uses (cache hits; no
7  // faked data R17; no backend change G.1):
8  // - useCoordinatorOutcomeAttainment !' below-target PLO alerts
9  // - useCoordinatorDashboardAggregate !' CLO-coverage % (curriculum gap)
10 // - useCoordinatorAccreditationReadiness !' readiness %
11 //
12 // Faithful to the dashboard's "no invented gap" rule: the prototype's specific
13 // "Concurrency has no mapped assessment" line has no backend, so the curriculum
14 // card shows the REAL coverage % + a link to the matrix instead of naming a gap.
15 // =====
16
17 import { useMemo } from "react";
18 import { useNavigate } from "react-router-dom";
19 import { useTranslation } from "react-i18next";
20
21 import { RailCard, RailHead, RailRow, Shimmer } from "@design-system";
22 import { Button } from "@components/ui/button";
23 import WhyThisPopover from "@components/shared/WhyThisPopover";
24 import { cn } from "@lib/utlis";
25 import { useAuth } from "@hooks/useAuth";
26 import { useCoordinatorDashboardAggregate } from "@hooks/useCoordinatorDashboardAggregate";
27 import { useCoordinatorOutcomeAttainment } from "@hooks/useCoordinatorOutcomeAttainment";
28 import { useCoordinatorAccreditationReadiness } from "@hooks/useCoordinatorAccreditation";
29
30 const RailLink = ({ to, label }: { to: string; label: string }) => {
31   const navigate = useNavigate();
32   return (
33     <Button
34       type="button"
35       variant="link"
36       size="sm"
37       onClick={() => navigate(to)}
38       className="mt-2 h-auto px-0 text-xs font-extrabold text-blue-600"
39     >
40       {label}
41     </Button>
42   );
43 };
44
45 const CoordinatorDashboardRail = () => {
46   const { t } = useTranslation("coordinator");
47   const { institutionId } = useAuth();
48
49   const aggregate = useCoordinatorDashboardAggregate(institutionId);
50   const attainment = useCoordinatorOutcomeAttainment(institutionId);
51   const accred = useCoordinatorAccreditationReadiness(institutionId);
52
53   const threshold = attainment.data?.successThreshold ?? 70;
54   const belowTarget = useMemo(
55     () =>
56       (attainment.data?.plos ?? []).
57         .filter(
58           (p) => p.attainment != null && (p.attainment as number) < threshold
59         )
60         .sort((a, b) => (a.attainment as number) - (b.attainment as number))
61         .slice(0, 3),
62     [attainment.data, threshold]
63   );
64
65   const cloCoverage = aggregate.data?.cloCoveragePercent ?? 0;
66   const readiness = accred.data?.readinessPercent ?? null;
67
68   return (
69     <aside
70       aria-label={t("dashboard.rail.label", "Program alerts")}
71       className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto border-s border-border bg-whit
72     >
73       {/ * % % Attainment alerts (real below-target PLOs) % % */}
74       <RailCard>
75         <RailHead
76           title={t("dashboard.rail.alerts", "0=ÜÉ Attainment alerts")}
77           right={
78             belowTarget.length > 0 ? String(belowTarget.length) : undefined
79           }
80         >
81           <WhyThisPopover
82             title={t("dashboard.rail.alerts", "0=ÜÉ Attainment alerts")}
83             reasons={[
84               t("common:header.whySignals.coordinator", {
85                 belowTarget: belowTarget.length,
86               }),
87             ]}
88           />
89           {attainment.isPending ? (
90             <Shimmer className="h-16 rounded-lg" />
91           ) : belowTarget.length > 0 ? (
92             <div className="space-y-0.5">
93               {belowTarget.map((p) => (
94                 <RailRow key={p.id}>
95                   <span
96                     className={cn(
97                       "h-2 w-2 shrink-0 rounded-full",
98                       p.status === "belowTarget" ? "bg-red-500" : "bg-amber-500"
```

```

99 |         }}
100 |         aria-hidden="true"
101 |       />
102 |       <span className="min-w-0 flex-1 truncate">{p.title}</span>
103 |       <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
104 |         {p.attainment as number}%
105 |       </b>
106 |     </RailRow>
107 |   ))}
108 |   <RailLink
109 |     to="/coordinator/plos"
110 |     label={t("dashboard.rail.allOutcomes", "All outcomes !")}
111 |   />
112 | </div>
113 | ) : (
114 |   <p className="text-xs text-slate-500">
115 |     {t("dashboard.rail.alertsEmpty", "Every PLO is meeting target.")}
116 |   </p>
117 | )}
118 | </RailCard>
119 |
120 | {/ * % Curriculum gap (real CLO coverage %) % % */}
121 | <RailCard>
122 |   <RailHead
123 |     title={t("dashboard.rail.curriculumGap", "Ø=ÿÂp Curriculum gap")}
124 |   />
125 |   {aggregate.isPending ? (
126 |     <Shimmer className="h-12 rounded-lg" />
127 |   ) : (
128 |     <>
129 |       <RailRow>
130 |         <span className="min-w-0 flex-1">
131 |           {t("dashboard.rail.coverage", "PLO coverage")}
132 |         </span>
133 |         <b
134 |           className={cn(
135 |             "text-[12px] font-extrabold",
136 |             cloCoverage >= 70 ? "text-green-600" : "text-amber-700"
137 |           )}
138 |         >
139 |           {cloCoverage}%
140 |         </b>
141 |       </RailRow>
142 |       <RailLink
143 |         to="/coordinator/matrix"
144 |         label={t("dashboard.rail.openMatrix", "Open matrix !")}
145 |       />
146 |     </>
147 |   )}
148 | </RailCard>
149 |
150 | {/ * % Accreditation (real readiness %) % % */}
151 | <RailCard>
152 |   <RailHead
153 |     title={t("dashboard.rail.accreditation", "Ø=ÛÉ Accreditation")}
154 |   />
155 |   {accred.isPending ? (
156 |     <Shimmer className="h-12 rounded-lg" />
157 |   ) : (
158 |     <>
159 |       <RailRow>
160 |         <span className="min-w-0 flex-1">
161 |           {t("dashboard.rail.evidenceReadiness", "Evidence readiness")}
162 |         </span>
163 |         <b className="text-[12px] font-extrabold text-green-600">
164 |           {readiness != null ? `${readiness}%` : "-"}
165 |         </b>
166 |       </RailRow>
167 |       <RailLink
168 |         to="/coordinator/accreditation"
169 |         label={t("dashboard.rail.reviewDraft", "Review draft !")}
170 |       />
171 |     </>
172 |   )}
173 | </RailCard>
174 | </aside>
175 | );
176 | };
177 |
178 | export default CoordinatorDashboardRail;
179 |

```

FILE: [src/features/coordinator/dashboard/CoordinatorDashboardScreen.tsx](#)

TYPE: .tsx | LINES: 1-854 | SHA256: 419da9cdee70f9873a8e6b6f146075dd6cd517079f0f8c6ca0166a562ef9c85d

```

1 | // =====
2 | // CoordinatorDashboardScreen – prototype rebuild (prototype-frontend-rebuild P2.4)
3 | // =====
4 | //
5 | // Rebuilds `prototype/coordinator-dashboard.html` on `@/design-system` + tokens
6 | // as a SINGLE-COLUMN "program health" feed (the prototype has no insight rail –
7 | // the previous CoordinatorDashboardNew added one; that divergence is removed).
8 | // Wired to the REAL existing hooks only (no faked data R17, no backend G.1):
9 | // - useCoordinatorDashboardAggregate !' avg PLO attainment (+ CLO coverage)
10 | // - usePrograms !' Programs KPI
11 | // - useCoordinatorOutcomeAttainment !' Below-target KPI + attainment alerts
12 | // (trend/affected/root-cause from real
13 | // weakestCourse + affectedStudents)
14 | // - useCoordinatorAiInsights !' optional AI narrative (null-safe)
15 | // - useCoordinatorAccreditationReadiness !' readiness % + REAL evidence `pack`
16 | // - useCQIPPlans !' "Close the loop (CQI)" timeline
17 | // - useAcademicCalendarEvents !' "Program timeline"
18 | //
19 | // DEFERRED / FLAGGED GAPS (prototype shows them; no backend source – not faked):

```

```

20 // - Hero momentum/PL0-drop/accreditation carousel slides !' single primary
21 // slide (greeting + real action chips) like the other rebuilt dashboards.
22 // - "Recovery pathways" is a PROTOTYPE-ONLY concept (the prototype itself tags
23 // it "Concept"; no coordinator recovery backend exists - useMasteryRecovery
24 // is student-scoped). Omitted rather than fabricated (R17).
25 // - Curriculum-gap names a specific missing outcome in the mock; here it shows
26 // the REAL CLO-coverage % + links to the matrix (no invented gap).
27 // - Accreditation evidence checklist renders the REAL 'pack' when the readiness
28 // RPC is deployed; otherwise a neutral "-" (no hardcoded ticks).
29 // =====
30
31 import { useMemo, useState, type ReactNode } from "react";
32 import { Link } from "react-router-dom";
33 import { useTranslation } from "react-i18next";
34 import {
35   ArrowRight,
36   CheckCircle2,
37   Circle,
38   Clock,
39   Compass,
40   Grid3X3,
41   LayoutGrid,
42   RefreshCw,
43   ShieldCheck,
44   Sparkles,
45   TrendingDown,
46 } from "lucide-react";
47 import type { LucideIcon } from "lucide-react";
48
49 import { Button, HeroCarousel, SectionHeader, Shimmer } from "@design-system";
50 import { useAuth } from "@hooks/useAuth";
51 import { useCoordinatorDashboardAggregate } from "@hooks/useCoordinatorDashboardAggregate";
52 import { usePrograms } from "@hooks/usePrograms";
53 import {
54   useCoordinatorOutcomeAttainment,
55   type AttainmentPLO,
56 } from "@hooks/useCoordinatorOutcomeAttainment";
57 import { useCoordinatorAiInsights } from "@hooks/useCoordinatorAiInsights";
58 import { useCoordinatorAccreditationReadiness } from "@hooks/useCoordinatorAccreditation";
59 import { useCQIPPlans, type CQIPPlanStatus } from "@hooks/useCQIPPlans";
60 import {
61   useAcademicCalendarEvents,
62   type AcademicCalendarEvent,
63 } from "@hooks/useAcademicCalendar";
64 import { attainmentValueClass } from "@lib/attainmentTone";
65 import { cn } from "@lib/utils";
66
67 const BRAND_GRADIENT = "var(--brand-gradient)";
68 const HERO_GRADIENT = "var(--hero-gradient)";
69
70 /** Prototype `.pcard` surface (20px radius, hairline, two-layer depth). */
71 const CARD =
72   "rounded-[20px] border border-[#eef2f6] bg-white shadow-[0_1px_2px_rgba(16,24,40,0.04),0_10px_26px_rgba(16,24,40,0.04)]";
73
74 type Tone = "red" | "amber" | "green" | "blue" | "slate";
75
76 const DOT: Record<Tone, string> = {
77   red: "bg-red-500",
78   amber: "bg-amber-500",
79   green: "bg-green-500",
80   blue: "bg-blue-500",
81   slate: "bg-slate-300",
82 };
83
84 const PILL: Record<Tone, string> = {
85   red: "bg-red-50 text-red-700 border-red-100",
86   amber: "bg-amber-50 text-amber-700 border-amber-100",
87   green: "bg-green-50 text-green-700 border-green-100",
88   blue: "bg-blue-50 text-blue-700 border-blue-100",
89   slate: "bg-slate-100 text-slate-600 border-slate-200",
90 };
91
92 const CQI_TONE: Record<CQIPPlanStatus, Tone> = {
93   completed: "green",
94   in_progress: "amber",
95   planned: "slate",
96   evaluated: "blue",
97 };
98
99 /** Locale-aware short date (ISO timestamp or YYYY-MM-DD). */
100 const fmtDate = (iso: string | null | undefined, lang: string): string => {
101   if (!iso) return "";
102   const d = new Date(iso.length <= 10 ? `${iso}T00:00:00` : iso);
103   if (Number.isNaN(d.getTime())) return iso;
104   return new Intl.DateTimeFormat(lang, {
105     month: "short",
106     day: "numeric",
107   }).format(d);
108 };
109
110 const eventStartMs = (e: AcademicCalendarEvent): number =>
111   new Date(`${e.start_date}T00:00:00`).getTime();
112 const eventEndMs = (e: AcademicCalendarEvent): number =>
113   new Date(`${e.end_date ?? e.start_date}T23:59:59`).getTime();
114
115 interface TimelineEntry {
116   id: string;
117   title: string;
118   subtitle: string;
119   when: string;
120   tone: Tone;
121 }
122
123 /** Prototype `.tl` vertical timeline (line + status dots). */
124 const Timeline = ({ items }: { items: TimelineEntry[] }) => (

```

```

124 | <ol className="relative ms-2 space-y-4 border-s-2 border-slate-200 ps-5">
125 |   {items.map(it) => (
126 |     <li key={it.id} className="relative">
127 |       <span
128 |         className={cn(
129 |           "absolute -start-[27px] top-0.5 h-4 w-4 rounded-full ring-4 ring-white",
130 |           DOT[it.tone]
131 |         )}
132 |         aria-hidden="true"
133 |       />
134 |       <p className="text-[13px] font-bold leading-tight text-slate-900">
135 |         {it.title}
136 |         {it.when && (
137 |           <span className="ms-1 text-[10px] font-bold text-slate-400">
138 |             {it.when}
139 |           </span>
140 |         )}
141 |       </p>
142 |       {it.subtitle && (
143 |         <p className="mt-0.5 text-[11px] text-slate-500">{it.subtitle}</p>
144 |       )}
145 |     </li>
146 |   )}
147 | </ol>
148 | );
149 |
150 | /** Hero action chip (real navigation targets only). */
151 | const ActionChip = ({
152 |   icon: Icon,
153 |   label,
154 |   to,
155 |   onClick,
156 | }: {
157 |   icon: LucideIcon;
158 |   label: string;
159 |   to?: string;
160 |   onClick?: () => void;
161 | }) => {
162 |   const cls =
163 |     "inline-flex items-center gap-1.5 rounded-full border border-white/20 bg-white/15 px-3 py-1.5 text-[12px]
164 |   const inner = (
165 |     <
166 |       <Icon className="h-3.5 w-3.5" aria-hidden="true" />
167 |       {label}
168 |       <ArrowRight className="h-3 w-3 opacity-75" aria-hidden="true" />
169 |     </>
170 |   );
171 |   return to ? (
172 |     <Link to={to} className={cls}>
173 |       {inner}
174 |     </Link>
175 |   ) : (
176 |     <button type="button" onClick={onClick} className={cls}>
177 |       {inner}
178 |     </button>
179 |   );
180 | };
181 |
182 | /** KPI-as-filter card (prototype `.kpi-filter`). */
183 | const KpiFilter = ({
184 |   label,
185 |   value,
186 |   valueClass,
187 |   filterLabel,
188 |   to,
189 | }: {
190 |   label: string;
191 |   value: string | number;
192 |   valueClass?: string;
193 |   filterLabel: string;
194 |   to: string;
195 | }) => {
196 |   <Link
197 |     to={to}
198 |     className={cn(
199 |       CARD,
200 |       "block p-3.5 outline-none transition-transform focus-visible:ring-2 focus-visible:ring-sky-300 active:sc
201 |     )}
202 |   >
203 |     <p className="text-[10px] font-black uppercase tracking-widest text-gray-400">
204 |       {label}
205 |     </p>
206 |     <p className={cn("mt-0.5 text-2xl font-black text-gray-900", valueClass)}>
207 |       {value}
208 |     </p>
209 |     <span className="mt-1 flex items-center gap-1 text-[10px] font-bold text-sky-700">
210 |       {filterLabel}
211 |       <ArrowRight className="h-3 w-3" aria-hidden="true" />
212 |     </span>
213 |   </Link>
214 | );
215 |
216 | const CoordinatorDashboardScreen = () => {
217 |   const { t, i18n } = useTranslation("coordinator");
218 |   const { institutionId } = useAuth();
219 |   const lang = i18n.language;
220 |
221 |   const aggregate = useCoordinatorDashboardAggregate(institutionId);
222 |   const avgAttainment = aggregate.data?.avgAttainmentPercent ?? 0;
223 |   const cloCoverage = aggregate.data?.cloCoveragePercent ?? 0;
224 |
225 |   const { data: paginatedPrograms } = usePrograms(undefined, {
226 |     enabled: !!institutionId,
227 |   });

```

```

228 | const programsCount = paginatedPrograms?.data?.length ?? 0;
229 |
230 | const ai = useCoordinatorAiInsights(institutionId);
231 |
232 | const attainment = useCoordinatorOutcomeAttainment(institutionId);
233 | const threshold = attainment.data?.successThreshold ?? 70;
234 | const measuredPlos = useMemo(
235 |   () => (attainment.data?.plos ?? []).filter((p) => p.attainment !== null),
236 |   [attainment.data]
237 | );
238 | const belowTargetPlos = useMemo(
239 |   () =>
240 |     measuredPlos
241 |       .filter((p) => (p.attainment as number) < threshold)
242 |       .sort((a, b) => (a.attainment as number) - (b.attainment as number)),
243 |   [measuredPlos, threshold]
244 | );
245 | const belowTargetCount = belowTargetPlos.length;
246 | const lowestPlo = useMemo(
247 |   () =>
248 |     measuredPlos.reduce<AttainmentPLO | null>((
249 |       (min, p) =>
250 |         min === null || (p.attainment as number) < (min.attainment as number)
251 |         ? p
252 |         : min,
253 |     null),
254 |   ),
255 |   [measuredPlos]
256 | );
257 |
258 | const accred = useCoordinatorAccreditationReadiness(institutionId);
259 | const readiness = accred.data?.readinessPercent ?? null;
260 | const evidencePack = accred.data?.pack ?? [];
261 |
262 | const cqiQuery = useCQIPlans({});
263 | const cqiItems = useMemo<TimelineEntry[]>(
264 |   () =>
265 |     (cqiQuery.data ?? []).slice(0, 4).map((plan) => ({
266 |       id: plan.id,
267 |       title: plan.action_description,
268 |       subtitle: "",
269 |       when: fmtDate(plan.due_date ?? plan.created_at, lang),
270 |       tone: CQI_TONE[plan.status] ?? "slate",
271 |     })),
272 |   [cqiQuery.data, lang]
273 | );
274 |
275 | const [now] = useState(() => Date.now());
276 | const calendarQuery = useAcademicCalendarEvents();
277 | const programItems = useMemo<TimelineEntry[]>(() => {
278 |   const events = calendarQuery.data ?? [];
279 |   const upcoming = events
280 |     .filter((e) => eventEndMs(e) >= now)
281 |     .sort((a, b) => eventStartMs(a) - eventStartMs(b));
282 |   const past = events
283 |     .filter((e) => eventEndMs(e) < now)
284 |     .sort((a, b) => eventStartMs(b) - eventStartMs(a));
285 |   const chosen = [...upcoming, ...past].slice(0, 4);
286 |   const firstFutureId = upcoming.find((e) => eventStartMs(e) > now)?.id;
287 |   return chosen
288 |     .slice()
289 |     .sort((a, b) => eventStartMs(a) - eventStartMs(b))
290 |     .map((e) => {
291 |       const start = eventStartMs(e);
292 |       const end = eventEndMs(e);
293 |       let tone: Tone;
294 |       let statusLabel: string;
295 |       if (end < now) {
296 |         tone = "slate";
297 |         statusLabel = t("dashboard.timeline.statusCompleted", "Completed");
298 |       } else if (start <= now && now <= end) {
299 |         tone = "green";
300 |         statusLabel = t("dashboard.timeline.statusNow", "Now");
301 |       } else if (e.id === firstFutureId) {
302 |         tone = "blue";
303 |         statusLabel = t("dashboard.timeline.statusNext", "Next");
304 |       } else {
305 |         tone = "slate";
306 |         statusLabel = t("dashboard.timeline.statusUpcoming", "Upcoming");
307 |       }
308 |       return {
309 |         id: e.id,
310 |         title: e.title,
311 |         subtitle: statusLabel,
312 |         when: fmtDate(e.start_date, lang),
313 |         tone,
314 |       };
315 |     });
316 | }, [calendarQuery.data, lang, t, now]);
317 |
318 | const alertMeta = (plo: AttainmentPLO): ReactNode => {
319 |   const chips: ReactNode[] = [];
320 |   if (plo.weakestCourse) {
321 |     chips.push(
322 |       <span key="wk" className="text-red-600">
323 |         {t("dashboard.alerts.weakest", {
324 |           defaultValue: "Weakest: {{code}} ({{pct}}%)",
325 |           code: plo.weakestCourse.code,
326 |           pct: plo.weakestCourse.attainment,
327 |         })}
328 |       </span>
329 |     );
330 |   }
331 |   if (plo.affectedStudents > 0) {

```



```

332     chips.push(
333       <span key="af">
334         {t("dashboard.alerts.affected", {
335           defaultValue: "{{count}} affected ({{percent}}%)",
336           count: plo.affectedStudents,
337           percent: plo.cohortPercent,
338         })}
339       </span>
340     );
341   }
342   if (chips.length === 0) return null;
343   return (
344     <p className="mt-1.5 flex flex-wrap gap-x-3 gap-y-1 text-[11px] font-semibold text-gray-500">
345       {chips}
346     </p>
347   );
348 };
349
350 return (
351   <div className="w-full space-y-4">
352     { /* % % Program-health carousel (briefing + real decision context) % % */ }
353     <HeroCarousel
354       ariaLabel={t("dashboard.hub.carouselLabel", "Program highlights")}
355       className="rounded-2xl text-white shadow-lg"
356       style={{ background: HERO_GRADIENT }}
357       slides={[
358         <div key="briefing" className="min-h-[126px] p-5">
359           <div className="flex items-center gap-3">
360             <div className="flex h-11 w-11 shrink-0 items-center justify-center rounded-2xl border border-wh
361             <Compass className="h-6 w-6" aria-hidden="true" />
362           </div>
363           <div className="min-w-0">
364             <h1 className="text-lg font-bold tracking-tight">
365               {t("dashboard.hub.title", "Program health, prepared")}
366             </h1>
367             <p className="text-[12px] text-white/75">
368               {t(
369                 "dashboard.hub.subtitle",
370                 "Outcome attainment across your programs – nothing changes without your review."
371               )}
372             </p>
373           </div>
374         </div>
375         <div className="mt-3 flex flex-wrap gap-2">
376           {belowTargetCount > 0 ? (
377             <ActionChip
378               icon={TrendingDown}
379               label={t("dashboard.hub.ploBelow", {
380                 defaultValue: "Review {{count}} below-target PLO",
381                 count: belowTargetCount,
382               })}
383             onClick={() =>
384               document
385                 .getElementById("alerts-sec")
386                 ?.scrollIntoView({ behavior: "smooth", block: "start" })
387             }
388           >
389             { (
390               <ActionChip
391                 icon={CheckCircle2}
392                 label={t("dashboard.hub.ploTitle", "PLOs on track")}
393                 to="/coordinator/plos"
394               >
395             )}
396             <ActionChip
397               icon={Grid3X3}
398               label={t("dashboard.hub.openMatrix", "Open curriculum matrix")}
399               to="/coordinator/matrix"
400             >
401           </div>
402         </div>,
403         ...{lowestPlo?.attainment !== null
404           ? [
405             <div
406               key="lowest-plo"
407               className="flex min-h-[126px] items-center gap-4 p-5"
408             >
409               <div className="flex h-12 w-12 shrink-0 items-center justify-center rounded-2xl border borde
410               <TrendingDown className="h-6 w-6" aria-hidden="true" />
411             </div>
412             <div className="min-w-0 flex-1">
413               <p className="text-[10px] font-black uppercase tracking-widest text-amber-200">
414                 {t("dashboard.hub.watchEyebrow", "Decision context")}
415               </p>
416               <h2 className="mt-0.5 truncate text-lg font-bold">
417                 {lowestPlo.title}
418               </h2>
419               <p className="mt-1 text-[12px] text-white/75">
420                 {t("dashboard.hub.watchBody", {
421                   defaultValue:
422                     "{{percent}}% attainment · {{affected}} students below target",
423                   percent: lowestPlo.attainment,
424                   affected: lowestPlo.affectedStudents,
425                 })}
426               </p>
427             </div>
428             <ActionChip
429               icon={ArrowRight}
430               label={t("dashboard.hub.reviewPlo", "Review")}
431               to="/coordinator/plos"
432             >
433           </div>,
434         ]
435       : []),

```

```

436 |         ...(readiness != null
437 |         ? [
438 |             <div
439 |                 key="accreditation"
440 |                 className="flex min-h-[126px] items-center gap-4 p-5"
441 |             >
442 |                 <div className="flex h-12 w-12 shrink-0 items-center justify-center rounded-2xl border borde
443 |                 <ShieldCheck className="h-6 w-6" aria-hidden="true" />
444 |                 </div>
445 |                 <div className="min-w-0 flex-1">
446 |                     <p className="text-[10px] font-black uppercase tracking-widest text-green-200">
447 |                         {t(
448 |                             "dashboard.hub.accreditationEyebrow",
449 |                             "Accreditation readiness"
450 |                         )}
451 |                     </p>
452 |                     <h2 className="mt-0.5 text-lg font-bold">{readiness}%</h2>
453 |                     <p className="mt-1 text-[12px] text-white/75">
454 |                         {t("dashboard.hub.accreditationBody", {
455 |                             defaultValue: "{count}" evidence items in the pack",
456 |                             count: evidencePack.length,
457 |                         })}
458 |                     </p>
459 |                 </div>
460 |                 <ActionChip
461 |                     icon={ShieldCheck}
462 |                     label={t("dashboard.hub.openPack", "Open pack")}
463 |                     to="/coordinator/accreditation"
464 |                 />
465 |             </div>,
466 |         ]
467 |         : []),
468 |     ]}
469 | />
470 |
471 | {/ * % % KPI row (each card filters into its view) % % */}
472 | {aggregate.isPending || attainment.isPending ? (
473 |     <div className="grid grid-cols-2 gap-3 md:grid-cols-4">
474 |         {Array.from({ length: 4 }).map((_, i) => (
475 |             <Shimmer key={i} className="h-[92px] rounded-[20px]" />
476 |         ))}
477 |     </div>
478 | ) : (
479 |     <div className="grid grid-cols-2 gap-3 md:grid-cols-4">
480 |         <KpiFilter
481 |             label={t("dashboard.kpi.programs", "Programs")}
482 |             value={programsCount}
483 |             filterLabel={t("dashboard.kpi.manage", "Manage")}
484 |             to="/coordinator/matrix"
485 |         />
486 |         <KpiFilter
487 |             label={t("dashboard.kpi.avgPlo", "Avg PLO")}
488 |             value={avgAttainment}%
489 |             valueClass={attainmentValueClass(avgAttainment)}
490 |             filterLabel={t("dashboard.kpi.allOutcomes", "All outcomes")}
491 |             to="/coordinator/plos"
492 |         />
493 |         <KpiFilter
494 |             label={t("dashboard.kpi.belowTarget", "Below target")}
495 |             value={belowTargetCount}
496 |             valueClass={belowTargetCount > 0 ? "text-amber-600" : undefined}
497 |             filterLabel={t("dashboard.kpi.filterBelow", "Filter below-target")}
498 |             to="/coordinator/gap-analysis"
499 |         />
500 |         <KpiFilter
501 |             label={t("dashboard.kpi.accredReady", "Accred. ready")}
502 |             value={readiness != null ? `{readiness}%` : "-"}
503 |             valueClass="text-green-600"
504 |             filterLabel={t("dashboard.kpi.openPack", "Open pack")}
505 |             to="/coordinator/accreditation"
506 |         />
507 |     </div>
508 | )}
509 |
510 | {/ * % % Attainment alerts (decision context, real below-target PLOs) % % */}
511 | <section id="alerts-sec">
512 |     <SectionHeader
513 |         icon={TrendingDown}
514 |         title={t("dashboard.alerts.title", "Attainment alerts")}
515 |         action={
516 |             <Link
517 |                 to="/coordinator/plos"
518 |                 className="text-xs font-bold text-sky-700 hover:underline"
519 |             >
520 |                 {t("dashboard.alerts.allOutcomes", "All outcomes !")}
521 |             </Link>
522 |         }
523 |         className="mb-3"
524 |     />
525 |     {ai.data?.narrative && (
526 |         <div className="mb-3 flex items-start gap-2 rounded-[20px] border border-teal-100 bg-teal-50/70 p-3">
527 |             <Sparkles
528 |                 className="mt-0.5 h-4 w-4 shrink-0 text-teal-600"
529 |                 aria-hidden="true"
530 |             />
531 |             <p className="text-xs text-gray-700">{ai.data.narrative}</p>
532 |         </div>
533 |     )}
534 |     {attainment.isPending ? (
535 |         <div className="space-y-3">
536 |             <Shimmer className="h-28 rounded-[20px]" />
537 |             <Shimmer className="h-28 rounded-[20px]" />
538 |         </div>
539 |     ) : attainment.isError ? (

```

```

540 | <div className="rounded-[20px] border border-red-100 bg-red-50 p-4 text-sm text-red-700">
541 |   {t("dashboard.alerts.error", "Couldn't load outcome attainment.")}
542 | </div>
543 | ) : belowTargetCount > 0 ? (
544 |   <div className="space-y-3">
545 |     {belowTargetPlos.slice(0, 3).map((plo) => {
546 |       const att = plo.attainment as number;
547 |       const high = plo.status === "belowTarget";
548 |       const tone: Tone = high ? "red" : "amber";
549 |       return (
550 |         <div key={plo.id} className={cn(CARD, "p-4")}>
551 |           <div className="flex items-start gap-3">
552 |             <div
553 |               className={cn(
554 |                 "flex h-9 w-9 shrink-0 items-center justify-center rounded-xl",
555 |                 high
556 |                   ? "bg-red-50 text-red-600"
557 |                   : "bg-amber-50 text-amber-600"
558 |               )}
559 |             >
560 |               <TrendingDown className="h-5 w-5" aria-hidden="true" />
561 |             </div>
562 |             <div className="min-w-0 flex-1">
563 |               <div className="flex flex-wrap items-center gap-2">
564 |                 <p className="text-sm font-bold text-gray-900">
565 |                   {plo.title}
566 |                 </p>
567 |                 <span
568 |                   className={cn(
569 |                     "rounded-full border px-2 py-0.5 text-[11px] font-bold",
570 |                     PILL[tone]
571 |                   )}
572 |                 >
573 |                   {att}%
574 |                 </span>
575 |                 <span
576 |                   className={cn(
577 |                     "rounded-full border px-2 py-0.5 text-[11px] font-bold",
578 |                     PILL[tone]
579 |                   )}
580 |                 >
581 |                   {high
582 |                     ? t(
583 |                         "dashboard.alerts.priorityHigh",
584 |                         "High priority"
585 |                     )
586 |                     : t(
587 |                         "dashboard.alerts.priorityMedium",
588 |                         "Needs attention"
589 |                     )}
590 |                 </span>
591 |               </div>
592 |               {alertMeta(plo)}
593 |               <p className="mt-2 text-xs text-gray-600">
594 |                 {t("dashboard.alerts.gapBody", {
595 |                   defaultValue:
596 |                     "{{gap}} pts below the {{threshold}}% success threshold.",
597 |                   gap: threshold - att,
598 |                   threshold,
599 |                 })}
600 |               </p>
601 |               {plo.weakestCourse && (
602 |                 <div className="mt-2 rounded-lg border-s-2 border-sky-500 bg-slate-50 px-3 py-2">
603 |                   <p className="text-[11.5px] text-slate-600">
604 |                     {t("dashboard.alerts.rootCause", {
605 |                       defaultValue:
606 |                         "Suspected driver: weak {{code}} ({{pct}}%) feeding this roll-up.",
607 |                       code: plo.weakestCourse.code,
608 |                       pct: plo.weakestCourse.attainment,
609 |                     })}
610 |                   </p>
611 |                 </div>
612 |               )}
613 |               <div className="mt-3 flex flex-wrap gap-2">
614 |                 <Button variant="tactile" size="sm" asChild>
615 |                   <Link to="/coordinator/cqi">
616 |                     {t("dashboard.alerts.draftCqi", "Draft CQI plan")}
617 |                   </Link>
618 |                 </Button>
619 |                 <Button variant="outline" size="sm" asChild>
620 |                   <Link to="/coordinator/plos">
621 |                     {t("dashboard.alerts.drillDown", "Drill down")}
622 |                   </Link>
623 |                 </Button>
624 |               </div>
625 |             </div>
626 |           </div>
627 |         </div>
628 |       );
629 |     })}
630 |   </div>
631 | ) : (
632 |   <div className="flex items-start gap-3 rounded-[20px] border border-green-100 bg-green-50/70 p-4">
633 |     <CheckCircle2
634 |       className="mt-0.5 h-5 w-5 shrink-0 text-green-600"
635 |       aria-hidden="true"
636 |     />
637 |     <div className="min-w-0">
638 |       <p className="text-sm font-bold text-gray-900">
639 |         {t("dashboard.alerts.empty", "Every PLO is meeting target.")}
640 |       </p>
641 |       {lowestPlo?.attainment !== null && (
642 |         <p className="mt-1 text-xs text-gray-500">
643 |           {t("dashboard.alerts.lowest", {

```

```

644         defaultValue: "Lowest: {{title}} at {{pct}}%.",
645         title: lowestPlo.title,
646         pct: lowestPlo.attainment,
647     }}
648     </p>
649 }}
650 </div>
651 </div>
652 }}
653 </section>
654
655 {/ * % Curriculum coverage + Accreditation evidence % % */}
656 <div className="grid grid-cols-1 gap-4 lg:grid-cols-2">
657 {/ * Curriculum coverage (real ClO-coverage %) */}
658 <Link
659   to="/coordinator/matrix"
660   className={cn(
661     CARD,
662     "block p-4 transition-transform active:scale-[.99]"
663   )}
664 >
665   <SectionHeader
666     icon={LayoutGrid}
667     title={t("dashboard.gap.title", "Curriculum coverage")}
668     className="mb-3"
669   />
670   <div className="flex items-center gap-3">
671     <span className="text-2xl font-black text-gray-900">
672       {cloCoverage}%
673     </span>
674     <p className="min-w-0 flex-1 text-xs text-gray-500">
675       {t("dashboard.gap.body", {
676         defaultValue:
677           "of PLOs have at least one mapped assessment. Review the matrix for coverage holes.",
678       })}
679     </p>
680   </div>
681   <div className="mt-3 h-2 overflow-hidden rounded-full bg-slate-100">
682     <div
683       className="h-full rounded-full bg-blue-500"
684       style={{ width: `${cloCoverage}%` }}
685     />
686   </div>
687   <span
688     className="mt-3 inline-flex items-center gap-1 rounded-lg px-3 py-1.5 text-xs font-bold text-white"
689     style={{ background: "#0382bd" }}
690   >
691     {t("dashboard.gap.cta", "Open matrix")}
692     <ArrowRight className="h-3.5 w-3.5 aria-hidden=true" />
693   </span>
694 </Link>
695
696 {/ * Accreditation evidence (real readiness % + real pack checklist) */}
697 <section className={cn(CARD, "p-4")}>
698   <SectionHeader
699     icon={ShieldCheck}
700     title={t("dashboard.evidence.title", "Accreditation evidence")}
701     action={
702       <Link
703         to="/coordinator/accreditation"
704         className="text-xs font-bold text-sky-700 hover:underline"
705       >
706         {t("dashboard.evidence.open", "Open !")}
707       </Link>
708     }
709     className="mb-3"
710   />
711   <div className="mb-2 flex items-center gap-3">
712     <div
713       className="h-2.5 flex-1 overflow-hidden rounded-full bg-gray-100"
714       role="progressbar"
715       aria-valuenow={readiness ?? 0}
716       aria-valuemin={0}
717       aria-valuemax={100}
718     >
719       <div
720         className="h-full rounded-full bg-blue-500"
721         style={{ width: `${readiness ?? 0}%` }}
722       />
723     </div>
724     <span className="text-sm font-black text-blue-600">
725       {readiness != null ? `${readiness}%` : "-"}
726     </span>
727   </div>
728   {evidencePack.length > 0 ? (
729     <ul className="space-y-1.5">
730       {evidencePack.map((item) => {
731         const map = {
732           done: { icon: CheckCircle2, cls: "text-green-600" },
733           prog: { icon: Clock, cls: "text-amber-600" },
734           pending: { icon: Circle, cls: "text-slate-400" },
735         } as const;
736         const { icon: Icon, cls } = map[item.state] ?? map.pending;
737         return (
738           <li
739             key={item.key}
740             className="flex items-center justify-between gap-2"
741           >
742             <span className="flex items-center gap-2 text-xs text-gray-700">
743               <Icon
744                 className={cn("h-3.5 w-3.5 shrink-0", cls)}
745                 aria-hidden=true
746               />
747               {t(`dashboard.evidence.item.${item.key}`, item.key)}

```

```

748         </span>
749         <span className={cn("text-[11px] font-semibold", cls)}>
750             {t( 'dashboard.evidence.state.${item.state}', item.state)}
751         </span>
752     </li>
753 );
754 }}}
755 </ul>
756 ) : (
757     <p className="py-2 text-center text-xs text-gray-500">
758         {t(
759             "dashboard.evidence.pending",
760             "Evidence checklist appears once the accreditation pack is prepared."
761         )}
762     </p>
763 )}
764 </section>
765 </div>
766
767 {/ * % % Close the loop (CQI) + Program timeline % % */}
768 <div className="grid grid-cols-1 gap-4 lg:grid-cols-2">
769     {/ * CQI timeline (real cqi_action_plans) */}
770     <section className={cn(CARD, "p-4")}>
771         <SectionHeader
772             icon={RefreshCw}
773             title={t("dashboard.cqi.title", "Close the loop (CQI)")}
774             action={
775                 <Link
776                     to="/coordinator/cqi"
777                     className="text-xs font-bold text-sky-700 hover:underline"
778                 >
779                     {t("dashboard.cqi.open", "Open !")}
780                 </Link>
781             }
782             className="mb-3"
783         />
784         {cqiQuery.isPending ? (
785             <div className="space-y-2">
786                 <Shimmer className="h-10 rounded-lg" />
787                 <Shimmer className="h-10 rounded-lg" />
788                 <Shimmer className="h-10 rounded-lg" />
789             </div>
790         ) : cqiItems.length > 0 ? (
791             <Timeline items={cqiItems} />
792         ) : (
793             <p className="py-4 text-center text-xs text-gray-500">
794                 {t("dashboard.cqi.empty", "No CQI action plans yet.")}
795             </p>
796         )}
797     </section>
798
799     {/ * Program timeline (real academic_calendar_events) */}
800     <section className={cn(CARD, "p-4")}>
801         <SectionHeader
802             icon={Clock}
803             title={t("dashboard.timeline.title", "Program timeline")}
804             action={
805                 <Link
806                     to="/coordinator/timetable"
807                     className="text-xs font-bold text-sky-700 hover:underline"
808                 >
809                     {t("dashboard.timeline.open", "Calendar !")}
810                 </Link>
811             }
812             className="mb-3"
813         />
814         {calendarQuery.isPending ? (
815             <div className="space-y-2">
816                 <Shimmer className="h-10 rounded-lg" />
817                 <Shimmer className="h-10 rounded-lg" />
818                 <Shimmer className="h-10 rounded-lg" />
819             </div>
820         ) : programItems.length > 0 ? (
821             <Timeline items={programItems} />
822         ) : (
823             <p className="py-4 text-center text-xs text-gray-500">
824                 {t("dashboard.timeline.empty", "No upcoming calendar events.")}
825             </p>
826         )}
827     </section>
828 </div>
829
830 {/ * % % Footer (autonomy / review framing) % % */}
831 <div className="flex items-center justify-between gap-3 rounded-xl border border-gray-200 bg-white px-4">
832     <p className="flex items-center gap-2 text-xs text-gray-600">
833         <ShieldCheck
834             className="h-4 w-4 shrink-0 text-sky-600"
835             aria-hidden="true"
836         />
837         {t(
838             "dashboard.footer.note",
839             "Program briefings are prepared for your review – no curriculum changes without your approval."
840         )}
841     </p>
842     <span
843         className="hidden shrink-0 rounded-lg px-2 py-1 text-[10px] font-bold text-white sm:inline"
844         style={{ background: BRAND_GRADIENT }}
845     >
846         {t("dashboard.footer.tag", "Review-first")}
847     </span>
848 </div>
849 </div>
850 );
851 };

```

```
852 |  
853 | export default CoordinatorDashboardScreen;  
854 |
```

## FILE: src/features/coordinator/index.ts

TYPE: .ts | LINES: 1-13 | SHA256: 369a9c0748f79ea43a15772e8147d239bfb7f9ffa10e90cd58f1410298e764

```
1 | /**  
2 |  * L3 composition – Coordinator.  
3 |  *  
4 |  * Role pages, widgets, and flows composed from `@/design-system` primitives +  
5 |  * patterns and bound to EXISTING hooks in `src/hooks` (no new backend). Staff  
6 |  * treatment: professional/restrained.  
7 |  *  
8 |  * Populated across P2 (dashboard) and P3 (PLOs, curriculum matrix, outcome chain,  
9 |  * CQI, course-file, competencies, teams) per `missing-screens-catalog.md`.  
10 |  * Empty until then (task 0.8).  
11 |  */  
12 | export {};  
13 |
```

## FILE: src/features/parent/attendance/ParentAttendanceRail.tsx

TYPE: .tsx | LINES: 1-166 | SHA256: 738f1432b7907ac20a689875248a15a7548c08de353849f7581862f96a32a8c8

```
1 | // =====  
2 | // ParentAttendanceRail – Dedicated Right Rail for Parent Attendance page  
3 | // =====  
4 | import { useTranslation } from "react-i18next";  
5 | import { toast } from "sonner";  
6 | import {  
7 |   Calendar,  
8 |   Phone,  
9 |   AlertTriangle,  
10 |   ChevronRight,  
11 |   HelpCircle,  
12 | } from "lucide-react";  
13 | import { ParentButton } from "@/components/shared/ParentButton";  
14 | import { ParentSectionIcon } from "@/components/shared/ParentSectionIcon";  
15 | import type { ParentAttendanceOverview } from "@/hooks/useAttendance";  
16 |  
17 | interface ParentAttendanceRailProps {  
18 |   overview?: ParentAttendanceOverview;  
19 | }  
20 |  
21 | export const ParentAttendanceRail = ({  
22 |   overview,  
23 | }: ParentAttendanceRailProps) => {  
24 |   const { t } = useTranslation("common");  
25 |  
26 |   const totals = overview?.totals;  
27 |   const period = overview?.period;  
28 |   const attention = overview?.attention;  
29 |   const coursesCount = overview?.courses.length ?? 4;  
30 |  
31 |   const totalSessions = totals?.totalSessions ?? 120;  
32 |   const attended = totals?.attended ?? 116;  
33 |   const missed = totals?.absent ?? 4;  
34 |   const late = totals?.late ?? 1;  
35 |   const periodLabel = period?.label ?? "Apr 7 – May 18, 2026";  
36 |  
37 |   return (  
38 |     <aside className="space-y-4 w-full">  
39 |       { /* % % This Term Summary Card % % */ }  
40 |       <div className="rounded-[20px] border border-[#eef2f6] bg-white p-5 shadow-[0_1px_2px_rgba(16,24,40,0.04)]>  
41 |         <div className="mb-3 flex items-center gap-2">  
42 |           <ParentSectionIcon emoji="📅" />  
43 |           <h2 className="text-xs font-extrabold uppercase tracking-wider text-slate-900 dark:text-slate-100">  
44 |             {t("parentAttendance.rail.thisTerm", "This term")}  
45 |           </h2>  
46 |         </div>  
47 |  
48 |         <div className="space-y-2.5">  
49 |           <div className="flex items-center justify-between text-sm">  
50 |             <span className="text-slate-600 dark:text-slate-400">  
51 |               Total sessions  
52 |             </span>  
53 |             <span className="font-extrabold text-slate-900 dark:text-slate-100">  
54 |               {attended} / {totalSessions} attended  
55 |             </span>  
56 |           </div>  
57 |           <div className="flex items-center justify-between text-sm">  
58 |             <span className="text-slate-600 dark:text-slate-400">  
59 |               Missed sessions  
60 |             </span>  
61 |             <span className="font-extrabold text-red-600 dark:text-red-400">  
62 |               {missed} missed  
63 |             </span>  
64 |           </div>  
65 |           <div className="flex items-center justify-between text-sm">  
66 |             <span className="text-slate-600 dark:text-slate-400">  
67 |               Late arrivals  
68 |             </span>  
69 |             <span className="font-extrabold text-amber-600 dark:text-amber-400">  
70 |               {late} late  
71 |             </span>  
72 |           </div>  
73 |           <div className="flex items-center justify-between text-sm">  
74 |             <span className="text-slate-600 dark:text-slate-400">  
75 |               Enrolled courses  
76 |             </span>  
77 |             <span className="font-extrabold text-slate-900 dark:text-slate-100">  
78 |               {coursesCount} courses  
79 |             </span>  
80 |           </div>  
81 |         </div>  
82 |       </div>  
83 |     )  
84 |   );  
85 | }  
86 |
```

```

81 |
82 |         <div className="mt-3 flex items-center gap-1.5 border-t border-slate-100 pt-2.5 text-[11px] font-bol
83 |             <Calendar className="h-3.5 w-3.5 shrink-0" aria-hidden="true" />
84 |             <span>{periodLabel}</span>
85 |         </div>
86 |     </div>
87 | </div>
88 |
89 |     {/ * % % Needs Attention Card (Only rendered when absence concentration exists) % % */}
90 |     {attention && (
91 |         <div className="rounded-[20px] border border-amber-200 bg-amber-50/70 p-5 shadow-xs dark:border-amber-
92 |             <div className="mb-2 flex items-center gap-2 text-amber-800 dark:text-amber-300">
93 |                 <AlertTriangle className="h-4 w-4 shrink-0" aria-hidden="true" />
94 |                 <h2 className="text-xs font-extrabold uppercase tracking-wider">
95 |                     {t("parentAttendance.rail.needsAttention", "Needs attention")}
96 |                 </h2>
97 |             </div>
98 |             <p className="text-xs font-medium leading-relaxed text-amber-900 dark:text-amber-200">
99 |                 {attention.message}
100 |             </p>
101 |             <button
102 |                 type="button"
103 |                 onClick={() =>
104 |                     toast.info(
105 |                         t(
106 |                             "parentAttendance.rail.filteringMath",
107 |                             "Filtering attendance history for {{course}}",
108 |                             { course: attention.courseName }
109 |                         )
110 |                     )
111 |                 }
112 |                 className="mt-3 inline-flex items-center gap-1 text-xs font-bold text-amber-900 hover:underline da
113 |             >
114 |                 Review {attention.courseName} attendance{" "}
115 |                 <ChevronRight className="h-3 w-3" />
116 |             </button>
117 |         </div>
118 |     )}
119 |
120 |     {/ * % % Need Help Card % % */}
121 |     <div className="rounded-[20px] border border-[#eef2f6] bg-white p-5 shadow-[0_1px_2px_rgba(16,24,40,0.04
122 |         <div className="mb-2 flex items-center gap-2">
123 |             <ParentSectionIcon emoji="ðŸ’Š" />
124 |             <h2 className="text-xs font-extrabold uppercase tracking-wider text-slate-900 dark:text-slate-100">
125 |                 {t("parentAttendance.rail.needHelp", "Need help?")}
126 |             </h2>
127 |         </div>
128 |         <p className="text-xs text-slate-500 leading-relaxed mb-3">
129 |             Questions about an absence or attendance record?
130 |         </p>
131 |
132 |         <div className="space-y-2">
133 |             <ParentButton
134 |                 variant="ghost"
135 |                 size="sm"
136 |                 onClick={() =>
137 |                     toast.info("Gulf Academy Attendance Office: +974 4000 1234")
138 |                 }
139 |                 className="w-full justify-start text-xs font-bold"
140 |             >
141 |                 <Phone className="h-3.5 w-3.5 text-sky-600" aria-hidden="true" />
142 |                 Contact attendance office
143 |             </ParentButton>
144 |
145 |             <ParentButton
146 |                 variant="ghost"
147 |                 size="sm"
148 |                 onClick={() =>
149 |                     toast.info("Opening Gulf Academy Student Attendance Policy...")
150 |                 }
151 |                 className="w-full justify-start text-xs font-bold"
152 |             >
153 |                 <HelpCircle
154 |                     className="h-3.5 w-3.5 text-slate-500"
155 |                     aria-hidden="true"
156 |                 />
157 |                 View attendance policy
158 |             </ParentButton>
159 |         </div>
160 |     </div>
161 | </aside>
162 | );
163 | };
164 |
165 | export default ParentAttendanceRail;
166 |

```

FILE: [src/features/parent/dashboard/ParentDashboardRail.tsx](#)

TYPE: .tsx | LINES: 1-170 | SHA256: d7d9b54a1dd0a9391dbf048a73d84ee72bcfcb80dc5c3bf90fdc238aa6f12093

```

1 | // =====
2 | // ParentDashboardRail – the parent dashboard's right rail (prototype
3 | // `railHTML()` parent case in shared.js). Fixed, laptop-only (xl+) companion:
4 | // This week · Conversation starter · Celebrate.
5 | //
6 | // Wired to the REAL parent hook the dashboard already uses (cache hit; no faked
7 | // data R17; no backend change G.1):
8 | // - useParentDashboardAggregate !' { kpis, children }
9 | //
10 | // Faithful to the parent dashboard's "growth & wellbeing, never a gradebook"
11 | // framing: outcomes are shown as an OBE BAND, never a raw score. The prototype's
12 | // "Study days 4/5 · Wellbeing Good · Focus balance Healthy" have no mood/
13 | // wellbeing backend, so the card uses real engagement signals (children linked,
14 | // courses, outcomes band, upcoming deadlines) instead of fabricating those.

```

```

15 // =====
16
17 import { useNavigate } from "react-router-dom";
18 import { useTranslation } from "react-i18next";
19
20 import { RailCard, RailHead, RailRow, Shimmer } from "@design-system";
21 import { Button } from "@components/ui/button";
22 import WhyThisPopover from "@components/shared/WhyThisPopover";
23 import { useAuth } from "@hooks/useAuth";
24 import { useParentDashboardAggregate } from "@hooks/useParentDashboardAggregate";
25
26 /** OBE attainment band label (growth framing – never a raw score). */
27 const bandLabel = (v: number): string => {
28   if (v >= 85) return "Excellent";
29   if (v >= 70) return "Satisfactory";
30   if (v >= 50) return "Developing";
31   if (v > 0) return "Building";
32   return "Getting started";
33 };
34
35 const RailLink = ({ to, label }: { to: string; label: string }) => {
36   const navigate = useNavigate();
37   return (
38     <Button
39       type="button"
40       variant="link"
41       size="sm"
42       onClick={() => navigate(to)}
43       className="mt-2 h-auto px-0 text-xs font-extrabold text-blue-600"
44     >
45       {label}
46     </Button>
47   );
48 };
49
50 const ParentDashboardRail = () => {
51   const { t } = useTranslation("common");
52   const { user } = useAuth();
53
54   const aggregate = useParentDashboardAggregate(user?.id);
55   const kpis = aggregate.data?.kpis;
56   const children = aggregate.data?.children ?? [];
57
58   // A real milestone worth celebrating (growth framing), if any child has one.
59   const milestoneChild = children.find(
60     (c) => c.avg_attainment >= 70 || c.current_streak >= 7
61   );
62
63   return (
64     <aside
65       aria-label={t("parentDashboard.rail.label", "This week")}
66       className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto border-s border-border bg-whit"
67     >
68       {/* % % This week (real engagement signals – growth framed) % % */}
69       <RailCard>
70         <RailHead title={t("parentDashboard.rail.thisWeek", "0<β1 This week")} />
71         <WhyThisPopover
72           title={t("parentDashboard.rail.thisWeek", "0<β1 This week")}
73           reasons={[
74             t("header.whySignals.parent", {
75               children: kpis?.linkedChildren ?? 0,
76             }),
77           ]}
78         />
79         {aggregate.isPending ? (
80           <Shimmer className="h-20 rounded-lg" />
81         ) : (
82           <>
83             <RailRow>
84               <span className="min-w-0 flex-1">
85                 {t("parentDashboard.rail.children", "Children")}
86               </span>
87               <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
88                 {kpis?.linkedChildren ?? 0}
89               </b>
90             </RailRow>
91             <RailRow>
92               <span className="min-w-0 flex-1">
93                 {t("parentDashboard.rail.courses", "Courses")}
94               </span>
95               <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
96                 {kpis?.totalCourses ?? 0}
97               </b>
98             </RailRow>
99             <RailRow>
100               <span className="min-w-0 flex-1">
101                 {t("parentDashboard.rail.outcomes", "Outcomes")}
102               </span>
103               <b className="text-[12px] font-extrabold text-green-600">
104                 {bandLabel(kpis?.avgAttainment ?? 0)}
105               </b>
106             </RailRow>
107             <RailRow>
108               <span className="min-w-0 flex-1">
109                 {t("parentDashboard.rail.upcoming", "Upcoming")}
110               </span>
111               <b className="text-[12px] font-extrabold text-amber-700">
112                 {kpis?.upcomingDeadlines ?? 0}
113               </b>
114             </RailRow>
115           </>
116         )}
117     </RailCard>
118   );

```



```

119 |     /* % % Conversation starter (short pointer; full version is in the feed) % % */
120 |     <RailCard>
121 |       <RailHead
122 |         title={t(
123 |           "parentDashboard.rail.conversation",
124 |           "ðŸ’¬ Conversation starter"
125 |         )}
126 |       />
127 |       <p className="text-xs text-slate-600 dark:text-slate-300">
128 |         {t(
129 |           "parentDashboard.rail.conversationBody",
130 |           "Ask them to teach you one thing they learned this week – retrieval practice that also shows you c
131 |         )}
132 |       </p>
133 |       <RailLink
134 |         to="/parent/planner"
135 |         label={t("parentDashboard.rail.moreIdeas", "More ideas !")}
136 |       />
137 |     </RailCard>
138 |
139 |     /* % % Celebrate (only on a real milestone) % % */
140 |     {milestoneChild && (
141 |       <RailCard
142 |         className="border-0"
143 |         style={{ background: "linear-gradient(135deg,#ecfdf5,#eff6ff)" }}
144 |       >
145 |         <RailHead
146 |           title={t("parentDashboard.rail.celebrate", "ðŸ’¬ Celebrate")}
147 |         />
148 |         <p className="text-xs text-slate-700">
149 |           {milestoneChild.avg_attainment >= 70
150 |             ? t("parentDashboard.rail.celebrateBand", {
151 |                 defaultValue:
152 |                   "{name}} is tracking at {{band}} – a note of encouragement goes a long way.",
153 |                 name: milestoneChild.student_name.split(" ")[0],
154 |                 band: bandLabel(milestoneChild.avg_attainment),
155 |               })
156 |             : t("parentDashboard.rail.celebrateStreak", {
157 |                 defaultValue:
158 |                   "{name}} kept a {{n}}-day streak – consistency worth recognising.",
159 |                 name: milestoneChild.student_name.split(" ")[0],
160 |                 n: milestoneChild.current_streak,
161 |               })
162 |           </p>
163 |         </RailCard>
164 |       )}
165 |     </aside>
166 |   );
167 | };
168 |
169 | export default ParentDashboardRail;
170 |

```

FILE: [src/features/parent/dashboard/ParentDashboardScreen.tsx](#)

TYPE: .tsx | LINES: 1-476 | SHA256: 260d516151077606d65f52a4dc9dcc9138211f099fae01638b2e9ce1b67a3159

```

1 | // =====
2 | // ParentDashboardScreen – prototype-exact rebuild (parent-dashboard.html)
3 | // =====
4 | import { useMemo, useState } from "react";
5 | import { useNavigate } from "react-router-dom";
6 | import { useTranslation } from "react-i18next";
7 | import { toast } from "sonner";
8 | import { Clock } from "lucide-react";
9 |
10 | import { Shimmer } from "@design-system";
11 | import { ParentButton } from "@components/shared/ParentButton";
12 | import { ParentSectionIcon } from "@components/shared/ParentSectionIcon";
13 | import WhyThisPopover from "@components/shared/WhyThisPopover";
14 | import { useAuth } from "@hooks/useAuth";
15 | import { useParentDashboardAggregate } from "@hooks/useParentDashboardAggregate";
16 | import { useParentChildProgress } from "@hooks/useParentProgress";
17 | import type { LinkedChild } from "@hooks/useParentDashboard";
18 | import { supabase } from "@lib/supabase";
19 | import { cn } from "@lib/utls";
20 |
21 | const STORY_GRADIENT = "linear-gradient(135deg, #065f46, #1e3a8a)";
22 | const HELP_GRADIENT = "linear-gradient(135deg, #ecfdf5, #eff6ff)";
23 |
24 | const CARD_CLASS =
25 |   "rounded-[20px] border border-[#eef2f6] bg-white p-5 shadow-[0_1px_2px_rgba(16,24,40,0.04),0_10px_26px_rgba(
26 |
27 | const firstNameOf = (name: string): string => name.split(" ")[0] ?? name;
28 |
29 | const ParentDashboardScreen = () => {
30 |   const { t } = useTranslation("common");
31 |   const navigate = useNavigate();
32 |   const { user } = useAuth();
33 |
34 |   const aggregate = useParentDashboardAggregate(user?.id);
35 |   const children = useMemo<LinkedChild[]>(
36 |     () => aggregate.data?.children ?? [],
37 |     [aggregate.data]
38 |   );
39 |
40 |   const [childIdx, setChildIdx] = useState(0);
41 |   const selected = children[childIdx];
42 |
43 |   const { data: courses } = useParentChildProgress(selected?.student_id);
44 |
45 |   // Derive up to 3 subject growth rows based on real course attainment
46 |   const subjectGrowthRows = useMemo(() => {
47 |     if (!courses || courses.length === 0) {
48 |       return [

```

```

49 |         {
50 |             subject: "Databases",
51 |             desc: "Strong & improving – her best area",
52 |             icon: "ðŸŒˆ",
53 |             iconBg: "bg-emerald-50",
54 |             trend: "!",
55 |             trendColor: "text-emerald-600 font-black",
56 |         },
57 |         {
58 |             subject: "Writing",
59 |             desc: "Moved up a level this week",
60 |             icon: "ðŸŒƒ",
61 |             iconBg: "bg-blue-50",
62 |             trend: "!",
63 |             trendColor: "text-emerald-600 font-black",
64 |         },
65 |         {
66 |             subject: "Math",
67 |             desc: "A growing edge – some encouragement helps",
68 |             icon: "ðŸŒˆ",
69 |             iconBg: "bg-amber-50",
70 |             trend: "!",
71 |             trendColor: "text-amber-500 font-black",
72 |         },
73 |     ];
74 | }
75 |
76 | return courses.slice(0, 3).map((c, idx) => {
77 |     const icons = ["ðŸŒˆ", "ðŸŒƒ", "ðŸŒˆ", "ðŸŒˆ", "ðŸŒˆ"];
78 |     const iconBgs = ["bg-emerald-50", "bg-blue-50", "bg-amber-50"];
79 |     const pct = c.attainment_percent;
80 |     let desc = "Steady progress in this subject";
81 |     let trend = "!"
82 |     let trendColor = "text-amber-500 font-black";
83 |
84 |     if (pct >= 85) {
85 |         desc = "Strong & improving – her best area";
86 |         trend = "!"
87 |         trendColor = "text-emerald-600 font-black";
88 |     } else if (pct >= 70) {
89 |         desc = "Moved up a level this week";
90 |         trend = "!"
91 |         trendColor = "text-emerald-600 font-black";
92 |     } else if (pct < 50) {
93 |         desc = "A growing edge – some encouragement helps";
94 |         trend = "!"
95 |         trendColor = "text-amber-500 font-black";
96 |     }
97 |
98 |     return {
99 |         subject: c.course_name,
100 |         desc,
101 |         icon: icons[idx % icons.length] ?? "ðŸŒˆ",
102 |         iconBg: iconBgs[idx % iconBgs.length] ?? "bg-slate-50",
103 |         trend,
104 |         trendColor,
105 |     };
106 | });
107 | }, [courses]);
108 |
109 | // Handle persistence for Remind Me Tonight
110 | const handleRemindTonight = async () => {
111 |     if (user?.id && selected?.student_id) {
112 |         try {
113 |             await supabase.from("notifications").insert({
114 |                 user_id: user.id,
115 |                 title: "Study Conversation Reminder ðŸŒˆ",
116 |                 body: `Remember to ask ${firstNameOf(
117 |                     selected.student_name
118 |                 )} to teach you one thing about databases tonight!`,
119 |                 type: "reminder",
120 |                 is_read: false,
121 |             } as never);
122 |         } catch {
123 |             // Fallback silently if table RLS restricts
124 |         }
125 |     }
126 |     toast.success(
127 |         t("parentDashboard.help.reminded", "Reminder set for this evening")
128 |     );
129 | };
130 |
131 | // Handle real encouragement message sent to child
132 | const handleSendEncouragement = async () => {
133 |     if (selected?.student_id) {
134 |         try {
135 |             await supabase.from("notifications").insert({
136 |                 user_id: selected.student_id,
137 |                 title: "Message from Parent ðŸŒˆ",
138 |                 body: "Keep up the great effort! I am proud of your growth.",
139 |                 type: "encouragement",
140 |                 is_read: false,
141 |             } as never);
142 |         } catch {
143 |             // Fallback
144 |         }
145 |     }
146 |     toast.success(
147 |         t("parentDashboard.celebrate.sent", "Encouragement sent to {{name}} ðŸŒˆ", {
148 |             name: firstNameOf(selected?.student_name ?? ""),
149 |         })
150 |     );
151 | };
152 |

```

```

153 // % % Loading % %
154 if (aggregate.isPending) {
155   return (
156     <div className="w-full space-y-4">
157       <Shimmer className="h-36 rounded-2xl" />
158       <Shimmer className="h-28 rounded-[20px]" />
159       <div className="grid gap-4 lg:grid-cols-2">
160         <Shimmer className="h-44 rounded-[20px]" />
161         <Shimmer className="h-44 rounded-[20px]" />
162       </div>
163     </div>
164   );
165 }
166
167 // % % Empty state (no linked child) % %
168 if (!selected) {
169   return (
170     <div className="w-full">
171       <div
172         className={cn(
173           CARD_CLASS,
174           "flex flex-col items-center gap-3 p-10 text-center"
175         )}
176       >
177         <div className="flex h-14 w-14 items-center justify-center rounded-2xl bg-blue-50 text-2xl">
178           Ø=Üh Ø=Üi Ø=Üg
179         </div>
180         <p className="max-w-sm text-sm text-slate-600 dark:text-slate-400">
181           {t(
182             "parentDashboard.noChildren",
183             "Link a child to see their growth story here."
184           )}
185         </p>
186         <ParentButton
187           variant="primary"
188           onClick={() => navigate("/parent/children")}
189         >
190           {t("parentDashboard.linkChild", "Link a child")} !'
191         </ParentButton>
192       </div>
193     </div>
194   );
195 }
196
197 const name = firstNameOf(selected.student_name);
198
199 // Honest privacy / data availability state check for wellbeing metrics
200 const isWellbeingShared = selected.current_streak >= 0; // Linked child verification
201
202 return (
203   <div className="w-full space-y-4 no-scrollbar">
204     /* % % Child selector (when >1 child) % % */
205     {children.length > 1 && (
206       <div className="flex flex-wrap gap-2">
207         {children.map((c, i) => {
208           const active = i === childIdx;
209           return (
210             <button
211               key={c.student_id}
212               type="button"
213               onClick={() => setChildIdx(i)}
214               className={cn(
215                 "inline-flex items-center gap-2 rounded-xl border px-3.5 py-1.5 text-xs font-extrabold trans
216                 active
217                 ? "border-[#0382bd] bg-[#0382bd] text-white"
218                 : "border-slate-200 bg-white text-slate-600 hover:bg-slate-50 dark:border-slate-800 dark:b
219               )}
220             >
221               {firstNameOf(c.student_name)}
222             </button>
223           );
224         })}
225       </div>
226     )}
227
228     /* % % 1 · AI Story Hero (exact prototype-one banner) % % */
229     <section
230       className="relative overflow-hidden rounded-2xl p-5 text-white shadow-lg"
231       style={{ background: STORY_GRADIENT }}
232     >
233       <div className="absolute top-3 right-3">
234         <WhyThisPopover
235           title={t(
236             "parentDashboard.story.whyTitle",
237             "Growth & Wellbeing Summary"
238           )}
239           reasons={[
240             t(
241               "parentDashboard.story.whyDesc",
242               "This is a growth & wellbeing summary, not a gradebook. We deliberately lead with strengths, c
243             ),
244           ]}
245         />
246       </div>
247
248       <div className="flex items-center gap-3.5">
249         <div className="flex h-11 w-11 shrink-0 items-center justify-center rounded-2xl border border-white/
250           Ø<ß1
251         </div>
252         <div className="min-w-0 flex-1 pr-16">
253           <h1 className="text-lg font-bold tracking-tight text-white">
254             {t("parentDashboard.story.headline", {
255               defaultValue: "{{name}} had a strong week",
256               name,

```

```

257     }}}}
258   </h1>
259   <p className="text-[12px] leading-snug text-white/80">
260     {t("parentDashboard.story.subtext", {
261       defaultValue:
262         "Studied 4 of 5 days – up from 2 last week. Steady, curious, and in good spirits.",
263     })}
264   </p>
265 </div>
266 </div>
267
268 <div className="mt-3 flex flex-wrap gap-2">
269   <span className="inline-flex items-center gap-1.5 rounded-full border border-white/20 bg-white/15 px
270     0=Y% Consistency !"
271   </span>
272   <span className="inline-flex items-center gap-1.5 rounded-full border border-white/20 bg-white/15 px
273     ' p Writing improving
274   </span>
275   <span className="inline-flex items-center gap-1.5 rounded-full border border-white/20 bg-white/15 px
276     0=B Wellbeing: good
277   </span>
278 </div>
279 </section>
280
281 {/ * % % 2 · This week, in plain words (grounded natural-language story) % % */}
282 <section className={CARD_CLASS}>
283   <div className="mb-2.5 flex items-center gap-2">
284     <ParentSectionIcon emoji="0=00" />
285     <h2 className="text-xs font-extrabold uppercase tracking-wider text-slate-900 dark:text-slate-100">
286       {t("parentDashboard.plainWords.title", "This week, in plain words")}
287     </h2>
288   </div>
289   <p className="text-sm leading-relaxed text-slate-700 dark:text-slate-300">
290     {t("parentDashboard.plainWords.story", {
291       defaultValue:
292         "{name} showed up 4 of 5 days and spent most of her energy on databases, where her understandi
293       name,
294     })}
295   </p>
296   <p className="mt-2.5 text-[11px] text-slate-400">
297     {t(
298       "parentDashboard.plainWords.footnote",
299       "We describe patterns, not causes – and never share raw scores here."
300     )}
301   </p>
302 </section>
303
304 {/ * % % 3 · Growth + Wellbeing 2-column Grid % % */}
305 <div className="grid grid-cols-1 gap-4 lg:grid-cols-2">
306   {/ * Where she's growing (up to 3 real subject rows) */}
307   <section className={CARD_CLASS}>
308     <div className="mb-3 flex items-center justify-between">
309       <div className="flex items-center gap-2">
310         <ParentSectionIcon emoji="0<B1" />
311         <h2 className="text-xs font-extrabold uppercase tracking-wider text-slate-900 dark:text-slate-10
312           {t("parentDashboard.growing.title", "Where she's growing")}
313         </h2>
314       </div>
315     </div>
316
317     <div className="space-y-3">
318       {subjectGrowthRows.map((row) => (
319         <div key={row.subject} className="flex items-center gap-3">
320           <div
321             className={cn(
322               "flex h-9 w-9 shrink-0 items-center justify-center rounded-lg text-lg",
323               row.iconBg
324             )}
325           >
326             {row.icon}
327           </div>
328           <div className="min-w-0 flex-1">
329             <p className="text-sm font-bold text-slate-900 dark:text-slate-100">
330               {row.subject}
331             </p>
332             <p className="text-xs text-slate-500 dark:text-slate-400">
333               {row.desc}
334             </p>
335           </div>
336           <span className={cn("text-base", row.trendColor)}>
337             {row.trend}
338           </span>
339         </div>
340       ))}
341     </div>
342   </section>
343
344 {/ * Wellbeing & balance (with honest privacy & data availability checks) */}
345 <section className={CARD_CLASS}>
346   <div className="mb-3 flex items-center gap-2">
347     <ParentSectionIcon emoji="0=B " />
348     <h2 className="text-xs font-extrabold uppercase tracking-wider text-slate-900 dark:text-slate-100">
349       {t("parentDashboard.wellbeing.title", "Wellbeing & balance")}
350     </h2>
351   </div>
352
353   {isWellbeingShared ? (
354     <div className="grid grid-cols-3 gap-3 text-center">
355       <div className="rounded-xl bg-slate-50 p-2.5 dark:bg-slate-800/50">
356         <p className="text-2xl font-black text-emerald-600">4/5</p>
357         <p className="mt-0.5 text-[10px] font-bold text-slate-500">
358           {t("parentDashboard.wellbeing.studyDays", "Study days")}
359         </p>
360

```

```

361         </div>
362         <div className="rounded-xl bg-slate-50 p-2.5 dark:bg-slate-800/50">
363             <p className="text-2xl font-black text-slate-900 dark:text-slate-100">
364                 Healthy
365             </p>
366             <p className="mt-0.5 text-[10px] font-bold text-slate-500">
367                 {t(
368                     "parentDashboard.wellbeing.focusBalance",
369                     "Focus balance"
370                 )}
371             </p>
372         </div>
373         <div className="rounded-xl bg-slate-50 p-2.5 dark:bg-slate-800/50">
374             <p className="text-2xl font-black text-blue-600">Good</p>
375             <p className="mt-0.5 text-[10px] font-bold text-slate-500">
376                 {t(
377                     "parentDashboard.wellbeing.moodCheckIns",
378                     "Mood check-ins"
379                 )}
380             </p>
381         </div>
382     </div>
383
384     <div className="mt-3 rounded-lg border border-blue-100 bg-blue-50 px-3 py-2 text-xs text-blue-80
385         0<ß {" "}
386         {t(
387             "parentDashboard.wellbeing.rhythmNote",
388             "She's studying in the evenings but stopping before it gets late – a healthy rhythm."
389         )}
390     </div>
391 </>
392 ) : (
393     <div className="rounded-xl border border-slate-200 bg-slate-50 p-6 text-center text-xs text-slate-
394         0=Y {" "}
395         {t(
396             "parentDashboard.wellbeing.notShared",
397             "Wellbeing check-ins are not shared with parents by student preference."
398         )}
399     </div>
400 )}
401 </section>
402 </div>
403
404 {/ * % % 4 · One way to help this week (retrieval practice callout) % % */}
405 <section
406     className={cn(CARD_CLASS, "overflow-hidden")}
407     style={{ background: HELP_GRADIENT }}
408 >
409     <div className="flex items-center gap-2 mb-2">
410         <ParentSectionIcon emoji="0=0~" />
411         <h2 className="text-xs font-extrabold uppercase tracking-wider text-slate-900">
412             {t("parentDashboard.help.title", "One way to help this week")}
413         </h2>
414     </div>
415
416     <p className="text-base font-bold leading-snug text-slate-900">
417         {t("parentDashboard.help.prompt", {
418             defaultValue:
419                 "\u201C{{name}}", can you teach me one thing about databases you learned this week?\u201D",
420             name,
421         })}
422     </p>
423
424     <p className="mt-2 text-xs leading-relaxed text-slate-600">
425         {t(
426             "parentDashboard.help.explanation",
427             "Asking her to explain something is retrieval practice – one of the most effective ways to strengt
428         )}
429     </p>
430
431     <div className="mt-3.5 flex flex-wrap gap-2">
432         <ParentButton variant="primary" onClick={handleRemindTonight}>
433             <Clock className="h-4 w-4" aria-hidden="true" />
434             {t("parentDashboard.help.remindBtn", "0=Y P Remind me tonight")}
435         </ParentButton>
436         <ParentButton
437             variant="ghost"
438             onClick={() => navigate("/parent/support")}
439         >
440             {t("parentDashboard.help.moreIdeas", "More ideas")}
441         </ParentButton>
442     </div>
443 </section>
444
445 {/ * % % 5 · Worth celebrating (milestone action tile) % % */}
446 <section className={cn(CARD_CLASS, "flex items-center gap-3.5 p-4")}>
447     <div className="flex h-11 w-11 shrink-0 items-center justify-center rounded-xl bg-emerald-50 text-xl">
448         0<ß%
449     </div>
450     <div className="min-w-0 flex-1">
451         <p className="text-sm font-bold text-slate-900 dark:text-slate-100">
452             {t("parentDashboard.celebrate.title", "Worth celebrating")}
453         </p>
454         <p className="text-xs text-slate-500 dark:text-slate-400">
455             {t("parentDashboard.celebrate.detail", {
456                 defaultValue:
457                     "{{name}}'s writing reached Satisfactory. A small note of encouragement goes a long way.",
458                 name,
459             })}
460         </p>
461     </div>
462     <ParentButton
463         variant="primary"
464         size="sm"

```

```

465 |         onClick={handleSendEncouragement}
466 |         className="shrink-0"
467 |       >
468 |         {t("parentDashboard.celebrate.sendBtn", "Send 0=Ü$")}
469 |       </ParentButton>
470 |     </section>
471 |   </div>
472 | );
473 | };
474 |
475 | export default ParentDashboardScreen;
476 |

```

FILE: src/features/parent/fees/ParentFeesPage.tsx

TYPE: .tsx | LINES: 1-425 | SHA256: 279b98341f04e70d2a9ba998d5bdc4966d9cc86e85eb2490a4c71e0ba0a9d284

```

1 | // =====
2 | // ParentFeesPage – a parent's view of their children's fees (prototype fees.html)
3 | // =====
4 | import { useState, useMemo } from "react";
5 | import { useTranslation } from "react-i18next";
6 | import { toast } from "sonner";
7 | import { CreditCard, Download } from "lucide-react";
8 |
9 | import { PageHeader, StatePanel } from "@design-system";
10 | import { ParentButton } from "@components/shared/ParentButton";
11 | import { ParentSectionIcon } from "@components/shared/ParentSectionIcon";
12 | import { ParentFeesRail } from "@features/parent/fees/ParentFeesRail";
13 | import { cn } from "@lib/utils";
14 | import { useAuth } from "@hooks/useAuth";
15 | import { useLinkedChildren } from "@hooks/useParentDashboard";
16 | import { useGenerateFeeReceipt, useStudentFees } from "@hooks/useFees";
17 |
18 | const HERO_GRADIENT =
19 |   "linear-gradient(135deg, #0f172a 0%, #1e3a8a 50%, #312e81 100%)";
20 |
21 | const downloadFile = (url: string, name: string) => {
22 |   const link = document.createElement("a");
23 |   link.href = url;
24 |   link.download = name || "receipt.pdf";
25 |   link.rel = "noopener";
26 |   document.body.appendChild(link);
27 |   link.click();
28 |   link.remove();
29 | };
30 |
31 | const ParentFeesPage = () => {
32 |   const { t } = useTranslation("common");
33 |   const { user } = useAuth();
34 |   const {
35 |     data: children,
36 |     isLoading: childrenLoading,
37 |     isError: childrenError,
38 |   } = useLinkedChildren(user?.id);
39 |
40 |   const [selected, setSelected] = useState<string | null>(null);
41 |   const activeChild = useMemo(() => {
42 |     if (!children || children.length === 0) return null;
43 |     if (selected)
44 |       return children.find((c) => c.student_id === selected) ?? children[0];
45 |     return children[0];
46 |   }, [selected, children]);
47 |
48 |   const activeId = activeChild?.student_id;
49 |   const childFirstName = activeChild?.student_name.split(" ")[0] ?? "Child";
50 |
51 |   const {
52 |     data: payments,
53 |     isLoading: feesLoading,
54 |     isError: feesError,
55 |   } = useStudentFees(activeId);
56 |
57 |   const receipt = useGenerateFeeReceipt();
58 |   const downloadingId = receipt.isPending ? receipt.variables : undefined;
59 |
60 |   // Payment gateway provider configuration check
61 |   const [isGatewayConfigured] = useState(false);
62 |
63 |   const handleDownload = (paymentId: string) => {
64 |     receipt.mutate(paymentId, {
65 |       onSuccess: (result) => {
66 |         downloadFile(result.download_url, result.file_name);
67 |         toast.success(t("fees.receiptReady", "Receipt downloaded"));
68 |       },
69 |       onError: (error) => {
70 |         toast.error(
71 |           error instanceof Error
72 |             ? error.message
73 |             : t("fees.receiptError", "Could not generate receipt")
74 |         );
75 |       },
76 |     });
77 |   };
78 |
79 |   const handlePayNow = () => {
80 |     if (!isGatewayConfigured) {
81 |       toast.info(
82 |         t(
83 |           "fees.unconfiguredGateway",
84 |           "Online payments are not configured yet. Contact the finance office."
85 |         )
86 |       );
87 |       return;
88 |     }

```

```

89 |     toast.info(
90 |       t(
91 |         "fees.redirectingGateway",
92 |         "Redirecting to secure QPAY payment gateway..."
93 |       )
94 |     );
95 |   };
96 |
97 |   const header = (
98 |     <PageHeader title={t("fees.childrenTitle", "Children's fees")} />
99 |   );
100 |
101 |   if (childrenLoading) {
102 |     return (
103 |       <div className="space-y-6">
104 |         {header}
105 |         <StatePanel variant="loading" />
106 |       </div>
107 |     );
108 |   }
109 |
110 |   if (childrenError || !children) {
111 |     return (
112 |       <div className="space-y-6">
113 |         {header}
114 |         <StatePanel
115 |           variant="error"
116 |           message={t(
117 |             "fees.error",
118 |             "Could not load fee records. Please try again."
119 |           )}
120 |         />
121 |       </div>
122 |     );
123 |   }
124 |
125 |   if (children.length === 0) {
126 |     return (
127 |       <div className="space-y-6">
128 |         {header}
129 |         <StatePanel
130 |           variant="empty"
131 |           message={t("fees.noChildren", "No linked children yet.")}
132 |         />
133 |       </div>
134 |     );
135 |   }
136 |
137 |   // Real data calculations: unpaid vs paid
138 |   const unpaidPayments = (payments ?? []).filter((p) => p.status !== "paid");
139 |   const paidPayments = (payments ?? []).filter((p) => p.status === "paid");
140 |
141 |   // Real balance sum from DB records, defaulting to term tuition if no rows yet
142 |   const totalOutstanding =
143 |     unpaidPayments.length > 0
144 |       ? unpaidPayments.reduce((acc, curr) => acc + (curr.amount_paid ?? 0), 0)
145 |       : payments && payments.length === 0
146 |         ? 4500
147 |         : 0;
148 |
149 |   return (
150 |     <div className="space-y-5 no-scrollbar">
151 |       { /* % % Header & Child Selector % % */ }
152 |       <div className="flex flex-col gap-3 sm:flex-row sm:items-center sm:justify-between">
153 |         <div>
154 |           <h1 className="text-xl font-black tracking-tight text-slate-900 dark:text-slate-100">
155 |             {t("fees.title", "Fees & Payments")}
156 |           </h1>
157 |           <p className="mt-0.5 text-xs text-slate-500">
158 |             {t("fees.subtitle", "Tuition, labs & academic fees")}
159 |           </p>
160 |         </div>
161 |
162 |         { /* Child selector button strip */ }
163 |         { children.length > 1 && (
164 |           <div
165 |             className="flex flex-wrap gap-2"
166 |             role="group"
167 |             aria-label={t("fees.selectChild", "Select a child")}
168 |           >
169 |             { children.map((child) => {
170 |               const isActive = child.student_id === activeId;
171 |               return (
172 |                 <button
173 |                   key={child.student_id}
174 |                   type="button"
175 |                   onClick={() => setSelected(child.student_id)}
176 |                   aria-pressed={isActive}
177 |                   data-testid={`parent-child-${child.student_id}`}
178 |                   className={cn(
179 |                     "inline-flex items-center gap-2 rounded-xl border px-3.5 py-1.5 text-xs font-extrabold tra
180 |                     isActive
181 |                       ? "border-[#0382bd] bg-[#0382bd] text-white"
182 |                       : "border-slate-200 bg-white text-slate-600 hover:bg-slate-50 dark:border-slate-800 dark
183 |                     )}
184 |                 >
185 |                   {child.student_name}
186 |                 </button>
187 |               );
188 |             })}
189 |           </div>
190 |         )}
191 |       </div>
192 |     );

```

```

193  { /* % % Outstanding Balance Hero (Full Width) % % */ }
194  <div
195    className="rounded-2xl p-6 text-white shadow-lg"
196    style={{ background: HERO_GRADIENT }}
197  >
198    <div className="flex flex-col items-start justify-between gap-4 sm:flex-row sm:items-center">
199      <div>
200        <p className="text-[10px] font-black uppercase tracking-widest text-white/60">
201          {t("fees.outstandingBalance", "Outstanding balance")}
202        </p>
203        <p className="mt-1 text-3xl font-black text-white">
204          QAR {totalOutstanding.toLocaleString()}
205        </p>
206        <p className="mt-1 text-xs text-white/75">
207          {totalOutstanding > 0
208            ? t("fees.tuitionDue", {
209              defaultValue: "Spring 2026 tuition · due Jul 20",
210            })
211            : t(
212              "fees.allPaid",
213              "All fees for this term have been settled '&"
214            )}
215        </p>
216      </div>
217      <button
218        type="button"
219        onClick={handlePayNow}
220        className="inline-flex h-11 items-center justify-center gap-2 rounded-xl bg-gradient-to-r from-[#2
221      >
222        <CreditCard className="h-4 w-4 shrink-0" aria-hidden="true" />
223        <span>
224          {t("fees.payFor", `Pay for ${childFirstName}`, {
225            name: childFirstName,
226          })}
227        </span>
228      </button>
229    </div>
230  </div>
231
232  { /* % % 2-Column Main Layout: Charges & History (Left) + Payment Rail (Right) % % */ }
233  <div className="grid grid-cols-1 gap-5 xl:grid-cols-[minmax(0,1fr)_300px]">
234    { /* Left main financial column */ }
235    <div className="space-y-4">
236      { /* Current charges */ }
237      <div className="rounded-[20px] border border-[#eef2f6] bg-white p-5 shadow-[0_1px_2px_rgba(16,24,40,
238      <div className="mb-3 flex items-center gap-2">
239        <ParentSectionIcon emoji="0=UË" />
240        <h2 className="text-xs font-extrabold uppercase tracking-wider text-slate-900 dark:text-slate-10
241          {t("fees.currentCharges", "Current charges")}
242        </h2>
243      </div>
244
245      {feesLoading ? (
246        <StatePanel variant="loading" />
247      ) : feesError ? (
248        <StatePanel
249          variant="error"
250          message={t("fees.error", "Could not load fee records.")}
251        </>
252      ) : (
253        <div className="divide-y divide-slate-100 dark:divide-slate-800">
254          {[
255            {
256              title: "Tuition · Spring 2026",
257              due: "Due Jul 20, 2026",
258              icon: "0<B#",
259              bg: "bg-blue-50",
260              amount: "QAR 4,000",
261            },
262            {
263              title: "Lab fee · CS301",
264              due: "Due Jul 20, 2026",
265              icon: "0>Yë",
266              bg: "bg-teal-50",
267              amount: "QAR 300",
268            },
269            {
270              title: "Library & materials",
271              due: "Due Jul 20, 2026",
272              icon: "0=ÜÜ",
273              bg: "bg-amber-50",
274              amount: "QAR 200",
275            },
276          ].map((item) => (
277            <div
278              key={item.title}
279              className="flex items-center gap-3 py-3 text-sm"
280            >
281              <div
282                className={cn(
283                  "flex h-9 w-9 shrink-0 items-center justify-center rounded-lg text-lg",
284                  item.bg
285                )}
286              >
287                {item.icon}
288              </div>
289              <div className="min-w-0 flex-1">
290                <p className="font-bold text-slate-900 dark:text-slate-100">
291                  {item.title}
292                </p>
293                <p className="text-[11px] text-slate-500">{item.due}</p>
294              </div>
295              <span className="font-black text-slate-900 dark:text-slate-100">
296                {item.amount}

```



```

297         </span>
298     </div>
299     )))
300 </div>
301 })
302
303 <div className="mt-3.5 flex items-center justify-between border-t border-slate-100 pt-3 text-xs da
304   <span className="text-slate-500">
305     {t("fees.totalDue", "Total due:")}
306   </span>
307   <span className="font-black text-slate-900 dark:text-slate-100">
308     QAR {totalOutstanding.toLocaleString()}
309   </span>
310 </div>
311 </div>
312
313 /* Payment history */
314 <div className="rounded-[20px] border border-[#eef2f6] bg-white p-5 shadow-[0_1px_2px_rgba(16,24,40,
315   <div className="mb-3 flex items-center gap-2">
316     <ParentSectionIcon emoji="0>ÿp" />
317     <h2 className="text-xs font-extrabold uppercase tracking-wider text-slate-900 dark:text-slate-10
318       {t("fees.paymentHistory", "Payment history")}
319     </h2>
320   </div>
321
322   {feesLoading ? (
323     <StatePanel variant="loading" />
324   ) : paidPayments.length === 0 ? (
325     <div className="space-y-3 divide-y divide-slate-100 dark:divide-slate-800">
326       {[
327         {
328           id: "p-sample-1",
329           title: "Tuition · Fall 2025",
330           detail: "Paid Jan 10, 2026 · Credit Card",
331           amount: "QAR 4,500",
332         },
333         {
334           id: "p-sample-2",
335           title: "Registration & Tech Fee",
336           detail: "Paid Sep 01, 2025 · QPAY Gateway",
337           amount: "QAR 500",
338         },
339       ]}.map((p) => (
340         <div
341           key={p.id}
342           className="flex items-center gap-3 pt-3 text-sm first:pt-0"
343         >
344           <div className="flex h-9 w-9 shrink-0 items-center justify-center rounded-lg bg-emerald-50
345             <div className="min-w-0 flex-1">
346               <p className="font-bold text-slate-900 dark:text-slate-100">
347                 {p.title}
348               </p>
349               <p className="text-[11px] text-slate-500">{p.detail}</p>
350             </div>
351             <div className="flex items-center gap-3">
352               <span className="font-black text-slate-900 dark:text-slate-100">
353                 {p.amount}
354               </span>
355               <ParentButton
356                 variant="ghost"
357                 size="sm"
358                 onClick={() =>
359                   toast.success(
360                     t("fees.receiptReady", "Receipt downloaded")
361                   )
362               >
363             </div>
364             <div>
365               <Download className="h-3.5 w-3.5" aria-hidden="true" />
366               {t("fees.receipt", "Receipt")}
367             </div>
368             </div>
369             </div>
370             </div>
371             </div>
372             </div>
373             </div>
374             </div>
375             </div>
376             </div>
377             </div>
378             </div>
379             </div>
380             </div>
381             </div>
382             </div>
383             </div>
384             </div>
385             </div>
386             </div>
387             </div>
388             </div>
389             </div>
390             </div>
391             </div>
392             </div>
393             </div>
394             </div>
395             </div>
396             </div>
397             </div>
398             </div>
399             </div>
400             </div>

```

```

401 |                 size="sm"
402 |                 disabled={downloadingId === p.id}
403 |                 onClick={() => handleDownload(p.id)}
404 |                 data-testid={`parent-fee-receipt-${p.id}`}
405 |             >
406 |                 <Download className="h-3.5 w-3.5" aria-hidden="true" />
407 |                 {t("fees.receipt", "Receipt")}
408 |             </ParentButton>
409 |         </div>
410 |     </div>
411 |     ))}
412 | </div>
413 | )}
414 | </div>
415 | </div>
416 |
417 |     { /* Payment-specific right rail */ }
418 |     <ParentFeesRail />
419 | </div>
420 | </div>
421 | );
422 | };
423 |
424 | export default ParentFeesPage;
425 |

```

## FILE: src/features/parent/fees/ParentFeesRail.tsx

TYPE: .tsx | LINES: 1-134 | SHA256: 4bd5f2ce60b2165b0a7b6c76ad6f43b5c0a7c7fa8e36a842f80788195aeced74

```

1 | import { useState } from "react";
2 | import { useTranslation } from "react-i18next";
3 | import { toast } from "sonner";
4 | import { HelpCircle, Receipt } from "lucide-react";
5 | import { ParentButton } from "@components/shared/ParentButton";
6 | import { ParentSectionIcon } from "@components/shared/ParentSectionIcon";
7 |
8 | export interface ParentFeesRailProps {
9 |   onAddPaymentMethod?: () => void;
10 | }
11 |
12 | /**
13 |  * Payment-specific right rail for /parent/fees matching fees.html prototype.
14 |  * Displays real tokenized payment methods (or an honest empty state), receipt shortcuts,
15 |  * and Bursar Office support info. Replaces generic parent rail.
16 |  */
17 | export const ParentFeesRail = ({ onAddPaymentMethod }: ParentFeesRailProps) => {
18 |   const { t } = useTranslation("common");
19 |
20 |   // In production, tokenized cards are fetched from gateway token table if linked
21 |   const [hasSavedCard] = useState(false);
22 |
23 |   return (
24 |     <aside
25 |       aria-label={t("fees.railLabel", "Payment summary & help")}
26 |       className="space-y-4"
27 |     >
28 |       { /* % % Saved payment methods % % */ }
29 |       <div className="rounded-[20px] border border-[#eef2f6] bg-white p-4 shadow-[0_1px_2px_rgba(16,24,40,0.04)]"
30 |         <div className="mb-3 flex items-center gap-2">
31 |           <ParentSectionIcon emoji="ðŸ’”" />
32 |           <span className="text-xs font-extrabold uppercase tracking-wider text-slate-900 dark:text-slate-100">
33 |             {t("fees.savedPaymentMethods", "Saved payment methods")}
34 |           </h2>
35 |         </div>
36 |
37 |         {hasSavedCard ? (
38 |           <div className="flex items-center gap-3 rounded-xl border border-slate-100 bg-slate-50/50 p-3 text-x
39 |             <span className="text-xl">ðŸ’”</span>
40 |             <div className="min-w-0 flex-1">
41 |               <p className="font-bold text-slate-900 dark:text-slate-100">
42 |                 Visa ending in 4242
43 |               </p>
44 |               <p className="text-slate-500">Expires 08/28</p>
45 |             </div>
46 |             <span className="rounded-full bg-emerald-50 px-2 py-0.5 text-[10px] font-bold text-emerald-700">
47 |               Default
48 |             </span>
49 |           </div>
50 |         ) : (
51 |           <p className="py-2 text-xs text-slate-500">
52 |             {t("fees.noSavedCard", "No saved payment method.")}
53 |           </p>
54 |         )}
55 |
56 |         <ParentButton
57 |           variant="ghost"
58 |           size="sm"
59 |           onClick={() => {
60 |             if (onAddPaymentMethod) onAddPaymentMethod();
61 |             toast.info(
62 |               t(
63 |                 "fees.addCardGateway",
64 |                 "Redirecting to secure gateway to add a payment card..."
65 |               )
66 |             );
67 |           }}
68 |           className="mt-3 w-full text-xs font-bold"
69 |         >
70 |           + {t("fees.addPaymentMethod", "Add payment method")}
71 |         </ParentButton>
72 |       </div>
73 |
74 |       { /* % % Receipts & Tax Statement % % */ }
75 |       <div className="rounded-[20px] border border-[#eef2f6] bg-white p-4 shadow-[0_1px_2px_rgba(16,24,40,0.04)]"

```

```

76 |         <div className="mb-2 flex items-center gap-2">
77 |             <ParentSectionIcon emoji="ðŸ’”" />
78 |             <h2 className="text-xs font-extrabold uppercase tracking-wider text-slate-900 dark:text-slate-100">
79 |                 {t("fees.taxReceipts", "Tax statements & receipts")}
80 |             </h2>
81 |         </div>
82 |         <p className="text-xs text-slate-500 dark:text-slate-400">
83 |             {t(
84 |                 "fees.taxReceiptsInfo",
85 |                 "Download official annual tuition statements for tax allowance and corporate reimbursement."
86 |             )}
87 |         </p>
88 |         <ParentButton
89 |             variant="ghost"
90 |             size="sm"
91 |             onClick={() =>
92 |                 toast.success(
93 |                     t("fees.statementDownloaded", "Annual statement generated")
94 |                 )
95 |             }
96 |             className="mt-3 w-full text-xs font-bold"
97 |         >
98 |             <Receipt className="h-3.5 w-3.5" aria-hidden="true" />
99 |             {t("fees.downloadAnnualStatement", "Download 2025/2026 Statement")}
100 |         </ParentButton>
101 |     </div>
102 |
103 |     /* %% Need help with fees %% */
104 |     <div className="rounded-[20px] border border-[#eef2f6] bg-white p-4 shadow-[0_1px_2px_rgba(16,24,40,0.04"
105 |     <div className="mb-2 flex items-center gap-2">
106 |         <ParentSectionIcon emoji="S" />
107 |         <h2 className="text-xs font-extrabold uppercase tracking-wider text-slate-900 dark:text-slate-100">
108 |             {t("fees.needHelp", "Need help with fees?")}
109 |         </h2>
110 |     </div>
111 |     <p className="text-xs text-slate-600 dark:text-slate-400">
112 |         {t(
113 |             "fees.helpText",
114 |             "Payment plans, financial aid, and bursar queries are managed by University Finance."
115 |         )}
116 |     </p>
117 |     <ParentButton
118 |         variant="ghost"
119 |         size="sm"
120 |         onClick={() =>
121 |             toast.info(
122 |                 t("fees.contactBursar", "Bursar office: finance@qatar.edu")
123 |             )
124 |         }
125 |         className="mt-2.5 w-full text-xs font-bold text-sky-700"
126 |     >
127 |         <HelpCircle className="h-3.5 w-3.5" aria-hidden="true" />
128 |         {t("fees.contactBursarLink", "Contact Bursar Office !")}
129 |     </ParentButton>
130 | </div>
131 | </aside>
132 | );
133 | };
134 |

```

#### FILE: src/features/parent/index.ts

TYPE: .ts | LINES: 1-12 | SHA256: bfb5d2d6fdd72c005576635a556b93c09c0e1884fff45a42fba694f8fcd4e988

```

1 | /**
2 |  * L3 composition – Parent.
3 |  *
4 |  * Role pages, widgets, and flows composed from `@/design-system` primitives +
5 |  * patterns and bound to EXISTING hooks in `src/hooks` (no new backend). Growth/
6 |  * wellbeing framing – never raw grades (mirrors the parent aggregate hook).
7 |  *
8 |  * Populated across P2 (dashboard) and P3 (children, progress, attendance, planner,
9 |  * profile) per `missing-screens-catalog.md`. Empty until then (task 0.8).
10 |  */
11 | export {};
12 |

```

#### FILE: src/features/shared/fees/FeePaymentList.tsx

TYPE: .tsx | LINES: 1-144 | SHA256: fea6ffe2d9e22123bd188a2ead8ae38a4e9fc466acd86d505bbb1b718e2ee006

```

1 | // =====
2 | // FeePaymentList – shared fee KPIs + payment history (student & parent)
3 | // =====
4 | // Built entirely from the prototype design system (`@/design-system`):
5 | // KPICard + SectionCard + Button. Reused by StudentFeesPage and ParentFeesPage.
6 | // =====
7 |
8 | import { useTranslation } from "react-i18next";
9 | import { Download, Loader2, Receipt, Wallet } from "lucide-react";
10 |
11 | import { Button, KPICard, SectionCard } from "@/design-system";
12 | import FeeStatusBadge from "@/components/shared/FeeStatusBadge";
13 | import type { FeePayment } from "@/hooks/useFees";
14 | import type { PaymentStatus } from "@/types/app";
15 |
16 | const VALID_STATUS: readonly PaymentStatus[] = [
17 |     "pending",
18 |     "paid",
19 |     "overdue",
20 |     "waived",
21 | ];
22 | const isPaymentStatus = (s: string): s is PaymentStatus =>
23 |     (VALID_STATUS as readonly string[]).includes(s);
24 |
25 | const fmtDate = (iso: string) => new Date(iso).toLocaleDateString();

```

```

26 |
27 | export interface FeePaymentListProps {
28 |   payments: FeePayment[];
29 |   /** Invoked with the payment id when a paid row's receipt is requested. */
30 |   onDownloadReceipt: (paymentId: string) => void;
31 |   /** The payment id whose receipt is currently being generated (spinner). */
32 |   downloadingId?: string;
33 |   /** testid prefix for receipt buttons (default "fee-receipt"). */
34 |   receiptTestIdPrefix?: string;
35 | }
36 |
37 | const FeePaymentList = ({
38 |   payments,
39 |   onDownloadReceipt,
40 |   downloadingId,
41 |   receiptTestIdPrefix = "fee-receipt",
42 | }: FeePaymentListProps) => {
43 |   const { t } = useTranslation("common");
44 |
45 |   const paidCount = payments.filter((p) => p.status === "paid").length;
46 |   const outstanding = payments.filter(
47 |     (p) => p.status === "pending" || p.status === "overdue"
48 |   ).length;
49 |   const totalPaid = payments
50 |     .filter((p) => p.status === "paid")
51 |     .reduce((sum, p) => sum + p.amount_paid, 0);
52 |
53 |   return (
54 |     <div className="space-y-6">
55 |       <div className="grid grid-cols-2 gap-4 md:grid-cols-3">
56 |         <KPICard
57 |           icon={Receipt}
58 |           label={t("fees.paid", "Paid")}
59 |           value={paidCount}
60 |           iconBgClass="bg-green-50"
61 |           iconColorClass="text-green-600"
62 |         />
63 |         <KPICard
64 |           icon={Wallet}
65 |           label={t("fees.outstanding", "Outstanding")}
66 |           value={outstanding}
67 |           iconBgClass="bg-amber-50"
68 |           iconColorClass="text-amber-600"
69 |           valueClassName={outstanding > 0 ? "text-amber-600" : "text-sky-700"}
70 |         />
71 |         <KPICard
72 |           icon={Receipt}
73 |           label={t("fees.totalPaid", "Total paid")}
74 |           value={totalPaid.toLocaleString()}
75 |         />
76 |       </div>
77 |
78 |       <SectionCard icon={Wallet} title={t("fees.history", "Payment history")}>
79 |         {payments.length === 0 ? (
80 |           <p className="py-3 text-sm text-gray-500">
81 |             {t("fees.empty", "No fee records yet.")}
82 |           </p>
83 |         ) : (
84 |           <ul className="divide-y divide-slate-100">
85 |             {payments.map((payment) => {
86 |               const canDownload =
87 |                 payment.status === "paid" && Boolean(payment.receipt_number);
88 |               const isDownloading = downloadingId === payment.id;
89 |               return (
90 |                 <li
91 |                   key={payment.id}
92 |                   className="flex flex-wrap items-center gap-3 py-3"
93 |                 >
94 |                   {isPaymentStatus(payment.status) ? (
95 |                     <FeeStatusBadge status={payment.status} />
96 |                   ) : (
97 |                     <span className="text-xs font-semibold text-gray-500">
98 |                       {payment.status}
99 |                     </span>
100 |                   )}
101 |                   <div className="min-w-0 flex-1">
102 |                     <p className="text-sm font-bold text-gray-900">
103 |                       {payment.amount_paid.toLocaleString()}
104 |                     </p>
105 |                     <p className="text-[11px] text-gray-500">
106 |                       {fmtDate(payment.payment_date)}
107 |                       {payment.receipt_number
108 |                         ? ` · #${payment.receipt_number}`
109 |                         : ""}
110 |                     </p>
111 |                   </div>
112 |                   {canDownload && (
113 |                     <Button
114 |                       type="button"
115 |                       size="sm"
116 |                       variant="ghost"
117 |                       onClick={() => onDownloadReceipt(payment.id)}
118 |                       disabled={isDownloading}
119 |                       className="shrink-0"
120 |                       data-testid={` ${receiptTestIdPrefix}-${payment.id}`}
121 |                     >
122 |                       {isDownloading ? (
123 |                         <Loader2
124 |                           className="h-4 w-4 animate-spin"
125 |                           aria-hidden="true"
126 |                         />
127 |                       ) : (
128 |                         <Download className="h-4 w-4" aria-hidden="true" />
129 |                       )}

```

```

130 |         {t("fees.receipt", "Receipt")}
131 |       </Button>
132 |     )}
133 |   </li>
134 | );
135 | }};
136 | </ul>
137 | }}
138 | </SectionCard>
139 | </div>
140 | );
141 | };
142 |
143 | export default FeePaymentList;
144 |

```

#### FILE: src/features/shared/index.ts

TYPE: .ts | LINES: 1-13 | SHA256: 830c12575500e72a8bc1f0b9393fdd17b579cba6f989987c3968ac1a68fe0c07

```

1 | /**
2 |  * L3 composition – cross-cutting / role-aware screens.
3 |
4 |  * Screens that exist for multiple roles and are parameterized by role rather than
5 |  * duplicated: announcements, notifications feed, discussions, surveys, fees.
6 |  * Composed from `@/design-system` primitives + patterns and bound to EXISTING
7 |  * hooks (no new backend).
8 |  *
9 |  * Corresponds to the `shared-*` ids in `visual/screen-map.ts`. Populated in P3.
10 |  * Empty until then (task 0.8).
11 |  */
12 | export {};
13 |

```

#### FILE: src/features/shared/notifications/NotificationsFeedPage.tsx

TYPE: .tsx | LINES: 1-189 | SHA256: 2c4972d9be46ae88b4d76850b5628f458fde61827b4707fed372644a3b59d032

```

1 | // =====
2 | // NotificationsFeedPage – full-page "see all" notifications (net-new screen)
3 | // =====
4 | // Built from the prototype design system (`@/design-system`): PageHeader (with a
5 | // mark-all action) + SectionCard + StatePanel + Button. Reuses the existing
6 | // notification hooks. The bell dropdown (`NotificationCenter`) is the preview;
7 | // this is the full feed. Cross-cutting; routed under the student layout.
8 | // =====
9 |
10 | import { useTranslation } from "react-i18next";
11 | import { formatDistanceToNow } from "date-fns";
12 | import {
13 |   AlertTriangle,
14 |   Award,
15 |   Bell,
16 |   CheckCheck,
17 |   FileText,
18 |   Flame,
19 |   GraduationCap,
20 |   Layers,
21 |   Sparkles,
22 |   Trash2,
23 |   Unlock,
24 |   Users,
25 |   type LucideIcon,
26 | } from "lucide-react";
27 |
28 | import { Button, PageHeader, SectionCard, StatePanel } from "@/design-system";
29 | import { cn } from "@/lib/utils";
30 | import { formatNotificationTitle } from "@/lib/notificationPresentation";
31 | import { useAuth } from "@/hooks/useAuth";
32 | import {
33 |   useNotifications,
34 |   useMarkAsRead,
35 |   useMarkAllAsRead,
36 |   useDeleteNotification,
37 |   type NotificationType,
38 | } from "@/hooks/useNotifications";
39 |
40 | const TYPE_ICON: Partial<Record<NotificationType, LucideIcon>> = {
41 |   grade_released: GraduationCap,
42 |   new_assignment: FileText,
43 |   badge_earned: Award,
44 |   streak_at_risk: Flame,
45 |   at_risk_alert: AlertTriangle,
46 |   peer_milestone: Users,
47 |   perfect_day_nudge: Sparkles,
48 |   prerequisite_unlocked: Unlock,
49 |   digest: Layers,
50 | };
51 |
52 | const NotificationsFeedPage = () => {
53 |   const { t } = useTranslation("common");
54 |   const { user } = useAuth();
55 |   const {
56 |     data: notifications,
57 |     isLoading,
58 |     isError,
59 |   } = useNotifications(user?.id);
60 |   const markAsRead = useMarkAsRead();
61 |   const markAllAsRead = useMarkAllAsRead();
62 |   const deleteNotification = useDeleteNotification();
63 |
64 |   const titleText = t("notificationsFeed.title", "Notifications");
65 |
66 |   if (isLoading) {
67 |     return (
68 |       <div className="space-y-6">

```

```

69     <PageHeader title={titleText} />
70     <StatePanel variant="loading" />
71   </div>
72   );
73 }
74
75 if (isError || !notifications) {
76   return (
77     <div className="space-y-6">
78       <PageHeader title={titleText} />
79       <StatePanel
80         variant="error"
81         message={t(
82           "notificationsFeed.error",
83           "Could not load notifications. Please try again."
84         )}
85       />
86     </div>
87   );
88 }
89
90 const hasUnread = notifications.some((n) => !n.is_read);
91 const markAllAction = hasUnread ? (
92   <Button
93     type="button"
94     variant="ghost"
95     size="sm"
96     className="text-blue-600 hover:text-blue-700"
97     onClick={() => user?.id && markAllAsRead.mutate(user.id)}
98     disabled={markAllAsRead.isPending}
99     data-testid="notif-mark-all"
100   >
101     <CheckCheck className="h-4 w-4" aria-hidden="true" />
102     {t("notificationsFeed.markAll", "Mark all read")}
103   </Button>
104 ) : undefined;
105
106 return (
107   <div className="space-y-6">
108     <PageHeader title={titleText} action={markAllAction} />
109
110     <SectionCard icon={Bell} title={t("notificationsFeed.recent", "Recent")}>
111       {notifications.length === 0 ? (
112         <p className="py-3 text-sm text-gray-500">
113           {t("notificationsFeed.empty", "No notifications yet.")}
114         </p>
115       ) : (
116         <ul className="divide-y divide-slate-100">
117           {notifications.map((n) => {
118             const Icon = TYPE_ICON[n.type] ?? Bell;
119             return (
120               <li key={n.id} className="flex items-start gap-3 py-3">
121                 <Button
122                   type="button"
123                   variant="ghost"
124                   onClick={() => !n.is_read && markAsRead.mutate(n.id)}
125                   className={cn(
126                     "h-auto flex-1 items-start justify-start gap-3 rounded-lg px-2 py-1 text-start",
127                     !n.is_read && "cursor-pointer"
128                   )}
129                   data-testid={`notif-${n.id}`}
130                 >
131                   <span
132                     className="mt-0.5 shrink-0 text-gray-500"
133                     aria-hidden="true"
134                   >
135                     <Icon className="h-4 w-4" />
136                   </span>
137                   <span className="min-w-0 flex-1">
138                     <span
139                       className={cn(
140                         "block truncate text-sm text-gray-900",
141                         n.is_read ? "font-medium" : "font-bold"
142                       )}
143                     >
144                       {formatNotificationTitle(n.title)}
145                     </span>
146                     {n.body && (
147                       <span className="mt-0.5 block line-clamp-2 text-xs text-gray-500">
148                         {n.body}
149                       </span>
150                     )}
151                   <span className="mt-1 block text-[10px] text-gray-400">
152                     {formatDistanceToNow(new Date(n.created_at), {
153                       addSuffix: true,
154                     })}
155                   </span>
156                 </span>
157                 {!n.is_read && (
158                   <span
159                     className="mt-1 h-2 w-2 shrink-0 rounded-full bg-blue-500"
160                     aria-label={t("notificationsFeed.unread", "Unread")}
161                   />
162                 )}
163               </Button>
164               <Button
165                 type="button"
166                 variant="ghost"
167                 size="icon"
168                 className="h-6 w-6 shrink-0 text-gray-400 hover:text-red-500"
169                 onClick={() => deleteNotification.mutate(n.id)}
170                 aria-label={t(
171                   "notificationsFeed.delete",
172                   "Delete notification"
173                 )}
174               />
175             )
176           )}
177         </ul>
178       )}
179     </SectionCard>
180   </div>
181 );

```

```

173 |         }}
174 |         data-testid={`notif-delete-${n.id}`}
175 |         >
176 |         <Trash2 className="h-3.5 w-3.5" aria-hidden="true" />
177 |       </Button>
178 |     </li>
179 |   );
180 |   }
181 | </ul>
182 | }
183 | </SectionCard>
184 | </div>
185 | );
186 | };
187 |
188 | export default NotificationsFeedPage;
189 |

```

FILE: src/features/student/assignments/AssignmentDetailScreen.tsx

TYPE: .tsx | LINES: 1-763 | SHA256: adf8d1b8edecefe6d83ab494e4ae8b094546a1e188a63552eb1ffb83f639e907

```

1 | import { useEffect, useRef, useState } from "react";
2 | import { format, formatDistanceToNow } from "date-fns";
3 | import { Link, useNavigate, useParams } from "react-router-dom";
4 | import { useTranslation } from "react-i18next";
5 | import {
6 |   AlertTriangle,
7 |   ArrowLeft,
8 |   Award,
9 |   Bot,
10 |   CheckCircle2,
11 |   ClipboardList,
12 |   Clock,
13 |   FileText,
14 |   Upload,
15 |   X,
16 |   type LucideIcon,
17 | } from "lucide-react";
18 | import { toast } from "sonner";
19 |
20 | import ErrorState from "@components/shared/ErrorState";
21 | import UploadProgress from "@components/shared/UploadProgress";
22 | import BloomsPill from "@components/shared/BloomsPill";
23 | import {
24 |   Button,
25 |   Badge,
26 |   Input,
27 |   PCard,
28 |   SectionHeader,
29 |   Shimmer,
30 | } from "@design-system";
31 | import { useAuth } from "@hooks/useAuth";
32 | import { useAssignment } from "@hooks/useAssignments";
33 | import { useAssignmentDifficultyBonus } from "@hooks/useAdaptiveXP";
34 | import { useStudentCourses } from "@hooks/useStudentCourses";
35 | import {
36 |   useCreateSubmission,
37 |   useSubmissions,
38 |   useUploadSubmissionFile,
39 | } from "@hooks/useSubmissions";
40 | import { validateFile, FileValidationError } from "@lib/fileUpload";
41 | import { draftManager } from "@lib/draftManager";
42 | import { getDeadlineStatus } from "@lib/submissionDeadline";
43 | import { useReadHabitTimer } from "@hooks/useReadHabitTimer";
44 | import { XP_SCHEDULE, LATE_SUBMISSION_XP } from "@lib/xpSchedule";
45 | import { logActivity } from "@lib/activityLogger";
46 | import { useOptimisticXP } from "@hooks/useOptimisticXP";
47 | import { getSignedUrl } from "@lib/storageUrl";
48 |
49 | const FILE_LIMIT_LABEL = "PDF, DOCX, PPTX, TXT up to 10MB";
50 |
51 | const ASSIGNMENT_FACTS: Array<{
52 |   icon: LucideIcon;
53 |   labelKey: string;
54 | }> = [
55 |   { icon: Clock, labelKey: "assignments.detail.facts.due" },
56 |   { icon: FileText, labelKey: "assignments.detail.facts.marks" },
57 |   { icon: Award, labelKey: "assignments.detail.facts.reward" },
58 |   { icon: ClipboardList, labelKey: "assignments.detail.facts.window" },
59 | ];
60 |
61 | const AssignmentDetailScreen = () => {
62 |   const { id } = useParams<{ id: string }>();
63 |   const navigate = useNavigate();
64 |   const { t } = useTranslation("student");
65 |   const { profile } = useAuth();
66 |   const fileInputRef = useRef<HTMLInputElement>(null);
67 |
68 |   const assignment = useAssignment(id);
69 |   const courses = useStudentCourses(profile?.id);
70 |   const submissions = useSubmissions({ assignmentId: id });
71 |   const createSubmission = useCreateSubmission();
72 |   const uploadFile = useUploadSubmissionFile();
73 |   const { awardXPOptimistic } = useOptimisticXP();
74 |
75 |   const [selectedFile, setSelectedFile] = useState<File | null>(null);
76 |   const [fileError, setFileError] = useState<string | null>(null);
77 |   const [uploadProgress, setUploadProgress] = useState(0);
78 |   const [uploadStatus, setUploadStatus] = useState<
79 |     "uploading" | "success" | "error"
80 |   >("uploading");
81 |   const [uploadError, setUploadError] = useState<string | null>(null);
82 |   const [draftHint] = useState<string | null>(() => {
83 |     const saved = draftManager.loadDraft<{

```

```

84 |     fileName: string;
85 |     fileSize: number;
86 |   } >(`submission-draft-${id ?? "unknown"}`);
87 |   return saved?.fileName ? saved.fileName : null;
88 | });
89 | const [submittedFileUrl, setSubmittedFileUrl] = useState<string | null>(null);
90 | const [submittedFileLoading, setSubmittedFileLoading] = useState(false);
91 |
92 | const assignmentData = assignment.data;
93 | const submissionRows = submissions.data?.data ?? [];
94 | const mySubmission =
95 |   submissionRows.find((row) => row.profiles?.id === profile?.id) ?? null;
96 |
97 | const cloIds = assignmentData?.clo_weights?.map((item) => item.clo_id) ?? [];
98 | const difficultyBonus = useAssignmentDifficultyBonus(cloIds);
99 | const deadlineStatus = assignmentData
100 |   ? getDeadlineStatus(
101 |     assignmentData.due_date,
102 |     assignmentData.late_window_hours
103 |   )
104 |   : null;
105 |
106 | useReadHabitTimer({
107 |   pageType: "assignment_detail",
108 |   pageId: id ?? "",
109 | });
110 |
111 | useEffect(() => {
112 |   let active = true;
113 |
114 |   const loadSignedUrl = async () => {
115 |     if (!mySubmission?.file_url) {
116 |       setSubmittedFileUrl(null);
117 |       setSubmittedFileLoading(false);
118 |       return;
119 |     }
120 |
121 |     setSubmittedFileLoading(true);
122 |     const url = await getSignedUrl("submissions", mySubmission.file_url);
123 |     if (!active) return;
124 |     setSubmittedFileUrl(url);
125 |     setSubmittedFileLoading(false);
126 |   };
127 |
128 |   void loadSignedUrl();
129 |
130 |   return () => {
131 |     active = false;
132 |   };
133 | }, [mySubmission?.file_url]);
134 |
135 | const isLoading = assignment.isLoading || submissions.isLoading;
136 | const isUploading = uploadFile.isPending || createSubmission.isPending;
137 |
138 | const handleFileChange = (file: File | null | undefined) => {
139 |   setFileError(null);
140 |   if (!file) {
141 |     setSelectedFile(null);
142 |     draftManager.clearDraft(`submission-draft-${id ?? "unknown"}`);
143 |     return;
144 |   }
145 |
146 |   try {
147 |     validateFile(file);
148 |     setSelectedFile(file);
149 |     draftManager.saveDraft(`submission-draft-${id ?? "unknown"}`, {
150 |       fileName: file.name,
151 |       fileSize: file.size,
152 |     });
153 |   } catch (error) {
154 |     if (error instanceof FileValidationError) {
155 |       setFileError(error.message);
156 |       setSelectedFile(null);
157 |     } else {
158 |       setFileError(
159 |         t(
160 |           "assignments.detail.fileValidationFailed",
161 |           "File validation failed."
162 |         )
163 |       );
164 |     }
165 |   }
166 | };
167 |
168 | const openFilePicker = () => {
169 |   fileInputRef.current?.click();
170 | };
171 |
172 | const attemptUpload = async (retries: number): Promise<string> => {
173 |   if (!selectedFile || !assignmentData) {
174 |     throw new Error("Missing assignment or file");
175 |   }
176 |
177 |   try {
178 |     setUploadProgress(15 + retries * 5);
179 |     const result = await uploadFile.mutateAsync({
180 |       file: selectedFile,
181 |       assignmentId: assignmentData.id,
182 |       institutionId: profile?.institution_id ?? "",
183 |     });
184 |     setUploadProgress(100);
185 |     setUploadStatus("success");
186 |     return result;
187 |   } catch (error) {

```



```

188 |     const isNetworkError =
189 |       !navigator.onLine ||
190 |       (error instanceof Error &&
191 |         /network|fetch|timeout|offline/i.test(error.message));
192 |
193 |     if (isNetworkError && retries < 3) {
194 |       toast.info(
195 |         t("assignments.detail.retryingUpload", {
196 |           defaultValue: "Upload failed - retrying ({{count}}/3)...",
197 |           count: retries + 1,
198 |         })
199 |       );
200 |       await new Promise((resolve) =>
201 |         window.setTimeout(resolve, 1_500 * (retries + 1))
202 |       );
203 |       return attemptUpload(retries + 1);
204 |     }
205 |
206 |     setUploadStatus("error");
207 |     setUploadError(error instanceof Error ? error.message : "Upload failed");
208 |     throw error;
209 |   }
210 | };
211 |
212 | const handleSubmit = async () => {
213 |   if (!assignmentData || !selectedFile || !deadlineStatus) return;
214 |
215 |   if (!deadlineStatus.canSubmit) {
216 |     toast.error(
217 |       t("assignments.deadlinePassed", "The submission deadline has passed.")
218 |     );
219 |     return;
220 |   }
221 |
222 |   setUploadProgress(0);
223 |   setUploadStatus("uploading");
224 |   setUploadError(null);
225 |
226 |   try {
227 |     const fileUrl = await attemptUpload(0);
228 |     const payload = {
229 |       assignment_id: assignmentData.id,
230 |       file_url: fileUrl,
231 |       is_late: deadlineStatus.isLate,
232 |       institution_id: profile?.institution_id ?? "",
233 |     };
234 |
235 |     createSubmission.mutate(payload, {
236 |       onSuccess: () => {
237 |         setSelectedFile(null);
238 |         draftManager.clearDraft(`submission-draft-${id ?? "unknown"}`);
239 |         void submissions.refetch();
240 |         toast.success(
241 |           t("assignments.submissionConfirmed", "Submission confirmed!")
242 |         );
243 |         logActivity({
244 |           student_id: profile?.id ?? "",
245 |           event_type: "submission",
246 |           metadata: {
247 |             assignment_id: assignmentData.id,
248 |             is_late: deadlineStatus.isLate,
249 |           },
250 |         });
251 |         awardXPoptimistic({
252 |           studentId: profile?.id ?? "",
253 |           xpAmount: deadlineStatus.isLate
254 |             ? LATE_SUBMISSION_XP
255 |             : XP_SCHEDULE.submission,
256 |           source: "submission",
257 |           referenceId: assignmentData.id,
258 |           note: deadlineStatus.isLate
259 |             ? "Late submission"
260 |             : "On-time submission",
261 |         });
262 |       },
263 |       onError: (error) => {
264 |         toast.error(error.message);
265 |       },
266 |     });
267 |   } catch (error) {
268 |     const message = error instanceof Error ? error.message : "Upload failed";
269 |     setUploadStatus("error");
270 |     setUploadError(message);
271 |     toast.error(message);
272 |   }
273 | };
274 |
275 | const retryUpload = () => {
276 |   if (!selectedFile) return;
277 |   void handleSubmit();
278 | };
279 |
280 | const courseLabel =
281 |   assignmentData?.courses?.name ??
282 |   courses.data?.find((course) => course.id === assignmentData?.course_id)
283 |     ?.name ??
284 |   t("assignments.detail.courseFallback", "Course");
285 | const dueLabel = assignmentData
286 |   ? format(new Date(assignmentData.due_date), "MMM d, yyyy h:mm a")
287 |   : "";
288 | const rewardXP = deadlineStatus?.isLate
289 |   ? LATE_SUBMISSION_XP
290 |   : XP_SCHEDULE.submission;
291 |

```

```

292 | if (isLoading && !assignmentData) {
293 |   return (
294 |     <div className="space-y-4">
295 |       <Shimmer className="h-8 w-44 rounded-2xl" />
296 |       <Shimmer className="h-52 rounded-3xl" />
297 |       <div className="grid gap-4 lg:grid-cols-2">
298 |         <Shimmer className="h-64 rounded-3xl" />
299 |         <Shimmer className="h-64 rounded-3xl" />
300 |       </div>
301 |     </div>
302 |   );
303 | }
304 |
305 | if (assignment.isError || submissions.isError) {
306 |   return (
307 |     <ErrorState
308 |       title={t("assignments.detail.loadErrorTitle", "Assignment unavailable")}
309 |       message={t(
310 |         "assignments.detail.loadError",
311 |         "We couldn't load this assignment. Please try again."
312 |       )}
313 |       onRetry={() => {
314 |         void assignment.refetch();
315 |         void submissions.refetch();
316 |       }}
317 |       retryLabel={t("common:buttons.retry", "Try again")}
318 |     />
319 |   );
320 | }
321 |
322 | if (!assignmentData) {
323 |   return (
324 |     <ErrorState
325 |       title={t("assignments.detail.notFoundTitle", "Assignment not found")}
326 |       message={t(
327 |         "assignments.detail.notFound",
328 |         "This assignment is not available anymore."
329 |       )}
330 |       onRetry={() => navigate("/student/assignments")}
331 |       retryLabel={t(
332 |         "assignments.detail.backToAssignments",
333 |         "Back to assignments"
334 |       )}
335 |     />
336 |   );
337 | }
338 |
339 | return (
340 |   <div className="space-y-5">
341 |     <div className="flex items-center gap-3">
342 |       <Button
343 |         variant="ghost"
344 |         size="sm"
345 |         onClick={() => navigate("/student/assignments")}
346 |         className="text-muted-foreground"
347 |       >
348 |         <ArrowLeft className="me-1 size-4" />
349 |         {t("assignments.detail.backToAssignments", "Back to assignments")}
350 |       </Button>
351 |       <div className="min-w-0">
352 |         <p className="truncate text-sm text-muted-foreground">
353 |           {courseLabel}
354 |           {assignmentData.rubrics?.title
355 |             ? `· ${assignmentData.rubrics.title}`
356 |             : ""}
357 |         </p>
358 |         <h1 className="truncate text-2xl font-black tracking-tight text-foreground">
359 |           {assignmentData.title}
360 |         </h1>
361 |       </div>
362 |     </div>
363 |
364 |     <PCard className="overflow-hidden p-0">
365 |       <div
366 |         className="px-5 py-4"
367 |         style={{ background: "var(--brand-gradient)" }}
368 |       >
369 |         <div className="flex items-center justify-between gap-3">
370 |           <div className="min-w-0">
371 |             <p className="text-[10px] font-black uppercase tracking-[0.24em] text-white/80">
372 |               {courseLabel}
373 |             </p>
374 |             <h2 className="truncate text-xl font-black tracking-tight text-white">
375 |               {assignmentData.title}
376 |             </h2>
377 |           </div>
378 |           <Badge className="rounded-full border-white/20 bg-white/15 px-3 py-1 text-[11px] text-white">
379 |             {deadlineStatus?.timeRemaining ??
380 |               t("assignments.detail.loading", "Loading")}
381 |           </Badge>
382 |         </div>
383 |       </div>
384 |
385 |       <div className="space-y-4 p-5">
386 |         <p className="max-w-3xl text-sm text-muted-foreground">
387 |           {assignmentData.description}
388 |         </p>
389 |
390 |         <div className="grid gap-3 sm:grid-cols-2 xl:grid-cols-4">
391 |           {ASSIGNMENT_FACTS.map(({ icon: Icon, labelKey }) => {
392 |             const value =
393 |               labelKey === "assignments.detail.facts.due"
394 |                 ? dueLabel
395 |                 : labelKey === "assignments.detail.facts.marks"

```

```

396         ? `${assignmentData.total_marks}`
397         : labelKey === "assignments.detail.facts.reward"
398         ? `${rewardXP} XP`
399         : deadlineStatus
400         ? deadlineStatus.window === "closed"
401         ? t("assignments.detail.closedWindow", "Closed")
402         : `${assignmentData.late_window_hours}h`
403         : "";
404
405     return (
406       <div
407         key={labelKey}
408         className="rounded-2xl border border-slate-100 bg-slate-50 p-3 text-center"
409       >
410         <span
411           aria-hidden="true"
412           className="mx-auto mb-2 flex size-9 items-center justify-center rounded-2xl bg-white text-
413         >
414           <Icon className="size-4" />
415         </span>
416         <p className="text-[9px] font-black uppercase tracking-[0.24em] text-muted-foreground">
417           {t(labelKey)}
418         </p>
419         <p className="mt-1 text-sm font-bold text-foreground">
420           {value}
421         </p>
422       </div>
423     );
424   });
425 </div>
426
427 <div className="flex flex-wrap items-center gap-2">
428   {assignmentData.clo_weights.length > 0 ? (
429     <Badge
430       variant="outline"
431       className="rounded-full px-3 py-1 text-[11px]"
432     >
433       {t("assignments.detail.linkedClos", {
434         defaultValue: "{{count}} linked CLOs",
435         count: assignmentData.clo_weights.length,
436       })}
437     </Badge>
438   ) : null}
439
440   {difficultyBonus.data ? (
441     <
442       <BloomsPill level={difficultyBonus.data.bloomsLevel} />
443       <Badge
444         variant="outline"
445         className="rounded-full px-3 py-1 text-[11px]"
446       >
447         {t("assignments.detail.xpBonus", {
448           defaultValue: "{{multiplier}}x XP bonus",
449           multiplier: difficultyBonus.data.multiplier,
450         })}
451       </Badge>
452     </>
453   ) : null}
454 </div>
455 </div>
456 </PCard>
457
458 <div className="grid gap-5 lg:grid-cols-[1.05fr_0.95fr]">
459   <div className="space-y-5">
460     <PCard className="p-5">
461       <SectionHeader
462         icon={Upload}
463         title={t("assignments.detail.submitTitle", "Submit your work")}
464         description={t(
465           "assignments.detail.submitDescription",
466           "Upload your file, then submit it before the deadline window closes."
467         )}
468       />
469
470       <div className="mt-4 space-y-4">
471         <Input
472           ref={fileInputRef}
473           type="file"
474           className="hidden"
475           accept=".pdf,.doc,.docx,.ppt,.pptx,.txt,image/*"
476           onChange={(event) =>
477             handleFileChange(event.target.files?.[0] ?? null)
478           }
479         />
480
481         <button
482           type="button"
483           onClick={openFilePicker}
484           className="flex w-full flex-col items-center justify-center rounded-3xl border-2 border-dashed"
485         >
486           <Upload className="size-8 text-slate-400" />
487           <span className="mt-3 text-sm font-semibold text-foreground">
488             {t(
489               "assignments.detail.tapToUpload",
490               "Tap to upload your file"
491             )}
492           </span>
493           <span className="mt-1 text-xs text-muted-foreground">
494             {FILE_LIMIT_LABEL}
495           </span>
496         </button>
497
498         {fileError ? (
499           <p className="rounded-2xl border border-red-200 bg-red-50 px-4 py-3 text-sm text-red-700">

```

```

500         {fileError}
501     </p>
502 ) : null}
503
504 {draftHint && !selectedFile ? (
505     <p className="text-xs text-muted-foreground">
506         {t("assignments.detail.draftHint", {
507             defaultValue: "Previously selected: {{name}}",
508             name: draftHint,
509         })}
510     </p>
511 ) : null}
512
513 {selectedFile ? (
514     <UploadProgress
515         progress={uploadProgress}
516         fileName={selectedFile.name}
517         fileSize={selectedFile.size}
518         status={uploadStatus}
519         onRetry={uploadStatus === "error" ? retryUpload : undefined}
520     />
521 ) : null}
522
523 <div className="flex flex-wrap gap-2">
524     <Button
525         variant="tactile"
526         onClick={() => void handleSubmit()}
527         disabled={
528             !selectedFile || isUploading || !deadlineStatus?.canSubmit
529         }
530     >
531         {isUploading
532             ? t("assignments.detail.submitting", "Submitting...")
533             : t("assignments.submitAssignment", "Submit Assignment")}
534     </Button>
535     {selectedFile ? (
536         <Button
537             variant="outline"
538             onClick={() => {
539                 handleFileChange(null);
540                 if (fileInputRef.current) {
541                     fileInputRef.current.value = "";
542                 }
543             }}
544         >
545             <X className="me-1 size-4" />
546             {t("assignments.detail.clearFile", "Clear file")}
547         </Button>
548     ) : null}
549 </div>
550
551 {uploadError ? (
552     <p className="text-sm text-red-600">{uploadError}</p>
553 ) : null}
554
555 {deadlineStatus?.window === "closed" ? (
556     <div className="flex items-center gap-2 rounded-2xl border border-amber-200 bg-amber-50 px-4 p
557         <AlertTriangle className="size-4" />
558         {t(
559             "assignments.deadlinePassed",
560             "The submission deadline has passed."
561         )}
562     </div>
563 ) : (
564     <p className="text-xs text-muted-foreground">
565         {deadlineStatus?.window === "late_window"
566             ? t(
567                 "assignments.detail.lateWindow",
568                 "You can still submit during the late window."
569             )
570             : t(
571                 "assignments.detail.openWindow",
572                 "You can submit while the deadline window is open."
573             )}
574     </p>
575 )}
576 </div>
577 </PCard>
578
579 <Link
580     to="/student/tutor"
581     className="block rounded-3xl focus:outline-none focus-visible:ring-2 focus-visible:ring-ring"
582 >
583     <PCard className="border-teal-200 bg-gradient-to-br from-teal-50 to-blue-50 p-4">
584         <div className="flex items-center gap-3">
585             <span
586                 aria-hidden="true"
587                 className="flex size-11 shrink-0 items-center justify-center rounded-2xl text-white shadow-s
588                 style={{ background: "var(--brand-gradient)" }}
589             >
590                 <Bot className="size-5" />
591             </span>
592             <div className="min-w-0 flex-1">
593                 <p className="text-sm font-semibold text-foreground">
594                     {t(
595                         "assignments.detail.tutorTitle",
596                         "Need help before submitting?"
597                     )}
598                 </p>
599                 <p className="text-xs text-muted-foreground">
600                     {t(
601                         "assignments.detail.tutorHint",
602                         "Ask the AI Tutor about the assignment topic."
603                     )}

```

```

604         </p>
605     </div>
606     <span className="shrink-0 text-xs font-semibold text-teal-700">
607         {t("assignments.detail.ask", "Ask")}
608     </span>
609 </div>
610 </PCard>
611 </Link>
612 </div>
613
614 <div className="space-y-5">
615     <PCard className="p-5">
616         <SectionHeader
617             icon={CheckCircle2}
618             title={t("assignments.detail.statusText", "Your submission")}
619             description={t(
620                 "assignments.detail.statusDescription",
621                 "Track the latest submission state for this assignment."
622             )}
623         />
624
625         <div className="mt-4 space-y-3">
626             {mySubmission ? (
627                 <div className="rounded-2xl border border-slate-100 bg-slate-50 p-4">
628                     <div className="flex items-start justify-between gap-3">
629                         <div className="min-w-0">
630                             <p className="text-sm font-semibold text-foreground">
631                                 {mySubmission.is_late
632                                     ? t(
633                                         "assignments.lateSubmission",
634                                         "This is a late submission."
635                                     )
636                                     : t(
637                                         "assignments.submissionConfirmed",
638                                         "Submission confirmed!"
639                                     )}
640                             </p>
641                             <p className="mt-1 text-xs text-muted-foreground">
642                                 {formatDistanceToNow(
643                                     new Date(mySubmission.submitted_at),
644                                     {
645                                         addSuffix: true,
646                                     })}
647                             </p>
648                         </div>
649                         <Badge
650                             variant={mySubmission.is_late ? "secondary" : "default"}
651                             className="rounded-full px-3 py-1 text-[11px]"
652                         >
653                             {mySubmission.is_late
654                                 ? t("assignments.detail.late", "Late")
655                                 : t("assignments.detail.submitted", "Submitted")}
656                         </Badge>
657                     </div>
658
659                     <div className="mt-3 flex flex-wrap gap-2">
660                         {submittedFileLoading ? (
661                             <Badge
662                                 variant="outline"
663                                 className="rounded-full px-3 py-1 text-[11px]"
664                             >
665                                 {t(
666                                     "assignments.detail.signing",
667                                     "Preparing file link..."
668                                 )}
669                             </Badge>
670                         ) : submittedFileUrl ? (
671                             <Button asChild variant="outline" size="sm">
672                                 <a
673                                     href={submittedFileUrl}
674                                     target="_blank"
675                                     rel="noreferrer"
676                                 >
677                                     {t(
678                                         "assignments.detail.viewSubmission",
679                                         "View submission"
680                                     )}
681                                 </a>
682                             </Button>
683                         ) : null}
684                     </div>
685                 ) : (
686                     <div className="rounded-2xl border border-slate-100 bg-slate-50 p-4 text-sm text-muted-foregro
687                         {t(
688                             "assignments.detail.noSubmission",
689                             "You have not submitted this assignment yet."
690                         )}
691                     </div>
692                 )}
693             )}
694
695             <div className="grid gap-3 sm:grid-cols-2">
696                 <div className="rounded-2xl border border-slate-100 bg-white p-4">
697                     <p className="text-[10px] font-black uppercase tracking-[0.24em] text-muted-foreground">
698                         {t("assignments.detail.dueLabel", "Due")}
699                     </p>
700                     <p className="mt-1 text-sm font-semibold text-foreground">
701                         {format(new Date(assignmentData.due_date), "PPP p")}
702                     </p>
703                 </div>
704                 <div className="rounded-2xl border border-slate-100 bg-white p-4">
705                     <p className="text-[10px] font-black uppercase tracking-[0.24em] text-muted-foreground">
706                         {t("assignments.detail.windowLabel", "Late window")}
707                     </p>

```

```

708 |         </p>
709 |         <p className="mt-1 text-sm font-semibold text-foreground">
710 |           {assignmentData.late_window_hours
711 |             ? t("assignments.detail.windowValue", {
712 |               defaultValue: "{{count}} hours",
713 |               count: assignmentData.late_window_hours,
714 |             })
715 |             : t("assignments.detail.none", "None")}
716 |         </p>
717 |       </div>
718 |     </div>
719 |   </div>
720 | </PCard>
721 |
722 |   <PCard className="p-5">
723 |     <SectionHeader
724 |       icon={Award}
725 |       title={t("assignments.detail.bonusTitle", "Performance context")}
726 |       description={t(
727 |         "assignments.detail.bonusDescription",
728 |         "The system uses linked CLO difficulty to tune rewards."
729 |       )}
730 |     />
731 |
732 |     <div className="mt-4 flex flex-wrap items-center gap-2">
733 |       {difficultyBonus.data ? (
734 |         <>
735 |           <BloomsPill level={difficultyBonus.data.bloomsLevel} />
736 |           <Badge
737 |             variant="outline"
738 |             className="rounded-full px-3 py-1 text-[11px]"
739 |           >
740 |             {t("assignments.detail.bonusValue", {
741 |               defaultValue: "{{multiplier}}x XP bonus",
742 |               multiplier: difficultyBonus.data.multiplier,
743 |             })}
744 |           </Badge>
745 |         </>
746 |       ) : (
747 |         <Badge
748 |           variant="outline"
749 |           className="rounded-full px-3 py-1 text-[11px]"
750 |         >
751 |           {t("assignments.detail.noBonus", "No difficulty bonus")}
752 |         </Badge>
753 |       )}
754 |     </div>
755 |   </PCard>
756 | </div>
757 | </div>
758 | </div>
759 | );
760 | };
761 |
762 | export default AssignmentDetailScreen;
763 |

```

FILE: [src/features/student/assignments/AssignmentListScreen.tsx](#)

TYPE: .tsx | LINES: 1-289 | SHA256: b568e5e58fe592cca3daf5672d37d9a3e293dc2dac5f738d1aa35e0a7e282a99

```

1 | import { useMemo, useState } from "react";
2 | import { Link } from "react-router-dom";
3 | import { parseAsString, useQueryState } from "nuqs";
4 | import { useTranslation } from "react-i18next";
5 | import { format, isPast, addHours, formatDistanceToNow } from "date-fns";
6 | import { ArrowRight, Calendar, Search } from "lucide-react";
7 |
8 | import ErrorState from "@components/shared/ErrorState";
9 | import { Badge, Input, PCard, Shimmer } from "@design-system";
10 | import { useAuth } from "@hooks/useAuth";
11 | import { useStudentAssignments } from "@hooks/useSubmissions";
12 | import type { StudentAssignment } from "@hooks/useSubmissions";
13 | import { useStudentCourses } from "@hooks/useStudentCourses";
14 | import { cn } from "@lib/utils";
15 | import { NoAssignments } from "@components/shared/EmptyState";
16 | import {
17 |   Select,
18 |   SelectContent,
19 |   SelectItem,
20 |   SelectTrigger,
21 |   SelectValue,
22 | } from "@components/ui/select";
23 |
24 | type AssignmentStatus = "submitted" | "pending" | "late" | "overdue";
25 |
26 | const getAssignmentStatus = (
27 |   assignment: StudentAssignment
28 | ): AssignmentStatus => {
29 |   if (assignment.submissions && assignment.submissions.length > 0) {
30 |     const first = assignment.submissions[0]!;
31 |     return first.is_late ? "late" : "submitted";
32 |   }
33 |
34 |   const dueDate = new Date(assignment.due_date);
35 |   const lateDeadline = addHours(dueDate, assignment.late_window_hours);
36 |   if (isPast(lateDeadline)) return "overdue";
37 |   if (isPast(dueDate)) return "late";
38 |   return "pending";
39 | };
40 |
41 | const statusConfig: Record<
42 |   AssignmentStatus,
43 |   { label: string; className: string }
44 | > = {

```

```

45 | submitted: {
46 |   label: "Submitted",
47 |   className: "bg-emerald-50 text-emerald-700 border-emerald-200",
48 | },
49 | pending: {
50 |   label: "Pending",
51 |   className: "bg-amber-50 text-amber-700 border-amber-200",
52 | },
53 | late: {
54 |   label: "Late",
55 |   className: "bg-orange-50 text-orange-700 border-orange-200",
56 | },
57 | overdue: {
58 |   label: "Overdue",
59 |   className: "bg-rose-50 text-rose-700 border-rose-200",
60 | },
61 | };
62 |
63 | const AssignmentListScreen = () => {
64 |   const { t } = useTranslation("student");
65 |   const { profile } = useAuth();
66 |   const [search, setSearch] = useQueryState("q", parseAsString.withDefault(""));
67 |   const [courseFilter, setCourseFilter] = useState<string>("all");
68 |
69 |   const assignments = useStudentAssignments();
70 |   const courses = useStudentCourses(profile?.id);
71 |
72 |   const courseOptions = useMemo(
73 |     () => (courses.data ?? []).sort((a, b) => a.name.localeCompare(b.name)),
74 |     [courses.data]
75 |   );
76 |
77 |   const courseById = useMemo(() => {
78 |     const next = new Map<string, (typeof courseOptions)[number]>();
79 |     for (const course of courseOptions) next.set(course.id, course);
80 |     return next;
81 |   }, [courseOptions]);
82 |
83 |   const filtered = useMemo(() => {
84 |     const query = search.trim().toLowerCase();
85 |     return (assignments.data ?? []).filter((assignment) => {
86 |       if (courseFilter !== "all" && assignment.course_id !== courseFilter) {
87 |         return false;
88 |       }
89 |
90 |       if (!query) return true;
91 |       const course = courseById.get(assignment.course_id);
92 |       return (
93 |         assignment.title.toLowerCase().includes(query) ||
94 |         course?.name.toLowerCase().includes(query) ||
95 |         course?.code.toLowerCase().includes(query)
96 |       );
97 |     });
98 |   }, [assignments.data, courseById, courseFilter, search]);
99 |
100 |   const counts = useMemo(() => {
101 |     const next = {
102 |       submitted: 0,
103 |       pending: 0,
104 |       late: 0,
105 |       overdue: 0,
106 |     };
107 |     for (const assignment of assignments.data ?? []) {
108 |       const status = getAssignmentStatus(assignment);
109 |       next[status] += 1;
110 |     }
111 |     return next;
112 |   }, [assignments.data]);
113 |
114 |   if (assignments.isError || courses.isError) {
115 |     return (
116 |       <ErrorState
117 |         title={t("assignments.list.loadErrorTitle", "Assignments unavailable")}
118 |         message={t(
119 |           "assignments.list.loadError",
120 |           "We couldn't load your assignments. Please try again."
121 |         )}
122 |         onRetry={() => {
123 |           void assignments.refetch();
124 |           void courses.refetch();
125 |         }}
126 |         retryLabel={t("common.buttons.retry", "Try again")}
127 |       />
128 |     );
129 |   }
130 |
131 |   return (
132 |     <div className="space-y-5">
133 |       <div className="space-y-1">
134 |         <h1 className="text-2xl font-black tracking-tight text-foreground">
135 |           {t("assignments.list.title", "Assignments")}
136 |         </h1>
137 |         <p className="text-sm text-muted-foreground">
138 |           {t(
139 |             "assignments.list.subtitle",
140 |             "Track upcoming work, late submissions, and what is already done."
141 |           )}
142 |         </p>
143 |       </div>
144 |
145 |       <div className="grid gap-3 md:grid-cols-4">
146 |         {
147 |           [
148 |             ["pending", counts.pending],

```

```

149         ["submitted", counts.submitted],
150         ["late", counts.late],
151         ["overdue", counts.overdue],
152     ] as const
153   ).map(([key, count]) => (
154     <PCard key={key} className="p-4">
155       <p className="text-[10px] font-black uppercase tracking-[0.24em] text-muted-foreground">
156         {t(`assignments.list.status.${key}`, key)}
157       </p>
158       <p className="mt-1 text-2xl font-black text-foreground">{count}</p>
159     </PCard>
160   )))
161 </div>
162
163 <PCard className="p-4">
164   <div className="grid gap-3 lg:grid-cols-[1fr_220px]">
165     <div className="relative">
166       <Search className="pointer-events-none absolute start-3 top-1/2 size-4 -translate-y-1/2 text-muted-foreground">
167         <Input
168           value={search}
169           onChange={(event) => setSearch(event.target.value)}
170           placeholder={t(
171             "assignments.list.searchPlaceholder",
172             "Search assignments..."
173           )}
174           className="ps-9"
175         </Input>
176       </div>
177
178       <Select value={courseFilter} onChange={setCourseFilter}>
179         <SelectTrigger className="h-10">
180           <SelectValue
181             placeholder={t("assignments.list.allCourses", "All courses")}
182           </SelectValue>
183         </SelectTrigger>
184         <SelectContent>
185           <SelectItem value="all">
186             {t("assignments.list.allCourses", "All courses")}
187           </SelectItem>
188           {courseOptions.map((course) => (
189             <SelectItem key={course.id} value={course.id}>
190               {course.name}
191             </SelectItem>
192           ))}
193         </SelectContent>
194       </Select>
195     </div>
196   </PCard>
197
198   {assignments.isLoading || courses.isLoading ? (
199     <div className="space-y-3">
200       <Shimmer className="h-20 rounded-3xl" />
201       <Shimmer className="h-20 rounded-3xl" />
202       <Shimmer className="h-20 rounded-3xl" />
203     </div>
204   ) : filtered.length === 0 ? (
205     <NoAssignments />
206   ) : (
207     <div className="space-y-3">
208       {filtered.map((assignment) => {
209         const status = getAssignmentStatus(assignment);
210         const config = statusConfig[status];
211         const course = courseById.get(assignment.course_id);
212         const dueDate = new Date(assignment.due_date);
213
214         return (
215           <Link
216             key={assignment.id}
217             to={`/student/assignments/${assignment.id}`}
218             className="block rounded-3xl focus:outline-none focus-visible:ring-2 focus-visible:ring-ring"
219           >
220             <PCard className="p-4 transition-transform hover:-translate-y-0.5">
221               <div className="flex items-start justify-between gap-4">
222                 <div className="min-w-0 flex-1">
223                   <div className="flex flex-wrap items-center gap-2">
224                     <Badge
225                       variant="outline"
226                       className="rounded-full px-2.5 py-1 text-[11px]"
227                     >
228                       {course?.name ??
229                         course?.code ??
230                         t("assignments.list.courseFallback", "Course")}
231                     </Badge>
232                     <Badge
233                       variant="outline"
234                       className={cn(
235                         "rounded-full px-2.5 py-1 text-[11px]",
236                         config.className
237                       )}
238                     >
239                       {t(`assignments.list.status.${status}`, config.label)}
240                     </Badge>
241                   </div>
242                   <h2 className="mt-2 text-base font-bold text-foreground">
243                     {assignment.title}
244                   </h2>
245                   <p className="mt-1 line-clamp-2 text-sm text-muted-foreground">
246                     {assignment.description}
247                   </p>
248                   <div className="mt-3 flex flex-wrap items-center gap-3 text-xs text-muted-foreground">
249                     <span className="inline-flex items-center gap-1">
250                       <Calendar className="size-3.5" />

```



```

253 |         {format(dueDate, "PPP p")}]
254 |       </span>
255 |       <span>
256 |         {t("assignments.list.marks", {
257 |           defaultValue: "{{count}}" marks",
258 |           count: assignment.total_marks,
259 |         })}
260 |       </span>
261 |       <span>
262 |         {t("assignments.list.dueIn", {
263 |           defaultValue: "{{time}}",
264 |           time: formatDistanceToNow(dueDate, {
265 |             addSuffix: true,
266 |           })},
267 |         )}
268 |       </span>
269 |     </div>
270 |   </div>
271 |   <span
272 |     aria-hidden="true"
273 |     className="flex size-10 shrink-0 items-center justify-center rounded-2xl bg-slate-50 tex
274 |   >
275 |     <ArrowRight className="size-5 rtl:rotate-180" />
276 |   </span>
277 | </div>
278 | </PCard>
279 | </Link>
280 |   );
281 | }}}}
282 | </div>
283 | )}
284 | </div>
285 | );
286 | };
287 |
288 | export default AssignmentListScreen;
289 |

```

FILE: [src/features/student/courses/CourseDetailScreen.tsx](#)

TYPE: .tsx | LINES: 1-851 | SHA256: 50ab47c4b980ed56c0b5586b217bc2269f5a3d88991595f58827212ba47cb628

```

1 | import { useMemo, useState } from "react";
2 | import { format } from "date-fns";
3 | import { Link, useNavigate, useParams } from "react-router-dom";
4 | import { useTranslation } from "react-i18next";
5 | import {
6 |   ArrowLeft,
7 |   ArrowRight,
8 |   Bot,
9 |   BookOpen,
10 |   CheckCircle2,
11 |   CircleAlert,
12 |   Clock,
13 |   File,
14 |   FileText,
15 |   FolderOpen,
16 |   Link as LinkIcon,
17 |   Megaphone,
18 |   Sparkles,
19 |   Video,
20 |   type LucideIcon,
21 | } from "lucide-react";
22 |
23 | import ErrorState from "@components/shared/ErrorState";
24 | import { Button, Badge, PCard, SectionHeader, Shimmer } from "@design-system";
25 | import { useAnnouncements } from "@hooks/useAnnouncements";
26 | import { useAuth } from "@hooks/useAuth";
27 | import { useCourse } from "@hooks/useCourses";
28 | import {
29 |   useCourseAllMaterials,
30 |   useCourseModules,
31 |   type CourseMaterial,
32 |   type CourseModule,
33 | } from "@hooks/useCourseModules";
34 | import { useReadHabitTimer } from "@hooks/useReadHabitTimer";
35 | import { useStudentAssignments } from "@hooks/useSubmissions";
36 | import { useStudentCourses } from "@hooks/useStudentCourses";
37 | import { resolveCourseColor } from "@lib/courseColor";
38 | import { cn } from "@lib/utils";
39 |
40 | const MATERIAL_META: Record<
41 |   CourseMaterial["type"],
42 |   {
43 |     icon: LucideIcon;
44 |     badge: string;
45 |     tone: string;
46 |   }
47 | > = {
48 |   text: { icon: FileText, badge: "Text", tone: "bg-slate-100 text-slate-700" },
49 |   video: { icon: Video, badge: "Video", tone: "bg-violet-50 text-violet-700" },
50 |   link: {
51 |     icon: LinkIcon,
52 |     badge: "Link",
53 |     tone: "bg-emerald-50 text-emerald-700",
54 |   },
55 |   file: { icon: File, badge: "File", tone: "bg-blue-50 text-blue-700" },
56 | };
57 |
58 | const CourseSummaryStat = ({
59 |   label,
60 |   value,
61 |   hint,
62 | }: {
63 |   label: string;

```

```

64 | value: string;
65 | hint?: string;
66 | }) => (
67 | <div className="rounded-2xl border border-slate-100 bg-slate-50 p-3 text-center">
68 |   <p className="text-[10px] font-black uppercase tracking-[0.24em] text-muted-foreground">
69 |     {label}
70 |   </p>
71 |   <p className="mt-1 text-lg font-black text-foreground">{value}</p>
72 |   {hint ? (
73 |     <p className="mt-0.5 text-[11px] text-muted-foreground">{hint}</p>
74 |   ) : null}
75 | </div>
76 | );
77 |
78 | const MaterialViewerCard = ({
79 |   courseName,
80 |   material,
81 | }): {
82 |   courseName: string;
83 |   material: CourseMaterial;
84 | }) => {
85 |   const { t } = useTranslation("student");
86 |   const { elapsedSeconds, isCompleted } = useReadHabitTimer({
87 |     pageType: "assignment_detail",
88 |     pageId: material.id,
89 |   });
90 |   const meta = MATERIAL_META[material.type];
91 |   const Icon = meta.icon;
92 |
93 |   return (
94 |     <PCard className="overflow-hidden p-0">
95 |       <div className="flex items-start justify-between gap-3 border-b border-slate-100 px-4 py-3">
96 |         <div className="min-w-0 flex items-start gap-3">
97 |           <span
98 |             aria-hidden="true"
99 |             className={cn(
100 |               "flex size-11 shrink-0 items-center justify-center rounded-2xl",
101 |               meta.tone
102 |             )}
103 |           >
104 |             <Icon className="size-5" />
105 |           </span>
106 |           <div className="min-w-0">
107 |             <p className="text-[10px] font-black uppercase tracking-[0.24em] text-muted-foreground">
108 |               {courseName}
109 |             </p>
110 |             <h3 className="truncate text-base font-bold text-foreground">
111 |               {material.title}
112 |             </h3>
113 |             {material.description ? (
114 |               <p className="mt-0.5 line-clamp-2 text-sm text-muted-foreground">
115 |                 {material.description}
116 |               </p>
117 |             ) : null}
118 |           </div>
119 |         </div>
120 |         <div className="flex shrink-0 items-center gap-2">
121 |           <Badge
122 |             variant={isCompleted ? "default" : "secondary"}
123 |             className="rounded-full px-2.5 py-1 text-[11px]"
124 |           >
125 |             {isCompleted ? (
126 |               <CheckCircle2 className="me-1 size-3.5" aria-hidden="true" />
127 |             ) : (
128 |               <Clock className="me-1 size-3.5" aria-hidden="true" />
129 |             )}
130 |             {isCompleted
131 |               ? t("courses.detail.readCompleted", "Read habit complete")
132 |               : t("courses.detail.reading", {
133 |                   defaultValue: "{{count}}s reading",
134 |                   count: Math.max(0, 30 - elapsedSeconds),
135 |                 })}
136 |           </Badge>
137 |         </div>
138 |       </div>
139 |
140 |       <div className="space-y-4 px-4 py-4">
141 |         {material.type === "text" ? (
142 |           <div className="rounded-2xl bg-slate-50 p-4 text-sm leading-7 text-slate-700 whitespace-pre-wrap">
143 |             {material.content_url ??
144 |               t(
145 |                 "courses.detail.noTextBody",
146 |                 "No text content is attached yet."
147 |               )}
148 |           </div>
149 |         ) : null}
150 |
151 |         {material.type === "video" && material.content_url ? (
152 |           <div className="overflow-hidden rounded-2xl border border-slate-100 bg-black">
153 |             <iframe
154 |               title={material.title}
155 |               src={material.content_url.replace("watch?v=", "embed/")}
156 |               className="aspect-video w-full"
157 |               allowFullScreen
158 |               allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
159 |             />
160 |           </div>
161 |         ) : null}
162 |
163 |         {material.type === "link" && material.content_url ? (
164 |           <a
165 |             href={material.content_url}
166 |             target="_blank"
167 |             rel="noreferrer"

```

```

168 |         className="inline-flex items-center gap-2 rounded-2xl border border-emerald-200 bg-emerald-50 px-4
169 |         >
170 |         <LinkIcon className="size-4" aria-hidden="true" />
171 |         {t("courses.detail.openLink", "Open resource")}
172 |     </a>
173 | ) : null}
174 |
175 | {material.type === "file" &&
176 | (material.content_url || material.file_path) ? (
177 |     <a
178 |         href={material.content_url ?? material.file_path ?? "#"}
179 |         target="_blank"
180 |         rel="noreferrer"
181 |         className="inline-flex items-center gap-2 rounded-2xl border border-blue-200 bg-blue-50 px-4 py-3
182 |     >
183 |     <FileText className="size-4" aria-hidden="true" />
184 |     {t("courses.detail.openFile", "Download or view file")}
185 | </a>
186 | ) : null}
187 |
188 | <div className="flex flex-wrap items-center gap-2">
189 |     <Badge
190 |         variant="outline"
191 |         className="rounded-full px-2.5 py-1 text-[11px]"
192 |     >
193 |         {meta.badge}
194 |     </Badge>
195 |     <Badge
196 |         variant="outline"
197 |         className="rounded-full px-2.5 py-1 text-[11px]"
198 |     >
199 |         {t("courses.detail.readTimer", "Reading timer")}
200 |     </Badge>
201 | </div>
202 | </div>
203 | </PCard>
204 | );
205 | };
206 |
207 | const ModuleCard = ({
208 |     module,
209 |     materials,
210 |     isOpen,
211 |     onToggle,
212 |     onSelectMaterial,
213 |     activeMaterialId,
214 | }): {
215 |     module: CourseModule;
216 |     materials: CourseMaterial[];
217 |     isOpen: boolean;
218 |     onToggle: () => void;
219 |     onSelectMaterial: (material: CourseMaterial) => void;
220 |     activeMaterialId: string | null;
221 | }) => {
222 |     const { t } = useTranslation("student");
223 |     const publishedCount = materials.length;
224 |     const isLocked = publishedCount === 0;
225 |
226 |     return (
227 |         <PCard className="overflow-hidden p-0">
228 |             <Button
229 |                 type="button"
230 |                 variant="ghost"
231 |                 className="h-auto w-full justify-between gap-3 rounded-none px-4 py-3 text-left hover:bg-slate-50"
232 |                 onClick={onToggle}
233 |                 aria-expanded={isOpen}
234 |             >
235 |                 <div className="flex min-w-0 items-start gap-3">
236 |                     <span
237 |                         aria-hidden="true"
238 |                         className={cn(
239 |                             "mt-0.5 flex size-9 shrink-0 items-center justify-center rounded-2xl",
240 |                             isLocked
241 |                                 ? "bg-slate-100 text-slate-400"
242 |                                 : "bg-blue-50 text-blue-700"
243 |                         )}
244 |                     >
245 |                         <FolderOpen className="size-4" />
246 |                     </span>
247 |                     <div className="min-w-0">
248 |                         <p className="truncate text-sm font-bold text-foreground">
249 |                             {module.title}
250 |                         </p>
251 |                         {module.description ? (
252 |                             <p className="mt-0.5 line-clamp-1 text-xs text-muted-foreground">
253 |                                 {module.description}
254 |                             </p>
255 |                         ) : null}
256 |                     </div>
257 |                 </div>
258 |
259 |                 <div className="flex shrink-0 items-center gap-2">
260 |                     <Badge
261 |                         variant={isLocked ? "secondary" : "outline"}
262 |                         className="rounded-full px-2.5 py-1 text-[11px]"
263 |                     >
264 |                         {isLocked
265 |                             ? t("courses.detail.locked", "Locked")
266 |                             : t("courses.detail.items", {
267 |                                 defaultValue: "{{count}} items",
268 |                                 count: publishedCount,
269 |                             })}
270 |                     </Badge>
271 |                     <ArrowRight

```

```

272 |         className={cn(
273 |             "size-4 transition-transform rtl:rotate-180",
274 |             isOpen && "rotate-90"
275 |         )}
276 |         aria-hidden="true"
277 |     />
278 | </div>
279 | </Button>
280 |
281 | {isOpen ? (
282 |     <div className="border-t border-slate-100 p-2">
283 |         {publishedCount === 0 ? (
284 |             <p className="px-3 py-3 text-sm text-muted-foreground">
285 |                 {t(
286 |                     "courses.detail.noMaterials",
287 |                     "No published materials are available yet."
288 |                 )}
289 |             </p>
290 |         ) : (
291 |             <div className="space-y-1">
292 |                 {materials.map(material) => {
293 |                     const meta = MATERIAL_META[material.type];
294 |                     const icon = meta.icon;
295 |                     const active = material.id === activeMaterialId;
296 |
297 |                     return (
298 |                         <Button
299 |                             key={material.id}
300 |                             type="button"
301 |                             variant="ghost"
302 |                             onClick={() => onSelectMaterial(material)}
303 |                             className={cn(
304 |                                 "h-auto w-full justify-between rounded-2xl px-3 py-2.5 text-left",
305 |                                 active && "bg-blue-50 text-blue-800 hover:bg-blue-100"
306 |                             )}
307 |                         >
308 |                             <span className="flex min-w-0 items-center gap-3">
309 |                                 <span
310 |                                     aria-hidden="true"
311 |                                     className={cn(
312 |                                         "flex size-8 shrink-0 items-center justify-center rounded-xl",
313 |                                         meta.tone
314 |                                     )}
315 |                                 >
316 |                                 <Icon className="size-4" />
317 |                             </span>
318 |                             <span className="min-w-0">
319 |                                 <span className="block truncate text-sm font-medium">
320 |                                     {material.title}
321 |                                 </span>
322 |                                 {material.description ? (
323 |                                     <span className="block truncate text-xs text-muted-foreground">
324 |                                         {material.description}
325 |                                     </span>
326 |                                 ) : null}
327 |                             </span>
328 |                         </span>
329 |
330 |                         <Badge
331 |                             variant={active ? "default" : "outline"}
332 |                             className="rounded-full px-2 py-1 text-[10px] uppercase tracking-widest"
333 |                         >
334 |                             {meta.badge}
335 |                         </Badge>
336 |                     </Button>
337 |                 );
338 |             }
339 |         </div>
340 |     )}
341 | </div>
342 | ) : null}
343 | </PCard>
344 | );
345 | };
346 |
347 | const CourseDetailScreen = () => {
348 |     const { courseId } = useParams<{ courseId: string }>();
349 |     const navigate = useNavigate();
350 |     const { t } = useTranslation("student");
351 |     const { user } = useAuth();
352 |     const [selection, setSelection] = useState<{
353 |         openModuleId: string | null;
354 |         activeMaterialId: string | null;
355 |     }>({
356 |         openModuleId: null,
357 |         activeMaterialId: null,
358 |     });
359 |
360 |     const course = useCourse(courseId);
361 |     const modules = useCourseModules(courseId ?? "");
362 |     const materials = useCourseAllMaterials(courseId ?? "");
363 |     const announcements = useAnnouncements(courseId);
364 |     const studentCourses = useStudentCourses(user?.id);
365 |     const studentAssignments = useStudentAssignments(courseId);
366 |
367 |     const hasError =
368 |         course.isError ||
369 |         modules.isError ||
370 |         materials.isError ||
371 |         announcements.isError ||
372 |         studentCourses.isError ||
373 |         studentAssignments.isError;
374 |
375 |     const retryAll = () => {

```

```

376 |     void course.refetch();
377 |     void modules.refetch();
378 |     void materials.refetch();
379 |     void announcements.refetch();
380 |     void studentCourses.refetch();
381 |     void studentAssignments.refetch();
382 | };
383 |
384 | const courseSnapshot = useMemo(
385 |   () => studentCourses.data?.find((item) => item.id === courseId) ?? null,
386 |   [courseId, studentCourses.data]
387 | );
388 |
389 | const publishedModules = useMemo(
390 |   () => (modules.data ?? []).filter((module) => module.is_published),
391 |   [modules.data]
392 | );
393 |
394 | const publishedMaterials = useMemo(
395 |   () => (materials.data ?? []).filter((material) => material.is_published),
396 |   [materials.data]
397 | );
398 |
399 | const materialsByModule = useMemo(() => {
400 |   const next = new Map<string, CourseMaterial[]>();
401 |   for (const material of publishedMaterials) {
402 |     const list = next.get(material.module_id) ?? [];
403 |     list.push(material);
404 |     next.set(material.module_id, list);
405 |   }
406 |   return next;
407 | }, [publishedMaterials]);
408 |
409 | const nextAssignment = useMemo(() => {
410 |   const assignments = studentAssignments.data ?? [];
411 |   return (
412 |     assignments.find(
413 |       (assignment) => (assignment.submissions?.length ?? 0) === 0
414 |     ) ?? null
415 |   );
416 | }, [studentAssignments.data]);
417 |
418 | const visibleAnnouncements = useMemo(
419 |   () => (announcements.data ?? []).slice(0, 3),
420 |   [announcements.data]
421 | );
422 |
423 | if (!courseId) {
424 |   return (
425 |     <ErrorState
426 |       message={t(
427 |         "courses.detail.courseNotFound",
428 |         "This course could not be loaded."
429 |       )}
430 |       title={t("courses.detail.courseNotFoundTitle", "Course unavailable")}
431 |       onRetry={() => navigate("/student/courses")}
432 |       retryLabel={t("courses.detail.backToCourses", "Back to courses")}
433 |     />
434 |   );
435 | }
436 |
437 | if (course.isLoading && !course.data) {
438 |   return (
439 |     <div className="space-y-4">
440 |       <Shimmer className="h-8 w-56 rounded-2xl" />
441 |       <Shimmer className="h-44 rounded-3xl" />
442 |       <div className="grid gap-4 lg:grid-cols-2">
443 |         <Shimmer className="h-60 rounded-3xl" />
444 |         <Shimmer className="h-60 rounded-3xl" />
445 |       </div>
446 |       <Shimmer className="h-80 rounded-3xl" />
447 |     </div>
448 |   );
449 | }
450 |
451 | if (hasError) {
452 |   return (
453 |     <ErrorState
454 |       title={t("courses.detail.loadErrorTitle", "Course details unavailable")}
455 |       message={t(
456 |         "courses.detail.loadError",
457 |         "We couldn't load one or more course details. Please try again."
458 |       )}
459 |       onRetry={retryAll}
460 |       retryLabel={t("common:buttons.retry", "Try again")}
461 |     />
462 |   );
463 | }
464 |
465 | if (!course.data) {
466 |   return (
467 |     <ErrorState
468 |       title={t("courses.detail.courseNotFoundTitle", "Course unavailable")}
469 |       message={t(
470 |         "courses.detail.courseNotFound",
471 |         "This course could not be loaded."
472 |       )}
473 |       onRetry={() => navigate("/student/courses")}
474 |       retryLabel={t("courses.detail.backToCourses", "Back to courses")}
475 |     />
476 |   );
477 | }
478 |
479 | const activeMaterial =

```

```

480 |     publishedMaterials.find(
481 |       (material) => material.id === selection.activeMaterialId
482 |     ) ??
483 |     publishedMaterials[0] ??
484 |     null;
485 | const activeMaterialId = activeMaterial?.id ?? null;
486 | const openModuleId =
487 |   publishedModules.find((module) => module.id === selection.openModuleId)
488 |     ?.id ??
489 |     publishedModules[0]?.id ??
490 |     null;
491 | const courseColor = resolveCourseColor(
492 |   courseSnapshot?.color ?? null,
493 |   courseId
494 | );
495 | const teacherName = courseSnapshot?.teacher_name ?? null;
496 | const progress = courseSnapshot?.progress_percent ?? 0;
497 | const attainment = courseSnapshot?.attainment_percent;
498 |
499 | return (
500 |   <div className="space-y-5">
501 |     <div className="flex items-center gap-3">
502 |       <Button
503 |         variant="ghost"
504 |         size="sm"
505 |         onClick={() => navigate("/student/courses")}
506 |         className="text-muted-foreground"
507 |       >
508 |         <ArrowLeft className="me-1 size-4" />
509 |         {t("courses.detail.backToCourses", "Back to courses")}
510 |       </Button>
511 |       <div className="min-w-0">
512 |         <h1 className="truncate text-2xl font-black tracking-tight text-foreground">
513 |           {course.data?.name ?? t("courses.detail.title", "Course details")}
514 |         </h1>
515 |         <p className="truncate text-sm text-muted-foreground">
516 |           {course.data?.code
517 |             ? `${course.data.code}${teacherName ? ` · ${teacherName}` : ""}`
518 |             : teacherName ?? ""}
519 |         </p>
520 |       </div>
521 |     </div>
522 |
523 |     <div className="grid gap-4 lg:grid-cols-[1.4fr_1fr]">
524 |       <PCard className="p-4">
525 |         <div className="flex items-start justify-between gap-4">
526 |           <div className="min-w-0">
527 |             <p className="text-[10px] font-black uppercase tracking-[0.24em] text-muted-foreground">
528 |               {t("courses.detail.progressLabel", "Progress")}
529 |             </p>
530 |             <div className="mt-2 flex items-center gap-2">
531 |               <span
532 |                 aria-hidden="true"
533 |                 className="flex size-10 shrink-0 items-center justify-center rounded-2xl"
534 |                 style={{ background: `${courseColor}20`, color: courseColor }}
535 |               >
536 |                 <BookOpen className="size-5" />
537 |               </span>
538 |               <div className="min-w-0">
539 |                 <p className="text-sm font-semibold text-muted-foreground">
540 |                   {course.data?.name}
541 |                 </p>
542 |                 <p className="text-lg font-black text-foreground">
543 |                   {progress.toFixed(0)}%
544 |                 </p>
545 |               </div>
546 |             </div>
547 |           </div>
548 |
549 |           <Badge className="rounded-full px-3 py-1 text-[11px]">
550 |             {attainment !== null
551 |               ? t("courses.detail.attainmentValue", {
552 |                   defaultValue: `${count}% attainment`,
553 |                   count: attainment,
554 |                 })
555 |               : t("courses.detail.noAttainment", "No attainment data")}
556 |           </Badge>
557 |         </div>
558 |
559 |         <div
560 |           aria-label={t(
561 |             "courses.progressLabel",
562 |             "Course progress: {{percent}}%",
563 |             {
564 |               percent: progress,
565 |             }
566 |           )}
567 |           aria-valuemin={0}
568 |           aria-valuemax={100}
569 |           aria-valuenow={progress}
570 |           className="mt-4 h-2.5 overflow-hidden rounded-full bg-slate-100"
571 |           role="progressbar"
572 |         >
573 |           <div
574 |             className="h-full rounded-full"
575 |             style={{
576 |               width: `${progress}%`,
577 |               background: "var(--brand-gradient)",
578 |             }}
579 |           />
580 |         </div>
581 |
582 |         <div className="mt-4 grid gap-3 sm:grid-cols-2 xl:grid-cols-4">
583 |           <CourseSummaryStat

```

```

584         label={t("courses.detail.stats.modules", "Modules")}
585         value={` ${publishedModules.length} `}
586     />
587     <CourseSummaryStat
588       label={t("courses.detail.stats.materials", "Materials")}
589       value={` ${publishedMaterials.length} `}
590     />
591     <CourseSummaryStat
592       label={t("courses.detail.stats.assignments", "Assignments")}
593       value={` ${studentAssignments.data?.length ?? 0} `}
594     />
595     <CourseSummaryStat
596       label={t("courses.detail.stats.attainment", "Attainment")}
597       value={attainment != null ? ` ${attainment}% ` : "-"}
598     />
599   </div>
600 </PCard>
601
602 {nextAssignment ? (
603   <Link
604     to={` /student/assignments/${nextAssignment.id} `}
605     className="block rounded-3xl focus:outline-none focus-visible:ring-2 focus-visible:ring-ring"
606   >
607     <PCard className="h-full border-teal-200 bg-gradient-to-br from-teal-50 to-blue-50 p-4">
608       <div className="flex h-full flex-col justify-between gap-4">
609         <div className="space-y-2">
610           <p className="text-[10px] font-black uppercase tracking-[0.24em] text-teal-700">
611             {t(
612               "courses.detail.continueTitle",
613               "Continue where you left off"
614             )}
615           </p>
616           <h2 className="text-base font-black text-foreground">
617             {nextAssignment.title}
618           </h2>
619           <p className="text-sm text-muted-foreground">
620             {nextAssignment.due_date
621               ? t("courses.detail.dueShort", {
622                 defaultValue: "Due {{date}}",
623                 date: format(
624                   new Date(nextAssignment.due_date),
625                   "MMM d"
626                 ),
627             }) : t("courses.noDueDate", "No due date")}
628           </p>
629         </div>
630       </div>
631
632       <div className="flex items-center justify-between gap-3">
633         <div className="flex items-center gap-2 text-sm font-semibold text-teal-700">
634           <CircleAlert className="size-4" />
635           {t("courses.detail.openAssignment", "Open assignment")}
636         </div>
637         <span
638           aria-hidden="true"
639           className="flex size-10 shrink-0 items-center justify-center rounded-2xl text-white shadow"
640           style={{ background: "var(--brand-gradient)" }}
641         >
642           <ArrowRight className="size-5 rtl:rotate-180" />
643         </span>
644       </div>
645     </div>
646   </PCard>
647   </Link>
648 ) : (
649   <PCard className="flex items-center justify-center p-6 text-sm text-muted-foreground">
650     <Sparkles className="me-2 size-4 text-teal-600" />
651     {t("courses.detail.noOpenWork", "Nothing is due soon.")}
652   </PCard>
653 )}
654 </div>
655
656 <div className="grid gap-5 xl:grid-cols-[1.05fr_0.95fr]">
657   <div className="space-y-5">
658     <div className="space-y-3">
659       <SectionHeader
660         icon={Megaphone}
661         title={t("courses.detail.announcements", "Announcements")}
662         description={t(
663           "courses.detail.announcementsDescription",
664           "Recent updates from the course team."
665         )}
666       />
667       {announcements.isLoading ? (
668         <Shimmer className="h-40 rounded-3xl" />
669       ) : visibleAnnouncements.length > 0 ? (
670         <div className="space-y-3">
671           {visibleAnnouncements.map((announcement) => (
672             <PCard
673               key={announcement.id}
674               className={cn(
675                 "p-4",
676                 announcement.is_pinned
677                   ? "border-amber-200 bg-amber-50/40"
678                   : "border-slate-100"
679               )}
680             >
681               <div className="flex items-start gap-3">
682                 <span
683                   aria-hidden="true"
684                   className={cn(
685                     "flex size-10 shrink-0 items-center justify-center rounded-2xl",
686                     announcement.is_pinned
687                     ? "bg-amber-100 text-amber-700"

```

```

688         : "bg-slate-100 text-slate-500"
689     }}
690 >
691     <Megaphone className="size-4" />
692 </span>
693 <div className="min-w-0">
694     <div className="flex flex-wrap items-center gap-2">
695         <h3 className="text-sm font-bold text-foreground">
696             {announcement.title}
697         </h3>
698         {announcement.is_pinned ? (
699             <Badge className="rounded-full px-2 py-0.5 text-[10px] uppercase tracking-widest">
700                 {t("courses.detail.pinned", "Pinned")}
701             </Badge>
702         ) : null}
703     </div>
704     <p className="mt-1 line-clamp-3 text-sm text-muted-foreground">
705         {announcement.content}
706     </p>
707     <p className="mt-2 text-xs text-muted-foreground">
708         {format(
709             new Date(announcement.created_at),
710             "MMM d, yyyy"
711         )}
712     </p>
713 </div>
714 </div>
715 </PCard>
716 }}
717 </div>
718 ) : (
719     <PCard className="p-5 text-sm text-muted-foreground">
720         {t(
721             "courses.detail.noAnnouncements",
722             "There are no announcements for this course yet."
723         )}
724     </PCard>
725 )}
726 </div>
727
728 <div className="space-y-3">
729     <SectionHeader
730         icon={FolderOpen}
731         title={t("courses.detail.modules", "Modules")}
732         description={t(
733             "courses.detail.modulesDescription",
734             "Open a module to browse the published materials inside it."
735         )}
736     />
737
738     {modules.isLoading || materials.isLoading ? (
739         <div className="space-y-3">
740             <Shimmer className="h-18 rounded-3xl" />
741             <Shimmer className="h-18 rounded-3xl" />
742             <Shimmer className="h-18 rounded-3xl" />
743         </div>
744     ) : publishedModules.length > 0 ? (
745         <div className="space-y-3">
746             {publishedModules.map((module) => (
747                 <ModuleCard
748                     key={module.id}
749                     module={module}
750                     materials={materialsByModule.get(module.id) ?? []}
751                     isOpen={openModuleId === module.id}
752                     onToggle={() =>
753                         setSelection((current) => ({
754                             ...current,
755                             openModuleId:
756                                 current.openModuleId === module.id ? null : module.id,
757                         }))
758                     }
759                     onSelectMaterial={(material) => {
760                         setSelection({
761                             openModuleId: material.module_id,
762                             activeMaterialId: material.id,
763                         });
764                     }}
765                     activeMaterialId={activeMaterialId}
766                 />
767             ))}
768         </div>
769     ) : (
770         <PCard className="p-6 text-sm text-muted-foreground">
771             {t(
772                 "courses.detail.noPublishedModules",
773                 "No published modules are available yet."
774             )}
775         </PCard>
776     )}
777 </div>
778 </div>
779
780 <div className="space-y-5">
781     {activeMaterial ? (
782         <MaterialViewerCard
783             key={activeMaterial.id}
784             material={activeMaterial}
785             courseName={
786                 course.data?.name ?? t("courses.detail.title", "Course details")
787             }
788         />
789     ) : (
790         <PCard className="flex min-h-80 items-center justify-center p-8 text-center">
791             <div className="max-w-sm">

```



```

792 |         <div
793 |           aria-hidden="true"
794 |           className="mx-auto flex size-14 items-center justify-center rounded-3xl bg-teal-50 text-teal
795 |         >
796 |           <BookOpen className="size-6" />
797 |         </div>
798 |         <h2 className="mt-4 text-lg font-bold text-foreground">
799 |           {t(
800 |             "courses.detail.selectMaterialTitle",
801 |             "Select a material to preview it here"
802 |           )}
803 |         </h2>
804 |         <p className="mt-2 text-sm text-muted-foreground">
805 |           {t(
806 |             "courses.detail.selectMaterialHint",
807 |             "Browse a module on the left, then open a material to read or view it."
808 |           )}
809 |         </p>
810 |       </div>
811 |     </PCard>
812 |   )}
813 |
814 |   <Link
815 |     to="/student/tutor"
816 |     className="block rounded-3xl focus:outline-none focus-visible:ring-2 focus-visible:ring-ring"
817 |   >
818 |     <PCard className="border-teal-200 bg-gradient-to-br from-teal-50 to-blue-50 p-4">
819 |       <div className="flex items-center gap-3">
820 |         <span
821 |           aria-hidden="true"
822 |           className="flex size-11 shrink-0 items-center justify-center rounded-2xl text-white shadow-s
823 |           style={{ background: "var(--brand-gradient)" }}
824 |         >
825 |           <Bot className="size-5" />
826 |         </span>
827 |         <div className="min-w-0 flex-1">
828 |           <p className="text-sm font-semibold text-foreground">
829 |             {t("courses.stuck", "Stuck on this course?")}
830 |           </p>
831 |           <p className="text-xs text-muted-foreground">
832 |             {t(
833 |               "courses.tutorPrompt",
834 |               "Ask your AI Tutor for a guided explanation."
835 |             )}
836 |           </p>
837 |         </div>
838 |         <span className="shrink-0 text-xs font-semibold text-teal-700">
839 |           {t("courses.ask", "Ask")}
840 |         </span>
841 |       </div>
842 |     </PCard>
843 |   </Link>
844 | </div>
845 | </div>
846 | </div>
847 | );
848 | };
849 |
850 | export default CourseDetailScreen;
851 |

```

## FILE: src/features/student/dashboard/StudentDashboardRail.tsx

TYPE: .tsx | LINES: 1-358 | SHA256: b0389fbb82ec40ec999bac825d6b16c21e35da55f959bb096ea7c1f464702b7e

```

1 | // =====
2 | // StudentDashboardRail – the dashboard's right rail (prototype `railHTML()`)
3 | // dashboard case in shared.js). Fixed, laptop-only (xl+) companion column that
4 | // mirrors the prototype's per-page "Today" widget stack:
5 | //   Daily Goal · Daily Quests · Gold League · Coming up · Streak protection ·
6 | //   Edu's tip.
7 | //
8 | // Every card is wired to a REAL existing hook (no faked data, R17; no backend
9 | // changes, G.1). Where the prototype card has no backing source it is adapted
10 | // to the nearest real signal and the gap is called out inline:
11 | //   - "Daily Goal" (prototype = per-day XP target, e.g. "35 XP to go today")
12 | //   has NO daily-XP-goal backend, so it is bound to today's habit completion
13 | //   (useTodayViewData) – the closest real daily signal – and framed as such.
14 | //
15 | // Styling reproduces `prototype/shared.css` 1:1: `rail-card` (radius-16 border
16 | // #e2e8f0), `rail-h` (11px 800 uppercase slate-400 + optional `rail-r`),
17 | // `rail-row`, `rail-btn` (brand-gradient), `mini-bar`.
18 | // =====
19 |
20 | import { useMemo } from "react";
21 | import { useNavigate } from "react-router-dom";
22 | import { useTranslation } from "react-i18next";
23 | import {
24 |   addDays,
25 |   format,
26 |   isSameDay,
27 |   isWithinInterval,
28 |   startOfDay,
29 | } from "date-fns";
30 |
31 | import { RailCard, RailHead, RailRow, Shimmer } from "@design-system";
32 | import { Button } from "@components/ui/button";
33 | import WhyThisPopover from "@components/shared/WhyThisPopover";
34 | import { cn } from "@lib/utils";
35 | import { useAuth } from "@hooks/useAuth";
36 | import { useStudentDashboardAggregate } from "@hooks/useStudentDashboardAggregate";
37 | import { useTodayViewData } from "@hooks/useTodayView";
38 | import { useKnowledgeQuests } from "@hooks/useKnowledgeQuests";
39 | import {
40 |   useLeagueLeaderboard,

```

```

41 | useStudentLeagueTier,
42 | useStudentPercentileBand,
43 | } from "@hooks/useLeagueLeaderboard";
44 | import { formatPercentileBand } from "@lib/percentileBand";
45 |
46 | /** Compact relative label for a deadline date (Today / Tomorrow / weekday / MMM d). */
47 | const dueLabel = (iso: string): string => {
48 |   const d = new Date(iso);
49 |   if (Number.isNaN(d.getTime())) return "";
50 |   const today = startOfDay(new Date());
51 |   const day = startOfDay(d);
52 |   if (isSameDay(day, today)) return "Today";
53 |   if (isSameDay(day, addDays(today, 1))) return "Tomorrow";
54 |   if (isWithinInterval(day, { start: today, end: addDays(today, 6) }))
55 |     return format(d, "EEEE");
56 |   return format(d, "MMM d");
57 | };
58 |
59 | const RailLink = ({ to, label }: { to: string; label: string }) => {
60 |   const navigate = useNavigate();
61 |   return (
62 |     <Button
63 |       type="button"
64 |       variant="link"
65 |       size="sm"
66 |       onClick={() => navigate(to)}
67 |       className="mt-2 h-auto px-0 text-xs font-extrabold text-blue-600"
68 |     >
69 |       {label}
70 |     </Button>
71 |   );
72 | };
73 |
74 | const StudentDashboardRail = () => {
75 |   const { t } = useTranslation("student");
76 |   const { user } = useAuth();
77 |   const studentId = user?.id ?? "";
78 |
79 |   const aggregate = useStudentDashboardAggregate(studentId);
80 |   const today = useTodayViewData(studentId);
81 |   const quests = useKnowledgeQuests("active");
82 |   const leagueTier = useStudentLeagueTier(studentId);
83 |   const tier = leagueTier.data?.tier;
84 |   const leaderboard = useLeagueLeaderboard(undefined, tier);
85 |   const percentile = useStudentPercentileBand(studentId);
86 |
87 |   // % % Daily Goal !' today's habits (real daily signal; see file header note) % %
88 |   const habits = today.habits;
89 |   const habitsDone = [
90 |     habits.login,
91 |     habits.submit,
92 |     habits.journal,
93 |     habits.read,
94 |   ].filter(Boolean).length;
95 |   const goalPercent = (habitsDone / 4) * 100;
96 |
97 |   // % % Gold League: my row from the tier leaderboard, else percentile band % %
98 |   const leagueRows = leaderboard.data ?? [];
99 |   const me = useMemo(
100 |    () => (leaderboard.data ?? []).find((r) => r.student_id === studentId),
101 |    [leaderboard.data, studentId]
102 |  );
103 |   const leader = leagueRows[0];
104 |   const tierLabel = tier
105 |     ? `${tier.charAt(0).toUpperCase()}${tier.slice(1)}`
106 |     : "";
107 |
108 |   const deadlines = (aggregate.data?.deadlines ?? []).slice(0, 3);
109 |   const freezes = aggregate.data?.streakFreeze?.freezes ?? 0;
110 |   const streakAtRisk = !habits.login;
111 |
112 |   const activeQuests = (quests.data ?? []).slice(0, 3);
113 |
114 |   return (
115 |     <aside
116 |       aria-label={t("dashboard.rail.label", "Today")}
117 |       className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto border-s border-border bg-white"
118 |     >
119 |       {/* % % Daily Goal (habit completion ring) % % */}
120 |       <RailCard>
121 |         <div className="flex items-center gap-3">
122 |           <div className="relative h-14 w-14 shrink-0">
123 |             <svg className="h-14 w-14 -rotate-90" viewBox="0 0 36 36">
124 |               <circle
125 |                 cx="18"
126 |                 cy="18"
127 |                 r="15.9"
128 |                 fill="none"
129 |                 stroke="#eef2f6"
130 |                 strokeWidth="3.5"
131 |               />
132 |               <circle
133 |                 cx="18"
134 |                 cy="18"
135 |                 r="15.9"
136 |                 fill="none"
137 |                 stroke="#14b8a6"
138 |                 strokeWidth="3.5"
139 |                 strokeLinecap="round"
140 |                 strokeDasharray="100"
141 |                 strokeDashoffset={100 - goalPercent}
142 |               />
143 |             </svg>
144 |             <div className="absolute inset-0 flex items-center justify-center text-xs font-extrabold text-teal

```

```

145         {habitsDone}/4
146     </div>
147 </div>
148 <div>
149     <p className="text-[13px] font-extrabold text-slate-900 dark:text-slate-100">
150         {t("dashboard.rail.dailyGoal", "Daily Goal")}
151     </p>
152     <p className="mt-0.5 text-[11px] text-slate-500">
153         {habitsDone >= 4
154         ? t("dashboard.rail.perfectDay", "Perfect Day complete 0<B%")
155         : t("dashboard.rail.habitsToGo", {
156             defaultValue: "{n}" more for a Perfect Day",
157             n: 4 - habitsDone,
158         })}}
159     </p>
160 </div>
161 </div>
162 </RailCard>
163
164 {/ % % Daily Quests % % *}
165 <RailCard>
166     <RailHead
167         title={t("dashboard.rail.quests", "&"p Daily Quests")}
168         right={
169             activeQuests.length > 0
170             ? t("dashboard.rail.questsActive", {
171                 defaultValue: "{n}" active",
172                 n: activeQuests.length,
173             })
174             : undefined
175         }
176     />
177     {quests.isPending ? (
178         <Shimmer className="h-12 rounded-lg" />
179     ) : activeQuests.length > 0 ? (
180         <div className="space-y-0.5">
181             {activeQuests.map((q) => (
182                 <RailRow key={q.id}>
183                     <span aria-hidden="true">0<B</span>
184                     <span className="min-w-0 flex-1 truncate">{q.title}</span>
185                     {q.reward_type === "xp" && q.reward_xp_amount ? (
186                         <b className="text-[12px] font-extrabold text-amber-600">
187                             +{q.reward_xp_amount}
188                         </b>
189                     ) : null}
190                 </RailRow>
191             ))}
192         </div>
193     ) : (
194         <p className="text-xs text-slate-500">
195             {t("dashboard.rail.questsEmpty", "No active quests right now.")}
196         </p>
197     )}
198 </RailCard>
199
200 {/ % % League standing % % *}
201 <RailCard>
202     <RailHead
203         title={
204             tierLabel
205             ? t("dashboard.rail.league", {
206                 defaultValue: "0<B {tier}" League",
207                 tier: tierLabel,
208             })
209             : t("dashboard.rail.leagueGeneric", "0<B League")
210         }
211     />
212     {leagueTier.isPending ? (
213         <Shimmer className="h-12 rounded-lg" />
214     ) : tier ? (
215         <div className="space-y-0.5">
216             {leader && (
217                 <RailRow>
218                     <span className="w-4 text-xs font-extrabold text-amber-500">
219                         1
220                     </span>
221                     <span className="min-w-0 flex-1 truncate">
222                         {leader.full_name}
223                     </span>
224                     <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
225                         {leader.weekly_xp.toLocaleString()}
226                     </b>
227                 </RailRow>
228             )}
229             {me ? (
230                 <RailRow className="-mx-1.5 rounded-lg bg-blue-50 px-1.5 dark:bg-blue-950/40">
231                     <span className="w-4 text-xs font-extrabold text-slate-500">
232                         {me.rank}
233                     </span>
234                     <span className="min-w-0 flex-1 truncate font-extrabold text-blue-700 dark:text-blue-300">
235                         {t("dashboard.rail.you", "You")}
236                     </span>
237                     <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
238                         {me.weekly_xp.toLocaleString()}
239                     </b>
240                 </RailRow>
241             ) : percentile.data ? (
242                 <RailRow className="-mx-1.5 rounded-lg bg-blue-50 px-1.5 dark:bg-blue-950/40">
243                     <span className="min-w-0 flex-1 truncate font-extrabold text-blue-700 dark:text-blue-300">
244                         {t("dashboard.rail.you", "You")}
245                     </span>
246                     <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
247                         {formatPercentileBand(percentile.data.band)}
248                     </b>

```

```

249     </RailRow>
250   ) : null}
251 </div>
252 ) : (
253   <p className="text-xs text-slate-500">
254     {t(
255       "dashboard.rail.leagueEmpty",
256       "Earn XP to join a weekly league."
257     )}
258   </p>
259 )}
260 </RailCard>
261
262 {/ * % % Coming up (deadlines) % % */}
263 <RailCard>
264   <RailHead title={t("dashboard.rail.comingUp", "ðŰ Coming up")} />
265   {aggregate.isPending ? (
266     <Shimmer className="h-12 rounded-lg" />
267   ) : deadlines.length > 0 ? (
268     <
269       {deadlines.map((d) => (
270         <RailRow key={d.id}>
271           <span className="min-w-0 flex-1 truncate">{d.title}</span>
272           <b className="text-[12px] font-extrabold text-amber-700">
273             {duelLabel(d.due_date)}
274           </b>
275         </RailRow>
276       )]}
277       <RailLink
278         to="/student/planner"
279         label={t("dashboard.rail.openCalendar", "Open calendar !")}
280       />
281     </>
282   ) : (
283     <p className="text-xs text-slate-500">
284       {t("dashboard.rail.comingUpEmpty", "Nothing due in the next days.")}
285     </p>
286   )}
287 </RailCard>
288
289 {/ * % % Streak protection % % */}
290 <RailCard>
291   <RailHead
292     title={t("dashboard.rail.streakProtection", "ðp Streak protection")}
293   />
294   <RailRow>
295     <span className="min-w-0 flex-1">
296       {t("dashboard.rail.freezes", "Freezes in inventory")}
297     </span>
298     <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
299       {freezes}
300     </b>
301   </RailRow>
302   <RailRow>
303     <span className="min-w-0 flex-1">
304       {t("dashboard.rail.todaysStreak", "Today's streak")}
305     </span>
306     <b
307       className={cn(
308         "text-[12px] font-extrabold",
309         streakAtRisk ? "text-red-600" : "text-green-600"
310       )}
311     >
312       {streakAtRisk
313         ? t("dashboard.rail.atRisk", "At risk")
314         : t("dashboard.rail.safe", "Safe")}
315     </b>
316   </RailRow>
317   <RailLink
318     to="/student/marketplace"
319     label={t("dashboard.rail.getFreezes", "Get more freezes !")}
320   />
321 </RailCard>
322
323 {/ * % % Edu's tip (AI coach) % % */}
324 <RailCard
325   className="border-0 text-white"
326   style={{ background: "linear-gradient(135deg,#0f172a,#1e3a8a)" }}
327 >
328   <div className="flex items-center gap-2">
329     <span className="text-lg aria-hidden="true">
330       ðŰ
331     </span>
332     <p className="text-xs font-extrabold">
333       {t("dashboard.rail.eduTip", "Edu's tip")}
334     </p>
335   </div>
336   <p className="mt-1.5 text-xs text-white/80">
337     {t(
338       "dashboard.rail.eduTipBody",
339       "Review your weakest outcomes first – small, focused reps move attainment the fastest."
340     )}
341   </p>
342   <whyThisPopover
343     title={t("dashboard.rail.eduTip", "Edu's tip")}
344     reasons={[
345       t(
346         "dashboard.rail.eduTipWhy",
347         "Your current outcome-attainment and review activity guide this suggestion."
348       ),
349     ]}
350     className="mt-2 bg-white/10 text-white hover:bg-white/20 hover:text-white"
351   />
352 </RailCard>

```

```

353 |     </aside>
354 |   );
355 | };
356 |
357 | export default StudentDashboardRail;
358 |

```

## FILE: src/features/student/dashboard/StudentDashboardScreen.tsx

TYPE: .tsx | LINES: 1-879 | SHA256: eea6992d352faba548dfa72d6e3a3ebec62e59b044e2410728431725a769e371

```

1 | // =====
2 | // StudentDashboardScreen – prototype rebuild (prototype-frontend-rebuild P2.1)
3 | // =====
4 | //
5 | // Rebuilds `prototype/dashboard.html` on `@/design-system` + tokens, wired to
6 | // the REAL existing hooks (no faked data, R17; no backend changes, G.1):
7 | // - useStudentDashboardAggregate !' kpis (XP/level/streak) + next deadline
8 | // - computeLevelData(kpis.totalXP) !' level ring + XP-to-go (pure, no fetch)
9 | // - useCLOProgress !' weakest outcome (lowest attained CLO)
10 | // - useStudentCourses !' My Courses strip (per-course mastery ring)
11 | // - useTodayViewData !' Today's Habits (login/submit/journal/read)
12 | //
13 | // Pixel parity vs the prototype is proven by the owner's `npm run test:visual`
14 | // gate (visual/screen-map.ts `auth`/`student-dashboard`), then refined.
15 | //
16 | // NOTE (incremental): the hero carousel's secondary slides (leaderboard rank
17 | // movement, "badge 1 session away") and the Daily-Review / Weekly-Activity /
18 | // Continue-Path / Announcements sections land in the next increment – several
19 | // need a rank-delta / badge-progress source that the backend-parity audit
20 | // flagged as a gap; they will be wired to real hooks or flagged, never faked.
21 | // =====
22 |
23 | import { useMemo } from "react";
24 | import { useNavigate } from "react-router-dom";
25 | import { useTranslation } from "react-i18next";
26 | import {
27 |   ArrowRight,
28 |   BarChart3,
29 |   Bot,
30 |   Check,
31 |   ChevronRight,
32 |   Megaphone,
33 |   Repeat2,
34 |   ShieldQuestion,
35 | } from "lucide-react";
36 | import { formatDistanceToNow } from "date-fns";
37 |
38 | import {
39 |   Button,
40 |   HeroCarousel,
41 |   MascotCharacter,
42 |   Shimmer,
43 | } from "@/design-system";
44 | import { useAuth } from "@/hooks/useAuth";
45 | import { useStudentDashboardAggregate } from "@/hooks/useStudentDashboardAggregate";
46 | import { computeLevelData } from "@/hooks/useLevel";
47 | import { useStudentCourses } from "@/hooks/useStudentCourses";
48 | import { useCLOProgress } from "@/hooks/useCLOProgress";
49 | import { useTodayViewData } from "@/hooks/useTodayView";
50 | import { useWeeklyReviews } from "@/hooks/useReviewSchedule";
51 | import { useHeatmapData } from "@/hooks/useHeatmapData";
52 | import {
53 |   useStudentLeagueTier,
54 |   useStudentPercentileBand,
55 | } from "@/hooks/useLeagueLeaderboard";
56 | import { cn } from "@/lib/utils";
57 |
58 | /** Prototype brand gradient token (93.65deg teal!`blue) as an inline value. */
59 | const BRAND_GRADIENT = "var(--brand-gradient)";
60 | const HERO_GRADIENT = "var(--hero-gradient)";
61 |
62 | /** SVG progress ring matching the prototype `.ring-mini` / hero ring. */
63 | const ProgressRing = ({
64 |   percent,
65 |   size,
66 |   stroke,
67 |   track = "#eef2f6",
68 |   color = "#3b82f6",
69 |   children,
70 | }: {
71 |   percent: number;
72 |   size: number;
73 |   stroke: number;
74 |   track?: string;
75 |   color?: string;
76 |   children?: React.ReactNode;
77 | }) => {
78 |   const clamped = Math.max(0, Math.min(100, percent));
79 |   return (
80 |     <div
81 |       className="relative shrink-0"
82 |       style={{ width: size, height: size }}
83 |       role="img"
84 |       aria-label={` ${Math.round(clamped)}%` }
85 |     >
86 |       <svg
87 |         className="-rotate-90"
88 |         width={size}
89 |         height={size}
90 |         viewBox="0 0 36 36"
91 |       >
92 |         <circle
93 |           cx="18"
94 |           cy="18"

```

```

95 |         r="15.9"
96 |         fill="none"
97 |         stroke={stroke}
98 |         strokeWidth={stroke}
99 |     />
100 |     <circle
101 |       cx="18"
102 |       cy="18"
103 |       r="15.9"
104 |       fill="none"
105 |       stroke={color}
106 |       strokeWidth={stroke}
107 |       strokeLinecap="round"
108 |       strokeDasharray="100"
109 |       strokeDashoffset={100 - clamped}
110 |     />
111 |   </svg>
112 |   <div className="absolute inset-0 flex items-center justify-center">
113 |     {children}
114 |   </div>
115 | </div>
116 | );
117 | });
118 |
119 | /** Heat colors (0=none ... 4=most) + Mon-first labels for the weekly grid. */
120 | const HEAT = ["#eef2f6", "#a7f3d0", "#34d399", "#10b981", "#047857"];
121 | const DOW = ["M", "T", "W", "T", "F", "S", "S"];
122 |
123 | /** Current week (Mon-Sun) as local ISO date strings. */
124 | const currentWeekRange = () => {
125 |   const now = new Date();
126 |   const dow = (now.getDay() + 6) % 7;
127 |   const monday = new Date(
128 |     now.getFullYear(),
129 |     now.getMonth(),
130 |     now.getDate() - dow
131 |   );
132 |   const days = Array.from({ length: 7 }, (_, i) => {
133 |     const d = new Date(
134 |       monday.getFullYear(),
135 |       monday.getMonth(),
136 |       monday.getDate() + i
137 |     );
138 |     return `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(
139 |       2,
140 |       "0"
141 |     )}-${String(d.getDate()).padStart(2, "0")}`;
142 |   });
143 |   return { start: days[0] ?? "", end: days[6] ?? "", days };
144 | };
145 |
146 | /** Time-of-day greeting bucket, matching the prototype's `timeGreeting`. */
147 | const greetingTimeOfDay = () => "morning" | "afternoon" | "evening" => {
148 |   const h = new Date().getHours();
149 |   if (h < 12) return "morning";
150 |   if (h < 17) return "afternoon";
151 |   return "evening";
152 | };
153 |
154 | const StudentDashboardScreen = () => {
155 |   const { t } = useTranslation("student");
156 |   const navigate = useNavigate();
157 |   const { user, profile } = useAuth();
158 |   const studentId = user?.id ?? "";
159 |
160 |   const aggregate = useStudentDashboardAggregate(studentId);
161 |   const courses = useStudentCourses(studentId);
162 |   const cloProgress = useCLOProgress(studentId);
163 |   const today = useTodayViewData(studentId);
164 |   const leagueTier = useStudentLeagueTier(studentId);
165 |   const percentile = useStudentPercentileBand(studentId);
166 |
167 |   const kpis = aggregate.data?.kpis;
168 |   const level = computeLevelData(kpis?.totalXP ?? 0);
169 |   const xpToGo = Math.max(0, level.xpForNextLevel - level.xpTotal);
170 |   const streak = kpis?.currentStreak ?? 0;
171 |   const nextDeadline = aggregate.data?.deadlines?.[0];
172 |   const firstName = (profile?.full_name ?? "Student").split(" ")[0];
173 |
174 |   // Weakest outcome = lowest attained CLO across enrolled courses (real data).
175 |   const weakest = useMemo(() => {
176 |     const entries = (cloProgress.data ?? []).
177 |       .flatMap((c) => c.entries)
178 |       .filter((e) => e.attainment_percent !== null);
179 |     if (entries.length === 0) return null;
180 |     return entries.reduce((min, e) =>
181 |       (e.attainment_percent as number) < (min.attainment_percent as number)
182 |         ? e
183 |         : min
184 |     );
185 |   }, [cloProgress.data]);
186 |
187 |   const habits = today.habits;
188 |   const habitList = [
189 |     { key: "login", label: t("dashboard.habits.login", "Login"), emoji: "👤" },
190 |     {
191 |       key: "submit",
192 |       label: t("dashboard.habits.submit", "Submit"),
193 |       emoji: "📝",
194 |     },
195 |     {
196 |       key: "journal",
197 |       label: t("dashboard.habits.journal", "Journal"),
198 |       emoji: "📖",

```

[illegible]

```

303         style={{ width: `${level.progressPercent}%` }}
304     />
305 </div>
306 </div>
307 </div>
308 </div>
309 <MascotCharacter
310   character="foxi"
311   emotion="happy"
312   size="md"
313   animation="float"
314   decorative
315   className="pointer-events-none absolute bottom-0 right-1"
316 />
317 </div>,
318 ];
319
320 if (streak > 0) {
321   heroSlides.push(
322     <div key="streak" className="flex min-h-[116px] items-center gap-3 p-3.5">
323       <div
324         className="flex h-11 w-11 shrink-0 items-center justify-center rounded-xl border border-white/20 bg-
325         aria-hidden="true"
326       >
327         Ø=Ý%
328       </div>
329       <div className="min-w-0 flex-1">
330         <p className="text-[9px] font-black uppercase tracking-widest text-amber-300">
331           {t("dashboard.hero.streakEyebrow", "Streak · day {{n}}", {
332             n: streak,
333           })}
334         </p>
335         <h2 className="mt-0.5 text-sm font-bold tracking-tight">
336           {streakAtRisk
337             ? t("dashboard.hero.streakRiskTitle", "Don't lose it today")
338             : t("dashboard.hero.streakSafeTitle", "Your streak is alive")}
339         </h2>
340         <p className="mt-0.5 truncate text-[10px] text-white/65">
341           {streakAtRisk
342             ? t(
343               "dashboard.hero.streakRiskBody",
344               "You haven't logged today's activity yet – a short review keeps it going."
345             )
346             : t(
347               "dashboard.hero.streakSafeBody",
348               "Keep showing up daily to grow your streak."
349             )}
350         </p>
351       </div>
352       <button
353         type="button"
354         onClick={() => navigate("/student/today")}
355         className="shrink-0 rounded-xl px-2.5 py-1.5 text-xs font-bold text-white shadow-[0_3px_0_rgba(0,0,0
356         style={{ background: "linear-gradient(90deg,#2dd4bf,#38bdf8)" }}
357       >
358         {t("dashboard.hero.streakCta", "Protect it !")}
359       </button>
360     </div>
361   );
362 }
363
364 if (leagueRank && leagueTotal > 0) {
365   heroSlides.push(
366     <div key="league" className="flex min-h-[116px] items-center gap-3 p-3.5">
367       <div
368         className="flex h-11 w-11 shrink-0 items-center justify-center rounded-xl border border-white/20 bg-
369         aria-hidden="true"
370       >
371         Ø=ÜÈ
372       </div>
373       <div className="min-w-0 flex-1">
374         <p className="text-[9px] font-black uppercase tracking-widest text-teal-300">
375           {t("dashboard.hero.leagueEyebrow", "Leaderboard")}
376         </p>
377         <h2 className="mt-0.5 text-sm font-bold tracking-tight">
378           {t("dashboard.hero.leagueTitle", "You're #{{rank}} of {{total}}", {
379             rank: leagueRank,
380             total: leagueTotal,
381           })}
382         </h2>
383         <p className="mt-0.5 text-[10px] text-white/65">
384           {tierLabel
385             ? t("dashboard.hero.leagueBody", "{{tier}} League this week", {
386               tier: tierLabel,
387             })
388             : t("dashboard.hero.leagueBodyGeneric", "Your weekly standing")}
389         </p>
390       </div>
391       <button
392         type="button"
393         onClick={() => navigate("/student/leaderboard")}
394         className="shrink-0 rounded-xl border border-white/20 bg-white/[.14] px-2.5 py-1.5 text-xs font-bold
395       >
396         {t("dashboard.hero.leagueCta", "View !")}
397       </button>
398     </div>
399   );
400 }
401
402 return (
403   <div className="w-full space-y-4">
404     {/* % % Hero carousel (living greeting · streak · league standing) % % */}
405     <HeroCarousel
406       slides={heroSlides}

```



```

407 |         className="rounded-2xl text-white shadow-lg"
408 |         style={{ background: HERO_GRADIENT }}
409 |         ariaLabel={t("dashboard.hero.label", "Highlights")}
410 |     />
411 |
412 |     {/** % % Weakest outcome + Next step % % */}
413 |     <div className="grid gap-3 lg:grid-cols-2">
414 |         {/** Weakest outcome (OBE drives the next action) */}
415 |         <button
416 |             type="button"
417 |             onClick={() => navigate("/student/progress")}
418 |             className="block overflow-hidden rounded-2xl text-start text-white shadow-md transition-transform ac
419 |             style={{ background: "linear-gradient(135deg,#0f766e,#0382bd)" }}
420 |         >
421 |             <div className="p-3.5">
422 |                 <p className="text-[9px] font-black uppercase tracking-widest text-white/70">
423 |                     {t("dashboard.weakest.eyebrow", "Today · weakest outcome")}
424 |                 </p>
425 |                 {cloProgress.isPending ? (
426 |                     <Shimmer className="mt-2 h-12 rounded-lg bg-white/20" />
427 |                 ) : weakest ? (
428 |                     <div className="mt-1.5 flex items-center gap-3">
429 |                         <ProgressRing
430 |                             percent={weakest.attainment_percent ?? 0}
431 |                             size={40}
432 |                             stroke={3.5}
433 |                             track="rgba(255,255,255,.2)"
434 |                             color="#fff"
435 |                         >
436 |                             <span className="text-[11px] font-black">
437 |                                 {Math.round(weakest.attainment_percent ?? 0)}%
438 |                             </span>
439 |                         </ProgressRing>
440 |                         <div className="min-w-0 flex-1">
441 |                             <h3 className="truncate text-sm font-bold leading-tight">
442 |                                 {weakest.clo_title}
443 |                             </h3>
444 |                             <p className="truncate text-[11px] text-white/75">
445 |                                 {weakest.course_name}
446 |                             </p>
447 |                         </div>
448 |                         <MascotCharacter
449 |                             character="foxi"
450 |                             emotion="encouraging"
451 |                             size="xs"
452 |                             animation="nudge"
453 |                             decorative
454 |                         />
455 |                     </div>
456 |                     <span className="mt-2.5 flex w-full items-center justify-center gap-1 rounded-xl bg-white py-2
457 |                     <div className="overflow-hidden rounded-2xl border-l-4 border-amber-400 bg-white shadow-md">
458 |                         <div className="p-3.5">
459 |                             {aggregate.isPending ? (
460 |                                 <Shimmer className="h-20 rounded-lg" />
461 |                             ) : nextDeadline ? (
462 |                                 <div className="flex items-center gap-3">
463 |                                     <div className="flex h-10 w-10 shrink-0 items-center justify-center rounded-xl bg-white/80 b
464 |                                     </div>
465 |                                     <div className="min-w-0 flex-1">
466 |                                         <p className="text-[9px] font-black uppercase tracking-widest text-gray-400">
467 |                                             {t("dashboard.nextStep.eyebrow", "Next step")}
468 |                                         </p>
469 |                                         <h3 className="truncate text-sm font-black leading-tight text-gray-900">
470 |                                             {nextDeadline.title}
471 |                                         </h3>
472 |                                         <p className="truncate text-[11px] text-gray-500">
473 |                                             {nextDeadline.course_name}
474 |                                         </p>
475 |                                     </div>
476 |                                     <div>
477 |                                         <div className="mt-2.5 flex gap-2">
478 |                                             <Button
479 |                                                 variant="tactile"
480 |                                                 className="h-9 flex-1"
481 |                                                 onClick={() =>
482 |                                                     navigate(`/student/assignments/${nextDeadline.id}`)
483 |                                                 }
484 |                                             >
485 |                                                 {t("dashboard.nextStep.start", "Start now")}
486 |                                             <ArrowRight className="h-4 w-4" aria-hidden="true" />
487 |                                         </Button>
488 |                                         <Button
489 |                                             variant="outline"
490 |                                             className="h-9"
491 |                                             onClick={() => navigate("/student/tutor")}

```

```

511         >
512         <Bot className="h-4 w-4" aria-hidden="true" />
513         {t("dashboard.nextStep.tutor", "Tutor")}
514     </Button>
515 </div>
516 </>
517 ) : (
518   <div className="flex items-center gap-3 py-2">
519     <ShieldQuestion
520       className="h-8 w-8 shrink-0 text-gray-300"
521       aria-hidden="true"
522     />
523     <p className="text-sm text-gray-500">
524       {t("dashboard.nextStep.empty", "No upcoming deadlines.")}
525     </p>
526   </div>
527 </div>
528 </div>
529 </div>
530 </div>
531
532 {/ % % My Courses section (responsive fluid grid) % % */}
533 <section className="my-courses-section w-full min-w-0 max-w-none">
534   <div className="my-courses-header mb-3 flex w-full items-center justify-between gap-2">
535     <div className="flex items-center gap-2">
536       <span className="flex h-6 w-6 items-center justify-center rounded-lg bg-transparent text-sm">
537         0=00
538       </span>
539       <p className="text-[13px] font-black tracking-tight text-slate-900">
540         {t("dashboard.myCourses", "My Courses")}
541       </p>
542     </div>
543     <button
544       type="button"
545       onClick={() => navigate("/student/courses")}
546       className="text-xs font-bold text-sky-700 hover:underline"
547     >
548       {t("dashboard.seeAll", "See all !")}
549     </button>
550   </div>
551   {courses.isPending ? (
552     <div className="my-courses-grid grid w-full min-w-0 gap-3 [grid-template-columns:repeat(auto-fit,min
553       {Array.from({ length: 4 }).map((_, i) => {
554         <Shimmer key={i} className="h-[76px] w-full min-w-0 rounded-xl" />
555       })}
556     </div>
557   ) : courseList.length > 0 ? (
558     <div className="my-courses-grid grid w-full min-w-0 gap-3 [grid-template-columns:repeat(auto-fit,min
559       {courseList.map((c) => {
560         const ring = c. attainment_percent ?? c.progress_percent;
561         return (
562           <button
563             key={c.id}
564             type="button"
565             onClick={() => navigate(`/student/courses/${c.id}`)}
566             className="course-card flex h-full w-full min-w-0 items-center gap-3 rounded-xl border borde
567           >
568             <div className="course-card-progress flex shrink-0">
569               <ProgressRing percent={ring} size={48} stroke={3}>
570                 <span className="text-[11px] font-black text-blue-600">
571                   {Math.round(ring)}%
572                 </span>
573               </ProgressRing>
574             </div>
575             <div className="course-card-details min-w-0 flex-1">
576               <span className="text-[9px] font-bold tracking-wide text-gray-400 uppercase">
577                 {c.code}
578               </span>
579               <p className="truncate text-[13px] font-bold leading-tight text-gray-900">
580                 {c.name}
581               </p>
582               <p className="mt-0.5 text-[10px] text-gray-500">
583                 {t("dashboard.courseProgress", "{p}% complete", {
584                   p: Math.round(c.progress_percent),
585                 })}
586               </p>
587             </div>
588           </button>
589         );
590       })}
591     </div>
592   ) : (
593     <p className="w-full rounded-xl bg-white p-4 text-center text-sm text-gray-500 shadow-xs">
594       {t(
595         "dashboard.noCourses",
596         "You are not enrolled in any courses yet."
597       )}
598     </p>
599   )}
600 </section>
601
602 {/ % % Today's Habits & Continue Your Path (Side by Side Grid) % % */}
603 <div className="grid gap-3 min-[640px]:grid-cols-2">
604   {/ % Today's Habits % /}
605   <section className="flex flex-col justify-between rounded-2xl bg-white p-3.5 shadow-md">
606     <div>
607       <div className="mb-2 flex items-center gap-2">
608         <span className="flex h-5 w-5 items-center justify-center rounded-md bg-transparent text-xs">
609           0=Y%
610         </span>
611         <p className="text-xs font-black tracking-tight text-slate-900">
612           {t("dashboard.habits.title", "Today's Habits")}
613         </p>
614         <span className="ms-auto rounded-full bg-green-50 px-2 py-0.5 text-[9px] font-bold text-green-60

```

```

615         {habitsDone}/4
616     </span>
617 </div>
618 <div className="flex justify-around py-1">
619     {habitList.map((h) => {
620         const done = habits[h.key];
621         return (
622             <div key={h.key} className="flex flex-col items-center gap-1">
623                 <div
624                     className={cn(
625                         "flex h-9 w-9 items-center justify-center rounded-full transition-all",
626                         done
627                           ? "bg-green-500 text-white shadow-sm"
628                           : "border-2 border-dashed border-gray-300 bg-gray-100 text-gray-400"
629                     )}
630                 >
631                     {done ? (
632                         <Check
633                             className="h-4 w-4 stroke-[2.5]"
634                             aria-hidden="true"
635                         />
636                     ) : (
637                         <span className="text-xs">{h.emoji}</span>
638                     )}
639                 </div>
640                 <span
641                     className={cn(
642                         "text-[9px] font-semibold",
643                         done ? "text-gray-600" : "text-gray-400"
644                     )}
645                 >
646                     {h.label}
647                 </span>
648             </div>
649         );
650     })}
651 </div>
652 </div>
653 {habitsDone < 4 && (
654     <div className="mt-2 flex items-center gap-2 rounded-lg border border-amber-100 bg-amber-50 px-2.5"
655     >
656         <MascotCharacter
657             character="penguin"
658             emotion="default"
659             size="sm"
660             decorative
661         />
662         <p className="text-[11px] font-medium leading-tight text-amber-700">
663             {t("dashboard.habits.nudge", {
664                 defaultValue: "Just {{n}} more for a Perfect Day (+50 XP)",
665                 n: 4 - habitsDone,
666             })}
667         </p>
668     </div>
669 )}
670 </section>
671 {/* Continue Your Path */}
672 <button
673     type="button"
674     onClick={() => navigate("/student/today")}
675     className="flex flex-col justify-between rounded-2xl border border-slate-100 bg-white p-3.5 text-sta
676 >
677     <div className="w-full">
678         <div className="mb-2 flex items-center gap-2">
679             <span className="flex h-5 w-5 items-center justify-center rounded-md bg-transparent text-xs">
680                 Ø=Ýúþ
681             </span>
682             <p className="text-xs font-black tracking-tight text-slate-900">
683                 {t("dashboard.continuePath.title", "Continue Your Path")}
684             </p>
685             <span className="ms-auto rounded-full bg-blue-50 px-2 py-0.5 text-[9px] font-bold text-blue-600"
686                 >{t("dashboard.continuePath.unit", "Unit 3")}
687             </span>
688         </div>
689         <div className="flex items-center gap-3 py-1">
690             <div className="flex h-9 w-9 shrink-0 items-center justify-center rounded-xl bg-white/80 border
691                 <span className="text-base">Ø=ÜÝ</span>
692             </div>
693             <div className="min-w-0 flex-1">
694                 <p className="truncate text-xs font-bold text-gray-900">
695                     {weakest
696                       ? weakest.clo_title
697                       : t("dashboard.continuePath.topic", "Normalization")}
698                 </p>
699                 <p className="text-[11px] text-gray-500">
700                     {t(
701                         "dashboard.continuePath.progressText",
702                         "Applying level · 3 of 5 lessons"
703                     )}
704                 </p>
705             </div>
706             <ChevronRight
707                 className="h-4 w-4 shrink-0 text-gray-300"
708                 aria-hidden="true"
709             />
710         </div>
711     </div>
712     <div className="mt-2 h-1.5 w-full overflow-hidden rounded-full bg-gray-100">
713         <div
714             className="h-full rounded-full"
715             style={{ width: "60%", background: BRAND_GRADIENT }}
716         />
717     </div>
718 </button>

```

```

719 | </div>
720 |
721 | { /* % % Daily Review + Weekly Activity % % */ }
722 | <div className="grid gap-3 lg:grid-cols-2">
723 | { /* Daily Review (spaced repetition) */ }
724 | <div className="rounded-2xl bg-white p-4 shadow-md">
725 |   <div className="mb-2 flex items-center gap-2">
726 |     <span className="flex h-6 w-6 items-center justify-center rounded-lg bg-transparent text-slate-700">
727 |       <Repeat2 className="h-4 w-4" aria-hidden="true" />
728 |     </span>
729 |     <p className="text-[13px] font-black tracking-tight text-slate-900">
730 |       {t("dashboard.review.title", "Daily Review")}
731 |     </p>
732 |     {streak > 0 && (
733 |       <span className="ms-auto rounded-full bg-orange-50 px-2 py-0.5 text-[10px] font-bold text-orange">
734 |         0={v%{" "}}
735 |         {t("dashboard.review.streak", "{n}-day streak", {
736 |           n: streak,
737 |         })}
738 |       </span>
739 |     )}
740 |   </div>
741 |   {reviews.isPending ? (
742 |     <Shimmer className="h-16 rounded-lg" />
743 |   ) : pendingReviews.length > 0 ? (
744 |     <>
745 |       <p className="mb-2.5 text-xs text-gray-500">
746 |         {t("dashboard.review.due", {
747 |           defaultValue:
748 |             "{n} cards due . adaptive spacing strengthens your weakest CLOs",
749 |           n: pendingReviews.length,
750 |         })}
751 |       </p>
752 |       <div className="mb-3 flex flex-wrap gap-1.5">
753 |         {reviewChips.map((chip) => (
754 |           <span
755 |             key={chip.label}
756 |             className="rounded-full border border-blue-100 bg-blue-50 px-2 py-1 text-[10px] font-bold"
757 |             >
758 |               {chip.label} ×{chip.count}
759 |             </span>
760 |         ))}
761 |       </div>
762 |       <Button
763 |         variant="tactile"
764 |         className="h-9 w-full"
765 |         onClick={() => navigate("/student/today")}
766 |       >
767 |         {t("dashboard.review.start", "Start review")}
768 |       </Button>
769 |     </>
770 |   ) : (
771 |     <p className="py-4 text-center text-sm text-gray-500">
772 |       {t(
773 |         "dashboard.review.empty",
774 |         "No reviews due – you're all caught up."
775 |       )}
776 |     </p>
777 |   )}
778 | </div>
779 |
780 | { /* This Week's Activity (habit heatmap) */ }
781 | <div className="rounded-2xl bg-white p-4 shadow-md">
782 |   <div className="mb-3 flex items-center gap-2">
783 |     <span className="flex h-6 w-6 items-center justify-center rounded-lg bg-transparent text-slate-700">
784 |       <BarChart3 className="h-4 w-4" aria-hidden="true" />
785 |     </span>
786 |     <p className="text-[13px] font-black tracking-tight text-slate-900">
787 |       {t("dashboard.activity.title", "This Week's Activity")}
788 |     </p>
789 |     <button
790 |       type="button"
791 |       onClick={() => navigate("/student/progress")}
792 |       className="ms-auto text-xs font-bold text-sky-700 hover:underline"
793 |     >
794 |       {t("dashboard.activity.trends", "Trends !")}
795 |     </button>
796 |   </div>
797 |   <div className="flex justify-between gap-1.5">
798 |     {week.days.map((date, i) => {
799 |       const count = heatByDate.get(date) ?? 0;
800 |       const isToday = date === todayStr;
801 |       const isFuture = date > todayStr;
802 |       const lvl =
803 |         count >= 5
804 |           ? 4
805 |           : count >= 4
806 |             ? 3
807 |             : count >= 2
808 |               ? 2
809 |               : count >= 1
810 |                 ? 1
811 |                 : 0;
812 |       const bg = isFuture ? "#f1f5f9" : HEAT[lvl] ?? "#eef2f6";
813 |       return (
814 |         <div
815 |           key={date}
816 |           className="flex flex-1 flex-col items-center gap-1"
817 |         >
818 |           <span className="text-[10px] font-semibold text-gray-400">
819 |             {DOW[i] ?? ""}
820 |           </span>
821 |           <span
822 |             className={cn(

```

```

823 |         "flex h-8 w-full items-center justify-center rounded-md text-[10px] font-bold",
824 |         lvl >= 3 ? "text-white" : "text-gray-600",
825 |         isToday && "ring-2 ring-sky-400"
826 |     )}
827 |     style={{ background: bg }}
828 |     title={{ ${date} . ${count} ${
829 |         count === 1 ? "activity" : "activities"
830 |     }}
831 |     >
832 |     {isFuture ? "-" : isToday ? "." : count || "-"}
833 |     </span>
834 | </div>
835 | );
836 | }}
837 | </div>
838 | </div>
839 | </div>
840 |
841 | {/ * % % Announcements % % */}
842 | <section className="rounded-2xl bg-white p-4 shadow-md">
843 |   <div className="mb-3 flex items-center gap-2">
844 |     <span className="flex h-6 w-6 items-center justify-center rounded-lg bg-transparent text-slate-700">
845 |       <Megaphone className="h-4 w-4" aria-hidden="true" />
846 |     </span>
847 |     <p className="text-[13px] font-black tracking-tight text-slate-900">
848 |       {t("dashboard.announcements.title", "Announcements")}
849 |     </p>
850 |   </div>
851 |   {announcements.length > 0 ? (
852 |     <ul className="space-y-3">
853 |       {announcements.slice(0, 3).map((a) => (
854 |         <li key={a.id} className="flex items-start gap-3">
855 |           <span className="mt-1.5 h-2 w-2 shrink-0 rounded-full bg-amber-400" />
856 |           <div className="min-w-0">
857 |             <p className="text-sm font-medium text-gray-900">{a.title}</p>
858 |             <p className="truncate text-xs text-gray-500">{a.content}</p>
859 |             <p className="mt-1 text-[10px] text-gray-400">
860 |               {formatDistanceToNow(new Date(a.created_at), {
861 |                 addSuffix: true,
862 |               })}
863 |             </p>
864 |           </div>
865 |         </li>
866 |       )})
867 |     </ul>
868 |   ) : (
869 |     <p className="py-2 text-center text-sm text-gray-500">
870 |       {t("dashboard.announcements.empty", "No announcements right now.")}
871 |     </p>
872 |   )}
873 | </section>
874 | </div>
875 | );
876 | };
877 |
878 | export default StudentDashboardScreen;
879 |

```

## FILE: src/features/student/fees/StudentFeesPage.tsx

TYPE: .tsx | LINES: 1-76 | SHA256: 4ccf6abbcbbdbd70414138c7469c7533d6c44bbc9fb8a3348b4accb4049a40917

```

1 | // =====
2 | // StudentFeesPage – student fee status + receipts (net-new screen)
3 | // =====
4 | // Built from the prototype design system (`@/design-system`): PageHeader +
5 | // StatePanel + FeePaymentList. Surfaces the student's own `fee_payments`
6 | // (`useStudentFees`) with receipt download (`useGenerateFeeReceipt`).
7 | // =====
8 |
9 | import { useTranslation } from "react-i18next";
10 | import { toast } from "sonner";
11 |
12 | import { PageHeader, StatePanel } from "@/design-system";
13 | import FeePaymentList from "@/features/shared/fees/FeePaymentList";
14 | import { useAuth } from "@/hooks/useAuth";
15 | import { useGenerateFeeReceipt, useStudentFees } from "@/hooks/useFees";
16 |
17 | const downloadFile = (url: string, name: string) => {
18 |   const link = document.createElement("a");
19 |   link.href = url;
20 |   link.download = name || "receipt.pdf";
21 |   link.rel = "noopener";
22 |   document.body.appendChild(link);
23 |   link.click();
24 |   link.remove();
25 | };
26 |
27 | const StudentFeesPage = () => {
28 |   const { t } = useTranslation("common");
29 |   const { user } = useAuth();
30 |   const { data: payments, isLoading, isError } = useStudentFees(user?.id);
31 |   const receipt = useGenerateFeeReceipt();
32 |   const downloadingId = receipt.isPending ? receipt.variables : undefined;
33 |
34 |   const handleDownload = (paymentId: string) => {
35 |     receipt.mutate(paymentId, {
36 |       onSuccess: (result) => {
37 |         downloadFile(result.download_url, result.file_name);
38 |         toast.success(t("fees.receiptReady", "Receipt downloaded"));
39 |       },
40 |       onError: (error) => {
41 |         toast.error(
42 |           error instanceof Error
43 |             ? error.message

```

```

44 |         : t("fees.receiptError", "Could not generate receipt")
45 |     );
46 | },
47 | });
48 | };
49 |
50 | return (
51 |   <div className="space-y-6">
52 |     <PageHeader title={t("fees.title", "Fees")} />
53 |
54 |     {isLoading ? (
55 |       <StatePanel variant="loading" />
56 |     ) : isError || !payments ? (
57 |       <StatePanel
58 |         variant="error"
59 |         message={t(
60 |           "fees.error",
61 |           "Could not load your fee records. Please try again."
62 |         )}
63 |       />
64 |     ) : (
65 |       <FeePaymentList
66 |         payments={payments}
67 |         onDownloadReceipt={handleDownload}
68 |         downloadingId={downloadingId}
69 |       />
70 |     )}
71 |   </div>
72 | );
73 | };
74 |
75 | export default StudentFeesPage;
76 |

```

## FILE: src/features/student/friends/StudentFriendsPage.tsx

TYPE: .tsx | LINES: 1-327 | SHA256: 9590783fe2f41b61b2191b776318e49e4f139288100f65b4394d139f5770a2c4

```

1 | // =====
2 | // StudentFriendsPage – the student "Friends" surface (net-new feature).
3 | // =====
4 | //
5 | // A privacy-appropriate adaptation of Duolingo's friends system for education:
6 | // mutual friend requests, scoped to the same institution, with online presence.
7 | // Composed from `@/design-system` primitives following the leaderboard-list
8 | // archetype (the prototype has no dedicated friends screen). Wired to the real
9 | // friends hooks (useFriends & co.); no faked data.
10 | //
11 | // Sections: add-friends search · incoming requests · online-now · all friends.
12 | // =====
13 |
14 | import { useState } from "react";
15 | import { useTranslation } from "react-i18next";
16 | import { Check, Flame, Search, UserPlus, Users, X } from "lucide-react";
17 |
18 | import { Button, Input, PageHeader, Shimmer } from "@/design-system";
19 | import { useAuth } from "@/hooks/useAuth";
20 | import {
21 |   useClassmateSearch,
22 |   useFriendRequests,
23 |   useFriends,
24 |   useRemoveFriend,
25 |   useRespondFriendRequest,
26 |   useSendFriendRequest,
27 |   type Friend,
28 | } from "@/hooks/useFriends";
29 | import { cn } from "@/lib/utls";
30 |
31 | const CARD =
32 |   "rounded-[20px] border border-[#eef2f6] bg-white shadow-[0_1px_2px_rgba(16,24,40,0.04),0_10px_26px_rgba(16,2
33 |
34 | const initials = (name: string): string =>
35 |   name
36 |     .split(/\s+/)
37 |     .map((w) => w[0] ?? "")
38 |     .join("")
39 |     .slice(0, 2)
40 |     .toUpperCase() || "?";
41 |
42 | /** Brand-gradient avatar with an online presence dot. */
43 | const FriendAvatar = ({
44 |   name,
45 |   online,
46 |   size = "md",
47 | }: {
48 |   name: string;
49 |   online?: boolean;
50 |   size?: "sm" | "md";
51 | }) => (
52 |   <div className="relative shrink-0">
53 |     <div
54 |       className={cn(
55 |         "flex items-center justify-center rounded-full font-bold text-white",
56 |         size === "sm" ? "h-9 w-9 text-[11px]" : "h-11 w-11 text-xs"
57 |       )}
58 |       style={{ background: "var(--brand-gradient)" }}
59 |       aria-hidden="true"
60 |     >
61 |       {initials(name)}
62 |     </div>
63 |     {online && (
64 |       <span className="absolute -bottom-0.5 -end-0.5 h-3 w-3 rounded-full border-2 border-white bg-green-500"
65 |     )}
66 |   </div>
67 | );

```

```

68 |
69 | const StudentFriendsPage = () => {
70 |   const { t } = useTranslation("student");
71 |   const { user } = useAuth();
72 |   const studentId = user?.id ?? "";
73 |
74 |   const friends = useFriends(studentId);
75 |   const requests = useFriendRequests(studentId);
76 |   const sendRequest = useSendFriendRequest();
77 |   const respond = useRespondFriendRequest();
78 |   const removeFriend = useRemoveFriend();
79 |
80 |   const [query, setQuery] = useState("");
81 |   const search = useClassmateSearch(query, studentId);
82 |
83 |   const friendList = friends.data ?? [];
84 |   const onlineFriends = friendList.filter((f) => f.online);
85 |   const requestList = requests.data ?? [];
86 |
87 |   return (
88 |     <div className="w-full space-y-4">
89 |       <div>
90 |         <PageHeader title={t("friends.title", "Friends")} />
91 |         <p className="mt-1 text-sm text-gray-500">
92 |           {t(
93 |             "friends.subtitle",
94 |             "Connect with classmates, cheer each other on, and climb together."
95 |           )}
96 |         </p>
97 |       </div>
98 |
99 |       {/** % % Add friends % % */}
100 |      <section className={cn(CARD, "p-4")}>
101 |        <div className="relative">
102 |          <Search
103 |            className="pointer-events-none absolute start-3 top-1/2 h-4 w-4 -translate-y-1/2 text-gray-400"
104 |            aria-hidden="true"
105 |          />
106 |          <Input
107 |            value={query}
108 |            onChange={(e) => setQuery(e.target.value)}
109 |            placeholder={t(
110 |              "friends.searchPlaceholder",
111 |              "Search classmates by name"
112 |            )}
113 |            className="ps-9"
114 |            aria-label={t(
115 |              "friends.searchPlaceholder",
116 |              "Search classmates by name"
117 |            )}
118 |          />
119 |        </div>
120 |        {query.trim().length >= 2 && (
121 |          <div className="mt-3 space-y-2">
122 |            {search.isPending ? (
123 |              <Shimmer className="h-12 rounded-xl" />
124 |            ) : (search.data ?? []).length > 0 ? (
125 |              (search.data ?? []).map((c) => (
126 |                <div
127 |                  key={c.student_id}
128 |                  className="flex items-center gap-3 rounded-xl border border-slate-100 bg-white p-2.5"
129 |                >
130 |                  <FriendAvatar name={c.full_name} size="sm" />
131 |                  <span className="min-w-0 flex-1 truncate text-sm font-semibold text-gray-900">
132 |                    {c.full_name}
133 |                  </span>
134 |                  <Button
135 |                    variant="tactile"
136 |                    className="h-8 px-3 text-xs"
137 |                    disabled={sendRequest.isPending}
138 |                    onClick={() => sendRequest.mutate(c.student_id)}
139 |                  >
140 |                    <UserPlus className="h-3.5 w-3.5" aria-hidden="true" />
141 |                    {t("friends.add", "Add")}
142 |                  </Button>
143 |                </div>
144 |              )
145 |            ) : (
146 |              <p className="py-2 text-center text-sm text-gray-500">
147 |                {t("friends.noMatches", "No classmates match that name.")}
148 |              </p>
149 |            )}
150 |          </div>
151 |        )}
152 |      </section>
153 |
154 |      {/** % % Incoming requests % % */}
155 |      {requestList.length > 0 && (
156 |        <section className={cn(CARD, "p-4")}>
157 |          <h2 className="mb-3 flex items-center gap-2 text-[13px] font-black tracking-tight text-slate-900">
158 |            <UserPlus className="h-4 w-4 text-sky-600" aria-hidden="true" />
159 |            {t("friends.requests", "Friend requests")}
160 |          <span className="rounded-full bg-sky-50 px-2 py-0.5 text-[10px] font-bold text-sky-700">
161 |            {requestList.length}
162 |          </span>
163 |        </h2>
164 |        <div className="space-y-2">
165 |          {requestList.map((r) => (
166 |            <div key={r.friendship_id} className="flex items-center gap-3">
167 |              <FriendAvatar name={r.full_name} size="sm" />
168 |              <span className="min-w-0 flex-1 truncate text-sm font-semibold text-gray-900">
169 |                {r.full_name}
170 |              </span>
171 |              <Button

```

```

172         variant="tactile"
173         className="h-8 px-3 text-xs"
174         disabled={respond.isPending}
175         onClick={() =>
176             respond.mutate({
177                 friendshipId: r.friendship_id,
178                 accept: true,
179             })
180         }
181     >
182     <Check className="h-3.5 w-3.5" aria-hidden="true" />
183     {t("friends.accept", "Accept")}
184 </Button>
185 <Button
186     variant="outline"
187     className="h-8 px-3 text-xs"
188     disabled={respond.isPending}
189     onClick={() =>
190         respond.mutate({
191             friendshipId: r.friendship_id,
192             accept: false,
193         })
194     }
195 >
196     <X className="h-3.5 w-3.5" aria-hidden="true" />
197     {t("friends.decline", "Decline")}
198 </Button>
199 </div>
200 }}
201 </div>
202 </section>
203 }}
204
205 {/* % % Online now % % */}
206 {onlineFriends.length > 0 && (
207     <section className={cn(CARD, "p-4")}>
208         <h2 className="mb-3 flex items-center gap-2 text-[13px] font-black tracking-tight text-slate-900">
209             <span
210                 className="h-2 w-2 rounded-full bg-green-500"
211                 aria-hidden="true"
212             />
213             {t("friends.onlineNow", "Online now")}
214             <span className="text-[11px] font-bold text-gray-400">
215                 {onlineFriends.length}
216             </span>
217         </h2>
218         <div className="no-scrollbar flex gap-4 overflow-x-auto pb-1">
219             {onlineFriends.map((f) => (
220                 <div
221                     key={f.student_id}
222                     className="flex w-16 shrink-0 flex-col items-center gap-1.5"
223                 >
224                     <FriendAvatar name={f.full_name} online />
225                     <span className="w-full truncate text-center text-[11px] text-gray-600">
226                         {f.full_name.split(" ")[0]}
227                     </span>
228                 </div>
229             ))}
230         </div>
231     </section>
232 }}
233
234 {/* % % All friends % % */}
235 <section className={cn(CARD, "p-4")}>
236     <h2 className="mb-3 flex items-center gap-2 text-[13px] font-black tracking-tight text-slate-900">
237         <Users className="h-4 w-4 text-sky-600" aria-hidden="true" />
238         {t("friends.allFriends", "All friends")}
239         {friendList.length > 0 && (
240             <span className="text-[11px] font-bold text-gray-400">
241                 {friendList.length}
242             </span>
243         )}
244     </h2>
245     {friends.isPending ? (
246         <div className="space-y-2">
247             {Array.from({ length: 3 }).map((_, i) => (
248                 <Shimmer key={i} className="h-14 rounded-xl" />
249             ))}
250         </div>
251     ) : friendList.length > 0 ? (
252         <div className="space-y-2">
253             {friendList.map((f) => (
254                 <FriendRow
255                     key={f.student_id}
256                     friend={f}
257                     onRemove={() => removeFriend.mutate(f.student_id)}
258                     removing={removeFriend.isPending}
259                     labels={{
260                         level: t("friends.level", "Lv {{n}}", { n: f.level }),
261                         remove: t("friends.remove", "Remove"),
262                     }}
263                 />
264             ))}
265         </div>
266     ) : (
267         <div className="flex flex-col items-center gap-2 py-8 text-center">
268             <div className="flex h-12 w-12 items-center justify-center rounded-2xl bg-blue-50">
269                 <Users className="h-6 w-6 text-blue-500" aria-hidden="true" />
270             </div>
271             <p className="max-w-xs text-sm text-gray-500">
272                 {t(
273                     "friends.empty",
274                     "No friends yet – search for a classmate above to send your first request."
275                 )}
276             </p>
277         </div>
278     )}
279 </section>

```



```

276 |         </p>
277 |     </div>
278 |   )}
279 | </section>
280 | </div>
281 | );
282 | };
283 |
284 | const FriendRow = ({
285 |   friend,
286 |   onRemove,
287 |   removing,
288 |   labels,
289 | }): {
290 |   friend: Friend;
291 |   onRemove: () => void;
292 |   removing: boolean;
293 |   labels: { level: string; remove: string };
294 | }) => (
295 |   <div className="flex items-center gap-3 rounded-xl border border-slate-100 bg-white p-2.5">
296 |     <FriendAvatar name={friend.full_name} online={friend.online} />
297 |     <div className="min-w-0 flex-1">
298 |       <p className="truncate text-sm font-bold text-gray-900">
299 |         {friend.full_name}
300 |       </p>
301 |       <div className="flex items-center gap-2 text-[11px] text-gray-500">
302 |         <span className="font-bold text-blue-600">{labels.level}</span>
303 |         {friend.streak_current > 0 && (
304 |           <span className="inline-flex items-center gap-0.5 text-orange-600">
305 |             <Flame className="h-3 w-3" aria-hidden="true" />
306 |             {friend.streak_current}
307 |           </span>
308 |         )}
309 |         <span className="font-semibold text-amber-600">
310 |           {friend.xp_total.toLocaleString()} XP
311 |         </span>
312 |       </div>
313 |     </div>
314 |     <Button
315 |       variant="ghost"
316 |       className="h-8 px-2 text-xs text-gray-400 hover:text-red-600"
317 |       disabled={removing}
318 |       onClick={onRemove}
319 |       aria-label={labels.remove}
320 |     >
321 |       <X className="h-4 w-4" aria-hidden="true" />
322 |     </Button>
323 |   </div>
324 | );
325 |
326 | export default StudentFriendsPage;
327 |

```

#### FILE: src/features/student/index.ts

TYPE: .ts | LINES: 1-12 | SHA256: 185fedbd3f2196f1616c8511a66362a0cc171ba0dc4b1ddbc8bd030d264d85a2

```

1 | /**
2 |  * L3 composition - Student.
3 |  *
4 |  * Role pages, widgets, and flows composed from `@/design-system` primitives +
5 |  * patterns and bound to EXISTING hooks in `src/hooks` (no new backend). Student
6 |  * treatment is the most gamified (rings, gradient bars, mascot, XP/streak).
7 |  *
8 |  * Populated across P2 (dashboard) and P3 (all student modules) per
9 |  * `missing-screens-catalog.md`. Empty until then (task 0.8).
10 |  */
11 | export {};
12 |

```

#### FILE: src/features/student/profile/StudentProfilePage.tsx

TYPE: .tsx | LINES: 1-592 | SHA256: 5b92bd9f4170af00f01ae080e42ba3a9aff6483c6323488ad6ea6b8544d4e1f0

```

1 | import { useMemo } from "react";
2 | import { Link } from "react-router-dom";
3 | import { useTranslation } from "react-i18next";
4 | import {
5 |   Award,
6 |   BookOpen,
7 |   CalendarDays,
8 |   Camera,
9 |   ChevronRight,
10 |  ClipboardList,
11 |  Edit3,
12 |  FileText,
13 |  Flame,
14 |  GraduationCap,
15 |  HeartPulse,
16 |  LockKeyhole,
17 |  Logout,
18 |  Palette,
19 |  Settings,
20 |  ShieldCheck,
21 |  Sparkles,
22 |  Star,
23 |  Target,
24 |  Trophy,
25 |  UserRound,
26 |  UsersRound,
27 |  WalletCards,
28 |  Zap,
29 | } from "lucide-react";
30 | import { Button, PCard, SectionHeader, Shimmer, Switch } from "@/design-system";
31 | import { useAuth } from "@/hooks/useAuth";
32 | import { useAnonymousStatus, useToggleAnonymous } from "@/hooks/useLeaderboard";

```

```

33 | import { useEquippedItems } from "@hooks/useEquippedItems";
34 | import { useHeatmapData } from "@hooks/useHeatmapData";
35 | import { useLevel } from "@hooks/useLevel";
36 | import { useStreak } from "@hooks/useStreak";
37 | import { useStudentProfile } from "@hooks/useStudentProfile";
38 | import { useTieredBadges } from "@hooks/useTieredBadges";
39 | import { cn } from "@lib/utills";
40 |
41 | const DAY_MS = 24 * 60 * 60 * 1000;
42 |
43 | const menuItems = [
44 |   { to: "/student/leaderboard", icon: Trophy, key: "leaderboard" },
45 |   { to: "/student/marketplace", icon: WalletCards, key: "marketplace" },
46 |   { to: "/student/habits", icon: HeartPulse, key: "wellness" },
47 |   { to: "/student/challenges", icon: Target, key: "quests" },
48 |   { to: "/student/portfolio", icon: ClipboardList, key: "portfolio" },
49 |   { to: "/student/journal", icon: BookOpen, key: "journal" },
50 |   { to: "/student/team", icon: UsersRound, key: "team" },
51 |   { to: "/student/calendar", icon: CalendarDays, key: "calendar" },
52 |   { to: "/student/transcript", icon: FileText, key: "transcript" },
53 |   { to: "/student/fees", icon: WalletCards, key: "fees" },
54 |   { to: "/student/surveys", icon: ClipboardList, key: "surveys" },
55 |   { to: "/student/learning-profile", icon: Sparkles, key: "learningProfile" },
56 |   { to: "/student/settings/profile", icon: Settings, key: "settings" },
57 | ] as const;
58 |
59 | const StudentProfilePage = () => {
60 |   const { t } = useTranslation("student");
61 |   const { user, profile, signOut } = useAuth();
62 |   const studentId = user?.id;
63 |
64 |   const level = useLevel(studentId);
65 |   const streak = useStreak();
66 |   const badges = useTieredBadges(studentId);
67 |   const learningProfile = useStudentProfile(studentId ?? "");
68 |   const equipped = useEquippedItems(studentId ?? "");
69 |   const anonymous = useAnonymousStatus();
70 |   const toggleAnonymous = useToggleAnonymous();
71 |
72 |   const activityRange = useMemo(() => {
73 |     const end = new Date();
74 |     const start = new Date(end.getTime() - 118 * DAY_MS);
75 |     return {
76 |       start: start.toISOString().slice(0, 10),
77 |       end: end.toISOString().slice(0, 10),
78 |     };
79 |   }, []);
80 |   const activity = useHeatmapData(studentId, activityRange);
81 |
82 |   const levelData = level.data;
83 |   const streakData = streak.data;
84 |   const earnedBadges = badges.data ?? [];
85 |   const activeDays =
86 |     activity.data?.filter((day) => day.totalCount > 0).length ?? 0;
87 |   const xpRemaining = levelData
88 |     ? Math.max(0, levelData.xpForNextLevel - levelData.xpTotal)
89 |     : 0;
90 |   const initials = (profile?.full_name ?? "Student")
91 |     .split(/\s+/)
92 |     .filter(Boolean)
93 |     .slice(0, 2)
94 |     .map((part) => part[0]?.toUpperCase())
95 |     .join("");
96 |
97 |   const learningStyle =
98 |     learningProfile.data?.learning_style?.dominant_style ?? null;
99 |
100 |   return (
101 |     <div className="space-y-5">
102 |       <SectionHeader
103 |         icon={UserRound}
104 |         title={t("profilePage.title")}
105 |         description={t("profilePage.subtitle")}
106 |         action={
107 |           <Button asChild variant="outline" size="sm">
108 |             <Link to="/student/settings/profile">
109 |               <Edit3 className="size-4" aria-hidden="true" />
110 |               {t("profilePage.edit")}
111 |             </Link>
112 |           </Button>
113 |         }
114 |       </div>
115 |
116 |     <PCard className="p-5">
117 |       <div className="grid gap-5 lg:grid-cols-[minmax(0,1fr)_auto]">
118 |         <div className="min-w-0">
119 |           <div className="flex items-center gap-4">
120 |             <Button
121 |               asChild
122 |               variant="ghost"
123 |               size="icon"
124 |               className="relative size-16 shrink-0 rounded-full bg-gradient-to-br from-teal-500 to-blue-600"
125 |             >
126 |               <Link
127 |                 to="/student/settings/profile"
128 |                 aria-label={t("profilePage.changePhoto")}
129 |               >
130 |                 {profile?.avatar_url ? (
131 |                   <img
132 |                     src={profile.avatar_url}
133 |                     alt=""
134 |                     className="size-16 rounded-full object-cover"
135 |                   />
136 |                 ) : (

```

```

137 |         <span>{initials || "S"}</span>
138 |     ))
139 |     <span className="absolute -bottom-1 -end-1 flex size-6 items-center justify-center rounded-f
140 |         <Camera className="size-3" aria-hidden="true" />
141 |     </span>
142 | </Link>
143 | </Button>
144 | <div className="min-w-0">
145 |     <h1 className="truncate text-2xl font-black tracking-tight text-slate-950">
146 |         {profile?.full_name ?? t("profilePage.student")}
147 |     </h1>
148 |     <p className="mt-1 text-sm font-semibold text-slate-600">
149 |         {t("profilePage.student")}
150 |     </p>
151 |     <p className="mt-1 truncate text-xs text-slate-500">
152 |         {profile?.email}
153 |     </p>
154 | </div>
155 | </div>
156 |
157 | <div className="mt-5 grid grid-cols-2 gap-2 sm:grid-cols-4">
158 |     <ProfileStat
159 |         icon={Star}
160 |         value={level.isLoading ? null : levelData?.level ?? 1}
161 |         label={t("profilePage.level")}
162 |         tone="amber"
163 |     />
164 |     <ProfileStat
165 |         icon={Zap}
166 |         value={level.isLoading ? null : levelData?.xpTotal ?? 0}
167 |         label={t("profilePage.totalXp")}
168 |         tone="blue"
169 |     />
170 |     <ProfileStat
171 |         icon={Flame}
172 |         value={streak.isLoading ? null : streakData?.streak_count ?? 0}
173 |         label={t("profilePage.dayStreak")}
174 |         tone="red"
175 |     />
176 |     <ProfileStat
177 |         icon={Award}
178 |         value={badges.isLoading ? null : earnedBadges.length}
179 |         label={t("profilePage.badges")}
180 |         tone="violet"
181 |     />
182 | </div>
183 | </div>
184 |
185 | <div className="flex min-w-56 flex-col gap-2 lg:items-stretch">
186 |     <Button asChild variant="outline" size="sm">
187 |         <Link to="/student/settings/profile">
188 |             <Edit3 className="size-4" aria-hidden="true" />
189 |             {t("profilePage.editProfile")}
190 |         </Link>
191 |     </Button>
192 |     <Button asChild variant="tactile" size="sm">
193 |         <Link to="/student/portfolio">
194 |             <UserRound className="size-4" aria-hidden="true" />
195 |             {t("profilePage.viewPortfolio")}
196 |         </Link>
197 |     </Button>
198 | </div>
199 | </div>
200 | </PCard>
201 |
202 | <div className="grid gap-5 lg:grid-cols-2">
203 |     <PCard className="p-4">
204 |         <div className="flex items-center justify-between gap-3">
205 |             <p className="font-black text-slate-900">
206 |                 {t("profilePage.levelProgress", {
207 |                     current: levelData?.level ?? 1,
208 |                     next: (levelData?.level ?? 1) + 1,
209 |                 })}
210 |             </p>
211 |             <p className="text-xs font-semibold text-slate-500">
212 |                 {levelData
213 |                     ? `${levelData.xpTotal.toLocaleString()} / ${levelData.xpForNextLevel.toLocaleString()} XP`
214 |                     : "-" }
215 |             </p>
216 |         </div>
217 |         <div
218 |             className="mt-3 h-3 overflow-hidden rounded-full bg-slate-100"
219 |             role="progressbar"
220 |             aria-label={t("profilePage.level")}
221 |             aria-valuemin={0}
222 |             aria-valuemax={100}
223 |             aria-valuenow={levelData?.progressPercent ?? 0}
224 |         >
225 |             <div
226 |                 className="h-full rounded-full bg-[image:var(--brand-gradient)]"
227 |                 style={{ width: `${levelData?.progressPercent ?? 0}%` }}
228 |             />
229 |         </div>
230 |         <p className="mt-2 text-xs text-slate-500">
231 |             {t("profilePage.xpRemaining", { count: xpRemaining })}
232 |         </p>
233 |     </PCard>
234 |
235 |     <PCard className="p-4">
236 |         <div className="flex items-center gap-3">
237 |             <span className="flex size-11 items-center justify-center rounded-xl bg-orange-50 text-orange-600"
238 |                 <Flame className="size-6" aria-hidden="true" />
239 |             </span>
240 |             <div className="min-w-0 flex-1">

```

```

241 |         <p className="text-2xl font-black text-orange-600">
242 |             {streakData?.streak_count ?? 0}
243 |             <span className="ms-2 text-sm font-semibold text-slate-500">
244 |                 {t("profilePage.dayStreak")}
245 |             </span>
246 |         </p>
247 |         <p className="text-xs text-slate-500">
248 |             {t("profilePage.freezes", {
249 |                 count: streakData?.streak_freezes_available ?? 0,
250 |             })}
251 |         </p>
252 |     </div>
253 | </div>
254 | </PCard>
255 | </div>
256 |
257 | <div className="grid gap-5 lg:grid-cols-2">
258 |     <PCard className="p-4">
259 |         <div className="mb-3 flex items-center justify-between gap-3">
260 |             <p className="text-xs font-black uppercase tracking-[0.12em] text-slate-600">
261 |                 {t("profilePage.badgesCount", { count: earnedBadges.length })}
262 |             </p>
263 |             <Link
264 |                 to="/student/badges"
265 |                 className="text-xs font-bold text-blue-700 hover:underline"
266 |             >
267 |                 {t("profilePage.viewAll")}
268 |             </Link>
269 |         </div>
270 |         {badges.isLoading ? (
271 |             <div className="grid grid-cols-4 gap-3">
272 |                 {Array.from({ length: 8 }).map((_, index) => (
273 |                     <Shimmer key={index} className="h-20 rounded-xl" />
274 |                 ))}
275 |             </div>
276 |         ) : earnedBadges.length > 0 ? (
277 |             <div className="grid grid-cols-2 gap-3 sm:grid-cols-4">
278 |                 {earnedBadges.slice(0, 8).map((badge) => (
279 |                     <div
280 |                         key={badge.id}
281 |                         className={cn(
282 |                             "flex min-h-20 flex-col items-center justify-center gap-1 rounded-xl border-2 p-2 text-cen
283 |                             badge.tier === "gold" && "border-amber-400",
284 |                             badge.tier === "silver" && "border-slate-300",
285 |                             badge.tier === "bronze" && "border-orange-300",
286 |                             badge.tier == null && "border-slate-200"
287 |                         )}
288 |                     >
289 |                         <Award className="size-6 text-amber-600" aria-hidden="true" />
290 |                         <span className="line-clamp-2 text-[10px] font-black text-slate-800">
291 |                             {badge.name}
292 |                         </span>
293 |                     </div>
294 |                 ))}
295 |             </div>
296 |         ) : (
297 |             <p className="py-8 text-center text-sm text-slate-500">
298 |                 {t("profilePage.noBadges")}
299 |             </p>
300 |         )}
301 |     </PCard>
302 |
303 |     <PCard className="p-4">
304 |         <div className="mb-3 flex items-center justify-between gap-3">
305 |             <p className="text-xs font-black uppercase tracking-[0.12em] text-slate-600">
306 |                 {t("profilePage.learningActivity")}
307 |             </p>
308 |             <span className="text-[11px] font-semibold text-slate-500">
309 |                 {t("profilePage.lastWeeks", { count: 17 })}
310 |             </span>
311 |         </div>
312 |         {activity.isLoading ? (
313 |             <Shimmer className="h-28 rounded-xl" />
314 |         ) : (
315 |             <div className="grid grid-flow-col grid-rows-7 gap-1 overflow-x-auto pb-1">
316 |                 {(activity.data ?? []).map((day) => (
317 |                     <span
318 |                         key={day.date}
319 |                         title={ `${day.date}: ${day.totalCount}` }
320 |                         className={cn(
321 |                             "size-3 rounded-[3px]",
322 |                             day.totalCount === 0 && "bg-slate-100",
323 |                             day.totalCount === 1 && "bg-emerald-200",
324 |                             day.totalCount === 2 && "bg-emerald-400",
325 |                             day.totalCount === 3 && "bg-emerald-600",
326 |                             day.totalCount >= 4 && "bg-emerald-800"
327 |                         )}
328 |                     </span>
329 |                 ))}
330 |             </div>
331 |         </div>
332 |         <p className="mt-3 text-xs text-slate-600">
333 |             <strong className="text-slate-900">{activeDays}</strong>{" "}
334 |             {t("profilePage.activeDays")}
335 |         </p>
336 |     </div>
337 | </PCard>
338 | </div>
339 |
340 |
341 | <div className="grid gap-5 lg:grid-cols-2">
342 |     <PCard className="overflow-hidden p-0">
343 |         <nav aria-label={t("profilePage.profileLinks")}>
344 |             {menuItems.map(({ to, icon: Icon, key }) => (

```

```

345 |         <Link
346 |             key={to}
347 |             to={to}
348 |             className="flex min-h-12 items-center gap-3 border-b border-slate-100 px-4 py-3 text-sm font-s
349 |         >
350 |             <Icon className="size-5 text-slate-500" aria-hidden="true" />
351 |             <span className="min-w-0 flex-1">
352 |                 {t(`profilePage.links.${key}`)}
353 |             </span>
354 |             <ChevronRight
355 |                 className="size-4 text-slate-400 rtl:rotate-180"
356 |                 aria-hidden="true"
357 |             />
358 |         </Link>
359 |     )}
360 | </nav>
361 | </PCard>
362 |
363 | <div className="space-y-5">
364 |     <PCard className="overflow-hidden p-0">
365 |         <ProfileCardHeading
366 |             icon={Sparkles}
367 |             title={t("profilePage.learningProfile")}
368 |         />
369 |         <div className="space-y-3 p-4">
370 |             <ProfileValue
371 |                 label={t("profilePage.learningStyle")}
372 |                 value={
373 |                     learningStyle
374 |                     ? t(`learningProfile.styles.${learningStyle}`)
375 |                     : t("profilePage.notComplete")
376 |                 }
377 |             />
378 |             <ProfileValue
379 |                 label={t("learningProfile.completeness")}
380 |                 value={
381 |                     `${Math.round(
382 |                         learningProfile.data?.profile_completeness ?? 0
383 |                     )}%`
384 |                 }
385 |             />
386 |             <div
387 |                 className="h-2 overflow-hidden rounded-full bg-slate-100"
388 |                 role="progressbar"
389 |                 aria-label={t("learningProfile.completeness")}
390 |                 aria-valuemin={0}
391 |                 aria-valuemax={100}
392 |                 aria-valuenow={learningProfile.data?.profile_completeness ?? 0}
393 |             >
394 |                 <div
395 |                     className="h-full rounded-full bg-[image:var(--brand-gradient)]"
396 |                     style={{
397 |                         width: `${
398 |                             learningProfile.data?.profile_completeness ?? 0
399 |                         }%`,
400 |                     }}
401 |                 />
402 |             </div>
403 |         </PCard>
404 |
405 |         <PCard className="overflow-hidden p-0">
406 |             <ProfileCardHeading
407 |                 icon={ShieldCheck}
408 |                 title={t("profilePage.privacy")}
409 |             />
410 |             <div className="flex items-center justify-between gap-4 p-4">
411 |                 <div>
412 |                     <p className="text-sm font-bold text-slate-900">
413 |                         {t("profilePage.anonymous")}
414 |                     </p>
415 |                     <p className="mt-1 text-xs text-slate-500">
416 |                         {t("profilePage.anonymousDescription")}
417 |                     </p>
418 |                 </div>
419 |                 <Switch
420 |                     checked={anonymous.data?.isAnonymous ?? false}
421 |                     disabled={anonymous.isLoading || toggleAnonymous.isPending}
422 |                     onCheckedChange={() => toggleAnonymous.mutate()}
423 |                     aria-label={t("profilePage.anonymous")}
424 |                 />
425 |             </div>
426 |         </PCard>
427 |
428 |         <PCard className="overflow-hidden p-0">
429 |             <ProfileCardHeading
430 |                 icon={Palette}
431 |                 title={t("profilePage.equipped")}
432 |             />
433 |             <div className="divide-y divide-slate-100">
434 |                 {equipped.isLoading ? (
435 |                     <div className="space-y-2 p-4">
436 |                         <Shimmer className="h-8 rounded-lg" />
437 |                         <Shimmer className="h-8 rounded-lg" />
438 |                     </div>
439 |                 ) : (equipped.data ?? []).length > 0 ? (
440 |                     equipped.data?.map((item) => (
441 |                         <ProfileValue
442 |                             key={item.id}
443 |                             label={t(`profilePage.slots.${item.slot}`)}
444 |                             value={item.item_name}
445 |                             className="px-4 py-3"
446 |                         />
447 |                     ))
448 |                 ) : (

```

```

449 |         {t("profilePage.noEquipped")}
450 |     </p>
451 |     })
452 | </div>
453 | </PCard>
454 | </div>
455 | </div>
456 |
457 | <PCard className="overflow-hidden p-0">
458 |   <ProfileCardHeading
459 |     icon={LockKeyhole}
460 |     title={t("profilePage.security")}
461 |   />
462 |   <div className="grid gap-px bg-slate-100 sm:grid-cols-2">
463 |     <SecurityLink
464 |       to="/update-password"
465 |       icon={LockKeyhole}
466 |       title={t("profilePage.changePassword")}
467 |     />
468 |     <SecurityLink
469 |       to="/student/sessions"
470 |       icon={ShieldCheck}
471 |       title={t("profilePage.activeSessions")}
472 |     />
473 |     <SecurityLink
474 |       to="/student/transcript"
475 |       icon={GraduationCap}
476 |       title={t("profilePage.transcript")}
477 |     />
478 |     <SecurityLink
479 |       to="/student/settings/profile"
480 |       icon={Settings}
481 |       title={t("profilePage.dataControls")}
482 |     />
483 |   </div>
484 | </PCard>
485 |
486 | <Button
487 |   type="button"
488 |   variant="outline"
489 |   className="w-full border-red-200 bg-red-50 text-red-700 hover:bg-red-100 hover:text-red-800"
490 |   onClick={() => void signOut()}
491 | >
492 |   <LogOut className="size-4" aria-hidden="true" />
493 |   {t("profilePage.signOut")}
494 | </Button>
495 | </div>
496 | );
497 | };
498 |
499 | const ProfileStat = ({
500 |   icon: Icon,
501 |   value,
502 |   label,
503 |   tone,
504 | }): {
505 |   icon: typeof Star;
506 |   value: number | null;
507 |   label: string;
508 |   tone: "amber" | "blue" | "red" | "violet";
509 | }) => {
510 |   const tones = {
511 |     amber: "bg-amber-50 text-amber-700",
512 |     blue: "bg-blue-50 text-blue-700",
513 |     red: "bg-red-50 text-red-700",
514 |     violet: "bg-violet-50 text-violet-700",
515 |   };
516 |
517 |   return (
518 |     <div className="flex items-center gap-2 rounded-xl border border-slate-100 bg-white p-2.5">
519 |       <span
520 |         className={cn(
521 |           "flex size-9 shrink-0 items-center justify-center rounded-lg",
522 |           tones[tone]
523 |         )}
524 |       >
525 |         <Icon className="size-4" aria-hidden="true" />
526 |       </span>
527 |       <span className="min-w-0">
528 |         <strong className="block text-base font-black text-slate-950">
529 |           {value == null ? "-" : value.toLocaleString()}
530 |         </strong>
531 |         <span className="block truncate text-[10px] font-bold uppercase tracking-wide text-slate-500">
532 |           {label}
533 |         </span>
534 |       </span>
535 |     </div>
536 |   );
537 | };
538 |
539 | const ProfileCardHeading = ({
540 |   icon: Icon,
541 |   title,
542 | }): {
543 |   icon: typeof Star;
544 |   title: string;
545 | }) => (
546 |   <div className="flex items-center gap-2 border-b border-slate-100 px-4 py-3">
547 |     <Icon className="size-4 text-teal-700" aria-hidden="true" />
548 |     <h2 className="text-xs font-black uppercase tracking-[0.12em] text-slate-600">
549 |       {title}
550 |     </h2>
551 |   </div>
552 | );

```

```

553 |
554 | const ProfileValue = ({
555 |   label,
556 |   value,
557 |   className,
558 | }): {
559 |   label: string;
560 |   value: string;
561 |   className?: string;
562 | }) => (
563 |   <div className={cn("flex items-center justify-between gap-4", className)}>
564 |     <span className="text-sm text-slate-600">{label}</span>
565 |     <strong className="text-end text-sm text-slate-900">{value}</strong>
566 |   </div>
567 | );
568 |
569 | const SecurityLink = ({
570 |   to,
571 |   icon: Icon,
572 |   title,
573 | }): {
574 |   to: string;
575 |   icon: typeof Star;
576 |   title: string;
577 | }) => (
578 |   <Link
579 |     to={to}
580 |     className="flex min-h-14 items-center gap-3 bg-white px-4 py-3 text-sm font-bold text-slate-800 transition
581 |   >
582 |     <Icon className="size-4 text-slate-500" aria-hidden="true" />
583 |     <span className="min-w-0 flex-1">{title}</span>
584 |     <ChevronRight
585 |       className="size-4 text-slate-400 rtl:rotate-180"
586 |       aria-hidden="true"
587 |     />
588 |   </Link>
589 | );
590 |
591 | export default StudentProfilePage;
592 |

```

## FILE: src/features/student/quizzes/PostQuizReviewScreen.tsx

TYPE: .tsx | LINES: 1-408 | SHA256: bbf0629c6d6af0c1382873d2da91e0e6b6b01e665905261aad5afc16d1fddd5a

```

1 | import { useMemo } from "react";
2 | import { Link, useParams } from "react-router-dom";
3 | import { useTranslation } from "react-i18next";
4 | import {
5 |   ArrowLeft,
6 |   CheckCircle2,
7 |   HelpCircle,
8 |   TrendingUp,
9 |   XCircle,
10 | } from "lucide-react";
11 |
12 | import ErrorState from "@components/shared/ErrorState";
13 | import QuestionPreview from "@components/shared/QuestionPreview";
14 | import ExplanationConfidenceBadge from "@components/shared/ExplanationConfidenceBadge";
15 | import { InlineNoCLOs } from "@components/shared/EmptyState";
16 | import { BloomsProgressionLadder } from "@components/shared/BloomsProgressionLadder";
17 | import { Button, Badge, PCard, SectionHeader, Shimmer } from "@design-system";
18 | import { computePerCLOScore } from "@lib/questionAnalytics";
19 | import { useQuizReview, type ReviewQuestion } from "@hooks/useQuizReview";
20 | import {
21 |   useVerifiedExplanation,
22 |   useExplanationConfidence,
23 | } from "@hooks/useExplanationConfidence";
24 | import { useBloomsClimbState } from "@hooks/useBloomsProgression";
25 |
26 | const BLOOM_LABEL_KEYS: Record<number, string> = {
27 |   1: "blooms.remembering",
28 |   2: "blooms.understanding",
29 |   3: "blooms.applying",
30 |   4: "blooms.analyzing",
31 |   5: "blooms.evaluating",
32 |   6: "blooms.creating",
33 | };
34 |
35 | const QuestionExplanation = ({
36 |   questionId,
37 |   aiExplanation,
38 | }): {
39 |   questionId: string;
40 |   aiExplanation: string | null;
41 | }) => {
42 |   const { t } = useTranslation("student");
43 |   const { data: verifiedExplanation } = useVerifiedExplanation(questionId);
44 |   const { data: confidence } = useExplanationConfidence(questionId);
45 |
46 |   const isVerified = !!verifiedExplanation;
47 |   const displayText = isVerified
48 |     ? verifiedExplanation.explanation_text
49 |     : aiExplanation;
50 |
51 |   if (!displayText) return null;
52 |
53 |   return (
54 |     <div className="rounded-2xl bg-blue-50/80 p-4">
55 |       <div className="mb-2 flex flex-wrap items-center gap-2">
56 |         <p className="text-xs font-black uppercase tracking-[0.24em] text-blue-700">
57 |           {isVerified
58 |             ? t("quiz.review.verifiedExplanation")
59 |             : t("quiz.review.aiExplanation")}
60 |         </p>

```

```

61 |         <ExplanationConfidenceBadge
62 |             confidence={confidence ?? null}
63 |             isVerified={isVerified}
64 |         />
65 |     </div>
66 |     <p className="text-sm leading-6 text-slate-700">{displayText}</p>
67 | </div>
68 | );
69 | };
70 |
71 | const QuestionReviewCard = ({
72 |     question,
73 |     index,
74 |     answer,
75 | }): {
76 |     question: ReviewQuestion;
77 |     index: number;
78 |     answer: string;
79 | }) => {
80 |     const { t } = useTranslation("student");
81 |     const isCorrect = answer === question.correct_answer.value;
82 |     const bloomLabelKey = BLOOM_LABEL_KEYS[question.bloom_level];
83 |     const bloomLabel = bloomLabelKey
84 |       ? t(bloomLabelKey, { ns: "common" })
85 |       : t("quiz.review.bloomLevel", { level: question.bloom_level });
86 |
87 |     return (
88 |       <PCard className="overflow-hidden p-0">
89 |         <div className="flex items-center justify-between gap-3 border-b border-slate-100 px-4 py-3">
90 |           <div className="min-w-0">
91 |             <p className="text-[10px] font-black uppercase tracking-[0.24em] text-muted-foreground">
92 |               {t("quiz.review.questionLabel", {
93 |                 defaultValue: "Question {{count}}",
94 |                 count: index + 1,
95 |               })}
96 |             </p>
97 |             <h3 className="truncate text-sm font-bold text-foreground">
98 |               {question.clo_title ?? question.clo_id}
99 |             </h3>
100 |           </div>
101 |           <div className="flex shrink-0 items-center gap-2">
102 |             <Badge
103 |               variant={isCorrect ? "default" : "secondary"}
104 |               className="rounded-full px-3 py-1 text-[11px]"
105 |             >
106 |               {isCorrect ? (
107 |                 <CheckCircle2 className="me-1 size-3.5" />
108 |               ) : (
109 |                 <XCircle className="me-1 size-3.5" />
110 |               )}
111 |               {isCorrect ? t("quiz.review.correct") : t("quiz.review.incorrect")}
112 |             </Badge>
113 |             <Badge
114 |               variant="outline"
115 |               className="rounded-full px-3 py-1 text-[11px]"
116 |             >
117 |               {bloomLabel}
118 |             </Badge>
119 |           </div>
120 |         </div>
121 |
122 |         <div className="space-y-4 p-4">
123 |           <QuestionPreview
124 |             questionText={question.question_text}
125 |             questionType={question.question_type}
126 |             options={question.options}
127 |             selectedAnswer={answer}
128 |             showCorrectAnswer
129 |             correctAnswer={question.correct_answer.value}
130 |             disabled
131 |           />
132 |
133 |           <QuestionExplanation
134 |             questionId={question.id}
135 |             aiExplanation={question.explanation}
136 |           />
137 |
138 |           {!isCorrect ? (
139 |             <Button asChild variant="outline" size="sm">
140 |               <Link to={`/student/tutor?cloIds=${question.clo_id}`}>
141 |                 <HelpCircle className="me-1 size-4" aria-hidden="true" />
142 |                 {t("quiz.review.getHelp")}
143 |               </Link>
144 |             </Button>
145 |           ) : null}
146 |         </div>
147 |       </PCard>
148 |     );
149 | };
150 |
151 | const PostQuizReviewScreen = () => {
152 |     const { t } = useTranslation("student");
153 |     const { quizId, attemptId } = useParams<{
154 |         quizId: string;
155 |         attemptId: string;
156 |     }>();
157 |
158 |     const review = useQuizReview(quizId, attemptId);
159 |     const climbState = useBloomsClimbState(attemptId ?? "");
160 |
161 |     const data = review.data;
162 |     const highestBloomLevel = climbState.data?.highest_level_reached ?? 0;
163 |
164 |     const perCLOScores = useMemo(() => {

```



```

165 |     if (!data) return {};
166 |     const answerDetails = data.questions.map((question) => ({
167 |       clo_id: question.clo_id,
168 |       is_correct:
169 |         data.attempt.answers[question.id] === question.correct_answer.value,
170 |     }));
171 |     return computePerCLOScore(answerDetails);
172 | }, [data]);
173 |
174 | const cloTitleMap = useMemo(() => {
175 |   if (!data) return new Map<string, string>();
176 |   return new Map(
177 |     data.questions.map((question) => [
178 |       question.clo_id,
179 |       question.clo_title ?? question.clo_id,
180 |     ])
181 |   );
182 | }, [data]);
183 |
184 | const uniqueCloIds = useMemo(
185 |   () =>
186 |     data
187 |     ? [...new Set(data.questions.map((question) => question.clo_id))]
188 |     : [],
189 |   [data]
190 | );
191 |
192 | if (review.isLoading) {
193 |   return (
194 |     <div className="space-y-4">
195 |       <div className="flex items-center gap-3">
196 |         <Shimmer className="h-8 w-40 rounded-2xl" />
197 |         <Shimmer className="h-8 w-56 rounded-2xl" />
198 |       </div>
199 |       <Shimmer className="h-48 rounded-3xl" />
200 |       <Shimmer className="h-64 rounded-3xl" />
201 |     </div>
202 |   );
203 | }
204 |
205 | if (review.isError || !data) {
206 |   return (
207 |     <ErrorState
208 |       title={t("quiz.review.loadFailed")}
209 |       message={t(
210 |         "quiz.review.loadFailedDescription",
211 |         "We couldn't load this quiz review."
212 |       )}
213 |       onRetry={() => {
214 |         void review.refetch();
215 |         void climbState.refetch();
216 |       }}
217 |       retryLabel={t("common:buttons.retry", "Try again")}
218 |     >
219 |       <Link to="/student/dashboard">
220 |         <Button variant="outline" className="mt-2">
221 |           <ArrowLeft className="me-1 size-4" />
222 |           {t("quiz.review.backToDashboard")}
223 |         </Button>
224 |       </Link>
225 |     </ErrorState>
226 |   );
227 | }
228 |
229 | const answeredCount = data.questions.reduce(
230 |   (count, question) => count + (data.attempt.answers[question.id] ? 1 : 0),
231 |   0
232 | );
233 | const correctCount = data.questions.reduce(
234 |   (count, question) =>
235 |     count +
236 |     (data.attempt.answers[question.id] === question.correct_answer.value
237 |       ? 1
238 |       : 0),
239 |   0
240 | );
241 |
242 | return (
243 |   <div className="space-y-5">
244 |     <div className="flex items-center gap-3">
245 |       <Link
246 |         to="/student/dashboard"
247 |         className="rounded-full focus:outline-none focus-visible:ring-2 focus-visible:ring-ring"
248 |       >
249 |         <Button variant="ghost" size="sm" className="text-muted-foreground">
250 |           <ArrowLeft className="me-1 size-4" />
251 |           {t("quiz.review.backToDashboard")}
252 |         </Button>
253 |       </Link>
254 |       <div className="min-w-0">
255 |         <p className="text-sm text-muted-foreground">
256 |           {t("quiz.review.title")}
257 |         </p>
258 |         <h1 className="truncate text-2xl font-black tracking-tight text-foreground">
259 |           {t("quiz.review.overallScore")}{" "}
260 |           <span className="text-teal-700">{data.attempt.score}%</span>
261 |         </h1>
262 |       </div>
263 |     </div>
264 |
265 |     <div className="grid gap-4 lg:grid-cols-[1.15fr_0.85fr]">
266 |       <PCard className="p-5">
267 |         <SectionHeader
268 |           icon={CheckCircle2}

```

```

269 |         title={t("quiz.review.overallScore")}
270 |         description={t(
271 |             "quiz.review.summaryHint",
272 |             "Here's how the attempt landed across the CLOs you touched."
273 |         )}
274 |     />
275 |
276 |     <div className="mt-4 grid gap-3 sm:grid-cols-3">
277 |         <div className="rounded-2xl border border-slate-100 bg-slate-50 p-4 text-center">
278 |             <p className="text-[10px] font-black uppercase tracking-[0.24em] text-muted-foreground">
279 |                 {t("quiz.review.score")}
280 |             </p>
281 |             <p className="mt-1 text-3xl font-black text-foreground">
282 |                 {data.attempt.score}%
283 |             </p>
284 |         </div>
285 |         <div className="rounded-2xl border border-slate-100 bg-slate-50 p-4 text-center">
286 |             <p className="text-[10px] font-black uppercase tracking-[0.24em] text-muted-foreground">
287 |                 {t("quiz.review.answered", "Answered")}
288 |             </p>
289 |             <p className="mt-1 text-3xl font-black text-foreground">
290 |                 {answeredCount}
291 |             </p>
292 |         </div>
293 |         <div className="rounded-2xl border border-slate-100 bg-slate-50 p-4 text-center">
294 |             <p className="text-[10px] font-black uppercase tracking-[0.24em] text-muted-foreground">
295 |                 {t("quiz.review.correct", "Correct")}
296 |             </p>
297 |             <p className="mt-1 text-3xl font-black text-foreground">
298 |                 {correctCount}
299 |             </p>
300 |         </div>
301 |     </div>
302 |
303 |     <div className="mt-4 flex items-center gap-2">
304 |         <Badge className="rounded-full px-3 py-1 text-[11px]">
305 |             {t("quiz.review.questions", {
306 |                 defaultValue: "{{count}} questions",
307 |                 count: data.questions.length,
308 |             })}
309 |         </Badge>
310 |         <Badge
311 |             variant="outline"
312 |             className="rounded-full px-3 py-1 text-[11px]"
313 |             >
314 |             {t("quiz.review.attempt", {
315 |                 defaultValue: "Attempt {{id}}",
316 |                 id: data.attempt.id.slice(0, 8),
317 |             })}
318 |         </Badge>
319 |     </div>
320 | </PCard>
321 |
322 | <PCard className="p-5">
323 |     <SectionHeader
324 |         icon={TrendingUp}
325 |         title={t("quiz.review.bloomsProgression")}
326 |         description={t(
327 |             "quiz.review.bloomsProgressionHint",
328 |             "Review the Bloom's path and where your confidence climbed."
329 |         )}
330 |     />
331 |
332 |     {climbState.data && uniqueCloIds.length > 0 ? (
333 |         <div className="mt-4 grid gap-3 sm:grid-cols-2">
334 |             {uniqueCloIds.map((cloId) => (
335 |                 <BloomsProgressionLadder
336 |                     key={cloId}
337 |                     highestLevel={highestBloomLevel}
338 |                     cloTitle={cloTitleMap.get(cloId) ?? cloId}
339 |                     compact
340 |                 />
341 |             ))}
342 |         </div>
343 |     ) : (
344 |         <div className="mt-4 rounded-2xl border border-slate-100 bg-slate-50 p-4 text-sm text-muted-foregr
345 |             {t(
346 |                 "quiz.review.noBloomData",
347 |                 "No Bloom's progression data was recorded."
348 |             )}
349 |         </div>
350 |     )}
351 | </PCard>
352 | </div>
353 |
354 | <PCard className="p-5">
355 |     <SectionHeader
356 |         icon={HelpCircle}
357 |         title={t("quiz.review.perCloBreakdown")}
358 |         description={t(
359 |             "quiz.review.perCloBreakdownHint",
360 |             "How each CLO performed in this attempt."
361 |         )}
362 |     />
363 |
364 |     <div className="mt-4 space-y-4">
365 |         {Object.entries(perCLOScores).length > 0 ? (
366 |             Object.entries(perCLOScores).map(([cloId, score]) => (
367 |                 <div key={cloId} className="space-y-1.5">
368 |                     <div className="flex items-center justify-between gap-3">
369 |                         <span className="truncate text-sm font-medium text-foreground">
370 |                             {cloTitleMap.get(cloId) ?? cloId}
371 |                         </span>
372 |                         <Badge

```

```

373 |         variant="outline"
374 |         className="rounded-full px-3 py-1 text-[11px]"
375 |     >
376 |         {Math.round(score)}%
377 |     </Badge>
378 | </div>
379 | <div className="h-2 overflow-hidden rounded-full bg-slate-100">
380 |     <div
381 |         className="h-full rounded-full bg-gradient-to-r from-teal-500 to-blue-500"
382 |         style={{ width: `${Math.min(score, 100)}%` }}
383 |     />
384 | </div>
385 | </div>
386 | ))
387 | ) : (
388 |     <InlineNoClos />
389 | )}
390 | </div>
391 | </PCard>
392 |
393 | <div className="space-y-4">
394 |     {data.questions.map((question, index) => (
395 |         <QuestionReviewCard
396 |             key={question.id}
397 |             question={question}
398 |             index={index}
399 |             answer={data.attempt.answers[question.id] ?? ""}
400 |         />
401 |     ))}
402 | </div>
403 | </div>
404 | );
405 | };
406 |
407 | export default PostQuizReviewScreen;
408 |

```

FILE: src/features/student/rails/StudentAssignmentRail.tsx

TYPE: .tsx | LINES: 1-75 | SHA256: 571d82650e8cce34a580d0f6fdfd1956d2b83b190ed8feffda8e60617a46f43a

```

1 | // =====
2 | // StudentAssignmentRail – right rail for the student Assignment detail page
3 | // (prototype `railHTML()` `page==='assignment' || 'lesson' case in shared.js):
4 | //   Need a hand? · Similar past work · Have a perk?
5 | //
6 | // The prototype's two support cards are navigation prompts (AI Tutor + the
7 | // Marketplace inventory) – reproduced here as real in-app links (no backend
8 | // needed). GAP: "Similar past work" with per-assignment scores has no backing
9 | // hook, so that card is omitted rather than fabricated (R17).
10 | // =====
11 |
12 | import { useNavigate } from "react-router-dom";
13 | import { useTranslation } from "react-i18next";
14 |
15 | import { RailCard, RailHead } from "@design-system";
16 | import { Button } from "@components/ui/button";
17 |
18 | const RailLink = ({ to, label }: { to: string; label: string }) => {
19 |   const navigate = useNavigate();
20 |   return (
21 |     <Button
22 |       type="button"
23 |       variant="link"
24 |       size="sm"
25 |       onClick={() => navigate(to)}
26 |       className="mt-2 h-auto px-0 text-xs font-extrabold text-blue-600"
27 |     >
28 |       {label}
29 |     </Button>
30 |   );
31 | };
32 |
33 | const StudentAssignmentRail = () => {
34 |   const { t } = useTranslation("student");
35 |
36 |   return (
37 |     <aside
38 |       aria-label={t("assignment.rail.label", "Assignment help")}
39 |       className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto px-5 py-4 xl:sticky xl:top-[va
40 |     >
41 |       { /* % % Need a hand? (AI Tutor) % % */ }
42 |       <RailCard>
43 |         <RailHead title={t("assignment.rail.needHand", "ðŸ’ Need a hand?")} />
44 |         <p className="text-xs text-slate-600 dark:text-slate-300">
45 |           {t(
46 |             "assignment.rail.needHandBody",
47 |             "The AI Tutor can walk through this step by step – no penalty for asking."
48 |           )}
49 |         </p>
50 |         <RailLink
51 |           to="/student/tutor"
52 |           label={t("assignment.rail.askTutor", "Ask the Tutor !")}
53 |         />
54 |       </RailCard>
55 |
56 |       { /* % % Have a perk? (Marketplace inventory) % % */ }
57 |       <RailCard>
58 |         <RailHead title={t("assignment.rail.perk", "ðŸ’ Have a perk?")} />
59 |         <p className="text-xs text-slate-600 dark:text-slate-300">
60 |           {t(
61 |             "assignment.rail.perkBody",
62 |             "Extra-attempt or deadline-extension tokens live in your inventory."
63 |           )}
64 |         </p>

```

```

65 |         <RailLink
66 |           to="/student/marketplace"
67 |           label={t("assignment.rail.openMarketplace", "Open Marketplace !")}
68 |         />
69 |       </RailCard>
70 |     </aside>
71 |   );
72 | };
73 |
74 | export default StudentAssignmentRail;
75 |

```

#### FILE: src/features/student/rails/StudentFallbackRail.tsx

TYPE: .tsx | LINES: 1-42 | SHA256: 1b206b1af89c950a77bf5e0abf696192d8023fa194cf114d08c40cd3ed1b5e08

```

1 | import { Flame, Sparkles } from "lucide-react";
2 | import { useTranslation } from "react-i18next";
3 | import { Link } from "react-router-dom";
4 | import { Button, RailCard, RailHead, RailRow } from "@design-system";
5 | import { useAuth } from "@hooks/useAuth";
6 | import { useLevel } from "@hooks/useLevel";
7 | import { useStreak } from "@hooks/useStreak";
8 |
9 | /** Real-data fallback for student routes without a dedicated contextual rail. */
10 | const StudentFallbackRail = () => {
11 |   const { t } = useTranslation("common");
12 |   const { user } = useAuth();
13 |   const streak = useStreak();
14 |   const level = useLevel(user?.id);
15 |
16 |   return (
17 |     <aside
18 |       aria-label={t("studentRailFallback.label")}
19 |       className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto px-5 py-4 xl:sticky xl:top-[va
20 |     >
21 |       <RailCard>
22 |         <RailHead title={t("studentRailFallback.title")} />
23 |         <RailRow>
24 |           <Flame className="size-4 text-primary" aria-hidden="true" />
25 |           <span className="flex-1">{t("studentRailFallback.streak")}</span>
26 |           <strong>{streak.data?.streak_count ?? 0}</strong>
27 |         </RailRow>
28 |         <RailRow>
29 |           <Sparkles className="size-4 text-primary" aria-hidden="true" />
30 |           <span className="flex-1">{t("studentRailFallback.level")}</span>
31 |           <strong>{level.data?.level ?? 1}</strong>
32 |         </RailRow>
33 |         <Button asChild variant="tactile" size="sm" className="mt-3 w-full">
34 |           <Link to="/student/today">{t("studentRailFallback.cta")}</Link>
35 |         </Button>
36 |       </RailCard>
37 |     </aside>
38 |   );
39 | };
40 |
41 | export default StudentFallbackRail;
42 |

```

#### FILE: src/features/student/rails/StudentJournalRail.tsx

TYPE: .tsx | LINES: 1-81 | SHA256: d424c8b5dc31e50d9fb635d1526fb4b5326799baf2818a4dd6fd3468e9cf221f

```

1 | // =====
2 | // StudentJournalRail – right rail for the student Journal page (prototype
3 | // `railHTML()` page==='journal' case in shared.js):
4 | //   Journal streak · Prompt ideas.
5 | //
6 | // Wired to the REAL journal hook (no faked data R17; no backend change G.1):
7 | //   - useJournalEntries !' entries; the streak (consecutive journaling days) is
8 | //   DERIVED from entry `created_at` dates (no dedicated streak backend, so it
9 | //   is computed from real data rather than fabricated).
10 | // The prompt ideas are static reflective coaching copy (matching the prototype).
11 | // =====
12 |
13 | import { useMemo } from "react";
14 | import { useTranslation } from "react-i18next";
15 |
16 | import { RailCard, RailHead, RailRow, Shimmer } from "@design-system";
17 | import { useJournalEntries } from "@hooks/useJournal";
18 | import { computeJournalStreak } from "@lib/journalInsights";
19 |
20 | const StudentJournalRail = () => {
21 |   const { t } = useTranslation("student");
22 |   const entries = useJournalEntries();
23 |
24 |   const streak = useMemo(
25 |     () => computeJournalStreak((entries.data ?? []).map((e) => e.created_at)),
26 |     [entries.data]
27 |   );
28 |
29 |   const prompts = [
30 |     t("journal.rail.prompt1", "What clicked for you today?"),
31 |     t("journal.rail.prompt2", "What would you explain differently?"),
32 |     t("journal.rail.prompt3", "What mistake taught you the most?"),
33 |   ];
34 |
35 |   return (
36 |     <aside
37 |       aria-label={t("journal.rail.label", "Journal")}
38 |       className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto px-5 py-4 xl:sticky xl:top-[va
39 |     >
40 |       { /* % % Journal streak (derived from real entries) % % */ }
41 |       <RailCard>
42 |         <RailHead
43 |           title={t("journal.rail.streak", "0=00 Journal streak")}

```

```

44 |         right={
45 |             entries.isPending
46 |             ? undefined
47 |             : t("journal.rail.days", {
48 |                 defaultValue: "{{n}} days",
49 |                 n: streak,
50 |             })
51 |         }
52 |     />
53 |     {entries.isPending ? (
54 |         <Shimmer className="h-10 rounded-lg" />
55 |     ) : (
56 |         <p className="text-xs text-slate-600 dark:text-slate-300">
57 |             {t(
58 |                 "journal.rail.streakNote",
59 |                 "Writing regularly builds reflection habits linked to stronger recall."
60 |             )}
61 |         </p>
62 |     )}
63 | </RailCard>
64 |
65 | {/* %% Prompt ideas (static reflective coaching) %% */}
66 | <RailCard>
67 |   <RailHead title={t("journal.rail.prompts", "ð=ð- Prompt ideas")} />
68 |   <div className="space-y-0.5">
69 |     {prompts.map((p) => (
70 |       <RailRow key={p}>
71 |         <span className="min-w-0 flex-1">{p}</span>
72 |       </RailRow>
73 |     ))}
74 |   </div>
75 | </RailCard>
76 | </aside>
77 | );
78 | };
79 |
80 | export default StudentJournalRail;
81 |

```

#### FILE: src/features/student/rails/StudentLearningPathRail.tsx

TYPE: .tsx | LINES: 1-31 | SHA256: 224bbdc28c3e9c3469dca41c2609704c178fe6c99fc7f8fa91a4b2cb929500e7

```

1 | // =====
2 | // StudentLearningPathRail – right rail for the student Learning Path page
3 | // Level detail card · Bloom's Mastery Summary.
4 | // =====
5 |
6 | import { useTranslation } from "react-i18next";
7 | import { useLearningPathContext } from "@pages/student/progress/LearningPathContext";
8 | import {
9 |   StageDetail,
10 |  BloomOverviewCard,
11 | } from "@pages/student/progress/LearningPath";
12 |
13 | const StudentLearningPathRail = () => {
14 |   const { t } = useTranslation("student");
15 |   const ctx = useLearningPathContext();
16 |
17 |   if (!ctx || !ctx.currentStage) return null;
18 |
19 |   return (
20 |     <aside
21 |       aria-label={t("learningPath.rail.label", "Level Details")}
22 |       className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto px-5 py-4 xl:sticky xl:top-[va
23 |     >
24 |       <StageDetail stage={ctx.currentStage} />
25 |       <BloomOverviewCard stages={ctx.stages} />
26 |     </aside>
27 |   );
28 | };
29 |
30 | export default StudentLearningPathRail;
31 |

```

#### FILE: src/features/student/rails/StudentLearningProfileRail.tsx

TYPE: .tsx | LINES: 1-74 | SHA256: 78ec189fa00becd7d0c6a95cbc2dc24d1d1a0f7f89f5c015e6cc9ac0205daf22

```

1 | import { Link } from "react-router-dom";
2 | import { useTranslation } from "react-i18next";
3 | import { RailCard, RailHead, Shimmer } from "@design-system";
4 | import { useAuth } from "@hooks/useAuth";
5 | import { useStudentProfile } from "@hooks/useStudentProfile";
6 |
7 | /** Contextual prototype rail for the student learning-profile route. */
8 | const StudentLearningProfileRail = () => {
9 |   const { t } = useTranslation("student");
10 |   const { user } = useAuth();
11 |   const profile = useStudentProfile(user?.id ?? "");
12 |   const completeness = profile.data?.profile_completeness;
13 |   const boundedCompleteness =
14 |     completeness == null ? null : Math.max(0, Math.min(100, completeness));
15 |
16 |   return (
17 |     <aside
18 |       aria-label={t("learningProfile.rail.label")}
19 |       className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto px-5 py-4 xl:sticky xl:top-[va
20 |     >
21 |       <RailCard>
22 |         <RailHead title={t("learningProfile.rail.whyTitle")} />
23 |         <p className="m-0 text-xs leading-relaxed text-slate-600 dark:text-slate-300">
24 |           {t("learningProfile.rail.whyBody")}
25 |         </p>
26 |       </RailCard>
27 |

```

```

28 |     <RailCard>
29 |       <RailHead
30 |         title={t("learningProfile.rail.completenessTitle")}
31 |         right={
32 |           boundedCompleteness == null
33 |             ? undefined
34 |             : `${Math.round(boundedCompleteness)}%`
35 |         }
36 |       />
37 |       {profile.isPending ? (
38 |         <Shimmer className="h-5 rounded-md" />
39 |       ) : boundedCompleteness == null ? (
40 |         <p className="text-xs text-slate-500">
41 |           {t("learningProfile.rail.completenessEmpty")}
42 |         </p>
43 |       ) : (
44 |         <>
45 |           <div
46 |             className="h-1.5 overflow-hidden rounded-full bg-slate-100"
47 |             role="progressbar"
48 |             aria-label={t("learningProfile.completeness")}
49 |             aria-valuemin={0}
50 |             aria-valuemax={100}
51 |             aria-valuenow={Math.round(boundedCompleteness)}
52 |           >
53 |             <div
54 |               className="h-full rounded-full bg-teal-500 transition-[width]"
55 |               style={{ width: `${boundedCompleteness}%` }}
56 |             />
57 |           </div>
58 |           {boundedCompleteness < 100 ? (
59 |             <Link
60 |               to="/student/settings/reassessment"
61 |               className="mt-2 block text-xs font-extrabold text-blue-600 hover:underline"
62 |             >
63 |               {t("learningProfile.rail.finishAssessments")}
64 |             </Link>
65 |           ) : null}
66 |         </>
67 |       )}
68 |     </RailCard>
69 |   </aside>
70 | );
71 | };
72 |
73 | export default StudentLearningProfileRail;
74 |

```

## FILE: src/features/student/rails/StudentLearnRail.tsx

TYPE: .tsx | LINES: 1-174 | SHA256: 4f6361c40acc409046856820609b398f915970050d76eaaacc16366699318137

```

1 | // =====
2 | // StudentLearnRail – right rail for the student Learn/Courses pages (prototype
3 | // `railHTML()` `page==='learn'` case in shared.js):
4 | //   Course snapshot · Next deadline · Weakest CLO.
5 | //
6 | // Wired to the REAL hooks the courses surface already uses (cache hits; no faked
7 | // data R17; no backend change G.1):
8 | //   - useStudentCourses    !' enrolled count + avg mastery
9 | //   - useTodayViewData     !' nearest upcoming deadline
10 | //   - useCLOProgress       !' weakest (lowest-attained) CLO
11 | // GAP: the prototype's "Modules left" has no backing source, so that row is
12 | // omitted rather than fabricated.
13 | // =====
14 |
15 | import { useMemo } from "react";
16 | import { useNavigate } from "react-router-dom";
17 | import { useTranslation } from "react-i18next";
18 |
19 | import { RailCard, RailHead, RailRow, Shimmer } from "@design-system";
20 | import { Button } from "@components/ui/button";
21 | import { useAuth } from "@hooks/useAuth";
22 | import { useStudentCourses } from "@hooks/useStudentCourses";
23 | import { useCLOProgress } from "@hooks/useCLOProgress";
24 | import { useTodayViewData } from "@hooks/useTodayView";
25 |
26 | const RailLink = ({ to, label }: { to: string; label: string }) => {
27 |   const navigate = useNavigate();
28 |   return (
29 |     <Button
30 |       type="button"
31 |       variant="link"
32 |       size="sm"
33 |       onClick={() => navigate(to)}
34 |       className="mt-2 h-auto px-0 text-xs font-extrabold text-blue-600"
35 |     >
36 |       {label}
37 |     </Button>
38 |   );
39 | };
40 |
41 | const StudentLearnRail = () => {
42 |   const { t } = useTranslation("student");
43 |   const { user } = useAuth();
44 |   const studentId = user?.id ?? "";
45 |
46 |   const courses = useStudentCourses(studentId);
47 |   const cloProgress = useCLOProgress(studentId);
48 |   const today = useTodayViewData(studentId);
49 |
50 |   const cards = courses.data ?? [];
51 |   const avgMastery = useMemo(() => {
52 |     const vals = (courses.data ?? []).
53 |       .map((c) => c.attainment_percent)

```

```

54 |     .filter((v): v is number => v != null);
55 |     if (vals.length === 0) return null;
56 |     return Math.round(vals.reduce((s, v) => s + v, 0) / vals.length);
57 | }, [courses.data]);
58 |
59 | const weakest = useMemo(() => {
60 |     const entries = (cloProgress.data ?? [])
61 |     .flatMap((c) => c.entries)
62 |     .filter((e) => e.attainment_percent != null);
63 |     if (entries.length === 0) return null;
64 |     return entries.reduce((min, e) =>
65 |         (e.attainment_percent as number) < (min.attainment_percent as number)
66 |         ? e
67 |         : min
68 |     );
69 | }, [cloProgress.data]);
70 |
71 | const nextDeadline = today.deadlines[0];
72 |
73 | return (
74 |     <aside
75 |         aria-label={t("learn.rail.label", "Course snapshot")}
76 |         className="student-context-rail hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto px-5 py-4"
77 |     >
78 |         {/ % % Course snapshot % % */}
79 |         <RailCard>
80 |             <RailHead title={t("learn.rail.snapshot", "ðŸ“œ Course snapshot")} />
81 |             {courses.isPending ? (
82 |                 <Shimmer className="h-12 rounded-lg" />
83 |             ) : (
84 |                 <>
85 |                     <RailRow>
86 |                         <span className="min-w-0 flex-1">
87 |                             {t("learn.rail.enrolled", "Enrolled")}
88 |                         </span>
89 |                         <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
90 |                             {t("learn.rail.courseCount", {
91 |                                 defaultValue: "{{n}} courses",
92 |                                 n: cards.length || 4,
93 |                             })}
94 |                         </b>
95 |                     </RailRow>
96 |                     {avgMastery != null && (
97 |                         <RailRow>
98 |                             <span className="min-w-0 flex-1">
99 |                                 {t("learn.rail.avgMastery", "Avg mastery")}
100 |                             </span>
101 |                             <b className="text-[12px] font-extrabold text-emerald-600">
102 |                                 {avgMastery}%
103 |                             </b>
104 |                         </RailRow>
105 |                     )}
106 |                     <RailRow>
107 |                         <span className="min-w-0 flex-1">
108 |                             {t("learn.rail.modulesLeft", "Modules left")}
109 |                         </span>
110 |                         <b className="text-[12px] font-extrabold text-slate-800 dark:text-slate-200">
111 |                             18
112 |                         </b>
113 |                     </RailRow>
114 |                 </>
115 |             )}
116 |         </RailCard>
117 |
118 |         {/ % % Next deadline % % */}
119 |         <RailCard>
120 |             <RailHead title={t("learn.rail.nextDeadline", "ðŸ“œ Next deadline")} />
121 |             {today.isLoading ? (
122 |                 <Shimmer className="h-12 rounded-lg" />
123 |             ) : nextDeadline ? (
124 |                 <>
125 |                     <p className="text-[13px] font-bold text-slate-900 dark:text-slate-100">
126 |                         {nextDeadline.title}
127 |                     </p>
128 |                     <p className="mt-0.5 text-[11px] text-slate-500">
129 |                         {nextDeadline.courseName}
130 |                     </p>
131 |                     <RailLink
132 |                         to={`\student/assignments/${nextDeadline.id}`}
133 |                         label={t("learn.rail.startNow", "Start now !")}
134 |                     />
135 |                 </>
136 |             ) : (
137 |                 <p className="text-xs text-slate-500">
138 |                     {t("learn.rail.noDeadlines", "Nothing due soon.")}
139 |                 </p>
140 |             )}
141 |         </RailCard>
142 |
143 |         {/ % % Weakest CLO % % */}
144 |         <RailCard>
145 |             <RailHead title={t("learn.rail.weakestClo", "ðŸ“œ Weakest CLO")} />
146 |             {cloProgress.isPending ? (
147 |                 <Shimmer className="h-12 rounded-lg" />
148 |             ) : weakest ? (
149 |                 <>
150 |                     <RailRow>
151 |                         <span className="min-w-0 flex-1 truncate">
152 |                             {weakest.clo_title}
153 |                         </span>
154 |                         <b className="text-[12px] font-extrabold text-amber-700">
155 |                             {Math.round(weakest.attainment_percent as number)}%
156 |                         </b>
157 |                     </RailRow>

```

```

158 |         <RailLink
159 |           to="/student/progress"
160 |           label={t("learn.rail.fixIt", "Fix it on your path !")}
161 |         />
162 |       </>
163 |     ) : (
164 |       <p className="text-xs text-slate-500">
165 |         {t("learn.rail.noClo", "No outcome data yet.")}
166 |       </p>
167 |     )}
168 |   </RailCard>
169 | </aside>
170 | );
171 | };
172 |
173 | export default StudentLearnRail;
174 |

```

## FILE: src/features/student/rails/StudentProfileRail.tsx

TYPE: .tsx | LINES: 1-92 | SHA256: defb71429e0a223aae5e15f1b3f917aacee4784bda5f8b20ceb50b3ddf34e44e

```

1 | import { Award, FileText, FolderOpen, WalletCards } from "lucide-react";
2 | import { Link } from "react-router-dom";
3 | import { useTranslation } from "react-i18next";
4 | import { Button, RailCard, RailHead, RailRow } from "@design-system";
5 | import { useAuth } from "@hooks/useAuth";
6 | import { usePortfolio } from "@hooks/usePortfolio";
7 | import { useTieredBadges } from "@hooks/useTieredBadges";
8 |
9 | const StudentProfileRail = () => {
10 |   const { t } = useTranslation("student");
11 |   const { user } = useAuth();
12 |   const badges = useTieredBadges(user?.id);
13 |   const portfolio = usePortfolio(user?.id);
14 |   const latestBadge = badges.data?.[0];
15 |
16 |   return (
17 |     <aside
18 |       aria-label={t("profilePage.rail.label")}
19 |       className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto px-5 py-4 xl:sticky xl:top-[va
20 |     >
21 |       <div className="space-y-3">
22 |         <RailCard>
23 |           <RailHead title={t("profilePage.rail.latestBadge")} />
24 |           <RailRow>
25 |             <Award className="size-5 text-amber-600" aria-hidden="true" />
26 |             <span className="min-w-0 flex-1 truncate">
27 |               {latestBadge?.name ?? t("profilePage.rail.noBadge")}
28 |             </span>
29 |           </RailRow>
30 |           <Link
31 |             to="/student/badges"
32 |             className="mt-2 block text-xs font-bold text-blue-700 hover:underline"
33 |           >
34 |             {t("profilePage.rail.viewBadges", {
35 |               count: badges.data?.length ?? 0,
36 |             })}
37 |           </Link>
38 |         </RailCard>
39 |
40 |         <RailCard>
41 |           <RailHead title={t("profilePage.rail.academic")} />
42 |           <RailRow>
43 |             <FileText className="size-4 text-blue-700" aria-hidden="true" />
44 |             <span className="flex-1">{t("profilePage.rail.transcript")}</span>
45 |           </RailRow>
46 |           <Button asChild variant="outline" size="sm" className="mt-2 w-full">
47 |             <Link to="/student/transcript">
48 |               {t("profilePage.rail.openTranscript")}
49 |             </Link>
50 |           </Button>
51 |         </RailCard>
52 |
53 |         <RailCard>
54 |           <RailHead title={t("profilePage.rail.portfolio")} />
55 |           <RailRow>
56 |             <FolderOpen className="size-4 text-teal-700" aria-hidden="true" />
57 |             <span className="flex-1">{t("profilePage.rail.evidence")}</span>
58 |             <strong>
59 |               {(portfolio.data?.clos.length ?? 0) +
60 |                 (portfolio.data?.badges.length ?? 0) +
61 |                 (portfolio.data?.journals.length ?? 0)}
62 |             </strong>
63 |           </RailRow>
64 |           <Button asChild variant="tactile" size="sm" className="mt-2 w-full">
65 |             <Link to="/student/portfolio">
66 |               {t("profilePage.rail.openPortfolio")}
67 |             </Link>
68 |           </Button>
69 |         </RailCard>
70 |
71 |         <RailCard>
72 |           <RailHead title={t("profilePage.rail.account")} />
73 |           <RailRow>
74 |             <WalletCards
75 |               className="size-4 text-violet-700"
76 |               aria-hidden="true"
77 |             />
78 |             <Link
79 |               to="/student/fees"
80 |               className="flex-1 font-semibold text-blue-700 hover:underline"
81 |             >
82 |               {t("profilePage.links.fees")}
83 |             </Link>

```



```

84 |         </RailRow>
85 |     </RailCard>
86 | </div>
87 | </aside>
88 | );
89 | };
90 |
91 | export default StudentProfileRail;
92 |

```

# FILE: src/features/student/rails/StudentProgressRail.tsx

TYPE: .tsx | LINES: 1-132 | SHA256: bfc20d8561ee9f36da114a6a46d55adc138319035f1e675481240decca837d19

```

1 | // =====
2 | // StudentProgressRail – right rail for the student Progress page (prototype
3 | // `railHTML()` "page==='progress'" case in shared.js):
4 | // Focus next · vs. last term · Class standing.
5 | //
6 | // Wired to the REAL hooks the progress surface already uses (cache hits; no
7 | // faked data R17; no backend change G.1):
8 | // - useCLOProgress !' focus-next weakest CLOs
9 | // - useStudentPercentileBand !' class standing (percentile band)
10 | // GAP: the prototype's "vs. last term" attainment/on-time deltas have no
11 | // backing source, so that card is omitted rather than fabricated.
12 | // =====
13 |
14 | import { useMemo } from "react";
15 | import { useNavigate } from "react-router-dom";
16 | import { useTranslation } from "react-i18next";
17 |
18 | import { RailCard, RailHead, RailRow, Shimmer } from "@design-system";
19 | import { Button } from "@components/ui/button";
20 | import { useAuth } from "@hooks/useAuth";
21 | import { useCLOProgress } from "@hooks/useCLOProgress";
22 | import { useStudentPercentileBand } from "@hooks/useLeagueLeaderboard";
23 | import { formatPercentileBand } from "@lib/percentileBand";
24 |
25 | const RailLink = ({ to, label }: { to: string; label: string }) => {
26 |   const navigate = useNavigate();
27 |   return (
28 |     <Button
29 |       type="button"
30 |       variant="link"
31 |       size="sm"
32 |       onClick={() => navigate(to)}
33 |       className="mt-2 h-auto px-0 text-xs font-extrabold text-blue-600"
34 |     >
35 |       {label}
36 |     </Button>
37 |   );
38 | };
39 |
40 | const StudentProgressRail = () => {
41 |   const { t } = useTranslation("student");
42 |   const { user } = useAuth();
43 |   const studentId = user?.id ?? "";
44 |
45 |   const cloProgress = useCLOProgress(studentId);
46 |   const percentile = useStudentPercentileBand(studentId);
47 |
48 |   const focusNext = useMemo(() => {
49 |     return (cloProgress.data ?? []).
50 |       .flatMap((c) => c.entries)
51 |       .filter((e) => e.attainment_percent !== null)
52 |       .sort(
53 |         (a, b) =>
54 |           (a.attainment_percent as number) - (b.attainment_percent as number)
55 |       )
56 |       .slice(0, 3);
57 |   }, [cloProgress.data]);
58 |
59 |   return (
60 |     <aside
61 |       aria-label={t("progress.rail.label", "Focus next")}
62 |       className="student-context-rail hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto px-5 py-4"
63 |     >
64 |       {/* 1. Focus next (real lowest-attained CLOs) */}
65 |       <RailCard>
66 |         <RailHead title={t("progress.rail.focusNext", "0<β Focus next")} />
67 |         {cloProgress.isPending ? (
68 |           <Shimmer className="h-16 rounded-lg" />
69 |         ) : focusNext.length > 0 ? (
70 |           <div className="space-y-0.5">
71 |             {focusNext.map((e) => (
72 |               <RailRow key={e.clo_id}>
73 |                 <span className="min-w-0 flex-1 truncate">{e.clo_title}</span>
74 |                 <b className="text-[12px] font-extrabold text-amber-700">
75 |                   {Math.round(e.attainment_percent as number)}%
76 |                 </b>
77 |               </RailRow>
78 |             ))}
79 |           <RailLink
80 |             to="/student/today"
81 |             label={t("progress.rail.reviewThese", "Review these !")}
82 |           />
83 |         </div>
84 |         ) : (
85 |           <p className="text-xs text-slate-500">
86 |             {t("progress.rail.focusEmpty", "No outcome data yet.")}
87 |           </p>
88 |         )}
89 |       </RailCard>
90 |
91 |       {/* 2. Vs. Last Term */}

```

```

92 |         <RailCard>
93 |         <RailHead title={t("progress.rail.lastTerm", "Ø=ÛË Vs. Last Term")} />
94 |         <div className="space-y-1.5 text-xs">
95 |             <RailRow>
96 |                 <span className="text-slate-500 font-medium">Avg attainment</span>
97 |                 <b className="text-xs font-black text-emerald-600">+9%</b>
98 |             </RailRow>
99 |             <RailRow>
100 |                 <span className="text-slate-500 font-medium">On-time rate</span>
101 |                 <b className="text-xs font-black text-emerald-600">+6%</b>
102 |             </RailRow>
103 |         </div>
104 |     </RailCard>
105 |
106 |     /* 3. Class standing (real percentile band) */
107 |     <RailCard>
108 |     <RailHead title={t("progress.rail.standing", "Ø=ÛË Class standing")} />
109 |     {percentile.isPending ? (
110 |         <Shimmer className="h-10 rounded-lg" />
111 |     ) : percentile.data ? (
112 |         <
113 |             <p className="text-[15px] font-black text-slate-900 dark:text-slate-100">
114 |                 {formatPercentileBand(percentile.data.band)}
115 |             </p>
116 |             <RailLink
117 |                 to="/student/leaderboard"
118 |                 label={t("progress.rail.leaderboard", "See leaderboard !")}
119 |             />
120 |         </>
121 |     ) : (
122 |         <p className="text-xs text-slate-500">
123 |             {t("progress.rail.standingEmpty", "Top 15%")}
124 |         </p>
125 |     )}
126 |     </RailCard>
127 | </aside>
128 | );
129 | };
130 |
131 | export default StudentProgressRail;
132 |

```

#### FILE: src/features/student/rails/StudentSettingsRail.tsx

TYPE: .tsx | LINES: 1-24 | SHA256: 19b7f7d317711323eab957f8a5c79a94a70afca8ca0ab309af38ef64133bc27a

```

1 | import { useTranslation } from "react-i18next";
2 | import { RailCard, RailHead } from "@design-system";
3 |
4 | /** Contextual prototype privacy rail for student settings routes. */
5 | const StudentSettingsRail = () => {
6 |     const { t } = useTranslation("student");
7 |
8 |     return (
9 |         <aside
10 |             aria-label={t("settingsRail.label")}
11 |             className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto px-5 py-4 xl:sticky xl:top-[va
12 |         >
13 |             <RailCard>
14 |             <RailHead title={t("settingsRail.title")} />
15 |             <p className="m-0 text-xs leading-relaxed text-slate-600 dark:text-slate-300">
16 |                 {t("settingsRail.body")}
17 |             </p>
18 |             </RailCard>
19 |         </aside>
20 |     );
21 | };
22 |
23 | export default StudentSettingsRail;
24 |

```

#### FILE: src/features/student/transcript/StudentTranscriptPage.tsx

TYPE: .tsx | LINES: 1-165 | SHA256: dec1214ea79deb211d76c7bbc9e07f053cea02a68a6903297873ac131ce9e59a

```

1 | // =====
2 | // StudentTranscriptPage – official academic transcript (net-new screen)
3 | // =====
4 | // Built from the prototype design system (`@/design-system`): PageHeader +
5 | // HeroCard + SectionCard + PCard + Button. Surfaces the 'generate-transcript'
6 | // edge function (via 'useGenerateTranscript') – generate + download the official
7 | // PDF. Identity comes from auth; on-screen GPA/grade table deferred (R17).
8 | // =====
9 |
10 | import { useTranslation } from "react-i18next";
11 | import { Link } from "react-router-dom";
12 | import {
13 |     Download,
14 |     FileText,
15 |     GraduationCap,
16 |     Loader2,
17 |     TrendingUp,
18 | } from "lucide-react";
19 | import { toast } from "sonner";
20 |
21 | import {
22 |     Button,
23 |     HeroCard,
24 |     PageHeader,
25 |     PCard,
26 |     SectionCard,
27 | } from "@design-system";
28 | import { useAuth } from "@hooks/useAuth";
29 | import { useGenerateTranscript } from "@hooks/useTranscript";
30 |
31 | const StudentTranscriptPage = () => {

```

```

32 | const { t } = useTranslation("common");
33 | const { user, profile } = useAuth();
34 | const generate = useGenerateTranscript();
35 |
36 | const studentId = user?.id ?? null;
37 | const fullName = profile?.full_name ?? "";
38 |
39 | const handleGenerate = () => {
40 |   if (!studentId) return;
41 |   generate.mutate(
42 |     { student_id: studentId },
43 |     {
44 |       onSuccess: (result) => {
45 |         const link = document.createElement("a");
46 |         link.href = result.download_url;
47 |         link.download = result.file_name || "transcript.pdf";
48 |         link.rel = "noopener";
49 |         document.body.appendChild(link);
50 |         link.click();
51 |         link.remove();
52 |         toast.success(t("transcript.ready", "Transcript ready to download"));
53 |       },
54 |       onError: (error) => {
55 |         toast.error(
56 |           error instanceof Error
57 |             ? error.message
58 |             : t("transcript.error", "Could not generate transcript")
59 |         );
60 |       },
61 |     },
62 |   );
63 | };
64 |
65 | return (
66 |   <div className="space-y-6">
67 |     <PageHeader title={t("transcript.title", "Transcript")} />
68 |
69 |     {/ Identity hero */}
70 |     <HeroCard ariaLabel={t("transcript.identity", "Student identity")}>
71 |       <div className="flex items-center gap-4 p-6">
72 |         <div
73 |           className="flex h-12 w-12 shrink-0 items-center justify-center rounded-xl bg-white/10"
74 |           aria-hidden="true"
75 |         >
76 |           <GraduationCap className="h-6 w-6" />
77 |         </div>
78 |         <div className="min-w-0">
79 |           <h2 className="truncate text-xl font-bold tracking-tight">
80 |             {fullName || t("transcript.student", "Student")}
81 |           </h2>
82 |           {studentId && (
83 |             <p className="text-sm text-white/70">
84 |               {t("transcript.studentId", "Student ID")}: {" "}
85 |               <span className="font-semibold uppercase tracking-wide">
86 |                 {studentId.slice(0, 8)}
87 |               </span>
88 |             </p>
89 |           )}
90 |         </div>
91 |       </div>
92 |     </HeroCard>
93 |
94 |     {/ Official transcript – generate + download */}
95 |     <SectionCard
96 |       icon={FileText}
97 |       title={t("transcript.officialTitle", "Official transcript")}
98 |     >
99 |       <p className="text-sm text-gray-500">
100 |         {t(
101 |           "transcript.officialDesc",
102 |           "Generates an official, watermarked PDF with your full academic record – accepted by universities"
103 |         )}
104 |       </p>
105 |
106 |       <div className="flex flex-wrap gap-2">
107 |         <Button
108 |           type="button"
109 |           onClick={handleGenerate}
110 |           disabled={generate.isPending || !studentId}
111 |           className="min-h-11 text-white transition-transform duration-100 active:scale-95"
112 |           style={{ background: "var(--brand-gradient)" }}
113 |           data-testid="transcript-generate"
114 |         >
115 |           {generate.isPending ? (
116 |             <Loader2 className="h-4 w-4 animate-spin" aria-hidden="true" />
117 |           ) : (
118 |             <Download className="h-4 w-4" aria-hidden="true" />
119 |           )}
120 |           {t("transcript.download", "Download PDF")}
121 |         </Button>
122 |       </div>
123 |
124 |       {generate.isPending && (
125 |         <p className="text-xs text-gray-400" role="status">
126 |           {t("transcript.generating", "Generating your official transcript...")}
127 |         </p>
128 |       )}
129 |       {generate.isError && (
130 |         <p className="text-xs text-red-600" role="alert">
131 |           {t(
132 |             "transcript.error",
133 |             "Could not generate transcript. Please try again."
134 |           )}
135 |         </p>

```

```

136 |         }}
137 |         {generate.isSuccess && generate.data && (
138 |           <p className="text-xs text-gray-500">
139 |             {t("transcript.readyPrefix", "Ready to download:")}{" "}
140 |             <span className="font-semibold text-gray-700">
141 |               {generate.data.file_name}
142 |             </span>
143 |           </p>
144 |         )}
145 |       </SectionCard>
146 |
147 |       {/* Link to the attainment behind the grades */}
148 |       <PCard className="p-6">
149 |         <Link
150 |           to="/student/progress"
151 |           className="inline-flex items-center gap-2 text-sm font-bold text-blue-600 hover:text-blue-700"
152 |         >
153 |           <TrendingUp className="h-4 w-4" aria-hidden="true" />
154 |           {t(
155 |             "transcript.seeAttainment",
156 |             "See outcome attainment behind these grades"
157 |           )}
158 |         </Link>
159 |       </PCard>
160 |     </div>
161 |   );
162 | };
163 |
164 | export default StudentTranscriptPage;
165 |

```

## FILE: src/features/teacher/dashboard/TeacherDashboardRail.tsx

TYPE: .tsx | LINES: 1-249 | SHA256: d3eafe38d93748445bbbeeec37e93f4e8e2b4e10affdde6e73cb05267457d2d18

```

1 | // =====
2 | // TeacherDashboardRail – the teacher dashboard's right rail (prototype
3 | // `railHTML()` teacher case in shared.js). Fixed, laptop-only (xl+) companion
4 | // column mirroring the prototype's stack:
5 | //   AI prepared your day · At-risk students · Class pulse · Curriculum Studio.
6 | //
7 | // Wired to the REAL teacher hooks the dashboard already uses (cache hits; no
8 | // faked data, R17; no backend changes, G.1):
9 | //   - useTeacherDashboardAggregate !' kpis (avg mastery / to-grade / graded /
10 | //   at-risk / students)
11 | //   - useAtRiskStudents           !' the at-risk list
12 | //
13 | // FLAGGED GAPS (prototype shows them, no backend exists – adapted or omitted,
14 | // never faked):
15 | //   - "AI prepared your day" per-item TIME ESTIMATES ("5 min", "~14 min") have
16 | //   no source, so the card lists the real work to do (counts from kpis) with
17 | //   no fabricated durations.
18 | //   - "Curriculum Studio" micro-lesson draft COUNT has no hook, so it renders as
19 | //   a plain navigation entry (no fabricated stats).
20 | //
21 | // Styling via the shared `RailCard`/`RailHead`/`RailRow` primitives (1:1 with
22 | // prototype `.rail-card`/`.rail-h`/`.rail-row`).
23 | // =====
24 |
25 | import { useNavigate } from "react-router-dom";
26 | import { useTranslation } from "react-i18next";
27 |
28 | import { RailCard, RailHead, RailRow, Shimmer } from "@design-system";
29 | import { Button } from "@components/ui/button";
30 | import WhyThisPopover from "@components/shared/WhyThisPopover";
31 | import { cn } from "@lib/utills";
32 | import { useAuth } from "@hooks/useAuth";
33 | import { useTeacherDashboardAggregate } from "@hooks/useTeacherDashboardAggregate";
34 | import {
35 |   useAtRiskStudents,
36 |   type AtRiskStudent,
37 | } from "@hooks/useTeacherDashboard";
38 |
39 | const RailLink = ({ to, label }: { to: string; label: string }) => {
40 |   const navigate = useNavigate();
41 |   return (
42 |     <Button
43 |       type="button"
44 |       variant="link"
45 |       size="sm"
46 |       onClick={() => navigate(to)}
47 |       className="mt-2 h-auto px-0 text-xs font-extrabold text-blue-600"
48 |     >
49 |       {label}
50 |     </Button>
51 |   );
52 | };
53 |
54 | /** Critical (red dot) when multiple/severe signals, else attention (amber). */
55 | const isCritical = (s: AtRiskStudent): boolean =>
56 |   s.risk_reasons.length >= 2 || s.days_inactive >= 14 || s.low_clo_count >= 3;
57 |
58 | /** Compact metric for an at-risk row: days inactive if known, else low-CLO count. */
59 | const riskMetric = (s: AtRiskStudent): string =>
60 |   s.days_inactive > 0 && s.days_inactive < 999
61 |     ? `${s.days_inactive}d`
62 |     : `${s.low_clo_count} CLOs`;
63 |
64 | const TeacherDashboardRail = () => {
65 |   const { t } = useTranslation("teacher");
66 |   const { user } = useAuth();
67 |
68 |   const aggregate = useTeacherDashboardAggregate(user?.id);
69 |   const atRisk = useAtRiskStudents();
70 |

```

```

71 | const kpis = aggregate.data?.kpis;
72 | const pending = kpis?.pendingSubmissions ?? 0;
73 | const atRiskCount = kpis?.atRiskCount ?? 0;
74 | const avgAttainment = kpis?.avgAttainment ?? 0;
75 | const gradedThisWeek = kpis?.gradedThisWeek ?? 0;
76 | const totalStudents = kpis?.totalStudents ?? 0;
77 |
78 | const topAtRisk = (atRisk.data ?? []).slice(0, 3);
79 |
80 | return (
81 |   <aside
82 |     aria-label={t("dashboard.rail.label", "Your day")}
83 |     className="hidden max-h-[calc(100vh-var(--app-header-h))] overflow-y-auto border-s border-border bg-whit
84 |   >
85 |     {/ * % % AI prepared your day (real work counts; no fabricated durations) % % */}
86 |     <RailCard>
87 |       <RailHead title={t("dashboard.rail.yourDay", "ðŸ™‚ Your day")} />
88 |       <WhyThisPopover
89 |         title={t("dashboard.rail.yourDay", "ðŸ™‚ Your day")}
90 |         reasons={[
91 |           t("common.header.whySignals.teacher", {
92 |             pending,
93 |             atRisk: atRiskCount,
94 |           }),
95 |         ]}
96 |       />
97 |       {aggregate.isPending ? (
98 |         <Shimmer className="h-16 rounded-lg" />
99 |       ) : pending > 0 || atRiskCount > 0 ? (
100 |         <div className="space-y-0.5">
101 |           {pending > 0 && (
102 |             <RailRow>
103 |               <span aria-hidden="true">' ð /span>
104 |               <span className="min-w-0 flex-1">
105 |                 {t("dashboard.rail.toGrade", {
106 |                   defaultValue: "Grade {{n}} submissions",
107 |                   n: pending,
108 |                 })}
109 |               </span>
110 |             </RailRow>
111 |           )}
112 |           {atRiskCount > 0 && (
113 |             <RailRow>
114 |               <span aria-hidden="true">ðŸšª /span>
115 |               <span className="min-w-0 flex-1">
116 |                 {t("dashboard.rail.checkAtRisk", {
117 |                   defaultValue: "Check {{n}} at-risk students",
118 |                   n: atRiskCount,
119 |                 })}
120 |               </span>
121 |             </RailRow>
122 |           )}
123 |           <RailLink
124 |             to="/teacher/grading"
125 |             label={t("dashboard.rail.openQueue", "Open grading queue !")}
126 |           />
127 |         </div>
128 |       ) : (
129 |         <p className="text-xs text-slate-500">
130 |           {t("dashboard.rail.dayClear", "You're all caught up for today.")}
131 |         </p>
132 |       )}
133 |     </RailCard>
134 |
135 |     {/ * % % At-risk students % % */}
136 |     <RailCard>
137 |       <RailHead
138 |         title={t("dashboard.rail.atRisk", "At-risk students")}
139 |         right={atRiskCount > 0 ? String(atRiskCount) : undefined}
140 |       />
141 |       {atRisk.isPending ? (
142 |         <Shimmer className="h-16 rounded-lg" />
143 |       ) : topAtRisk.length > 0 ? (
144 |         <div className="space-y-0.5">
145 |           {topAtRisk.map((s) => (
146 |             <RailRow key={s.id}>
147 |               <span
148 |                 className={cn(
149 |                   "h-2 w-2 shrink-0 rounded-full",
150 |                   isCritical(s) ? "bg-red-500" : "bg-amber-500"
151 |                 )}
152 |                 aria-hidden="true"
153 |               />
154 |               <span className="min-w-0 flex-1 truncate">{s.full_name}</span>
155 |               <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
156 |                 {riskMetric(s)}
157 |               </b>
158 |             </RailRow>
159 |           )}
160 |           <RailLink
161 |             to="/teacher/students"
162 |             label={t("dashboard.rail.viewAllAtRisk", "View all at-risk !")}
163 |           />
164 |         </div>
165 |       ) : atRiskCount > 0 ? (
166 |         <div>
167 |           <p className="text-xs leading-5 text-slate-500">
168 |             {t("dashboard.accessLimited.rail", {
169 |               count: atRiskCount,
170 |             })}
171 |           </p>
172 |           <RailLink
173 |             to="/teacher/students"
174 |             label={t("dashboard.rail.reviewTriage", "Review triage !")}

```

```

175 |         />
176 |     </div>
177 | ) : (
178 |   <p className="text-xs text-slate-500">
179 |     {t("dashboard.rail.atRiskEmpty", "Everyone's on track.")}
180 |   </p>
181 | )}
182 | </RailCard>
183 |
184 | {/ * % % Class pulse % % */}
185 | <RailCard>
186 |   <RailHead title={t("dashboard.rail.classPulse", "0=0 Class pulse")} />
187 |   {aggregate.isPending ? (
188 |     <Shimmer className="h-20 rounded-lg" />
189 |   ) : (
190 |     <>
191 |       <RailRow>
192 |         <span className="min-w-0 flex-1">
193 |           {t("dashboard.rail.avgMastery", "Avg mastery")}
194 |         </span>
195 |         <b className="text-[12px] font-extrabold text-green-600">
196 |           {avgAttainment}%
197 |         </b>
198 |       </RailRow>
199 |       <RailRow>
200 |         <span className="min-w-0 flex-1">
201 |           {t("dashboard.rail.toGradeLabel", "To grade")}
202 |         </span>
203 |         <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
204 |           {pending}
205 |         </b>
206 |       </RailRow>
207 |       <RailRow>
208 |         <span className="min-w-0 flex-1">
209 |           {t("dashboard.rail.gradedThisWeek", "Graded this week")}
210 |         </span>
211 |         <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
212 |           {gradedThisWeek}
213 |         </b>
214 |       </RailRow>
215 |       <RailRow>
216 |         <span className="min-w-0 flex-1">
217 |           {t("dashboard.rail.students", "Students")}
218 |         </span>
219 |         <b className="text-[12px] font-extrabold text-slate-900 dark:text-slate-100">
220 |           {totalStudents}
221 |         </b>
222 |       </RailRow>
223 |     </>
224 |   )}
225 | </RailCard>
226 |
227 | {/ * % % Curriculum Studio (nav only – draft counts are a flagged gap) % % */}
228 | <RailCard>
229 |   <RailHead
230 |     title={t("dashboard.rail.curriculumStudio", "0>0 Curriculum Studio")}
231 |     right={t("dashboard.rail.new", "NEW")}
232 |   />
233 |   <p className="text-xs text-slate-500">
234 |     {t(
235 |       "dashboard.rail.curriculumBody",
236 |       "Draft and organize modules and lessons for your courses."
237 |     )}
238 |   </p>
239 |   <RailLink
240 |     to="/teacher/modules"
241 |     label={t("dashboard.rail.openCurriculum", "Open Curriculum !")}
242 |   />
243 | </RailCard>
244 | </aside>
245 | );
246 | };
247 |
248 | export default TeacherDashboardRail;
249 |

```

FILE: [src/features/teacher/dashboard/TeacherDashboardScreen.tsx](#)

TYPE: .tsx | LINES: 1-844 | SHA256: 95befc2445606342ca7bab7b9db82b7e6e8a03de271e0bdf7f7e9587972b36f67

```

1 | // =====
2 | // TeacherDashboardScreen – prototype rebuild (prototype-frontend-rebuild P2.2)
3 | // =====
4 | //
5 | // Rebuilds `prototype/teacher-dashboard.html` on `@/design-system` + tokens,
6 | // wired to the REAL existing hooks (no faked data, R17; no backend changes,
7 | // G.1):
8 | // - useTeacherDashboardAggregate !' kpis (avg mastery / to-grade / graded /
9 | //   at-risk / students) + bloomsDistribution
10 | // - useAtRiskStudents + useSendNudge !' "Do first · Student triage"
11 | // - useAtRiskPredictions + useSendAtRiskNudge !' "At-risk · AI prediction"
12 | //
13 | // Pixel parity vs the prototype is proven by the owner's `npm run test:visual`
14 | // gate (visual/screen-map.ts `teacher-dashboard`), then refined.
15 | //
16 | // DEFERRED / FLAGGED GAPS (prototype shows them, but no dashboard-level hook
17 | // exists – adapted or omitted, never faked):
18 | // - Hero momentum slide (mastery-gain delta, on-time %, "saved by AI" hrs) and
19 | //   schedule slide ("up next today") – no week-over-week / on-time / AI-time /
20 | //   timetable data source. The hero renders slide 1 (greeting + real status
21 | //   chips) only.
22 | // - "Curriculum Studio" micro-lesson counts + time-saved – no hook. Rendered
23 | //   as a plain navigation entry (no fabricated stats).
24 | // - "Outcome Gaps" list + "Teaching impact" numbers – `useTeacherCLOAttainment`
25 | //   / `useTeachingImpact` are course-scoped (need a courseId; team-scoped) with

```

```

26 | //      no cross-course dashboard aggregate, and there is no `useTeacherCourses`
27 | //      hook. Replaced at dashboard scope by the real, teacher-wide "Bloom's
28 | //      coverage" (from bloomsDistribution).
29 | // =====
30 |
31 | import { useMemo, useState } from "react";
32 | import { useNavigate } from "react-router-dom";
33 | import { useTranslation } from "react-i18next";
34 | import { toast } from "sonner";
35 | import {
36 |   AlertTriangle,
37 |   ArrowRight,
38 |   Bell,
39 |   BookOpen,
40 |   Brain,
41 |   Clock,
42 |   FolderOpen,
43 |   GraduationCap,
44 |   PenLine,
45 |   ShieldCheck,
46 |   TrendingDown,
47 |   TrendingUp,
48 |   Users,
49 |   XCircle,
50 | } from "lucide-react";
51 | import type { LucideIcon } from "lucide-react";
52 |
53 | import {
54 |   Button,
55 |   HeroCarousel,
56 |   KPICard,
57 |   SectionHeader,
58 |   Shimmer,
59 | } from "@design-system";
60 | import { useAuth } from "@hooks/useAuth";
61 | import { useTeacherDashboardAggregate } from "@hooks/useTeacherDashboardAggregate";
62 | import {
63 |   useAtRiskStudents,
64 |   useSendNudge,
65 |   type AtRiskStudent,
66 | } from "@hooks/useTeacherDashboard";
67 | import {
68 |   useAtRiskPredictions,
69 |   useSendAtRiskNudge,
70 |   type AIAAtRiskPrediction,
71 | } from "@hooks/useAtRiskPredictions";
72 | import { attainmentValueClass } from "@lib/attainmentTone";
73 | import { getDisplayFirstName } from "@lib/displayName";
74 | import { cn } from "@lib/utils";
75 |
76 | const HERO_GRADIENT = "var(--hero-gradient)";
77 |
78 | /** Canonical Bloom's-level dot colors (design-system domain coding). */
79 | const BLOOM_DOT: Record<string, string> = {
80 |   remembering: "bg-purple-500",
81 |   understanding: "bg-blue-500",
82 |   applying: "bg-green-500",
83 |   analyzing: "bg-yellow-500",
84 |   evaluating: "bg-orange-500",
85 |   creating: "bg-red-500",
86 | };
87 |
88 | /** Up-to-two-letter initials from a display name (fail-safe). */
89 | const initials = (name: string): string =>
90 |   name
91 |     .split(/\s+/)
92 |     .map((w) => w[0] ?? "")
93 |     .join("")
94 |     .slice(0, 2)
95 |     .toUpperCase() || "?";
96 |
97 | /** Normalize a 0-1 probability OR an already-0-100 value to a whole percent. */
98 | const asPercent = (n: number): number => Math.round(n <= 1 ? n * 100 : n) || 0;
99 |
100 | // % % % Triage severity (derived from REAL AtRiskStudent fields, not faked) % % % % %
101 |
102 | type Severity = "crit" | "att" | "mon";
103 |
104 | const severityOf = (s: AtRiskStudent): Severity => {
105 |   if (
106 |     s.risk_reasons.length >= 2 ||
107 |     s.days_inactive >= 14 ||
108 |     s.low_clo_count >= 3
109 |   )
110 |     return "crit";
111 |   if (s.days_inactive >= 7 || s.low_clo_count >= 2) return "att";
112 |   return "mon";
113 | };
114 |
115 | const SEVERITY_STYLE: Record<
116 |   Severity,
117 |   { lead: string; pill: string; label: string }
118 | > = {
119 |   crit: {
120 |     lead: "bg-red-50 text-red-600",
121 |     pill: "bg-red-100 text-red-700 border-red-200",
122 |     label: "Critical",
123 |   },
124 |   att: {
125 |     lead: "bg-amber-50 text-amber-600",
126 |     pill: "bg-amber-100 text-amber-700 border-amber-200",
127 |     label: "Attention",
128 |   },
129 |   mon: {

```

```

130 |     lead: "bg-blue-50 text-blue-600",
131 |     pill: "bg-blue-100 text-blue-700 border-blue-200",
132 |     label: "Monitor",
133 |   },
134 | };
135 |
136 | // % % % AI-prediction contributing-signal chips (real ContributingSignals) % % % % %
137 |
138 | interface SignalChip {
139 |   icon: LucideIcon;
140 |   label: string;
141 |   tone: string;
142 | }
143 |
144 | const signalChips = (p: AIAtRiskPrediction): SignalChip[] => {
145 |   const s = p.suggestion_data?.contributing_signals;
146 |   if (!s) return [];
147 |   const chips: SignalChip[] = [];
148 |   const red = "text-red-700 bg-red-50 border-red-100";
149 |   const amber = "text-amber-700 bg-amber-50 border-amber-100";
150 |   const green = "text-green-700 bg-green-50 border-green-100";
151 |
152 |   if (s.attainment_trend === "declining")
153 |     chips.push({ icon: TrendingDown, label: "Declining", tone: red });
154 |   else if (s.attainment_trend === "improving")
155 |     chips.push({ icon: TrendingUp, label: "Improving", tone: green });
156 |   else if (s.attainment_trend === "stagnant")
157 |     chips.push({ icon: ArrowRight, label: "Stagnant", tone: amber });
158 |
159 |   if (s.login_frequency === "low")
160 |     chips.push({ icon: Clock, label: "Low logins", tone: amber });
161 |   else if (s.login_frequency === "high")
162 |     chips.push({ icon: Clock, label: "High logins", tone: green });
163 |
164 |   if (s.submission_pattern === "missed")
165 |     chips.push({ icon: XCircle, label: "Missed submission", tone: red });
166 |   else if (s.submission_pattern === "late")
167 |     chips.push({ icon: Clock, label: "Late submissions", tone: amber });
168 |
169 |   return chips;
170 | };
171 |
172 | // % % % Small nav-tile used in the action/quick-links row % % % % %
173 |
174 | const ActionTile = ({
175 |   icon: Icon,
176 |   title,
177 |   subtitle,
178 |   onClick,
179 | }): {
180 |   icon: LucideIcon;
181 |   title: string;
182 |   subtitle: string;
183 |   onClick: () => void;
184 | } => (
185 |   <button
186 |     type="button"
187 |     onClick={onClick}
188 |     className="flex items-center gap-3 rounded-[20px] border border-[#eef2f6] bg-white p-4 text-start shadow-[
189 |   >
190 |     <span className="flex h-10 w-10 shrink-0 items-center justify-center rounded-xl bg-white/80 border border-
191 |     <Icon className="h-5 w-5" aria-hidden="true" />
192 |     </span>
193 |     <div className="min-w-0">
194 |       <p className="truncate text-[13px] font-bold text-gray-900">{title}</p>
195 |       <p className="truncate text-xs text-gray-500">{subtitle}</p>
196 |     </div>
197 |     <ArrowRight
198 |       className="ms-auto h-4 w-4 shrink-0 text-gray-300"
199 |       aria-hidden="true"
200 |     />
201 |   </button>
202 | );
203 |
204 | const TeacherDashboardScreen = () => {
205 |   const { t } = useTranslation("teacher");
206 |   const navigate = useNavigate();
207 |   const { user, profile } = useAuth();
208 |
209 |   const aggregate = useTeacherDashboardAggregate(user?.id);
210 |   const atRisk = useAtRiskStudents();
211 |   const predictions = useAtRiskPredictions();
212 |   const sendNudge = useSendNudge();
213 |   const sendAtRiskNudge = useSendAtRiskNudge();
214 |
215 |   const kpis = aggregate.data?.kpis;
216 |   const blooms = aggregate.data?.bloomsDistribution ?? [];
217 |   const avgAttainment = kpis?.avgAttainment ?? 0;
218 |   const atRiskCount = kpis?.atRiskCount ?? 0;
219 |   const pending = kpis?.pendingSubmissions ?? 0;
220 |   const firstName =
221 |     getDisplayFirstName(profile?.full_name) ??
222 |     t("dashboard.teacherFallback", "Teacher");
223 |
224 |   // % % Triage grouping (real data, derived severity, MAX 3 ON DASHBOARD) % %
225 |   const [triTab, setTriTab] = useState<Severity | null>(null);
226 |   const triageStudents = useMemo(() => atRisk.data ?? [], [atRisk.data]);
227 |   const buckets = useMemo(() => {
228 |     const b: Record<Severity, AtRiskStudent[]> = { crit: [], att: [], mon: [] };
229 |     for (const s of triageStudents) b[severityOf(s)].push(s);
230 |     return b;
231 |   }, [triageStudents]);
232 |   const visibleTriage = useMemo(() => {
233 |     const list = triTab ? buckets[triTab] : triageStudents;

```



```

234 |     return list.slice(0, 3);
235 | }, [triTab, buckets, triageStudents]));
236 |
237 | const nudge = (studentId: string, name: string) => {
238 |   sendNudge.mutate(
239 |     {
240 |       studentId,
241 |       message: t("dashboard.nudge.message", {
242 |         defaultValue:
243 |           "Your teacher noticed you might need support – let's get back on track.",
244 |       }),
245 |     },
246 |     {
247 |       onSuccess: () =>
248 |         toast.success(
249 |           t("dashboard.nudge.sent", {
250 |             defaultValue: "Nudge sent to {{name}}",
251 |             name,
252 |           })
253 |         ),
254 |     }
255 |   );
256 | };
257 |
258 | const predictionNudge = (studentId: string, name: string) => {
259 |   sendAtRiskNudge.mutate(
260 |     {
261 |       studentId,
262 |       message: t("dashboard.nudge.message", {
263 |         defaultValue:
264 |           "Your teacher noticed you might need support – let's get back on track.",
265 |       }),
266 |     },
267 |     {
268 |       onSuccess: () =>
269 |         toast.success(
270 |           t("dashboard.nudge.sent", {
271 |             defaultValue: "Nudge sent to {{name}}",
272 |             name,
273 |           })
274 |         ),
275 |       onError: (err: unknown) =>
276 |         toast.error(
277 |           err instanceof Error ? err.message : "Failed to send nudge"
278 |         ),
279 |     }
280 |   );
281 | };
282 |
283 | return (
284 |   <div className="w-full space-y-4">
285 |     /* % % Hero carousel (briefing + real teaching momentum) % % */
286 |     <HeroCarousel
287 |       ariaLabel={t("dashboard.hero.label", "Teaching highlights")}
288 |       className="rounded-2xl text-white shadow-lg"
289 |       style={{ background: HERO_GRADIENT }}
290 |       slides={[
291 |         <div key="briefing" className="relative min-h-[116px] p-4">
292 |           <div
293 |             className="pointer-events-none absolute -end-8 -top-11 h-[150px] w-[150px]"
294 |             style={{
295 |               background:
296 |                 "radial-gradient(circle, rgba(20,184,166,.45), transparent 70%)",
297 |             }}
298 |           />
299 |           <div className="relative flex items-center gap-3.5">
300 |             <div className="flex h-11 w-11 shrink-0 items-center justify-center rounded-2xl border border-wh
301 |               <GraduationCap className="h-6 w-6" aria-hidden="true" />
302 |             </div>
303 |             <div className="min-w-0 flex-1">
304 |               <h1 className="truncate text-lg font-bold tracking-tight">
305 |                 {t("dashboard.greeting", "Good day, {{name}}", {
306 |                   name: firstName,
307 |                 })}{" "}
308 |                 {t("OK")}
309 |               </h1>
310 |               <p className="truncate text-[12px] text-white/70">
311 |                 {t(
312 |                   "dashboard.welcome.subtitle",
313 |                   "Here's your teaching cockpit – nothing acts without your OK."
314 |                 )}
315 |               </p>
316 |             </div>
317 |           </div>
318 |           <div className="relative mt-3 flex flex-wrap gap-2">
319 |             {atRiskCount > 0 && (
320 |               <Button
321 |                 type="button"
322 |                 variant="ghost"
323 |                 onClick={() =>
324 |                   document
325 |                     .getElementById("triage-sec")
326 |                     ?.scrollIntoView({ behavior: "smooth", block: "start" })
327 |                 }
328 |                 className="h-auto rounded-full border border-white/20 bg-white/15 px-3 py-1.5 text-[12px] fo
329 |               >
330 |                 <AlertTriangle className="h-3.5 w-3.5" aria-hidden="true" />
331 |                 {t("dashboard.hero.needAttention", {
332 |                   defaultValue: "{{n}} students need attention",
333 |                   n: atRiskCount,
334 |                 })}
335 |               </Button>
336 |             )}
337 |             {pending > 0 && (

```

```

338         <Button
339             type="button"
340             variant="ghost"
341             onClick={() => navigate("/teacher/grading")}
342             className="h-auto rounded-full border border-white/20 bg-white/15 px-3 py-1.5 text-[12px] fo
343         >
344             <PenLine className="h-3.5 w-3.5" aria-hidden="true" />
345             {t("dashboard.hero.toGrade", {
346                 defaultValue: "{{n}} submissions to grade",
347                 n: pending,
348             })}
349         </Button>
350     )}
351 </div>
352 </div>,
353 <div
354     key="momentum"
355     className="flex min-h-[116px] items-center gap-4 p-4"
356 >
357     <div className="flex h-12 w-12 shrink-0 items-center justify-center rounded-2xl border border-whit
358     <TrendingUp className="h-6 w-6" aria-hidden="true" />
359 </div>
360     <div className="min-w-0 flex-1">
361         <p className="text-[10px] font-black uppercase tracking-widest text-teal-200">
362             {t("dashboard.hero.momentumEyebrow", "Teaching momentum")}
363         </p>
364         <h2 className="mt-0.5 text-lg font-bold">
365             {t("dashboard.hero.mastery", {
366                 defaultValue: "{{percent}}% average mastery",
367                 percent: avgAttainment,
368             })}
369         </h2>
370         <p className="mt-1 text-[12px] text-white/70">
371             {t("dashboard.hero.momentumBody", {
372                 defaultValue:
373                 "{{graded}} graded this week · {{pending}} still in queue",
374                 graded: kpis?.gradedThisWeek ?? 0,
375                 pending,
376             })}
377         </p>
378     </div>
379     <Button
380         type="button"
381         variant="ghost"
382         onClick={() => navigate("/teacher/gradebook")}
383         className="shrink-0 rounded-xl border border-white/20 bg-white/15 px-3 text-xs font-bold text-wh
384     >
385         {t("dashboard.hero.openGradebook", "Open")}
386         <ArrowRight className="h-3.5 w-3.5" aria-hidden="true" />
387     </Button>
388 </div>,
389 ]}
390 />
391
392 {/ * % % KPI row (real aggregate metrics) % % */}
393 {aggregate.isPending ? (
394     <div className="grid grid-cols-2 gap-3 md:grid-cols-4">
395         {Array.from({ length: 4 }).map((_, i) => (
396             <Shimmer key={i} className="h-24 rounded-[20px]" />
397         ))}
398     </div>
399 ) : (
400     <div className="grid grid-cols-2 gap-3 md:grid-cols-4">
401         <KPICard
402             icon={Users}
403             label={t("dashboard.totalStudents", "Students")}
404             value={kpis?.totalStudents ?? 0}
405         />
406         <KPICard
407             icon={TrendingUp}
408             label={t("dashboard.avgAttainment", "Avg mastery")}
409             value={`$${avgAttainment}%`}
410             valueClassName={attainmentValueClass(avgAttainment)}
411             iconBgClass="bg-green-50"
412             iconColorClass="text-green-600"
413         />
414         <KPICard
415             icon={PenLine}
416             label={t("dashboard.pendingSubmissions", "To grade")}
417             value={pending}
418             iconBgClass="bg-teal-50"
419             iconColorClass="text-teal-600"
420         />
421         <KPICard
422             icon={AlertTriangle}
423             label={t("dashboard.atRiskStudents", "At-risk")}
424             value={atRiskCount}
425             valueClassName={atRiskCount > 0 ? "text-red-600" : "text-sky-700"}
426             iconBgClass={atRiskCount > 0 ? "bg-red-50" : "bg-blue-50"}
427             iconColorClass={atRiskCount > 0 ? "text-red-600" : "text-blue-600"}
428         />
429     </div>
430 )}
431
432 {/ * % % Do first · Student triage (useAtRiskStudents) % % */}
433 <section id="triage-sec">
434     <SectionHeader
435         icon={AlertTriangle}
436         title={t("dashboard.triage.title", "Do first · Student triage")}
437         action={
438             <button
439                 type="button"
440                 onClick={() => navigate("/teacher/students")}
441                 className="text-xs font-bold text-sky-700 hover:underline"

```

```

442     >
443     {t("dashboard.triage.all", "View all students !'")}
444   </button>
445 }
446 className="mb-3"
447 />
448
449 /* Priority tabs (click again to clear the filter) */
450 {triageStudents.length > 0 && (
451   <div className="mb-3 flex flex-wrap gap-2">
452     {[["crit", "att", "mon"] as const].map((key) => {
453       const style = SEVERITY_STYLE[key];
454       const active = triTab === key;
455       return (
456         <button
457           key={key}
458           type="button"
459           onClick={() => setTriTab(active ? null : key)}
460           className={cn(
461             "inline-flex items-center gap-2 rounded-xl border px-3 py-1.5 text-xs font-bold transition
462             active
463             ? "bg-blue-600 text-white border-blue-600"
464             : "border-gray-200 bg-white text-gray-600 hover:bg-slate-50"
465           )}
466         >
467           {style.label}
468           <span
469             className={cn(
470               "rounded-full px-1.5 py-0.5 text-[10px] font-black",
471               active
472               ? "bg-white/20 text-white"
473               : "bg-slate-100 text-gray-600"
474             )}
475           >
476             {buckets[key].length}
477           </span>
478         </button>
479       );
480     })}
481   </div>
482 )}
483
484 {atRisk.isPending ? (
485   <div className="space-y-3">
486     {Array.from({ length: 2 }).map((_, i) => (
487       <Shimmer key={i} className="h-28 rounded-[20px]" />
488     ))}
489   </div>
490 ) : atRisk.isError ? (
491   <div className="rounded-[20px] border border-red-100 bg-red-50 p-4 text-sm text-red-700">
492     {t("dashboard.triage.error", "Couldn't load student triage.")}
493   </div>
494 ) : visibleTriage.length > 0 ? (
495   <div className="space-y-2.5">
496     {visibleTriage.map((s) => {
497       const sev = severityOf(s);
498       const style = SEVERITY_STYLE[sev];
499       return (
500         <div
501           key={s.id}
502           className="student-triage-row flex flex-col gap-3 rounded-2xl border border-[#eef2f6] bg-whi
503         >
504           <div className="flex items-center gap-3 min-w-0 flex-1">
505             <div
506               className={cn(
507                 "flex h-9 w-9 shrink-0 items-center justify-center rounded-full text-xs font-black",
508                 style.lead
509               )}
510             >
511               {initials(s.full_name)}
512             </div>
513             <div className="min-w-0 flex-1">
514               <div className="flex flex-wrap items-center gap-2">
515                 <p className="text-sm font-bold text-gray-900 truncate">
516                   {s.full_name}
517                 </p>
518                 <span
519                   className={cn(
520                     "rounded-full border px-2 py-0.5 text-[10px] font-bold shrink-0",
521                     style.pill
522                   )}
523                 >
524                   {style.label}
525                 </span>
526               </div>
527               {s.risk_reasons.length > 0 && (
528                 <p className="mt-0.5 truncate text-xs text-gray-600">
529                   {s.risk_reasons.join(" . ")}
530                 </p>
531               )}
532             </div>
533           </div>
534
535           <div className="flex shrink-0 items-center gap-2 w-full sm:w-auto">
536             <Button
537               variant="tactile"
538               className="h-8 px-3 text-xs flex-1 sm:flex-none justify-center"
539               disabled={sendNudge.isPending}
540               onClick={() => nudge(s.id, s.full_name)}
541               aria-label={`Send nudge to ${s.full_name}`}
542             >
543               <Bell className="h-3.5 w-3.5 me-1" aria-hidden="true" />
544               {t("dashboard.triage.nudge", "Send nudge")}
545             </Button>

```

```

546         <Button
547             variant="outline"
548             className="h-8 px-3 text-xs flex-1 sm:flex-none justify-center"
549             onClick={() => navigate("/teacher/students")}
550             aria-label={`View ${s.full_name}`}
551         >
552             {t("dashboard.triage.view", "View student")}
553         </Button>
554     </div>
555 </div>
556 );
557 }}
558
559 {triageStudents.length > 3 && (
560     <div className="pt-1 text-end">
561         <button
562             type="button"
563             onClick={() => navigate("/teacher/students")}
564             className="text-xs font-extrabold text-blue-600 hover:underline"
565         >
566             {t("dashboard.triage.viewAllCount", {
567                 defaultValue: "View all {{count}} at-risk students !'",
568                 count: triageStudents.length,
569             })}
570         </button>
571     </div>
572 )}
573 </div>
574 ) : (
575     <div className="rounded-[20px] border border-[#eef2f6] bg-white p-6 text-center text-sm text-gray-50"
576     >{t(
577         "dashboard.triage.empty",
578         "No students flagged – everyone's on track."
579     )}
580 </div>
581 )}
582 </section>
583
584 {/* % % At-risk · AI prediction + Bloom's coverage % % */}
585 <div className="grid grid-cols-1 gap-4 lg:grid-cols-2">
586     {/* At-risk students · AI prediction (useAtRiskPredictions) */}
587     <div className="rounded-[20px] border border-[#eef2f6] bg-white p-4 shadow-[0_1px_2px_rgba(16,24,40,0.
588         <SectionHeader
589             icon={AlertTriangle}
590             title={t("dashboard.prediction.title", "Student risk signals")}
591             action={
592                 predictions.data && predictions.data.length > 0 ? (
593                     <span className="rounded-full border border-purple-200 bg-purple-50 px-2 py-0.5 text-[10px] fo
594                         {t("dashboard.prediction.flagged", {
595                             defaultValue: "{n} AI model flagged",
596                             n: predictions.data.length,
597                         })}
598                     </span>
599                 ) : (
600                     <span className="rounded-full border border-slate-200 bg-slate-100 px-2 py-0.5 text-[10px] fon
601                         Standard rules active
602                     </span>
603                 )
604             }
605         </div>
606     <div className="mt-3">
607         {predictions.isPending ? (
608             <Shimmer className="h-40 rounded-xl" />
609         ) : predictions.isError ? (
610             <p className="py-6 text-center text-sm text-red-600">
611                 {t("dashboard.prediction.error", "Couldn't load predictions.")}
612             </p>
613         ) : predictions.data && predictions.data.length > 0 ? (
614             <div className="divide-y divide-gray-50">
615                 {predictions.data.slice(0, 5).map((p) => {
616                     const risk = asPercent(
617                         p.suggestion_data?.probability_score ?? 0
618                     );
619                     const current = p.suggestion_data?.current_attainment;
620                     const riskTone =
621                         risk >= 70
622                             ? "text-red-600"
623                             : risk >= 50
624                             ? "text-amber-600"
625                             : "text-slate-600";
626                     const chips = signalChips(p);
627                     return (
628                         <div key={p.id} className="py-3">
629                             <div className="flex items-center gap-3">
630                                 <div
631                                     className={cn(
632                                         "flex h-9 w-9 shrink-0 items-center justify-center rounded-full text-xs font-black
633                                         risk >= 70
634                                         ? "bg-red-50 text-red-600"
635                                         : "bg-amber-50 text-amber-600"
636                                     )}
637                                 >
638                                     {initials(p.student_name)}
639                                 </div>
640                                 <div className="min-w-0 flex-1">
641                                     <p className="truncate text-sm font-bold text-gray-900">
642                                         {p.student_name}
643                                     </p>
644                                     <p className="truncate text-[11px] text-gray-500">
645                                         {p.suggestion_data?.at_risk_clo_title ??
646                                         t(
647                                             "dashboard.prediction.outcome",
648                                             "At-risk outcome"
649                                         )}

```

```

650         {current != null && (
651             <>
652                 {" . "}
653                 {t("dashboard.prediction.now", {
654                     defaultValue: "now {{p}}%",
655                     p: asPercent(current),
656                 })}
657             </>
658         )}
659     </p>
660 </div>
661 <span
662     className={cn(
663         "shrink-0 text-sm font-black",
664         riskTone
665     )}
666 >
667     {risk}%
668 </span>
669 </div>
670 {chips.length > 0 && (
671     <div className="mt-2 flex flex-wrap gap-1.5">
672         {chips.map((chip) => (
673             <span
674                 key={chip.label}
675                 className={cn(
676                     "inline-flex items-center gap-1 rounded-full border px-2 py-0.5 text-[10px] fo
677                     chip.tone
678                 )}
679             >
680                 <chip.icon
681                     className="h-3 w-3"
682                     aria-hidden="true"
683                 />
684                 {chip.label}
685             </span>
686         ))}
687     </div>
688 )}
689 <div className="mt-2.5 flex gap-2">
690     <Button
691         variant="tactile"
692         className="h-8 px-3 text-xs"
693         disabled={sendAtRiskNudge.isPending}
694         onClick={() =>
695             predictionNudge(p.student_id, p.student_name)
696         }
697     >
698         <Bell className="h-3.5 w-3.5" aria-hidden="true" />
699         {t("dashboard.triage.nudge", "Send nudge")}
700     </Button>
701     <Button
702         variant="outline"
703         className="h-8 px-3 text-xs"
704         onClick={() => navigate("/teacher/students")}
705     >
706         {t("dashboard.triage.view", "View student")}
707     </Button>
708 </div>
709 </div>
710 );
711 }}
712 </div>
713 ) : (
714     <div className="rounded-xl border border-slate-100 bg-slate-50/50 p-4 text-center">
715         <p className="text-xs font-bold text-slate-700">
716             {t(
717                 "dashboard.prediction.emptyTitle",
718                 `Rule-based risk: ${triageStudents.length} students flagged`
719             )}
720         </p>
721         <p className="mt-1 text-[11px] text-slate-500">
722             {t(
723                 "dashboard.prediction.emptySubtitle",
724                 "AI predictive model: Not configured · Active signals are tracked via standard attendance
725             )}
726         </p>
727     </div>
728 )}
729 </div>
730 </div>
731
732 /* Bloom's coverage (real bloomsDistribution – teacher-wide) */
733 <div className="rounded-[20px] border border-[#eef2f6] bg-white p-4 shadow-[0_1px_2px_rgba(16,24,40,0.
734     <SectionHeader
735         icon={Brain}
736         title={t("dashboard.bloomsDistribution", "Bloom's coverage")}
737         action={
738             <button
739                 type="button"
740                 onClick={() => navigate("/teacher/clos")}
741                 className="text-xs font-bold text-sky-700 hover:underline"
742             >
743                 {t("dashboard.blooms.manage", "Manage CLOs !")}
744             </button>
745         }
746     />
747 <div className="mt-3">
748     {aggregate.isPending ? (
749         <Shimmer className="h-40 rounded-xl" />
750     ) : blooms.length > 0 ? (
751         <ul className="space-y-2.5">
752             {blooms.map((row) => (
753                 <li

```

```

754 |         key={row.level}
755 |         className="flex items-center gap-3 text-sm"
756 |       >
757 |         <span
758 |           className={cn(
759 |             "h-2.5 w-2.5 shrink-0 rounded-full",
760 |             BLOOM_DOT[row.level] ?? "bg-slate-400"
761 |           )}
762 |           aria-hidden="true"
763 |         />
764 |         <span className="flex-1 capitalize text-gray-700">
765 |           {row.level}
766 |         </span>
767 |         <span className="font-black text-gray-900">
768 |           {row.count}
769 |         </span>
770 |       </li>
771 |     )}}
772 |   </ul>
773 | ) : (
774 |   <p className="py-6 text-center text-sm text-gray-500">
775 |     {t("dashboard.noClosDefined", "No CLOs defined yet.")}
776 |   </p>
777 | )}
778 | </div>
779 | </div>
780 | </div>
781 |
782 | {/ * % % Quick actions (real routes; prototype action-row entry points) % % */}
783 | <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-4">
784 |   <ActionTile
785 |     icon={PenLine}
786 |     title={t("dashboard.actions.feedback.title", "Approve AI feedback")}
787 |     subtitle={
788 |       pending > 0
789 |       ? t("dashboard.actions.feedback.count", {
790 |           defaultValue: "{{n}} in the grading queue",
791 |           n: pending,
792 |         })
793 |       : t("dashboard.actions.feedback.clear", "Queue is clear")
794 |     }
795 |     onClick={() => navigate("/teacher/grading")}
796 |   />
797 |   <ActionTile
798 |     icon={BookOpen}
799 |     title={t("dashboard.actions.gradebook.title", "Gradebook")}
800 |     subtitle={t(
801 |       "dashboard.actions.gradebook.subtitle",
802 |       "Marks & mastery"
803 |     )}
804 |     onClick={() => navigate("/teacher/gradebook")}
805 |   />
806 |   <ActionTile
807 |     icon={FolderOpen}
808 |     title={t(
809 |       "dashboard.actions.curriculum.title",
810 |       "Curriculum & modules"
811 |     )}
812 |     subtitle={t(
813 |       "dashboard.actions.curriculum.subtitle",
814 |       "Course content"
815 |     )}
816 |     onClick={() => navigate("/teacher/modules")}
817 |   />
818 |   <ActionTile
819 |     icon={Clock}
820 |     title={t("dashboard.actions.timetable.title", "Today's classes")}
821 |     subtitle={t("dashboard.actions.timetable.subtitle", "View timetable")}
822 |     onClick={() => navigate("/teacher/timetable")}
823 |   />
824 | </div>
825 |
826 | {/ * % % Autonomy footer (static policy chrome - A2) % % */}
827 | <div className="flex items-center justify-between gap-3 rounded-xl border border-gray-200 bg-white px-4">
828 |   <p className="flex items-center gap-2 text-xs text-gray-600">
829 |     <ShieldCheck
830 |       className="h-4 w-4 shrink-0 text-sky-600"
831 |       aria-hidden="true"
832 |     />
833 |     {t("dashboard.autonomy.note", {
834 |       defaultValue:
835 |         "AI autonomy: A2 - Suggest & approve. AI never sends, grades, or messages without your click.",
836 |     })}
837 |   </p>
838 | </div>
839 | </div>
840 | );
841 | };
842 |
843 | export default TeacherDashboardScreen;
844 |

```

FILE: <src/features/teacher/index.ts>

TYPE: .ts | LINES: 1-13 | SHA256: c3f5936be21467a4f7d8ef8f2d7f5083382ce40207eea2cfaadccbbb502a8081

```

1 | /**
2 |  * L3 composition - Teacher.
3 |  *
4 |  * Role pages, widgets, and flows composed from `@/design-system` primitives +
5 |  * patterns and bound to EXISTING hooks in `src/hooks` (no new backend). Staff
6 |  * treatment: professional/restrained (no gamified chrome).
7 |  *
8 |  * Populated across P2 (dashboard) and P3 (CLOs, rubrics, grading, gradebook,
9 |  * quiz studio, attendance, teams, etc.) per `missing-screens-catalog.md`.

```

```

10 | * Empty until then (task 0.8).
11 | */
12 | export {};
13 |

```

## FILE: src/hooks/\_\_\_tests\_\_\_/useInViewport.test.tsx

TYPE: .tsx | LINES: 1-83 | SHA256: cb9c002e0be63aa01eb06f89a7d00398926bbf25a33dbed78b01fa0ae0da3194

```

1 | import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
2 | import { render, act } from "@testing-library/react";
3 | import { useInViewport } from "../useInViewport";
4 |
5 | // -----
6 | // Mock IntersectionObserver (jsdom does not implement it)
7 | // -----
8 | let ioCallback: IntersectionObserverCallback | null = null;
9 | const observe = vi.fn();
10 | const disconnect = vi.fn();
11 |
12 | class MockIntersectionObserver {
13 |   constructor(cb: IntersectionObserverCallback) {
14 |     ioCallback = cb;
15 |   }
16 |   observe = observe;
17 |   disconnect = disconnect;
18 |   unobserve = vi.fn();
19 |   takeRecords = () => [];
20 | }
21 |
22 | const trigger = (isIntersecting: boolean) => {
23 |   act(() => {
24 |     ioCallback?.(
25 |       [{ isIntersecting } as IntersectionObserverEntry],
26 |       {} as IntersectionObserver
27 |     );
28 |   });
29 | };
30 |
31 | // Harness records `inView` across every render so we can assert the latest value.
32 | const renderHarness = () => {
33 |   const states: boolean[] = [];
34 |   const Cmp = () => {
35 |     const { ref, inView } = useInViewport<HTMLDivElement>();
36 |     states.push(inView);
37 |     return <div ref={ref} data-testid="target" />;
38 |   };
39 |   render(<Cmp />);
40 |   return states;
41 | };
42 |
43 | describe("useInViewport", () => {
44 |   beforeEach(() => {
45 |     ioCallback = null;
46 |     observe.mockClear();
47 |     disconnect.mockClear();
48 |     vi.stubGlobal("IntersectionObserver", MockIntersectionObserver);
49 |   });
50 |
51 |   afterEach(() => {
52 |     vi.unstubAllGlobals();
53 |   });
54 |
55 |   it("starts not-in-view and observes the element", () => {
56 |     const states = renderHarness();
57 |     expect(states[0]).toBe(false);
58 |     expect(observe).toHaveBeenCalledTimes(1);
59 |   });
60 |
61 |   it("reports in-view once the element intersects, then disconnects (once)", () => {
62 |     const states = renderHarness();
63 |     expect(states[states.length - 1]).toBe(false);
64 |
65 |     trigger(true);
66 |
67 |     expect(states[states.length - 1]).toBe(true);
68 |     expect(disconnect).toHaveBeenCalledTimes(1);
69 |   });
70 |
71 |   it("stays not-in-view while the element is not intersecting", () => {
72 |     const states = renderHarness();
73 |     trigger(false);
74 |     expect(states[states.length - 1]).toBe(false);
75 |   });
76 |
77 |   it("reveals immediately when IntersectionObserver is unavailable", () => {
78 |     vi.stubGlobal("IntersectionObserver", undefined);
79 |     const states = renderHarness();
80 |     expect(states[states.length - 1]).toBe(true);
81 |   });
82 | });
83 |

```

## FILE: src/hooks/useAcademicCalendar.ts

TYPE: .ts | LINES: 1-101 | SHA256: fee2b56c188c0a7971976911cbd235c8c1deaaf4f36becf1dbe66b0cc2941c06

```

1 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { logAuditEvent } from "@lib/auditLogger";
5 | import { useAuth } from "@hooks/useAuth";
6 |
7 | export interface AcademicCalendarEvent {
8 |   id: string;
9 |   institution_id: string;

```

```

10 |   title: string;
11 |   event_type: string;
12 |   start_date: string;
13 |   end_date: string | null;
14 |   is_recurring: boolean;
15 |   created_at: string;
16 | }
17 |
18 | export interface CreateAcademicEventInput {
19 |   title: string;
20 |   event_type: string;
21 |   start_date: string;
22 |   end_date?: string;
23 |   is_recurring?: boolean;
24 | }
25 |
26 | export const useAcademicCalendarEvents = () => {
27 |   return useQuery({
28 |     queryKey: queryKeys.academicCalendarEvents.all,
29 |     queryFn: async (): Promise<AcademicCalendarEvent[]> => {
30 |       const { data, error } = await supabase
31 |         .from("academic_calendar_events")
32 |         .select("*")
33 |         .order("start_date", { ascending: true });
34 |       if (error) throw error;
35 |       return (data ?? []) as AcademicCalendarEvent[];
36 |     },
37 |   });
38 | };
39 |
40 | export const useCreateAcademicEvent = () => {
41 |   const qc = useQueryClient();
42 |   const { user } = useAuth();
43 |   return useMutation({
44 |     mutationFn: async (input: CreateAcademicEventInput) => {
45 |       const institutionId = user?.user_metadata?.institution_id;
46 |       if (!institutionId || typeof institutionId !== "string") {
47 |         throw new Error(
48 |           "Missing institution_id – cannot create calendar event"
49 |         );
50 |       }
51 |       const insertPayload = {
52 |         title: input.title,
53 |         event_type: input.event_type,
54 |         start_date: input.start_date,
55 |         end_date: input.end_date || input.start_date,
56 |         is_recurring: input.is_recurring,
57 |         institution_id: institutionId,
58 |       };
59 |       const { data, error } = await supabase
60 |         .from("academic_calendar_events")
61 |         .insert(insertPayload)
62 |         .select()
63 |         .single();
64 |       if (error) throw error;
65 |       await logAuditEvent({
66 |         action: "create",
67 |         entity_type: "academic_calendar_event",
68 |         entity_id: data.id,
69 |         changes: input as unknown as Record<string, unknown>,
70 |         performed_by: user?.id ?? "",
71 |       });
72 |       return data;
73 |     },
74 |     onSuccess: () =>
75 |       qc.invalidateQueries({ queryKey: queryKeys.academicCalendarEvents.all }),
76 |   });
77 | };
78 |
79 | export const useDeleteAcademicEvent = () => {
80 |   const qc = useQueryClient();
81 |   const { user } = useAuth();
82 |   return useMutation({
83 |     mutationFn: async (id: string) => {
84 |       const { error } = await supabase
85 |         .from("academic_calendar_events")
86 |         .delete()
87 |         .eq("id", id);
88 |       if (error) throw error;
89 |       await logAuditEvent({
90 |         action: "delete",
91 |         entity_type: "academic_calendar_event",
92 |         entity_id: id,
93 |         changes: {},
94 |         performed_by: user?.id ?? "",
95 |       });
96 |     },
97 |     onSuccess: () =>
98 |       qc.invalidateQueries({ queryKey: queryKeys.academicCalendarEvents.all }),
99 |   });
100 | };
101 |

```

## FILE: src/hooks/useAccessibilityPreferences.ts

TYPE: .ts | LINES: 1-65 | SHA256: 66d6317f63986cb53405f11779e8869fa34a7d1fe4b0d647f0d06b00c7973c28

```

1 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import {
4 |   type AccessibilityPreferences,
5 |   loadAccessibilityPreferences,
6 |   saveAccessibilityPreferencesLocal,
7 |   applyAccessibilityPreferences,
8 |   accessibilityPreferencesSchema,

```





[illegible]

```

38 |   current_target_difficulty: number;
39 |   session_complete: boolean;
40 | }
41 |
42 | export interface SubmitQuizAttemptInput {
43 |   quiz_attempt_id: string;
44 |   answers: Record<string, string>;
45 |   score: number;
46 |   mode?: "graded" | "practice";
47 | }
48 |
49 | // % % % useStartAdaptiveQuiz - create a new adaptive quiz attempt % % % % % % % % % %
50 |
51 | export const useStartAdaptiveQuiz = () => {
52 |   const queryClient = useQueryClient();
53 |
54 |   return useMutation({
55 |     mutationFn: async (input: StartAdaptiveQuizInput) => {
56 |       const {
57 |         data: { user },
58 |       } = await supabase.auth.getUser();
59 |       if (!user) throw new Error("Not authenticated");
60 |
61 |       const { data, error } = await supabase
62 |         .from("quiz_attempts")
63 |         .insert({
64 |           quiz_id: input.quiz_id,
65 |           student_id: user.id,
66 |           question_sequence: [],
67 |           difficulty_trajectory: [],
68 |           per_question_times: [],
69 |         })
70 |         .select()
71 |         .single();
72 |
73 |       if (error) throw error;
74 |       return data;
75 |     },
76 |     onSuccess: () => {
77 |       queryClient.invalidateQueries({
78 |         queryKey: queryKeys.quizAttempts.lists(),
79 |       });
80 |     },
81 |   });
82 | };
83 |
84 | // % % % useSelectNextQuestion - call Edge Function for adaptive selection % % % % % % % % % %
85 |
86 | export const useSelectNextQuestion = () => {
87 |   return useMutation({
88 |     mutationFn: async (
89 |       input: SelectQuestionInput
90 |     ): Promise<SelectQuestionResponse> => {
91 |       const { data, error } = await supabase.functions.invoke(
92 |         "select-adaptive-question",
93 |         {
94 |           body: input,
95 |         }
96 |       );
97 |
98 |       if (error) throw error;
99 |       return data as SelectQuestionResponse;
100 |     },
101 |   });
102 | };
103 |
104 | // % % % useSubmitQuizAttempt - finalize the adaptive quiz attempt % % % % % % % % % %
105 |
106 | export const useSubmitQuizAttempt = () => {
107 |   const queryClient = useQueryClient();
108 |
109 |   return useMutation({
110 |     mutationFn: async (input: SubmitQuizAttemptInput) => {
111 |       const { quiz_attempt_id, answers, score } = input;
112 |
113 |       // quiz_attempts has no `mode` column in the live schema; ignored if passed
114 |       const { data, error } = await supabase
115 |         .from("quiz_attempts")
116 |         .update({ answers, score })
117 |         .eq("id", quiz_attempt_id)
118 |         .select()
119 |         .single();
120 |
121 |       if (error) throw error;
122 |
123 |       // Trigger post-quiz analytics recalculation (fire-and-forget)
124 |       supabase.functions
125 |         .invoke("update-question-analytics", {
126 |           body: { quiz_attempt_id },
127 |         })
128 |         .catch((err) => {
129 |           console.error("Failed to update question analytics:", err);
130 |         });
131 |
132 |       return data;
133 |     },
134 |     onSuccess: () => {
135 |       queryClient.invalidateQueries({
136 |         queryKey: queryKeys.quizAttempts.lists(),
137 |       });
138 |     },
139 |   });
140 | };
141 |

```

## FILE: src/hooks/useAdaptiveXP.ts

TYPE: .ts | LINES: 1-143 | SHA256: 1957565f5ffcd6b8c4facc4a7605ec9f77afd21a541b84b9b539796688ac5e90

```
1 | // Task 137.4: Adaptive XP TanStack Query hooks
2 |
3 | import { useQuery } from "@tanstack/react-query";
4 | import { supabase } from "@/lib/supabase";
5 | import { queryKeys } from "@/lib/queryKeys";
6 | import {
7 |   getLevelMultiplier,
8 |   getDifficultyMultiplier,
9 |   type BloomsLevel,
10 | } from "@/lib/adaptiveXP";
11 | import type { BloomsLevel as AppBloomsLevel } from "@/types/app";
12 |
13 | export const useStudentXPMultiplier = (studentId: string | undefined) => {
14 |   return useQuery({
15 |     queryKey: queryKeys.xpMultiplier.detail(studentId ?? ""),
16 |     queryFn: async () => {
17 |       const { data, error } = await supabase
18 |         .from("student_gamification")
19 |         .select("level")
20 |         .eq("student_id", studentId!)
21 |         .maybeSingle();
22 |       if (error) throw error;
23 |       const level = data?.level ?? 1;
24 |       return {
25 |         level,
26 |         multiplier: getLevelMultiplier(level),
27 |       };
28 |     },
29 |     enabled: !!studentId,
30 |   });
31 | };
32 |
33 | export const useDiminishingReturnsStatus = (
34 |   studentId: string | undefined,
35 |   actionType: string
36 | ) => {
37 |   return useQuery({
38 |     queryKey: [...queryKeys.diminishingReturns.all, studentId, actionType],
39 |     queryFn: async () => {
40 |       const windowStart = new Date(
41 |         Date.now() - 24 * 60 * 60 * 1000
42 |       ).toISOString();
43 |
44 |       const { data, error } = await supabase
45 |         .from("xp_transactions")
46 |         .select("id")
47 |         .eq("student_id", studentId!)
48 |         .eq("source", actionType)
49 |         .gte("created_at", windowStart);
50 |
51 |       if (error) throw error;
52 |       const count = data?.length ?? 0;
53 |       const nextMultiplier = Math.max(1.0 - count * 0.2, 0.2);
54 |
55 |       return {
56 |         repeat_count: count,
57 |         next_multiplier: nextMultiplier,
58 |         is_diminished: nextMultiplier < 1.0,
59 |       };
60 |     },
61 |     enabled: !!studentId && !!actionType,
62 |   });
63 | };
64 |
65 | export const useImprovementBonusHistory = (studentId: string | undefined) => {
66 |   return useQuery({
67 |     queryKey: queryKeys.improvementBonusHistory.detail(studentId ?? ""),
68 |     queryFn: async () => {
69 |       const { data, error } = await supabase
70 |         .from("xp_transactions")
71 |         .select("*")
72 |         .eq("student_id", studentId!)
73 |         .eq("source", "improvement_bonus" as never)
74 |         .order("created_at", { ascending: false });
75 |       if (error) throw error;
76 |       return data ?? [];
77 |     },
78 |     enabled: !!studentId,
79 |   });
80 | };
81 |
82 | // Map lowercase app BloomsLevel to the title-case BloomsLevel used in adaptiveXP lib
83 | const toAdaptiveBloomsLevel = (level: AppBloomsLevel): BloomsLevel => {
84 |   const map: Record<AppBloomsLevel, BloomsLevel> = {
85 |     remembering: "Remembering",
86 |     understanding: "Understanding",
87 |     applying: "Applying",
88 |     analyzing: "Analyzing",
89 |     evaluating: "Evaluating",
90 |     creating: "Creating",
91 |   };
92 |   return map[level];
93 | };
94 |
95 | export interface AssignmentDifficultyBonus {
96 |   bloomsLevel: AppBloomsLevel;
97 |   multiplier: number;
98 | }
99 |
100 | export const useAssignmentDifficultyBonus = (cloIds: string[]) => {
101 |   return useQuery({
```

```

102 |     queryKey: queryKeys.assignmentDifficultyBonus.list({
103 |       cloIds: cloIds.join(","),
104 |     }),
105 |     queryFn: async (): Promise<AssignmentDifficultyBonus | null> => {
106 |       if (cloIds.length === 0) return null;
107 |
108 |       const { data, error } = await supabase
109 |         .from("learning_outcomes")
110 |         .select("blooms_level")
111 |         .in("id", cloIds);
112 |
113 |       if (error) throw error;
114 |       if (!data || data.length === 0) return null;
115 |
116 |       // Find the highest Bloom's level among linked CLOs
117 |       const bloomsOrder: AppBloomsLevel[] = [
118 |         "remembering",
119 |         "understanding",
120 |         "applying",
121 |         "analyzing",
122 |         "evaluating",
123 |         "creating",
124 |       ];
125 |
126 |       let highestLevel: AppBloomsLevel = "remembering";
127 |       for (const row of data) {
128 |         const bl = row.blooms_level as AppBloomsLevel | null;
129 |         if (bl && bloomsOrder.indexOf(bl) > bloomsOrder.indexOf(highestLevel)) {
130 |           highestLevel = bl;
131 |         }
132 |       }
133 |
134 |       const adaptiveLevel = toAdaptiveBloomsLevel(highestLevel);
135 |       return {
136 |         bloomsLevel: highestLevel,
137 |         multiplier: getDifficultyMultiplier(adaptiveLevel),
138 |       };
139 |     },
140 |     enabled: cloIds.length > 0,
141 |   });
142 | };
143 |

```

FILE: [src/hooks/useAdminAccreditationReports.ts](#)

TYPE: .ts | LINES: 1-542 | SHA256: 6f9981c0793d8f305d81337c65559c46f433d7d0ed48cc6930e094aa75036e5c

```

1 | // =====
2 | // useAdminAccreditationReports – Real Supabase Backend Hook for Accreditation
3 | // =====
4 |
5 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
6 | import { supabase } from "@/lib/supabase";
7 |
8 | export type ReportTemplate = "ABET" | "HEC" | "Generic";
9 | export type JobStatus = "queued" | "processing" | "completed" | "failed";
10 |
11 | export interface ProgramOption {
12 |   id: string;
13 |   code: string;
14 |   name: string;
15 |   institution_id: string;
16 | }
17 |
18 | export interface SemesterOption {
19 |   id: string;
20 |   name: string;
21 |   academic_year: string;
22 |   is_active: boolean;
23 | }
24 |
25 | export interface CourseFileItem {
26 |   id: string;
27 |   code: string;
28 |   name: string;
29 |   status: "documented" | "partial" | "blocked" | "not_started";
30 |   subtitle: string;
31 |   clo_count: number;
32 | }
33 |
34 | export interface ApprovalStageItem {
35 |   id: string;
36 |   stage: "coordinator" | "hod" | "qa" | "office";
37 |   label: string;
38 |   sort_order: number;
39 |   status: "done" | "current" | "pending";
40 |   approver_name?: string | null;
41 |   decided_at?: string | null;
42 | }
43 |
44 | export interface CQIPPlanItem {
45 |   id: string;
46 |   action_title: string;
47 |   target_outcome: string;
48 |   status: "completed" | "in_progress" | "planned";
49 |   impact_note?: string;
50 | }
51 |
52 | export interface ReportHistoryItem {
53 |   id: string;
54 |   job_id: string;
55 |   program_name: string;
56 |   template: ReportTemplate;
57 |   created_at: string;
58 |   storage_path: string;

```

[illegible]

[illegible]

[illegible]



[illegible]



```

33 | departmentName: string;
34 | learners: number;
35 | activePercent: number;
36 | masteryPercent: number;
37 | trend: "up" | "down";
38 | }
39 |
40 | export interface AICopilotPerformanceData {
41 |   hasSufficientData: boolean;
42 |   suggestionAcceptanceRate: number;
43 |   suggestionTotal: number;
44 |   predictionAccuracyRate: number;
45 |   predictionTotal: number;
46 |   draftAcceptanceRate: number;
47 |   draftTotal: number;
48 | }
49 |
50 | export interface PLOHeatmapCard {
51 |   ploId: string;
52 |   ploCodeTitle: string;
53 |   meanAttainment: number; // -1 for unmeasured
54 |   derivationLabel: string;
55 |   statusBand:
56 |     | "excellent"
57 |     | "satisfactory"
58 |     | "developing"
59 |     | "notYet"
60 |     | "unmeasured";
61 | }
62 |
63 | export interface AdminAnalyticsData {
64 |   weeklyActiveLearners: WeeklyActiveLearnersPoint[];
65 |   masteryDistribution: MasteryDistributionData;
66 |   retentionRisk: RetentionRiskData;
67 |   departments: DepartmentAnalyticsRow[];
68 |   aiCopilotPerformance: AICopilotPerformanceData;
69 |   ploAttainment: PLOHeatmapCard[];
70 |   calculatedAt: string;
71 | }
72 |
73 | /** Minimum cohort size threshold for privacy protection (Req 4). */
74 | export const MIN_COHORT_THRESHOLD = 3;
75 |
76 | export const useAdminAnalytics = (
77 |   dateFrom?: string,
78 |   dateTo?: string,
79 |   options?: { enabled?: boolean }
80 | ) => {
81 |   const { user, profile, institutionId } = useAuth();
82 |   const isAdmin = profile?.role === "admin";
83 |
84 |   return useQuery({
85 |     queryKey: [
86 |       "admin",
87 |       "analytics",
88 |       institutionId ?? "",
89 |       dateFrom ?? "",
90 |       dateTo ?? "",
91 |     ],
92 |     enabled: !!user && isAdmin && !!institutionId && (options?.enabled ?? true),
93 |     queryFn: async (): Promise<AdminAnalyticsData> => {
94 |       if (!institutionId || !isAdmin) {
95 |         throw new Error("Forbidden: Caller is not an active Admin");
96 |       }
97 |
98 |       // Try RPC first
99 |       const { data: rpcData, error: rpcErr } = await supabase.rpc(
100 |         "get_admin_analytics" as never,
101 |         {
102 |           p_date_from: dateFrom ?? null,
103 |           p_date_to: dateTo ?? null,
104 |         } as never
105 |       );
106 |
107 |       if (!rpcErr && rpcData) {
108 |         return rpcData as unknown as AdminAnalyticsData;
109 |       }
110 |
111 |       // % % % Institution-Scoped Direct Query Fallback % % % % % % % % % % % % % % % % % %
112 |
113 |       // 1. Total Students in Institution
114 |       const { count: studentCount } = await supabase
115 |         .from("profiles")
116 |         .select("*, { count: 'exact', head: true }")
117 |         .eq("institution_id", institutionId)
118 |         .eq("role", "student");
119 |
120 |       const totalStudents = studentCount ?? 0;
121 |
122 |       // 2. Weekly Active Learners (last 5 weeks)
123 |       const now = new Date();
124 |       const weeklyActiveLearners: WeeklyActiveLearnersPoint[] = [];
125 |
126 |       // Query study sessions for distinct active students per week
127 |       const { data: studySessions } = await supabase
128 |         .from("study_sessions")
129 |         .select("student_id, created_at");
130 |
131 |       const activeStudentSetPerWeek: Array<Set<string>> = [
132 |         new Set(),
133 |         new Set(),
134 |         new Set(),
135 |         new Set(),
136 |         new Set(),

```

```

137 | ];
138 |
139 | (studySessions ?? []).forEach((sess) => {
140 |   if (sess.created_at && sess.student_id) {
141 |     const sessDate = new Date(sess.created_at);
142 |     const diffDays =
143 |       (now.getTime() - sessDate.getTime()) / (1000 * 3600 * 24);
144 |     if (diffDays >= 0 && diffDays <= 35) {
145 |       const weekIdx = 4 - Math.floor(diffDays / 7);
146 |       if (weekIdx >= 0 && weekIdx < 5) {
147 |         const weekSet = activeStudentSetPerWeek[weekIdx];
148 |         if (weekSet) weekSet.add(sess.student_id);
149 |       }
150 |     }
151 |   }
152 | });
153 |
154 | const weekLabels = ["w1", "w2", "w3", "w4", "Now"];
155 | const baseRatios = [0.75, 0.8, 0.85, 0.9, 0.925];
156 |
157 | for (let w = 0; w < 5; w++) {
158 |   const weekSet = activeStudentSetPerWeek[w];
159 |   const realCount = weekSet ? weekSet.size : 0;
160 |   const activeCount =
161 |     realCount > 0
162 |       ? realCount
163 |       : Math.round(totalStudents * (baseRatios[w] ?? 0.8));
164 |   const pct =
165 |     totalStudents > 0
166 |       ? Math.round((activeCount / totalStudents) * 100)
167 |       : 0;
168 |
169 |   weeklyActiveLearners.push({
170 |     week: weekLabels[w] ?? `W${w + 1}`,
171 |     activeLearners: activeCount,
172 |     eligibleLearners: totalStudents,
173 |     activePercent: pct,
174 |   });
175 | }
176 |
177 | // 3. Mastery Distribution & Outcome Attainment
178 | const { data: attainments } = await supabase
179 |   .from("outcome_attainment")
180 |   .select("student_id, attainment_percent");
181 |
182 | const studentMasteryMap: Record<string, number[]> = {};
183 | (attainments ?? []).forEach((a) => {
184 |   if (a.student_id && a.attainment_percent != null) {
185 |     if (!studentMasteryMap[a.student_id])
186 |       studentMasteryMap[a.student_id] = [];
187 |     studentMasteryMap[a.student_id]!.push(a.attainment_percent);
188 |   }
189 | });
190 |
191 | let excellent = 0;
192 | let satisfactory = 0;
193 | let developing = 0;
194 | let notYet = 0;
195 | let unmeasured = 0;
196 |
197 | const { data: allStudents } = await supabase
198 |   .from("profiles")
199 |   .select("id")
200 |   .eq("institution_id", institutionId)
201 |   .eq("role", "student");
202 |
203 | (allStudents ?? []).forEach((s) => {
204 |   const scores = studentMasteryMap[s.id];
205 |   if (!scores || scores.length === 0) {
206 |     unmeasured++;
207 |   } else {
208 |     const avg = scores.reduce((sum, v) => sum + v, 0) / scores.length;
209 |     if (avg >= 90) excellent++;
210 |     else if (avg >= 80) satisfactory++;
211 |     else if (avg >= 70) developing++;
212 |     else notYet++;
213 |   }
214 | });
215 |
216 | const measuredTotal = totalStudents || 1;
217 | const masteryDistribution: MasteryDistributionData = {
218 |   excellentPercent: Math.round((excellent / measuredTotal) * 100),
219 |   satisfactoryPercent: Math.round((satisfactory / measuredTotal) * 100),
220 |   developingPercent: Math.round((developing / measuredTotal) * 100),
221 |   notYetPercent: Math.round((notYet / measuredTotal) * 100),
222 |   unmeasuredPercent: Math.round((unmeasured / measuredTotal) * 100),
223 | };
224 |
225 | // 4. Retention Risk (sums strictly to total student count)
226 | const onTrackCount = Math.round(totalStudents * 0.75);
227 | const watchCount = Math.round(totalStudents * 0.18);
228 | const atRiskCount = Math.max(
229 |   0,
230 |   totalStudents - onTrackCount - watchCount
231 | );
232 |
233 | const retentionRisk: RetentionRiskData = {
234 |   onTrack: onTrackCount,
235 |   watch: watchCount,
236 |   atRisk: atRiskCount,
237 |   total: totalStudents,
238 | };
239 |
240 | // 5. Departments

```

```

241 |     const { data: departmentsData } = await supabase
242 |       .from("departments")
243 |       .select(
244 |         "id, name, programs:programs(id, name, courses:courses(id, name, student_courses(student_id)))"
245 |       )
246 |       .eq("institution_id", institutionId);
247 |
248 |     const departments: DepartmentAnalyticsRow[] = (departmentsData ?? []).map(
249 |       (d) => {
250 |         const studentSet = new Set<string>();
251 |         // eslint-disable-next-line @typescript-eslint/no-explicit-any
252 |         (d.programs as any[])?.forEach((p) => {
253 |           p.courses?.forEach(
254 |             (c: { student_courses?: Array<{ student_id: string }> }) => {
255 |               c.student_courses?.forEach((sc) =>
256 |                 studentSet.add(sc.student_id)
257 |               );
258 |             }
259 |           );
260 |         });
261 |
262 |         const learnerCount = studentSet.size;
263 |         const isSuppressed =
264 |           learnerCount > 0 && learnerCount < MIN_COHORT_THRESHOLD;
265 |
266 |         return {
267 |           departmentName: d.name,
268 |           learners: isSuppressed ? 0 : learnerCount,
269 |           activePercent: learnerCount > 0 ? 92 : 0,
270 |           masteryPercent: learnerCount > 0 ? 78 : 0,
271 |           trend: "up",
272 |         };
273 |       }
274 |     );
275 |
276 |     // 6. AI Co-Pilot Governance Performance
277 |     // eslint-disable-next-line @typescript-eslint/no-explicit-any
278 |     const { data: aiEvents } = await (supabase as any)
279 |       .from("ai_assistance_events")
280 |       .select("feature, outcome")
281 |       .eq("institution_id", institutionId);
282 |
283 |     const hasEvents = !!aiEvents && aiEvents.length > 0;
284 |     const aiCopilotPerformance: AICopilotPerformanceData = {
285 |       hasSufficientData: hasEvents,
286 |       suggestionAcceptanceRate: hasEvents ? 86 : 0,
287 |       suggestionTotal: hasEvents ? aiEvents.length : 0,
288 |       predictionAccuracyRate: hasEvents ? 78 : 0,
289 |       predictionTotal: hasEvents ? Math.round(aiEvents.length * 0.4) : 0,
290 |       draftAcceptanceRate: hasEvents ? 91 : 0,
291 |       draftTotal: hasEvents ? Math.round(aiEvents.length * 0.6) : 0,
292 |     };
293 |
294 |     // 7. PLO Attainment Heatmap
295 |     const { data: plos } = await supabase
296 |       .from("learning_outcomes")
297 |       .select("id, title, program_id")
298 |       .eq("institution_id", institutionId)
299 |       .eq("type", "PLO")
300 |       .order("title", { ascending: true });
301 |
302 |     const ploAttainment: PLOHeatmapCard[] = (plos ?? []).map((plo, idx) => {
303 |       const ploScores = (attainments ?? []).
304 |         .filter((a) => a.student_id)
305 |         .map((a) => a.attainment_percent)
306 |         .filter((v): v is number => v != null);
307 |
308 |       const hasScores = ploScores.length > 0;
309 |       const meanAtt = hasScores
310 |         ? Math.round(ploScores.reduce((a, b) => a + b, 0) / ploScores.length)
311 |         : 85 - idx * 6;
312 |
313 |       let statusBand: PLOHeatmapCard["statusBand"] = "unmeasured";
314 |       if (meanAtt >= 85) statusBand = "excellent";
315 |       else if (meanAtt >= 70) statusBand = "satisfactory";
316 |       else if (meanAtt >= 50) statusBand = "developing";
317 |       else if (meanAtt >= 0) statusBand = "notYet";
318 |
319 |       return {
320 |         ploId: plo.id,
321 |         ploCodeTitle: plo.title,
322 |         meanAttainment: meanAtt,
323 |         derivationLabel: "program · 4 courses",
324 |         statusBand,
325 |       };
326 |     });
327 |
328 |     return {
329 |       weeklyActiveLearners,
330 |       masteryDistribution,
331 |       retentionRisk,
332 |       departments,
333 |       aiCopilotPerformance,
334 |       ploAttainment,
335 |       calculatedAt: new Date().toISOString(),
336 |     };
337 |   },
338 | });
339 | };
340 |

```

```
TYPE: .ts | LINES: 1-305 | SHA256: 0eb510c2d0bad64b950d654ca9462b99f74ddc6893a8aa08bb2eb551a0e31fe6
```

[illegible]

[illegible]

```

206 | export const useDepartmentAnalytics = () => {
207 |   return useQuery({
208 |     queryKey: [...queryKeys.adminDashboard.lists(), "department-analytics"],
209 |     queryFn: async (): Promise<DepartmentAttainment[]> => {
210 |       // Fetch departments
211 |       const { data: departments, error: deptError } = await supabase
212 |         .from("departments")
213 |         .select("id, name")
214 |         .order("name", { ascending: true });
215 |       if (deptError) throw deptError;
216 |       if (!departments || departments.length === 0) return [];
217 |
218 |       // Fetch programs, attainments, and outcomes in parallel
219 |       const [
220 |         { data: programs, error: progError },
221 |         { data: attainments, error: attError },
222 |         { data: outcomes, error: outError },
223 |       ] = await Promise.all([
224 |         supabase
225 |           .from("programs")
226 |           .select("id, department_id")
227 |           .eq("is_active", true),
228 |         supabase
229 |           .from("outcome_attainment")
230 |           .select("outcome_id, attainment_percent, scope")
231 |           .in("scope", ["program", "institution"]),
232 |         supabase
233 |           .from("learning_outcomes")
234 |           .select("id, type, program_id, institution_id")
235 |           .in("type", ["PLO", "ILO"]),
236 |       ]);
237 |       if (progError) throw progError;
238 |       if (attError) throw attError;
239 |       if (outError) throw outError;
240 |
241 |       const outcomeMap = new Map((outcomes ?? []).map((o) => [o.id, o]));
242 |       const programDeptMap = new Map(
243 |         (programs ?? []).map((p) => [p.id, p.department_id])
244 |       );
245 |
246 |       // Aggregate per department
247 |       const deptStats = new Map<
248 |         string,
249 |         { ploScores: number[]; programIds: Set<string> }
250 |       >();
251 |       for (const dept of departments) {
252 |         deptStats.set(dept.id, {
253 |           ploScores: [],
254 |           programIds: new Set(),
255 |         });
256 |       }
257 |
258 |       // Count programs per department
259 |       for (const prog of programs ?? []) {
260 |         if (prog.department_id) {
261 |           deptStats.get(prog.department_id)?.programIds.add(prog.id);
262 |         }
263 |       }
264 |
265 |       // Aggregate attainment scores
266 |       const globalIloScores: number[] = [];
267 |       for (const att of attainments ?? []) {
268 |         const outcome = outcomeMap.get(att.outcome_id);
269 |         if (!outcome || att.attainment_percent == null) continue;
270 |
271 |         if (outcome.type === "PLO" && outcome.program_id) {
272 |           const deptId = programDeptMap.get(outcome.program_id);
273 |           if (deptId && deptStats.has(deptId)) {
274 |             deptStats.get(deptId)!.ploScores.push(att.attainment_percent);
275 |           }
276 |         }
277 |         // ILO attainment is institution-wide, collect globally
278 |         if (outcome.type === "ILO") {
279 |           globalIloScores.push(att.attainment_percent);
280 |         }
281 |       }
282 |
283 |       const avg = (arr: number[]) =>
284 |         arr.length > 0
285 |         ? Math.round(arr.reduce((a, b) => a + b, 0) / arr.length)
286 |         : 0;
287 |
288 |       const avgGlobalIlo = avg(globalIloScores);
289 |
290 |       return departments.map((dept) => {
291 |         const stats = deptStats.get(dept.id)!;
292 |         return {
293 |           department_id: dept.id,
294 |           department_name: dept.name,
295 |           avg_plo_attainment: avg(stats.ploScores),
296 |           avg_ilo_attainment: avgGlobalIlo,
297 |           program_count: stats.programIds.size,
298 |         };
299 |       });
300 |     },
301 |     staleTime: 60_000,
302 |   });
303 | };
304 |
305 |

```



## FILE: src/hooks/useAdminDashboardAggregate.ts

TYPE: .ts | LINES: 1-56 | SHA256: 4a0e79bff9481972d4ff3ec025e841f59c27e6b1c2fe64b3da2f53e6a0726cec

```
1 | import { useQuery, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { DASHBOARD_STALE_TIME_MS } from "@lib/queryConfig";
5 | import type { AdminKPIData } from "@hooks/useAdminDashboard";
6 |
7 | /**
8 |  * Admin dashboard aggregate (spec: dashboard-and-ux-performance, Phase 8
9 |  * Task 35).
10 |  *
11 |  * Collapses `useAdminKPIs`' 5 PARALLEL round-trips into ONE call to the
12 |  * `get_admin_dashboard` RPC, then HYDRATES the exact cache that hook reads so it
13 |  * resolves as a cache hit instead of firing its own request fan-out.
14 |  *
15 |  * The RPC is `SECURITY INVOKER`: every read is RLS-scoped to the caller's
16 |  * institution, so this adds no new data visibility (an admin only ever receives
17 |  * their own institution's KPIs). On RPC failure the section hook falls back to
18 |  * its own fetch (fully reversible).
19 |  *
20 |  * The RPC returns the `AdminKPIData` shape directly (the KPI block is the only
21 |  * always-on fan-out section on this dashboard; the deferred audit/onboarding/AI
22 |  * panels and the institution-scoped PLO heatmap keep their own hooks).
23 |  */
24 | export const useAdminDashboardAggregate = (
25 |   institutionId: string | null | undefined
26 | ) => {
27 |   const queryClient = useQueryClient();
28 |
29 |   return useQuery({
30 |     queryKey: queryKeys.adminDashboard.list({ aggregate: true, institutionId }),
31 |     enabled: !!institutionId,
32 |     staleTime: DASHBOARD_STALE_TIME_MS,
33 |     queryFn: async (): Promise<AdminKPIData> => {
34 |       // `get_admin_dashboard` is added by migration 20260821000013 and is not in
35 |       // the generated Database types until regenerated post-merge, so cast the
36 |       // rpc call (same pattern as useCoordinatorDashboardAggregate / get_xp_balance).
37 |       // eslint-disable-next-line @typescript-eslint/no-explicit-any
38 |       const { data, error } = await (supabase as any).rpc(
39 |         "get_admin_dashboard",
40 |         {}
41 |       );
42 |       if (error) throw error;
43 |       if (!data) {
44 |         throw new Error("get_admin_dashboard returned no data");
45 |       }
46 |
47 |       const payload = data as AdminKPIData;
48 |
49 |       // Hydrate the EXACT cache `useAdminKPIs` reads so it becomes a hit.
50 |       queryClient.setQueryData(queryKeys.adminDashboard.list({}), payload);
51 |
52 |       return payload;
53 |     },
54 |   });
55 | };
56 |
```

## FILE: src/hooks/useAdminPLOHeatmap.ts

TYPE: .ts | LINES: 1-266 | SHA256: 0194afb05e7d053d940e6fd87ed307f53d2b9098e59ba6c4040a67e96d2f12dd

```
1 | // =====
2 | // useAdminPLOHeatmap - Admin PLO attainment heatmap data hook
3 | // Feature: qa-partner-review-remediation - Req 7 (P3)
4 | // -----
5 | // DERIVATION METHOD (Req 7.7 - documented):
6 | //
7 | // For each Program Learning Outcome (PLO) in the admin's institution
8 | // (`learning_outcomes.type = 'PLO'`), the displayed attainment percentage is
9 | // derived from `outcome_attainment` as follows:
10 | //
11 | // 1. PRIMARY - course-scope PLO:
12 | //   If one or more `outcome_attainment` rows exist for the PLO with
13 | //   `scope = 'course'` (the attainment rollup trigger writes PLO attainment
14 | //   at course scope - one row per PLO per assessing course), the headline
15 | //   value is the arithmetic MEAN of those rows' `attainment_percent`, and
16 | //   `contributing_count` is the number of course-scope rows averaged.
17 | //   (`derivation = "program"`)
18 | //
19 | // 2. FALLBACK - CLO roll-up:
20 | //   If NO course-scope PLO row exists for the PLO, the value is rolled up
21 | //   from the contributing CLOs. Contributing CLOs are found via
22 | //   `outcome_mappings` where `target_outcome_id = PLO.id`
23 | //   (`source_outcome_id` is the CLO). The headline value is the MEAN of the
24 | //   `scope = 'student_course'` `attainment_percent` rows for those CLOs (the
25 | //   scope at which CLO attainment is written), and `contributing_count` is
26 | //   the number of CLO rows averaged. (`derivation = "clo_rollup"`)
27 | //
28 | // 3. NONE - no measurable data:
29 | //   If neither a course-scope PLO row nor any CLO student_course-scope row
30 | //   exists, the PLO is returned with `attainment_percent = -1` (sentinel for
31 | //   "unmeasured") and `contributing_count = 0`. (`derivation = "none"`)
32 | //   Callers treat `-1` as a neutral/grey cell, distinct from a real 0%.
33 | //
34 | // The `contributors` array ALWAYS carries the CLO-level breakdown (every mapped
35 | // CLO that has a course-scope attainment row, with its course name) so the
36 | // drill-down view is meaningful regardless of which derivation produced the
37 | // headline number.
38 | //
39 | // All resolution is performed with a fixed, small number of batched queries
40 | // (no per-PLO / per-CLO round trips !' no N+1), institution-scoped throughout.
```

```

41 | // =====
42 |
43 | import { useQuery } from "@tanstack/react-query";
44 | import { supabase } from "@lib/supabase";
45 | import { queryKeys } from "@lib/queryKeys";
46 | import { useAuth } from "@hooks/useAuth";
47 |
48 | /** Sentinel returned in `attainment_percent` when a PLO has no measurable data. */
49 | export const PLO_ATTAINMENT_UNMEASURED = -1;
50 |
51 | /** How a PLO's headline attainment value was derived (Req 7.7). */
52 | export type PLODerivation = "program" | "clo_rollup" | "none";
53 |
54 | /** A single contributing CLO/course row shown in the drill-down (Req 7.3). */
55 | export interface PLOContributor {
56 |   clo_id: string;
57 |   clo_title: string;
58 |   course_id: string | null;
59 |   course_name: string | null;
60 |   attainment_percent: number;
61 | }
62 |
63 | /** One PLO cell in the heatmap. */
64 | export interface AdminPLOHeatmapRow {
65 |   plo_id: string;
66 |   plo_title: string;
67 |   program_id: string | null;
68 |   attainment_percent: number;
69 |   contributing_count: number;
70 |   derivation: PLODerivation;
71 |   contributors: PLOContributor[];
72 | }
73 |
74 | const mean = (values: number[]): number =>
75 |   values.length === 0
76 |     ? 0
77 |     : values.reduce((sum, v) => sum + v, 0) / values.length;
78 |
79 | const uniq = (values: string[]): string[] => Array.from(new Set(values));
80 |
81 | /**
82 |  * Derives PLO attainment for the admin's institution from `outcome_attainment`
83 |  * joined (logically) to `learning_outcomes` where `type = 'PLO'`.
84 |  *
85 |  * @param programId Optional program filter; when provided only PLOs belonging to
86 |  *   that program are returned.
87 |  */
88 | export const useAdminPLOHeatmap = (
89 |   programId?: string,
90 |   options?: { enabled?: boolean }
91 | ) => {
92 |   const { institutionId } = useAuth();
93 |
94 |   return useQuery({
95 |     queryKey: queryKeys.adminPLOHeatmap.list({ institutionId, programId }),
96 |     // PERF (dashboard-and-ux-performance): still institution-gated, but the admin
97 |     // dashboard's critical-first sequencing can additionally defer this 6-query
98 |     // heatmap until the KPI aggregate settles. Defaults to enabled for any other
99 |     // caller — fully backward-compatible.
100 |    enabled: !!institutionId && (options?.enabled ?? true),
101 |    queryFn: async (): Promise<AdminPLOHeatmapRow[]> => {
102 |      // 1. PLOs in this institution (explicit column select — no `*`, no TS2589).
103 |      let ploQuery = supabase
104 |        .from("learning_outcomes")
105 |        .select("id, title, program_id, sort_order")
106 |        .eq("institution_id", institutionId)
107 |        .eq("type", "PLO")
108 |        .order("sort_order", { ascending: true })
109 |        .order("title", { ascending: true });
110 |      if (programId) ploQuery = ploQuery.eq("program_id", programId);
111 |
112 |      const { data: plos, error: ploError } = await ploQuery;
113 |      if (ploError) throw ploError;
114 |      if (!plos || plos.length === 0) return [];
115 |
116 |      const ploIds = plos.map((p) => p.id);
117 |
118 |      // 2. PLO attainment for these PLOs (one batched query).
119 |      //   The attainment rollup trigger writes PLO rows at scope='course'
120 |      //   (one row per PLO per course that assesses it), so that is the scope
121 |      //   we read and average here. (Verified against live data 2026-06:
122 |      //   PLO rows exist only at scope='course', not 'program'.)
123 |      const { data: programRows, error: programError } = await supabase
124 |        .from("outcome_attainment")
125 |        .select("outcome_id, attainment_percent, course_id")
126 |        .eq("scope", "course")
127 |        .in("outcome_id", ploIds);
128 |      if (programError) throw programError;
129 |
130 |      const programByPlo = new Map<string, number[]>();
131 |      for (const row of programRows ?? []) {
132 |        const list = programByPlo.get(row.outcome_id) ?? [];
133 |        list.push(row.attainment_percent);
134 |        programByPlo.set(row.outcome_id, list);
135 |      }
136 |
137 |      // 3. CLO !' PLO mappings for ALL PLOs (one batched query). Used both for the
138 |      //   roll-up fallback and to build the drill-down contributor breakdown.
139 |      const { data: mappings, error: mapError } = await supabase
140 |        .from("outcome_mappings")
141 |        .select("source_outcome_id, target_outcome_id")
142 |        .in("target_outcome_id", ploIds);
143 |      if (mapError) throw mapError;
144 |

```

```

145 | const cloIdsByPlo = new Map<string, string[]>();
146 | for (const m of mappings ?? []) {
147 |   const list = cloIdsByPlo.get(m.target_outcome_id) ?? [];
148 |   list.push(m.source_outcome_id);
149 |   cloIdsByPlo.set(m.target_outcome_id, list);
150 | }
151 | const allCloIds = uniq((mappings ?? []).map((m) => m.source_outcome_id));
152 |
153 | // 4. CLO titles + their course (one batched query), 5. course-scope CLO
154 | // attainment (one batched query). Skipped entirely if no mapped CLOs.
155 | const cloTitleById = new Map<string, string>();
156 | const cloCourseById = new Map<string, string | null>();
157 | const cloAttainmentById = new Map<string, number>();
158 | let courseIds: string[] = [];
159 |
160 | if (allCloIds.length > 0) {
161 |   const { data: cloOutcomes, error: cloError } = await supabase
162 |     .from("learning_outcomes")
163 |     .select("id, title, course_id")
164 |     .in("id", allCloIds);
165 |   if (cloError) throw cloError;
166 |   for (const clo of cloOutcomes ?? []) {
167 |     cloTitleById.set(clo.id, clo.title);
168 |     cloCourseById.set(clo.id, clo.course_id);
169 |   }
170 |
171 |   const { data: cloAttRows, error: cloAttError } = await supabase
172 |     .from("outcome_attainment")
173 |     .select("outcome_id, attainment_percent")
174 |     .eq("scope", "student_course")
175 |     .in("outcome_id", allCloIds);
176 |   if (cloAttError) throw cloAttError;
177 |   // If a CLO has multiple course-scope rows, average them per CLO.
178 |   const cloAttLists = new Map<string, number[]>();
179 |   for (const row of cloAttRows ?? []) {
180 |     const list = cloAttLists.get(row.outcome_id) ?? [];
181 |     list.push(row.attainment_percent);
182 |     cloAttLists.set(row.outcome_id, list);
183 |   }
184 |   for (const [cloId, list] of cloAttLists) {
185 |     cloAttainmentById.set(cloId, mean(list));
186 |   }
187 |
188 |   courseIds = uniq(
189 |     (cloOutcomes ?? []).
190 |       .map((c) => c.course_id)
191 |       .filter((id): id is string => !!id)
192 |   );
193 | }
194 |
195 | // 6. Course names for the drill-down (one batched query).
196 | const courseNameById = new Map<string, string>();
197 | if (courseIds.length > 0) {
198 |   const { data: courses, error: courseError } = await supabase
199 |     .from("courses")
200 |     .select("id, name")
201 |     .in("id", courseIds);
202 |   if (courseError) throw courseError;
203 |   for (const c of courses ?? []) courseNameById.set(c.id, c.name);
204 | }
205 |
206 | // 7. Assemble one row per PLO using the documented derivation.
207 | return plos.map((plo): AdminPLOHeatmapRow => {
208 |   const mappedCloIds = cloIdsByPlo.get(plo.id) ?? [];
209 |
210 |   // Contributor breakdown – every mapped CLO that has a course-scope value.
211 |   const contributors: PLOContributor[] = mappedCloIds
212 |     .filter((cloId) => cloAttainmentById.has(cloId))
213 |     .map((cloId) => {
214 |       const courseId = cloCourseById.get(cloId) ?? null;
215 |       return {
216 |         clo_id: cloId,
217 |         clo_title: cloTitleById.get(cloId) ?? cloId,
218 |         course_id: courseId,
219 |         course_name: courseId
220 |           ? courseNameById.get(courseId) ?? null
221 |           : null,
222 |         attainment_percent: cloAttainmentById.get(cloId) ?? 0,
223 |       };
224 |     });
225 |
226 |   const programValues = programByPlo.get(plo.id) ?? [];
227 |   if (programValues.length > 0) {
228 |     return {
229 |       plo_id: plo.id,
230 |       plo_title: plo.title,
231 |       program_id: plo.program_id,
232 |       attainment_percent: mean(programValues),
233 |       contributing_count: programValues.length,
234 |       derivation: "program",
235 |       contributors,
236 |     };
237 |   }
238 |
239 |   if (contributors.length > 0) {
240 |     return {
241 |       plo_id: plo.id,
242 |       plo_title: plo.title,
243 |       program_id: plo.program_id,
244 |       attainment_percent: mean(
245 |         contributors.map((c) => c.attainment_percent)
246 |       ),
247 |       contributing_count: contributors.length,
248 |       derivation: "clo_rollup",

```

```

249 |         contributors,
250 |     };
251 | }
252 |
253 | return {
254 |     plo_id: plo.id,
255 |     plo_title: plo.title,
256 |     program_id: plo.program_id,
257 |     attainment_percent: PLO_ATTAINMENT_UNMEASURED,
258 |     contributing_count: 0,
259 |     derivation: "none",
260 |     contributors,
261 | };
262 | });
263 | },
264 | });
265 | };
266 |

```

## FILE: src/hooks/useAdminSecurity.ts

TYPE: .ts | LINES: 1-84 | SHA256: fbc627995e6c0cab7b4a9c375ef955bbbfcfb29552178ff5d54fda7294deb946d

```

1 | // =====
2 | // useAdminSecurity – read hook for the Admin Security console (net-new UI)
3 | // =====
4 | //
5 | // Surfaces three existing, admin-scoped tables that previously had no UI:
6 | //   • blocked_ips      – IPs blocked by the login rate-limiter
7 | //   • login_attempts   – per-email failed-login / lockout tracking
8 | //   • rate_limit_events – recent rate-limit / abuse signals
9 | //
10 | // No new backend logic – a plain read over existing tables (catalog §7). RLS on
11 | // these tables enforces admin-only access server-side; this hook just queries.
12 | // =====
13 |
14 | import { useQuery } from "@tanstack/react-query";
15 | import { supabase } from "@lib/supabase";
16 |
17 | export interface BlockedIpRow {
18 |   ip_address: string;
19 |   blocked_until: string;
20 |   reason: string;
21 |   blocked_by: string | null;
22 |   created_at: string;
23 | }
24 |
25 | export interface LoginAttemptRow {
26 |   email: string;
27 |   attempt_count: number;
28 |   locked_until: string | null;
29 |   updated_at: string;
30 | }
31 |
32 | export interface RateLimitEventRow {
33 |   id: number;
34 |   ip_address: string;
35 |   event_type: string;
36 |   user_id: string | null;
37 |   metadata: unknown;
38 |   occurred_at: string;
39 | }
40 |
41 | export interface AdminSecurityData {
42 |   blockedIps: BlockedIpRow[];
43 |   lockedAccounts: LoginAttemptRow[];
44 |   rateLimitEvents: RateLimitEventRow[];
45 | }
46 |
47 | /** Max recent rate-limit events to pull for the console. */
48 | export const RATE_LIMIT_EVENT_LIMIT = 100;
49 |
50 | export const useAdminSecurity = () =>
51 |   useQuery<AdminSecurityData>({
52 |     queryKey: ["admin", "security", "overview"],
53 |     queryFn: async () => {
54 |       const [blocked, attempts, events] = await Promise.all([
55 |         supabase
56 |           .from("blocked_ips")
57 |           .select("")
58 |           .order("created_at", { ascending: false })
59 |           .returns<BlockedIpRow[]>(),
60 |         supabase
61 |           .from("login_attempts")
62 |           .select("")
63 |           .order("updated_at", { ascending: false })
64 |           .returns<LoginAttemptRow[]>(),
65 |         supabase
66 |           .from("rate_limit_events")
67 |           .select("")
68 |           .order("occurred_at", { ascending: false })
69 |           .limit(RATE_LIMIT_EVENT_LIMIT)
70 |           .returns<RateLimitEventRow[]>(),
71 |       ]);
72 |
73 |       if (blocked.error) throw blocked.error;
74 |       if (attempts.error) throw attempts.error;
75 |       if (events.error) throw events.error;
76 |
77 |       return {
78 |         blockedIps: blocked.data ?? [],
79 |         lockedAccounts: attempts.data ?? [],
80 |         rateLimitEvents: events.data ?? [],
81 |       };
82 |     },

```

```
83 | });
84 |
```

FILE: src/hooks/useAIFeedbackDraft.ts

TYPE: .ts | LINES: 1-84 | SHA256: cf5e5c0fef72a8035ccabe2a77bd01616ca16c1fbd14a3c3331358323044ac65

```
// =====  
2 // useAIFeedbackDraft - TanStack Query hooks for AI feedback draft generation  
3 // Validates: Requirements 48.4, 48.6  
4 // =====  
5  
6 import { useMutation, useQueryClient } from "@tanstack/react-query";  
7 import { supabase } from "@/lib/supabase";  
8 import { queryKeys } from "@/lib/queryKeys";  
9  
10 // % % % Types % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
11  
12 export interface GenerateDraftInput {  
13   submission_id: string;  
14   rubric_id: string;  
15   rubric_selections: Array<{ criterion_id: string; level_index: number }>;  
16   student_id: string;  
17   clo_id: string;  
18 }  
19  
20 export interface CriterionDraft {  
21   criterion_id: string;  
22   criterion_title: string;  
23   draft_comment: string;  
24 }  
25  
26 export interface GenerateDraftResponse {  
27   criterion_drafts: CriterionDraft[];  
28   overall_draft: string;  
29 }  
30  
31 export interface LogDraftActionInput {  
32   student_id: string;  
33   suggestion_type: "feedback_draft";  
34   suggestion_text: string;  
35   suggestion_data: Record<string, string | undefined>;  
36   feedback: "thumbs_up" | "thumbs_down" | null;  
37   validated_outcome: "accepted" | "edited" | "rejected" | null;  
38 }  
39  
40 // % % % useGenerateFeedbackDraft % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
41  
42 export const useGenerateFeedbackDraft = () => {  
43   return useMutation({  
44     mutationFn: async (  
45       input: GenerateDraftInput  
46     ): Promise<GenerateDraftResponse> => {  
47       const { data, error } = await supabase.functions.invoke(  
48         "ai-feedback-draft",  
49         {  
50           body: input,  
51         },  
52       );  
53       if (error) throw error;  
54       if (data?.error) throw new Error(data.error);  
55  
56       return {  
57         criterion_drafts: data.criterion_drafts ?? [],  
58         overall_draft: data.overall_draft ?? "",  
59       };  
60     },  
61   });  
62 };  
63  
64 // % % % useLogDraftAction % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
65  
66 export const useLogDraftAction = () => {  
67   const queryClient = useQueryClient();  
68  
69   return useMutation({  
70     mutationFn: async (input: LogDraftActionInput): Promise<void> => {  
71       const { error } = await supabase.from("ai_feedback").insert(input);  
72       if (error) throw error;  
73     },  
74     onSuccess: () => {  
75       queryClient.invalidateQueries({ queryKey: queryKeys.aiFeedback.lists() });  
76       queryClient.invalidateQueries({  
77         queryKey: queryKeys.feedbackDrafts.lists(),  
78       });  
79     },  
80   });  
81 };  
82 };  
83 };  
84
```

FILE: src/hooks/useAIGovernance.ts

```
TYPE: .ts | LINES: 1-89 | SHA256: 5f2bb8326717c489347835520ece244981c8e9fa2a49ec58daa206d0a702e0bc
```

```

1 | import { useQuery } from "@tanstack/react-query";
2 |
3 | import { supabase } from "@lib/supabase";
4 | import { queryKeys } from "@lib/queryKeys";
5 | import type { Database } from "@types/database";
6 |
7 | type TutorLog = Database["public"]["Tables"]["tutor_llm_logs"]["Row"];
8 |
9 | export interface GovernanceModelUsage {
10 |   model: string;

```



[illegible]

FILE: `src/hooks/useAISuggestions.ts`

TYPE: .ts | LINES: 1-83 | SHA256: e09aa3887746c0b2b36ea6b811e95c265a668c7841d8b25f96aaf0fcd28c2145

```

1 // =====
2 // useAISuggestions – TanStack Query hooks for AI module suggestions
3 // Validates: Requirements 46.5, 46.6
4 // =====
5
6 import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
7 import { supabase } from "@lib/supabase";
8 import { queryKeys } from "@lib/queryKeys";
9
10 // % % % Types % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
11
12 export interface SuggestionData {
13   weak_clo_id: string;
14   weak_clo_title: string;
15   weak_clo_attainment: number;
16   prerequisite_clo_id: string | null;
17   prerequisite_clo_title: string | null;
18   social_proof_text: string | null;
19 }
20
21 export interface AISuggestion {
22   id: string;
23   student_id: string;
24   suggestion_type: string;
25   suggestion_text: string;
26   suggestion_data: SuggestionData;
27   feedback: "thumbs_up" | "thumbs_down" | null;
28   validated_outcome: string | null;
29   created_at: string;
30 }
31
32 export interface SubmitFeedbackInput {
33   feedbackId: string;
34   feedback: "thumbs up" | "thumbs down";

```

```

35 }
36 // % % % useAISuggestions % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % ?
37
38 export const useAISuggestions = (studentId: string | undefined) => {
39   return useQuery({
40     queryKey: queryKeys.aiSuggestions.list({ studentId }),
41     queryFn: async (): Promise<AISuggestion[]> => {
42       if (!studentId) return [];
43
44       const { data, error } = await supabase
45         .from("ai_feedback")
46         .select(
47           "id, student_id, suggestion_type, suggestion_text, suggestion_data, feedback, validated_outcome, cre
48         )
49         .eq("student_id", studentId)
50         .eq("suggestion_type", "module_suggestion")
51         .order("created_at", { ascending: false });
52
53       if (error) throw error;
54       return (data ?? []) as unknown as AISuggestion[];
55     },
56     enabled: !!studentId,
57   });
58 };
59
60 // % % % useSubmitAIFeedback % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
61
62 export const useSubmitAIFeedback = () => {
63   const queryClient = useQueryClient();
64
65   return useMutation({
66     mutationFn: async ({
67       feedbackId,
68       feedback,
69     }: SubmitFeedbackInput): Promise<void> => {
70       const { error } = await supabase
71         .from("ai_feedback")
72         .update({ feedback })
73         .eq("id", feedbackId);
74
75       if (error) throw error;
76     },
77     onSuccess: () => {
78       queryClient.invalidateQueries({ queryKey: queryKeys.aiSuggestions.all });
79     },
80   });
81 };
82
83

```

FILE: src/hooks/useAnnouncements.ts

```
TYPE: .ts | LINES: 1-440 | SHA256: 4b57c5a00b9aea32f90124b99a71efba50fa0a41bd85da6924a994a31fba27e1
```

[illegible]



[illegible]

```

156 |     },
157 |   });
158 | };
159 |
160 | export const useUpdateAnnouncement = () => {
161 |   const queryClient = useQueryClient();
162 |   return useMutation({
163 |     mutationFn: async (
164 |       input: UpdateAnnouncementInput & { performedBy: string }
165 |     ) => {
166 |       const { id, performedBy, ...updates } = input;
167 |       const { data, error } = await supabase
168 |         .from("announcements")
169 |         .update(updates)
170 |         .eq("id", id)
171 |         .select()
172 |         .single();
173 |       if (error) throw error;
174 |       await logAuditEvent({
175 |         action: "update",
176 |         entity_type: "announcement",
177 |         entity_id: id,
178 |         changes: updates,
179 |         performed_by: performedBy,
180 |       });
181 |       return castAnnouncement(data as unknown as Record<string, unknown>);
182 |     },
183 |     onSuccess: () => {
184 |       queryClient.invalidateQueries({
185 |         queryKey: queryKeys.announcements.lists(),
186 |       });
187 |     },
188 |   });
189 | };
190 |
191 | export const useDeleteAnnouncement = () => {
192 |   const queryClient = useQueryClient();
193 |   return useMutation({
194 |     mutationFn: async ({
195 |       id,
196 |       performedBy,
197 |     }: {
198 |       id: string;
199 |       performedBy: string;
200 |     }) => {
201 |       const { error } = await supabase
202 |         .from("announcements")
203 |         .delete()
204 |         .eq("id", id);
205 |       if (error) throw error;
206 |       await logAuditEvent({
207 |         action: "delete",
208 |         entity_type: "announcement",
209 |         entity_id: id,
210 |         changes: null,
211 |         performed_by: performedBy,
212 |       });
213 |     },
214 |     onSuccess: () => {
215 |       queryClient.invalidateQueries({
216 |         queryKey: queryKeys.announcements.lists(),
217 |       });
218 |     },
219 |   });
220 | };
221 |
222 | // % % % Student-facing: recent announcements across enrolled courses % % % % % % % % %
223 |
224 | export const useStudentAnnouncements = (
225 |   studentId: string | undefined,
226 |   limit = 5,
227 |   options?: { enabled?: boolean }
228 | ) => {
229 |   return useQuery({
230 |     queryKey: queryKeys.announcements.list({ studentId, limit }),
231 |     queryFn: async () => {
232 |       if (!studentId) return [];
233 |       // Get enrolled course IDs
234 |       const { data: enrollments, error: enrollErr } = await supabase
235 |         .from("student_courses")
236 |         .select("course_id")
237 |         .eq("student_id", studentId)
238 |         .eq("status", "active");
239 |       if (enrollErr) throw enrollErr;
240 |       const courseIds = (enrollments ?? []).map((e) => e.course_id);
241 |       if (courseIds.length === 0) return [];
242 |       const { data, error } = await supabase
243 |         .from("announcements")
244 |         .select(
245 |           "id, course_id, author_id, title, content, is_pinned, created_at, updated_at"
246 |         )
247 |         .in("course_id", courseIds)
248 |         .order("is_pinned", { ascending: false })
249 |         .order("created_at", { ascending: false })
250 |         .limit(limit);
251 |       if (error) throw error;
252 |       return (data ?? []).map((r) =>
253 |         castAnnouncement(r as unknown as Record<string, unknown>)
254 |       );
255 |     },
256 |     // Caller-overridable gate (backward-compatible): 1- and 2-arg callers pass
257 |     // no 'options', so 'options?.enabled ?? true' collapses to the prior
258 |     // 'enabled: !!studentId'. The StudentDashboard passes `{ enabled: false }`
259 |     // on the happy path so the aggregate-hydrated cache is used without this

```

[illegible]

```

364 | };
365 |
366 | /**
367 |  * Create a short-lived signed URL for downloading a private attachment.
368 |  * Reuses the shared getSignedUrl helper (the bucket is private – no public URL).
369 |  */
370 | export const getAnnouncementAttachmentUrl = (
371 |   storagePath: string
372 | ): Promise<string | null> => {
373 |   getSignedUrl(ANNOUNCEMENT_ATTACHMENT_BUCKET, storagePath);
374 | }
375 | /**
376 |  * Mark an announcement as read for the current student. Idempotent via the
377 |  * UNIQUE(announcement_id, student_id) constraint – repeated views upsert the
378 |  * same row instead of erroring or duplicating (Req 15.4).
379 |  */
380 | export const useMarkAnnouncementRead = () => {
381 |   const queryClient = useQueryClient();
382 |   return useMutation({
383 |     mutationFn: async ({
384 |       announcementId,
385 |       studentId,
386 |     }): {
387 |       announcementId: string;
388 |       studentId: string;
389 |     }: Promise<void> => {
390 |       const { error } = await supabase
391 |         .from("announcement_reads")
392 |         .upsert(
393 |           { announcement_id: announcementId, student_id: studentId },
394 |           { onConflict: "announcement_id,student_id" }
395 |         );
396 |       if (error) throw error;
397 |       // Audit trail (domain-knowledge.md – audit logs are append-only). A read
398 |       // receipt is idempotent (one row per student+announcement via the upsert
399 |       // conflict target), so this is low-volume. The student is the actor.
400 |       await logAuditEvent({
401 |         action: "read",
402 |         entity_type: "announcement_read",
403 |         entity_id: announcementId,
404 |         changes: { announcement_id: announcementId, student_id: studentId },
405 |         performed_by: studentId,
406 |       });
407 |     },
408 |     onSuccess: (_data, variables) => {
409 |       queryClient.invalidateQueries({
410 |         queryKey: queryKeys.announcementReads.list({
411 |           announcementId: variables.announcementId,
412 |         }),
413 |       });
414 |     },
415 |   });
416 | };
417 |
418 | /**
419 |  * Teacher-facing: read receipts for an announcement the caller authored.
420 |  * RLS limits this to the authoring teacher.
421 |  */
422 | export const useAnnouncementReadReceipts = (
423 |   announcementId: string | undefined
424 | ) => {
425 |   return useQuery({
426 |     queryKey: queryKeys.announcementReads.list({ announcementId }),
427 |     queryFn: async (): Promise<AnnouncementRead[]> => {
428 |       if (!announcementId) return [];
429 |       const { data, error } = await supabase
430 |         .from("announcement_reads")
431 |         .select("id, announcement_id, student_id, read_at")
432 |         .eq("announcement_id", announcementId)
433 |         .order("read_at", { ascending: false });
434 |       if (error) throw error;
435 |       return data ?? [];
436 |     },
437 |     enabled: !!announcementId,
438 |   });
439 | };
440 |

```

## FILE: src/hooks/useApproachableWording.ts

TYPE: .ts | LINES: 1-42 | SHA256: 20e199b2f8d540ad0f70ae4977dfff3240e06dd5c07cc9d073bf013448eb60352

```

1 | // =====
2 | // useApproachableWording – younger-student wording with cross-language fallback
3 | // =====
4 | //
5 | // The younger-student-friendly wording (R22) must display even when the active
6 | // language lacks a key but the other supported language has it (R22.6). This
7 | // hook returns a translator that applies that fallback policy via
8 | // `resolveLocalizedOrFail`, and re-renders on language change (through
9 | // `useTranslation`) so the wording stays correct after a runtime switch.
10 |
11 | import { useTranslation } from "react-i18next";
12 | import { resolveLocalizedOrFail } from "@lib/localization";
13 |
14 | export type ApproachableTranslator = (
15 |   key: string,
16 |   params?: Record<string, unknown>
17 | ) => string;
18 |
19 | /**
20 |  * Returns a translator for younger-student wording in the given namespace.
21 |  *
22 |  * The translator resolves the active language first and falls back to the other
23 |  * supported language when only that one has the key (R22.6). If neither has it,

```

FILE: src/hooks/useAssessmentIntro.ts

FILE: src/hooks/useAssignments.ts

[illegible]

```

24 |   clo_weights: Array<{ clo_id: string; weight: number }>;
25 |   late_window_hours: number;
26 |   prerequisites: Array<{ clo_id: string; required_attainment: number }> | null;
27 |   created_at: string;
28 | }
29 |
30 | export interface AssignmentWithRelations extends Assignment {
31 |   rubrics: { title: string } | null;
32 |   courses: { name: string } | null;
33 | }
34 |
35 | // % % % useAssignments - list assignments, optionally filtered by course_id % % % %
36 |
37 | export const useAssignments = (
38 |   courseId?: string,
39 |   pagination?: { page?: number; pageSize?: number }
40 | ) => {
41 |   const { page, pageSize, from, to } = getPaginationRange(
42 |     pagination?.page,
43 |     pagination?.pageSize
44 |   );
45 |
46 |   return useQuery({
47 |     queryKey: queryKeys.assignments.list({ courseId, page, pageSize }),
48 |     placeholderData: keepPreviousData,
49 |     queryFn: async (): Promise<PaginatedResult<AssignmentWithRelations>> => {
50 |       let query = supabase
51 |         .from("assignments")
52 |         .select("*, rubrics(title), courses!assignments_course_id_fkey(name)", {
53 |           count: "exact",
54 |         })
55 |         .order("due_date", { ascending: true })
56 |         .range(from, to);
57 |
58 |       if (courseId) {
59 |         query = query.eq("course_id", courseId);
60 |       }
61 |
62 |       const { data, error, count } = await query;
63 |
64 |       if (error) throw error;
65 |       return {
66 |         data: (data ?? []) as unknown as AssignmentWithRelations[],
67 |         count: count ?? 0,
68 |         page,
69 |         pageSize,
70 |       };
71 |     },
72 |   });
73 | };
74 |
75 | // % % % useAssignment - single assignment detail with rubric join % % % % % % % % % %
76 |
77 | export const useAssignment = (id?: string) => {
78 |   return useQuery({
79 |     queryKey: queryKeys.assignments.detail(id ?? ""),
80 |     queryFn: async (): Promise<AssignmentWithRelations | null> => {
81 |       const { data, error } = await supabase
82 |         .from("assignments")
83 |         .select("*, rubrics(title)")
84 |         .eq("id", id!)
85 |         .maybeSingle();
86 |
87 |       if (error) throw error;
88 |       return data as AssignmentWithRelations | null;
89 |     },
90 |     enabled: !!id,
91 |   });
92 | };
93 |
94 | // % % % useCreateAssignment - insert assignment % % % % % % % % % % % % % % % % % % % % % %
95 |
96 | export const useCreateAssignment = () => {
97 |   const queryClient = useQueryClient();
98 |   const { user } = useAuth();
99 |
100 |   return useMutation({
101 |     mutationFn: async (data: CreateAssignmentFormData): Promise<Assignment> => {
102 |       // NOTE: tutor_autonomy_level column exists in DB but database.ts types have not been
103 |       // regenerated yet. Using type assertion until `scripts/regen-types.ps1` is run.
104 |       // eslint-disable-next-line @typescript-eslint/no-explicit-any
105 |       const { data: result, error } = await (supabase as any)
106 |         .from("assignments")
107 |         .insert(data)
108 |         .select()
109 |         .single();
110 |
111 |       if (error) throw error;
112 |
113 |       const assignment = result as unknown as Assignment;
114 |
115 |       await logAuditEvent({
116 |         action: "create",
117 |         entity_type: "assignment",
118 |         entity_id: assignment.id,
119 |         changes: data,
120 |         performed_by: user?.id ?? "unknown",
121 |       });
122 |
123 |       return assignment;
124 |     },
125 |     onSuccess: () => {
126 |       queryClient.invalidateQueries({
127 |         queryKey: queryKeys.assignments.lists(),

```

[illegible]

FILE: src/hooks/useAtRiskPredictions.ts

TYPE: .ts | LINES: 1-145 | SHA256: 06678f252558e5b513a1b5d11d00ea74ac5ee5aa7132c73f4c8f40382347d36a

[illegible]

[illegible]



```

128     }) => {
129         const { error } = await supabase.from("notifications").insert({
130             user_id: studentId,
131             type: "nudge",
132             title: "At-Risk Alert: Your teacher wants to help",
133             body: message,
134             is_read: false,
135         });
136         if (error) throw error;
137     },
138     onSuccess: () => {
139         queryClient.invalidateQueries({
140             queryKey: queryKeys.atRiskPredictions.lists(),
141         });
142     },
143 });
144 };
145

```

FILE: src/hooks/useAttendance.ts

```
TYPE: .ts | LINES: 1-1057 | SHA256: 774e2340960417b13f77d47962886e42be9f887855c6bdeece9ed0fc430053d5
```

[illegible]

[illegible]

[illegible]

[illegible]

```

395 * course: course_sections !' class_sessions !' attendance_records, i.e. 1 + 3×N
396 * round-trips) into a fixed set of bounded queries regardless of course count:
397 * 1) active enrollments (the complete course list, incl. zero-session courses),
398 * 2) all sections for those courses % run in parallel
399 * + this student's attendance records % (records filter by student_id only,
400 * using the idx_attendance_student index – no large session_id `in()`),
401 * 3) all sessions for those sections.
402 * This was the #2 DB-time consumer (143 calls / 4.77 s max in pg_stat_statements);
403 * the new path is O(1) round-trips. RLS is unchanged (same tables, same caller).
404 */
405 export const useStudentAttendance = (
406   studentId: string | undefined,
407   options?: { enabled?: boolean }
408 ) => {
409   return useQuery({
410     queryKey: queryKeys.attendanceRecords.list({
411       studentId,
412       type: "student_courses",
413     }),
414     queryFn: async (): Promise<StudentCourseAttendance[]> => {
415       if (!studentId) return [];
416
417       // 1) Active enrollments – the complete course list (incl. zero-session
418       // courses, which render at 100% / 0 sessions, preserving prior behavior).
419       const { data: enrollments, error: enrErr } = await supabase
420         .from("student_courses")
421         .select("course_id, courses:course_id(name)")
422         .eq("student_id", studentId)
423         .eq("status", "active");
424       if (enrErr) throw enrErr;
425       if (!enrollments || enrollments.length === 0) return [];
426
427       const courses: StudentAttendanceCourse[] = enrollments.map((e) => ({
428         courseId: e.course_id,
429         courseName:
430           (e.courses as unknown as { name: string } | null)?.name ?? "Unknown",
431       }));
432       const courseIds = courses.map((c) => c.courseId);
433
434       // 2) Sections for those courses + this student's attendance records.
435       // Independent, so fetch in parallel.
436       const [sectionsRes, recordsRes] = await Promise.all([
437         supabase
438           .from("course_sections")
439           .select("id, course_id")
440           .in("course_id", courseIds),
441         supabase
442           .from("attendance_records")
443           .select("status, session_id")
444           .eq("student_id", studentId),
445       ]);
446       if (sectionsRes.error) throw sectionsRes.error;
447       if (recordsRes.error) throw recordsRes.error;
448
449       const sections = (sectionsRes.data ?? []) as {
450         id: string;
451         course_id: string;
452       }[];
453
454       // 3) Sessions for those sections (needs the section ids from step 2).
455       let sessions: { id: string; section_id: string }[] = [];
456       const sectionIds = sections.map((s) => s.id);
457       if (sectionIds.length > 0) {
458         const { data: sessionData, error: sessErr } = await supabase
459           .from("class_sessions")
460           .select("id, section_id")
461           .in("section_id", sectionIds);
462         if (sessErr) throw sessErr;
463         sessions = (sessionData ?? []) as { id: string; section_id: string }[];
464       }
465
466       const records = (recordsRes.data ?? []) as {
467         status: string;
468         session_id: string;
469       }[];
470
471       return aggregateStudentAttendance(courses, sections, sessions, records);
472     },
473     // Backward-compatible: callers that omit `options` keep the prior
474     // `enabled: !!studentId` behavior. The optional `enabled` lets callers (e.g.
475     // StudentDashboard, where the aggregate RPC already returns attendance) gate
476     // this hook to a fallback-only fetch without changing any other call site.
477     enabled: !!studentId && (options?.enabled ?? true),
478     staleTime: 60_000,
479   });
480 };
481
482 // % % % Parent-facing: per-course attendance summary for a linked child % % % % %
483
484 export interface ParentAttendanceSummary {
485   course_id: string;
486   course_name: string;
487   course_code: string;
488   total_sessions: number;
489   present: number;
490   late: number;
491   absent: number;
492   attendance_rate: number;
493 }
494
495 interface ParentEnrolledCourse {
496   course_id: string;
497   course_name: string;
498   course_code: string;

```

```

499 | }
500 |
501 | interface ParentAttendanceRecord {
502 |   course_id: string;
503 |   status: string;
504 | }
505 |
506 | /**
507 |  * Pure aggregation of a child's attendance records into per-course summaries.
508 |  * Seeds one entry per ENROLLED course (so courses with zero records still
509 |  * render at 0%), then counts present/late/absent and derives the rate.
510 |  *
511 |  * Extracted + exported so the parity test can prove the consolidated
512 |  * (2-query) `useChildAttendance` returns byte-for-byte the same shape the prior
513 |  * 4-step waterfall produced. `total_sessions` intentionally counts the child's
514 |  * attendance RECORDS (any status, including excused) per course, matching the
515 |  * pre-existing behavior.
516 |  */
517 | export function aggregateParentAttendance(
518 |   courses: ParentEnrolledCourse[],
519 |   records: ParentAttendanceRecord[]
520 | ): ParentAttendanceSummary[] {
521 |   const summary = new Map<string, ParentAttendanceSummary>();
522 |   for (const c of courses) {
523 |     summary.set(c.course_id, {
524 |       course_id: c.course_id,
525 |       course_name: c.course_name,
526 |       course_code: c.course_code,
527 |       total_sessions: 0,
528 |       present: 0,
529 |       late: 0,
530 |       absent: 0,
531 |       attendance_rate: 0,
532 |     });
533 |   }
534 |
535 |   for (const r of records) {
536 |     const s = summary.get(r.course_id);
537 |     if (!s) continue;
538 |     s.total_sessions += 1;
539 |     if (r.status === "present") s.present += 1;
540 |     else if (r.status === "late") s.late += 1;
541 |     else if (r.status === "absent") s.absent += 1;
542 |   }
543 |
544 |   for (const s of summary.values()) {
545 |     s.attendance_rate =
546 |       s.total_sessions > 0
547 |       ? Math.round(((s.present + s.late) / s.total_sessions) * 100)
548 |       : 0;
549 |   }
550 |
551 |   return Array.from(summary.values());
552 | }
553 |
554 | /**
555 |  * Parent view: attendance summary per course for one linked child.
556 |  *
557 |  * Consolidated from the prior 4-step client waterfall (enrollments !'
558 |  * course_sections !' class_sessions !' attendance_records with a large
559 |  * `.in(sessionIds)`), which could exceed URL/`in`-list limits and stall. Now
560 |  * two queries: (1) the child's active enrollments (the complete course list,
561 |  * incl. zero-record courses) and (2) a single JOINED attendance query filtered
562 |  * by `student_id` that walks `attendance_records !' class_sessions !'
563 |  * course_sections` to resolve each record's course – no unbounded `.in()`.
564 |  *
565 |  * RLS is preserved: both queries read the same tables under the caller's
566 |  * (parent's) row-level security, so only verified-linked children resolve.
567 |  */
568 | export const useChildAttendance = (studentId: string | undefined) => {
569 |   return useQuery({
570 |     queryKey: queryKeys.attendanceRecords.list({
571 |       studentId,
572 |       view: "parent-summary",
573 |     }),
574 |     queryFn: async (): Promise<ParentAttendanceSummary[]> => {
575 |       if (!studentId) return [];
576 |
577 |       // 1) Active enrollments – the complete course list (incl. zero-record).
578 |       const { data: enrollments, error: enrErr } = await supabase
579 |         .from("student_courses")
580 |         .select("course_id, courses!inner(id, name, code)")
581 |         .eq("student_id", studentId)
582 |         .eq("status", "active");
583 |       if (enrErr) throw enrErr;
584 |       if (!enrollments || enrollments.length === 0) return [];
585 |
586 |       const courses: ParentEnrolledCourse[] = enrollments.map((e) => {
587 |         const c = e.courses as unknown as {
588 |           id: string;
589 |           name: string;
590 |           code: string;
591 |         };
592 |         return { course_id: c.id, course_name: c.name, course_code: c.code };
593 |       });
594 |
595 |       // 2) One joined query for THIS child's attendance – resolves the course
596 |       // via the FK chain instead of a giant session-id `.in()` list.
597 |       const { data: rawRecords, error: recErr } = await supabase
598 |         .from("attendance_records")
599 |         .select(
600 |           "status, class_sessions!inner(course_sections!inner(course_id))"
601 |         )
602 |         .eq("student_id", studentId);

```

```

603 |         if (recErr) throw recErr;
604 |
605 |         const records: ParentAttendanceRecord[] = (rawRecords ?? []).map((r) => {
606 |             const courseId =
607 |                 (
608 |                     r.class_sessions as unknown as {
609 |                         course_sections: { course_id: string } | null;
610 |                     } | null
611 |                 )?.course_sections?.course_id ?? "";
612 |             return { course_id: courseId, status: r.status as string };
613 |         });
614 |
615 |         return aggregateParentAttendance(courses, records);
616 |     },
617 |     enabled: !!studentId,
618 |     staleTime: 60_000,
619 | });
620 | };
621 |
622 | // % % % Single Canonical Parent Attendance Overview Hook & Types % % % % % % % % % %
623 |
624 | export interface ParentAttendanceOverview {
625 |     child: {
626 |         id: string;
627 |         name: string;
628 |     };
629 |     period: {
630 |         dateFrom: string;
631 |         dateTo: string;
632 |         label: string;
633 |     };
634 |     totals: {
635 |         totalSessions: number;
636 |         present: number;
637 |         late: number;
638 |         absent: number;
639 |         excused: number;
640 |         attended: number;
641 |         attendanceRate: number;
642 |         punctualityRate: number;
643 |         absenceRate: number;
644 |     };
645 |     trend: Array<{
646 |         periodLabel: string;
647 |         periodStart: string;
648 |         periodEnd: string;
649 |         present: number;
650 |         late: number;
651 |         absent: number;
652 |         attendanceRate: number;
653 |     }>;
654 |     courses: Array<{
655 |         courseId: string;
656 |         code: string;
657 |         name: string;
658 |         present: number;
659 |         late: number;
660 |         absent: number;
661 |         excused: number;
662 |         totalSessions: number;
663 |         attendanceRate: number;
664 |         trend: "up" | "stable" | "down" | "insufficient_data";
665 |     }>;
666 |     recentExceptions: Array<{
667 |         attendanceRecordId: string;
668 |         sessionId: string;
669 |         courseId: string;
670 |         courseName: string;
671 |         sessionDate: string;
672 |         sessionType: string;
673 |         topic?: string;
674 |         status: "present" | "late" | "absent" | "excused";
675 |     }>;
676 |     attention?: {
677 |         courseId: string;
678 |         courseName: string;
679 |         absenceCount: number;
680 |         message: string;
681 |     };
682 | }
683 |
684 | export interface RawAttendanceRecordItem {
685 |     id: string;
686 |     session_id: string;
687 |     status: string;
688 |     created_at: string;
689 |     class_sessions: {
690 |         id: string;
691 |         session_date: string;
692 |         session_type: string;
693 |         topic?: string;
694 |         course_sections: {
695 |             id: string;
696 |             course_id: string;
697 |             courses: {
698 |                 id: string;
699 |                 name: string;
700 |                 code: string;
701 |             };
702 |         };
703 |     };
704 | }
705 |
706 | /**

```

```

707 | * Pure canonical aggregation function for Parent Attendance Overview.
708 | * Enforces explicit formulas:
709 | *   Attended = Present + Late
710 | *   Attendance Rate = Math.round(((Present + Late) / Total) * 100)
711 | *   Punctuality Rate = Attended > 0 ? Math.round((Present / Attended) * 100) : 100
712 | *   Absence Rate = Math.round((Absent / Total) * 100)
713 | */
714 | export function buildParentAttendanceOverview(
715 |   child: { id: string; name: string },
716 |   enrolledCourses: Array<{ courseId: string; code: string; name: string }>,
717 |   records: RawAttendanceRecordItem[],
718 |   dateFrom?: string,
719 |   dateTo?: string
720 | ): ParentAttendanceOverview {
721 |   // Filter by date range if provided
722 |   const filteredRecords = records.filter((r) => {
723 |     const d = r.class_sessions?.session_date;
724 |     if (!d) return false;
725 |     if (dateFrom && d < dateFrom) return false;
726 |     if (dateTo && d > dateTo) return false;
727 |     return true;
728 |   });
729 |
730 |   // Calculate totals
731 |   let present = 0;
732 |   let late = 0;
733 |   let absent = 0;
734 |   let excused = 0;
735 |
736 |   for (const r of filteredRecords) {
737 |     if (r.status === "present") present++;
738 |     else if (r.status === "late") late++;
739 |     else if (r.status === "absent") absent++;
740 |     else if (r.status === "excused") excused++;
741 |   }
742 |
743 |   const totalSessions = filteredRecords.length;
744 |   const attended = present + late;
745 |   const attendanceRate =
746 |     totalSessions > 0 ? Math.round((attended / totalSessions) * 100) : 100;
747 |   const punctualityRate =
748 |     attended > 0 ? Math.round((present / attended) * 100) : 100;
749 |   const absenceRate =
750 |     totalSessions > 0 ? Math.round((absent / totalSessions) * 100) : 0;
751 |
752 |   // Aggregate by Course
753 |   const courseMap = new Map<
754 |     string,
755 |     {
756 |       courseId: string;
757 |       code: string;
758 |       name: string;
759 |       present: number;
760 |       late: number;
761 |       absent: number;
762 |       excused: number;
763 |       totalSessions: number;
764 |     }
765 |   >();
766 |
767 |   for (const c of enrolledCourses) {
768 |     courseMap.set(c.courseId, {
769 |       courseId: c.courseId,
770 |       code: c.code,
771 |       name: c.name,
772 |       present: 0,
773 |       late: 0,
774 |       absent: 0,
775 |       excused: 0,
776 |       totalSessions: 0,
777 |     });
778 |   }
779 |
780 |   for (const r of filteredRecords) {
781 |     const course = r.class_sessions?.course_sections?.courses;
782 |     if (!course) continue;
783 |     let item = courseMap.get(course.id);
784 |     if (!item) {
785 |       item = {
786 |         courseId: course.id,
787 |         code: course.code,
788 |         name: course.name,
789 |         present: 0,
790 |         late: 0,
791 |         absent: 0,
792 |         excused: 0,
793 |         totalSessions: 0,
794 |       };
795 |       courseMap.set(course.id, item);
796 |     }
797 |     item.totalSessions++;
798 |     if (r.status === "present") item.present++;
799 |     else if (r.status === "late") item.late++;
800 |     else if (r.status === "absent") item.absent++;
801 |     else if (r.status === "excused") item.excused++;
802 |   }
803 |
804 |   const courseSummaries = Array.from(courseMap.values()).map((c) => {
805 |     const cAttended = c.present + c.late;
806 |     const rate =
807 |       c.totalSessions > 0
808 |       ? Math.round((cAttended / c.totalSessions) * 100)
809 |       : 100;
810 |     let trend: "up" | "stable" | "down" | "insufficient_data" = "stable";

```



```

811 |     if (c.totalSessions < 3) trend = "insufficient_data";
812 |     else if (rate >= 95) trend = "up";
813 |     else if (rate < 85) trend = "down";
814 |
815 |     return {
816 |       courseId: c.courseId,
817 |       code: c.code,
818 |       name: c.name,
819 |       present: c.present,
820 |       late: c.late,
821 |       absent: c.absent,
822 |       excused: c.excused,
823 |       totalSessions: c.totalSessions,
824 |       attendanceRate: rate,
825 |       trend,
826 |     };
827 |   });
828 |
829 |   // Recent Exceptions (sorted descending by date)
830 |   const recentExceptions: ParentAttendanceOverview["recentExceptions"] =
831 |     filteredRecords
832 |       .map((r) => ({
833 |         attendanceRecordId: r.id,
834 |         sessionId: r.class_sessions.id,
835 |         courseId: r.class_sessions.course_sections.courses.id,
836 |         courseName: r.class_sessions.course_sections.courses.name,
837 |         sessionId: r.class_sessions.session_date,
838 |         sessionType: r.class_sessions.session_type,
839 |         topic: r.class_sessions.topic,
840 |         status:
841 |           (r.status as "present" | "late" | "absent" | "excused") ?? "present",
842 |       }))
843 |       .sort((a, b) => b.sessionDate.localeCompare(a.sessionDate));
844 |
845 |   // Determine highest absence course for Attention card
846 |   let maxAbsenceCourse: (typeof courseSummaries)[0] | null = null;
847 |   for (const c of courseSummaries) {
848 |     if (c.absent > 0) {
849 |       if (!maxAbsenceCourse || c.absent > maxAbsenceCourse.absent) {
850 |         maxAbsenceCourse = c;
851 |       }
852 |     }
853 |   }
854 |
855 |   const attention = maxAbsenceCourse
856 |     ? {
857 |       courseId: maxAbsenceCourse.courseId,
858 |       courseName: maxAbsenceCourse.name,
859 |       absenceCount: maxAbsenceCourse.absent,
860 |       message: `${maxAbsenceCourse.name} has ${maxAbsenceCourse.absent} of the ${absent} missed sessions.`,
861 |     }
862 |     : undefined;
863 |
864 |   // Trend aggregation (weekly blocks)
865 |   const weekMap = new Map<
866 |     string,
867 |     {
868 |       label: string;
869 |       start: string;
870 |       end: string;
871 |       present: number;
872 |       late: number;
873 |       absent: number;
874 |     }
875 |   >();
876 |
877 |   for (const r of filteredRecords) {
878 |     const d = new Date(r.class_sessions.session_date);
879 |     if (isNaN(d.getTime())) continue;
880 |     // Get week label (e.g., W1, W2)
881 |     const weekNum = Math.ceil(d.getDate() / 7);
882 |     const weekKey = `${d.getFullYear()}-${M${d.getMonth() + 1}}-W${weekNum}`;
883 |     let w = weekMap.get(weekKey);
884 |     if (!w) {
885 |       w = {
886 |         label: `W${weekNum}`,
887 |         start: r.class_sessions.session_date,
888 |         end: r.class_sessions.session_date,
889 |         present: 0,
890 |         late: 0,
891 |         absent: 0,
892 |       };
893 |       weekMap.set(weekKey, w);
894 |     }
895 |     if (r.status === "present") w.present++;
896 |     else if (r.status === "late") w.late++;
897 |     else if (r.status === "absent") w.absent++;
898 |   }
899 |
900 |   const trend = Array.from(weekMap.values()).map((w) => {
901 |     const tot = w.present + w.late + w.absent;
902 |     const rate = tot > 0 ? Math.round(((w.present + w.late) / tot) * 100) : 100;
903 |     return {
904 |       periodLabel: w.label,
905 |       periodStart: w.start,
906 |       periodEnd: w.end,
907 |       present: w.present,
908 |       late: w.late,
909 |       absent: w.absent,
910 |       attendanceRate: rate,
911 |     };
912 |   });
913 |
914 |   // Calculate actual period label

```

```

915 | const dates = filteredRecords
916 |   .map((r) => r.class_sessions?.session_date)
917 |   .filter(Boolean)
918 |   .sort();
919 | const actualFrom = dateFrom || dates[0] || "2026-04-07";
920 | const actualTo = dateTo || dates[dates.length - 1] || "2026-05-18";
921 | const label = `${new Date(actualFrom).toLocaleDateString("en-US", {
922 |   month: "short",
923 |   day: "numeric",
924 | })} - ${new Date(actualTo).toLocaleDateString("en-US", {
925 |   month: "short",
926 |   day: "numeric",
927 |   year: "numeric",
928 | })}`;
929 |
930 | return {
931 |   child,
932 |   period: {
933 |     dateFrom: actualFrom,
934 |     dateTo: actualTo,
935 |     label,
936 |   },
937 |   totals: {
938 |     totalSessions,
939 |     present,
940 |     late,
941 |     absent,
942 |     excused,
943 |     attended,
944 |     attendanceRate,
945 |     punctualityRate,
946 |     absenceRate,
947 |   },
948 |   trend,
949 |   courses: courseSummaries,
950 |   recentExceptions,
951 |   attention,
952 | };
953 | }
954 |
955 | /**
956 |  * Single Canonical Parent Attendance Hook
957 |  * Consolidates Overview, Trend, Course Breakdown, Exceptions, and Right Rail data
958 |  * into a single unified query with strict authorization checks.
959 |  */
960 | export const useParentAttendanceOverview = (
961 |   studentId: string | undefined,
962 |   options?: { dateFrom?: string; dateTo?: string; courseId?: string }
963 | ) => {
964 |   return useQuery({
965 |     queryKey: queryKeys.attendanceRecords.list({
966 |       studentId,
967 |       dateFrom: options?.dateFrom,
968 |       dateTo: options?.dateTo,
969 |       courseId: options?.courseId,
970 |       view: "parent-canonical-overview",
971 |     }),
972 |     queryFn: async (): Promise<ParentAttendanceOverview> => {
973 |       if (!studentId) {
974 |         return buildParentAttendanceOverview({ id: "", name: "Child" }, [], []);
975 |       }
976 |
977 |       // 1) Fetch child profile info
978 |       const { data: profile, error: profErr } = await supabase
979 |         .from("profiles")
980 |         .select("id, full_name")
981 |         .eq("id", studentId)
982 |         .single();
983 |       if (profErr) throw profErr;
984 |
985 |       const childName = profile?.full_name ?? "Child";
986 |
987 |       // 2) Fetch active course enrollments for this child
988 |       const { data: enrollments, error: enrErr } = await supabase
989 |         .from("student_courses")
990 |         .select("course_id, courses!inner(id, name, code)")
991 |         .eq("student_id", studentId)
992 |         .eq("status", "active");
993 |       if (enrErr) throw enrErr;
994 |
995 |       const enrolledCourses = (enrollments ?? []).map((e) => {
996 |         const c = e.courses as unknown as {
997 |           id: string;
998 |           name: string;
999 |           code: string;
1000 |         };
1001 |         return { courseId: c.id, name: c.name, code: c.code };
1002 |       });
1003 |
1004 |       // 3) Fetch attendance records with joined session details
1005 |       let query = supabase
1006 |         .from("attendance_records")
1007 |         .select(
1008 |           id,
1009 |           session_id,
1010 |           status,
1011 |           created_at,
1012 |           class_sessions!inner(
1013 |             id,
1014 |             session_date,
1015 |             session_type,
1016 |             topic,
1017 |             course_sections!inner(

```

```

1019 |         id,
1020 |         course_id,
1021 |         courses!inner(
1022 |             id,
1023 |             name,
1024 |             code
1025 |         )
1026 |     )
1027 | )
1028 |
1029 | )
1030 | .eq("student_id", studentId);
1031 |
1032 | if (options?.courseId) {
1033 |     query = query.eq(
1034 |         "class_sessions.course_sections.course_id",
1035 |         options.courseId
1036 |     );
1037 | }
1038 |
1039 | const { data: recordsData, error: recErr } = await query;
1040 | if (recErr) throw recErr;
1041 |
1042 | const rawRecords = (recordsData ??
1043 |     []) as unknown as RawAttendanceRecordItem[];
1044 |
1045 | return buildParentAttendanceOverview(
1046 |     { id: studentId, name: childName },
1047 |     enrolledCourses,
1048 |     rawRecords,
1049 |     options?.dateFrom,
1050 |     options?.dateTo
1051 | );
1052 | },
1053 | enabled: !!studentId,
1054 | staleTime: 60_000,
1055 | });
1056 | };
1057 |

```

## FILE: src/hooks/useAuditLogs.ts

TYPE: .ts | LINES: 1-76 | SHA256: 70ef5d176411651913627c5e60c102511a1b1a2a21206cd8a28680ffba54e5e2

```

1 | import { useQuery, keepPreviousData } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import type { PaginatedResult } from "@types/pagination";
5 | import { getPaginationRange } from "@types/pagination";
6 | import { sanitizePostgresValue } from "@lib/sanitizeFilter";
7 |
8 | export interface AuditLogRecord {
9 |     id: string;
10 |     actor_id: string;
11 |     action: string;
12 |     target_type: string;
13 |     target_id: string;
14 |     diff: Record<string, unknown> | null;
15 |     ip_address: string | null;
16 |     created_at: string;
17 | }
18 |
19 | export interface AuditLogFilters {
20 |     search?: string;
21 |     action?: string;
22 |     entityType?: string;
23 |     page?: number;
24 |     pageSize?: number;
25 | }
26 |
27 | export const useAuditLogs = (filters: AuditLogFilters = {}) => {
28 |     const { page, pageSize, from, to } = getPaginationRange(
29 |         filters.page,
30 |         filters.pageSize
31 |     );
32 |
33 |     return useQuery({
34 |         queryKey: queryKeys.auditLogs.list({ ...filters, page, pageSize } as Record<
35 |             string,
36 |             unknown
37 |         >),
38 |         placeholderData: keepPreviousData,
39 |         queryFn: async (): Promise<PaginatedResult<AuditLogRecord>> => {
40 |             let query = supabase
41 |                 .from("audit_logs")
42 |                 .select(
43 |                     "id, actor_id, action, target_type, target_id, diff, ip_address, created_at",
44 |                     { count: "exact" }
45 |                 )
46 |                 .order("created_at", { ascending: false })
47 |                 .range(from, to);
48 |
49 |             if (filters.action) {
50 |                 query = query.eq("action", filters.action);
51 |             }
52 |
53 |             if (filters.entityType) {
54 |                 query = query.eq("target_type", filters.entityType);
55 |             }
56 |
57 |             if (filters.search) {
58 |                 const safe = sanitizePostgresValue(filters.search);
59 |                 query = query.or(
60 |                     `action.ilike.${safe}%,target_type.ilike.${safe}%,target_id.ilike.${safe}%`
61 |                 );

```

```

62 |     }
63 |
64 |     const { data, error, count } = await query;
65 |     if (error) throw error;
66 |     return {
67 |       data: (data ?? []) as AuditLogRecord[],
68 |       count: count ?? 0,
69 |       page,
70 |       pageSize,
71 |     };
72 |   },
73 |   staleTime: 30_000,
74 | });
75 | });
76 |

```

#### FILE: src/hooks/useAuth.ts

TYPE: .ts | LINES: 1-12 | SHA256: 9b101f19a4075832cf71052fc7e09018ec2027ecf0d68bf23ee039b2b41d3b

```

1 | import { useContext } from "react";
2 | import { AuthContext } from "@providers/AuthProvider";
3 | import type { AuthContextValue } from "@providers/AuthProvider";
4 |
5 | export const useAuth = (): AuthContextValue => {
6 |   const context = useContext(AuthContext);
7 |   if (context === undefined) {
8 |     throw new Error("useAuth must be used within an AuthProvider");
9 |   }
10 |   return context;
11 | };
12 |

```

#### FILE: src/hooks/useAvatarUpload.ts

TYPE: .ts | LINES: 1-153 | SHA256: 02ef7f2c1ced23177a90e7a7ff56eac0d7554cdb5db87a1b8c00ac8818de848e

```

1 | import { useCallback } from "react";
2 | import { useMutation, useQueryClient } from "@tanstack/react-query";
3 | import { supabase } from "@lib/supabase";
4 | import { useAuth } from "@hooks/useAuth";
5 | import { queryKeys } from "@lib/queryKeys";
6 | import { toast } from "sonner";
7 |
8 | interface AvatarUploadConfig {
9 |   file: File;
10 | }
11 |
12 | interface UseAvatarUploadReturn {
13 |   uploadAvatar: (config: AvatarUploadConfig) => Promise<void>;
14 |   deleteAvatar: () => Promise<void>;
15 |   isPending: boolean;
16 |   error: Error | null;
17 | }
18 |
19 | const validateAvatarFile = (file: File): { valid: boolean; error?: string } => {
20 |   const validTypes = ["image/png", "image/jpeg", "image/webp"];
21 |   const maxSize = 2 * 1024 * 1024; // 2 MB
22 |
23 |   if (!validTypes.includes(file.type)) {
24 |     return {
25 |       valid: false,
26 |       error: "Invalid file type. Please upload a PNG, JPG, or WebP image.",
27 |     };
28 |   }
29 |
30 |   if (file.size > maxSize) {
31 |     return {
32 |       valid: false,
33 |       error: "File is too large. Maximum size is 2 MB.",
34 |     };
35 |   }
36 |
37 |   return { valid: true };
38 | };
39 |
40 | const resizeImage = async (file: File): Promise<File> => {
41 |   try {
42 |     // eslint-disable-next-line @typescript-eslint/no-explicit-any
43 |     const imageCompression = (await import("browser-image-compression" as any))
44 |       .default;
45 |
46 |     const options = {
47 |       maxSizeMB: 0.15,
48 |       maxWidthOrHeight: 512,
49 |       useWebWorker: true,
50 |     };
51 |
52 |     return await imageCompression(file, options);
53 |   } catch {
54 |     return file;
55 |   }
56 | };
57 |
58 | export const useAvatarUpload = (): UseAvatarUploadReturn => {
59 |   const { user, refetchProfile } = useAuth();
60 |   const queryClient = useQueryClient();
61 |
62 |   const uploadMutation = useMutation({
63 |     mutationFn: async (config: AvatarUploadConfig) => {
64 |       if (!user) {
65 |         throw new Error("User not authenticated");
66 |       }
67 |
68 |       const { file } = config;
69 |

```

```

70 |     const validation = validateAvatarFile(file);
71 |     if (!validation.valid) {
72 |       throw new Error(validation.error);
73 |     }
74 |
75 |     const resizedFile = await resizeImage(file);
76 |     const path = `${user.id}/avatar`;
77 |
78 |     const { error: uploadError } = await supabase.storage
79 |       .from("avatars")
80 |       .upload(path, resizedFile, {
81 |         cacheControl: "public, max-age=31536000, immutable",
82 |         upsert: true,
83 |       });
84 |
85 |     if (uploadError) {
86 |       throw uploadError;
87 |     }
88 |
89 |     const { data } = supabase.storage.from("avatars").getPublicUrl(path);
90 |     const avatarUrl = `${data.publicUrl}?t=${Date.now()}`;
91 |
92 |     const { error: updateError } = await supabase
93 |       .from("profiles")
94 |       .update({ avatar_url: avatarUrl })
95 |       .eq("id", user.id);
96 |
97 |     if (updateError) {
98 |       throw updateError;
99 |     }
100 |   },
101 |   onSuccess: async () => {
102 |     await refetchProfile?();
103 |     queryClient.invalidateQueries({ queryKey: queryKeys.profiles.all });
104 |     queryClient.invalidateQueries({ queryKey: queryKeys.users.all });
105 |     toast.success("Avatar updated successfully");
106 |   },
107 |   onError: (error: Error) => {
108 |     console.error("Avatar upload error:", error);
109 |     toast.error(error.message || "Failed to upload avatar");
110 |   },
111 | });
112 |
113 | const deleteMutation = useMutation({
114 |   mutationFn: async () => {
115 |     if (!user) throw new Error("User not authenticated");
116 |
117 |     const { error: updateError } = await supabase
118 |       .from("profiles")
119 |       .update({ avatar_url: null })
120 |       .eq("id", user.id);
121 |
122 |     if (updateError) throw updateError;
123 |   },
124 |   onSuccess: async () => {
125 |     await refetchProfile?();
126 |     queryClient.invalidateQueries({ queryKey: queryKeys.profiles.all });
127 |     queryClient.invalidateQueries({ queryKey: queryKeys.users.all });
128 |     toast.success("Avatar removed");
129 |   },
130 |   onError: (error: Error) => {
131 |     toast.error(error.message || "Failed to remove avatar");
132 |   },
133 | });
134 |
135 | const uploadAvatar = useCallback(
136 |   async (config: AvatarUploadConfig) => {
137 |     return uploadMutation.mutateAsync(config);
138 |   },
139 |   [uploadMutation]
140 | );
141 |
142 | const deleteAvatar = useCallback(async () => {
143 |   return deleteMutation.mutateAsync();
144 | }, [deleteMutation]);
145 |
146 | return {
147 |   uploadAvatar,
148 |   deleteAvatar,
149 |   isPending: uploadMutation.isPending || deleteMutation.isPending,
150 |   error: uploadMutation.error || deleteMutation.error,
151 | };
152 | };
153 |

```

## FILE: src/hooks/useBadgeDefinitions.ts

TYPE: .ts | LINES: 1-133 | SHA256: 383f8e1d876d90ceb1e74fce032e52e2a19b320473c6b91183a4021e11d64a66

```

1 | import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
2 |
3 | import { supabase } from "@/lib/supabase";
4 | import { queryKeys } from "@/lib/queryKeys";
5 | import { logAuditEvent } from "@/lib/auditLogger";
6 | import { useAuth } from "@/hooks/useAuth";
7 | import type { Database, Json } from "@/types/database";
8 |
9 | export type BadgeDefinition =
10 |   Database["public"]["Tables"]["badge_definitions"]["Row"];
11 |
12 | export interface BadgeDefinitionInput {
13 |   badge_key: string;
14 |   name: string;
15 |   description: string;
16 |   emoji: string;

```

```

17 |   category: string | null;
18 |   tier_conditions: Json;
19 | }
20 |
21 | const invalidateDefinitions = (
22 |   queryClient: ReturnType<typeof useQueryClient>
23 | ) =>
24 |   queryClient.invalidateQueries({
25 |     queryKey: queryKeys.badgeDefinitions.lists(),
26 |   });
27 |
28 | export const useBadgeDefinitions = (institutionId?: string) =>
29 |   useQuery({
30 |     queryKey: queryKeys.badgeDefinitions.list({ institutionId }),
31 |     queryFn: async (): Promise<BadgeDefinition[]> => {
32 |       const { data, error } = await supabase
33 |         .from("badge_definitions")
34 |         .select("*")
35 |         .eq("institution_id", institutionId!)
36 |         .order("is_archived")
37 |         .order("name");
38 |
39 |       if (error) throw error;
40 |       return data ?? [];
41 |     },
42 |     enabled: !!institutionId,
43 |   });
44 |
45 | export const useCreateBadgeDefinition = () => {
46 |   const queryClient = useQueryClient();
47 |   const { user } = useAuth();
48 |
49 |   return useMutation({
50 |     mutationFn: async ({
51 |       institutionId,
52 |       input,
53 |     }: {
54 |       institutionId: string;
55 |       input: BadgeDefinitionInput;
56 |     }) => {
57 |       const { data, error } = await supabase
58 |         .from("badge_definitions")
59 |         .insert({ ...input, institution_id: institutionId })
60 |         .select()
61 |         .single();
62 |
63 |       if (error) throw error;
64 |       await logAuditEvent({
65 |         action: "create",
66 |         entity_type: "badge_definition",
67 |         entity_id: data.id,
68 |         changes: { ...input },
69 |         performed_by: user?.id ?? "",
70 |       });
71 |       return data;
72 |     },
73 |     onSuccess: () => invalidateDefinitions(queryClient),
74 |   });
75 | };
76 |
77 | export const useUpdateBadgeDefinition = () => {
78 |   const queryClient = useQueryClient();
79 |   const { user } = useAuth();
80 |
81 |   return useMutation({
82 |     mutationFn: async ({
83 |       id,
84 |       input,
85 |     }: {
86 |       id: string;
87 |       input: Partial<BadgeDefinitionInput> & { is_archived?: boolean };
88 |     }) => {
89 |       const { data, error } = await supabase
90 |         .from("badge_definitions")
91 |         .update(input)
92 |         .eq("id", id)
93 |         .select()
94 |         .single();
95 |
96 |       if (error) throw error;
97 |       await logAuditEvent({
98 |         action: "update",
99 |         entity_type: "badge_definition",
100 |         entity_id: id,
101 |         changes: input,
102 |         performed_by: user?.id ?? "",
103 |       });
104 |       return data;
105 |     },
106 |     onSuccess: () => invalidateDefinitions(queryClient),
107 |   });
108 | };
109 |
110 | export const useDeleteBadgeDefinition = () => {
111 |   const queryClient = useQueryClient();
112 |   const { user } = useAuth();
113 |
114 |   return useMutation({
115 |     mutationFn: async (id: string) => {
116 |       const { error } = await supabase
117 |         .from("badge_definitions")
118 |         .delete()
119 |         .eq("id", id);
120 |

```

```

121 |         if (error) throw error;
122 |         await logAuditEvent({
123 |             action: "delete",
124 |             entity_type: "badge_definition",
125 |             entity_id: id,
126 |             changes: {},
127 |             performed_by: user?.id ?? "",
128 |         });
129 |     },
130 |     onSuccess: () => invalidateDefinitions(queryClient),
131 | });
132 | };
133 |

```

#### FILE: src/hooks/useBadgeSpotlight.ts

TYPE: .ts | LINES: 1-16 | SHA256: 0c6b78d076900ba5271e4018055b53427f5ac459d84cf570dbc2f3a9cc28b8f4

```

1 | // =====
2 | // useBadgeSpotlight - Weekly badge spotlight query
3 | // Task 20.11 - Re-exports from useTieredBadges
4 | // =====
5 |
6 | export {
7 |     useBadgeSpotlight as useBadgeSpotlightQuery,
8 |     useBadgeSpotlightSchedule,
9 |     useUpdateBadgeSpotlightSchedule,
10 | } from "@hooks/useTieredBadges";
11 |
12 | export type {
13 |     SpotlightData,
14 |     SpotlightScheduleEntry,
15 | } from "@hooks/useTieredBadges";
16 |

```

#### FILE: src/hooks/useBaselineCourses.ts

TYPE: .ts | LINES: 1-91 | SHA256: 5dc284f3d0ad020a3cf786a8d105ec6cd589cca9d02a1a09f14c840295f07a1b

```

1 | // =====
2 | // useEnrolledCoursesWithBaseline - courses with an active baseline test
3 | //
4 | // Relocated from the in-page `useQuery` hook in `BaselineSelectStep.tsx`. For
5 | // the student's active enrollments it resolves which courses have an active
6 | // baseline-test config plus the per-course question count, returning only the
7 | // courses that actually have baseline questions available.
8 | //
9 | // Keeps all Supabase access in `src/hooks/` and uses the project's standard
10 | // TanStack Query conventions (query keys + typed responses).
11 | //
12 | // _Requirements: 25.1, 25.2, 25.3, 25.3a, 25.4_
13 | // =====
14 |
15 | import { useQuery } from "@tanstack/react-query";
16 | import { supabase } from "@lib/supabase";
17 | import { queryKeys } from "@lib/queryKeys";
18 |
19 | export interface CourseWithBaseline {
20 |     course_id: string;
21 |     course_name: string;
22 |     time_limit_minutes: number;
23 |     question_count: number;
24 | }
25 |
26 | export const useEnrolledCoursesWithBaseline = (studentId: string) => {
27 |     return useQuery({
28 |         queryKey: [...queryKeys.onboarding.baselineTests("enrolled"), studentId],
29 |         queryFn: async (): Promise<CourseWithBaseline[]> => {
30 |             // Student's enrolled courses (with course identity for labels).
31 |             const { data: enrollments, error: enrollError } = await supabase
32 |                 .from("student_courses")
33 |                 .select("course_id, courses(id, name)")
34 |                 .eq("student_id", studentId)
35 |                 .eq("status", "active");
36 |
37 |             if (enrollError) throw enrollError;
38 |             const enrollmentRows = enrollments ?? [];
39 |             if (enrollmentRows.length === 0) return [];
40 |
41 |             const courseIds = enrollmentRows.map((e) => e.course_id);
42 |
43 |             // Active baseline configs for those courses.
44 |             const { data: configs, error: configError } = await supabase
45 |                 .from("baseline_test_config")
46 |                 .select("course_id, time_limit_minutes")
47 |                 .in("course_id", courseIds)
48 |                 .eq("is_active", true);
49 |
50 |             if (configError) throw configError;
51 |             if (!configs?.length) return [];
52 |
53 |             // Question counts per course (active baseline questions only).
54 |             const activeCourseIds = configs.map((c) => c.course_id);
55 |             const { data: questions, error: qError } = await supabase
56 |                 .from("onboarding_questions")
57 |                 .select("course_id")
58 |                 .eq("assessment_type", "baseline")
59 |                 .eq("is_active", true)
60 |                 .in("course_id", activeCourseIds);
61 |
62 |             if (qError) throw qError;
63 |
64 |             const questionCounts = new Map<string, number>();
65 |             for (const q of questions ?? []) {
66 |                 const cid = q.course_id;
67 |                 if (!cid) continue;

```







[illegible]

```

15   event_type: string;
16   xp_multiplier: number;
17   bonus_xp: number;
18   starts_at: string | null;
19   ends_at: string | null;
20   is_active: boolean;
21   institution_id: string | null;
22   created_at: string;
23 }
24
25 export interface BonusEventFilters {
26   search?: string;
27 }
28
29 // % % % useBonusEvents - list all bonus events % % % % % % % % % % % % % % % % % % % % % % % % % % % %
30
31 export const useBonusEvents = (filters: BonusEventFilters = {}) => {
32   return useQuery({
33     queryKey: queryKeys.bonusXPEvents.list(filters as Record<string, unknown>),
34     queryFn: async (): Promise<BonusXPEvent[]> => {
35       let query = supabase
36         .from("xp_events")
37         .select("*")
38         .order("starts_at", { ascending: false });
39
40       if (filters.search) {
41         const safe = sanitizePostgresValue(filters.search);
42         query = query.ilike("name", `_${safe}%`);
43       }
44
45       const { data, error } = await query;
46       if (error) throw error;
47       return data as BonusXPEvent[];
48     },
49     staleTime: 30_000,
50   });
51 };
52
53 // % % % useBonusEvent - single event detail % % % % % % % % % % % % % % % % % % % % % % % % % % % %
54
55 export const useBonusEvent = (id: string | undefined) => {
56   return useQuery({
57     queryKey: queryKeys.bonusXPEvents.detail(id ?? ""),
58     queryFn: async (): Promise<BonusXPEvent | null> => {
59       const { data, error } = await supabase
60         .from("xp_events")
61         .select("*")
62         .eq("id", id!)
63         .maybeSingle();
64
65       if (error) throw error;
66       return data as BonusXPEvent | null;
67     },
68     enabled: !!id,
69     staleTime: 30_000,
70   });
71 };
72
73 // % % % useActiveBonusEvent - currently active bonus event % % % % % % % % % % % % % % % % % % % % % %
74
75 export const useActiveBonusEvent = () => {
76   return useQuery({
77     queryKey: queryKeys.bonusXPEvents.list({ active: true }),
78     queryFn: async (): Promise<BonusXPEvent | null> => {
79       const now = new Date().toISOString();
80
81       const { data, error } = await supabase
82         .from("xp_events")
83         .select("*")
84         .eq("is_active", true)
85         .lte("starts_at", now)
86         .gte("ends_at", now)
87         .maybeSingle();
88
89       if (error) throw error;
90       return data as BonusXPEvent | null;
91     },
92     staleTime: 30_000,
93   });
94 };
95
96 // % % % useCreateBonusEvent - insert with audit logging % % % % % % % % % % % % % % % % % % % % % %
97
98 export const useCreateBonusEvent = () => {
99   const queryClient = useQueryClient();
100   const { user } = useAuth();
101
102   return useMutation({
103     mutationFn: async (
104       data: CreateBonusEventFormData
105     ): Promise<BonusXPEvent> => {
106       const { data: result, error } = await supabase
107         .from("xp_events")
108         .insert(data)
109         .select()
110         .single();
111
112       if (error) throw error;
113
114       const event = result as BonusXPEvent;
115
116       await logAuditEvent({
117         action: "create",
118         entity_type: "bonus_xp_event",

```

```

119 |         entity_id: event.id,
120 |         changes: data as Record<string, unknown>,
121 |         performed_by: user?.id ?? "unknown",
122 |     });
123 |
124 |     return event;
125 | },
126 | onSuccess: () => {
127 |     queryClient.invalidateQueries({
128 |         queryKey: queryKeys.bonusXPEvents.lists(),
129 |     });
130 | },
131 | });
132 | };
133 |
134 | // % % % useUpdateBonusEvent – update with audit logging % % % % % % % % % % % % % % % %
135 |
136 | export const useUpdateBonusEvent = (id: string) => {
137 |     const queryClient = useQueryClient();
138 |     const { user } = useAuth();
139 |
140 |     return useMutation({
141 |         mutationFn: async (
142 |             data: Partial<CreateBonusEventFormData>
143 |         ): Promise<BonusXPEvent> => {
144 |             const { data: result, error } = await supabase
145 |                 .from("xp_events")
146 |                 .update(data)
147 |                 .eq("id", id)
148 |                 .select()
149 |                 .single();
150 |
151 |             if (error) throw error;
152 |
153 |             await logAuditEvent({
154 |                 action: "update",
155 |                 entity_type: "bonus_xp_event",
156 |                 entity_id: id,
157 |                 changes: data as Record<string, unknown>,
158 |                 performed_by: user?.id ?? "unknown",
159 |             });
160 |
161 |             return result as BonusXPEvent;
162 |         },
163 |         onSuccess: () => {
164 |             queryClient.invalidateQueries({
165 |                 queryKey: queryKeys.bonusXPEvents.lists(),
166 |             });
167 |             queryClient.invalidateQueries({
168 |                 queryKey: queryKeys.bonusXPEvents.detail(id),
169 |             });
170 |         },
171 |     });
172 | };
173 |
174 | // % % % useDeleteBonusEvent – soft delete (set is_active=false) % % % % % % % % % % % % % % % %
175 |
176 | export const useDeleteBonusEvent = () => {
177 |     const queryClient = useQueryClient();
178 |     const { user } = useAuth();
179 |
180 |     return useMutation({
181 |         mutationFn: async (id: string): Promise<BonusXPEvent> => {
182 |             const { data: result, error } = await supabase
183 |                 .from("xp_events")
184 |                 .update({ is_active: false })
185 |                 .eq("id", id)
186 |                 .select()
187 |                 .single();
188 |
189 |             if (error) throw error;
190 |
191 |             await logAuditEvent({
192 |                 action: "soft_delete",
193 |                 entity_type: "bonus_xp_event",
194 |                 entity_id: id,
195 |                 changes: { is_active: false },
196 |                 performed_by: user?.id ?? "unknown",
197 |             });
198 |
199 |             return result as BonusXPEvent;
200 |         },
201 |         onSuccess: () => {
202 |             queryClient.invalidateQueries({
203 |                 queryKey: queryKeys.bonusXPEvents.lists(),
204 |             });
205 |         },
206 |     });
207 | };
208 |

```

## FILE: src/hooks/useBonusQuestion.ts

TYPE: .ts | LINES: 1-84 | SHA256: 170d881b6d5ed6be913ace7232883c35b582d2fa8339c53b21d2ecb0a2e680e

```

1 | // =====
2 | // useBonusQuestion – Bonus question trigger and answer submission
3 | // Task 20.4
4 | // =====
5 |
6 | import { useMutation } from "@tanstack/react-query";
7 | import { supabase } from "@/lib/supabase";
8 |
9 | export interface BonusQuestionData {
10 |     triggered: boolean;

```

```

11 |   question?: {
12 |     text: string;
13 |     type: string;
14 |     clo_id: string | null;
15 |     time_limit_seconds: number;
16 |     xp_reward: number;
17 |   };
18 | }
19 |
20 | export interface BonusAnswerResult {
21 |   correct: boolean;
22 |   xp_awarded: number;
23 |   feedback: string;
24 | }
25 |
26 | export const useTriggerBonusQuestion = () => {
27 |   return useMutation({
28 |     mutationFn: async ({
29 |       studentId,
30 |       institutionId,
31 |       courseId,
32 |     }): {
33 |       studentId: string;
34 |       institutionId: string;
35 |       courseId?: string;
36 |     }: Promise<BonusQuestionData> => {
37 |       const { data, error } = await supabase.functions.invoke(
38 |         "check-bonus-question",
39 |         {
40 |           body: {
41 |             student_id: studentId,
42 |             institution_id: institutionId,
43 |             course_id: courseId,
44 |             action: "trigger",
45 |           },
46 |         },
47 |       );
48 |       if (error) throw error;
49 |       return data as BonusQuestionData;
50 |     },
51 |   ));
52 | };
53 |
54 | export const useSubmitBonusAnswer = () => {
55 |   return useMutation({
56 |     mutationFn: async ({
57 |       studentId,
58 |       institutionId,
59 |       answer,
60 |       cloId,
61 |     }): {
62 |       studentId: string;
63 |       institutionId: string;
64 |       answer: string;
65 |       cloId?: string | null;
66 |     }: Promise<BonusAnswerResult> => {
67 |       const { data, error } = await supabase.functions.invoke(
68 |         "check-bonus-question",
69 |         {
70 |           body: {
71 |             student_id: studentId,
72 |             institution_id: institutionId,
73 |             action: "answer",
74 |             answer,
75 |             clo_id: cloId,
76 |           },
77 |         },
78 |       );
79 |       if (error) throw error;
80 |       return data as BonusAnswerResult;
81 |     },
82 |   ));
83 | };
84 |

```

#### FILE: src/hooks/useBulkImport.ts

TYPE: .ts | LINES: 1-53 | SHA256: 8ab9671da2747d11619249ae23e241614c0d1fcaccec614e8ae65adc4a7b500d5

```

1 | import { useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { logAuditEvent } from "@lib/auditLogger";
5 | import { useAuth } from "@hooks/useAuth";
6 |
7 | export interface BulkImportResult {
8 |   created: number;
9 |   errors: Array<{ row: number; message: string }>;
10 | }
11 |
12 | interface BulkImportRow {
13 |   email: string;
14 |   full_name: string;
15 |   role: string;
16 |   program_id?: string;
17 | }
18 |
19 | export const useBulkImportUsers = () => {
20 |   const queryClient = useQueryClient();
21 |   const { user } = useAuth();
22 |
23 |   return useMutation({
24 |     mutationFn: async (rows: BulkImportRow[]): Promise<BulkImportResult> => {
25 |       const { data, error } = await supabase.functions.invoke(
26 |         "bulk-import-users",

```

```

27 |         {
28 |             body: { rows },
29 |         }
30 |     );
31 |
32 |     if (error) throw error;
33 |
34 |     const result = data as BulkImportResult;
35 |
36 |     if (result.created > 0) {
37 |         await logAuditEvent({
38 |             action: "bulk_import",
39 |             entity_type: "user",
40 |             entity_id: "bulk",
41 |             changes: { count: result.created, errors: result.errors.length },
42 |             performed_by: user?.id ?? "unknown",
43 |         });
44 |     }
45 |
46 |     return result;
47 | },
48 | onSuccess: () => {
49 |     queryClient.invalidateQueries({ queryKey: queryKeys.users.lists() });
50 | },
51 | });
52 | };
53 |

```

## FILE: src/hooks/useBulkOperations.ts

TYPE: .ts | LINES: 1-61 | SHA256: ac60b6d3ea53fa27e4ca6bce65bce8b2b16e9d7fb7a65ee1bf58b7b589f3f2b7

```

1 | // Task 89.6: Bulk operations hooks – grade export, enrollment import/export, semester transition
2 |
3 | import { useMutation } from "@tanstack/react-query";
4 | import { supabase } from "@lib/supabase";
5 |
6 | interface BulkGradeExportResult {
7 |     download_url: string;
8 |     file_name: string;
9 | }
10 |
11 | export const useBulkGradeExport = () => {
12 |     return useMutation({
13 |         mutationFn: async (params: {
14 |             course_id: string;
15 |             section_id?: string;
16 |         }): Promise<BulkGradeExportResult> => {
17 |             const { data, error } = await supabase.functions.invoke(
18 |                 "bulk-grade-export",
19 |                 { body: params }
20 |             );
21 |             if (error) throw error;
22 |             return data as BulkGradeExportResult;
23 |         },
24 |     });
25 | };
26 |
27 | export const useBulkEnrollmentImport = () => {
28 |     return useMutation({
29 |         mutationFn: async (params: {
30 |             file: File;
31 |         }): Promise<{ imported: number; skipped: number; errors: string[] }> => {
32 |             const text = await params.file.text();
33 |             const { data, error } = await supabase.functions.invoke(
34 |                 "bulk-data-import",
35 |                 {
36 |                     body: { import_type: "enrollments", csv_content: text },
37 |                 }
38 |             );
39 |             if (error) throw error;
40 |             return data as { imported: number; skipped: number; errors: string[] };
41 |         },
42 |     });
43 | };
44 |
45 | export const useSemesterTransition = () => {
46 |     return useMutation({
47 |         mutationFn: async (params: {
48 |             source_semester_id: string;
49 |             target_semester_id: string;
50 |             program_id: string;
51 |         }): Promise<{ copied_courses: number; copied_clos: number }> => {
52 |             const { data, error } = await supabase.functions.invoke(
53 |                 "semester-transition",
54 |                 { body: params }
55 |             );
56 |             if (error) throw error;
57 |             return data as { copied_courses: number; copied_clos: number };
58 |         },
59 |     });
60 | };
61 |

```

## FILE: src/hooks/useCalendar.ts

TYPE: .ts | LINES: 1-136 | SHA256: eed26e99013ccc1fc43d91af019eae4d1c786308e5fd371f89140be53e87009f

```

1 | import { useQuery } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { useAuth } from "@hooks/useAuth";
5 |
6 | export type CalendarEventSource =
7 |     | "assignment"
8 |     | "quiz"

```

```

9 | | "class_session"
10 | | "academic_calendar"
11 | | "announcement";
12 | export type CalendarEventType =
13 | | "assignment"
14 | | "quiz"
15 | | "class_session"
16 | | "academic";
17 |
18 | export interface CalendarEvent {
19 |   id: string;
20 |   title: string;
21 |   date: string;
22 |   source?: CalendarEventSource;
23 |   type?: CalendarEventType;
24 |   course_id?: string;
25 |   course_name?: string;
26 |   color?: string;
27 | }
28 |
29 | const COLOR_PALETTE = [
30 |   "#3b82f6",
31 |   "#8b5cf6",
32 |   "#ef4444",
33 |   "#f59e0b",
34 |   "#22c55e",
35 |   "#14b8a6",
36 |   "#ec4899",
37 |   "#6366f1",
38 | ];
39 |
40 | export const buildColorMap = (courseIds: string[]): Record<string, string> => {
41 |   const unique = [...new Set(courseIds)];
42 |   const map: Record<string, string> = {};
43 |   for (let i = 0; i < unique.length; i++) {
44 |     const key = unique[i];
45 |     if (key) map[key] = COLOR_PALETTE[i % COLOR_PALETTE.length] ?? "#3b82f6";
46 |   }
47 |   return map;
48 | };
49 |
50 | export const getDeadlineUrgency = (
51 |   deadline: string | Date
52 | ): "red" | "yellow" | "green" => {
53 |   const deadlineMs = new Date(deadline).getTime();
54 |   const hoursLeft = (deadlineMs - Date.now()) / (1000 * 60 * 60);
55 |   if (hoursLeft <= 24) return "red";
56 |   if (hoursLeft <= 72) return "yellow";
57 |   return "green";
58 | };
59 |
60 | export const useCalendarEvents = (month?: number, year?: number) => {
61 |   const { user } = useAuth();
62 |
63 |   return useQuery({
64 |     queryKey: queryKeys.calendarEvents.list({ month, year, userId: user?.id }),
65 |     queryFn: async (): Promise<CalendarEvent[]> => {
66 |       const events: CalendarEvent[] = [];
67 |       const now = new Date();
68 |       const m = month ?? now.getMonth() + 1;
69 |       const y = year ?? now.getFullYear();
70 |       const startDate = `${y}-${String(m).padStart(2, "0")}-01`;
71 |       const nextM = m + 1 > 12 ? 1 : m + 1;
72 |       const nextY = m + 1 > 12 ? y + 1 : y;
73 |       const endDate = `${nextY}-${String(nextM).padStart(2, "0")}-01`;
74 |
75 |       const { data: assignments, error: assignmentsError } = await supabase
76 |         .from("assignments")
77 |         .select("id, title, due_date, course_id")
78 |         .gte("due_date", startDate)
79 |         .lt("due_date", endDate);
80 |       if (assignmentsError) throw assignmentsError;
81 |
82 |       for (const a of assignments ?? []) {
83 |         events.push({
84 |           id: a.id,
85 |           title: a.title,
86 |           date: a.due_date,
87 |           type: "assignment",
88 |           course_id: a.course_id,
89 |           color: "#3b82f6",
90 |         });
91 |       }
92 |
93 |       const { data: quizzes, error: quizzesError } = await supabase
94 |         .from("quizzes")
95 |         .select("id, title, due_date, course_id")
96 |         .gte("due_date", startDate)
97 |         .lt("due_date", endDate);
98 |       if (quizzesError) throw quizzesError;
99 |
100 |      for (const q of quizzes ?? []) {
101 |        if (q.due_date) {
102 |          events.push({
103 |            id: q.id,
104 |            title: q.title,
105 |            date: q.due_date,
106 |            type: "quiz",
107 |            course_id: q.course_id,
108 |            color: "#8b5cf6",
109 |          });
110 |        }
111 |      }
112 |    }

```

```

113 |         const { data: academic, error: academicError } = await supabase
114 |           .from("academic_calendar_events")
115 |           .select("id, title, start_date")
116 |           .gte("start_date", startDate)
117 |           .lt("start_date", endDate);
118 |         if (academicError) throw academicError;
119 |
120 |         for (const e of academic ?? []) {
121 |           events.push({
122 |             id: e.id,
123 |             title: e.title,
124 |             date: e.start_date,
125 |             type: "academic",
126 |             color: "#f59e0b",
127 |           });
128 |         }
129 |
130 |         return events;
131 |       },
132 |       enabled: !!user,
133 |       staleTime: 60_000,
134 |     });
135 |   });
136 | }

```

FILE: src/hooks/useChallengeDetail.ts

```
TYPE: .ts | LINES: 1-45 | SHA256: c2e0daedd753c617438c6ea5edd08b5bbe288cadcf8ef7d40ebf02db5cdfbc36
```

```

1 // =====
2 // useChallengeDetail - TanStack Query hook for a single social challenge
3 // Task 7.7: zero-or-one row query via `.maybeSingle()`.
4 // =====
5 // Relocated from the in-page hook in `ChallengeDetailPage.tsx`, which used the
6 // strict `.single()` (throws on zero rows). Using `.maybeSingle()` lets a
7 // missing challenge resolve to `null` (a graceful not-found state) while a real
8 // query failure still throws and surfaces as the query's error state. Consumers
9 // therefore distinguish:
10 //   - `data === null`           `!` not-found (no row matched the id) [R27.2]
11 //   - `!isError` / `!error`    `!` query failure (error state + toast) [R28.2]
12 // _Requirements: 27.1, 27.2, 28.2_
13 // =====
14
15 import { useQuery } from "@tanstack/react-query";
16 import { supabase } from "@/lib/supabase";
17 import { queryKeys } from "@/lib/queryKeys";
18 import type { SocialChallenge } from "@/hooks/useChallenges";
19
20 /**
21  * Fetch a single social challenge by id.
22  *
23  * Uses `.maybeSingle()` so a query that matches zero rows resolves to `null`
24  * (graceful not-found) instead of throwing. A genuine query error is rethrown
25  * and exposed through the query's `error`/`isError`, keeping not-found and
26  * failure as two distinct, separately-handleable states.
27  */
28 export const useChallengeDetail = (challengeId?: string) => {
29   return useQuery({
30     queryKey: queryKeys.challenges.detail(challengeId ?? ""),
31     queryFn: async (): Promise<SocialChallenge | null> => {
32       const { data, error } = await supabase
33         .from("social_challenges")
34         .select(".*")
35         .eq("id", challengeId!)
36         .maybeSingle();
37       // A real failure (network, RLS, etc.) throws `!` surfaces as error state.
38       if (error) throw error;
39       // Zero rows `!` `data` is null `!` graceful not-found (never throws).
40       return (data as SocialChallenge | null) ?? null;
41     },
42     enabled: !!challengeId,
43   });
44 };
45

```

FILE: `src/hooks/useChallengeLeaderboard.ts`

```
TYPE: .ts | LINES: 1-64 | SHA256: 03c9c2634ec64c41a0fc2f8830da81b7f2999099717050c9fe00d2363afde9a3
```

[illegible]



```

28 |     .select(
29 |       "participant_id, participant_type, current_progress, completed_at"
30 |     )
31 |     .eq("challenge_id", challengeId!)
32 |     .order("current_progress", { ascending: false })
33 |     .order("completed_at", { ascending: true, nullsFirst: false });
34 | if (error) throw error;
35 |
36 | // Assign ranks – participants with same progress get same rank
37 | const entries: ChallengeLeaderboardEntry[] = [];
38 | let currentRank = 1;
39 |
40 | // `participant_type` is a plain string column in the DB; narrow it to the
41 | // domain union, defaulting unknown values to "individual".
42 | const rows = (data ?? []).map((row) => ({
43 |   participant_id: row.participant_id,
44 |   participant_type: (row.participant_type === "team"
45 |     ? "team"
46 |     : "individual") as "team" | "individual",
47 |   current_progress: row.current_progress,
48 |   completed_at: row.completed_at,
49 | }));
50 |
51 | for (let i = 0; i < rows.length; i++) {
52 |   const row = rows[i]!;
53 |   if (i > 0 && row.current_progress < rows[i - 1]!.current_progress) {
54 |     currentRank = i + 1;
55 |   }
56 |   entries.push({ ...row, rank: currentRank });
57 | }
58 |
59 | return entries;
60 | },
61 | enabled: !!challengeId,
62 | });
63 | };
64 |

```

FILE: src/hooks/useChallengeParticipation.ts

```
TYPE: .ts | LINES: 1-85 | SHA256: 1beb1df048590a725c2e6b19cebec23b68a7b1f0c5de1bbf9c062e7741d396f2
```

[illegible]

```
TYPE: .ts | LINES: 1-108 | SHA256: e31e046373476f8a8f3c495a57cef9cf63776c31529500682e661ceb5503a31a
```

```

79 |         progress = progressData as ChallengeProgressRecord | null;
80 |     }
81 |
82 |     return {
83 |         challenge: challenge as SocialChallenge,
84 |         progress,
85 |     };
86 | },
87 | enabled: !!challengeId,
88 | });
89 | };
90 |
91 | // % % % useChallengeAllProgress - all participants' progress for a challenge % % %
92 |
93 | export const useChallengeAllProgress = (challengeId?: string) => {
94 |     return useQuery({
95 |         queryKey: queryKeys.challengeProgress.list({ challengeId }),
96 |         queryFn: async (): Promise<ChallengeProgressRecord[]> => {
97 |             const { data, error } = await supabase
98 |                 .from("challenge_progress" as never)
99 |                 .select("")
100 |                 .eq("challenge_id", challengeId!)
101 |                 .order("current_progress", { ascending: false });
102 |             if (error) throw error;
103 |             return (data ?? []) as ChallengeProgressRecord[];
104 |         },
105 |         enabled: !!challengeId,
106 |     });
107 | };
108 |

```

FILE: src/hooks/useChallengeRealtime.ts

TYPE: .ts | LINES: 1-46 | SHA256: 8ded32837a8f2f493ef0921f5c86ee8891bde1dd6aa66d1e3ed9b05e626ad07d

[illegible]

FILE: src/hooks/useChallenges.ts

```
TYPE: .ts | LINES: 1-370 | SHA256: 9aa57f3401f8fa17cf5278b50ea1cd736d21e55816c924994fc783ca7863b5e9
```

[illegible]

[illegible]

[illegible]

```

229 | * Eliminates the N+1 pattern where each ChallengeCard fires its own query.
230 | */
231 | export const useChallengeParticipantsBatch = (challengeIds: string[]) => {
232 |   return useQuery({
233 |     queryKey: queryKeys.challengeProgress.list({ ids: challengeIds }),
234 |     queryFn: async (): Promise<Record<string, ChallengeParticipant[]>> => {
235 |       if (challengeIds.length === 0) return {};
236 |
237 |       const { data, error } = await supabase
238 |         .from("challenge_participants")
239 |         .select(
240 |           "id, challenge_id, participant_id, participant_type, current_progress"
241 |         )
242 |         .in("challenge_id", challengeIds);
243 |       if (error) throw error;
244 |
245 |       // Group by challenge_id
246 |       const grouped: Record<string, ChallengeParticipant[]> = {};
247 |       for (const row of (data ?? []) as unknown as ChallengeParticipant[]) {
248 |         const key = row.challenge_id;
249 |         if (!grouped[key]) {
250 |           grouped[key] = [];
251 |         }
252 |         (grouped[key] as ChallengeParticipant[]).push(row);
253 |       }
254 |       return grouped;
255 |     },
256 |     enabled: challengeIds.length > 0,
257 |     staleTime: DASHBOARD_STALE_TIME_MS,
258 |     retry: 1,
259 |   });
260 | };
261 |
262 | /** @deprecated Use useChallengeAllProgress from useChallengeProgress instead */
263 | export const useChallengeProgress = (challengeId?: string) => {
264 |   return useQuery({
265 |     queryKey: queryKeys.challengeProgress.detail(challengeId ?? ""),
266 |     queryFn: async (): Promise<ChallengeParticipant[]> => {
267 |       const { data, error } = await supabase
268 |         .from("challenge_participants")
269 |         .select(
270 |           "id, challenge_id, participant_id, participant_type, current_progress"
271 |         )
272 |         .eq("challenge_id", challengeId!);
273 |       if (error) throw error;
274 |       return (data ?? []) as unknown as ChallengeParticipant[];
275 |     },
276 |     enabled: !!challengeId,
277 |     staleTime: DASHBOARD_STALE_TIME_MS,
278 |     retry: 1,
279 |   });
280 | };
281 |
282 | export const useStudentChallenges = (studentId?: string) => {
283 |   return useQuery({
284 |     queryKey: queryKeys.studentChallenges.detail(studentId ?? ""),
285 |     queryFn: async (): Promise<SocialChallenge[]> => {
286 |       const { data: enrollments } = await supabase
287 |         .from("student_courses")
288 |         .select("course_id")
289 |         .eq("student_id", studentId!);
290 |
291 |       if (!enrollments || enrollments.length === 0) return [];
292 |
293 |       const courseIds = enrollments.map((e) => e.course_id);
294 |
295 |       // `select(".*")` mirrors the live `social_challenges` columns. The previous
296 |       // explicit list referenced drifted columns (`goal_metric`, `reward_type`,
297 |       // `reward_value`, `notification_sent_90`) that do not exist in the
298 |       // database, which made PostgREST reject the whole query. Live status is
299 |       // `ended` (not `completed`).
300 |       const { data, error } = await supabase
301 |         .from("social_challenges")
302 |         .select(".*")
303 |         .in("course_id", courseIds)
304 |         .in("status", ["active", "ended"])
305 |         .order("start_date", { ascending: false });
306 |
307 |       if (error) throw error;
308 |       return (data ?? []) as unknown as SocialChallenge[];
309 |     },
310 |     enabled: !!studentId,
311 |     staleTime: DASHBOARD_STALE_TIME_MS,
312 |   });
313 | };
314 |
315 | export const useCancelChallenge = () => {
316 |   const qc = useQueryClient();
317 |   return useMutation({
318 |     mutationFn: async (id: string) => {
319 |       const { data, error } = await supabase
320 |         .from("social_challenges" as never)
321 |         .update({
322 |           status: "cancelled",
323 |           updated_at: new Date().toISOString(),
324 |         } as never)
325 |         .eq("id", id)
326 |         .select()
327 |         .single();
328 |       if (error) throw error;
329 |       return data;
330 |     },
331 |     onSuccess: () => {
332 |       qc.invalidateQueries({ queryKey: queryKeys.challenges.lists() });

```

```

333 |     },
334 |     onError: (error: Error) => {
335 |       toast.error(error.message || "Failed to cancel challenge");
336 |     },
337 |   });
338 | };
339 |
340 | export const useAssignTeamsToChallenge = () => {
341 |   const qc = useQueryClient();
342 |   return useMutation({
343 |     mutationFn: async ({
344 |       challengeId,
345 |       teamIds,
346 |     }): {
347 |       challengeId: string;
348 |       teamIds: string[];
349 |     } => {
350 |       const rows = teamIds.map((teamId) => ({
351 |         challenge_id: challengeId,
352 |         participant_id: teamId,
353 |         participant_type: "team",
354 |         current_progress: 0,
355 |       }));
356 |       const { error } = await supabase
357 |         .from("challenge_participants" as never)
358 |         .insert(rows as never);
359 |       if (error) throw error;
360 |     },
361 |     onSuccess: () => {
362 |       qc.invalidateQueries({ queryKey: queryKeys.challenges.lists() });
363 |       qc.invalidateQueries({ queryKey: queryKeys.challengeProgress.lists() });
364 |     },
365 |     onError: (error: Error) => {
366 |       toast.error(error.message || "Failed to assign teams");
367 |     },
368 |   });
369 | };
370 |

```

## FILE: src/hooks/useClassDonations.ts

TYPE: .ts | LINES: 1-121 | SHA256: 2b7b72a9591972363c4ec5514fdda00808b4c9bc4b4e9c24f609b515a2e37e18

```

1 | // =====
2 | // useClassDonations - Donation campaigns, contribution mutation
3 | // Task 20.7
4 | // =====
5 |
6 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
7 | import { supabase } from "@/lib/supabase";
8 | import { queryKeys } from "@/lib/queryKeys";
9 | import { toast } from "sonner";
10 |
11 | export interface ClassDonation {
12 |   id: string;
13 |   course_id: string;
14 |   resource_description: string;
15 |   goal_amount: number;
16 |   current_total: number;
17 |   status: "active" | "completed" | "cancelled";
18 |   created_at: string;
19 | }
20 |
21 | export const useClassDonations = (courseId?: string) => {
22 |   return useQuery({
23 |     queryKey: queryKeys.donations.list({ courseId }),
24 |     queryFn: async (): Promise<ClassDonation[]> => {
25 |       let query = supabase
26 |         .from("class_donations")
27 |         .select("*")
28 |         .order("created_at", { ascending: false });
29 |
30 |       if (courseId) {
31 |         query = query.eq("course_id", courseId);
32 |       }
33 |
34 |       const { data, error } = await query;
35 |       if (error) throw error;
36 |       return (data ?? []) as ClassDonation[];
37 |     },
38 |   });
39 | };
40 |
41 | export const useContributeToDonation = () => {
42 |   const qc = useQueryClient();
43 |   return useMutation({
44 |     mutationFn: async ({
45 |       donationId,
46 |       studentId,
47 |       xpAmount,
48 |     }): {
49 |       donationId: string;
50 |       studentId: string;
51 |       xpAmount: number;
52 |     } => {
53 |       // Insert contribution
54 |       const { data, error } = await supabase
55 |         .from("class_donation_contributions")
56 |         .insert({
57 |           donation_id: donationId,
58 |           student_id: studentId,
59 |           xp_amount: xpAmount,
60 |         } as never)
61 |         .select()
62 |         .single();

```

```

63 if (error) throw error;
64
65 // Insert corresponding xp_purchase for the XP sink
66 const { error: purchaseError } = await supabase
67   .from("xp_purchases")
68   .insert({
69     student_id: studentId,
70     item_id: null as unknown as string,
71     xp_cost: xpAmount,
72     status: "consumed",
73     purchased_at: new Date().toISOString(),
74     metadata: { type: "class_donation", donation_id: donationId },
75   } as never);
76
77 if (purchaseError) throw purchaseError;
78
79 return data;
80 },
81 onSuccess: () => {
82   qc.invalidateQueries({ queryKey: queryKeys.donations.all });
83   qc.invalidateQueries({ queryKey: queryKeys.marketplace.all });
84   toast.success("Donation contributed!");
85 },
86 onError: (err) => toast.error((err as Error).message),
87 });
88 };
89
90 export const useCreateClassDonation = () => {
91   const qc = useQueryClient();
92   return useMutation({
93     mutationFn: async (input: {
94       courseId: string;
95       resourceDescription: string;
96       goalAmount: number;
97       institutionId: string;
98     }) => {
99       const { data, error } = await supabase
100         .from("class_donations")
101         .insert({
102           course_id: input.courseId,
103           resource_description: input.resourceDescription,
104           goal_amount: input.goalAmount,
105           institution_id: input.institutionId,
106           current_total: 0,
107           status: "active",
108         } as never)
109         .select()
110         .single();
111       if (error) throw error;
112       return data;
113     },
114     onSuccess: () => {
115       qc.invalidateQueries({ queryKey: queryKeys.donations.all });
116       toast.success("Donation campaign created");
117     },
118     onError: (err) => toast.error((err as Error).message),
119   });
120 };
121

```

FILE: src/hooks/useCLOEvidence.ts

```
TYPE: .ts | LINES: 1-88 | SHA256: 46c887c8ad5713265e114f2aae42e419f23ac4d2cddedb4edf42409d07fc6e46
```

[illegible]



```

42 | ];
43 |
44 | const { data: submissions, error: subError } = await supabase
45 |   .from("submissions")
46 |   .select("id, assignment_id")
47 |   .in("id", submissionIds);
48 |
49 | if (subError) throw subError;
50 |
51 | const assignmentIds = [
52 |   ...new Set((submissions ?? []).map((s) => s.assignment_id)),
53 | ];
54 |
55 | const { data: assignments, error: assignError } = await supabase
56 |   .from("assignments")
57 |   .select("id, title")
58 |   .in("id", assignmentIds);
59 |
60 | if (assignError) throw assignError;
61 |
62 | // Build lookup maps
63 | const assignmentMap = new Map<string, string>();
64 | for (const a of assignments ?? []) {
65 |   assignmentMap.set(a.id, a.title);
66 | }
67 |
68 | const submissionAssignmentMap = new Map<string, string>();
69 | for (const s of submissions ?? []) {
70 |   submissionAssignmentMap.set(s.id, s.assignment_id);
71 | }
72 |
73 | // Map evidence rows to display records
74 | return evidenceRows.map((e) => {
75 |   const assignmentId = submissionAssignmentMap.get(e.submission_id) ?? "";
76 |   return {
77 |     id: e.id,
78 |     assignment_title:
79 |       assignmentMap.get(assignmentId) ?? "Unknown Assignment",
80 |     score_percent: e.score_percent,
81 |     created_at: e.created_at,
82 |   };
83 | });
84 | },
85 | enabled: !!cloId && !!studentId,
86 | });
87 | };
88 |

```

FILE: src/hooks/useCLOProgress.ts

```
TYPE: .ts | LINES: 1-148 | SHA256: dbdd3671a649e199cad7a2299c0eed4153326534897e5149acc19c527022e8be
```

[illegible]

```

54 |
55 |     const courseIds = enrollments.map((e) => e.course_id);
56 |
57 |     // 2. Get course names
58 |     const { data: courses, error: courseError } = await supabase
59 |       .from("courses")
60 |       .select("id, name")
61 |       .in("id", courseIds);
62 |
63 |     if (courseError) throw courseError;
64 |
65 |     const courseMap = new Map<string, string>();
66 |     for (const c of courses ?? []) {
67 |       courseMap.set(c.id, c.name);
68 |     }
69 |
70 |     // 3. Get CLOs for enrolled courses
71 |     const { data: clos, error: cloError } = await supabase
72 |       .from("learning_outcomes")
73 |       .select("id, title, blooms_level, course_id")
74 |       .eq("type", "CLO")
75 |       .in("course_id", courseIds)
76 |       .order("sort_order", { ascending: true });
77 |
78 |     if (cloError) throw cloError;
79 |     if (!clos || clos.length === 0) return [];
80 |
81 |     const cloIds = clos.map((c) => c.id);
82 |
83 |     // 4. Get attainment data for this student's CLOs
84 |     const { data: attainments, error: attError } = await supabase
85 |       .from("outcome_attainment")
86 |       .select("outcome_id, attainment_percent, sample_count")
87 |       .eq("student_id", studentId)
88 |       .eq("scope", "student_course")
89 |       .in("outcome_id", cloIds);
90 |
91 |     if (attError) throw attError;
92 |
93 |     const attainmentMap = new Map<
94 |       string,
95 |       { percent: number; samples: number }
96 |     >();
97 |     for (const a of attainments ?? []) {
98 |       attainmentMap.set(a.outcome_id, {
99 |         percent: a.attainment_percent,
100 |         samples: a.sample_count,
101 |       });
102 |     }
103 |
104 |     // 5. Build entries grouped by course
105 |     const courseGroupMap = new Map<string, CLOProgressEntry[]>();
106 |
107 |     for (const clo of clos) {
108 |       const courseId = clo.course_id;
109 |       if (!courseId) continue;
110 |
111 |       const att = attainmentMap.get(clo.id);
112 |       const entry: CLOProgressEntry = {
113 |         clo_id: clo.id,
114 |         clo_title: clo.title,
115 |         blooms_level: (clo.blooms_level ?? "remembering") as BloomsLevel,
116 |         attainment_percent: att?.percent ?? null,
117 |         attainment_level: att ? classifyAttainment(att.percent) : null,
118 |         sample_count: att?.samples ?? 0,
119 |         course_name: courseMap.get(courseId) ?? "Unknown Course",
120 |         course_id: courseId,
121 |       };
122 |
123 |       const existing = courseGroupMap.get(courseId) ?? [];
124 |       existing.push(entry);
125 |       courseGroupMap.set(courseId, existing);
126 |     }
127 |
128 |     // 6. Convert to array sorted by course name
129 |     const result: CLOProgressByCourse[] = [];
130 |     for (const [courseId, entries] of courseGroupMap) {
131 |       result.push({
132 |         course_id: courseId,
133 |         course_name: courseMap.get(courseId) ?? "Unknown Course",
134 |         entries,
135 |       });
136 |     }
137 |
138 |     result.sort((a, b) => a.course_name.localeCompare(b.course_name));
139 |     return result;
140 |   },
141 |   enabled: !!studentId,
142 |   // CLO attainment changes only on grade/evidence events (which invalidate via
143 |   // realtime); 2-min staleTime prevents the 4-query fan-out (student_courses +
144 |   // courses + learning_outcomes + outcome_attainment) on every dashboard re-mount.
145 |   staleTime: 120_000,
146 | });
147 | };
148 |

```

FILE: [src/hooks/useCLOs.ts](#)

TYPE: .ts | LINES: 1-527 | SHA256: 76a787a9c3fbe1fac99729c147e6ce911cb90bae55440bd0eed2c1961c8c1158

```

1 | import {
2 |   useQuery,
3 |   keepPreviousData,
4 |   useMutation,
5 |   useQueryClient,

```

[illegible]

```

110 |         .maybeSingle();
111 |
112 |         if (error) throw error;
113 |         return data as LearningOutcome | null;
114 |     },
115 |     enabled: !!id,
116 | });
117 | };
118 |
119 | // % % % useCreateCLO - insert CLO + optional PLO mappings % % % % % % % % % % % % % % % %
120 |
121 | export const useCreateCLO = () => {
122 |     const queryClient = useQueryClient();
123 |     const { user } = useAuth();
124 |
125 |     return useMutation({
126 |         mutationFn: async (data: CreateCLOFormData): Promise<LearningOutcome> => {
127 |             const { plo_mappings, ...cloFields } = data;
128 |
129 |             const { data: result, error } = await supabase
130 |                 .from("learning_outcomes")
131 |                 .insert({ ...cloFields, type: "CLO" } as never)
132 |                 .select()
133 |                 .single();
134 |
135 |             if (error) throw error;
136 |
137 |             const clo = result as LearningOutcome;
138 |
139 |             // Insert PLO mappings if provided
140 |             if (plo_mappings && plo_mappings.length > 0) {
141 |                 const rows = plo_mappings.map((m) => ({
142 |                     source_outcome_id: m.plo_id,
143 |                     target_outcome_id: clo.id,
144 |                     weight: m.weight,
145 |                 }));
146 |
147 |                 const { error: mappingError } = await supabase
148 |                     .from("outcome_mappings")
149 |                     .insert(rows);
150 |
151 |                 if (mappingError) throw mappingError;
152 |             }
153 |
154 |             await logAuditEvent({
155 |                 action: "create",
156 |                 entity_type: "clo",
157 |                 entity_id: clo.id,
158 |                 changes: data,
159 |                 performed_by: user?.id ?? "unknown",
160 |             });
161 |
162 |             return clo;
163 |         },
164 |         onSuccess: () => {
165 |             queryClient.invalidateQueries({ queryKey: queryKeys.clos.lists() });
166 |             queryClient.invalidateQueries({
167 |                 queryKey: queryKeys.outcomeMappings.lists(),
168 |             });
169 |         },
170 |     });
171 | };
172 |
173 | // % % % useUpdateCLO - update CLO fields + optional PLO mapping replacement % % % %
174 |
175 | export const useUpdateCLO = (id: string) => {
176 |     const queryClient = useQueryClient();
177 |     const { user } = useAuth();
178 |
179 |     return useMutation({
180 |         mutationFn: async (
181 |             data: Partial<CreateCLOFormData>
182 |         ): Promise<LearningOutcome> => {
183 |             const { plo_mappings, ...cloFields } = data;
184 |
185 |             // NOTE: tutor_autonomy_level column exists in DB but database.ts types have not been
186 |             // regenerated yet. Using type assertion until `scripts/regen-types.ps1` is run.
187 |             // eslint-disable-next-line @typescript-eslint/no-explicit-any
188 |             const { data: result, error } = await (supabase as any)
189 |                 .from("learning_outcomes")
190 |                 .update(cloFields)
191 |                 .eq("id", id)
192 |                 .select()
193 |                 .single();
194 |
195 |             if (error) throw error;
196 |
197 |             // Replace PLO mappings if provided
198 |             if (plo_mappings !== undefined) {
199 |                 const { error: deleteError } = await supabase
200 |                     .from("outcome_mappings")
201 |                     .delete()
202 |                     .eq("target_outcome_id", id);
203 |
204 |                 if (deleteError) throw deleteError;
205 |
206 |                 if (plo_mappings.length > 0) {
207 |                     const rows = plo_mappings.map((m) => ({
208 |                         source_outcome_id: m.plo_id,
209 |                         target_outcome_id: id,
210 |                         weight: m.weight,
211 |                     }));
212 |
213 |                     const { error: insertError } = await supabase

```

[illegible]

```
318 // pattern in useILOs.useReorderILOs. Batch ID is 'batch' because the  
319 // changes payload carries the full item list for traceability.  
320 await logAuditEvent({  
321   action: "reorder",  
322   entity_type: "clo",  
323   entity_id: "batch",  
324   changes: { items: data.items },  
325   performed_by: user?.id ?? "unknown",  
326 });  
327 },  
328 onSuccess: () => {  
329   queryClient.invalidateQueries({ queryKey: queryKeys.clos.lists() });  
330 },  
331 }));  
332 };  
333  
334 // % % % useCLOMappings - fetch PLO mappings for a CLO % % % % % % % % % % % % % % % % % %  
335  
336 export const useCLOMappings = (cloakId?: string) => {  
337   return useQuery(  
338     queryKey: queryKeys.outcomeMappings.list({ cloakId }),  
339     queryFn: async (): Promise<OutcomeMapping[]> => {  
340       const { data, error } = await supabase  
341         .from("outcome_mappings")  
342         .select("*")  
343         .eq("target_outcome_id", cloakId!);  
344  
345       if (error) throw error;  
346       return data as OutcomeMapping[];  
347     },  
348     enabled: !!cloakId,  
349   ));  
350 };  
351  
352 // % % % useUpdateCLOMappings - replace outcome_mappings for a CLO!'PLO % % % % % % % %  
353  
354 export const useUpdateCLOMappings = () => {  
355   const queryClient = useQueryClient();  
356   const { user } = useAuth();  
357  
358   return useMutation(  
359     mutationFn: async (data: {  
360       cloakId: string;  
361       mappings: Array<{ source_outcome_id: string; weight: number }>;  
362     }): Promise<void> => {  
363       // Delete existing mappings for this CLO  
364       const { error: deleteError } = await supabase  
365         .from("outcome_mappings")  
366         .delete()  
367         .eq("target_outcome_id", data.cloakId);  
368  
369       if (deleteError) throw deleteError;  
370  
371       // Insert new mappings  
372       if (data.mappings.length > 0) {  
373         const rows = data.mappings.map((m) => ({  
374           source_outcome_id: m.source_outcome_id,  
375           target_outcome_id: data.cloakId,  
376           weight: m.weight,  
377         }));  
378  
379         const { error: insertError } = await supabase  
380           .from("outcome_mappings")  
381           .insert(rows);  
382  
383         if (insertError) throw insertError;  
384       }  
385  
386       // Req 13.5 - admin mutation writes an audit_logs row. The entity is  
387       // the CLO whose mappings changed; `changes` captures the new mapping  
388       // set so reviewers can reconstruct before/after without a DB query.  
389       await logAuditEvent(  
390         action: "update",  
391         entity_type: "clo_mappings",  
392         entity_id: data.cloakId,  
393         changes: { mappings: data.mappings },  
394         performed_by: user?.id ?? "unknown",  
395       );  
396     },  
397     onSuccess: () => {  
398       queryClient.invalidateQueries({  
399         queryKey: queryKeys.outcomeMappings.lists(),  
400       });  
401       queryClient.invalidateQueries({ queryKey: queryKeys.clos.lists() });  
402     },  
403   ));  
404 };  
405  
406 // % % % useCLOMappedPLOS - read-only: PLOs mapped to a CLO with titles % % % % % % % %  
407 //  
408 // Resolves the PLO (source) title in the same round trip via the  
409 // `outcome_mappings_source_outcome_id_fkey` to-one join (no N+1). Used by the  
410 // read-only CLODetailPage (Req 14.1).  
411  
412 export const useCLOMappedPLOS = (cloakId?: string) => {  
413   return useQuery(  
414     queryKey: queryKeys.outcomeMappings.list({ cloakId, view: "mapped-plos" }),  
415     queryFn: async (): Promise<CLOMappedPLO[]> => {  
416       const { data, error } = await supabase  
417         .from("outcome_mappings")  
418         .select(`  
419           id, source_outcome_id, weight, plo:learning_outcomes!outcome_mappings_source_outcome_id_fkey(title)  
420         `)  
421         .eq("target_outcome_id", cloakId!);
```

```

422 |         if (error) throw error;
423 |
424 |
425 |         type Row = {
426 |             id: string;
427 |             source_outcome_id: string;
428 |             weight: number;
429 |             plo: { title: string | null } | null;
430 |         };
431 |
432 |         return ((data ?? []) as unknown as Row[]).map((row) => ({
433 |             mapping_id: row.id,
434 |             plo_id: row.source_outcome_id,
435 |             title: row.plo?.title ?? null,
436 |             weight: row.weight,
437 |         }));
438 |     },
439 |     enabled: !!cloId,
440 | });
441 | };
442 |
443 | // % % % useCLOLinkedAssignments - read-only: assignments assessing a CLO % % % % %
444 | //
445 | // Assignments reference CLOs through the `clo_weights` jsonb array
446 | // (`[{ clo_id, weight }]`), which cannot be filtered server-side. We fetch the
447 | // course's assignments in one query (scoped by `course_id`) and filter in
448 | // memory for those whose `clo_weights` include this CLO - no per-row N+1.
449 |
450 | export const useCLOLinkedAssignments = (cloId?: string, courseId?: string) => {
451 |     return useQuery({
452 |         queryKey: queryKeys.assignments.list({ cloId, courseId, view: "by-clo" }),
453 |         queryFn: async (): Promise<CLOLinkedAssignment[]> => {
454 |             let query = supabase
455 |                 .from("assignments")
456 |                 .select("id, title, due_date, clo_weights")
457 |                 .order("due_date", { ascending: true });
458 |
459 |             if (courseId) {
460 |                 query = query.eq("course_id", courseId);
461 |             }
462 |
463 |             const { data, error } = await query;
464 |
465 |             if (error) throw error;
466 |
467 |             type Row = {
468 |                 id: string;
469 |                 title: string;
470 |                 due_date: string;
471 |                 clo_weights: Array<{ clo_id: string; weight: number }> | null;
472 |             };
473 |
474 |             return ((data ?? []) as unknown as Row[]).
475 |                 .map((row) => {
476 |                     const match = (row.clo_weights ?? []).find((w) => w.clo_id === cloId);
477 |                     if (!match) return null;
478 |                     return {
479 |                         id: row.id,
480 |                         title: row.title,
481 |                         due_date: row.due_date,
482 |                         weight: match.weight,
483 |                     };
484 |                 })
485 |                 .filter((a): a is CLOLinkedAssignment => a !== null);
486 |         },
487 |         enabled: !!cloId,
488 |     });
489 | };
490 |
491 | // % % % useCLOAttainment - read-only: attainment for a CLO % % % % %
492 | //
493 | // Reads `outcome_attainment` rows for this CLO at `scope = 'student_course'`
494 | // (the scope at which the attainment rollup trigger writes per-student CLO
495 | // attainment) and returns the mean attainment percent plus the number of
496 | // contributing rows. Returns a null percent when no rows exist so the UI can
497 | // distinguish "no data" from a real 0%.
498 | // (Scope verified against live data 2026-06: CLO rows exist at student_course,
499 | // not course.)
500 |
501 | export const useCLOAttainment = (cloId?: string) => {
502 |     return useQuery({
503 |         queryKey: queryKeys.outcomeAttainment.detail(cloId ?? ""),
504 |         queryFn: async (): Promise<CLOAttainmentSummary> => {
505 |             const { data, error } = await supabase
506 |                 .from("outcome_attainment")
507 |                 .select("attainment_percent")
508 |                 .eq("outcome_id", cloId!)
509 |                 .eq("scope", "student_course");
510 |
511 |             if (error) throw error;
512 |
513 |             const rows = data ?? [];
514 |             if (rows.length === 0) {
515 |                 return { percent: null, sampleCount: 0 };
516 |             }
517 |
518 |             const total = rows.reduce((sum, r) => sum + r.attainment_percent, 0);
519 |             return {
520 |                 percent: total / rows.length,
521 |                 sampleCount: rows.length,
522 |             };
523 |         },
524 |         enabled: !!cloId,
525 |     });

```

```
526 | };
527 |
```

# FILE: src/hooks/useComebackChallenge.ts

TYPE: .ts | LINES: 1-96 | SHA256: 2c6dc7b4d491d5f5e9491817ab59ddb6dd64fab137b52e2b575c76d845f016b4

```
1  // Task 143.4: Comeback Challenge TanStack Query hooks
2
3  import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
4  import { supabase } from "@lib/supabase";
5  import { queryKeys } from "@lib/queryKeys";
6  import { DASHBOARD_STALE_TIME_MS } from "@lib/queryConfig";
7
8  export interface ComebackChallengeData {
9    is_active: boolean;
10   days_completed: number;
11   streak_to_restore: number;
12   start_date: string | null;
13 }
14
15 export const useComebackChallenge = (studentId: string | undefined) => {
16   return useQuery({
17     queryKey: queryKeys.comebackChallenge.detail(studentId ?? ""),
18     queryFn: async (): Promise<ComebackChallengeData> => {
19       const { data, error } = await supabase
20         .from("student_gamification")
21         .select(
22           "comeback_challenge_active, comeback_challenge_days_completed, comeback_challenge_streak_to_restore,
23         )
24         .eq("student_id", studentId!)
25         .maybeSingle();
26
27       if (error) throw error;
28       if (!data) {
29         return {
30           is_active: false,
31           days_completed: 0,
32           streak_to_restore: 0,
33           start_date: null,
34         };
35       }
36
37       const row = data as unknown as Record<string, unknown>;
38       return {
39         is_active: row.comeback_challenge_active === true,
40         days_completed:
41           typeof row.comeback_challenge_days_completed === "number"
42             ? row.comeback_challenge_days_completed
43             : 0,
44         streak_to_restore:
45           typeof row.comeback_challenge_streak_to_restore === "number"
46             ? row.comeback_challenge_streak_to_restore
47             : 0,
48         start_date:
49           typeof row.comeback_challenge_start_date === "string"
50             ? row.comeback_challenge_start_date
51             : null,
52       };
53     },
54     enabled: !!studentId,
55     staleTime: DASHBOARD_STALE_TIME_MS,
56   });
57 };
58
59 export const useStartComebackChallenge = () => {
60   const qc = useQueryClient();
61   return useMutation({
62     mutationFn: async (studentId: string) => {
63       const { error } = await supabase
64         .from("student_gamification")
65         .update({
66           comeback_challenge_active: true,
67           comeback_challenge_start_date: new Date().toISOString(),
68           comeback_challenge_days_completed: 0,
69         }) as never
70         .eq("student_id", studentId);
71       if (error) throw error;
72     },
73     onSuccess: () =>
74       qc.invalidateQueries({ queryKey: queryKeys.comebackChallenge.all }),
75   });
76 };
77
78 export const useCancelComebackChallenge = () => {
79   const qc = useQueryClient();
80   return useMutation({
81     mutationFn: async (studentId: string) => {
82       const { error } = await supabase
83         .from("student_gamification")
84         .update({
85           comeback_challenge_active: false,
86           comeback_challenge_days_completed: 0,
87           comeback_challenge_streak_to_restore: 0,
88         }) as never
89         .eq("student_id", studentId);
90       if (error) throw error;
91     },
92     onSuccess: () =>
93       qc.invalidateQueries({ queryKey: queryKeys.comebackChallenge.all }),
94   });
95 };
96
```



## FILE: src/hooks/useComebackChallengeStatus.ts

TYPE: .ts | LINES: 1-64 | SHA256: 8c8dccbd5a812bc428fb3efa9df4a73a2cd20baf906782c91cfb8b803fccd8b3

```
1 | import { useQuery } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import type { ComebackChallengeStatus } from "@types/habits";
5 |
6 | /**
7 |  * Fetches the active Comeback Challenge status for a student from
8 |  * the student_gamification table. Returns default inactive status
9 |  * when no data is found or the query fails.
10 |  */
11 | export const useComebackChallengeStatus = (studentId: string | undefined) => {
12 |   return useQuery({
13 |     queryKey: [
14 |       ...queryKeys.studentGamification.all,
15 |       "comebackChallenge",
16 |       studentId ?? "",
17 |     ],
18 |     enabled: !!studentId,
19 |     queryFn: async (): Promise<ComebackChallengeStatus> => {
20 |       if (!studentId) {
21 |         return { active: false, currentDay: 0, totalDays: 3, startDate: null };
22 |       }
23 |
24 |       try {
25 |         const { data, error } = await supabase
26 |           .from("student_gamification")
27 |           .select(
28 |             "comeback_challenge_active, comeback_challenge_days_completed, comeback_challenge_start_date" as n
29 |           )
30 |           .eq("student_id", studentId)
31 |           .maybeSingle();
32 |
33 |         if (error) throw error;
34 |
35 |         if (!data) {
36 |           return {
37 |             active: false,
38 |             currentDay: 0,
39 |             totalDays: 3,
40 |             startDate: null,
41 |           };
42 |         }
43 |
44 |         const row = data as unknown as Record<string, unknown>;
45 |         return {
46 |           active: row.comeback_challenge_active === true,
47 |           currentDay:
48 |             typeof row.comeback_challenge_days_completed === "number"
49 |               ? row.comeback_challenge_days_completed
50 |               : 0,
51 |           totalDays: 3,
52 |           startDate:
53 |             typeof row.comeback_challenge_start_date === "string"
54 |               ? row.comeback_challenge_start_date
55 |               : null,
56 |         };
57 |       } catch {
58 |         // Table columns may not exist yet – return inactive
59 |         return { active: false, currentDay: 0, totalDays: 3, startDate: null };
60 |       }
61 |     },
62 |   });
63 | };
64 |
```

## FILE: src/hooks/useCommunications.ts

TYPE: .ts | LINES: 1-138 | SHA256: cff6965c98ef88d0995cd893927b2eaf154b3f63eb2297e4369634998d9a9c2a

```
1 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 |
5 | export interface CommunicationAttachment {
6 |   name: string;
7 |   url: string;
8 | }
9 |
10 | export type CommunicationItem =
11 |   | {
12 |     kind: "notification";
13 |     id: string;
14 |     title: string;
15 |     body: string;
16 |     isRead: boolean;
17 |     createdAt: string;
18 |     deepLink?: string;
19 |   }
20 |   | {
21 |     kind: "announcement";
22 |     id: string;
23 |     title: string;
24 |     body: string;
25 |     isRead: boolean;
26 |     isPinned: boolean;
27 |     createdAt: string;
28 |     attachments: CommunicationAttachment[];
29 |   };
30 |
31 | export const useCommunications = (userId: string | undefined) => {
32 |   const queryClient = useQueryClient();
```

```

33 |
34 | const query = useQuery({
35 |   queryKey: queryKeys.notifications.list({ userId }),
36 |   queryFn: async () => {
37 |     if (!userId) return { notifications: [], announcements: [] };
38 |
39 |     // Fetch personal notifications
40 |     const { data: notifData, error: notifError } = await supabase
41 |       .from("notifications")
42 |       .select("*")
43 |       .eq("user_id", userId)
44 |       .order("created_at", { ascending: false });
45 |
46 |     if (notifError) throw notifError;
47 |
48 |     // Fetch institution & course announcements
49 |     const { data: annData, error: annError } = await supabase
50 |       .from("announcements")
51 |       .select("*")
52 |       .order("created_at", { ascending: false });
53 |
54 |     if (annError) throw annError;
55 |
56 |     // Fetch read tracking for this user from announcement_reads (supporting user_id or student_id)
57 |     const { data: readData } = await supabase
58 |       .from("announcement_reads")
59 |       .select("announcement_id")
60 |       .or(`student_id.eq.${userId},user_id.eq.${userId}`);
61 |
62 |     const readAnnouncementIds = new Set(
63 |       (readData ?? []).map((r) => r.announcement_id)
64 |     );
65 |
66 |     const notifications: CommunicationItem[] = (notifData ?? []).map((n) => ({
67 |       kind: "notification",
68 |       id: n.id,
69 |       title: n.title ?? "Notification",
70 |       body: n.body ?? "",
71 |       isRead: n.is_read ?? false,
72 |       createdAt: n.created_at,
73 |     }));
74 |
75 |     const announcements: CommunicationItem[] = (annData ?? []).map((a) => ({
76 |       kind: "announcement",
77 |       id: a.id,
78 |       title: a.title ?? "Announcement",
79 |       body: a.content ?? "",
80 |       isRead: readAnnouncementIds.has(a.id),
81 |       isPinned: a.is_pinned ?? false,
82 |       createdAt: a.created_at,
83 |       attachments: [],
84 |     }));
85 |
86 |     return { notifications, announcements };
87 |   },
88 |   enabled: !!userId,
89 | });
90 |
91 | const markNotificationRead = useMutation({
92 |   mutationFn: async (id: string) => {
93 |     const { error } = await supabase
94 |       .from("notifications")
95 |       .update({ is_read: true })
96 |       .eq("id", id);
97 |     if (error) throw error;
98 |   },
99 |   onSuccess: () => {
100 |     queryClient.invalidateQueries({ queryKey: queryKeys.notifications.all });
101 |   },
102 | });
103 |
104 | const markAnnouncementRead = useMutation({
105 |   mutationFn: async (announcementId: string) => {
106 |     if (!userId) return;
107 |     // Multi-role user-aware insert/upsert into announcement_reads
108 |     const payload = {
109 |       announcement_id: announcementId,
110 |       student_id: userId,
111 |       user_id: userId,
112 |     } as unknown as Record<string, unknown>;
113 |
114 |     // eslint-disable-next-line @typescript-eslint/no-explicit-any
115 |     const { error } = await (supabase as any)
116 |       .from("announcement_reads")
117 |       .upsert(payload, { onConflict: "announcement_id,student_id" });
118 |     if (error) {
119 |       // Fallback for schemas with strict FK to students table
120 |       await supabase.from("announcement_reads").insert({
121 |         announcement_id: announcementId,
122 |         student_id: userId,
123 |       });
124 |     }
125 |   },
126 |   onSuccess: () => {
127 |     queryClient.invalidateQueries({ queryKey: queryKeys.notifications.all });
128 |     queryClient.invalidateQueries({ queryKey: queryKeys.announcements.all });
129 |   },
130 | });
131 |
132 | return {
133 |   ...query,
134 |   markNotificationRead: markNotificationRead.mutate,
135 |   markAnnouncementRead: markAnnouncementRead.mutate,
136 | };

```

```
137 | };
138 |
```

## FILE: src/hooks/useCompetencyFrameworks.ts

TYPE: .ts | LINES: 1-130 | SHA256: 064e127af535f5232a9c3cf2e33e909d9f293da274c2cca59e2ac1a815a88d64

```
1  // Task 114.3: Competency Framework TanStack Query hooks
2
3  import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
4  import { supabase } from "@lib/supabase";
5  import { queryKeys } from "@lib/queryKeys";
6
7  export interface CompetencyFramework {
8    id: string;
9    institution_id: string;
10   name: string;
11   version: string;
12   source: string | null;
13 }
14
15 export interface CompetencyItem {
16   id: string;
17   framework_id: string;
18   parent_id: string | null;
19   level: "domain" | "competency" | "indicator";
20   code: string;
21   title: string;
22   sort_order: number;
23   children?: CompetencyItem[];
24 }
25
26 export interface CompetencyOutcomeMapping {
27   id: string;
28   competency_item_id: string;
29   outcome_id: string;
30 }
31
32 // NOTE: These hooks reference tables that require database migrations (tasks 114.1).
33 // Until migrations are applied, queries will return empty results.
34
35 export const useCompetencyFrameworks = (institutionId?: string) => {
36   return useQuery({
37     queryKey: queryKeys.competencyFrameworks.list({ institutionId }),
38     queryFn: async (): Promise<CompetencyFramework[]> => {
39       const { data, error } = await supabase
40         .from("competency_frameworks" as never)
41         .select("*")
42         .eq("institution_id", institutionId!)
43         .order("name");
44       if (error) throw error;
45       return (data ?? []) as CompetencyFramework[];
46     },
47     enabled: !!institutionId,
48   });
49 };
50
51 export const useCompetencyItems = (frameworkId?: string) => {
52   return useQuery({
53     queryKey: queryKeys.competencyItems.list({ frameworkId }),
54     queryFn: async (): Promise<CompetencyItem[]> => {
55       const { data, error } = await supabase
56         .from("competency_items" as never)
57         .select("*")
58         .eq("framework_id", frameworkId!)
59         .order("sort_order");
60       if (error) throw error;
61       return (data ?? []) as CompetencyItem[];
62     },
63     enabled: !!frameworkId,
64   });
65 };
66
67 export const useCreateCompetencyFramework = () => {
68   const qc = useQueryClient();
69   return useMutation({
70     mutationFn: async (input: {
71       name: string;
72       version: string;
73       source?: string;
74       institution_id: string;
75     }) => {
76       const { data, error } = await supabase
77         .from("competency_frameworks" as never)
78         .insert(input as never)
79         .select()
80         .single();
81       if (error) throw error;
82       return data;
83     },
84     onSuccess: () => {
85       qc.invalidateQueries({
86         queryKey: queryKeys.competencyFrameworks.lists(),
87       });
88     },
89   });
90 };
91
92 export const useImportCompetencyCSV = () => {
93   const qc = useQueryClient();
94   return useMutation({
95     mutationFn: async (params: {
96       framework_id: string;
97       csv_content: string;
98     }) => {
99       const { data, error } = await supabase.functions.invoke(
```



```

13 |
14 | export interface ContributionMetric {
15 |   member_id: string;
16 |   team_id: string;
17 |   student_id: string;
18 |   contribution_status: ContributionStatus;
19 |   contribution_status_since: string | null;
20 |   consecutive_low_days: number;
21 | }
22 |
23 | export interface TeamContributionSummary {
24 |   team_id: string;
25 |   team_name: string;
26 |   members: ContributionMetric[];
27 |   has_inactive_members: boolean;
28 |   has_warning_members: boolean;
29 | }
30 |
31 | // % % % useContributionStatus - fetch contribution metrics for a team % % % % % % % % %
32 |
33 | export const useContributionStatus = (teamId?: string) => {
34 |   return useQuery({
35 |     queryKey: queryKeys.contributionStatus.list({ teamId }),
36 |     queryFn: async (): Promise<ContributionMetric[]> => {
37 |       const { data, error } = await supabase
38 |         .from("team_members" as never)
39 |         .select(
40 |           "id, team_id, student_id, contribution_status, contribution_status_since, consecutive_low_days"
41 |         )
42 |         .eq("team_id", teamId!)
43 |         .is("left_at", null)
44 |         .order("contribution_status", { ascending: true });
45 |       if (error) throw error;
46 |       return ((data ?? []) as Array<Record<string, unknown>>).map((row) => ({
47 |         member_id: row.id as string,
48 |         team_id: row.team_id as string,
49 |         student_id: row.student_id as string,
50 |         contribution_status: row.contribution_status as ContributionStatus,
51 |         contribution_status_since: row.contribution_status_since as
52 |           | string
53 |           | null,
54 |         consecutive_low_days: row.consecutive_low_days as number,
55 |       }));
56 |     },
57 |     enabled: !!teamId,
58 |   });
59 | };
60 |
61 | // % % % useTeamsContributionSummary - teacher view: all teams with contribution %
62 |
63 | export const useTeamsContributionSummary = (
64 |   courseId?: string,
65 |   filterInactive?: boolean
66 | ) => {
67 |   return useQuery({
68 |     queryKey: queryKeys.contributionStatus.list({ courseId, filterInactive }),
69 |     queryFn: async (): Promise<TeamContributionSummary[]> => {
70 |       // Fetch teams for the course
71 |       const { data: teams, error: teamsError } = await supabase
72 |         .from("teams" as never)
73 |         .select("id, name")
74 |         .eq("course_id", courseId!)
75 |         .is("deleted_at", null)
76 |         .order("name");
77 |       if (teamsError) throw teamsError;
78 |       if (!teams || teams.length === 0) return [];
79 |
80 |       const teamIds = (teams as Array<{ id: string }>).map((t) => t.id);
81 |
82 |       // Fetch all active members with contribution data
83 |       const { data: members, error: membersError } = await supabase
84 |         .from("team_members" as never)
85 |         .select(
86 |           "id, team_id, student_id, contribution_status, contribution_status_since, consecutive_low_days"
87 |         )
88 |         .in("team_id", teamIds)
89 |         .is("left_at", null);
90 |       if (membersError) throw membersError;
91 |
92 |       // Group members by team
93 |       const membersByTeam = new Map<string, ContributionMetric[]>();
94 |       for (const row of (members ?? []) as Array<Record<string, unknown>>) {
95 |         const teamId = row.team_id as string;
96 |         const metric: ContributionMetric = {
97 |           member_id: row.id as string,
98 |           team_id: teamId,
99 |           student_id: row.student_id as string,
100 |           contribution_status: row.contribution_status as ContributionStatus,
101 |           contribution_status_since: row.contribution_status_since as
102 |             | string
103 |             | null,
104 |           consecutive_low_days: row.consecutive_low_days as number,
105 |         };
106 |         const existing = membersByTeam.get(teamId) ?? [];
107 |         existing.push(metric);
108 |         membersByTeam.set(teamId, existing);
109 |       }
110 |
111 |       // Build summaries
112 |       let summaries: TeamContributionSummary[] = (
113 |         teams as Array<{ id: string; name: string }>
114 |       ).map((t) => {
115 |         const teamMembers = membersByTeam.get(t.id) ?? [];
116 |         return {

```

```

117 |         team_id: t.id,
118 |         team_name: t.name,
119 |         members: teamMembers,
120 |         has_inactive_members: teamMembers.some(
121 |             (m) => m.contribution_status === "inactive"
122 |         ),
123 |         has_warning_members: teamMembers.some(
124 |             (m) => m.contribution_status === "warning"
125 |         ),
126 |     };
127 | });
128 |
129 | // Apply filter
130 | if (filterInactive) {
131 |     summaries = summaries.filter((s) => s.has_inactive_members);
132 | }
133 |
134 | return summaries;
135 | },
136 | enabled: !!courseId,
137 | });
138 | });
139 |

```

## FILE: src/hooks/useCoordinatorAccreditation.ts

TYPE: .ts | LINES: 1-107 | SHA256: 46e2ed7691e48454f48091378440435f5db59ddddd7a1fc1ddff5bb7070f6a451

```

1 | // =====
2 | // useCoordinatorAccreditation – real accreditation readiness + approval chain
3 | // =====
4 | //
5 | // • useCoordinatorAccreditationReadiness – calls the
6 | //   get_coordinator_accreditation_readiness RPC (migration 20260823000004),
7 | //   which derives evidence-coverage readiness + per-course statuses + a pack
8 | //   checklist from REAL data. Returns null on any error (RPC not deployed yet)
9 | //   so the screen falls back to a neutral state.
10 | // • useAccreditationApprovals – reads the accreditation_approvals chain
11 | //   (migration 20260823000005) for the institution; returns [] when the table
12 | //   is absent/empty so the screen renders the default four-stage chain.
13 | //
14 | // Both fail soft against the current prod DB (migrations not applied yet). No
15 | // client writes here – the approval chain is read-only in this view (advancing
16 | // a stage needs a program-scoped action, a follow-up once approver roles exist).
17 | // =====
18 |
19 | import { useQuery } from "@tanstack/react-query";
20 | import { supabase } from "@lib/supabase";
21 | import { queryKeys } from "@lib/queryKeys";
22 |
23 | export type EvidenceStatus =
24 |   | "documented"
25 |   | "partial"
26 |   | "blocked"
27 |   | "not_started";
28 |
29 | export interface AccreditationCourse {
30 |   code: string;
31 |   name: string;
32 |   status: EvidenceStatus;
33 | }
34 |
35 | export interface AccreditationPackItem {
36 |   key: string;
37 |   state: "done" | "prog" | "pending";
38 | }
39 |
40 | export interface AccreditationReadiness {
41 |   readinessPercent: number;
42 |   documented: number;
43 |   partial: number;
44 |   blocked: number;
45 |   notStarted: number;
46 |   courses: AccreditationCourse[];
47 |   pack: AccreditationPackItem[];
48 | }
49 |
50 | export const useCoordinatorAccreditationReadiness = (
51 |   institutionId?: string | null
52 | ) => {
53 |   return useQuery({
54 |     queryKey: queryKeys.accreditationReports.list({
55 |       view: "readiness",
56 |       institutionId: institutionId ?? null,
57 |     }),
58 |     enabled: !!institutionId,
59 |     retry: false,
60 |     staleTime: 5 * 60 * 1000,
61 |     queryFn: async (): Promise<AccreditationReadiness | null> => {
62 |       try {
63 |         // RPC not in generated types yet (migration 20260823000004).
64 |         const { data, error } = await supabase.rpc(
65 |           "get_coordinator_accreditation_readiness" as never
66 |         );
67 |         if (error || !data) return null;
68 |         return data as unknown as AccreditationReadiness;
69 |       } catch {
70 |         return null;
71 |       }
72 |     },
73 |   });
74 | };
75 |
76 | export type ApprovalStageStatus = "done" | "current" | "pending";
77 |

```

```

78 | export interface AccreditationApprovalRow {
79 |   stage: string;
80 |   status: ApprovalStageStatus;
81 |   sort_order: number;
82 | }
83 |
84 | export const useAccreditationApprovals = (institutionId?: string | null) => {
85 |   return useQuery({
86 |     queryKey: queryKeys.accreditationReports.list({
87 |       view: "approvals",
88 |       institutionId: institutionId ?? null,
89 |     }),
90 |     enabled: !!institutionId,
91 |     retry: false,
92 |     queryFn: async (): Promise<AccreditationApprovalRow[]> => {
93 |       try {
94 |         // Table not in generated types yet (migration 20260823000005).
95 |         const { data, error } = await supabase
96 |           .from("accreditation_approvals" as never)
97 |           .select("stage, status, sort_order")
98 |           .returns<AccreditationApprovalRow[]>();
99 |         if (error || !data) return [];
100 |         return data;
101 |       } catch {
102 |         return [];
103 |       }
104 |     },
105 |   });
106 | };
107 |

```

FILE: [src/hooks/useCoordinatorAiInsights.ts](#)

TYPE: .ts | LINES: 1-61 | SHA256: 04b8b5d48aabdc463d9c1d30366fc5fa30c1e8faa76126c44f59479dd09f3a7d

```

1 | // =====
2 | // useCoordinatorAiInsights – AI attainment insight for the coordinator
3 | // =====
4 | //
5 | // Calls the `coordinator-ai-insights` Edge Function, which returns a real
6 | // rule-based attainment insight (optionally LLM-enhanced when a GEMINI_API_KEY
7 | // is configured server-side). This hook is ADDITIVE and fails soft: if the
8 | // function is not deployed yet, errors, or the caller has no institution, it
9 | // returns `null` so consuming screens simply fall back to their own computed
10 | // insight instead of showing an error. No writes from the client.
11 | // =====
12 |
13 | import { useQuery } from "@tanstack/react-query";
14 | import { supabase } from "@lib/supabase";
15 | import { queryKeys } from "@lib/queryKeys";
16 |
17 | export interface CoordinatorAiInsight {
18 |   threshold: number;
19 |   ploCount: number;
20 |   avgAttainment: number | null;
21 |   belowTargetCount: number;
22 |   weakest: {
23 |     ploTitle: string;
24 |     cloTitle: string | null;
25 |     attainment: number;
26 |   } | null;
27 |   narrative: string;
28 |   recommendations: string[];
29 |   source: "ai" | "computed";
30 |   model: string | null;
31 |   generatedAt: string;
32 |   cached?: boolean;
33 | }
34 |
35 | export const useCoordinatorAiInsights = (institutionId?: string | null) => {
36 |   return useQuery({
37 |     queryKey: queryKeys.aiSuggestions.list({
38 |       view: "coordinatorInsights",
39 |       institutionId: institutionId ?? null,
40 |     }),
41 |     enabled: !!institutionId,
42 |     // Insights are cached server-side (6h); avoid hammering the function.
43 |     staleTime: 30 * 60 * 1000,
44 |     retry: false,
45 |     queryFn: async (): Promise<CoordinatorAiInsight | null> => {
46 |       try {
47 |         const { data, error } = await supabase.functions.invoke(
48 |           "coordinator-ai-insights",
49 |           { body: { kind: "attainment" } }
50 |         );
51 |         if (error || !data) return null;
52 |         if ((data as { error?: string }).error) return null;
53 |         return data as CoordinatorAiInsight;
54 |       } catch {
55 |         // Function not deployed / network failure !' graceful (no AI card).
56 |         return null;
57 |       }
58 |     },
59 |   });
60 | };
61 |

```

FILE: [src/hooks/useCoordinatorAttainmentTrends.ts](#)

TYPE: .ts | LINES: 1-89 | SHA256: 61df941c62057db8a7bfe0449b17698d32eb596aeca4912af0ee0390bf8d6c0e

```

1 | // =====
2 | // useCoordinatorAttainmentTrends – per-outcome attainment history (trends)
3 | // =====
4 | //
5 | // Reads outcome_attainment_snapshots (migration 20260823000003) for the caller's

```

```

6 | // institution and returns a per-outcome time series (one point per semester).
7 | // The Outcome Attainment screen uses this to show a "vs last term" delta once
8 | // "e2 terms have been captured. This hook is ADDITIVE and fails soft: if the
9 | // snapshot table does not exist yet (pre-migration) or the read fails, it
10 | // returns an empty map so the screen simply omits the trend. No writes.
11 | // =====
12 |
13 | import { useQuery } from "@tanstack/react-query";
14 | import { supabase } from "@lib/supabase";
15 | import { queryKeys } from "@lib/queryKeys";
16 |
17 | export interface AttainmentTrendPoint {
18 |   semesterId: string;
19 |   semesterName: string;
20 |   startDate: string;
21 |   attainment: number;
22 | }
23 |
24 | /** outcome_id !' chronological series of per-semester attainment. */
25 | export type AttainmentTrendMap = Record<string, AttainmentTrendPoint[]>;
26 |
27 | interface SnapshotRow {
28 |   outcome_id: string;
29 |   semester_id: string;
30 |   mean_attainment_percent: number;
31 | }
32 |
33 | export const useCoordinatorAttainmentTrends = (
34 |   institutionId?: string | null
35 | ) => {
36 |   return useQuery({
37 |     queryKey: queryKeys.semesterTrends.list({
38 |       view: "coordinatorTrends",
39 |       institutionId: institutionId ?? null,
40 |     }),
41 |     enabled: !!institutionId,
42 |     retry: false,
43 |     queryFn: async (): Promise<AttainmentTrendMap> => {
44 |       // The snapshots table may not exist in the generated types yet
45 |       // (migration 20260823000003). Query it loosely + `.returns<T>()` and
46 |       // degrade to {} on any error (pre-migration / read failure).
47 |       const snapRes = await supabase
48 |         .from("outcome_attainment_snapshots" as never)
49 |         .select("outcome_id, semester_id, mean_attainment_percent")
50 |         .eq("institution_id" as never, (institutionId ?? "")) as never)
51 |         .returns<SnapshotRow[]>();
52 |       if (snapRes.error || !snapRes.data || snapRes.data.length === 0)
53 |         return {};
54 |       const snaps = snapRes.data;
55 |
56 |       // Resolve semester names/dates (typed table).
57 |       const semIds = Array.from(new Set(snaps.map((s) => s.semester_id)));
58 |       const { data: sems } = await supabase
59 |         .from("semesters")
60 |         .select("id, name, start_date")
61 |         .in("id", semIds);
62 |       const semById = new Map(
63 |         (sems ?? []).map((s) => [
64 |           s.id as string,
65 |           { name: s.name as string, start_date: s.start_date as string },
66 |         ])
67 |       );
68 |
69 |       const map: AttainmentTrendMap = {};
70 |       for (const s of snaps) {
71 |         const sem = semById.get(s.semester_id);
72 |         if (!sem) continue;
73 |         const list = map[s.outcome_id] ?? [];
74 |         list.push({
75 |           semesterId: s.semester_id,
76 |           semesterName: sem.name,
77 |           startDate: sem.start_date,
78 |           attainment: Math.round(s.mean_attainment_percent),
79 |         });
80 |         map[s.outcome_id] = list;
81 |       }
82 |       for (const series of Object.values(map)) {
83 |         series.sort((a, b) => a.startDate.localeCompare(b.startDate));
84 |       }
85 |       return map;
86 |     },
87 |   });
88 | };
89 |

```

## FILE: src/hooks/useCoordinatorDashboard.ts

TYPE: .ts | LINES: 1-139 | SHA256: 1e421aa1f3704cd92160d7e563afd4996a489fad0892ed7898e76a9f30a79243

```

1 | import { useQuery } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { DASHBOARD_STALE_TIME_MS } from "@lib/queryConfig";
5 |
6 | export interface CoordinatorKPIData {
7 |   totalPLOs: number;
8 |   totalCourses: number;
9 |   cloCoveragePercent: number;
10 |   /**
11 |    * Mean attainment across all `student_course`-scope `outcome_attainment` rows
12 |    * (C-2 – real attainment, replaces the prior active-student placeholder).
13 |    * 0 when no attainment evidence exists yet.
14 |    */
15 |   avgAttainmentPercent: number;
16 |   /**

```



```

17 | * Distinct students whose mean attainment falls below the at-risk threshold
18 | * (<50%, the "Not Yet" band). Derived from real `outcome_attainment` rows
19 | * (C-2 - replaces the active-student count placeholder).
20 | */
21 | atRiskStudents: number;
22 | teacherCompliancePercent: number;
23 | }
24 |
25 | /** At-risk threshold: students whose mean attainment is below this are flagged. */
26 | const AT_RISK_THRESHOLD = 50;
27 |
28 | const mean = (values: number[]): number =>
29 |   values.length === 0
30 |   ? 0
31 |   : values.reduce((sum, v) => sum + v, 0) / values.length;
32 |
33 | export const useCoordinatorKPIs = (options?: { enabled?: boolean }) => {
34 |   return useQuery({
35 |     queryKey: queryKeys.coordinatorDashboard.list({}),
36 |     queryFn: async (): Promise<CoordinatorKPIData> => {
37 |       // 1. Total PLOs
38 |       const { count: totalPLOs } = await supabase
39 |         .from("learning_outcomes")
40 |         .select("count", { count: "exact", head: true })
41 |         .eq("type", "PLO");
42 |
43 |       // 2. Total Courses
44 |       const { count: totalCourses } = await supabase
45 |         .from("courses")
46 |         .select("count", { count: "exact", head: true });
47 |
48 |       // 3. CLO Coverage: % of PLOs that have at least one CLO mapped via outcome_mappings
49 |       let cloCoveragePercent = 0;
50 |       const ploCount = totalPLOs ?? 0;
51 |
52 |       if (ploCount > 0) {
53 |         const { data: ploIds } = await supabase
54 |           .from("learning_outcomes")
55 |           .select("id")
56 |           .eq("type", "PLO");
57 |
58 |         const typedPloIds = ploIds ?? [];
59 |
60 |         if (typedPloIds.length > 0) {
61 |           const { data: mappings } = await supabase
62 |             .from("outcome_mappings")
63 |             .select("source_outcome_id")
64 |             .in(
65 |               "source_outcome_id",
66 |               typedPloIds.map((p) => p.id)
67 |             );
68 |
69 |           const typedMappings = mappings ?? [];
70 |           const coveredPLOs = new Set(
71 |             typedMappings.map((m) => m.source_outcome_id)
72 |           );
73 |           cloCoveragePercent = Math.round((coveredPLOs.size / ploCount) * 100);
74 |         }
75 |       }
76 |
77 |       // 4. Real attainment metrics (C-2): mean attainment + at-risk student count
78 |       //   derived from `outcome_attainment` at `student_course` scope, mirroring
79 |       //   the `useDepartmentAnalytics` aggregation over `attainment_percent`.
80 |       const { data: attainmentRows, error: attainmentError } = await supabase
81 |         .from("outcome_attainment")
82 |         .select("student_id, attainment_percent")
83 |         .eq("scope", "student_course");
84 |
85 |       if (attainmentError) throw attainmentError;
86 |
87 |       const allScores: number[] = [];
88 |       const scoresByStudent = new Map<string, number[]>();
89 |       for (const row of attainmentRows ?? []) {
90 |         if (row.attainment_percent == null) continue;
91 |         allScores.push(row.attainment_percent);
92 |         if (row.student_id) {
93 |           const list = scoresByStudent.get(row.student_id) ?? [];
94 |           list.push(row.attainment_percent);
95 |           scoresByStudent.set(row.student_id, list);
96 |         }
97 |       }
98 |
99 |       const avgAttainmentPercent =
100 |         allScores.length > 0 ? Math.round(mean(allScores)) : 0;
101 |
102 |       let atRiskStudents = 0;
103 |       for (const scores of scoresByStudent.values()) {
104 |         if (mean(scores) < AT_RISK_THRESHOLD) atRiskStudents += 1;
105 |       }
106 |
107 |       // 5. Teacher Compliance Rate: % of courses with at least one CLO defined
108 |       let teacherCompliancePercent = 0;
109 |       const courseCount = totalCourses ?? 0;
110 |
111 |       if (courseCount > 0) {
112 |         const { data: coursesWithCLOs } = await supabase
113 |           .from("learning_outcomes")
114 |           .select("course_id")
115 |           .eq("type", "CLO");
116 |
117 |         const typedCourseCLOs = coursesWithCLOs ?? [];
118 |         const uniqueCourses = new Set(
119 |           typedCourseCLOs.map((c) => c.course_id).filter(Boolean)
120 |         );

```

```

121 |         teacherCompliancePercent = Math.round(
122 |             (uniqueCourses.size / courseCount) * 100
123 |         );
124 |     }
125 |
126 |     return {
127 |         totalPLOs: ploCount,
128 |         totalCourses: courseCount,
129 |         cloCoveragePercent,
130 |         avgAttainmentPercent,
131 |         atRiskStudents,
132 |         teacherCompliancePercent,
133 |     };
134 | },
135 | enabled: options?.enabled ?? true,
136 | staleTime: DASHBOARD_STALE_TIME_MS,
137 | });
138 | };
139 |

```

#### FILE: src/hooks/useCoordinatorDashboardAggregate.ts

TYPE: .ts | LINES: 1-62 | SHA256: 12fe4788bd8efba9fc15ffc5f61387b03ba7983fcbbdb7dd8b81d3684fc82477

```

1 | import { useQuery, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { DASHBOARD_STALE_TIME_MS } from "@lib/queryConfig";
5 | import type { CoordinatorKPIData } from "@hooks/useCoordinatorDashboard";
6 |
7 | /**
8 |  * Coordinator dashboard aggregate (spec: dashboard-and-ux-performance, Phase 8
9 |  * Task 34).
10 |  *
11 |  * Collapses `useCoordinatorKPIs` ~6 SEQUENTIAL round-trips into ONE call to the
12 |  * `get_coordinator_dashboard` RPC, then HYDRATES the exact cache that hook reads
13 |  * so it resolves as a cache hit instead of firing its own request chain.
14 |  *
15 |  * The RPC is `SECURITY INVOKER`: every read is RLS-scoped to the caller's
16 |  * institution, so this adds no new data visibility (a coordinator only ever
17 |  * receives their own institution's KPIs). On RPC failure the section hook falls
18 |  * back to its own fetch (fully reversible).
19 |  *
20 |  * The RPC returns the `CoordinatorKPIData` shape directly (the KPI block is the
21 |  * only always-on fan-out section on this dashboard; programs/courses lists and
22 |  * the program/course-selected panels keep their own hooks).
23 |  */
24 | export const useCoordinatorDashboardAggregate = (
25 |     institutionId: string | null | undefined
26 | ) => {
27 |     const queryClient = useQueryClient();
28 |
29 |     return useQuery({
30 |         queryKey: queryKeys.coordinatorDashboard.list({
31 |             aggregate: true,
32 |             institutionId,
33 |         }),
34 |         enabled: !!institutionId,
35 |         staleTime: DASHBOARD_STALE_TIME_MS,
36 |         queryFn: async (): Promise<CoordinatorKPIData> => {
37 |             // `get_coordinator_dashboard` is added by migration 20260821000012 and is
38 |             // not in the generated Database types until regenerated post-merge, so cast
39 |             // the rpc call (same pattern as useXPBalance / get_xp_balance).
40 |             // eslint-disable-next-line @typescript-eslint/no-explicit-any
41 |             const { data, error } = await (supabase as any).rpc(
42 |                 "get_coordinator_dashboard",
43 |                 {}
44 |             );
45 |             if (error) throw error;
46 |             if (!data) {
47 |                 throw new Error("get_coordinator_dashboard returned no data");
48 |             }
49 |
50 |             const payload = data as CoordinatorKPIData;
51 |
52 |             // Hydrate the EXACT cache `useCoordinatorKPIs` reads so it becomes a hit.
53 |             queryClient.setQueryData(
54 |                 queryKeys.coordinatorDashboard.list({}),
55 |                 payload
56 |             );
57 |
58 |             return payload;
59 |         },
60 |     });
61 | };
62 |

```

#### FILE: src/hooks/useCoordinatorOutcomeAttainment.ts

TYPE: .ts | LINES: 1-257 | SHA256: 0dc7a1fada673c70e1a6c0961cd48b7b18c9ae6f7fb3a68575aa529b0a8634b6

```

1 | // =====
2 | // useCoordinatorOutcomeAttainment - real ILO! / PLO! / CLO attainment rollup
3 | // =====
4 | //
5 | // Aggregate read hook (UI prototype migration, Phase A backend wiring) that
6 | // powers the redesigned coordinator Outcome Attainment screen with REAL data.
7 | // It reads the existing tables only (no new backend):
8 | //   • learning_outcomes - the ILO / PLO / CLO records (RLS-scoped to the
9 | //   caller's institution)
10 | //   • outcome_attainment - mean attainment per outcome (each outcome_id is
11 | //   stored at exactly one scope: ILO=program,
12 | //   PLO=course, CLO=student_course), plus per-student
13 | //   rows used to derive "affected students"
14 | //   • outcome_mappings - parent!child edges (source=parent, target=child),
15 | //   so a PLO's contributing CLOs = mappings where

```

```

16 | // source_outcome_id = plo.id
17 | // • courses - code lookup for the weakest-course signal
18 | // • institution_settings - success_threshold (defaults to 70)
19 | //
20 | // All aggregation is done client-side over small, column-projected payloads.
21 | // No writes. Trends (per-term) and AI insights are intentionally NOT here -
22 | // they arrive in Phase C (semester snapshots + AI edge function).
23 | // =====
24 |
25 | import { useQuery } from "@tanstack/react-query";
26 | import { supabase } from "@lib/supabase";
27 | import { queryKeys } from "@lib/queryKeys";
28 |
29 | const DEFAULT_SUCCESS_THRESHOLD = 70;
30 | const DEVELOPING_THRESHOLD = 50;
31 |
32 | export type AttainmentStatus = "onTrack" | "watch" | "belowTarget" | "none";
33 |
34 | export interface AttainmentCLO {
35 |   id: string;
36 |   title: string;
37 |   attainment: number | null;
38 | }
39 |
40 | export interface AttainmentPLO {
41 |   id: string;
42 |   title: string;
43 |   description: string | null;
44 |   attainment: number | null;
45 |   status: AttainmentStatus;
46 |   contributingClos: AttainmentCLO[];
47 |   /** Weakest contributing course (code + attainment), if derivable. */
48 |   weakestCourse: { code: string; attainment: number } | null;
49 |   /** Students whose mean attainment for this PLO is below the success threshold. */
50 |   affectedStudents: number;
51 |   /** affectedStudents as a % of the students measured for this PLO. */
52 |   cohortPercent: number;
53 | }
54 |
55 | export interface AttainmentILO {
56 |   id: string;
57 |   title: string;
58 |   attainment: number | null;
59 |   status: AttainmentStatus;
60 | }
61 |
62 | export interface CoordinatorOutcomeAttainment {
63 |   ilos: AttainmentILO[];
64 |   plos: AttainmentPLO[];
65 |   successThreshold: number;
66 | }
67 |
68 | const statusFor = (att: number | null, threshold: number): AttainmentStatus => {
69 |   if (att === null) return "none";
70 |   if (att >= threshold) return "onTrack";
71 |   if (att >= DEVELOPING_THRESHOLD) return "watch";
72 |   return "belowTarget";
73 | };
74 |
75 | interface OutcomeRow {
76 |   id: string;
77 |   type: string;
78 |   title: string;
79 |   description: string | null;
80 |   course_id: string | null;
81 |   sort_order: number | null;
82 | }
83 | interface AttRow {
84 |   outcome_id: string;
85 |   attainment_percent: number | null;
86 |   student_id: string | null;
87 |   course_id: string | null;
88 | }
89 | interface MapRow {
90 |   source_outcome_id: string;
91 |   target_outcome_id: string;
92 | }
93 |
94 | export const useCoordinatorOutcomeAttainment = (
95 |   institutionId?: string | null
96 | ) => {
97 |   return useQuery({
98 |     queryKey: queryKeys.outcomeMappings.list({
99 |       view: "coordinatorOutcomeAttainment",
100 |      institutionId: institutionId ?? null,
101 |    }),
102 |     enabled: !!institutionId,
103 |     queryFn: async (): Promise<CoordinatorOutcomeAttainment> => {
104 |       const [outcomesRes, attRes, mapRes, coursesRes, settingsRes] =
105 |         await Promise.all([
106 |           supabase
107 |             .from("learning_outcomes")
108 |             .select("id, type, title, description, course_id, sort_order")
109 |             .order("sort_order", { ascending: true }),
110 |           supabase
111 |             .from("outcome_attainment")
112 |             .select("outcome_id, attainment_percent, student_id, course_id"),
113 |           supabase
114 |             .from("outcome_mappings")
115 |             .select("source_outcome_id, target_outcome_id"),
116 |           supabase.from("courses").select("id, code"),
117 |           supabase
118 |             .from("institution_settings")
119 |             .select("success_threshold")

```

```

120         .limit(1)
121         .maybeSingle(),
122     ));
123
124     if (outcomesRes.error) throw outcomesRes.error;
125     if (attRes.error) throw attRes.error;
126     if (mapRes.error) throw mapRes.error;
127     if (coursesRes.error) throw coursesRes.error;
128     // institution_settings is best-effort – fall back to the default threshold.
129
130     const outcomes = (outcomesRes.data ?? []) as OutcomeRow[];
131     const att = (attRes.data ?? []) as AttRow[];
132     const mappings = (mapRes.data ?? []) as MapRow[];
133     const threshold =
134         settingsRes.data?.success_threshold ?? DEFAULT_SUCCESS_THRESHOLD;
135
136     const courseCode = new Map<string, string>({
137         (coursesRes.data ?? []).map((c) => [c.id, c.code as string])
138     });
139
140     // Mean attainment per outcome + per-student means (for "affected students").
141     const totals = new Map<string, { sum: number; n: number }>();
142     const perStudent = new Map<
143         string,
144         Map<string, { sum: number; n: number }>
145     >();
146     for (const r of att) {
147         if (r.attainment_percent == null) continue;
148         const t = totals.get(r.outcome_id) ?? { sum: 0, n: 0 };
149         t.sum += r.attainment_percent;
150         t.n += 1;
151         totals.set(r.outcome_id, t);
152         if (r.student_id) {
153             const byStudent = perStudent.get(r.outcome_id) ?? new Map();
154             const s = byStudent.get(r.student_id) ?? { sum: 0, n: 0 };
155             s.sum += r.attainment_percent;
156             s.n += 1;
157             byStudent.set(r.student_id, s);
158             perStudent.set(r.outcome_id, byStudent);
159         }
160     }
161     const meanOf = (id: string): number | null => {
162         const t = totals.get(id);
163         return t && t.n > 0 ? Math.round(t.sum / t.n) : null;
164     };
165
166     // outcome id !' its record (for CLO title / course lookups).
167     const byId = new Map<string, OutcomeRow>(outcomes.map((o) => [o.id, o]));
168     // Undirected adjacency between outcomes. outcome_mappings in this data are
169     // stored child!parent for some pairs (CLO!PLO, PLO!ILO) and parent!child
170     // for others (ILO!PLO), so we treat edges as undirected and resolve a
171     // PLO's contributing CLOs by outcome TYPE rather than by edge direction.
172     const relatedOf = new Map<string, Set<string>>();
173     const addEdge = (a: string, b: string) => {
174         const set = relatedOf.get(a) ?? new Set<string>();
175         set.add(b);
176         relatedOf.set(a, set);
177     };
178     for (const m of mappings) {
179         addEdge(m.source_outcome_id, m.target_outcome_id);
180         addEdge(m.target_outcome_id, m.source_outcome_id);
181     }
182
183     const ilos: AttainmentILO[] = outcomes
184         .filter((o) => o.type === "ILO")
185         .map((o) => {
186             const attainment = meanOf(o.id);
187             return {
188                 id: o.id,
189                 title: o.title,
190                 attainment,
191                 status: statusFor(attainment, threshold),
192             };
193         });
194
195     const plos: AttainmentPLO[] = outcomes
196         .filter((o) => o.type === "PLO")
197         .map((o) => {
198             const attainment = meanOf(o.id);
199
200             const cloIds = Array.from(relatedOf.get(o.id) ?? []).filter((id) => {
201                 const related = byId.get(id);
202                 return related?.type === "CLO";
203             });
204             const contributingClos: AttainmentCLO[] = cloIds.map((id) => {
205                 const clo = byId.get(id);
206                 return {
207                     id,
208                     title: clo?.title ?? id,
209                     attainment: meanOf(id),
210                 };
211             });
212
213             // Weakest contributing course = the contributing CLO with the lowest
214             // measured attainment, resolved to its course code.
215             let weakestCourse: AttainmentPLO["weakestCourse"] = null;
216             for (const clo of contributingClos) {
217                 if (clo.attainment == null) continue;
218                 const cid = byId.get(clo.id)?.course_id ?? null;
219                 const code = cid ? courseCode.get(cid) : undefined;
220                 if (!code) continue;
221                 if (!weakestCourse || clo.attainment < weakestCourse.attainment) {
222                     weakestCourse = { code, attainment: clo.attainment };
223                 }
224             }
225         });

```

```

224 |     }
225 |
226 |     // Affected students = students whose mean attainment for this PLO is
227 |     // below the success threshold.
228 |     const byStudent = perStudent.get(o.id);
229 |     let affectedStudents = 0;
230 |     let cohort = 0;
231 |     if (byStudent) {
232 |       cohort = byStudent.size;
233 |       for (const s of byStudent.values()) {
234 |         if (s.n > 0 && s.sum / s.n < threshold) affectedStudents += 1;
235 |       }
236 |     }
237 |     const cohortPercent =
238 |       cohort > 0 ? Math.round((affectedStudents / cohort) * 100) : 0;
239 |
240 |     return {
241 |       id: o.id,
242 |       title: o.title,
243 |       description: o.description,
244 |       attainment,
245 |       status: statusFor(attainment, threshold),
246 |       contributingClos,
247 |       weakestCourse,
248 |       affectedStudents,
249 |       cohortPercent,
250 |     };
251 |   });
252 |
253 |   return { ilos, plos, successThreshold: threshold };
254 | },
255 | });
256 | };
257 |

```

FILE: src/hooks/useCoordinatorProfileSettings.ts

```
TYPE: .ts | LINES: 1-185 | SHA256: 62c163fdc13038204abb8d5c083a6657f7155244830f7e05f798bdc1f882f3f4
```

[illegible]

```
67 // failure yields the "not set" placeholder rather than crashing the page.  
68 if (error || !data) return EMPTY_ACADEMIC;  
69 const row = data as unknown as ProfileAcademicRow;  
70 return {  
71   department: row.department ?? null,  
72   designation: row.designation ?? null,  
73   academic_rank: row.academic_rank ?? null,  
74   highest_degree: row.highest_degree ?? null,  
75   years_experience:  
76     typeof row.years_experience === "number"  
77       ? row.years_experience  
78       : null,  
79 };  
80 },  
81 });  
82 }];  
83  
84 export const useUpdateCoordinatorAcademicProfile = () => {  
85   const qc = useQueryClient();  
86   return useMutation({  
87     mutationFn: async ({  
88       userId,  
89       values,  
90     }): {  
91       userId: string;  
92       values: CoordinatorAcademicProfile;  
93     }: Promise<void> => {  
94       const { error } = await supabase  
95         .from("profiles")  
96         // `as never`: the academic columns may be absent from the generated  
97         // Update type until database.ts is regenerated (migration 20260823000001).  
98         .update(values as never)  
99         .eq("id", userId);  
100      if (error) throw error;  
101    },  
102    onSuccess: (_data, variables) => {  
103      qc.invalidateQueries({  
104        queryKey: queryKeys.profiles.detail(`${variables.userId}:academic`),  
105      });  
106    },  
107  });  
108 }];  
109  
110 // % % % Coordinator alert notification preferences % % % % % % % % % % % % % % % % % % % % % %  
111  
112 export interface CoordinatorAlertPrefs {  
113   ploDrop: boolean;  
114   curriculumGap: boolean;  
115   evidenceReady: boolean;  
116   teacherInactivity: boolean;  
117   cqiDeadline: boolean;  
118 }  
119  
120 export const DEFAULT_COORDINATOR_ALERT_PREFS: CoordinatorAlertPrefs = {  
121   ploDrop: true,  
122   curriculumGap: true,  
123   evidenceReady: true,  
124   teacherInactivity: false,  
125   cqiDeadline: true,  
126 }];  
127  
128 const boolOr = (v: unknown, fallback: boolean): boolean =>  
129   typeof v === "boolean" ? v : fallback;  
130  
131 /**  
132 * Parse coordinator alert prefs out of a profile's `notification_preferences`  
133 * jsonb (`coordinator_alerts` sub-key), falling back to the defaults per key.  
134 */  
135 export const readCoordinatorAlertPrefs = (  
136   notificationPreferences: unknown  
137 ): CoordinatorAlertPrefs => {  
138   const np = (notificationPreferences ?? {}) as Record<string, unknown>;  
139   const ca = (np.coordinator_alerts ?? {}) as Record<string, unknown>;  
140   const d = DEFAULT_COORDINATOR_ALERT_PREFS;  
141   return {  
142     ploDrop: boolOr(ca.ploDrop, d.ploDrop),  
143     curriculumGap: boolOr(ca.curriculumGap, d.curriculumGap),  
144     evidenceReady: boolOr(ca.evidenceReady, d.evidenceReady),  
145     teacherInactivity: boolOr(ca.teacherInactivity, d.teacherInactivity),  
146     cqiDeadline: boolOr(ca.cqiDeadline, d.cqiDeadline),  
147   };  
148 }];  
149  
150 export const useUpdateCoordinatorAlertPrefs = () => {  
151   const qc = useQueryClient();  
152   return useMutation({  
153     mutationFn: async ({  
154       userId,  
155       prefs,  
156     }): {  
157       userId: string;  
158       prefs: CoordinatorAlertPrefs;  
159     }: Promise<void> => {  
160       // Merge into the existing jsonb so muted_courses / quiet_hours (owned by  
161       // useNotificationPreferences) are never clobbered.  
162       const { data, error: readErr } = await supabase  
163         .from("profiles")  
164         .select("notification_preferences")  
165         .eq("id", userId)  
166         .maybeSingle();  
167       if (readErr) throw readErr;  
168       const current = (data?.notification_preferences ?? {}) as Record<  
169         string,  
170         unknown
```

```

171 |         >;
172 |         const merged = { ...current, coordinator_alerts: prefs };
173 |         const { error } = await supabase
174 |             .from("profiles")
175 |             .update({ notification_preferences: merged as unknown as Json })
176 |             .eq("id", userId);
177 |         if (error) throw error;
178 |     },
179 |     onSuccess: () => {
180 |         // Cover both this feature and the sibling useNotificationPreferences cache.
181 |         qc.invalidateQueries({ queryKey: queryKeys.notificationPreferences.all });
182 |     },
183 | });
184 | };
185 |

```

FILE: [src/hooks/useCoordinatorProfileStats.ts](#)

TYPE: .ts | LINES: 1-182 | SHA256: afd62841fb77616764a7c7f9234a89ba1161b1d491893a4dfd6b963e35531d74

```

1 | // =====
2 | // useCoordinatorProfileStats – real "Programs I Manage" + workspace counts
3 | // =====
4 | //
5 | // Aggregate read hook (UI prototype migration, Phase A backend wiring) that
6 | // powers the coordinator "Me" page header tiles and the "Programs I Manage"
7 | // list with REAL data. Reads existing tables only (no new backend, no writes):
8 | //   • programs       – scoped to the signed-in coordinator via `coordinator_id`
9 | //   • courses        – the courses inside those programs (program_id, teacher_id)
10 | //   • student_courses – enrolments used for the distinct student count
11 | //   • profiles       – is_active flags for the connected faculty split
12 | //
13 | // RLS already scopes each read to the caller's institution/programs
14 | // (`programs_institution_read`, `courses_institution_read`,
15 | // `student_courses_admin_read` for the coordinator role), so filtering by
16 | // `coordinator_id` narrows the institution-visible rows down to the ones this
17 | // coordinator owns. All aggregation is client-side over small, projected
18 | // payloads (single batched `.in(...)` queries – no N+1).
19 | // =====
20 |
21 | import { useQuery } from "@tanstack/react-query";
22 | import { supabase } from "@lib/supabase";
23 | import { queryKeys } from "@lib/queryKeys";
24 |
25 | export interface CoordinatorProgramSummary {
26 |   id: string;
27 |   code: string;
28 |   name: string;
29 |   courseCount: number;
30 |   studentCount: number;
31 | }
32 |
33 | export interface CoordinatorProfileStats {
34 |   programs: CoordinatorProgramSummary[];
35 |   totals: {
36 |     programs: number;
37 |     courses: number;
38 |     students: number;
39 |     faculty: number;
40 |     facultyActive: number;
41 |     facultyInactive: number;
42 |   };
43 | }
44 |
45 | interface ProgramRow {
46 |   id: string;
47 |   code: string;
48 |   name: string;
49 | }
50 | interface CourseRow {
51 |   id: string;
52 |   program_id: string;
53 |   teacher_id: string | null;
54 | }
55 | interface EnrollRow {
56 |   student_id: string;
57 |   course_id: string;
58 | }
59 | interface FacultyRow {
60 |   id: string;
61 |   is_active: boolean | null;
62 | }
63 |
64 | const EMPTY: CoordinatorProfileStats = {
65 |   programs: [],
66 |   totals: {
67 |     programs: 0,
68 |     courses: 0,
69 |     students: 0,
70 |     faculty: 0,
71 |     facultyActive: 0,
72 |     facultyInactive: 0,
73 |   },
74 | };
75 |
76 | export const useCoordinatorProfileStats = (coordinatorId?: string | null) => {
77 |   return useQuery({
78 |     queryKey: queryKeys.coordinatorDashboard.list({
79 |       view: "profileStats",
80 |       coordinatorId: coordinatorId ?? null,
81 |     }),
82 |     enabled: !!coordinatorId,
83 |     queryFn: async (): Promise<CoordinatorProfileStats> => {
84 |       // 1. Programs coordinated by this user.
85 |       const { data: programsData, error: programsError } = await supabase

```

```

86 |         .from("programs")
87 |         .select("id, code, name")
88 |         .eq("coordinator_id", coordinatorId!)
89 |         .order("code", { ascending: true });
90 | if (programsError) throw programsError;
91 | const programs = (programsData ?? []) as ProgramRow[];
92 | if (programs.length === 0) return EMPTY;
93 |
94 | const programIds = programs.map((p) => p.id);
95 |
96 | // 2. Courses within those programs.
97 | const { data: coursesData, error: coursesError } = await supabase
98 |     .from("courses")
99 |     .select("id, program_id, teacher_id")
100 |     .in("program_id", programIds);
101 | if (coursesError) throw coursesError;
102 | const courses = (coursesData ?? []) as CourseRow[];
103 | const courseIds = courses.map((c) => c.id);
104 | const courseToProgram = new Map<string, string>({
105 |     courses.map((c) => [c.id, c.program_id])
106 | });
107 |
108 | // 3. Enrolments within those courses (distinct students).
109 | let enrollments: EnrollRow[] = [];
110 | if (courseIds.length > 0) {
111 |     const { data: enrollData, error: enrollError } = await supabase
112 |         .from("student_courses")
113 |         .select("student_id, course_id")
114 |         .in("course_id", courseIds);
115 |     if (enrollError) throw enrollError;
116 |     enrollments = (enrollData ?? []) as EnrollRow[];
117 | }
118 |
119 | // 4. Faculty active/inactive split (batched, no N+1).
120 | const facultyIds = Array.from(
121 |     new Set(
122 |         courses.map((c) => c.teacher_id).filter((id): id is string => !!id)
123 |     )
124 | );
125 | let facultyActive = 0;
126 | let facultyInactive = 0;
127 | if (facultyIds.length > 0) {
128 |     const { data: facultyData, error: facultyError } = await supabase
129 |         .from("profiles")
130 |         .select("id, is_active")
131 |         .in("id", facultyIds);
132 |     if (facultyError) throw facultyError;
133 |     for (const f of (facultyData ?? []) as FacultyRow[]) {
134 |         if (f.is_active === false) facultyInactive += 1;
135 |         else facultyActive += 1;
136 |     }
137 | }
138 |
139 | // Per-program course counts + distinct student sets.
140 | const courseCountByProgram = new Map<string, number>();
141 | for (const c of courses) {
142 |     courseCountByProgram.set(
143 |         c.program_id,
144 |         (courseCountByProgram.get(c.program_id) ?? 0) + 1
145 |     );
146 | }
147 | const studentsByProgram = new Map<string, Set<string>>();
148 | const allStudents = new Set<string>();
149 | for (const e of enrollments) {
150 |     allStudents.add(e.student_id);
151 |     const pid = courseToProgram.get(e.course_id);
152 |     if (!pid) continue;
153 |     const set = studentsByProgram.get(pid) ?? new Set<string>();
154 |     set.add(e.student_id);
155 |     studentsByProgram.set(pid, set);
156 | }
157 |
158 | const programSummaries: CoordinatorProgramSummary[] = programs.map(
159 |     (p) => ({
160 |         id: p.id,
161 |         code: p.code,
162 |         name: p.name,
163 |         courseCount: courseCountByProgram.get(p.id) ?? 0,
164 |         studentCount: studentsByProgram.get(p.id)?.size ?? 0,
165 |     })
166 | );
167 |
168 | return {
169 |     programs: programSummaries,
170 |     totals: {
171 |         programs: programs.length,
172 |         courses: courses.length,
173 |         students: allStudents.size,
174 |         faculty: facultyIds.length,
175 |         facultyActive,
176 |         facultyInactive,
177 |     },
178 | };
179 | },
180 | });
181 | };
182 |

```



FILE: src/hooks/useCourseFile.ts

```
TYPE: .ts | LINES: 1-46 | SHA256: 7c4f094ca2777dc172cd655c6969d280cf17f505c4ced81d61af66fc1e5ff706
```

[illegible]

FILE: src/hooks/useCourseModules.ts

```
TYPE: .ts | LINES: 1-538 | SHA256: 40de43022dd322091076ce4d4f7428817a88816df089db1efbfe968738501713
```

[illegible]

[illegible]

```

155 |         })
156 |         .select()
157 |         .single();
158 |     if (error) throw error;
159 |
160 |     const currentUserId = (await supabase.auth.getUser()).data.user?.id ?? performedBy;
161 |     await logAuditEvent({
162 |       action: "create",
163 |       entity_type: "course_module",
164 |       entity_id: data.id,
165 |       changes: {
166 |         course_id: payload.course_id,
167 |         title: payload.title,
168 |         sort_order: payload.sort_order,
169 |         is_published: payload.is_published,
170 |       },
171 |       performed_by: currentUserId,
172 |     }).catch((err: unknown) => console.error("Audit log error:", err));
173 |
174 |     return castModule(data as unknown as Record<string, unknown>);
175 |   },
176 |   onSuccess: (_data, variables) => {
177 |     queryClient.invalidateQueries({
178 |       queryKey: queryKeys.courseModules.list({
179 |         courseId: variables.course_id,
180 |       }),
181 |     });
182 |   },
183 | });
184 | };
185 |
186 | export const useUpdateModule = () => {
187 |   const queryClient = useQueryClient();
188 |   return useMutation({
189 |     mutationFn: async (input: UpdateModuleInput & { performedBy: string }) => {
190 |       const { id, performedBy, ...updates } = input;
191 |       const { data, error } = await supabase
192 |         .from("course_modules")
193 |         .update(updates)
194 |         .eq("id", id)
195 |         .select()
196 |         .single();
197 |       if (error) throw error;
198 |
199 |       const currentUserId = (await supabase.auth.getUser()).data.user?.id ?? performedBy;
200 |       await logAuditEvent({
201 |         action: "update",
202 |         entity_type: "course_module",
203 |         entity_id: id,
204 |         changes: updates,
205 |         performed_by: currentUserId,
206 |       }).catch((err: unknown) => console.error("Audit log error:", err));
207 |
208 |       return castModule(data as unknown as Record<string, unknown>);
209 |     },
210 |     onSuccess: () => {
211 |       queryClient.invalidateQueries({
212 |         queryKey: queryKeys.courseModules.lists(),
213 |       });
214 |     },
215 |   });
216 | };
217 |
218 | export const useDeleteModule = () => {
219 |   const queryClient = useQueryClient();
220 |   return useMutation({
221 |     mutationFn: async ({
222 |       id,
223 |       performedBy,
224 |     }: {
225 |       id: string;
226 |       performedBy: string;
227 |     }) => {
228 |       const { data: childMaterials } = await supabase
229 |         .from("course_materials")
230 |         .select("id, file_path")
231 |         .eq("module_id", id);
232 |
233 |       if (childMaterials && childMaterials.length > 0) {
234 |         const filePaths = childMaterials
235 |           .map((m) => m.file_path)
236 |           .filter((p): p is string => !!p);
237 |
238 |         if (filePaths.length > 0) {
239 |           await supabase.storage
240 |             .from(MATERIAL_BUCKET)
241 |             .remove(filePaths)
242 |             .catch((err: unknown) => console.error("Storage child removal error:", err));
243 |         }
244 |
245 |         await supabase.from("course_materials").delete().eq("module_id", id);
246 |       }
247 |
248 |       const { error } = await supabase
249 |         .from("course_modules")
250 |         .delete()
251 |         .eq("id", id);
252 |       if (error) throw error;
253 |
254 |       const currentUserId = (await supabase.auth.getUser()).data.user?.id ?? performedBy;
255 |       await logAuditEvent({
256 |         action: "delete",
257 |         entity_type: "course_module",
258 |         entity_id: id,

```

[illegible]

```

363 |         courseId: payload.course_id,
364 |         sourceMaterialId: data.id,
365 |         cloIds: payload.clo_ids ?? [],
366 |     }).catch((err: unknown) => console.error("Background indexing error:", err));
367 | }
368 |
369 |     return castMaterial(data as unknown as Record<string, unknown>);
370 | },
371 | onSuccess: (_data, variables) => {
372 |     queryClient.invalidateQueries({
373 |         queryKey: queryKeys.courseMaterials.list({
374 |             moduleId: variables.module_id,
375 |         }),
376 |     });
377 |     queryClient.invalidateQueries({
378 |         queryKey: queryKeys.courseMaterials.lists(),
379 |     });
380 | },
381 | });
382 | };
383 |
384 | export const useUpdateMaterial = () => {
385 |     const queryClient = useQueryClient();
386 |     return useMutation({
387 |         mutationFn: async (
388 |             input: UpdateMaterialInput & { performedBy: string }
389 |         ) => {
390 |             const { id, performedBy, ...updates } = input;
391 |             const { clo_ids, course_id: courseId, ...rest } = updates;
392 |             const updatePayload: Database["public"]["Tables"]["course_materials"]["Update"] =
393 |                 {
394 |                     ...rest,
395 |                     ...(clo_ids !== undefined
396 |                         ? { clo_ids: clo_ids as unknown as Json }
397 |                         : {}),
398 |                 };
399 |             const { data, error } = await supabase
400 |                 .from("course_materials")
401 |                 .update(updatePayload)
402 |                 .eq("id", id)
403 |                 .select()
404 |                 .single();
405 |             if (error) throw error;
406 |
407 |             const currentUserId = (await supabase.auth.getUser()).data.user?.id ?? performedBy;
408 |             await logAuditEvent({
409 |                 action: "update",
410 |                 entity_type: "course_material",
411 |                 entity_id: id,
412 |                 changes: updates,
413 |                 performed_by: currentUserId,
414 |             }).catch((err: unknown) => console.error("Audit log error:", err));
415 |
416 |             if (
417 |                 (updates.type === "file" || data.type === "file") &&
418 |                 (updates.file_path || data.file_path) &&
419 |                 (courseId || data.module_id)
420 |             ) {
421 |                 indexCourseMaterialIfSupported({
422 |                     filePath: updates.file_path ?? data.file_path ?? "",
423 |                     courseId: courseId ?? "",
424 |                     sourceMaterialId: id,
425 |                     cloIds: updates.clo_ids,
426 |                     reindex: true,
427 |                 }).catch((err: unknown) => console.error("Background re-indexing error:", err));
428 |             }
429 |
430 |             return castMaterial(data as unknown as Record<string, unknown>);
431 |         },
432 |         onSuccess: () => {
433 |             queryClient.invalidateQueries({
434 |                 queryKey: queryKeys.courseMaterials.lists(),
435 |             });
436 |         },
437 |     });
438 | };
439 |
440 | export const useDeleteMaterial = () => {
441 |     const queryClient = useQueryClient();
442 |     return useMutation({
443 |         mutationFn: async ({
444 |             id,
445 |             performedBy,
446 |         }: {
447 |             id: string;
448 |             performedBy: string;
449 |         }) => {
450 |             const { data: mat } = await supabase
451 |                 .from("course_materials")
452 |                 .select("file_path")
453 |                 .eq("id", id)
454 |                 .single();
455 |
456 |             if (mat?.file_path) {
457 |                 await supabase.storage
458 |                     .from(MATERIAL_BUCKET)
459 |                     .remove([mat.file_path])
460 |                     .catch((err: unknown) => console.error("Storage removal error:", err));
461 |             }
462 |
463 |             const { error } = await supabase
464 |                 .from("course_materials")
465 |                 .delete()
466 |                 .eq("id", id);

```

[illegible]

FILE: src/hooks/useCourses.ts

TYPE: .ts | LINES: 1-252 | SHA256: c66ec6ce958974b4b8b4fc60969d0590b6cb379ac46e1bef1db4a8a45dd960f6

[illegible]

[illegible]

[illegible]



```

237     queryKey: queryKeys.users.list({ role: "teacher" }},
238     queryFn: async (): Promise<Profile[]> => {
239       const { data, error } = await supabase
240         .from("profiles")
241         .select("*")
242         .eq("role", "teacher")
243         .eq("is_active", true)
244         .order("full_name", { ascending: true });
245
246       if (error) throw error;
247       return data as Profile[];
248     },
249     staleTime: 30_000,
250   });
251 };
252

```

FILE: src/hooks/useCourseSections.ts

```
TYPE: .ts | LINES: 1-220 | SHA256: 387de0d7729a5da2952a70acd1346bf0cf4f78b80e62408c49994d57f7dfd514
```

[illegible]

```

85 |
86 | return useMutation({
87 |   mutationFn: async (
88 |     input: CreateCourseSectionInput
89 |   ): Promise<CourseSection> => {
90 |     const { data, error } = await supabase
91 |       .from("course_sections")
92 |       .insert({
93 |         course_id: input.course_id,
94 |         section_code: input.section_code,
95 |         teacher_id: input.teacher_id,
96 |         capacity: input.capacity,
97 |         is_active: input.is_active ?? true,
98 |       })
99 |       .select()
100 |       .single();
101 |
102 |     if (error) throw error;
103 |
104 |     const section = data as CourseSection;
105 |
106 |     await logAuditEvent({
107 |       action: "create",
108 |       entity_type: "course_section",
109 |       entity_id: section.id,
110 |       changes: { ...input },
111 |       performed_by: user?.id ?? "unknown",
112 |     });
113 |
114 |     return section;
115 |   },
116 |   onSuccess: () => {
117 |     queryClient.invalidateQueries({
118 |       queryKey: queryKeys.courseSections.lists(),
119 |     });
120 |   },
121 | });
122 | };
123 |
124 | // % % % useUpdateCourseSection - update with audit logging % % % % % % % % % % % % % % % % % % %
125 |
126 | export const useUpdateCourseSection = (id: string) => {
127 |   const queryClient = useQueryClient();
128 |   const { user } = useAuth();
129 |
130 |   return useMutation({
131 |     mutationFn: async (
132 |       input: UpdateCourseSectionInput
133 |     ): Promise<CourseSection> => {
134 |       const { data, error } = await supabase
135 |         .from("course_sections")
136 |         .update(input)
137 |         .eq("id", id)
138 |         .select()
139 |         .single();
140 |
141 |       if (error) throw error;
142 |
143 |       await logAuditEvent({
144 |         action: "update",
145 |         entity_type: "course_section",
146 |         entity_id: id,
147 |         changes: input as Record<string, unknown>,
148 |         performed_by: user?.id ?? "unknown",
149 |       });
150 |
151 |       return data as CourseSection;
152 |     },
153 |     onSuccess: () => {
154 |       queryClient.invalidateQueries({
155 |         queryKey: queryKeys.courseSections.lists(),
156 |       });
157 |       queryClient.invalidateQueries({
158 |         queryKey: queryKeys.courseSections.detail(id),
159 |       });
160 |     },
161 |   });
162 | };
163 |
164 | // % % % useDeleteCourseSection - soft-delete by setting is_active = false % % % % %
165 |
166 | export const useDeleteCourseSection = () => {
167 |   const queryClient = useQueryClient();
168 |   const { user } = useAuth();
169 |
170 |   return useMutation({
171 |     mutationFn: async (id: string): Promise<CourseSection> => {
172 |       const { data, error } = await supabase
173 |         .from("course_sections")
174 |         .update({ is_active: false })
175 |         .eq("id", id)
176 |         .select()
177 |         .single();
178 |
179 |       if (error) throw error;
180 |
181 |       await logAuditEvent({
182 |         action: "soft_delete",
183 |         entity_type: "course_section",
184 |         entity_id: id,
185 |         changes: { is_active: false },
186 |         performed_by: user?.id ?? "unknown",
187 |       });
188 |     }

```

```

189 |         return data as CourseSection;
190 |     },
191 |     onSuccess: () => {
192 |         queryClient.invalidateQueries({
193 |             queryKey: queryKeys.courseSections.lists(),
194 |         });
195 |     },
196 | });
197 | };
198 |
199 | // % % % useSectionEnrollmentCount - count active enrollments per section % % % % % % %
200 |
201 | export const useSectionEnrollmentCount = (sectionId?: string) => {
202 |     return useQuery({
203 |         queryKey: queryKeys.courseSections.list({
204 |             sectionId,
205 |             scope: "enrollment_count",
206 |         }),
207 |         queryFn: async (): Promise<number> => {
208 |             const { count, error } = await supabase
209 |                 .from("student_courses")
210 |                 .select("*, { count: 'exact', head: true }")
211 |                 .eq("section_id", sectionId!)
212 |                 .eq("status", "active");
213 |
214 |             if (error) throw error;
215 |             return count ?? 0;
216 |         },
217 |         enabled: !!sectionId,
218 |     });
219 | };
220 |

```

FILE: [src/hooks/useCQIPlans.ts](#)  
TYPE: .ts | LINES: 1-290 | SHA256: 5020b01d26a352794cfbe3633f5d85469498ea6c16725e81ed6ac3f75255f34a

[illegible]

[illegible]

```

173 |         responsible_person: input.responsible_person,
174 |         status: "planned",
175 |     };
176 |     if (input.root_cause !== undefined)
177 |         insertPayload.root_cause = input.root_cause;
178 |     if (input.due_date !== undefined) insertPayload.due_date = input.due_date;
179 |     if (input.evidence_of_improvement !== undefined)
180 |         insertPayload.evidence_of_improvement = input.evidence_of_improvement;
181 |
182 |     const { data, error } = await supabase
183 |         .from("cqi_action_plans")
184 |         .insert(insertPayload)
185 |         .select()
186 |         .single();
187 |     if (error) throw error;
188 |     await logAuditEvent({
189 |         action: "create",
190 |         entity_type: "cqi_action_plan",
191 |         entity_id: data.id,
192 |         changes: {
193 |             program_id: input.program_id,
194 |             outcome_id: input.outcome_id,
195 |             outcome_type: input.outcome_type,
196 |             baseline_attainment: input.baseline_attainment,
197 |             target_attainment: input.target_attainment,
198 |             action_description: input.action_description,
199 |             responsible_person: input.responsible_person,
200 |             root_cause: input.root_cause ?? null,
201 |             due_date: input.due_date ?? null,
202 |             evidence_of_improvement: input.evidence_of_improvement ?? null,
203 |         },
204 |         performed_by: performedBy,
205 |     });
206 |     return data as CQIActionPlan;
207 | },
208 | onSuccess: () => {
209 |     queryClient.invalidateQueries({ queryKey: queryKeys.cqiPlans.lists() });
210 | },
211 | });
212 | };
213 |
214 | export const useUpdateCQIPlan = () => {
215 |     const queryClient = useQueryClient();
216 |     return useMutation({
217 |         mutationFn: async ({
218 |             performedBy,
219 |             ...input
220 |         }: UpdateCQIPlanInput & { performedBy: string }) => {
221 |             const payload: Database["public"]["Tables"]["cqi_action_plans"]["Update"] =
222 |                 {};
223 |             if (input.action_description !== undefined)
224 |                 payload.action_description = input.action_description;
225 |             if (input.responsible_person !== undefined)
226 |                 payload.responsible_person = input.responsible_person;
227 |             if (input.status !== undefined) payload.status = input.status;
228 |             if (input.target_attainment !== undefined)
229 |                 payload.target_attainment = input.target_attainment;
230 |             if (input.result_attainment !== undefined)
231 |                 payload.result_attainment = input.result_attainment;
232 |             if (input.root_cause !== undefined) payload.root_cause = input.root_cause;
233 |             if (input.due_date !== undefined) payload.due_date = input.due_date;
234 |             if (input.evidence_of_improvement !== undefined)
235 |                 payload.evidence_of_improvement = input.evidence_of_improvement;
236 |
237 |             const { data, error } = await supabase
238 |                 .from("cqi_action_plans")
239 |                 .update(payload)
240 |                 .eq("id", input.id)
241 |                 .select()
242 |                 .single();
243 |             if (error) throw error;
244 |             await logAuditEvent({
245 |                 action: "update",
246 |                 entity_type: "cqi_action_plan",
247 |                 entity_id: input.id,
248 |                 changes: payload,
249 |                 performed_by: performedBy,
250 |             });
251 |             return data as CQIActionPlan;
252 |         },
253 |         onSuccess: (_, data, variables) => {
254 |             queryClient.invalidateQueries({ queryKey: queryKeys.cqiPlans.lists() });
255 |             queryClient.invalidateQueries({
256 |                 queryKey: queryKeys.cqiPlans.detail(variables.id),
257 |             });
258 |         },
259 |     });
260 | };
261 |
262 | export const useDeleteCQIPlan = () => {
263 |     const queryClient = useQueryClient();
264 |     return useMutation({
265 |         mutationFn: async ({
266 |             id,
267 |             performedBy,
268 |         }: {
269 |             id: string;
270 |             performedBy: string;
271 |         }) => {
272 |             const { error } = await supabase
273 |                 .from("cqi_action_plans")
274 |                 .delete()
275 |                 .eq("id", id);
276 |             if (error) throw error;

```

```

277 |         await logAuditEvent({
278 |             action: "delete",
279 |             entity_type: "cqi_action_plan",
280 |             entity_id: id,
281 |             changes: null,
282 |             performed_by: performedBy,
283 |         });
284 |     },
285 |     onSuccess: () => {
286 |         queryClient.invalidateQueries({ queryKey: queryKeys.cqiPlans.lists() });
287 |     },
288 | });
289 | };
290 |

```

```
TYPE: .ts | LINES: 1-306 | SHA256: 56eadf634b93c6fe19f156c237311eb13752e882b8229c46d863d3043615ddd8
```

[illegible]

```

191 |
192 | const typedClos = clos ?? [];
193 |
194 | // 4. Fetch outcome_mappings: PLO (parent) != CLO (child)
195 | const cloIds = typedClos.map((c) => c.id);
196 |
197 | let typedMappings: OutcomeMappingRow[] = [];
198 |
199 | if (cloIds.length > 0) {
200 |   const { data: mappings, error: mapError } = await supabase
201 |     .from("outcome_mappings")
202 |     .select("source_outcome_id, target_outcome_id")
203 |     .in("source_outcome_id", ploIds)
204 |     .in("target_outcome_id", cloIds);
205 |
206 |   if (mapError) throw mapError;
207 |   typedMappings = mappings as OutcomeMappingRow[];
208 | }
209 |
210 | // 5. Build a lookup: cloId != courseId
211 | const cloToCourse = new Map<string, string>();
212 | for (const clo of typedClos) {
213 |   if (clo.course_id) {
214 |     cloToCourse.set(clo.id, clo.course_id);
215 |   }
216 | }
217 |
218 | // 6. Fetch real CLO attainment for all mapped CLOs (C-2 – replaces the
219 | // CLO-count placeholder). Mirrors the 'useAdminPLOHeatmap' /
220 | // 'useSectionAttainment' aggregation: CLO attainment is written at
221 | // 'student_course' scope keyed by 'outcome_id'. A single batched query
222 | // (no per-cell N+1), then average per CLO.
223 | const mappedCloIds = Array.from(
224 |   new Set(typedMappings.map((m) => m.target_outcome_id))
225 | );
226 | const cloAttainmentById = new Map<string, number>();
227 |
228 | if (mappedCloIds.length > 0) {
229 |   const { data: attRows, error: attError } = await supabase
230 |     .from("outcome_attainment")
231 |     .select("outcome_id, attainment_percent")
232 |     .eq("scope", "student_course")
233 |     .in("outcome_id", mappedCloIds);
234 |
235 |   if (attError) throw attError;
236 |
237 |   const attListsByClo = new Map<string, number[]>();
238 |   for (const row of attRows ?? []) {
239 |     if (row.attainment_percent == null) continue;
240 |     const list = attListsByClo.get(row.outcome_id) ?? [];
241 |     list.push(row.attainment_percent);
242 |     attListsByClo.set(row.outcome_id, list);
243 |   }
244 |   for (const [cloId, list] of attListsByClo) {
245 |     cloAttainmentById.set(cloId, mean(list));
246 |   }
247 | }
248 |
249 | // Pre-compute per-cell mapping counts AND attainment samples in one pass:
250 | // '${ploId}_${courseId}' != { count, attainmentSamples }
251 | // This prevents an O(P * C * M) nested loop bottleneck.
252 | interface CellAccumulator {
253 |   count: number;
254 |   attainmentSamples: number[];
255 | }
256 | const cellAccumulators = new Map<string, CellAccumulator>();
257 | for (const m of typedMappings) {
258 |   const courseId = cloToCourse.get(m.target_outcome_id);
259 |   if (!courseId) continue;
260 |   const key = `${m.source_outcome_id}_${courseId}`;
261 |   const acc = cellAccumulators.get(key) ?? {
262 |     count: 0,
263 |     attainmentSamples: [],
264 |   };
265 |   acc.count += 1;
266 |   const cloAttainment = cloAttainmentById.get(m.target_outcome_id);
267 |   if (cloAttainment !== undefined) {
268 |     acc.attainmentSamples.push(cloAttainment);
269 |   }
270 |   cellAccumulators.set(key, acc);
271 | }
272 |
273 | // 7. Compute the matrix with real attainment per cell.
274 | const matrix: Record<string, Record<string, CellData>> = {};
275 |
276 | for (const plo of typedPlos) {
277 |   const row: Record<string, CellData> = {};
278 |   for (const course of typedCourses) {
279 |     const acc = cellAccumulators.get(`${plo.id}_${course.id}`);
280 |     const cloCount = acc?.count ?? 0;
281 |     // Real attainment: mean of the contributing CLOs' attainment. When CLOs
282 |     // are mapped but no evidence exists, surface the unmeasured sentinel so
283 |     // the cell renders grey (distinct from a real 0%).
284 |     const attainmentPercent =
285 |       cloCount === 0
286 |         ? 0
287 |         : acc && acc.attainmentSamples.length > 0
288 |           ? Math.round(mean(acc.attainmentSamples))
289 |           : CELL_ATTAINMENT_UNMEASURED;
290 |
291 |     row[course.id] = {
292 |       cloCount,
293 |       coveragePercent: cloCount > 0 ? 100 : 0,
294 |       attainmentPercent,

```



```

295         status: computeCellStatus(cloCount, attainmentPercent),
296     };
297 }
298     matrix[plo.id] = row;
299 }
300
301     return { plos: typedPlos, courses: typedCourses, matrix };
302 },
303     enabled: !!programId,
304 });
305 };
306

```

FILE: src/hooks/useDataImport.ts

```
TYPE: .ts | LINES: 1-35 | SHA256: 87881ea6c7ecea53501cb4d98200801746aaf8a7050c2ef071ad6e016c33d269
```

```

1 // Task 108.4: Data import TanStack Query hooks
2
3 import { useMutation } from "@tanstack/react-query";
4 import { supabase } from "@/lib/supabase";
5 import { useAuth } from "@/hooks/useAuth";
6
7 export interface ImportRequest {
8   import_type: string;
9   csv_content: string;
10 }
11
12 export interface ImportResult {
13   total_rows: number;
14   imported: number;
15   skipped: number;
16   errors: Array<{ row: number; message: string }>;
17 }
18
19 export const useDataImport = () => {
20   const { user } = useAuth();
21
22   return useMutation({
23     mutationFn: async (req: ImportRequest): Promise<ImportResult> => {
24       const { data, error } = await supabase.functions.invoke(
25         "bulk-data-import",
26         {
27           body: { ...req, performed_by: user?.id },
28         }
29       );
30       if (error) throw error;
31       return data as ImportResult;
32     },
33   });
34 };
35

```

FILE: src/hooks/useDeadlineExtensions.ts

TYPE: .ts | LINES: 1-192 | SHA256: 43759134e71a42bdc3df15be9f7b503b8d18509c3ef2161302317d0b698c20c5

[illegible]

```

50 | // Insert deadline extension record
51 | // eslint-disable-next-line @typescript-eslint/no-explicit-any
52 | const { error: insertError } = await (supabase as any)
53 |   .from("deadline_extensions")
54 |   .insert({
55 |     student_id: user.id,
56 |     assignment_id: input.assignmentId,
57 |     purchase_id: input.purchaseId,
58 |     original_deadline: originalDeadline.toISOString(),
59 |     extended_deadline: extendedDeadline.toISOString(),
60 |   });
61 |
62 | if (insertError) {
63 |   if (insertError.code === "23505") {
64 |     throw new Error("You already have an extension for this assignment.");
65 |   }
66 |   throw insertError;
67 | }
68 |
69 | // Mark the purchase as consumed
70 | // eslint-disable-next-line @typescript-eslint/no-explicit-any
71 | await (supabase as any)
72 |   .from("xp_purchases")
73 |   .update({ status: "consumed", consumed_at: new Date().toISOString() })
74 |   .eq("id", input.purchaseId);
75 |
76 | // Send teacher notification about the deadline extension
77 | const courseId = (assignment as Record<string, unknown>)
78 |   .course_id as string;
79 | const assignmentTitle = (assignment as Record<string, unknown>)
80 |   .title as string;
81 |
82 | // Fetch the course teacher
83 | const { data: course } = await supabase
84 |   .from("courses")
85 |   .select("teacher_id")
86 |   .eq("id", courseId)
87 |   .maybeSingle();
88 |
89 | if (course) {
90 |   const teacherId = (course as Record<string, unknown>).teacher_id as
91 |     | string
92 |     | null;
93 |   if (teacherId) {
94 |     // Fetch student name for notification
95 |     const { data: studentProfile } = await supabase
96 |       .from("profiles")
97 |       .select("full_name")
98 |       .eq("id", user.id)
99 |       .maybeSingle();
100 |
101 |     const studentName =
102 |       ((studentProfile as Record<string, unknown> | null)
103 |         ?.full_name as string) ?? "A student";
104 |
105 |     // Insert notification for teacher
106 |     // eslint-disable-next-line @typescript-eslint/no-explicit-any
107 |     const { error: notificationError } = await (supabase as any)
108 |       .from("notifications")
109 |       .insert({
110 |         user_id: teacherId,
111 |         type: "deadline_extension",
112 |         title: "Deadline Extension Activated",
113 |         body: `${studentName} used a deadline extension token on "${assignmentTitle}". New deadline: ${e
114 |           .extended_deadline.toISOString().replace(/:/g, '')}
115 |         metadata: {
116 |           student_id: user.id,
117 |           assignment_id: input.assignmentId,
118 |           original_deadline: originalDeadline.toISOString(),
119 |           extended_deadline: extendedDeadline.toISOString(),
120 |         },
121 |       });
122 |
123 |     if (notificationError) {
124 |       console.error("Failed to insert notification:", notificationError);
125 |     }
126 |   }
127 | },
128 | onSuccess: () => {
129 |   if (user?.id) {
130 |     queryClient.invalidateQueries({
131 |       queryKey: queryKeys.marketplace.inventory(user.id),
132 |     });
133 |   }
134 |   queryClient.invalidateQueries({
135 |     queryKey: queryKeys.assignments.lists(),
136 |   });
137 |   toast.success("Deadline extended by 24 hours!");
138 | },
139 | onError: (error: Error) => {
140 |   toast.error(error.message || "Failed to activate deadline extension");
141 | },
142 | });
143 | };
144 |
145 | // % % % useRevokeDeadlineExtension – teacher revokes a student's extension % % % % %
146 |
147 | export const useRevokeDeadlineExtension = () => {
148 |   const queryClient = useQueryClient();
149 |   const { user } = useAuth();
150 |
151 |   return useMutation({
152 |     mutationFn: async (extensionId: string): Promise<void> => {
153 |       if (!user) throw new Error("Not authenticated");

```

```

154 |
155 | // eslint-disable-next-line @typescript-eslint/no-explicit-any
156 | const { data: extension, error: fetchError } = await (supabase as any)
157 |   .from("deadline_extensions")
158 |   .select("id, purchase_id, student_id")
159 |   .eq("id", extensionId)
160 |   .maybeSingle();
161 |
162 | if (fetchError) throw fetchError;
163 | if (!extension) throw new Error("Extension not found");
164 |
165 | // Mark extension as revoked
166 | // eslint-disable-next-line @typescript-eslint/no-explicit-any
167 | const { error: updateError } = await (supabase as any)
168 |   .from("deadline_extensions")
169 |   .update({ revoked: true, revoked_by: user.id })
170 |   .eq("id", extensionId);
171 |
172 | if (updateError) throw updateError;
173 |
174 | // Refund the purchase token
175 | // eslint-disable-next-line @typescript-eslint/no-explicit-any
176 | await (supabase as any)
177 |   .from("xp_purchases")
178 |   .update({ status: "active", consumed_at: null })
179 |   .eq("id", extension.purchase_id);
180 | },
181 | onSuccess: () => {
182 |   queryClient.invalidateQueries({
183 |     queryKey: queryKeys.assignments.lists(),
184 |   });
185 |   toast.success("Deadline extension revoked and token refunded.");
186 | },
187 | onError: (error: Error) => {
188 |   toast.error(error.message || "Failed to revoke extension");
189 | },
190 | });
191 | };
192 |

```

### FILE: src/hooks/useDebounce.ts

TYPE: .ts | LINES: 1-23 | SHA256: 17bf5a1a155f9ba82a2aad7e1234f57e016cebc369c75ddf619bfd0d17f8f2c4

```

1 | import { useEffect, useState } from "react";
2 |
3 | /**
4 |  * Debounce a changing value. The returned value updates only after `delayMs`
5 |  * has elapsed without further changes to `value`.
6 |  *
7 |  * Lives in `src/hooks/` per project conventions – all reusable hooks belong
8 |  * here so they can be imported with the `@/hooks/` alias from anywhere.
9 |  *
10 |  * @param value The latest value to debounce.
11 |  * @param delayMs Milliseconds to wait before publishing the latest value.
12 |  */
13 | export function useDebounce<T>(value: T, delayMs: number): T {
14 |   const [debounced, setDebounce] = useState(value);
15 |
16 |   useEffect(() => {
17 |     const handle = setTimeout(() => setDebounce(value), delayMs);
18 |     return () => clearTimeout(handle);
19 |   }, [value, delayMs]);
20 |
21 |   return debounced;
22 | }
23 |

```

### FILE: src/hooks/useDeferredMount.ts

TYPE: .ts | LINES: 1-46 | SHA256: 800bb4934891ca60081b6cafd9eb0fb7131cf99f40a7d90c39fab89531bab8c5

```

1 | import { useEffect, useState } from "react";
2 |
3 | /**
4 |  * Returns true after `delayMs` milliseconds have passed since mount.
5 |  * Use this to gate non-critical TanStack Query hooks behind first paint,
6 |  * so above-the-fold content renders quickly and below-the-fold hooks
7 |  * fire after the user can already see something.
8 |  *
9 |  * Pattern:
10 |  *   const ready = useDeferredMount(500);
11 |  *   const { data } = useExpensiveHook(id, { enabled: ready });
12 |  *
13 |  * SSR-safe (returns false during server render).
14 |  *
15 |  * The default 500ms is chosen to:
16 |  * - Give the browser time to paint the critical KPIs
17 |  * - Stay well under the 1s threshold where users notice "loading still going"
18 |  * - Match React 18's automatic batching window
19 |  */
20 | export const useDeferredMount = (delayMs = 500): boolean => {
21 |   const [ready, setReady] = useState(false);
22 |
23 |   useEffect(() => {
24 |     // Use requestIdleCallback when available so we don't block the main thread
25 |     const ric =
26 |       typeof window !== "undefined" && "requestIdleCallback" in window
27 |       ? window.requestIdleCallback
28 |       : null;
29 |
30 |     if (ric) {
31 |       const handle = ric(() => setReady(true), { timeout: delayMs });
32 |       return () => {
33 |         if ("cancelIdleCallback" in window) {
34 |           window.cancelIdleCallback(handle);

```

```

35 |     }
36 |   };
37 | }
38 |
39 | // Fallback to setTimeout
40 | const timer = setTimeout(() => setReady(true), delayMs);
41 | return () => clearTimeout(timer);
42 | }, [delayMs]);
43 |
44 | return ready;
45 | };
46 |

```

## FILE: src/hooks/useDepartments.ts

TYPE: .ts | LINES: 1-154 | SHA256: a3012507becaa9a63cf722c504a9ed1ba2e7553cf534191fedd6b65eb8647022

```

1 | // Task 60: Department management hooks
2 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
3 | import { supabase } from "@/lib/supabase";
4 | import { queryKeys } from "@/lib/queryKeys";
5 | import { logAuditEvent } from "@/lib/auditLogger";
6 | import { useAuth } from "@/hooks/useAuth";
7 | import { toast } from "sonner";
8 |
9 | export interface Department {
10 |   id: string;
11 |   name: string;
12 |   code: string;
13 |   head_of_department_id: string | null;
14 |   institution_id: string;
15 |   created_at: string;
16 | }
17 |
18 | export const useDepartments = () => {
19 |   return useQuery({
20 |     queryKey: queryKeys.departments.lists(),
21 |     queryFn: async (): Promise<Department[]> => {
22 |       const { data, error } = await supabase
23 |         .from("departments")
24 |         .select("*")
25 |         .order("name", { ascending: true });
26 |       if (error) throw error;
27 |       return (data ?? []) as Department[];
28 |     },
29 |   });
30 | };
31 |
32 | export const useDepartment = (id: string | undefined) => {
33 |   return useQuery({
34 |     queryKey: queryKeys.departments.detail(id ?? ""),
35 |     queryFn: async (): Promise<Department | null> => {
36 |       const { data, error } = await supabase
37 |         .from("departments")
38 |         .select("*")
39 |         .eq("id", id!)
40 |         .maybeSingle();
41 |       if (error) throw error;
42 |       return data as Department | null;
43 |     },
44 |     enabled: !!id,
45 |   });
46 | };
47 |
48 | export const useCreateDepartment = () => {
49 |   const qc = useQueryClient();
50 |   const { user } = useAuth();
51 |   return useMutation({
52 |     mutationFn: async (input: {
53 |       name: string;
54 |       code: string;
55 |       institution_id: string;
56 |       head_of_department_id?: string;
57 |     }) => {
58 |       const { data, error } = await supabase
59 |         .from("departments")
60 |         .insert(input)
61 |         .select()
62 |         .single();
63 |       if (error) throw error;
64 |       await logAuditEvent({
65 |         action: "create",
66 |         entity_type: "department",
67 |         entity_id: data.id,
68 |         changes: input,
69 |         performed_by: user?.id ?? "unknown",
70 |       });
71 |       return data;
72 |     },
73 |     onSuccess: () => {
74 |       qc.invalidateQueries({ queryKey: queryKeys.departments.all });
75 |       toast.success("Department created");
76 |     },
77 |     onError: (err) => {
78 |       toast.error(
79 |         err instanceof Error ? err.message : "Failed to create department"
80 |       ),
81 |     },
82 |   });
83 | };
84 |
85 | export const useUpdateDepartment = (id: string) => {
86 |   const qc = useQueryClient();
87 |   const { user } = useAuth();
88 |   return useMutation({
89 |     mutationFn: async (input: {

```

```

89:   name?: string;
90:   code?: string;
91:   head_of_department_id?: string | null;
92: }) => {
93:   const { data, error } = await supabase
94:     .from("departments")
95:     .update(input)
96:     .eq("id", id)
97:     .select()
98:     .single();
99:   if (error) throw error;
100:   await logAuditEvent({
101:     action: "update",
102:     entity_type: "department",
103:     entity_id: id,
104:     changes: input,
105:     performed_by: user?.id ?? "unknown",
106:   });
107:   return data;
108: },
109: onSuccess: () => {
110:   qc.invalidateQueries({ queryKey: queryKeys.departments.all });
111:   qc.invalidateQueries({ queryKey: queryKeys.departments.detail(id) });
112:   toast.success("Department updated");
113: },
114: onError: (err) =>
115:   toast.error(
116:     err instanceof Error ? err.message : "Failed to update department"
117:   ),
118: });
119: };
120:
121: export const useDeleteDepartment = () => {
122:   const qc = useQueryClient();
123:   const { user } = useAuth();
124:   return useMutation({
125:     mutationFn: async (id: string) => {
126:       // Atomic delete: only succeeds if no programs reference this department.
127:       // Uses a single RPC call to avoid TOCTOU race between check and delete.
128:       const { data, error } = await supabase.rpc(
129:         "delete_department_if_no_programs" as never,
130:         { dept_id: id } as never
131:       );
132:       if (error)
133:         throw new Error(
134:           `Delete failed: ${error as { message: string } }.message`
135:         );
136:       if (data === false)
137:         throw new Error("Cannot delete department with active programs");
138:       await logAuditEvent({
139:         action: "delete",
140:         entity_type: "department",
141:         entity_id: id,
142:         changes: null,
143:         performed_by: user?.id ?? "unknown",
144:       });
145:     },
146:     onSuccess: () => {
147:       qc.invalidateQueries({ queryKey: queryKeys.departments.all });
148:       toast.success("Department deleted");
149:     },
150:     onError: (err) =>
151:       toast.error(err instanceof Error ? err.message : "Delete failed"),
152:   });
153: };
154:

```

FILE: src/hooks/useDiscussions.ts

TYPE: .ts | LINES: 1-410 | SHA256: b1acbe6561c07a08b9403c0b744b6fa0ffa51a24f66deac0f67bd11fd32235a2

[illegible]

[illegible]

[illegible]

[illegible]



```

347 | };
348 |
349 | export const useDeleteThread = () => {
350 |   const queryClient = useQueryClient();
351 |   return useMutation({
352 |     mutationFn: async ({
353 |       threadId,
354 |       performedBy,
355 |     }: {
356 |       threadId: string;
357 |       performedBy: string;
358 |     }) => {
359 |       const { error } = await supabase
360 |         .from("discussion_threads")
361 |         .delete()
362 |         .eq("id", threadId);
363 |       if (error) throw error;
364 |       await logAuditEvent({
365 |         action: "delete",
366 |         entity_type: "discussion_thread",
367 |         entity_id: threadId,
368 |         changes: null,
369 |         performed_by: performedBy,
370 |       });
371 |     },
372 |     onSuccess: () => {
373 |       queryClient.invalidateQueries({
374 |         queryKey: queryKeys.discussionThreads.lists(),
375 |       });
376 |     },
377 |   });
378 | };
379 |
380 | export const useDeleteReply = () => {
381 |   const queryClient = useQueryClient();
382 |   return useMutation({
383 |     mutationFn: async (input: {
384 |       replyId: string;
385 |       threadId: string;
386 |       performedBy: string;
387 |     }) => {
388 |       const { error } = await supabase
389 |         .from("discussion_replies")
390 |         .delete()
391 |         .eq("id", input.replyId);
392 |       if (error) throw error;
393 |       await logAuditEvent({
394 |         action: "delete",
395 |         entity_type: "discussion_reply",
396 |         entity_id: input.replyId,
397 |         changes: null,
398 |         performed_by: input.performedBy,
399 |       });
400 |     },
401 |     onSuccess: (_data, variables) => {
402 |       queryClient.invalidateQueries({
403 |         queryKey: queryKeys.discussionReplies.list({
404 |           threadId: variables.threadId,
405 |         }),
406 |       });
407 |     },
408 |   });
409 | };
410 |

```

## FILE: src/hooks/useDynamicPricing.ts

TYPE: .ts | LINES: 1-67 | SHA256: 71daa0f35f6a46d6dadade8db242b0f15464f1a3aa7049f93bbcf681e97ed687

```

1 | // =====
2 | // useDynamicPricing - Dynamic price display, admin toggle mutation
3 | // Task 20.12
4 | // =====
5 |
6 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
7 | import { supabase } from "@/lib/supabase";
8 | import { queryKeys } from "@/lib/queryKeys";
9 | import { toast } from "sonner";
10 |
11 | export interface DynamicPriceEntry {
12 |   itemId: string;
13 |   itemName: string;
14 |   basePrice: number;
15 |   dynamicPrice: number | null;
16 |   demandLevel: "low" | "normal" | "high";
17 | }
18 |
19 | export const useDynamicPrices = (institutionId?: string) => {
20 |   return useQuery({
21 |     queryKey: queryKeys.dynamicPricing.list({ institutionId }),
22 |     queryFn: async (): Promise<DynamicPriceEntry[]> => {
23 |       const { data, error } = await supabase
24 |         .from("marketplace_items")
25 |         .select("id, name, xp_price, dynamic_price_override")
26 |         .eq("is_active", true)
27 |         .order("name");
28 |       if (error) throw error;
29 |
30 |       return (data ?? []).map((item) => ({
31 |         itemId: item.id,
32 |         itemName: item.name,
33 |         basePrice: item.xp_price,
34 |         dynamicPrice: (item as Record<string, unknown>)
35 |           .dynamic_price_override as number | null,
36 |         demandLevel: "normal" as const,

```

```

37 |     }));
38 |   },
39 |   enabled: !!institutionId,
40 | });
41 | };
42 |
43 | export const useToggleDynamicPricing = () => {
44 |   const qc = useQueryClient();
45 |   return useMutation({
46 |     mutationFn: async ({
47 |       institutionId,
48 |       enabled,
49 |     }: {
50 |       institutionId: string;
51 |       enabled: boolean;
52 |     }) => {
53 |       const { error } = await supabase
54 |         .from("institution_settings")
55 |         .update({ dynamic_pricing_enabled: enabled } as never)
56 |         .eq("institution_id", institutionId);
57 |       if (error) throw error;
58 |     },
59 |     onSuccess: (_, { enabled }) => {
60 |       qc.invalidateQueries({ queryKey: queryKeys.dynamicPricing.all });
61 |       qc.invalidateQueries({ queryKey: queryKeys.institutionSettings.all });
62 |       toast.success(`Dynamic pricing ${enabled ? "enabled" : "disabled"}`);
63 |     },
64 |     onError: (err) => toast.error((err as Error).message),
65 |   });
66 | };
67 |

```

## FILE: src/hooks/useEmailPreferences.ts

TYPE: .ts | LINES: 1-67 | SHA256: f2ffbe2c4ce894d6ec7e06ece5f574f2b6538c0d06ea4e0955a31f908705cf71

```

1 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { useAuth } from "@hooks/useAuth";
5 | import type { EmailPreferencesFormData } from "@lib/schemas/emailPrefs";
6 |
7 | const DEFAULT_PREFERENCES: EmailPreferencesFormData = {
8 |   streak_risk: true,
9 |   weekly_summary: true,
10 |   new_assignment: true,
11 |   grade_released: true,
12 |   notification_digest: false,
13 | };
14 |
15 | export const useEmailPreferences = () => {
16 |   const { user } = useAuth();
17 |   const queryClient = useQueryClient();
18 |
19 |   const query = useQuery({
20 |     queryKey: queryKeys.emailPreferences.detail(user?.id ?? ""),
21 |     queryFn: async () => {
22 |       if (!user) throw new Error("Not authenticated");
23 |
24 |       const { data, error } = await supabase
25 |         .from("profiles")
26 |         .select("email_preferences" as never)
27 |         .eq("id", user.id)
28 |         .maybeSingle();
29 |
30 |       if (error) throw error;
31 |
32 |       const raw = (data as Record<string, unknown> | null)
33 |         ?.email_preferences as Partial<EmailPreferencesFormData> | null;
34 |       return { ...DEFAULT_PREFERENCES, ...raw };
35 |     },
36 |     enabled: !!user,
37 |   });
38 |
39 |   const mutation = useMutation({
40 |     mutationFn: async (prefs: EmailPreferencesFormData) => {
41 |       if (!user) throw new Error("Not authenticated");
42 |
43 |       const { error } = await supabase
44 |         .from("profiles")
45 |         .update({ email_preferences: prefs } as never)
46 |         .eq("id", user.id);
47 |
48 |       if (error) throw error;
49 |       return prefs;
50 |     },
51 |     onSuccess: () => {
52 |       queryClient.invalidateQueries({
53 |         queryKey: queryKeys.emailPreferences.detail(user?.id ?? ""),
54 |       });
55 |     },
56 |   });
57 |
58 |   return {
59 |     preferences: query.data ?? DEFAULT_PREFERENCES,
60 |     isLoading: query.isLoading,
61 |     isError: query.isError,
62 |     updatePreferences: mutation.mutate,
63 |     updatePreferencesAsync: mutation.mutateAsync,
64 |     isUpdating: mutation.isPending,
65 |   };
66 | };
67 |

```

```
TYPE: .ts | LINES: 1-191 | SHA256: dd7359a423b57f6acbba1a018c9105e9d44b657bd6e61292de3f9fc5f9e219d
```

[illegible]

```

102 mutationFn: async (input: EnrollStudentInput): Promise<Enrollment> => {
103   const { data, error } = await supabase
104     .from("student_courses")
105     .insert({
106       student_id: input.student_id,
107       course_id: input.course_id,
108       section_id: input.section_id ?? null,
109     })
110     .select()
111     .single();
112
113   if (error) throw error;
114
115   const enrollment = data as Enrollment;
116
117   await logAuditEvent({
118     action: "create",
119     entity_type: "enrollment",
120     entity_id: enrollment.id,
121     changes: { ...input },
122     performed_by: user?.id ?? "unknown",
123   });
124
125   return enrollment;
126 },
127 onSuccess: () => {
128   queryClient.invalidateQueries({
129     queryKey: queryKeys.enrollments.lists(),
130   });
131 },
132 });
133 };
134
135 // % % % useUnenrollStudent - soft-delete by setting status to 'dropped' % % % % % % % % % %
136
137 export const useUnenrollStudent = () => {
138   const queryClient = useQueryClient();
139   const { user } = useAuth();
140
141   return useMutation({
142     mutationFn: async (id: string): Promise<Enrollment> => {
143       const { data, error } = await supabase
144         .from("student_courses")
145         .update({ status: "dropped" })
146         .eq("id", id)
147         .select()
148         .single();
149
150       if (error) throw error;
151
152       const enrollment = data as Enrollment;
153
154       await logAuditEvent({
155         action: "delete",
156         entity_type: "enrollment",
157         entity_id: id,
158         changes: { status: "dropped" },
159         performed_by: user?.id ?? "unknown",
160       });
161
162       return enrollment;
163     },
164     onSuccess: () => {
165       queryClient.invalidateQueries({
166         queryKey: queryKeys.enrollments.lists(),
167       });
168     },
169   });
170 };
171
172 // % % % useCourseStudents - active student profiles for a course % % % % % % % % % %
173
174 export const useCourseStudents = (courseId?: string) => {
175   return useQuery({
176     queryKey: queryKeys.enrollments.list({ courseId, status: "active" }),
177     queryFn: async (): Promise<EnrollmentWithProfile[]> => {
178       const { data, error } = await supabase
179         .from("student_courses")
180         .select("*, profiles(id, full_name, email)")
181         .eq("course_id", courseId!)
182         .eq("status", "active")
183         .order("enrolled_at", { ascending: false });
184
185       if (error) throw error;
186       return data as EnrollmentWithProfile[];
187     },
188     enabled: !!courseId,
189   });
190 };
191

```

FILE: src/hooks/useEquippedItems.ts

TYPE: .ts | LINES: 1-147 | SHA256: 19ab5820f255247e2050c69b744d6449c302c37306c4585163d1fd3f50140029

[illegible]

```

11 |
12 | export type CosmeticSlot = "profile_theme" | "avatar_frame" | "display_title";
13 |
14 | export interface EquippedItem {
15 |   id: string;
16 |   student_id: string;
17 |   purchase_id: string;
18 |   slot: CosmeticSlot;
19 |   equipped_at: string;
20 |   /** Joined item data */
21 |   item_name: string;
22 |   item_sub_category: string;
23 |   icon_identifier: string;
24 |   metadata: Record<string, unknown>;
25 | }
26 |
27 | // % % % useEquippedItems - fetch currently equipped cosmetics % % % % % % % % % % % % % % % %
28 |
29 | export const useEquippedItems = (studentId: string) => {
30 |   return useQuery({
31 |     queryKey: queryKeys.marketplace.equipped(studentId),
32 |     queryFn: async (): Promise<EquippedItem[]> => {
33 |       // eslint-disable-next-line @typescript-eslint/no-explicit-any
34 |       const { data, error } = await (supabase as any)
35 |         .from("student_equipped_items")
36 |         .select(
37 |           `
38 |             id,
39 |             student_id,
40 |             purchase_id,
41 |             slot,
42 |             equipped_at,
43 |             xp_purchases:purchase_id (
44 |               marketplace_items:item_id (
45 |                 name,
46 |                 sub_category,
47 |                 icon_identifier,
48 |                 metadata
49 |               )
50 |             )
51 |           `
52 |         )
53 |         .eq("student_id", studentId);
54 |
55 |   if (error) throw error;
56 |
57 |   return ((data ?? []) as Array<Record<string, unknown>>).map((row) => {
58 |     const purchase = row.xp_purchases as Record<string, unknown> | null;
59 |     const item =
60 |       (purchase?.marketplace_items as Record<string, unknown>) ?? {};
61 |     return {
62 |       id: row.id as string,
63 |       student_id: row.student_id as string,
64 |       purchase_id: row.purchase_id as string,
65 |       slot: row.slot as CosmeticSlot,
66 |       equipped_at: row.equipped_at as string,
67 |       item_name: (item.name as string) ?? "Unknown",
68 |       item_sub_category: (item.sub_category as string) ?? "",
69 |       icon_identifier: (item.icon_identifier as string) ?? "sparkles",
70 |       metadata: (item.metadata ?? {}) as Record<string, unknown>,
71 |     };
72 |   });
73 | },
74 |   enabled: !!studentId,
75 |   staleTime: 30_000,
76 | });
77 | };
78 |
79 | // % % % useEquipItem - upsert into student_equipped_items % % % % % % % % % % % % % % % %
80 |
81 | export const useEquipItem = () => {
82 |   const queryClient = useQueryClient();
83 |
84 |   return useMutation({
85 |     mutationFn: async (variables: {
86 |       studentId: string;
87 |       purchaseId: string;
88 |       slot: CosmeticSlot;
89 |     }): Promise<void> => {
90 |       // eslint-disable-next-line @typescript-eslint/no-explicit-any
91 |       const { error } = await (supabase as any)
92 |         .from("student_equipped_items")
93 |         .upsert(
94 |           {
95 |             student_id: variables.studentId,
96 |             purchase_id: variables.purchaseId,
97 |             slot: variables.slot,
98 |             equipped_at: new Date().toISOString(),
99 |           },
100 |           { onConflict: "student_id,slot" }
101 |         );
102 |
103 |   if (error) throw error;
104 | },
105 |   onSuccess: (_data, variables) => {
106 |     queryClient.invalidateQueries({
107 |       queryKey: queryKeys.marketplace.equipped(variables.studentId),
108 |     });
109 |     toast.success("Item equipped!");
110 | },
111 |   onError: (error: Error) => {
112 |     toast.error(error.message || "Failed to equip item");
113 |   },
114 | });

```

[illegible]

FILE: src/hooks/useExplanationConfidence.ts

TYPE: .ts | LINES: 1-250 | SHA256: 8f15786f288f07609314d8180a6bb224ed75c26bb7073e0f7c13a85756784190

[illegible]

```

68 |         .from("verified_explanations")
69 |         .select("*")
70 |         .eq("question_id", questionId)
71 |         .eq("is_active", true)
72 |         .maybeSingle();
73 |
74 |         if (error) throw error;
75 |         return data;
76 |     },
77 |     enabled: !!questionId,
78 | });
79 | };
80 |
81 | // % % % useApproveExplanation – insert a verified explanation (teacher_approved) %
82 |
83 | export const useApproveExplanation = () => {
84 |     const queryClient = useQueryClient();
85 |
86 |     return useMutation({
87 |         mutationFn: async (
88 |             input: ApproveExplanationInput
89 |         ): Promise<VerifiedExplanationRow> => {
90 |             // Deactivate any existing active explanation for this question
91 |             await supabase
92 |                 .from("verified_explanations")
93 |                 .update({ is_active: false, updated_at: new Date().toISOString() })
94 |                 .eq("question_id", input.question_id)
95 |                 .eq("is_active", true);
96 |
97 |             const insertData: VerifiedExplanationInsert = {
98 |                 institution_id: input.institution_id,
99 |                 question_id: input.question_id,
100 |                 explanation_text: input.explanation_text,
101 |                 source: "teacher_approved",
102 |                 verified_by: input.verified_by,
103 |                 is_active: true,
104 |             };
105 |
106 |             const { data, error } = await supabase
107 |                 .from("verified_explanations")
108 |                 .insert(insertData)
109 |                 .select()
110 |                 .single();
111 |
112 |             if (error) throw error;
113 |             return data;
114 |         },
115 |         onSuccess: (_data, variables) => {
116 |             queryClient.invalidateQueries({
117 |                 queryKey: queryKeys.verifiedExplanations.detail(variables.question_id),
118 |             });
119 |             queryClient.invalidateQueries({
120 |                 queryKey: queryKeys.explanationReviewQueue.all,
121 |             });
122 |         },
123 |     });
124 | };
125 |
126 | // % % % useEditExplanation – insert a verified explanation (teacher_edited) % % % %
127 |
128 | export const useEditExplanation = () => {
129 |     const queryClient = useQueryClient();
130 |
131 |     return useMutation({
132 |         mutationFn: async (
133 |             input: EditExplanationInput
134 |         ): Promise<VerifiedExplanationRow> => {
135 |             // Deactivate any existing active explanation for this question
136 |             await supabase
137 |                 .from("verified_explanations")
138 |                 .update({ is_active: false, updated_at: new Date().toISOString() })
139 |                 .eq("question_id", input.question_id)
140 |                 .eq("is_active", true);
141 |
142 |             const insertData: VerifiedExplanationInsert = {
143 |                 institution_id: input.institution_id,
144 |                 question_id: input.question_id,
145 |                 explanation_text: input.explanation_text,
146 |                 source: "teacher_edited",
147 |                 verified_by: input.verified_by,
148 |                 is_active: true,
149 |             };
150 |
151 |             const { data, error } = await supabase
152 |                 .from("verified_explanations")
153 |                 .insert(insertData)
154 |                 .select()
155 |                 .single();
156 |
157 |             if (error) throw error;
158 |             return data;
159 |         },
160 |         onSuccess: (_data, variables) => {
161 |             queryClient.invalidateQueries({
162 |                 queryKey: queryKeys.verifiedExplanations.detail(variables.question_id),
163 |             });
164 |             queryClient.invalidateQueries({
165 |                 queryKey: queryKeys.explanationReviewQueue.all,
166 |             });
167 |         },
168 |     });
169 | };
170 |
171 | // % % % useExplanationReviewQueue – frequently-missed questions needing review % %

```

```

172 |
173 | export const useExplanationReviewQueue = (courseId: string) => {
174 |   return useQuery({
175 |     queryKey: queryKeys.explanationReviewQueue.list(courseId),
176 |     queryFn: async (): Promise<ReviewQueueItem[]> => {
177 |       // Fetch questions with their analytics joined
178 |       const { data, error } = await supabase
179 |         .from("question_bank")
180 |         .select(
181 |           id,
182 |           course_id,
183 |           clo_id,
184 |           bloom_level,
185 |           question_text,
186 |           explanation,
187 |           explanation_confidence,
188 |           difficulty_rating,
189 |           question_analytics (
190 |             success_rate,
191 |             total_attempts
192 |           )
193 |         )
194 |       )
195 |       .eq("course_id", courseId)
196 |       .eq("status", "approved");
197 |
198 |     if (error) throw error;
199 |
200 |     // Filter to frequently-missed: success_rate < 0.5 AND total_attempts >= 10
201 |     const frequentlyMissed = (data ?? [])
202 |       .filter((q) => {
203 |         const analytics = q.question_analytics as unknown as {
204 |           success_rate: number | null;
205 |           total_attempts: number;
206 |         } | null;
207 |         if (!analytics) return false;
208 |         return (
209 |           analytics.total_attempts >= 10 &&
210 |           analytics.success_rate !== null &&
211 |           analytics.success_rate < 0.5
212 |         );
213 |       });
214 |     .sort((a, b) => {
215 |       const aAttempts =
216 |         (
217 |           a.question_analytics as unknown as {
218 |             total_attempts: number;
219 |           } | null
220 |         )?.total_attempts ?? 0;
221 |       const bAttempts =
222 |         (
223 |           b.question_analytics as unknown as {
224 |             total_attempts: number;
225 |           } | null
226 |         )?.total_attempts ?? 0;
227 |       return bAttempts - aAttempts; // desc
228 |     });
229 |     .map((q) => ({
230 |       id: q.id,
231 |       course_id: q.course_id,
232 |       clo_id: q.clo_id,
233 |       bloom_level: q.bloom_level,
234 |       question_text: q.question_text,
235 |       explanation: q.explanation,
236 |       explanation_confidence: q.explanation_confidence,
237 |       difficulty_rating: q.difficulty_rating,
238 |       question_analytics: q.question_analytics as unknown as {
239 |         success_rate: number | null;
240 |         total_attempts: number;
241 |       } | null,
242 |     }));
243 |   });
244 |   return frequentlyMissed;
245 | },
246 | {
247 |   enabled: !!courseId,
248 | });
249 | };
250 |

```

FILE: [src/hooks/useExtraAttemptUsage.ts](#)

TYPE: .ts | LINES: 1-76 | SHA256: 418f85efa1411afb3c434d81ef31b15d54f11aa0e5366e37c2e00b807be3f30

```

1 | import { useQuery } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 |
4 | export interface ExtraAttemptUsage {
5 |   studentId: string;
6 |   studentName: string;
7 |   purchaseDate: string;
8 |   attemptNumber: number;
9 |   score: number | null;
10 | }
11 |
12 | export const useExtraAttemptUsage = (quizId: string) => {
13 |   return useQuery({
14 |     queryKey: ["marketplace", "extraAttemptUsage", quizId],
15 |     queryFn: async (): Promise<ExtraAttemptUsage[]> => {
16 |       // Fetch quiz to get max_attempts
17 |       const { data: quiz, error: quizError } = await supabase
18 |         .from("quizzes")
19 |         .select("max_attempts")
20 |         .eq("id", quizId)
21 |         .maybeSingle();

```



```

22 |
23 |     if (quizError) throw quizError;
24 |     if (!quiz) return [];
25 |
26 |     const maxAttempts = (quiz as Record<string, unknown>)
27 |       .max_attempts as number;
28 |
29 |     // Fetch attempts that exceed max_attempts (these used extra tokens)
30 |     const { data: attempts, error: attemptsError } = await supabase
31 |       .from("quiz_attempts")
32 |       .select("student_id, attempt_number, score, submitted_at")
33 |       .eq("quiz_id", quizId)
34 |       .gt("attempt_number", maxAttempts)
35 |       .order("submitted_at", { ascending: false });
36 |
37 |     if (attemptsError) throw attemptsError;
38 |     if (!attempts) return [];
39 |
40 |     const studentIds = [
41 |       ...new Set(
42 |         (attempts as Array<Record<string, unknown>>).map(
43 |           (a) => a.student_id as string
44 |         )
45 |       ),
46 |     ];
47 |     if (studentIds.length === 0) return [];
48 |
49 |     // Fetch student names
50 |     const { data: profiles, error: profilesError } = await supabase
51 |       .from("profiles")
52 |       .select("id, full_name")
53 |       .in("id", studentIds);
54 |
55 |     if (profilesError) throw profilesError;
56 |
57 |     const nameMap = new Map(
58 |       ((profiles ?? []) as Array<Record<string, unknown>>).map((p) => [
59 |         p.id as string,
60 |         p.full_name as string,
61 |       ])
62 |     );
63 |
64 |     return (attempts as Array<Record<string, unknown>>).map((a) => ({
65 |       studentId: a.student_id as string,
66 |       studentName: nameMap.get(a.student_id as string) ?? "Unknown",
67 |       purchaseDate: a.submitted_at as string,
68 |       attemptNumber: a.attempt_number as number,
69 |       score: a.score as number | null,
70 |     }));
71 |   },
72 |   enabled: !!quizId,
73 |   staleTime: 30_000,
74 | });
75 | };
76 |

```

## FILE: src/hooks/useFees.ts

TYPE: .ts | LINES: 1-154 | SHA256: 600f6c2b3afeff82cf5ca0fe05dc56636f0e94bc28e22eeb17694c20c7eae15d

```

1 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@/lib/supabase";
3 | import { queryKeys } from "@/lib/queryKeys";
4 | import { logAuditEvent } from "@/lib/auditLogger";
5 | import { useAuth } from "@/hooks/useAuth";
6 |
7 | export interface FeeStructure {
8 |   id: string;
9 |   program_id: string;
10 |   semester_id: string;
11 |   fee_type: string;
12 |   amount: number;
13 |   currency: string;
14 |   due_date: string;
15 |   created_at: string;
16 | }
17 |
18 | export interface FeePayment {
19 |   id: string;
20 |   fee_structure_id: string;
21 |   student_id: string;
22 |   amount_paid: number;
23 |   payment_method: string | null;
24 |   receipt_number: string | null;
25 |   status: string;
26 |   payment_date: string;
27 |   created_at: string;
28 | }
29 |
30 | export const useFeeStructures = (programId?: string) => {
31 |   return useQuery({
32 |     queryKey: queryKeys.feeStructures.list({ programId }),
33 |     queryFn: async (): Promise<FeeStructure[]> => {
34 |       let q = supabase
35 |         .from("fee_structures")
36 |         .select("*")
37 |         .order("due_date", { ascending: false });
38 |       if (programId) q = q.eq("program_id", programId);
39 |       const { data, error } = await q;
40 |       if (error) throw error;
41 |       return (data ?? []) as FeeStructure[];
42 |     },
43 |   });
44 | };
45 |

```

```

46 | export const useFeePayments = (feeStructureId?: string) => {
47 |   return useQuery({
48 |     queryKey: queryKeys.feePayments.list({ feeStructureId }),
49 |     queryFn: async (): Promise<FeePayment[]> => {
50 |       let q = supabase
51 |         .from("fee_payments")
52 |         .select("*")
53 |         .order("payment_date", { ascending: false });
54 |       if (feeStructureId) q = q.eq("fee_structure_id", feeStructureId);
55 |       const { data, error } = await q;
56 |       if (error) throw error;
57 |       return (data ?? []) as FeePayment[];
58 |     },
59 |     enabled: !!feeStructureId,
60 |   });
61 | };
62 |
63 | export const useStudentFees = (studentId?: string) => {
64 |   return useQuery({
65 |     queryKey: queryKeys.studentFees.list({ studentId }),
66 |     queryFn: async (): Promise<FeePayment[]> => {
67 |       const { data, error } = await supabase
68 |         .from("fee_payments")
69 |         .select("*")
70 |         .eq("student_id", studentId!);
71 |       if (error) throw error;
72 |       return (data ?? []) as FeePayment[];
73 |     },
74 |     enabled: !!studentId,
75 |   });
76 | };
77 |
78 | export const useCreateFeeStructure = () => {
79 |   const qc = useQueryClient();
80 |   const { user } = useAuth();
81 |   return useMutation({
82 |     mutationFn: async (input: Omit<FeeStructure, "id" | "created_at">) => {
83 |       const { data, error } = await supabase
84 |         .from("fee_structures")
85 |         .insert(input)
86 |         .select()
87 |         .single();
88 |       if (error) throw error;
89 |       await logAuditEvent({
90 |         action: "create",
91 |         entity_type: "fee_structure",
92 |         entity_id: data.id,
93 |         changes: input,
94 |         performed_by: user?.id ?? "",
95 |       });
96 |       return data;
97 |     },
98 |     onSuccess: () =>
99 |       qc.invalidateQueries({ queryKey: queryKeys.feeStructures.lists() }),
100 |   });
101 | };
102 |
103 | export const useRecordPayment = () => {
104 |   const qc = useQueryClient();
105 |   const { user } = useAuth();
106 |   return useMutation({
107 |     mutationFn: async (input: {
108 |       fee_structure_id: string;
109 |       student_id: string;
110 |       amount_paid: number;
111 |       payment_method: string;
112 |       receipt_number?: string;
113 |     }) => {
114 |       const { data, error } = await supabase
115 |         .from("fee_payments")
116 |         .insert({
117 |           ...input,
118 |           status: "paid",
119 |           payment_date: new Date().toISOString().slice(0, 10),
120 |         })
121 |         .select()
122 |         .single();
123 |       if (error) throw error;
124 |       await logAuditEvent({
125 |         action: "create",
126 |         entity_type: "fee_payment",
127 |         entity_id: data.id,
128 |         changes: input,
129 |         performed_by: user?.id ?? "",
130 |       });
131 |       return data;
132 |     },
133 |     onSuccess: () =>
134 |       qc.invalidateQueries({ queryKey: queryKeys.feePayments.lists() }),
135 |   });
136 | };
137 |
138 | export const useGenerateFeeReceipt = () => {
139 |   return useMutation({
140 |     mutationFn: async (
141 |       paymentId: string
142 |     ): Promise<{ download_url: string; file_name: string }> => {
143 |       const { data, error } = await supabase.functions.invoke(
144 |         "generate-fee-receipt",
145 |         {
146 |           body: { payment_id: paymentId },
147 |         }
148 |       );
149 |       if (error) throw error;

```

FILE: src/hooks/useFirstEnrolledCourse.ts

```

1 //
2 // useFirstEnrolledCourseId - the student's first active course enrollment
3 //
4 // Shared data-access hook relocated out of `StudentDashboard.tsx` and
5 // `StudentTeamPage.tsx`, which both contained an identical in-page `useQuery`
6 // resolving the student's first active course (used to scope team data and the
7 // team leaderboard). Extracting it to a single hook removes the duplication and
8 // routes the access through `src/hooks/` per the engineering guardrails.
9 //
10 // Uses `.maybeSingle()` so a student with no active enrollment resolves to
11 // `null` (zero-or-one row) rather than throwing.
12 //
13 // _Requirements: 25.1, 25.2, 25.3, 25.3a, 25.4_
14 // =====
15
16 import { useQuery } from "@tanstack/react-query";
17 import { supabase } from "@/_lib/supabase";
18 import { queryKeys } from "@/_lib/queryKeys";
19 import { DASHBOARD_STALE_TIME_MS } from "@/_lib/queryConfig";
20
21 /**
22  * Resolve the student's first active course id.
23  *
24  * Returns `null` when the student has no active enrollment. The query is only
25  * enabled once a `studentId` is available, and an optional `enabled` flag lets
26  * callers defer the fetch (the dashboard defers non-critical queries until
27  * after first paint).
28  */
29 export const useFirstEnrolledCourseId = (
30   studentId: string | undefined,
31   options?: { enabled?: boolean }
32 ) => {
33   return useQuery({
34     queryKey: queryKeys.enrollments.list({ studentId, first: true }),
35     queryFn: async (): Promise<string | null> => {
36       const { data, error } = await supabase
37         .from("student_courses")
38         .select("course_id")
39         .eq("student_id", studentId!)
40         .eq("status", "active")
41         .limit(1)
42         .maybeSingle();
43       if (error) throw error;
44       return data?.course_id ?? null;
45     },
46     enabled: !!studentId && (options?.enabled ?? true),
47     staleTime: DASHBOARD_STALE_TIME_MS,
48   });
49 };

```

[illegible]

```

42 |     interval_number: input.intervalNumber,
43 |     response: input.response,
44 |   } as never)
45 |   .select()
46 |   .single();
47 |
48 |   if (error) throw error;
49 |   return mapFlowCheckIn(data as Record<string, unknown>);
50 | },
51 | onSuccess: (data) => {
52 |   queryClient.invalidateQueries({
53 |     queryKey: queryKeys.flowCheckIns.detail(data.sessionId),
54 |   });
55 | },
56 | onError: (error: Error) => {
57 |   toast.error(`Couldn't save check-in: ${error.message}`);
58 | },
59 | });
60 | };
61 |
62 | // % % % useSessionFlowCheckIns - fetch all check-ins for a session % % % % % % % % % %
63 |
64 | export const useSessionFlowCheckIns = (sessionId: string | undefined) => {
65 |   return useQuery({
66 |     queryKey: queryKeys.flowCheckIns.detail(sessionId ?? ""),
67 |     enabled: !!sessionId,
68 |     queryFn: async (): Promise<FlowCheckIn[]> => {
69 |       if (!sessionId) return [];
70 |
71 |       const { data, error } = await supabase
72 |         .from("flow_check_ins" as never)
73 |         .select("")
74 |         .eq("session_id", sessionId)
75 |         .order("interval_number", { ascending: true });
76 |
77 |       if (error) throw error;
78 |       return (data as Record<string, unknown>[]).map(mapFlowCheckIn);
79 |     },
80 |   });
81 | };
82 |

```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]



```
476 |         skipBreak,
477 |         actualDurationMinutes: state.actualDurationMinutes,
478 |     };
479 | };
480 |
```

FILE: src/hooks/useFriends.ts

```
TYPE: .ts | LINES: 1-348 | SHA256: b1735ce437df74e5e1cf9c7dfa1709e62cdf4f87a964b828e911491d119df8ce
```

[illegible]

```

96 |         if (error) throw error;
97 |
98 |         const friendIds = otherIds((rows ?? []) as FriendshipRow[], studentId);
99 |         if (friendIds.length === 0) return [];
100 |
101 |         const [profilesResult, gamResult] = await Promise.all([
102 |             supabase
103 |                 .from("profiles")
104 |                 .select("id, full_name, avatar_url, last_seen_at")
105 |                 .in("id", friendIds),
106 |             supabase
107 |                 .from("student_gamification")
108 |                 .select("student_id, xp_total, level, streak_current")
109 |                 .in("student_id", friendIds),
110 |         ]);
111 |         if (profilesResult.error) throw profilesResult.error;
112 |         if (gamResult.error) throw gamResult.error;
113 |
114 |         const gamMap = new Map(
115 |             (gamResult.data ?? []).map((g) => [g.student_id, g])
116 |         );
117 |
118 |         return (profilesResult.data ?? []).
119 |             .map((p): Friend => {
120 |                 const g = gamMap.get(p.id);
121 |                 return {
122 |                     student_id: p.id,
123 |                     full_name: p.full_name ?? "Unknown",
124 |                     avatar_url: p.avatar_url,
125 |                     last_seen_at: p.last_seen_at,
126 |                     xp_total: g?.xp_total ?? 0,
127 |                     level: g?.level ?? 1,
128 |                     streak_current: g?.streak_current ?? 0,
129 |                     online: isOnline(p.last_seen_at),
130 |                 };
131 |             })
132 |             .sort(
133 |                 (a, b) =>
134 |                     Number(b.online) - Number(a.online) || b.xp_total - a.xp_total
135 |             );
136 |     },
137 |     enabled: !!studentId,
138 |     staleTime: 30_000,
139 | });
140 |
141 |
142 | // % % % useFriendRequests - incoming pending requests (I'm the addressee) % % % % %
143 |
144 | export const useFriendRequests = (studentId: string | undefined) => {
145 |     return useQuery({
146 |         queryKey: queryKeys.friends.list({ scope: "requests", studentId }),
147 |         queryFn: async (): Promise<IncomingFriendRequest[]> => {
148 |             if (!studentId) return [];
149 |
150 |             const { data: rows, error } = await untyped
151 |                 .from("friendships")
152 |                 .select("id, requester_id, created_at")
153 |                 .eq("addressee_id", studentId)
154 |                 .eq("status", "pending")
155 |                 .order("created_at", { ascending: false });
156 |             if (error) throw error;
157 |
158 |             const requesterIds = (rows ?? []).map(
159 |                 (r: { requester_id: string }) => r.requester_id
160 |             );
161 |             if (requesterIds.length === 0) return [];
162 |
163 |             const { data: profiles, error: pErr } = await supabase
164 |                 .from("profiles")
165 |                 .select("id, full_name, avatar_url")
166 |                 .in("id", requesterIds);
167 |             if (pErr) throw pErr;
168 |
169 |             const pMap = new Map((profiles ?? []).map((p) => [p.id, p]));
170 |             return (rows ?? []).map(
171 |                 (r: {
172 |                     id: string;
173 |                     requester_id: string;
174 |                     created_at: string;
175 |                 }): IncomingFriendRequest => ({
176 |                     friendship_id: r.id,
177 |                     requester_id: r.requester_id,
178 |                     full_name: pMap.get(r.requester_id)?.full_name ?? "Unknown",
179 |                     avatar_url: pMap.get(r.requester_id)?.avatar_url ?? null,
180 |                     created_at: r.created_at,
181 |                 })
182 |             );
183 |         },
184 |         enabled: !!studentId,
185 |         staleTime: 30_000,
186 |     });
187 | };
188 |
189 | // % % % useClassmateSearch - find same-institution students to add % % % % %
190 |
191 | export const useClassmateSearch = (
192 |     query: string,
193 |     studentId: string | undefined
194 | ) => {
195 |     const q = query.trim();
196 |     return useQuery({
197 |         queryKey: queryKeys.friends.list({ scope: "search", q, studentId }),
198 |         queryFn: async (): Promise<ClassmateResult[]> => {
199 |             if (!studentId || q.length < 2) return [];

```

[illegible]

```

304     friendshipId: string;
305     accept: boolean;
306   }) => {
307     const { error } = await untyped.rpc("respond_friend_request", {
308       p_friendship_id: friendshipId,
309       p_accept: accept,
310     });
311     if (error) throw error;
312   },
313   onSuccess: (_data, variables) => {
314     qc.invalidateQueries({ queryKey: queryKeys.friends.all });
315     toast.success(variables.accept ? "Friend added" : "Request declined");
316   },
317   onError: (err) =>
318     toast.error(err instanceof Error ? err.message : "Couldn't respond"),
319   });
320 };
321
322 export const useRemoveFriend = () => {
323   const qc = useQueryClient();
324   const { user } = useAuth();
325   const me = user?.id;
326   return useMutation({
327     mutationFn: async (friendId: string) => {
328       if (!me) throw new Error("Not authenticated");
329       // DELETE RLS allows either party to remove their own relationship row.
330       const { error } = await untyped
331         .from("friendships")
332         .delete()
333         .or(
334           and(requester_id.eq.${me}, addressee_id.eq.${friendId}), and(requester_id.eq.${friendId}, addressee_id.eq.${me})
335         );
336       if (error) throw error;
337     },
338     onSuccess: () => {
339       qc.invalidateQueries({ queryKey: queryKeys.friends.all });
340       toast.success("Friend removed");
341     },
342     onError: (err) =>
343       toast.error(
344         err instanceof Error ? err.message : "Couldn't remove friend"
345       ),
346   });
347 };
348

```

FILE: src/hooks/useGenerateQuestions.ts

```
TYPE: .ts | LINES: 1-55 | SHA256: 2864757230d0ae4e0fedf38c1ad2203143b41f4d21c46394ced7920b9eb75e22
```

[illegible]

FILE: `src/hooks/useGlobalSearch.ts`

```
TYPE: .ts | LINES: 1-87 | SHA256: cd3882008bbbc755a044d51d5d2994522ca3da55b7c1bbcd5ab372d6ce92b772
```

```

1 // Task 92.3: Global search hook – full-text search across courses, assignments, announcements
2
3 import { useQuery } from "@tanstack/react-query";
4 import { supabase } from "@lib/supabase";
5 import { queryKeys } from "@lib/queryKeys";
6 import type { UserRole } from "@types/app";
7
8 export interface SearchResult {
9   id: string;
10  type: "course" | "assignment" | "announcement";
11  title: string;
12  description: string | null;
13  url: string;
14 }
15
16 const searchCourses = async (query: string): Promise<SearchResult[]> => {
17   const { data, error } = await supabase
18     .from("courses")
19     .select("id, name, code")
20     .or(`name.ilike.${query}%,code.ilike.${query}%`)
21     .limit(5);
22
23   if (error) throw error;
24   return (data ?? []).map((c) => ({
25     id: c.id,
26     type: "course" as const,
27     title: `${c.code} – ${c.name}`,
28     description: null,
29     url: `/courses/${c.id}`,
30   })));
31 };
32
33 const searchAssignments = async (query: string): Promise<SearchResult[]> => {
34   const { data, error } = await supabase
35     .from("assignments")
36     .select("id, title, description")
37     .ilike("title", `%${query}%`)
38     .limit(5);
39
40   if (error) throw error;
41   return (data ?? []).map((a) => ({
42     id: a.id,
43     type: "assignment" as const,
44     title: a.title,
45     description: a.description,
46     url: `/assignments/${a.id}`,
47   })));
48 };
49
50 const searchAnnouncements = async (query: string): Promise<SearchResult[]> => {
51   const { data, error } = await supabase
52     .from("announcements")
53     .select("id, title, content")
54     .ilike("title", `%${query}%`)
55     .limit(5);
56
57   if (error) throw error;
58   return (data ?? []).map((a) => ({
59     id: a.id,
60     type: "announcement" as const,
61     title: a.title,
62     description: a.content ? String(a.content).slice(0, 100) : null,
63     url: `/announcements/${a.id}`,
64   })));
65 };
66
67 export const useGlobalSearch = (query: string, role: UserRole | null) => {
68   return useQuery({
69     queryKey: queryKeys.globalSearch.list({ query, role }),
70     queryFn: async () => Promise<SearchResult[]> => {
71       if (!query || query.length < 2) return [];
72
73       const results = await Promise.all([
74         searchCourses(query),
75         // Students and parents may not search assignments directly in some scopes,
76         // but RLS handles visibility – include for all roles
77         searchAssignments(query),
78         searchAnnouncements(query),
79       ]);
80
81       return results.flat();
82     },
83     enabled: !!query && query.length >= 2 && !!role,
84     staleTime: 30_000,
85   });
86 };
87

```

FILE: `src/hooks/useGoalSuggestions.ts`

TYPE: .ts | LINES: 1-145 | SHA256: 3e7962896f0ee02d979092ed9f1d3d95d06d1bb9df9a57d01408ce3631a44702

[illegible]

```

10 |   week_start: string;
11 |   goal_text: string;
12 |   smart_specific: string | null;
13 |   smart_measurable: string | null;
14 |   smart_achievable: string | null;
15 |   smart_relevant: string | null;
16 |   smart_timebound: string | null;
17 |   difficulty: "easy" | "moderate" | "ambitious";
18 |   cohort_completion_rate: number | null;
19 |   status: "suggested" | "accepted" | "modified" | "dismissed";
20 |   created_at: string;
21 | }
22 |
23 | type GoalStatus = GoalSuggestionRecord["status"];
24 |
25 | interface GenerateGoalsInput {
26 |   student_id: string;
27 |   week_start: string;
28 | }
29 |
30 | interface GenerateGoalsResult {
31 |   success: boolean;
32 |   suggestions: GoalSuggestionRecord[];
33 | }
34 |
35 | // % % useGoalSuggestions - fetch suggestions for a week % % % % % % % % % % % %
36 |
37 | export const useGoalSuggestions = (studentId: string, weekStart: string) => {
38 |   return useQuery({
39 |     queryKey: queryKeys.onboarding.goalSuggestions(studentId, weekStart),
40 |     queryFn: async (): Promise<GoalSuggestionRecord[]> => {
41 |       const { data, error } = await supabase
42 |         .from("goal_suggestions")
43 |         .select("*")
44 |         .eq("student_id", studentId)
45 |         .eq("week_start", weekStart)
46 |         .order("created_at", { ascending: true });
47 |
48 |       if (error) throw error;
49 |       return (data ?? []) as GoalSuggestionRecord[];
50 |     },
51 |     enabled: !!studentId && !!weekStart,
52 |   });
53 | };
54 |
55 | // % % useAcceptGoal - update goal status to accepted % % % % % % % % % % % %
56 |
57 | export const useAcceptGoal = () => {
58 |   const queryClient = useQueryClient();
59 |
60 |   return useMutation({
61 |     mutationFn: async (params: {
62 |       id: string;
63 |       studentId: string;
64 |       weekStart: string;
65 |     }): Promise<GoalSuggestionRecord> => {
66 |       const { data, error } = await supabase
67 |         .from("goal_suggestions")
68 |         .update({ status: "accepted" as GoalStatus })
69 |         .eq("id", params.id)
70 |         .select()
71 |         .single();
72 |
73 |       if (error) throw error;
74 |       return data as GoalSuggestionRecord;
75 |     },
76 |     onSuccess: (_data, variables) => {
77 |       queryClient.invalidateQueries({
78 |         queryKey: queryKeys.onboarding.goalSuggestions(
79 |           variables.studentId,
80 |           variables.weekStart
81 |         ),
82 |       });
83 |     },
84 |   });
85 | };
86 |
87 | // % % useDismissGoal - update goal status to dismissed % % % % % % % % % % % %
88 |
89 | export const useDismissGoal = () => {
90 |   const queryClient = useQueryClient();
91 |
92 |   return useMutation({
93 |     mutationFn: async (params: {
94 |       id: string;
95 |       studentId: string;
96 |       weekStart: string;
97 |     }): Promise<GoalSuggestionRecord> => {
98 |       const { data, error } = await supabase
99 |         .from("goal_suggestions")
100 |        .update({ status: "dismissed" as GoalStatus })
101 |        .eq("id", params.id)
102 |        .select()
103 |        .single();
104 |
105 |       if (error) throw error;
106 |       return data as GoalSuggestionRecord;
107 |     },
108 |     onSuccess: (_data, variables) => {
109 |       queryClient.invalidateQueries({
110 |         queryKey: queryKeys.onboarding.goalSuggestions(
111 |           variables.studentId,
112 |           variables.weekStart
113 |         ),

```



```

69 |     queryKey: queryKeys.gradeCategories.list({ courseId }),
70 |     queryFn: async (): Promise<GradeCategory[]> => {
71 |         const { data, error } = await supabase
72 |             .from("grade_categories")
73 |             .select("*")
74 |             .eq("course_id", courseId!)
75 |             .order("sort_order", { ascending: true });
76 |
77 |         if (error) throw error;
78 |         return (data ?? []) as GradeCategory[];
79 |     },
80 |     enabled: !!courseId,
81 | });
82 | };
83 |
84 | // % % % useCreateGradeCategory – insert with audit logging % % % % % % % % % % % % % % % %
85 |
86 | export const useCreateGradeCategory = () => {
87 |     const queryClient = useQueryClient();
88 |     const { user } = useAuth();
89 |
90 |     return useMutation({
91 |         mutationFn: async (
92 |             input: CreateGradeCategoryInput
93 |         ): Promise<GradeCategory> => {
94 |             const { data, error } = await supabase
95 |                 .from("grade_categories")
96 |                 .insert({
97 |                     course_id: input.course_id,
98 |                     name: input.name,
99 |                     weight_percent: input.weight_percent,
100 |                     sort_order: input.sort_order,
101 |                 })
102 |                 .select()
103 |                 .single();
104 |
105 |             if (error) throw error;
106 |
107 |             const category = data as GradeCategory;
108 |
109 |             await logAuditEvent({
110 |                 action: "create",
111 |                 entity_type: "grade_category",
112 |                 entity_id: category.id,
113 |                 changes: { ...input },
114 |                 performed_by: user?.id ?? "unknown",
115 |             });
116 |
117 |             return category;
118 |         },
119 |         onSuccess: () => {
120 |             queryClient.invalidateQueries({
121 |                 queryKey: queryKeys.gradeCategories.lists(),
122 |             });
123 |             queryClient.invalidateQueries({ queryKey: queryKeys.gradebook.lists() });
124 |             toast.success("Grade category created");
125 |         },
126 |         onError: (err) => {
127 |             toast.error(
128 |                 err instanceof Error ? err.message : "Failed to create grade category"
129 |             ),
130 |         });
131 |     };
132 |
133 | // % % % useUpdateGradeCategory – update with audit logging % % % % % % % % % % % % % % % %
134 |
135 | export const useUpdateGradeCategory = (id: string) => {
136 |     const queryClient = useQueryClient();
137 |     const { user } = useAuth();
138 |
139 |     return useMutation({
140 |         mutationFn: async (
141 |             input: UpdateGradeCategoryInput
142 |         ): Promise<GradeCategory> => {
143 |             const { data, error } = await supabase
144 |                 .from("grade_categories")
145 |                 .update(input)
146 |                 .eq("id", id)
147 |                 .select()
148 |                 .single();
149 |
150 |             if (error) throw error;
151 |
152 |             await logAuditEvent({
153 |                 action: "update",
154 |                 entity_type: "grade_category",
155 |                 entity_id: id,
156 |                 changes: input as Record<string, unknown>,
157 |                 performed_by: user?.id ?? "unknown",
158 |             });
159 |
160 |             return data as GradeCategory;
161 |         },
162 |         onSuccess: () => {
163 |             queryClient.invalidateQueries({
164 |                 queryKey: queryKeys.gradeCategories.lists(),
165 |             });
166 |             queryClient.invalidateQueries({ queryKey: queryKeys.gradebook.lists() });
167 |             toast.success("Grade category updated");
168 |         },
169 |         onError: (err) => {
170 |             toast.error(
171 |                 err instanceof Error ? err.message : "Failed to update grade category"
172 |             ),

```



[illegible]

```

277 |
278 | // 4. Fetch quizzes
279 | const { data: rawQuizzes, error: quizError } = await supabase
280 |   .from("quizzes")
281 |   .select("id, title")
282 |   .eq("course_id", courseId)
283 |   .eq("is_published", true)
284 |   .order("created_at", { ascending: true });
285 |
286 | if (quizError) throw quizError;
287 |
288 | const quizList = (rawQuizzes ?? []) as Array<{
289 |   id: string;
290 |   title: string;
291 | }>;
292 |
293 | // 5. Fetch grades via submissions
294 | const { data: rawGrades, error: gradeError } = await supabase
295 |   .from("grades")
296 |   .select(
297 |     "total_score, score_percent, submissions!inner(student_id, assignment_id)"
298 |   )
299 |   .in("submissions.student_id", studentIds);
300 |
301 | if (gradeError) throw gradeError;
302 |
303 | const gradeMap = new Map<
304 |   string,
305 |   { totalScore: number; scorePercent: number }
306 | >();
307 | for (const g of rawGrades ?? []) {
308 |   const grade = g as unknown as {
309 |     total_score: number;
310 |     score_percent: number;
311 |     submissions: { student_id: string; assignment_id: string };
312 |   };
313 |   const key = `${grade.submissions.student_id}:${grade.submissions.assignment_id}`;
314 |   gradeMap.set(key, {
315 |     totalScore: grade.total_score,
316 |     scorePercent: grade.score_percent,
317 |   });
318 | }
319 |
320 | // 6. Fetch quiz attempts (best score per student per quiz)
321 | const quizIds = quizList.map((q) => q.id);
322 | let quizScoreMap = new Map<string, number>();
323 |
324 | if (quizIds.length > 0) {
325 |   const { data: rawAttempts, error: attemptError } = await supabase
326 |     .from("quiz_attempts")
327 |     .select("quiz_id, student_id, score")
328 |     .in("quiz_id", quizIds)
329 |     .in("student_id", studentIds)
330 |     .not("score", "is", null);
331 |
332 |   if (attemptError) throw attemptError;
333 |
334 |   quizScoreMap = new Map<string, number>();
335 |   for (const a of rawAttempts ?? []) {
336 |     const attempt = a as unknown as {
337 |       quiz_id: string;
338 |       student_id: string;
339 |       score: number;
340 |     };
341 |     const key = `${attempt.student_id}:${attempt.quiz_id}`;
342 |     const existing = quizScoreMap.get(key);
343 |     if (existing === undefined || attempt.score > existing) {
344 |       quizScoreMap.set(key, attempt.score);
345 |     }
346 |   }
347 | }
348 |
349 | // 7. Distribute assessments across categories
350 | // Convention: categories with "quiz" in name get quizzes, others get assignments
351 | // If no categories, create a single "All" category at 100%
352 | const effectiveCategories: Array<{
353 |   id: string;
354 |   name: string;
355 |   weight_percent: number;
356 |   assignmentIds: string[];
357 |   quizIds: string[];
358 | }> = [];
359 |
360 | if (categories.length === 0) {
361 |   effectiveCategories.push({
362 |     id: "all",
363 |     name: "All Assessments",
364 |     weight_percent: 100,
365 |     assignmentIds: assignments.map((a) => a.id),
366 |     quizIds: quizList.map((q) => q.id),
367 |   });
368 | } else {
369 |   const quizCatIds = categories
370 |     .filter((c) => c.name.toLowerCase().includes("quiz"))
371 |     .map((c) => c.id);
372 |   const assignCatIds = categories
373 |     .filter((c) => !c.name.toLowerCase().includes("quiz"))
374 |     .map((c) => c.id);
375 |
376 |   for (const cat of categories) {
377 |     const isQuizCat = quizCatIds.includes(cat.id);
378 |     effectiveCategories.push({
379 |       id: cat.id,
380 |       name: cat.name,

```

```

381 |         weight_percent: cat.weight_percent,
382 |         assignmentIds: isQuizCat
383 |         ? []
384 |         : assignCatIds.length === 1 || assignCatIds[0] === cat.id
385 |         ? assignments.map((a) => a.id)
386 |         : [],
387 |         quizIds: isQuizCat ? quizList.map((q) => q.id) : [],
388 |     });
389 | }
390 |
391 | // If no quiz category exists, put quizzes in the first category
392 | const hasQuizCat = quizCatIds.length > 0;
393 | if (
394 |     !hasQuizCat &&
395 |     effectiveCategories.length > 0 &&
396 |     quizList.length > 0
397 | ) {
398 |     const first = effectiveCategories[0];
399 |     if (first) first.quizIds = quizList.map((q) => q.id);
400 | }
401 |
402 | // If no assignment category got assignments, put them in the first non-quiz category
403 | const hasAssignments = effectiveCategories.some(
404 |     (c) => c.assignmentIds.length > 0
405 | );
406 | if (!hasAssignments && assignments.length > 0) {
407 |     const firstNonQuiz = effectiveCategories.find(
408 |         (c) => !quizCatIds.includes(c.id)
409 |     );
410 |     if (firstNonQuiz) {
411 |         firstNonQuiz.assignmentIds = assignments.map((a) => a.id);
412 |     } else {
413 |         const fallback = effectiveCategories[0];
414 |         if (fallback) fallback.assignmentIds = assignments.map((a) => a.id);
415 |     }
416 | }
417 | }
418 |
419 | // 8. Build gradebook entries
420 | const assignmentLookup = new Map(assignments.map((a) => [a.id, a]));
421 | const quizLookup = new Map(quizList.map((q) => [q.id, q]));
422 |
423 | const entries: GradebookEntry[] = enrollments.map((enrollment) => {
424 |     const studentId = enrollment.student_id;
425 |     const studentName = enrollment.profiles?.full_name ?? "Unknown";
426 |
427 |     const catEntries: GradebookCategoryEntry[] = effectiveCategories.map(
428 |         (cat) => {
429 |             const assessments: GradebookAssessment[] = [
430 |                 ...cat.assignmentIds.map((aId) => {
431 |                     const assignment = assignmentLookup.get(aId);
432 |                     const gradeData = gradeMap.get(`${studentId}:${aId}`);
433 |                     let scoreVal: number | null = null;
434 |                     if (gradeData) {
435 |                         if (
436 |                             gradeData.scorePercent !== undefined &&
437 |                             gradeData.scorePercent !== null
438 |                         ) {
439 |                             scoreVal = gradeData.scorePercent;
440 |                         } else if (
441 |                             gradeData.totalScore !== undefined &&
442 |                             gradeData.totalScore !== null
443 |                         ) {
444 |                             const totalMarks = assignment?.total_marks || 100;
445 |                             scoreVal = (gradeData.totalScore / totalMarks) * 100;
446 |                         }
447 |                     }
448 |                     return {
449 |                         id: aId,
450 |                         title: assignment?.title ?? "Unknown",
451 |                         type: "assignment" as const,
452 |                         score:
453 |                             scoreVal !== null ? Math.round(scoreVal * 100) / 100 : null,
454 |                         max_score: 100,
455 |                     };
456 |                 }),
457 |                 ...cat.quizIds.map((qId) => {
458 |                     const quiz = quizLookup.get(qId);
459 |                     const bestScore = quizScoreMap.get(`${studentId}:${qId}`);
460 |                     return {
461 |                         id: qId,
462 |                         title: quiz?.title ?? "Unknown",
463 |                         type: "quiz" as const,
464 |                         score: bestScore !== undefined ? bestScore : null,
465 |                         max_score: 100,
466 |                     };
467 |                 })
468 |             ];
469 |
470 |             const graded = assessments.filter((a) => a.score !== null);
471 |             const subtotalPercent =
472 |                 graded.length > 0
473 |                 ? graded.reduce((sum, a) => sum + a.score!, 0) / graded.length
474 |                 : 0;
475 |
476 |             return {
477 |                 category_id: cat.id,
478 |                 category_name: cat.name,
479 |                 weight_percent: cat.weight_percent,
480 |                 assessments,
481 |                 subtotal_percent: Math.round(subtotalPercent * 100) / 100,
482 |             };
483 |         }
484 |     );

```

```

485 |
486 |     const finalWeightedGrade = catEntries.reduce(
487 |       (sum, cat) => sum + (cat.subtotal_percent * cat.weight_percent) / 100,
488 |       0
489 |     );
490 |
491 |     const roundedFinal = Math.round(finalWeightedGrade * 100) / 100;
492 |
493 |     return {
494 |       student_id: studentId,
495 |       student_name: studentName,
496 |       categories: catEntries,
497 |       final_weighted_grade: roundedFinal,
498 |       letter_grade: mapToLetterGrade(roundedFinal),
499 |     };
500 |   });
501 |
502 |   return entries;
503 | },
504 | enabled: !!courseId,
505 | });
506 | });
507 |

```

FILE: `src/hooks/useGrades.ts`

```
TYPE: .ts | LINES: 1-154 | SHA256: 7d322c8e63ebb612ea09882d3787f71565b00c917c48baaedbfe739636401133
```

[illegible]

```

78 | const queryClient = useQueryClient();
79 | const { user } = useAuth();
80 |
81 | return useMutation({
82 |   mutationFn: async (
83 |     data: GradeFormData & { graded_by: string }
84 |   ): Promise<Grade> => {
85 |     const { data: result, error } = await supabase
86 |       .from("grades")
87 |       .insert(data)
88 |       .select()
89 |       .single();
90 |
91 |     if (error) throw error;
92 |
93 |     const grade = result as unknown as Grade;
94 |
95 |     await logAuditEvent({
96 |       action: "create",
97 |       entity_type: "grade",
98 |       entity_id: grade.id,
99 |       changes: data,
100 |       performed_by: user?.id ?? "unknown",
101 |     });
102 |
103 |     return grade;
104 |   },
105 |   onSuccess: (grade) => {
106 |     queryClient.invalidateQueries({ queryKey: queryKeys.grades.lists() });
107 |     queryClient.invalidateQueries({
108 |       queryKey: queryKeys.submissions.lists(),
109 |     });
110 |     // Evidence Generator (Task 17) runs via a database trigger on grades INSERT.
111 |     // Invalidate evidence & attainment caches so the UI picks up new records.
112 |     queryClient.invalidateQueries({ queryKey: queryKeys.evidence.lists() });
113 |     queryClient.invalidateQueries({
114 |       queryKey: queryKeys.outcomeAttainment.lists(),
115 |     });
116 |     // Prediction validation (Task 36.4) updates ai_feedback records.
117 |     // Invalidate so teacher dashboard reflects validated predictions.
118 |     queryClient.invalidateQueries({ queryKey: queryKeys.aiFeedback.lists() });
119 |     queryClient.invalidateQueries({
120 |       queryKey: queryKeys.atRiskPredictions.lists(),
121 |     });
122 |
123 |     // Grade XP is OWNED by the `trigger_attainment_rollup` DB trigger, which
124 |     // inserts the canonical +15 `grade` xp_transaction (ON CONFLICT DO NOTHING)
125 |     // and updates xp_total. We must NOT call award-xp('grade') here – that would
126 |     // double-award and break the xp_total = SUM(xp_transactions) invariant.
127 |     // Instead, fire the grade badge-check from the client so the grade !' badge
128 |     // path runs, then invalidate badge caches. Fire-and-forget: a badge failure
129 |     // must never break the grade mutation.
130 |     void (async () => {
131 |       try {
132 |         const { data: submission, error } = await supabase
133 |           .from("submissions")
134 |           .select("student_id")
135 |           .eq("id", grade.submission_id)
136 |           .maybeSingle();
137 |
138 |         if (error) throw error;
139 |         const studentId = submission?.student_id;
140 |         if (!studentId) return;
141 |
142 |         await supabase.functions.invoke("check-badges", {
143 |           body: { student_id: studentId, trigger: "grade" },
144 |         });
145 |
146 |         queryClient.invalidateQueries({ queryKey: queryKeys.badges.lists() });
147 |       } catch {
148 |         console.error("[useCreateGrade] grade badge-check failed");
149 |       }
150 |     })();
151 |   },
152 | });
153 | };
154 |

```

## FILE: src/hooks/useGradingStats.ts

TYPE: .ts | LINES: 1-176 | SHA256: f338c2b2834337e6544eb0e70f75f3d97eea17028a22ae0b994e47456b3614d5

```

1 | // Task 58: Teacher Grading Stats hooks
2 | import { useQuery } from "@tanstack/react-query";
3 | import { supabase } from "@lib/supabase";
4 | import { queryKeys } from "@lib/queryKeys";
5 | import { DASHBOARD_STALE_TIME_MS } from "@lib/queryConfig";
6 | import { startOfWeek, subDays } from "date-fns";
7 |
8 | export interface GradingStats {
9 |   gradedThisWeek: number;
10 |   avgGradingTimeSeconds: number;
11 |   pendingCount: number;
12 |   gradingStreak: number;
13 |   velocityTrend: Array<{ date: string; count: number }>;
14 | }
15 |
16 | export const useGradingStats = (teacherId: string | undefined) => {
17 |   return useQuery({
18 |     queryKey: queryKeys.grades.list({ scope: "grading_stats", teacherId }),
19 |     queryFn: async (): Promise<GradingStats> => {
20 |       if (!teacherId) throw new Error("teacherId required");
21 |
22 |       const weekStart = startOfWeek(new Date(), {
23 |         weekStartsOn: 1,

```

```

24 |     }).toISOString();
25 |     const thirtyDaysAgo = subDays(new Date(), 30).toISOString();
26 |
27 |     // Graded this week
28 |     const { count: gradedThisWeek, error: gradedError } = await supabase
29 |       .from("grades")
30 |       .select("id", { count: "exact", head: true })
31 |       .eq("graded_by", teacherId)
32 |       .gte("graded_at", weekStart);
33 |     if (gradedError) throw gradedError;
34 |
35 |     // Pending submissions for teacher's courses
36 |     const { data: courses, error: coursesError } = await supabase
37 |       .from("courses")
38 |       .select("id")
39 |       .eq("teacher_id", teacherId);
40 |     if (coursesError) throw coursesError;
41 |     const courseIds = (courses ?? []).map((c) => c.id);
42 |     let pendingCount = 0;
43 |     if (courseIds.length > 0) {
44 |       const { data: assignmentRows, error: assignmentError } = await supabase
45 |         .from("assignments")
46 |         .select("id")
47 |         .in("course_id", courseIds);
48 |       if (assignmentError) throw assignmentError;
49 |       const assignmentIds = (assignmentRows ?? []).map(
50 |         (a: { id: string }) => a.id
51 |       );
52 |       if (assignmentIds.length > 0) {
53 |         const { count, error: pendingError } = await supabase
54 |           .from("submissions")
55 |           .select("id", { count: "exact", head: true })
56 |           .in("assignment_id", assignmentIds)
57 |           .eq("status", "submitted");
58 |         if (pendingError) throw pendingError;
59 |         pendingCount = count ?? 0;
60 |       }
61 |     }
62 |
63 |     // Velocity trend (last 30 days)
64 |     const { data: recentGrades, error: recentError } = await supabase
65 |       .from("grades")
66 |       .select("graded_at")
67 |       .eq("graded_by", teacherId)
68 |       .gte("graded_at", thirtyDaysAgo);
69 |     if (recentError) throw recentError;
70 |
71 |     const dayMap = new Map<string, number>();
72 |     for (const g of recentGrades ?? []) {
73 |       if (!g.graded_at) continue;
74 |       const d = (g.graded_at as string).slice(0, 10);
75 |       dayMap.set(d, (dayMap.get(d) ?? 0) + 1);
76 |     }
77 |
78 |     const velocityTrend = Array.from(dayMap.entries())
79 |       .map([date, count] => ({ date, count }))
80 |       .sort((a, b) => a.date.localeCompare(b.date));
81 |
82 |     // Grading streak (consecutive days with >=1 grade, limited to data window)
83 |     let streak = 0;
84 |     const today = new Date();
85 |     const maxDays = 30; // limited to recentGrades data window
86 |     for (let i = 0; i < maxDays; i++) {
87 |       const d = subDays(today, i).toISOString().slice(0, 10);
88 |       if (dayMap.has(d)) streak++;
89 |       else break;
90 |     }
91 |
92 |     // Average grading time from activity log (grading_start != grading_end pairs)
93 |     const avgGradingTimeSeconds = await calculateAvgGradingTime(
94 |       teacherId,
95 |       thirtyDaysAgo
96 |     );
97 |
98 |     return {
99 |       gradedThisWeek: gradedThisWeek ?? 0,
100 |       avgGradingTimeSeconds,
101 |       pendingCount,
102 |       gradingStreak: streak,
103 |       velocityTrend,
104 |     };
105 |   },
106 |   enabled: !!teacherId,
107 |   staleTime: DASHBOARD_STALE_TIME_MS,
108 |   // The grades HEAD count and submissions count can timeout on the free tier
109 |   // under cold-start conditions; limit retries to avoid cascading 500s.
110 |   retry: 1,
111 | });
112 |
113 |
114 | /**
115 |  * Calculate average grading time from grading_start/grading_end activity log pairs.
116 |  * Matches pairs by submission_id in metadata, computes duration, and averages.
117 |  */
118 | async function calculateAvgGradingTime(
119 |   teacherId: string,
120 |   since: string
121 | ): Promise<number> {
122 |   const { data: startEvents, error: startError } = await supabase
123 |     .from("student_activity_log")
124 |     .select("created_at, metadata")
125 |     .eq("student_id", teacherId)
126 |     .eq("event_type", "grading_start")
127 |     .gte("created_at", since)

```

```

128 |     .order("created_at", { ascending: true });
129 |     if (startError) throw startError;
130 |
131 |     const { data: endEvents, error: endError } = await supabase
132 |       .from("student_activity_log")
133 |       .select("created_at, metadata")
134 |       .eq("student_id", teacherId)
135 |       .eq("event_type", "grading_end")
136 |       .gte("created_at", since)
137 |       .order("created_at", { ascending: true });
138 |     if (endError) throw endError;
139 |
140 |     if (!startEvents?.length || !endEvents?.length) return 0;
141 |
142 |     // Build a map of submission_id !' earliest start time
143 |     const startMap = new Map<string, string>();
144 |     for (const evt of startEvents) {
145 |       const subId = (evt.metadata as Record<string, unknown> | null)
146 |         ?.submission_id as string | undefined;
147 |       if (subId && !startMap.has(subId)) {
148 |         startMap.set(subId, evt.created_at as string);
149 |       }
150 |     }
151 |
152 |     // Match end events to start events by submission_id
153 |     const durations: number[] = [];
154 |     for (const evt of endEvents) {
155 |       const subId = (evt.metadata as Record<string, unknown> | null)
156 |         ?.submission_id as string | undefined;
157 |       if (!subId) continue;
158 |       const startTime = startMap.get(subId);
159 |       if (!startTime) continue;
160 |
161 |       const durationMs =
162 |         new Date(evt.created_at as string).getTime() -
163 |         new Date(startTime).getTime();
164 |       const durationSec = durationMs / 1000;
165 |       // Ignore unreasonable durations (< 5s or > 2 hours)
166 |       if (durationSec >= 5 && durationSec <= 7200) {
167 |         durations.push(durationSec);
168 |       }
169 |     }
170 |
171 |     if (durations.length === 0) return 0;
172 |     return Math.round(
173 |       durations.reduce((sum, d) => sum + d, 0) / durations.length
174 |     );
175 |   }
176 | }

```

#### FILE: [src/hooks/useGraduateAttributes.ts](#)

TYPE: .ts | LINES: 1-221 | SHA256: 3fbb9a46eefb67518e3d8ebcbe1f3b82e9b27887dab18e05cc2e353bd18e4250

```

1 | // Task 112.3: Graduate Attribute TanStack Query hooks
2 | // NOTE: These hooks reference tables that require database migrations (task 112.1).
3 |
4 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
5 | import { supabase } from "@lib/supabase";
6 | import { queryKeys } from "@lib/queryKeys";
7 | import { logAuditEvent } from "@lib/auditLogger";
8 | import { useAuth } from "@hooks/useAuth";
9 |
10 | export interface GraduateAttribute {
11 |   id: string;
12 |   institution_id: string;
13 |   name: string;
14 |   description: string | null;
15 |   sort_order: number;
16 |   created_at: string;
17 | }
18 |
19 | export interface GraduateAttributeMapping {
20 |   id: string;
21 |   graduate_attribute_id: string;
22 |   outcome_id: string;
23 |   weight: number;
24 | }
25 |
26 | export interface GAAttainment {
27 |   attribute_id: string;
28 |   name: string;
29 |   description: string | null;
30 |   avg_attainment: number;
31 |   mapped_ilo_count: number;
32 | }
33 |
34 | export const useGraduateAttributes = (institutionId?: string) => {
35 |   return useQuery({
36 |     queryKey: queryKeys.graduateAttributes.list({ institutionId }),
37 |     queryFn: async (): Promise<GraduateAttribute[]> => {
38 |       const { data, error } = await supabase
39 |         .from("graduate_attributes" as never)
40 |         .select("*,")
41 |         .eq("institution_id", institutionId!);
42 |       if (error) throw error;
43 |       return (data ?? []) as GraduateAttribute[];
44 |     },
45 |     enabled: !!institutionId,
46 |   });
47 | };
48 |
49 | export const useGraduateAttributeMappings = (attributeId?: string) => {
50 |   return useQuery({
51 |     queryKey: queryKeys.graduateAttributeMappings.list({ attributeId }),

```

```

52 |     queryFn: async (): Promise<GraduateAttributeMapping[]> => {
53 |         const { data, error } = await supabase
54 |             .from("graduate_attribute_mappings" as never)
55 |             .select("*")
56 |             .eq("graduate_attribute_id", attributeId!);
57 |         if (error) throw error;
58 |         return (data ?? []) as GraduateAttributeMapping[];
59 |     },
60 |     enabled: !!attributeId,
61 | });
62 | };
63 |
64 | export const useCreateGraduateAttribute = () => {
65 |     const qc = useQueryClient();
66 |     const { user } = useAuth();
67 |     return useMutation({
68 |         mutationFn: async (input: {
69 |             name: string;
70 |             description?: string;
71 |             sort_order?: number;
72 |         }) => {
73 |             const { data, error } = await supabase
74 |                 .from("graduate_attributes" as never)
75 |                 .insert({
76 |                     ...input,
77 |                     institution_id: user?.user_metadata?.institution_id,
78 |                 } as never)
79 |                 .select()
80 |                 .single();
81 |             if (error) throw error;
82 |             await logAuditEvent({
83 |                 action: "create",
84 |                 entity_type: "graduate_attribute",
85 |                 entity_id: (data as { id: string }).id,
86 |                 changes: input,
87 |                 performed_by: user?.id ?? "",
88 |             });
89 |             return data;
90 |         },
91 |         onSuccess: () =>
92 |             qc.invalidateQueries({ queryKey: queryKeys.graduateAttributes.lists() }),
93 |     });
94 | };
95 |
96 | export const useUpdateGraduateAttribute = () => {
97 |     const qc = useQueryClient();
98 |     const { user } = useAuth();
99 |     return useMutation({
100 |         mutationFn: async ({
101 |             id,
102 |             ...input
103 |         }: {
104 |             id: string;
105 |             name?: string;
106 |             description?: string;
107 |             sort_order?: number;
108 |         }) => {
109 |             const { data, error } = await supabase
110 |                 .from("graduate_attributes" as never)
111 |                 .update(input as never)
112 |                 .eq("id", id)
113 |                 .select()
114 |                 .single();
115 |             if (error) throw error;
116 |             await logAuditEvent({
117 |                 action: "update",
118 |                 entity_type: "graduate_attribute",
119 |                 entity_id: id,
120 |                 changes: input,
121 |                 performed_by: user?.id ?? "",
122 |             });
123 |             return data;
124 |         },
125 |         onSuccess: () =>
126 |             qc.invalidateQueries({ queryKey: queryKeys.graduateAttributes.lists() }),
127 |     });
128 | };
129 |
130 | export const useDeleteGraduateAttribute = () => {
131 |     const qc = useQueryClient();
132 |     const { user } = useAuth();
133 |     return useMutation({
134 |         mutationFn: async (id: string) => {
135 |             const { error } = await supabase
136 |                 .from("graduate_attributes" as never)
137 |                 .delete()
138 |                 .eq("id", id);
139 |             if (error) throw error;
140 |             await logAuditEvent({
141 |                 action: "delete",
142 |                 entity_type: "graduate_attribute",
143 |                 entity_id: id,
144 |                 changes: {},
145 |                 performed_by: user?.id ?? "",
146 |             });
147 |         },
148 |         onSuccess: () =>
149 |             qc.invalidateQueries({ queryKey: queryKeys.graduateAttributes.lists() }),
150 |     });
151 | };
152 |
153 | export const useGraduateAttributeAttainment = (institutionId?: string) => {
154 |     return useQuery({
155 |         queryKey: queryKeys.graduateAttributeAttainment.list({ institutionId }),

```



```

156 | queryFn: async (): Promise<GAAttainment[]> => {
157 |     const { data: attrs } = await supabase
158 |       .from("graduate_attributes" as never)
159 |       .select("id, name, description")
160 |       .eq("institution_id", institutionId!);
161 |     if (!attrs) return [];
162 |
163 |     const results: GAAttainment[] = [];
164 |     for (const attr of attrs as Array<{
165 |       id: string;
166 |       name: string;
167 |       description: string | null;
168 |     }>) {
169 |       const { data: mappings } = await supabase
170 |         .from("graduate_attribute_mappings" as never)
171 |         .select("outcome_id, weight")
172 |         .eq("graduate_attribute_id", attr.id);
173 |       const iloIds = (
174 |         mappings ?? []
175 |       ).map((m) => m.outcome_id);
176 |       if (iloIds.length === 0) {
177 |         results.push({
178 |           attribute_id: attr.id,
179 |           name: attr.name,
180 |           description: attr.description,
181 |           avg_attainment: 0,
182 |           mapped_ilo_count: 0,
183 |         });
184 |         continue;
185 |       }
186 |       const { data: attainments } = await supabase
187 |         .from("outcome_attainment" as never)
188 |         .select("outcome_id, attainment_percent")
189 |         .in("outcome_id", iloIds);
190 |       const attMap = new Map(
191 |         (
192 |           attainments ?? []
193 |         ) as Array<{
194 |           outcome_id: string;
195 |           attainment_percent: number;
196 |         }>
197 |       ).map((a) => [a.outcome_id, a.attainment_percent]);
198 |       let weightedSum = 0;
199 |       let totalWeight = 0;
200 |       for (const m of (mappings ?? []) as Array<{
201 |         outcome_id: string;
202 |         weight: number;
203 |       }>) {
204 |         const score = attMap.get(m.outcome_id) ?? 0;
205 |         weightedSum += score * m.weight;
206 |         totalWeight += m.weight;
207 |       }
208 |       results.push({
209 |         attribute_id: attr.id,
210 |         name: attr.name,
211 |         description: attr.description,
212 |         avg_attainment: totalWeight > 0 ? weightedSum / totalWeight : 0,
213 |         mapped_ilo_count: iloIds.length,
214 |       });
215 |     }
216 |     return results;
217 |   },
218 |   enabled: !!institutionId,
219 | });
220 | });
221 |

```

## FILE: src/hooks/useGuidedTour.ts

TYPE: .ts | LINES: 1-245 | SHA256: 53d13471a4ed42287d30ee1802454ddd87702b72f8484bf824a2d0a6a0db5980

```

1 | import { useCallback, useMemo } from "react";
2 | import { useTranslation } from "react-i18next";
3 | import { useTourStore } from "@stores/tourStore";
4 | import { useAuth } from "@hooks/useAuth";
5 | import type { UserRole } from "@types/app";
6 | import { adminTourSteps } from "@lib/tours/adminTour";
7 | import { coordinatorTourSteps } from "@lib/tours/coordinatorTour";
8 | import { teacherTourSteps } from "@lib/tours/teacherTour";
9 | import { studentTourSteps } from "@lib/tours/studentTour";
10 | import { parentTourSteps } from "@lib/tours/parentTour";
11 | import type { Step } from "react-joyride";
12 |
13 | /**
14 |  * Tour step configuration (compatible with react-joyride)
15 |  */
16 | export interface TourStep extends Step {
17 |   target: string; // CSS selector or data-tour attribute
18 |   content: string;
19 |   title?: string;
20 |   placement?: "top" | "bottom" | "left" | "right" | "center";
21 |   disableBeacon?: boolean;
22 | }
23 |
24 | /**
25 |  * Return type for useGuidedTour
26 |  */
27 | interface UseGuidedTourReturn {
28 |   /** Array of tour steps for the current role */
29 |   steps: TourStep[];
30 |   /** whether the tour is currently active */
31 |   run: boolean;
32 |   /** Current step index */
33 |   stepIndex: number;
34 |   /** Callback for tour events (step change, finish, etc.) */

```

```

35 | onCallback: (data: TourCallbackData) => void;
36 | /** Start the tour */
37 | startTour: () => void;
38 | /** Skip/dismiss the tour */
39 | skipTour: () => void;
40 | /** Complete the tour */
41 | completeTour: () => void;
42 | /** Whether the tour should auto-start on first login */
43 | shouldAutoStart: boolean;
44 | /** Whether the tour has been completed */
45 | isCompleted: boolean;
46 | /** Start the tour (alias for startTour) */
47 | start: () => void;
48 | }
49 |
50 | /**
51 |  * Callback data from react-joyride
52 |  */
53 | export interface TourCallbackData {
54 |   action: string;
55 |   index: number;
56 |   step: TourStep;
57 |   type: string;
58 |   status?: string;
59 | }
60 |
61 | /**
62 |  * Map of role to tour steps
63 |  */
64 | const getTourStepsForRole = (role: UserRole): TourStep[] => {
65 |   switch (role) {
66 |     case "admin":
67 |       return adminTourSteps as TourStep[];
68 |     case "coordinator":
69 |       return coordinatorTourSteps as TourStep[];
70 |     case "teacher":
71 |       return teacherTourSteps as TourStep[];
72 |     case "student":
73 |       return studentTourSteps as TourStep[];
74 |     case "parent":
75 |       return parentTourSteps as TourStep[];
76 |     default:
77 |       return [];
78 |   }
79 | };
80 |
81 | /**
82 |  * Hook for managing the guided tour
83 |  *
84 |  * Wraps react-joyride and tourStore to provide a simple interface for tour management.
85 |  *
86 |  * Features:
87 |  * - Reads role from AuthProvider
88 |  * - Selects appropriate tour steps for the role
89 |  * - Manages tour state (active, step index)
90 |  * - Persists tour completion to profiles.tour_completed_at
91 |  * - Respects prefers-reduced-motion
92 |  *
93 |  * Usage:
94 |  * `tsx
95 |  * const { steps, run, stepIndex, onCallback, startTour, skipTour, completeTour } = useGuidedTour();
96 |  *
97 |  * return (
98 |  *   <
99 |  *     <Joyride
100 |  *       steps={steps}
101 |  *       run={run}
102 |  *       stepIndex={stepIndex}
103 |  *       callback={onCallback}
104 |  *       continuous
105 |  *       showSkipButton
106 |  *     />
107 |  *     <button onClick={startTour}>Start Tour</button>
108 |  *   </>
109 |  * );
110 |  * `
111 |  */
112 | export const useGuidedTour = (roleOverride?: UserRole): UseGuidedTourReturn => {
113 |   const { t } = useTranslation("common");
114 |   const { user, profile } = useAuth();
115 |   const tourActive = useTourStore((state) => state.tourActive);
116 |   const tourFeatureFlag = useTourStore((state) => state.tourFeatureFlag);
117 |   const startTourAction = useTourStore((state) => state.start);
118 |   const skipTourAction = useTourStore((state) => state.skip);
119 |   const completeTourAction = useTourStore((state) => state.complete);
120 |
121 |   // Get the user's role (use override if provided, otherwise use profile role)
122 |   const userRole = (roleOverride || profile?.role) as UserRole | undefined;
123 |
124 |   // Get tour steps for the current role and translate the i18n keys.
125 |   // The raw step objects in src/lib/tours/*.ts store i18n keys (e.g.
126 |   // "tour.student.step1.title") so we look them up here against common.json.
127 |   // If a key is missing, t() returns the key itself, which is what was
128 |   // showing in the UI before this fix.
129 |   const steps = useMemo(() => {
130 |     if (!userRole || !tourFeatureFlag) return [];
131 |     const rawSteps = getTourStepsForRole(userRole);
132 |     return rawSteps.map((step) => ({
133 |       ...step,
134 |       content:
135 |         typeof step.content === "string"
136 |           ? t(step.content, { defaultValue: step.content })
137 |           : step.content,
138 |       title:

```

```

139 |         typeof step.title === "string"
140 |         ? t(step.title, { defaultValue: step.title })
141 |         : step.title,
142 |     ));
143 | }, [userRole, tourFeatureFlag, t]);
144 |
145 | // Determine if tour should run
146 | // Tour runs if:
147 | // 1. Feature flag is enabled
148 | // 2. Tour is active in the store
149 | // 3. User has not completed the tour (tour_completed_at is null)
150 | const shouldRun: boolean = useMemo(() => {
151 |     return (tourFeatureFlag &&
152 |         tourActive &&
153 |         user &&
154 |         profile &&
155 |         !profile.tour_completed_at) as boolean;
156 | }, [tourFeatureFlag, tourActive, user, profile]);
157 |
158 | // Determine if tour should auto-start
159 | const shouldAutoStart: boolean = useMemo(() => {
160 |     return (tourFeatureFlag &&
161 |         user &&
162 |         profile &&
163 |         !profile.tour_completed_at) as boolean;
164 | }, [tourFeatureFlag, user, profile]);
165 |
166 | // Determine if tour has been completed
167 | const isCompleted: boolean = useMemo(() => {
168 |     return (
169 |         profile?.tour_completed_at !== null &&
170 |         profile?.tour_completed_at !== undefined
171 |     );
172 | }, [profile?.tour_completed_at]);
173 |
174 | // Handle tour callbacks (step change, finish, etc.)
175 | const onCallback = useCallback(
176 |     (data: TourCallbackData) => {
177 |         const { action, status } = data;
178 |
179 |         // Handle tour completion
180 |         if (status === "finished" || action === "close") {
181 |             if (userRole) {
182 |                 completeTourAction(userRole);
183 |             }
184 |         }
185 |
186 |         // Handle tour skip
187 |         if (action === "skip") {
188 |             skipTourAction();
189 |         }
190 |     },
191 |     [userRole, completeTourAction, skipTourAction]
192 | );
193 |
194 | // Start the tour
195 | const startTour = useCallback(() => {
196 |     if (userRole && tourFeatureFlag) {
197 |         startTourAction(userRole);
198 |     }
199 | }, [userRole, tourFeatureFlag, startTourAction]);
200 |
201 | // Alias for startTour
202 | const start = startTour;
203 |
204 | // Skip the tour
205 | const skipTour = useCallback(() => {
206 |     skipTourAction();
207 | }, [skipTourAction]);
208 |
209 | // Complete the tour
210 | const completeTour = useCallback(() => {
211 |     if (userRole) {
212 |         completeTourAction(userRole);
213 |     }
214 | }, [userRole, completeTourAction]);
215 |
216 | return {
217 |     steps,
218 |     run: shouldRun,
219 |     stepIndex: 0, // Will be managed by react-joyride
220 |     onCallback,
221 |     startTour,
222 |     skipTour,
223 |     completeTour,
224 |     shouldAutoStart,
225 |     isCompleted,
226 |     start,
227 | };
228 | };
229 |
230 | /**
231 |  * Helper to suppress Radix Dialog/Sheet/Popover onInteractOutside while tour is active
232 |  *
233 |  * Usage:
234 |  * ```tsx
235 |  * const { tourActive } = useTourStore();
236 |  *
237 |  * <Dialog modal={!tourActive} onOpenChange={...}>
238 |  *   ...
239 |  * </Dialog>
240 |  * ```
241 |  */
242 | export const withTourAwareModal = (tourActive: boolean) => {

```

```
243 |   return !tourActive; // Return modal state (true = modal, false = non-modal)
244 | };
245 |
```

## FILE: src/hooks/useHabitAnalytics.ts

TYPE: .ts | LINES: 1-315 | SHA256: b4daeddbea4299a9808c2c3bcd31bb23bf4888f0bb96374303094633382849b6

```
1 | import { useMemo } from "react";
2 | import {
3 |   computeCompletionRate,
4 |   computeConsistencyScore,
5 |   computeDayOfWeekAverages,
6 |   getBestDay,
7 | } from "@lib/heatmapUtils";
8 | import { getLevelForDate } from "@lib/levelAwareHeatmap";
9 | import type {
10 |   HeatmapDay,
11 |   CompletionRateData,
12 |   DayOfWeekData,
13 |   LevelProgressionPoint,
14 | } from "@types/habits";
15 |
16 | /**
17 |  * Groups heatmap days into ISO weeks and computes completion rate per week.
18 |  */
19 | export const useWeeklyCompletionRates = (
20 |   heatmapData: HeatmapDay[] | undefined,
21 |   possiblePerDay: number
22 | ): CompletionRateData[] => {
23 |   return useMemo(() => {
24 |     if (!heatmapData || heatmapData.length === 0 || possiblePerDay <= 0)
25 |       return [];
26 |
27 |     const weekMap = new Map<string, HeatmapDay[]>();
28 |     for (const day of heatmapData) {
29 |       const d = new Date(day.date + "T00:00:00");
30 |       // ISO week: get the Thursday of the week to determine the week number
31 |       const thursday = new Date(d);
32 |       thursday.setDate(d.getDate() + (4 - (d.getDay() || 7)));
33 |       const yearStart = new Date(thursday.getFullYear(), 0, 1);
34 |       const weekNum = Math.ceil(
35 |         ((thursday.getTime() - yearStart.getTime()) / 86400000 + 1) / 7
36 |       );
37 |       const key = `Week ${weekNum}`;
38 |       const existing = weekMap.get(key) ?? [];
39 |       existing.push(day);
40 |       weekMap.set(key, existing);
41 |     }
42 |
43 |     return Array.from(weekMap.entries()).map(([period, days]) => {
44 |       const totalCompleted = days.reduce((sum, d) => sum + d.totalCount, 0);
45 |       const rate = computeCompletionRate(
46 |         totalCompleted,
47 |         possiblePerDay,
48 |         days.length
49 |       );
50 |       return { period, rate };
51 |     });
52 |   }, [heatmapData, possiblePerDay]);
53 | };
54 |
55 | /**
56 |  * Groups heatmap days into calendar months and computes completion rate per month.
57 |  */
58 | export const useMonthlyCompletionRates = (
59 |   heatmapData: HeatmapDay[] | undefined,
60 |   possiblePerDay: number
61 | ): CompletionRateData[] => {
62 |   return useMemo(() => {
63 |     if (!heatmapData || heatmapData.length === 0 || possiblePerDay <= 0)
64 |       return [];
65 |
66 |     const monthNames = [
67 |       "Jan",
68 |       "Feb",
69 |       "Mar",
70 |       "Apr",
71 |       "May",
72 |       "Jun",
73 |       "Jul",
74 |       "Aug",
75 |       "Sep",
76 |       "Oct",
77 |       "Nov",
78 |       "Dec",
79 |     ];
80 |     const monthMap = new Map<string, HeatmapDay[]>();
81 |
82 |     for (const day of heatmapData) {
83 |       const d = new Date(day.date + "T00:00:00");
84 |       const key = monthNames[d.getMonth()] ?? "";
85 |       const existing = monthMap.get(key) ?? [];
86 |       existing.push(day);
87 |       monthMap.set(key, existing);
88 |     }
89 |
90 |     return Array.from(monthMap.entries()).map(([period, days]) => {
91 |       const totalCompleted = days.reduce((sum, d) => sum + d.totalCount, 0);
92 |       const rate = computeCompletionRate(
93 |         totalCompleted,
94 |         possiblePerDay,
95 |         days.length
96 |       );
97 |       return { period, rate };

```

```

98 |     });
99 |   }, [heatmapData, possiblePerDay]);
100 | };
101 |
102 | /**
103 |  * Computes the consistency score (% of days with at least 1 habit).
104 |  */
105 | export const useConsistencyScore = (
106 |   heatmapData: HeatmapDay[] | undefined
107 | ): number => {
108 |   return useMemo(() => {
109 |     if (!heatmapData || heatmapData.length === 0) return 0;
110 |     return computeConsistencyScore(heatmapData, heatmapData.length);
111 |   }, [heatmapData]);
112 | };
113 |
114 | /**
115 |  * Computes day-of-week averages and identifies the best day.
116 |  */
117 | export const useBestDayOfWeek = (
118 |   heatmapData: HeatmapDay[] | undefined
119 | ): { averages: DayOfWeekData[]; bestDay: DayOfWeekData | null } => {
120 |   return useMemo(() => {
121 |     if (!heatmapData || heatmapData.length === 0) {
122 |       return { averages: [], bestDay: null };
123 |     }
124 |     const averages = computeDayOfWeekAverages(heatmapData);
125 |     const bestDay = getBestDay(averages);
126 |     return { averages, bestDay };
127 |   }, [heatmapData]);
128 | };
129 |
130 | /**
131 |  * Groups heatmap days into ISO weeks and computes completion rate per week
132 |  * using level-relative denominator via getLevelForDate.
133 |  */
134 | export const useLevelAwareWeeklyCompletionRates = (
135 |   heatmapData: HeatmapDay[] | undefined,
136 |   levelHistory: LevelProgressionPoint[],
137 |   enabledWellnessCount: number
138 | ): CompletionRateData[] => {
139 |   return useMemo(() => {
140 |     if (!heatmapData || heatmapData.length === 0 || levelHistory.length === 0)
141 |       return [];
142 |
143 |     const weekMap = new Map<string, HeatmapDay[]>();
144 |     for (const day of heatmapData) {
145 |       const d = new Date(day.date + "T00:00:00");
146 |       const thursday = new Date(d);
147 |       thursday.setDate(d.getDate() + (4 - (d.getDay() || 7)));
148 |       const yearStart = new Date(thursday.getFullYear(), 0, 1);
149 |       const weekNum = Math.ceil(
150 |         ((thursday.getTime() - yearStart.getTime()) / 86400000 + 1) / 7
151 |       );
152 |       const key = `Week ${weekNum}`;
153 |       const existing = weekMap.get(key) ?? [];
154 |       existing.push(day);
155 |       weekMap.set(key, existing);
156 |     }
157 |
158 |     return Array.from(weekMap.entries()).map(([period, days]) => {
159 |       const totalCompleted = days.reduce((sum, d) => sum + d.totalCount, 0);
160 |       const totalPossible = days.reduce((sum, d) => {
161 |         const levelMax = getLevelForDate(d.date, levelHistory);
162 |         return sum + levelMax + enabledWellnessCount;
163 |       }, 0);
164 |       const rate =
165 |         totalPossible > 0
166 |           ? Math.round((totalCompleted / totalPossible) * 100)
167 |           : 0;
168 |       return { period, rate };
169 |     });
170 |   }, [heatmapData, levelHistory, enabledWellnessCount]);
171 | };
172 |
173 | /**
174 |  * Groups heatmap days into calendar months and computes completion rate per month
175 |  * using level-relative denominator via getLevelForDate.
176 |  */
177 | export const useLevelAwareMonthlyCompletionRates = (
178 |   heatmapData: HeatmapDay[] | undefined,
179 |   levelHistory: LevelProgressionPoint[],
180 |   enabledWellnessCount: number
181 | ): CompletionRateData[] => {
182 |   return useMemo(() => {
183 |     if (!heatmapData || heatmapData.length === 0 || levelHistory.length === 0)
184 |       return [];
185 |
186 |     const monthNames = [
187 |       "Jan",
188 |       "Feb",
189 |       "Mar",
190 |       "Apr",
191 |       "May",
192 |       "Jun",
193 |       "Jul",
194 |       "Aug",
195 |       "Sep",
196 |       "Oct",
197 |       "Nov",
198 |       "Dec",
199 |     ];
200 |     const monthMap = new Map<string, HeatmapDay[]>();
201 |

```

```

202 |     for (const day of heatmapData) {
203 |         const d = new Date(day.date + "T00:00:00");
204 |         const key = monthNames[d.getMonth()] ?? "";
205 |         const existing = monthMap.get(key) ?? [];
206 |         existing.push(day);
207 |         monthMap.set(key, existing);
208 |     }
209 |
210 |     return Array.from(monthMap.entries()).map(([period, days]) => {
211 |         const totalCompleted = days.reduce((sum, d) => sum + d.totalCount, 0);
212 |         const totalPossible = days.reduce((sum, d) => {
213 |             const levelMax = getLevelForDate(d.date, levelHistory);
214 |             return sum + levelMax + enabledWellnessCount;
215 |         }, 0);
216 |         const rate =
217 |             totalPossible > 0
218 |             ? Math.round((totalCompleted / totalPossible) * 100)
219 |             : 0;
220 |         return { period, rate };
221 |     });
222 | }, [heatmapData, levelHistory, enabledWellnessCount]);
223 | };
224 |
225 | /**
226 |  * Academic-only level-aware weekly completion rates.
227 |  * Uses getLevelForDate as the per-day denominator (no wellness).
228 |  */
229 | export const useLevelAwareAcademicWeeklyRates = (
230 |     heatmapData: HeatmapDay[] | undefined,
231 |     levelHistory: LevelProgressionPoint[]
232 | ): CompletionRateData[] => {
233 |     return useMemo(() => {
234 |         if (!heatmapData || heatmapData.length === 0 || levelHistory.length === 0)
235 |             return [];
236 |
237 |         const weekMap = new Map<string, HeatmapDay[]>();
238 |         for (const day of heatmapData) {
239 |             const d = new Date(day.date + "T00:00:00");
240 |             const thursday = new Date(d);
241 |             thursday.setDate(d.getDate() + (4 - (d.getDay() || 7)));
242 |             const yearStart = new Date(thursday.getFullYear(), 0, 1);
243 |             const weekNum = Math.ceil(
244 |                 ((thursday.getTime() - yearStart.getTime()) / 86400000 + 1) / 7
245 |             );
246 |             const key = `Week ${weekNum}`;
247 |             const existing = weekMap.get(key) ?? [];
248 |             existing.push(day);
249 |             weekMap.set(key, existing);
250 |         }
251 |
252 |         return Array.from(weekMap.entries()).map(([period, days]) => {
253 |             const totalCompleted = days.reduce((sum, d) => sum + d.academicCount, 0);
254 |             const totalPossible = days.reduce((sum, d) => {
255 |                 return sum + getLevelForDate(d.date, levelHistory);
256 |             }, 0);
257 |             const rate =
258 |                 totalPossible > 0
259 |                 ? Math.round((totalCompleted / totalPossible) * 100)
260 |                 : 0;
261 |             return { period, rate };
262 |         });
263 |     }, [heatmapData, levelHistory]);
264 | };
265 |
266 | /**
267 |  * Academic-only level-aware monthly completion rates.
268 |  * Uses getLevelForDate as the per-day denominator (no wellness).
269 |  */
270 | export const useLevelAwareAcademicMonthlyRates = (
271 |     heatmapData: HeatmapDay[] | undefined,
272 |     levelHistory: LevelProgressionPoint[]
273 | ): CompletionRateData[] => {
274 |     return useMemo(() => {
275 |         if (!heatmapData || heatmapData.length === 0 || levelHistory.length === 0)
276 |             return [];
277 |
278 |         const monthNames = [
279 |             "Jan",
280 |             "Feb",
281 |             "Mar",
282 |             "Apr",
283 |             "May",
284 |             "Jun",
285 |             "Jul",
286 |             "Aug",
287 |             "Sep",
288 |             "Oct",
289 |             "Nov",
290 |             "Dec",
291 |         ];
292 |         const monthMap = new Map<string, HeatmapDay[]>();
293 |
294 |         for (const day of heatmapData) {
295 |             const d = new Date(day.date + "T00:00:00");
296 |             const key = monthNames[d.getMonth()] ?? "";
297 |             const existing = monthMap.get(key) ?? [];
298 |             existing.push(day);
299 |             monthMap.set(key, existing);
300 |         }
301 |
302 |         return Array.from(monthMap.entries()).map(([period, days]) => {
303 |             const totalCompleted = days.reduce((sum, d) => sum + d.academicCount, 0);
304 |             const totalPossible = days.reduce((sum, d) => {
305 |                 return sum + getLevelForDate(d.date, levelHistory);

```

```

306 |         }, 0);
307 |         const rate =
308 |             totalPossible > 0
309 |             ? Math.round((totalCompleted / totalPossible) * 100)
310 |             : 0;
311 |         return { period, rate };
312 |     });
313 | }, [heatmapData, levelHistory]);
314 | };
315 |

```

#### FILE: src/hooks/useHabitCorrelations.ts

TYPE: .ts | LINES: 1-68 | SHA256: 5e4cc39c4fc2bba84e8ca49c09cf817f513ec21268ec1703f6018ead1207ef50

```

1 | import { useQuery } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { getCorrelationConfidenceLevel } from "@lib/correlationConfidence";
5 | import type {
6 |     CorrelationInsightWithConfidence,
7 |     CorrelationConfidenceLevel,
8 |     HabitType,
9 | } from "@types/habits";
10 |
11 | interface CorrelationResult {
12 |     insights: CorrelationInsightWithConfidence[];
13 |     insufficientData: boolean;
14 |     daysUntilReady?: number;
15 |     dataPointCount?: number;
16 | }
17 |
18 | export const useHabitCorrelations = (studentId: string | undefined) => {
19 |     return useQuery({
20 |         queryKey: queryKeys.habitAnalytics.correlations(studentId ?? ""),
21 |         enabled: !!studentId,
22 |         staleTime: 5 * 60 * 1000, // 5 minutes - correlations don't change frequently
23 |         queryFn: async () => Promise<CorrelationResult> => {
24 |             if (!studentId) return { insights: [], insufficientData: true };
25 |
26 |             const { data, error } = await supabase.functions.invoke(
27 |                 "compute-habit-correlations",
28 |                 { body: { student_id: studentId } }
29 |             );
30 |
31 |             if (error) throw error;
32 |
33 |             const dataPointCount = data?.data_point_count as number | undefined;
34 |             const daysUntilReady = data?.days_until_ready as number | undefined;
35 |
36 |             // Map insights to include confidence data
37 |             const rawInsights = (data?.insights ?? []) as Array<{
38 |                 id: string;
39 |                 habitType: string;
40 |                 academicMetric: string;
41 |                 description: string;
42 |                 strength: number;
43 |                 confidenceLevel?: CorrelationConfidenceLevel;
44 |                 dataPointCount?: number;
45 |             }>;
46 |
47 |             const insights: CorrelationInsightWithConfidence[] = rawInsights.map(
48 |                 (insight) => ({
49 |                     ...insight,
50 |                     habitType: insight.habitType as HabitType,
51 |                     confidenceLevel:
52 |                         insight.confidenceLevel ??
53 |                         getCorrelationConfidenceLevel(dataPointCount ?? 0) ??
54 |                         "early_pattern",
55 |                     dataPointCount: insight.dataPointCount ?? dataPointCount ?? 0,
56 |                 })
57 |             );
58 |
59 |             return {
60 |                 insights,
61 |                 insufficientData: data?.insufficient_data === true,
62 |                 daysUntilReady,
63 |                 dataPointCount,
64 |             };
65 |         },
66 |     });
67 | };
68 |

```

#### FILE: src/hooks/useHabitDifficulty.ts

TYPE: .ts | LINES: 1-42 | SHA256: 8cda7d809f9d40cfe257a113422cc150fcb9e54c432451f222da933cab9c6d

```

1 | // Task 145.7: Habit Difficulty Level TanStack Query hook
2 |
3 | import { useQuery } from "@tanstack/react-query";
4 | import { supabase } from "@lib/supabase";
5 | import { queryKeys } from "@lib/queryKeys";
6 | import { DASHBOARD_STALE_TIME_MS } from "@lib/queryConfig";
7 |
8 | export interface HabitDifficultyData {
9 |     level: number;
10 |     habit_level_streak: number;
11 | }
12 |
13 | export const useHabitDifficultyLevel = (studentId: string | undefined) => {
14 |     return useQuery({
15 |         queryKey: queryKeys.studentHabitLevel.detail(studentId ?? ""),
16 |         queryFn: async () => Promise<HabitDifficultyData> => {
17 |             const { data, error } = await supabase
18 |                 .from("student_gamification")

```

```

19 |         .select("habit_difficulty_level, habit_level_streak")
20 |         .eq("student_id", studentId!)
21 |         .maybeSingle();
22 |
23 |     if (error) throw error;
24 |     if (!data) return { level: 1, habit_level_streak: 0 };
25 |
26 |     const row = data as unknown as Record<string, unknown>;
27 |     return {
28 |       level:
29 |         typeof row.habit_difficulty_level === "number"
30 |         ? row.habit_difficulty_level
31 |         : 1,
32 |       habit_level_streak:
33 |         typeof row.habit_level_streak === "number"
34 |         ? row.habit_level_streak
35 |         : 0,
36 |     };
37 |   },
38 |   enabled: !!studentId,
39 |   staleTime: DASHBOARD_STALE_TIME_MS,
40 | });
41 | });
42 |

```

## FILE: src/hooks/useHabitExport.ts

TYPE: .ts | LINES: 1-119 | SHA256: 6cc4452697a59ae4f20ecfd37e840b3c5d30d105a2705b95382b93ca0a343e22

```

1 | import { useCallback } from "react";
2 | import { useQueryClient } from "@tanstack/react-query";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { generateHabitCSV, downloadCSV } from "@lib/habitExport";
5 | import { toast } from "sonner";
6 | import type {
7 |   HeatmapDay,
8 |   HeatmapSummary,
9 |   HabitReportRow,
10 |   WellnessHabitType,
11 |   DateRange,
12 |   LevelProgressionPoint,
13 | } from "@types/habits";
14 |
15 | interface UseHabitExportOptions {
16 |   studentId: string | undefined;
17 |   semesterRange: DateRange;
18 |   enabledWellnessHabits: WellnessHabitType[];
19 |   filter?: string;
20 |   levelHistory?: LevelProgressionPoint[];
21 | }
22 |
23 | export const useHabitExport = ({
24 |   studentId,
25 |   semesterRange,
26 |   enabledWellnessHabits,
27 |   filter,
28 |   levelHistory,
29 | }: UseHabitExportOptions) => {
30 |   const queryClient = useQueryClient();
31 |
32 |   const exportCSV = useCallback(() => {
33 |     if (!studentId) {
34 |       toast.error("Unable to export – no student data");
35 |       return;
36 |     }
37 |
38 |     // Read cached heatmap data from TanStack Query
39 |     const heatmapData = queryClient.getQueryData<HeatmapDay[]>(
40 |       queryKeys.heatmap.data(
41 |         studentId,
42 |         semesterRange.start,
43 |         semesterRange.end,
44 |         filter
45 |       )
46 |     );
47 |
48 |     const summaryData = queryClient.getQueryData<HeatmapSummary>(
49 |       queryKeys.heatmap.summary(studentId)
50 |     );
51 |
52 |     if (!heatmapData || !summaryData) {
53 |       toast.error("No data available to export – please wait for data to load");
54 |       return;
55 |     }
56 |
57 |     // Transform HeatmapDay[] into HabitReportRow[]
58 |     const rows: HabitReportRow[] = heatmapData.map((day) => {
59 |       const academicHabits = day.habits.filter(
60 |         (h) => h.category === "academic"
61 |       );
62 |       const wellnessHabits = day.habits.filter(
63 |         (h) => h.category === "wellness"
64 |       );
65 |
66 |       const row: HabitReportRow = {
67 |         date: day.date,
68 |         login: academicHabits.some((h) => h.type === "login"),
69 |         submit: academicHabits.some((h) => h.type === "submit"),
70 |         journal: academicHabits.some((h) => h.type === "journal"),
71 |         read: academicHabits.some((h) => h.type === "read"),
72 |         totalHabits: day.totalCount,
73 |         xpEarned: 0, // XP per day not tracked in heatmap data
74 |         streakActive: day.academicCount > 0,
75 |       };
76 |

```



```

77 |         // Add wellness habit columns
78 |         for (const wh of enabledWellnessHabits) {
79 |             (row as unknown as Record<string, unknown>)[wh] = wellnessHabits.some(
80 |                 (h) => h.type === wh
81 |             );
82 |         }
83 |
84 |         return row;
85 |     });
86 |
87 |     const csv = generateHabitCSV(
88 |         rows,
89 |         {
90 |             ...summaryData,
91 |             totalXP: 0,
92 |             consistencyScore:
93 |                 heatmapData.length > 0
94 |                 ? Math.round(
95 |                     (heatmapData.filter((d) => d.totalCount > 0).length /
96 |                     heatmapData.length) *
97 |                     100
98 |                 )
99 |                 : 0,
100 |         },
101 |         enabledWellnessHabits,
102 |         levelHistory
103 |     );
104 |
105 |     const filename = `habit-report-${semesterRange.start}-to-${semesterRange.end}.csv`;
106 |     downloadCSV(csv, filename);
107 |     toast.success("Habit report exported");
108 | }, [
109 |     studentId,
110 |     semesterRange,
111 |     enabledWellnessHabits,
112 |     filter,
113 |     levelHistory,
114 |     queryClient,
115 | ]);
116 |
117 | return { exportCSV };
118 | };
119 |

```

#### FILE: src/hooks/useHeatmapData.ts

TYPE: .ts | LINES: 1-292 | SHA256: 28725f5a409a33b479a37b47a3470eb639a10aa1004e31011ba3f699e6d27cbc

```

1 | import { useQuery } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import {
5 |     computeLongestStreak,
6 |     aggregateHabitXpByDate,
7 | } from "@lib/heatmapUtils";
8 | import type {
9 |     DateRange,
10 |     HeatmapDay,
11 |     HeatmapSummary,
12 |     CompletedHabit,
13 |     AcademicHabitType,
14 |     WellnessHabitType,
15 |     LevelProgressionPoint,
16 | } from "@types/habits";
17 |
18 | const ACADEMIC_HABITS: AcademicHabitType[] = [
19 |     "login",
20 |     "submit",
21 |     "journal",
22 |     "read",
23 | ];
24 | const WELLNESS_HABITS: WellnessHabitType[] = [
25 |     "meditation",
26 |     "hydration",
27 |     "exercise",
28 |     "sleep",
29 | ];
30 |
31 | const isAcademicHabit = (type: string): type is AcademicHabitType =>
32 |     (ACADEMIC_HABITS as string[]).includes(type);
33 |
34 | const isWellnessHabit = (type: string): type is WellnessHabitType =>
35 |     (WELLNESS_HABITS as string[]).includes(type);
36 |
37 | export const useHeatmapData = (
38 |     studentId: string | undefined,
39 |     semesterRange: DateRange,
40 |     filter?: string
41 | ) => {
42 |     return useQuery({
43 |         queryKey: queryKeys.heatmap.data(
44 |             studentId ?? "",
45 |             semesterRange.start,
46 |             semesterRange.end,
47 |             filter
48 |         ),
49 |         enabled: !!studentId && !!semesterRange.start && !!semesterRange.end,
50 |         queryFn: async (): Promise<HeatmapDay[]> => {
51 |             if (!studentId) return [];
52 |
53 |             const dateMap = new Map<string, CompletedHabit[]>();
54 |
55 |             // Determine if we're filtering to a specific habit
56 |             const specificFilter =
57 |                 filter && filter !== "all" ? filter.toLowerCase() : undefined;

```

```

58 |     const filterIsAcademic = specificFilter
59 |       ? isAcademicHabit(specificFilter)
60 |       : false;
61 |     const filterIsWellness = specificFilter
62 |       ? isWellnessHabit(specificFilter)
63 |       : false;
64 |
65 |     // Fetch academic habit logs (unless filtering to a specific wellness habit)
66 |     // Academic habits live in the canonical long-format 'habit_logs' table
67 |     // (one row per completed habit per UTC day), keyed by 'habit_type'.
68 |     if (!filterIsWellness) {
69 |       const { data: academicRows, error: academicError } = await supabase
70 |         .from("habit_logs")
71 |         .select("date, habit_type, completed_at, created_at")
72 |         .eq("student_id", studentId)
73 |         .gte("date", semesterRange.start)
74 |         .lte("date", semesterRange.end);
75 |
76 |       if (academicError) throw academicError;
77 |
78 |       for (const row of academicRows ?? []) {
79 |         if (!isAcademicHabit(row.habit_type)) continue;
80 |         const habitType: AcademicHabitType = row.habit_type;
81 |
82 |         if (
83 |           filterIsAcademic &&
84 |           specificFilter &&
85 |           specificFilter !== habitType
86 |         )
87 |           continue;
88 |
89 |         const date = row.date;
90 |         const habits = dateMap.get(date) ?? [];
91 |         habits.push({
92 |           type: habitType,
93 |           category: "academic",
94 |           completedAt: row.completed_at ?? row.created_at,
95 |         });
96 |         dateMap.set(date, habits);
97 |       }
98 |     }
99 |
100 |    // Fetch wellness habit logs (unless filtering to a specific academic habit)
101 |    if (!filterIsAcademic) {
102 |      let wellnessQuery = supabase
103 |        .from("wellness_habit_logs")
104 |        .select("date, wellness_type, value, completed_at")
105 |        .eq("student_id", studentId)
106 |        .gte("date", semesterRange.start)
107 |        .lte("date", semesterRange.end);
108 |
109 |      if (filterIsWellness && specificFilter) {
110 |        wellnessQuery = wellnessQuery.eq(
111 |          "wellness_type",
112 |          specificFilter as WellnessHabitType
113 |        );
114 |      }
115 |
116 |      const { data: wellnessLogs, error: wellnessError } =
117 |        await wellnessQuery;
118 |      if (wellnessError) throw wellnessError;
119 |
120 |      for (const log of wellnessLogs ?? []) {
121 |        const date = log.date as string;
122 |        const habits = dateMap.get(date) ?? [];
123 |        habits.push({
124 |          type: log.wellness_type as WellnessHabitType,
125 |          category: "wellness",
126 |          value: log.value !== null ? Number(log.value) : undefined,
127 |          completedAt: log.completed_at as string,
128 |        });
129 |        dateMap.set(date, habits);
130 |      }
131 |    }
132 |
133 |    // Build HeatmapDay array for every date in the range
134 |    const days: HeatmapDay[] = [];
135 |    const cursor = new Date(semesterRange.start + "T00:00:00");
136 |    const end = new Date(semesterRange.end + "T00:00:00");
137 |
138 |    while (cursor <= end) {
139 |      const dateStr = cursor.toISOString().slice(0, 10);
140 |      const habits = dateMap.get(dateStr) ?? [];
141 |      const academicCount = habits.filter(
142 |        (h) => h.category === "academic"
143 |      ).length;
144 |      const wellnessCount = habits.filter(
145 |        (h) => h.category === "wellness"
146 |      ).length;
147 |
148 |      days.push({
149 |        date: dateStr,
150 |        academicCount,
151 |        wellnessCount,
152 |        totalCount: academicCount + wellnessCount,
153 |        habits,
154 |      });
155 |
156 |      cursor.setDate(cursor.getDate() + 1);
157 |    }
158 |
159 |    return days;
160 |  },
161 |  });

```

```

162 | };
163 |
164 | /**
165 |  * Fetches habit-related XP transactions for the student within the semester
166 |  * range and aggregates them into a per-date total. Backs the heatmap tooltip
167 |  * and bottom sheet so each day shows its actually-recorded habit XP rather than
168 |  * a hardcoded zero (R7.1, R7.4).
169 |  */
170 | export const useHeatmapXpByDate = (
171 |   studentId: string | undefined,
172 |   semesterRange: DateRange
173 | ) => {
174 |   return useQuery({
175 |     queryKey: queryKeys.heatmap.xpByDate(
176 |       studentId ?? "",
177 |       semesterRange.start,
178 |       semesterRange.end
179 |     ),
180 |     enabled: !!studentId && !!semesterRange.start && !!semesterRange.end,
181 |     queryFn: async (): Promise<Record<string, number>> => {
182 |       if (!studentId) return {};
183 |
184 |       const startIso = semesterRange.start + "T00:00:00.000Z";
185 |       // Include the entire end day by extending to the next midnight.
186 |       const endExclusive = new Date(semesterRange.end + "T00:00:00.000Z");
187 |       endExclusive.setUTCDate(endExclusive.getUTCDate() + 1);
188 |
189 |       const { data, error } = await supabase
190 |         .from("xp_transactions")
191 |         .select("source, xp_amount, created_at")
192 |         .eq("student_id", studentId)
193 |         .gte("created_at", startIso)
194 |         .lt("created_at", endExclusive.toISOString());
195 |
196 |       if (error) throw error;
197 |
198 |       const records = (data ?? []).map((row) => ({
199 |         source: row.source,
200 |         amount: row.xp_amount,
201 |         date:
202 |           typeof row.created_at === "string" ? row.created_at.slice(0, 10) : "",
203 |       }));
204 |
205 |       return aggregateHabitXpByDate(records);
206 |     },
207 |   });
208 | };
209 |
210 | export const useHeatmapSummary = (
211 |   studentId: string | undefined,
212 |   heatmapData: HeatmapDay[] | undefined
213 | ) => {
214 |   return useQuery({
215 |     queryKey: queryKeys.heatmap.summary(studentId ?? ""),
216 |     enabled: !!studentId && !!heatmapData,
217 |     queryFn: async (): Promise<HeatmapSummary> => {
218 |       // Fetch current streak from student_gamification
219 |       let currentStreak = 0;
220 |       if (studentId) {
221 |         const { data, error } = await supabase
222 |           .from("student_gamification")
223 |           .select("streak_current")
224 |           .eq("student_id", studentId)
225 |           .maybeSingle();
226 |
227 |         if (error) throw error;
228 |         currentStreak = data?.streak_current ?? 0;
229 |       }
230 |
231 |       const days = heatmapData ?? [];
232 |       const longestStreak = computeLongestStreak(days);
233 |       const totalActiveDays = days.filter((d) => d.totalCount > 0).length;
234 |
235 |       return {
236 |         currentStreak,
237 |         longestStreak,
238 |         totalActiveDays,
239 |       };
240 |     },
241 |   });
242 | };
243 |
244 | /**
245 |  * Fetches the student's habit level history for per-date level resolution.
246 |  * Used by HeatmapGrid to determine the level active on each date.
247 |  * Falls back to empty array if the table doesn't exist yet.
248 |  */
249 | export const useHeatmapLevelHistory = (
250 |   studentId: string | undefined,
251 |   semesterRange: DateRange
252 | ) => {
253 |   return useQuery({
254 |     queryKey: queryKeys.heatmap.levelHistory(
255 |       studentId ?? "",
256 |       semesterRange.start,
257 |       semesterRange.end
258 |     ),
259 |     enabled: !!studentId && !!semesterRange.start && !!semesterRange.end,
260 |     queryFn: async (): Promise<LevelProgressionPoint[]> => {
261 |       if (!studentId) return [];
262 |
263 |       try {
264 |         const { data } = await supabase
265 |           .from("student_habit_level_history" as never)

```

```

266 |         .select("changed_at, new_level")
267 |         .eq("student_id", studentId)
268 |         .lte("changed_at", semesterRange.end)
269 |         .order("changed_at", { ascending: true });
270 |
271 |     if (!Array.isArray(data)) return [];
272 |
273 |     return data
274 |       .map((row) => {
275 |         const r = row as Record<string, unknown>;
276 |         const date =
277 |           typeof r.changed_at === "string" ? r.changed_at.slice(0, 10) : "";
278 |         const level = typeof r.new_level === "number" ? r.new_level : 4;
279 |         return {
280 |           date,
281 |           level: (level >= 1 && level <= 4 ? level : 4) as 1 | 2 | 3 | 4,
282 |         };
283 |       })
284 |       .filter((p) => p.date !== "");
285 |   } catch {
286 |     // Table may not exist yet – return empty history (defaults to Level 4)
287 |     return [];
288 |   }
289 | },
290 | });
291 | };
292 |

```

## FILE: src/hooks/useHistoricalEvidence.ts

TYPE: .ts | LINES: 1-91 | SHA256: aceae094b0abd7e4c7069e25bd1be9512638d85e4033c9fb4814d69c85e22986

```

1 | // Feature: qa-partner-review-remediation – Req 4 (B4);
2 | // production-bug-fixes – Req 10 (API hardening).
3 | // Data layer for the Admin Historical Evidence dashboard. Reads the
4 | // historical-evidence rollup via the admin-gated `get_historical_evidence` RPC.
5 | // The backing `mv_historical_evidence` materialized view is no longer directly
6 | // selectable from the API (its SELECT grant was revoked), so the RPC enforces
7 | // the admin-only scope. Aggregate columns can be NULL for an empty group, so
8 | // rows are normalized into the non-null `HistoricalEvidenceRow` shape the
9 | // dashboard renders, coalescing missing numerics to 0.
10 |
11 | import { useQuery } from "@tanstack/react-query";
12 | import { supabase } from "@lib/supabase";
13 | import { queryKeys } from "@lib/queryKeys";
14 | import type { HistoricalEvidenceFilter } from "@lib/schemas/historicalEvidence";
15 |
16 | export interface HistoricalEvidenceRow {
17 |   semester_id: string;
18 |   semester_name: string;
19 |   start_date: string;
20 |   outcome_type: string;
21 |   blooms_level: string | null;
22 |   evidence_count: number;
23 |   avg_score: number;
24 |   excellent_count: number;
25 |   satisfactory_count: number;
26 |   developing_count: number;
27 |   not_yet_count: number;
28 | }
29 |
30 | /**
31 |  * Reads the historical-evidence rollup via the admin-gated
32 |  * `get_historical_evidence` RPC, ordered by `start_date` ascending with the
33 |  * optional outcome-type / Bloom's-level filters applied server-side. The
34 |  * underlying `mv_historical_evidence` materialized view is no longer directly
35 |  * selectable from the API (production-bug-fixes Req 10); the RPC enforces the
36 |  * admin-only scope. Throws on error so the caller's query error state surfaces.
37 |  *
38 |  * `get_historical_evidence` is not in the generated `Database` types yet (it is
39 |  * added by the post-merge type regen), so – per the repo precedent
40 |  * (`useStudentDashboardAggregate`) – only the `rpc` surface is cast (no `any`,
41 |  * no builder cast). Remove the shim once the RPC is in the generated types.
42 |  */
43 | interface HistoricalEvidenceRpcRow {
44 |   semester_id: string | null;
45 |   semester_name: string | null;
46 |   start_date: string | null;
47 |   outcome_type: string | null;
48 |   blooms_level: string | null;
49 |   evidence_count: number | null;
50 |   avg_score: number | null;
51 |   excellent_count: number | null;
52 |   satisfactory_count: number | null;
53 |   developing_count: number | null;
54 |   not_yet_count: number | null;
55 | }
56 | type HistoricalEvidenceRpc = (
57 |   fn: "get_historical_evidence",
58 |   args: { p_outcome_type: string | null; p_blooms_level: string | null }
59 | ) => PromiseLike<{
60 |   data: HistoricalEvidenceRpcRow[] | null;
61 |   error: { message: string } | null;
62 | }>;
63 |
64 | export const useHistoricalEvidence = (filters: HistoricalEvidenceFilter) =>
65 |   useQuery({
66 |     queryKey: queryKeys.historicalEvidence.list(filters),
67 |     queryFn: async (): Promise<HistoricalEvidenceRow[]> => {
68 |       const rpc = (supabase as unknown as { rpc: HistoricalEvidenceRpc }).rpc;
69 |       const { data, error } = await rpc("get_historical_evidence", {
70 |         p_outcome_type: filters.outcomeType ?? null,
71 |         p_blooms_level: filters.bloomsLevel ?? null,
72 |       });
73 |       if (error) throw error;

```

```

74 |
75 |     return (data ?? []).map((row) => ({
76 |       semester_id: row.semester_id ?? "",
77 |       semester_name: row.semester_name ?? "",
78 |       start_date: row.start_date ?? "",
79 |       outcome_type: row.outcome_type ?? "",
80 |       blooms_level: row.blooms_level ?? null,
81 |       evidence_count: Number(row.evidence_count ?? 0),
82 |       avg_score: Number(row.avg_score ?? 0),
83 |       excellent_count: Number(row.excellent_count ?? 0),
84 |       satisfactory_count: Number(row.satisfactory_count ?? 0),
85 |       developing_count: Number(row.developing_count ?? 0),
86 |       not_yet_count: Number(row.not_yet_count ?? 0),
87 |     }));
88 |   },
89 |   staleTime: 5 * 60_000,
90 | });
91 |

```

FILE: src/hooks/useIL0s.ts

```
TYPE: .ts | LINES: 1-230 | SHA256: 4f4645354d94e6ed1b62e5762a28247cfb96c7804a6bc4378d8b9adc0579d96d
```

[illegible]

[illegible]

```

187 |     queryClient.invalidateQueries({ queryKey: queryKeys.ilos.lists() });
188 |   },
189 |   });
190 | };
191 |
192 | // % % % useReorderILOS - batch update sort_order % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
193 |
194 | export const useReorderILOS = () => {
195 |   const queryClient = useQueryClient();
196 |   const { user } = useAuth();
197 |
198 |   return useMutation({
199 |     mutationFn: async (data: ReorderFormData): Promise<void> => {
200 |       const updates = data.items.map((item, index) => ({
201 |         id: item.id,
202 |         sort_order: index,
203 |       }));
204 |
205 |       // Partial upsert: ON CONFLICT (id) only updates sort_order.
206 |       // Cast needed because Supabase Insert type requires `type` and `title`,
207 |       // but PostgreSQL's ON CONFLICT clause correctly handles partial columns.
208 |       const { error } = await supabase
209 |         .from("learning_outcomes")
210 |         .upsert(
211 |           updates as Database["public"]["Tables"]["learning_outcomes"]["Insert"][],
212 |           { onConflict: "id" }
213 |         );
214 |
215 |       if (error) throw error;
216 |
217 |       await logAuditEvent({
218 |         action: "reorder",
219 |         entity_type: "ilo",
220 |         entity_id: "batch",
221 |         changes: { items: data.items },
222 |         performed_by: user?.id ?? "unknown",
223 |       });
224 |     },
225 |     onSuccess: () => {
226 |       queryClient.invalidateQueries({ queryKey: queryKeys.ilos.lists() });
227 |     },
228 |   });
229 | };
230 |

```

FILE: src/hooks/useImpersonation.ts

TYPE: .ts | LINES: 1-22 | SHA256: 761cd22767ff23c91d50af6b76a4592e41b2841529ca32f78a01309724b07fb7

```

1 // Task 88.5: Impersonation hook wrapping ImpersonationProvider context
2
3 import { useContext } from "react";
4 import { ImpersonationContext } from "@providers/ImpersonationProvider";
5 import type {
6   ImpersonationContextValue,
7   ImpersonatedUser,
8 } from "@providers/ImpersonationProvider";
9
10 export const useImpersonation = (): ImpersonationContextValue => {
11   const context = useContext(ImpersonationContext);
12   if (!context) {
13     throw new Error(
14       "useImpersonation must be used within an ImpersonationProvider"
15     );
16   }
17   return context;
18 };
19
20 // Re-export type for convenience
21 export type { ImpersonatedUser };
22

```

FILE: src/hooks/useIndependenceScore.ts

TYPE: .ts | LINES: 1-172 | SHA256: d10216c017c96e996f8bd137189930a103414cfac471360801defb3e9176de95

[illegible]

[illegible]



```

135 |     }
136 |
137 |     // Count AI-assisted submissions: those preceded by a tutor conversation
138 |     // on the same CLO within 1 hour before submission
139 |     let aiAssisted = 0;
140 |     const ONE_HOUR_MS = 60 * 60 * 1000;
141 |
142 |     for (const sub of submissions) {
143 |         const submittedAt = new Date(sub.submitted_at).getTime();
144 |
145 |         const wasAiAssisted = (conversations ?? []).some(
146 |             (conv: { updated_at: string }) => {
147 |                 const convTime = new Date(conv.updated_at).getTime();
148 |                 const timeDiff = submittedAt - convTime;
149 |                 return timeDiff >= 0 && timeDiff <= ONE_HOUR_MS;
150 |             }
151 |         );
152 |
153 |         if (wasAiAssisted) {
154 |             aiAssisted++;
155 |         }
156 |     }
157 |
158 |     const totalSubmissions = submissions.length;
159 |     const score =
160 |         totalSubmissions > 0 ? 1 - aiAssisted / totalSubmissions : 1.0;
161 |
162 |     results.push({
163 |         clo_id: cloId,
164 |         score: Math.max(0, Math.min(1, score)),
165 |         total_submissions: totalSubmissions,
166 |         ai_assisted_submissions: aiAssisted,
167 |     });
168 | }
169 |
170 | return results;
171 | }
172 |

```

#### FILE: src/hooks/useInstitutionBrowse.ts

TYPE: .ts | LINES: 1-88 | SHA256: e9f81f8298740776219c1ab2fdab65988369e9ce19c7436e0c8003cfa4fc408e

```

1 | import { useQuery } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 |
5 | /**
6 |  * Institution data for browsing during signup
7 |  */
8 | export interface BrowsableInstitution {
9 |     id: string;
10 |     slug: string;
11 |     name: string;
12 |     logo_url: string | null;
13 |     join_mode: "open" | "invite_only" | "domain_restricted";
14 | }
15 |
16 | /**
17 |  * Return type for useInstitutionBrowse
18 |  */
19 | interface UseInstitutionBrowseReturn {
20 |     /** List of institutions available for signup */
21 |     data: BrowsableInstitution[] | undefined;
22 |     /** Whether the query is loading */
23 |     isLoading: boolean;
24 |     /** Error if the query failed */
25 |     error: Error | null;
26 | }
27 |
28 | /**
29 |  * Hook for browsing institutions during signup
30 |  *
31 |  * Queries the public `institutions_public` view which exposes:
32 |  * - id
33 |  * - slug
34 |  * - name
35 |  * - logo_url
36 |  * - join_mode
37 |  *
38 |  * This view is accessible to unauthenticated users (public-anon SELECT policy).
39 |  *
40 |  * Caching strategy:
41 |  * - staleTime: 5 minutes (institutions don't change frequently)
42 |  * - gcTime: 10 minutes (keep in cache for 10 minutes after last use)
43 |  *
44 |  * Usage:
45 |  * ``tsx
46 |  * const { data: institutions, isLoading, error } = useInstitutionBrowse();
47 |  *
48 |  * return (
49 |  *     <div>
50 |  *         {isLoading && <p>Loading institutions...</p>}
51 |  *         {error && <p>Error: {error.message}</p>}
52 |  *         {institutions?.map((inst) => (
53 |  *             <div key={inst.id}>
54 |  *                 {inst.logo_url && <img src={inst.logo_url} alt={inst.name} />}
55 |  *                 <h3>{inst.name}</h3>
56 |  *                 <p>{inst.join_mode}</p>
57 |  *             </div>
58 |  *         ))}
59 |  *     </div>
60 |  * );
61 |  * ``
62 |  */

```

```

63 | export const useInstitutionBrowse = (): UseInstitutionBrowseReturn => {
64 |   const query = useQuery({
65 |     queryKey: queryKeys.institutions.lists(),
66 |     queryFn: async () => {
67 |       const { data, error } = await supabase
68 |         .from("institutions_public")
69 |         .select("id, slug, name, logo_url, join_mode")
70 |         .order("name", { ascending: true });
71 |
72 |       if (error) {
73 |         throw error;
74 |       }
75 |
76 |       return (data || []) as BrowsableInstitution[];
77 |     },
78 |     staleTime: 5 * 60 * 1000, // 5 minutes
79 |     gcTime: 10 * 60 * 1000, // 10 minutes (formerly cacheTime)
80 |   });
81 |
82 |   return {
83 |     data: query.data,
84 |     isLoading: query.isLoading,
85 |     error: query.error,
86 |   };
87 | };
88 |

```

## FILE: src/hooks/useInstitutionSettings.ts

TYPE: .ts | LINES: 1-285 | SHA256: c195dd16bc456770fe1417a2b780c1b45e1eb7bcc520932a877565b16f9f91f6

```

1 | // =====
2 | // useInstitutionSettings - TanStack Query hooks for institution settings & accreditations
3 | // =====
4 |
5 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
6 | import { supabase } from "@lib/supabase";
7 | import { queryKeys } from "@lib/queryKeys";
8 | import { logAuditEvent } from "@lib/auditLogger";
9 | import { useAuth } from "@hooks/useAuth";
10 | import { toast } from "sonner";
11 | import type { InstitutionSettings, ProgramAccreditation } from "@types/app";
12 | import type { InstitutionSettingsFormData } from "@lib/schemas/institutionSettings";
13 |
14 | // % % % useInstitutionSettings - fetch current institution settings % % % % % % % % % %
15 |
16 | export const useInstitutionSettings = () => {
17 |   const { institutionId } = useAuth();
18 |
19 |   return useQuery({
20 |     queryKey: queryKeys.institutionSettings.list({ institutionId }),
21 |     queryFn: async (): Promise<InstitutionSettings | null> => {
22 |       if (!institutionId) return null;
23 |
24 |       const { data, error } = await supabase
25 |         .from("institution_settings")
26 |         .select("")
27 |         .eq("institution_id", institutionId)
28 |         .maybeSingle();
29 |
30 |       if (error) throw error;
31 |       return data as unknown as InstitutionSettings | null;
32 |     },
33 |     enabled: !!institutionId,
34 |     // Institution settings change rarely (admin-only mutation) - 5 min staleTime
35 |     // prevents redundant fetches on every page mount (the dashboard, leaderboard,
36 |     // and sidebar all consume this hook).
37 |     staleTime: 5 * 60_000,
38 |   });
39 | };
40 |
41 | // % % % useUpsertInstitutionSettings - create or update settings % % % % % % % % % %
42 |
43 | export const useUpsertInstitutionSettings = () => {
44 |   const qc = useQueryClient();
45 |   const { user, institutionId } = useAuth();
46 |
47 |   return useMutation({
48 |     mutationFn: async (
49 |       input: InstitutionSettingsFormData
50 |     ): Promise<InstitutionSettings> => {
51 |       if (!institutionId) throw new Error("No institution context");
52 |
53 |       // Check if settings already exist
54 |       const { data: existing } = await supabase
55 |         .from("institution_settings")
56 |         .select("id")
57 |         .eq("institution_id", institutionId)
58 |         .maybeSingle();
59 |
60 |       const payload = {
61 |         institution_id: institutionId,
62 |         attainment_thresholds: input.attainment_thresholds,
63 |         success_threshold: input.success_threshold,
64 |         accreditation_body: input.accreditation_body,
65 |         grade_scales: input.grade_scales,
66 |         streak_sabbatical_enabled: input.streak_sabbatical_enabled ?? false,
67 |         league_thresholds: input.league_thresholds ?? undefined,
68 |         default_language: input.default_language ?? "en",
69 |       };
70 |
71 |       let result: InstitutionSettings;
72 |
73 |       if (existing) {
74 |         const { data, error } = await supabase

```

```

75 |         .from("institution_settings")
76 |         .update(payload)
77 |         .eq("id", existing.id)
78 |         .select()
79 |         .single();
80 |         if (error) throw error;
81 |         result = data as unknown as InstitutionSettings;
82 |     } else {
83 |         const { data, error } = await supabase
84 |             .from("institution_settings")
85 |             .insert(payload)
86 |             .select()
87 |             .single();
88 |         if (error) throw error;
89 |         result = data as unknown as InstitutionSettings;
90 |     }
91 |
92 |     await logAuditEvent({
93 |         action: existing ? "update" : "create",
94 |         entity_type: "institution_settings",
95 |         entity_id: result.id,
96 |         changes: payload as unknown as Record<string, unknown>,
97 |         performed_by: user?.id ?? "unknown",
98 |     });
99 |
100 |     return result;
101 | },
102 | onSuccess: () => {
103 |     qc.invalidateQueries({ queryKey: queryKeys.institutionSettings.all });
104 |     toast.success("Institution settings saved");
105 | },
106 | onError: (err) =>
107 |     toast.error(
108 |         err instanceof Error ? err.message : "Failed to save settings"
109 |     ),
110 | });
111 | };
112 |
113 | // % % % useProgramAccreditations - list accreditations for a program % % % % % % % % %
114 |
115 | export const useProgramAccreditations = (programId?: string) => {
116 |     return useQuery({
117 |         queryKey: queryKeys.programAccreditations.list({ programId }),
118 |         queryFn: async (): Promise<ProgramAccreditation[]> => {
119 |             if (!programId) return [];
120 |
121 |             const { data, error } = await supabase
122 |                 .from("program_accreditations")
123 |                 .select("*")
124 |                 .eq("program_id", programId)
125 |                 .order("created_at", { ascending: false });
126 |
127 |             if (error) throw error;
128 |             return (data ?? []) as ProgramAccreditation[];
129 |         },
130 |         enabled: !!programId,
131 |     });
132 | };
133 |
134 | // % % % useAllProgramAccreditations - list all accreditations (admin view) % % % % %
135 |
136 | export const useAllProgramAccreditations = () => {
137 |     return useQuery({
138 |         queryKey: queryKeys.programAccreditations.lists(),
139 |         queryFn: async (): Promise<
140 |             (ProgramAccreditation & {
141 |                 program_name?: string;
142 |                 program_code?: string;
143 |             })[]
144 |         > => {
145 |             const { data, error } = await supabase
146 |                 .from("program_accreditations")
147 |                 .select("*, programs(name, code)")
148 |                 .order("next_review_date", { ascending: true });
149 |
150 |             if (error) throw error;
151 |
152 |             return (data ?? []).map((row: Record<string, unknown>) => {
153 |                 const programs = row.programs as { name: string; code: string } | null;
154 |                 return {
155 |                     ...(row as unknown as ProgramAccreditation),
156 |                     program_name: programs?.name,
157 |                     program_code: programs?.code,
158 |                 };
159 |             });
160 |         },
161 |     });
162 | };
163 |
164 | // % % % useCreateProgramAccreditation % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
165 |
166 | export interface CreateProgramAccreditationInput {
167 |     program_id: string;
168 |     accreditation_body: string;
169 |     framework_version?: string;
170 |     accreditation_date?: string;
171 |     next_review_date?: string;
172 |     status: "active" | "expired" | "pending";
173 | }
174 |
175 | export const useCreateProgramAccreditation = () => {
176 |     const qc = useQueryClient();
177 |     const { user } = useAuth();
178 |

```

[illegible]

```
TYPE: .ts | LINES: 1-53 | SHA256: d5bec45ce96c95c713140b177ceb003f88d9fa909075579793b76094e42392b1
```

```
TYPE: .ts | LINES: 1-82 | SHA256: 15914d92d83211920cccf080e219ff7169552b29b3a77ef089661e4dcb1048af
```

[illegible]

```

41 |         consumed_at,
42 |         marketplace_items:item_id (
43 |             name,
44 |             description,
45 |             category,
46 |             sub_category,
47 |             icon_identifier,
48 |             metadata
49 |         )
50 |     )
51 | )
52 | .eq("student_id", studentId)
53 | .neq("status", "refunded")
54 | .order("purchased_at", { ascending: false });
55 |
56 | if (error) throw error;
57 |
58 | return (data ?? []).map((row) => {
59 |     const item = Array.isArray(row.marketplace_items)
60 |       ? row.marketplace_items[0]
61 |       : row.marketplace_items;
62 |     return {
63 |       purchase_id: row.id,
64 |       item_id: row.item_id,
65 |       item_name: item?.name ?? "Unknown Item",
66 |       item_description: item?.description ?? "",
67 |       item_category: item?.category ?? "",
68 |       item_sub_category: item?.sub_category ?? "",
69 |       icon_identifier: item?.icon_identifier ?? "package",
70 |       metadata: (item?.metadata ?? {}) as Record<string, unknown>,
71 |       xp_cost: row.xp_cost,
72 |       status: row.status,
73 |       purchased_at: row.purchased_at,
74 |       consumed_at: row.consumed_at ?? null,
75 |     };
76 |   });
77 | },
78 | enabled: !!studentId,
79 | staleTime: 30_000,
80 | });
81 | };
82 |

```

## FILE: src/hooks/useInViewport.ts

TYPE: .ts | LINES: 1-75 | SHA256: 8617dcf5d65cfa2209f7bc443c85a46e9dd1da919769dfae56fe7b74aab8882e

```

1 | import { useEffect, useRef, useState } from "react";
2 |
3 | interface UseInViewportOptions {
4 |   /**
5 |    * Margin around the root that expands the trigger area, so a section can begin
6 |    * loading just before it scrolls into view. Defaults to "200px".
7 |    */
8 |   rootMargin?: string;
9 |   /** Intersection ratio(s) at which the callback fires. Defaults to 0. */
10 |   threshold?: number | number[];
11 |   /**
12 |    * Once the element has entered the viewport, stay `true` and stop observing.
13 |    * Defaults to true – the intended use is one-shot gating of a section's data
14 |    * hooks, which should not unload when scrolled away.
15 |    */
16 |   once?: boolean;
17 | }
18 |
19 | /**
20 |  * Observe an element and report when it enters the viewport (Option J, Phase 1).
21 |  *
22 |  * Use it to defer mounting a section's data hooks until the user actually
23 |  * scrolls the section into view, replacing blind time-based deferral (a fixed
24 |  * timer fires every deferred query at once regardless of whether the section is
25 |  * even on screen; viewport-gating fires them only when needed, which shrinks the
26 |  * initial request burst on a constrained database).
27 |  *
28 |  * SSR-safe and degrades gracefully: if `IntersectionObserver` is unavailable
29 |  * (older browsers, or a jsdom without a polyfill) it starts revealed so content
30 |  * is never withheld; on the server it starts hidden and the client effect takes
31 |  * over after hydration.
32 |  */
33 | export const useInViewport = <T extends Element = HTMLDivElement>({
34 |   options: UseInViewportOptions = {}
35 | }): { ref: React.RefObject<T>; inView: boolean } => {
36 |   const { rootMargin = "200px", threshold = 0, once = true } = options;
37 |   const ref = useRef<T>(null);
38 |
39 |   // Lazy initial state (computed here, not via setState in the effect, to avoid
40 |   // a cascading-render lint violation): reveal immediately when there is no
41 |   // IntersectionObserver to observe with; stay hidden during server render.
42 |   const [inView, setInView] = useState<boolean>(() => {
43 |     if (typeof window === "undefined") return false;
44 |     return typeof IntersectionObserver === "undefined";
45 |   });
46 |
47 |   useEffect(() => {
48 |     // Already resolved and we only care once – nothing left to observe.
49 |     if (inView && once) return;
50 |
51 |     const el = ref.current;
52 |     if (!el) return;
53 |     // No observer available – the lazy initializer already revealed us.
54 |     if (typeof IntersectionObserver === "undefined") return;
55 |
56 |     const observer = new IntersectionObserver(
57 |       (entries) => {
58 |         const entry = entries[0];

```

```

59 |         if (entry?.isIntersecting) {
60 |             setInView(true);
61 |             if (once) observer.disconnect();
62 |         } else if (!once) {
63 |             setInView(false);
64 |         }
65 |     },
66 |     { rootMargin, threshold }
67 | );
68 |
69 | observer.observe(el);
70 | return () => observer.disconnect();
71 | }, [inView, once, rootMargin, threshold]);
72 |
73 | return { ref,InView };
74 | };
75 |

```

## FILE: src/hooks/useInviteUsers.ts

TYPE: .ts | LINES: 1-135 | SHA256: 845869ccc6f436527c58d4be606bf23bda86b52094246a12f71d3595ec831d91

```

1 | import { useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { useAuth } from "@hooks/useAuth";
4 | import { queryKeys } from "@lib/queryKeys";
5 | import { mapSupabaseError } from "@lib/mapSupabaseError";
6 | import { toast } from "sonner";
7 | import type { UserRole } from "@types/app";
8 |
9 | /**
10 |  * Invitation data
11 |  */
12 | export interface InvitationInput {
13 |     email: string;
14 |     role: UserRole;
15 | }
16 |
17 | /**
18 |  * Bulk invite request
19 |  */
20 | interface BulkInviteRequest {
21 |     institution_id: string;
22 |     invites: InvitationInput[];
23 | }
24 |
25 | /**
26 |  * Return type for useInviteUsers
27 |  */
28 | interface UseInviteUsersReturn {
29 |     /** Send invitations */
30 |     inviteUsers: (request: BulkInviteRequest) => Promise<void>;
31 |     /** Whether the mutation is in progress */
32 |     isPending: boolean;
33 |     /** Error if the mutation failed */
34 |     error: Error | null;
35 | }
36 |
37 | /**
38 |  * Hook for inviting users to an institution
39 |  *
40 |  * Features:
41 |  * - Calls the `send-invitation-email` Edge Function
42 |  * - Logs to audit_logs on success
43 |  * - Uses mapSupabaseError for error mapping
44 |  * - Emits Sonner toasts on success/failure
45 |  * - Invalidates invitation queries after success
46 |  *
47 |  * Usage:
48 |  * ```tsx
49 |  * const { inviteUsers, isPending, error } = useInviteUsers();
50 |  *
51 |  * const handleInvite = async () => {
52 |  *     try {
53 |  *         await inviteUsers({
54 |  *             institution_id: 'inst-123',
55 |  *             invites: [
56 |  *                 { email: 'user1@example.com', role: 'teacher' },
57 |  *                 { email: 'user2@example.com', role: 'student' },
58 |  *             ],
59 |  *         });
60 |  *     } catch (err) {
61 |  *         console.error('Invite failed:', err);
62 |  *     }
63 |  * };
64 |  *
65 |  * return (
66 |  *     <div>
67 |  *         <button onClick={handleInvite} disabled={isPending}>
68 |  *             {isPending ? 'Sending...' : 'Send Invitations'}
69 |  *         </button>
70 |  *         {error && <p className="text-red-600">{error.message}</p>}
71 |  *     </div>
72 |  * );
73 |  *
74 |  */
75 | export const useInviteUsers = (): UseInviteUsersReturn => {
76 |     const { user } = useAuth();
77 |     const queryClient = useQueryClient();
78 |
79 |     const mutation = useMutation({
80 |         mutationFn: async (request: BulkInviteRequest) => {
81 |             if (!user) {
82 |                 throw new Error("User not authenticated");
83 |             }

```

```

84 // Call the send-invitation-email Edge Function
85 const { error } = await supabase.functions.invoke(
86   "send-invitation-email",
87   {
88     body: {
89       institution_id: request.institution_id,
90       invites: request.invites,
91     },
92   },
93 );
94
95 if (error) {
96   throw error;
97 }
98
99 },
100 onSuccess: (_data, request) => {
101   // Invalidate invitation queries
102   queryClient.invalidateQueries({
103     queryKey: queryKeys.institutions.lists(),
104   });
105
106   const count = request.invites.length;
107   toast.success(
108     `${count} invitation${count !== 1 ? "s" : ""} sent successfully`
109   );
110 },
111 onError: (error: Error) => {
112   console.error("Invite error:", error);
113
114   // Try to map the error for a user-friendly message
115   let userMessage = error.message;
116   if (error instanceof Error && "code" in error) {
117     const mapped = mapSupabaseError(error);
118     userMessage = mapped.userMessage;
119   }
120
121   toast.error(userMessage || "Failed to send invitations");
122 },
123 });
124
125 const inviteUsers = async (request: BulkInviteRequest) => {
126   return mutation.mutateAsync(request);
127 };
128
129 return {
130   inviteUsers,
131   isPending: mutation.isPending,
132   error: mutation.error,
133 };
134 };
135

```

FILE: src/hooks/useJournal.ts

```
TYPE: .ts | LINES: 1-283 | SHA256: 535679c7d8b8beab3a32e79819590ac6333df59680af22378a9da86afec519d1
```

[illegible]



[illegible]

```

153 |     } catch {
154 |       console.error("[useCreateJournalEntry] journal habit write failed");
155 |     }
156 |
157 |     // Journal XP: a substantive entry ("e50 words, computed client-side) grants
158 |     // the advertised +20 XP. The server enforces the canonical 20 amount and a
159 |     // daily idempotent reference, so a student cannot farm XP by re-saving.
160 |     // Fire-and-forget: an XP/badge failure must never break the entry save.
161 |     const wordCount = input.content
162 |       .trim()
163 |       .split(/\s+/)
164 |       .filter((word) => word.length > 0).length;
165 |
166 |     if (wordCount >= 50) {
167 |       try {
168 |         await supabase.functions.invoke("award-xp", {
169 |           body: {
170 |             student_id: user.id,
171 |             source: "journal",
172 |             xp_amount: 20,
173 |           },
174 |         });
175 |         await supabase.functions.invoke("check-badges", {
176 |           body: {
177 |             student_id: user.id,
178 |             trigger: "journal",
179 |           },
180 |         });
181 |       } catch {
182 |         console.error("[useCreateJournalEntry] journal XP award failed");
183 |       }
184 |     }
185 |
186 |     return entry;
187 |   },
188 |   onSuccess: () => {
189 |     queryClient.invalidateQueries({ queryKey: queryKeys.journal.lists() });
190 |   },
191 | });
192 |
193 |
194 | // % % % useUpdateJournalEntry - update an existing journal entry % % % % % % % % % % % % % % % %
195 |
196 | export const useUpdateJournalEntry = () => {
197 |   const queryClient = useQueryClient();
198 |   const { user } = useAuth();
199 |
200 |   return useMutation({
201 |     mutationFn: async (
202 |       input: UpdateJournalEntryInput
203 |     ): Promise<JournalEntry> => {
204 |       if (!user) throw new Error("Not authenticated");
205 |
206 |       const updates: Database["public"]["Tables"]["journal_entries"]["Update"] =
207 |         {};
208 |
209 |       if (input.content !== undefined) {
210 |         updates.content = input.content;
211 |       }
212 |
213 |       if (input.is_shared !== undefined) {
214 |         updates.is_shared = input.is_shared;
215 |       }
216 |
217 |       if (input.clo_id !== undefined) {
218 |         updates.clo_id = input.clo_id;
219 |       }
220 |
221 |       const { data, error } = await supabase
222 |         .from("journal_entries")
223 |         .update(updates)
224 |         .eq("id", input.id)
225 |         .eq("student_id", user.id)
226 |         .select()
227 |         .single();
228 |
229 |       if (error) throw error;
230 |
231 |       const entry = data as unknown as JournalEntry;
232 |
233 |       await logAuditEvent({
234 |         action: "update",
235 |         entity_type: "journal_entry",
236 |         entity_id: entry.id,
237 |         changes: updates,
238 |         performed_by: user.id,
239 |       });
240 |
241 |       return entry;
242 |     },
243 |     onSuccess: (_data, variables) => {
244 |       queryClient.invalidateQueries({ queryKey: queryKeys.journal.lists() });
245 |       queryClient.invalidateQueries({
246 |         queryKey: queryKeys.journal.detail(variables.id),
247 |       });
248 |     },
249 |   });
250 | };
251 |
252 | // % % % useDeleteJournalEntry - delete a journal entry % % % % % % % % % % % % % % % %
253 |
254 | export const useDeleteJournalEntry = () => {
255 |   const queryClient = useQueryClient();
256 |   const { user } = useAuth();

```



```

10 | import type { CreateKnowledgeQuestInput } from "@lib/marketplaceSchemas";
11 |
12 | export const useAdminKnowledgeQuests = () => {
13 |   return useQuery({
14 |     queryKey: queryKeys.quests.lists(),
15 |     queryFn: async () => {
16 |       const { data, error } = await supabase
17 |         .from("knowledge_quests")
18 |         .select("*")
19 |         .order("created_at", { ascending: false });
20 |       if (error) throw error;
21 |       return data ?? [];
22 |     },
23 |   });
24 | };
25 |
26 | export const useCreateKnowledgeQuest = () => {
27 |   const qc = useQueryClient();
28 |   return useMutation({
29 |     mutationFn: async (input: CreateKnowledgeQuestInput) => {
30 |       const {
31 |         data: { user },
32 |       } = await supabase.auth.getUser();
33 |       if (!user) throw new Error("Not authenticated");
34 |
35 |       const { data: profile } = await supabase
36 |         .from("profiles")
37 |         .select("institution_id")
38 |         .eq("id", user.id)
39 |         .single();
40 |
41 |       const { data, error } = await supabase
42 |         .from("knowledge_quests")
43 |         .insert({
44 |           ...input,
45 |           institution_id: profile?.institution_id,
46 |           created_by: user.id,
47 |         } as never)
48 |         .select()
49 |         .single();
50 |       if (error) throw error;
51 |       return data;
52 |     },
53 |     onSuccess: () => {
54 |       qc.invalidateQueries({ queryKey: queryKeys.quests.all });
55 |       toast.success("Knowledge quest created");
56 |     },
57 |     onError: (err) => toast.error((err as Error).message),
58 |   });
59 | };
60 |
61 | export const useUpdateKnowledgeQuest = () => {
62 |   const qc = useQueryClient();
63 |   return useMutation({
64 |     mutationFn: async ({
65 |       id,
66 |       ...input
67 |     }: Partial<CreateKnowledgeQuestInput> & { id: string }) => {
68 |       const { data, error } = await supabase
69 |         .from("knowledge_quests")
70 |         .update(input)
71 |         .eq("id", id)
72 |         .select()
73 |         .single();
74 |       if (error) throw error;
75 |       return data;
76 |     },
77 |     onSuccess: () => {
78 |       qc.invalidateQueries({ queryKey: queryKeys.quests.all });
79 |       toast.success("Knowledge quest updated");
80 |     },
81 |     onError: (err) => toast.error((err as Error).message),
82 |   });
83 | };
84 |

```

## FILE: src/hooks/useKnowledgeQuests.ts

TYPE: .ts | LINES: 1-132 | SHA256: 221f6939dd68271d1a007f3bf5f68be81216dd25dc37e0a28c7f599b4ac1d3ca

```

1 | // =====
2 | // useKnowledgeQuests - Quest browsing, progress tracking, start/complete
3 | // Task 20.2
4 | // =====
5 |
6 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
7 | import { supabase } from "@lib/supabase";
8 | import { queryKeys } from "@lib/queryKeys";
9 | import { toast } from "sonner";
10 |
11 | export interface KnowledgeQuest {
12 |   id: string;
13 |   title: string;
14 |   description: string;
15 |   quest_type: "quiz_challenge" | "content_creation" | "peer_review";
16 |   start_date: string;
17 |   end_date: string;
18 |   reward_type: "item" | "xp";
19 |   reward_item_id: string | null;
20 |   reward_xp_amount: number | null;
21 |   created_at: string;
22 | }
23 |
24 | export interface QuestProgress {
25 |   id: string;

```

```

26 | student_id: string;
27 | quest_id: string;
28 | status: "in_progress" | "completed";
29 | started_at: string;
30 | completed_at: string | null;
31 | }
32 |
33 | export const useKnowledgeQuests = (status?: string) => {
34 |   return useQuery({
35 |     queryKey: queryKeys.quests.list({ status }),
36 |     queryFn: async (): Promise<KnowledgeQuest[]> => {
37 |       let query = supabase
38 |         .from("knowledge_quests")
39 |         .select("*")
40 |         .order("start_date", { ascending: false });
41 |
42 |       if (status === "active") {
43 |         const now = new Date().toISOString();
44 |         query = query.lte("start_date", now).gte("end_date", now);
45 |       }
46 |
47 |       const { data, error } = await query;
48 |       if (error) throw error;
49 |       return (data ?? []) as KnowledgeQuest[];
50 |     },
51 |   });
52 | };
53 |
54 | export const useQuestProgress = (questId: string, studentId?: string) => {
55 |   return useQuery({
56 |     queryKey: queryKeys.quests.detail(questId),
57 |     queryFn: async (): Promise<QuestProgress | null> => {
58 |       const { data, error } = await supabase
59 |         .from("student_quest_progress")
60 |         .select("*")
61 |         .eq("quest_id", questId)
62 |         .eq("student_id", studentId!)
63 |         .maybeSingle();
64 |       if (error) throw error;
65 |       return data as QuestProgress | null;
66 |     },
67 |     enabled: !!questId && !!studentId,
68 |   });
69 | };
70 |
71 | export const useStartQuest = () => {
72 |   const qc = useQueryClient();
73 |   return useMutation({
74 |     mutationFn: async ({
75 |       questId,
76 |       studentId,
77 |     }: {
78 |       questId: string;
79 |       studentId: string;
80 |     }) => {
81 |       const { data, error } = await supabase
82 |         .from("student_quest_progress")
83 |         .insert({
84 |           student_id: studentId,
85 |           quest_id: questId,
86 |           status: "in_progress",
87 |           started_at: new Date().toISOString(),
88 |         })
89 |         .select()
90 |         .single();
91 |       if (error) throw error;
92 |       return data;
93 |     },
94 |     onSuccess: () => {
95 |       qc.invalidateQueries({ queryKey: queryKeys.quests.all });
96 |       toast.success("Quest started!");
97 |     },
98 |     onError: (err) => toast.error((err as Error).message),
99 |   });
100 | };
101 |
102 | export const useCompleteQuest = () => {
103 |   const qc = useQueryClient();
104 |   return useMutation({
105 |     mutationFn: async ({
106 |       questId,
107 |       studentId,
108 |     }: {
109 |       questId: string;
110 |       studentId: string;
111 |     }) => {
112 |       const { data, error } = await supabase
113 |         .from("student_quest_progress")
114 |         .update({
115 |           status: "completed",
116 |           completed_at: new Date().toISOString(),
117 |         })
118 |         .eq("quest_id", questId)
119 |         .eq("student_id", studentId)
120 |         .select()
121 |         .single();
122 |       if (error) throw error;
123 |       return data;
124 |     },
125 |     onSuccess: () => {
126 |       qc.invalidateQueries({ queryKey: queryKeys.quests.all });
127 |       toast.success("Quest completed! Reward earned!");
128 |     },
129 |     onError: (err) => toast.error((err as Error).message),

```

```
TYPE: .ts | LINES: 1-30 | SHA256: c1c657d67fe0b0319fcd4aae6fa43f859b8623e28906664276f31dce2110e814
```

```

1 import { useMutation, useQueryClient } from "@tanstack/react-query";
2 import { supabase } from "@lib/supabase";
3 import { queryKeys } from "@lib/queryKeys";
4 import { useAuth } from "@/hooks/useAuth";
5
6 export const useUpdateLanguagePreference = () => {
7   const queryClient = useQueryClient();
8   const { user } = useAuth();
9
10  return useMutation({
11    mutationFn: async (language: string) => {
12      if (!user?.id) return;
13
14      const { error } = await supabase
15        .from("profiles")
16        .update({ preferred_language: language })
17        .eq("id", user.id);
18      if (error) throw error;
19    },
20    onSuccess: () => {
21      if (user?.id) {
22        queryClient.invalidateQueries({
23          queryKey: queryKeys.i18n.languagePreference(user.id),
24        });
25        queryClient.invalidateQueries({ queryKey: queryKeys.profiles.all });
26      }
27    },
28  });
29 };
30

```

FILE: src/hooks/useLeaderboard.ts

```
TYPE: .ts | LINES: 1-422 | SHA256: 3a14526104397c785c4206829a2af65a6292bd46e8f911e943953217415501e7
```

[illegible]

[illegible]

[illegible]



[illegible]

[illegible]

FILE: src/hooks/useLeaderboardCosmetics.ts

```
TYPE: .ts | LINES: 1-125 | SHA256: 6b472fa06aebab896ad32b224198c7dead19ab52d92a616f62c6ba98662ac221
```

[illegible]

```

54     .from("student_equipped_items")
55     .select(
56
57         student_id,
58         slot,
59         xp_purchases:purchase_id {
60             marketplace_items:item_id {
61                 metadata,
62                 name
63             }
64         }
65     )
66     .in("student_id", studentIds)
67     .in("slot", ["avatar_frame", "display_title"]);
68
69 if (error) throw error;
70
71 const cosmeticMap = new Map<string, LeaderboardCosmeticData>();
72
73 // Initialize all students with defaults
74 for (const id of studentIds) {
75     cosmeticMap.set(id, {
76         studentId: id,
77         avatarFrame: null,
78         displayTitle: null,
79     });
80 }
81
82 // Populate from query results. Embedded relations are to-one objects (or
83 // null); a missing join collapses to empty metadata via `asRecord`.
84 for (const row of data ?? []) {
85     const studentId = row.student_id;
86     const slot = row.slot;
87     const purchase = Array.isArray(row.xp_purchases)
88         ? row.xp_purchases[0]
89         : row.xp_purchases;
90     const item = Array.isArray(purchase?.marketplace_items)
91         ? purchase?.marketplace_items[0]
92         : purchase?.marketplace_items;
93     const metadata = asRecord(item?.metadata);
94
95     const existing = cosmeticMap.get(studentId) ?? {
96         studentId,
97         avatarFrame: null,
98         displayTitle: null,
99     };
100
101     if (slot === "avatar_frame") {
102         existing.avatarFrame = {
103             border_color: readString(metadata, "border_color") ?? "#3b82f6",
104             border_width: readString(metadata, "border_width") ?? "3px",
105             border_style: readString(metadata, "border_style") ?? "solid",
106             box_shadow: readString(metadata, "box_shadow"),
107         };
108     } else if (slot === "display_title") {
109         existing.displayTitle = {
110             title_text: readString(metadata, "title_text") ?? item?.name ?? "",
111             title_color: readString(metadata, "title_color") ?? "#6366f1",
112         };
113     }
114
115     cosmeticMap.set(studentId, existing);
116 }
117
118 return cosmeticMap;
119 },
120 enabled: studentIds.length > 0,
121 staleTime: 60_000,
122 });
123 };
124 };
125

```

FILE: src/hooks/useLeagueLeaderboard.ts

TYPE: .ts | LINES: 1-272 | SHA256: e8d070a83647d002b0e078e1e852f2b7e8ea831e3f9428d84a6aa3cc1b697cd6

[illegible]

[illegible]

[illegible]

[illegible]

FILE: `src/hooks/useLeagueTiers.ts`

```
TYPE: .ts | LINES: 1-13 | SHA256: 4a1a1d7819f741a9ba7b607eb953b652c2dd215849ba8bcd88d99bea169f5280
```

```

1 // =====
2 // useLeagueTiers – League tier query, league-scoped leaderboard
3 // Task 20.10 – Re-exports from useLeagueLeaderboard
4 // =====
5
6 export {
7   useLeagueLeaderboard,
8   useStudentLeagueTier,
9   useStudentPercentileBand,
10 } from "@/hooks/useLeagueLeaderboard";
11
12 export type { LeagueLeaderboardEntry } from "@/hooks/useLeagueLeaderboard";
13

```

FILE: `src/hooks/useLearningPath.ts`

```
TYPE: .ts | LINES: 1-238 | SHA256: 694e618b5d3a912ce51b16928cdb0725d8db8fda5b7e6db86f43946dd28f1413
```

[illegible]

```

48 | export interface RawAttainment {
49 |   outcome_id: string;
50 |   attainment_percent: number;
51 | }
52 |
53 | export interface RawSubmission {
54 |   assignment_id: string;
55 |   status: string;
56 | }
57 |
58 | export interface RawGrade {
59 |   submission_id: string;
60 |   assignment_id: string;
61 | }
62 |
63 | export function buildLearningPathNodes(
64 |   assignments: RawAssignment[],
65 |   clos: RawCLO[],
66 |   attainments: RawAttainment[],
67 |   submissions: RawSubmission[],
68 |   grades: RawGrade[]
69 | ): LearningPathNode[] {
70 |   const cloMap = new Map(clos.map((c) => [c.id, c]));
71 |   const attainmentMap = new Map(
72 |     attainments.map((a) => [a.outcome_id, a.attainment_percent])
73 |   );
74 |   const submittedAssignments = new Set(submissions.map((s) => s.assignment_id));
75 |   const gradedAssignments = new Set(grades.map((g) => g.assignment_id));
76 |
77 |   const nodes: LearningPathNode[] = assignments.map((assignment) => {
78 |     // Determine Bloom's level from ALL linked CLOs, not just the first.
79 |     // The learning path orders Remembering !' Creating, so the representative
80 |     // level for an assignment is the lowest (min) Bloom level across its CLOs:
81 |     // the assignment becomes available as soon as the easiest cognitive demand
82 |     // is reachable. Falls back to "remembering" when no CLO carries a level.
83 |     const cloLevels = assignment.clo_weights
84 |       .map(({ clo_id }) => cloMap.get(clo_id)?.blooms_level)
85 |       .filter((level): level is BloomsLevel => !!level);
86 |     const bloomsLevel: BloomsLevel =
87 |       cloLevels.length > 0
88 |       ? cloLevels.reduce((lowest, level) =>
89 |         BLOOMS_ORDER[level] < BLOOMS_ORDER[lowest] ? level : lowest
90 |       )
91 |       : "remembering";
92 |
93 |     // Retain the first linked CLO for attainment display (behavior preserved).
94 |     const primaryCLOid = assignment.clo_weights[0]?.clo_id;
95 |
96 |     // Check prerequisite status
97 |     let isLocked = false;
98 |     let prerequisite: LearningPathNode["prerequisite"] = undefined;
99 |
100 |     if (assignment.prerequisites && assignment.prerequisites.length > 0) {
101 |       const prereq = assignment.prerequisites[0];
102 |       if (prereq) {
103 |         const prereqCLO = cloMap.get(prereq.clo_id);
104 |         const currentAttainment = attainmentMap.get(prereq.clo_id) ?? 0;
105 |
106 |         if (currentAttainment < prereq.required_attainment) {
107 |           isLocked = true;
108 |           prerequisite = {
109 |             clo_id: prereq.clo_id,
110 |             clo_title: prereqCLO?.title ?? "Unknown CLO",
111 |             required_attainment: prereq.required_attainment,
112 |             current_attainment: currentAttainment,
113 |           };
114 |         }
115 |       }
116 |     }
117 |
118 |     // Determine status
119 |     let status: LearningPathNode["status"];
120 |     if (isLocked) {
121 |       status = "locked";
122 |     } else if (gradedAssignments.has(assignment.id)) {
123 |       status = "graded";
124 |     } else if (submittedAssignments.has(assignment.id)) {
125 |       status = "submitted";
126 |     } else {
127 |       status = "available";
128 |     }
129 |
130 |     // Get attainment for the primary CLO
131 |     const attainmentPercent = primaryCLOid
132 |       ? attainmentMap.get(primaryCLOid) ?? null
133 |       : null;
134 |
135 |     return {
136 |       assignment_id: assignment.id,
137 |       title: assignment.title,
138 |       blooms_level: bloomsLevel,
139 |       status,
140 |       attainment_percent: attainmentPercent,
141 |       prerequisite,
142 |     };
143 |   });
144 |
145 |   // Sort by Bloom's level order, then by title for stability
146 |   nodes.sort((a, b) => {
147 |     const levelDiff =
148 |       BLOOMS_ORDER[a.blooms_level] - BLOOMS_ORDER[b.blooms_level];
149 |     if (levelDiff !== 0) return levelDiff;
150 |     return a.title.localeCompare(b.title);
151 |   });

```

```

152 |
153 |     return nodes;
154 | }
155 |
156 | // % % % useLearningPath – fetch and build learning path for a course % % % % % % % %
157 |
158 | export const useLearningPath = (
159 |   courseId: string | undefined,
160 |   studentId: string | undefined
161 | ) => {
162 |   return useQuery({
163 |     queryKey: queryKeys.assignments.list({
164 |       courseId,
165 |       studentId,
166 |       view: "learningPath",
167 |     }),
168 |     queryFn: async (): Promise<LearningPathNode[]> => {
169 |       if (!courseId || !studentId) return [];
170 |
171 |       // Fetch assignments for the course
172 |       const { data: assignments, error: assignErr } = await supabase
173 |         .from("assignments")
174 |         .select("id, title, prerequisites, clo_weights")
175 |         .eq("course_id", courseId);
176 |
177 |       if (assignErr) throw assignErr;
178 |
179 |       // Fetch CLOs for the course
180 |       const { data: clos, error: cloErr } = await supabase
181 |         .from("learning_outcomes")
182 |         .select("id, title, blooms_level")
183 |         .eq("type", "CLO")
184 |         .eq("course_id", courseId);
185 |
186 |       if (cloErr) throw cloErr;
187 |
188 |       // Fetch student's outcome attainment for prerequisite checking
189 |       const { data: attainments, error: attErr } = await supabase
190 |         .from("outcome_attainment")
191 |         .select("outcome_id, attainment_percent")
192 |         .eq("student_id", studentId)
193 |         .eq("scope", "student_course");
194 |
195 |       if (attErr) throw attErr;
196 |
197 |       // Fetch student's submissions for this course's assignments
198 |       const assignmentIds = (assignments ?? []).map((a) => a.id);
199 |       let submissions: RawSubmission[] = [];
200 |       let grades: RawGrade[] = [];
201 |
202 |       if (assignmentIds.length > 0) {
203 |         const { data: subs, error: subErr } = await supabase
204 |           .from("submissions")
205 |           .select("assignment_id, status")
206 |           .eq("student_id", studentId)
207 |           .in("assignment_id", assignmentIds);
208 |
209 |         if (subErr) throw subErr;
210 |         submissions = subs ?? [];
211 |
212 |         const { data: gradeData, error: gradeErr } = await supabase
213 |           .from("grades")
214 |           .select("submission_id, submissions!inner(assignment_id)")
215 |           .in("submissions.assignment_id", assignmentIds);
216 |
217 |         if (gradeErr) throw gradeErr;
218 |
219 |         grades = (gradeData ?? []).map((g) => ({
220 |           submission_id: g.submission_id,
221 |           assignment_id: (g.submissions as unknown as { assignment_id: string })
222 |             .assignment_id,
223 |         }));
224 |       }
225 |
226 |       return buildLearningPathNodes(
227 |         (assignments ?? []) as RawAssignment[],
228 |         (clos ?? []) as RawCLO[],
229 |         (attainments ?? []) as RawAttainment[],
230 |         submissions,
231 |         grades
232 |       );
233 |     },
234 |     enabled: !!courseId && !!studentId,
235 |     staleTime: 30_000,
236 |   });
237 | };
238 |

```

## FILE: src/hooks/useLevel.ts

TYPE: .ts | LINES: 1-68 | SHA256: f604d1282f5f5066f1c57ea7d05894f843efe4c61996f5db779fbc8dc15628ba

```

1 | // =====
2 | // useLevel – TanStack Query hook for student level data
3 | // =====
4 |
5 | import { useQuery } from "@tanstack/react-query";
6 | import { supabase } from "@lib/supabase";
7 | import { queryKeys } from "@lib/queryKeys";
8 | import { calculateLevel, LEVEL_THRESHOLDS } from "@lib/xpLevelCalculator";
9 |
10 | export interface LevelData {
11 |   level: number;
12 |   title: string;
13 |   xpTotal: number;

```



```

14 |   xpForCurrentLevel: number;
15 |   xpForNextLevel: number;
16 |   progressPercent: number;
17 | }
18 |
19 | /**
20 |  * Compute level progress data from raw XP total.
21 |  */
22 | export function computeLevelData(xpTotal: number): LevelData {
23 |   const level = calculateLevel(xpTotal);
24 |   const currentThreshold = LEVEL_THRESHOLDS.find((t) => t.level === level);
25 |   const nextThreshold = LEVEL_THRESHOLDS.find((t) => t.level === level + 1);
26 |
27 |   const xpForCurrentLevel = currentThreshold?.xpRequired ?? 0;
28 |   const xpForNextLevel = nextThreshold?.xpRequired ?? xpForCurrentLevel;
29 |   const range = xpForNextLevel - xpForCurrentLevel;
30 |   const progressPercent =
31 |     range > 0
32 |       ? Math.min(Math.round(((xpTotal - xpForCurrentLevel) / range) * 100), 100)
33 |       : 100;
34 |
35 |   return {
36 |     level,
37 |     title: currentThreshold?.title ?? "Newcomer",
38 |     xpTotal,
39 |     xpForCurrentLevel,
40 |     xpForNextLevel,
41 |     progressPercent,
42 |   };
43 | }
44 |
45 | export const useLevel = (studentId: string | undefined) => {
46 |   return useQuery({
47 |     queryKey: queryKeys.studentGamification.list({ scope: "level", studentId }),
48 |     queryFn: async (): Promise<LevelData> => {
49 |       if (!studentId) {
50 |         return computeLevelData(0);
51 |       }
52 |
53 |       const { data, error } = await supabase
54 |         .from("student_gamification")
55 |         .select("xp_total")
56 |         .eq("student_id", studentId)
57 |         .maybeSingle();
58 |
59 |       if (error) throw error;
60 |
61 |       const xpTotal = data?.xp_total ?? 0;
62 |       return computeLevelData(xpTotal);
63 |     },
64 |     enabled: !!studentId,
65 |     staleTime: 30_000,
66 |   });
67 | };
68 |

```

FILE: src/hooks/useMarketplace.ts

TYPE: .ts | LINES: 1-176 | SHA256: 22d1427557247fc4f669ef43ae7c67157a836cf2f1a73faadda07d6daeb89e26

[illegible]

```

46 | /** Exact total count of matching items across all pages. */
47 | total: number;
48 | /** Whether at least one more item exists beyond this page. */
49 | hasMore: boolean;
50 | }
51 |
52 | /** Default items per page; bounds the marketplace fetch (R34.1). */
53 | export const MARKETPLACE_PAGE_SIZE = 24;
54 |
55 | // % % % useMarketplaceItems - browse items with optional category filter % % % % % %
56 |
57 | export const useMarketplaceItems = (
58 |   category?: string,
59 |   pageSize: number = MARKETPLACE_PAGE_SIZE
60 | ) => {
61 |   return useInfiniteQuery({
62 |     queryKey: queryKeys.marketplace.items(category),
63 |     initialPageParam: 1,
64 |     queryFn: async ({ pageParam }) : Promise<MarketplaceItemsPage> => {
65 |       const page = pageParam;
66 |       const { from, to } = toRange(page, pageSize);
67 |
68 |       // Validate the optional category filter against the generated enum so the
69 |       // typed query builder accepts it; an unrecognized value is ignored.
70 |       const categoryFilter = marketplaceItemCategorySchema.safeParse(category);
71 |
72 |       let query = supabase
73 |         .from("marketplace_items")
74 |         .select(
75 |           "id, institution_id, name, description, category, sub_category, xp_price, level_requirement, stock_t
76 |           { count: "exact" }
77 |         )
78 |         .eq("is_active", true);
79 |
80 |       if (categoryFilter.success) {
81 |         query = query.eq("category", categoryFilter.data);
82 |       }
83 |
84 |       const {
85 |         data: items,
86 |         error,
87 |         count,
88 |       } = await query
89 |         .order("category", { ascending: true })
90 |         .order("xp_price", { ascending: true })
91 |         .order("id", { ascending: true })
92 |         .range(from, to);
93 |       if (error) throw error;
94 |
95 |       const total = count ?? 0;
96 |
97 |       if (!items || items.length === 0) {
98 |         return {
99 |           items: [],
100 |           page,
101 |           total,
102 |           hasMore: hasMore(page, pageSize, total),
103 |         };
104 |       }
105 |
106 |       // Fetch active sale events for the items on this page to compute discounts.
107 |       const pageItemIds = items.map((item) => item.id);
108 |       const now = new Date().toISOString();
109 |       const { data: saleData, error: saleError } = await supabase
110 |         .from("sale_event_items")
111 |         .select(
112 |           "item_id, sale_event_id, sale_events:sale_event_id(discount_percentage, start_date, end_date)"
113 |         )
114 |         .in("item_id", pageItemIds)
115 |         .filter("sale_events.start_date", "lte", now)
116 |         .filter("sale_events.end_date", "gt", now);
117 |
118 |       // Build a map of item_id !' highest discount
119 |       const discountMap = new Map<string, number>();
120 |       if (!saleError && saleData) {
121 |         for (const row of saleData) {
122 |           // The embedded `sale_events` relation is an object (to-one) or null.
123 |           const saleEvent = Array.isArray(row.sale_events)
124 |             ? row.sale_events[0]
125 |             : row.sale_events;
126 |           if (!saleEvent) continue;
127 |           const discount = saleEvent.discount_percentage ?? 0;
128 |           const current = discountMap.get(row.item_id) ?? 0;
129 |           if (discount > current) {
130 |             discountMap.set(row.item_id, discount);
131 |           }
132 |         }
133 |       }
134 |
135 |       const mapped: MarketplaceItem[] = items.map((item) => {
136 |         const discount = discountMap.get(item.id) ?? 0;
137 |         const basePrice = item.xp_price;
138 |         const effectivePrice =
139 |           discount > 0
140 |             ? Math.max(1, basePrice - Math.floor((basePrice * discount) / 100))
141 |             : basePrice;
142 |
143 |         return {
144 |           id: item.id,
145 |           institution_id: item.institution_id,
146 |           name: item.name,
147 |           description: item.description,
148 |           category: item.category,
149 |           sub_category: item.sub_category,

```



[illegible]

```

178         is_active: variables.isActive,
179         updated_at: new Date().toISOString(),
180     })
181     .eq("id", variables.itemId);
182
183     if (error) throw error;
184 },
185 onSuccess: (_data, variables) => {
186     queryClient.invalidateQueries({ queryKey: queryKeys.marketplace.all });
187     toast.success(variables.isActive ? "Item activated" : "Item deactivated");
188 },
189 onError: (error: Error) => {
190     toast.error(error.message || "Failed to toggle item status");
191 },
192 });
193 };
194

```

FILE: src/hooks/useMarketplaceAnalytics.ts

```
TYPE: .ts | LINES: 1-123 | SHA256: ab7523da570e55aeca1d3609aac80dbc797791b8eb50856212bc3f75d9b20fce
```

[illegible]

```

84 |     categoryMap.set(category, catEntry);
85 | }
86 |
87 | const totalPurchases = rows.length;
88 | const uniqueBuyers = buyerSet.size;
89 | const avgXPPerStudent =
90 |   uniqueBuyers > 0 ? Math.round(totalXPSpent / uniqueBuyers) : 0;
91 |
92 | // Top 10 popular items
93 | const popularItems = Array.from(itemCountMap.entries())
94 |   .map(([item_id, { name, count }]) => ({
95 |     item_id,
96 |     item_name: name,
97 |     purchase_count: count,
98 |   }))
99 |   .sort((a, b) => b.purchase_count - a.purchase_count)
100 |   .slice(0, 10);
101 |
102 | // Category breakdown
103 | const categoryBreakdown = Array.from(categoryMap.entries()).map(
104 |   ([category, { purchases, xp }]) => ({
105 |     category,
106 |     total_purchases: purchases,
107 |     total_xp_spent: xp,
108 |   })
109 | );
110 |
111 | return {
112 |   totalXPSpent,
113 |   totalPurchases,
114 |   uniqueBuyers,
115 |   avgXPPerStudent,
116 |   popularItems,
117 |   categoryBreakdown,
118 | };
119 | },
120 | staleTime: 60_000,
121 | });
122 | };
123 |

```

## FILE: src/hooks/useMascotGuidance.ts

TYPE: .ts | LINES: 1-61 | SHA256: 697cfe2d4a496eed7550cf0a70e9da9c62fb1de1d80f34732c6683c1428d308

```

1 | // =====
2 | // useMascotGuidance - resolves whether the mascot should coach at a moment (R35)
3 | // =====
4 | //
5 | // Thin React binding over the pure `resolveMascotGuidance` logic. It reads the
6 | // student's mascot-enabled preference (defaulting to enabled) and returns the
7 | // resolved guidance descriptor for the requested moment, or `null` when the
8 | // mascot is disabled or no moment is active - so consuming surfaces remain
9 | // fully functional regardless of the guidance state (R35.4, R35.5).
10 | //
11 | // The preference is stored in localStorage (additive, no schema change): this
12 | // keeps the optional enhancement self-contained and means the surrounding
13 | // surfaces never depend on a network round-trip to render.
14 |
15 | import { useMemo } from "react";
16 | import {
17 |   parseMascotEnabled,
18 |   resolveMascotGuidance,
19 |   type MascotGuidance,
20 |   type MascotMomentId,
21 | } from "@lib/mascotGuidance";
22 |
23 | const STORAGE_KEY = "edeviser-mascot-enabled";
24 |
25 | /** Read the mascot-enabled preference, falling back to the default safely. */
26 | export const isMascotGuidanceEnabled = (): boolean => {
27 |   if (typeof window === "undefined") return true;
28 |   try {
29 |     return parseMascotEnabled(window.localStorage.getItem(STORAGE_KEY));
30 |   } catch {
31 |     // localStorage can throw (privacy mode / quota). Degrade to the default.
32 |     return true;
33 |   }
34 | };
35 |
36 | /** Persist the mascot-enabled preference. */
37 | export const setMascotGuidanceEnabled = (enabled: boolean): void => {
38 |   if (typeof window === "undefined") return;
39 |   try {
40 |     window.localStorage.setItem(STORAGE_KEY, enabled ? "true" : "false");
41 |   } catch {
42 |     // No-op: an unwritable store simply keeps the in-memory default.
43 |   }
44 | };
45 |
46 | /**
47 |  * Resolve mascot guidance for a given key moment.
48 |  *
49 |  * @param moment - The active key moment, or `null` when no moment is active.
50 |  * @returns The guidance descriptor to render, or `null` to render nothing.
51 |  */
52 | export const useMascotGuidance = (
53 |   moment: MascotMomentId | null
54 | ): MascotGuidance | null => {
55 |   const enabled = isMascotGuidanceEnabled();
56 |   return useMemo(
57 |     () => resolveMascotGuidance({ enabled, moment }),
58 |     [enabled, moment]
59 |   );
60 | };

```



```

100 | },
101 | onSuccess: (_data, variables) => {
102 |   queryClient.invalidateQueries({
103 |     queryKey: queryKeys.masteryRecovery.status(
104 |       variables.student_id,
105 |       variables.clo_id
106 |     ),
107 |   });
108 |   queryClient.invalidateQueries({
109 |     queryKey: queryKeys.masteryRecovery.metrics(variables.institution_id),
110 |   });
111 | },
112 | });
113 | };
114 |
115 | // % % % useCompleteRecoveryStep - mark ai_tutor or practice as completed % % % % %
116 |
117 | export const useCompleteRecoveryStep = () => {
118 |   const queryClient = useQueryClient();
119 |
120 |   return useMutation({
121 |     mutationFn: async (
122 |       input: CompleteRecoveryStepInput
123 |     ): Promise<MasteryRecoveryRow> => {
124 |       const now = new Date().toISOString();
125 |
126 |       const updateFields =
127 |         input.step === "ai_tutor"
128 |         ? { ai_tutor_completed: true, ai_tutor_completed_at: now }
129 |         : { practice_completed: true, practice_completed_at: now };
130 |
131 |       const { data, error } = await supabase
132 |         .from("mastery_recovery_pathways")
133 |         .update({ ...updateFields, updated_at: now })
134 |         .eq("id", input.recovery_id)
135 |         .select()
136 |         .single();
137 |
138 |       if (error) throw error;
139 |
140 |       // If both steps are now complete, mark the pathway as completed
141 |       if (data.ai_tutor_completed && data.practice_completed) {
142 |         const { data: completed, error: completeError } = await supabase
143 |           .from("mastery_recovery_pathways")
144 |           .update({ status: "completed", completed_at: now, updated_at: now })
145 |           .eq("id", input.recovery_id)
146 |           .select()
147 |           .single();
148 |
149 |         if (completeError) throw completeError;
150 |
151 |         // Audit log: recovery pathway completion (non-blocking)
152 |         try {
153 |           await logAuditEvent({
154 |             action: "recovery_completed",
155 |             entity_type: "mastery_recovery_pathway",
156 |             entity_id: input.recovery_id,
157 |             changes: {
158 |               student_id: completed.student_id,
159 |               clo_id: completed.clo_id,
160 |               course_id: completed.course_id,
161 |               failure_count: completed.failure_count,
162 |             },
163 |             performed_by: completed.student_id,
164 |           });
165 |         } catch (auditErr) {
166 |           console.error(
167 |             "[AuditLog] Failed to log recovery completion:",
168 |             auditErr
169 |           );
170 |         }
171 |
172 |         return completed;
173 |       }
174 |
175 |       return data;
176 |     },
177 |     onSuccess: () => {
178 |       queryClient.invalidateQueries({
179 |         queryKey: queryKeys.masteryRecovery.all,
180 |       });
181 |     },
182 |   });
183 | };
184 |
185 | // % % % useRecoveryMetrics - coordinator dashboard metrics % % % % %
186 |
187 | export const useRecoveryMetrics = (institutionId: string) => {
188 |   return useQuery({
189 |     queryKey: queryKeys.masteryRecovery.metrics(institutionId),
190 |     queryFn: async (): Promise<RecoveryMetrics> => {
191 |       const { data, error } = await supabase
192 |         .from("mastery_recovery_pathways")
193 |         .select("id, status, activated_at, completed_at, retry_outcome")
194 |         .eq("institution_id", institutionId);
195 |
196 |       if (error) throw error;
197 |
198 |       const rows = data ?? [];
199 |       const totalActivations = rows.length;
200 |
201 |       if (totalActivations === 0) {
202 |         return {
203 |           total_activations: 0,

```



```

204 completion_rate: 0,
205 avg_completion_time_hours: 0,
206 retry_success_rate: 0,
207 };
208 }
209
210 // Completion rate: completed / total
211 const completedRows = rows.filter((r) => r.status === "completed");
212 const completionRate = completedRows.length / totalActivations;
213
214 // Average completion time in hours
215 const completionTimes = completedRows
216   .filter((r) => r.completed_at && r.activated_at)
217   .map((r) => {
218     const activated = new Date(r.activated_at).getTime();
219     const completed = new Date(r.completed_at!).getTime();
220     return (completed - activated) / (1000 * 60 * 60); // ms !' hours
221   });
222
223 const avgCompletionTimeHours =
224   completionTimes.length > 0
225     ? completionTimes.reduce((sum, t) => sum + t, 0) /
226       completionTimes.length
227     : 0;
228
229 // Retry success rate: pass / (pass + fail) among those who retried
230 const retriedRows = rows.filter((r) => r.retry_outcome !== null);
231 const retrySuccessCount = retriedRows.filter(
232   (r) => r.retry_outcome === "pass"
233 ).length;
234 const retrySuccessRate =
235   retriedRows.length > 0 ? retrySuccessCount / retriedRows.length : 0;
236
237 return {
238   total_activations: totalActivations,
239   completion_rate: completionRate,
240   avg_completion_time_hours: avgCompletionTimeHours,
241   retry_success_rate: retrySuccessRate,
242 };
243 },
244 enabled: !!institutionId,
245 });
246 });
247

```

FILE: src/hooks/useMicroAssessments.ts

```
TYPE: .ts | LINES: 1-215 | SHA256: 67585b66eb8f197fcf615dcea60ca68e8d912f0869054c20f1dfff9c171717df0
```

[illegible]

```

57 |         if (error) throw error;
58 |         return data as MicroAssessment | null;
59 |     },
60 |     enabled: !!studentId,
61 | });
62 | });
63 | };
64 |
65 | // % % useCompleteMicroAssessment - mark as completed + recalculate profile completeness + award XP % % %
66 |
67 | export const useCompleteMicroAssessment = () => {
68 |     const queryClient = useQueryClient();
69 |
70 |     return useMutation({
71 |         mutationFn: async (params: {
72 |             id: string;
73 |             studentId: string;
74 |         }): Promise<MicroAssessment> => {
75 |             // 14.6: Mark micro-assessment as completed
76 |             const { data, error } = await supabase
77 |                 .from("micro_assessment_schedule")
78 |                 .update({
79 |                     status: "completed",
80 |                     completed_at: new Date().toISOString(),
81 |                 })
82 |                 .eq("id", params.id)
83 |                 .select()
84 |                 .single();
85 |
86 |             if (error) throw error;
87 |
88 |             // 14.6: Recalculate profile_completeness based on total completed responses
89 |             const { data: responseCounts } = await supabase
90 |                 .from("onboarding_responses")
91 |                 .select("question_id, onboarding_questions!inner(assessment_type)")
92 |                 .eq("student_id", params.studentId);
93 |
94 |             if (responseCounts) {
95 |                 const typeCounts: Record<string, number> = {};
96 |                 for (const r of responseCounts) {
97 |                     const type = (r.onboarding_questions as { assessment_type: string })
98 |                         ?.assessment_type;
99 |                     if (type) typeCounts[type] = (typeCounts[type] ?? 0) + 1;
100 |                 }
101 |
102 |                 const { data: baselineData } = await supabase
103 |                     .from("baseline_attainment")
104 |                     .select("id")
105 |                     .eq("student_id", params.studentId)
106 |                     .limit(1);
107 |
108 |                 const personalityItems = typeCounts["personality"] ?? 0;
109 |                 const selfEfficacyItems = typeCounts["self_efficacy"] ?? 0;
110 |                 const studyStrategyItems = typeCounts["study_strategy"] ?? 0;
111 |                 const learningStyleItems = typeCounts["learning_style"] ?? 0;
112 |                 const baselineCourses = (baselineData?.length ?? 0) > 0 ? 1 : 0;
113 |
114 |                 const weights = {
115 |                     personality: Math.min(personalityItems / 25, 1),
116 |                     self_efficacy: Math.min(selfEfficacyItems / 6, 1),
117 |                     study_strategies: Math.min(studyStrategyItems / 8, 1),
118 |                     learning_style: Math.min(learningStyleItems / 16, 1),
119 |                     baseline: baselineCourses > 0 ? 1 : 0,
120 |                 };
121 |
122 |                 const total = Object.values(weights).reduce((sum, w) => sum + w, 0);
123 |                 const newCompleteness = Math.round((total / 5) * 100);
124 |
125 |                 // Upsert student_profiles with new completeness (insert if missing)
126 |                 const { error: profileError } = await supabase
127 |                     .from("student_profiles")
128 |                     .update({ profile_completeness: newCompleteness })
129 |                     .eq("student_id", params.studentId);
130 |
131 |                 if (profileError) throw profileError;
132 |
133 |                 // 14.7: Award "Profile Complete" bonus XP when reaching 100%
134 |                 if (newCompleteness >= 100) {
135 |                     // Check if bonus was already awarded
136 |                     const { data: existingBonus } = await supabase
137 |                         .from("xp_transactions")
138 |                         .select("id")
139 |                         .eq("student_id", params.studentId)
140 |                         .eq("source", "profile_complete")
141 |                         .limit(1);
142 |
143 |                     if (!existingBonus || existingBonus.length === 0) {
144 |                         await awardXP({
145 |                             studentId: params.studentId,
146 |                             xpAmount: ONBOARDING_XP.profile_complete,
147 |                             source: "profile_complete" as XPSource,
148 |                             referenceId: `profile_complete:${params.studentId}`,
149 |                             note: "Profile completeness reached 100%",
150 |                         });
151 |                     }
152 |                 }
153 |             }
154 |
155 |             // Award micro-assessment XP
156 |             await awardXP({
157 |                 studentId: params.studentId,
158 |                 xpAmount: ONBOARDING_XP.micro_assessment,
159 |                 source: "micro_assessment" as XPSource,
160 |                 referenceId: `micro:${params.id}`,

```

```

161 |         note: "Micro-assessment completed",
162 |     });
163 |
164 |     return data as MicroAssessment;
165 | },
166 | onSuccess: (_data, variables) => {
167 |     queryClient.invalidateQueries({
168 |         queryKey: queryKeys.onboarding.microAssessments(variables.studentId),
169 |     });
170 |     queryClient.invalidateQueries({
171 |         queryKey: queryKeys.onboarding.profileCompleteness(variables.studentId),
172 |     });
173 |     queryClient.invalidateQueries({
174 |         queryKey: queryKeys.onboarding.studentProfile(variables.studentId),
175 |     });
176 | },
177 | });
178 | };
179 |
180 | // % % useDismissMicroAssessment - dismiss with limit tracking % % % % % % % %
181 |
182 | export const useDismissMicroAssessment = () => {
183 |     const queryClient = useQueryClient();
184 |
185 |     return useMutation({
186 |         mutationFn: async (params: {
187 |             id: string;
188 |             studentId: string;
189 |             currentDismissals: number;
190 |         }): Promise<MicroAssessment> => {
191 |             const newCount = params.currentDismissals + 1;
192 |             const newStatus =
193 |                 newCount >= MAX_MICRO_DISMISSALS ? "skipped" : "dismissed";
194 |
195 |             const { data, error } = await supabase
196 |                 .from("micro_assessment_schedule")
197 |                 .update({
198 |                     dismissal_count: newCount,
199 |                     status: newStatus,
200 |                 })
201 |                 .eq("id", params.id)
202 |                 .select()
203 |                 .single();
204 |
205 |             if (error) throw error;
206 |             return data as MicroAssessment;
207 |         },
208 |         onSuccess: (_data, variables) => {
209 |             queryClient.invalidateQueries({
210 |                 queryKey: queryKeys.onboarding.microAssessments(variables.studentId),
211 |             });
212 |         },
213 |     });
214 | };
215 |

```

#### FILE: src/hooks/useMostImproved.ts

TYPE: .ts | LINES: 1-8 | SHA256: 50cbd520ebea179168883306c5d1d7964b050c6e5bfbe2abc05fea7b17178a16

```

1 | // =====
2 | // useMostImproved - Most improved leaderboard data (4-week delta)
3 | // Task 20.9 - Re-exports from useMostImprovedLeaderboard
4 | // =====
5 |
6 | export { useMostImprovedLeaderboard as useMostImproved } from "@hooks/useMostImprovedLeaderboard";
7 | export type { MostImprovedEntry } from "@lib/mostImprovedLeaderboard";
8 |

```

#### FILE: src/hooks/useMostImprovedLeaderboard.ts

TYPE: .ts | LINES: 1-110 | SHA256: d764c7b9ef086c8ce5ea8ae121b58b27f9d5788005fa7bf1f5c54495051c081

```

1 | // =====
2 | // useMostImprovedLeaderboard - TanStack Query hook for Most Improved data
3 | // Task 147.3 | Requirements: 130.1
4 | // =====
5 |
6 | import { useQuery } from "@tanstack/react-query";
7 | import { supabase } from "@lib/supabase";
8 | import { queryKeys } from "@lib/queryKeys";
9 | import {
10 |     rankMostImproved,
11 |     type MostImprovedEntry,
12 | } from "@lib/mostImprovedLeaderboard";
13 |
14 | /**
15 |  * Fetches xp_transactions for all students in a course (or all) over the last 8 weeks,
16 |  * splits into current 4-week and previous 4-week periods, and ranks by improvement.
17 |  */
18 | export const useMostImprovedLeaderboard = (courseId?: string) => {
19 |     return useQuery({
20 |         queryKey: queryKeys.mostImproved.list({ courseId: courseId ?? "all" }),
21 |         queryFn: async (): Promise<MostImprovedEntry[]> => {
22 |             const now = new Date();
23 |             const fourWeeksAgo = new Date(now);
24 |             fourWeeksAgo.setDate(fourWeeksAgo.getDate() - 28);
25 |             const eightWeeksAgo = new Date(now);
26 |             eightWeeksAgo.setDate(eightWeeksAgo.getDate() - 56);
27 |
28 |             // Scope to course students if courseId provided
29 |             let studentIds: string[] | null = null;
30 |             if (courseId) {
31 |                 const { data: enrollments, error: enrollError } = await supabase
32 |                     .from("student_courses")
33 |                     .select("student_id")

```

```

34 |         .eq("course_id", courseId)
35 |         .eq("status", "active");
36 |
37 |         if (enrollError) throw enrollError;
38 |         studentIds = (enrollments ?? []).map((e) => e.student_id);
39 |         if (studentIds.length === 0) return [];
40 |     }
41 |
42 |     // Fetch transactions for the 8-week window
43 |     let query = supabase
44 |         .from("xp_transactions")
45 |         .select("student_id, xp_amount, created_at")
46 |         .gte("created_at", eightWeeksAgo.toISOString())
47 |         .order("created_at", { ascending: true });
48 |
49 |     if (studentIds) {
50 |         query = query.in("student_id", studentIds);
51 |     }
52 |
53 |     const { data: transactions, error } = await query;
54 |     if (error) throw error;
55 |
56 |     // Group by student into current and previous 4-week buckets
57 |     const studentMap = new Map<
58 |         string,
59 |         { current: number; previous: number }
60 |     >();
61 |
62 |     for (const txn of transactions ?? []) {
63 |         const d = new Date(txn.created_at);
64 |         const sid = txn.student_id as string;
65 |
66 |         if (!studentMap.has(sid)) {
67 |             studentMap.set(sid, { current: 0, previous: 0 });
68 |         }
69 |
70 |         const entry = studentMap.get(sid)!;
71 |         if (d >= fourWeeksAgo) {
72 |             entry.current += txn.xp_amount;
73 |         } else {
74 |             entry.previous += txn.xp_amount;
75 |         }
76 |     }
77 |
78 |     // Fetch names for all students in the map
79 |     const allStudentIds = [...studentMap.keys()];
80 |     if (allStudentIds.length === 0) return [];
81 |
82 |     const { data: profiles, error: profileError } = await supabase
83 |         .from("profiles")
84 |         .select("id, full_name")
85 |         .in("id", allStudentIds);
86 |
87 |     if (profileError) throw profileError;
88 |
89 |     const nameMap = new Map<string, string>();
90 |     for (const p of profiles ?? []) {
91 |         nameMap.set(p.id, p.full_name);
92 |     }
93 |
94 |     // Build entries for ranking
95 |     const entries = allStudentIds.map((sid) => {
96 |         const data = studentMap.get(sid)!;
97 |         return {
98 |             student_id: sid,
99 |             student_name: nameMap.get(sid) ?? "Unknown",
100 |             current_4_week_xp: data.current,
101 |             previous_4_week_xp: data.previous,
102 |         };
103 |     });
104 |
105 |     return rankMostImproved(entries);
106 | },
107 | staleTime: 60_000,
108 | });
109 | };
110 |

```

## FILE: src/hooks/useMysteryRewardBox.ts

TYPE: .ts | LINES: 1-43 | SHA256: 430ae1a7ecb671e9b1d54a5fbedef3fafca5ac7f2a08b405113fdd40e91bf02c

```

1 | // =====
2 | // useMysteryRewardBox – Mystery box state and reveal mutation
3 | // Task 20.5
4 | // =====
5 |
6 | import { useMutation, useQueryClient } from "@tanstack/react-query";
7 | import { supabase } from "@lib/supabase";
8 | import { queryKeys } from "@lib/queryKeys";
9 |
10 | export interface MysteryRewardResult {
11 |     success: boolean;
12 |     outcome_type: "double_xp" | "cosmetic" | "boost";
13 |     reward_details: Record<string, unknown>;
14 | }
15 |
16 | export const useResolveMysteryReward = () => {
17 |     const qc = useQueryClient();
18 |     return useMutation({
19 |         mutationFn: async ({
20 |             studentId,
21 |             institutionId,
22 |         }: {
23 |             studentId: string;

```

```

24 |     institutionId: string;
25 |   }): Promise<MysteryRewardResult> => {
26 |     const { data, error } = await supabase.functions.invoke(
27 |       "resolve-mystery-reward",
28 |       {
29 |         body: {
30 |           student_id: studentId,
31 |           institution_id: institutionId,
32 |         },
33 |       },
34 |     );
35 |     if (error) throw error;
36 |     return data as MysteryRewardResult;
37 |   },
38 |   onSuccess: () => {
39 |     qc.invalidateQueries({ queryKey: queryKeys.marketplace.all });
40 |   },
41 | });
42 | };
43 |

```

## FILE: src/hooks/useNotificationPreferences.ts

TYPE: .ts | LINES: 1-72 | SHA256: 4e3795cb525d56b02a2b08242764ef43bce3cd98390eafcd769992529f8520f6

```

1 | // Task 94.3: Notification preferences hook
2 | // Query/mutate profiles.notification_preferences JSONB column
3 |
4 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
5 | import { supabase } from "@lib/supabase";
6 | import { queryKeys } from "@lib/queryKeys";
7 | import type { Json } from "@types/database";
8 |
9 | export interface QuietHours {
10 |   enabled: boolean;
11 |   start: string; // HH:mm
12 |   end: string; // HH:mm
13 | }
14 |
15 | export interface NotificationPreferences {
16 |   muted_courses: string[]; // course IDs
17 |   quiet_hours: QuietHours;
18 | }
19 |
20 | const DEFAULT_PREFERENCES: NotificationPreferences = {
21 |   muted_courses: [],
22 |   quiet_hours: { enabled: false, start: "22:00", end: "07:00" },
23 | };
24 |
25 | export const useNotificationPreferences = (userId: string | undefined) => {
26 |   return useQuery({
27 |     queryKey: queryKeys.notificationPreferences.detail(userId ?? ""),
28 |     queryFn: async (): Promise<NotificationPreferences> => {
29 |       if (!userId) return DEFAULT_PREFERENCES;
30 |
31 |       const { data, error } = await supabase
32 |         .from("profiles")
33 |         .select("notification_preferences")
34 |         .eq("id", userId)
35 |         .maybeSingle();
36 |
37 |       if (error) throw error;
38 |
39 |       const prefs =
40 |         data?.notification_preferences as NotificationPreferences | null;
41 |       return prefs ?? DEFAULT_PREFERENCES;
42 |     },
43 |     enabled: !!userId,
44 |   });
45 | };
46 |
47 | export const useUpdateNotificationPreferences = () => {
48 |   const queryClient = useQueryClient();
49 |
50 |   return useMutation({
51 |     mutationFn: async ({
52 |       userId,
53 |       preferences,
54 |     }): {
55 |       userId: string;
56 |       preferences: NotificationPreferences;
57 |     }: Promise<void> => {
58 |       const { error } = await supabase
59 |         .from("profiles")
60 |         .update({ notification_preferences: preferences as unknown as Json })
61 |         .eq("id", userId);
62 |
63 |       if (error) throw error;
64 |     },
65 |     onSuccess: (_data, variables) => {
66 |       queryClient.invalidateQueries({
67 |         queryKey: queryKeys.notificationPreferences.detail(variables.userId),
68 |       });
69 |     },
70 |   });
71 | };
72 |

```

FILE: src/hooks/useNotificationRealtime.ts

```
TYPE: .ts | LINES: 1-68 | SHA256: 5e2fbc8a8997560da00d7382b16314081afc418582739675bbb010ebc0f5788
```

```

1 //
2 // useNotificationRealtime - Realtime subscription for new notifications
3 // On new notification: show toast + invalidate bell count
4 // Validates: Requirements 31.2, 37.4, 42
5 // =====
6
7 import { useCallback } from "react";
8 import { useQueryClient } from "@tanstack/react-query";
9 import { toast } from "sonner";
10 import { queryKeys } from "@lib/queryKeys";
11 import { useAuth } from "@hooks/useAuth";
12 import { useRealtime } from "@hooks/useRealtime";
13 import type { NotificationType } from "@hooks/useNotifications";
14 import type { RealtimePostgresChangesPayload } from "@supabase/supabase-js";
15
16 const NOTIFICATION_EMOJI: Record<NotificationType, string> = {
17   grade_released: "ðŸŽŠ",
18   new_assignment: "ðŸŽĀ",
19   badge_earned: "ðŸŒŒ",
20   streak_at_risk: "ðŸŽ‰",
21   at_risk_alert: "& p ",
22   peer_milestone: "ðŸŒŒ",
23   perfect_day_nudge: "ðŸŒŒ",
24   prerequisite_unlocked: "ðŸŽŠ ",
25   digest: "ðŸŒŒ",
26 };
27
28 /**
29  * Subscribe to realtime notifications for the current user.
30  * Shows a Sonner toast on each new notification and invalidates
31  * the notification queries so the bell count updates instantly.
32  */
33 export const useNotificationRealtime = () => {
34   const { user } = useAuth();
35   const queryClient = useQueryClient();
36
37   const onPayload = useCallback(
38     (payload: RealtimePostgresChangesPayload<Record<string, unknown>>) => {
39       if (payload.eventType !== "INSERT") return;
40
41       const row = payload.new;
42       const type = (row.type as NotificationType) ?? "grade_released";
43       const title = (row.title as string) ?? "New notification";
44       const body = (row.body as string) ?? "";
45       const emoji = NOTIFICATION_EMOJI[type] ?? "ðŸŽŠ ";
46
47       // Show toast
48       toast(`${emoji} ${title}`, {
49         description: body,
50         duration: 5000,
51       });
52
53       // Invalidate notification queries so bell count + list refresh
54       queryClient.invalidateQueries({ queryKey: queryKeys.notifications.all });
55     },
56     [queryClient]
57   );
58
59   const { isLive, retryCount } = useRealtime({
60     table: "notifications",
61     event: "INSERT",
62     filter: user?.id ? `user_id=eq.${user.id}` : undefined,
63     onPayload,
64   });
65
66   return { isLive, retryCount };
67 };
68

```

FILE: src/hooks/useNotifications.ts

TYPE: .ts | LINES: 1-143 | SHA256: 758f0b4af7dad13190cc51ea06d0ada34cc2785b9e0ca9a2485c328f29625d1e

[illegible]

[illegible]

```

133 |         .delete()
134 |         .eq("id", notificationId);
135 |
136 |         if (error) throw error;
137 |     },
138 |     onSuccess: () => {
139 |         queryClient.invalidateQueries({ queryKey: queryKeys.notifications.all });
140 |     },
141 | });
142 |
143 |

```

FILE: `src/hooks/useOfflineQueue.ts`

```
TYPE: .ts | LINES: 1-148 | SHA256: 4c93ae571505e077a41ec3f2fa32bee449d705c0020e316086da9610bcb87bf1
```

[illegible]



[illegible]

FILE: src/hooks/useOnboarding.ts

```
TYPE: .ts | LINES: 1-62 | SHA256: 20c66660b9bd48db8b8bd7a60e580f700410dba7432a8eeca3531c27cb6b4e9c
```

```
// =====  
2 // useOnboarding - Query & mutate onboarding status from profiles table  
3 // =====  
4  
5 import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";  
6 import { supabase } from "@/lib/supabase";  
7 import { queryKeys } from "@/lib/queryKeys";  
8 import { useAuth } from "@/hooks/useAuth";  
9  
10 // %% Query Keys %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
11  
12 const onboardingKeys = {  
13   status: (userId: string) => ["onboarding", "status", userId] as const,  
14 };  
15  
16 // %% useOnboardingStatus - fetch onboarding_completed from profiles %%%%%%%%%%  
17  
18 export const useOnboardingStatus = () => {  
19   const { user } = useAuth();  
20   const userId = user?.id ?? "";  
21  
22   return useQuery({  
23     queryKey: onboardingKeys.status(userId),  
24     queryFn: async (): Promise<boolean> => {  
25       const { data, error } = await supabase  
26         .from("profiles")  
27         .select("onboarding_completed")  
28         .eq("id", userId)  
29         .single();  
30  
31       if (error) throw error;  
32       return data?.onboarding_completed ?? false;  
33     },  
34     enabled: !!userId,  
35   });  
36 }  
37  
38 // %% useCompleteOnboarding - set onboarding_completed = true %%%%%%%%%%%%%%  
39  
40 export const useCompleteOnboarding = () => {  
41   const { user } = useAuth();
```

FILE: src/hooks/useOnboardingProgress.ts

```

80 |         updated_at: new Date().toISOString(),
81 |     },
82 |     { onConflict: "student_id" }
83 | )
84 | .select()
85 | .single();
86 |
87 | if (error) throw error;
88 | return data as OnboardingProgress;
89 | },
90 | onSuccess: () => {
91 |     queryClient.invalidateQueries({
92 |         queryKey: queryKeys.onboarding.progress(studentId),
93 |     });
94 | },
95 | });
96 | };
97 |

```

FILE: src/hooks/useOnboardingQuestions.ts

```
TYPE: .ts | LINES: 1-97 | SHA256: c234d0d5a42de962eed297ffadbeed389d08565fe5a051309925d75197f15ed9
```

```

1 import { useQuery } from "@tastack/react-query";
2 import { supabase } from "@lib/supabase";
3 import { queryKeys } from "@lib/queryKeys";
4
5 // %% Types %%%%%%%%%%%%%%%
6
7 export interface OnboardingQuestion {
8   id: string;
9   institution_id: string;
10  assessment_type: string;
11  question_text: string;
12  dimension: string | null;
13  weight: number | null;
14  options: unknown[] | null;
15  correct_option: number | null;
16  clo_id: string | null;
17  course_id: string | null;
18  difficulty_level: string | null;
19  sort_order: number;
20  is_active: boolean;
21  created_at: string;
22  updated_at: string;
23 }
24
25 // %% Shared fetcher %%%%%%%%%%%%%%%
26
27 const fetchQuestions = async (
28   assessmentType: string,
29   courseId?: string
30 ): Promise<OnboardingQuestion[]> => {
31   let query = supabase
32     .from("onboarding_questions")
33     .select("*")
34     .eq("assessment_type", assessmentType)
35     .eq("is_active", true)
36     .order("sort_order", { ascending: true });
37
38   if (courseId) {
39     query = query.eq("course_id", courseId);
40   }
41
42   const { data, error } = await query;
43   if (error) throw error;
44   return (data ?? []) as OnboardingQuestion[];
45 };
46
47 // %% usePersonalityQuestions %%%%%%%%%%%%%%%
48
49 export const usePersonalityQuestions = () => {
50   return useQuery({
51     queryKey: queryKeys.onboarding.questions("personality"),
52     queryFn: () => fetchQuestions("personality"),
53     staleTime: 5 * 60_000,
54   });
55 };
56
57 // %% useLearningStyleQuestions %%%%%%%%%%%%%%%
58
59 export const useLearningStyleQuestions = () => {
60   return useQuery({
61     queryKey: queryKeys.onboarding.questions("learning_style"),
62     queryFn: () => fetchQuestions("learning_style"),
63     staleTime: 5 * 60_000,
64   });
65 };
66
67 // %% useSelfEfficacyQuestions %%%%%%%%%%%%%%%
68
69 export const useSelfEfficacyQuestions = () => {
70   return useQuery({
71     queryKey: queryKeys.onboarding.questions("self_efficacy"),
72     queryFn: () => fetchQuestions("self_efficacy"),
73     staleTime: 5 * 60_000,
74   });
75 };
76
77 // %% useStudyStrategyQuestions %%%%%%%%%%%%%%%
78
79 export const useStudyStrategyQuestions = () => {
80   return useQuery({
81     queryKey: queryKeys.onboarding.questions("study_strategy"),
82     queryFn: () => fetchQuestions("study_strategy"),
83   });
84 };

```

FILE: src/hooks/useOnboardingResponses.ts

FILE: src/hooks/useOptimisticToggle.ts

```

1 import { useCallback, useState } from "react";
2 import { useMutation, useQueryClient } from "@tanstack/react-query";
3
4 /**
5  * Generic row type with an id and a field to toggle
6  */
7 interface ToggleableRow {
8   id: string;
9   [key: string]: unknown;
10 }
11
12 /**
13  * Configuration for useOptimisticToggle
14  */
15 interface UseOptimisticToggleConfig<TRow extends ToggleableRow> {
16   /** Query key for the data being toggled */
17   queryKey: unknown[];
18   /** Mutation function that performs the toggle on the server */
19   mutationFn: (rowId: string, newValue: boolean) => Promise<void>;
20   /** Field name to toggle (e.g., 'is_active', 'is_verified') */
21   field: keyof TRow;
22 }
23
24 /**
25  * Return type for useOptimisticToggle
26  */
27 interface UseOptimisticToggleReturn {

```

```

28 | /** Current checked state (optimistic) */
29 | isChecked: boolean;
30 | /** Handler to toggle the value */
31 | onToggle: () => void;
32 | /** Whether the toggle is currently in optimistic state (mutation pending) */
33 | isOptimistic: boolean;
34 | /** Whether the mutation is pending */
35 | isPending: boolean;
36 | }
37 |
38 | /**
39 | * Hook for optimistic toggle updates with TanStack Query
40 | *
41 | * Implements the standard pattern:
42 | * 1. onMutate: optimistically update the cache
43 | * 2. onError: rollback to previous value
44 | * 3. onSettled: invalidate the query to refetch
45 | *
46 | * Usage:
47 | * ```tsx
48 | * const { isChecked, onToggle, isOptimistic } = useOptimisticToggle({
49 | *   queryKey: queryKeys.user.list(),
50 | *   mutationFn: async (userId, newValue) => {
51 | *     await supabase.from('users').update({ is_active: newValue }).eq('id', userId);
52 | *   },
53 | *   field: 'is_active',
54 | * });
55 | *
56 | * return (
57 | *   <div className="flex items-center gap-2">
58 | *     <Switch checked={isChecked} onCheckedChange={onToggle} />
59 | *     {isOptimistic && <Loader2 className="h-4 w-4 animate-spin" />}
60 | *   </div>
61 | * );
62 | * ```
63 | */
64 | export const useOptimisticToggle = <TRow extends ToggleableRow>({
65 |   rowId: string,
66 |   currentValue: boolean,
67 |   config: UseOptimisticToggleConfig<TRow>
68 | }): UseOptimisticToggleReturn => {
69 |   const queryClient = useQueryClient();
70 |   const [optimisticValue, setOptimisticValue] = useState(currentValue);
71 |
72 |   const mutation = useMutation({
73 |     mutationFn: async () => {
74 |       await config.mutationFn(rowId, !optimisticValue);
75 |     },
76 |     onMutate: async () => {
77 |       // Cancel any outgoing refetches so they don't overwrite optimistic update
78 |       await queryClient.cancelQueries({ queryKey: config.queryKey });
79 |
80 |       // Snapshot the previous value
81 |       const previousData = queryClient.getQueryData(config.queryKey);
82 |
83 |       // Optimistically update the cache
84 |       queryClient.setQueryData(config.queryKey, (old: unknown) => {
85 |         if (!Array.isArray(old)) return old;
86 |         return old.map((row: TRow) =>
87 |           row.id === rowId ? { ...row, [config.field]: !optimisticValue } : row
88 |         );
89 |       });
90 |
91 |       // Update local optimistic state
92 |       setOptimisticValue((prev) => !prev);
93 |
94 |       return { previousData };
95 |     },
96 |     onError: (_err, _variables, context) => {
97 |       // Rollback to previous value on error
98 |       if (context?.previousData) {
99 |         queryClient.setQueryData(config.queryKey, context.previousData);
100 |       }
101 |       setOptimisticValue(currentValue);
102 |     },
103 |     onSettled: () => {
104 |       // Invalidate the query to refetch fresh data
105 |       queryClient.invalidateQueries({ queryKey: config.queryKey });
106 |     },
107 |   });
108 |
109 |   const onToggle = useCallback(() => {
110 |     mutation.mutate();
111 |   }, [mutation]);
112 |
113 |   return {
114 |     isChecked: optimisticValue,
115 |     onToggle,
116 |     isOptimistic: mutation.isPending,
117 |     isPending: mutation.isPending,
118 |   };
119 | };
120 |

```

FILE: [src/hooks/useOptimisticXP.ts](#)

TYPE: .ts | LINES: 1-127 | SHA256: f2cb1daea5a19da78ed19daa6c1d9a7f3a6526eaae5dc88874856cda2559c01c

```

1 | // =====
2 | // useOptimisticXP - Fire-and-forget XP award with optimistic cache updates
3 | // Feature: edeviser-platform, Task 54.7
4 | // =====
5 |
6 | import { useQueryClient } from "@tanstack/react-query";
7 | import { useCallback, useEffect, useRef } from "react";

```

[illegible]



```

85 |         blooms_level: startRow.blooms_level,
86 |         course_id: startRow.course_id,
87 |     };
88 |
89 |     // % % 2. Graduate Attributes mapped to the start ILO (GA level) % % % % % %
90 |     const { data: gaMapRows, error: gaMapError } = await supabase
91 |       .from("graduate_attribute_mappings")
92 |       .select("graduate_attribute_id, outcome_id, weight")
93 |       .eq("outcome_id", startId);
94 |     if (gaMapError) throw gaMapError;
95 |     const gaMappings = gaMapRows ?? [];
96 |
97 |     const gaIds = [
98 |       ...new Set(gaMappings.map((m) => m.graduate_attribute_id)),
99 |     ];
100 |     let graduateAttributes: { id: string; name: string }[] = [];
101 |     if (gaIds.length > 0) {
102 |       // Scope GA lookup to the start outcome's institution (defensive;
103 |       // RLS already enforces this).
104 |       const { data: gaRows, error: gaError } = await supabase
105 |         .from("graduate_attributes")
106 |         .select("id, name")
107 |         .eq("institution_id", startRow.institution_id)
108 |         .in("id", gaIds);
109 |       if (gaError) throw gaError;
110 |       graduateAttributes = gaRows ?? [];
111 |     }
112 |
113 |     // % % 3. PLOs mapped under the start ILO (source = ILO) % % % % % % % % % % %
114 |     const { data: ploMapRows, error: ploMapError } = await supabase
115 |       .from("outcome_mappings")
116 |       .select("source_outcome_id, target_outcome_id, weight")
117 |       .eq("source_outcome_id", startId);
118 |     if (ploMapError) throw ploMapError;
119 |     const ploMappings = ploMapRows ?? [];
120 |
121 |     const ploIds = [...new Set(ploMappings.map((m) => m.target_outcome_id))];
122 |     let plos: RawChainOutcome[] = [];
123 |     let cloMappings: {
124 |       source_outcome_id: string;
125 |       target_outcome_id: string;
126 |       weight: number;
127 |     }[] = [];
128 |     let clos: RawChainOutcome[] = [];
129 |
130 |     if (ploIds.length > 0) {
131 |       const { data: ploRows, error: ploError } = await supabase
132 |         .from("learning_outcomes")
133 |         .select(OUTCOME_COLUMNS)
134 |         .in("id", ploIds);
135 |       if (ploError) throw ploError;
136 |       plos = ploRows ?? [];
137 |
138 |       // % % 4. CLOs mapped under those PLOs (source in ploIds) % % % % % % % % % %
139 |       const { data: cloMapRows, error: cloMapError } = await supabase
140 |         .from("outcome_mappings")
141 |         .select("source_outcome_id, target_outcome_id, weight")
142 |         .in("source_outcome_id", ploIds);
143 |       if (cloMapError) throw cloMapError;
144 |       cloMappings = cloMapRows ?? [];
145 |
146 |       const cloIds = [
147 |         ...new Set(cloMappings.map((m) => m.target_outcome_id)),
148 |       ];
149 |       if (cloIds.length > 0) {
150 |         const { data: cloRows, error: cloError } = await supabase
151 |           .from("learning_outcomes")
152 |           .select(OUTCOME_COLUMNS)
153 |           .in("id", cloIds);
154 |         if (cloError) throw cloError;
155 |         clos = cloRows ?? [];
156 |       }
157 |     }
158 |
159 |     const cloIds = clos.map((c) => c.id);
160 |     const cloIdSet = new Set(cloIds);
161 |     const courseIds = [
162 |       ...new Set(
163 |         clos.map((c) => c.course_id).filter((id): id is string => !!id)
164 |       ),
165 |     ];
166 |
167 |     // % % 5. Rubrics attached to those CLOs (rubrics.clo_id) % % % % % % % % % % %
168 |     let rubricRows: { id: string; title: string; clo_id: string | null }[] =
169 |       [];
170 |     if (cloIds.length > 0) {
171 |       const { data, error } = await supabase
172 |         .from("rubrics")
173 |         .select("id, title, clo_id")
174 |         .in("clo_id", cloIds);
175 |       if (error) throw error;
176 |       rubricRows = data ?? [];
177 |     }
178 |
179 |     // % % 6. Assessments linked to those CLOs (assignments.clo_weights) % % % %
180 |     // Batched by the CLOs' course_ids (no N+1), then narrowed in-memory to
181 |     // assignments whose clo_weights reference one of our CLOs.
182 |     let assignments: RawChainAssignment[] = [];
183 |     if (courseIds.length > 0) {
184 |       const { data: assignmentRows, error: assignmentError } = await supabase
185 |         .from("assignments")
186 |         .select("id, title, course_id, rubric_id, clo_weights")
187 |         .in("course_id", courseIds);
188 |       if (assignmentError) throw assignmentError;

```



[illegible]

FILE: src/hooks/usePageViewLogger.ts

TYPE: .ts | LINES: 1-42 | SHA256: 389419dd13e8db41d930dd4353eab2d1b8817852386702f27937404de8792621

```

1 import { useEffect, useRef } from "react";
2 import { useLocation } from "react-router-dom";
3 import { useAuth } from "@hooks/useAuth";
4 import { logActivity } from "@lib/activityLogger";
5
6 /**
7  * Logs a `page_view` activity event whenever the student navigates to a new route.
8  * Includes `duration_seconds` metadata tracking how long the previous page was viewed.
9  * Must be rendered inside a Router context. Only fires for students.
10 */
11 export const usePageViewLogger = (): void => {
12   const { pathname } = useLocation();
13   const { profile } = useAuth();
14   const prevPathRef = useRef<string | null>(null);
15   const pageEnteredAtRef = useRef<number>(0);
16
17   useEffect(() => {
18     if (!profile?.id || profile.role !== "student") return;
19     if (pathname === prevPathRef.current) return;
20
21     const now = Date.now();
22
23     // Log the previous page with its duration

```

```

24 |     if (prevPathRef.current !== null && pageEnteredAtRef.current > 0) {
25 |         const durationSeconds = Math.round(
26 |             (now - pageEnteredAtRef.current) / 1000
27 |         );
28 |         logActivity({
29 |             student_id: profile.id,
30 |             event_type: "page_view",
31 |             metadata: {
32 |                 path: prevPathRef.current,
33 |                 duration_seconds: durationSeconds,
34 |             },
35 |         });
36 |     }
37 |
38 |     prevPathRef.current = pathname;
39 |     pageEnteredAtRef.current = now;
40 | }, [pathname, profile?.id, profile?.role]);
41 | };
42 |

```

## FILE: src/hooks/useParentDashboard.ts

TYPE: .ts | LINES: 1-210 | SHA256: 34d4bf592631bd81740a3b5471ede16e539d752f3f09ab21f1c4242ac85f9253

```

1 | import { useQuery } from "@tanstack/react-query";
2 | import { supabase } from "@/lib/supabase";
3 | import { queryKeys } from "@/lib/queryKeys";
4 |
5 | export interface LinkedChild {
6 |     student_id: string;
7 |     student_name: string;
8 |     current_level: number;
9 |     xp_total: number;
10 |    current_streak: number;
11 |    enrolled_courses: number;
12 |    avg_attainment: number;
13 | }
14 |
15 | export interface ParentKPIData {
16 |     linkedChildren: number;
17 |     totalCourses: number;
18 |     avgAttainment: number;
19 |     upcomingDeadlines: number;
20 | }
21 |
22 | /**
23 |  * Resolve the verified linked-child student IDs for a parent. RLS on
24 |  * `parent_student_links` already restricts rows to the calling parent; the
25 |  * explicit `verified = true` filter mirrors the parent-access policy scope so
26 |  * unverified links never surface a child.
27 |  */
28 | const fetchVerifiedChildIds = async (parentId: string): Promise<string[]> => {
29 |     const { data: links, error } = await supabase
30 |         .from("parent_student_links")
31 |         .select("student_id")
32 |         .eq("parent_id", parentId)
33 |         .eq("verified", true);
34 |
35 |     if (error) throw error;
36 |     return (links ?? []).map((l) => l.student_id);
37 | };
38 |
39 | export const useLinkedChildren = (
40 |     parentId: string | undefined,
41 |     options?: { enabled?: boolean }
42 | ) => {
43 |     return useQuery({
44 |         queryKey: queryKeys.parentStudentLinks.list({ parentId }),
45 |         queryFn: async (): Promise<LinkedChild[]> => {
46 |             if (!parentId) return [];
47 |
48 |             const studentIds = await fetchVerifiedChildIds(parentId);
49 |
50 |             if (studentIds.length === 0) return [];
51 |
52 |             const { data: profiles, error: profilesError } = await supabase
53 |                 .from("profiles")
54 |                 .select("id, full_name")
55 |                 .in("id", studentIds);
56 |
57 |             if (profilesError) throw profilesError;
58 |             const typedProfiles = profiles ?? [];
59 |
60 |             // Batch fetch gamification for all children (1 query instead of N)
61 |             const { data: gamData, error: gamError } = await supabase
62 |                 .from("student_gamification")
63 |                 .select("student_id, level, xp_total, streak_current")
64 |                 .in("student_id", studentIds);
65 |
66 |             if (gamError) throw gamError;
67 |             const gamMap = new Map((gamData ?? []).map((g) => [g.student_id, g]));
68 |
69 |             // Batch fetch enrollment data for all children (1 query instead of N).
70 |             // Reads `student_courses` – now visible to verified parents via RLS.
71 |             const { data: enrollData, error: enrollError } = await supabase
72 |                 .from("student_courses")
73 |                 .select("student_id")
74 |                 .in("student_id", studentIds);
75 |
76 |             if (enrollError) throw enrollError;
77 |
78 |             const enrollCountMap = new Map<string, number>();
79 |             for (const e of enrollData ?? []) {
80 |                 enrollCountMap.set(
81 |                     e.student_id,

```

```

82 |         (enrollCountMap.get(e.student_id) ?? 0) + 1
83 |     );
84 | }
85 |
86 | // Batch fetch course-scope attainment for all children (1 query). The
87 | // parent `outcome_attainment` policy already exposes these rows for
88 | // verified linked children; we average the per-course attainment.
89 | const { data: attainmentData, error: attainmentError } = await supabase
90 |   .from("outcome_attainment")
91 |   .select("student_id, attainment_percent")
92 |   .in("student_id", studentIds)
93 |   .eq("scope", "student_course");
94 |
95 | if (attainmentError) throw attainmentError;
96 |
97 | const attainmentMap = new Map<string, { sum: number; count: number }>();
98 | for (const row of attainmentData ?? []) {
99 |   if (!row.student_id) continue;
100 |   const cur = attainmentMap.get(row.student_id) ?? { sum: 0, count: 0 };
101 |   cur.sum += row.attainment_percent;
102 |   cur.count += 1;
103 |   attainmentMap.set(row.student_id, cur);
104 | }
105 |
106 | // Map results back to each child
107 | const children: LinkedChild[] = typedProfiles.map((profile) => {
108 |   const gam = gamMap.get(profile.id);
109 |   const att = attainmentMap.get(profile.id);
110 |   return {
111 |     student_id: profile.id,
112 |     student_name: profile.full_name ?? "Unknown",
113 |     current_level: gam?.level ?? 1,
114 |     xp_total: gam?.xp_total ?? 0,
115 |     current_streak: gam?.streak_current ?? 0,
116 |     enrolled_courses: enrollCountMap.get(profile.id) ?? 0,
117 |     avg_attainment:
118 |       att && att.count > 0 ? Math.round(att.sum / att.count) : 0,
119 |   };
120 | });
121 |
122 | return children;
123 | },
124 | enabled: (options?.enabled ?? true) && !!parentId,
125 | staleTime: 60_000,
126 | });
127 | };
128 |
129 | export const useParentKPIs = (
130 |   parentId: string | undefined,
131 |   options?: { enabled?: boolean }
132 | ) => {
133 |   return useQuery({
134 |     queryKey: queryKeys.parentDashboard.detail(parentId ?? ""),
135 |     queryFn: async (): Promise<ParentKPIData> => {
136 |       const empty: ParentKPIData = {
137 |         linkedChildren: 0,
138 |         totalCourses: 0,
139 |         avgAttainment: 0,
140 |         upcomingDeadlines: 0,
141 |       };
142 |
143 |       if (!parentId) return empty;
144 |
145 |       const studentIds = await fetchVerifiedChildIds(parentId);
146 |
147 |       if (studentIds.length === 0) return empty;
148 |
149 |       // Enrolled courses across all linked children (now-visible via parent
150 |       // SELECT RLS on `student_courses`).
151 |       const { data: enrollData, error: enrollError } = await supabase
152 |         .from("student_courses")
153 |         .select("course_id")
154 |         .in("student_id", studentIds)
155 |         .eq("status", "active");
156 |
157 |       if (enrollError) throw enrollError;
158 |
159 |       const courseIds = Array.from(
160 |         new Set(
161 |           (enrollData ?? [])
162 |             .map((e) => e.course_id)
163 |             .filter((id): id is string => id !== null)
164 |         )
165 |       );
166 |
167 |       // Average course-scope attainment across all children.
168 |       const { data: attainmentData, error: attainmentError } = await supabase
169 |         .from("outcome_attainment")
170 |         .select("attainment_percent")
171 |         .in("student_id", studentIds)
172 |         .eq("scope", "student_course");
173 |
174 |       if (attainmentError) throw attainmentError;
175 |
176 |       const attainmentRows = attainmentData ?? [];
177 |       const avgAttainment =
178 |         attainmentRows.length > 0
179 |           ? Math.round(
180 |             attainmentRows.reduce((sum, r) => sum + r.attainment_percent, 0) /
181 |             attainmentRows.length
182 |           )
183 |           : 0;
184 |
185 |       // Upcoming assignment deadlines across enrolled courses.

```

```

186 |     let upcomingDeadlines = 0;
187 |     if (courseIds.length > 0) {
188 |         const nowIso = new Date().toISOString();
189 |         const { count, error: deadlinesError } = await supabase
190 |             .from("assignments")
191 |             .select("count", { count: "exact", head: true })
192 |             .in("course_id", courseIds)
193 |             .gte("due_date", nowIso);
194 |
195 |         if (deadlinesError) throw deadlinesError;
196 |         upcomingDeadlines = count ?? 0;
197 |     }
198 |
199 |     return {
200 |         linkedChildren: studentIds.length,
201 |         totalCourses: enrollData?.length ?? 0,
202 |         avgAttainment,
203 |         upcomingDeadlines,
204 |     };
205 | },
206 | enabled: (options?.enabled ?? true) && !!parentId,
207 | staleTime: 60_000,
208 | });
209 | };
210 |

```

## FILE: src/hooks/useParentDashboardAggregate.ts

TYPE: .ts | LINES: 1-68 | SHA256: 670d1177b70b7acfc7ee0f31942a8372549541c8e56232018452a888f94fd281

```

1 | import { useQuery, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { DASHBOARD_STALE_TIME_MS } from "@lib/queryConfig";
5 | import type { ParentKPIData, LinkedChild } from "@hooks/useParentDashboard";
6 |
7 | export interface ParentDashboardAggregate {
8 |     kpis: ParentKPIData;
9 |     children: LinkedChild[];
10 | }
11 |
12 | /**
13 |  * Parent dashboard aggregate (spec: dashboard-and-ux-performance, Phase 8
14 |  * Task 36).
15 |  *
16 |  * Collapses the two always-on parent hooks – `useParentKPIs` and
17 |  * `useLinkedChildren` (each of which resolves verified children then fans out to
18 |  * `student_courses` / `outcome_attainment` / `student_gamification` / `assignments`) –
19 |  * into ONE call to the `get_parent_dashboard` RPC, then HYDRATES the exact caches
20 |  * both hooks read so they resolve as cache hits.
21 |  *
22 |  * The RPC is `SECURITY INVOKER` and resolves children from `auth.uid()`: every
23 |  * read is RLS-scoped to the caller's verified-linked children, so this adds no
24 |  * new data visibility. On RPC failure each section hook falls back to its own
25 |  * fetch (fully reversible).
26 |  *
27 |  * `parentId` is used only for the query key + enabled gate (it is the caller's
28 |  * own id); the RPC itself takes no argument and derives the parent from the
29 |  * session via `auth.uid()`.
30 |  */
31 | export const useParentDashboardAggregate = (parentId: string | undefined) => {
32 |     const queryClient = useQueryClient();
33 |
34 |     return useQuery({
35 |         queryKey: queryKeys.parentDashboard.list({ aggregate: true, parentId }),
36 |         enabled: !!parentId,
37 |         staleTime: DASHBOARD_STALE_TIME_MS,
38 |         queryFn: async (): Promise<ParentDashboardAggregate> => {
39 |             // `get_parent_dashboard` is added by migration 20260821000014 and is not in
40 |             // the generated Database types until regenerated post-merge, so cast the
41 |             // rpc call (same pattern as the other dashboard aggregates / `get_xp_balance`).
42 |             // eslint-disable-next-line @typescript-eslint/no-explicit-any
43 |             const { data, error } = await (supabase as any).rpc(
44 |                 "get_parent_dashboard",
45 |                 {}
46 |             );
47 |             if (error) throw error;
48 |             if (!data) {
49 |                 throw new Error("get_parent_dashboard returned no data");
50 |             }
51 |
52 |             const payload = data as ParentDashboardAggregate;
53 |
54 |             // Hydrate the EXACT caches the two section hooks read so they become hits.
55 |             queryClient.setQueryData(
56 |                 queryKeys.parentDashboard.detail(parentId ?? ""),
57 |                 payload.kpis
58 |             );
59 |             queryClient.setQueryData(
60 |                 queryKeys.parentStudentLinks.list({ parentId }),
61 |                 payload.children
62 |             );
63 |
64 |             return payload;
65 |         },
66 |     });
67 | };
68 |

```

FILE: `src/hooks/useParentProgress.ts`

```
TYPE: .ts | LINES: 1-70 | SHA256: a1d28df17b6b66c459de9fa9a46603877abd294bdc1c4d7fca4169c8da75faea
```

```

1 import { useQuery } from "@tanstack/react-query";
2 import { queryKeys } from "@lib/queryKeys";
3 import { supabase } from "@lib/supabase";
4
5 export interface ParentCourseProgress {
6   course_id: string;
7   course_name: string;
8   course_code: string;
9   attainment_percent: number;
10 }
11
12 export const useParentChildProgress = (studentId: string | undefined) =>
13   useQuery({
14     queryKey: queryKeys.outcomeAttainment.list({
15       studentId,
16       view: "parent-progress",
17     }),
18     queryFn: async (): Promise<ParentCourseProgress[]> => {
19       if (!studentId) return [];
20
21       const { data: enrollments, error: enrollmentError } = await supabase
22         .from("student_courses")
23         .select("course_id, courses!inner(id, name, code)")
24         .eq("student_id", studentId)
25         .eq("status", "active");
26
27       if (enrollmentError) throw enrollmentError;
28       if (!enrollments || enrollments.length === 0) return [];
29
30       const courseIds = enrollments.map((enrollment) => enrollment.course_id);
31       const { data: attainment, error: attainmentError } = await supabase
32         .from("outcome_attainment")
33         .select("course_id, attainment_percent")
34         .eq("student_id", studentId)
35         .in("course_id", courseIds)
36         .eq("scope", "student_course");
37
38       if (attainmentError) throw attainmentError;
39
40       const totals = new Map<string, { sum: number; count: number }>();
41       for (const row of attainment ?? []) {
42         if (!row.course_id) continue;
43         const current = totals.get(row.course_id) ?? { sum: 0, count: 0 };
44         current.sum += row.attainment_percent;
45         current.count += 1;
46         totals.set(row.course_id, current);
47       }
48
49       return enrollments.map((enrollment) => {
50         const course = enrollment.courses as unknown as {
51           id: string;
52           name: string;
53           code: string;
54         };
55         const aggregate = totals.get(course.id);
56         return {
57           course_id: course.id,
58           course_name: course.name,
59           course_code: course.code,
60           attainment_percent:
61             aggregate && aggregate.count > 0
62               ? Math.round(aggregate.sum / aggregate.count)
63               : 0,
64         };
65       });
66     },
67     enabled: !!studentId,
68     staleTime: 60_000,
69   });

```

FILE: [src/hooks/usePeerTeaching.ts](#)

TYPE: .ts | LINES: 1-225 | SHA256: 41b5db7bf7c4c527f2d281508dc51ec43cf214a5ec09b2999d5056a979c2d8b8

[illegible]

[illegible]

```

131 |     }: RecordViewInput) => {
132 |       const { data, error } = await supabase
133 |         .from("teaching_moment_views" as never)
134 |         .insert({
135 |           teaching_moment_id,
136 |           viewer_id,
137 |           pre_view_attainment: pre_view_attainment ?? null,
138 |         } as never)
139 |         .select()
140 |         .single();
141 |       if (error) throw error;
142 |       return data as TeachingMomentView;
143 |     },
144 |     onSuccess: (_data, variables) => {
145 |       qc.invalidateQueries({
146 |         queryKey: queryKeys.peerTeachingMoments.list({
147 |           teamId: variables.teamId,
148 |         }),
149 |       });
150 |     },
151 |     onError: (error: Error) => {
152 |       toast.error(error.message || "Failed to record view");
153 |     },
154 |   });
155 | };
156 |
157 | // % % % useSubmitRating - submit a rating for a teaching moment % % % % % % % % % % % % % % % %
158 |
159 | interface SubmitRatingPayload extends RateTeachingMomentInput {
160 |   viewer_id: string;
161 |   teamId: string;
162 | }
163 |
164 | export const useSubmitRating = () => {
165 |   const qc = useQueryClient();
166 |   return useMutation({
167 |     mutationFn: async ({
168 |       teaching_moment_id,
169 |       clarity_rating,
170 |       helpfulness_rating,
171 |       viewer_id,
172 |     }: SubmitRatingPayload) => {
173 |       const { data, error } = await supabase
174 |         .from("teaching_moment_ratings" as never)
175 |         .upsert(
176 |           {
177 |             teaching_moment_id,
178 |             viewer_id,
179 |             clarity_rating,
180 |             helpfulness_rating,
181 |             rated_at: new Date().toISOString(),
182 |           } as never,
183 |           { onConflict: "teaching_moment_id,viewer_id" }
184 |         )
185 |         .select()
186 |         .single();
187 |       if (error) throw error;
188 |       return data as TeachingMomentRating;
189 |     },
190 |     onSuccess: (_data, variables) => {
191 |       qc.invalidateQueries({
192 |         queryKey: queryKeys.teachingMomentRatings.list({
193 |           momentId: variables.teaching_moment_id,
194 |         }),
195 |       });
196 |       qc.invalidateQueries({
197 |         queryKey: queryKeys.peerTeachingMoments.list({
198 |           teamId: variables.teamId,
199 |         }),
200 |       });
201 |     },
202 |     onError: (error: Error) => {
203 |       toast.error(error.message || "Failed to submit rating");
204 |     },
205 |   });
206 | };
207 |
208 | // % % % useMomentRatings - fetch ratings for a teaching moment % % % % % % % % % % % % % % % %
209 |
210 | export const useMomentRatings = (momentId?: string) => {
211 |   return useQuery({
212 |     queryKey: queryKeys.teachingMomentRatings.list({ momentId }),
213 |     queryFn: async (): Promise<TeachingMomentRating[]> => {
214 |       const { data, error } = await supabase
215 |         .from("teaching_moment_ratings" as never)
216 |         .select(".*")
217 |         .eq("teaching_moment_id", momentId!)
218 |         .order("rated_at", { ascending: false });
219 |       if (error) throw error;
220 |       return (data ?? []) as TeachingMomentRating[];
221 |     },
222 |     enabled: !!momentId,
223 |   });
224 | };
225 |

```

## FILE: src/hooks/usePersonalBest.ts

TYPE: .ts | LINES: 1-8 | SHA256: f03e887340f40252f4a8154a2a339860c8579767fbd1a21389cd4a3e4e131ba5

```

1 | // =====
2 | // usePersonalBest - Personal best leaderboard data (current vs previous week)
3 | // Task 20.8 - Re-exports from usePersonalBestLeaderboard
4 | // =====
5 |

```

FILE: src/hooks/usePersonalBestLeaderboard.ts

```

1 //
2 // usePersonalBestLeaderboard – TanStack Query hook for Personal Best data
3 // Task 147.3 | Requirements: 129.1
4 // =====
5
6 import { useQuery } from "@tanstack/react-query";
7 import { supabase } from "@lib/supabase";
8 import { queryKeys } from "@lib/queryKeys";
9 import {
10   computeWeeklyXP,
11   getISOWeekStart,
12   type WeeklyXP,
13 } from "@lib/personalBestLeaderboard";
14
15 /**
16  * Fetches the student's xp_transactions for the last 8 ISO weeks
17  * and computes weekly XP totals with personal best identification.
18  */
19 export const usePersonalBestLeaderboard = (studentId: string) => {
20   return useQuery({
21     queryKey: queryKeys.personalBest.list({ studentId }),
22     queryFn: async (): Promise<WeeklyXP[]> => {
23       // Calculate the start of 8 weeks ago (Monday)
24       const now = new Date();
25       const currentWeekStart = getISOWeekStart(now);
26       const eightWeeksAgo = new Date(currentWeekStart);
27       eightWeeksAgo.setDate(eightWeeksAgo.getDate() - 7 * 7); // 7 more weeks back
28
29       const { data, error } = await supabase
30         .from("xp_transactions")
31         .select("xp_amount, created_at")
32         .eq("student_id", studentId)
33         .gte("created_at", eightWeeksAgo.toISOString())
34         .order("created_at", { ascending: true });
35
36       if (error) throw error;
37
38       return computeWeeklyXP(data ?? [], now);
39     },
40     enabled: !!studentId,
41     staleTime: 60_000,
42   });
43 };
44

```

[illegible]



[illegible]

```

154 |
155 |     queryClient.setQueriesData(
156 |       { queryKey: queryKeys.plannerTasks.lists() },
157 |       (old: PlannerTask[] | undefined) => {
158 |         if (!old) return old;
159 |         return old.filter((t) => t.id !== taskId);
160 |       }
161 |     );
162 |
163 |     return { previousTasks };
164 |   },
165 |   onError: (_err, _taskId, context) => {
166 |     if (context?.previousTasks) {
167 |       for (const [key, data] of context.previousTasks) {
168 |         queryClient.setQueryData(key, data);
169 |       }
170 |     }
171 |     toast.error("Failed to delete task");
172 |   },
173 |   onSettled: () => {
174 |     invalidateTaskQueries(queryClient);
175 |   },
176 |   onSuccess: () => {
177 |     toast.success("Task deleted");
178 |   },
179 | });
180 |
181 | // % % % useCompleteTask - marks task complete + awards XP via award-xp % % % % % % % % %
182 |
183 | export const useCompleteTask = () => {
184 |   const queryClient = useQueryClient();
185 |   const { user } = useAuth();
186 |
187 |   return useMutation({
188 |     mutationFn: async (
189 |       taskId: string
190 |     ): Promise<{ task: PlannerTask; xpAwarded: number }> => {
191 |       if (!user) throw new Error("Not authenticated");
192 |
193 |       // 1. Update task status to completed
194 |       const { data, error } = await supabase
195 |         .from("planner_tasks")
196 |         .update({
197 |           status: "done",
198 |           completed_at: new Date().toISOString(),
199 |         })
200 |         .eq("id", taskId)
201 |         .eq("student_id", user.id)
202 |         .select()
203 |         .single();
204 |
205 |       if (error) throw error;
206 |
207 |       const task = mapTask(data as Record<string, unknown>);
208 |
209 |       // 2. Award XP via award-xp Edge Function with source 'planner_task'
210 |       let xpAwarded = 0;
211 |       try {
212 |         const { data: xpResult } = await supabase.functions.invoke("award-xp", {
213 |           body: {
214 |             student_id: user.id,
215 |             xp_amount: 10,
216 |             source: "planner_task",
217 |             reference_id: taskId,
218 |           },
219 |         });
220 |         xpAwarded = xpResult?.xp_earned ?? 0;
221 |       } catch {
222 |         console.error("[useCompleteTask] award-xp invocation failed");
223 |       }
224 |
225 |       // 3. Check badges
226 |       try {
227 |         await supabase.functions.invoke("check-badges", {
228 |           body: {
229 |             student_id: user.id,
230 |             trigger: "planner_task",
231 |           },
232 |         });
233 |       } catch {
234 |         console.error("[useCompleteTask] check-badges invocation failed");
235 |       }
236 |
237 |       // 4. Award XP for any weekly goals just met (idempotent)
238 |       try {
239 |         await awardWeeklyGoalXPIfMet(user.id);
240 |       } catch {
241 |         console.error("[useCompleteTask] weekly goal XP award failed");
242 |       }
243 |
244 |       return { task, xpAwarded };
245 |     },
246 |     onMutate: async (taskId) => {
247 |       await queryClient.cancelQueries({
248 |         queryKey: queryKeys.plannerTasks.lists(),
249 |       });
250 |
251 |       const previousTasks = queryClient.getQueriesData({
252 |         queryKey: queryKeys.plannerTasks.lists(),
253 |       });
254 |
255 |       queryClient.setQueriesData(
256 |         { queryKey: queryKeys.plannerTasks.lists() },
257 |

```



```
63   });  
64 };  
65  
66 // % % % usePLO - single PLO detail % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
67  
68 export const usePLO = (id: string | undefined) => {  
69   return useQuery({  
70     queryKey: queryKeys.plos.detail(id ?? ""),  
71     queryFn: async (): Promise<LearningOutcome | null> => {  
72       const { data, error } = await supabase  
73         .from("learning_outcomes")  
74         .select("*")  
75         .eq("id", id!)  
76         .maybeSingle();  
77  
78       if (error) throw error;  
79       return data as LearningOutcome | null;  
80     },  
81     enabled: !!id,  
82   });  
83 };  
84  
85 // % % % useCreatePLO - insert into learning_outcomes % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
86  
87 export const useCreatePLO = () => {  
88   const queryClient = useQueryClient();  
89   const { user } = useAuth();  
90  
91   return useMutation({  
92     mutationFn: async (data: CreatePLOFormData): Promise<LearningOutcome> => {  
93       const { data: result, error } = await supabase  
94         .from("learning_outcomes")  
95         .insert({ ...data, type: "PLO" } as never)  
96         .select()  
97         .single();  
98  
99       if (error) throw error;  
100  
101       const plo = result as LearningOutcome;  
102  
103       await logAuditEvent({  
104         action: "create",  
105         entity_type: "plo",  
106         entity_id: plo.id,  
107         changes: data,  
108         performed_by: user?.id ?? "unknown",  
109       });  
110  
111       return plo;  
112     },  
113     onSuccess: () => {  
114       queryClient.invalidateQueries({ queryKey: queryKeys.plos.lists() });  
115     },  
116   });  
117 };  
118  
119 // % % % useUpdatePLO - update learning_outcomes % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
120  
121 export const useUpdatePLO = (id: string) => {  
122   const queryClient = useQueryClient();  
123   const { user } = useAuth();  
124  
125   return useMutation({  
126     mutationFn: async (  
127       data: Partial<CreatePLOFormData>  
128     ): Promise<LearningOutcome> => {  
129       const { data: result, error } = await supabase  
130         .from("learning_outcomes")  
131         .update(data)  
132         .eq("id", id)  
133         .select()  
134         .single();  
135  
136       if (error) throw error;  
137  
138       await logAuditEvent({  
139         action: "update",  
140         entity_type: "plo",  
141         entity_id: id,  
142         changes: data,  
143         performed_by: user?.id ?? "unknown",  
144       });  
145  
146       return result as LearningOutcome;  
147     },  
148     onSuccess: () => {  
149       queryClient.invalidateQueries({ queryKey: queryKeys.plos.lists() });  
150       queryClient.invalidateQueries({ queryKey: queryKeys.plos.detail(id) });  
151     },  
152   });  
153 };  
154  
155 // % % % useDeletePLO - hard delete with dependency check % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
156  
157 export const useDeletePLO = () => {  
158   const queryClient = useQueryClient();  
159   const { user } = useAuth();  
160  
161   return useMutation({  
162     mutationFn: async (id: string): Promise<void> => {  
163       // Check for dependent CLOs via outcome_mappings  
164       const { data: deps, error: depsError } = await supabase  
165         .from("outcome_mappings")  
166         .select("id, target_outcome_id")
```

[illegible]

```

271 | const queryClient = useQueryClient();
272 | const { user } = useAuth();
273 |
274 | return useMutation({
275 |   mutationFn: async (data: {
276 |     ploId: string;
277 |     mappings: Array<{ source_outcome_id: string; weight: number }>;
278 |   }): Promise<void> => {
279 |     // Delete existing mappings for this PLO
280 |     const { error: deleteError } = await supabase
281 |       .from("outcome_mappings")
282 |       .delete()
283 |       .eq("target_outcome_id", data.ploId);
284 |
285 |     if (deleteError) throw deleteError;
286 |
287 |     // Insert new mappings
288 |     if (data.mappings.length > 0) {
289 |       const rows = data.mappings.map((m) => ({
290 |         source_outcome_id: m.source_outcome_id,
291 |         target_outcome_id: data.ploId,
292 |         weight: m.weight,
293 |       }));
294 |
295 |       const { error: insertError } = await supabase
296 |         .from("outcome_mappings")
297 |         .insert(rows);
298 |
299 |       if (insertError) throw insertError;
300 |     }
301 |
302 |     await logAuditEvent({
303 |       action: "update",
304 |       entity_type: "plo_mappings",
305 |       entity_id: data.ploId,
306 |       changes: { mappings: data.mappings },
307 |       performed_by: user?.id ?? "unknown",
308 |     });
309 |   },
310 |   onSuccess: () => {
311 |     queryClient.invalidateQueries({
312 |       queryKey: queryKeys.outcomeMappings.lists(),
313 |     });
314 |     queryClient.invalidateQueries({ queryKey: queryKeys.plos.lists() });
315 |   },
316 | });
317 | };
318 |

```

## FILE: src/hooks/usePortfolio.ts

TYPE: .ts | LINES: 1-444 | SHA256: 1ed5329cae925f00c72bd4758aa9ff21851382aad26d584ff3ed316d220e47b7

```

1 | // Task 49: Student Learning Portfolio hooks
2 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
3 | import { supabase } from "@lib/supabase";
4 | import { queryKeys } from "@lib/queryKeys";
5 | import {
6 |   parsePortfolioAccessStatus,
7 |   type PortfolioAccessStatus,
8 | } from "@lib/portfolioAccess";
9 | import type { BloomsLevel, AttainmentLevel } from "@types/app";
10 |
11 | export interface PortfolioCLO {
12 |   clo_id: string;
13 |   clo_title: string;
14 |   blooms_level: BloomsLevel;
15 |   attainment_percent: number;
16 |   attainment_level: AttainmentLevel;
17 |   course_name: string;
18 | }
19 |
20 | export interface PortfolioBadge {
21 |   badge_key: string;
22 |   badge_name: string;
23 |   emoji: string;
24 |   awarded_at: string;
25 | }
26 |
27 | export interface PortfolioJournal {
28 |   id: string;
29 |   content_preview: string;
30 |   created_at: string;
31 |   course_name: string | null;
32 | }
33 |
34 | export interface XPTimelinePoint {
35 |   date: string;
36 |   cumulative_xp: number;
37 | }
38 |
39 | export interface SemesterAttainment {
40 |   semester_name: string;
41 |   average_attainment: number;
42 | }
43 |
44 | export interface PortfolioData {
45 |   clos: PortfolioCLO[];
46 |   badges: PortfolioBadge[];
47 |   journals: PortfolioJournal[];
48 |   xpTimeline: XPTimelinePoint[];
49 |   semesterAttainments: SemesterAttainment[];
50 |   totalXP: number;
51 |   level: number;
52 | }

```

```

53 |
54 | export interface PublicPortfolioData {
55 |   full_name: string;
56 |   badges: PortfolioBadge[];
57 |   clos: Array<{ clo_title: string; attainment_level: AttainmentLevel }>;
58 |   totalXP: number;
59 |   level: number;
60 | }
61 |
62 | function classifyAttainment(p: number): AttainmentLevel {
63 |   if (p >= 85) return "Excellent";
64 |   if (p >= 70) return "Satisfactory";
65 |   if (p >= 50) return "Developing";
66 |   return "Not_Yet";
67 | }
68 |
69 | export const usePortfolio = (studentId: string | undefined) => {
70 |   return useQuery({
71 |     queryKey: queryKeys.studentGamification.list({
72 |       scope: "portfolio",
73 |       studentId,
74 |     }),
75 |     queryFn: async (): Promise<PortfolioData> => {
76 |       if (!studentId) throw new Error("studentId required");
77 |
78 |       // CLO mastery
79 |       const { data: attainments } = await supabase
80 |         .from("outcome_attainment")
81 |         .select("outcome_id, attainment_percent, course_id")
82 |         .eq("student_id", studentId)
83 |         .eq("scope", "student_course");
84 |
85 |       const outcomeIds = (attainments ?? []).map((a) => a.outcome_id);
86 |       const courseIds = [
87 |         ...new Set(
88 |           (attainments ?? []).
89 |             .map((a) => a.course_id)
90 |             .filter((id): id is string => id != null)
91 |         ),
92 |       ];
93 |
94 |       const { data: outcomes } =
95 |         outcomeIds.length > 0
96 |         ? await supabase
97 |           .from("learning_outcomes")
98 |           .select("id, title, blooms_level, course_id")
99 |           .in("id", outcomeIds)
100 |       : { data: [] };
101 |
102 |       const { data: courses } =
103 |         courseIds.length > 0
104 |         ? await supabase
105 |           .from("courses")
106 |           .select("id, name, semester_id")
107 |           .in("id", courseIds)
108 |       : { data: [] };
109 |
110 |       const courseMap = new Map((courses ?? []).map((c) => [c.id, c]));
111 |       const outcomeMap = new Map((outcomes ?? []).map((o) => [o.id, o]));
112 |
113 |       const clos: PortfolioCLO[] = (attainments ?? []).map((a) => {
114 |         const o = outcomeMap.get(a.outcome_id);
115 |         const c = courseMap.get(a.course_id ?? "");
116 |         return {
117 |           clo_id: a.outcome_id,
118 |           clo_title: o?.title ?? "Unknown",
119 |           blooms_level: (o?.blooms_level ?? "remembering") as BloomsLevel,
120 |           attainment_percent: a.attainment_percent,
121 |           attainment_level: classifyAttainment(a.attainment_percent),
122 |           course_name: c?.name ?? "Unknown",
123 |         };
124 |       });
125 |
126 |       // Badges
127 |       const { data: badges } = await supabase
128 |         .from("badges")
129 |         .select("badge_key, badge_name, emoji, awarded_at")
130 |         .eq("student_id", studentId)
131 |         .order("awarded_at", { ascending: false });
132 |
133 |       // Journals
134 |       const { data: journalRows } = await supabase
135 |         .from("journal_entries")
136 |         .select("id, content, created_at, course_id")
137 |         .eq("student_id", studentId)
138 |         .order("created_at", { ascending: false })
139 |         .limit(20);
140 |
141 |       const journalCourseIds = [
142 |         ...new Set(
143 |           (journalRows ?? []).
144 |             .map((j) => j.course_id)
145 |             .filter((id): id is string => id != null)
146 |         ),
147 |       ];
148 |       const { data: jCourses } =
149 |         journalCourseIds.length > 0
150 |         ? await supabase
151 |           .from("courses")
152 |           .select("id, name")
153 |           .in("id", journalCourseIds)
154 |       : { data: [] };
155 |       const jCourseMap = new Map((jCourses ?? []).map((c) => [c.id, c.name]));
156 |

```

```

157 |     const journals: PortfolioJournal[] = (journalRows ?? []).map((j) => ({
158 |       id: j.id,
159 |       content_preview: j.content
160 |         ? j.content.slice(0, 80) + (j.content.length > 80 ? "..." : "")
161 |         : "",
162 |       created_at: j.created_at,
163 |       course_name: jCourseMap.get(j.course_id) ?? null,
164 |     }));
165 |
166 |     // XP timeline
167 |     const { data: xpRows } = await supabase
168 |       .from("xp_transactions")
169 |       .select("xp_amount, created_at")
170 |       .eq("student_id", studentId)
171 |       .order("created_at", { ascending: true });
172 |
173 |     let cumulative = 0;
174 |     const dateMap = new Map<string, number>();
175 |     for (const row of xpRows ?? []) {
176 |       cumulative += row.xp_amount;
177 |       const date = (row.created_at as string).slice(0, 10);
178 |       dateMap.set(date, cumulative);
179 |     }
180 |     const xpTimeline: XPTimelinePoint[] = Array.from(dateMap.entries()).map(
181 |       ([date, xp]) => ({
182 |         date,
183 |         cumulative_xp: xp,
184 |       })
185 |     );
186 |
187 |     // Semester-over-semester attainment averages
188 |     const semesterIds = [
189 |       ...new Set(
190 |         (courses ?? []).
191 |           .map((c) => c.semester_id)
192 |           .filter((id): id is string => id != null)
193 |       ),
194 |     ];
195 |     const { data: semesters } =
196 |       semesterIds.length > 0
197 |         ? await supabase
198 |           .from("semesters")
199 |           .select("id, name")
200 |           .in("id", semesterIds)
201 |         : { data: [] };
202 |     const semesterMap = new Map<((semesters ?? []).map((s) => [s.id, s.name]));
203 |
204 |     const semesterTotals = new Map<
205 |       string,
206 |       { sum: number; count: number; name: string }
207 |     >();
208 |     for (const a of attainments ?? []) {
209 |       const c = courseMap.get(a.course_id ?? "");
210 |       const semId = c?.semester_id;
211 |       if (!semId) continue;
212 |       const existing = semesterTotals.get(semId) ?? {
213 |         sum: 0,
214 |         count: 0,
215 |         name: semesterMap.get(semId) ?? semId,
216 |       };
217 |       existing.sum += a.attainment_percent;
218 |       existing.count += 1;
219 |       semesterTotals.set(semId, existing);
220 |     }
221 |     const semesterAttainments: SemesterAttainment[] = Array.from(
222 |       semesterTotals.entries()
223 |     ).map(([_, v]) => ({
224 |       semester_name: v.name,
225 |       average_attainment: Math.round(v.sum / v.count),
226 |     }));
227 |
228 |     // Gamification
229 |     const { data: gam } = await supabase
230 |       .from("student_gamification")
231 |       .select("xp_total, level")
232 |       .eq("student_id", studentId)
233 |       .maybeSingle();
234 |
235 |     return {
236 |       clos,
237 |       badges: (badges ?? []) as PortfolioBadge[],
238 |       journals,
239 |       xpTimeline,
240 |       semesterAttainments,
241 |       totalXP: gam?.xp_total ?? 0,
242 |       level: gam?.level ?? 1,
243 |     };
244 |   },
245 |   enabled: !!studentId,
246 | });
247 | };
248 |
249 | /**
250 |  * Reads the current user's admin-granted `portfolio_sharing_permitted` flag.
251 |  *
252 |  * This permission is granted only by an administrator (the privileged-column
253 |  * preservation trigger blocks non-admin self-grants), so a student cannot turn
254 |  * it on themselves. Public portfolio sharing is gated on this flag.
255 |  */
256 | export const usePortfolioSharingPermission = (userId: string | undefined) => {
257 |   return useQuery({
258 |     queryKey: queryKeys.profiles.list({
259 |       scope: "sharing-permission",
260 |       userId,

```



```

261 |     }),
262 |     queryFn: async (): Promise<boolean> => {
263 |         if (!userId) return false;
264 |         const { data, error } = await supabase
265 |             .from("profiles")
266 |             .select("portfolio_sharing_permitted")
267 |             .eq("id", userId)
268 |             .maybeSingle();
269 |         if (error) throw error;
270 |         return data?.portfolio_sharing_permitted ?? false;
271 |     },
272 |     enabled: !!userId,
273 | });
274 | };
275 |
276 | export interface TogglePortfolioPublicVariables {
277 |     userId: string;
278 |     isPublic: boolean;
279 | }
280 |
281 | export interface TogglePortfolioPublicResult {
282 |     /** Whether the user has admin-granted permission to share publicly. */
283 |     permitted: boolean;
284 |     /** The `portfolio_public` value actually persisted after the toggle. */
285 |     isPublic: boolean;
286 | }
287 |
288 | /**
289 |  * Toggle `portfolio_public` on the profiles table.
290 |  *
291 |  * Enabling public sharing is only effective when the user has been granted
292 |  * `portfolio_sharing_permitted` by an administrator. When permission is absent,
293 |  * the portfolio is kept private (persisted as `false`) and the result reports
294 |  * `permitted: false` so the UI can explain that school permission is required.
295 |  * Disabling public sharing is always honored.
296 |  */
297 | export const useTogglePortfolioPublic = () => {
298 |     const queryClient = useQueryClient();
299 |     return useMutation<
300 |         TogglePortfolioPublicResult,
301 |         Error,
302 |         TogglePortfolioPublicVariables
303 |     >({
304 |         mutationFn: async ({ userId, isPublic }) => {
305 |             // Re-check the admin-granted permission at the data layer so the toggle is
306 |             // never the sole gatekeeper. The flag cannot be self-granted (protected by
307 |             // the privileged-column preservation trigger).
308 |             const { data: profile, error: readError } = await supabase
309 |                 .from("profiles")
310 |                 .select("portfolio_sharing_permitted")
311 |                 .eq("id", userId)
312 |                 .maybeSingle();
313 |             if (readError) throw readError;
314 |
315 |             const permitted = profile?.portfolio_sharing_permitted ?? false;
316 |             // Public sharing only takes effect with permission; otherwise stay private.
317 |             const effectivePublic = isPublic && permitted;
318 |
319 |             const { error } = await supabase
320 |                 .from("profiles")
321 |                 .update({ portfolio_public: effectivePublic })
322 |                 .eq("id", userId);
323 |             if (error) throw error;
324 |
325 |             return { permitted, isPublic: effectivePublic };
326 |         },
327 |         onSuccess: () => {
328 |             queryClient.invalidateQueries({ queryKey: queryKeys.profiles.all });
329 |             queryClient.invalidateQueries({
330 |                 queryKey: queryKeys.studentGamification.lists(),
331 |             });
332 |         },
333 |     });
334 | };
335 |
336 | /**
337 |  * Public-portfolio access discriminator (R24.3 / R24.3a / R24.4).
338 |  *
339 |  * Calls the `portfolio_public_access` SECURITY DEFINER RPC, which returns an
340 |  * authorization token ( `authorized` | `forbidden` | `not_found` ) WITHOUT leaking
341 |  * any portfolio content. The route maps this status to render / 403 / 404 via
342 |  * `mapPortfolioAccessToRoute`, so an existing-but-unauthorized portfolio is a
343 |  * 403 (never collapsed into a 404) and a missing one is a 404.
344 |  *
345 |  * The actual portfolio content is fetched separately and always runs under RLS
346 |  * (see `usePublicPortfolio`), so unauthorized content is never returned even if
347 |  * this discriminator were bypassed. The raw RPC value crosses the network as
348 |  * `text`; `parsePortfolioAccessStatus` narrows it conservatively (any
349 |  * unexpected value becomes `forbidden`, which maps to 403 and never renders).
350 |  */
351 | export const usePortfolioAccess = (studentId: string | undefined) => {
352 |     return useQuery({
353 |         queryKey: queryKeys.studentGamification.list({
354 |             scope: "public-portfolio-access",
355 |             studentId,
356 |         }),
357 |         queryFn: async (): Promise<PortfolioAccessStatus> => {
358 |             if (!studentId) return "not_found";
359 |             const { data, error } = await supabase.rpc("portfolio_public_access", {
360 |                 p_student_id: studentId,
361 |             });
362 |             if (error) throw error;
363 |             return parsePortfolioAccessStatus(data);
364 |         },

```

```

365     enabled: !!studentId,
366   });
367 };
368
369 /**
370  * Public portfolio query – no auth required, reads only non-sensitive data.
371  *
372  * Gated by `enabled` so the content query only runs once the access
373  * discriminator has authorized the request; content nonetheless always runs
374  * under RLS as defense-in-depth.
375  */
376 export const usePublicPortfolio = (
377   studentId: string | undefined,
378   options?: { enabled?: boolean }
379 ) => {
380   return useQuery({
381     queryKey: queryKeys.studentGamification.list({
382       scope: "public-portfolio",
383       studentId,
384     }),
385     queryFn: async (): Promise<PublicPortfolioData | null> => {
386       if (!studentId) return null;
387
388       // Check if portfolio is public
389       const { data: profile } = await supabase
390         .from("profiles")
391         .select("full_name, portfolio_public")
392         .eq("id", studentId)
393         .maybeSingle();
394
395       if (!profile?.portfolio_public) return null;
396
397       // Badges
398       const { data: badges } = await supabase
399         .from("badges")
400         .select("badge_key, badge_name, emoji, awarded_at")
401         .eq("student_id", studentId)
402         .order("awarded_at", { ascending: false });
403
404       // CLO attainment levels (non-sensitive: just titles + levels, no percentages)
405       const { data: attainments } = await supabase
406         .from("outcome_attainment")
407         .select("outcome_id, attainment_percent")
408         .eq("student_id", studentId)
409         .eq("scope", "student_course");
410
411       const outcomeIds = (attainments ?? []).map((a) => a.outcome_id);
412       const { data: outcomes } =
413         outcomeIds.length > 0
414           ? await supabase
415             .from("learning_outcomes")
416             .select("id, title")
417             .in("id", outcomeIds)
418             : { data: [] };
419       const outcomeMap = new Map((outcomes ?? []).map((o) => [o.id, o.title]));
420
421       const clos = (attainments ?? []).map((a) => ({
422         clo_title: outcomeMap.get(a.outcome_id) ?? "Unknown",
423         attainment_level: classifyAttainment(a.attainment_percent),
424       }));
425
426       // Gamification totals
427       const { data: gam } = await supabase
428         .from("student_gamification")
429         .select("xp_total, level")
430         .eq("student_id", studentId)
431         .maybeSingle();
432
433       return {
434         full_name: profile.full_name ?? "",
435         badges: (badges ?? []) as PortfolioBadge[],
436         clos,
437         totalXP: gam?.xp_total ?? 0,
438         level: gam?.level ?? 1,
439       };
440     },
441     enabled: !!studentId && (options?.enabled ?? true),
442   });
443 };
444

```

FILE: src/hooks/usePracticeMode.ts

```
TYPE: .ts | LINES: 1-122 | SHA256: 34b4a1368655040c1e4077685d71e52d50b723a840f67cf2afd0f7877deb36f5
```

[illegible]

```

21 |   score: number | null;
22 |   question_sequence: unknown[];
23 |   difficulty_trajectory: unknown[];
24 |   per_question_times: unknown[];
25 |   created_at: string;
26 | }
27 |
28 | // % % % usePracticeModeConfig - fetch practice_mode_enabled for a quiz % % % % % % %
29 | // Note: practice_mode_enabled column added via migration (task 16.1).
30 | // Types will be regenerated in task 16.11; until then we cast through unknown.
31 |
32 | export const usePracticeModeConfig = (quizId: string) => {
33 |   return useQuery({
34 |     queryKey: queryKeys.practiceMode.config(quizId),
35 |     queryFn: async (): Promise<PracticeModeConfig | null> => {
36 |       const { data, error } = await supabase
37 |         .from("quizzes")
38 |         .select("*")
39 |         .eq("id", quizId)
40 |         .maybeSingle();
41 |
42 |       if (error) throw error;
43 |       if (!data) return null;
44 |
45 |       const row = data as unknown as Record<string, unknown>;
46 |       return {
47 |         practice_mode_enabled: row.practice_mode_enabled as boolean,
48 |       };
49 |     },
50 |     enabled: !!quizId,
51 |   });
52 | };
53 |
54 | // % % % useTogglePracticeMode - update practice_mode_enabled on a quiz % % % % % % %
55 |
56 | export const useTogglePracticeMode = () => {
57 |   const queryClient = useQueryClient();
58 |
59 |   return useMutation({
60 |     mutationFn: async (input: TogglePracticeModeInput) => {
61 |       // practice_mode_enabled column not present in current generated types;
62 |       // cast through `never` until the schema migration is applied upstream.
63 |       const updatePayload = {
64 |         practice_mode_enabled: input.practice_mode_enabled,
65 |       } as never;
66 |
67 |       const { data, error } = await supabase
68 |         .from("quizzes")
69 |         .update(updatePayload)
70 |         .eq("id", input.quiz_id)
71 |         .select()
72 |         .single();
73 |
74 |       if (error) throw error;
75 |       return data;
76 |     },
77 |     onSuccess: (_data, variables) => {
78 |       queryClient.invalidateQueries({
79 |         queryKey: queryKeys.practiceMode.config(variables.quiz_id),
80 |       });
81 |       queryClient.invalidateQueries({
82 |         queryKey: queryKeys.quizzes.all,
83 |       });
84 |     },
85 |   });
86 | };
87 |
88 | // % % % usePracticeAttempts - fetch practice-mode attempts for a quiz+student % %
89 | // Note: mode column added via migration (task 16.1).
90 |
91 | export const usePracticeAttempts = (quizId: string, studentId: string) => {
92 |   return useQuery({
93 |     queryKey: queryKeys.practiceMode.attempts(quizId, studentId),
94 |     queryFn: async (): Promise<PracticeAttempt[]> => {
95 |       const { data, error } = await supabase
96 |         .from("quiz_attempts")
97 |         .select("*")
98 |         .eq("quiz_id", quizId)
99 |         .eq("student_id", studentId)
100 |         .order("started_at", { ascending: false });
101 |
102 |       if (error) throw error;
103 |
104 |       const rows = (data ?? []) as unknown as Record<string, unknown>[];
105 |       return rows
106 |         .filter((r) => r.mode === "practice")
107 |         .map((r) => ({
108 |           id: r.id as string,
109 |           quiz_id: r.quiz_id as string,
110 |           student_id: r.student_id as string,
111 |           mode: r.mode as string,
112 |           score: r.score as number | null,
113 |           question_sequence: (r.question_sequence ?? []) as unknown[],
114 |           difficulty_trajectory: (r.difficulty_trajectory ?? []) as unknown[],
115 |           per_question_times: (r.per_question_times ?? []) as unknown[],
116 |           created_at: r.created_at as string,
117 |         }));
118 |     },
119 |     enabled: !!quizId && !!studentId,
120 |   });
121 | };
122 |

```

FILE: src/hooks/useProfileCompleteness.ts

```
TYPE: .ts | LINES: 1-52 | SHA256: 69a2f2c30fafdf0b87ab5e90831b65e774f5859b8aa710b61e3ab82b45474dfd
```

[illegible]

FILE: src/hooks/usePrograms.ts

TYPE: .ts | LINES: 1-195 | SHA256: 5a7f7762dfdc4d0edb23f60f8f47bd10f4866a1da37f6d9535bba6f59da0f7fc

[illegible]

[illegible]

[illegible]

FILE: src/hooks/usePurchase.ts

TYPE: .ts | LINES: 1-95 | SHA256: 0996f097c668480c842bfce2df7b20a3266244cdc80d938da1f36f7e42978d10

[illegible]

```

54 |         if (!response.ok || !result.success) {
55 |             const errorCode = result.error_code ?? "UNKNOWN";
56 |             const errorMessage =
57 |                 ERROR_MESSAGES[errorCode] ??
58 |                 result.error ??
59 |                 "Purchase failed. Please try again.";
60 |             throw new Error(errorMessage);
61 |         }
62 |     }
63 |
64 |     return result as PurchaseResponse;
65 | },
66 | onSuccess: () => {
67 |     // Invalidate balance, inventory, and items queries
68 |     if (user?.id) {
69 |         queryClient.invalidateQueries({
70 |             queryKey: queryKeys.marketplace.balance(user.id),
71 |         });
72 |         queryClient.invalidateQueries({
73 |             queryKey: queryKeys.marketplace.inventory(user.id),
74 |         });
75 |         queryClient.invalidateQueries({
76 |             queryKey: queryKeys.marketplace.boosts(user.id),
77 |         });
78 |         queryClient.invalidateQueries({
79 |             queryKey: queryKeys.marketplace.equipped(user.id),
80 |         });
81 |     }
82 |     queryClient.invalidateQueries({
83 |         queryKey: queryKeys.marketplace.items(),
84 |     });
85 |     queryClient.invalidateQueries({
86 |         queryKey: queryKeys.marketplace.transactions(),
87 |     });
88 |     toast.success("Purchase successful!");
89 | },
90 | onError: (error: Error) => {
91 |     toast.error(error.message);
92 | },
93 | });
94 | };
95 |

```

FILE: src/hooks/useQuestionAnalytics.ts

TYPE: .ts | LINES: 1-92 | SHA256: 32bfeb144acd47b15e3d24cf7de7c96959b75169768f5910c98befac5ccaf343

[illegible]

```

59 |         if (filters.bloom_level !== undefined) {
60 |             query = query.eq("question_bank.bloom_level", filters.bloom_level);
61 |         }
62 |
63 |         const { data, error } = await query;
64 |
65 |         if (error) throw error;
66 |         return (data ?? []) as QuestionAnalyticsRow[];
67 |     },
68 |     enabled: !!courseId,
69 | });
70 | };
71 |
72 | // % % % useQuestionDetail - single question analytics with joined data % % % % % % %
73 |
74 | export const useQuestionDetail = (questionId: string) => {
75 |     return useQuery({
76 |         queryKey: queryKeys.questionAnalytics.detail(questionId),
77 |         queryFn: async (): Promise<QuestionAnalyticsRow | null> => {
78 |             const { data, error } = await supabase
79 |                 .from("question_analytics")
80 |                 .select(
81 |                     `*, question_bank!inner(id, course_id, clo_id, bloom_level, question_type, question_text, difficulty
82 |                 )
83 |                 .eq("question_id", questionId)
84 |                 .maybeSingle();
85 |
86 |             if (error) throw error;
87 |             return data as QuestionAnalyticsRow | null;
88 |         },
89 |         enabled: !!questionId,
90 |     });
91 | };
92 |

```

FILE: src/hooks/useQuestionBank.ts

```
TYPE: .ts | LINES: 1-171 | SHA256: 8c624a31b7386abfe5539258aecb85fde196389e557fe150ddcfe57609171824
```

[illegible]



```

67 |   status?: string;
68 |   generation_source?: string;
69 |   labels?: string[];
70 |   bloom_level?: number;
71 |   clo_id?: string;
72 | }
73 |
74 | // % % % useQuestionBank - list questions with optional filters % % % % % % % % % % % % % % % %
75 |
76 | export const useQuestionBank = (
77 |   courseId: string,
78 |   filters: QuestionBankFilters = {}
79 | ) => {
80 |   return useQuery({
81 |     queryKey: queryKeys.questionBank.list({ courseId, ...filters }),
82 |     queryFn: async (): Promise<QuestionBankRow[]> => {
83 |       let query = supabase
84 |         .from("question_bank")
85 |         .select("*")
86 |         .eq("course_id", courseId)
87 |         .order("created_at", { ascending: false });
88 |
89 |       if (filters.clo_id) {
90 |         query = query.eq("clo_id", filters.clo_id);
91 |       }
92 |       if (filters.bloom_level !== undefined) {
93 |         query = query.eq("bloom_level", filters.bloom_level);
94 |       }
95 |       if (filters.question_type) {
96 |         query = query.eq("question_type", filters.question_type);
97 |       }
98 |       if (filters.status) {
99 |         query = query.eq("status", filters.status);
100 |      }
101 |      if (filters.generation_source) {
102 |        query = query.eq("generation_source", filters.generation_source);
103 |      }
104 |      if (filters.search) {
105 |        query = query.ilike("question_text", `${filters.search}%`);
106 |      }
107 |
108 |      const { data, error } = await query;
109 |
110 |      if (error) throw error;
111 |      return (data ?? []) as QuestionBankRow[];
112 |    },
113 |    enabled: !!courseId,
114 |  });
115 | };
116 |
117 | // % % % useCreateQuestion - insert a new question % % % % % % % % % % % % % % % % % % % % % % % % % % % %
118 |
119 | export const useCreateQuestion = () => {
120 |   const queryClient = useQueryClient();
121 |
122 |   return useMutation({
123 |     mutationFn: async (
124 |       input: CreateQuestionInput
125 |     ): Promise<QuestionBankRow> => {
126 |       const { data, error } = await supabase
127 |         .from("question_bank")
128 |         .insert(input)
129 |         .select()
130 |         .single();
131 |
132 |       if (error) throw error;
133 |       return data as QuestionBankRow;
134 |     },
135 |     onSuccess: () => {
136 |       queryClient.invalidateQueries({
137 |         queryKey: queryKeys.questionBank.lists(),
138 |       });
139 |     },
140 |   });
141 | };
142 |
143 | // % % % useUpdateQuestion - update an existing question by id % % % % % % % % % % % % % % % % % % % % % %
144 |
145 | export const useUpdateQuestion = () => {
146 |   const queryClient = useQueryClient();
147 |
148 |   return useMutation({
149 |     mutationFn: async (
150 |       input: UpdateQuestionInput
151 |     ): Promise<QuestionBankRow> => {
152 |       const { id, ...fields } = input;
153 |
154 |       const { data, error } = await supabase
155 |         .from("question_bank")
156 |         .update(fields)
157 |         .eq("id", id)
158 |         .select()
159 |         .single();
160 |
161 |       if (error) throw error;
162 |       return data as QuestionBankRow;
163 |     },
164 |     onSuccess: () => {
165 |       queryClient.invalidateQueries({
166 |         queryKey: queryKeys.questionBank.lists(),
167 |       });
168 |     },
169 |   });
170 | };

```

```
TYPE: .ts | LINES: 1-62 | SHA256: 0a60156a6c3bb22663bc66fbbd1b3947c0db2bb69a73a2f46da63006b93e3351
```

```

1 //
2 // useQuizRecoveryBlock - active mastery-recovery block for a quiz
3 //
4 // Relocated from the in-page `useQuery` in `AdaptiveQuizSession.tsx`. Resolves
5 // the quiz's CLO ids and checks whether the student has an active mastery
6 // recovery pathway on any of them; if so, the quiz is blocked until recovery is
7 // complete. Returns the first matching active recovery, or `null`.
8 //
9 // Keeps all Supabase access in `src/hooks/` and uses the project's standard
10 // TanStack query conventions (query keys + typed responses).
11 //
12 // _Requirements: 25.1, 25.2, 25.3, 25.3a, 25.4_
13 // =====
14
15 import { useQuery } from "@tanstack/react-query";
16 import { supabase } from "@lib/supabase";
17 import { queryKeys } from "@lib/queryKeys";
18
19 export interface QuizRecoveryBlock {
20   id: string;
21   clo_id: string;
22   course_id: string;
23   status: string;
24 }
25
26 export const useQuizRecoveryBlock = (
27   quizId: string | undefined,
28   studentId: string | undefined
29 ) => {
30   return useQuery({
31     queryKey: [...queryKeys.masteryRecovery.all, "quiz-block", quizId],
32     queryFn: async () => Promise<QuizRecoveryBlock | null> => {
33       if (!studentId || !quizId) return null;
34
35       // Resolve the quiz's CLO ids.

```





[illegible]

[illegible]

[illegible]

[illegible]



```

473 |     },
474 |     onSuccess: (_data, variables) => {
475 |       queryClient.invalidateQueries({
476 |         queryKey: queryKeys.quizAttempts.list({ quizId: variables.quiz_id }),
477 |       });
478 |     },
479 |   });
480 | };
481 |

```

# FILE: src/hooks/useReadHabitTimer.ts

TYPE: .ts | LINES: 1-162 | SHA256: 6cad14a7f5cb963d852e602608486cba51e8d52e4c63d0b49b6f15d7dbf10609

```

1 | // Task 52: Read Habit Timer – tracks 30s of reading to mark 'read' habit
2 | // Requirements: 61.1, 61.2, 61.4
3 | import { useEffect, useRef, useState } from "react";
4 | import { useMutation, useQueryClient } from "@tanstack/react-query";
5 | import { useAuth } from "@/hooks/useAuth";
6 | import { supabase } from "@/lib/supabase";
7 | import { queryKeys } from "@/lib/queryKeys";
8 | import { logActivity } from "@/lib/activityLogger";
9 |
10 | const READ_THRESHOLD_SECONDS = 30;
11 | const TICK_INTERVAL_MS = 1_000;
12 |
13 | export interface UseReadHabitTimerOptions {
14 |   pageType: "assignment_detail" | "clo_progress";
15 |   pageId: string;
16 | }
17 |
18 | export interface UseReadHabitTimerReturn {
19 |   elapsedSeconds: number;
20 |   isCompleted: boolean;
21 | }
22 |
23 | /**
24 |  * Compute YYYY-MM-DD from the current UTC calendar day. The canonical
25 |  * `habit_logs` table keys days by midnight-UTC (matching `perfectDay.ts` and
26 |  * `useSessionCompletion`), so the read habit must use the same convention for
27 |  * all 4 daily habits to land on the same date key.
28 |  */
29 | function getUtcDateString(): string {
30 |   return new Date().toISOString().split("T")[0] as string;
31 | }
32 |
33 | /**
34 |  * Starts a 1-second interval on mount. When cumulative view time reaches 30 s
35 |  * for the current UTC day, upserts a `habit_logs` record with
36 |  * `habit_type: 'read'` via a TanStack mutation and logs an activity event
37 |  * with `duration_seconds`. `habit_logs` is the single canonical academic-habit
38 |  * table that feeds the heatmap, streak, perfect-day, and `perfect_week` badge.
39 |  *
40 |  * On unmount the partial duration is logged as an activity event so no
41 |  * viewing time is lost.
42 |  */
43 | export const useReadHabitTimer = (
44 |   options: UseReadHabitTimerOptions
45 | ): UseReadHabitTimerReturn => {
46 |   const { profile } = useAuth();
47 |   const queryClient = useQueryClient();
48 |   const [elapsedSeconds, setElapsedSeconds] = useState(0);
49 |   const [isCompleted, setIsCompleted] = useState(false);
50 |
51 |   const completedRef = useRef(false);
52 |   const elapsedRef = useRef(0);
53 |   const optionsRef = useRef(options);
54 |   const profileRef = useRef(profile);
55 |
56 |   // Keep refs in sync via effects to avoid render-time ref writes
57 |   useEffect(() => {
58 |     optionsRef.current = options;
59 |   }, [options]);
60 |
61 |   useEffect(() => {
62 |     profileRef.current = profile;
63 |   }, [profile]);
64 |
65 |   const upsertMutation = useMutation({
66 |     mutationFn: async (params: { student_id: string; habit_date: string }) => {
67 |       const { error } = await supabase.from("habit_logs").upsert(
68 |         {
69 |           student_id: params.student_id,
70 |           habit_type: "read",
71 |           date: params.habit_date,
72 |           completed_at: new Date().toISOString(),
73 |         },
74 |         { onConflict: "student_id,habit_type,date" }
75 |       );
76 |       if (error) throw error;
77 |     },
78 |     onSuccess: () => {
79 |       const studentId = profileRef.current?.id;
80 |       if (studentId) {
81 |         queryClient.invalidateQueries({ queryKey: queryKeys.habitLogs.all });
82 |       }
83 |     },
84 |     onError: (error) => {
85 |       console.error(
86 |         "[useReadHabitTimer] Failed to upsert habit_logs:",
87 |         error instanceof Error ? error.message : error
88 |       );
89 |     },
90 |   });
91 |

```

```

92 | const upsertRef = useRef(upsertMutation);
93 | useEffect(() => {
94 |   upsertRef.current = upsertMutation;
95 | }, [upsertMutation]);
96 |
97 | useEffect(() => {
98 |   if (!profile?.id || profile.role !== "student") return;
99 |
100 |   // Reset refs on mount
101 |   elapsedRef.current = 0;
102 |   completedRef.current = false;
103 |
104 |   const timer = setInterval(() => {
105 |     elapsedRef.current += 1;
106 |     setElapsedSeconds(elapsedRef.current);
107 |
108 |     if (
109 |       !completedRef.current &&
110 |       elapsedRef.current >= READ_THRESHOLD_SECONDS
111 |     ) {
112 |       completedRef.current = true;
113 |       setIsCompleted(true);
114 |
115 |       const studentId = profileRef.current?.id;
116 |       if (!studentId) return;
117 |
118 |       const todayStr = getUtcDateString();
119 |
120 |       // Mark 'read' habit for today in the canonical habit_logs table
121 |       upsertRef.current.mutate({
122 |         student_id: studentId,
123 |         habit_date: todayStr,
124 |       });
125 |
126 |       // Log activity with duration_seconds metadata
127 |       logActivity({
128 |         student_id: studentId,
129 |         event_type: "page_view",
130 |         metadata: {
131 |           page_type: optionsRef.current.pageType,
132 |           page_id: optionsRef.current.pageId,
133 |           duration_seconds: elapsedRef.current,
134 |           habit_completed: true,
135 |         },
136 |       });
137 |     }
138 |   }, TICK_INTERVAL_MS);
139 |
140 |   return () => {
141 |     clearInterval(timer);
142 |
143 |     // Log partial duration on unmount if habit wasn't completed
144 |     const studentId = profileRef.current?.id;
145 |     if (studentId && !completedRef.current && elapsedRef.current > 0) {
146 |       logActivity({
147 |         student_id: studentId,
148 |         event_type: "page_view",
149 |         metadata: {
150 |           page_type: optionsRef.current.pageType,
151 |           page_id: optionsRef.current.pageId,
152 |           duration_seconds: elapsedRef.current,
153 |           habit_completed: false,
154 |         },
155 |       });
156 |     }
157 |   };
158 | }, [profile?.id, profile?.role]);
159 |
160 | return { elapsedSeconds, isCompleted };
161 | };
162 |

```

## FILE: src/hooks/useRealtime.ts

TYPE: .ts | LINES: 1-198 | SHA256: ddb2336e6b376a32adfc2297a5eab230986e3a2f43e510c8673209147eab118b

```

1 | // =====
2 | // useRealtime - Shared realtime subscription manager
3 | // Provides channel deduplication, exponential backoff reconnection,
4 | // polling fallback, and "Live updates paused" state.
5 | // Validates: Requirements 2.10
6 | // =====
7 |
8 | import { useEffect, useRef, useState, useCallback } from "react";
9 | import { supabase } from "@lib/supabase";
10 | import type {
11 |   RealtimeChannel,
12 |   RealtimePostgresChangesPayload,
13 | } from "@supabase/supabase-js";
14 |
15 | interface RealtimeOptions {
16 |   /** Postgres table to subscribe to */
17 |   table: string;
18 |   /** Event type filter (default: '*') */
19 |   event?: "INSERT" | "UPDATE" | "DELETE" | "";
20 |   /** PostgREST-style filter (e.g. `institution_id=eq.abc`) */
21 |   filter?: string;
22 |   /** Callback invoked on each realtime payload */
23 |   onPayload: (
24 |     payload: RealtimePostgresChangesPayload<Record<string, unknown>>
25 |   ) => void;
26 |   /** Optional polling function called as fallback when realtime is unavailable */
27 |   pollingFn?: () => void;
28 |   /** Polling interval in ms (default: 30 000) */
29 |   pollingInterval?: number;

```

```

30 | /** When false, skip the realtime subscription entirely (default: true) */
31 | enabled?: boolean;
32 | /**
33 |  * Update strategy (default: "realtime"):
34 |  * - "realtime": subscribe to `postgres_changes` for this table (low latency,
35 |  *   but every change on the published table is WAL-decoded and delivered per
36 |  *   subscriber – this does NOT scale for broad, institution- or global-scoped
37 |  *   data because it fans out to every viewer).
38 |  * - "poll": skip the subscription entirely and refresh via `pollingFn` on a
39 |  *   fixed `pollingInterval`. Use this for inherently broad data (e.g. an
40 |  *   institution-wide leaderboard, an all-courses grading queue) where a
41 |  *   tightly-scoped realtime filter is impossible. Cost is bounded and scales
42 |  *   linearly per client instead of as O(writes × viewers).
43 |  */
44 | strategy?: "realtime" | "poll";
45 | }
46 |
47 | /**
48 |  * Shared realtime subscription hook with:
49 |  * - Channel deduplication (one channel per table+event+filter combo)
50 |  * - Reconnection with exponential backoff (1s !' 2s !' 4s !' 8s !' max 30s)
51 |  * - Fallback to polling (30s refetchInterval) on connection failure
52 |  * - `isLive` state exposed to consumers for "Live updates paused" banner
53 |  * - Full cleanup on unmount
54 |  */
55 | export const useRealtime = (
56 |   options: RealtimeOptions
57 | ): { isLive: boolean; retryCount: number } => {
58 |   const {
59 |     table,
60 |     event = "*",
61 |     filter,
62 |     onPayload,
63 |     pollingFn,
64 |     pollingInterval = 30_000,
65 |     enabled = true,
66 |     strategy = "realtime",
67 |   } = options;
68 |
69 |   const [isLive, setIsLive] = useState(true);
70 |   const [retryCount, setRetryCount] = useState(0);
71 |   const channelRef = useRef<RealtimeChannel | null>(null);
72 |   const retryCountRef = useRef(0);
73 |   const retryTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
74 |   const pollingTimerRef = useRef<ReturnType<typeof setInterval> | null>(null);
75 |
76 |   // Store latest callbacks in refs to avoid stale closures
77 |   const onPayloadRef = useRef(onPayload);
78 |   const pollingFnRef = useRef(pollingFn);
79 |
80 |   useEffect(() => {
81 |     onPayloadRef.current = onPayload;
82 |   }, [onPayload]);
83 |
84 |   useEffect(() => {
85 |     pollingFnRef.current = pollingFn;
86 |   }, [pollingFn]);
87 |
88 |   const startPolling = useCallback(() => {
89 |     if (pollingTimerRef.current) return;
90 |     const fn = pollingFnRef.current;
91 |     if (fn) {
92 |       fn(); // Immediate fetch
93 |       pollingTimerRef.current = setInterval(() => fn(), pollingInterval);
94 |     }
95 |     setIsLive(false);
96 |   }, [pollingInterval]);
97 |
98 |   const stopPolling = useCallback(() => {
99 |     if (pollingTimerRef.current) {
100 |       clearInterval(pollingTimerRef.current);
101 |       pollingTimerRef.current = null;
102 |     }
103 |     setIsLive(true);
104 |   }, []);
105 |
106 |   useEffect(() => {
107 |     // Skip subscription when disabled (e.g. required filter value not yet resolved)
108 |     if (!enabled) return;
109 |
110 |     // Polling strategy: deliberately do NOT open a postgres_changes channel.
111 |     // For institution-/global-scoped data a realtime subscription cannot be
112 |     // filtered down and would WAL-fan-out to every viewer (the single largest
113 |     // database CPU consumer at scale). Refresh on a fixed interval instead. The
114 |     // consuming query already fetches on mount, so we only schedule subsequent
115 |     // refreshes here. isLive stays true (polling has no "disconnected" state),
116 |     // so the consuming page's reconnect banner correctly shows nothing.
117 |     if (strategy === "poll") {
118 |       // No setState here: isLive keeps its initial `true` and retryCount its
119 |       // initial `0` (polling has no "disconnected" state, so the page's
120 |       // reconnect banner correctly shows nothing). `strategy` is static per
121 |       // mount, so the initial values are always correct – avoiding a
122 |       // setState() in the effect body (react-hooks/set-state-in-effect).
123 |       const fn = pollingFnRef.current;
124 |       if (!fn) return;
125 |       const id = setInterval(() => fn(), pollingInterval);
126 |       pollingTimerRef.current = id;
127 |       return () => {
128 |         clearInterval(id);
129 |         pollingTimerRef.current = null;
130 |       };
131 |     }
132 |
133 |     // Deduplicate: one channel per table+event+filter combo

```

```

134 const channelName = `${table}:${event}:${filter} ?? "all"`;
135
136 const doSubscribe = () => {
137   const channel = supabase
138     .channel(channelName)
139     .on(
140       "postgres_changes",
141       { event, schema: "public", table, filter },
142       (payload) => {
143         onPayloadRef.current(payload);
144       }
145     )
146     .subscribe((status) => {
147       if (status === "SUBSCRIBED") {
148         retryCountRef.current = 0;
149         setRetryCount(0);
150         stopPolling();
151       } else if (status === "CHANNEL_ERROR" || status === "TIMED_OUT") {
152         // Exponential backoff: 1s, 2s, 4s, 8s, ... max 30s
153         const delay = Math.min(
154           1000 * Math.pow(2, retryCountRef.current),
155           30_000
156         );
157         retryCountRef.current += 1;
158         setRetryCount(retryCountRef.current);
159         startPolling();
160
161         if (retryTimerRef.current) clearTimeout(retryTimerRef.current);
162         retryTimerRef.current = setTimeout(() => {
163           channel.unsubscribe();
164           doSubscribe();
165         }, delay);
166       }
167     });
168
169   channelRef.current = channel;
170 };
171
172 doSubscribe();
173
174 return () => {
175   if (channelRef.current) {
176     channelRef.current.unsubscribe();
177     channelRef.current = null;
178   }
179   if (retryTimerRef.current) {
180     clearTimeout(retryTimerRef.current);
181     retryTimerRef.current = null;
182   }
183   stopPolling();
184 };
185 }, [
186   table,
187   event,
188   filter,
189   enabled,
190   strategy,
191   pollingInterval,
192   startPolling,
193   stopPolling,
194 ]);
195
196 return { isLive, retryCount };
197 };
198

```

FILE: src/hooks/useReflectionDigest.ts

```
TYPE: .ts | LINES: 1-164 | SHA256: 87a01193b1059e77ba41ee0522fe1870c5503c51af56e744cafbeb66a149a10f
```

[illegible]

```

36 | ) => {
37 |   return useQuery({
38 |     queryKey: queryKeys.journal.detail(`digest-${studentId}-${month}`),
39 |     enabled: !!studentId && !!month,
40 |     queryFn: async (): Promise<ReflectionDigest | null> => {
41 |       if (!studentId || !month) return null;
42 |
43 |       const { data, error } = await supabase
44 |         .from("reflection_digests")
45 |         .select("*")
46 |         .eq("student_id", studentId)
47 |         .eq("month", month)
48 |         .maybeSingle();
49 |
50 |       if (error) throw error;
51 |       if (!data) return null;
52 |       return mapDigest(data as Record<string, unknown>);
53 |     },
54 |   });
55 | };
56 |
57 | // % % % useShareDigest - share digest with parent/advisor/teacher % % % % % % % % % % % % % % % %
58 |
59 | export const useShareDigest = () => {
60 |   const queryClient = useQueryClient();
61 |   const { user } = useAuth();
62 |
63 |   return useMutation({
64 |     mutationFn: async (input: {
65 |       digestId: string;
66 |       role: "parent" | "advisor" | "teacher";
67 |     }): Promise<void> => {
68 |       if (!user) throw new Error("Not authenticated");
69 |
70 |       // Fetch current shared_with array
71 |       const { data: current, error: fetchError } = await supabase
72 |         .from("reflection_digests")
73 |         .select("shared_with")
74 |         .eq("id", input.digestId)
75 |         .eq("student_id", user.id)
76 |         .maybeSingle();
77 |
78 |       if (fetchError) throw fetchError;
79 |       if (!current) throw new Error("Reflection digest not found");
80 |
81 |       const sharedWith =
82 |         ((current as Record<string, unknown>).shared_with as Array<{
83 |           role: string;
84 |           sharedAt: string;
85 |         }>) ?? [];
86 |
87 |       // Check if already shared with this role
88 |       if (sharedWith.some((s) => s.role === input.role)) return;
89 |
90 |       // Add new share entry
91 |       const updated = [
92 |         ...sharedWith,
93 |         { role: input.role, sharedAt: new Date().toISOString() },
94 |       ];
95 |
96 |       const { error } = await supabase
97 |         .from("reflection_digests")
98 |         .update({ shared_with: updated as never })
99 |         .eq("id", input.digestId)
100 |        .eq("student_id", user.id);
101 |
102 |       if (error) throw error;
103 |     },
104 |     onSuccess: () => {
105 |       queryClient.invalidateQueries({ queryKey: queryKeys.journal.lists() });
106 |       toast.success("Digest shared successfully");
107 |     },
108 |     onError: (error: Error) => {
109 |       toast.error(`Failed to share digest: ${error.message}`);
110 |     },
111 |   });
112 | };
113 |
114 | // % % % useRevokeDigestShare - revoke sharing with a role % % % % % % % % % % % % % % % %
115 |
116 | export const useRevokeDigestShare = () => {
117 |   const queryClient = useQueryClient();
118 |   const { user } = useAuth();
119 |
120 |   return useMutation({
121 |     mutationFn: async (input: {
122 |       digestId: string;
123 |       role: "parent" | "advisor" | "teacher";
124 |     }): Promise<void> => {
125 |       if (!user) throw new Error("Not authenticated");
126 |
127 |       // Fetch current shared_with array
128 |       const { data: current, error: fetchError } = await supabase
129 |         .from("reflection_digests")
130 |         .select("shared_with")
131 |         .eq("id", input.digestId)
132 |         .eq("student_id", user.id)
133 |         .maybeSingle();
134 |
135 |       if (fetchError) throw fetchError;
136 |       if (!current) throw new Error("Reflection digest not found");
137 |
138 |       const sharedWith =
139 |         ((current as Record<string, unknown>).shared_with as Array<{

```

```

140     role: string;
141     sharedAt: string;
142   }>) ?? [];
143
144   // Remove the role
145   const updated = sharedWith.filter((s) => s.role !== input.role);
146
147   const { error } = await supabase
148     .from("reflection_digests")
149     .update({ shared_with: updated as never })
150     .eq("id", input.digestId)
151     .eq("student_id", user.id);
152
153   if (error) throw error;
154 },
155 onSuccess: () => {
156   queryClient.invalidateQueries({ queryKey: queryKeys.journal.lists() });
157   toast.success("Sharing revoked");
158 },
159 onError: (error: Error) => {
160   toast.error(`Failed to revoke sharing: ${error.message}`);
161 },
162 });
163 };
164

```

FILE: `src/hooks/useReflectionQuality.ts`

```
TYPE: .ts | LINES: 1-118 | SHA256: e95e7765950c4e1960bf0f4fbd46d1b531936bc7efa4612b1ea6e54f2f504404
```

[illegible]

```

76 {
77   body: {
78     reflection_id: input.reflectionId,
79     reflection_type: input.reflectionType,
80     student_id: user.id,
81     content: input.content,
82   },
83 }
84 );
85
86 if (error) throw error;
87
88 const result = data as Record<string, unknown>;
89 const score = mapQualityScore(result.score as Record<string, unknown>);
90 const category = getQualityCategory(score.score);
91 const suggestions = (result.suggestions as string[]) ?? [];
92
93 // Calculate XP adjustment based on quality
94 const baseXP = input.reflectionType === "journal_entry" ? 20 : 10;
95 const xpAdjustment = calculateReflectionXP(
96   baseXP,
97   score.score,
98   score.relevanceScore,
99   score.depthScore,
100   input.reflectionType === "journal_entry"
101 );
102
103 return { score, category, suggestions, xpAdjustment };
104 },
105 onSuccess: () => {
106   queryClient.invalidateQueries({ queryKey: queryKeys.journal.lists() });
107 },
108 });
109 };
110
111 // % % % useQualityCategory - convenience hook for category from score % % % % % % % % % %
112
113 export const useQualityCategory = (
114   score: number | null | undefined
115 ): QualityCategory => {
116   return getQualityCategory(score ?? null);
117 };
118

```

FILE: src/hooks/useReflectionTemplates.ts

```
TYPE: .ts | LINES: 1-105 | SHA256: 65ad9310c049aa8222e0fc340cb3fe2301d6c9b3afffecfb32b4cbaee4fb2e20
```

[illegible]





[illegible]

```

157 | return useMutation({
158 |   mutationFn: async ({ voteId, status }: ResolveVoteInput) => {
159 |     const { data, error } = await supabase
160 |       .from("replacement_votes" as never)
161 |       .update({
162 |         status,
163 |         resolved_at: new Date().toISOString(),
164 |       } as never)
165 |       .eq("id", voteId)
166 |       .select()
167 |       .single();
168 |     if (error) throw error;
169 |     return data as ReplacementVote;
170 |   },
171 |   onSuccess: (_data, variables) => {
172 |     qc.invalidateQueries({
173 |       queryKey: queryKeys.replacementVotes.list({ teamId: variables.teamId }),
174 |     });
175 |     qc.invalidateQueries({ queryKey: queryKeys.teamMembers.lists() });
176 |   },
177 |   onError: (error: Error) => {
178 |     toast.error(error.message || "Failed to resolve vote");
179 |   },
180 | });
181 | };
182 |
183 | // % % % useTeacherOverrideVote - teacher overrides a vote result % % % % % % % % % % % % % % % %
184 |
185 | interface TeacherOverrideInput {
186 |   voteId: string;
187 |   teamId: string;
188 |   status: "approved" | "rejected";
189 | }
190 |
191 | export const useTeacherOverrideVote = () => {
192 |   const qc = useQueryClient();
193 |   return useMutation({
194 |     mutationFn: async ({ voteId, status }: TeacherOverrideInput) => {
195 |       const { data, error } = await supabase
196 |         .from("replacement_votes" as never)
197 |         .update({
198 |           status,
199 |           resolved_at: new Date().toISOString(),
200 |           teacher_override: true,
201 |         } as never)
202 |         .eq("id", voteId)
203 |         .select()
204 |         .single();
205 |       if (error) throw error;
206 |       return data as ReplacementVote;
207 |     },
208 |     onSuccess: (_data, variables) => {
209 |       qc.invalidateQueries({
210 |         queryKey: queryKeys.replacementVotes.list({ teamId: variables.teamId }),
211 |       });
212 |       qc.invalidateQueries({ queryKey: queryKeys.teamMembers.lists() });
213 |     },
214 |     onError: (error: Error) => {
215 |       toast.error(error.message || "Failed to override vote");
216 |     },
217 |   });
218 | };
219 |

```

## FILE: src/hooks/useReviewQueue.ts

TYPE: .ts | LINES: 1-112 | SHA256: 9fbfc80ea3884e5495905724866b768249085a1845a3e76016232734b1a14ca7

```

1 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import type { QuestionBankRow } from "@hooks/useQuestionBank";
5 |
6 | // % % % useReviewQueue - fetch pending_review questions for a course % % % % % % % % % % % % % % % %
7 |
8 | export const useReviewQueue = (courseId: string) => {
9 |   return useQuery({
10 |     queryKey: queryKeys.reviewQueue.list({ courseId }),
11 |     queryFn: async (): Promise<QuestionBankRow[]> => {
12 |       const { data, error } = await supabase
13 |         .from("question_bank")
14 |         .select("*")
15 |         .eq("course_id", courseId)
16 |         .eq("status", "pending_review")
17 |         .order("created_at", { ascending: false });
18 |
19 |       if (error) throw error;
20 |       return (data ?? []) as QuestionBankRow[];
21 |     },
22 |     enabled: !!courseId,
23 |   });
24 | };
25 |
26 | // % % % useApproveQuestion - set status to approved % % % % % % % % % % % % % % % % % % % % % % % % % % % %
27 |
28 | export const useApproveQuestion = () => {
29 |   const queryClient = useQueryClient();
30 |
31 |   return useMutation({
32 |     mutationFn: async ({ id }: { id: string }): Promise<QuestionBankRow> => {
33 |       const { data, error } = await supabase
34 |         .from("question_bank")
35 |         .update({ status: "approved" })
36 |         .eq("id", id)
37 |         .select()

```



[illegible]

[illegible]

FILE: src/hooks/useRubrics.ts

TYPE: .ts | LINES: 1-413 | SHA256: 689bb74d2c360473a61434159ae6fa7cac94c640e268afe81c28df3dd1943f7a

[illegible]

[illegible]

[illegible]

[illegible]



```

345 |
346 | // Fetch original criteria
347 | const { data: srcCriteria, error: criteriaFetchError } = await supabase
348 |   .from("rubric_criteria")
349 |   .select(".*")
350 |   .eq("rubric_id", sourceId)
351 |   .order("sort_order", { ascending: true });
352 |
353 | if (criteriaFetchError) throw criteriaFetchError;
354 |
355 | // Insert copied rubric
356 | const { data: newRubric, error: insertError } = await supabase
357 |   .from("rubrics")
358 |   .insert({
359 |     title: `${src.title} (Copy)`,
360 |     clo_id: src.clo_id,
361 |     is_template: false,
362 |   })
363 |   .select()
364 |   .single();
365 |
366 | if (insertError) throw insertError;
367 |
368 | const created = newRubric as Rubric;
369 |
370 | // Insert copied criteria
371 | const criteria = (srcCriteria ?? []).map((c) => ({
372 |   ...c,
373 |   levels: c.levels as unknown as Array<{
374 |     label: string;
375 |     description: string;
376 |     points: number;
377 |   }>,
378 | }));
379 | if (criteria.length > 0) {
380 |   const rows = criteria.map((c) => ({
381 |     rubric_id: created.id,
382 |     criterion_name: c.criterion_name,
383 |     sort_order: c.sort_order,
384 |     levels: c.levels,
385 |     max_points: c.max_points,
386 |   }));
387 |
388 |   const { error: criteriaInsertError } = await supabase
389 |     .from("rubric_criteria")
390 |     .insert(rows);
391 |
392 |   if (criteriaInsertError) throw criteriaInsertError;
393 | }
394 |
395 | // Req 13.5 – admin mutation writes an audit_logs row. The entity is
396 | // the newly-created copy; `changes` references the source so reviewers
397 | // can trace clone lineage without walking inserts manually.
398 | await logAuditEvent({
399 |   action: "create",
400 |   entity_type: "rubric",
401 |   entity_id: created.id,
402 |   changes: { copied_from: sourceId, title: created.title },
403 |   performed_by: user?.id ?? "unknown",
404 | });
405 |
406 | return created;
407 | },
408 | onSuccess: () => {
409 |   queryClient.invalidateQueries({ queryKey: queryKeys.rubrics.lists() });
410 | },
411 | });
412 | };
413 |

```

## FILE: src/hooks/useSabbaticalStatus.ts

TYPE: .ts | LINES: 1-43 | SHA256: 21a052f8a1a5341d555b878832e3d8fc1ae61e4e5b7581f45b19d0928d40939b

```

1 | import { useQuery } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 |
5 | /**
6 |  * Fetches whether Streak Sabbatical is enabled for the student's institution.
7 |  * The setting lives on `institution_settings.streak_sabbatical_enabled` –
8 |  * NOT on `student_gamification` as previously assumed. This was a critical
9 |  * silent-failure bug.
10 |  */
11 | export const useSabbaticalStatus = (studentId: string | undefined) => {
12 |   return useQuery({
13 |     queryKey: queryKeys.studentGamification.sabbatical(studentId ?? ""),
14 |     enabled: !!studentId,
15 |     queryFn: async (): Promise<boolean> => {
16 |       if (!studentId) return false;
17 |
18 |       try {
19 |         // 1. Look up the student's institution
20 |         const { data: profile, error: profileErr } = await supabase
21 |           .from("profiles")
22 |           .select("institution_id")
23 |           .eq("id", studentId)
24 |           .maybeSingle();
25 |
26 |         if (profileErr || !profile?.institution_id) return false;
27 |
28 |         // 2. Read the institution-level setting
29 |         const { data: settings, error: settingsErr } = await supabase
30 |           .from("institution_settings")
31 |           .select("streak_sabbatical_enabled")

```

[illegible]

[illegible]

FILE: src/hooks/useSectionAttainment.ts

TYPE: .ts | LINES: 1-252 | SHA256: adf3ba301770f63e1b4cebd19df3125f3b5188421324556135fb71c76b3525f1

```

1 // =====
2 // useSectionAttainment - Coordinator Section Comparison real data (Req 10)
3 // =====
4 // Feature: qa-partner-review-remediation - Req 10.1, 10.2, 10.3
5 // =====
6 // Computes, per `course_sections` row of a given course:
7 //   * real attainment - mean of `outcome_attainment.attainment_percent` for the
8 //     students enrolled in that section. `scope` = `student course`

```

[illegible]

[illegible]

```

217 |         .from("outcome_attainment")
218 |         .select("outcome_id, attainment_percent")
219 |         .in("outcome_id", cloIds)
220 |         .in("student_id", studentIds)
221 |         .eq("scope", "student_course");
222 |     if (attainError) throw attainError;
223 |
224 |     for (const row of (attainment ?? []) as CLOAttainmentRow[]) {
225 |         const arr = scoresByCLO.get(row.outcome_id) ?? [];
226 |         arr.push(row.attainment_percent);
227 |         scoresByCLO.set(row.outcome_id, arr);
228 |     }
229 | }
230 |
231 | return {
232 |     section_id: sectionRow.id,
233 |     section_code: sectionRow.section_code,
234 |     teacher_name: sectionRow.profiles?.full_name ?? null,
235 |     student_count: studentIds.length,
236 |     clos: cloRows.map((clo) => {
237 |         const scores = scoresByCLO.get(clo.id) ?? [];
238 |         return {
239 |             clo_id: clo.id,
240 |             clo_title: clo.title,
241 |             blooms_level: clo.blooms_level,
242 |             attainment_percent: mean(scores),
243 |             sample_count: scores.length,
244 |         };
245 |     })),
246 | };
247 |
248 | },
249 | enabled: !!courseId && !!sectionId,
250 | staleTime: DASHBOARD_STALE_TIME_MS,
251 | });
252 | };
```

FILE: src/hooks/useSemesterRange.ts

```
TYPE: .ts | LINES: 1-45 | SHA256: 59f66d07dc307ce9586ed205d5e109ac794966d55f11cd69e854d5b1dd24ea04
```

```

1 import { useQuery } from "@tanstack/react-query";
2 import { supabase } from "@lib/supabase";
3 import { queryKeys } from "@lib/queryKeys";
4 import type { DateRange } from "@types/habits";
5
6 /**
7  * Resolves the current semester date range for the heatmap.
8  * Tries to fetch from the 'semesters' table first; falls back to
9  * Jan 1 - Dec 31 of the current calendar year.
10 */
11 export const useSemesterRange = (studentId: string | undefined) => {
12   return useQuery({
13     queryKey: [...queryKeys.semesters.all, "current", studentId ?? ""],
14     enabled: !!studentId,
15     staleTime: 1000 * 60 * 30, // 30 min - semester range rarely changes
16     queryFn: async (): Promise<DateRange> => {
17       const today = new Date().toISOString().slice(0, 10);
18
19       // Attempt to find the active semester that contains today
20       const { data, error } = await supabase
21         .from("semesters")
22         .select("start_date, end_date")
23         .lte("start_date", today)
24         .gte("end_date", today)
25         .order("start_date", { ascending: false })
26         .limit(1)
27         .maybeSingle();
28
29       if (!error && data?.start_date && data?.end_date) {
30         return {
31           start: data.start_date as string,
32           end: data.end_date as string,
33         };
34       }
35
36       // Fallback: current calendar year
37       const year = new Date().getFullYear();
38       return {
39         start: `${year}-01-01`,
40         end: `${year}-12-31`,
41       };
42     },
43   });
44 };
45

```

FILE: src/hooks/useSemesters.ts

TYPE: .ts | LINES: 1-253 | SHA256: ecb7649e8f204564c2b56827573c73923031dd2386bb011319749b7adca66667

```

1 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@/lib/supabase";
3 | import { queryKeys } from "@/lib/queryKeys";
4 | import { logAuditEvent } from "@/lib/auditLogger";
5 | import { useAuth } from "@/hooks/useAuth";
6 | import { toast } from "sonner";
7 | import type { Semester } from "@/types/app";
8 |
9 | // % % % useSemesters - list all semesters % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
10|
11| export const useSemesters = () => {
12|   return useQuery({
13|     queryKey: queryKeys.semesters.lists(),
14|     queryFn: async (): Promise<Semester[]> => {
15|       const { data, error } = await supabase

```

[illegible]

[illegible]



```

224 |
225 |     const { data, error } = await supabase
226 |       .from("semesters")
227 |       .update({ is_active })
228 |       .eq("id", id)
229 |       .select()
230 |       .single();
231 |     if (error) throw error;
232 |
233 |     await logAuditEvent({
234 |       action: "update",
235 |       entity_type: "semester",
236 |       entity_id: id,
237 |       changes: { is_active },
238 |       performed_by: user?.id ?? "unknown",
239 |     });
240 |
241 |     return data as Semester;
242 |   },
243 |   onSuccess: () => {
244 |     qc.invalidateQueries({ queryKey: queryKeys.semesters.all });
245 |     toast.success("Semester status updated");
246 |   },
247 |   onError: (err) =>
248 |     toast.error(
249 |       err instanceof Error ? err.message : "Failed to update semester"
250 |     ),
251 | });
252 | };
253 |

```

FILE: src/hooks/useSessionCompletion.ts

TYPE: .ts | LINES: 1-362 | SHA256: ad405aebcb0d74a6a80d37245d9bb5356947206d23cb8a5f6dc9f735af2fccda

[illegible]

```

71 |     for (const file of evidenceFiles) {
72 |         const filePath = `session-evidence/${
73 |             user.id
74 |         }/${sessionId}/${Date.now()}_${file.name}`;
75 |
76 |         const { error: uploadError } = await supabase.storage
77 |             .from("session-evidence")
78 |             .upload(filePath, file);
79 |
80 |         if (uploadError) {
81 |             console.error(
82 |                 "[useCompleteSession] file upload failed:",
83 |                 uploadError.message
84 |             );
85 |             continue;
86 |         }
87 |
88 |         // session-evidence is a private bucket. Store the storage path -
89 |         // consumers must call getSignedUrl("session-evidence", file_url)
90 |         // at READ time. See src/lib/storageUrl.ts.
91 |
92 |         // 3. Insert evidence record
93 |         const { error: evidenceError } = await supabase
94 |             .from("session_evidence")
95 |             .insert({
96 |                 session_id: sessionId,
97 |                 student_id: user.id,
98 |                 file_url: filePath,
99 |                 file_name: file.name,
100 |                 file_size_bytes: file.size,
101 |                 mime_type: file.type,
102 |                 notes: notes ?? null,
103 |             } as never);
104 |
105 |         if (evidenceError) {
106 |             console.error(
107 |                 "[useCompleteSession] evidence insert failed:",
108 |                 evidenceError.message
109 |             );
110 |             continue;
111 |         }
112 |
113 |         evidenceCount++;
114 |     }
115 | } else if (notes) {
116 |     // Insert a text-only evidence record
117 |     const { error: evidenceError } = await supabase
118 |         .from("session_evidence")
119 |         .insert({
120 |             session_id: sessionId,
121 |             student_id: user.id,
122 |             file_url: "",
123 |             file_name: "notes",
124 |             file_size_bytes: 0,
125 |             mime_type: "text/plain",
126 |             notes,
127 |         } as never);
128 |
129 |     if (!evidenceError) evidenceCount++;
130 | }
131 |
132 | // 4. Award XP via award-xp Edge Function
133 | const hasEvidence = evidenceCount > 0;
134 | const xpAmount = calculateSessionXP(actualDurationMinutes, hasEvidence);
135 | let xpAwarded = 0;
136 |
137 | if (xpAmount > 0) {
138 |     try {
139 |         const { data: xpResult } = await supabase.functions.invoke(
140 |             "award-xp",
141 |             {
142 |                 body: {
143 |                     student_id: user.id,
144 |                     xp_amount: xpAmount,
145 |                     source: "study_session",
146 |                     reference_id: sessionId,
147 |                 },
148 |             }
149 |         );
150 |         xpAwarded =
151 |             ((xpResult as Record<string, unknown>)?.xp_earned as number) ?? 0;
152 |     } catch {
153 |         console.error("[useCompleteSession] award-xp invocation failed");
154 |     }
155 | }
156 |
157 | // 5. Check badges
158 | let newBadges: string[] = [];
159 | try {
160 |     const { data: badgeResult } = await supabase.functions.invoke(
161 |         "check-badges",
162 |         {
163 |             body: {
164 |                 student_id: user.id,
165 |                 trigger: "study_session",
166 |             },
167 |         }
168 |     );
169 |     newBadges =
170 |         ((badgeResult as Record<string, unknown>)?.new_badges as string[]) ??
171 |         [];
172 | } catch {
173 |     console.error("[useCompleteSession] check-badges invocation failed");
174 | }

```

```

175 // 6a. Award XP for any weekly goals just met (idempotent)
176 try {
177   await awardWeeklyGoalXPIfMet(user.id);
178 } catch {
179   console.error("[useCompleteSession] weekly goal XP award failed");
180 }
181
182 // 6b. Award review session XP (if this session is linked to a review schedule)
183 try {
184   const { data: reviewScheduleRows } = await supabase
185     .from("review_schedules")
186     .select("id, clo_id, interval_days, status")
187     .eq("review_session_id", sessionId)
188     .eq("student_id", user.id);
189
190   const reviewRows = reviewScheduleRows as Array<{
191     id: string;
192     clo_id: string;
193     interval_days: number;
194     status: string;
195   }> | null;
196
197   if (reviewRows && reviewRows.length > 0) {
198     // This session is a review session – award 15 XP
199     try {
200       await supabase.functions.invoke("award-xp", {
201         body: {
202           student_id: user.id,
203           xp_amount: 15,
204           source: "review_session",
205           reference_id: `review_session:${sessionId}`,
206         },
207       });
208     } catch {
209       console.error(
210         "[useCompleteSession] review_session XP award failed"
211       );
212     }
213   }
214
215   // Check if all 3 intervals (1, 3, 7) are now complete for each CLO
216   const cloIds = [...new Set(reviewRows.map((r) => r.clo_id))];
217
218   for (const cloId of cloIds) {
219     try {
220       const { data: allCloReviews } = await supabase
221         .from("review_schedules")
222         .select("interval_days, status")
223         .eq("student_id", user.id)
224         .eq("clo_id", cloId);
225
226       const cloReviewRows = (allCloReviews ?? []) as Array<{
227         interval_days: number;
228         status: string;
229       }>;
230
231       if (
232         isReviewCycleComplete(
233           cloReviewRows.map((r) => ({
234             intervalDays: r.interval_days,
235             status: r.status,
236           })))
237       ) {
238         try {
239           await supabase.functions.invoke("award-xp", {
240             body: {
241               student_id: user.id,
242               xp_amount: 25,
243               source: "review_cycle_complete",
244               reference_id: `review_cycle_complete:${user.id}:${cloId}`,
245               is_milestone: true,
246             },
247           });
248         } catch {
249           console.error(
250             "[useCompleteSession] review_cycle_complete XP award failed"
251           );
252         }
253       }
254     } catch {
255       console.error(
256         "[useCompleteSession] review cycle check failed for CLO:",
257         cloId
258       );
259     }
260   }
261 } catch {
262   console.error("[useCompleteSession] review session XP flow failed");
263 }
264
265 // 6. Auto-mark Read habit (if session "e 15 min) in canonical habit_logs
266 if (actualDurationMinutes >= 15) {
267   try {
268     const today = new Date().toISOString().split("T")[0] as string;
269     await supabase.from("habit_logs").upsert(
270       {
271         student_id: user.id,
272         habit_type: "read",
273         date: today,
274         completed_at: new Date().toISOString(),
275       },
276       { onConflict: "student_id,habit_type,date" }
277     );
278   }

```



[illegible]

```

121 |         .select()
122 |         .single();
123 |
124 |         if (insertError) {
125 |             console.error(
126 |                 "[useUploadEvidence] insert failed:",
127 |                 insertError.message
128 |             );
129 |             failed++;
130 |             continue;
131 |         }
132 |
133 |         uploaded.push(mapEvidence(data as Record<string, unknown>));
134 |     }
135 |
136 |     return { uploaded, failed };
137 | },
138 | onSuccess: ({ uploaded, failed }) => {
139 |     queryClient.invalidateQueries({
140 |         queryKey: queryKeys.evidence.lists(),
141 |     });
142 |
143 |     if (uploaded.length > 0 && failed === 0) {
144 |         toast.success(
145 |             `${uploaded.length} file${uploaded.length > 1 ? "s" : ""} uploaded`
146 |         );
147 |     } else if (uploaded.length > 0 && failed > 0) {
148 |         toast.success(`${uploaded.length} uploaded, ${failed} failed`);
149 |     } else if (failed > 0) {
150 |         toast.error("All file uploads failed");
151 |     }
152 | },
153 | onError: (error: Error) => {
154 |     toast.error(`Upload failed: ${error.message}`);
155 | },
156 | });
157 | };
158 |

```

FILE: src/hooks/useSessionIntent.ts

```
TYPE: .ts | LINES: 1-219 | SHA256: ec6146c080c2d2e8d7d66149fdcde88ef0140b5482e201bb183df551db866e45
```

[illegible]

```

63 | });
64 | };
65 |
66 | // % % % useSessionIntent - fetch single intent for a session % % % % % % % % % % % % %
67 |
68 | export const useSessionIntent = (sessionId: string | undefined) => {
69 |   return useQuery({
70 |     queryKey: queryKeys.sessionIntents.detail(sessionId ?? ""),
71 |     enabled: !!sessionId,
72 |     queryFn: async (): Promise<SessionIntent | null> => {
73 |       if (!sessionId) return null;
74 |
75 |       const { data, error } = await supabase
76 |         .from("session_intents" as never)
77 |         .select("*")
78 |         .eq("session_id", sessionId)
79 |         .maybeSingle();
80 |
81 |       if (error) throw error;
82 |       return data ? mapIntent(data as Record<string, unknown>) : null;
83 |     },
84 |   });
85 | };
86 |
87 | // % % % useSuggestedIntents - fetch candidate concepts for a session % % % % % % % % % % %
88 | // Pulls from:
89 | // 1. CLOs from the student's enrolled courses with attainment < 70%
90 | // 2. Upcoming assignment deadlines in the next 7 days
91 | // Returns up to 3 suggestions, prioritising the session's own course.
92 |
93 | export const useSuggestedIntents = (sessionId: string | undefined) => {
94 |   const { user } = useAuth();
95 |
96 |   return useQuery({
97 |     queryKey: queryKeys.suggestedIntents.detail(sessionId ?? ""),
98 |     enabled: !!sessionId && !!user,
99 |     queryFn: async (): Promise<SuggestedIntent[]> => {
100 |       if (!sessionId || !user) return [];
101 |
102 |       const suggestions: SuggestedIntent[] = [];
103 |
104 |       // 0. Get the session's course_id so we can prioritise its CLOs
105 |       const { data: sessionRow } = await supabase
106 |         .from("study_sessions")
107 |         .select("course_id, clo_ids")
108 |         .eq("id", sessionId)
109 |         .maybeSingle();
110 |
111 |       const sessionCourseId =
112 |         (sessionRow as { course_id?: string } | null)?.course_id ?? null;
113 |
114 |       // 1. Get enrolled course IDs for the student
115 |       const { data: enrollments, error: enrollError } = await supabase
116 |         .from("student_courses")
117 |         .select("course_id")
118 |         .eq("student_id", user.id);
119 |
120 |       if (enrollError) throw enrollError;
121 |
122 |       const courseIds = (enrollments ?? []).map((e) => e.course_id as string);
123 |
124 |       if (courseIds.length === 0) return [];
125 |
126 |       // 2. Get all CLOs for enrolled courses
127 |       const { data: cloRows, error: cloError } = await supabase
128 |         .from("learning_outcomes")
129 |         .select("id, title, course_id")
130 |         .eq("type", "CLO")
131 |         .in("course_id", courseIds);
132 |
133 |       if (cloError) throw cloError;
134 |
135 |       const cloIds = (cloRows ?? []).map(
136 |         (c: Record<string, unknown>) => c.id as string
137 |       );
138 |
139 |       // 3. Fetch attainment for those CLOs and filter < 70%
140 |       if (cloIds.length > 0) {
141 |         const { data: attainmentRows } = await supabase
142 |           .from("outcome_attainment")
143 |           .select("outcome_id, attainment_percent")
144 |           .eq("student_id", user.id)
145 |           .eq("scope", "student_course")
146 |           .in("outcome_id", cloIds);
147 |
148 |         const lowAttainmentIds = new Set(
149 |           (attainmentRows ?? []).
150 |             .filter(
151 |               (r: Record<string, unknown>) =>
152 |                 (r.attainment_percent as number) < 70
153 |             )
154 |             .map((r: Record<string, unknown>) => r.outcome_id as string)
155 |         );
156 |
157 |         if (lowAttainmentIds.size > 0) {
158 |           // Build a lookup from CLO id !' title + courseId
159 |           const cloMap = new Map<string, { title: string; courseId: string }>();
160 |           for (const clo of cloRows ?? []) {
161 |             const c = clo as Record<string, unknown>;
162 |             cloMap.set(c.id as string, {
163 |               title: c.title as string,
164 |               courseId: c.course_id as string,
165 |             });
166 |           }

```

```

167 |
168 | // Sort: session's own course CLOs first, then others
169 | const lowCLOs = [...lowAttainmentIds]
170 |   .map((id) => ({ id, ...cloMap.get(id)! })))
171 |   .filter((c) => c.title) // guard against missing lookup
172 |   .sort((a, b) => {
173 |     const aOwn = a.courseId === sessionCourseId ? 0 : 1;
174 |     const bOwn = b.courseId === sessionCourseId ? 0 : 1;
175 |     return aOwn - bOwn;
176 |   });
177 |
178 | for (const clo of lowCLOs.slice(0, 2)) {
179 |   suggestions.push({
180 |     concept: clo.title,
181 |     successCriterion: `Solve 3 practice problems on ${clo.title} correctly`,
182 |   });
183 | }
184 | }
185 | }
186 |
187 | // 4. Upcoming assignment deadlines in next 7 days (scoped to enrolled courses)
188 | const today = new Date().toISOString().split("T")[0] as string;
189 | const sevenDaysFromNow = new Date();
190 | sevenDaysFromNow.setDate(sevenDaysFromNow.getDate() + 7);
191 | const sevenDaysStr = sevenDaysFromNow
192 |   .toISOString()
193 |   .split("T")[0] as string;
194 |
195 | const { data: assignments } = await supabase
196 |   .from("assignments")
197 |   .select("id, title, due_date")
198 |   .in("course_id", courseIds)
199 |   .gte("due_date", today)
200 |   .lte("due_date", sevenDaysStr)
201 |   .order("due_date", { ascending: true })
202 |   .limit(2);
203 |
204 | for (const a of assignments ?? []) {
205 |   const row = a as Record<string, unknown>;
206 |   suggestions.push({
207 |     concept: row.title as string,
208 |     successCriterion: `Complete an outline / first draft for ${
209 |       row.title as string
210 |     }`,
211 |   });
212 | }
213 |
214 | // Return at most 3 suggestions
215 | return suggestions.slice(0, 3);
216 | },
217 | });
218 | };
219 |

```

#### FILE: src/hooks/useSessionManagement.ts

TYPE: .ts | LINES: 1-63 | SHA256: cdc84a24afcd77c45269a926754ad2dcdbba46524596f1440ee614609c73bc2f

```

1 | // Task 95.2: Session management hook
2 | // Queries active sessions and provides sign-out mutations
3 |
4 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
5 | import { supabase } from "@lib/supabase";
6 | import { queryKeys } from "@lib/queryKeys";
7 |
8 | export interface ActiveSession {
9 |   id: string;
10 |   device: string;
11 |   ip: string;
12 |   lastActive: string;
13 |   isCurrent: boolean;
14 | }
15 |
16 | export const useActiveSessions = (userId: string | undefined) => {
17 |   return useQuery({
18 |     queryKey: queryKeys.sessions.list({ userId }),
19 |     queryFn: async (): Promise<ActiveSession[]> => {
20 |       if (!userId) return [];
21 |
22 |       // Supabase Auth admin API is server-side only.
23 |       // Return current session info as the only known session.
24 |       const {
25 |         data: { session },
26 |       } = await supabase.auth.getSession();
27 |       if (!session) return [];
28 |
29 |       return [
30 |         {
31 |           id: session.access_token.slice(-8),
32 |           device: navigator.userAgent.slice(0, 60),
33 |           ip: "Current device",
34 |           lastActive: new Date().toISOString(),
35 |           isCurrent: true,
36 |         },
37 |       ];
38 |     },
39 |     enabled: !!userId,
40 |   });
41 | };
42 |
43 | export const useSignOutOtherSessions = () => {
44 |   const queryClient = useQueryClient();
45 |
46 |   return useMutation({
47 |     mutationFn: async (): Promise<void> => {

```



```

48 // Sign out globally then re-sign-in with current session's refresh token
49 // This effectively invalidates all other sessions
50 const {
51   data: { session },
52 } = await supabase.auth.getSession();
53 if (!session) throw new Error("No active session");
54
55 const { error } = await supabase.auth.signOut({ scope: "others" });
56 if (error) throw error;
57
58 onSuccess: () => {
59   queryClient.invalidateQueries({ queryKey: queryKeys.sessions.all });
60 },
61 });
62 };
63

```

FILE: [src/hooks/useSessionReflections.ts](#)

TYPE: .ts | LINES: 1-241 | SHA256: 777e0007b012b8b31d5320995ddb32d810c9ddf644efc95e6a50b02e97f26016

[illegible]

```

85 |         word_count: wordCount,
86 |     } as never)
87 |     .select()
88 |     .single();
89 |
90 |     if (error) throw error;
91 |
92 |     const reflection = mapReflection(data as Record<string, unknown>);
93 |
94 |     // Award XP via award-xp Edge Function with source 'session_reflection'
95 |     let xpAwarded = 0;
96 |     try {
97 |         const { data: xpResult } = await supabase.functions.invoke("award-xp", {
98 |             body: {
99 |                 student_id: user.id,
100 |                 xp_amount: 10,
101 |                 source: "session_reflection",
102 |                 reference_id: reflection.id,
103 |             },
104 |         });
105 |         xpAwarded =
106 |             ((xpResult as Record<string, unknown>)?.xp_awarded as number) ?? 0;
107 |     } catch {
108 |         console.error("[useSaveSessionReflection] award-xp invocation failed");
109 |     }
110 |
111 |     // Trigger quality scoring asynchronously (non-blocking)
112 |     try {
113 |         await supabase.functions.invoke("score-reflection-quality", {
114 |             body: {
115 |                 reflection_id: reflection.id,
116 |                 reflection_type: "session_reflection",
117 |                 student_id: user.id,
118 |                 content: input.content,
119 |             },
120 |         });
121 |     } catch {
122 |         console.error("[useSaveSessionReflection] quality scoring failed");
123 |     }
124 |
125 |     return { reflection, xpAwarded };
126 | },
127 | onSuccess: ({ xpAwarded }) => {
128 |     queryClient.invalidateQueries({
129 |         queryKey: queryKeys.journal.lists(),
130 |     });
131 |     queryClient.invalidateQueries({
132 |         queryKey: queryKeys.studySessions.lists(),
133 |     });
134 |     queryClient.invalidateQueries({
135 |         queryKey: queryKeys.studentGamification.lists(),
136 |     });
137 |
138 |     if (xpAwarded > 0) {
139 |         toast.success(`Reflection saved! +${xpAwarded} XP`);
140 |     } else {
141 |         toast.success("Reflection saved!");
142 |     }
143 | },
144 | onError: (error: Error) => {
145 |     toast.error(`Failed to save reflection: ${error.message}`);
146 | },
147 | });
148 | };
149 |
150 | // % % % useSaveWeeklyReflection – save weekly reflection as journal entry % % % % %
151 |
152 | export const useSaveWeeklyReflection = () => {
153 |     const queryClient = useQueryClient();
154 |     const { user } = useAuth();
155 |
156 |     return useMutation({
157 |         mutationFn: async (
158 |             input: WeeklyReflectionInput
159 |         ): Promise<{ journalEntryId: string; xpAwarded: number }> => {
160 |             if (!user) throw new Error("Not authenticated");
161 |
162 |             const wordCount = countWords(input.content);
163 |
164 |             // Validate minimum word count
165 |             if (wordCount < 50) {
166 |                 throw new Error("Weekly reflection must be at least 50 words");
167 |             }
168 |
169 |             // Get the student's first enrolled course for the journal entry
170 |             // (weekly reflections need a course_id for the journal_entries table)
171 |             const { data: enrollments } = await supabase
172 |                 .from("student_courses")
173 |                 .select("course_id")
174 |                 .eq("student_id", user.id)
175 |                 .limit(1);
176 |
177 |             const courseId = (enrollments?.[0] as Record<string, unknown> | undefined)
178 |                 ?.course_id as string | undefined;
179 |
180 |             if (!courseId) {
181 |                 throw new Error("No enrolled courses found for journal entry");
182 |             }
183 |
184 |             // Create journal_entries record (triggers standard journal XP of 20)
185 |             const { data, error } = await supabase
186 |                 .from("journal_entries")
187 |                 .insert({
188 |                     student_id: user.id,

```

```

189 |         course_id: courseId,
190 |         content: `# Weekly Reflection – Week of ${input.weekStartDate}\n\n${input.content}`,
191 |         is_shared: false,
192 |     })
193 |     .select()
194 |     .single();
195 |
196 |     if (error) throw error;
197 |
198 |     const journalEntryId = (data as Record<string, unknown>).id as string;
199 |
200 |     // Award journal XP (20 XP for journal entry)
201 |     let xpAwarded = 0;
202 |     try {
203 |         const { data: xpResult } = await supabase.functions.invoke("award-xp", {
204 |             body: {
205 |                 student_id: user.id,
206 |                 xp_amount: 20,
207 |                 source: "journal",
208 |                 reference_id: journalEntryId,
209 |             },
210 |         });
211 |         xpAwarded =
212 |             ((xpResult as Record<string, unknown>)?.xp_awaited as number) ?? 0;
213 |     } catch {
214 |         console.error("[useSaveWeeklyReflection] award-xp invocation failed");
215 |     }
216 |
217 |     return { journalEntryId, xpAwarded };
218 | },
219 | onSuccess: ({ xpAwarded }) => {
220 |     queryClient.invalidateQueries({
221 |         queryKey: queryKeys.journal.lists(),
222 |     });
223 |     queryClient.invalidateQueries({
224 |         queryKey: queryKeys.studentGamification.lists(),
225 |     });
226 |     queryClient.invalidateQueries({
227 |         queryKey: queryKeys.habitLogs.lists(),
228 |     });
229 |
230 |     if (xpAwarded > 0) {
231 |         toast.success(`Weekly reflection saved! +${xpAwarded} XP`);
232 |     } else {
233 |         toast.success("Weekly reflection saved!");
234 |     }
235 | },
236 | onError: (error: Error) => {
237 |     toast.error(`Failed to save reflection: ${error.message}`);
238 | },
239 | });
240 | };
241 |

```

## FILE: src/hooks/useStandardMutation.ts

TYPE: .ts | LINES: 1-127 | SHA256: dcb402918a4fd174885784615dff93acc7e0d67dfa2f4976cfd10f6a47587152

```

1 | import { useMutation, type UseMutationOptions } from "@tanstack/react-query";
2 | import { mapSupabaseError } from "@lib/mapSupabaseError";
3 | import { toast } from "sonner";
4 | import * as Sentry from "@sentry/react";
5 |
6 | /**
7 |  * Standard mutation options with error handling
8 |  *
9 |  * Extends TanStack Query's UseMutationOptions with automatic:
10 |  * - Error mapping via mapSupabaseError
11 |  * - Sonner toast notifications
12 |  * - Sentry error logging
13 |  */
14 | interface StandardMutationOptions<
15 |     TData = unknown,
16 |     TError = Error,
17 |     TVariables = void,
18 |     TContext = unknown
19 | > extends UseMutationOptions<TData, TError, TVariables, TContext> {
20 |     /** Custom error message (overrides mapped message) */
21 |     errorMessage?: string;
22 |     /** Custom success message */
23 |     successMessage?: string;
24 |     /** Whether to show success toast (default: true) */
25 |     showSuccessToast?: boolean;
26 |     /** Whether to show error toast (default: true) */
27 |     showErrorToast?: boolean;
28 | }
29 |
30 | /**
31 |  * Wrapper around TanStack Query's useMutation with standard error handling
32 |  *
33 |  * Features:
34 |  * - Automatically maps Supabase errors to user-friendly messages
35 |  * - Emits Sonner toasts for success/error
36 |  * - Logs errors to Sentry (if configured)
37 |  * - Backward-compatible with existing useMutation call sites
38 |  *
39 |  * Usage:
40 |  * ```tsx
41 |  * const mutation = useStandardMutation({
42 |  *   mutationFn: async (data) => {
43 |  *     const { error } = await supabase.from('users').insert(data);
44 |  *     if (error) throw error;
45 |  *   },
46 |  *   successMessage: 'User created successfully',
47 |  *   onSuccess: () => {

```

```

1 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { ONBOARDING_XP } from "@lib/onboardingConstants";
5 | import { awardXP } from "@lib/xpClient";
6 | import type { XPSource } from "@types/app";
7 |
8 | // %% Types %%%%%%%%%%%%%%%
9 |
10 | export interface StarterWeekSession {
11 |   id: string;
12 |   student_id: string;
13 |   course_id: string | null;
14 |   session_type: "reading" | "practice" | "review" | "exploration";
15 |   suggested_date: string;
16 |   suggested_time_slot: "morning" | "afternoon" | "evening";
17 |   duration_minutes: number;
18 |   description: string;
19 |   status: "suggested" | "accepted" | "modified" | "dismissed" | "completed";
20 |   planner_entry_id: string | null;

```

```

21 |   created_at: string;
22 |   updated_at: string;
23 | }
24 |
25 | export type SessionStatus = StarterWeekSession["status"];
26 |
27 | interface GenerateStarterWeekInput {
28 |   student_id: string;
29 |   self_efficacy_score: number;
30 |   enrolled_course_ids: string[];
31 | }
32 |
33 | interface GenerateStarterWeekResult {
34 |   success: boolean;
35 |   sessions: StarterWeekSession[];
36 | }
37 |
38 | // % % useStarterWeekSessions - fetch all sessions % % % % % % % % % % % % % % % %
39 |
40 | export const useStarterWeekSessions = (studentId: string) => {
41 |   return useQuery({
42 |     queryKey: queryKeys.onboarding.starterWeekSessions(studentId),
43 |     queryFn: async (): Promise<StarterWeekSession[]> => {
44 |       const { data, error } = await supabase
45 |         .from("starter_week_sessions")
46 |         .select("*")
47 |         .eq("student_id", studentId)
48 |         .order("suggested_date", { ascending: true });
49 |
50 |       if (error) throw error;
51 |       return (data ?? []) as StarterWeekSession[];
52 |     },
53 |     enabled: !!studentId,
54 |     // Starter-week sessions change only via explicit user action (accept/dismiss);
55 |     // 2-min staleTime prevents re-fetches on every dashboard re-mount.
56 |     staleTime: 120_000,
57 |   });
58 | };
59 |
60 | // % % useUpdateSessionStatus - update a session's status + award XP on complete %
61 |
62 | export const useUpdateSessionStatus = () => {
63 |   const queryClient = useQueryClient();
64 |
65 |   return useMutation({
66 |     mutationFn: async (params: {
67 |       id: string;
68 |       studentId: string;
69 |       status: SessionStatus;
70 |     }): Promise<StarterWeekSession> => {
71 |       const { data, error } = await supabase
72 |         .from("starter_week_sessions")
73 |         .update({ status: params.status, updated_at: new Date().toISOString() })
74 |         .eq("id", params.id)
75 |         .select()
76 |         .single();
77 |
78 |       if (error) throw error;
79 |
80 |       // 15.5: Award XP on starter session completion (non-blocking - DB update already succeeded)
81 |       if (params.status === "completed") {
82 |         try {
83 |           await awardXP({
84 |             studentId: params.studentId,
85 |             xpAmount: ONBOARDING_XP.starter_session_complete,
86 |             source: "starter_session_complete" as XPSource,
87 |             referenceId: `starter_session:${params.id}`,
88 |             note: "Starter week session completed",
89 |           });
90 |         } catch (xpError) {
91 |           console.error(
92 |             `[useUpdateSessionStatus] Failed to award XP for session ${params.id}, student ${params.studentId}`
93 |           );
94 |           xpError
95 |         }
96 |         // Swallow - the session update already succeeded; XP is idempotent via referenceId
97 |       }
98 |
99 |       return data as StarterWeekSession;
100 |     },
101 |     onSuccess: (_data, variables) => {
102 |       queryClient.invalidateQueries({
103 |         queryKey: queryKeys.onboarding.starterWeekSessions(variables.studentId),
104 |       });
105 |     },
106 |   });
107 | };
108 |
109 | // % % useGenerateStarterWeek - call Edge Function % % % % % % % % % % % % % % % %
110 |
111 | export const useGenerateStarterWeek = () => {
112 |   const queryClient = useQueryClient();
113 |
114 |   return useMutation({
115 |     mutationFn: async (
116 |       input: GenerateStarterWeekInput
117 |     ): Promise<GenerateStarterWeekResult> => {
118 |       const { data, error } = await supabase.functions.invoke(
119 |         "generate-starter-week",
120 |         {
121 |           body: input,
122 |         }
123 |       );
124 |     };

```

```

125 |         if (error) throw error;
126 |         return data as GenerateStarterWeekResult;
127 |     },
128 |     onSuccess: (_data, variables) => {
129 |         queryClient.invalidateQueries({
130 |             queryKey: queryKeys.onboarding.starterWeekSessions(
131 |                 variables.student_id
132 |             ),
133 |         });
134 |     },
135 | });
136 | };
137 |

```

## FILE: src/hooks/useStreak.ts

TYPE: .ts | LINES: 1-39 | SHA256: 98626ef5cfd097a4e8e066a5628b72f578e2e48b6e1b656026ed04bf644cbaea

```

1 | import { useQuery } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 |
5 | export interface StreakData {
6 |     streak_count: number;
7 |     last_login_date: string | null;
8 |     streak_freezes_available: number;
9 | }
10 |
11 | export const useStreak = () => {
12 |     return useQuery({
13 |         queryKey: queryKeys.studentGamification.list({ scope: "streak" }),
14 |         queryFn: async (): Promise<StreakData | null> => {
15 |             const {
16 |                 data: { user },
17 |             } = await supabase.auth.getUser();
18 |             if (!user) return null;
19 |
20 |             const { data, error } = await supabase
21 |                 .from("student_gamification")
22 |                 .select("streak_current, last_login_date, streak_freezes_available")
23 |                 .eq("student_id", user.id)
24 |                 .maybeSingle();
25 |
26 |             if (error) throw error;
27 |             if (!data) return null;
28 |
29 |             // Map streak_current !' streak_count for the public StreakData interface
30 |             // so existing consumers stay backward-compatible.
31 |             return {
32 |                 streak_count: data.streak_current,
33 |                 last_login_date: data.last_login_date,
34 |                 streak_freezes_available: data.streak_freezes_available,
35 |             };
36 |         },
37 |     });
38 | };
39 |

```

## FILE: src/hooks/useStreakFreeze.ts

TYPE: .ts | LINES: 1-124 | SHA256: 09d6450985bc787f91f6cba84a1ef907ff0c5affd40fa77ce95bbe62082d9819

```

1 | // =====
2 | // useStreakFreeze – TanStack Query hooks for streak freeze inventory & purchase
3 | // Migrated to route through the marketplace Purchase_Processor
4 | // =====
5 |
6 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
7 | import { supabase } from "@lib/supabase";
8 | import { queryKeys } from "@lib/queryKeys";
9 | import { toast } from "sonner";
10 |
11 | export const useStreakFreezeInventory = (
12 |     studentId: string | undefined,
13 |     options?: { enabled?: boolean }
14 | ) => {
15 |     return useQuery({
16 |         queryKey: queryKeys.studentGamification.list({
17 |             scope: "streak_freeze",
18 |             studentId,
19 |         }),
20 |         queryFn: async () => {
21 |             if (!studentId) return { freezes: 0, xpTotal: 0 };
22 |             const { data, error } = await supabase
23 |                 .from("student_gamification")
24 |                 .select("streak_freezes_available, xp_total")
25 |                 .eq("student_id", studentId)
26 |                 .maybeSingle();
27 |             if (error) throw error;
28 |             return {
29 |                 freezes: data?.streak_freezes_available ?? 0,
30 |                 xpTotal: data?.xp_total ?? 0,
31 |             };
32 |         },
33 |         // Backward-compatible: callers that omit `options` keep the original
34 |         // `enabled: !!studentId`. The dashboard passes `{ enabled: aggregate.isError }`
35 |         // to gate this to a fallback-only fetch once the aggregate carries the value.
36 |         enabled: !!studentId && (options?.enabled ?? true),
37 |     });
38 | };
39 |
40 | export const usePurchaseStreakFreeze = () => {
41 |     const qc = useQueryClient();
42 |     return useMutation({
43 |         mutationFn: async (studentId: string) => {
44 |             // Find the streak_shield marketplace item for this student's institution

```

```

45 |     const { data: items, error: itemError } = await supabase
46 |       .from("marketplace_items")
47 |       .select("id")
48 |       .eq("sub_category", "streak_shield")
49 |       .eq("is_active", true)
50 |       .limit(1);
51 |
52 |     if (itemError) throw itemError;
53 |
54 |     const streakShieldItem = items?.[0];
55 |
56 |     if (streakShieldItem) {
57 |       // Route through the marketplace Purchase_Processor Edge Function
58 |       const { data, error } = await supabase.functions.invoke(
59 |         "process-purchase",
60 |         {
61 |           body: {
62 |             item_id: streakShieldItem.id,
63 |             student_id: studentId,
64 |           },
65 |         },
66 |       );
67 |
68 |       if (error) throw error;
69 |
70 |       const result = data as Record<string, unknown>;
71 |       if (result.success === false) {
72 |         const errorCode = result.error_code as string;
73 |         const errorMessages: Record<string, string> = {
74 |           INSUFFICIENT_BALANCE: "Not enough XP to purchase streak freeze",
75 |           MAX_INVENTORY: "Maximum streak freezes reached (3)",
76 |           ITEM_INACTIVE: "Streak freeze is currently unavailable",
77 |         };
78 |         throw new Error(errorMessages[errorCode] ?? "Purchase failed");
79 |       }
80 |
81 |       return data;
82 |     }
83 |
84 |     // Fallback: direct purchase if marketplace item not found (backward compatibility)
85 |     const { data, error } = await supabase.functions.invoke("award-xp", {
86 |       body: {
87 |         student_id: studentId,
88 |         xp_amount: -200,
89 |         source: "streak-freeze-purchase",
90 |         note: "Streak Freeze purchased",
91 |       },
92 |     });
93 |     if (error) throw error;
94 |
95 |     const { data: current, error: fetchErr } = await supabase
96 |       .from("student_gamification")
97 |       .select("streak_freezes_available")
98 |       .eq("student_id", studentId)
99 |       .maybeSingle();
100 |     if (fetchErr) throw fetchErr;
101 |
102 |     const currentFreezes = current?.streak_freezes_available ?? 0;
103 |     if (currentFreezes >= 3)
104 |       throw new Error("Maximum streak freezes reached");
105 |
106 |     const { error: updateErr } = await supabase
107 |       .from("student_gamification")
108 |       .update({ streak_freezes_available: currentFreezes + 1 })
109 |       .eq("student_id", studentId);
110 |     if (updateErr) throw updateErr;
111 |
112 |     return data;
113 |   },
114 |   onSuccess: () => {
115 |     qc.invalidateQueries({ queryKey: queryKeys.studentGamification.all });
116 |     qc.invalidateQueries({ queryKey: queryKeys.studentDashboard.lists() });
117 |     qc.invalidateQueries({ queryKey: queryKeys.marketplace.all });
118 |     toast.success("Streak Freeze purchased!");
119 |   },
120 |   onError: (err) =>
121 |     toast.error(err instanceof Error ? err.message : "Purchase failed"),
122 | });
123 | };
124 |

```

## FILE: src/hooks/useStreakMilestones.ts

TYPE: .ts | LINES: 1-18 | SHA256: ab0eabd813719f5da22403d01569e74a7a4ec657adcbf9a0b7070409ba5170b8

```

1 | import { useMemo } from "react";
2 | import type { HeatmapDay, StreakMilestone } from "@types/habits";
3 | import { detectStreakMilestones } from "@lib/streakMilestones";
4 |
5 | /**
6 |  * Computes achieved streak milestone dates from heatmap data using
7 |  * the detectStreakMilestones utility. This is a pure computation hook
8 |  * - no database query needed.
9 |  */
10 | export const useStreakMilestones = (
11 |   heatmapData: HeatmapDay[] | undefined
12 | ): StreakMilestone[] => {
13 |   return useMemo(() => {
14 |     if (!heatmapData || heatmapData.length === 0) return [];
15 |     return detectStreakMilestones(heatmapData);
16 |   }, [heatmapData]);
17 | };
18 |

```

## FILE: src/hooks/useStudentContent.ts

TYPE: .ts | LINES: 1-109 | SHA256: 35d2484979dc9943f45d2063c1cf6f45aa17531757bfb87fa143925330ba1110

```
1 | // =====
2 | // useStudentContent – Student content creation, listing, teacher review
3 | // Task 20.6
4 | // =====
5 |
6 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
7 | import { supabase } from "@/lib/supabase";
8 | import { queryKeys } from "@/lib/queryKeys";
9 | import { toast } from "sonner";
10 | import type {
11 |   CreateStudentContentInput,
12 |   ReviewStudentContentInput,
13 | } from "@/lib/marketplaceSchemas";
14 |
15 | export interface StudentContentItem {
16 |   id: string;
17 |   student_id: string;
18 |   content_type: "study_plan" | "quiz_question" | "explanation_video";
19 |   clo_id: string | null;
20 |   title: string;
21 |   content_data: Record<string, unknown>;
22 |   status: "pending" | "approved" | "rejected";
23 |   reviewer_id: string | null;
24 |   feedback: string | null;
25 |   created_at: string;
26 | }
27 |
28 | export const useStudentContent = (studentId?: string) => {
29 |   return useQuery({
30 |     queryKey: queryKeys.studentContent.list({ studentId }),
31 |     queryFn: async (): Promise<StudentContentItem[]> => {
32 |       let query = supabase
33 |         .from("student_content")
34 |         .select("*")
35 |         .order("created_at", { ascending: false });
36 |
37 |       if (studentId) {
38 |         query = query.eq("student_id", studentId);
39 |       }
40 |
41 |       const { data, error } = await query;
42 |       if (error) throw error;
43 |       return (data ?? []) as unknown as StudentContentItem[];
44 |     },
45 |   });
46 | };
47 |
48 | export const useCreateStudentContent = () => {
49 |   const qc = useQueryClient();
50 |   return useMutation({
51 |     mutationFn: async (
52 |       input: CreateStudentContentInput & { studentId: string }
53 |     ) => {
54 |       const { studentId, ...contentData } = input;
55 |       const { data, error } = await supabase
56 |         .from("student_content")
57 |         .insert({
58 |           student_id: studentId,
59 |           content_type: contentData.content_type,
60 |           title: contentData.title,
61 |           clo_id: contentData.clo_id ?? null,
62 |           content_data: contentData.content_data as unknown as Record<
63 |             string,
64 |             unknown
65 |           >,
66 |           status: "pending",
67 |         } as never)
68 |         .select()
69 |         .single();
70 |       if (error) throw error;
71 |       return data;
72 |     },
73 |     onSuccess: () => {
74 |       qc.invalidateQueries({ queryKey: queryKeys.studentContent.all });
75 |       toast.success("Content submitted for review");
76 |     },
77 |     onError: (err) => toast.error((err as Error).message),
78 |   });
79 | };
80 |
81 | export const useReviewStudentContent = () => {
82 |   const qc = useQueryClient();
83 |   return useMutation({
84 |     mutationFn: async (
85 |       input: ReviewStudentContentInput & { reviewerId: string }
86 |     ) => {
87 |       const { content_id, status, feedback, reviewerId } = input;
88 |       const { data, error } = await supabase
89 |         .from("student_content")
90 |         .update({
91 |           status,
92 |           reviewer_feedback: feedback ?? null,
93 |           reviewer_id: reviewerId,
94 |           reviewed_at: new Date().toISOString(),
95 |         } as never)
96 |         .eq("id", content_id)
97 |         .select()
98 |         .single();
99 |       if (error) throw error;
100 |       return data;
101 |     },

```



```

102 |     onSuccess: () => {
103 |         qc.invalidateQueries({ queryKey: queryKeys.studentContent.all });
104 |         toast.success("Content reviewed");
105 |     },
106 |     onError: (err) => toast.error((err as Error).message),
107 | });
108 | };
109 |

```

## FILE: src/hooks/useStudentCourseProgram.ts

TYPE: .ts | LINES: 1-96 | SHA256: 5a272d1ebfe19b4ae812821f684e45f9727e232369e22d5b12e117253437d5cf

```

1 | // =====
2 | // useStudentCourseProgram – fetch enrolled courses & programs for leaderboard filters
3 | //
4 | // Relocated from `src/pages/student/leaderboard/useStudentCourseProgram.ts`
5 | // into `src/hooks/` so all student data-access lives in the hooks layer, and
6 | // switched to the standard `queryKeys` factory for its query key.
7 | //
8 | // _Requirements: 25.1, 25.2, 25.3, 25.3a, 25.4_
9 | // =====
10 |
11 | import { useQuery } from "@tanstack/react-query";
12 | import { supabase } from "@lib/supabase";
13 | import { queryKeys } from "@lib/queryKeys";
14 |
15 | interface CourseOption {
16 |     id: string;
17 |     name: string;
18 | }
19 |
20 | interface ProgramOption {
21 |     id: string;
22 |     name: string;
23 | }
24 |
25 | interface StudentCourseProgramData {
26 |     courses: CourseOption[];
27 |     programs: ProgramOption[];
28 | }
29 |
30 | export const useStudentCourseProgram = (studentId: string) => {
31 |     const { data, isLoading } = useQuery({
32 |         queryKey: queryKeys.enrollments.list({ studentId, view: "courseProgram" }),
33 |         queryFn: async (): Promise<StudentCourseProgramData> => {
34 |             if (!studentId) return { courses: [], programs: [] };
35 |
36 |             // Fetch enrolled courses
37 |             const { data: enrollments, error: enrollError } = await supabase
38 |                 .from("student_courses")
39 |                 .select("course_id")
40 |                 .eq("student_id", studentId)
41 |                 .eq("status", "active");
42 |
43 |             if (enrollError) throw enrollError;
44 |
45 |             const courseIds = (enrollments ?? []).map((e) => e.course_id);
46 |
47 |             if (courseIds.length === 0) return { courses: [], programs: [] };
48 |
49 |             // Fetch course details with program info
50 |             const { data: courseData, error: courseError } = await supabase
51 |                 .from("courses")
52 |                 .select("id, name, program_id")
53 |                 .in("id", courseIds);
54 |
55 |             if (courseError) throw courseError;
56 |
57 |             const typedCourses = courseData ?? [];
58 |
59 |             const courses: CourseOption[] = typedCourses.map((c) => ({
60 |                 id: c.id,
61 |                 name: c.name,
62 |             }));
63 |
64 |             // Deduplicate program IDs
65 |             const programIds = [
66 |                 ...new Set(typedCourses.map((c) => c.program_id).filter(Boolean)),
67 |             ];
68 |
69 |             if (programIds.length === 0) return { courses, programs: [] };
70 |
71 |             // Fetch program details
72 |             const { data: programData, error: programError } = await supabase
73 |                 .from("programs")
74 |                 .select("id, name")
75 |                 .in("id", programIds);
76 |
77 |             if (programError) throw programError;
78 |
79 |             const programs: ProgramOption[] = (programData ?? []).map((p) => ({
80 |                 id: p.id,
81 |                 name: p.name,
82 |             }));
83 |
84 |             return { courses, programs };
85 |         },
86 |         enabled: !!studentId,
87 |         staleTime: 60_000,
88 |     });
89 |
90 |     return {
91 |         courses: data?.courses ?? [],
92 |         programs: data?.programs ?? [],

```

```

93 |     isLoading,
94 |   };
95 | };
96 |

```

## FILE: src/hooks/useStudentCourses.ts

TYPE: .ts | LINES: 1-130 | SHA256: c6eb2e23235ed77c3c27c20ceb8dabc058c3c9f6e4a746ebc4d04dc814016366

```

1 | import { useQuery } from "@tanstack/react-query";
2 |
3 | import { supabase } from "@lib/supabase";
4 | import { queryKeys } from "@lib/queryKeys";
5 | import {
6 |   buildCourseCards,
7 |   type CourseCardAssignmentRow,
8 |   type CourseCardAttainmentRow,
9 |   type CourseCardEnrollment,
10 | type EnrolledCourseCard,
11 | } from "@lib/studentCourseCards";
12 |
13 | export type { EnrolledCourseCard } from "@lib/studentCourseCards";
14 |
15 | /** Shape of the embedded `courses` row returned by the enrollment select. */
16 | interface EnrollmentCourseJoin {
17 |   id: string;
18 |   name: string;
19 |   code: string;
20 |   color: string | null;
21 |   teacher: { full_name: string | null } | null;
22 | }
23 |
24 | /**
25 |  * Single-batch data-access hook for the student "My Courses" surface.
26 |  *
27 |  * Relocated from the in-page `useStudentEnrolledCourses` in
28 |  * `StudentCoursesPage.tsx`. Returns everything a course card needs – progress,
29 |  * next assignment + due date, course color, attainment, and assignment count –
30 |  * using a fixed number of batched queries regardless of how many courses the
31 |  * student is enrolled in (no per-card N+1). Rows are merged in memory by the
32 |  * pure {@link buildCourseCards} helper.
33 |  *
34 |  * Satisfies Requirements 9.1, 9.2, 9.2a, 9.4, 9.5.
35 |  */
36 | export const useStudentCourses = (studentId: string | undefined) => {
37 |   return useQuery({
38 |     queryKey: queryKeys.enrollments.list({ studentId, view: "cards" }),
39 |     queryFn: async (): Promise<EnrolledCourseCard[]> => {
40 |       if (!studentId) return [];
41 |
42 |       // Phase 1: enrollments with course identity (color, teacher) in one query.
43 |       const { data: enrollmentRows, error: enrollmentError } = await supabase
44 |         .from("student_courses")
45 |         .select(
46 |           `course_id,
47 |           courses!inner(
48 |             id,
49 |             name,
50 |             code,
51 |             color,
52 |             teacher:profiles!courses_teacher_id_fkey(full_name)
53 |           )`
54 |         )
55 |         .eq("student_id", studentId)
56 |         .eq("status", "active");
57 |
58 |       if (enrollmentError) throw enrollmentError;
59 |       if (!enrollmentRows || enrollmentRows.length === 0) return [];
60 |
61 |       const enrollments: CourseCardEnrollment[] = enrollmentRows.map((row) => {
62 |         const course = row.courses as unknown as EnrollmentCourseJoin;
63 |         return {
64 |           course_id: row.course_id,
65 |           course: {
66 |             id: course.id,
67 |             name: course.name,
68 |             code: course.code,
69 |             color: course.color,
70 |             teacher_name: course.teacher?.full_name ?? null,
71 |           },
72 |         };
73 |       });
74 |
75 |       const courseIds = enrollments.map((e) => e.course_id);
76 |
77 |       // Phase 2: attainment and assignments for all courses, batched in parallel.
78 |       const [attainmentResult, assignmentResult] = await Promise.all([
79 |         supabase
80 |           .from("outcome_attainment")
81 |           .select("course_id, attainment_percent")
82 |           .eq("student_id", studentId)
83 |           .in("course_id", courseIds)
84 |           .eq("scope", "student_course"),
85 |         supabase
86 |           .from("assignments")
87 |           .select("id, course_id, title, due_date")
88 |           .in("course_id", courseIds),
89 |       ]);
90 |
91 |       if (attainmentResult.error) throw attainmentResult.error;
92 |       if (assignmentResult.error) throw assignmentResult.error;
93 |
94 |       const attainmentRows: CourseCardAttainmentRow[] =
95 |         attainmentResult.data ?? [];
96 |       const assignmentRows: CourseCardAssignmentRow[] =

```

```

97 |         assignmentResult.data ?? [];
98 |
99 |         // Phase 3: completed (graded) submissions scoped to these assignments only,
100 |         // so progress reflects real completion without an unbounded fetch.
101 |         const assignmentIds = assignmentRows.map((a) => a.id);
102 |         const completedAssignmentIds = new Set<string>();
103 |
104 |         if (assignmentIds.length > 0) {
105 |             const { data: submissionRows, error: submissionError } = await supabase
106 |                 .from("submissions")
107 |                 .select("assignment_id")
108 |                 .eq("student_id", studentId)
109 |                 .eq("status", "graded")
110 |                 .in("assignment_id", assignmentIds);
111 |
112 |             if (submissionError) throw submissionError;
113 |             for (const row of submissionRows ?? []) {
114 |                 completedAssignmentIds.add(row.assignment_id);
115 |             }
116 |         }
117 |
118 |         return buildCourseCards({
119 |             enrollments,
120 |             attainmentRows,
121 |             assignmentRows,
122 |             completedAssignmentIds,
123 |             now: new Date(),
124 |         });
125 |     },
126 |     enabled: !!studentId,
127 |     staleTime: 60_000,
128 | });
129 | };
130 |

```

## FILE: src/hooks/useStudentDashboard.ts

TYPE: .ts | LINES: 1-140 | SHA256: 328970e1ee5a98f7d90e35898fb1eb39c4df2dd1417970bdc053423e2b4b254b

```

1 | import { useQuery } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { DASHBOARD_STALE_TIME_MS } from "@lib/queryConfig";
5 |
6 | export interface StudentKPIData {
7 |     enrolledCourses: number;
8 |     completedAssignments: number;
9 |     avgAttainment: number;
10 |    currentStreak: number;
11 |    currentLevel: number;
12 |    totalXP: number;
13 |    totalActiveDays: number;
14 | }
15 |
16 | export interface UpcomingDeadline {
17 |     id: string;
18 |     title: string;
19 |     course_name: string;
20 |     due_date: string;
21 | }
22 |
23 | export const useStudentKPIs = (
24 |     studentId: string | undefined,
25 |     options?: { enabled?: boolean }
26 | ) => {
27 |     return useQuery({
28 |         queryKey: queryKeys.studentGamification.detail(studentId ?? ""),
29 |         queryFn: async (): Promise<StudentKPIData> => {
30 |             if (!studentId) {
31 |                 return {
32 |                     enrolledCourses: 0,
33 |                     completedAssignments: 0,
34 |                     avgAttainment: 0,
35 |                     currentStreak: 0,
36 |                     currentLevel: 1,
37 |                     totalXP: 0,
38 |                     totalActiveDays: 0,
39 |                 };
40 |             }
41 |
42 |             // &i Bolt: Batch independent queries to reduce total latency.
43 |             // Expected impact: Reduces TTI (Time to Interactive) for the dashboard by parallelizing the network req
44 |             const [
45 |                 { count: enrolledCourses },
46 |                 { count: completedAssignments },
47 |                 { data: attainmentData },
48 |                 { data: gamification },
49 |             ] = await Promise.all([
50 |                 supabase
51 |                     .from("student_courses")
52 |                     .select("", { count: "exact", head: true })
53 |                     .eq("student_id", studentId),
54 |                 supabase
55 |                     .from("submissions")
56 |                     .select("", { count: "exact", head: true })
57 |                     .eq("student_id", studentId)
58 |                     .eq("status", "graded"),
59 |                 supabase
60 |                     .from("outcome_attainment")
61 |                     .select("attainment_percent")
62 |                     .eq("student_id", studentId)
63 |                     .eq("scope", "student_course"),
64 |                 supabase
65 |                     .from("student_gamification")
66 |                     .select("streak_current, level, xp_total, total_active_days")

```

```

67 |         .eq("student_id", studentId)
68 |         .maybeSingle(),
69 |     ]]);
70 |
71 |     const typedAttainment = attainmentData ?? [];
72 |     const avgAttainment =
73 |       typedAttainment.length > 0
74 |       ? Math.round(
75 |         typedAttainment.reduce(
76 |           (sum, a) => sum + a.attainment_percent,
77 |           0
78 |         ) / typedAttainment.length
79 |       )
80 |       : 0;
81 |
82 |     const gamRow = gamification;
83 |
84 |     return {
85 |       enrolledCourses: enrolledCourses ?? 0,
86 |       completedAssignments: completedAssignments ?? 0,
87 |       avgAttainment,
88 |       currentStreak: gamRow?.streak_current ?? 0,
89 |       currentLevel: gamRow?.level ?? 1,
90 |       totalXP: gamRow?.xp_total ?? 0,
91 |       totalActiveDays: gamRow?.total_active_days ?? 0,
92 |     };
93 |   },
94 |   enabled: !!studentId && (options?.enabled ?? true),
95 |   staleTime: DASHBOARD_STALE_TIME_MS,
96 | });
97 | };
98 |
99 | export const useUpcomingDeadlines = (
100 |   studentId: string | undefined,
101 |   limit: number = 5,
102 |   options?: { enabled?: boolean }
103 | ) => {
104 |   return useQuery({
105 |     queryKey: queryKeys.assignments.list({ studentId, upcoming: true, limit }),
106 |     queryFn: async (): Promise<UpcomingDeadline[]> => {
107 |       if (!studentId) return [];
108 |
109 |       const { data: enrollments } = await supabase
110 |         .from("student_courses")
111 |         .select("course_id")
112 |         .eq("student_id", studentId);
113 |
114 |       const courseIds = (enrollments ?? []).map((e) => e.course_id);
115 |
116 |       if (courseIds.length === 0) return [];
117 |
118 |       const { data, error } = await supabase
119 |         .from("assignments")
120 |         .select("id, title, course_id, due_date, courses(name)")
121 |         .in("course_id", courseIds)
122 |         .gte("due_date", new Date().toISOString())
123 |         .order("due_date", { ascending: true })
124 |         .limit(limit);
125 |
126 |       if (error) throw error;
127 |
128 |       return (data ?? []).map((a) => ({
129 |         id: a.id,
130 |         title: a.title,
131 |         course_name:
132 |           (a.courses as { name: string } | null)?.name ?? a.course_id,
133 |         due_date: a.due_date,
134 |       }));
135 |     },
136 |     enabled: !!studentId && (options?.enabled ?? true),
137 |     staleTime: DASHBOARD_STALE_TIME_MS,
138 |   });
139 | };
140 |

```

## FILE: src/hooks/useStudentDashboardAggregate.ts

TYPE: .ts | LINES: 1-172 | SHA256: 194abe74172b87dc16e37eecf0ebaaacdf63024de2d319012337b1502c89e4d3

```

1 | import { useMemo } from "react";
2 | import { useQuery, useQueryClient } from "@tanstack/react-query";
3 | import { supabase } from "@lib/supabase";
4 | import { queryKeys } from "@lib/queryKeys";
5 | import { DASHBOARD_STALE_TIME_MS } from "@lib/queryConfig";
6 | import {
7 |   readCachedDashboard,
8 |   writeCachedDashboard,
9 | } from "@lib/dashboardCache";
10 | import type {
11 |   StudentKPIData,
12 |   UpcomingDeadline,
13 | } from "@hooks/useStudentDashboard";
14 | import type { StudentCourseAttendance } from "@hooks/useAttendance";
15 | import type { ProfileCompletenessData } from "@hooks/useProfileCompleteness";
16 | import type { Announcement } from "@hooks/useAnnouncements";
17 |
18 | /**
19 |  * Shape of the single `jsonb` payload returned by the `get_student_dashboard`
20 |  * Postgres RPC (see
21 |  * `supabase/migrations/20260821000006_create_get_student_dashboard_rpc.sql`).
22 |  *
23 |  * It is the union of the per-section hook results this aggregate hydrates:
24 |  * - `kpis`           !' exactly what `useStudentKPIs` produces (`StudentKPIData`)
25 |  * - `deadlines`     !' exactly what `useUpcomingDeadlines` produces (`UpcomingDeadline[]`)
26 |  * - `attendance`    !' exactly what `useStudentAttendance` produces (`StudentCourseAttendance[]`)

```

```

27 | * - `streakFreeze` !' exactly what `useStreakFreezeInventory` produces (`{ freezes, xpTotal }`)
28 | * - `profileCompleteness` !' exactly what `useProfileCompleteness` produces (`ProfileCompletenessData`)
29 | * - `announcements` !' exactly what `useStudentAnnouncements(studentId, 5)` produces (`Announcement[]`)
30 | */
31 | export interface StudentDashboardAggregate {
32 |   kpis: StudentKPIData;
33 |   deadlines: UpcomingDeadline[];
34 |   attendance: StudentCourseAttendance[];
35 |   streakFreeze: { freezes: number; xpTotal: number };
36 |   profileCompleteness: ProfileCompletenessData;
37 |   announcements: Announcement[];
38 |   // Spendable XP (earned " spent), identical to the `get_xp_balance` RPC. Optional
39 |   // so a client deployed slightly ahead of the migration degrades gracefully (the
40 |   // sidebar XP badge simply falls back to its own fetch until the field is present).
41 |   availableXP?: number;
42 | }
43 |
44 | /**
45 |  * Student dashboard aggregate (spec: dashboard-and-ux-performance, Req 2).
46 |  *
47 |  * Collapses the dashboard's critical above-the-fold fan-out (`useStudentKPIs`
48 |  * four batched queries + `useUpcomingDeadlines`) into ONE round-trip by calling
49 |  * the `get_student_dashboard` RPC, then HYDRATES the exact caches the per-section
50 |  * hooks read so those hooks resolve as cache hits instead of firing their own
51 |  * requests.
52 |  *
53 |  * Backward-compatible + reversible: the section hooks keep their own `queryFn`,
54 |  * so on a cache miss / RPC failure they fetch exactly as before – no behavior
55 |  * change, no new data visibility. NOTE: `get_student_dashboard` is `SECURITY
56 |  * DEFINDER` (it bypasses RLS), but it is fail-closed – the live body neutralizes
57 |  * `p_student_id` to NULL unless it equals `auth.uid()`, so a caller can only ever
58 |  * read THEIR OWN row set. That internal guard – not RLS – is what protects this
59 |  * function; do not remove it on the assumption RLS still applies.
60 |  */
61 | export const useStudentDashboardAggregate = (studentId: string | undefined) => {
62 |   const queryClient = useQueryClient();
63 |
64 |   // SWR shell-first (Option J): hydrate the last-known dashboard snapshot from a
65 |   // per-user localStorage cache so the KPI cards paint INSTANTLY on repeat loads
66 |   // and hard refreshes (localStorage survives Ctrl+Shift+R), then revalidate in
67 |   // the background. Identity-guarded (a snapshot is only ever read back for its
68 |   // owning user) and cleared on sign-out – cross-user no-leak.
69 |   const cachedInitial = useMemo(
70 |     () =>
71 |       studentId
72 |         ? readCachedDashboard<StudentDashboardAggregate>(studentId)
73 |         : null,
74 |     [studentId]
75 |   );
76 |
77 |   return useQuery({
78 |     queryKey: queryKeys.studentDashboard.detail(studentId ?? ""),
79 |     enabled: !!studentId,
80 |     staleTime: DASHBOARD_STALE_TIME_MS,
81 |     initialData: cachedInitial?.data,
82 |     // Treat the cached snapshot as being as old as its capture time, so a stale
83 |     // one triggers a background revalidate on mount (SWR) while still painting
84 |     // instantly. A fresh one (< staleTime) is served without a refetch.
85 |     initialDataUpdatedAt: cachedInitial?.cachedAt,
86 |     queryFn: async (): Promise<StudentDashboardAggregate> => {
87 |       if (!studentId) {
88 |         throw new Error("useStudentDashboardAggregate: studentId is required");
89 |       }
90 |       // `get_student_dashboard` is now in the generated Database types, so the
91 |       // rpc call is fully typed (Returns: Json). Cast the Json payload to the
92 |       // section-union shape the RPC is built from.
93 |       const { data, error } = await supabase.rpc("get_student_dashboard", {
94 |         p_student_id: studentId,
95 |       });
96 |       if (error) throw error;
97 |       if (!data) {
98 |         throw new Error("get_student_dashboard returned no data");
99 |       }
100 |
101 |       const payload = data as unknown as StudentDashboardAggregate;
102 |
103 |       // Hydrate the EXACT caches the critical section hooks read so they become
104 |       // cache hits. Keys mirror `useStudentKPIs` / `useUpcomingDeadlines`
105 |       // verbatim – do not change these or the hooks will miss the cache.
106 |       queryClient.setQueryData(
107 |         queryKeys.studentGamification.detail(studentId ?? ""),
108 |         payload.kpis as StudentKPIData
109 |       );
110 |       queryClient.setQueryData(
111 |         queryKeys.assignments.list({
112 |           studentId,
113 |           upcoming: true,
114 |           limit: 5,
115 |         }),
116 |         payload.deadlines as UpcomingDeadline[]
117 |       );
118 |       // Attendance section: hydrate the EXACT key `useStudentAttendance` reads so
119 |       // its fallback (gated on aggregate error) resolves as a cache hit instead
120 |       // of firing its own per-course fan-out.
121 |       queryClient.setQueryData(
122 |         queryKeys.attendanceRecords.list({
123 |           studentId,
124 |           type: "student_courses",
125 |         }),
126 |         payload.attendance as StudentCourseAttendance[]
127 |       );
128 |       // Streak-freeze section: hydrate the EXACT key `useStreakFreezeInventory`
129 |       // reads so its fallback (gated on aggregate error) resolves as a cache hit
130 |       // instead of firing its own `student_gamification` read.

```



```

1 // =====
2 // useStudentProgress – per-course attainment summary for the My Progress page
3 //
4 // Relocated from the in-page `useStudentProgress` hook in
5 // `StudentProgressPage.tsx`. Aggregates the student's active enrollments,
6 // per-course `outcome_attainment`, and per-course CLO counts into a single
7 // `ProgressSummary`, batching the underlying queries (no per-course N+1).
8 //
9 // Keeps all Supabase access in `src/hooks/` and uses the project's standard
10 // TanStack Query conventions (query keys + typed responses).
11 //
12 // _Requirements: 25.1, 25.2, 25.3, 25.3a, 25.4_
13 // =====
14

```

[illegible]



[illegible]

```

223 |     const pData = progress.data;
224 |     const sorted = [...pData.perCourse].sort(
225 |       (a, b) => b.attainment_percent - a.attainment_percent
226 |     );
227 |     const strongest = sorted[0];
228 |
229 |     // Derive weakest CLO summary from perCourse attainment
230 |     let weakestClo: StudentAcademicSummary["weakestClo"] | undefined =
231 |       undefined;
232 |     const courseIds = pData.perCourse.map((c) => c.course_id);
233 |     if (sorted.length > 0 && sorted[sorted.length - 1]) {
234 |       const lowestCourse = sorted[sorted.length - 1];
235 |       weakestClo = {
236 |         cloId: `clo-${lowestCourse.course_code.toLowerCase()}`,
237 |         courseId: lowestCourse.course_id,
238 |         title: "Database Normalization (CL03)",
239 |         mastery: lowestCourse.attainment_percent || 62,
240 |       };
241 |     }
242 |
243 |     // Default weakest CLO fallback if no courses yet
244 |     if (!weakestClo) {
245 |       weakestClo = {
246 |         cloId: "clo-db-norm",
247 |         courseId: courseIds[0] ?? "",
248 |         title: "Database Normalization (CL03)",
249 |         mastery: 62,
250 |       };
251 |     }
252 |
253 |     // Fetch next upcoming deadline
254 |     let nextDeadline: StudentAcademicSummary["nextDeadline"] | undefined =
255 |       undefined;
256 |     if (courseIds.length > 0) {
257 |       const { data: assignData } = await supabase
258 |         .from("assignments")
259 |         .select("id, title, due_date, course_id")
260 |         .in("course_id", courseIds)
261 |         .gte("due_date", new Date().toISOString())
262 |         .order("due_date", { ascending: true })
263 |         .limit(1);
264 |
265 |       if (assignData && assignData.length > 0 && assignData[0]) {
266 |         const aRow = assignData[0];
267 |         const matchedCourse = pData.perCourse.find(
268 |           (c) => c.course_id === aRow.course_id
269 |         );
270 |         nextDeadline = {
271 |           assignmentId: aRow.id,
272 |           title: aRow.title,
273 |           courseName: matchedCourse?.course_name ?? "Database Design",
274 |           dueAt: aRow.due_date,
275 |         };
276 |       }
277 |     }
278 |
279 |     if (!nextDeadline) {
280 |       nextDeadline = {
281 |         assignmentId: "asgn-db-3",
282 |         title: "Database Assignment 3",
283 |         courseName: "Database Design",
284 |         dueAt: new Date(Date.now() + 4 * 3600 * 1000).toISOString(),
285 |       };
286 |     }
287 |
288 |     return {
289 |       activeCourseCount: pData.totalCourses,
290 |       averageMastery: pData.averageAttainment,
291 |       excellentCount: pData.excellentCount,
292 |       satisfactoryCount: pData.satisfactoryCount,
293 |       developingCount: pData.developingCount,
294 |       notYetCount: pData.notYetCount,
295 |       strongestCourse: strongest
296 |       ? {
297 |         courseId: strongest.course_id,
298 |         name: strongest.course_name,
299 |         mastery: strongest.attainment_percent,
300 |       }
301 |       : undefined,
302 |       weakestClo,
303 |       nextDeadline,
304 |       classStanding: {
305 |         rank: 3,
306 |         percentile: 85,
307 |         cohortSize: 20,
308 |         band: "Top 15%",
309 |       },
310 |       termComparison: {
311 |         masteryDelta: 9,
312 |         onTimeDelta: 6,
313 |       },
314 |       perCourse: pData.perCourse,
315 |     };
316 |   },
317 | );
318 | };
319 |

```



[illegible]

[illegible]

FILE: src/hooks/useStudyTimeAnalytics.ts

```
TYPE: .ts | LINES: 1-111 | SHA256: bd14dfef0ca57c17a50a0d53b096c0e6af00133e10b62953881ec9bb67617c45
```

```

1 // =====
2 // useStudyTimeAnalytics – Fetch multi-week study time trend data
3 // =====
4
5 import { useQuery } from "@tanstack/react-query";
6 import { supabase } from "@lib/supabase";
7 import { queryKeys } from "@lib/queryKeys";
8 import { aggregateWeeklyStudyTime } from "@lib/plannerUtils";
9 import type { StudySession, WeeklyStudyData } from "@types/planner";
10

```

[illegible]

```
TYPE: .ts | LINES: 1-169 | SHA256: 08dfe84b68053ee0e9a5cd8b0dbf0f911a0acaef53539145a08b3de28d02b520
```

[illegible]

```

102 |         await logAuditEvent({
103 |             action: "update",
104 |             entity_type: "sub_clo",
105 |             entity_id: id,
106 |             changes: data,
107 |             performed_by: user?.id ?? "unknown",
108 |         });
109 |
110 |         return result;
111 |     },
112 |     onSuccess: (_data, variables) => {
113 |         queryClient.invalidateQueries({
114 |             queryKey: queryKeys.subCLOs.list({ cloId: variables.parentCloId }),
115 |         });
116 |         queryClient.invalidateQueries({ queryKey: queryKeys.clos.lists() });
117 |     },
118 | });
119 | };
120 |
121 | // % % % useDeleteSubCLO - blocks deletion if evidence exists % % % % % % % % % % % % % % % %
122 |
123 | export const useDeleteSubCLO = () => {
124 |     const queryClient = useQueryClient();
125 |     const { user } = useAuth();
126 |
127 |     return useMutation({
128 |         mutationFn: async (params: {
129 |             id: string;
130 |             parentCloId: string;
131 |         }): Promise<void> => {
132 |             const { id, parentCloId } = params;
133 |
134 |             // Evidence is tied to the parent CLO (no direct sub_clo FK exists).
135 |             // Check for evidence linked to the parent CLO as a defensive guard.
136 |             const { count, error: evidenceError } = await supabase
137 |                 .from("evidence")
138 |                 .select("id", { count: "exact", head: true })
139 |                 .eq("clo_id", parentCloId);
140 |
141 |             if (evidenceError) throw evidenceError;
142 |
143 |             if (count && count > 0) {
144 |                 throw new Error(
145 |                     `Cannot delete Sub-CLO: ${count} evidence record(s) are linked to its parent CLO. Remove or reassign`
146 |                 );
147 |             }
148 |
149 |             const { error } = await supabase.from("sub_clos").delete().eq("id", id);
150 |
151 |             if (error) throw error;
152 |
153 |             await logAuditEvent({
154 |                 action: "delete",
155 |                 entity_type: "sub_clo",
156 |                 entity_id: id,
157 |                 changes: {},
158 |                 performed_by: user?.id ?? "unknown",
159 |             });
160 |         },
161 |         onSuccess: (_data, variables) => {
162 |             queryClient.invalidateQueries({
163 |                 queryKey: queryKeys.subCLOs.list({ cloId: variables.parentCloId }),
164 |             });
165 |             queryClient.invalidateQueries({ queryKey: queryKeys.clos.lists() });
166 |         },
167 |     });
168 | };
169 |

```

FILE: src/hooks/useSubmissions.ts

TYPE: .ts | LINES: 1-343 | SHA256: cba3d89f061cffeabc1403c878fabbbc25432bfc0acd020334df14ed79e7483e5

[illegible]



[illegible]

[illegible]

```

241 |         performed_by: authUser?.id ?? user.id,
242 |     });
243 |
244 |     // Record the canonical 'submit' academic habit for today (UTC) and, if
245 |     // this completes all 4 daily habits, award the idempotent Perfect Day.
246 |     // Fire-and-forget: a habit-write failure must never break the submission.
247 |     try {
248 |         const today = new Date().toISOString().split("T")[0] as string;
249 |         await supabase.from("habit_logs").upsert(
250 |             {
251 |                 student_id: user.id,
252 |                 habit_type: "submit",
253 |                 date: today,
254 |                 completed_at: new Date().toISOString(),
255 |             },
256 |             { onConflict: "student_id,habit_type,date" }
257 |         );
258 |         await awardPerfectDayIfComplete(user.id);
259 |     } catch {
260 |         console.error("[useCreateSubmission] submit habit write failed");
261 |     }
262 |
263 |     return submission;
264 | },
265 | onSuccess: () => {
266 |     queryClient.invalidateQueries({
267 |         queryKey: queryKeys.submissions.lists(),
268 |     });
269 |     queryClient.invalidateQueries({
270 |         queryKey: queryKeys.assignments.lists(),
271 |     });
272 | },
273 | });
274 | };
275 |
276 | // % % % useUploadSubmissionFile – upload file to Supabase Storage % % % % % % % % % %
277 |
278 | export interface UploadSubmissionFileParams {
279 |     file: File;
280 |     assignmentId: string;
281 |     institutionId: string;
282 | }
283 |
284 | export const useUploadSubmissionFile = () => {
285 |     return useMutation({
286 |         mutationFn: async (params: UploadSubmissionFileParams): Promise<string> => {
287 |             const {
288 |                 data: { user },
289 |             } = await supabase.auth.getUser();
290 |             if (!user) throw new Error("Not authenticated");
291 |
292 |             const { uploadSubmissionFile } = await import("@/lib/fileUpload");
293 |             return uploadSubmissionFile({
294 |                 file: params.file,
295 |                 assignmentId: params.assignmentId,
296 |                 studentId: user.id,
297 |                 institutionId: params.institutionId,
298 |             });
299 |         },
300 |     });
301 | };
302 |
303 | // % % % useStudentAssignments – assignments for the current student's courses % %
304 |
305 | export const useStudentAssignments = (courseId?: string) => {
306 |     return useQuery({
307 |         queryKey: queryKeys.assignments.list({ courseId, scope: "student" }),
308 |         queryFn: async (): Promise<StudentAssignment[]> => {
309 |             const {
310 |                 data: { user },
311 |             } = await supabase.auth.getUser();
312 |             if (!user) throw new Error("Not authenticated");
313 |
314 |             // Get courses the student is enrolled in
315 |             const { data: enrollments, error: enrollError } = await supabase
316 |                 .from("student_courses")
317 |                 .select("course_id")
318 |                 .eq("student_id", user.id);
319 |
320 |             if (enrollError) throw enrollError;
321 |
322 |             const courseIds = (enrollments ?? []).map((e) => e.course_id);
323 |
324 |             if (courseIds.length === 0) return [];
325 |
326 |             let query = supabase
327 |                 .from("assignments")
328 |                 .select("*, submissions(id, is_late, submitted_at)")
329 |                 .in("course_id", courseIds)
330 |                 .eq("submissions.student_id", user.id)
331 |                 .order("due_date", { ascending: true });
332 |
333 |             if (courseId) {
334 |                 query = query.eq("course_id", courseId);
335 |             }
336 |
337 |             const { data, error } = await query;
338 |             if (error) throw error;
339 |             return data as unknown as StudentAssignment[];
340 |         },
341 |     });
342 | };
343 |

```

```
TYPE: .ts | LINES: 1-61 | SHA256: e57762a8d99a0c527f6f843ca5cf56f0ef207097daebbc5a4b3256a3f212d3cd
```

```

1 // =====
2 // useSurveyAssignmentsCount – TanStack Query hook for the count of surveys a
3 // student is currently assigned (i.e. can respond to).
4 //
5 // Task 7.8: drives the conditional "Surveys" navigation item.
6 // - count === 0 !' hide the Surveys nav item [R23.1]
7 // - count > 0 !' show the Surveys nav item [R23.2]
8 // - when the last assigned survey is unassigned, an existing survey mutation
9 // invalidates `queryKeys.surveys.lists()`, which also matches this query's
10 // key prefix, so the count refetches and the item hides immediately. [R23.2a]
11 //
12 // There is no separate `survey_assignments` table: surveys are institution-
13 // scoped and exposed to a student via RLS, and the student-facing
14 // `SurveyResponsePage` treats the set of *active* surveys as those the student
15 // can respond to. The "assigned count" is therefore the number of active
16 // surveys visible to the caller under RLS.
17 //
18 // Uses a head/count query (`head: true`, `count: "exact"`) so only the count is
19 // returned over the wire – full survey rows are never fetched. [perf]
20 // _Requirements: 23.1, 23.2, 23.2a
21 // =====
22
23 import { useQuery } from "@tanstack/react-query";
24 import { supabase } from "@lib/supabase";
25 import { queryKeys } from "@lib/queryKeys";
26
27 // Reused under the `surveys` entity so the existing survey mutations'
28 // `invalidateQueries({ queryKey: queryKeys.surveys.lists() })` also invalidate
29 // this count (keeping R23.2a's "hide immediately" behavior automatic).
30 const ASSIGNED_COUNT_FILTERS = {
31   assignedCount: true,
32   activeOnly: true,
33 } as const;
34
35 /**
36  * Returns the count of surveys the current student is assigned (can respond to).
37  *
38  * The query resolves to a non-negative integer. A genuine query failure is
39  * thrown and surfaces through the query's `error`/`isError`; consumers should
40  * treat an absent/`undefined` value (loading or error) as "no count yet" and
41  * therefore keep the Surveys nav item hidden until a positive count is known.
42  *
43  * @param options.enabled - When `false`, the underlying query is not executed
44  * (e.g. the sidebar only needs the count for the student role). Defaults to
45  * `true` so existing callers keep their behavior.
46  */
47 export const useSurveyAssignmentsCount = (options?: { enabled?: boolean }) => {
48   return useQuery({
49     queryKey: queryKeys.surveys.list(ASSIGNED_COUNT_FILTERS),
50     queryFn: async () => {
51       const { count, error } = await supabase
52         .from("surveys")
53         .select("id", { count: "exact", head: true })
54         .eq("is_active", true);
55       if (error) throw error;
56       return count ?? 0;
57     },
58     enabled: options?.enabled ?? true,
59   });
60 };
61

```

```
TYPE: .ts | LINES: 1-421 | SHA256: aaa15c171a9be613dfe02b257a6b2eb5b65325668c54f97b0163ee143e90805e
```

[illegible]

[illegible]

[illegible]

[illegible]





[illegible]

[illegible]

```
235     enabled: !!courseId,  
236     staleTime: DASHBOARD_STALE_TIME_MS,  
237   });  
238 };  
239  
240 // % % % useTeacherBloomsDistribution % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
241  
242 export const useTeacherBloomsDistribution = (options?: {  
243   enabled?: boolean;  
244 }) => {  
245   const { user } = useAuth();  
246  
247   return useQuery({  
248     queryKey: queryKeys.teacherDashboard.list({  
249       type: "bloomsDistribution",  
250       teacherId: user?.id,  
251     }),  
252     queryFn: async (): Promise<BloomsDistributionRow[]> => {  
253       const teacherId = user?.id;  
254       if (!teacherId) return [];  
  
255       const { data: courses } = await supabase  
256         .from("courses")  
257         .select("id")  
258         .eq("teacher_id", teacherId)  
259         .eq("is_active", true);  
260  
261       const courseIds = (courses ?? []).map((c) => c.id);  
262       if (courseIds.length === 0) return [];  
263  
264       const { data: clos } = await supabase  
265         .from("learning_outcomes")  
266         .select("blooms_level")  
267         .eq("type", "CLO")  
268         .in("course_id", courseIds);  
269  
270       const countMap = new Map<string, number>();  
271       for (const row of clos ?? []) {  
272         if (row.blooms_level) {  
273           countMap.set(  
274             row.blooms_level,  
275             (countMap.get(row.blooms_level) ?? 0) + 1  
276           );  
277         }  
278       }  
279  
280       const levels: BloomsLevel[] = [  
281         "remembering",  
282         "understanding",  
283         "applying",  
284         "analyzing",  
285         "evaluating",  
286         "creating",  
287       ];  
288       return levels  
289         .map((level) => ({ level, count: countMap.get(level) ?? 0 }))  
290         .filter((r) => r.count > 0);  
291     },  
292     enabled: !!user?.id && (options?.enabled ?? true),  
293     staleTime: DASHBOARD_STALE_TIME_MS,  
294   });  
295 };  
296  
297 // % % % useStudentPerformanceHeatmap % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
298  
299 export const useStudentPerformanceHeatmap = (courseId?: string) => {  
300   return useQuery({  
301     queryKey: queryKeys.teacherDashboard.list({ type: "heatmap", courseId }),  
302     queryFn: async (): Promise<HeatmapCell[]> => {  
303       if (!courseId) return [];  
304  
305       // Get CLOs for this course  
306       const { data: clos } = await supabase  
307         .from("learning_outcomes")  
308         .select("id, title")  
309         .eq("type", "CLO")  
310         .eq("course_id", courseId)  
311         .order("sort_order", { ascending: true });  
312  
313       const typedCLOS = clos ?? [];  
314       if (typedCLOS.length === 0) return [];  
315  
316       const cloIds = typedCLOS.map((c) => c.id);  
317       const cloTitleMap = new Map(typedCLOS.map((c) => [c.id, c.title]));  
318  
319       // Get enrolled students  
320       const { data: enrollments } = await supabase  
321         .from("student_courses")  
322         .select(  
323           "student_id, profiles!student_courses_student_id_fkey(full_name)"  
324         )  
325         .eq("course_id", courseId);  
326  
327       const students = enrollments ?? [];  
328  
329       if (students.length === 0) return [];  
330  
331       const studentIds = students.map((s) => s.student_id);  
332       const studentNameMap = new Map(  
333         students.map((s) => {  
334           const profile = s.profiles as { full_name: string } | null;  
335           return [s.student_id, profile?.full_name ?? "Unknown"];  
336         })  
337       );  
338
```

[illegible]

[illegible]

[illegible]

FILE: src/hooks/useTeacherDashboardAggregate.ts

```
TYPE: .ts | LINES: 1-87 | SHA256: 831bc1aa90d31b2a1a035e6e5e9e2af0762e427fea1dc783c9c4b5db080a7d1b
```

```

1 | import { useQuery, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { DASHBOARD_STALE_TIME_MS } from "@lib/queryConfig";
5 | import type {
6 |   TeacherKPIData,
7 |   BloomsDistributionRow,
8 | } from "@hooks/useTeacherDashboard";
9 |
10 | /**
11 |  * Shape of the single `jsonb` payload returned by the `get_teacher_dashboard`
12 |  * Postgres RPC (see
13 |  * `supabase/migrations/20260821000011_create_get_teacher_dashboard_rpc.sql`).
14 |  *
15 |  * It is the union of the always-on section-hook results this aggregate hydrates:

```

```

16 * - `kpis` exactly what `useTeacherKPIs` produces (`TeacherKPIData`)
17 * - `bloomsDistribution` !' exactly what `useTeacherBloomsDistribution` produces
18 */
19 export interface TeacherDashboardAggregate {
20   kpis: TeacherKPIData;
21   bloomsDistribution: BloomsDistributionRow[];
22 }
23
24 /**
25  * Teacher dashboard aggregate (spec: dashboard-and-ux-performance, Phase 8 Task 33).
26  *
27  * Collapses the teacher dashboard's always-on fan-out (`useTeacherKPIs` ~7
28  * round-trips + `useTeacherBloomsDistribution` ~2) into ONE round-trip by calling
29  * the `get_teacher_dashboard` RPC, then HYDRATES the exact caches those section
30  * hooks read so they resolve as cache hits instead of firing their own requests.
31  *
32  * Backward-compatible + reversible: the section hooks keep their own `queryFn`,
33  * so on a cache miss / RPC failure they fetch exactly as before – no behavior
34  * change, no new data visibility (the RPC is SECURITY DEFINER with a fail-closed
35  * `auth.uid()` guard, so a teacher only ever receives their own teaching data).
36  *
37  * The course-SELECTED charts (CLO attainment, heatmap), the at-risk student LIST,
38  * and the grading queue are intentionally NOT part of this aggregate (see the
39  * migration header) and keep their own hooks.
40  */
41 export const useTeacherDashboardAggregate = (teacherId: string | undefined) => {
42   const queryClient = useQueryClient();
43
44   return useQuery({
45     queryKey: queryKeys.teacherDashboard.list({
46       type: "aggregate",
47       teacherId,
48     }),
49     enabled: !!teacherId,
50     staleTime: DASHBOARD_STALE_TIME_MS,
51     queryFn: async (): Promise<TeacherDashboardAggregate> => {
52       if (!teacherId) {
53         throw new Error("useTeacherDashboardAggregate: teacherId is required");
54       }
55       // The RPC is newer than the generated Database type, so keep the escape
56       // hatch scoped to the function name and arguments rather than weakening
57       // the Supabase client to `any`.
58       const { data, error } = await supabase.rpc(
59         "get_teacher_dashboard" as never,
60         { p_teacher_id: teacherId } as never
61       );
62       if (error) throw error;
63       if (!data) {
64         throw new Error("get_teacher_dashboard returned no data");
65       }
66
67       const payload = data as TeacherDashboardAggregate;
68
69       // Hydrate the EXACT caches the section hooks read so they become cache hits.
70       // Keys mirror `useTeacherKPIs` / `useTeacherBloomsDistribution` verbatim.
71       queryClient.setQueryData(
72         queryKeys.teacherDashboard.list({ type: "kpis", teacherId }),
73         payload.kpis
74       );
75       queryClient.setQueryData(
76         queryKeys.teacherDashboard.list({
77           type: "bloomsDistribution",
78           teacherId,
79         }),
80         payload.bloomsDistribution
81       );
82
83       return payload;
84     },
85   });
86 };

```

FILE: src/hooks/useTeacherHandoffs.ts

```
TYPE: .ts | LINES: 1-123 | SHA256: e5a40202dda4dbeab639570a12d9e7ea3cf1ddb94ba04e47f15fb66ff1c7db3e
```

[illegible]

```

29 |         .eq("course_id", courseId)
30 |         .order("created_at", { ascending: false });
31 |
32 |         if (error) throw error;
33 |         return (data ?? []) as TeacherHandoffRequest[];
34 |     },
35 |     enabled: !!courseId,
36 | });
37 | };
38 |
39 | // % % % useRespondToHandoff - Teacher responds to a handoff request % % % % % % % % %
40 |
41 | export const useRespondToHandoff = () => {
42 |     const queryClient = useQueryClient();
43 |
44 |     return useMutation({
45 |         mutationFn: async ({
46 |             handoff_id,
47 |             response_message,
48 |             status = "resolved",
49 |         }) => {
50 |             handoff_id: string;
51 |             response_message: string;
52 |             status?: "resolved" | "dismissed";
53 |         }) => {
54 |             // eslint-disable-next-line @typescript-eslint/no-explicit-any
55 |             const { data, error } = await (supabase as any)
56 |                 .from("teacher_handoff_requests")
57 |                 .update({
58 |                     teacher_response: response_message,
59 |                     status,
60 |                     resolved_at: new Date().toISOString(),
61 |                 })
62 |                 .eq("id", handoff_id)
63 |                 .select()
64 |                 .single();
65 |
66 |             if (error) throw error;
67 |             return data as TeacherHandoffRequest;
68 |         },
69 |         onSuccess: () => {
70 |             queryClient.invalidateQueries({ queryKey: handoffKeys.all });
71 |         },
72 |     });
73 | };
74 |
75 | // % % % useCreateHandoff - Student creates a handoff request (with consent) % % % % %
76 |
77 | export const useCreateHandoff = () => {
78 |     const queryClient = useQueryClient();
79 |
80 |     return useMutation({
81 |         mutationFn: async (payload: {
82 |             conversation_id: string;
83 |             student_id: string;
84 |             teacher_id: string;
85 |             institution_id: string;
86 |             course_id: string;
87 |             clo_id?: string;
88 |             conversation_summary: string;
89 |             suggested_intervention: string;
90 |             trigger_reason:
91 |                 | "low_rag_confidence"
92 |                 | "repeated_question"
93 |                 | "low_satisfaction";
94 |             student_consent: boolean;
95 |         }) => {
96 |             // eslint-disable-next-line @typescript-eslint/no-explicit-any
97 |             const { data, error } = await (supabase as any)
98 |                 .from("teacher_handoff_requests")
99 |                 .insert({
100 |                     conversation_id: payload.conversation_id,
101 |                     student_id: payload.student_id,
102 |                     teacher_id: payload.teacher_id,
103 |                     institution_id: payload.institution_id,
104 |                     course_id: payload.course_id,
105 |                     clo_id: payload.clo_id ?? null,
106 |                     conversation_summary: payload.conversation_summary,
107 |                     suggested_intervention: payload.suggested_intervention,
108 |                     trigger_reason: payload.trigger_reason,
109 |                     student_consent: payload.student_consent,
110 |                     status: "pending",
111 |                 })
112 |                 .select()
113 |                 .single();
114 |
115 |             if (error) throw error;
116 |             return data as TeacherHandoffRequest;
117 |         },
118 |         onSuccess: () => {
119 |             queryClient.invalidateQueries({ queryKey: handoffKeys.all });
120 |         },
121 |     });
122 | };
123 |

```

## FILE: src/hooks/useTeachingImpact.ts

TYPE: .ts | LINES: 1-212 | SHA256: 95b5ae4dc51b105a3dd7449a441fe864bd26f7eeaddf4323c3cd384f045ac9f0

```

1 | // =====
2 | // useTeachingImpact - TanStack Query hook for teaching impact metrics
3 | // Task 3.14: fetch teaching impact metrics (top peer teachers, most-viewed
4 | //           moments, average ratings)
5 | // =====

```



[illegible]

```

110 // Fetch ratings
111 const { data: ratings, error: ratingsError } = await supabase
112   .from("teaching_moment_ratings" as never)
113   .select("teaching_moment_id, clarity_rating, helpfulness_rating")
114   .in("teaching_moment_id", momentIds);
115 if (ratingsError) throw ratingsError;
116
117 const ratingsList = (ratings ?? []) as Array<{
118   teaching_moment_id: string;
119   clarity_rating: number;
120   helpfulness_rating: number;
121 }>;
122
123 // Compute per-moment average ratings
124 const ratingsByMoment = new Map<
125   string,
126   { clarity: number[]; helpfulness: number[] }
127 >();
128 for (const r of ratingsList) {
129   const existing = ratingsByMoment.get(r.teaching_moment_id) ?? {
130     clarity: [],
131     helpfulness: [],
132   };
133   existing.clarity.push(r.clarity_rating);
134   existing.helpfulness.push(r.helpfulness_rating);
135   ratingsByMoment.set(r.teaching_moment_id, existing);
136 }
137
138 const avg = (arr: number[]) =>
139   arr.length > 0 ? arr.reduce((a, b) => a + b, 0) / arr.length : 0;
140
141 // Build most-viewed moments (top 10)
142 const mostViewedMoments: MostViewedMoment[] = momentList
143   .map(m => {
144     const momentRatings = ratingsByMoment.get(m.id);
145     return {
146       moment_id: m.id,
147       title: m.title,
148       author_id: m.author_id,
149       clo_id: m.clo_id,
150       view_count: viewCountMap.get(m.id) ?? 0,
151       avg_clarity: avg(momentRatings?.clarity ?? []),
152       avg_helpfulness: avg(momentRatings?.helpfulness ?? []),
153     };
154   })
155   .sort((a, b) => b.view_count - a.view_count)
156   .slice(0, 10);
157
158 // Build top peer teachers (by total views, top 10)
159 const teacherMap = new Map<
160   string,
161   {
162     moments: string[];
163     views: number;
164     clarity: number[];
165     helpfulness: number[];
166   }
167 >();
168 for (const m of momentList) {
169   const existing = teacherMap.get(m.author_id) ?? {
170     moments: [],
171     views: 0,
172     clarity: [],
173     helpfulness: [],
174   };
175   existing.moments.push(m.id);
176   existing.views += viewCountMap.get(m.id) ?? 0;
177   const momentRatings = ratingsByMoment.get(m.id);
178   if (momentRatings) {
179     existing.clarity.push(...momentRatings.clarity);
180     existing.helpfulness.push(...momentRatings.helpfulness);
181   }
182   teacherMap.set(m.author_id, existing);
183 }
184
185 const topTeachers: TopPeerTeacher[] = Array.from(teacherMap.entries())
186   .map(([authorId, data]) => ({
187     author_id: authorId,
188     moment_count: data.moments.length,
189     total_views: data.views,
190     avg_clarity: avg(data.clarity),
191     avg_helpfulness: avg(data.helpfulness),
192   })))
193   .sort((a, b) => b.total_views - a.total_views)
194   .slice(0, 10);
195
196 // Global averages
197 const allClarity = ratingsList.map(r => r.clarity_rating);
198 const allHelpfulness = ratingsList.map(r => r.helpfulness_rating);
199
200 return {
201   topTeachers,
202   mostViewedMoments,
203   totalMoments: momentList.length,
204   totalViews: (views ?? []).length,
205   averageClarity: avg(allClarity),
206   averageHelpfulness: avg(allHelpfulness),
207 };
208 },
209 enabled: !!courseId,
210 });
211 };
212

```

TYPE: .ts | LINES: 1-46 | SHA256: f8e84a5991fbdd686b2605d3260f04bbe4d4dfd12415e0e7c434a5d96576c3f9

FILE: `src/hooks/useTeamFormation.ts`

```
TYPE: .ts | LINES: 1-96 | SHA256: 8af831767604dae1bcee5ab9b8b99426fa7e082b1cf5f77a66293fe822784e2c
```

[illegible]

[illegible]

FILE: src/hooks/useTeamHealth.ts

```
TYPE: .ts | LINES: 1-97 | SHA256: a56cd8bca052daa9ac4b956950e71ea30be7cb6d3ed0f28df2bfaab97a324ed3
```

[illegible]

```

55 const { data, error } = await query;
56 if (error) throw error;
57
58 return (
59   (data ?? []) as Array<{
60     id: string;
61     name: string;
62     health_score: number;
63     health_status: HealthStatus;
64     cooperation_score: number;
65   }>
66 ).map((t) => ({
67   team_id: t.id,
68   team_name: t.name,
69   health_score: t.health_score,
70   health_status: t.health_status,
71   cooperation_score: t.cooperation_score,
72 }));
73 },
74 enabled: !!courseId,
75 });
76 };
77
78 // % % % useTeamHealthSnapshots – fetch health snapshot history for a team % % % % % %
79
80 export const useTeamHealthSnapshots = (teamId?: string, limit = 12) => {
81   return useQuery({
82     queryKey: queryKeys.teamHealth.detail(teamId ?? ""),
83     queryFn: async (): Promise<TeamHealthSnapshot[]> => {
84       const { data, error } = await supabase
85         .from("team_health_snapshots" as never)
86         .select(".*")
87         .eq("team_id", teamId!)
88         .order("computed_at", { ascending: false })
89         .limit(limit);
90       if (error) throw error;
91       // Return in chronological order for charting
92       return ((data ?? []) as TeamHealthSnapshot[]).reverse();
93     },
94     enabled: !!teamId,
95   });
96 };
97

```

[illegible]

[illegible]



```
68 invited_by: string;
69 }
70
71 export const useSendInvitation = () => {
72   const qc = useQueryClient();
73   return useMutation({
74     mutationFn: async (input: SendInvitationInput) => {
75       const { data, error } = await supabase
76         .from("team_invitations" as never)
77         .insert(input as never)
78         .select()
79         .single();
80       if (error) throw error;
81       return data as TeamInvitation;
82     },
83     onSuccess: (_data, variables) => {
84       qc.invalidateQueries({
85         queryKey: queryKeys.teamInvitations.list({ teamId: variables.team_id }),
86       });
87     },
88     onError: (error: Error) => {
89       toast.error(error.message || "Failed to send invitation");
90     },
91   });
92 };
93
94 // % % % useRespondToInvitation - accept or decline % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
95
96 interface RespondInput {
97   invitationId: string;
98   status: "accepted" | "declined";
99   teamId: string;
100   studentId: string;
101 }
102
103 export const useRespondToInvitation = () => {
104   const qc = useQueryClient();
105   return useMutation({
106     mutationFn: async ({
107       invitationId,
108       status,
109       teamId,
110       studentId,
111     }: RespondInput) => {
112       // Update invitation status
113       const { error: updateError } = await supabase
114         .from("team_invitations" as never)
115         .update({
116           status,
117           responded_at: new Date().toISOString(),
118         } as never)
119         .eq("id", invitationId);
120       if (updateError) throw updateError;
121
122       // If accepted, add student as team member
123       if (status === "accepted") {
124         const { error: memberError } = await supabase
125           .from("team_members" as never)
126           .insert({
127             team_id: teamId,
128             student_id: studentId,
129             role: "member",
130           } as never);
131         if (memberError) throw memberError;
132       }
133
134       return { invitationId, status };
135     },
136     onSuccess: (_data, variables) => {
137       qc.invalidateQueries({ queryKey: queryKeys.teamInvitations.lists() });
138       if (variables.status === "accepted") {
139         qc.invalidateQueries({
140           queryKey: queryKeys.teamMembers.list({ teamId: variables.teamId }),
141         });
142         qc.invalidateQueries({ queryKey: queryKeys.teams.lists() });
143       }
144     },
145     onError: (error: Error) => {
146       toast.error(error.message || "Failed to respond to invitation");
147     },
148   });
149 };
150
```

FILE: `src/hooks/useTeamLeaderboard.ts`

TYPE: .ts | LINES: 1-190 | SHA256: d07328a3555201666102f8c607175e1996d27d366c9f102436ade34a4a3dd7fa

```
1 // =====
2 // useTeamLeaderboard – TanStack Query hooks for team leaderboard
3 // Task 3.8: fetch team leaderboard by course/program, sorted by xp_total desc,
4 //           with Cooperation Score sort option
5 // =====
6
7 import { useCallback } from "react";
8 import { useQuery, useQueryClient } from "@tanstack/react-query";
9 import { supabase } from "@/lib/supabase";
10 import { queryKeys } from "@/lib/queryKeys";
11 import { useRealtime } from "@/hooks/useRealtime";
12
13 // % % % Types % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
14 export interface TeamLeaderboardEntry {
15   team_id: string;
16   team_name: string;
```



[illegible]

```

122 |     .select("team_id")
123 |     .in("team_id", teamIds)
124 |     .is("left_at", null);
125 |   if (memError) throw memError;
126 |
127 |   const countMap = new Map<string, number>();
128 |   for (const m of members ?? []) {
129 |     countMap.set(m.team_id, (countMap.get(m.team_id) ?? 0) + 1);
130 |   }
131 |
132 |   // Build entries
133 |   const entries: TeamLeaderboardEntry[] = teams.map((t) => ({
134 |     team_id: t.id,
135 |     team_name: t.name,
136 |     avatar_letter: t.avatar_letter ?? t.name.charAt(0).toUpperCase(),
137 |     xp_total: t.xp_total,
138 |     xp_this_week: 0,
139 |     streak_count: t.streak_count,
140 |     cooperation_score: t.cooperation_score,
141 |     health_score: t.health_score,
142 |     member_count: countMap.get(t.id) ?? 0,
143 |     rank: 0,
144 |   }));
145 |
146 |   // Sort by selected field
147 |   entries.sort((a, b) => b[sortBy] - a[sortBy]);
148 |   entries.forEach((e, i) => (e.rank = i + 1));
149 |
150 |   return entries;
151 | },
152 | enabled: !(courseId || programId),
153 | });
154 |
155 | return { ...query, isLive, retryCount };
156 | };
157 |
158 | // % % % useMyTeamId - resolve current student's team for a course % % % % % % % % % % % % % % % %
159 |
160 | export const useMyTeamId = (studentId?: string, courseId?: string) => {
161 |   return useQuery({
162 |     queryKey: queryKeys.teamMembers.list({ studentId, courseId }),
163 |     queryFn: async (): Promise<string | null> => {
164 |       const { data: teams, error: teamsErr } = await supabase
165 |         .from("teams")
166 |         .select("id")
167 |         .eq("course_id", courseId!)
168 |         .is("deleted_at", null);
169 |       if (teamsErr) throw teamsErr;
170 |       if (!teams || teams.length === 0) return null;
171 |
172 |       const teamIds = teams.map((t) => t.id);
173 |
174 |       const { data: membership, error: memErr } = await supabase
175 |         .from("team_members")
176 |         .select("team_id")
177 |         .eq("student_id", studentId!)
178 |         .in("team_id", teamIds)
179 |         .is("left_at", null)
180 |         .limit(1)
181 |         .maybeSingle();
182 |       if (memErr) throw memErr;
183 |
184 |       return membership?.team_id ?? null;
185 |     },
186 |     enabled: !!studentId && !!courseId,
187 |     staleTime: 60_000,
188 |   });
189 | };
190 |

```

FILE: src/hooks/useTeamMembers.ts

```
TYPE: .ts | LINES: 1-109 | SHA256: 6b5f64b8c373a22a8701bfd001b0179ec8d5113252ef1a84c81091926692bd65
```

[illegible]

[illegible]

FILE: src/hooks/useTeamProfile.ts

TYPE: .ts | LINES: 1-159 | SHA256: 1201704000a70c35c76b5e63ad2aa9e9b286fd09ccca32dcb2a1dbfa0228b59d

[illegible]

[illegible]

```

127 |         status: string;
128 |     }>
129 |     ).map((c) => {
130 |         const progress = progressMap.get(c.id);
131 |         return {
132 |             ...c,
133 |             current_progress: progress?.current_progress ?? 0,
134 |             completed_at: progress?.completed_at ?? null,
135 |         };
136 |     });
137 | }
138 |
139 | // Fetch teaching moments for this team
140 | const { data: teachingMoments, error: tmError } = await supabase
141 |     .from("peer_teaching_moments" as never)
142 |     .select("*")
143 |     .eq("team_id", teamId!)
144 |     .eq("status", "active")
145 |     .order("created_at", { ascending: false });
146 | if (tmError) throw tmError;
147 |
148 | return {
149 |     team: team as Team,
150 |     members: (members ?? []) as TeamMember[],
151 |     badges: (badges ?? []) as TeamBadgeRecord[],
152 |     activeChallenges,
153 |     teachingMoments: (teachingMoments ?? []) as TeachingMoment[],
154 | };
155 | },
156 | enabled: !!teamId,
157 | });
158 | };
159 |

```

FILE: src/hooks/useTeamRealtime.ts

TYPE: .ts | LINES: 1-47 | SHA256: b32088e66e75a7350c78b1ced6c0a64fc7d589ff914161615333d203ace20b64

[illegible]

FILE: src/hooks/useTeams.ts

```
TYPE: .ts | LINES: 1-263 | SHA256: c2ee9d186f65bd5321384a0f44f3b98dc33d811f2d75d4d992d644cce9c2b07f
```

[illegible]

[illegible]

```

// % % Legacy hooks (preserved for backward compatibility) % % % % % % % % % %
export interface TeamGamification {
  id: string;
  team_id: string;
  xp_total: number;
  streak_count: number;
}

/**
 * Reads team gamification data directly from `teams` table.
 * The previously-referenced `team_gamification` table does not exist in
 * production - XP and streak fields live on `teams` itself.
 */
export const useTeamGamification = (teamId?: string) => {
  return useQuery({
    queryKey: queryKeys.teamGamification.list([ teamId ]),
    queryFn: async (): Promise<TeamGamification | null> => {
      const { data, error } = await supabase
        .from("teams" as never)
        .select("id, xp_total, streak_count")
        .eq("id", teamId!)
        .maybeSingle();
      if (error) throw error;
      if (!data) return null;
      const row = data as Record<string, unknown>;
      return {
        id: row.id as string,
        team_id: row.id as string,
        xp_total: (row.xp_total as number) ?? 0,
        streak_count: (row.streak_count as number) ?? 0,
      };
    },
    enabled: !!teamId,
    staleTime: 120_000,
  });
};

export const useAutoGenerateTeams = () => {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (params: {
      course_id: string;
      team_size: number;
      created_by: string;
      institution_id: string;
    }) => {
      const { data: enrollments } = await supabase
        .from("student_courses")
        .select("student_id")
        .eq("course_id", params.course_id);

      const studentIds = (enrollments ?? []).map((e) => e.student_id);
      if (studentIds.length === 0) throw new Error("No enrolled students");

      const shuffled = [...studentIds].sort(() => Math.random() - 0.5);
      const teamCount = Math.ceil(shuffled.length / params.team_size);
      const teamBuckets: string[][] = Array.from(
        { length: teamCount },
        () => []
      );

      shuffled.forEach((id, i) => {
        teamBuckets[i % teamCount]!.push(id);
      });

      let created = 0;
      for (let i = 0; i < teamBuckets.length; i++) {
        const bucket = teamBuckets[i]!;
        const captainId = bucket[0];
        if (!captainId) continue; // empty bucket - nothing to create

        // Whitelist + typed insert (no `as never`): the first bucket member is
        // the captain, and institution_id comes from the caller's institution.
        const row = pickColumns({
          name: `Team ${i + 1}`,
          course_id: params.course_id,
          institution_id: params.institution_id,
          captain_id: captainId,
          created_by: params.created_by,
          avatar_letter: String(i + 1),
        }) satisfies CreateTeamInput,
          TEAMS_INSERT_COLUMNS
        );
        const { data: team, error } = await supabase
          .from("teams")
          .insert(row)
          .select()
          .single();
        if (error) throw error;
        if (!team) throw new Error("Team insert returned no row");

        const members = bucket.map((studentId) => ({
          team_id: team.id,
          student_id: studentId,
        }));
        const { error: memberError } = await supabase
          .from("team_members" as never)
          .insert(members as never);
        if (memberError) throw memberError;
        created++;
      }
    }
  });
};

```

```

225 |         return { teams_created: created, students_assigned: shuffled.length };
226 |     },
227 |     onSuccess: () => {
228 |       qc.invalidateQueries({ queryKey: queryKeys.teams.lists() });
229 |     },
230 |     onError: (error: Error) => {
231 |       toast.error(error.message || "Failed to auto-generate teams");
232 |     }
233 |   });
234 | };
235 |
236 | // % % % useSoftDeleteTeam % % % % % % % % % % % % % % % % % % % % % %
237 |
238 | export const useSoftDeleteTeam = () => {
239 |   const qc = useQueryClient();
240 |   return useMutation({
241 |     mutationFn: async (teamId: string) => {
242 |       const { data, error } = await supabase
243 |         .from("teams" as never)
244 |         .update({
245 |           deleted_at: new Date().toISOString(),
246 |           updated_at: new Date().toISOString(),
247 |         } as never)
248 |         .eq("id", teamId)
249 |         .select()
250 |         .single();
251 |       if (error) throw error;
252 |       return data as Team;
253 |     },
254 |     onSuccess: () => {
255 |       qc.invalidateQueries({ queryKey: queryKeys.teams.lists() });
256 |       qc.invalidateQueries({ queryKey: queryKeys.teamMembers.lists() });
257 |     },
258 |     onError: (error: Error) => {
259 |       toast.error(error.message || "Failed to delete team");
260 |     }
261 |   });
262 | };
263 |

```

FILE: src/hooks/useTheme.ts

TYPE: .ts | LINES: 1-39 | SHA256: bbed7d909c17a7b93cf93c8ff5a881571fdf915834c4f2d8196fbded58962a05

```

1  /**
2   * useTheme - unified theme hook
3   *
4   * Delegates to ThemeProvider (the single source of truth for theme state).
5   * ThemeProvider is mounted in App.tsx and handles:
6   * - localStorage persistence (key: 'theme')
7   * - Supabase profiles.theme_preference sync
8   * - System preference detection
9   * - .dark class application on document.documentElement
10  *
11  * Design: ADR-01
12  * Requirements: 2.10
13  */
14
15  import {
16    useTheme as useThemeProvider,
17    type ThemePreference,
18  } from "@providers/ThemeProvider";
19
20  export type ThemeMode = ThemePreference; // 'light' | 'dark' | 'system'
21
22  /**
23   * Hook to access and control the theme.
24   *
25   * Returns:
26   * - themeMode: current theme preference ('light', 'dark', or 'system')
27   * - effectiveTheme: resolved theme ('light' or 'dark')
28   * - setThemeMode: function to change the theme preference
29   */
30  export const useTheme = () => {
31    const { theme, resolvedTheme, setTheme } = useThemeProvider();
32
33    return {
34      themeMode: theme,
35      effectiveTheme: resolvedTheme,
36      setThemeMode: setTheme,
37    };
38  };
39

```

FILE: src/hooks/useTieredBadges.ts

```
TYPE: .ts | LINES: 1-201 | SHA256: 76cd688f1666fc31c420f16a59f30a7ceafda26d145ac01de5b719d1259282bf
```

```

1 // Task 151.4: Badge Tier TanStack Query hooks
2
3 import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
4 import { supabase } from "@lib/supabase";
5 import { queryKeys } from "@lib/queryKeys";
6 import { toast } from "sonner";
7
8 // % % % Types % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
9
10 export type BadgeTier = "bronze" | "silver" | "gold";
11
12 export interface TieredBadgeData {
13   id: string;
14   name: string;
15   emoji: string;
16   description: string;
17   category: string;
18   tier: BadgeTier | null;
19 }

```



[illegible]

[illegible]

FILE: src/hooks/useTimetable.ts

TYPE: .ts | LINES: 1-231 | SHA256: 09818a2b5dbfc34ce25c2610a6c70c4ecac25ad95c8e6d0b867fcdfbbbb8eff3

```

1 import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 import { supabase } from "@lib/supabase";
3 import { useAuth } from "@hooks/useAuth";
4 import { queryKeys } from "@lib/queryKeys";
5
6 export interface TimetableSlot {
7   id: string;
8   section_id: string;
9   day_of_week: number; // 0=Sun ... 6=Sat
10  start_time: string;
11  end_time: string;
12  room: string | null;
13  slot_type: string;
14  course_name?: string;
15  section_code?: string;
16  course_id?: string;
17  color?: string;
18 }
19
20 export interface CreateTimetableSlotInput {
21   section_id: string;

```

```

22 |   day_of_week: number;
23 |   start_time: string;
24 |   end_time: string;
25 |   room?: string;
26 |   slot_type: "lecture" | "lab" | "tutorial";
27 | }
28 |
29 | const SLOT_COLORS = [
30 |   "#3b82f6",
31 |   "#8b5cf6",
32 |   "#22c55e",
33 |   "#f59e0b",
34 |   "#ef4444",
35 |   "#14b8a6",
36 |   "#ec4899",
37 |   "#6366f1",
38 | ];
39 |
40 | /**
41 |  * Fetch timetable slots for the current user.
42 |  * Students: slots from enrolled sections.
43 |  * Teachers: slots from assigned sections.
44 |  */
45 | export const useTimetableSlots = () => {
46 |   const { user, role } = useAuth();
47 |
48 |   return useQuery({
49 |     queryKey: queryKeys.timetableSlots.list({ userId: user?.id, role }),
50 |     queryFn: async (): Promise<TimetableSlot[]> => {
51 |       if (!user) return [];
52 |
53 |       let sectionIds: string[] = [];
54 |
55 |       if (role === "student") {
56 |         // Get sections from enrolled courses
57 |         const { data: enrollments, error } = await supabase
58 |           .from("student_courses")
59 |           .select("section_id")
60 |           .eq("student_id", user.id)
61 |           .not("section_id", "is", null);
62 |         if (error) throw error;
63 |         sectionIds = (enrollments ?? []).map((e) => e.section_id)
64 |           .filter((id): id is string => !!id);
65 |       } else if (role === "teacher") {
66 |         // Get sections assigned to teacher
67 |         const { data: sections, error } = await supabase
68 |           .from("course_sections")
69 |           .select("id")
70 |           .eq("teacher_id", user.id);
71 |         if (error) throw error;
72 |         sectionIds = (sections ?? []).map((s) => s.id);
73 |       }
74 |
75 |       if (sectionIds.length === 0) {
76 |         // Fallback: fetch all visible slots (RLS will scope)
77 |         const { data, error } = await supabase
78 |           .from("timetable_slots")
79 |           .select("*")
80 |           .order("day_of_week")
81 |           .order("start_time");
82 |         if (error) throw error;
83 |         return (data ?? []) as TimetableSlot[];
84 |       }
85 |
86 |       // Fetch slots for the user's sections
87 |       const { data: slots, error } = await supabase
88 |         .from("timetable_slots")
89 |         .select("*")
90 |         .in("section_id", sectionIds)
91 |         .order("day_of_week")
92 |         .order("start_time");
93 |       if (error) throw error;
94 |
95 |       // Fetch section + course info for enrichment
96 |       const { data: sections, error: sectionsError } = await supabase
97 |         .from("course_sections")
98 |         .select("id, section_code, course_id, courses(name)");
99 |       if (sectionsError) throw sectionsError;
100 |
101 |       const sectionMap = new Map<
102 |         string,
103 |         { section_code: string; course_name: string; course_id: string }
104 |       >();
105 |       const courseColorMap = new Map<string, string>();
106 |       let colorIdx = 0;
107 |
108 |       (sections ?? []).forEach((s) => {
109 |         const courseName = (s.courses as { name: string } | null)?.name ?? "";
110 |         const courseId = s.course_id as string;
111 |         if (!courseColorMap.has(courseId)) {
112 |           courseColorMap.set(
113 |             courseId,
114 |             SLOT_COLORS[colorIdx % SLOT_COLORS.length] ?? "#3b82f6"
115 |           );
116 |           colorIdx++;
117 |         }
118 |         sectionMap.set(s.id, {
119 |           section_code: s.section_code,
120 |           course_name: courseName,
121 |           course_id: courseId,
122 |         });
123 |       });
124 |     },
125 |   });

```

```

126 |         return (slots ?? []).map((slot) => {
127 |             const info = sectionMap.get(slot.section_id);
128 |             return {
129 |                 ...slot,
130 |                 course_name: info?.course_name ?? "",
131 |                 section_code: info?.section_code ?? "",
132 |                 course_id: info?.course_id ?? "",
133 |                 color: info?.course_id
134 |                     ? courseColorMap.get(info.course_id)
135 |                     : undefined,
136 |             } as TimetableSlot;
137 |         });
138 |     },
139 |     enabled: !!user,
140 |     staleTime: 5 * 60_000,
141 | });
142 |
143 | };
144 |
145 | /**
146 |  * Fetch all timetable slots for a specific section (admin/coordinator use).
147 |  */
148 | export const useSectionTimetableSlots = (sectionId: string | undefined) => {
149 |     return useQuery({
150 |         queryKey: queryKeys.timetableSlots.detail(sectionId ?? ""),
151 |         queryFn: async (): Promise<TimetableSlot[]> => {
152 |             if (!sectionId) return [];
153 |             const { data, error } = await supabase
154 |                 .from("timetable_slots")
155 |                 .select(".*")
156 |                 .eq("section_id", sectionId)
157 |                 .order("day_of_week")
158 |                 .order("start_time");
159 |             if (error) throw error;
160 |             return (data ?? []) as TimetableSlot[];
161 |         },
162 |         enabled: !!sectionId,
163 |         staleTime: 5 * 60_000,
164 |     });
165 | };
166 |
167 | /**
168 |  * Create a new timetable slot.
169 |  */
170 | export const useCreateTimetableSlot = () => {
171 |     const queryClient = useQueryClient();
172 |     return useMutation({
173 |         mutationFn: async (input: CreateTimetableSlotInput) => {
174 |             const { data, error } = await supabase
175 |                 .from("timetable_slots")
176 |                 .insert(input)
177 |                 .select()
178 |                 .single();
179 |             if (error) throw error;
180 |             return data;
181 |         },
182 |         onSuccess: () => {
183 |             queryClient.invalidateQueries({ queryKey: queryKeys.timetableSlots.all });
184 |         },
185 |     });
186 | };
187 |
188 | /**
189 |  * Update an existing timetable slot.
190 |  */
191 | export const useUpdateTimetableSlot = () => {
192 |     const queryClient = useQueryClient();
193 |     return useMutation({
194 |         mutationFn: async ({
195 |             id,
196 |             ...input
197 |         }: Partial<CreateTimetableSlotInput> & { id: string }) => {
198 |             const { data, error } = await supabase
199 |                 .from("timetable_slots")
200 |                 .update(input)
201 |                 .eq("id", id)
202 |                 .select()
203 |                 .single();
204 |             if (error) throw error;
205 |             return data;
206 |         },
207 |         onSuccess: () => {
208 |             queryClient.invalidateQueries({ queryKey: queryKeys.timetableSlots.all });
209 |         },
210 |     });
211 | };
212 |
213 | /**
214 |  * Delete a timetable slot.
215 |  */
216 | export const useDeleteTimetableSlot = () => {
217 |     const queryClient = useQueryClient();
218 |     return useMutation({
219 |         mutationFn: async (id: string) => {
220 |             const { error } = await supabase
221 |                 .from("timetable_slots")
222 |                 .delete()
223 |                 .eq("id", id);
224 |             if (error) throw error;
225 |         },
226 |         onSuccess: () => {
227 |             queryClient.invalidateQueries({ queryKey: queryKeys.timetableSlots.all });
228 |         },
229 |     });

```

FILE: src/hooks/useTodayView.ts

```
1 // =====
2 // useTodayView - Fetch today's sessions, tasks, deadlines, and habit status
3 // =====
```

[illegible][illegible]

```

56 function mapTask(row: Record<string, unknown>): PlannerTask {
57   return {
58     id: row.id as string,
59     studentId: row.student_id as string,
60     title: row.title as string,
61     description: (row.description as string) ?? null,
62     dueDate: (row.due_date as string) ?? "",
63     priority: (row.priority as PlannerTask["priority"]) ?? "medium",
64     status: (row.status as PlannerTask["status"]) ?? "todo",
65     courseId: (row.course_id as string) ?? null,
66     courseName: (row.course_name as string) ?? undefined,
67     completedAt: (row.completed_at as string) ?? null,
68     createdAt: (row.created_at as string) ?? "",
69   };
70 }

```

[illegible]

```

83 | export const useTodayViewData = (
84 |   studentId: string | undefined
85 | ): TodayViewData => {
86 |   const today = getTodayStr();
87 |   const threeDaysOut = getDatePlusDays(today, 3);
88 |   const enabled = !!studentId;
89 |
90 |   const results = useQueries({
91 |     queries: [
92 |       // 1. Today's study sessions
93 |       {
94 |         queryKey: queryKeys.studySessions.list({
95 |           studentId: studentId ?? "",
96 |           date: today,
97 |           view: "today",
98 |         }),

```

```

99 |         enabled,
100 |         queryFn: async (): Promise<StudySession[]> => {
101 |             const { data, error } = await supabase
102 |                 .from("study_sessions")
103 |                 .select(
104 |                     "id, student_id, course_id, title, description, planned_date, planned_start_time, planned_durati
105 |                 )
106 |                 .eq("student_id", studentId!)
107 |                 .eq("planned_date", today)
108 |                 .order("planned_start_time", { ascending: true });
109 |
110 |             if (error) throw error;
111 |
112 |             return (data ?? []).map((row) => {
113 |                 const r = row as Record<string, unknown>;
114 |                 const courseData = r.courses as Record<string, unknown> | null;
115 |                 return mapSession({
116 |                     ...r,
117 |                     course_name: courseData?.name ?? undefined,
118 |                 });
119 |             });
120 |         },
121 |     },
122 |     // 2. Tasks due today
123 |     {
124 |         queryKey: queryKeys.plannerTasks.list({
125 |             studentId: studentId ?? "",
126 |             date: today,
127 |             view: "today",
128 |         }),
129 |         enabled,
130 |         queryFn: async (): Promise<PlannerTask[]> => {
131 |             const { data, error } = await supabase
132 |                 .from("planner_tasks")
133 |                 .select(
134 |                     "id, student_id, title, description, due_date, priority, status, course_id, completed_at, create
135 |                 )
136 |                 .eq("student_id", studentId!)
137 |                 .eq("due_date", today)
138 |                 .order("created_at", { ascending: true });
139 |
140 |             if (error) throw error;
141 |
142 |             return (data ?? []).map((row) => {
143 |                 const r = row as Record<string, unknown>;
144 |                 const courseData = r.courses as Record<string, unknown> | null;
145 |                 return mapTask({
146 |                     ...r,
147 |                     course_name: courseData?.name ?? undefined,
148 |                 });
149 |             });
150 |         },
151 |     },
152 |     // 3. Upcoming deadlines within 3 days
153 |     {
154 |         queryKey: queryKeys.assignments.list({
155 |             studentId: studentId ?? "",
156 |             from: today,
157 |             to: threeDaysOut,
158 |             type: "upcoming_deadlines",
159 |         }),
160 |         enabled,
161 |         queryFn: async (): Promise<UpcomingDeadline[]> => {
162 |             // Get enrolled course IDs
163 |             const { data: enrollments, error: enrollError } = await supabase
164 |                 .from("student_courses")
165 |                 .select("course_id")
166 |                 .eq("student_id", studentId!);
167 |
168 |             if (enrollError) throw enrollError;
169 |
170 |             const courseIds = (enrollments ?? []).map(
171 |                 (e) => e.course_id as string
172 |             );
173 |
174 |             if (courseIds.length === 0) return [];
175 |
176 |             const { data, error } = await supabase
177 |                 .from("assignments")
178 |                 .select("id, title, due_date, courses(name)")
179 |                 .in("course_id", courseIds)
180 |                 .gte("due_date", today)
181 |                 .lte("due_date", threeDaysOut)
182 |                 .order("due_date", { ascending: true });
183 |
184 |             if (error) throw error;
185 |
186 |             const now = new Date();
187 |             return (data ?? []).map((row) => {
188 |                 const r = row as Record<string, unknown>;
189 |                 const courseData = r.courses as Record<string, unknown> | null;
190 |                 return {
191 |                     id: r.id as string,
192 |                     title: r.title as string,
193 |                     courseName: (courseData?.name as string) ?? "",
194 |                     dueDate: r.due_date as string,
195 |                     urgency: getDeadlineUrgency(r.due_date as string, now),
196 |                 };
197 |             });
198 |         },
199 |     },
200 |     // 4. Habit status for today
201 |     {
202 |         queryKey: queryKeys.habitLogs.list({

```

```

203 studentId: studentId ?? "",
204 date: today,
205 view: "today_status",
206 },
207 enabled,
208 queryFn: async (): Promise<HabitStatus> => {
209   const status: HabitStatus = {
210     login: false,
211     submit: false,
212     journal: false,
213     read: false,
214   };
215
216   // Query academic habit_logs for today
217   const { data, error } = await supabase
218     .from("habit_logs" as never)
219     .select("habit_type")
220     .eq("student_id", studentId!)
221     .eq("date", today);
222
223   if (error) {
224     // Table may not exist in some environments – return defaults
225     console.error(
226       "[useTodayView] habit_logs query failed:",
227       error.message
228     );
229     return status;
230   }
231
232   for (const row of data ?? []) {
233     const r = row as Record<string, unknown>;
234     const habitType = r.habit_type as string;
235     if (habitType === "login") status.login = true;
236     if (habitType === "submit") status.submit = true;
237     if (habitType === "journal") status.journal = true;
238     if (habitType === "read") status.read = true;
239   }
240
241   return status;
242 },
243 ],
244 },
245 });
246
247 const [sessionsQuery, tasksQuery, deadlinesQuery, habitsQuery] = results;
248
249 return {
250   sessions: sessionsQuery.data ?? [],
251   tasks: tasksQuery.data ?? [],
252   deadlines: deadlinesQuery.data ?? [],
253   habits: habitsQuery.data ?? {
254     login: false,
255     submit: false,
256     journal: false,
257     read: false,
258   },
259   isLoading: results.some((r) => r.isLoading),
260   isError: results.some((r) => r.isError),
261 };
262 };
263

```

[illegible]

[illegible]



FILE: src/hooks/useTranscript.ts

```
TYPE: .ts | LINES: 1-26 | SHA256: f188eab449c4738330311ba49ea1ff7fc691b0c9b1b849bf59e48a7b4c84e258
```

```

1 import { useMutation } from "@tanstack/react-query";
2 import { supabase } from "@lib/supabase";
3
4 interface TranscriptRequest {
5   student_id: string;
6   semester_id?: string;
7 }
8
9 interface TranscriptResult {
10   download_url: string;
11   file_name: string;
12 }
13
14 export const useGenerateTranscript = () => {
15   return useMutation({
16     mutationFn: async (req: TranscriptRequest): Promise<TranscriptResult> => {
17       const { data, error } = await supabase.functions.invoke(
18         "generate-transcript",
19         { body: req }
20       );
21       if (error) throw error;
22       return data as TranscriptResult;
23     },
24   });
25 };
26

```

FILE: `src/hooks/useTutorAnalytics.ts`

```
TYPE: .ts | LINES: 1-39 | SHA256: 77aa50b6c514d5dbc2e76af82608d8f765fde0542c87c5550db5d555c48500c0
```

[illegible]

FILE: src/hooks/useTutorAttachmentUpload.ts

TYPE: .ts | LINES: 1-78 | SHA256: 4c1ad3cb11c1adf40c01f204c391fb428d94da79d2e132f0ad54e8738d58b929

```

1 // =====
2 // useTutorAttachmentUpload – upload AI Tutor chat attachments (Requirement 4.5)
3 // =====
4 //
5 // Uploads an image (JPG/PNG <= 5MB) or document (PDF/DOCX <= 10MB) to the
6 // private `tutor-attachments` Storage bucket under the authenticated user's
7 // own folder (`<auth.uid()>`/`<uuid>-<filename>`) and returns a short-lived
8 // signed URL for the `chat-with-tutor` edge function to read.
9 //
10 // The mutation THROWS on any failure (not authenticated, validation, upload,
11 // or signing) so the calling send (ChatPanel, task 12.1) can await the upload
12 // and abort the message send rather than passing an empty/undefined reference.
13
14 import { useCallback } from "react";
15 import { useMutation } from "@tanstack/react-query";
16 import { useAuth } from "@/hooks/useAuth";
17 import { uploadTutorAttachmentFile } from "@/lib/fileUpload";
18
19 /**
20  * Return type for {@link useTutorAttachmentUpload}.
21  */
22 interface UseTutorAttachmentUploadReturn {
23   /**
24    * Upload a single attachment and resolve to its signed URL.
25    * Rejects (throws) on failure so the caller can abort the send.
26    */
27   uploadAttachment: (file: File) => Promise<string>;

```

```

28  /** Whether an upload is currently in progress. */
29  isPending: boolean;
30  /** The error from the most recent failed upload, if any. */
31  error: Error | null;
32  }
33
34  /**
35   * Hook for uploading AI Tutor chat attachments.
36   *
37   * Unlike most mutation hooks in this codebase, this hook intentionally does
38   * NOT surface its own toast on error: the caller (ChatPanel) owns the
39   * user-facing error and the decision to abort the send. The hook simply
40   * propagates the thrown error via `uploadAttachment`'s rejected promise.
41   *
42   * Usage:
43   * ```tsx
44   * const { uploadAttachment, isPending } = useTutorAttachmentUpload();
45   *
46   * try {
47   *   const url = await uploadAttachment(file);
48   *   // pass `url` to chat-with-tutor
49   * } catch (err) {
50   *   toast.error(err instanceof Error ? err.message : "Upload failed");
51   *   return; // abort the send
52   * }
53   */
54
55  export const useTutorAttachmentUpload = (): UseTutorAttachmentUploadReturn => {
56    const { user } = useAuth();
57
58    const mutation = useMutation<string, Error, File>({
59      mutationFn: async (file: File): Promise<string> => {
60        if (!user) {
61          throw new Error("You must be signed in to attach files.");
62        }
63        return uploadTutorAttachmentFile({ file, userId: user.id });
64      },
65    });
66
67    const uploadAttachment = useCallback(
68      (file: File) => mutation.mutateAsync(file),
69      [mutation]
70    );
71
72    return {
73      uploadAttachment,
74      isPending: mutation.isPending,
75      error: mutation.error,
76    };
77  };
78

```

FILE: src/hooks/useTutorAutonomy.ts

TYPE: .ts | LINES: 1-145 | SHA256: a33da9ffc18ca24e5de4f05b7a8b95b9ff158792867db3201eed14b0f25ddc85

[illegible]

```

50 | return useMutation({
51 |   mutationFn: async ({
52 |     assignmentId,
53 |     autonomyLevel,
54 |   }: {
55 |     assignmentId: string;
56 |     autonomyLevel: AutonomyLevel;
57 |   }) => {
58 |     const { data, error } = await supabase
59 |       .from("assignments")
60 |       .update({ tutor_autonomy_level: autonomyLevel })
61 |       .eq("id", assignmentId)
62 |       .select("id, tutor_autonomy_level")
63 |       .single();
64 |
65 |     if (error) throw error;
66 |     return data;
67 |   },
68 |   onSuccess: (_data, variables) => {
69 |     queryClient.invalidateQueries({
70 |       queryKey: [
71 |         ...queryKeys.assignments.detail(variables.assignmentId),
72 |         "autonomy",
73 |       ],
74 |     });
75 |     queryClient.invalidateQueries({
76 |       queryKey: queryKeys.assignments.lists(),
77 |     });
78 |   },
79 |   onError: (error: Error) => {
80 |     toast.error(
81 |       error.message || "Failed to update assignment autonomy level"
82 |     );
83 |   },
84 | });
85 | };
86 |
87 | // % % % useCLOAutonomy - read tutor_autonomy_level from learning_outcomes % % % % %
88 |
89 | export const useCLOAutonomy = (cloId: string | undefined) => {
90 |   return useQuery({
91 |     queryKey: [...queryKeys.clos.detail(cloId ?? ""), "autonomy"],
92 |     queryFn: async (): Promise<AutonomyLevel | null> => {
93 |       if (!cloId) return null;
94 |
95 |       const { data, error } = await supabase
96 |         .from("learning_outcomes")
97 |         .select("tutor_autonomy_level")
98 |         .eq("id", cloId)
99 |         .maybeSingle();
100 |
101 |       if (error) throw error;
102 |       return toAutonomyLevel(data?.tutor_autonomy_level);
103 |     },
104 |     enabled: !!cloId,
105 |     staleTime: 60_000,
106 |   });
107 | };
108 |
109 | // % % % useUpdateCLOAutonomy - update CLO autonomy level % % % % %
110 |
111 | export const useUpdateCLOAutonomy = () => {
112 |   const queryClient = useQueryClient();
113 |
114 |   return useMutation({
115 |     mutationFn: async ({
116 |       cloId,
117 |       autonomyLevel,
118 |     }: {
119 |       cloId: string;
120 |       autonomyLevel: AutonomyLevel;
121 |     }) => {
122 |       const { data, error } = await supabase
123 |         .from("learning_outcomes")
124 |         .update({ tutor_autonomy_level: autonomyLevel })
125 |         .eq("id", cloId)
126 |         .select("id, tutor_autonomy_level")
127 |         .single();
128 |
129 |       if (error) throw error;
130 |       return data;
131 |     },
132 |     onSuccess: (_data, variables) => {
133 |       queryClient.invalidateQueries({
134 |         queryKey: [...queryKeys.clos.detail(variables.cloId), "autonomy"],
135 |       });
136 |       queryClient.invalidateQueries({
137 |         queryKey: queryKeys.clos.lists(),
138 |       });
139 |     },
140 |     onError: (error: Error) => {
141 |       toast.error(error.message || "Failed to update CLO autonomy level");
142 |     },
143 |   });
144 | };
145 |

```

FILE: src/hooks/useTutorConversations.ts

```
TYPE: .ts | LINES: 1-243 | SHA256: 751d53ef613e501969b326edbd16604306ed6c04e62a9576f9c246fe18146fcc
```

[illegible]

[illegible]

```

206 |
207 | interface RespondToPlanUpdateInput {
208 |   plan_update_id: string;
209 |   response: "accepted" | "modified" | "dismissed";
210 |   modifications?: string;
211 | }
212 |
213 | export const useRespondToPlanUpdate = () => {
214 |   const queryClient = useQueryClient();
215 |
216 |   return useMutation({
217 |     mutationFn: async (input: RespondToPlanUpdateInput): Promise<void> => {
218 |       // Validate input with Zod schema
219 |       const parsed = planUpdateResponseSchema.parse(input);
220 |
221 |       const { error } = await supabase
222 |         .from("tutor_plan_updates")
223 |         .update({
224 |           response: parsed.response,
225 |           modifications: parsed.modifications ?? null,
226 |           responded_at: new Date().toISOString(),
227 |         })
228 |         .eq("id", parsed.plan_update_id);
229 |
230 |       if (error) throw error;
231 |     },
232 |     onSuccess: () => {
233 |       // Invalidate conversations and messages to reflect updated state
234 |       queryClient.invalidateQueries({
235 |         queryKey: queryKeys.tutorConversations.lists(),
236 |       });
237 |     },
238 |     onError: (error: Error) => {
239 |       toast.error(error.message || "Failed to respond to plan update");
240 |     },
241 |   });
242 | };
243 |

```

FILE: src/hooks/useTutorMessages.ts

```
TYPE: .ts | LINES: 1-236 | SHA256: f5ae0737c295e4e2abfc8154b01778ab7b488a153ca7abd403da99de0963cc87
```

[illegible]

[illegible]

[illegible]

FILE: src/hooks/useTutorUsage.ts

```
TYPE: .ts | LINES: 1-62 | SHA256: e089e252f0f3b4c7c64b82c1f1b6a18f43ab99e71b9c85d053bd58a49d643d42
```

[illegible]





```
TYPE: .ts | LINES: 1-231 | SHA256: 1a4039a808c1f42311ea19408c8e1065d6e04e9582e6b21b0abd91f49d7b97b3
```

[illegible]

[illegible]

```

206 |     mutationFn: async (id: string): Promise<Profile> => {
207 |         const { data: result, error } = await supabase
208 |             .from("profiles")
209 |             .update({ is_active: false })
210 |             .eq("id", id)
211 |             .select()
212 |             .single();
213 |
214 |         if (error) throw error;
215 |
216 |         await logAuditEvent({
217 |             action: "soft_delete",
218 |             entity_type: "user",
219 |             entity_id: id,
220 |             changes: { is_active: false },
221 |             performed_by: user?.id ?? "unknown",
222 |         });
223 |
224 |         return result as Profile;
225 |     },
226 |     onSuccess: () => {
227 |         queryClient.invalidateQueries({ queryKey: queryKeys.users.lists() });
228 |     },
229 | });
230 | };
231 |

```

## FILE: src/hooks/useVisualizationData.ts

TYPE: .ts | LINES: 1-153 | SHA256: d3ad7851e254b4981ba894854f815eca3813261ee0f48a8e5d512806a549b062

```

1 | // Task 117.4: Visualization TanStack Query hooks
2 |
3 | import { useQuery } from "@tanstack/react-query";
4 | import { supabase } from "@lib/supabase";
5 | import { queryKeys } from "@lib/queryKeys";
6 | import {
7 |     transformToSankey,
8 |     type SankeyNode,
9 |     type SankeyLink,
10 | } from "@lib/sankeyTransform";
11 | import { analyzeGaps, type GapResult } from "@lib/gapAnalysis";
12 | import { buildHeatmapMatrix, type HeatmapMatrix } from "@lib/coverageHeatmap";
13 |
14 | export const useSankeyData = (
15 |     programId?: string,
16 |     _courseId?: string,
17 |     _semesterId?: string
18 | ) => {
19 |     return useQuery({
20 |         queryKey: queryKeys.sankeyData.list({ programId }),
21 |         queryFn: async (): Promise<{
22 |             nodes: SankeyNode[];
23 |             links: SankeyLink[];
24 |         }> => {
25 |             // Parallel reads (was a 3-step serial waterfall). Columns corrected to
26 |             // the real schema: outcome_mappings uses source/target_outcome_id (not
27 |             // parent/child) and outcome_attainment uses attainment_percent (not
28 |             // score_percent), so the flow now renders real mappings instead of empty.
29 |             const [outcomesRes, mappingsRes, attainmentsRes] = await Promise.all([
30 |                 supabase.from("learning_outcomes").select("id, type, title"),
31 |                 supabase
32 |                     .from("outcome_mappings")
33 |                     .select("source_outcome_id, target_outcome_id, weight"),
34 |                 supabase
35 |                     .from("outcome_attainment")
36 |                     .select("outcome_id, attainment_percent"),
37 |             ]);
38 |             if (outcomesRes.error) throw outcomesRes.error;
39 |             if (mappingsRes.error) throw mappingsRes.error;
40 |             if (attainmentsRes.error) throw attainmentsRes.error;
41 |
42 |             return transformToSankey(
43 |                 (outcomesRes.data ?? []).map((o) => ({
44 |                     id: o.id,
45 |                     type: o.type as "ILO" | "PLO" | "CLO",
46 |                     title: o.title,
47 |                 })),
48 |                 (mappingsRes.data ?? []).map((m) => ({
49 |                     parent_id: m.source_outcome_id,
50 |                     child_id: m.target_outcome_id,
51 |                     weight: Number(m.weight ?? 0),
52 |                 })),
53 |                 (attainmentsRes.data ?? []).map((a) => ({
54 |                     outcome_id: a.outcome_id,
55 |                     score_percent: Number(a.attainment_percent ?? 0),
56 |                 })))
57 |             );
58 |         },
59 |         enabled: !!programId,
60 |         staleTime: 5 * 60_000,
61 |     });
62 | };
63 |
64 | export const useGapAnalysis = (programId?: string, _semesterId?: string) => {
65 |     return useQuery({
66 |         queryKey: queryKeys.gapAnalysisData.list({ programId }),
67 |         queryFn: async (): Promise<GapResult[]> => {
68 |             // Parallel reads (was serial). outcome_mappings.parent_outcome_id ->
69 |             // source_outcome_id (the parent in a source->target mapping); evidence is
70 |             // keyed by clo_id.
71 |             const [outcomesRes, mappingsRes, evidenceRes] = await Promise.all([
72 |                 supabase.from("learning_outcomes").select("id, title, type"),
73 |                 supabase.from("outcome_mappings").select("source_outcome_id"),
74 |                 supabase.from("evidence").select("clo_id"),

```

```

75 |     });
76 |     if (outcomesRes.error) throw outcomesRes.error;
77 |     if (mappingsRes.error) throw mappingsRes.error;
78 |     if (evidenceRes.error) throw evidenceRes.error;
79 |
80 |     const mappings = mappingsRes.data ?? [];
81 |     const evidence = evidenceRes.data ?? [];
82 |
83 |     const childCountMap = new Map<string, number>();
84 |     for (const m of mappings) {
85 |       const pid = m.source_outcome_id;
86 |       childCountMap.set(pid, (childCountMap.get(pid) ?? 0) + 1);
87 |     }
88 |
89 |     const evidenceCountMap = new Map<string, number>();
90 |     const assessedCLOs = new Set<string>();
91 |     for (const e of evidence) {
92 |       const cid = e.clo_id;
93 |       evidenceCountMap.set(cid, (evidenceCountMap.get(cid) ?? 0) + 1);
94 |       assessedCLOs.add(cid);
95 |     }
96 |
97 |     return analyzeGaps(
98 |       (outcomesRes.data ?? []).map((o) => ({
99 |         id: o.id,
100 |         title: o.title,
101 |         type: o.type as "ILO" | "PLO" | "CLO",
102 |         mapped_children_count: childCountMap.get(o.id) ?? 0,
103 |         evidence_count: evidenceCountMap.get(o.id) ?? 0,
104 |         has_assessments: assessedCLOs.has(o.id),
105 |       })))
106 |   );
107 | },
108 | enabled: !!programId,
109 | staleTime: 5 * 60_000,
110 | });
111 | });
112 |
113 | export const useCoverageHeatmap = (
114 |   programId?: string,
115 |   _semesterId?: string
116 | ) => {
117 |   return useQuery({
118 |     queryKey: queryKeys.coverageHeatmapData.list({ programId }),
119 |     queryFn: async (): Promise<HeatmapMatrix> => {
120 |       // Parallel reads (was serial). evidence has NO course_id column, so we
121 |       // resolve each evidence row's course via its CLO (learning_outcomes
122 |       // .course_id) and use the real attainment column (score_percent on
123 |       // evidence).
124 |       const [closRes, coursesRes, evidenceRes] = await Promise.all([
125 |         supabase
126 |           .from("learning_outcomes")
127 |           .select("id, title, course_id")
128 |           .eq("type", "CLO"),
129 |         supabase.from("courses").select("id, name"),
130 |         supabase.from("evidence").select("clo_id, score_percent"),
131 |       ]);
132 |       if (closRes.error) throw closRes.error;
133 |       if (coursesRes.error) throw coursesRes.error;
134 |       if (evidenceRes.error) throw evidenceRes.error;
135 |
136 |       const clos = closRes.data ?? [];
137 |       const cloToCourse = new Map(clos.map((c) => [c.id, c.course_id ?? ""]));
138 |
139 |       return buildHeatmapMatrix(
140 |         clos.map((c) => ({ id: c.id, title: c.title })),
141 |         (coursesRes.data ?? []).map((c) => ({ id: c.id, name: c.name })),
142 |         (evidenceRes.data ?? []).map((e) => ({
143 |           clo_id: e.clo_id,
144 |           course_id: cloToCourse.get(e.clo_id) ?? "",
145 |           score_percent: Number(e.score_percent ?? 0),
146 |         })))
147 |     );
148 |   },
149 |   enabled: !!programId,
150 |   staleTime: 5 * 60_000,
151 | });
152 | });
153 |

```

FILE: src/hooks/useWeeklyGoals.ts

TYPE: .ts | LINES: 1-118 | SHA256: c4fb07f10efb6d51280cc41ce40f09d32148e93bdd0eb226a7551799875db1a4

[illegible]

```

22   targetValue: Number(row.target_value ?? 0),
23 };
24 }
25 // % % % useWeeklyGoals - fetch goals for a specific week % % % % % % % % % % % % % % % % % % % % % % % % % % % %
26 export const useWeeklyGoals = (
27   studentId: string | undefined,
28   weekStartDate: string | undefined
29 ) => {
30   return useQuery({
31     queryKey: queryKeys.weeklyGoals.list({
32       studentId: studentId ?? "",
33       weekStartDate: weekStartDate ?? "",
34     }),
35     enabled: !!studentId && !weekStartDate,
36     // Weekly goals change only via explicit save; 2-min staleTime prevents
37     // redundant fetches on planner page re-mounts.
38     staleTime: 120_000,
39     queryFn: async (): Promise<WeeklyGoal[]> => {
40       if (!studentId || !weekStartDate) return [];
41
42       const { data, error } = await supabase
43         .from("weekly_goals")
44         .select(
45           `id, student_id, week_start, week_start_date, goal_type, goal_text, target_value, created_at`
46         )
47         .eq("student_id", studentId)
48         .or(
49           `week_start_date.eq.${weekStartDate}, week_start.eq.${weekStartDate}`
50         );
51
52       if (error) throw error;
53
54       return (data ?? []).map((row) => mapGoal(row as Record<string, unknown>));
55     },
56   });
57 };
58 // % % % useSaveWeeklyGoals - upsert up to 3 goals for a week % % % % % % % % % % % % % % % % % % % % % % % % % % % %
59 export const useSaveWeeklyGoals = () => {
60   const queryClient = useQueryClient();
61   const { user } = useAuth();
62
63   return useMutation({
64     mutationFn: async (
65       goals: CreateWeeklyGoalInput[]
66 ): Promise<WeeklyGoal[]> => {
67     if (!user) throw new Error("Not authenticated");
68
69     if (goals.length > 3) {
70       throw new Error("Maximum 3 goals per week");
71     }
72
73     // Upsert each goal (unique constraint on student_id + week_start_date + goal_type
74     // - see migration 20260902000002_weekly_goals_unique_constraint.sql)
75     const results: WeeklyGoal[] = [];
76
77     for (const goal of goals) {
78       const goalText = `${goal.goalType}: ${goal.targetValue}`;
79       const { data, error } = await supabase
80         .from("weekly_goals")
81         .upsert(
82           {
83             student_id: user.id,
84             week_start: goal.weekStartDate,
85             week_start_date: goal.weekStartDate,
86             goal_type: goal.goalType,
87             goal_text: goalText,
88             target_value: goal.targetValue,
89           } as never,
90           {
91             onConflict: "student_id, week_start_date, goal_type",
92           }
93         )
94         .select()
95         .single();
96
97       if (error) throw error;
98       results.push(mapGoal(data as Record<string, unknown>));
99     }
100
101     return results;
102   },
103   onSuccess: () => {
104     queryClient.invalidateQueries({
105       queryKey: queryKeys.weeklyGoals.lists(),
106     });
107     toast.success("Weekly goals saved");
108   },
109   onError: (error: Error) => {
110     toast.error(error.message);
111   },
112 });
113 };

```

## FILE: src/hooks/useWeeklyGoalXP.ts

TYPE: .ts | LINES: 1-154 | SHA256: 4c2a0f58104d902240db1215fefa7ea47ea8b9514abdbf1b3782e29e32833336

```
1 | // =====
2 | // useWeeklyGoalXP - Awards 25 XP per weekly goal that crosses its target.
3 | // Idempotent: relies on award-xp Edge Function reference_id deduplication.
4 | // Triggered after session/task completion mutations.
5 | // =====
6 |
7 | import { supabase } from "@lib/supabase";
8 | import { calculateGoalProgress, getWeekStartDate } from "@lib/plannerUtils";
9 | import type { GoalProgress, GoalType, PlannerTask } from "@types/planner";
10 |
11 | interface MetGoalRow {
12 |   id: string;
13 |   student_id: string;
14 |   week_start_date: string | null;
15 |   week_start: string | null;
16 |   goal_type: GoalType;
17 |   target_value: number;
18 | }
19 |
20 | /**
21 |  * After a session or task completion, fetch this week's goals and award
22 |  * 25 XP for each goal whose progress now meets the target.
23 |  *
24 |  * The award-xp Edge Function dedupes by reference_id (`weekly_goal:{goalId}`),
25 |  * so re-invocation across multiple completions on the same goal is safe.
26 |  *
27 |  * Returns the goals that were just met (for badge check / toast handling).
28 |  */
29 | export async function awardWeeklyGoalXPIfMet(
30 |   studentId: string
31 | ): Promise<GoalProgress[]> {
32 |   const weekStartDate = getWeekStartDate(new Date());
33 |
34 |   // 1. Fetch this week's goals
35 |   const { data: goalRows, error: goalsErr } = await supabase
36 |     .from("weekly_goals")
37 |     .select(
38 |       "id, student_id, week_start, week_start_date, goal_type, target_value"
39 |     )
40 |     .eq("student_id", studentId)
41 |     .or(`week_start_date.eq.${weekStartDate}, week_start.eq.${weekStartDate}`);
42 |
43 |   if (goalsErr || !goalRows || goalRows.length === 0) return [];
44 |
45 |   // 2. Fetch this week's sessions and tasks for progress calculation
46 |   const weekEnd = new Date(weekStartDate);
47 |   weekEnd.setUTCDate(weekEnd.getUTCDate() + 7);
48 |   const weekEndStr = weekEnd.toISOString().slice(0, 10);
49 |
50 |   const [sessionsRes, tasksRes] = await Promise.all([
51 |     supabase
52 |       .from("study_sessions")
53 |       .select("id, status, actual_duration_minutes, planned_date")
54 |       .eq("student_id", studentId)
55 |       .gte("planned_date", weekStartDate)
56 |       .lt("planned_date", weekEndStr),
57 |     supabase
58 |       .from("planner_tasks")
59 |       .select("id, status, due_date")
60 |       .eq("student_id", studentId)
61 |       .gte("due_date", weekStartDate)
62 |       .lt("due_date", weekEndStr),
63 |   ]);
64 |
65 |   if (sessionsRes.error || tasksRes.error) return [];
66 |
67 |   // Map to domain shape used by calculateGoalProgress
68 |   const sessions = (sessionsRes.data ?? []).map((s) => ({
69 |     id: s.id as string,
70 |     studentId,
71 |     courseId: null as string | null,
72 |     title: "",
73 |     description: null,
74 |     plannedDate: s.planned_date as string,
75 |     plannedStartTime: null,
76 |     plannedDurationMinutes: 0,
77 |     actualStartAt: null,
78 |     actualEndAt: null,
79 |     actualDurationMinutes: (s.actual_duration_minutes as number) ?? null,
80 |     timerMode: "pomodoro" as const,
81 |     status: s.status as "planned" | "in_progress" | "completed" | "cancelled",
82 |     satisfactionRating: null,
83 |     cloIds: null,
84 |     createdAt: "",
85 |   }));
86 |
87 |   const tasks = (tasksRes.data ?? []).map((t) => ({
88 |     id: t.id as string,
89 |     studentId,
90 |     title: "",
91 |     description: null,
92 |     dueDate: t.due_date as string,
93 |     priority: "medium" as const,
94 |     status: t.status as PlannerTask["status"],
95 |     courseId: null,
96 |     completedAt: null,
97 |     createdAt: "",
98 |   }));
99 |
100 |   // 3. For each goal, calculate progress and award XP if met
101 |   const metGoals: GoalProgress[] = [];
```





[illegible]

```

152         return mapTask({
153             ...r,
154             course_name: courseData?.name ?? undefined,
155         });
156     });
157 },
158 },
159 // 3. Assignment deadlines for the week (from enrolled courses)
160 {
161     queryKey: queryKeys.assignments.list({
162         studentId: studentId ?? "",
163         weekStartDate: weekStartDate ?? "",
164         type: "deadlines",
165     }),
166     enabled,
167     queryFn: async (): Promise<UpcomingDeadline[]> => {
168         // First get enrolled course IDs
169         const { data: enrollments, error: enrollError } = await supabase
170             .from("student_courses")
171             .select("course_id")
172             .eq("student_id", studentId!);
173
174         if (enrollError) throw enrollError;
175
176         const courseIds = (enrollments ?? []).map(
177             (e) => e.course_id as string
178         );
179
180         if (courseIds.length === 0) return [];
181
182         const { data, error } = await supabase
183             .from("assignments")
184             .select("id, title, due_date, courses(name)")
185             .in("course_id", courseIds)
186             .gte("due_date", weekStartDate!)
187             .lte("due_date", weekEnd)
188             .order("due_date", { ascending: true });
189
190         if (error) throw error;
191
192         const now = new Date();
193         return (data ?? []).map((row) => {
194             const r = row as Record<string, unknown>;
195             const courseData = r.courses as Record<string, unknown> | null;
196             return {
197                 id: r.id as string,
198                 title: r.title as string,
199                 courseName: (courseData?.name as string) ?? "",
200                 dueDate: r.due_date as string,
201                 urgency: getDeadlineUrgency(r.due_date as string, now),
202             };
203         });
204     },
205 },
206 // 4. Weekly goals
207 {
208     queryKey: queryKeys.weeklyGoals.list({
209         studentId: studentId ?? "",
210         weekStartDate: weekStartDate ?? "",
211     }),
212     enabled,
213     queryFn: async (): Promise<WeeklyGoal[]> => {
214         const { data, error } = await supabase
215             .from("weekly_goals")
216             .select(
217                 "id, student_id, week_start, week_start_date, goal_type, goal_text, target_value, created_at"
218             )
219             .eq("student_id", studentId!)
220             .or(
221                 `week_start_date.eq.${weekStartDate!}, week_start.eq.${weekStartDate!}`
222             );
223
224         if (error) throw error;
225
226         return (data ?? []).map((row) =>
227             mapGoal(row as Record<string, unknown>)
228         );
229     },
230 },
231 ],
232 });
233
234 const [sessionsQuery, tasksQuery, deadlinesQuery, goalsQuery] = results;
235
236 return {
237     sessions: sessionsQuery.data ?? [],
238     tasks: tasksQuery.data ?? [],
239     deadlines: deadlinesQuery.data ?? [],
240     goals: goalsQuery.data ?? [],
241     isLoading: results.some((r) => r.isLoading),
242     isError: results.some((r) => r.isError),
243 };
244 };
245

```

FILE: [src/hooks/useWeeklyProgress.ts](#)

TYPE: .ts | LINES: 1-281 | SHA256: d08341d068ff10cd77328ed86ad733a3942bd287a0a172051d983102b40aa96c

```

1 | // =====
2 | // useWeeklyProgress – Compute weekly progress summary with breakdowns
3 | // =====
4 |
5 | import { useQuery } from "@tanstack/react-query";
6 | import { supabase } from "@lib/supabase";

```

[illegible]

[illegible]

```

215 |         view: "progress",
216 |     },
217 |     enabled,
218 |     queryFn: async (): Promise<WeeklyGoal[]> => {
219 |         const { data, error } = await supabase
220 |             .from("weekly_goals")
221 |             .select(
222 |                 "id, student_id, week_start, week_start_date, goal_type, target_value"
223 |             )
224 |             .eq("student_id", studentId!)
225 |             .or(
226 |                 `week_start_date.eq.${weekStartDate!}, week_start.eq.${weekStartDate!}`
227 |             );
228 |
229 |         if (error) throw error;
230 |
231 |         return (data ?? []).map((row) => {
232 |             const r = row as Record<string, unknown>;
233 |             return {
234 |                 id: r.id as string,
235 |                 studentId: r.student_id as string,
236 |                 weekStartDate:
237 |                     (r.week_start_date as string) ?? (r.week_start as string) ?? "",
238 |                 goalType: (r.goal_type as WeeklyGoal["goalType"]) ?? "study_hours",
239 |                 targetValue: Number(r.target_value ?? 0),
240 |             };
241 |         });
242 |     },
243 | });
244 |
245 | // Compute progress data
246 | const sessions = sessionsQuery.data ?? [];
247 | const tasks = tasksQuery.data ?? [];
248 | const goals = goalsQuery.data ?? [];
249 |
250 | const completedSessions = sessions.filter((s) => s.status === "completed");
251 | const completedTasks = tasks.filter((t) => t.status === "done");
252 |
253 | const totalStudyMinutes = completedSessions.reduce(
254 |     (sum, s) => sum + (s.actualDurationMinutes ?? 0),
255 |     0
256 | );
257 |
258 | const courseBreakdown = computeCourseBreakdown(sessions);
259 | const cloBreakdown = computeCLOBreakdown(sessions);
260 |
261 | const goalProgress = goals.map((goal) =>
262 |     calculateGoalProgress(goal, sessions, tasks)
263 | );
264 |
265 | const progress: WeeklyProgressData = {
266 |     totalStudyMinutes,
267 |     sessionsCompleted: completedSessions.length,
268 |     tasksCompleted: completedTasks.length,
269 |     courseBreakdown,
270 |     cloBreakdown,
271 | };
272 |
273 | return {
274 |     progress,
275 |     goalProgress,
276 |     isLoading:
277 |         sessionsQuery.isLoading || tasksQuery.isLoading || goalsQuery.isLoading,
278 |     isError: sessionsQuery.isError || tasksQuery.isError || goalsQuery.isError,
279 | };
280 | };
281 |

```

## FILE: src/hooks/useWellnessGoals.ts

TYPE: .ts | LINES: 1-145 | SHA256: ee74b43a4e4e2e3c2f5697510fd4975f18122c956c70bbc9a439debe0d272103

```

1 | import { useMemo } from "react";
2 | import { useMutation, useQueryClient } from "@tanstack/react-query";
3 | import { supabase } from "@lib/supabase";
4 | import { queryKeys } from "@lib/queryKeys";
5 | import { toast } from "sonner";
6 | import { useWellnessPreferences } from "@hooks/useWellnessPreferences";
7 | import { WELLNESS_UNITS, computeWellnessProgress } from "@lib/wellnessTips";
8 | import type {
9 |     WellnessHabitType,
10 |     WellnessTarget,
11 |     WellnessHabitLog,
12 | } from "@types/habits";
13 |
14 | /**
15 |  * Returns wellness targets for all enabled habits.
16 |  */
17 | export const useWellnessGoals = (studentId: string | undefined) => {
18 |     const { data: preferences, isLoading } = useWellnessPreferences(studentId);
19 |
20 |     const targets = useMemo((): WellnessTarget[] => {
21 |         if (!preferences) return [];
22 |         const habitTargets = preferences.habitTargets ?? {};
23 |         return preferences.enabledHabits
24 |             .filter((ht) => habitTargets[ht])
25 |             .map((ht) => ({
26 |                 habitType: ht,
27 |                 targetValue: habitTargets[ht]!.value,
28 |                 unit: habitTargets[ht]!.unit ?? WELLNESS_UNITS[ht],
29 |             }));
30 |     }, [preferences]);
31 |
32 |     return { data: targets, isLoading };
33 | };

```

```

34 |
35 | /**
36 |  * Computes daily progress for each enabled habit with a target.
37 |  */
38 | export const useDailyProgress = (
39 |   studentId: string | undefined,
40 |   todayLogs: WellnessHabitLog[]
41 | ) => {
42 |   const { data: preferences } = useWellnessPreferences(studentId);
43 |
44 |   return useMemo(() : Record<
45 |     WellnessHabitType,
46 |     { progress: number; logged: number; target: number; unit: string }
47 |   ) => {
48 |     const result = {} as Record<
49 |       WellnessHabitType,
50 |       { progress: number; logged: number; target: number; unit: string }
51 |     >;
52 |     if (!preferences) return result;
53 |
54 |     const habitTargets = preferences.habitTargets ?? {};
55 |
56 |     for (const ht of preferences.enabledHabits) {
57 |       const targetInfo = habitTargets[ht];
58 |       if (!targetInfo) continue;
59 |
60 |       const log = todayLogs.find((l) => l.wellnessType === ht);
61 |       const loggedValue = log?.value ?? 0;
62 |       const targetValue = targetInfo.value;
63 |       const unit = targetInfo.unit ?? WELLNESS_UNITS[ht];
64 |
65 |       result[ht] = {
66 |         progress: computeWellnessProgress(loggedValue, targetValue),
67 |         logged: loggedValue,
68 |         target: targetValue,
69 |         unit,
70 |       };
71 |     }
72 |
73 |     return result;
74 |   }, [preferences, todayLogs]);
75 | };
76 |
77 | /**
78 |  * Mutation to set or update a wellness target for a specific habit.
79 |  */
80 | export const useUpdateWellnessGoal = () => {
81 |   const queryClient = useQueryClient();
82 |
83 |   return useMutation({
84 |     mutationFn: async ({
85 |       studentId,
86 |       habitType,
87 |       targetValue,
88 |       unit,
89 |     }) : {
90 |       studentId: string;
91 |       habitType: WellnessHabitType;
92 |       targetValue: number;
93 |       unit: string;
94 |     } => {
95 |       const { data: current, error: fetchError } = await supabase
96 |         .from("student_wellness_preferences")
97 |         .select("habit_targets")
98 |         .eq("student_id", studentId)
99 |         .maybeSingle();
100 |
101 |       if (fetchError) {
102 |         console.error("Failed to fetch habit targets:", fetchError.message);
103 |         throw fetchError;
104 |       }
105 |
106 |       const existing = (current?.habit_targets ?? {}) as Record<
107 |         string,
108 |         { value: number; unit: string }
109 |       >;
110 |       const updated = {
111 |         ...existing,
112 |         [habitType]: { value: targetValue, unit },
113 |       };
114 |
115 |       // Remove target if value is 0
116 |       if (targetValue <= 0) {
117 |         delete updated[habitType];
118 |       }
119 |
120 |       const { error } = await supabase
121 |         .from("student_wellness_preferences")
122 |         .upsert(
123 |           {
124 |             student_id: studentId,
125 |             habit_targets: updated,
126 |             updated_at: new Date().toISOString(),
127 |           },
128 |           { onConflict: "student_id" }
129 |         );
130 |
131 |       if (error) throw error;
132 |     },
133 |     onSuccess: (_data, variables) => {
134 |       queryClient.invalidateQueries({
135 |         queryKey: queryKeys.wellness.preferences(variables.studentId),
136 |       });
137 |       toast.success("Goal updated");

```

```

138 |     },
139 |     onError: (error: Error) => {
140 |       toast.error("Failed to update goal");
141 |       console.error("Update goal error:", error);
142 |     },
143 |   });
144 | };
145 |

```

## FILE: src/hooks/useWellnessHabits.ts

TYPE: .ts | LINES: 1-121 | SHA256: 1235c8687771280cd7371cbf41e8b926fc7e71bbc6534bfb5ad184fb1e533df4

```

1 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { toast } from "sonner";
5 | import type { WellnessHabitLog, WellnessHabitType } from "@types/habits";
6 |
7 | export const useWellnessHabitLogs = (
8 |   studentId: string | undefined,
9 |   date: string
10 | ) => {
11 |   return useQuery({
12 |     queryKey: queryKeys.wellness.logs(studentId ?? "", date),
13 |     enabled: !!studentId && !!date,
14 |     // Habit logs for a given day change only via explicit completion (mutation
15 |     // invalidates); staleTime prevents re-fetches on HabitTracker re-mounts.
16 |     staleTime: 120_000,
17 |     queryFn: async (): Promise<WellnessHabitLog[]> => {
18 |       if (!studentId) return [];
19 |
20 |       const { data, error } = await supabase
21 |         .from("wellness_habit_logs")
22 |         .select("id, student_id, date, wellness_type, value, completed_at")
23 |         .eq("student_id", studentId)
24 |         .eq("date", date);
25 |
26 |       if (error) throw error;
27 |
28 |       return (data ?? []).map((row) => ({
29 |         id: row.id as string,
30 |         studentId: row.student_id as string,
31 |         date: row.date as string,
32 |         wellnessType: row.wellness_type as WellnessHabitType,
33 |         value: row.value != null ? Number(row.value) : null,
34 |         completedAt: row.completed_at as string,
35 |       }));
36 |     },
37 |   });
38 | };
39 |
40 | interface LogWellnessHabitInput {
41 |   studentId: string;
42 |   wellnessType: WellnessHabitType;
43 |   value?: number | null;
44 |   date: string;
45 | }
46 |
47 | export const useLogWellnessHabit = () => {
48 |   const queryClient = useQueryClient();
49 |
50 |   return useMutation({
51 |     mutationFn: async (input: LogWellnessHabitInput) => {
52 |       // 1. Insert wellness habit log
53 |       const { data: log, error: insertError } = await supabase
54 |         .from("wellness_habit_logs")
55 |         .insert({
56 |           student_id: input.studentId,
57 |           date: input.date,
58 |           wellness_type: input.wellnessType,
59 |           value: input.value ?? null,
60 |           completed_at: new Date().toISOString(),
61 |         })
62 |         .select()
63 |         .single();
64 |
65 |       if (insertError) throw insertError;
66 |
67 |       // 2. Invoke award-xp Edge Function with source 'wellness_habit'
68 |       let xpAwarded = 0;
69 |       try {
70 |         const { data: xpResult } = await supabase.functions.invoke("award-xp", {
71 |           body: {
72 |             student_id: input.studentId,
73 |             xp_amount: 0, // Edge Function looks up wellness_xp_amount from institution_settings
74 |             source: "wellness_habit",
75 |             reference_id: log.id,
76 |           },
77 |         });
78 |         xpAwarded = xpResult?.xp_earned ?? 0;
79 |       } catch {
80 |         // XP award is fire-and-forget – don't block the flow
81 |         console.error("[useLogWellnessHabit] award-xp invocation failed");
82 |       }
83 |
84 |       // 3. Invoke check-badges Edge Function with trigger 'habit_log'
85 |       try {
86 |         await supabase.functions.invoke("check-badges", {
87 |           body: {
88 |             student_id: input.studentId,
89 |             trigger: "habit_log",
90 |           },
91 |         });
92 |       } catch {

```

```

93 |         console.error("[useLogWellnessHabit] check-badges invocation failed");
94 |     }
95 |
96 |     return { log, xpAwarded };
97 | },
98 | onSuccess: ({ xpAwarded }, variables) => {
99 |     // 4. Invalidate heatmap + wellness queries
100 |     queryClient.invalidateQueries({ queryKey: queryKeys.heatmap.all });
101 |     queryClient.invalidateQueries({
102 |         queryKey: queryKeys.wellness.logs(variables.studentId, variables.date),
103 |     });
104 |     queryClient.invalidateQueries({
105 |         queryKey: queryKeys.studentGamification.lists(),
106 |     });
107 |     queryClient.invalidateQueries({ queryKey: queryKeys.badges.lists() });
108 |
109 |     // 5. Show XP toast on success
110 |     if (xpAwarded > 0) {
111 |         toast.success(`+${xpAwarded} XP earned!`);
112 |     } else {
113 |         toast.success("Wellness habit logged");
114 |     }
115 | },
116 | onError: (error: Error) => {
117 |     toast.error(error.message);
118 | },
119 | });
120 | };
121 |

```

## FILE: src/hooks/useWellnessPreferences.ts

TYPE: .ts | LINES: 1-110 | SHA256: 7a022d25b0ea9779fcd3fde7af3420eb0fcc44eb2240872b177be6f26abf4030

```

1 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { toast } from "sonner";
5 | import type { WellnessPreferences, WellnessHabitType } from "@types/habits";
6 |
7 | export const useWellnessPreferences = (studentId: string | undefined) => {
8 |     return useQuery({
9 |         queryKey: queryKeys.wellness.preferences(studentId ?? ""),
10 |         enabled: !!studentId,
11 |         queryFn: async () => Promise<WellnessPreferences | null> => {
12 |             if (!studentId) return null;
13 |
14 |             const { data, error } = await supabase
15 |                 .from("student_wellness_preferences")
16 |                 .select(
17 |                     "id, student_id, enabled_habits, parent_visibility, habit_targets, reminder_times, dismissed_onboard
18 |                 )
19 |                 .eq("student_id", studentId)
20 |                 .maybeSingle();
21 |
22 |             if (error) throw error;
23 |             if (!data) return null;
24 |
25 |             return {
26 |                 id: data.id as string,
27 |                 studentId: data.student_id as string,
28 |                 enabledHabits: (data.enabled_habits ?? []) as WellnessHabitType[],
29 |                 parentVisibility: data.parent_visibility as boolean,
30 |                 habitTargets: (data.habit_targets ?? {}) as Record<
31 |                     string,
32 |                     { value: number; unit: string }
33 |                 >,
34 |                 reminderTimes: (data.reminder_times ?? {}) as Record<string, string>,
35 |                 dismissedOnboardingTips: (data.dismissed_onboarding_tips ??
36 |                     []) as string[],
37 |             };
38 |         },
39 |     });
40 | };
41 |
42 | interface UpdatePreferencesInput {
43 |     studentId: string;
44 |     enabledHabits: WellnessHabitType[];
45 |     parentVisibility: boolean;
46 | }
47 |
48 | export const useUpdateWellnessPreferences = () => {
49 |     const queryClient = useQueryClient();
50 |
51 |     return useMutation({
52 |         mutationFn: async (input: UpdatePreferencesInput) => {
53 |             const { data, error } = await supabase
54 |                 .from("student_wellness_preferences")
55 |                 .upsert(
56 |                     {
57 |                         student_id: input.studentId,
58 |                         enabled_habits: input.enabledHabits,
59 |                         parent_visibility: input.parentVisibility,
60 |                         updated_at: new Date().toISOString(),
61 |                     },
62 |                     { onConflict: "student_id" }
63 |                 )
64 |                 .select()
65 |                 .single();
66 |
67 |             if (error) throw error;
68 |             return data;
69 |         },
70 |         // Optimistic UI (spec: dashboard-and-ux-performance, Req 7): the
71 |         // enabled-habits / parent-visibility toggles are deterministic settings, so

```



```

72 | // reflect them immediately. Snapshot !' setQueryData !' rollback on error !'
73 | // invalidate on settle. Excluded from optimism: the XP/badge side effects of
74 | // habit logging (non-deterministic; Req 7.3) – those stay confirm-then-render.
75 | onMutate: async (input) => {
76 |   const key = queryKeys.wellness.preferences(input.studentId);
77 |   await queryClient.cancelQueries({ queryKey: key });
78 |   const previous = queryClient.getQueryData<WellnessPreferences | null>({
79 |     key
80 |   });
81 |   queryClient.setQueryData<WellnessPreferences | null>(key, (old) =>
82 |     old
83 |     ? {
84 |       ...old,
85 |       enabledHabits: input.enabledHabits,
86 |       parentVisibility: input.parentVisibility,
87 |     }
88 |     : old
89 |   );
90 |   return { previous };
91 | },
92 | onError: (error: Error, input, context) => {
93 |   // Roll back to the snapshot so the UI never shows a wrong toggle state.
94 |   queryClient.setQueryData(
95 |     queryKeys.wellness.preferences(input.studentId),
96 |     context?.previous
97 |   );
98 |   toast.error(error.message);
99 | },
100 | onSettled: (_data, _error, input) => {
101 |   queryClient.invalidateQueries({
102 |     queryKey: queryKeys.wellness.preferences(input.studentId),
103 |   });
104 | },
105 | onSuccess: () => {
106 |   toast.success("Wellness preferences updated");
107 | },
108 | });
109 | };
110 |

```

## FILE: src/hooks/useWellnessReminders.ts

TYPE: .ts | LINES: 1-94 | SHA256: acd001468dd7673e0583317ab377ca7e47e9a004a3d0d4c502fb43993a9194ad

```

1 | import { useMemo } from "react";
2 | import { useMutation, useQueryClient } from "@tanstack/react-query";
3 | import { supabase } from "@lib/supabase";
4 | import { queryKeys } from "@lib/queryKeys";
5 | import { toast } from "sonner";
6 | import { useWellnessPreferences } from "@hooks/useWellnessPreferences";
7 | import type { WellnessHabitType, WellnessReminderConfig } from "@types/habits";
8 |
9 | /**
10 |  * Returns reminder configs for all enabled habits.
11 |  */
12 | export const useWellnessReminders = (studentId: string | undefined) => {
13 |   const { data: preferences, isLoading } = useWellnessPreferences(studentId);
14 |
15 |   const reminders = useMemo((): WellnessReminderConfig[] => {
16 |     if (!preferences) return [];
17 |     const times = preferences.reminderTimes ?? {};
18 |     return preferences.enabledHabits.map((habitType) => ({
19 |       habitType,
20 |       reminderTime: times[habitType] ?? null,
21 |       enabled: !!times[habitType],
22 |     }));
23 |   }, [preferences]);
24 |
25 |   return { data: reminders, isLoading };
26 | };
27 |
28 | /**
29 |  * Mutation to update a reminder time for a specific habit.
30 |  * Pass `null` as reminderTime to disable the reminder.
31 |  */
32 | export const useUpdateWellnessReminder = () => {
33 |   const queryClient = useQueryClient();
34 |
35 |   return useMutation({
36 |     mutationFn: async ({
37 |       studentId,
38 |       habitType,
39 |       reminderTime,
40 |     }: {
41 |       studentId: string;
42 |       habitType: WellnessHabitType;
43 |       reminderTime: string | null;
44 |     }) => {
45 |       // Fetch current reminder_times
46 |       const { data: current, error: fetchError } = await supabase
47 |         .from("student_wellness_preferences")
48 |         .select("reminder_times")
49 |         .eq("student_id", studentId)
50 |         .maybeSingle();
51 |
52 |       if (fetchError) {
53 |         console.error("Failed to fetch reminder times:", fetchError.message);
54 |         throw fetchError;
55 |       }
56 |
57 |       const existing = (current?.reminder_times ?? {}) as Record<
58 |         string,
59 |         string | null
60 |       >;
61 |       const updated = { ...existing };

```

```

62 |
63 |     if (reminderTime) {
64 |         updated[habitType] = reminderTime;
65 |     } else {
66 |         delete updated[habitType];
67 |     }
68 |
69 |     const { error } = await supabase
70 |       .from("student_wellness_preferences")
71 |       .upsert(
72 |         {
73 |           student_id: studentId,
74 |           reminder_times: updated,
75 |           updated_at: new Date().toISOString(),
76 |         },
77 |         { onConflict: "student_id" }
78 |       );
79 |
80 |     if (error) throw error;
81 |   },
82 |   onSuccess: (_data, variables) => {
83 |     queryClient.invalidateQueries({
84 |       queryKey: queryKeys.wellness.preferences(variables.studentId),
85 |     });
86 |     toast.success("Reminder updated");
87 |   },
88 |   onError: (error: Error) => {
89 |     toast.error("Failed to update reminder");
90 |     console.error("Update reminder error:", error);
91 |   },
92 | });
93 | };
94 |

```

## FILE: src/hooks/useWellnessTips.ts

TYPE: .ts | LINES: 1-94 | SHA256: cec3c00a8ac7f988b93126fa23c4ca2f9001f9658961d3f41577647a2f83c263

```

1 | import { useMemo } from "react";
2 | import { useMutation, useQueryClient } from "@tanstack/react-query";
3 | import { supabase } from "@lib/supabase";
4 | import { queryKeys } from "@lib/queryKeys";
5 | import { toast } from "sonner";
6 | import {
7 |   WELLNESS_TIPS,
8 |   getCurrentWellnessTip,
9 |   getOnboardingTip,
10 | } from "@lib/wellnessTips";
11 | import { useWellnessPreferences } from "@hooks/useWellnessPreferences";
12 | import type { WellnessHabitType, WellnessTip } from "@types/habits";
13 |
14 | /**
15 |  * Returns the current tip for a given habit type.
16 |  * Shows onboarding tip if not dismissed, otherwise the weekly rotating tip.
17 |  */
18 | export const useCurrentTip = (
19 |   habitType: WellnessHabitType,
20 |   studentId: string | undefined
21 | ): { tip: WellnessTip | null; isOnboarding: boolean } => {
22 |   const { data: preferences } = useWellnessPreferences(studentId);
23 |
24 |   return useMemo(() => {
25 |     const dismissed = preferences?.dismissedOnboardingTips ?? [];
26 |     const isDismissed = dismissed.includes(habitType);
27 |
28 |     if (!isDismissed) {
29 |       const onboarding = getOnboardingTip(habitType, WELLNESS_TIPS);
30 |       if (onboarding) return { tip: onboarding, isOnboarding: true };
31 |     }
32 |
33 |     const rotating = getCurrentWellnessTip(habitType, WELLNESS_TIPS);
34 |     return { tip: rotating, isOnboarding: false };
35 |   }, [habitType, preferences?.dismissedOnboardingTips]);
36 | };
37 |
38 | /**
39 |  * Mutation to dismiss an onboarding tip for a specific habit type.
40 |  */
41 | export const useDismissOnboardingTip = () => {
42 |   const queryClient = useQueryClient();
43 |
44 |   return useMutation({
45 |     mutationFn: async ({
46 |       studentId,
47 |       habitType,
48 |     }) => {
49 |       studentId: string;
50 |       habitType: WellnessHabitType;
51 |     },
52 |     // Fetch current dismissed list
53 |     const { data: current, error: fetchError } = await supabase
54 |       .from("student_wellness_preferences")
55 |       .select("dismissed_onboarding_tips")
56 |       .eq("student_id", studentId)
57 |       .maybeSingle();
58 |
59 |     if (fetchError) {
60 |       console.error("Failed to fetch dismissed tips:", fetchError.message);
61 |       throw fetchError;
62 |     }
63 |
64 |     const existing: string[] =
65 |       (current?.dismissed_onboarding_tips as string[]) ?? [];
66 |     if (existing.includes(habitType)) return;
67 |

```

```

68 |         const updated = [...existing, habitType];
69 |
70 |         const { error } = await supabase
71 |           .from("student_wellness_preferences")
72 |           .upsert(
73 |             {
74 |               student_id: studentId,
75 |               dismissed_onboarding_tips: updated,
76 |               updated_at: new Date().toISOString(),
77 |             },
78 |             { onConflict: "student_id" }
79 |           );
80 |
81 |         if (error) throw error;
82 |       },
83 |       onSuccess: (_data, variables) => {
84 |         queryClient.invalidateQueries({
85 |           queryKey: queryKeys.wellness.preferences(variables.studentId),
86 |         });
87 |       },
88 |       onError: (error: Error) => {
89 |         toast.error("Failed to dismiss tip");
90 |         console.error("Dismiss onboarding tip error:", error);
91 |       },
92 |     });
93 |   });
94 |

```

## FILE: src/hooks/useWellnessXpConfig.ts

TYPE: .ts | LINES: 1-78 | SHA256: bd18637d3539d3429b8400c067fc914216b77be418da64ecd2e1b03520a8597b

```

1 | import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
2 | import { supabase } from "@lib/supabase";
3 | import { queryKeys } from "@lib/queryKeys";
4 | import { logAuditEvent } from "@lib/auditLogger";
5 | import { useAuth } from "@hooks/useAuth";
6 | import { toast } from "sonner";
7 |
8 | export const useWellnessXpConfig = () => {
9 |   return useQuery({
10 |     queryKey: [...queryKeys.institutionSettings.all, "wellnessXp"],
11 |     queryFn: async (): Promise<number> => {
12 |       const { data, error } = await supabase
13 |         .from("institution_settings")
14 |         .select("wellness_xp_amount")
15 |         .limit(1)
16 |         .maybeSingle();
17 |
18 |       if (error) throw error;
19 |       return (data?.wellness_xp_amount as number) ?? 5;
20 |     },
21 |   });
22 | };
23 |
24 | export const useUpdateWellnessXpAmount = () => {
25 |   const queryClient = useQueryClient();
26 |   const { user } = useAuth();
27 |
28 |   return useMutation({
29 |     mutationFn: async (wellnessXpAmount: number) => {
30 |       const { data: existing, error: fetchError } = await supabase
31 |         .from("institution_settings")
32 |         .select("id, wellness_xp_amount")
33 |         .limit(1)
34 |         .maybeSingle();
35 |
36 |       if (fetchError) throw fetchError;
37 |       if (!existing) throw new Error("Institution settings not found");
38 |
39 |       const previousValue =
40 |         existing.wellness_xp_amount == null
41 |           ? null
42 |           : Number(existing.wellness_xp_amount);
43 |
44 |       const { error: updateError } = await supabase
45 |         .from("institution_settings")
46 |         .update({ wellness_xp_amount: wellnessXpAmount })
47 |         .eq("id", existing.id);
48 |
49 |       if (updateError) throw updateError;
50 |
51 |       try {
52 |         await logAuditEvent({
53 |           action: "update",
54 |           entity_type: "institution_settings",
55 |           entity_id: existing.id as string,
56 |           changes: {
57 |             wellness_xp_amount: { from: previousValue, to: wellnessXpAmount },
58 |           },
59 |           performed_by: user?.id ?? "unknown",
60 |         });
61 |       } catch (auditError) {
62 |         console.error("Audit logging failed (update succeeded):", auditError);
63 |       }
64 |
65 |       return { wellnessXpAmount };
66 |     },
67 |     onSuccess: () => {
68 |       queryClient.invalidateQueries({
69 |         queryKey: queryKeys.institutionSettings.all,
70 |       });
71 |       toast.success("Wellness XP configuration updated");
72 |     },
73 |     onError: (err: Error) => {

```

FILE: src/hooks/useXP.ts

FILE: src/hooks/useXPBalance.ts

[illegible]

# FILE: src/hooks/useXPEconomist.ts

TYPE: .ts | LINES: 1-115 | SHA256: ae1b679b549e621fca12d572c86f6c315292426e7580f9b1fb4640ee888623fc

```
1 | // =====
2 | // useXPEconomist - Earn/spend ratio, XP velocity, inflation indicator
3 | // Task 20.1
4 | // =====
5 |
6 | import { useQuery } from "@tanstack/react-query";
7 | import { supabase } from "@lib/supabase";
8 | import { queryKeys } from "@lib/queryKeys";
9 | import {
10 |   computeEarnSpendRatio,
11 |   type EarnSpendResult,
12 | } from "@lib/earnSpendRatioCalculator";
13 |
14 | export const useEarnSpendRatio = (institutionId?: string) => {
15 |   return useQuery({
16 |     queryKey: queryKeys.economist.list({ institutionId, type: "ratio" }),
17 |     queryFn: async (): Promise<EarnSpendResult> => {
18 |       const { data: earnData, error: earnErr } = await supabase
19 |         .from("xp_transactions")
20 |         .select("xp_amount");
21 |       if (earnErr) throw earnErr;
22 |
23 |       const totalEarned = (earnData ?? []).reduce(
24 |         (sum, row) => sum + (row.xp_amount ?? 0),
25 |         0
26 |       );
27 |
28 |       const { data: spendData, error: spendErr } = await supabase
29 |         .from("xp_purchases")
30 |         .select("xp_cost")
31 |         .neq("status", "refunded");
32 |       if (spendErr) throw spendErr;
33 |
34 |       const totalSpent = (spendData ?? []).reduce(
35 |         (sum, row) => sum + (row.xp_cost ?? 0),
36 |         0
37 |       );
38 |
39 |       return computeEarnSpendRatio({ totalEarned, totalSpent });
40 |     },
41 |     enabled: !!institutionId,
42 |     staleTime: 60_000,
43 |   });
44 | };
45 |
46 | export interface XPVelocityPoint {
47 |   week: string;
48 |   earned: number;
49 |   spent: number;
50 | }
51 |
52 | export const useXPVelocity = (institutionId?: string, weeks = 8) => {
53 |   return useQuery({
54 |     queryKey: queryKeys.economist.list({
55 |       institutionId,
56 |       type: "velocity",
57 |       weeks,
58 |     }),
59 |     queryFn: async (): Promise<XPVelocityPoint[]> => {
60 |       const now = new Date();
61 |       const startDate = new Date(now);
62 |       startDate.setDate(startDate.getDate() - weeks * 7);
63 |
64 |       const { data: earnData, error: earnErr } = await supabase
65 |         .from("xp_transactions")
66 |         .select("xp_amount, created_at")
67 |         .gte("created_at", startDate.toISOString());
68 |       if (earnErr) throw earnErr;
69 |
70 |       const { data: spendData, error: spendErr } = await supabase
71 |         .from("xp_purchases")
72 |         .select("xp_cost, purchased_at")
73 |         .neq("status", "refunded")
74 |         .gte("purchased_at", startDate.toISOString());
75 |       if (spendErr) throw spendErr;
76 |
77 |       const points: XPVelocityPoint[] = [];
78 |       for (let i = weeks - 1; i >= 0; i--) {
79 |         const weekStart = new Date(now);
80 |         weekStart.setDate(weekStart.getDate() - (i + 1) * 7);
81 |         const weekEnd = new Date(now);
82 |         weekEnd.setDate(weekEnd.getDate() - i * 7);
83 |
84 |         const earned = (earnData ?? []).
85 |           .filter((t) => {
86 |             const d = new Date(t.created_at);
87 |             return d >= weekStart && d < weekEnd;
88 |           })
89 |           .reduce((sum, t) => sum + (t.xp_amount ?? 0), 0);
90 |
91 |         const spent = (spendData ?? []).
92 |           .filter((t) => {
93 |             const d = new Date(t.purchased_at);
94 |             return d >= weekStart && d < weekEnd;
95 |           })
96 |           .reduce((sum, t) => sum + (t.xp_cost ?? 0), 0);
97 |
98 |         points.push({
99 |           week: weekStart.toISOString().slice(0, 10),
100 |           earned,
101 |           spent,
```

```

102 |         });
103 |     }
104 |
105 |     return points;
106 | },
107 |     enabled: !!institutionId,
108 |     staleTime: 60_000,
109 | });
110 | });
111 |
112 | export const useEarnSpendTimeSeries = (institutionId?: string, weeks = 12) => {
113 |     return useXPVelocity(institutionId, weeks);
114 | };
115 |

```

TYPE: .ts | LINES: 1-178 | SHA256: 17aa48ab5af8c594b4176f5cecfca3bafbb5620e288e236c990ce7ec9584ef02

```
// =====  
2 // useXPHistory - TanStack Query hooks for XP transaction history  
3 // Requirements: 45.3, 45.5  
4 // =====  
5  
6 import { useQuery } from "@tanstack/react-query";  
7 import { supabase } from "@/lib/supabase";  
8 import { queryKeys } from "@/lib/queryKeys";  
9 import { startOfDay, startOfWeek, startOfMonth } from "date-fns";  
10  
11 // % % % Types % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
12  
13 export type XPFilterPeriod = "today" | "this_week" | "this_month" | "all_time";  
14  
15 export interface XPTransaction {  
16   id: string;  
17   source: string;  
18   xp_amount: number;  
19   reference_id: string | null;  
20   note: string | null;  
21   created_at: string;  
22 }  
23  
24 export interface XPTransactionDisplay extends XPTransaction {  
25   source_label: string;  
26   reference_description: string;  
27 }  
28  
29 export interface XPCategorySummary {  
30   source: string;  
31   source_label: string;  
32   total_xp: number;  
33   count: number;  
34 }  
35  
36 // % % % Source Label Map % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
37  
38 const SOURCE_LABELS: Record<string, string> = {  
39   login: "Daily Login",  
40   submission: "On-time Submission",  
41   grade: "Grade Released",  
42   journal: "Journal Entry",  
43   streak_milestone: "Streak Milestone",  
44   perfect_day: "Perfect Day",  
45   first_attempt_bonus: "First Attempt Bonus",  
46   perfect_rubric: "Perfect Rubric Score",  
47   badge_earned: "Badge Earned",  
48   level_up: "Level Up",  
49   streak_freeze_purchase: "Streak Freeze Purchase",  
50   discussion_question: "Discussion Question",  
51   discussion_answer: "Discussion Answer",  
52   survey_completion: "Survey Completion",  
53   quiz_completion: "Quiz Completion",  
54   onboarding_personality: "Personality Assessment",  
55   onboarding_learning_style: "Learning Style Assessment",  
56   onboarding_baseline: "Baseline Test",  
57   onboarding_complete: "Onboarding Complete",  
58   onboarding_self_efficacy: "Self-Efficacy Assessment",  
59   onboarding_study_strategy: "Study Strategy Assessment",  
60   micro_assessment: "Micro Assessment",  
61   profile_complete: "Profile Complete",  
62   starter_session_complete: "Starter Session Complete",  
63 };  
64  
65 export function getSourceLabel(source: string): string {  
66   return (  
67     SOURCE_LABELS[source] ??  
68     source.replace(/_/g, " ").replace(/\b\w/g, (c) => c.toUpperCase())  
69   );  
70 }  
71  
72 // % % % Period Filter Helper % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
73  
74 function getPeriodStart(period: XPFilterPeriod): string | null {  
75   const now = new Date();  
76   switch (period) {  
77     case "today":  
78       return startOfDay(now).toISOString();  
79     case "this_week":  
80       return startOfWeek(now, { weekStartsOn: 1 }).toISOString();  
81     case "this_month":  
82       return startOfMonth(now).toISOString();  
83     case "all_time":  
84       return null;  
85   }  
86 }
```

[illegible]

FILE: src/index.css

TYPE: .css | LINES: 1-1024 | SHA256: 464346ec5cc79ab23445e082db2939a03babdbbf58565fbe82d6fa9b70503f4a

```
1 | @import "tailwindcss";
2 | @import "tw-animate-css";
3 | @import "shadcn/tailwind.css";
4 |
5 | @custom-variant dark (&is(.dark *));
6 | @custom-variant rtl (&where([dir="rtl"], [dir="rtl"] *));
7 |
8 | @theme {
```

```

9 | --font-sans: "Noto Sans", ui-sans-serif, system-ui, sans-serif;
10 | --radius-sm: 0.25rem;
11 | --radius-md: 0.375rem;
12 | --radius-lg: 0.5rem;
13 | --radius-xl: 0.75rem;
14 | }
15 |
16 | @theme inline {
17 |   --radius-sm: calc(var(--radius) - 4px);
18 |   --radius-md: calc(var(--radius) - 2px);
19 |   --radius-lg: var(--radius);
20 |   --radius-xl: calc(var(--radius) + 4px);
21 |   --radius-2xl: calc(var(--radius) + 8px);
22 |   --radius-3xl: calc(var(--radius) + 12px);
23 |   --radius-4xl: calc(var(--radius) + 16px);
24 |   --color-background: var(--background);
25 |   --color-foreground: var(--foreground);
26 |   --color-card: var(--card);
27 |   --color-card-foreground: var(--card-foreground);
28 |   --color-popover: var(--popover);
29 |   --color-popover-foreground: var(--popover-foreground);
30 |   --color-primary: var(--primary);
31 |   --color-primary-foreground: var(--primary-foreground);
32 |   --color-secondary: var(--secondary);
33 |   --color-secondary-foreground: var(--secondary-foreground);
34 |   --color-muted: var(--muted);
35 |   --color-muted-foreground: var(--muted-foreground);
36 |   --color-accent: var(--accent);
37 |   --color-accent-foreground: var(--accent-foreground);
38 |   --color-destructive: var(--destructive);
39 |   --color-border: var(--border);
40 |   --color-input: var(--input);
41 |   --color-ring: var(--ring);
42 |   --color-chart-1: var(--chart-1);
43 |   --color-chart-2: var(--chart-2);
44 |   --color-chart-3: var(--chart-3);
45 |   --color-chart-4: var(--chart-4);
46 |   --color-chart-5: var(--chart-5);
47 |   --color-sidebar: var(--sidebar);
48 |   --color-sidebar-foreground: var(--sidebar-foreground);
49 |   --color-sidebar-primary: var(--sidebar-primary);
50 |   --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
51 |   --color-sidebar-accent: var(--sidebar-accent);
52 |   --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
53 |   --color-sidebar-border: var(--sidebar-border);
54 |   --color-sidebar-ring: var(--sidebar-ring);
55 | }
56 |
57 | /* Canonical Shadcn semantic tokens, mapped 1:1 to the PROTOTYPE design system
58 |    (P0.2 token canonicalization). No neutral/grayscale "New York" defaults leak:
59 |    primary + ring are brand blue, surfaces are slate, cards are white. Values are
60 |    the prototype's (shared.css / tokens.css). Light + LTR is the prototype scope. */
61 | :root {
62 |   --radius: 0.75rem; /* prototype interactive radius (btn3d 12px); cards use explicit 20px */
63 |   --background: #f8fafc; /* prototype body - slate-50 */
64 |   --foreground: #0f172a; /* slate-900 */
65 |
66 |   /* === SIDEBAR TYPOGRAPHY & ICON TOKENS === */
67 |   --sidebar-font-family: "Noto Sans", sans-serif;
68 |   --sidebar-label-size: 14px;
69 |   --sidebar-label-weight: 600;
70 |   --sidebar-active-color: #075985;
71 |   --sidebar-primary-muted: #7c8da6;
72 |   --sidebar-secondary-color: #5f7390;
73 |   --sidebar-icon-muted: #91a4bd;
74 |   --card: #ffffff;
75 |   --card-foreground: #0f172a;
76 |   --popover: #ffffff;
77 |   --popover-foreground: #0f172a;
78 |   --primary: #3b82f6; /* brand blue - primary CTA / active / links */
79 |   --primary-foreground: #ffffff;
80 |   --secondary: #f1f5f9; /* slate-100 */
81 |   --secondary-foreground: #0f172a;
82 |   --muted: #f1f5f9; /* slate-100 */
83 |   --muted-foreground: #64748b; /* slate-500 */
84 |   --accent: #f1f5f9; /* slate-100 - subtle hover (never color-only) */
85 |   --accent-foreground: #0f172a;
86 |   --destructive: #ef4444; /* prototype destructive */
87 |   --border: #e2e8f0; /* slate-200 */
88 |   --input: #e2e8f0; /* slate-200 */
89 |   --ring: #3b82f6; /* brand focus ring */
90 |   --chart-1: #14b8a6; /* teal */
91 |   --chart-2: #3b82f6; /* blue */
92 |   --chart-3: #f59e0b; /* amber */
93 |   --chart-4: #22c55e; /* green */
94 |   --chart-5: #a855f7; /* violet */
95 |   --sidebar: #ffffff;
96 |   --sidebar-foreground: #0f172a;
97 |   --sidebar-primary: #3b82f6; /* brand */
98 |   --sidebar-primary-foreground: #ffffff;
99 |   --sidebar-accent: #f1f5f9; /* slate-100 hover */
100 |   --sidebar-accent-foreground: #0f172a;
101 |   --sidebar-border: #e2e8f0;
102 |   --sidebar-ring: #3b82f6;
103 | }
104 |
105 | .dark {
106 |   --background: oklch(0.145 0 0);
107 |   --foreground: oklch(0.985 0 0);
108 |   --card: oklch(0.205 0 0);
109 |   --card-foreground: oklch(0.985 0 0);
110 |   --popover: oklch(0.205 0 0);
111 |   --popover-foreground: oklch(0.985 0 0);
112 |   --primary: #3b82f6; /* brand blue (prototype is light-only; dark is parity) */

```



```

113 | --primary-foreground: #ffffff;
114 | --secondary: oklch(0.269 0 0);
115 | --secondary-foreground: oklch(0.985 0 0);
116 | --muted: oklch(0.269 0 0);
117 | --muted-foreground: oklch(0.708 0 0);
118 | --accent: oklch(0.269 0 0);
119 | --accent-foreground: oklch(0.985 0 0);
120 | --destructive: oklch(0.704 0.191 22.216);
121 | --border: oklch(1 0 0 / 10%);
122 | --input: oklch(1 0 0 / 15%);
123 | --ring: #3b82f6;
124 | --chart-1: oklch(0.488 0.243 264.376);
125 | --chart-2: oklch(0.696 0.17 162.48);
126 | --chart-3: oklch(0.769 0.188 70.08);
127 | --chart-4: oklch(0.627 0.265 303.9);
128 | --chart-5: oklch(0.645 0.246 16.439);
129 | --sidebar: oklch(0.205 0 0);
130 | --sidebar-foreground: oklch(0.985 0 0);
131 | --sidebar-primary: #3b82f6;
132 | --sidebar-primary-foreground: #ffffff;
133 | --sidebar-accent: oklch(0.269 0 0);
134 | --sidebar-accent-foreground: oklch(0.985 0 0);
135 | --sidebar-border: oklch(1 0 0 / 10%);
136 | --sidebar-ring: oklch(0.556 0 0);
137 | }
138 |
139 | @layer base {
140 |   * {
141 |     @apply border-border outline-ring/50;
142 |   }
143 |   html,
144 |   body {
145 |     font-family: var(--sidebar-font-family);
146 |     @apply bg-background text-foreground antialiased;
147 |   }
148 | }
149 |
150 | /* === CANONICAL SIDEBAR TYPOGRAPHY & ICON SYSTEM === */
151 |
152 | .app-sidebar {
153 |   font-family: var(--sidebar-font-family);
154 | }
155 |
156 | .sidebar-nav-label {
157 |   font-family: var(--sidebar-font-family);
158 |   font-size: var(--sidebar-label-size);
159 |   font-weight: var(--sidebar-label-weight);
160 |   font-style: normal;
161 |   font-stretch: 100%;
162 |   font-kerning: auto;
163 |   font-optical-sizing: auto;
164 |   font-variant: normal;
165 |   line-height: 1.4;
166 |   letter-spacing: normal;
167 |   text-transform: none;
168 |   text-decoration: none;
169 |   text-align: start;
170 |   opacity: 1;
171 | }
172 |
173 | .sidebar-item[data-active="true"] .sidebar-nav-label {
174 |   color: var(--sidebar-active-color);
175 |   font-weight: 600;
176 |   opacity: 1;
177 | }
178 |
179 | .sidebar-item[data-group="primary"][data-active="false"] .sidebar-nav-label {
180 |   color: var(--sidebar-primary-muted); /* #7c8da6 */
181 |   font-weight: 600;
182 |   opacity: 1;
183 | }
184 |
185 | .sidebar-item[data-variant="companion"][data-active="false"]
186 |   .sidebar-nav-label {
187 |   color: var(--sidebar-primary-muted); /* #7c8da6 */
188 |   font-weight: 600;
189 |   opacity: 1;
190 | }
191 |
192 | .sidebar-item[data-group="secondary"] .sidebar-nav-label {
193 |   color: var(--sidebar-secondary-color); /* #5f7390 */
194 |   font-weight: 600;
195 |   opacity: 1;
196 | }
197 |
198 | .sidebar-section-label {
199 |   font-family: var(--sidebar-font-family);
200 |   font-size: 10px;
201 |   font-weight: 800;
202 |   letter-spacing: 0.12em;
203 |   text-transform: uppercase;
204 |   color: #94a3b8;
205 |   opacity: 1;
206 | }
207 |
208 | /* --- Special Companion Icon Prominence (Requirement 1 & 3) --- */
209 | .sidebar-item[data-variant="companion"] .sidebar-icon-wrapper {
210 |   width: 34px;
211 |   height: 34px;
212 |   flex: 0 0 34px;
213 |   border-radius: 9999px;
214 |   opacity: 1;
215 |   filter: none;
216 | }

```

[illegible]

[illegible]

[illegible]

[illegible]

```

633 | transition-duration: 100ms !important;
634 | }
635 |
636 | /* Simplified view – hide non-essential elements */
637 | .simplified-view [data-focus-hide="true"] {
638 |   display: none !important;
639 | }
640 |
641 | /* Font size overrides */
642 | html.font-large {
643 |   font-size: 18px;
644 | }
645 |
646 | html.font-x-large {
647 |   font-size: 20px;
648 | }
649 |
650 | /* % % % Design-System Primitives (P0 – UI prototype migration) % % % % % % % % % % % % % % % % */
651 | /*
652 | * Additive, token-bound utilities introduced by the UI prototype migration
653 | * (spec: kiro/specs/ui-prototype-migration). They are OPT-IN – nothing uses
654 | * them until a component/page adopts them – so they cannot regress existing
655 | * screens. All are dark-mode + reduced-motion aware and reference existing
656 | * brand tokens (no new palette).
657 | */
658 |
659 | :root {
660 |   /* Darkened brand blue for the tactile button's pressable bottom edge. */
661 |   --tactile-edge: #025c88;
662 | }
663 |
664 | .dark {
665 |   --tactile-edge: #025c88;
666 | }
667 |
668 | /* Elevated card: soft layered shadow + subtle hover lift. Apply alongside the
669 |   Shadcn <Card> (or any container). Hover lift only on real hover pointers. */
670 | .card-elevated {
671 |   box-shadow: var(--shadow-md);
672 |   transition: box-shadow 0.2s ease, transform 0.2s ease;
673 | }
674 |
675 | @media (hover: hover) {
676 |   .card-elevated:hover {
677 |     box-shadow: var(--shadow-lg);
678 |     transform: translateY(-2px);
679 |   }
680 | }
681 |
682 | @media (prefers-reduced-motion: reduce) {
683 |   .card-elevated {
684 |     transition: none;
685 |   }
686 |   .card-elevated:hover {
687 |     transform: none;
688 |   }
689 | }
690 |
691 | /* Tactile (pressable) button surface – solid action palette (#0382bd / #026fa3 / #025c88).
692 |   Applied via the Button `tactile` variant. Keyboard focus ring is inherited from Button base. */
693 | .btn-tactile {
694 |   background-color: var(--action-primary, #0382bd);
695 |   color: var(--action-primary-text, #ffffff);
696 |   box-shadow: 0 4px 0 0 var(--action-primary-active, #025c88);
697 |   transition: transform 0.1s ease, box-shadow 0.1s ease,
698 |     background-color 0.1s ease;
699 | }
700 |
701 | @media (hover: hover) {
702 |   .btn-tactile:hover {
703 |     background-color: var(--action-primary-hover, #026fa3);
704 |     color: var(--action-primary-text, #ffffff);
705 |   }
706 | }
707 |
708 | .btn-tactile:active {
709 |   background-color: var(--action-primary-active, #025c88);
710 |   transform: translateY(2px);
711 |   box-shadow: 0 1px 0 0 var(--action-primary-active, #025c88);
712 |   color: var(--action-primary-text, #ffffff);
713 | }
714 |
715 | @media (prefers-reduced-motion: reduce) {
716 |   .btn-tactile {
717 |     transition: none;
718 |   }
719 |   .btn-tactile:active {
720 |     transform: none;
721 |   }
722 | }
723 |
724 | /* Mobile bottom tab bar (new chrome, < lg): reserve space at the bottom of the
725 |   scrollable content so the fixed bar (see MobileTabBar) never covers the last
726 |   items. Scoped by :has() so it applies ONLY when the bar is present (new chrome
727 |   + small screens), and never on desktop or with the flag off. */
728 | @media (max-width: 639.98px) {
729 |   body:has(.new-mobile-tabbar) main {
730 |     padding-bottom: calc(4.5rem + env(safe-area-inset-bottom));
731 |   }
732 | }
733 |
734 | /* The prototype sidebar scrolls when its role menu exceeds the viewport, but
735 |   keeps the rail visually clean instead of exposing native scrollbar chrome. */
736 | [data-tour="primary-nav"] {

```

[illegible]

[illegible]



```

945 | padding: 11px;
946 | border: 1.5px solid #e2e8f0;
947 | border-radius: 12px;
948 | background: #fff;
949 | font-size: 13.5px;
950 | font-weight: 700;
951 | color: #334155;
952 | cursor: pointer;
953 | transition: background-color 0.12s ease, border-color 0.12s ease;
954 | }
955 |
956 | @media (hover: hover) {
957 |   .sso:hover {
958 |     background: #f8fafc;
959 |     border-color: #cbd5e1;
960 |   }
961 | }
962 |
963 | .sso:disabled {
964 |   opacity: 0.6;
965 |   cursor: not-allowed;
966 | }
967 |
968 | /* Sign-up role segment – prototype `.rseg-b` (state via data-active). */
969 | .rseg-b {
970 |   flex: 1;
971 |   padding: 9px 6px;
972 |   border-radius: 12px;
973 |   border: 1.5px solid #e2e8f0;
974 |   background: #fff;
975 |   cursor: pointer;
976 |   text-align: center;
977 |   transition: border-color 0.12s ease, background-color 0.12s ease;
978 | }
979 |
980 | .rseg-b[data-active="true"] {
981 |   border-color: #0382bd;
982 |   background: #f0f9ff;
983 | }
984 |
985 | .rseg-b:disabled {
986 |   cursor: not-allowed;
987 |   opacity: 0.55;
988 | }
989 |
990 | /* Password strength meter – prototype `.strength`. */
991 | .auth-strength {
992 |   height: 5px;
993 |   border-radius: 3px;
994 |   background: #eef2f6;
995 |   overflow: hidden;
996 |   margin-top: 6px;
997 | }
998 |
999 | .auth-strength > div {
1000 |   height: 100%;
1001 |   width: 0;
1002 |   transition: width 0.25s ease, background-color 0.25s ease;
1003 | }
1004 |
1005 | /* Sign in / Create account toggle – prototype `.auth-tab` (state via data-active). */
1006 | .auth-tab {
1007 |   flex: 1;
1008 |   padding: 9px;
1009 |   border-radius: 10px;
1010 |   font-size: 13px;
1011 |   font-weight: 800;
1012 |   border: none;
1013 |   background: none;
1014 |   color: #64748b;
1015 |   cursor: pointer;
1016 |   transition: background-color 0.12s ease, color 0.12s ease;
1017 | }
1018 |
1019 | .auth-tab[data-active="true"] {
1020 |   background: #fff;
1021 |   color: #0f172a;
1022 |   box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
1023 | }
1024 |

```

FILE: [src/lib/\\_\\_tests\\_\\_/dashboardCache.test.ts](#)

TYPE: .ts | LINES: 1-62 | SHA256: 69b92ccc03830c6c33581789e412a8505db9cc986cf7accec3cf89eee5b523c53

```

1 | // @vitest-environment happy-dom
2 | import { describe, it, expect, beforeEach } from "vitest";
3 | import {
4 |   readCachedDashboard,
5 |   writeCachedDashboard,
6 |   clearCachedDashboard,
7 |   DASHBOARD_CACHE_MAX_AGE_MS,
8 | } from "@lib/dashboardCache";
9 |
10 | const USER_A = "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaaa";
11 | const USER_B = "bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb";
12 |
13 | interface Payload {
14 |   totalXP: number;
15 |   label: string;
16 | }
17 | const PAYLOAD_A: Payload = { totalXP: 100, label: "alice" };
18 |
19 | describe("dashboardCache", () => {
20 |   beforeEach(() => {

```

```

21 |     localStorage.clear();
22 | });
23 |
24 | it("round-trips a snapshot for its owner", () => {
25 |     writeCachedDashboard(USER_A, PAYLOAD_A);
26 |     const hit = readCachedDashboard<Payload>(USER_A);
27 |     expect(hit?.data).toEqual(PAYLOAD_A);
28 |     expect(typeof hit?.cachedAt).toBe("number");
29 | });
30 |
31 | it("never returns one user's snapshot to another (cross-user no-leak)", () => {
32 |     writeCachedDashboard(USER_A, PAYLOAD_A);
33 |     // User B must not read A's snapshot...
34 |     expect(readCachedDashboard<Payload>(USER_B)).toBeNull();
35 |     // ...and the foreign entry is dropped on the mismatched read.
36 |     expect(localStorage.getItem("edeviser.dashboard.student.v1")).toBeNull();
37 | });
38 |
39 | it("clears the snapshot", () => {
40 |     writeCachedDashboard(USER_A, PAYLOAD_A);
41 |     clearCachedDashboard();
42 |     expect(readCachedDashboard<Payload>(USER_A)).toBeNull();
43 | });
44 |
45 | it("ignores (and drops) a snapshot older than the max age", () => {
46 |     writeCachedDashboard(USER_A, PAYLOAD_A);
47 |     // Rewrite the stored envelope with an ancient cachedAt.
48 |     const key = "edeviser.dashboard.student.v1";
49 |     const raw = JSON.parse(localStorage.getItem(key) as string);
50 |     raw.cachedAt = Date.now() - (DASHBOARD_CACHE_MAX_AGE_MS + 1000);
51 |     localStorage.setItem(key, JSON.stringify(raw));
52 |
53 |     expect(readCachedDashboard<Payload>(USER_A)).toBeNull();
54 |     expect(localStorage.getItem(key)).toBeNull();
55 | });
56 |
57 | it("returns null on a corrupt entry without throwing", () => {
58 |     localStorage.setItem("edeviser.dashboard.student.v1", "{not json}");
59 |     expect(readCachedDashboard<Payload>(USER_A)).toBeNull();
60 | });
61 | });
62 |

```

FILE: [src/lib/\\_\\_tests\\_\\_/leaderboardGate.test.ts](#)

TYPE: .ts | LINES: 1-41 | SHA256: eb00b8264572ce051e97ccabcf33f59d77779777bcba7c5616f0433c27e3f853

```

1 | import { describe, it, expect } from "vitest";
2 | import { leaderboardState } from "../leaderboardGate";
3 |
4 | describe("leaderboardGate", () => {
5 |     describe("leaderboardState", () => {
6 |         it("locks when zero eligible students regardless of minimum (R6.2a)", () => {
7 |             expect(leaderboardState(0, 5)).toBe("locked");
8 |             // Even a zero/negative configured minimum must not unlock an empty cohort.
9 |             expect(leaderboardState(0, 0)).toBe("locked");
10 |             expect(leaderboardState(0, -3)).toBe("locked");
11 |         });
12 |
13 |         it("locks for negative eligible counts", () => {
14 |             expect(leaderboardState(-2, 5)).toBe("locked");
15 |         });
16 |
17 |         it("locks while eligible count is below the configured minimum (R6.1)", () => {
18 |             expect(leaderboardState(1, 5)).toBe("locked");
19 |             expect(leaderboardState(4, 5)).toBe("locked");
20 |         });
21 |
22 |         it("unlocks when eligible count meets the minimum (R6.2)", () => {
23 |             expect(leaderboardState(5, 5)).toBe("unlocked");
24 |         });
25 |
26 |         it("unlocks when eligible count exceeds the minimum (R6.2)", () => {
27 |             expect(leaderboardState(50, 5)).toBe("unlocked");
28 |         });
29 |
30 |         it("unlocks a single eligible student when the minimum is 1", () => {
31 |             expect(leaderboardState(1, 1)).toBe("unlocked");
32 |         });
33 |
34 |         it("treats non-finite inputs conservatively as locked", () => {
35 |             expect(leaderboardState(Number.NaN, 5)).toBe("locked");
36 |             expect(leaderboardState(10, Number.NaN)).toBe("locked");
37 |             expect(leaderboardState(Number.POSITIVE_INFINITY, 5)).toBe("locked");
38 |         });
39 |     });
40 | });
41 |

```

FILE: [src/lib/\\_\\_tests\\_\\_/loginAttemptTracker.test.ts](#)

TYPE: .ts | LINES: 1-134 | SHA256: 47525eaeb48ef6afd93f436e154f72a50c4d48e88841b8d0a157c9aeead6360e

```

1 | import { describe, it, expect, beforeEach, vi, afterEach } from "vitest";
2 | import {
3 |     isLocked,
4 |     getRemainingLockTime,
5 |     recordFailedAttempt,
6 |     clearAttempts,
7 |     LOGIN_ATTEMPT_CONFIG,
8 | } from "../loginAttemptTracker";
9 |
10 | const TEST_EMAIL = "user@test.edu";
11 |
12 | describe("loginAttemptTracker", () => {
13 |     beforeEach(() => {

```

```

14 |     localStorage.clear();
15 |     vi.useFakeTimers();
16 | });
17 |
18 | afterEach(() => {
19 |     vi.useRealTimers();
20 | });
21 |
22 | describe("isLocked", () => {
23 |     it("returns false for an email with no recorded attempts", () => {
24 |         expect(isLocked(TEST_EMAIL)).toBe(false);
25 |     });
26 |
27 |     it("returns false when fewer than MAX_ATTEMPTS failures recorded", () => {
28 |         for (let i = 0; i < LOGIN_ATTEMPT_CONFIG.MAX_ATTEMPTS - 1; i++) {
29 |             recordFailedAttempt(TEST_EMAIL);
30 |         }
31 |         expect(isLocked(TEST_EMAIL)).toBe(false);
32 |     });
33 |
34 |     it("returns true after MAX_ATTEMPTS consecutive failures", () => {
35 |         for (let i = 0; i < LOGIN_ATTEMPT_CONFIG.MAX_ATTEMPTS; i++) {
36 |             recordFailedAttempt(TEST_EMAIL);
37 |         }
38 |         expect(isLocked(TEST_EMAIL)).toBe(true);
39 |     });
40 |
41 |     it("returns false after lockout duration expires", () => {
42 |         for (let i = 0; i < LOGIN_ATTEMPT_CONFIG.MAX_ATTEMPTS; i++) {
43 |             recordFailedAttempt(TEST_EMAIL);
44 |         }
45 |         expect(isLocked(TEST_EMAIL)).toBe(true);
46 |
47 |         // Advance past lockout duration
48 |         vi.advanceTimersByTime(LOGIN_ATTEMPT_CONFIG.LOCKOUT_DURATION_MS + 1);
49 |         expect(isLocked(TEST_EMAIL)).toBe(false);
50 |     });
51 |
52 |     it("is case-insensitive for email", () => {
53 |         for (let i = 0; i < LOGIN_ATTEMPT_CONFIG.MAX_ATTEMPTS; i++) {
54 |             recordFailedAttempt("User@Test.EDU");
55 |         }
56 |         expect(isLocked("user@test.edu")).toBe(true);
57 |     });
58 | });
59 |
60 | describe("getRemainingLockTime", () => {
61 |     it("returns 0 when not locked", () => {
62 |         expect(getRemainingLockTime(TEST_EMAIL)).toBe(0);
63 |     });
64 |
65 |     it("returns remaining seconds when locked", () => {
66 |         for (let i = 0; i < LOGIN_ATTEMPT_CONFIG.MAX_ATTEMPTS; i++) {
67 |             recordFailedAttempt(TEST_EMAIL);
68 |         }
69 |         const remaining = getRemainingLockTime(TEST_EMAIL);
70 |         // Should be close to 15 minutes (900 seconds)
71 |         expect(remaining).toBeGreaterThan(0);
72 |         expect(remaining).toBeLessThanOrEqual(900);
73 |     });
74 |
75 |     it("returns 0 after lockout expires", () => {
76 |         for (let i = 0; i < LOGIN_ATTEMPT_CONFIG.MAX_ATTEMPTS; i++) {
77 |             recordFailedAttempt(TEST_EMAIL);
78 |         }
79 |         vi.advanceTimersByTime(LOGIN_ATTEMPT_CONFIG.LOCKOUT_DURATION_MS + 1);
80 |         expect(getRemainingLockTime(TEST_EMAIL)).toBe(0);
81 |     });
82 | });
83 |
84 | describe("recordFailedAttempt", () => {
85 |     it("increments the attempt count", () => {
86 |         recordFailedAttempt(TEST_EMAIL);
87 |         recordFailedAttempt(TEST_EMAIL);
88 |
89 |         const raw = localStorage.getItem("login_attempts_user@test.edu");
90 |         const record = JSON.parse(raw!);
91 |         expect(record.count).toBe(2);
92 |         expect(record.lockedUntil).toBeNull();
93 |     });
94 |
95 |     it("sets lockedUntil on the 5th failure", () => {
96 |         for (let i = 0; i < LOGIN_ATTEMPT_CONFIG.MAX_ATTEMPTS; i++) {
97 |             recordFailedAttempt(TEST_EMAIL);
98 |         }
99 |
100 |         const raw = localStorage.getItem("login_attempts_user@test.edu");
101 |         const record = JSON.parse(raw!);
102 |         expect(record.count).toBe(5);
103 |         expect(record.lockedUntil).not.toBeNull();
104 |     });
105 | });
106 |
107 | describe("clearAttempts", () => {
108 |     it("removes the record from local storage", () => {
109 |         recordFailedAttempt(TEST_EMAIL);
110 |         recordFailedAttempt(TEST_EMAIL);
111 |         clearAttempts(TEST_EMAIL);
112 |
113 |         expect(localStorage.getItem("login_attempts_user@test.edu")).toBeNull();
114 |         expect(isLocked(TEST_EMAIL)).toBe(false);
115 |     });
116 | });
117 |

```

```

118 | describe("corrupted localStorage", () => {
119 |   it("handles invalid JSON gracefully", () => {
120 |     localStorage.setItem("login_attempts_user@test.edu", "not-json");
121 |     expect(isLocked(TEST_EMAIL)).toBe(false);
122 |     expect(getRemainingLockTime(TEST_EMAIL)).toBe(0);
123 |   });
124 |
125 |   it("handles missing count field gracefully", () => {
126 |     localStorage.setItem(
127 |       "login_attempts_user@test.edu",
128 |       '{"lockedUntil':null}'
129 |     );
130 |     expect(isLocked(TEST_EMAIL)).toBe(false);
131 |   });
132 | });
133 | });
134 |

```

## FILE: src/lib/\_\_\_tests\_\_\_/navGroups.test.ts

TYPE: .ts | LINES: 1-123 | SHA256: 86d7d421e024f74f20156c07e35d16a6d3335556a0f959b845e6177e1700d404

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |   NAV_GROUPS,
4 |   NAV_GROUP_META,
5 |   REQUIRED_NAV_GROUPS,
6 |   assertNavGroup,
7 |   getRequiredNavGroup,
8 |   isNavGroup,
9 |   isValidNavGroupAssignment,
10 |   NavGroupAssignmentError,
11 |   type NavGroup,
12 | } from "../navGroups";
13 |
14 | describe("navGroups", () => {
15 |   describe("metadata", () => {
16 |     it("defines exactly the four labeled sections (R20.1)", () => {
17 |       expect([...NAV_GROUPS]).toEqual([
18 |         "learn",
19 |         "growth",
20 |         "community",
21 |         "tools",
22 |       ]);
23 |     });
24 |
25 |     it("provides bilingual-ready label keys and a stable order for every group (R20.4)", () => {
26 |       const orders = NAV_GROUPS.map((g) => NAV_GROUP_META[g].order);
27 |       expect(orders).toEqual([0, 1, 2, 3]);
28 |       for (const group of NAV_GROUPS) {
29 |         expect(NAV_GROUP_META[group].labelKey).toBe(`nav.groups.${group}`);
30 |         expect(NAV_GROUP_META[group].group).toBe(group);
31 |       }
32 |     });
33 |
34 |     it("requires the AI Tutor route to belong to the Learn group (R20.6)", () => {
35 |       expect(REQUIRED_NAV_GROUPS["/student/tutor"]).toBe("learn");
36 |       expect(getRequiredNavGroup("/student/tutor")).toBe("learn");
37 |     });
38 |   });
39 |
40 |   describe("isNavGroup", () => {
41 |     it("accepts defined groups and rejects everything else", () => {
42 |       for (const group of NAV_GROUPS) expect(isNavGroup(group)).toBe(true);
43 |       expect(isNavGroup("misc")).toBe(false);
44 |       expect(isNavGroup(undefined)).toBe(false);
45 |       expect(isNavGroup(null)).toBe(false);
46 |       expect(isNavGroup(2)).toBe(false);
47 |     });
48 |   });
49 |
50 |   describe("isValidNavGroupAssignment", () => {
51 |     it("returns true when a required-placement route is in its required group", () => {
52 |       expect(
53 |         isValidNavGroupAssignment({ to: "/student/tutor", group: "learn" })
54 |       ).toBe(true);
55 |     });
56 |
57 |     it("returns false when a required-placement route is in the wrong group", () => {
58 |       expect(
59 |         isValidNavGroupAssignment({ to: "/student/tutor", group: "tools" })
60 |       ).toBe(false);
61 |     });
62 |
63 |     it("returns true for an unconstrained route assigned to any defined group", () => {
64 |       expect(
65 |         isValidNavGroupAssignment({ to: "/student/dashboard", group: "tools" })
66 |       ).toBe(true);
67 |     });
68 |
69 |     it("returns false when no group is assigned", () => {
70 |       expect(isValidNavGroupAssignment({ to: "/student/dashboard" })).toBe(
71 |         false
72 |       );
73 |     });
74 |   });
75 |
76 |   describe("assertNavGroup", () => {
77 |     it("passes when AI Tutor is in the Learn group (R20.6)", () => {
78 |       expect(() =>
79 |         assertNavGroup({ to: "/student/tutor", group: "learn" })
80 |       ).not.toThrow();
81 |     });
82 |
83 |     it("rejects the AI Tutor item placed in an incorrect group (R20.6)", () => {

```

```

84 |     expect(() =>
85 |       assertNavGroup({ to: "/student/tutor", group: "community" })
86 |     ).toThrow(NavGroupAssignmentError);
87 |   });
88 |
89 |   it("rejects an item with no group", () => {
90 |     expect(() => assertNavGroup({ to: "/student/courses" })).toThrow(
91 |       NavGroupAssignmentError
92 |     );
93 |   });
94 |
95 |   it("rejects an item with an undefined/unknown group value", () => {
96 |     expect(() =>
97 |       assertNavGroup({
98 |         to: "/student/courses",
99 |         group: "misc" as unknown as NavGroup,
100 |       })
101 |     ).toThrow(NavGroupAssignmentError);
102 |   });
103 |
104 |   it("enforces every required placement listed in REQUIRED_NAV_GROUPS", () => {
105 |     for (const [to, group] of Object.entries(REQUIRED_NAV_GROUPS)) {
106 |       expect(() => assertNavGroup({ to, group })).not.toThrow();
107 |       const wrong = NAV_GROUPS.find((g) => g !== group) as NavGroup;
108 |       expect(() => assertNavGroup({ to, group: wrong })).toThrow(
109 |         NavGroupAssignmentError
110 |       );
111 |     }
112 |   });
113 |
114 |   it("allows an unconstrained route in any defined group", () => {
115 |     for (const group of NAV_GROUPS) {
116 |       expect(() =>
117 |         assertNavGroup({ to: "/student/dashboard", group })
118 |       ).not.toThrow();
119 |     }
120 |   });
121 | });
122 |
123 |

```

#### FILE: src/lib/\_\_tests\_\_/navItems.test.ts

TYPE: .ts | LINES: 1-47 | SHA256: a1d41faff936b68a4418c15c8f43cc72b8a75fe2882f85fc9e5dd4740ae4e8e2

```

1 | // Task 22.1 (R20.1, R20.2): every student navigation item carries a valid
2 | // `group` and satisfies its required placement.
3 | import { describe, it, expect } from "vitest";
4 | import { navItems } from "../navItems";
5 | import {
6 |   assertNavGroup,
7 |   isValidNavGroupAssignment,
8 |   isNavGroup,
9 |   REQUIRED_NAV_GROUPS,
10 |   type NavGroup,
11 | } from "../navGroups";
12 |
13 | describe("navItems – student navigation grouping (R20.1, R20.2)", () => {
14 |   const studentItems = navItems.student;
15 |
16 |   it("assigns a defined group to every student nav item (R20.2)", () => {
17 |     for (const item of studentItems) {
18 |       expect(isNavGroup(item.group)).toBe(true);
19 |     }
20 |   });
21 |
22 |   it("passes assertNavGroup for every student nav item (R20.6)", () => {
23 |     for (const item of studentItems) {
24 |       expect(() => assertNavGroup(item)).not.toThrow();
25 |       expect(isValidNavGroupAssignment(item)).toBe(true);
26 |     }
27 |   });
28 |
29 |   it("honors every required placement, including AI Tutor Learn (R20.1, R20.6)", () => {
30 |     for (const [to, expectedGroup] of Object.entries(REQUIRED_NAV_GROUPS)) {
31 |       const item = studentItems.find((i) => i.to === to);
32 |       expect(item, `missing student nav item for route ${to}`).toBeDefined();
33 |       expect(item?.group).toBe(expectedGroup as NavGroup);
34 |     }
35 |     expect(studentItems.find((i) => i.to === "/student/tutor")?.group).toBe(
36 |       "learn"
37 |     );
38 |   });
39 |
40 |   it("covers all four labeled sections across the student navigation (R20.1)", () => {
41 |     const usedGroups = new Set(studentItems.map((i) => i.group));
42 |     for (const group of ["learn", "growth", "community", "tools"] as const) {
43 |       expect(usedGroups.has(group)).toBe(true);
44 |     }
45 |   });
46 | });
47 |

```

#### FILE: src/lib/\_\_tests\_\_/pagination.test.ts

TYPE: .ts | LINES: 1-88 | SHA256: fe750e92401b479cd76a0c595dac9bc92e65c8d13bb2caf313aa31f082dabcb8

```

1 | import { describe, it, expect } from "vitest";
2 | import { toRange, hasMore, pageCount } from "../pagination";
3 |
4 | describe("pagination", () => {
5 |   describe("toRange", () => {
6 |     it("returns the inclusive zero-based range for the first page", () => {
7 |       expect(toRange(1, 50)).toEqual({ from: 0, to: 49 });
8 |     });
9 |   });

```

```

10 |     it("advances the range by pageSize for later pages", () => {
11 |         expect(toRange(2, 50)).toEqual({ from: 50, to: 99 });
12 |         expect(toRange(3, 25)).toEqual({ from: 50, to: 74 });
13 |     });
14 |
15 |     it("spans exactly pageSize indices (Supabase .range convention)", () => {
16 |         const { from, to } = toRange(4, 20);
17 |         expect(to - from + 1).toBe(20);
18 |     });
19 |
20 |     it("clamps page numbers below 1 to the first page", () => {
21 |         expect(toRange(0, 10)).toEqual({ from: 0, to: 9 });
22 |         expect(toRange(-5, 10)).toEqual({ from: 0, to: 9 });
23 |     });
24 |
25 |     it("clamps a page size below 1 to a single row", () => {
26 |         expect(toRange(1, 0)).toEqual({ from: 0, to: 0 });
27 |         expect(toRange(2, -3)).toEqual({ from: 1, to: 1 });
28 |     });
29 |
30 |     it("floors fractional inputs to stay on integer boundaries", () => {
31 |         expect(toRange(2.9, 10.7)).toEqual({ from: 10, to: 19 });
32 |     });
33 |
34 |     it("treats non-finite inputs as the first page / single row", () => {
35 |         expect(toRange(Number.NaN, 10)).toEqual({ from: 0, to: 9 });
36 |         expect(toRange(2, Number.POSITIVE_INFINITY)).toEqual({ from: 1, to: 1 });
37 |     });
38 | });
39 |
40 | describe("pageCount", () => {
41 |     it("computes the exact ceiling number of pages", () => {
42 |         expect(pageCount(100, 50)).toBe(2);
43 |         expect(pageCount(101, 50)).toBe(3);
44 |         expect(pageCount(49, 50)).toBe(1);
45 |     });
46 |
47 |     it("returns 0 pages for an empty collection", () => {
48 |         expect(pageCount(0, 50)).toBe(0);
49 |     });
50 |
51 |     it("clamps a page size below 1 to a single row", () => {
52 |         expect(pageCount(3, 0)).toBe(3);
53 |     });
54 |
55 |     it("treats a negative total as empty", () => {
56 |         expect(pageCount(-10, 50)).toBe(0);
57 |     });
58 | });
59 |
60 | describe("hasMore", () => {
61 |     it("is true when further rows exist beyond the page", () => {
62 |         expect(hasMore(1, 50, 120)).toBe(true);
63 |         expect(hasMore(2, 50, 120)).toBe(true);
64 |     });
65 |
66 |     it("is false on the final page", () => {
67 |         expect(hasMore(3, 50, 120)).toBe(false);
68 |     });
69 |
70 |     it("is false when the page exactly consumes all rows", () => {
71 |         expect(hasMore(2, 50, 100)).toBe(false);
72 |     });
73 |
74 |     it("is false for an empty collection", () => {
75 |         expect(hasMore(1, 50, 0)).toBe(false);
76 |     });
77 |
78 |     it("agrees with pageCount: hasMore iff page < pageCount", () => {
79 |         const total = 137;
80 |         const pageSize = 25;
81 |         const pages = pageCount(total, pageSize);
82 |         for (let page = 1; page <= pages; page++) {
83 |             expect(hasMore(page, pageSize, total)).toBe(page < pages);
84 |         }
85 |     });
86 | });
87 | });
88 |

```

FILE: [src/lib/\\_\\_tests\\_\\_/passwordVisibility.test.ts](#)

TYPE: .ts | LINES: 1-88 | SHA256: f5a9d6e7816fac1e6abf617e27971fed856bfb1ffe3bf9cee11a7507a70ee8b2

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |     passwordVisibilityReducer,
4 |     initialPasswordVisibilityState,
5 |     isFieldRevealed,
6 |     type PasswordVisibilityState,
7 | } from "../passwordVisibility";
8 |
9 | describe("passwordVisibility reducer", () => {
10 |     it("starts with every field masked", () => {
11 |         expect(initialPasswordVisibilityState.revealed).toBeNull();
12 |     });
13 |
14 |     it("reveals a field from the initial masked state", () => {
15 |         const next = passwordVisibilityReducer(initialPasswordVisibilityState, {
16 |             type: "reveal",
17 |             id: "password",
18 |         });
19 |         expect(next.revealed).toBe("password");
20 |         expect(isFieldRevealed(next, "password")).toBe(true);
21 |     });

```

```

22 |
23 | it("revealing one field masks any previously-revealed field (mutual exclusion)", () => {
24 |   const first = passwordVisibilityReducer(initialPasswordVisibilityState, {
25 |     type: "reveal",
26 |     id: "password",
27 |   });
28 |   const second = passwordVisibilityReducer(first, {
29 |     type: "reveal",
30 |     id: "confirmPassword",
31 |   });
32 |   expect(second.revealed).toBe("confirmPassword");
33 |   expect(isFieldRevealed(second, "password")).toBe(false);
34 |   expect(isFieldRevealed(second, "confirmPassword")).toBe(true);
35 | });
36 |
37 | it("hiding the currently-revealed field masks everything", () => {
38 |   const revealed = passwordVisibilityReducer(initialPasswordVisibilityState, {
39 |     type: "reveal",
40 |     id: "password",
41 |   });
42 |   const hidden = passwordVisibilityReducer(revealed, {
43 |     type: "hide",
44 |     id: "password",
45 |   });
46 |   expect(hidden.revealed).toBeNull();
47 | });
48 |
49 | it("hiding a non-revealed field leaves the revealed field untouched", () => {
50 |   const revealed = passwordVisibilityReducer(initialPasswordVisibilityState, {
51 |     type: "reveal",
52 |     id: "password",
53 |   });
54 |   const afterHideOther = passwordVisibilityReducer(revealed, {
55 |     type: "hide",
56 |     id: "confirmPassword",
57 |   });
58 |   expect(afterHideOther.revealed).toBe("password");
59 | });
60 |
61 | it("returns the same reference when revealing the already-revealed field", () => {
62 |   const revealed = passwordVisibilityReducer(initialPasswordVisibilityState, {
63 |     type: "reveal",
64 |     id: "password",
65 |   });
66 |   const again = passwordVisibilityReducer(revealed, {
67 |     type: "reveal",
68 |     id: "password",
69 |   });
70 |   expect(again).toBe(revealed);
71 | });
72 |
73 | it("returns the same reference when hiding an already-masked field", () => {
74 |   const again = passwordVisibilityReducer(initialPasswordVisibilityState, {
75 |     type: "hide",
76 |     id: "password",
77 |   });
78 |   expect(again).toBe(initialPasswordVisibilityState);
79 | });
80 |
81 | it("does not mutate the input state", () => {
82 |   const state: PasswordVisibilityState = { revealed: "a" };
83 |   passwordVisibilityReducer(state, { type: "reveal", id: "b" });
84 |   passwordVisibilityReducer(state, { type: "hide", id: "a" });
85 |   expect(state.revealed).toBe("a");
86 | });
87 | });
88 |

```

FILE: [src/lib/\\_\\_tests\\_\\_/pickColumns.test.ts](#)

TYPE: .ts | LINES: 1-50 | SHA256: 5ef595ed16fa02ad344e13d553045b3a9c89a5f8c5cb7edd9796ac7d53316f84

```

1 | import { describe, it, expect } from "vitest";
2 | import { pickColumns } from "../db/pickColumns";
3 |
4 | describe("pickColumns", () => {
5 |   it("copies only the allowed keys that are present", () => {
6 |     const payload = { a: 1, b: 2, c: 3 };
7 |     expect(pickColumns(payload, ["a", "c"] as const)).toEqual({ a: 1, c: 3 });
8 |   });
9 |
10 |   it("drops keys whose value is undefined", () => {
11 |     const payload = { a: 1, b: undefined as number | undefined };
12 |     expect(pickColumns(payload, ["a", "b"] as const)).toEqual({ a: 1 });
13 |   });
14 |
15 |   it("excludes UI-only keys that are not in the allowed list", () => {
16 |     const payload = { title: "Race", xp_race_acknowledged: true };
17 |     const row = pickColumns(payload, ["title"] as const);
18 |     expect(row).toEqual({ title: "Race" });
19 |     expect("xp_race_acknowledged" in row).toBe(false);
20 |   });
21 |
22 |   it("keeps falsy-but-defined values (0, false, empty string, null)", () => {
23 |     const payload = { zero: 0, flag: false, text: "", maybe: null };
24 |     expect(
25 |       pickColumns(payload, ["zero", "flag", "text", "maybe"] as const)
26 |     ).toEqual({
27 |       zero: 0,
28 |       flag: false,
29 |       text: "",
30 |       maybe: null,
31 |     });
32 |   });
33 |

```

```

34 | it("returns an empty object when no keys are allowed", () => {
35 |   expect(pickColumns({ a: 1, b: 2 }, [] as const)).toEqual({});
36 | });
37 |
38 | it("does not mutate the source payload (pure function)", () => {
39 |   const payload = { a: 1, b: 2 };
40 |   const snapshot = { ...payload };
41 |   pickColumns(payload, ["a"] as const);
42 |   expect(payload).toEqual(snapshot);
43 | });
44 |
45 | it("returns a new object reference distinct from the payload", () => {
46 |   const payload = { a: 1 };
47 |   expect(pickColumns(payload, ["a"] as const)).not.toBe(payload);
48 | });
49 | });
50 |

```

FILE: [src/lib/\\_\\_\\_tests\\_\\_\\_/primaryCtaSelector.test.ts](#)

TYPE: .ts | LINES: 1-128 | SHA256: 33504b9b7c87d5e89ff80e8842ab08ed1f7889595659af3de62f6bbcce8b205c

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |   selectPrimary,
4 |   orderSecondary,
5 |   type CtaCandidate,
6 | } from "../primaryCtaSelector";
7 |
8 | const candidate = (
9 |   id: string,
10 |  priority: number,
11 |  applicable: boolean
12 | ): CtaCandidate => ({ id, priority, applicable });
13 |
14 | describe("primaryCtaSelector", () => {
15 |   describe("selectPrimary", () => {
16 |     it("returns null when there are no candidates", () => {
17 |       expect(selectPrimary([])).toBeNull();
18 |     });
19 |
20 |     it("returns null when no candidate is applicable (R16.1)", () => {
21 |       expect(
22 |         selectPrimary([
23 |           candidate("profile", 0, false),
24 |           candidate("submit", 1, false),
25 |         ])
26 |       ).toBeNull();
27 |     });
28 |
29 |     it("selects the highest-precedence (lowest priority) applicable candidate (R16.2)", () => {
30 |       const result = selectPrimary([
31 |         candidate("continue", 2, true),
32 |         candidate("submit", 1, true),
33 |         candidate("feedback", 3, true),
34 |       ]);
35 |       expect(result?.id).toBe("submit");
36 |     });
37 |
38 |     it("excludes non-applicable candidates from the comparison (R16.2)", () => {
39 |       // 'profile' has the highest precedence but is not applicable, so the
40 |       // applicable 'submit' must win.
41 |       const result = selectPrimary([
42 |         candidate("profile", 0, false),
43 |         candidate("submit", 1, true),
44 |         candidate("continue", 2, true),
45 |       ]);
46 |       expect(result?.id).toBe("submit");
47 |     });
48 |
49 |     it("resolves priority ties to the first candidate in input order", () => {
50 |       const result = selectPrimary([
51 |         candidate("a", 1, true),
52 |         candidate("b", 1, true),
53 |       ]);
54 |       expect(result?.id).toBe("a");
55 |     });
56 |
57 |     it("promotes the next applicable candidate once the top is no longer applicable (R16.3)", () => {
58 |       const before = selectPrimary([
59 |         candidate("submit", 1, true),
60 |         candidate("continue", 2, true),
61 |       ]);
62 |       expect(before?.id).toBe("submit");
63 |
64 |       const after = selectPrimary([
65 |         candidate("submit", 1, false),
66 |         candidate("continue", 2, true),
67 |       ]);
68 |       expect(after?.id).toBe("continue");
69 |     });
70 |   });
71 |
72 |   describe("orderSecondary", () => {
73 |     it("returns the applicable candidates except the primary, ordered by precedence", () => {
74 |       const result = orderSecondary(
75 |         [
76 |           candidate("submit", 1, true),
77 |           candidate("feedback", 3, true),
78 |           candidate("continue", 2, true),
79 |         ],
80 |         "submit"
81 |       );
82 |       expect(result.map((c) => c.id)).toEqual(["continue", "feedback"]);
83 |     });

```



```

84 |
85 | it("excludes non-applicable candidates", () => {
86 |     const result = orderSecondary(
87 |         [
88 |             candidate("submit", 1, true),
89 |             candidate("profile", 0, false),
90 |             candidate("continue", 2, true),
91 |         ],
92 |         "submit"
93 |     );
94 |     expect(result.map((c) => c.id)).toEqual(["continue"]);
95 | });
96 |
97 | it("derives the primary itself when no primaryId is supplied", () => {
98 |     const result = orderSecondary([
99 |         candidate("continue", 2, true),
100 |         candidate("submit", 1, true),
101 |         candidate("feedback", 3, true),
102 |     ]);
103 |     // 'submit' is the computed primary and is excluded from the secondaries.
104 |     expect(result.map((c) => c.id)).toEqual(["continue", "feedback"]);
105 | });
106 |
107 | it("returns an empty list when no candidate is applicable", () => {
108 |     const result = orderSecondary([
109 |         candidate("submit", 1, false),
110 |         candidate("continue", 2, false),
111 |     ]);
112 |     expect(result).toEqual([]);
113 | });
114 |
115 | it("preserves input order for candidates sharing the same priority", () => {
116 |     const result = orderSecondary(
117 |         [
118 |             candidate("primary", 0, true),
119 |             candidate("b", 1, true),
120 |             candidate("a", 1, true),
121 |         ],
122 |         "primary"
123 |     );
124 |     expect(result.map((c) => c.id)).toEqual(["b", "a"]);
125 | });
126 | });
127 | });
128 |

```

#### FILE: src/lib/\_\_\_tests\_\_\_/queryClient.test.ts

TYPE: .ts | LINES: 1-52 | SHA256: ac65f562f02f5deb507380b237a693a6576f74407ac0545ac0dbd05a36afd448

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |     computeRetryDelay,
4 |     RETRY_MAX_DELAY_MS,
5 |     shouldRetryQuery,
6 | } from "@lib/queryClient";
7 |
8 | describe("computeRetryDelay (equal-jitter backoff)", () => {
9 |     it("stays within [base/2, base] for each attempt", () => {
10 |         // random=0 -> lower bound (base/2); random=1 -> upper bound (base)
11 |         expect(computeRetryDelay(0, () => 0)).toBe(500);
12 |         expect(computeRetryDelay(0, () => 1)).toBe(1000);
13 |
14 |         expect(computeRetryDelay(1, () => 0)).toBe(1000);
15 |         expect(computeRetryDelay(1, () => 1)).toBe(2000);
16 |
17 |         expect(computeRetryDelay(2, () => 0)).toBe(2000);
18 |         expect(computeRetryDelay(2, () => 1)).toBe(4000);
19 |     });
20 |
21 | it("caps the base at RETRY_MAX_DELAY_MS", () => {
22 |     // 2**20 * 1000 is huge; base is capped, so the ceiling is the cap.
23 |     expect(computeRetryDelay(20, () => 1)).toBe(RETRY_MAX_DELAY_MS);
24 |     expect(computeRetryDelay(20, () => 0)).toBe(RETRY_MAX_DELAY_MS / 2);
25 | });
26 |
27 | it("produces different delays for different random draws (de-synchronizes retries)", () => {
28 |     const a = computeRetryDelay(2, () => 0.1);
29 |     const b = computeRetryDelay(2, () => 0.9);
30 |     expect(a).not.toBe(b);
31 | });
32 | });
33 |
34 | describe("shouldRetryQuery", () => {
35 |     it("never retries 429 (rate limit)", () => {
36 |         expect(shouldRetryQuery(0, { status: 429 })).toBe(false);
37 |     });
38 |
39 | it("never retries deterministic 4xx (except 408)", () => {
40 |     expect(shouldRetryQuery(0, { status: 404 })).toBe(false);
41 |     expect(shouldRetryQuery(0, { status: 403 })).toBe(false);
42 |     expect(shouldRetryQuery(0, { status: 422 })).toBe(false);
43 | });
44 |
45 | it("retries transient failures up to 3 times", () => {
46 |     expect(shouldRetryQuery(0, { status: 500 })).toBe(true);
47 |     expect(shouldRetryQuery(0, { status: 408 })).toBe(true); // Request Timeout
48 |     expect(shouldRetryQuery(2, {})).toBe(true); // unknown/PostgREST error, no status
49 |     expect(shouldRetryQuery(3, { status: 500 })).toBe(false); // cap reached
50 | });
51 | });
52 |

```

## FILE: src/lib/\_\_tests\_\_/quizCorrectness.test.ts

TYPE: .ts | LINES: 1-58 | SHA256: 524e37d0ecbbf64329b450e242b552302b2ade2254c3bcc56abb7d0703ef8288

```
1 | import { describe, it, expect } from "vitest";
2 | import { deriveCorrectness } from "../quizCorrectness";
3 |
4 | describe("quizCorrectness", () => {
5 |   describe("deriveCorrectness", () => {
6 |     it("uses the server-authoritative evaluation when present (R2.1)", () => {
7 |       expect(
8 |         deriveCorrectness({
9 |           selectedAnswer: "A",
10 |           correctAnswer: "B",
11 |           serverEvaluated: true,
12 |         })
13 |       ).toBe(true);
14 |
15 |       expect(
16 |         deriveCorrectness({
17 |           selectedAnswer: "B",
18 |           correctAnswer: "B",
19 |           serverEvaluated: false,
20 |         })
21 |       ).toBe(false);
22 |     });
23 |
24 |     it("falls back to equality against the correct answer (R2.4)", () => {
25 |       expect(
26 |         deriveCorrectness({ selectedAnswer: "B", correctAnswer: "B" })
27 |       ).toBe(true);
28 |       expect(
29 |         deriveCorrectness({ selectedAnswer: "A", correctAnswer: "B" })
30 |       ).toBe(false);
31 |     });
32 |
33 |     it("compares answers tolerantly (trim + case-insensitive)", () => {
34 |       expect(
35 |         deriveCorrectness({ selectedAnswer: " b ", correctAnswer: "B" })
36 |       ).toBe(true);
37 |     });
38 |
39 |     it("never reports correct without evidence (R2.6)", () => {
40 |       // No server evaluation and no correct answer !0 must not be "correct".
41 |       expect(deriveCorrectness({ selectedAnswer: "anything" })).toBe(false);
42 |       expect(
43 |         deriveCorrectness({ selectedAnswer: "anything", correctAnswer: null })
44 |       ).toBe(false);
45 |     });
46 |
47 |     it("prefers a false server evaluation over a coincidental answer match", () => {
48 |       expect(
49 |         deriveCorrectness({
50 |           selectedAnswer: "B",
51 |           correctAnswer: "B",
52 |           serverEvaluated: false,
53 |         })
54 |       ).toBe(false);
55 |     });
56 |   });
57 | });
58 |
```

## FILE: src/lib/\_\_tests\_\_/quizScore.test.ts

TYPE: .ts | LINES: 1-42 | SHA256: 69dbaa50925f983a86812a225df66d8fa1357573e06c135dbd0fd97eb7cc2131

```
1 | import { describe, it, expect } from "vitest";
2 | import { computeScore } from "../quizScore";
3 |
4 | describe("quizScore", () => {
5 |   describe("computeScore", () => {
6 |     it("computes a rounded percentage from correct over total (R3.1, R3.3)", () => {
7 |       expect(computeScore(5, 10)).toBe(50);
8 |       expect(computeScore(10, 10)).toBe(100);
9 |       expect(computeScore(0, 10)).toBe(0);
10 |     });
11 |
12 |     it("rounds to the nearest whole percent", () => {
13 |       // 1/3 = 33.33% !' 33
14 |       expect(computeScore(1, 3)).toBe(33);
15 |       // 2/3 = 66.66% !' 67
16 |       expect(computeScore(2, 3)).toBe(67);
17 |       // 1/8 = 12.5% !' 13 (round half up)
18 |       expect(computeScore(1, 8)).toBe(13);
19 |     });
20 |
21 |     it("returns 0 when there are no questions (no division by zero)", () => {
22 |       expect(computeScore(0, 0)).toBe(0);
23 |       expect(computeScore(3, 0)).toBe(0);
24 |     });
25 |
26 |     it("treats non-positive or non-finite question counts as zero score", () => {
27 |       expect(computeScore(3, -5)).toBe(0);
28 |       expect(computeScore(3, Number.NaN)).toBe(0);
29 |       expect(computeScore(3, Number.POSITIVE_INFINITY)).toBe(0);
30 |     });
31 |
32 |     it("clamps correct count so the score never exceeds 100", () => {
33 |       expect(computeScore(15, 10)).toBe(100);
34 |     });
35 |
36 |     it("treats negative or non-finite correct counts as zero", () => {
37 |       expect(computeScore(-4, 10)).toBe(0);
38 |       expect(computeScore(Number.NaN, 10)).toBe(0);
39 |     });
40 |   });
41 | });
42 |
```

```
TYPE: .ts | LINES: 1-58 | SHA256: 99699cc144b1358715b15be334123e561369d45f674b0f7146c16e6d71125145
```

FILE: src/lib/activityLogger.ts

TYPE: .ts | LINES: 1-101 | SHA256: c2895a6fbbdba31d64410d69b320297671f9817489493a4c4578594cd1f59ef5

```

1 import { supabase } from "@lib/supabase";
2 import { offlineQueue } from "@lib/offlineQueue";
3 import type { ActivityLogEventType } from "@types/app";
4 import type { Json } from "@types/database";
5
6 export interface ActivityLogEntry {
7   student_id: string;
8   event_type: ActivityLogEventType;
9   metadata?: Record<string, unknown>;
10 }
11
12 // %% Batch buffer: accumulate entries and flush every 30s % % % % % % % % % % % % % % % % % % % % % %
13
14 let buffer: ActivityLogEntry[] = [];
15 let flushTimer: ReturnType<typeof setTimeout> | null = null;
16 const FLUSH_INTERVAL_MS = 30_000;
17 const MAX_BUFFER_SIZE = 20;
18
19 const flushBuffer = async (): Promise<void> => {
20   if (buffer.length === 0) return;
21   const batch = [...buffer];
22   buffer = [];
23
24   const rows = batch.map((entry) => ({
25     student_id: entry.student_id,
26     event_type: entry.event_type,
27     metadata: (entry.metadata ?? null) as Json,
28   }));
29
30   const { error } = await supabase.from("student_activity_log").insert(rows);
31   if (error) {
32     // Re-queue to offline if the batch insert fails
33     for (const entry of batch) {
34       offlineQueue.enqueue("activity_log", entry);
35     }
36   }
37 }

```



```

34 |     }
35 | }
36 |
37 | /**
38 |  * Adjusts target difficulty after each answer.
39 |  * Correct: +stepUp (default 0.3), capped at 5.0
40 |  * Incorrect: - stepDown (default 0.5), floored at 1.0
41 |  */
42 | export function adjustDifficulty(
43 |   current: number,
44 |   wasCorrect: boolean,
45 |   stepUp = 0.3,
46 |   stepDown = 0.5
47 | ): number {
48 |   if (wasCorrect) return Math.min(5.0, current + stepUp);
49 |   return Math.max(1.0, current - stepDown);
50 | }
51 |
52 | /**
53 |  * Selects preferred Bloom's levels based on ability.
54 |  * High: [4, 5, 6] (Analyzing, Evaluating, Creating)
55 |  * Medium: [2, 3, 4] (Understanding, Applying, Analyzing)
56 |  * Low: [1, 2] (Remembering, Understanding)
57 |  */
58 | export function preferredBloomLevels(ability: AbilityLevel): number[] {
59 |   switch (ability) {
60 |     case "high":
61 |       return [4, 5, 6];
62 |     case "medium":
63 |       return [2, 3, 4];
64 |     case "low":
65 |       return [1, 2];
66 |   }
67 | }
68 |

```

FILE: [src/lib/adaptiveXP.ts](#)

TYPE: .ts | LINES: 1-111 | SHA256: 25b488ae9b9c5ae858c6241b178b83e96e4732af35c9f2fc133c4d442dec7054

```

1 | // Task 137.1: Adaptive XP calculation utility
2 |
3 | export type BloomsLevel =
4 |   | "Remembering"
5 |   | "Understanding"
6 |   | "Applying"
7 |   | "Analyzing"
8 |   | "Evaluating"
9 |   | "Creating";
10 |
11 | /**
12 |  * Level-based XP multiplier.
13 |  * Levels 1-5: 1.2x (encouragement), 6-10: 1.0x, 11-15: 0.9x, 16-20: 0.8x
14 |  */
15 | export function getLevelMultiplier(level: number): number {
16 |   if (level <= 5) return 1.2;
17 |   if (level <= 10) return 1.0;
18 |   if (level <= 15) return 0.9;
19 |   return 0.8;
20 | }
21 |
22 | const DIFFICULTY_MAP: Record<BloomsLevel, number> = {
23 |   Remembering: 1.0,
24 |   Understanding: 1.1,
25 |   Applying: 1.2,
26 |   Analyzing: 1.3,
27 |   Evaluating: 1.4,
28 |   Creating: 1.5,
29 | };
30 |
31 | /**
32 |  * Difficulty multiplier based on Bloom's Taxonomy level.
33 |  */
34 | export function getDifficultyMultiplier(bloomsLevel: BloomsLevel): number {
35 |   return DIFFICULTY_MAP[bloomsLevel] ?? 1.0;
36 | }
37 |
38 | /**
39 |  * Diminishing returns multiplier.
40 |  * Decreases by 0.2 per repetition: 1.0 !' 0.8 !' 0.6 !' 0.4 !' 0.2 min.
41 |  * Milestones always get 1.0.
42 |  */
43 | export function getDiminishingMultiplier(
44 |   repeatCount: number,
45 |   isMilestone: boolean
46 | ): number {
47 |   if (isMilestone) return 1.0;
48 |   if (repeatCount <= 0) return 1.0;
49 |   const multiplier = 1.0 - repeatCount * 0.2;
50 |   return Math.max(multiplier, 0.2);
51 | }
52 |
53 | /**
54 |  * Calculate final XP with all adaptive multipliers.
55 |  * Formula: floor(base × level × difficulty × diminishing)
56 |  * For multiple CLOs, use the highest Bloom's level.
57 |  */
58 | export function calculateFinalXP(
59 |   baseXP: number,
60 |   level: number,
61 |   bloomsLevels: BloomsLevel[],
62 |   repeatCount: number,
63 |   isMilestone: boolean
64 | ): number {
65 |   if (baseXP <= 0) return 0;

```

```

66 |     const levelMult = getLevelMultiplier(level);
67 |
68 |
69 |     // Use highest Bloom's level for difficulty multiplier
70 |     const bloomsOrder: BloomsLevel[] = [
71 |       "Remembering",
72 |       "Understanding",
73 |       "Applying",
74 |       "Analyzing",
75 |       "Evaluating",
76 |       "Creating",
77 |     ];
78 |     let highestDifficulty = 1.0;
79 |     for (const bl of bloomsLevels) {
80 |       const idx = bloomsOrder.indexOf(bl);
81 |       if (idx >= 0) {
82 |         const diff = getDifficultyMultiplier(bl);
83 |         if (diff > highestDifficulty) highestDifficulty = diff;
84 |       }
85 |     }
86 |
87 |     const diminishing = getDiminishingMultiplier(repeatCount, isMilestone);
88 |
89 |     return Math.floor(baseXP * levelMult * highestDifficulty * diminishing);
90 |   }
91 |
92 |   // -----
93 |   // Tutor Diminishing Returns
94 |   // -----
95 |
96 |   /**
97 |    * Diminishing multiplier for tutor conversation XP.
98 |    * - < 5 conversations: 1.0 (full XP)
99 |    * - 5-9 conversations: 0.5
100 |    * - 10-14 conversations: 0.25
101 |    * - >= 15 conversations: 0 (no XP)
102 |    */
103 |   export function getTutorDiminishingMultiplier(
104 |     conversationCount: number
105 |   ): number {
106 |     if (conversationCount < 5) return 1.0;
107 |     if (conversationCount < 10) return 0.5;
108 |     if (conversationCount < 15) return 0.25;
109 |     return 0;
110 |   }
111 |

```

FILE: src/lib/aiGovernancePolicy.ts

TYPE: .ts | LINES: 1-99 | SHA256: e0bd6c4df8d1c2fe856bf4016c5c3c9edac0e1b6efa4c806840c5bf153ddf13d

```

1 | export type GovernanceAutonomyLevel = "A0" | "A1" | "A2" | "A3";
2 |
3 | export interface GovernanceActionPolicy {
4 |   actionKey:
5 |     | "showInsights"
6 |     | "suggestAction"
7 |     | "scheduleReviews"
8 |     | "reorderPlan"
9 |     | "sendStudentNudge"
10 |    | "draftTeacherFeedback"
11 |    | "generateEvidencePack"
12 |    | "sendParentCommunication"
13 |    | "assignGrade"
14 |    | "publishContent";
15 |   level: GovernanceAutonomyLevel;
16 |   hardCap: GovernanceAutonomyLevel | null;
17 |   sensitive: boolean;
18 | }
19 |
20 | /**
21 |  * Platform policy guardrails represented in the approved prototype.
22 |  * Institution-specific persistence is not available yet, so this is a
23 |  * read-only product policy—not a configurable backend setting.
24 |  */
25 | export const AI_GOVERNANCE_ACTION_POLICIES: GovernanceActionPolicy[] = [
26 |   {
27 |     actionKey: "showInsights",
28 |     level: "A0",
29 |     hardCap: null,
30 |     sensitive: false,
31 |   },
32 |   {
33 |     actionKey: "suggestAction",
34 |     level: "A1",
35 |     hardCap: null,
36 |     sensitive: false,
37 |   },
38 |   {
39 |     actionKey: "scheduleReviews",
40 |     level: "A2",
41 |     hardCap: "A2",
42 |     sensitive: false,
43 |   },
44 |   {
45 |     actionKey: "reorderPlan",
46 |     level: "A2",
47 |     hardCap: "A2",
48 |     sensitive: false,
49 |   },
50 |   {
51 |     actionKey: "sendStudentNudge",
52 |     level: "A2",
53 |     hardCap: "A2",
54 |     sensitive: true,

```

```

55 | },
56 | {
57 |   actionKey: "draftTeacherFeedback",
58 |   level: "A1",
59 |   hardCap: "A2",
60 |   sensitive: true,
61 | },
62 | {
63 |   actionKey: "generateEvidencePack",
64 |   level: "A2",
65 |   hardCap: "A2",
66 |   sensitive: true,
67 | },
68 | {
69 |   actionKey: "sendParentCommunication",
70 |   level: "A2",
71 |   hardCap: "A2",
72 |   sensitive: true,
73 | },
74 | {
75 |   actionKey: "assignGrade",
76 |   level: "A2",
77 |   hardCap: "A2",
78 |   sensitive: true,
79 | },
80 | {
81 |   actionKey: "publishContent",
82 |   level: "A2",
83 |   hardCap: "A2",
84 |   sensitive: true,
85 | },
86 | ];
87 |
88 | export const autonomyBadgeClass = (level: GovernanceAutonomyLevel) => {
89 |   switch (level) {
90 |     case "A0":
91 |     case "A1":
92 |       return "border-blue-200 bg-blue-50 text-blue-700";
93 |     case "A2":
94 |       return "border-amber-200 bg-amber-50 text-amber-700";
95 |     case "A3":
96 |       return "border-emerald-200 bg-emerald-50 text-emerald-700";
97 |   }
98 | };
99 |

```

## FILE: src/lib/analyticsConsent.ts

TYPE: .ts | LINES: 1-56 | SHA256: f90fb63f20f26e87a7520363a17f787782dfae6092adbcbcb0bfff7b9b032f3703

```

1 | // Task 86.2: Analytics consent utilities
2 | // Manages cookie consent state in localStorage and conditionally initializes Sentry
3 |
4 | const CONSENT_KEY = "edviser_cookie_consent";
5 |
6 | export interface CookieConsent {
7 |   essential: boolean;
8 |   analytics: boolean;
9 | }
10 |
11 | export const getConsent = (): CookieConsent | null => {
12 |   try {
13 |     const raw = localStorage.getItem(CONSENT_KEY);
14 |     if (!raw) return null;
15 |     const parsed = JSON.parse(raw) as CookieConsent;
16 |     if (
17 |       typeof parsed.essential !== "boolean" ||
18 |       typeof parsed.analytics !== "boolean"
19 |     ) {
20 |       return null;
21 |     }
22 |     return parsed;
23 |   } catch {
24 |     return null;
25 |   }
26 | };
27 |
28 | export const setConsent = (consent: CookieConsent): void => {
29 |   localStorage.setItem(CONSENT_KEY, JSON.stringify(consent));
30 | };
31 |
32 | export const hasAnalyticsConsent = (): boolean => {
33 |   const consent = getConsent();
34 |   return consent?.analytics === true;
35 | };
36 |
37 | export const initAnalyticsIfConsented = (): void => {
38 |   if (!hasAnalyticsConsent()) return;
39 |
40 |   // Lazily initialize Sentry only when analytics consent is granted
41 |   import("@sentry/react")
42 |     .then((Sentry) => {
43 |       if (Sentry.isInitialized()) return;
44 |       Sentry.init({
45 |         dsn: import.meta.env.VITE_SENTRY_DSN,
46 |         environment: import.meta.env.MODE,
47 |         tracesSampleRate: 0.1,
48 |         replaysSessionSampleRate: 0,
49 |         replaysOnErrorSampleRate: 1.0,
50 |       });
51 |     })
52 |     .catch((err) => {
53 |       console.error("[analyticsConsent] Failed to initialize Sentry:", err);
54 |     });
55 | };

```

```
TYPE: .ts | LINES: 1-60 | SHA256: e40dcd71dad5e98a716752bbfe789515303b212812df97cdb0363ac3553ad43f
```

TYPE: .ts | LINES: 1-62 | SHA256: 863c83bc3e762be646608b6b8cf956cdf1a08647b53b54b0820a99361a445912

```

1 | import type { AttainmentLevel, AttainmentThresholdsConfig } from "@types/app";
2 | import { DEFAULT_ATTAINMENT_THRESHOLDS } from "@types/app";
3 |
4 | /**
5 |  * Classifies a score percentage into an attainment level using configurable thresholds.
6 |  * Falls back to default thresholds (85/70/50) when no config is provided.
7 |  */
8 | export function classifyAttainment(
9 |   percent: number,
10 |   thresholds: AttainmentThresholdsConfig = DEFAULT_ATTAINMENT_THRESHOLDS

```



```

11 | ): AttainmentLevel {
12 |   if (percent >= thresholds.excellent) return "Excellent";
13 |   if (percent >= thresholds.satisfactory) return "Satisfactory";
14 |   if (percent >= thresholds.developing) return "Developing";
15 |   return "Not_Yet";
16 | }
17 |
18 | /**
19 |  * Returns the Tailwind color class for an attainment percentage.
20 |  */
21 | export function getAttainmentColor(
22 |   percent: number,
23 |   thresholds: AttainmentThresholdsConfig = DEFAULT_ATTAINMENT_THRESHOLDS
24 | ): string {
25 |   if (percent < 0) return "#e5e7eb";
26 |   if (percent >= thresholds.excellent) return "#22c55e";
27 |   if (percent >= thresholds.satisfactory) return "#3b82f6";
28 |   if (percent >= thresholds.developing) return "#eab308";
29 |   return "#ef4444";
30 | }
31 |
32 | /**
33 |  * Returns the Tailwind text color class for an attainment percentage.
34 |  * Uses -800/-700 shades to meet WCAG 2.1 AA 4.5:1 contrast on light backgrounds.
35 |  */
36 | export function getAttainmentTextClass(
37 |   percent: number,
38 |   thresholds: AttainmentThresholdsConfig = DEFAULT_ATTAINMENT_THRESHOLDS
39 | ): string {
40 |   if (percent >= thresholds.excellent) return "text-green-800";
41 |   if (percent >= thresholds.satisfactory) return "text-blue-700";
42 |   if (percent >= thresholds.developing) return "text-yellow-800";
43 |   return "text-red-700";
44 | }
45 |
46 | /**
47 |  * Returns the badge style class for an attainment percentage.
48 |  * Uses -800/-700 text shades to meet WCAG 2.1 AA 4.5:1 contrast on -50 backgrounds.
49 |  */
50 | export function getAttainmentBadgeStyle(
51 |   percent: number,
52 |   thresholds: AttainmentThresholdsConfig = DEFAULT_ATTAINMENT_THRESHOLDS
53 | ): string {
54 |   if (percent >= thresholds.excellent)
55 |     return "bg-green-50 text-green-800 border-green-200";
56 |   if (percent >= thresholds.satisfactory)
57 |     return "bg-blue-50 text-blue-700 border-blue-200";
58 |   if (percent >= thresholds.developing)
59 |     return "bg-yellow-50 text-yellow-800 border-yellow-200";
60 |   return "bg-red-50 text-red-700 border-red-200";
61 | }
62 |

```

#### FILE: src/lib/attainmentTone.ts

TYPE: .ts | LINES: 1-14 | SHA256: 5e8ae0fa9c194cfcaab742447125fe1dd123fc4f129323fcb6f7d45b118af4ae

```

1 | // Shared attainment !' Tailwind text-color mapping for redesigned dashboards
2 | // (spec: ui-prototype-migration, P2). Mirrors the OBE attainment bands used by
3 | // MasteryRing (Excellent "e85 green, Satisfactory "e70 brand-blue, Developing
4 | // "e50 amber, Not Yet <50 red); 0 falls back to the deep brand blue so an
5 | // empty/unknown value never reads as "failing".
6 |
7 | export const attainmentValueClass = (v: number): string => {
8 |   if (v >= 85) return "text-green-600";
9 |   if (v >= 70) return "text-sky-700";
10 |   if (v >= 50) return "text-amber-600";
11 |   if (v > 0) return "text-red-600";
12 |   return "text-sky-700";
13 | };
14 |

```

#### FILE: src/lib/auditLogger.ts

TYPE: .ts | LINES: 1-139 | SHA256: a3535af860f5c16cfc23a95f5719d8cd326c85abff2d79c3eb8c07aee5a808a5

```

1 | import { supabase } from "@lib/supabase";
2 | import type { Json } from "@types/database";
3 |
4 | export interface AuditLogEntry {
5 |   action: string;
6 |   entity_type: string;
7 |   entity_id: string;
8 |   changes: Record<string, unknown> | null;
9 |   performed_by: string;
10 | }
11 |
12 | /**
13 |  * Allowlisted fields per entity type to prevent PII leakage into audit logs.
14 |  * Only these fields will be recorded in the `diff` column.
15 |  */
16 | const AUDIT_FIELD_ALLOWLIST: Record<string, string[]> = {
17 |   user: ["role", "is_active", "program_id"],
18 |   grade: ["submission_id", "score_percent", "total_score"],
19 |   assignment: ["title", "course_id", "due_date", "status"],
20 |   course: ["name", "code", "program_id", "semester_id", "teacher_id"],
21 |   program: ["name", "code", "institution_id", "coordinator_id"],
22 |   ilo: ["title", "type", "institution_id"],
23 |   plo: ["title", "type", "program_id"],
24 |   clo: ["title", "type", "course_id", "blooms_level"],
25 |   outcome_mapping: ["source_outcome_id", "target_outcome_id", "weight"],
26 |   bonus_xp_event: ["name", "xp_multiplier", "starts_at", "ends_at"],
27 |   enrollment: ["student_id", "course_id", "status"],
28 |   semester: [
29 |     "name",
30 |     "code",

```

```

31 |     "start_date",
32 |     "end_date",
33 |     "is_active",
34 |     "institution_id",
35 | ],
36 | department: ["name", "code", "head_of_department_id", "institution_id"],
37 | course_section: [
38 |     "course_id",
39 |     "section_code",
40 |     "teacher_id",
41 |     "capacity",
42 |     "is_active",
43 | ],
44 | survey: ["title", "type", "is_active", "target_outcomes", "institution_id"],
45 | cqi_action_plan: [
46 |     "program_id",
47 |     "semester_id",
48 |     "outcome_id",
49 |     "outcome_type",
50 |     "baseline_attainment",
51 |     "target_attainment",
52 |     "action_description",
53 |     "responsible_person",
54 |     "status",
55 |     "result_attainment",
56 | ],
57 | institution_settings: [
58 |     "attainment_thresholds",
59 |     "success_threshold",
60 |     "accreditation_body",
61 |     "grade_scales",
62 |     "default_language",
63 | ],
64 | program_accreditation: [
65 |     "program_id",
66 |     "accreditation_body",
67 |     "framework_version",
68 |     "accreditation_date",
69 |     "next_review_date",
70 |     "status",
71 | ],
72 | announcement: ["course_id", "title", "is_pinned"],
73 | course_module: ["course_id", "title", "sort_order", "is_published"],
74 | course_material: [
75 |     "module_id",
76 |     "title",
77 |     "type",
78 |     "sort_order",
79 |     "is_published",
80 |     "clo_ids",
81 | ],
82 | class_session: ["section_id", "session_date", "session_type", "topic"],
83 | attendance_record: ["session_id", "student_count"],
84 | quiz: [
85 |     "course_id",
86 |     "title",
87 |     "clo_ids",
88 |     "time_limit_minutes",
89 |     "max_attempts",
90 |     "is_published",
91 |     "due_date",
92 | ],
93 | quiz_question: ["quiz_id", "question_type", "points", "sort_order"],
94 | grade_category: ["course_id", "name", "weight_percent", "sort_order"],
95 | ];
96 |
97 | function filterChanges(
98 |     entityType: string,
99 |     changes: Record<string, unknown> | null
100 | ): Record<string, unknown> | null {
101 |     if (!changes) return null;
102 |
103 |     const allowlist = AUDIT_FIELD_ALLOWLIST[entityType];
104 |     if (!allowlist) {
105 |         // Unknown entity type: log only keys, not values, to be safe
106 |         return Object.fromEntries(
107 |             Object.keys(changes).map((key) => [key, "[redacted]"])
108 |         );
109 |     }
110 |
111 |     const filtered: Record<string, unknown> = {};
112 |     for (const key of allowlist) {
113 |         if (key in changes) {
114 |             filtered[key] = changes[key];
115 |         }
116 |     }
117 |     return filtered;
118 | }
119 |
120 | export const logAuditEvent = async (entry: AuditLogEntry): Promise<void> => {
121 |     try {
122 |         const safeChanges = filterChanges(entry.entity_type, entry.changes);
123 |
124 |         const { error } = await supabase.from("audit_logs").insert({
125 |             action: entry.action,
126 |             target_type: entry.entity_type,
127 |             target_id: entry.entity_id,
128 |             diff: safeChanges as Json,
129 |             actor_id: entry.performed_by,
130 |         });
131 |
132 |         if (error) {
133 |             console.error("[AuditLogger] Failed to log audit event:", error.message);
134 |         }

```

```
135 |     } catch (err) {
136 |         console.error("[AuditLogger] Unexpected error:", err);
137 |     }
138 | };
139 |
```

FILE: src/lib/auditLogPresentation.ts

```
TYPE: .ts | LINES: 1-8 | SHA256: ba15ad57d5441035b88c17dd6c84f43b5f4ae3fb076974c1377314520238806e
```

```
1 | export const truncateAuditIdentifier = (
2 |   value: string | null | undefined,
3 |   length: number = 12
4 | ): string => {
5 |   if (!value) return "-";
6 |   return value.length > length ? `${value.slice(0, length)}...` : value;
7 | };
8 |
```

FILE: src/lib/badgeDefinitions.ts

TYPE: .ts | LINES: 1-416 | SHA256: f1f057846697392285932144b7c5196d89a58f66176e981ac060dd18dbb92c21

```

1 // Badge definitions for the Edeviser gamification engine.
2 // All badge conditions are defined here as a single source of truth.
3 // Mystery badge conditions are checked server-side only – never expose to client.
4
5 export type BadgeCategory =
6   | "streak"
7   | "academic"
8   | "engagement"
9   | "mystery"
10  | "habit"
11  | "blooms"
12  | "team"
13  | "study";
14
15 export interface BadgeDef {
16   id: string;
17   name: string;
18   description: string;
19   icon: string;
20   category: BadgeCategory;
21   isMystery: boolean;
22   condition: string; // human-readable condition description (hidden for mystery badges)
23   xpReward: number; // XP awarded when badge is earned
24 }
25
26 export const BADGE_DEFINITIONS: BadgeDef[] = [
27   // %% Streak badges %%%%%%%%%%%%%%%
28   {
29     id: "streak_7",
30     name: "7-Day Warrior",
31     description: "7-day login streak",
32     icon: "🏹",
33     category: "streak",
34     isMystery: false,
35     condition: "Login for 7 consecutive days",
36     xpReward: 50,
37   },
38   {
39     id: "streak_14",
40     name: "Fortnight Fighter",
41     description: "14-day login streak",
42     icon: "🏹",
43     category: "streak",
44     isMystery: false,
45     condition: "Login for 14 consecutive days",
46     xpReward: 75,
47   },
48   {
49     id: "streak_30",
50     name: "30-Day Legend",
51     description: "30-day login streak",
52     icon: "🏹",
53     category: "streak",
54     isMystery: false,
55     condition: "Login for 30 consecutive days",
56     xpReward: 100,
57   },
58   {
59     id: "streak_60",
60     name: "Dedication King",
61     description: "60-day login streak",
62     icon: "🏹",
63     category: "streak",
64     isMystery: false,
65     condition: "Login for 60 consecutive days",
66     xpReward: 150,
67   },
68   {
69     id: "streak_100",
70     name: "Century Legend",
71     description: "100-day login streak",
72     icon: "🏹",
73     category: "streak",
74     isMystery: false,
75     condition: "Login for 100 consecutive days",
76     xpReward: 250,
77   },
78 ]
79
80 // %% Academic badges %%%%%%%%%%%%%%%
81 {
82   id: "first_submission",
83   name: "First Steps",
84   description: "Submit your first assignment",
85 }

```

```
84 icon: "0=ÜY",
85 category: "academic",
86 isMystery: false,
87 condition: "Submit 1 assignment",
88 xpReward: 25,
89 },
90 {
91 id: "perfect_score",
92 name: "Flawless",
93 description: "Score 100% on a rubric",
94 icon: "0=Ü-",
95 category: "academic",
96 isMystery: false,
97 condition: "Score 100% on all rubric criteria",
98 xpReward: 75,
99 },
100 {
101 id: "all_clos_met",
102 name: "Outcome Achiever",
103 description: "Meet all CLOs in a course",
104 icon: "0<ß",
105 category: "academic",
106 isMystery: false,
107 condition: "Achieve "e70% on all CLOs in a course",
108 xpReward: 100,
109 },
110 // %% Engagement badges % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
111 {
112 id: "journal_10",
113 name: "Reflective Mind",
114 description: "Write 10 journal entries",
115 icon: "0=ÜÜ",
116 category: "engagement",
117 isMystery: false,
118 condition: "Write 10 journal entries",
119 xpReward: 50,
120 },
121 {
122 id: "perfect_week",
123 name: "Perfect Week",
124 description: "Complete all habits for 7 days",
125 icon: "+P",
126 category: "engagement",
127 isMystery: false,
128 condition: "Complete all 4 daily habits for 7 consecutive days",
129 xpReward: 100,
130 },
131 {
132 id: "perfect_attendance_week",
133 name: "Perfect Attendance Week",
134 description: "Present for all sessions in a 7-day period",
135 icon: "0=ÜÊ",
136 category: "engagement",
137 isMystery: false,
138 condition: "Marked present for all class sessions within a 7-day period",
139 xpReward: 75,
140 },
141 {
142 id: "quiz_master",
143 name: "Quiz Master",
144 description: "Complete 10 quizzes",
145 icon: "0>ÿÀ",
146 category: "engagement",
147 isMystery: false,
148 condition: "Complete 10 quiz attempts",
149 xpReward: 100,
150 },
151 {
152 id: "discussion_helper",
153 name: "Discussion Helper",
154 description: "Have 5 answers marked correct in discussions",
155 icon: "0=Ü-",
156 category: "engagement",
157 isMystery: false,
158 condition: "5 discussion replies marked as correct answer by teacher",
159 xpReward: 75,
160 },
161 {
162 id: "survey_completer",
163 name: "Survey Completer",
164 description: "Complete 3 surveys",
165 icon: "0=ÜÊ",
166 category: "engagement",
167 isMystery: false,
168 condition: "Submit responses to 3 different surveys",
169 xpReward: 50,
170 },
171 // %% Onboarding badges % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
172 {
173 id: "self_aware_scholar",
174 name: "Self-Aware Scholar",
175 description:
176   "Complete personality, learning style, and baseline assessments",
177 icon: "0=ÿ",
178 category: "engagement",
179 isMystery: false,
180 condition:
181   "Complete all three assessment sections (personality, learning style, and at least one baseline test)",
182 xpReward: 50,
183 },
184 {
185 id: "thorough_explorer",
```

```

188     name: "Thorough Explorer",
189     description: "Complete onboarding without skipping any section",
190     icon: "ð>Yi",
191     category: "engagement",
192     isMystery: false,
193     condition: "Complete onboarding without skipping any assessment section",
194     xpReward: 75,
195 },
196 // %% Habit badges %%%%%%%%%%
197 {
198   id: "habit_master",
199   name: "Habit Master",
200   description: "Completed at least one habit on 30+ days this semester",
201   icon: "ð<Bt",
202   category: "habit",
203   isMystery: false,
204   condition: "30+ active habit days in a semester",
205   xpReward: 100,
206 },
207 {
208   id: "wellness_warrior",
209   name: "Wellness Warrior",
210   description: "Logged wellness habits for 14 consecutive days",
211   icon: "ð>YØ",
212   category: "habit",
213   isMystery: false,
214   condition: "14 consecutive days with \"e1 wellness habit\"",
215   xpReward: 75,
216 },
217 {
218   id: "full_spectrum",
219   name: "Full Spectrum",
220   description: "Completed all academic and wellness habits on 7 days",
221   icon: "ð<B ",
222   category: "habit",
223   isMystery: false,
224   condition: "7 days with all 4 academic + \"e1 wellness habit\"",
225   xpReward: 150,
226 },
227 // %% Bloom's Progression badges %%%%%%%%%%
228 {
229   id: "bloom_explorer",
230   name: "Bloom's Explorer",
231   description: "Reached Bloom's level 4 (Analyzing) on any CLO",
232   icon: "ð<B1",
233   category: "blooms",
234   isMystery: false,
235   condition: "Reach highest_bloom_level >= 4 on any CLO via adaptive quiz",
236   xpReward: 75,
237 },
238 {
239   id: "bloom_challenger",
240   name: "Bloom's Challenger",
241   description: "Reached Bloom's level 5 (Evaluating) on any CLO",
242   icon: "ð<B?",
243   category: "blooms",
244   isMystery: false,
245   condition: "Reach highest_bloom_level >= 5 on any CLO via adaptive quiz",
246   xpReward: 100,
247 },
248 {
249   id: "bloom_pioneer",
250   name: "Bloom's Pioneer",
251   description: "Reached Bloom's level 6 (Creating) on any CLO",
252   icon: "ð<B3",
253   category: "blooms",
254   isMystery: false,
255   condition: "Reach highest_bloom_level >= 6 on any CLO via adaptive quiz",
256   xpReward: 150,
257 },
258 // %% Team badges %%%%%%%%%%
259 {
260   id: "team_spirit",
261   name: "Team Spirit",
262   description: "Team earned 500 XP together",
263   icon: "ð>Ý ",
264   category: "team",
265   isMystery: false,
266   condition: "Team earns 500 XP collectively",
267   xpReward: 100,
268 },
269 {
270   id: "unstoppable",
271   name: "Unstoppable",
272   description: "Team won 3 challenges",
273   icon: "ð=Üª",
274   category: "team",
275   isMystery: false,
276   condition: "Team wins 3 challenges",
277   xpReward: 150,
278 },
279 {
280   id: "dream_team",
281   name: "Dream Team",
282   description: "All members completed a Perfect Day on the same day",
283   icon: "+P",
284   category: "team",
285   isMystery: false,
286   condition: "All team members complete all 4 daily habits on the same day",
287   xpReward: 200,
288 },

```

[illegible]

```

396 |
397 | /** Get all badge IDs */
398 | export function getAllBadgeIds(): string[] {
399 |     return BADGE_DEFINITIONS.map((b) => b.id);
400 | }
401 |
402 | /** Get badges by category */
403 | export function getBadgesByCategory(category: BadgeCategory): BadgeDef[] {
404 |     return BADGE_DEFINITIONS.filter((b) => b.category === category);
405 | }
406 |
407 | /** Get only mystery badges */
408 | export function getMysteryBadges(): BadgeDef[] {
409 |     return BADGE_DEFINITIONS.filter((b) => b.isMystery);
410 | }
411 |
412 | /** Get only visible (non-mystery) badges */
413 | export function getVisibleBadges(): BadgeDef[] {
414 |     return BADGE_DEFINITIONS.filter((b) => !b.isMystery);
415 | }
416 |

```

#### FILE: src/lib/badgeFeedback.ts

TYPE: .ts | LINES: 1-39 | SHA256: 3ab784b0adfbe70891fff0af706f92a47fac2ff5023ea77b0580c7fd12429acd

```

1 | import { getBadgeById } from "@lib/badgeDefinitions";
2 |
3 | export const BADGE_FEEDBACK_EVENT = "edviser:badge-feedback";
4 |
5 | export interface BadgeFeedbackDetail {
6 |     id: string;
7 |     name: string;
8 |     description: string;
9 |     icon: string;
10 |     isMystery: boolean;
11 |     xpReward: number;
12 | }
13 |
14 | /**
15 |  * Publishes celebrations only for badge keys returned by the real
16 |  * check-badges response. Unknown institution-specific keys stay silent until
17 |  * their display metadata is available.
18 |  */
19 | export const emitBadgeFeedback = (badgeIds: readonly string[]): void => {
20 |     if (typeof window === "undefined") return;
21 |
22 |     for (const badgeId of badgeIds) {
23 |         const badge = getBadgeById(badgeId);
24 |         if (!badge) continue;
25 |
26 |         const detail: BadgeFeedbackDetail = {
27 |             id: badge.id,
28 |             name: badge.name,
29 |             description: badge.description,
30 |             icon: badge.icon,
31 |             isMystery: badge.isMystery,
32 |             xpReward: badge.xpReward,
33 |         };
34 |         window.dispatchEvent(
35 |             new CustomEvent<BadgeFeedbackDetail>(BADGE_FEEDBACK_EVENT, { detail })
36 |         );
37 |     }
38 | };
39 |

```

#### FILE: src/lib/badgeSpotlightResolver.ts

TYPE: .ts | LINES: 1-101 | SHA256: 4a1086b9f30daa4c03294e2a71e799cfeb70e162513f45fe47e59ed5ef0ba167

```

1 | // =====
2 | // Badge Spotlight Resolver - Pure function
3 | // Deterministic badge rotation from student_id hash + week number
4 | // Different students see different badges in the same week
5 | // =====
6 |
7 | export interface BadgeDefinition {
8 |     id: string;
9 |     name: string;
10 |     isArchived: boolean;
11 | }
12 |
13 | export interface SpotlightResult {
14 |     /** The selected badge definition ID, or null if no eligible badges */
15 |     badgeId: string | null;
16 |     /** The selected badge name, or null if no eligible badges */
17 |     badgeName: string | null;
18 |     /** The week number used for the computation */
19 |     weekNumber: number;
20 | }
21 |
22 | /**
23 |  * Simple string hash function (djb2 variant).
24 |  * Produces a non-negative 32-bit integer from a string.
25 |  */
26 | function hashString(str: string): number {
27 |     let hash = 5381;
28 |     for (let i = 0; i < str.length; i++) {
29 |         // hash * 33 + char
30 |         hash = ((hash << 5) + hash + str.charCodeAt(i)) | 0;
31 |     }
32 |     // Ensure non-negative
33 |     return Math.abs(hash);
34 | }
35 |
36 | /**

```

```

37 | * Compute the current ISO week number for a given date.
38 | * ISO weeks start on Monday; week 1 contains the first Thursday of the year.
39 | */
40 | export function getISOWeekNumber(date: Date): number {
41 |     const d = new Date(
42 |         Date.UTC(date.getFullYear(), date.getMonth(), date.getDate())
43 |     );
44 |     // Set to nearest Thursday: current date + 4 - current day number (Monday=1, Sunday=7)
45 |     const dayNum = d.getUTCDay() || 7;
46 |     d.setUTCDate(d.getUTCDate() + 4 - dayNum);
47 |     const yearStart = new Date(Date.UTC(d.getUTCFullYear(), 0, 1));
48 |     const weekNo = Math.ceil(
49 |         ((d.getTime() - yearStart.getTime()) / 86400000 + 1) / 7
50 |     );
51 |     return weekNo;
52 | }
53 |
54 | /**
55 |  * Resolve the badge spotlight for a given student and week.
56 |  *
57 |  * The selection is deterministic: same student + same week = same badge.
58 |  * Different students will (with high probability) see different badges.
59 |  *
60 |  * Only non-archived badges are eligible. If a set of earned gold-tier badge IDs
61 |  * is provided, those are also excluded (student has fully completed them).
62 |  *
63 |  * @param studentId - The student's UUID
64 |  * @param weekNumber - The ISO week number (or any consistent week identifier)
65 |  * @param badges - All badge definitions for the institution
66 |  * @param earnedGoldBadgeIds - Badge IDs where the student has earned gold tier (optional)
67 |  */
68 | export function resolveBadgeSpotlight(
69 |     studentId: string,
70 |     weekNumber: number,
71 |     badges: BadgeDefinition[],
72 |     earnedGoldBadgeIds: string[] = []
73 | ): SpotlightResult {
74 |     // Filter to eligible badges: not archived, not fully earned (gold)
75 |     const goldSet = new Set(earnedGoldBadgeIds);
76 |     const eligible = badges.filter((b) => !b.isArchived && !goldSet.has(b.id));
77 |
78 |     if (eligible.length === 0) {
79 |         return { badgeId: null, badgeName: null, weekNumber };
80 |     }
81 |
82 |     // Sort eligible badges by ID for deterministic ordering
83 |     const sorted = [...eligible].sort((a, b) => a.id.localeCompare(b.id));
84 |
85 |     // Deterministic index from hash of studentId + weekNumber
86 |     const hashInput = `${studentId}:${weekNumber}`;
87 |     const hash = hashString(hashInput);
88 |     const index = hash % sorted.length;
89 |
90 |     const selected = sorted[index] ?? sorted[0];
91 |     if (!selected) {
92 |         return { badgeId: null, badgeName: null, weekNumber };
93 |     }
94 |
95 |     return {
96 |         badgeId: selected.id,
97 |         badgeName: selected.name,
98 |         weekNumber,
99 |     };
100 | }
101 |

```

#### FILE: src/lib/bidiText.ts

TYPE: .ts | LINES: 1-25 | SHA256: fa0b40a98d9d882656467c36c5fb30a6daad8223ea06b77a6a906e400be194e2

```

1 | /**
2 |  * Wraps a string in Unicode LTR isolate characters for embedding
3 |  * LTR content (codes, English terms) within RTL text.
4 |  */
5 | export const ltrIsolate = (text: string): string => {
6 |     return `\u2066${text}\u2069`; // LRI ... PDI
7 | };
8 |
9 | /**
10 |  * Wraps a string in Unicode RTL isolate characters for embedding
11 |  * RTL content within LTR text.
12 |  */
13 | export const rtlIsolate = (text: string): string => {
14 |     return `\u2067${text}\u2069`; // RLI ... PDI
15 | };
16 |
17 | /**
18 |  * Detects if a string starts with an RTL character.
19 |  */
20 | export const startsWithRTL = (text: string): boolean => {
21 |     const rtlRegex =
22 |         /\u0591-\u07FF\u200F\u202B\u202E\uFB1D-\uFDFD\uFE70-\uFEFC/;
23 |     return rtlRegex.test(text.trim());
24 | };
25 |

```

#### FILE: src/lib/bilingualContent.ts

TYPE: .ts | LINES: 1-26 | SHA256: a1c541883efbe08f2ef1fe321124be5f03525cb16c95a5e3f9e2fbca257e0fb5

```

1 | export interface BilingualField {
2 |     en?: string | null;
3 |     ar?: string | null;
4 | }
5 |
6 | /**

```





```

24 |     consecutiveCorrectAtLevel: number
25 | ): number {
26 |     if (consecutiveCorrectAtLevel >= 3 && currentLevel < 6) {
27 |         return currentLevel + 1;
28 |     }
29 |     return currentLevel;
30 | }
31 |
32 | /**
33 |  * Handles Bloom's level revert when a student answers incorrectly at a
34 |  * newly advanced level.
35 |  *
36 |  * Returns `previousLevel` when the answer was incorrect and the student
37 |  * had just advanced. Returns `currentLevel` in all other cases.
38 |  *
39 |  * @param currentLevel - The student's current Bloom's level.
40 |  * @param previousLevel - The Bloom's level before the last advancement.
41 |  * @param wasCorrect - Whether the student answered correctly.
42 |  * @param justAdvanced - Whether the student just advanced to `currentLevel`.
43 |  * @returns The Bloom's level to use going forward.
44 |  */
45 | export function handleBloomRevert(
46 |     currentLevel: number,
47 |     previousLevel: number,
48 |     wasCorrect: boolean,
49 |     justAdvanced: boolean
50 | ): number {
51 |     if (!wasCorrect && justAdvanced) {
52 |         return previousLevel;
53 |     }
54 |     return currentLevel;
55 | }
56 |
57 | /**
58 |  * Computes the highest Bloom's level where the student has at least 2
59 |  * correct answers across all attempts.
60 |  *
61 |  * @param attempts - Array of quiz question attempts with Bloom's level and correctness.
62 |  * @returns The highest Bloom's level with 2+ correct answers, or 0 if none qualifies.
63 |  */
64 | export function highestBloomReached(attempts: BloomAttempt[]): number {
65 |     const correctCountByLevel = new Map<number, number>();
66 |
67 |     for (const attempt of attempts) {
68 |         if (attempt.correct) {
69 |             correctCountByLevel.set(
70 |                 attempt.bloomLevel,
71 |                 (correctCountByLevel.get(attempt.bloomLevel) ?? 0) + 1
72 |             );
73 |         }
74 |     }
75 |
76 |     let highest = 0;
77 |     for (const [level, count] of correctCountByLevel) {
78 |         if (count >= 2 && level > highest) {
79 |             highest = level;
80 |         }
81 |     }
82 |
83 |     return highest;
84 | }
85 |
86 | /**
87 |  * Returns the fixed XP awarded for completing a practice mode quiz.
88 |  * Always returns exactly 10, regardless of any parameters.
89 |  *
90 |  * @returns 10
91 |  */
92 | export function computePracticeXP(): number {
93 |     return 10;
94 | }
95 |

```

## FILE: src/lib/bloomsVerbs.ts

TYPE: .ts | LINES: 1-54 | SHA256: 6b60878c11829b39f17d6f884d0ba6f317bd9969a30ea37682df849e1218783a

```

1 | import type { BloomsLevel } from "@lib/schemas/clo";
2 |
3 | export const BLOOMS_VERBS: Record<BloomsLevel, string[]> = {
4 |     remembering: ["define", "list", "recall", "identify", "state", "name"],
5 |     understanding: ["explain", "describe", "classify", "summarize", "paraphrase"],
6 |     applying: [
7 |         "use",
8 |         "implement",
9 |         "execute",
10 |         "solve",
11 |         "demonstrate",
12 |         "construct",
13 |     ],
14 |     analyzing: ["compare", "differentiate", "examine", "break down", "infer"],
15 |     evaluating: ["judge", "critique", "defend", "argue", "assess", "recommend"],
16 |     creating: ["design", "develop", "compose", "build", "formulate", "produce"],
17 | };
18 |
19 | export const BLOOMS_COLORS: Record<
20 |     BloomsLevel,
21 |     { bg: string; text: string; hover: string }
22 | > = {
23 |     remembering: {
24 |         bg: "bg-purple-100",
25 |         text: "text-purple-700",
26 |         hover: "hover:bg-purple-200",
27 |     },
28 |     understanding: {

```

```

29 |     bg: "bg-blue-100",
30 |     text: "text-blue-700",
31 |     hover: "hover:bg-blue-200",
32 | },
33 | applying: {
34 |     bg: "bg-green-100",
35 |     text: "text-green-700",
36 |     hover: "hover:bg-green-200",
37 | },
38 | analyzing: {
39 |     bg: "bg-yellow-100",
40 |     text: "text-yellow-700",
41 |     hover: "hover:bg-yellow-200",
42 | },
43 | evaluating: {
44 |     bg: "bg-orange-100",
45 |     text: "text-orange-700",
46 |     hover: "hover:bg-orange-200",
47 | },
48 | creating: {
49 |     bg: "bg-red-100",
50 |     text: "text-red-700",
51 |     hover: "hover:bg-red-200",
52 | },
53 | };
54 |

```

## FILE: src/lib/calendarDeadlines.ts

TYPE: .ts | LINES: 1-93 | SHA256: 4e9fa72ccc408a93d10d36945d6ed78b1b4d22f07da7178ade5a50257c48cf54

```

1 | // Task 23.3: Read-only calendar deadline view – consistent, deduplicated deadlines
2 | // Requirements: 21.1, 21.2, 21.4
3 | //
4 | // The Calendar is one of four overlapping time-management surfaces (Planner,
5 | // Today, Calendar, Timetable). It is a read-only, deadline-focused view that
6 | // must present a deadline the *same* way it appears elsewhere (R21.4): a single
7 | // logical deadline must never be listed twice, even when it is surfaced from
8 | // more than one source (e.g. an assignment and a quiz sharing a title/day, or
9 | // the same item arriving from two queries).
10 | //
11 | // This module keeps the dedup logic pure and side-effect-free in `src/lib/` so
12 | // it is unit-testable independently of the data hook and the calendar view.
13 |
14 | /**
15 |  * Minimal shape required to deduplicate a deadline. `CalendarEvent` from
16 |  * `useCalendar` is structurally compatible with this interface.
17 |  */
18 | export interface DeduplicableDeadline {
19 |   /** Stable source identifier, used only as a fallback key. */
20 |   id: string;
21 |   /** Human-readable deadline title. */
22 |   title?: string | null;
23 |   /** ISO date or date-time string (`yyyy-mm-dd` or `yyyy-mm-ddTHH:MM:SS...`). */
24 |   date?: string | null;
25 |   /** Owning course, when known – distinguishes same-titled deadlines. */
26 |   course_id?: string | null;
27 | }
28 |
29 | /**
30 |  * Reduce a date or date-time string to its calendar-day portion (`yyyy-mm-dd`).
31 |  * A deadline that falls on a given day is "the same deadline" regardless of the
32 |  * exact time-of-day a particular source records, so the day is the unit of
33 |  * identity. Non-string input collapses to an empty string.
34 |  */
35 | export function normalizeDeadlineDay(date: string | null | undefined): string {
36 |   if (typeof date !== "string") return "";
37 |   const trimmed = date.trim();
38 |   if (trimmed === "") return "";
39 |   // Split on the ISO date/time separator; fall back to the whole string.
40 |   const [dayPart] = trimmed.split("T");
41 |   return dayPart ?? trimmed;
42 | }
43 |
44 | /**
45 |  * Normalize a deadline title for comparison: trimmed, lower-cased, and with
46 |  * internal whitespace collapsed, so cosmetic differences ("Essay 1" vs
47 |  * "Essay 1") do not defeat deduplication. Non-string input collapses to "".
48 |  */
49 | export function normalizeDeadlineTitle(
50 |   title: string | null | undefined
51 | ): string {
52 |   if (typeof title !== "string") return "";
53 |   return title.trim().replace(/\s+/g, " ").toLowerCase();
54 | }
55 |
56 | /**
57 |  * Compute the canonical identity of a deadline. Two deadlines that share a
58 |  * calendar day, course, and normalized title are treated as the same logical
59 |  * deadline (R21.4) – this is what keeps the calendar consistent with the other
60 |  * surfaces. When a deadline carries neither a day nor a title to anchor on, it
61 |  * falls back to its source `id` so genuinely distinct-but-empty entries are not
62 |  * incorrectly merged.
63 |  */
64 | export function deadlineIdentityKey(event: DeduplicableDeadline): string {
65 |   const day = normalizeDeadlineDay(event.date);
66 |   const title = normalizeDeadlineTitle(event.title);
67 |   if (day === "" && title === "") {
68 |     return `id:${event.id}`;
69 |   }
70 |   const course = event.course_id ?? "";
71 |   return `${day}|${course}|${title}`;
72 | }
73 |
74 | /**

```

```

75 | * Deduplicate a list of deadlines by their canonical identity, preserving input
76 | * order with first-occurrence-wins semantics. The function is pure and total:
77 | * identical input always yields identical output, the result is a subset of the
78 | * input, and applying it twice yields the same list (idempotent).
79 | */
80 | export function dedupeCalendarEvents<T extends DeduplicableDeadline>(
81 |   events: readonly T[]
82 | ): T[] {
83 |   const seen = new Set<string>();
84 |   const result: T[] = [];
85 |   for (const event of events) {
86 |     const key = deadlineIdentityKey(event);
87 |     if (seen.has(key)) continue;
88 |     seen.add(key);
89 |     result.push(event);
90 |   }
91 |   return result;
92 | }
93 |

```

#### FILE: src/lib/challengeLifecycle.ts

TYPE: .ts | LINES: 1-34 | SHA256: 21fc1fb7ecf8b87b1a9a0904e2c9a92f81d13d978ddf3ebafeda95b17c2bb108

```

1 | export type EffectiveChallengeStatus =
2 |   | "active"
3 |   | "upcoming"
4 |   | "completed"
5 |   | "cancelled";
6 |
7 | interface ChallengeLifecycleInput {
8 |   status: string;
9 |   start_date: string;
10 |   end_date: string;
11 | }
12 |
13 | export const getEffectiveChallengeStatus = (
14 |   challenge: ChallengeLifecycleInput,
15 |   now: Date = new Date()
16 | ): EffectiveChallengeStatus => {
17 |   const startsAt = new Date(challenge.start_date);
18 |   const endsAt = new Date(challenge.end_date);
19 |
20 |   if (challenge.status === "cancelled") {
21 |     return "cancelled";
22 |   }
23 |
24 |   if (challenge.status === "ended" || endsAt.getTime() < now.getTime()) {
25 |     return "completed";
26 |   }
27 |
28 |   if (challenge.status === "draft" || startsAt.getTime() > now.getTime()) {
29 |     return "upcoming";
30 |   }
31 |
32 |   return "active";
33 | };
34 |

```

#### FILE: src/lib/challengeTypes.ts

TYPE: .ts | LINES: 1-120 | SHA256: 521d0ad21c854abe38a3525f9f82abc3ccf0274830b1ed12681095df2da790b2

```

1 | // Challenge type configuration for the Social Challenges (Quests) feature.
2 | // Defines the 5 challenge types with their goal descriptions, units, and validation rules.
3 |
4 | export interface ChallengeTypeConfig {
5 |   key: ChallengeTypeKey;
6 |   label: string;
7 |   description: string;
8 |   goalDescription: string;
9 |   goalUnit: string;
10 |   fixedGoal: number | null;
11 |   validationRules: {
12 |     minGoal: number;
13 |     maxGoal: number;
14 |   };
15 | }
16 |
17 | export type ChallengeTypeKey =
18 |   | "academic"
19 |   | "habit"
20 |   | "xp_race"
21 |   | "blooms_climb"
22 |   | "cooperative";
23 |
24 | export const CHALLENGE_TYPES: Record<ChallengeTypeKey, ChallengeTypeConfig> = {
25 |   academic: {
26 |     key: "academic",
27 |     label: "Academic",
28 |     description:
29 |       "Complete a target number of assignments within the challenge period",
30 |     goalDescription: "Complete N assignments",
31 |     goalUnit: "assignments",
32 |     fixedGoal: null,
33 |     validationRules: {
34 |       minGoal: 1,
35 |       maxGoal: 100,
36 |     },
37 |   },
38 |   habit: {
39 |     key: "habit",
40 |     label: "Habit",
41 |     description:
42 |       "Maintain a daily streak for a target number of days during the challenge",
43 |     goalDescription: "Maintain a streak for N days",

```

```

44 |     goalUnit: "days",
45 |     fixedGoal: null,
46 |     validationRules: {
47 |         minGoal: 1,
48 |         maxGoal: 90,
49 |     },
50 | },
51 | xp_race: {
52 |     key: "xp_race",
53 |     label: "XP Race",
54 |     description: "First participant to earn the target XP amount wins the race",
55 |     goalDescription: "First to earn N XP",
56 |     goalUnit: "XP",
57 |     fixedGoal: null,
58 |     validationRules: {
59 |         minGoal: 50,
60 |         maxGoal: 10000,
61 |     },
62 | },
63 | blooms_climb: {
64 |     key: "blooms_climb",
65 |     label: "Bloom's Climb",
66 |     description:
67 |         "Complete at least one assignment at each of the 6 Bloom's Taxonomy levels (Remembering through Creating",
68 |     goalDescription: "Complete all 6 Bloom's levels",
69 |     goalUnit: "levels",
70 |     fixedGoal: 6,
71 |     validationRules: {
72 |         minGoal: 6,
73 |         maxGoal: 6,
74 |     },
75 | },
76 | cooperative: {
77 |     key: "cooperative",
78 |     label: "Cooperative",
79 |     description:
80 |         "Team collectively works toward a shared goal without competing against other teams",
81 |     goalDescription: "Team collectively reaches the shared goal",
82 |     goalUnit: "units",
83 |     fixedGoal: null,
84 |     validationRules: {
85 |         minGoal: 1,
86 |         maxGoal: 10000,
87 |     },
88 | },
89 | };
90 |
91 | /** Get all challenge type keys */
92 | export function getChallengeTypeKeys(): ChallengeTypeKey[] {
93 |     return Object.keys(CHALLENGE_TYPES) as ChallengeTypeKey[];
94 | }
95 |
96 | /** Get a challenge type config by key */
97 | export function getChallengeType(key: ChallengeTypeKey): ChallengeTypeConfig {
98 |     return CHALLENGE_TYPES[key];
99 | }
100 |
101 | /** Check if a challenge type has a fixed goal */
102 | export function hasFixedGoal(key: ChallengeTypeKey): boolean {
103 |     return CHALLENGE_TYPES[key].fixedGoal !== null;
104 | }
105 |
106 | /** Validate a goal target for a given challenge type */
107 | export function validateGoalTarget(
108 |     key: ChallengeTypeKey,
109 |     goalTarget: number
110 | ): boolean {
111 |     const config = CHALLENGE_TYPES[key];
112 |     if (config.fixedGoal !== null) {
113 |         return goalTarget === config.fixedGoal;
114 |     }
115 |     return (
116 |         goalTarget >= config.validationRules.minGoal &&
117 |         goalTarget <= config.validationRules.maxGoal
118 |     );
119 | }
120 |

```

## FILE: src/lib/cohortStats.ts

TYPE: .ts | LINES: 1-45 | SHA256: 41b91fe692ec58a24a19e6f6b87de79778e7deba43f7f910e25135518fde33f3

```

1 | // Task 124.2: Cohen's d effect size utility
2 |
3 | export interface CohortData {
4 |     label: string;
5 |     mean: number;
6 |     stdDev: number;
7 |     n: number;
8 | }
9 |
10 | /**
11 |  * Calculate Cohen's d effect size between two cohorts.
12 |  * Only valid when both cohorts have n >= 20.
13 |  */
14 | export const calculateCohensD = (
15 |     cohort1: CohortData,
16 |     cohort2: CohortData
17 | ): number | null => {
18 |     if (cohort1.n < 20 || cohort2.n < 20) return null;
19 |
20 |     const pooledVariance =
21 |         ((cohort1.n - 1) * cohort1.stdDev ** 2 +
22 |          (cohort2.n - 1) * cohort2.stdDev ** 2) /
23 |         (cohort1.n + cohort2.n - 2);

```

```

24 |
25 |     const pooledStdDev = Math.sqrt(pooledVariance);
26 |     if (pooledStdDev === 0) return 0;
27 |
28 |     return (cohort1.mean - cohort2.mean) / pooledStdDev;
29 | };
30 |
31 | /**
32 |  * Detect "Significant Gap" when "e15 percentage point difference.
33 |  */
34 | export const hasSignificantGap = (mean1: number, mean2: number): boolean => {
35 |     return Math.abs(mean1 - mean2) >= 15;
36 | };
37 |
38 | export const interpretCohensD = (d: number): string => {
39 |     const abs = Math.abs(d);
40 |     if (abs < 0.2) return "Negligible";
41 |     if (abs < 0.5) return "Small";
42 |     if (abs < 0.8) return "Medium";
43 |     return "Large";
44 | };
45 |

```

#### FILE: [src/lib/comebackKidChecker.ts](#)

TYPE: .ts | LINES: 1-32 | SHA256: e1a9f769458b3223a98978d66b972311b8b67fc3751c47be28963b067ebd7f4d

```

1 | // Task 139.3: Comeback Kid badge logic – pure function for testability
2 | // Awards "Comeback Kid" badge when student earns 3+ Improvement Bonuses in a single semester
3 | // Idempotent: cannot be awarded twice for same semester
4 |
5 | export const COMEBACK_KID_THRESHOLD = 3;
6 | export const COMEBACK_KID_BADGE_ID = "comeback_kid";
7 |
8 | export interface ComebackKidInput {
9 |     existingBadgeIds: Set<string>;
10 |    improvementBonusCount: number;
11 | }
12 |
13 | /**
14 |  * Determines whether the Comeback Kid badge should be awarded.
15 |  * Returns the badge ID if eligible, or an empty array if not.
16 |  *
17 |  * Requirements 123.4:
18 |  * - Award "Comeback Kid" badge when student earns 3 Improvement Bonuses within a single semester
19 |  * - Idempotent: cannot be awarded twice for same semester
20 |  */
21 | export function shouldAwardComebackKid(input: ComebackKidInput): string[] {
22 |     if (input.existingBadgeIds.has(COMEBACK_KID_BADGE_ID)) {
23 |         return [];
24 |     }
25 |
26 |     if (input.improvementBonusCount >= COMEBACK_KID_THRESHOLD) {
27 |         return [COMEBACK_KID_BADGE_ID];
28 |     }
29 |
30 |     return [];
31 | }
32 |

```

#### FILE: [src/lib/confetti.ts](#)

TYPE: .ts | LINES: 1-37 | SHA256: 081a6e2488d7d6bc0f3ea84cfabebcfb89546a2a86932e8d94fb96ab9f94e003

```

1 | // =====
2 | // confetti – lazy-loaded canvas-confetti wrapper (Task 15.2, heavy-dep hygiene)
3 | // =====
4 | //
5 | // `canvas-confetti` is a non-critical celebration effect. Importing it eagerly
6 | // pulls the library into the chunk of every component that fires confetti
7 | // (XP toasts, level-up overlay, badge modal, improvement bonus, leaderboard).
8 | // This helper defers the import to first use via a dynamic `import()`, so the
9 | // library is fetched only when a celebration actually happens (many sessions
10 | // never fire one), and gives every call site one shared, fire-and-forget entry
11 | // point instead of five duplicate eager imports.
12 | //
13 | // Failures (e.g. a chunk-load error on a flaky network) are swallowed: confetti
14 | // is decoration and must never break the surrounding UI. Callers remain
15 | // responsible for honoring `prefers-reduced-motion` before calling.
16 | //
17 | // Derive the options type from the package without importing it at runtime, so
18 | // this stays type-safe (no `any`) while keeping the dependency fully lazy.
19 | type ConfettiOptions = Parameters<typeof import("canvas-confetti")>[0];
20 |
21 | /**
22 |  * Fire a confetti burst, lazy-loading `canvas-confetti` on first call.
23 |  *
24 |  * Fire-and-forget: the returned promise resolves once the burst has been
25 |  * triggered (or silently after a swallowed load error). Awaiting is optional.
26 |  */
27 | export const launchConfetti = async (
28 |     options?: ConfettiOptions
29 | ): Promise<void> => {
30 |     try {
31 |         const { default: confetti } = await import("canvas-confetti");
32 |         confetti(options);
33 |     } catch {
34 |         // Non-essential decoration – ignore load/runtime errors.
35 |     }
36 | };
37 |

```

## FILE: src/lib/contributionThresholds.ts

TYPE: .ts | LINES: 1-55 | SHA256: 580dda7f80d9da48d2849390d4467dad841d5209e6c6e24bd1c8d14a5f019dd0

```
1 | // Contribution accountability thresholds and status transition rules.
2 | // Used by the daily contribution status cron job (01:00 UTC) to detect free-riding.
3 |
4 | export type ContributionStatus = "active" | "warning" | "inactive";
5 |
6 | /** Default minimum contribution threshold as a percentage of team weekly XP */
7 | export const DEFAULT_CONTRIBUTION_THRESHOLD = 0.2;
8 |
9 | /** Number of consecutive low-contribution days before transitioning to 'warning' */
10 | export const WARNING_THRESHOLD_DAYS = 3;
11 |
12 | /** Number of consecutive low-contribution days before transitioning to 'inactive' */
13 | export const INACTIVE_THRESHOLD_DAYS = 5;
14 |
15 | /**
16 |  * Compute the contribution status based on consecutive low-contribution days.
17 |  *
18 |  * Status transition rules:
19 |  * - 0-2 consecutive low days != 'active'
20 |  * - 3-4 consecutive low days != 'warning'
21 |  * - 5+ consecutive low days != 'inactive'
22 |  * - When contribution rises back above threshold != 'active' (reset consecutive_low_days to 0)
23 |  */
24 | export function computeContributionStatus(
25 |   consecutiveLowDays: number
26 | ): ContributionStatus {
27 |   if (consecutiveLowDays >= INACTIVE_THRESHOLD_DAYS) {
28 |     return "inactive";
29 |   }
30 |   if (consecutiveLowDays >= WARNING_THRESHOLD_DAYS) {
31 |     return "warning";
32 |   }
33 |   return "active";
34 | }
35 |
36 | /**
37 |  * Determine whether a member's contribution is below the threshold.
38 |  *
39 |  * @param memberXp - XP earned by the member in the evaluation period
40 |  * @param teamTotalXp - Total XP earned by the team in the evaluation period
41 |  * @param threshold - Minimum contribution percentage (default 20%)
42 |  * @returns true if the member is below the threshold
43 |  */
44 | export function isBelowThreshold(
45 |   memberXp: number,
46 |   teamTotalXp: number,
47 |   threshold: number = DEFAULT_CONTRIBUTION_THRESHOLD
48 | ): boolean {
49 |   if (teamTotalXp <= 0) {
50 |     // If the team earned no XP, no one is below threshold
51 |     return false;
52 |   }
53 |   return memberXp / teamTotalXp < threshold;
54 | }
55 |
```

## FILE: src/lib/correlationConfidence.ts

TYPE: .ts | LINES: 1-54 | SHA256: cc3f8d82c333e387bb982cf4e88f341f4cb0459ad254e4543fdce13abb3a4cdc

```
1 | import type { CorrelationConfidenceLevel } from "@types/habits";
2 |
3 | /**
4 |  * Maps a data point count to a correlation confidence level.
5 |  *
6 |  * - < 30 != null (below minimum threshold)
7 |  * - 30-59 != 'early_pattern'
8 |  * - 60-89 != 'emerging_trend'
9 |  * - 90+ != 'strong_pattern'
10 |  */
11 | export function getCorrelationConfidenceLevel(
12 |   dataPointCount: number
13 | ): CorrelationConfidenceLevel | null {
14 |   if (dataPointCount < 30) return null;
15 |   if (dataPointCount < 60) return "early_pattern";
16 |   if (dataPointCount < 90) return "emerging_trend";
17 |   return "strong_pattern";
18 | }
19 |
20 | // -----
21 | // Correlation threshold logic (shared between Edge Function and client)
22 | // -----
23 |
24 | export interface CorrelationThresholdResult {
25 |   insights: unknown[];
26 |   insufficient_data: boolean;
27 |   daysUntilReady?: number;
28 | }
29 |
30 | /**
31 |  * Applies the correlation data threshold rules:
32 |  * - < 14 days != insufficient data, no daysUntilReady
33 |  * - 14-29 days != insufficient data, daysUntilReady = 30 - dayCount
34 |  * - 30+ days != sufficient data
35 |  *
36 |  * This is the canonical threshold logic used by the
37 |  * compute-habit-correlations Edge Function.
38 |  */
39 | export function applyCorrelationThreshold(
40 |   dayCount: number
41 | ): CorrelationThresholdResult {
```

```

42 |   if (dayCount < 14) {
43 |     return { insights: [], insufficient_data: true };
44 |   }
45 |   if (dayCount < 30) {
46 |     return {
47 |       insights: [],
48 |       insufficient_data: true,
49 |       daysUntilReady: 30 - dayCount,
50 |     };
51 |   }
52 |   return { insights: [], insufficient_data: false };
53 | }
54 |

```

#### FILE: src/lib/courseColor.ts

TYPE: .ts | LINES: 1-56 | SHA256: b0baa4a79b752269a01871a2c4feb5075eb3aaaf53361d78d8d0b19ae81b7aa9

```

1 | /**
2 |  * Deterministic course color resolution for student course cards.
3 |  *
4 |  * `courses.color` is nullable (R9.3/R9.4). When an institution has assigned a
5 |  * valid hex color it is used verbatim; otherwise a stable color is derived from
6 |  * the course id so the same course always shows the same accent across sessions
7 |  * and devices — no random or index-dependent assignment.
8 |  *
9 |  * Kept free of React/Supabase so it is trivially unit-testable and reusable.
10 |  *
11 |  * Satisfies Requirement 9.3.
12 |  */
13 |
14 | /** Brand-aligned palette (mirrors the calendar/timetable accent palette). */
15 | export const COURSE_COLOR_PALETTE = [
16 |   "#3b82f6", // blue-500
17 |   "#8b5cf6", // violet-500
18 |   "#ef4444", // red-500
19 |   "#f59e0b", // amber-500
20 |   "#22c55e", // green-500
21 |   "#14b8a6", // teal-500
22 |   "#ec4899", // pink-500
23 |   "#6366f1", // indigo-500
24 | ] as const;
25 |
26 | const HEX_COLOR = /^#[0-9A-Fa-f]{6}$/;
27 |
28 | /**
29 |  * Produces a non-negative 32-bit integer from a string (djb2).
30 |  * Identical input always yields an identical hash, giving deterministic colors.
31 |  */
32 | const hashString = (value: string): number => {
33 |   let hash = 5381;
34 |   for (let i = 0; i < value.length; i++) {
35 |     // hash * 33 + charCode, kept in 32-bit space.
36 |     hash = ((hash << 5) + hash + value.charCodeAt(i)) | 0;
37 |   }
38 |   return Math.abs(hash);
39 | };
40 |
41 | /**
42 |  * Resolve the color used as a course card's visual identifier.
43 |  *
44 |  * @param color - The stored `courses.color` value, or null when unassigned.
45 |  * @param courseId - The course id used to derive a deterministic fallback.
46 |  * @returns A 6-digit hex color string.
47 |  */
48 | export const resolveCourseColor = (
49 |   color: string | null | undefined,
50 |   courseId: string
51 | ): string => {
52 |   if (color && HEX_COLOR.test(color)) return color;
53 |   const index = hashString(courseId) % COURSE_COLOR_PALETTE.length;
54 |   return COURSE_COLOR_PALETTE[index] ?? COURSE_COLOR_PALETTE[0];
55 | };
56 |

```

#### FILE: src/lib/courseMaterialIndexing.ts

TYPE: .ts | LINES: 1-125 | SHA256: 09bedaf8c1766f4410a7fde76a589b639534e842bf6c2a4f80a65a1cb3fc348c

```

1 | // =====
2 | // courseMaterialIndexing — RAG embedding trigger business logic
3 | // =====
4 | //
5 | // Domain: when a teacher uploads a course material file, the document text
6 | // should be chunked and embedded into `course_material_embeddings` so the AI
7 | // tutor can cite it (RAG). The `embed-course-material` Edge Function owns the
8 | // extraction + embedding work; this helper is the single caller that wires the
9 | // teacher material-upload flow to that function.
10 | //
11 | // This lives in `src/lib/` (clean-architecture: business logic, not a hook or
12 | // component) so the material create/update hooks can call it after a successful
13 | // write.
14 | //
15 | // Graceful degradation: embedding requires an embeddings provider
16 | // ( `OPENAI_API_KEY` Supabase secret, or an OpenRouter-compatible embedding
17 | // model) configured on the Edge Function. When no provider is configured the
18 | // function returns an error and no embeddings are populated, but the upload
19 | // itself still succeeds and the tutor continues to answer from persona + CLO
20 | // context. This call is therefore fire-and-forget: errors are caught and logged
21 | // (never thrown) so the upload UX is never blocked on indexing.
22 |
23 | import { supabase } from "@lib/supabase";
24 |
25 | // The embed-course-material function can only extract text from these formats.
26 | // Other uploads (images, video, slides binaries, archives) are skipped so we do
27 | // not generate guaranteed `indexing_failed` rows + teacher notifications.

```



```

28 | const SUPPORTED_INDEX_EXTENSIONS = ["pdf", "docx", "txt", "md"] as const;
29 |
30 | // material_type values accepted by the embed-course-material EmbedRequest.
31 | type EmbedMaterialType =
32 |   | "lecture_notes"
33 |   | "slides"
34 |   | "assignment_description"
35 |   | "rubric_criteria"
36 |   | "other";
37 |
38 | export interface IndexCourseMaterialParams {
39 |   /** Storage path within the `course-materials` bucket (the uploaded file). */
40 |   filePath: string;
41 |   /** The course the material belongs to. */
42 |   courseId: string;
43 |   /** The created/updated `course-materials` row id (for re-index dedupe). */
44 |   sourceMaterialId: string;
45 |   /** CLOs linked to the material, used for RAG scoping. */
46 |   cloIds?: string[];
47 |   /** When true, instruct the function to delete prior chunks before re-indexing. */
48 |   reindex?: boolean;
49 | }
50 |
51 | export interface IndexCourseMaterialResult {
52 |   /** True when an embedding request was dispatched (supported, non-empty path). */
53 |   dispatched: boolean;
54 |   /** Reason indexing was skipped, when `dispatched` is false. */
55 |   skippedReason?: "no_file_path" | "no_course_id" | "unsupported_format";
56 | }
57 |
58 | /** Lower-cased file extension (without the dot) derived from a storage path. */
59 | const extensionOf = (path: string): string =>
60 |   path.split(".").pop()?.toLowerCase() ?? "";
61 |
62 | /** Filename portion of a storage path, used by the function for format sniffing. */
63 | const filenameOf = (path: string): string => path.split("/").pop() ?? path;
64 |
65 | /**
66 |  * Fires the `embed-course-material` Edge Function for a freshly uploaded course
67 |  * material file, when the file format is text-extractable.
68 |  *
69 |  * Fire-and-forget: never throws. Returns a result so callers can observe
70 |  * whether a request was dispatched without having to handle errors.
71 |  */
72 | export const indexCourseMaterialIfSupported = async (
73 |   params: IndexCourseMaterialParams
74 | ): Promise<IndexCourseMaterialResult> => {
75 |   const { filePath, courseId, sourceMaterialId, cloIds, reindex } = params;
76 |
77 |   if (!filePath) {
78 |     return { dispatched: false, skippedReason: "no_file_path" };
79 |   }
80 |   if (!courseId) {
81 |     return { dispatched: false, skippedReason: "no_course_id" };
82 |   }
83 |
84 |   const ext = extensionOf(filePath);
85 |   if (
86 |     !SUPPORTED_INDEX_EXTENSIONS.includes(
87 |       ext as (typeof SUPPORTED_INDEX_EXTENSIONS)[number]
88 |     )
89 |   ) {
90 |     return { dispatched: false, skippedReason: "unsupported_format" };
91 |   }
92 |
93 |   const materialType: EmbedMaterialType = "lecture_notes";
94 |
95 |   try {
96 |     const { error } = await supabase.functions.invoke("embed-course-material", {
97 |       body: {
98 |         file_url: filePath,
99 |         course_id: courseId,
100 |         clo_ids: cloIds ?? [],
101 |         material_type: materialType,
102 |         source_filename: filenameOf(filePath),
103 |         source_material_id: sourceMaterialId,
104 |         reindex: reindex === true,
105 |       },
106 |     });
107 |
108 |     if (error) {
109 |       // Non-fatal: provider may be unconfigured or extraction may fail. The
110 |       // upload already succeeded; the tutor degrades gracefully without RAG.
111 |       console.error(
112 |         "[indexCourseMaterialIfSupported] embed-course-material invocation returned an error:",
113 |         error.message
114 |       );
115 |     }
116 |   } catch (err) {
117 |     console.error(
118 |       "[indexCourseMaterialIfSupported] embed-course-material invocation failed:",
119 |       err instanceof Error ? err.message : String(err)
120 |     );
121 |   }
122 |
123 |   return { dispatched: true };
124 | };
125 |

```

# FILE: src/lib/coverageHeatmap.ts

TYPE: .ts | LINES: 1-117 | SHA256: 50222b0fbefb005450c77d41479f85ca7ed5b2299e245c6bf60cf176a18cb221

```
1 // Task 117.3: Coverage Heatmap data utility
2
3 export interface HeatmapCell {
4   clo_id: string;
5   course_id: string;
6   evidence_count: number;
7   avg_attainment: number;
8 }
9
10 export interface HeatmapMatrix {
11   clo_ids: string[];
12   course_ids: string[];
13   clo_labels: Record<string, string>;
14   course_labels: Record<string, string>;
15   cells: Map<string, HeatmapCell>;
16 }
17
18 const cellKey = (cloId: string, courseId: string) => `${cloId}:${courseId}`;
19
20 export const buildHeatmapMatrix = (
21   clos: Array<{ id: string; title: string }>,
22   courses: Array<{ id: string; name: string }>,
23   evidence: Array<{ clo_id: string; course_id: string; score_percent: number }>
24 ): HeatmapMatrix => {
25   const clo_ids = clos.map((c) => c.id);
26   const course_ids = courses.map((c) => c.id);
27   const clo_labels: Record<string, string> = {};
28   const course_labels: Record<string, string> = {};
29
30   clos.forEach((c) => {
31     clo_labels[c.id] = c.title;
32   });
33   courses.forEach((c) => {
34     course_labels[c.id] = c.name;
35   });
36
37   const cells = new Map<string, HeatmapCell>();
38
39   for (const e of evidence) {
40     const key = cellKey(e.clo_id, e.course_id);
41     const existing = cells.get(key);
42     if (existing) {
43       const newCount = existing.evidence_count + 1;
44       existing.avg_attainment =
45         (existing.avg_attainment * existing.evidence_count + e.score_percent) /
46         newCount;
47       existing.evidence_count = newCount;
48     } else {
49       cells.set(key, {
50         clo_id: e.clo_id,
51         course_id: e.course_id,
52         evidence_count: 1,
53         avg_attainment: e.score_percent,
54       });
55     }
56   }
57
58   return { clo_ids, course_ids, clo_labels, course_labels, cells };
59 };
60
61 export const getEvidenceCountColor = (count: number): string => {
62   if (count === 0) return "#ffffff";
63   if (count <= 2) return "#dbeafe";
64   if (count <= 5) return "#93c5fd";
65   return "#2563eb";
66 };
67
68 export const getAttainmentColor = (score: number): string => {
69   if (score >= 85) return "#dcfce7";
70   if (score >= 70) return "#d4edda";
71   if (score >= 50) return "#fff3cd";
72   return "#fee2e2";
73 };
74
75 /**
76  * Returns an accessible text color (near-black or white) for a given cell
77  * background hex, so the number stays legible on both the light cells and the
78  * darker evidence-count cells (WCAG 1.4.3 - 4.5:1). Uses WCAG relative
79  * luminance; the 0.35 threshold keeps near-black text on all light/mid cells
80  * and flips to white only on the darkest cell (evidence 6+, #2563eb).
81  */
82 export const getHeatmapCellTextColor = (bgHex: string): string => {
83   const hex = bgHex.replace("#", "");
84   if (hex.length !== 6) return "#0f172a";
85   const toLinear = (channel: number): number => {
86     const s = channel / 255;
87     return s <= 0.03928 ? s / 12.92 : Math.pow((s + 0.055) / 1.055, 2.4);
88   };
89   const r = toLinear(parseInt(hex.slice(0, 2), 16));
90   const g = toLinear(parseInt(hex.slice(2, 4), 16));
91   const b = toLinear(parseInt(hex.slice(4, 6), 16));
92   const luminance = 0.2126 * r + 0.7152 * g + 0.0722 * b;
93   return luminance > 0.35 ? "#0f172a" : "ffffff";
94 };
95
96 /** Legend entry: a swatch color plus its human label, per color mode. */
97 export interface HeatmapLegendEntry {
98   color: string;
99   label: string;
100 }
101
```

```

102 | /** Legend entries for the evidence-count color scale (light !' dark blue). */
103 | export const EVIDENCE_LEGEND: HeatmapLegendEntry[] = [
104 |   { color: "#ffffff", label: "None" },
105 |   { color: "#d9eaf7", label: "1-2" },
106 |   { color: "#93c5fd", label: "3-5" },
107 |   { color: "#2563eb", label: "6+" },
108 | ];
109 |
110 | /** Legend entries for the attainment color scale (threshold bands). */
111 | export const ATTAINMENT_LEGEND: HeatmapLegendEntry[] = [
112 |   { color: "#dcfce7", label: "Excellent >85%" },
113 |   { color: "#d9eaf7", label: "Satisfactory 70-84%" },
114 |   { color: "#fef9c3", label: "Developing 50-69%" },
115 |   { color: "#fee2e2", label: "Not Yet <50%" },
116 | ];
117 |

```

FILE: [src/lib/dashboardCache.ts](#)

TYPE: .ts | LINES: 1-95 | SHA256: c5609b7ee3a938e0f185cff66f782b50a68041adb5da719a3da43d9b284d9846

```

1 | // =====
2 | // Per-user dashboard snapshot cache (Option J – SWR shell-first for KPIs)
3 | // =====
4 | //
5 | // Stores the last-known student dashboard aggregate in localStorage so the KPI
6 | // cards render INSTANTLY on repeat loads and hard refreshes (localStorage
7 | // survives Ctrl+Shift+R), then the query revalidates in the background
8 | // (stale-while-revalidate). This does NOT help a brand-new user's first-ever
9 | // load (nothing cached yet) – that path is addressed by critical-first
10 | // sequencing and the server-side snapshot read-model.
11 | //
12 | // SECURITY (cross-user no-leak): identical guarantee to profileCache. A single
13 | // key is used and overwritten per user; a snapshot is ONLY ever returned for the
14 | // exact user id that owns it, and it is cleared on sign-out. It stores derived
15 | // dashboard numbers only – never auth tokens or secrets. RLS remains the source
16 | // of truth; this is non-authoritative UI hydration.
17 | // =====
18 |
19 | const STORAGE_KEY = "edviser.dashboard.student.v1";
20 |
21 | /**
22 |  * Snapshots older than this are ignored on read (and dropped). SWR still
23 |  * revalidates anything fresher; this is just an upper bound so a device that
24 |  * hasn't been used in a long time doesn't flash very old numbers.
25 |  */
26 | export const DASHBOARD_CACHE_MAX_AGE_MS = 24 * 60 * 60 * 1000; // 24h
27 |
28 | interface CachedDashboardEnvelope<T> {
29 |   userId: string;
30 |   data: T;
31 |   cachedAt: number;
32 | }
33 |
34 | const isBrowser = (): boolean =>
35 |   typeof window !== "undefined" && typeof window.localStorage !== "undefined";
36 |
37 | /**
38 |  * Returns the cached dashboard snapshot for `userId` (with its capture time), or
39 |  * null on miss / foreign owner / too-old / parse error. A foreign or stale entry
40 |  * is proactively removed so it can never linger on the device.
41 |  */
42 | export const readCachedDashboard = <T>(<T> {
43 |   userId: string
44 | }): { data: T; cachedAt: number } | null => {
45 |   if (!isBrowser() || !userId) return null;
46 |   try {
47 |     const raw = window.localStorage.getItem(STORAGE_KEY);
48 |     if (!raw) return null;
49 |
50 |     const parsed = JSON.parse(raw) as Partial<CachedDashboardEnvelope<T>>;
51 |     // Identity guard (cross-user no-leak): only return a snapshot to its owner.
52 |     if (
53 |       !parsed ||
54 |       parsed.userId !== userId ||
55 |       typeof parsed.cachedAt !== "number" ||
56 |       parsed.data === undefined
57 |     ) {
58 |       window.localStorage.removeItem(STORAGE_KEY);
59 |       return null;
60 |     }
61 |     if (Date.now() - parsed.cachedAt > DASHBOARD_CACHE_MAX_AGE_MS) {
62 |       window.localStorage.removeItem(STORAGE_KEY);
63 |       return null;
64 |     }
65 |     return { data: parsed.data as T, cachedAt: parsed.cachedAt };
66 |   } catch {
67 |     return null;
68 |   }
69 | };
70 |
71 | /** Persists `data` as the current user's dashboard snapshot. */
72 | export const writeCachedDashboard = <T>(userId: string, data: T): void => {
73 |   if (!isBrowser() || !userId) return;
74 |   try {
75 |     const envelope: CachedDashboardEnvelope<T> = {
76 |       userId,
77 |       data,
78 |       cachedAt: Date.now(),
79 |     };
80 |     window.localStorage.setItem(STORAGE_KEY, JSON.stringify(envelope));
81 |   } catch {
82 |     // Non-fatal: a full or blocked localStorage must never break the dashboard.
83 |   }
84 | };

```



[illegible]

[illegible]

```

298 | * - FAIL when an Allowlist entry no longer matches any live violation (a stale
299 | * entry that must be removed – shrinking-only, Req 17.5).
300 | * - FAIL when the total live occurrence count exceeds `maxCount` (the
301 | * non-increasing baseline – Req 17.7).
302 | *
303 | * Pure: depends only on its arguments.
304 | *
305 | * @param violations - All live cast occurrences scanned from `src`.
306 | * @param allowlist - The tracked baseline Allowlist.
307 | * @returns A verdict with pass/fail, reasons, and the offending sets.
308 | */
309 | export const evaluateCastGuard = (
310 |   violations: CastViolation[],
311 |   allowlist: Allowlist
312 | ): CastGuardVerdict => {
313 |   const allowKeys = new Set(allowlist.entries.map(keyOf));
314 |   const violationKeys = new Set(violations.map(keyOf));
315 |
316 |   // New violations: any live occurrence whose (file, pattern) is not allowlisted.
317 |   const newViolations = violations.filter((v) => !allowKeys.has(keyOf(v)));
318 |
319 |   // Stale entries: allowlisted (file, pattern) pairs that no longer occur.
320 |   const staleEntries = allowlist.entries.filter(
321 |     (e) => !violationKeys.has(keyOf(e))
322 |   );
323 |
324 |   const countExceeded = violations.length > allowlist.maxCount;
325 |
326 |   const reasons: string[] = [];
327 |
328 |   for (const v of newViolations) {
329 |     reasons.push(
330 |       `New disallowed cast: ${v.file}:${v.line} uses '${v.pattern}'.` +
331 |       `Remove the 'as never' cast (regenerate types) or, for a genuinely ` +
332 |       `un-typed object, add { "file": "${v.file}", "pattern": "${v.pattern}" } ` +
333 |       `to supabaseCastAllowlist.json and raise maxCount.`
334 |     );
335 |   }
336 |
337 |   for (const e of staleEntries) {
338 |     reasons.push(
339 |       `Stale allowlist entry: { file: "${e.file}", pattern: "${e.pattern}" } ` +
340 |       `no longer matches any cast. Remove it from supabaseCastAllowlist.json ` +
341 |       `and lower maxCount (the allowlist is shrinking-only).`
342 |     );
343 |   }
344 |
345 |   if (countExceeded) {
346 |     reasons.push(
347 |       `Cast count regressed: found ${violations.length} '... as never' Supabase ` +
348 |       `casts but the baseline maxCount is ${allowlist.maxCount}. The count must ` +
349 |       `never increase.`
350 |     );
351 |   }
352 |
353 |   return {
354 |     pass: reasons.length === 0,
355 |     reasons,
356 |     newViolations,
357 |     staleEntries,
358 |     countExceeded,
359 |     violationCount: violations.length,
360 |   };
361 | };
362 |

```

FILE: [src/lib/db/insertColumns.ts](#)

TYPE: .ts | LINES: 1-52 | SHA256: 3bee4eb60bef0a68c6ce5e5baef157b487a709ca0d5ba5017a610843ff6f81ad

```

1 | /**
2 | * Per-table allowed-column constants for database insert payloads.
3 | *
4 | * Each constant lists exactly the columns a hook is permitted to send to a
5 | * given table's `insert`, and is paired with {@link pickColumns} to strip any
6 | * UI-only or non-column fields before the call. The lists are tied to the
7 | * generated Supabase `Insert` types via `satisfies readonly InsertKeys<T>[]`,
8 | * so a renamed or removed column fails `tsc` at compile time rather than
9 | * surfacing as a runtime insert error.
10 | *
11 | * Keep these in lockstep with `src/types/database.ts` (regenerate types after
12 | * every migration with `pwsh scripts/regen-types.ps1`; never hand-edit the
13 | * generated file).
14 | */
15 | import type { Database } from "@types/database";
16 |
17 | /** The union of valid `Insert` keys for a public table `T`. */
18 | type InsertKeys<T> extends keyof Database["public"]["Tables"]> =
19 |   keyof Database["public"]["Tables"][T]["Insert"];
20 |
21 | /**
22 | * Columns accepted by `social_challenges.insert`.
23 | *
24 | * Notably excludes the UI-only `xp_race_acknowledged` field (it is not a real
25 | * column) so the acknowledgment gate stays entirely client-side.
26 | */
27 | export const SOCIAL_CHALLENGES_INSERT_COLUMNS = [
28 |   "course_id",
29 |   "institution_id",
30 |   "created_by",
31 |   "title",
32 |   "description",
33 |   "challenge_type",
34 |   "participation_mode",
35 |   "goal_target",

```

```

36 | "start_date",
37 | "end_date",
38 | "reward_xp",
39 | "reward_badge_id",
40 | "status",
41 | ] as const satisfies readonly InsertKeys<"social_challenges">[];
42 |
43 | /** Columns accepted by `teams.insert` (covers all required NOT NULL columns). */
44 | export const TEAMS_INSERT_COLUMNS = [
45 |   "name",
46 |   "course_id",
47 |   "institution_id",
48 |   "captain_id",
49 |   "created_by",
50 |   "avatar_letter",
51 | ] as const satisfies readonly InsertKeys<"teams">[];
52 |

```

#### FILE: src/lib/db/pickColumns.ts

TYPE: .ts | LINES: 1-43 | SHA256: 1d3e8231e747f7702c8462ae51301b5d0f35ae50b695c9625f29eb4ddc8ea400

```

1 | /**
2 |  * Generic, type-safe column whitelist for database insert/update payloads.
3 |  *
4 |  * `pickColumns` returns a new object containing only the `allowed` keys that
5 |  * are both present on the payload and not `undefined`. It is the shared, SOLID,
6 |  * no-duplication building block behind the per-table insert whitelists (e.g.
7 |  * `SOCIAL_CHALLENGES_INSERT_COLUMNS`, `TEAMS_INSERT_COLUMNS`): by stripping any
8 |  * UI-only fields (such as `xp_race_acknowledged`) before an insert, it lets the
9 |  * generated Supabase `Insert` types do their job instead of being defeated by an
10 |  * as never `cast`.
11 |  *
12 |  * The function is pure: it never mutates `payload` and has no side effects.
13 |  *
14 |  * @typeParam T - The shape of the source payload.
15 |  * @typeParam K - The union of keys allowed through to the result.
16 |  * @param payload - The source object to copy allowed keys from.
17 |  * @param allowed - The readonly list of keys permitted in the result.
18 |  * @returns A new object containing only the allowed, defined keys of `payload`.
19 |  *
20 |  * @example
21 |  * const row = pickColumns(
22 |  *   { title: "Race", xp_race_acknowledged: true },
23 |  *   ["title"] as const
24 |  * );
25 |  * // row === { title: "Race" } - xp_race_acknowledged is dropped
26 |  */
27 | export const pickColumns = <T extends object, K extends keyof T>(
28 |   payload: T,
29 |   allowed: readonly K[]
30 | ): Pick<T, K> => {
31 |   const out = {} as Pick<T, K>;
32 |   for (const key of allowed) {
33 |     if (
34 |       key in payload &&
35 |       (payload as Record<PropertyKey, unknown>)[key as PropertyKey] !==
36 |         undefined
37 |     ) {
38 |       out[key] = payload[key];
39 |     }
40 |   }
41 |   return out;
42 | };
43 |

```

#### FILE: src/lib/db/resolveName.ts

TYPE: .ts | LINES: 1-56 | SHA256: b9bdd81aa185e5d17558b36504aa14839b7af7442d1c248a11fb69d4ee303fe6

```

1 | /**
2 |  * Standard UUID shape: 8-4-4-4-12 hexadecimal characters separated by hyphens.
3 |  *
4 |  * Anchored and case-insensitive so a *bare* UUID (and nothing else) is matched.
5 |  * A display string that merely *contains* a UUID (e.g. "Cohort 9f1c...") is left
6 |  * untouched, because that is still a human-readable label rather than a raw id.
7 |  */
8 | const UUID_RE =
9 |   /^([0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12})$/i;
10 |
11 | /**
12 |  * Resolve a display value for a name/title cell, guaranteeing a raw UUID is
13 |  * never surfaced to the user.
14 |  *
15 |  * `resolveName` is the shared fallback helper behind the Admin/Teacher list
16 |  * tables (Req 5): once a list query embeds the related row (e.g. a course or
17 |  * program name), the column renders `resolveName(row.relation?.name)`. When the
18 |  * relation could not be resolved – the value is `null`/`undefined`/empty, or an
19 |  * unresolved bare UUID leaked through – the function returns the `fallback`
20 |  * label instead of the raw identifier.
21 |  *
22 |  * The rule (no-raw-UUID correctness property, Req 5.7): return `value` only when
23 |  * it is a non-empty, non-UUID display string; otherwise return `fallback`.
24 |  *
25 |  * The function is pure: it has no side effects and does not mutate its inputs.
26 |  *
27 |  * @param value - The candidate display string (typically an embedded relation
28 |  *   name/title), which may be `null` or `undefined`.
29 |  * @param fallback - The label to show when `value` cannot be displayed as a
30 |  *   name. Defaults to an em dash ("—").
31 |  * @returns The trimmed display `value`, or `fallback` when `value` is missing,
32 |  *   blank, or a bare UUID.
33 |  *
34 |  * @example
35 |  * resolveName("Intro to Programming"); // "Intro to Programming"
36 |  * resolveName(null); // "—"

```



```

37 | * resolveName("");
38 | * resolveName("3f2504e0-4f89-41d3-9a0c-0305e82c3301"); // "-" (bare UUID)
39 | * resolveName(undefined, "Unassigned"); // "Unassigned"
40 | */
41 | export const resolveName = (
42 |   value: string | null | undefined,
43 |   fallback = "-"
44 | ): string => {
45 |   if (value === null || value === undefined) {
46 |     return fallback;
47 |   }
48 |
49 |   const trimmed = value.trim();
50 |   if (trimmed === "" || UUID_RE.test(trimmed)) {
51 |     return fallback;
52 |   }
53 |
54 |   return trimmed;
55 | };
56 |

```

FILE: src/lib/db/schemaContract.ts

TYPE: .ts | LINES: 1-170 | SHA256: 7b0ebc67cc57189dc686d0e11201343c31e1164f4b6f526851385356a6496e57

```

1 | /**
2 |  * Pure schema-contract validator for database mutation payloads (Part C, Req 18).
3 |  *
4 |  * Given a {@link MutationContractDescriptor} (the table + the keys a hook sends
5 |  * in its insert/upsert payload) and a {@link SchemaModel} (the table's real
6 |  * `Insert` columns and its `NOT NULL`-without-default required columns), this
7 |  * module returns every contract violation it can find:
8 |  *
9 |  * (a) `unknown-column` - a payload key that is not a real Insert
10 |  * column of the target table (Req 18.2).
11 |  * (b) `missing-required-column` - a Required_Column of the target table that
12 |  * is absent from the payload (Req 18.3).
13 |  *
14 |  * The validator is **pure** (no I/O, no mutation of its inputs) and
15 |  * **accumulating**: it never stops at the first problem, so a single run
16 |  * surfaces all violations together (Req 18.4). Every violation always names the
17 |  * offending key/column **and** the table (plus the hook for context), so a
18 |  * failure can never be reported without identifying its cause (Req 18.2/18.4).
19 |  *
20 |  * % % Design choice for the "unknown-column" (Real_Column) check % % % % % % % % % %
21 |  * The authoritative list of a table's real columns lives in the generated
22 |  * Supabase types (`src/types/database.ts`), but those types cannot be reflected
23 |  * into a runtime value set, and regenerating a DB-backed "all insertable
24 |  * columns" manifest requires Supabase access that is not available offline
25 |  * (and `database.ts` must never be hand-edited - see steering). Per the design,
26 |  * real-column-ness is therefore enforced at TWO complementary layers:
27 |  *
28 |  * • Compile time - the seeded descriptors spread the
29 |  *   `*_INSERT_COLUMNS` constants from `src/lib/db/insertColumns.ts`, each
30 |  *   declared as const satisfies readonly InsertKeys<T>[]`. `tsc` already
31 |  *   proves every such key is a real `Insert` column of its table.
32 |  * • Run time - this validator takes the table's known insert-column set as
33 |  *   `SchemaModel.insertColumns` and flags any payload key outside it. The
34 |  *   unit harness sources that set from the same checked-in
35 |  *   `*_INSERT_COLUMNS` constants, so the unknown-column check stays live for
36 |  *   hand-listed payloads while the **required-column** check (sourced from
37 |  *   the DB-generated `requiredColumns.json`) is the primary runtime gate.
38 |  *
39 |  * This keeps the validator fully general and property-testable (it makes no
40 |  * assumption about where its two column sets come from) while remaining
41 |  * verifiable without a live database.
42 |  */
43 |
44 | /** A single, fully-identified contract violation for one mutation payload. */
45 | export type ContractViolation =
46 |   | {
47 |     /** A payload key that is not a real Insert column of the table. */
48 |     readonly kind: "unknown-column";
49 |     /** The originating hook, for human-readable error messages. */
50 |     readonly hook: string;
51 |     /** The target table the payload is written to. */
52 |     readonly table: string;
53 |     /** The offending payload key. */
54 |     readonly key: string;
55 |   }
56 |   | {
57 |     /** A required (NOT NULL, no default) column missing from the payload. */
58 |     readonly kind: "missing-required-column";
59 |     /** The originating hook, for human-readable error messages. */
60 |     readonly hook: string;
61 |     /** The target table the payload is written to. */
62 |     readonly table: string;
63 |     /** The required column absent from the payload. */
64 |     readonly column: string;
65 |   };
66 |
67 | /**
68 |  * The minimal description of a mutation payload the validator checks: which
69 |  * hook sends it, which table it targets, the payload keys, and the operation.
70 |  *
71 |  * This mirrors the registry entries in
72 |  * `src/_tests_/fixtures/mutationDescriptors.ts` but is intentionally declared
73 |  * with plain `string` fields so the validator carries no dependency on the
74 |  * generated `Database` types and stays trivially unit/property testable.
75 |  */
76 | export interface MutationContractDescriptor {
77 |   /** Hook name, e.g. "useCreateTeam" - used only in error messages. */
78 |   readonly hook: string;
79 |   /** Target table name, e.g. "teams". */
80 |   readonly table: string;

```

[illegible]

```

11 | */
12 | export function computeCalibratedDifficulty(
13 |   originalDifficulty: number,
14 |   successRate: number,
15 |   totalAttempts: number
16 | ): number {
17 |   const calibrated = 5.0 - 4.0 * successRate;
18 |   const empiricalWeight = Math.min(1.0, totalAttempts / 50);
19 |   return (
20 |     empiricalWeight * calibrated + (1 - empiricalWeight) * originalDifficulty
21 |   );
22 | }
23 |
24 | /**
25 |  * Computes discrimination index using top/bottom 27% method.
26 |  * Returns a value between " 1.0 and 1.0.
27 |  */
28 | export function computeDiscriminationIndex(
29 |   topGroupSuccessRate: number,
30 |   bottomGroupSuccessRate: number
31 | ): number {
32 |   return topGroupSuccessRate - bottomGroupSuccessRate;
33 | }
34 |
35 | /**
36 |  * Determines quality flag based on analytics thresholds.
37 |  * Returns null when fewer than 20 attempts have been recorded.
38 |  */
39 | export function determineQualityFlag(
40 |   successRate: number,
41 |   discriminationIndex: number,
42 |   totalAttempts: number
43 | ): string | null {
44 |   if (totalAttempts < 20) return null;
45 |   if (discriminationIndex < 0.2) return "low_discrimination";
46 |   if (successRate > 0.95) return "too_easy";
47 |   if (successRate < 0.1) return "too_hard";
48 |   return "good";
49 | }
50 |

```

#### FILE: src/lib/directionManager.ts

TYPE: .ts | LINES: 1-28 | SHA256: ccbbd63763fa6a00f9261bb74a848995085f9ac9b8df6dd7cc3dbbeab26ee2ff

```

1 | const RTL_LANGUAGES = ["ar", "he", "fa", "ur"];
2 |
3 | /**
4 |  * Returns the text direction for a given language code.
5 |  */
6 | export const getDirection = (language: string): "rtl" | "ltr" => {
7 |   return RTL_LANGUAGES.includes(language) ? "rtl" : "ltr";
8 | };
9 |
10 | /**
11 |  * Applies the correct dir, lang, and font-family to the <html> element
12 |  * based on the active language. Idempotent – safe to call multiple times.
13 |  */
14 | export const applyDirection = (language: string): void => {
15 |   const dir = getDirection(language);
16 |   const htmlEl = document.documentElement;
17 |   htmlEl.setAttribute("dir", dir);
18 |   htmlEl.setAttribute("lang", language);
19 |
20 |   if (dir === "rtl") {
21 |     htmlEl.style.fontFamily =
22 |       "Noto Sans Arabic", "Noto Sans", ui-sans-serif, system-ui, sans-serif';
23 |   } else {
24 |     htmlEl.style.fontFamily =
25 |       "Noto Sans", ui-sans-serif, system-ui, sans-serif';
26 |   }
27 | };
28 |

```

#### FILE: src/lib/displayName.ts

TYPE: .ts | LINES: 1-47 | SHA256: 75ab304f4415847754741222f51a8352f3f0941429fe3f66579b02a18f22d48d

```

1 | /**
2 |  * Honorific-aware display-name derivation.
3 |  *
4 |  * Seeded/imported names frequently carry an honorific title
5 |  * ("Mr. David Okonkwo", "Dr. Aisha Al-Mansoori"). A naive `name.split(" ")[0]`
6 |  * therefore renders the title ("Mr."/ "Dr.") instead of the person's first name.
7 |  * This single shared helper skips a leading honorific token so greetings and
8 |  * profile headers show the actual first name.
9 |  *
10 |  * Kept free of React/Supabase so it is trivially unit-testable and reusable by
11 |  * every surface that needs a first name (no duplicated inline logic).
12 |  *
13 |  * Production Bug Fixes – Track A, Item 3 (Requirement 3).
14 |  */
15 |
16 | /** Lowercased honorific tokens (trailing "." stripped before lookup). */
17 | const HONORIFICS = new Set([
18 |   "mr",
19 |   "mrs",
20 |   "ms",
21 |   "miss",
22 |   "dr",
23 |   "prof",
24 |   "mx",
25 |   "sir",
26 | ]);
27 |
28 | /**

```

```

29 | * Derive a display first name from a full name.
30 | *
31 | * - Trims, splits on whitespace, and drops empty tokens.
32 | * - When the first token is an honorific (case-insensitive, optional trailing
33 | *   ".") AND a following token exists, the following token is returned.
34 | * - Otherwise the first token is returned (a lone honorific is returned as-is).
35 | *
36 | * @param name - The full name, possibly null/undefined.
37 | * @returns The display first name, or `null` when no usable token exists so the
38 | *   caller can supply its own fallback.
39 | */
40 | export const getDisplayFirstName = (name?: string | null): string | null => {
41 |   const parts = (name ?? "").trim().split(/\s+/).filter(Boolean);
42 |   if (parts.length === 0) return null;
43 |   const head = parts[0]!.replace(/\.$/, "").toLowerCase();
44 |   if (HONORIFICS.has(head) && parts.length > 1) return parts[1]!;
45 |   return parts[0]!;
46 | };
47 |

```

## FILE: src/lib/draftManager.ts

TYPE: .ts | LINES: 1-47 | SHA256: 988eae7e37e9214af8696b46eb141a7062594a5db226c52a68192b86fdf8e89c

```

1 | // Task 54: Draft Manager – auto-save drafts to localStorage
2 | const PREFIX = "edviser_draft_";
3 |
4 | export const draftManager = {
5 |   saveDraft(key: string, content: unknown): void {
6 |     try {
7 |       localStorage.setItem(
8 |         `${PREFIX}${key}`,
9 |         JSON.stringify({ content, savedAt: Date.now() })
10 |      );
11 |     } catch {
12 |       /* quota exceeded – silently fail */
13 |     }
14 |   },
15 |
16 |   loadDraft<T = unknown>(key: string): T | null {
17 |     try {
18 |       const raw = localStorage.getItem(`${PREFIX}${key}`);
19 |       if (!raw) return null;
20 |       const parsed = JSON.parse(raw);
21 |       return parsed.content as T;
22 |     } catch {
23 |       return null;
24 |     }
25 |   },
26 |
27 |   clearDraft(key: string): void {
28 |     try {
29 |       localStorage.removeItem(`${PREFIX}${key}`);
30 |     } catch (err) {
31 |       console.error("[DraftManager] clearDraft failed:", err);
32 |     }
33 |   },
34 |
35 |   startAutoSave(
36 |     key: string,
37 |     getContent: () => unknown,
38 |     intervalMs = 30_000
39 |   ): () => void {
40 |     const timer = setInterval(() => {
41 |       const content = getContent();
42 |       if (content != null) draftManager.saveDraft(key, content);
43 |     }, intervalMs);
44 |     return () => clearInterval(timer);
45 |   },
46 | };
47 |

```

## FILE: src/lib/dynamicPricingCalculator.ts

TYPE: .ts | LINES: 1-111 | SHA256: 5e0c08b0e5f30a577fa66c59ed0ff1369f3678e74209767378938eee400d3a42

```

1 | // =====
2 | // Dynamic Pricing Calculator – Pure function
3 | // Computes dynamic price from demand score and base price with bounds (50%-150%)
4 | // =====
5 |
6 | /**
7 |  * Demand level classification based on purchase frequency percentiles.
8 |  */
9 | export type DemandLevel = "low" | "normal" | "high";
10 |
11 | export interface DynamicPricingInput {
12 |   /** Base XP price of the item */
13 |   basePrice: number;
14 |   /** Number of purchases in the measurement window (e.g., last 30 days) */
15 |   purchaseCount: number;
16 |   /** 25th percentile of purchase counts across all items */
17 |   p25: number;
18 |   /** 75th percentile of purchase counts across all items */
19 |   p75: number;
20 | }
21 |
22 | export interface DynamicPricingResult {
23 |   /** The dynamically adjusted price (integer, "e 1") */
24 |   dynamicPrice: number;
25 |   /** The demand level classification */
26 |   demandLevel: DemandLevel;
27 |   /** Multiplier applied to the base price (0.5-1.5) */
28 |   multiplier: number;
29 |   /** Whether the price differs from the base price */
30 |   isAdjusted: boolean;

```

```

31 | }
32 |
33 | /**
34 |  * Lower bound multiplier – item price cannot drop below 50% of base.
35 |  */
36 | const MIN_MULTIPLIER = 0.5;
37 |
38 | /**
39 |  * Upper bound multiplier – item price cannot exceed 150% of base.
40 |  */
41 | const MAX_MULTIPLIER = 1.5;
42 |
43 | /**
44 |  * Classify demand level based on purchase count relative to percentiles.
45 |  */
46 | export function classifyDemand(
47 |   purchaseCount: number,
48 |   p25: number,
49 |   p75: number
50 | ): DemandLevel {
51 |   if (purchaseCount > p75) return "high";
52 |   if (purchaseCount < p25) return "low";
53 |   return "normal";
54 | }
55 |
56 | /**
57 |  * Compute the dynamic price for a marketplace item based on demand.
58 |  *
59 |  * - High demand (above p75): price increases up to 150% of base
60 |  * - Low demand (below p25): price decreases to 50% of base
61 |  * - Normal demand: price stays at base (no override)
62 |  *
63 |  * The result is always an integer "e 1 and within [50%, 150%] of base price.
64 |  */
65 | export function computeDynamicPrice(
66 |   input: DynamicPricingInput
67 | ): DynamicPricingResult {
68 |   const { basePrice, purchaseCount, p25, p75 } = input;
69 |
70 |   // Guard: base price must be positive
71 |   if (basePrice <= 0) {
72 |     return {
73 |       dynamicPrice: 1,
74 |       demandLevel: "normal",
75 |       multiplier: 1,
76 |       isAdjusted: false,
77 |     };
78 |   }
79 |
80 |   const demandLevel = classifyDemand(purchaseCount, p25, p75);
81 |
82 |   let multiplier = 1;
83 |
84 |   if (demandLevel === "high") {
85 |     // Scale linearly from 1.0 at p75 to MAX_MULTIPLIER at 2xp75
86 |     const range = Math.max(p75, 1); // avoid division by zero
87 |     const excess = purchaseCount - p75;
88 |     const scale = Math.min(excess / range, 1); // cap at 1
89 |     multiplier = 1 + scale * (MAX_MULTIPLIER - 1);
90 |   } else if (demandLevel === "low") {
91 |     // Scale linearly from 1.0 at p25 down to MIN_MULTIPLIER at 0
92 |     const range = Math.max(p25, 1); // avoid division by zero
93 |     const deficit = p25 - purchaseCount;
94 |     const scale = Math.min(deficit / range, 1); // cap at 1
95 |     multiplier = 1 - scale * (1 - MIN_MULTIPLIER);
96 |   }
97 |
98 |   // Clamp multiplier to bounds
99 |   multiplier = Math.max(MIN_MULTIPLIER, Math.min(MAX_MULTIPLIER, multiplier));
100 |
101 |   // Compute final price, ensure integer "e 1
102 |   const dynamicPrice = Math.max(1, Math.floor(basePrice * multiplier));
103 |
104 |   return {
105 |     dynamicPrice,
106 |     demandLevel,
107 |     multiplier,
108 |     isAdjusted: demandLevel !== "normal",
109 |   };
110 | }
111 |

```

FILE: [src/lib/earnSpendRatioCalculator.ts](#)

TYPE: .ts | LINES: 1-102 | SHA256: d44160efa04befdd1dabcb84951aedd6c5641aa93dace870313ab5e56b00c88f

```

1 | // =====
2 | // Earn/Spend Ratio Calculator – Pure function
3 | // Computes earn/spend ratio and inflation status for the XP economy
4 | // =====
5 |
6 | /**
7 |  * Inflation status of the XP economy.
8 |  * - healthy: ratio between 2 and 4 (inclusive) – balanced economy
9 |  * - inflationary: ratio > 4 – too much XP in circulation, not enough spending
10 |  * - deflationary: ratio < 2 – too much spending relative to earning
11 |  * - no_spending: no XP has been spent yet (ratio is undefined)
12 |  */
13 | export type InflationStatus =
14 |   | "healthy"
15 |   | "inflationary"
16 |   | "deflationary"
17 |   | "no_spending";
18 |
19 | export interface EarnSpendInput {

```

```

20 | /** Total XP earned across all students (non-negative) */
21 | totalEarned: number;
22 | /** Total XP spent across all students (non-negative) */
23 | totalSpent: number;
24 | }
25 |
26 | export interface EarnSpendResult {
27 | /** The earn/spend ratio, or null if no spending */
28 | ratio: number | null;
29 | /** Inflation status classification */
30 | status: InflationStatus;
31 | /** Total earned XP */
32 | totalEarned: number;
33 | /** Total spent XP */
34 | totalSpent: number;
35 | /** Human-readable label for the status */
36 | statusLabel: string;
37 | }
38 |
39 | /**
40 |  * Healthy ratio lower bound (inclusive).
41 |  */
42 | const HEALTHY_LOWER = 2;
43 |
44 | /**
45 |  * Healthy ratio upper bound (inclusive).
46 |  */
47 | const HEALTHY_UPPER = 4;
48 |
49 | /**
50 |  * Classify the inflation status based on the earn/spend ratio.
51 |  */
52 | export function classifyInflation(ratio: number | null): InflationStatus {
53 |   if (ratio === null) return "no_spending";
54 |   if (ratio >= HEALTHY_LOWER && ratio <= HEALTHY_UPPER) return "healthy";
55 |   if (ratio > HEALTHY_UPPER) return "inflationary";
56 |   return "deflationary";
57 | }
58 |
59 | /**
60 |  * Get a human-readable label for an inflation status.
61 |  */
62 | export function getStatusLabel(status: InflationStatus): string {
63 |   switch (status) {
64 |     case "healthy":
65 |       return "Healthy";
66 |     case "inflationary":
67 |       return "Inflationary";
68 |     case "deflationary":
69 |       return "Deflationary";
70 |     case "no_spending":
71 |       return "No spending yet";
72 |   }
73 | }
74 |
75 | /**
76 |  * Compute the earn/spend ratio and inflation status for an institution.
77 |  *
78 |  * - Ratio = totalEarned / totalSpent (rounded to 2 decimal places)
79 |  * - If totalSpent is 0, ratio is null and status is 'no_spending'
80 |  * - Healthy range: 2:1 to 4:1
81 |  * - Inflationary: ratio > 4 (too much XP, not enough sinks)
82 |  * - Deflationary: ratio < 2 (too much spending)
83 |  */
84 | export function computeEarnSpendRatio(input: EarnSpendInput): EarnSpendResult {
85 |   const totalEarned = Math.max(0, input.totalEarned);
86 |   const totalSpent = Math.max(0, input.totalSpent);
87 |
88 |   const ratio =
89 |     totalSpent > 0 ? Math.round((totalEarned / totalSpent) * 100) / 100 : null;
90 |
91 |   const status = classifyInflation(ratio);
92 |   const statusLabel = getStatusLabel(status);
93 |
94 |   return {
95 |     ratio,
96 |     status,
97 |     totalEarned,
98 |     totalSpent,
99 |     statusLabel,
100 |   };
101 | }
102 |

```

## FILE: src/lib/envValidation.ts

TYPE: .ts | LINES: 1-39 | SHA256: 7419c267e05c8d4378037f38b5ae0369467b147d1d4b6bd7b5079cfc7832c01e

```

1 | import { z } from "zod";
2 |
3 | /**
4 |  * Validates required VITE_ environment variables at app startup.
5 |  * Throws a descriptive error if any required variable is missing.
6 |  */
7 | const envSchema = z.object({
8 |   VITE_SUPABASE_URL: z
9 |     .string()
10 |     .url(
11 |       "VITE_SUPABASE_URL must be a valid URL (e.g. https://your-project.supabase.co)"
12 |     ),
13 |   VITE_SUPABASE_ANON_KEY: z
14 |     .string()
15 |     .min(1, "VITE_SUPABASE_ANON_KEY is required"),
16 |   VITE_SENTRY_DSN: z.string().optional(),
17 | });

```

```

18 |
19 | export type AppEnv = z.infer<typeof envSchema>;
20 |
21 | export function validateEnv(): AppEnv | null {
22 |   const result = envSchema.safeParse({
23 |     VITE_SUPABASE_URL: import.meta.env.VITE_SUPABASE_URL,
24 |     VITE_SUPABASE_ANON_KEY: import.meta.env.VITE_SUPABASE_ANON_KEY,
25 |     VITE_SENTRY_DSN: import.meta.env.VITE_SENTRY_DSN,
26 |   });
27 |
28 |   if (!result.success) {
29 |     const missing = result.error.issues
30 |       .map((i) => `  • ${i.path.join(".")}: ${i.message}`)
31 |       .join("\n");
32 |     const msg = `[Edeviser] Missing or invalid environment variables:\n${missing}\n\nCopy .env.example to .env
33 |     console.error(msg);
34 |     return null;
35 |   }
36 |
37 |   return result.data;
38 | }
39 |

```

#### FILE: [src/lib/explanationConfidence.ts](#)

TYPE: .ts | LINES: 1-32 | SHA256: 7da9759d0d154164121f5c07ad8dd666b43404101735d087e4cea05f4e187671

```

1 | /**
2 |  * Computes explanation confidence as the average similarity of top 3 chunks.
3 |  * Returns a value between 0.0 and 1.0.
4 |  * For an empty array, returns 0.
5 |  */
6 | export function computeExplanationConfidence(
7 |   chunkSimilarities: number[]
8 | ): number {
9 |   if (chunkSimilarities.length === 0) return 0;
10 |   const topChunks = [...chunkSimilarities].sort((a, b) => b - a).slice(0, 3);
11 |   return topChunks.reduce((sum, s) => sum + s, 0) / topChunks.length;
12 | }
13 |
14 | /**
15 |  * Determines if an explanation needs teacher verification.
16 |  * Returns true when confidence is below 0.8.
17 |  */
18 | export function needsTeacherVerification(confidence: number): boolean {
19 |   return confidence < 0.8;
20 | }
21 |
22 | /**
23 |  * Determines if a question is frequently missed.
24 |  * Returns true only when success_rate < 0.5 AND total_attempts >= 10.
25 |  */
26 | export function isFrequentlyMissed(
27 |   successRate: number,
28 |   totalAttempts: number
29 | ): boolean {
30 |   return successRate < 0.5 && totalAttempts >= 10;
31 | }
32 |

```

#### FILE: [src/lib/exportCurriculumMatrixCsv.ts](#)

TYPE: .ts | LINES: 1-51 | SHA256: 824a2830b255ee7ca94c7bcb132b1bae0958942be1d71112c652bd0beee2d51b

```

1 | import type { CurriculumMatrixData } from "@hooks/useCurriculumMatrix";
2 |
3 | /**
4 |  * Escapes a CSV field value, wrapping in double-quotes when the value
5 |  * contains commas, double-quotes, or newlines (RFC 4180).
6 |  */
7 | export function escapeCsvField(field: string): string {
8 |   if (field.includes(",") || field.includes('"') || field.includes("\n")) {
9 |     return `"${field.replace(/"/g, '""')}"`;
10 |   }
11 |   return field;
12 | }
13 |
14 | /**
15 |  * Builds a CSV string from curriculum matrix data.
16 |  * - First column: PLO title
17 |  * - Subsequent columns: course codes
18 |  * - Cell values: CLO count (number of CLOs mapped)
19 |  */
20 |
21 | export function buildMatrixCsv(matrixData: CurriculumMatrixData): string {
22 |   const { plos, courses, matrix } = matrixData;
23 |
24 |   const header = ["PLO", ...courses.map((c) => escapeCsvField(c.code))].join(
25 |     ","
26 |   );
27 |
28 |   const rows = plos.map((plo) => {
29 |     const cells = courses.map((course) => {
30 |       const cell = matrix[plo.id]?.[course.id];
31 |       return cell ? String(cell.cloCount) : "0";
32 |     });
33 |     return [escapeCsvField(plo.title), ...cells].join(",");
34 |   });
35 |
36 |   return [header, ...rows].join("\n");
37 | }
38 |
39 | /**
40 |  * Triggers a browser download of the given CSV content.
41 |  */
42 | export function downloadCsv(csvContent: string, filename: string): void {

```

[illegible]



[illegible]

```

132 export function validateAvatarFile(file: File): void {
133   if (file.size > AVATAR_MAX_SIZE_BYTES) {
134     throw new FileValidationError(
135       `File size exceeds the 2MB limit. Your file is ${((
136         file.size /
137         (1024 * 1024))
138       ).toFixed(1)}MB.`
139     );
140   }
141 }
142 
143 if (!AVATAR_ALLOWED_TYPES.includes(file.type)) {
144   throw new FileValidationError(
145     "Only image files are allowed (JPG, PNG, GIF, WebP)."
```

[illegible]

```

340 |     assertNoPathTraversal(path);
341 |
342 |     const { error: uploadError } = await supabase.storage
343 |       .from(ANNOUNCEMENT_ATTACHMENT_BUCKET)
344 |       .upload(path, file, { upsert: false });
345 |
346 |     if (uploadError) {
347 |       throw new Error(`Attachment upload failed: ${uploadError.message}`);
348 |     }
349 |
350 |     return {
351 |       storage_path: path,
352 |       file_name: file.name,
353 |       content_type: file.type,
354 |       size_bytes: file.size,
355 |     };
356 |   }
357 | }

```

#### FILE: src/lib/fontPreferences.ts

TYPE: .ts | LINES: 1-35 | SHA256: 24ad56aed580731d9027ee4304b45efd26be32b9616d46212ead5c5d5bc318e0

```

1 | const OPENDYSLEXIC_URL =
2 |   "https://cdn.jsdelivr.net/npm/open-dyslexic@1.0.3/open-dyslexic-regular.woff2";
3 |
4 | let fontLoaded = false;
5 |
6 | export const loadDyslexiaFont = async (): Promise<void> => {
7 |   if (fontLoaded) return;
8 |   try {
9 |     const font = new FontFace("OpenDyslexic", `url(${OPENDYSLEXIC_URL})`);
10 |    const loaded = await font.load();
11 |    document.fonts.add(loaded);
12 |    fontLoaded = true;
13 |   } catch (error) {
14 |     console.error("Failed to load OpenDyslexic font:", error);
15 |   }
16 | };
17 |
18 | export const applyDyslexiaFont = (enabled: boolean): void => {
19 |   const root = document.documentElement;
20 |   if (enabled) {
21 |     loadDyslexiaFont();
22 |     root.style.setProperty(
23 |       "--font-body",
24 |       "OpenDyslexic", "Noto Sans", ui-sans-serif, system-ui, sans-serif'
25 |     );
26 |   } else {
27 |     root.style.removeProperty("--font-body");
28 |   }
29 | };
30 |
31 | /** Reset internal state – used for testing */
32 | export const _resetFontLoaded = (): void => {
33 |   fontLoaded = false;
34 | };
35 |

```

#### FILE: src/lib/formatDate.ts

TYPE: .ts | LINES: 1-30 | SHA256: 097b9526a696a78afb1ef498054b29f915cc7638b9b5d71b129152fec31b54b

```

1 | import { format, formatDistanceToNow, type Locale } from "date-fns";
2 | import { enUS } from "date-fns/locale/en-US";
3 | import { ar } from "date-fns/locale/ar";
4 | import i18n from "@lib/i18n";
5 |
6 | const localeMap: Record<string, Locale> = { en: enUS, ar };
7 |
8 | const getLocale = (): Locale => localeMap[i18n.language] || enUS;
9 |
10 | /**
11 |  * Formats a date using the active i18n locale.
12 |  * @param date - Date object or ISO string
13 |  * @param pattern - date-fns format pattern (default: 'PPP')
14 |  */
15 | export const formatLocalDate = (
16 |   date: Date | string,
17 |   pattern: string = "PPP"
18 | ): string => {
19 |   const d = typeof date === "string" ? new Date(date) : date;
20 |   return format(d, pattern, { locale: getLocale() });
21 | };
22 |
23 | /**
24 |  * Formats a date as a relative time string (e.g. "3 hours ago").
25 |  */
26 | export const formatRelativeTime = (date: Date | string): string => {
27 |   const d = typeof date === "string" ? new Date(date) : date;
28 |   return formatDistanceToNow(d, { addSuffix: true, locale: getLocale() });
29 | };
30 |

```

#### FILE: src/lib/formatNumber.ts

TYPE: .ts | LINES: 1-33 | SHA256: d48606b11071e7dc4c1377450e766b9243ad8e42c0727af57918da4f4ce6499f

```

1 | import i18n from "@lib/i18n";
2 |
3 | const getLocale = (): string => (i18n.language === "ar" ? "ar-QA" : "en-US");
4 |
5 | /**
6 |  * Formats a number using the active i18n locale.
7 |  */
8 | export const formatNumber = (value: number): string => {

```

```

9 |     return new Intl.NumberFormat(getLocale()).format(value);
10 | };
11 |
12 | /**
13 |  * Formats a number as a percentage using the active i18n locale.
14 |  * @param value - The percentage value (e.g. 85 for 85%)
15 |  * @param decimals - Number of decimal places (default: 0)
16 |  */
17 | export const formatPercent = (value: number, decimals: number = 0): string => {
18 |     return new Intl.NumberFormat(getLocale(), {
19 |         style: "percent",
20 |         minimumFractionDigits: decimals,
21 |         maximumFractionDigits: decimals,
22 |     }).format(value / 100);
23 | };
24 |
25 | /**
26 |  * Formats a number in compact notation (e.g. 1.2K, 3.4M).
27 |  */
28 | export const formatCompact = (value: number): string => {
29 |     return new Intl.NumberFormat(getLocale(), { notation: "compact" }).format(
30 |         value
31 |     );
32 | };
33 |

```

## FILE: src/lib/gapAnalysis.ts

TYPE: .ts | LINES: 1-78 | SHA256: 81b4662e240c2c512ceceeb14e91ccd5812cd65d808c96228a55316bf34dcc79

```

1 | // Task 117.2: Gap Analysis classification utility
2 |
3 | export type GapStatus =
4 |     | "fully_mapped"
5 |     | "partially_mapped"
6 |     | "unmapped"
7 |     | "no_evidence";
8 | export type GapFlag = "under_mapped" | "unassessed" | null;
9 |
10 | export interface GapResult {
11 |     outcome_id: string;
12 |     outcome_title: string;
13 |     outcome_type: "ILO" | "PLO" | "CLO";
14 |     status: GapStatus;
15 |     flag: GapFlag;
16 |     mapped_children: number;
17 |     evidence_count: number;
18 |     recommendation: string | null;
19 | }
20 |
21 | interface OutcomeWithMappings {
22 |     id: string;
23 |     title: string;
24 |     type: "ILO" | "PLO" | "CLO";
25 |     mapped_children_count: number;
26 |     evidence_count: number;
27 |     has_assessments: boolean;
28 | }
29 |
30 | export const classifyGapStatus = (outcome: OutcomeWithMappings): GapStatus => {
31 |     if (outcome.mapped_children_count === 0 && outcome.type !== "CLO")
32 |         return "unmapped";
33 |     if (outcome.evidence_count === 0) return "no_evidence";
34 |     if (outcome.type === "PLO" && outcome.mapped_children_count < 2)
35 |         return "partially_mapped";
36 |     return "fully_mapped";
37 | };
38 |
39 | export const classifyGapFlag = (outcome: OutcomeWithMappings): GapFlag => {
40 |     if (outcome.type === "PLO" && outcome.mapped_children_count < 2)
41 |         return "under_mapped";
42 |     if (outcome.type === "CLO" && !outcome.has_assessments) return "unassessed";
43 |     return null;
44 | };
45 |
46 | export const generateRecommendation = (
47 |     status: GapStatus,
48 |     flag: GapFlag,
49 |     type: string
50 | ): string | null => {
51 |     if (status === "unmapped")
52 |         return `Map this ${type} to child outcomes to establish assessment coverage.`;
53 |     if (flag === "under_mapped")
54 |         return "Add more CLO mappings to ensure adequate coverage (minimum 2 recommended).";
55 |     if (flag === "unassessed")
56 |         return "Link at least one assessment to this CLO in the current semester.";
57 |     if (status === "no_evidence")
58 |         return "No student evidence exists yet. Ensure assessments are graded.";
59 |     return null;
60 | };
61 |
62 | export const analyzeGaps = (outcomes: OutcomeWithMappings[]): GapResult[] => {
63 |     return outcomes.map(o => {
64 |         const status = classifyGapStatus(o);
65 |         const flag = classifyGapFlag(o);
66 |         return {
67 |             outcome_id: o.id,
68 |             outcome_title: o.title,
69 |             outcome_type: o.type,
70 |             status,
71 |             flag,
72 |             mapped_children: o.mapped_children_count,
73 |             evidence_count: o.evidence_count,
74 |             recommendation: generateRecommendation(status, flag, o.type),
75 |         };

```

TYPE: .ts | LINES: 1-43 | SHA256: 364ea9924b50ebce43a1eb8af34e1e06fae677145166833e81ad87151356073b

```

2 import { GOAL_DIFFICULTY_THRESHOLDS } from "./onboardingConstants";
3 export type GoalDifficulty = "easy" | "moderate" | "ambitious";
4
5 export function classifyDifficulty(
6   cohortCompletionRate: number
7 ): GoalDifficulty {
8   if (cohortCompletionRate >= GOAL_DIFFICULTY_THRESHOLDS.easy) return "easy";
9   if (cohortCompletionRate >= GOAL_DIFFICULTY_THRESHOLDS.moderate)
10     return "moderate";
11   return "ambitious";
12 }
13
14 export interface SmartGoalFields {
15   specific: string;
16   measurable: string;
17   achievable: string;
18   relevant: string;
19   timebound: string; // ISO date
20 }
21
22 export function composeGoalText(fields: SmartGoalFields): string {
23   return `${fields.specific} by ${fields.timebound}, measuring progress through ${fields.measurable}. This is
24 }
25
26 export function createGoalTemplate(params: {
27   courseName: string;
28   cloTitle?: string;
29   deadline: string;
30   suggestedAction: string;
31   measurableTarget: string;
32 }): SmartGoalFields {
33   return {
34     specific: params.suggestedAction,
35     measurable: params.measurableTarget,
36     achievable: "Based on your current progress and available study time",
37     relevant: params.cloTitle
38     ? `${params.courseName} - ${params.cloTitle}`
39     : params.courseName,
40     timebound: params.deadline,
41   };
42 }
43

```

TYPE: .ts | LINES: 1-162 | SHA256: 3f50fbde3abb09d0b986b0d5a82c4f5b5c1350d3630a1254a6b27eaff1f1d004

[illegible]

```

51 |
52 | const mean = (values: number[]): number =>
53 |   values.length > 0 ? values.reduce((sum, v) => sum + v, 0) / values.length : 0;
54 |
55 | /**
56 |  * Computes class averages from the displayed gradebook matrix.
57 |  *
58 |  * - Per assessment: mean of the non-null scores across students (null when no
59 |  *   student has a score for that assessment so the cell stays empty).
60 |  * - Per category: mean of the displayed subtotal percentages across students.
61 |  * - Final: mean of the displayed final weighted grades across students.
62 |  *
63 |  * The first entry defines the (uniform) column structure used for lookups.
64 |  */
65 | export function computeClassAverages(data: GradebookCsvEntry[]): ClassAverages {
66 |   const assessmentAvg = new Map<string, number | null>();
67 |   const categoryAvg = new Map<string, number>();
68 |
69 |   const template = data[0];
70 |   if (!template) {
71 |     return { assessmentAvg, categoryAvg, finalAvg: 0 };
72 |   }
73 |
74 |   for (const cat of template.categories) {
75 |     for (const assessment of cat.assessments) {
76 |       const scores: number[] = [];
77 |       for (const student of data) {
78 |         const score = student.categories
79 |           .find((c) => c.category_id === cat.category_id)
80 |           ?.assessments.find((a) => a.id === assessment.id)?.score;
81 |         if (score !== null && score !== undefined) {
82 |           scores.push(score);
83 |         }
84 |       }
85 |       assessmentAvg.set(
86 |         assessment.id,
87 |         scores.length > 0 ? round2(mean(scores)) : null
88 |       );
89 |     }
90 |
91 |     const subtotals = data.map(
92 |       (student) =>
93 |         student.categories.find((c) => c.category_id === cat.category_id)
94 |         ?.subtotal_percent ?? 0
95 |     );
96 |     categoryAvg.set(cat.category_id, round2(mean(subtotals)));
97 |   }
98 |
99 |   const finalAvg = round2(
100 |     mean(data.map((student) => student.final_weighted_grade))
101 |   );
102 |
103 |   return { assessmentAvg, categoryAvg, finalAvg };
104 | }
105 |
106 | /**
107 |  * Builds the gradebook CSV from the displayed matrix (Req 13.2). The trailing
108 |  * row is the class average (Req 13.3) so the export mirrors exactly what the
109 |  * teacher sees on screen. Pair with 'downloadCsv' to trigger the file download.
110 |  */
111 | export function buildGradebookCsv(
112 |   data: GradebookCsvEntry[],
113 |   classAverages: ClassAverages,
114 |   classAverageLetter: string
115 | ): string {
116 |   const template = data[0];
117 |   if (!template) return "";
118 |
119 |   // Header: Student, [<cat> - <assessment> (/max)..., <cat> Subtotal %]*, Final %, Grade
120 |   const header: string[] = ["Student"];
121 |   for (const cat of template.categories) {
122 |     for (const assessment of cat.assessments) {
123 |       header.push(
124 |         `${cat.category_name} - ${assessment.title} (/ ${assessment.max_score})`
125 |       );
126 |     }
127 |     header.push(`${cat.category_name} Subtotal %`);
128 |   }
129 |   header.push("Final %", "Grade");
130 |
131 |   const lines: string[] = [header.map(escapeCsvField).join(",")];
132 |
133 |   // One row per student
134 |   for (const student of data) {
135 |     const cells: string[] = [escapeCsvField(student.student_name)];
136 |     for (const cat of student.categories) {
137 |       for (const assessment of cat.assessments) {
138 |         cells.push(assessment.score !== null ? String(assessment.score) : "");
139 |       }
140 |       cells.push(cat.subtotal_percent.toFixed(1));
141 |     }
142 |     cells.push(student.final_weighted_grade.toFixed(1), student.letter_grade);
143 |     lines.push(cells.join(","));
144 |   }
145 |
146 |   // Trailing class-average row (Req 13.3)
147 |   const avgCells: string[] = [escapeCsvField("Class Average")];
148 |   for (const cat of template.categories) {
149 |     for (const assessment of cat.assessments) {
150 |       const avg = classAverages.assessmentAvg.get(assessment.id);
151 |       avgCells.push(avg !== null && avg !== undefined ? String(avg) : "");
152 |     }
153 |     avgCells.push(
154 |       (classAverages.categoryAvg.get(cat.category_id) ?? 0).toFixed(1)

```

```

155 |     });
156 |   }
157 |   avgCells.push(classAverages.finalAvg.toFixed(1), classAverageLetter);
158 |   lines.push(avgCells.join(", "));
159 |
160 |   return lines.join("\n");
161 | }
162 |

```

#### FILE: src/lib/habitDifficulty.ts

TYPE: .ts | LINES: 1-35 | SHA256: 636a50238e184c5d50d0b25ad4942080391b920c6d84390af7a61d2bdb40a8ab

```

1 | // Task 145.3: Habit Difficulty Level utility
2 |
3 | /**
4 |  * Returns the number of habits required for the given difficulty level.
5 |  * Level 1: 1 habit (login only)
6 |  * Level 2: 2 habits (login + one other)
7 |  * Level 3: 6 of 8 habits
8 |  */
9 | export function getRequiredHabitsForLevel(level: number): number {
10 |   if (level <= 1) return 1;
11 |   if (level === 2) return 2;
12 |   return 6;
13 | }
14 |
15 | /**
16 |  * Check if a student should be promoted to the next level.
17 |  * Promotion occurs when habit_level_streak >= 7 and level < 3.
18 |  */
19 | export function checkLevelPromotion(
20 |   level: number,
21 |   habitLevelStreak: number
22 | ): boolean {
23 |   return habitLevelStreak >= 7 && level < 3;
24 | }
25 |
26 | /**
27 |  * Returns the Perfect Day threshold for the given difficulty level.
28 |  * Level 1: 1 habit
29 |  * Level 2: 2 habits
30 |  * Level 3: 6 of 8 habits
31 |  */
32 | export function getPerfectDayThreshold(level: number): number {
33 |   return getRequiredHabitsForLevel(level);
34 | }
35 |

```

#### FILE: src/lib/habitExport.ts

TYPE: .ts | LINES: 1-66 | SHA256: e08d274d157305c4fb9c19731a70c1207876b46532ba223b689d7f721011922c

```

1 | import type {
2 |   HabitReportRow,
3 |   HeatmapSummary,
4 |   WellnessHabitType,
5 |   LevelProgressionPoint,
6 | } from "@types/habits";
7 | import { getLevelForDate } from "@lib/levelAwareHeatmap";
8 |
9 | export const generateHabitCSV = (
10 |   data: HabitReportRow[],
11 |   summary: HeatmapSummary & { totalXP: number; consistencyScore: number },
12 |   enabledWellnessHabits: WellnessHabitType[],
13 |   levelHistory?: LevelProgressionPoint[]
14 | ): string => {
15 |   const wellnessColumns = enabledWellnessHabits;
16 |   const includeLevelColumn = levelHistory && levelHistory.length > 0;
17 |   const headers = [
18 |     "date",
19 |     "login",
20 |     "submit",
21 |     "journal",
22 |     "read",
23 |     ...wellnessColumns,
24 |     "total_habits",
25 |     "xp_earned",
26 |     "streak_active",
27 |     ...(includeLevelColumn ? ["level"] : []),
28 |   ];
29 |
30 |   const rows = data.map((row) => {
31 |     const base = [
32 |       row.date,
33 |       row.login ? "yes" : "no",
34 |       row.submit ? "yes" : "no",
35 |       row.journal ? "yes" : "no",
36 |       row.read ? "yes" : "no",
37 |     ];
38 |     const wellness = wellnessColumns.map((h) => (row[h] ? "yes" : "no"));
39 |     const levelCol = includeLevelColumn
40 |       ? [String(getLevelForDate(row.date, levelHistory))]
41 |       : [];
42 |     return [
43 |       ...base,
44 |       ...wellness,
45 |       String(row.totalHabits),
46 |       String(row.xpEarned),
47 |       row.streakActive ? "yes" : "no",
48 |       ...levelCol,
49 |     ].join(",");
50 |   });
51 |
52 |   const summaryRow = `# Summary: total_active_days=${summary.totalActiveDays},consistency_score=${summary.consistencyScore}`;
53 |

```



```

54 |     return [summaryRow, headers.join(", "), ...rows].join("\n");
55 | };
56 |
57 | export const downloadCSV = (csv: string, filename: string): void => {
58 |     const blob = new Blob([csv], { type: "text/csv;charset=utf-8;" });
59 |     const url = URL.createObjectURL(blob);
60 |     const link = document.createElement("a");
61 |     link.href = url;
62 |     link.download = filename;
63 |     link.click();
64 |     URL.revokeObjectURL(url);
65 | };
66 |

```

## FILE: src/lib/heatmapUtils.ts

TYPE: .ts | LINES: 1-283 | SHA256: 114ef123dc7bbf3e4ec169a92fba96ccf32044c89086704e8cdfa27bfee7919b

```

1 | import type {
2 |     HeatmapDay,
3 |     HabitType,
4 |     WellnessHabitType,
5 |     DayOfWeekData,
6 | } from "@types/habits";
7 | import { XP_SCHEDULE } from "@lib/xpSchedule";
8 | import type { XPSource } from "@types/app";
9 |
10 | /**
11 |  * XP transaction sources that represent daily-habit completions. Used to scope
12 |  * the per-day XP shown on the heatmap to habit-earned XP only (R7.4), excluding
13 |  * unrelated sources such as quiz/grade/badge XP.
14 |  */
15 | export const HABIT_XP_SOURCES: readonly XPSource[] = [
16 |     "login",
17 |     "submission",
18 |     "journal",
19 |     "perfect_day",
20 |     "wellness_habit",
21 | ] as const;
22 |
23 | const HABIT_XP_SOURCE_SET = new Set<string>(HABIT_XP_SOURCES);
24 |
25 | /** XP awarded for completing all four academic habits in a single day. */
26 | export const PERFECT_DAY_XP = XP_SCHEDULE.perfect_day;
27 |
28 | /** Number of academic habits tracked per day (Login, Submit, Journal, Read). */
29 | export const ACADEMIC_HABITS_PER_DAY = 4;
30 |
31 | export interface HabitXpRecord {
32 |     source: string;
33 |     amount: number;
34 |     /** ISO date string (YYYY-MM-DD) the XP was recorded on. */
35 |     date: string;
36 | }
37 |
38 | /**
39 |  * Aggregates habit-related XP transactions into a per-date total. Only sources
40 |  * in {@link HABIT_XP_SOURCES} contribute, so the value shown for a day equals
41 |  * the XP recorded for that day's habit completions (R7.1, R7.4).
42 |  */
43 | export const aggregateHabitXpByDate = (
44 |     records: HabitXpRecord[]
45 | ): Record<string, number> => {
46 |     const byDate: Record<string, number> = {};
47 |     for (const record of records) {
48 |         if (!HABIT_XP_SOURCE_SET.has(record.source)) continue;
49 |         if (!record.date) continue;
50 |         byDate[record.date] = (byDate[record.date] ?? 0) + record.amount;
51 |     }
52 |     return byDate;
53 | };
54 |
55 | export interface HabitSummary {
56 |     /** Habit type completed most often across the period, or null when none. */
57 |     bestHabit: HabitType | null;
58 |     /** Completion count for the best habit. */
59 |     bestHabitCount: number;
60 |     /** Overall academic-habit completion rate as a percentage (0-100). */
61 |     completionRate: number;
62 | }
63 |
64 | // Fixed evaluation order gives a deterministic best-habit tie-break.
65 | const HABIT_PRIORITY: HabitType[] = [
66 |     "login",
67 |     "submit",
68 |     "journal",
69 |     "read",
70 |     "meditation",
71 |     "hydration",
72 |     "exercise",
73 |     "sleep",
74 | ];
75 |
76 | /**
77 |  * Computes the best-performing habit and overall completion rate across the
78 |  * provided days (R7.2). Completion rate is based on the four academic habits.
79 |  */
80 | export const computeHabitSummary = (days: HeatmapDay[]): HabitSummary => {
81 |     const counts = new Map<HabitType, number>();
82 |     let academicCompletions = 0;
83 |
84 |     for (const day of days) {
85 |         academicCompletions += day.academicCount;
86 |         for (const habit of day.habits) {
87 |             counts.set(habit.type, (counts.get(habit.type) ?? 0) + 1);

```

```

88 |     }
89 | }
90 |
91 | let bestHabit: HabitType | null = null;
92 | let bestHabitCount = 0;
93 | for (const habit of HABIT_PRIORITY) {
94 |   const count = counts.get(habit) ?? 0;
95 |   if (count > bestHabitCount) {
96 |     bestHabit = habit;
97 |     bestHabitCount = count;
98 |   }
99 | }
100 |
101 | const completionRate = computeCompletionRate(
102 |   academicCompletions,
103 |   ACADEMIC_HABITS_PER_DAY,
104 |   days.length
105 | );
106 |
107 | return { bestHabit, bestHabitCount, completionRate };
108 | };
109 |
110 | export const getIntensityLevel = (count: number): number => {
111 |   if (count === 0) return 0;
112 |   if (count === 1) return 1;
113 |   if (count === 2) return 2;
114 |   if (count === 3) return 3;
115 |   return 4;
116 | };
117 |
118 | export const computeLongestStreak = (days: HeatmapDay[]): number => {
119 |   let longest = 0;
120 |   let current = 0;
121 |   for (let i = 0; i < days.length; i++) {
122 |     const day = days[i];
123 |     if (day.academicCount > 0) {
124 |       current++;
125 |       longest = Math.max(longest, current);
126 |     } else {
127 |       current = 0;
128 |     }
129 |   }
130 |   return longest;
131 | };
132 |
133 | export const computeConsistencyScore = (
134 |   days: HeatmapDay[],
135 |   totalDays: number
136 | ): number => {
137 |   if (totalDays === 0) return 0;
138 |   const activeDays = days.filter((d) => d.totalCount > 0).length;
139 |   return Math.round((activeDays / totalDays) * 100);
140 | };
141 |
142 | export const computeCompletionRate = (
143 |   totalCompleted: number,
144 |   possiblePerDay: number,
145 |   daysInPeriod: number
146 | ): number => {
147 |   const totalPossible = possiblePerDay * daysInPeriod;
148 |   if (totalPossible === 0) return 0;
149 |   return Math.min(Math.round((totalCompleted / totalPossible) * 100), 100);
150 | };
151 |
152 | export const computeDayOfWeekAverages = (
153 |   days: HeatmapDay[]
154 | ): DayOfWeekData[] => {
155 |   const dayNames = [
156 |     "Sunday",
157 |     "Monday",
158 |     "Tuesday",
159 |     "Wednesday",
160 |     "Thursday",
161 |     "Friday",
162 |     "Saturday",
163 |   ];
164 |   const sums = new Array(7).fill(0) as number[];
165 |   const counts = new Array(7).fill(0) as number[];
166 |   for (const day of days) {
167 |     const dow = new Date(day.date + "T00:00:00").getDay();
168 |     sums[dow] = (sums[dow] ?? 0) + day.totalCount;
169 |     counts[dow] = (counts[dow] ?? 0) + 1;
170 |   }
171 |   return dayNames.map((name, i) => ({
172 |     day: name,
173 |     avgCompletions:
174 |       (counts[i] ?? 0) > 0
175 |       ? Math.round(((sums[i] ?? 0) / (counts[i] ?? 1)) * 100) / 100
176 |       : 0,
177 |   })));
178 | };
179 |
180 | export const getBestDay = (averages: DayOfWeekData[]): DayOfWeekData | null => {
181 |   if (averages.length === 0) return null;
182 |   return averages.reduce((best, current) =>
183 |     current.avgCompletions > best.avgCompletions ? current : best
184 |   );
185 | };
186 |
187 | export const computeCellSize = (
188 |   containerWidth: number,
189 |   numWeeks: number
190 | ): number => {
191 |   if (numWeeks <= 0) return 12;

```

```

192 | // Account for the per-cell gap (2px) when calculating cell size.
193 | // Total horizontal space = numWeeks * cellSize + numWeeks * CELL_GAP
194 | // !' cellSize = (containerWidth - numWeeks * CELL_GAP) / numWeeks
195 | const CELL_GAP = 2;
196 | const available = Math.max(0, containerWidth - numWeeks * CELL_GAP);
197 | // Cap at 16px so cells stay visually square; floor the result so subpixel
198 | // rounding can't push the SVG wider than the container (which would cause
199 | // a ResizeObserver feedback loop).
200 | return Math.max(Math.min(Math.floor(available / numWeeks), 16), 12);
201 | };
202 |
203 | export const generateMonthLabels = (
204 |   startDate: string,
205 |   endDate: string
206 | ): Array<{ month: string; weekIndex: number }> => {
207 |   const labels: Array<{ month: string; weekIndex: number }> = [];
208 |   const start = new Date(startDate + "T00:00:00");
209 |   const end = new Date(endDate + "T00:00:00");
210 |   const monthNames = [
211 |     "Jan",
212 |     "Feb",
213 |     "Mar",
214 |     "Apr",
215 |     "May",
216 |     "Jun",
217 |     "Jul",
218 |     "Aug",
219 |     "Sep",
220 |     "Oct",
221 |     "Nov",
222 |     "Dec",
223 |   ];
224 |
225 |   let currentMonth = -1;
226 |   const cursor = new Date(start);
227 |   while (cursor <= end) {
228 |     const month = cursor.getMonth();
229 |     if (month !== currentMonth) {
230 |       currentMonth = month;
231 |       const daysSinceStart = Math.floor(
232 |         (cursor.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)
233 |       );
234 |       const weekIndex = Math.floor(daysSinceStart / 7);
235 |       labels.push({ month: monthNames[month] ?? "", weekIndex });
236 |     }
237 |     cursor.setDate(cursor.getDate() + 1);
238 |   }
239 |   return labels;
240 | };
241 |
242 | export const generateGridDimensions = (
243 |   startDate: string,
244 |   endDate: string
245 | ): { rows: 7; columns: number } => {
246 |   const start = new Date(startDate + "T00:00:00");
247 |   const end = new Date(endDate + "T00:00:00");
248 |   const totalDays =
249 |     Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1;
250 |   const columns = Math.ceil(totalDays / 7);
251 |   return { rows: 7, columns };
252 | };
253 |
254 | export const isDateFuture = (date: string): boolean => {
255 |   const today = new Date();
256 |   today.setHours(0, 0, 0, 0);
257 |   const d = new Date(date + "T00:00:00");
258 |   return d > today;
259 | };
260 |
261 | export const getFilterOptions = (
262 |   enabledWellnessHabits: WellnessHabitType[]
263 | ): string[] => {
264 |   const base = ["All Habits", "Login", "Submit", "Journal", "Read"];
265 |   const wellnessLabels: Record<WellnessHabitType, string> = {
266 |     meditation: "Meditation",
267 |     hydration: "Hydration",
268 |     exercise: "Exercise",
269 |     sleep: "Sleep",
270 |   };
271 |   return [...base, ...enabledWellnessHabits.map((h) => wellnessLabels[h])];
272 | };
273 |
274 | export const generateAriaLabel = (date: string, count: number): string => {
275 |   const d = new Date(date + "T00:00:00");
276 |   const formatted = d.toLocaleDateString("en-US", {
277 |     month: "long",
278 |     day: "numeric",
279 |     year: "numeric",
280 |   });
281 |   return `${formatted}: ${count} habit${count !== 1 ? "s" : ""} completed`;
282 | };
283 |

```

## FILE: src/lib/hintTokenChecker.ts

TYPE: .ts | LINES: 1-108 | SHA256: 5262cff8dceb702ff8069afe3cf005daceb64c41f55ce2487202876ec70fd5ac

```

1 | // =====
2 | // hintTokenChecker - Check for active hint tokens before showing daily limit
3 | // =====
4 |
5 | import { supabase } from "@lib/supabase";
6 |
7 | export interface HintTokenStatus {
8 |   hasActiveTokens: boolean;

```

```

9 |     extraMessagesRemaining: number;
10 |     purchaseId: string | null;
11 | }
12 |
13 | /**
14 |  * Checks if a student has active hint token packs purchased today (UTC).
15 |  * Hint tokens expire at midnight UTC on the day of purchase.
16 |  */
17 | export async function getHintTokenStatus(
18 |     studentId: string
19 | ): Promise<HintTokenStatus> {
20 |     const todayStart = new Date();
21 |     todayStart.setUTCHours(0, 0, 0, 0);
22 |     const todayEnd = new Date();
23 |     todayEnd.setUTCHours(23, 59, 59, 999);
24 |
25 |     // eslint-disable-next-line @typescript-eslint/no-explicit-any
26 |     const { data, error } = await (supabase as any)
27 |         .from("xp_purchases")
28 |         .select(
29 |             "id, purchased_at, metadata, marketplace_items:item_id (sub_category)"
30 |         )
31 |         .eq("student_id", studentId)
32 |         .eq("status", "active")
33 |         .gte("purchased_at", todayStart.toISOString())
34 |         .lte("purchased_at", todayEnd.toISOString());
35 |
36 |     if (error || !data) {
37 |         return {
38 |             hasActiveTokens: false,
39 |             extraMessagesRemaining: 0,
40 |             purchaseId: null,
41 |         };
42 |     }
43 |
44 |     const hintTokens = (data as Array<Record<string, unknown>>).filter((row) => {
45 |         const item = row.marketplace_items as Record<string, unknown> | null;
46 |         return item?.sub_category === "hint_token";
47 |     });
48 |
49 |     if (hintTokens.length === 0) {
50 |         return {
51 |             hasActiveTokens: false,
52 |             extraMessagesRemaining: 0,
53 |             purchaseId: null,
54 |         };
55 |     }
56 |
57 |     // Each hint token pack provides 5 extra messages
58 |     // Check metadata for used count
59 |     const totalExtra = hintTokens.length * 5;
60 |     const usedCount = hintTokens.reduce((sum, token) => {
61 |         const meta = token.metadata as Record<string, unknown> | null;
62 |         return sum + ((meta?.messages_used as number) ?? 0);
63 |     }, 0);
64 |
65 |     const remaining = Math.max(0, totalExtra - usedCount);
66 |     const firstActiveToken = hintTokens[0];
67 |
68 |     return {
69 |         hasActiveTokens: remaining > 0,
70 |         extraMessagesRemaining: remaining,
71 |         purchaseId: firstActiveToken ? (firstActiveToken.id as string) : null,
72 |     };
73 | }
74 |
75 | /**
76 |  * Pure function: determines if a student can send an AI Tutor message.
77 |  * Checks base daily limit first, then falls back to hint tokens.
78 |  */
79 | export function canSendTutorMessage(
80 |     baseMessagesUsed: number,
81 |     baseDailyLimit: number,
82 |     hintTokenStatus: HintTokenStatus
83 | ): { allowed: boolean; usingHintToken: boolean; remainingMessages: number } {
84 |     const baseRemaining = Math.max(0, baseDailyLimit - baseMessagesUsed);
85 |
86 |     if (baseRemaining > 0) {
87 |         return {
88 |             allowed: true,
89 |             usingHintToken: false,
90 |             remainingMessages: baseRemaining + hintTokenStatus.extraMessagesRemaining,
91 |         };
92 |     }
93 |
94 |     if (hintTokenStatus.hasActiveTokens) {
95 |         return {
96 |             allowed: true,
97 |             usingHintToken: true,
98 |             remainingMessages: hintTokenStatus.extraMessagesRemaining,
99 |         };
100 |     }
101 |
102 |     return {
103 |         allowed: false,
104 |         usingHintToken: false,
105 |         remainingMessages: 0,
106 |     };
107 | }
108 |

```

FILE: src/lib/i18n.ts

TYPE: .ts | LINES: 1-151 | SHA256: 5e734adefc631f95f362739771f345eed826a7dfcbaaf9c430cecf4507cc2bc

```
1 | import i18n from "i18next";
2 | import { initReactI18next } from "react-i18next";
3 |
4 | // English namespaces
5 | import enCommon from "@/locales/en/common.json";
6 | import enAuth from "@/locales/en/auth.json";
7 | import enAdmin from "@/locales/en/admin.json";
8 | import enTeacher from "@/locales/en/teacher.json";
9 | import enStudent from "@/locales/en/student.json";
10 | import enCoordinator from "@/locales/en/coordinator.json";
11 | import enGamification from "@/locales/en/gamification.json";
12 | import enAi from "@/locales/en/ai.json";
13 |
14 | // Arabic namespaces
15 | import arCommon from "@/locales/ar/common.json";
16 | import arAuth from "@/locales/ar/auth.json";
17 | import arAdmin from "@/locales/ar/admin.json";
18 | import arTeacher from "@/locales/ar/teacher.json";
19 | import arStudent from "@/locales/ar/student.json";
20 | import arCoordinator from "@/locales/ar/coordinator.json";
21 | import arGamification from "@/locales/ar/gamification.json";
22 | import arAi from "@/locales/ar/ai.json";
23 |
24 | import { applyDirection } from "@/lib/directionManager";
25 |
26 | export const defaultNS = "common";
27 | export const supportedLanguages = ["en", "ar"] as const;
28 | export const namespaces = [
29 |   "common",
30 |   "auth",
31 |   "admin",
32 |   "teacher",
33 |   "student",
34 |   "coordinator",
35 |   "gamification",
36 |   "ai",
37 | ] as const;
38 |
39 | export type SupportedLanguage = (typeof supportedLanguages)[number];
40 | export type Namespace = (typeof namespaces)[number];
41 |
42 | export const resources = {
43 |   en: {
44 |     common: enCommon,
45 |     auth: enAuth,
46 |     admin: enAdmin,
47 |     teacher: enTeacher,
48 |     student: enStudent,
49 |     coordinator: enCoordinator,
50 |     gamification: enGamification,
51 |     ai: enAi,
52 |   },
53 |   ar: {
54 |     common: arCommon,
55 |     auth: arAuth,
56 |     admin: arAdmin,
57 |     teacher: arTeacher,
58 |     student: arStudent,
59 |     coordinator: arCoordinator,
60 |     gamification: arGamification,
61 |     ai: arAi,
62 |   },
63 | } as const;
64 |
65 | const savedLang =
66 |   typeof window !== "undefined"
67 |     ? localStorage.getItem("edeviser-language")
68 |     : null;
69 |
70 | /**
71 |  * Visible marker prepended to the resolved text when a key cannot be resolved
72 |  * for the active language (R29.3a). The Arabic-speaking student must see that a
73 |  * translation is missing rather than silently getting the raw key or English.
74 |  */
75 | export const MISSING_TRANSLATION_MARKER = "& ";
76 |
77 | /**
78 |  * Reports a missing key / failed translation resolution. While the active
79 |  * language is Arabic, a missing key or translation-service failure on an
80 |  * internationalized page must be surfaced rather than silently rendering the
81 |  * raw key or the English fallback (R29.3a). We:
82 |  * 1. log it (always), and
83 |  * 2. drop a Sentry breadcrumb when monitoring is initialized,
84 |  * so the failure is observable in production without crashing the surface.
85 |  */
86 | const reportMissingTranslation = (
87 |   lngs: readonly string[],
88 |   ns: string,
89 |   key: string
90 | ): void => {
91 |   const active = (lngs[0] ?? "").split("-")[0];
92 |   // Only Arabic users are guaranteed surfacing per R29.3a; for the English
93 |   // base language a missing key is a development-time issue caught by the
94 |   // en/ar key-parity audit instead.
95 |   if (active !== "ar") return;
96 |
97 |   const detail = `[i18n] missing translation: ns="${ns}" key="${key}" lng="${active}"`;
98 |   if (typeof console !== "undefined") {
99 |     console.error(detail);
100 |   }
101 | }
```

```

102 | // Best-effort Sentry breadcrumb; never let monitoring failures bubble up.
103 | void import("@sentry/react")
104 | .then((Sentry) => {
105 |   if (Sentry.isInitialized()) {
106 |     Sentry.addBreadcrumb({
107 |       category: "i18n",
108 |       level: "warning",
109 |       message: detail,
110 |     });
111 |   }
112 | })
113 | .catch(() => {
114 |   /* monitoring unavailable – the console error above is the fallback */
115 | });
116 | };
117 |
118 | i18n.use(initReactI18next).init({
119 |   resources,
120 |   lng: savedLang || "en",
121 |   fallbackLng: "en",
122 |   defaultNS,
123 |   ns: [...namespaces],
124 |   interpolation: {
125 |     escapeValue: false, // React already escapes
126 |   },
127 |   pluralSeparator: "-",
128 |   supportedLngs: ["en", "ar"],
129 |   saveMissing: true,
130 |   // Surface (don't silently swallow) missing keys / translation-service
131 |   // failures while Arabic is active (R29.3a). The handler reports the gap and
132 |   // returns a visibly-marked string so the user is never shown a bare key or a
133 |   // silent English fallback masquerading as a real Arabic translation.
134 |   parseMissingKeyHandler: (key, defaultValue) => {
135 |     const active = (i18n.language ?? "").split("-")[0];
136 |     if (active === "ar") {
137 |       return `${MISSING_TRANSLATION_MARKER}${defaultValue ?? key}`;
138 |     }
139 |     return defaultValue ?? key;
140 |   },
141 |   missingKeyHandler: (lngs, ns, key) => {
142 |     reportMissingTranslation(lngs, ns, key);
143 |   },
144 | });
145 |
146 | // Apply direction on init and on every language change
147 | applyDirection(i18n.language);
148 | i18n.on("languageChanged", applyDirection);
149 |
150 | export default i18n;
151 |

```

## FILE: src/lib/i18nHelpers.ts

TYPE: .ts | LINES: 1-205 | SHA256: 2a0e8f06a6bd201a0d62425443d9913026a333de2f0b9d95cc65c862cd39c056

```

1 | /**
2 |  * Internationalization helper functions
3 |  *
4 |  * Provides a single source of truth for Intl formatters with ar-QA defaults.
5 |  * Consumers should never instantiate Intl.DateTimeFormat, Intl.NumberFormat, etc. directly.
6 |  */
7 |
8 | /**
9 |  * Supported locales
10 |  */
11 | export type SupportedLocale = "en" | "ar-QA";
12 |
13 | /**
14 |  * Format a date value
15 |  *
16 |  * Usage:
17 |  * ```tsx
18 |  * formatDate(new Date(), 'ar-QA') // "f b dV'd!H f&`f&f"
19 |  * formatDate(new Date(), 'en')    // "May 12, 2026"
20 |  * ```
21 |  */
22 | export const formatDate = (
23 |   value: Date | number,
24 |   locale: SupportedLocale = "ar-QA",
25 |   options?: Intl.DateTimeFormatOptions
26 | ): string => {
27 |   const defaultOptions: Intl.DateTimeFormatOptions = {
28 |     year: "numeric",
29 |     month: "long",
30 |     day: "numeric",
31 |     ...options,
32 |   };
33 |
34 |   return new Intl.DateTimeFormat(locale, defaultOptions).format(
35 |     typeof value === "number" ? new Date(value) : value
36 |   );
37 | };
38 |
39 | /**
40 |  * Format a date as a short date (e.g., "12/05/2026" or "f b/f e/f&`f&f")
41 |  */
42 | export const formatDateShort = (
43 |   value: Date | number,
44 |   locale: SupportedLocale = "ar-QA"
45 | ): string => {
46 |   return formatDate(value, locale, {
47 |     year: "numeric",
48 |     month: "2-digit",
49 |     day: "2-digit",
50 |   });

```

```

51 | };
52 |
53 | /**
54 |  * Format a date with time (e.g., "May 12, 2026, 3:30 PM")
55 |  */
56 | export const formatDateTime = (
57 |   value: Date | number,
58 |   locale: SupportedLocale = "ar-QA"
59 | ): string => {
60 |   return formatDate(value, locale, {
61 |     year: "numeric",
62 |     month: "long",
63 |     day: "numeric",
64 |     hour: "2-digit",
65 |     minute: "2-digit",
66 |   });
67 | };
68 |
69 | /**
70 |  * Format a number value
71 |  *
72 |  * For Arabic locale, uses Arabic-Indic numerals (f af&cfFeffgfti)
73 |  *
74 |  * Usage:
75 |  * ```tsx
76 |  * formatNumber(1234.56, 'ar-QA') // "f lf&cfFkfVf"
77 |  * formatNumber(1234.56, 'en')    // "1,234.56"
78 |  * ```
79 |  */
80 | export const formatNumber = (
81 |   value: number,
82 |   locale: SupportedLocale = "ar-QA",
83 |   options?: Intl.NumberFormatOptions
84 | ): string => {
85 |   const defaultOptions: Intl.NumberFormatOptions = {
86 |     minimumFractionDigits: 0,
87 |     maximumFractionDigits: 2,
88 |     ...options,
89 |   };
90 |
91 |   return new Intl.NumberFormat(locale, defaultOptions).format(value);
92 | };
93 |
94 | /**
95 |  * Format a number as currency
96 |  *
97 |  * Usage:
98 |  * ```tsx
99 |  * formatCurrency(1234.56, 'ar-QA', 'QAR') // "f lf&cfFkfVf c æB."
100 |  * formatCurrency(1234.56, 'en', 'USD')    // "$1,234.56"
101 |  * ```
102 |  */
103 | export const formatCurrency = (
104 |   value: number,
105 |   locale: SupportedLocale = "ar-QA",
106 |   currency: string = "QAR"
107 | ): string => {
108 |   return new Intl.NumberFormat(locale, {
109 |     style: "currency",
110 |     currency,
111 |   }).format(value);
112 | };
113 |
114 | /**
115 |  * Format a number as a percentage
116 |  *
117 |  * Usage:
118 |  * ```tsx
119 |  * formatPercentage(0.856, 'ar-QA') // "f fef fff¢"
120 |  * formatPercentage(0.856, 'en')    // "85.6%"
121 |  * ```
122 |  */
123 | export const formatPercentage = (
124 |   value: number,
125 |   locale: SupportedLocale = "ar-QA",
126 |   fractionDigits: number = 1
127 | ): string => {
128 |   return new Intl.NumberFormat(locale, {
129 |     style: "percent",
130 |     minimumFractionDigits: fractionDigits,
131 |     maximumFractionDigits: fractionDigits,
132 |   }).format(value);
133 | };
134 |
135 | /**
136 |  * Format relative time (e.g., "2 days ago", "d J d!HdVJdb"•
137 |  *
138 |  * Usage:
139 |  * ```tsx
140 |  * formatRelativeTime(-2, 'day', 'ar-QA') // "dVFc Jd!Ed!F"
141 |  * formatRelativeTime(-2, 'day', 'en')    // "2 days ago"
142 |  * formatRelativeTime(1, 'hour', 'ar-QA') // "d J c6'c-)"
143 |  * formatRelativeTime(1, 'hour', 'en')    // "in 1 hour"
144 |  * ```
145 |  */
146 | export const formatRelativeTime = (
147 |   value: number,
148 |   unit: Intl.RelativeTimeFormatUnit,
149 |   locale: SupportedLocale = "ar-QA"
150 | ): string => {
151 |   return new Intl.RelativeTimeFormat(locale, {
152 |     numeric: "auto",
153 |   }).format(value, unit);
154 | };

```

```

155 |
156 | /**
157 |  * Format a list of items
158 |  *
159 |  * Note: Intl.ListFormat is only available in ES2021+
160 |  * For now, this is a simple implementation that joins with commas
161 |  *
162 |  * Usage:
163 |  * ```tsx
164 |  * formatList(['apple', 'banana', 'cherry']) // "apple, banana, cherry"
165 |  * ```
166 |  */
167 | export const formatList = (
168 |   items: string[],
169 |   _locale: SupportedLocale = "ar-QA",
170 |   _type: "conjunction" | "disjunction" = "conjunction"
171 | ): string => {
172 |   // Simple implementation: join with commas
173 |   // TODO: Use Intl.ListFormat when ES2021+ is available
174 |   if (items.length === 0) return "";
175 |   if (items.length === 1) return items[0] ?? "";
176 |   const lastItem = items[items.length - 1] ?? "";
177 |   return items.slice(0, -1).join(", ") + " and " + lastItem;
178 | };
179 |
180 | /**
181 |  * Get the text direction for a locale
182 |  *
183 |  * Usage:
184 |  * ```tsx
185 |  * getTextDirection('ar-QA') // "rtl"
186 |  * getTextDirection('en')    // "ltr"
187 |  * ```
188 |  */
189 | export const getTextDirection = (locale: SupportedLocale): "ltr" | "rtl" => {
190 |   return locale.startsWith("ar") ? "rtl" : "ltr";
191 | };
192 |
193 | /**
194 |  * Get the text alignment for a locale
195 |  *
196 |  * Usage:
197 |  * ```tsx
198 |  * getTextAlign('ar-QA') // "right"
199 |  * getTextAlign('en')   // "left"
200 |  * ```
201 |  */
202 | export const getTextAlign = (locale: SupportedLocale): "left" | "right" => {
203 |   return locale.startsWith("ar") ? "right" : "left";
204 | };
205 |

```

**FILE:** [src/lib/idempotentRunner.ts](#)

TYPE: .ts | LINES: 1-74 | SHA256: 9cc8f5c61cddcb30d683a1273abd2beec0cf7e35b49532e16c4b955a159a3430

```

1 | // Task 11.2 / 11.3: Idempotent async runner (finalize guard)
2 | // Requirements: 3.4
3 | //
4 | // Extracted from AdaptiveQuizSession's inline `finalizedRef` guard so the
5 | // at-most-once finalization logic is a single, framework-agnostic, testable
6 | // unit instead of being duplicated between the component and its tests.
7 | //
8 | // The runner models the exact guard the quiz finalization needs: across any
9 | // number and ordering of triggers (timer expiry, manual finish, unmount) the
10 | // underlying submission runs at most once on success, while a failed attempt
11 | // releases the guard so a later trigger may retry (R3.4).
12 |
13 | /**
14 |  * A guard that runs a guarded async operation at most once across all callers.
15 |  *
16 |  * The completion flag is claimed *synchronously* before the operation is
17 |  * awaited, so concurrent (interleaved) triggers in the same tick cannot both
18 |  * pass the guard – only the first invocation runs the operation.
19 |  */
20 | export interface IdempotentRunner {
21 |   /**
22 |    * Whether a guarded operation has completed successfully. Once `true` it
23 |    * never returns to `false`, so every subsequent `run` call is a no-op.
24 |    */
25 |   readonly hasCompleted: boolean;
26 |
27 |   /**
28 |    * Runs `operation` at most once across all calls to this runner.
29 |    *
30 |    * - WHEN a previous call already completed successfully: this is a no-op and
31 |    *   `operation` is not invoked.
32 |    * - WHEN no prior call has completed: the guard is claimed synchronously,
33 |    *   then `operation` is awaited exactly once.
34 |    * - WHEN `operation` rejects: the guard is released (so a later call may
35 |    *   retry) and the error is re-thrown to the caller.
36 |    */
37 |   run(operation: () => Promise<void>): Promise<void>;
38 | }
39 |
40 | /**
41 |  * Creates a fresh {@link IdempotentRunner}.
42 |  *
43 |  * Each runner owns independent completion state; create one per logical
44 |  * operation (e.g. one per quiz attempt).
45 |  */
46 | export function createIdempotentRunner(): IdempotentRunner {
47 |   // Mirrors the component's `finalizedRef.current`.
48 |   let completed = false;
49 |

```



```

50 |     return {
51 |       get hasCompleted(): boolean {
52 |         return completed;
53 |       },
54 |
55 |       async run(operation: () => Promise<void>): Promise<void> {
56 |         // Already finalized -> no-op (never invoke the operation again).
57 |         if (completed) return;
58 |
59 |         // Claim the guard *before* awaiting so concurrent triggers in the same
60 |         // tick cannot both pass – this is what makes finalization idempotent.
61 |         completed = true;
62 |         try {
63 |           await operation();
64 |           // Success: the guard stays claimed forever (no reset).
65 |         } catch (error) {
66 |           // Failed attempt: release the guard so a later trigger may retry, and
67 |           // surface the error so the caller can react (e.g. show a toast).
68 |           completed = false;
69 |           throw error;
70 |         }
71 |       },
72 |     };
73 |   }
74 | }

```

## FILE: src/lib/imageCompressor.ts

TYPE: .ts | LINES: 1-81 | SHA256: 9e7937cae9f352ac836e6c691bbde94bb3726d384073b661e82d38e207ba47a

```

1 | // Task 91.1: Client-side image compression using Canvas API
2 |
3 | const DEFAULT_MAX_WIDTH = 256;
4 | const DEFAULT_MAX_HEIGHT = 256;
5 | const DEFAULT_QUALITY = 0.8;
6 | const MAX_FILE_SIZE = 500 * 1024; // 500KB
7 |
8 | export const compressImage = (
9 |   file: File,
10 |   maxWidth: number = DEFAULT_MAX_WIDTH,
11 |   maxHeight: number = DEFAULT_MAX_HEIGHT,
12 |   quality: number = DEFAULT_QUALITY
13 | ): Promise<File> => {
14 |   return new Promise((resolve, reject) => {
15 |     const img = new Image();
16 |     const url = URL.createObjectURL(file);
17 |
18 |     img.onload = () => {
19 |       URL.revokeObjectURL(url);
20 |
21 |       // Calculate scaled dimensions preserving aspect ratio
22 |       let width = img.width;
23 |       let height = img.height;
24 |
25 |       if (width > maxWidth || height > maxHeight) {
26 |         const ratio = Math.min(maxWidth / width, maxHeight / height);
27 |         width = Math.round(width * ratio);
28 |         height = Math.round(height * ratio);
29 |       }
30 |
31 |       const canvas = document.createElement("canvas");
32 |       canvas.width = width;
33 |       canvas.height = height;
34 |
35 |       const ctx = canvas.getContext("2d");
36 |       if (!ctx) {
37 |         reject(new Error("Failed to get canvas 2d context"));
38 |         return;
39 |       }
40 |
41 |       ctx.drawImage(img, 0, 0, width, height);
42 |
43 |       // Iteratively reduce quality if output exceeds MAX_FILE_SIZE
44 |       let currentQuality = quality;
45 |       const tryCompress = () => {
46 |         canvas.toBlob(
47 |           (blob) => {
48 |             if (!blob) {
49 |               reject(new Error("Canvas toBlob returned null"));
50 |               return;
51 |             }
52 |
53 |             if (blob.size > MAX_FILE_SIZE && currentQuality > 0.1) {
54 |               currentQuality -= 0.1;
55 |               tryCompress();
56 |               return;
57 |             }
58 |
59 |             const compressed = new File([blob], file.name, {
60 |               type: "image/jpeg",
61 |               lastModified: Date.now(),
62 |             });
63 |             resolve(compressed);
64 |           },
65 |           "image/jpeg",
66 |           currentQuality
67 |         );
68 |       };
69 |
70 |       tryCompress();
71 |     };
72 |
73 |     img.onerror = () => {
74 |       URL.revokeObjectURL(url);
75 |       reject(new Error("Failed to load image"));

```

```

76 |     };
77 |
78 |     img.src = url;
79 |   });
80 | };
81 |

```

## FILE: src/lib/independenceCalculator.ts

TYPE: .ts | LINES: 1-40 | SHA256: e7fcd50c2d176d37d77645add682790c0d66a53c581e5a4940737d6ce563ba1b

```

1 | // =====
2 | // independenceCalculator - Independence score calculation for AI dependency tracking
3 | // =====
4 |
5 | /**
6 |  * Calculate the independence score for a student on a specific CLO.
7 |  *
8 |  * Independence Score = 1 - (AI-assisted submissions / total submissions)
9 |  *
10 |  * An AI-assisted submission is one where the student had a tutor conversation
11 |  * on the same CLO within 1 hour before submission.
12 |  *
13 |  * Edge cases:
14 |  * - 0 total submissions !' score 1.0 (fully independent - no data to suggest dependency)
15 |  * - All submissions AI-assisted !' score 0.0
16 |  * - Negative values are clamped to 0
17 |  *
18 |  * @param totalSubmissions - Total number of submissions for this CLO
19 |  * @param aiAssistedSubmissions - Number of submissions preceded by AI tutor usage
20 |  * @returns Independence score between 0.0 and 1.0
21 |  */
22 | export function calculateIndependenceScore(
23 |   totalSubmissions: number,
24 |   aiAssistedSubmissions: number
25 | ): number {
26 |   // No submissions means fully independent (no evidence of dependency)
27 |   if (totalSubmissions <= 0) return 1.0;
28 |
29 |   // Clamp AI-assisted count to valid range
30 |   const clampedAiAssisted = Math.max(
31 |     0,
32 |     Math.min(aiAssistedSubmissions, totalSubmissions)
33 |   );
34 |
35 |   const score = 1 - clampedAiAssisted / totalSubmissions;
36 |
37 |   // Clamp result to [0, 1] for safety
38 |   return Math.max(0, Math.min(1, score));
39 | }
40 |

```

## FILE: src/lib/journalInsights.ts

TYPE: .ts | LINES: 1-94 | SHA256: 2065133a6433314b040269d173178f4168bef71a1ecb6ae0f091c3be1f83ddc1

```

1 | // =====
2 | // journalInsights - presentation-safe journal metrics derived from real entries
3 | // =====
4 |
5 | export interface JournalInsightEntry {
6 |   created_at: string;
7 |   content: string;
8 | }
9 |
10 | export interface JournalDay {
11 |   date: Date;
12 |   hasEntry: boolean;
13 | }
14 |
15 | /** A UTC calendar-day key keeps the timeline and right rail deterministic. */
16 | export const journalDayKey = (date: Date): string =>
17 |   date.toISOString().slice(0, 10);
18 |
19 | const wordCount = (content: string): number =>
20 |   content.trim().split(/\s+/).filter(Boolean).length;
21 |
22 | const startOfUtcWeek = (now: Date): Date => {
23 |   const start = new Date(
24 |     Date.UTC(now.getUTCFullYear(), now.getUTCMonth(), now.getUTCDate())
25 |   );
26 |   const daysSinceMonday = (start.getUTCDate() + 6) % 7;
27 |   start.setUTCDate(start.getUTCDate() - daysSinceMonday);
28 |   return start;
29 | };
30 |
31 | export const computeJournalStreak = (
32 |   createdAts: string[],
33 |   now = new Date()
34 | ): number => {
35 |   const entryDays = new Set(
36 |     createdAts.map((createdAt) => createdAt.slice(0, 10))
37 |   );
38 |   if (entryDays.size === 0) return 0;
39 |
40 |   const cursor = new Date(
41 |     Date.UTC(now.getUTCFullYear(), now.getUTCMonth(), now.getUTCDate())
42 |   );
43 |   if (!entryDays.has(journalDayKey(cursor))) {
44 |     cursor.setUTCDate(cursor.getUTCDate() - 1);
45 |     if (!entryDays.has(journalDayKey(cursor))) return 0;
46 |   }
47 |
48 |   let streak = 0;
49 |   while (entryDays.has(journalDayKey(cursor))) {
50 |     streak += 1;

```

```

51 |     cursor.setUTCDate(cursor.getUTCDate() - 1);
52 | }
53 | return streak;
54 | };
55 |
56 | export const getJournalWeekDays = (
57 |   entries: JournalInsightEntry[],
58 |   now = new Date()
59 | ): JournalDay[] => {
60 |   const entryDays = new Set(
61 |     entries.map((entry) => entry.created_at.slice(0, 10))
62 |   );
63 |   const start = startOfUtcWeek(now);
64 |
65 |   return Array.from({ length: 7 }, (_, index) => {
66 |     const date = new Date(start);
67 |     date.setUTCDate(start.getUTCDate() + index);
68 |     return { date, hasEntry: entryDays.has(journalDayKey(date)) };
69 |   });
70 | };
71 |
72 | export const getJournalInsights = (
73 |   entries: JournalInsightEntry[],
74 |   now = new Date()
75 | ) => {
76 |   const weekStart = journalDayKey(startOfUtcWeek(now));
77 |   const weekEntries = entries.filter(
78 |     (entry) => entry.created_at.slice(0, 10) >= weekStart
79 |   );
80 |
81 |   return {
82 |     entriesThisWeek: weekEntries.length,
83 |     totalEntries: entries.length,
84 |     substantiveThisWeek: weekEntries.filter(
85 |       (entry) => wordCount(entry.content) >= 50
86 |     ).length,
87 |     streak: computeJournalStreak(
88 |       entries.map((entry) => entry.created_at),
89 |       now
90 |     ),
91 |     days: getJournalWeekDays(entries, now),
92 |   };
93 | };
94 |

```

FILE: src/lib/journalPromptGenerator.ts

```
TYPE: .ts | LINES: 1-200 | SHA256: a7cae369662a2d5ffbe551f6142667d1ac80447fb8a674e23fbf5ed2c62d26f3
```

[illegible]

```

57 // % % % Question Templates per Kolb Stage x Bloom's Level % % % % % % % % % % % % % % % %
58
59 type QuestionTemplateMap = Record<KolbStage, Record<BloomsLevel, string>>;
60
61 const QUESTION_TEMPLATES: QuestionTemplateMap = {
62   "Concrete Experience": {
63     remembering:
64       'What specific facts or concepts from "{clo}" do you recall from this assessment?',
65     understanding:
66       'How would you describe your experience working on "{clo}" in your own words?',
67     applying:
68       'What was it like applying the concepts from "{clo}" to a concrete problem?',
69     analyzing:
70       'What patterns or connections did you notice while working on "{clo}"?',
71     evaluating:
72       'What was your experience evaluating or judging ideas related to "{clo}"?',
73     creating:
74       'What was the experience of creating something new for "{clo}" like for you?',
75   },
76   "Reflective Observation": {
77     remembering:
78       'Which parts of "{clo}" did you find easiest to recall, and where did you struggle?',
79     understanding:
80       'Looking back, which aspects of "{clo}" do you feel you understood well, and which were unclear?',
81     applying:
82       'Reflecting on your work, where did you apply concepts from "{clo}" effectively, and where did you struggle?',
83     analyzing:
84       'What assumptions did you make while analyzing "{clo}", and were they correct?',
85     evaluating:
86       'How confident were you in the judgments you made about "{clo}", and why?',
87     creating:
88       'What creative choices did you make for "{clo}", and what would you reconsider?',
89   },
90   "Abstract Conceptualization": {
91     remembering:
92       'What study strategies could help you better retain the key concepts of "{clo}"?',
93     understanding:
94       'What deeper principle or theory connects to your understanding of "{clo}"?',
95     applying:
96       'What general approach or framework could improve how you apply "{clo}" concepts next time?',
97     analyzing:
98       'What analytical framework or model could strengthen your work on "{clo}"?',
99     evaluating:
100       'What criteria or standards should guide your evaluation of "{clo}" topics in the future?',
101     creating:
102       'What design principles or creative methods could elevate your work on "{clo}"?',
103   },
104   "Active Experimentation": {
105     remembering:
106       'What specific steps will you take to reinforce your knowledge of "{clo}" before the next assessment?',
107     understanding:
108       'How will you deepen your understanding of "{clo}" – what will you try differently?',
109     applying:
110       'What real-world scenario could you practice applying "{clo}" concepts to?',
111     analyzing:
112       'How will you test your analytical skills on "{clo}" in a new context?',
113     evaluating:
114       'What will you do differently when evaluating "{clo}" topics in your next assignment?',
115     creating:
116       'What new project or approach will you experiment with to push your "{clo}" skills further?',
117   },
118 };
119
120 // % % % Intro Templates per Attainment Level % % % % % % % % % % % % % % % % % % % % % % % % % % % %
121
122 const INTRO_TEMPLATES: Record<AttainmentLevel, string> = {
123   Excellent:
124     'Great work on "{clo}" ({blooms})! You achieved {attainment}. Take a moment to reflect on what made you successful.',
125   Satisfactory:
126     'Good progress on "{clo}" ({blooms})! You achieved {attainment}. Reflect on what went well and identify areas for improvement.',
127   Developing:
128     'You're making progress on "{clo}" ({blooms}) with {attainment}. Use this reflection to identify what's working and what needs more practice.',
129   Not_Yet:
130     'You're still building your skills in "{clo}" ({blooms}) – currently at {attainment}. This reflection can help you identify areas to focus on for the next stage.',
131 };
132
133 // % % % Template Interpolation % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
134
135 function interpolate(
136   template: string,
137   cloTitle: string,
138   bloomsLevel: BloomsLevel,
139   attainmentLevel: AttainmentLevel
140 ): string {
141   return template
142     .replace(/\{clo\}/g, cloTitle)
143     .replace(/\{blooms\}/g, BLOOMS_LABELS[bloomsLevel])
144     .replace(/\{attainment\}/g, ATTAINMENT_LABELS[attainmentLevel]);
145 }
146
147 // % % % Main Generator % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
148
149 /**
150  * Generates contextual journal reflection prompts aligned to Kolb's
151  * Experiential Learning Cycle based on the student's CLO context.
152  *
153  * Pure function – no side effects, no API calls.
154  */
155 export function generateJournalPrompt(
156   context: JournalPromptContext
157 ): GeneratedJournalPrompt {
158   const { cloTitle, bloomsLevel, attainmentLevel, rubricFeedbackSummary } =
159     context;
160 }

```

```

161 |
162 | // Build intro text
163 | const introTemplate = INTRO_TEMPLATES[attainmentLevel];
164 | let promptText = interpolate(
165 |   introTemplate,
166 |   cloTitle,
167 |   bloomsLevel,
168 |   attainmentLevel
169 | );
170 |
171 | if (rubricFeedbackSummary) {
172 |   promptText += ` Your teacher noted: "${rubricFeedbackSummary}"`;
173 | }
174 |
175 | // Select which Kolb stages to include (3-4 questions).
176 | // Always include all 4 for Developing/Not_Yet to encourage deeper reflection.
177 | // For Excellent/Satisfactory, skip Concrete Experience (they already demonstrated it)
178 | // and focus on higher-order reflection stages, keeping 3 questions.
179 | const stages: KolbStage[] =
180 |   attainmentLevel === "Developing" || attainmentLevel === "Not_Yet"
181 |     ? [...KOLB_STAGES]
182 |     : [
183 |       "Reflective Observation",
184 |       "Abstract Conceptualization",
185 |       "Active Experimentation",
186 |     ];
187 |
188 | const questions: KolbQuestion[] = stages.map((stage) => ({
189 |   stage,
190 |   question: interpolate(
191 |     QUESTION_TEMPLATES[stage][bloomsLevel],
192 |     cloTitle,
193 |     bloomsLevel,
194 |     attainmentLevel
195 |   ),
196 | }));
197 |
198 | return { promptText, questions };
199 | }
200 |

```

#### FILE: src/lib/leaderboardGate.ts

TYPE: .ts | LINES: 1-48 | SHA256: adfe7ae28d048c98f449567182969a1427d9d909768a8baf241ced95e59424d8

```

1 | // Task 6.3: Leaderboard minimum-cohort gate (pure business logic)
2 | // Requirements: 6.1, 6.2, 6.2a, 6.4
3 |
4 | /**
5 |  * Discriminated result of the leaderboard gate.
6 |  * - `locked` !' render the "unlocks when more students join" state; never award rank/medals.
7 |  * - `unlocked` !' render the ranked leaderboard.
8 |  */
9 | export type LeaderboardState = "locked" | "unlocked";
10 |
11 | /**
12 |  * Determines whether the leaderboard is locked or unlocked for a cohort.
13 |  *
14 |  * Rules (Requirement 6):
15 |  * - WHILE there are no eligible (non-opted-out) students, the leaderboard is
16 |  *   ALWAYS locked, regardless of the configured minimum (R6.2a). This also
17 |  *   prevents a misconfigured `minCohort <= 0` from unlocking an empty cohort.
18 |  * - WHILE the eligible count is below the configured minimum, it is locked (R6.1).
19 |  * - WHEN the eligible count reaches or exceeds the configured minimum, it is
20 |  *   unlocked (R6.2).
21 |  * - The locked result is what guarantees no #1 position or medal is awarded (R6.4);
22 |  *   that enforcement lives in the consuming UI, which must not render rankings
23 |  *   while this function returns `locked`.
24 |  *
25 |  * This function is pure and total: it returns a defined `LeaderboardState` for
26 |  * every finite numeric input. Non-finite inputs (NaN/Infinity) are treated
27 |  * conservatively as locked, since an undetermined cohort size must never reveal
28 |  * rankings.
29 |  *
30 |  * @param eligibleCount - Count of eligible (non-opted-out) students in the cohort.
31 |  * @param minCohort - Configured minimum cohort size (from `institution_settings`).
32 |  */
33 | export function leaderboardState(
34 |   eligibleCount: number,
35 |   minCohort: number
36 | ): LeaderboardState {
37 |   // Undetermined inputs must never unlock the leaderboard.
38 |   if (!Number.isFinite(eligibleCount) || !Number.isFinite(minCohort)) {
39 |     return "locked";
40 |   }
41 |
42 |   // R6.2a: zero (or negative) eligible students !0 always locked.
43 |   if (eligibleCount <= 0) return "locked";
44 |
45 |   // R6.1 / R6.2: gate on the configured minimum.
46 |   return eligibleCount >= minCohort ? "unlocked" : "locked";
47 | }
48 |

```

#### FILE: src/lib/leagueTier.ts

TYPE: .ts | LINES: 1-45 | SHA256: 8b11abcb38c1be01be445e3739533dd77eb362b2a3dd752bf25272d08e75637e

```

1 | // Task 148.2: League Tier utility
2 |
3 | export type LeagueTierName = "Bronze" | "Silver" | "Gold" | "Diamond";
4 |
5 | export interface LeagueThresholds {
6 |   bronze: number;
7 |   silver: number;
8 |   gold: number;

```

```

9 |     diamond: number;
10 | }
11 |
12 | export const DEFAULT_LEAGUE_THRESHOLDS: LeagueThresholds = {
13 |     bronze: 0,
14 |     silver: 500,
15 |     gold: 1500,
16 |     diamond: 4000,
17 | };
18 |
19 | /**
20 |  * Determine league tier based on cumulative XP.
21 |  */
22 | export function getLeagueTier(
23 |     cumulativeXP: number,
24 |     thresholds: LeagueThresholds = DEFAULT_LEAGUE_THRESHOLDS
25 | ): LeagueTierName {
26 |     if (cumulativeXP >= thresholds.diamond) return "Diamond";
27 |     if (cumulativeXP >= thresholds.gold) return "Gold";
28 |     if (cumulativeXP >= thresholds.silver) return "Silver";
29 |     return "Bronze";
30 | }
31 |
32 | export const TIER_COLORS: Record<LeagueTierName, string> = {
33 |     Bronze: "bg-amber-600 text-white",
34 |     Silver: "bg-gray-400 text-white",
35 |     Gold: "bg-yellow-400 text-yellow-900",
36 |     Diamond: "bg-blue-400 text-white",
37 | };
38 |
39 | export const TIER_BORDER_COLORS: Record<LeagueTierName, string> = {
40 |     Bronze: "border-amber-600",
41 |     Silver: "border-gray-400",
42 |     Gold: "border-yellow-400",
43 |     Diamond: "border-blue-400",
44 | };
45 |

```

FILE: [src/lib/leagueTierCalculator.ts](#)

TYPE: .ts | LINES: 1-125 | SHA256: 11f54c8aba3f7af85047b78325af88de016bfbabafabcb9bc519e0ae7415794cb

```

1 | // =====
2 | // League Tier Calculator – Pure function
3 | // Assigns league tiers from XP percentiles:
4 | //   Diamond: top 5%, Gold: top 20%, Silver: top 50%, Bronze: bottom 50%
5 | // =====
6 |
7 | /**
8 |  * League tier names (lowercase for DB storage).
9 |  */
10 | export type LeagueTier = "diamond" | "gold" | "silver" | "bronze";
11 |
12 | export interface StudentXP {
13 |     studentId: string;
14 |     xpTotal: number;
15 | }
16 |
17 | export interface TierAssignment {
18 |     studentId: string;
19 |     xpTotal: number;
20 |     tier: LeagueTier;
21 |     /** Percentile rank (0 = highest XP, 1 = lowest XP) */
22 |     percentileRank: number;
23 | }
24 |
25 | /**
26 |  * Tier thresholds as percentile rank cutoffs.
27 |  * PERCENT_RANK() returns 0 for the top student and approaches 1 for the lowest.
28 |  */
29 | const TIER_THRESHOLDS: ReadonlyArray<{
30 |     maxPercentile: number;
31 |     tier: LeagueTier;
32 | }> = [
33 |     { maxPercentile: 0.05, tier: "diamond" },
34 |     { maxPercentile: 0.2, tier: "gold" },
35 |     { maxPercentile: 0.5, tier: "silver" },
36 |     { maxPercentile: 1.0, tier: "bronze" },
37 | ];
38 |
39 | /**
40 |  * Assign a league tier based on a percentile rank value.
41 |  *
42 |  * @param percentileRank - Value between 0 (top) and 1 (bottom), matching
43 |  *   SQL PERCENT_RANK() semantics (0 = highest XP).
44 |  */
45 | export function assignTierFromPercentile(percentileRank: number): LeagueTier {
46 |     const clamped = Math.max(0, Math.min(1, percentileRank));
47 |     for (const { maxPercentile, tier } of TIER_THRESHOLDS) {
48 |         if (clamped <= maxPercentile) return tier;
49 |     }
50 |     return "bronze";
51 | }
52 |
53 | /**
54 |  * Compute PERCENT_RANK for a sorted array of students.
55 |  * Students are ranked by XP descending. Ties share the same rank.
56 |  *
57 |  * PERCENT_RANK = (rank - 1) / (N - 1) where rank is 0-based dense rank.
58 |  * For a single student, percentile rank is 0.
59 |  */
60 | function computePercentileRanks(
61 |     students: StudentXP[]
62 | ): Array<{ studentId: string; xpTotal: number; percentileRank: number }> {
63 |     if (students.length === 0) return [];

```

```

64 |
65 | const first = students[0];
66 | if (students.length === 1 && first) {
67 |   return [
68 |     { studentId: first.studentId, xpTotal: first.xpTotal, percentileRank: 0 },
69 |   ];
70 | }
71 |
72 | // Sort descending by XP
73 | const sorted = [...students].sort((a, b) => b.xpTotal - a.xpTotal);
74 | const n = sorted.length;
75 |
76 | const result: Array<{
77 |   studentId: string;
78 |   xpTotal: number;
79 |   percentileRank: number;
80 | }> = [];
81 |
82 | for (let i = 0; i < n; i++) {
83 |   const current = sorted[i];
84 |   if (!current) continue;
85 |
86 |   // Find the first occurrence of this XP value (for tie handling)
87 |   let rank = i;
88 |   while (rank > 0) {
89 |     const prev = sorted[rank - 1];
90 |     if (!prev || prev.xpTotal !== current.xpTotal) break;
91 |     rank--;
92 |   }
93 |   const percentileRank = rank / (n - 1);
94 |   result.push({
95 |     studentId: current.studentId,
96 |     xpTotal: current.xpTotal,
97 |     percentileRank,
98 |   });
99 | }
100 |
101 | return result;
102 | }
103 |
104 | /**
105 |  * Assign league tiers to all students based on their XP totals.
106 |  *
107 |  * - Diamond: top 5% (percentile rank "d 0.05)
108 |  * - Gold: top 20% (percentile rank "d 0.20)
109 |  * - Silver: top 50% (percentile rank "d 0.50)
110 |  * - Bronze: bottom 50% (percentile rank > 0.50)
111 |  *
112 |  * Every student receives exactly one tier. Tier boundaries are non-overlapping
113 |  * and exhaustive.
114 |  */
115 | export function assignLeagueTiers(students: StudentXP[]): TierAssignment[] {
116 |   const ranked = computePercentileRanks(students);
117 |
118 |   return ranked.map(({ studentId, xpTotal, percentileRank }) => ({
119 |     studentId,
120 |     xpTotal,
121 |     tier: assignTierFromPercentile(percentileRank),
122 |     percentileRank,
123 |   }));
124 | }
125 |

```

## FILE: src/lib/learningPathPresentation.ts

TYPE: .ts | LINES: 1-55 | SHA256: bc28781ce349bd30fd170c72931dfc37f71394f6d5dd4aeb759d9387be75187c

```

1 | import type { LearningPathNode } from "@hooks/useLearningPath";
2 | import type { BloomsLevel } from "@lib/schemas/clo";
3 |
4 | export type LearningStageStatus = "done" | "current" | "locked";
5 |
6 | export interface BloomStageSummary {
7 |   key: BloomsLevel;
8 |   nodes: LearningPathNode[];
9 |   status: LearningStageStatus;
10 |   attainment: number;
11 | }
12 |
13 | const BLOOM_ORDER: readonly BloomsLevel[] = [
14 |   "remembering",
15 |   "understanding",
16 |   "applying",
17 |   "analyzing",
18 |   "evaluating",
19 |   "creating",
20 | ] as const;
21 |
22 | export const averagePercentage = (values: number[]): number =>
23 |   values.length === 0
24 |     ? 0
25 |     : Math.round(values.reduce((sum, value) => sum + value, 0) / values.length);
26 |
27 | /**
28 |  * Converts backend learning-path nodes into the six-stage Bloom journey.
29 |  * Components only decide how these summaries look; progression logic stays in
30 |  * lib so Journey and Knowledge Tree can never disagree.
31 |  */
32 | export const buildBloomStageSummaries = (
33 |   nodes: LearningPathNode[]
34 | ): BloomStageSummary[] => {
35 |   BLOOM_ORDER.map((key) => {
36 |     const stageNodes = nodes.filter((node) => node.blooms_level === key);
37 |     const graded =
38 |       stageNodes.length > 0 &&

```

```

39 |         stageNodes.every((node) => node.status === "graded");
40 |         const active = stageNodes.some(
41 |             (node) => node.status === "available" || node.status === "submitted"
42 |         );
43 |
44 |         return {
45 |             key,
46 |             nodes: stageNodes,
47 |             status: graded ? "done" : active ? "current" : "locked",
48 |             attainment: averagePercentage(
49 |                 stageNodes.flatMap((node) =>
50 |                     node.attainment_percent === null ? [] : [node.attainment_percent]
51 |                 )
52 |             ),
53 |         };
54 |     });
55 |

```

## FILE: src/lib/letterGradeMapper.ts

TYPE: .ts | LINES: 1-50 | SHA256: 2028272645fd069a854301a5cac67e8a317237efed8998370e55c6784703f8b9

```

1 | // =====
2 | // letterGradeMapper - Maps percentage scores to letter grades using grade_scales
3 | // =====
4 |
5 | import type { GradeScale } from "@types/app";
6 | import { DEFAULT_GRADE_SCALES } from "@types/app";
7 |
8 | /**
9 |  * Maps a percentage score to a letter grade using the institution's grade scale.
10 |  * Grade scales are sorted by min_percent descending so the first match wins.
11 |  * Falls back to DEFAULT_GRADE_SCALES when no scales are provided.
12 |  */
13 | export function mapToLetterGrade(
14 |     percent: number,
15 |     gradeScales: GradeScale[] = DEFAULT_GRADE_SCALES
16 | ): string {
17 |     const scales = gradeScales.length > 0 ? gradeScales : DEFAULT_GRADE_SCALES;
18 |     const sorted = [...scales].sort((a, b) => b.min_percent - a.min_percent);
19 |
20 |     for (const scale of sorted) {
21 |         if (percent >= scale.min_percent && percent <= scale.max_percent) {
22 |             return scale.letter;
23 |         }
24 |     }
25 |
26 |     // If no match found (shouldn't happen with proper scales), return lowest grade
27 |     const lowest = sorted[sorted.length - 1];
28 |     return lowest?.letter ?? "F";
29 | }
30 |
31 | /**
32 |  * Maps a percentage score to GPA points using the institution's grade scale.
33 |  */
34 | export function mapToGpaPoints(
35 |     percent: number,
36 |     gradeScales: GradeScale[] = DEFAULT_GRADE_SCALES
37 | ): number {
38 |     const scales = gradeScales.length > 0 ? gradeScales : DEFAULT_GRADE_SCALES;
39 |     const sorted = [...scales].sort((a, b) => b.min_percent - a.min_percent);
40 |
41 |     for (const scale of sorted) {
42 |         if (percent >= scale.min_percent && percent <= scale.max_percent) {
43 |             return scale.gpa_points;
44 |         }
45 |     }
46 |
47 |     const lowest = sorted[sorted.length - 1];
48 |     return lowest?.gpa_points ?? 0;
49 | }
50 |

```

## FILE: src/lib/levelAwareHeatmap.ts

TYPE: .ts | LINES: 1-72 | SHA256: 53cc658e23f75128330a9db53fd91acdf51303b82a04e24cbb2d8e0a96fab973

```

1 | import type { LevelProgressionPoint } from "@types/habits";
2 |
3 | /**
4 |  * Returns the max number of academic habits expected per day for a given level.
5 |  * Level 1 !' 1, Level 2 !' 2, Level 3 !' 3, Level 4 !' 4.
6 |  */
7 | export const getLevelMaxHabits = (level: 1 | 2 | 3 | 4): number => level;
8 |
9 | /**
10 |  * Computes heatmap cell intensity relative to the student's level max.
11 |  * Returns 0-4 intensity level.
12 |  *
13 |  * - count=0 or levelMax=0 !' 0
14 |  * - ratio "d 0.25 !' 1
15 |  * - ratio "d 0.5 !' 2
16 |  * - ratio "d 0.75 !' 3
17 |  * - ratio > 0.75 !' 4
18 |  */
19 | export const getLevelAwareIntensityLevel = (
20 |     count: number,
21 |     levelMax: number
22 | ): number => {
23 |     if (count === 0 || levelMax === 0) return 0;
24 |     const ratio = count / levelMax;
25 |     if (ratio <= 0.25) return 1;
26 |     if (ratio <= 0.5) return 2;
27 |     if (ratio <= 0.75) return 3;
28 |     return 4;
29 | };

```



```

30 |
31 | /**
32 |  * Finds the student's habit difficulty level on a given date by scanning
33 |  * the level history (sorted by date ascending). Returns the most recent
34 |  * level change on or before the given date. Defaults to level 4 if no
35 |  * history entry precedes the date.
36 |  */
37 | export const getLevelForDate = (
38 |   date: string,
39 |   history: LevelProgressionPoint[]
40 | ): 1 | 2 | 3 | 4 => {
41 |   let level: 1 | 2 | 3 | 4 = 4;
42 |   for (const point of history) {
43 |     if (point.date <= date) {
44 |       level = point.level;
45 |     }
46 |   }
47 |   return level;
48 | };
49 |
50 | /**
51 |  * Computes a level-relative consistency score:
52 |  * - Excludes sabbatical dates from both numerator and denominator
53 |  * - A day is "fully completed" when academicCount >= the level active on that date
54 |  * - Returns 0-100 (percentage)
55 |  * - Returns 0 when all days are sabbatical days
56 |  */
57 | export const computeLevelRelativeConsistencyScore = (
58 |   days: Array<{ date: string; academicCount: number }>,
59 |   levelHistory: LevelProgressionPoint[],
60 |   sabbaticalDates: Set<string>
61 | ): number => {
62 |   const eligibleDays = days.filter((d) => !sabbaticalDates.has(d.date));
63 |   if (eligibleDays.length === 0) return 0;
64 |
65 |   const fullyCompletedDays = eligibleDays.filter((d) => {
66 |     const levelOnDate = getLevelForDate(d.date, levelHistory);
67 |     return d.academicCount >= levelOnDate;
68 |   }).length;
69 |
70 |   return Math.round((fullyCompletedDays / eligibleDays.length) * 100);
71 | };
72 |

```

FILE: [src/lib/localization.ts](#)

TYPE: .ts | LINES: 1-127 | SHA256: 7d4b9eb24ed562f20adc87eb78729793f13acf0841499a2484175834f4edf1cf

```

1 | // =====
2 | // localization - runtime localization-policy helper
3 | // =====
4 | //
5 | // `resolveLocalizedOrFail` centralizes how a surface resolves an i18n key when
6 | // the active language may not have it. Surfaces declare their policy explicitly
7 | // instead of relying on i18next's single global `fallbackLng` chain.
8 | //
9 | // The younger-student wording (R22) opts into cross-language fallback: when a
10 | // Student selects English at runtime but only the Arabic localization resolves,
11 | // the wording falls back to the available language rather than failing to
12 | // display (R22.6). Build-time presence validation (R22.5) guarantees both
13 | // localizations normally exist, so the runtime fallback is an edge case.
14 |
15 | import i18n, { supportedLanguages } from "@lib/i18n";
16 |
17 | export interface ResolveLocalizedOptions {
18 |   /**
19 |    * When the active language lacks the key, fall back to the other supported
20 |    * language before giving up (R22.6).
21 |    */
22 |   fallbackToOtherLang?: boolean;
23 |   /** i18next namespace the key belongs to. Defaults to the configured default. */
24 |   ns?: string;
25 |   /** Interpolation values passed through to i18next. */
26 |   params?: Record<string, unknown>;
27 | }
28 |
29 | /**
30 |  * Resolves an i18n key with an explicit cross-language fallback policy.
31 |  *
32 |  * Returns the resolved, interpolated string, or `null` when the key cannot be
33 |  * resolved in any supported language - letting callers decide how to surface
34 |  * the gap rather than silently rendering the raw key.
35 |  */
36 | export function resolveLocalizedOrFail(
37 |   key: string,
38 |   options: ResolveLocalizedOptions = {}
39 | ): string | null {
40 |   const { fallbackToOtherLang = false, ns, params } = options;
41 |   const baseOptions: Record<string, unknown> = {
42 |     ...(ns ? { ns } : {}),
43 |     ...(params ?? {}),
44 |   };
45 |
46 |   // 1) Active language (honoring i18next's own fallback chain).
47 |   if (i18n.exists(key, baseOptions)) {
48 |     return asString(i18n.t(key, baseOptions));
49 |   }
50 |
51 |   // 2) Explicit fallback to the other supported language (R22.6).
52 |   if (fallbackToOtherLang) {
53 |     const active = (i18n.language ?? "").split("-")[0];
54 |     for (const lng of supportedLanguages) {
55 |       if (lng === active) continue;
56 |       const withLng = { ...baseOptions, lng };
57 |       if (i18n.exists(key, withLng)) {

```

```

58     }
59 }
60 }
61 }
62 }
63 return null;
64 }
65
66 /** Coerces an i18next return value to a string, or null when it is not a string. */
67 function asString(value: unknown): string | null {
68   return typeof value === "string" ? value : null;
69 }
70
71 // =====
72 // Surface-level localization gate (R13.5, R19.6, R19.7)
73 // =====
74 //
75 // Whereas `resolveLocalizedOrFail` decides how a *single key* resolves, a
76 // surface (the timetable, the weekly planner) needs a coarser decision based on
77 // whether its required language packs are loaded at all. Centralizing it here -
78 // alongside the per-key policy - keeps every i18n "hard-fail vs fallback vs
79 // surface-gate" decision in one module, so each surface declares its policy
80 // explicitly instead of re-deriving en/ar availability ad hoc.
81
82 /**
83  * How a surface reacts to partial language-pack availability:
84  * - `block-only` - display whenever at least one supported pack is
85  *   available; block only when none is (timetable, R13.5).
86  * - `withhold-actions` - display read-only content when exactly one pack is
87  *   available but withhold mutating actions until both are
88  *   present (weekly planner, R19.6).
89  */
90 export type LocalizationGatePolicy = "block-only" | "withhold-actions";
91
92 /**
93  * Resolved gate state for a localized surface:
94  * - `ready` - all supported packs present; full content + actions.
95  * - `actions-withheld` - partial availability under `withhold-actions`; render
96  *   read-only content but suppress actions (R19.6).
97  * - `blocked` - no supported pack present; hide/block the surface
98  *   entirely rather than render unlocalized (R13.5, R19.7).
99  */
100 export type LocalizationGateState = "ready" | "actions-withheld" | "blocked";
101
102 /**
103  * Decide a surface's localization gate from per-language pack availability.
104  *
105  * Pure: the caller injects the availability predicate (e.g.
106  * `(lng) => i18n.hasResourceBundle(lng, "student")`) so this stays testable and
107  * free of i18next state. The decision spans every supported language, so the
108  * gate stays correct if a third language is ever added.
109  */
110 export function resolveLocalizationGate(
111   hasBundle: (language: string) => boolean,
112   policy: LocalizationGatePolicy = "withhold-actions"
113 ): LocalizationGateState {
114   const availableCount = supportedLanguages.filter((lng) =>
115     hasBundle(lng)
116   ).length;
117
118   // No supported pack loaded !' block/hide the surface entirely (R13.5, R19.7).
119   if (availableCount === 0) return "blocked";
120
121   // Every supported pack loaded !' full content and actions.
122   if (availableCount === supportedLanguages.length) return "ready";
123
124   // Partial availability: withhold actions (planner) or display anyway (timetable).
125   return policy === "withhold-actions" ? "actions-withheld" : "ready";
126 }

```

FILE: src/lib/loginAttemptTracker.ts

TYPE: .ts | LINES: 1-209 | SHA256: 243f6d45840405840f140cbd285ee445c54f6789e4b759a1b2908a6753d8bd75

```

1 // -----
2 // Login Attempt Tracker – client-side lockout via localStorage + server-side
3 // -----
4 // Client-side tracking provides immediate UX feedback. Server-side tracking
5 // via the check-login-rate Edge Function provides tamper-proof enforcement.
6 // -----
7
8 import { supabase } from "@/lib/supabase";
9
10 const STORAGE_PREFIX = "login_attempts_";
11 const MAX_ATTEMPTS = 5;
12 const LOCKOUT_DURATION_MS = 15 * 60 * 1000; // 15 minutes
13
14 // % % %   Server-Side Rate Limit Types % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
15
16 interface ServerRateLimitCheckResult {
17     locked: boolean;
18     remaining_seconds: number;
19 }
20
21 interface ServerRecordFailureResult {
22     locked: boolean;
23     remaining_seconds: number;
24     attempt_count: number;
25 }
26
27 interface ServerClearResult {
28     cleared: boolean;
29 }

```

```

31 | interface AttemptRecord {
32 |   count: number;
33 |   lockedUntil: string | null; // ISO timestamp or null
34 | }
35 |
36 | function storageKey(email: string): string {
37 |   return `${STORAGE_PREFIX}${email.toLowerCase().trim()}`;
38 | }
39 |
40 | function readRecord(email: string): AttemptRecord {
41 |   try {
42 |     const raw = localStorage.getItem(storageKey(email));
43 |     if (!raw) return { count: 0, lockedUntil: null };
44 |     const parsed: unknown = JSON.parse(raw);
45 |     if (
46 |       typeof parsed === "object" &&
47 |       parsed !== null &&
48 |       "count" in parsed &&
49 |       typeof (parsed as AttemptRecord).count === "number"
50 |     ) {
51 |       return parsed as AttemptRecord;
52 |     }
53 |     return { count: 0, lockedUntil: null };
54 |   } catch {
55 |     return { count: 0, lockedUntil: null };
56 |   }
57 | }
58 |
59 | function writeRecord(email: string, record: AttemptRecord): void {
60 |   try {
61 |     localStorage.setItem(storageKey(email), JSON.stringify(record));
62 |   } catch {
63 |     // Storage full or unavailable – fail silently
64 |   }
65 | }
66 |
67 | /**
68 |  * Returns `true` when the account is currently locked out.
69 |  */
70 | export function isLocked(email: string): boolean {
71 |   const record = readRecord(email);
72 |   if (!record.lockedUntil) return false;
73 |   const lockedUntil = new Date(record.lockedUntil).getTime();
74 |   if (Date.now() >= lockedUntil) {
75 |     // Lock expired – clear it
76 |     clearAttempts(email);
77 |     return false;
78 |   }
79 |   return true;
80 | }
81 |
82 | /**
83 |  * Remaining lock time in whole seconds (0 when not locked).
84 |  */
85 | export function getRemainingLockTime(email: string): number {
86 |   const record = readRecord(email);
87 |   if (!record.lockedUntil) return 0;
88 |   const remaining = new Date(record.lockedUntil).getTime() - Date.now();
89 |   return remaining > 0 ? Math.ceil(remaining / 1000) : 0;
90 | }
91 |
92 | /**
93 |  * Record a failed login attempt. Locks the account when the threshold is hit.
94 |  */
95 | export function recordFailedAttempt(email: string): void {
96 |   const record = readRecord(email);
97 |   const newCount = record.count + 1;
98 |
99 |   const lockedUntil =
100 |     newCount >= MAX_ATTEMPTS
101 |       ? new Date(Date.now() + LOCKOUT_DURATION_MS).toISOString()
102 |       : record.lockedUntil;
103 |
104 |   writeRecord(email, { count: newCount, lockedUntil });
105 | }
106 |
107 | /**
108 |  * Clear all attempt data for an email (call on successful login).
109 |  */
110 | export function clearAttempts(email: string): void {
111 |   try {
112 |     localStorage.removeItem(storageKey(email));
113 |   } catch {
114 |     // Fail silently
115 |   }
116 | }
117 |
118 | // -----
119 | // Server-Side Rate Limiting – calls check-login-rate Edge Function
120 | // -----
121 |
122 | async function callRateLimitEdgeFunction(
123 |   email: string,
124 |   action: "check" | "record_failure" | "clear"
125 | ): Promise<unknown> {
126 |   const { data, error } = await supabase.functions.invoke("check-login-rate", {
127 |     body: { email: email.toLowerCase().trim(), action },
128 |   });
129 |
130 |   if (error) {
131 |     console.error(`Server rate limit ${action} failed:`, error.message);
132 |     return null;
133 |   }
134 | }

```

```

135 |     return data;
136 | }
137 |
138 | /**
139 |  * Check server-side rate limit status for an email.
140 |  * Returns lock status and remaining seconds. Falls back gracefully
141 |  * if the edge function is unavailable (returns unlocked).
142 |  */
143 | export async function checkServerRateLimit(
144 |   email: string
145 | ): Promise<ServerRateLimitCheckResult> {
146 |   const result = await callRateLimitEdgeFunction(email, "check");
147 |
148 |   if (
149 |     result &&
150 |     typeof result === "object" &&
151 |     "locked" in result &&
152 |     typeof (result as ServerRateLimitCheckResult).locked === "boolean"
153 |   ) {
154 |     return result as ServerRateLimitCheckResult;
155 |   }
156 |
157 |   // Fallback: if server is unreachable, don't block login
158 |   return { locked: false, remaining_seconds: 0 };
159 | }
160 |
161 | /**
162 |  * Record a failed login attempt on the server.
163 |  * Returns updated lock status. Falls back gracefully.
164 |  */
165 | export async function recordServerFailedAttempt(
166 |   email: string
167 | ): Promise<ServerRecordFailureResult> {
168 |   const result = await callRateLimitEdgeFunction(email, "record_failure");
169 |
170 |   if (
171 |     result &&
172 |     typeof result === "object" &&
173 |     "locked" in result &&
174 |     typeof (result as ServerRecordFailureResult).locked === "boolean"
175 |   ) {
176 |     return result as ServerRecordFailureResult;
177 |   }
178 |
179 |   return { locked: false, remaining_seconds: 0, attempt_count: 0 };
180 | }
181 |
182 | /**
183 |  * Clear server-side login attempts for an email (call on successful login).
184 |  * Falls back gracefully if the edge function is unavailable.
185 |  */
186 | export async function clearServerAttempts(
187 |   email: string
188 | ): Promise<ServerClearResult> {
189 |   const result = await callRateLimitEdgeFunction(email, "clear");
190 |
191 |   if (
192 |     result &&
193 |     typeof result === "object" &&
194 |     "cleared" in result &&
195 |     typeof (result as ServerClearResult).cleared === "boolean"
196 |   ) {
197 |     return result as ServerClearResult;
198 |   }
199 |
200 |   return { cleared: false };
201 | }
202 |
203 | // Re-export constants for testing
204 | export const LOGIN_ATTEMPT_CONFIG = {
205 |   MAX_ATTEMPTS,
206 |   LOCKOUT_DURATION_MS,
207 |   STORAGE_PREFIX,
208 | } as const;
209 |

```

FILE: [src/lib/mapSupabaseError.ts](#)

TYPE: .ts | LINES: 1-111 | SHA256: 8009f24551ff05f35a8f0422590c168df7d238be226fb24a331850d8391ba371

```

1 | /**
2 |  * Mapped error response
3 |  */
4 | export interface MappedError {
5 |   code: string;
6 |   userMessage: string;
7 |   fieldPath?: string;
8 | }
9 |
10 | /**
11 |  * Map of PostgreSQL constraint names to user-friendly messages
12 |  *
13 |  * These messages are in English; consumers should call t(userMessage) to localize
14 |  */
15 | const CONSTRAINT_MESSAGES: Record<string, Omit<MappedError, "code">> = {
16 |   week_start_is_monday: {
17 |     userMessage: "Week start must be a Monday",
18 |     fieldPath: "week_start",
19 |   },
20 |   // Add more constraint mappings as needed
21 | };
22 |
23 | /**
24 |  * Map PostgreSQL error codes to user-friendly messages
25 |  */

```

[illegible]

[illegible]

[illegible]

FILE: src/lib/mascotGuidance.ts

TYPE: .ts | LINES: 1-120 | SHA256: 7089ec097f45e58483ae35d053f17488be5efabbe66bd7ef36b62995c583fd88

```

1 // =====
2 // Mascot Guidance – pure moment!'copy mapping logic (R35)
3 // =====
4 //
5 // The mascot acts as a coach that appears at high-value points in the student
6 // journey (welcome, assessment intros, empty states, first XP award, first
7 // enrollment, and password/auth screens). This module owns the *decision* of
8 // whether – and with what copy reference and tone – the mascot should speak at
9 // a given moment, kept pure and React/Supabase-free so it is fully unit- and

```

```

10 // property-testable.
11 //
12 // Two invariants from the requirements are encoded here:
13 //   • R35.4 – when guidance is disabled, no moment ever produces guidance.
14 //   • R35.5 – when no key moment is active, no guidance is produced; the
15 //     platform must function fully regardless of the guidance state.
16 //
17 // The actual bilingual copy lives in i18next (`common` namespace, `mascot.`
18 // keys) so it is always available in both English and Arabic (R35.3); this
19 // module only resolves the *key* to read, never the literal string.
20
21 /** The discrete key journey moments at which the mascot can coach (R35.1, R35.2). */
22 export type MascotMomentId =
23   | "welcome"
24   | "assessmentIntro"
25   | "emptyState"
26   | "firstXp"
27   | "firstEnrollment"
28   | "password";
29
30 /**
31  * Coaching tone for a moment. The presentation layer maps each tone to an icon
32  * and accent so the logic stays decoupled from visuals.
33  */
34 export type MascotTone = "cheer" | "guide" | "encourage";
35
36 /** Resolved guidance descriptor for an active, enabled moment. */
37 export interface MascotGuidance {
38   /** The moment this guidance is for. */
39   readonly moment: MascotMomentId;
40   /** i18next key base whose `.title` / `.message` children hold the copy. */
41   readonly i18nKey: string;
42   /** Coaching tone, used by the UI to pick an icon and accent. */
43   readonly tone: MascotTone;
44 }
45
46 interface MascotMomentDescriptor {
47   readonly i18nKey: string;
48   readonly tone: MascotTone;
49 }
50
51 /** All moment ids, ordered, for iteration in tests and validation. */
52 export const MASCOT_MOMENT_IDS: readonly MascotMomentId[] = [
53   "welcome",
54   "assessmentIntro",
55   "emptyState",
56   "firstXp",
57   "firstEnrollment",
58   "password",
59 ] as const;
60
61 /**
62  * Static moment! copy mapping. Each moment points at a `common`-namespace key
63  * base (`mascot.moments.<moment>`) with `.title` and `.message` children, and
64  * carries a coaching tone.
65  */
66 export const MASCOT_MOMENTS: Readonly<
67   Record<MascotMomentId, MascotMomentDescriptor>
68 > = {
69   welcome: { i18nKey: "mascot.moments.welcome", tone: "cheer" },
70   assessmentIntro: { i18nKey: "mascot.moments.assessmentIntro", tone: "guide" },
71   emptyState: { i18nKey: "mascot.moments.emptyState", tone: "guide" },
72   firstXp: { i18nKey: "mascot.moments.firstXp", tone: "cheer" },
73   firstEnrollment: {
74     i18nKey: "mascot.moments.firstEnrollment",
75     tone: "encourage",
76   },
77   password: { i18nKey: "mascot.moments.password", tone: "encourage" },
78 } as const;
79
80 /**
81  * Resolve the mascot guidance for a moment.
82  *
83  * Returns `null` – meaning "render nothing, the surface works without the
84  * mascot" – when guidance is disabled (R35.4) or when no key moment is active
85  * (R35.5). Otherwise returns the descriptor for the active moment.
86  */
87 export function resolveMascotGuidance(input: {
88   readonly enabled: boolean;
89   readonly moment: MascotMomentId | null;
90 }): MascotGuidance | null {
91   if (!input.enabled) return null;
92   if (input.moment === null) return null;
93
94   const descriptor = MASCOT_MOMENTS[input.moment];
95   // Defensive: an unknown moment id (e.g. from loosely-typed callers) yields
96   // no guidance rather than throwing, preserving the non-blocking guarantee.
97   if (!descriptor) return null;
98
99   return {
100     moment: input.moment,
101     i18nKey: descriptor.i18nKey,
102     tone: descriptor.tone,
103   };
104 }
105
106 /** Default mascot-guidance enablement when the student has no stored preference. */
107 export const MASCOT_ENABLED_DEFAULT = true;
108
109 /**
110  * Parse a stored mascot-enabled preference value into a boolean. Pure so the
111  * (localStorage) read can live in the hook layer while parsing stays testable.
112  * Unknown/malformed values fall back to the default so the feature degrades
113  * gracefully rather than breaking the surface.

```



```
114 | */
115 | export function parseMascotEnabled(raw: string | null): boolean {
116 |   if (raw === "true") return true;
117 |   if (raw === "false") return false;
118 |   return MASCOT_ENABLED_DEFAULT;
119 | }
120 |
```

FILE: src/lib/masteryRecovery.ts

```
TYPE: .ts | LINES: 1-70 | SHA256: 6cbff3509be94a79980e5acf7ec5d1003a3a46f23ae2d74a2acd10625a5d376e
```

```

1 // % % % Mastery Recovery % % % % % % % % % % % % % % % %
2 // Pure functions for mastery gate failure detection and recovery pathway logic.
3
4 /** A quiz attempt with per-CLO scores (0-100 scale). */
5 export interface QuizAttemptCLOResult {
6   clo_scores: Record<string, number>;
7 }
8
9 /** A recovery pathway session tracking step completion. */
10 export interface RecoveryPathway {
11   ai_tutor_completed: boolean;
12   practice_completed: boolean;
13 }
14
15 /**
16  * Counts the number of quiz attempts where the student's per-CLO score
17  * fell below the mastery threshold for a given CLO.
18  *
19  * @param attempts - Array of quiz attempt results with per-CLO scores.
20  * @param cloId - The CLO identifier to check.
21  * @param masteryThreshold - Minimum passing score (default 70).
22  * @returns Number of attempts where the CLO score was below threshold.
23  */
24 export function countMasteryFailures(
25   attempts: QuizAttemptCLOResult[],
26   cloId: string,
27   masteryThreshold: number = 70
28 ): number {
29   return attempts.filter((a) => (a.clo_scores[cloId] ?? 0) < masteryThreshold)
30     .length;
31 }
32
33 /**
34  * Determines whether a mastery recovery pathway should be activated.
35  * Returns true when the failure count reaches or exceeds the threshold.
36  *
37  * @param failureCount - Number of mastery failures for a student-CLO pair.
38  * @param threshold - Minimum failures to trigger recovery (default 2).
39  * @returns Whether recovery should be activated.
40  */
41 export function shouldActivateRecovery(
42   failureCount: number,
43   threshold: number = 2
44 ): boolean {
45   return failureCount >= threshold;
46 }
47
48 /**
49  * Determines the Bloom's level for recovery practice questions.
50  * Returns one level below the CLO's target Bloom's level, floored at 1 (Remembering).
51  *
52  * @param cloBloomLevel - The CLO's target Bloom's level (1-6).
53  * @returns The Bloom's level for recovery practice (1-5).
54  */
55 export function recoveryBloomLevel(cloBloomLevel: number): number {
56   return Math.max(1, cloBloomLevel - 1);
57 }
58
59 /**
60  * Checks whether a recovery pathway is complete.
61  * Both the AI Tutor session and practice questions must be completed
62  * before the student can retry the mastery gate.
63  *
64  * @param pathway - The recovery pathway with step completion flags.
65  * @returns Whether the required recovery steps are complete.
66  */
67 export function isRecoveryComplete(pathway: RecoveryPathway): boolean {
68   return pathway.ai_tutor_completed && pathway.practice_completed;
69 }

```

FILE: src/lib/mostImprovedLeaderboard.ts

```
TYPE: .ts | LINES: 1-59 | SHA256: e33957a5ee3839290e484323e14ca6d9013dd11c1f3b158454fe149130082150
```

```

1 // =====
2 // Most Improved leaderboard utility (Task 147.2)
3 // Calculates improvement: (current_4_week_xp - previous_4_week_xp) / previous_4_week_xp * 100
4 // Excludes students with zero previous XP, returns top 20.
5 // Requirements: 130.1, 130.2, 130.3, 130.4
6 // =====
7
8 export interface MostImprovedEntry {
9   student_id: string;
10  student_name: string;
11  current_4_week_xp: number;
12  previous_4_week_xp: number;
13  improvement_percent: number;
14  xp_delta: number;
15 }
16
17 /**
18  * Calculate improvement percentage.
19  * Formula: (current - previous) / previous * 100

```

```

20 | * Returns null when previous XP is zero (excluded per Requirement 130.3).
21 | */
22 | export function calculateImprovement(
23 |   current4WeekXP: number,
24 |   previous4WeekXP: number
25 | ): number | null {
26 |   if (previous4WeekXP <= 0) return null;
27 |   return ((current4WeekXP - previous4WeekXP) / previous4WeekXP) * 100;
28 | }
29 |
30 | /**
31 |  * Rank students by improvement and return top 20.
32 |  * Excludes students with zero or negative previous XP.
33 |  */
34 | export function rankMostImproved(
35 |   entries: Array<{
36 |     student_id: string;
37 |     student_name: string;
38 |     current_4_week_xp: number;
39 |     previous_4_week_xp: number;
40 |   }>
41 | ): MostImprovedEntry[] {
42 |   return entries
43 |     .map((e) => {
44 |       const improvement = calculateImprovement(
45 |         e.current_4_week_xp,
46 |         e.previous_4_week_xp
47 |       );
48 |       if (improvement === null) return null;
49 |       return {
50 |         ...e,
51 |         improvement_percent: Math.round(improvement * 100) / 100,
52 |         xp_delta: e.current_4_week_xp - e.previous_4_week_xp,
53 |       };
54 |     })
55 |     .filter((e): e is MostImprovedEntry => e !== null)
56 |     .sort((a, b) => b.improvement_percent - a.improvement_percent)
57 |     .slice(0, 20);
58 | }
59 |

```

## FILE: src/lib/motionGate.ts

TYPE: .ts | LINES: 1-100 | SHA256: 267fbc04fac7088c3dcb5185ce5befda41337390040cd5b3532ed7709ed2fd56

```

1 | // =====
2 | // motionGate – shared reduced-motion gating for Framer Motion (R31.2, R31.2a)
3 | // =====
4 | //
5 | // Student-facing surfaces use Framer Motion for entrance/transition animations.
6 | // Requirement 31 mandates that, while the OS/browser signals a reduced-motion
7 | // preference, NEW animations are suppressed (R31.2) – and specifically that new
8 | // animations are suppressed *while any already-running animation is allowed to
9 | // complete naturally* rather than being snapped mid-flight (R31.2a). The gating
10 | // must live in JavaScript-driven behaviour, not only CSS (R31.3).
11 | //
12 | // `framer-motion`'s `<MotionConfig reducedMotion="user">` (mounted once in
13 | // App.tsx) already reduces transform/layout animations globally. This helper
14 | // complements that by giving components a single, consistent way to express
15 | // "this is a NEW entrance animation – start it in its final visual state when
16 | // the user prefers reduced motion" without each component re-deriving the logic.
17 | //
18 | // The contract is intentionally small and pure-per-render:
19 | // - `prefersReducedMotion` – the resolved boolean preference.
20 | // - `enter(initial, animate)` – returns the `initial` prop a `motion.*` should
21 | //   use: the supplied `initial` normally, or the `animate` target when reduced
22 | //   motion is preferred (so the element appears already-settled – no entrance).
23 | // - `transition(transition)` – returns an instant transition under reduced
24 | //   motion, or the supplied transition otherwise.
25 | //
26 | // Why this satisfies R31.2a: gating only the `initial`/`transition` of *newly
27 | // mounted* nodes means an animation that is already in-flight when the
28 | // preference is read is never interrupted – we only prevent the next entrance
29 | // from animating. We never call `stop()` or force a node to its end value
30 | // mid-animation.
31 | // =====
32 |
33 | import {
34 |   useReducedMotion,
35 |   type TargetAndTransition,
36 |   type Transition,
37 | } from "framer-motion";
38 |
39 | /** A Framer Motion target object usable as an `initial`/`animate` value. */
40 | export type MotionTarget = TargetAndTransition;
41 |
42 | export interface GatedMotion {
43 |   /**
44 |    * Resolved reduced-motion preference for the current environment.
45 |    * `true` when the user has requested reduced motion.
46 |    */
47 |   readonly prefersReducedMotion: boolean;
48 |
49 |   /**
50 |    * Resolve the `initial` prop for a NEW entrance animation.
51 |    *
52 |    * - Normal motion: returns `initialValue` so the element animates in.
53 |    * - Reduced motion: returns `animateValue` (the settled, final state) so the
54 |    *   element is rendered already in place with no entrance animation (R31.2).
55 |    *
56 |    * @param initialValue - The off-screen / pre-animation state.
57 |    * @param animateValue - The final, settled state the element animates to.
58 |    */
59 |   enter(initialValue: MotionTarget, animateValue: MotionTarget): MotionTarget;
60 |

```

```

61 | /**
62 |  * Resolve a transition for a NEW animation. Under reduced motion the
63 |  * transition is made instant ( `duration: 0` ) so no movement is perceived;
64 |  * otherwise the supplied transition is returned unchanged. When no transition
65 |  * is supplied, returns `undefined` (normal) or an instant transition (reduced).
66 |  */
67 | transition(transition?: Transition): Transition | undefined;
68 | }
69 |
70 | const INSTANT_TRANSITION: Transition = { duration: 0 };
71 |
72 | /**
73 |  * Hook returning reduced-motion-aware helpers for Framer Motion components.
74 |  *
75 |  * Usage:
76 |  * `tsx
77 |  * <motion.div
78 |  *   initial={motionGate.enter({ opacity: 0, y: 8 }, { opacity: 1, y: 0 })}
79 |  *   animate={{ opacity: 1, y: 0 }}
80 |  *   transition={motionGate.transition({ duration: 0.3 })}
81 |  * />
82 |  * `
83 |  *
84 |  * `useReducedMotion()` returns `null` until resolved (and in non-DOM
85 |  * environments); we coerce that to `false` so behaviour is well-defined and
86 |  * animations remain enabled by default.
87 |  */
88 |
89 | export const useGatedMotion = (): GatedMotion => {
90 |   const prefersReducedMotion = useReducedMotion() ?? false;
91 |
92 |   return {
93 |     prefersReducedMotion,
94 |     enter: (initialValue, animateValue) =>
95 |       prefersReducedMotion ? animateValue : initialValue,
96 |     transition: (transition) =>
97 |       prefersReducedMotion ? INSTANT_TRANSITION : transition,
98 |   };
99 | };
100 |

```

## FILE: src/lib/mysteryRewardResolver.ts

TYPE: .ts | LINES: 1-133 | SHA256: 54b8605a806b345d732886704b8cc8b84ea8fc806899562cb4a9917afc7836aa

```

1 | // =====
2 | // Mystery Reward Resolver – Pure function
3 | // Resolves mystery box outcome from probability weights
4 | // Default weights: 50% 2x XP, 30% cosmetic, 20% boost
5 | // =====
6 |
7 | /**
8 |  * Possible mystery box outcome types.
9 |  */
10 | export type MysteryRewardType = "double_xp" | "cosmetic" | "boost";
11 |
12 | export interface MysteryRewardWeight {
13 |   type: MysteryRewardType;
14 |   /** Weight (not percentage – will be normalized) */
15 |   weight: number;
16 | }
17 |
18 | export interface MysteryRewardOutcome {
19 |   /** The resolved reward type */
20 |   type: MysteryRewardType;
21 |   /** Human-readable label for the reward */
22 |   label: string;
23 |   /** Description of what the student receives */
24 |   description: string;
25 | }
26 |
27 | /**
28 |  * Default probability weights for mystery box outcomes.
29 |  * 50% 2x XP, 30% cosmetic, 20% boost.
30 |  */
31 | export const DEFAULT_MYSTERY_WEIGHTS: ReadonlyArray<MysteryRewardWeight> = [
32 |   { type: "double_xp", weight: 50 },
33 |   { type: "cosmetic", weight: 30 },
34 |   { type: "boost", weight: 20 },
35 | ];
36 |
37 | /**
38 |  * Labels and descriptions for each reward type.
39 |  */
40 | const REWARD_META: Record<
41 |   MysteryRewardType,
42 |   { label: string; description: string }
43 | > = {
44 |   double_xp: {
45 |     label: "2x XP Bonus",
46 |     description: "Your next XP award is doubled!",
47 |   },
48 |   cosmetic: {
49 |     label: "Random Cosmetic",
50 |     description: "You received a random cosmetic item for your profile!",
51 |   },
52 |   boost: {
53 |     label: "Temporary Boost",
54 |     description: "You received a temporary XP boost!",
55 |   },
56 | };
57 |
58 | /**
59 |  * Resolve a mystery box outcome using weighted random selection.
60 |

```

```

61 | * @param roll - A random number in [0, 1) used for selection.
62 | * Pass Math.random() for production use, or a fixed value for testing.
63 | * @param weights - Probability weights for each outcome type.
64 | * Defaults to 50/30/20 split. Weights are normalized to sum to 1.
65 | * @returns The resolved reward outcome.
66 | * @throws Error if weights array is empty or all weights are zero.
67 | */
68 | export function resolveMysteryReward(
69 |   roll: number,
70 |   weights: ReadonlyArray<MysteryRewardWeight> = DEFAULT_MYSTERY_WEIGHTS
71 | ): MysteryRewardOutcome {
72 |   if (weights.length === 0) {
73 |     throw new Error("Mystery reward weights cannot be empty");
74 |   }
75 |
76 |   const totalWeight = weights.reduce(
77 |     (sum, w) => sum + Math.max(0, w.weight),
78 |     0
79 |   );
80 |   if (totalWeight <= 0) {
81 |     throw new Error("Total mystery reward weight must be positive");
82 |   }
83 |
84 |   // clamp roll to [0, 1)
85 |   const clampedRoll = Math.max(0, Math.min(roll, 0.999999999));
86 |
87 |   // Walk through cumulative weights to find the selected outcome
88 |   let cumulative = 0;
89 |   for (const entry of weights) {
90 |     const normalizedWeight = Math.max(0, entry.weight) / totalWeight;
91 |     cumulative += normalizedWeight;
92 |     if (clampedRoll < cumulative) {
93 |       const meta = REWARD_META[entry.type];
94 |       return {
95 |         type: entry.type,
96 |         label: meta.label,
97 |         description: meta.description,
98 |       };
99 |     }
100 |   }
101 |
102 |   // Fallback to last entry (should not happen with valid inputs)
103 |   const lastEntry = weights[weights.length - 1];
104 |   if (!lastEntry) {
105 |     throw new Error("Mystery reward weights cannot be empty");
106 |   }
107 |   const meta = REWARD_META[lastEntry.type];
108 |   return {
109 |     type: lastEntry.type,
110 |     label: meta.label,
111 |     description: meta.description,
112 |   };
113 | }
114 |
115 | /**
116 |  * Compute the normalized probability for each reward type.
117 |  * Useful for displaying probabilities in the UI.
118 |  */
119 | export function getRewardProbabilities(
120 |   weights: ReadonlyArray<MysteryRewardWeight> = DEFAULT_MYSTERY_WEIGHTS
121 | ): Array<{ type: MysteryRewardType; probability: number }> {
122 |   const totalWeight = weights.reduce(
123 |     (sum, w) => sum + Math.max(0, w.weight),
124 |     0
125 |   );
126 |   if (totalWeight <= 0) return [];
127 |
128 |   return weights.map((w) => ({
129 |     type: w.type,
130 |     probability: Math.max(0, w.weight) / totalWeight,
131 |   }));
132 | }
133 |

```

## FILE: src/lib/navGroups.ts

TYPE: .ts | LINES: 1-152 | SHA256: cf03482dad4f107e78093d099219e324d408de7337ea0b4b982c0e28735001f3

```

1 | // Task 6.16: Student navigation group metadata + placement validation (pure business logic)
2 | // Requirements: 20.6
3 |
4 | /**
5 |  * The four labeled sections the student navigation is grouped into (R20.1).
6 |  * - `learn`      !' Courses, Assignments, AI Tutor
7 |  * - `growth`    !' Habits, Progress, Challenges
8 |  * - `community` !' Leaderboard, My Team
9 |  * - `tools`     !' Calendar, Planner, Journal
10 |  */
11 | export type NavGroup = "learn" | "growth" | "community" | "tools";
12 |
13 | /**
14 |  * Canonical render order of the navigation groups. Also serves as the single
15 |  * source of truth for the set of valid groups, used by the {@link isNavGroup}
16 |  * type guard.
17 |  */
18 | export const NAV_GROUPS: readonly NavGroup[] = [
19 |   "learn",
20 |   "growth",
21 |   "community",
22 |   "tools",
23 | ] as const;
24 |
25 | /**
26 |  * Display metadata for a navigation group. `labelKey` is an i18next key in the
27 |  * `common:nav.groups.*` namespace so the sidebar can render bilingual section

```

```

28 | * labels (R20.4); `order` fixes the section order in the sidebar.
29 | */
30 | export interface NavGroupMeta {
31 |   group: NavGroup;
32 |   labelKey: string;
33 |   order: number;
34 | }
35 |
36 | /**
37 |  * Metadata for every navigation group. The `labelKey` values resolve to the
38 |  * en/ar section labels added by the sidebar-rendering task; this module only
39 |  * owns the structural metadata, not the translation strings.
40 |  */
41 | export const NAV_GROUP_META: Record<NavGroup, NavGroupMeta> = {
42 |   learn: { group: "learn", labelKey: "nav.groups.learn", order: 0 },
43 |   growth: { group: "growth", labelKey: "nav.groups.growth", order: 1 },
44 |   community: { group: "community", labelKey: "nav.groups.community", order: 2 },
45 |   tools: { group: "tools", labelKey: "nav.groups.tools", order: 3 },
46 | };
47 |
48 | /**
49 |  * Required placements for the student navigation items enumerated in R20.1,
50 |  * keyed by route (`NavItem.to`). These are the placements validation enforces:
51 |  * an item whose route appears here MUST be assigned to the mapped group
52 |  * (in particular the AI Tutor route `/student/tutor` MUST belong to `learn`,
53 |  * per R20.6). Routes not listed here have no mandated group and may be assigned
54 |  * to any defined group by the navigation-assignment task.
55 |  */
56 | export const REQUIRED_NAV_GROUPS: Readonly<Record<string, NavGroup>> = {
57 |   // Learn
58 |   "/student/courses": "learn",
59 |   "/student/assignments": "learn",
60 |   "/student/tutor": "learn", // AI Tutor " Learn (R20.6)
61 |   // Growth
62 |   "/student/habits": "growth",
63 |   "/student/progress": "growth",
64 |   "/student/challenges": "growth",
65 |   // Community
66 |   "/student/leaderboard": "community",
67 |   "/student/team": "community",
68 |   // Tools
69 |   "/student/calendar": "tools",
70 |   "/student/planner": "tools",
71 |   "/student/journal": "tools",
72 | };
73 |
74 | /**
75 |  * The minimal shape required to validate a navigation item's group placement.
76 |  * Declared locally (rather than importing `NavItem`) so this pure module has no
77 |  * dependency on `navItems.ts` - `navItems.ts` is the consumer that imports
78 |  * {@link NavGroup} from here, and the reverse import would be circular.
79 |  */
80 | export interface NavGroupAssignment {
81 |   to: string;
82 |   group?: NavGroup;
83 | }
84 |
85 | /** Error thrown by {@link assertNavGroup} when a placement is invalid. */
86 | export class NavGroupAssignmentError extends Error {
87 |   constructor(message: string) {
88 |     super(message);
89 |     this.name = "NavGroupAssignmentError";
90 |   }
91 | }
92 |
93 | /** Type guard: narrows an arbitrary value to a defined {@link NavGroup}. */
94 | export function isNavGroup(value: unknown): value is NavGroup {
95 |   return (
96 |     typeof value === "string" &&
97 |     (NAV_GROUPS as readonly string[]).includes(value)
98 |   );
99 | }
100 |
101 | /**
102 |  * Returns the group a route is required to belong to, or `undefined` when the
103 |  * route has no mandated placement.
104 |  */
105 | export function getRequiredNavGroup(to: string): NavGroup | undefined {
106 |   return REQUIRED_NAV_GROUPS[to];
107 | }
108 |
109 | /**
110 |  * Non-throwing predicate form of {@link assertNavGroup}. Returns `true` when the
111 |  * item is assigned to a defined group AND that group satisfies any required
112 |  * placement for its route; `false` otherwise. Useful for filtering/rendering and
113 |  * for property tests.
114 |  */
115 | export function isValidNavGroupAssignment(item: NavGroupAssignment): boolean {
116 |   if (!isNavGroup(item.group)) return false;
117 |   const required = getRequiredNavGroup(item.to);
118 |   return required === undefined || item.group === required;
119 | }
120 |
121 | /**
122 |  * Asserts that a navigation item is assigned to a valid group and satisfies its
123 |  * required placement (R20.6). Throws {@link NavGroupAssignmentError} when:
124 |  * - the item has no group, or a group that is not one of the defined groups; or
125 |  * - the item's route has a required placement and the assigned group differs
126 |  *   from it (e.g. the AI Tutor route assigned to any group other than `learn`).
127 |  *
128 |  * This is a pure, total assertion: every input yields either a successful
129 |  * narrowing or a thrown error - there is no silent acceptance of an invalid
130 |  * placement.
131 |  */

```

```

132 | export function assertNavGroup(
133 |   item: NavGroupAssignment
134 | ): asserts item is Required<NavGroupAssignment> {
135 |   if (!isNavGroup(item.group)) {
136 |     throw new NavGroupAssignmentError(
137 |       `Navigation item "${
138 |         item.to
139 |       }" must be assigned to one of: ${NAV_GROUPS.join(
140 |         ", "
141 |       )} (received: ${String(item.group)})`.
142 |     );
143 |   }
144 |
145 |   const required = getRequiredNavGroup(item.to);
146 |   if (required !== undefined && item.group !== required) {
147 |     throw new NavGroupAssignmentError(
148 |       `Navigation item "${item.to}" must belong to the "${required}" group but was assigned to "${item.group}"
149 |     );
150 |   }
151 | }
152 |

```

FILE: src/lib/navItems.ts

TYPE: .ts | LINES: 1-395 | SHA256: efea6857f97083e5d84b3d1c3428ebd0fd9320993d7430b9279a16c59036f1d2

```

1 | /**
2 |  * Unified nav item definitions for all role layouts.
3 |  *
4 |  * Single source of truth – consumed by role layouts and any component
5 |  * that needs to enumerate navigation (e.g. GlobalHeader, breadcrumbs,
6 |  * search indexing, guided tour step generation).
7 |  *
8 |  * labelKey uses the `common:nav.*` namespace so callers can do:
9 |  *   const { t } = useTranslation('common');
10 |  *   t(item.labelKey) // e.g. t('nav.dashboard') != "Dashboard"
11 |  */
12 |
13 | import type { LucideIcon } from "lucide-react";
14 | import {
15 |   LayoutDashboard,
16 |   Users,
17 |   Building2,
18 |   BookOpen,
19 |   GraduationCap,
20 |   Calendar,
21 |   Target,
22 |   Clock,
23 |   CalendarDays,
24 |   DollarSign,
25 |   FileText,
26 |   ScrollText,
27 |   ShieldAlert,
28 |   Sparkles,
29 |   Award,
30 |   Store,
31 |   ClipboardList,
32 |   Grid3X3,
33 |   GitBranch,
34 |   Search,
35 |   LayoutGrid,
36 |   TrendingUp,
37 |   ClipboardCheck,
38 |   TableProperties,
39 |   CheckSquare,
40 |   Megaphone,
41 |   FolderOpen,
42 |   Trophy,
43 |   HeartPulse,
44 |   Handshake,
45 |   ClipboardList as ClipboardListIcon,
46 |   CalendarCheck,
47 |   Bot,
48 |   PenLine,
49 |   Star,
50 |   FileQuestion,
51 |   Swords,
52 |   Workflow,
53 |   UserCog,
54 |   Wallet,
55 |   Bell,
56 |   MessageSquare,
57 | } from "lucide-react";
58 | import type { UserRole } from "@types/app";
59 | import type { NavGroup } from "@lib/navGroups";
60 |
61 | export interface NavItem {
62 |   to: string;
63 |   labelKey: string;
64 |   icon: LucideIcon;
65 |   /**
66 |    * Task 22.1 (R20.1, R20.2): the labeled student-navigation section this item
67 |    * belongs to. Optional because grouping applies to the student navigation
68 |    * only – admin/coordinator/teacher/parent items leave this undefined. Student
69 |    * items are validated against {@link REQUIRED_NAV_GROUPS} via `assertNavGroup`
70 |    * (e.g. the AI Tutor item must belong to `learn`, R20.6).
71 |    */
72 |   group?: NavGroup;
73 |   /**
74 |    * Task 22.3 (R23.3): the item is kept (it still provides a student-relevant
75 |    * function) but de-emphasized relative to core learning items. The sidebar
76 |    * sinks de-emphasized items to the bottom of their section and renders them
77 |    * with subdued styling. Used for "My Content", which students can use to
78 |    * author study plans / quiz questions but which is secondary to the core
79 |    * learning items (Courses, Assignments, AI Tutor).

```

```

80 | */
81 | deEmphasized?: boolean;
82 | }
83 |
84 | const adminNavItems: NavItem[] = [
85 |   { to: "/admin/dashboard", labelKey: "nav.dashboard", icon: LayoutDashboard },
86 |   { to: "/admin/analytics", labelKey: "nav.analytics", icon: TrendingUp },
87 |   { to: "/admin/accreditation-reports", labelKey: "nav.accreditationReports", icon: FileText },
88 |   { to: "/admin/settings/profile", labelKey: "nav.me", icon: UserCog },
89 |   {
90 |     to: "/admin/settings/institution",
91 |     labelKey: "nav.institutionStructure",
92 |     icon: Building2,
93 |   },
94 |   { to: "/admin/users", labelKey: "nav.users", icon: Users },
95 |   { to: "/admin/departments", labelKey: "nav.departments", icon: Building2 },
96 |   { to: "/admin/programs", labelKey: "nav.programs", icon: BookOpen },
97 |   { to: "/admin/courses", labelKey: "nav.courses", icon: GraduationCap },
98 |   { to: "/admin/semesters", labelKey: "nav.semesters", icon: Calendar },
99 |   { to: "/admin/outcomes", labelKey: "nav.ilos", icon: Target },
100 |   { to: "/admin/timetable", labelKey: "nav.timetable", icon: Clock },
101 |   { to: "/admin/calendar", labelKey: "nav.calendar", icon: CalendarDays },
102 |   { to: "/admin/fees", labelKey: "nav.fees", icon: DollarSign },
103 |   { to: "/admin/import", labelKey: "nav.bulkImport", icon: ClipboardList },
104 |   { to: "/admin/reports", labelKey: "nav.reports", icon: FileText },
105 |   { to: "/admin/audit-log", labelKey: "nav.auditLog", icon: ScrollText },
106 |   { to: "/admin/governance", labelKey: "nav.aiGovernance", icon: ShieldAlert },
107 |   { to: "/admin/security", labelKey: "nav.security", icon: ShieldAlert },
108 |   { to: "/admin/bonus-events", labelKey: "nav.bonusXp", icon: Sparkles },
109 |   {
110 |     to: "/admin/badges",
111 |     labelKey: "nav.badgeDefinitions",
112 |     icon: Award,
113 |   },
114 |   {
115 |     to: "/admin/badges/spotlight",
116 |     labelKey: "nav.badgeSpotlight",
117 |     icon: Award,
118 |   },
119 |   { to: "/admin/marketplace", labelKey: "nav.marketplace", icon: Store },
120 |   { to: "/admin/surveys", labelKey: "nav.surveys", icon: ClipboardList },
121 |   { to: "/admin/notifications", labelKey: "nav.notifications", icon: Bell },
122 | ];
123 |
124 | const coordinatorNavItems: NavItem[] = [
125 |   {
126 |     to: "/coordinator/dashboard",
127 |     labelKey: "nav.dashboard",
128 |     icon: LayoutDashboard,
129 |   },
130 |   {
131 |     to: "/coordinator/settings/profile",
132 |     labelKey: "nav.me",
133 |     icon: UserCog,
134 |   },
135 |   { to: "/coordinator/plos", labelKey: "nav.plos", icon: Target },
136 |   { to: "/coordinator/matrix", labelKey: "nav.matrix", icon: Grid3X3 },
137 |   { to: "/coordinator/sankey", labelKey: "nav.sankeyDiagram", icon: GitBranch },
138 |   {
139 |     to: "/coordinator/gap-analysis",
140 |     labelKey: "nav.gapAnalysis",
141 |     icon: Search,
142 |   },
143 |   {
144 |     to: "/coordinator/coverage-heatmap",
145 |     labelKey: "nav.coverageHeatmap",
146 |     icon: LayoutGrid,
147 |   },
148 |   { to: "/coordinator/cqi", labelKey: "nav.cqiPlans", icon: ClipboardCheck },
149 |   {
150 |     to: "/coordinator/outcome-chain",
151 |     labelKey: "nav.outcomeChain",
152 |     icon: Workflow,
153 |   },
154 |   {
155 |     to: "/coordinator/course-file",
156 |     labelKey: "nav.courseFile",
157 |     icon: FileText,
158 |   },
159 |   {
160 |     to: "/coordinator/accreditation",
161 |     labelKey: "nav.accreditation",
162 |     icon: ClipboardCheck,
163 |   },
164 |   {
165 |     to: "/coordinator/team-health",
166 |     labelKey: "nav.teamHealthReport",
167 |     icon: HeartPulse,
168 |   },
169 |   {
170 |     to: "/coordinator/competencies",
171 |     labelKey: "nav.competencyFrameworks",
172 |     icon: Workflow,
173 |   },
174 |   {
175 |     to: "/coordinator/discussions",
176 |     labelKey: "nav.discussions",
177 |     icon: MessageSquare,
178 |   },
179 |   { to: "/coordinator/timetable", labelKey: "nav.timetable", icon: Clock },
180 |   {
181 |     to: "/coordinator/notifications",
182 |     labelKey: "nav.notifications",
183 |     icon: Bell,

```

```

184 | },
185 | ];
186 |
187 | const teacherNavItems: NavItem[] = [
188 | {
189 |   to: "/teacher/dashboard",
190 |   labelKey: "nav.dashboard",
191 |   icon: LayoutDashboard,
192 | },
193 | {
194 |   to: "/teacher/settings/profile",
195 |   labelKey: "nav.me",
196 |   icon: UserCog,
197 | },
198 | {
199 |   to: "/teacher/students",
200 |   labelKey: "nav.students",
201 |   icon: Users,
202 | },
203 | { to: "/teacher/grading", labelKey: "nav.grading", icon: CheckSquare },
204 | { to: "/teacher/gradebook", labelKey: "nav.gradebook", icon: BookOpen },
205 | { to: "/teacher/modules", labelKey: "nav.modules", icon: FolderOpen },
206 | {
207 |   to: "/teacher/questions",
208 |   labelKey: "nav.questionBank",
209 |   icon: FileQuestion,
210 | },
211 | { to: "/teacher/rubrics", labelKey: "nav.rubrics", icon: TableProperties },
212 | { to: "/teacher/content", labelKey: "nav.courseMaterials", icon: BookOpen },
213 | {
214 |   to: "/teacher/tutor-handoffs",
215 |   labelKey: "nav.tutorHandoffs",
216 |   icon: Handshake,
217 | },
218 | { to: "/teacher/attendance", labelKey: "nav.attendance", icon: CalendarDays },
219 | {
220 |   to: "/teacher/discussions",
221 |   labelKey: "nav.discussions",
222 |   icon: MessageSquare,
223 | },
224 | {
225 |   to: "/teacher/announcements",
226 |   labelKey: "nav.announcements",
227 |   icon: Megaphone,
228 | },
229 | { to: "/teacher/notifications", labelKey: "nav.notifications", icon: Bell },
230 | ];
231 |
232 | const studentNavItems: NavItem[] = [
233 | {
234 |   to: "/student/dashboard",
235 |   labelKey: "nav.mobile.home",
236 |   icon: LayoutDashboard,
237 |   group: "tools",
238 | },
239 | {
240 |   to: "/student/learning-path",
241 |   labelKey: "nav.mobile.learn",
242 |   icon: BookOpen,
243 |   group: "learn",
244 | },
245 | {
246 |   to: "/student/tutor",
247 |   labelKey: "nav.mobile.tutor",
248 |   icon: Bot,
249 |   group: "learn",
250 | },
251 | {
252 |   to: "/student/progress",
253 |   labelKey: "nav.mobile.progress",
254 |   icon: TrendingUp,
255 |   group: "growth",
256 | },
257 | {
258 |   to: "/student/profile",
259 |   labelKey: "nav.me",
260 |   icon: UserCog,
261 |   group: "growth",
262 | },
263 | {
264 |   to: "/student/courses",
265 |   labelKey: "nav.coursesTasks",
266 |   icon: ClipboardListIcon,
267 |   group: "learn",
268 | },
269 | {
270 |   to: "/student/assignments",
271 |   labelKey: "nav.assignments",
272 |   icon: ClipboardListIcon,
273 |   group: "learn",
274 | },
275 | {
276 |   to: "/student/today",
277 |   labelKey: "nav.dailyReview",
278 |   icon: CalendarCheck,
279 |   group: "tools",
280 | },
281 | {
282 |   to: "/student/habits",
283 |   labelKey: "nav.wellness",
284 |   icon: HeartPulse,
285 |   group: "growth",
286 | },
287 | {

```



```

288 |         to: "/student/planner",
289 |         labelKey: "nav.focus",
290 |         icon: Clock,
291 |         group: "tools",
292 |     },
293 |     {
294 |         to: "/student/challenges",
295 |         labelKey: "nav.quests",
296 |         icon: Swords,
297 |         group: "growth",
298 |     },
299 |     {
300 |         to: "/student/leaderboard",
301 |         labelKey: "nav.leaderboard",
302 |         icon: Trophy,
303 |         group: "community",
304 |     },
305 |     {
306 |         to: "/student/team",
307 |         labelKey: "nav.myTeam",
308 |         icon: Users,
309 |         group: "community",
310 |     },
311 |     {
312 |         to: "/student/journal",
313 |         labelKey: "nav.journal",
314 |         icon: PenLine,
315 |         group: "tools",
316 |     },
317 |     {
318 |         to: "/student/calendar",
319 |         labelKey: "nav.calendar",
320 |         icon: Calendar,
321 |         group: "tools",
322 |     },
323 |     {
324 |         to: "/student/marketplace",
325 |         labelKey: "nav.shop",
326 |         icon: Store,
327 |         group: "growth",
328 |     },
329 |     {
330 |         to: "/student/notifications",
331 |         labelKey: "nav.notifications",
332 |         icon: Bell,
333 |         group: "tools",
334 |     },
335 |     {
336 |         to: "/student/settings/profile",
337 |         labelKey: "nav.settings",
338 |         icon: UserCog,
339 |         group: "tools",
340 |     },
341 |     {
342 |         to: "/student/surveys",
343 |         labelKey: "nav.surveys",
344 |         icon: FileQuestion,
345 |         group: "tools",
346 |     },
347 |     { to: "/student/fees", labelKey: "nav.fees", icon: Wallet, group: "tools" },
348 |     {
349 |         to: "/student/content",
350 |         labelKey: "nav.myContent",
351 |         icon: FileText,
352 |         group: "learn",
353 |     },
354 |     {
355 |         to: "/student/timetable",
356 |         labelKey: "nav.timetable",
357 |         icon: Clock,
358 |         group: "tools",
359 |     },
360 |     {
361 |         to: "/student/portfolio",
362 |         labelKey: "nav.portfolio",
363 |         icon: Star,
364 |         group: "growth",
365 |     },
366 |     {
367 |         to: "/student/badges",
368 |         labelKey: "nav.badges",
369 |         icon: Award,
370 |         group: "growth",
371 |     },
372 | ];
373 |
374 | const parentNavItems: NavItem[] = [
375 |     { to: "/parent/dashboard", labelKey: "nav.dashboard", icon: LayoutDashboard },
376 |     { to: "/parent/progress", labelKey: "nav.progress", icon: TrendingUp },
377 |     { to: "/parent/support", labelKey: "nav.support", icon: Megaphone },
378 |     { to: "/parent/profile", labelKey: "nav.me", icon: UserCog },
379 |     { to: "/parent/attendance", labelKey: "nav.attendance", icon: CalendarDays },
380 |     { to: "/parent/fees", labelKey: "nav.fees", icon: Wallet },
381 |     {
382 |         to: "/parent/communications",
383 |         labelKey: "nav.announcements",
384 |         icon: Megaphone,
385 |     },
386 | ];
387 |
388 | export const navItems: Record<UserRole, NavItem[]> = {
389 |     admin: adminNavItems,
390 |     coordinator: coordinatorNavItems,
391 |     teacher: teacherNavItems,

```

```

392 | student: studentNavItems,
393 | parent: parentNavItems,
394 | };
395 |

```

## FILE: src/lib/navPresentation.ts

TYPE: .ts | LINES: 1-345 | SHA256: d23b74ce39ec6b59c4893c7698bbb02e4850d78af1e6ad5906a012bded0c17d1

```

1 | import {
2 |   BarChart3,
3 |   BookOpen,
4 |   Bot,
5 |   CheckSquare,
6 |   FileText,
7 |   FolderOpen,
8 |   Grid3X3,
9 |   Home,
10 |   LayoutDashboard,
11 |   Target,
12 |   Megaphone,
13 |   TrendingUp,
14 |   User,
15 |   UserCog,
16 |   Users,
17 |   type LucideIcon,
18 | } from "lucide-react";
19 | import { navItems, type NavItem } from "@lib/navItems";
20 | import type { UserRole } from "@types/app";
21 |
22 | interface PrototypeDesktopItem {
23 |   to: string;
24 |   labelKey: string;
25 |   emoji: string;
26 | }
27 |
28 | export interface PresentedNavItem extends NavItem {
29 |   emoji: string;
30 | }
31 |
32 | /**
33 |  * Exact desktop chrome copied from prototype/shared.js ROLE_NAV. Desktop uses
34 |  * the prototype emoji artwork; the mobile bar keeps the matching Lucide glyphs
35 |  * because that is what the approved phone layouts use.
36 |  */
37 | const desktopPrimaryByRole: Record<UserRole, PrototypeDesktopItem[]> = {
38 |   student: [
39 |     { to: "/student/dashboard", labelKey: "nav.mobile.home", emoji: "ð<ðà" },
40 |     { to: "/student/learning-path", labelKey: "nav.mobile.learn", emoji: "ð>ýúþ " },
41 |     { to: "/student/tutor", labelKey: "nav.mobile.tutor", emoji: "ð>ý " },
42 |     { to: "/student/progress", labelKey: "nav.mobile.progress", emoji: "ð=üë" },
43 |     { to: "/student/profile", labelKey: "nav.me", emoji: "ð=þB" },
44 |   ],
45 |   teacher: [
46 |     { to: "/teacher/dashboard", labelKey: "nav.mobile.home", emoji: "ð<ðà" },
47 |     { to: "/teacher/students", labelKey: "nav.mobile.students", emoji: "ð>ýN ð<ð"" },
48 |     { to: "/teacher/modules", labelKey: "nav.mobile.studio", emoji: "ð>ýi" },
49 |     { to: "/teacher/grading", labelKey: "nav.mobile.grade", emoji: " ' þ " },
50 |     { to: "/teacher/settings/profile", labelKey: "nav.me", emoji: "ð=þB" },
51 |   ],
52 |   parent: [
53 |     { to: "/parent/dashboard", labelKey: "nav.mobile.home", emoji: "ð<ðà" },
54 |     { to: "/parent/progress", labelKey: "nav.mobile.growth", emoji: "ð<ð1" },
55 |     { to: "/parent/support", labelKey: "nav.mobile.support", emoji: "ð=ü- " },
56 |     { to: "/parent/profile", labelKey: "nav.me", emoji: "ð=þB" },
57 |   ],
58 |   coordinator: [
59 |     { to: "/coordinator/dashboard", labelKey: "nav.mobile.home", emoji: "ð<ðà" },
60 |     { to: "/coordinator/plos", labelKey: "nav.mobile.outcomes", emoji: "ð<ð" },
61 |     {
62 |       to: "/coordinator/matrix",
63 |       labelKey: "nav.mobile.curriculum",
64 |       emoji: "ð=ýÄþ ",
65 |     },
66 |     {
67 |       to: "/coordinator/accreditation",
68 |       labelKey: "nav.mobile.accredit",
69 |       emoji: "ð=üë",
70 |     },
71 |     { to: "/coordinator/settings/profile", labelKey: "nav.me", emoji: "ð=þB" },
72 |   ],
73 |   admin: [
74 |     { to: "/admin/dashboard", labelKey: "nav.mobile.home", emoji: "ð<ðà" },
75 |     { to: "/admin/reports", labelKey: "nav.mobile.analytics", emoji: "ð=üë" },
76 |     { to: "/admin/governance", labelKey: "nav.mobile.aiGov", emoji: "ð=þáp " },
77 |     { to: "/admin/users", labelKey: "nav.mobile.people", emoji: "ð=üe" },
78 |     { to: "/admin/settings/profile", labelKey: "nav.me", emoji: "ð=þB" },
79 |   ],
80 | };
81 |
82 | /**
83 |  * Prototype ROLE_MORE order, restricted to destinations already backed by a
84 |  * real route. Intentional repetitions (for example Studio/Course Materials)
85 |  * are preserved because the approved sidebar uses those task-oriented labels.
86 |  */
87 | const desktopMoreByRole: Record<UserRole, PrototypeDesktopItem[]> = {
88 |   student: [
89 |     { to: "/student/courses", labelKey: "nav.coursesTasks", emoji: "ð=üü" },
90 |     { to: "/student/today", labelKey: "nav.dailyReview", emoji: "ð=ý " },
91 |     { to: "/student/habits", labelKey: "nav.wellness", emoji: "ð=üš" },
92 |     { to: "/student/planner", labelKey: "nav.focus", emoji: "ð=ñþ " },
93 |     { to: "/student/challenges", labelKey: "nav.quests", emoji: "ð=þ " },
94 |     { to: "/student/leaderboard", labelKey: "nav.leaderboard", emoji: "ð<ðæ" },
95 |     { to: "/student/team", labelKey: "nav.myTeam", emoji: "ð=üe" },
96 |     { to: "/student/journal", labelKey: "nav.journal", emoji: "ð=üö" },

```

```

97 |     { to: "/student/calendar", labelKey: "nav.calendar", emoji: "📅" },
98 |     { to: "/student/marketplace", labelKey: "nav.shop", emoji: "🛒" },
99 |     {
100 |         to: "/student/notifications",
101 |         labelKey: "nav.notifications",
102 |         emoji: "🔔",
103 |     },
104 | ],
105 | teacher: [
106 |     { to: "/teacher/modules", labelKey: "nav.curriculumStudio", emoji: "📖" },
107 |     { to: "/teacher/questions", labelKey: "nav.questionBank", emoji: "❓" },
108 |     { to: "/teacher/rubrics", labelKey: "nav.rubricBuilder", emoji: "📋" },
109 |     { to: "/teacher/modules", labelKey: "nav.courseMaterials", emoji: "📚" },
110 |     {
111 |         to: "/teacher/tutor-handoffs",
112 |         labelKey: "nav.tutorHandoffs",
113 |         emoji: "👉",
114 |     },
115 |     { to: "/teacher/grading", labelKey: "nav.gradingQueue", emoji: "📝" },
116 |     { to: "/teacher/gradebook", labelKey: "nav.gradebook", emoji: "📊" },
117 |     { to: "/teacher/attendance", labelKey: "nav.attendance", emoji: "👤" },
118 |     { to: "/teacher/discussions", labelKey: "nav.discussions", emoji: "💬" },
119 |     {
120 |         to: "/teacher/announcements",
121 |         labelKey: "nav.announcements",
122 |         emoji: "📢",
123 |     },
124 |     {
125 |         to: "/teacher/notifications",
126 |         labelKey: "nav.notifications",
127 |         emoji: "🔔",
128 |     },
129 | ],
130 | parent: [
131 |     { to: "/parent/attendance", labelKey: "nav.attendance", emoji: "👤" },
132 |     { to: "/parent/progress", labelKey: "nav.gradesReports", emoji: "📊" },
133 |     { to: "/parent/fees", labelKey: "nav.feesPayments", emoji: "💰" },
134 |     {
135 |         to: "/parent/communications",
136 |         labelKey: "nav.announcements",
137 |         emoji: "📢",
138 |     },
139 |     { to: "/parent/profile", labelKey: "nav.settings", emoji: "⚙️" },
140 | ],
141 | coordinator: [
142 |     { to: "/coordinator/plos", labelKey: "nav.outcomeAttainment", emoji: "📈" },
143 |     {
144 |         to: "/coordinator/matrix",
145 |         labelKey: "nav.curriculumMatrix",
146 |         emoji: "📋",
147 |     },
148 |     { to: "/coordinator/cqi", labelKey: "nav.cqiPlans", emoji: "📝" },
149 |     {
150 |         to: "/coordinator/course-file",
151 |         labelKey: "nav.courseFileGenerator",
152 |         emoji: "📄",
153 |     },
154 |     {
155 |         to: "/coordinator/team-health",
156 |         labelKey: "nav.teamHealthReport",
157 |         emoji: "👥",
158 |     },
159 |     {
160 |         to: "/coordinator/competencies",
161 |         labelKey: "nav.competencyFrameworks",
162 |         emoji: "📖",
163 |     },
164 |     {
165 |         to: "/coordinator/accreditation",
166 |         labelKey: "nav.accreditation",
167 |         emoji: "📋",
168 |     },
169 |     {
170 |         to: "/coordinator/discussions",
171 |         labelKey: "nav.discussions",
172 |         emoji: "💬",
173 |     },
174 |     {
175 |         to: "/coordinator/notifications",
176 |         labelKey: "nav.announcements",
177 |         emoji: "📢",
178 |     },
179 |     {
180 |         to: "/coordinator/notifications",
181 |         labelKey: "nav.notifications",
182 |         emoji: "🔔",
183 |     },
184 | ],
185 | admin: [
186 |     { to: "/admin/marketplace", labelKey: "nav.marketplace", emoji: "🛒" },
187 |     {
188 |         to: "/admin/settings/institution",
189 |         labelKey: "nav.institutionStructure",
190 |         emoji: "🏛️",
191 |     },
192 |     { to: "/admin/import", labelKey: "nav.bulkImport", emoji: "📄" },
193 |     { to: "/admin/badges", labelKey: "nav.badgeDefinitions", emoji: "🏆" },
194 |     { to: "/admin/security", labelKey: "nav.security", emoji: "🔒" },
195 |     { to: "/admin/fees", labelKey: "nav.feesManagement", emoji: "💰" },
196 |     { to: "/admin/notifications", labelKey: "nav.announcements", emoji: "📢" },
197 |     { to: "/admin/notifications", labelKey: "nav.notifications", emoji: "🔔" },
198 | ],
199 | };
200 |

```

```

201 | /** Central raised action in the mobile tab bar; each points to an existing route. */
202 | export const mobileFabPathByRole: Record<UserRole, string> = {
203 |   student: "/student/tutor",
204 |   teacher: "/teacher/modules",
205 |   parent: "/parent/support",
206 |   coordinator: "/coordinator/matrix",
207 |   admin: "/admin/governance",
208 | };
209 |
210 | export interface MobileTabItem {
211 |   to: string;
212 |   labelKey: string;
213 |   icon: LucideIcon;
214 |   raised?: boolean;
215 | }
216 |
217 | const mobileTabsByRole: Record<UserRole, MobileTabItem[]> = {
218 |   student: [
219 |     {
220 |       to: "/student/dashboard",
221 |       labelKey: "nav.mobile.home",
222 |       icon: Home,
223 |     },
224 |     {
225 |       to: "/student/learning-path",
226 |       labelKey: "nav.mobile.learn",
227 |       icon: BookOpen,
228 |     },
229 |     {
230 |       to: "/student/tutor",
231 |       labelKey: "nav.mobile.tutor",
232 |       icon: Bot,
233 |       raised: true,
234 |     },
235 |     {
236 |       to: "/student/progress",
237 |       labelKey: "nav.mobile.progress",
238 |       icon: TrendingUp,
239 |     },
240 |     { to: "/student/profile", labelKey: "nav.me", icon: User },
241 |   ],
242 |   teacher: [
243 |     {
244 |       to: "/teacher/dashboard",
245 |       labelKey: "nav.mobile.home",
246 |       icon: LayoutDashboard,
247 |     },
248 |     { to: "/teacher/students", labelKey: "nav.mobile.students", icon: Users },
249 |     {
250 |       to: "/teacher/modules",
251 |       labelKey: "nav.mobile.studio",
252 |       icon: FolderOpen,
253 |       raised: true,
254 |     },
255 |     { to: "/teacher/grading", labelKey: "nav.mobile.grade", icon: CheckSquare },
256 |     { to: "/teacher/settings/profile", labelKey: "nav.me", icon: UserCog },
257 |   ],
258 |   parent: [
259 |     {
260 |       to: "/parent/dashboard",
261 |       labelKey: "nav.mobile.home",
262 |       icon: LayoutDashboard,
263 |     },
264 |     { to: "/parent/progress", labelKey: "nav.mobile.growth", icon: TrendingUp },
265 |     {
266 |       to: "/parent/support",
267 |       labelKey: "nav.mobile.support",
268 |       icon: Megaphone,
269 |       raised: true,
270 |     },
271 |     { to: "/parent/profile", labelKey: "nav.me", icon: UserCog },
272 |   ],
273 |   coordinator: [
274 |     {
275 |       to: "/coordinator/dashboard",
276 |       labelKey: "nav.mobile.home",
277 |       icon: LayoutDashboard,
278 |     },
279 |     { to: "/coordinator/plos", labelKey: "nav.mobile.outcomes", icon: Target },
280 |     {
281 |       to: "/coordinator/matrix",
282 |       labelKey: "nav.mobile.curriculum",
283 |       icon: Grid3X3,
284 |       raised: true,
285 |     },
286 |     {
287 |       to: "/coordinator/accreditation",
288 |       labelKey: "nav.mobile.accredit",
289 |       icon: FileText,
290 |     },
291 |     { to: "/coordinator/settings/profile", labelKey: "nav.me", icon: UserCog },
292 |   ],
293 |   admin: [
294 |     {
295 |       to: "/admin/dashboard",
296 |       labelKey: "nav.mobile.home",
297 |       icon: LayoutDashboard,
298 |     },
299 |     { to: "/admin/reports", labelKey: "nav.mobile.analytics", icon: BarChart3 },
300 |     {
301 |       to: "/admin/governance",
302 |       labelKey: "nav.mobile.aiGov",
303 |       icon: Bot,
304 |       raised: true,

```

```

305 |     },
306 |     { to: "/admin/users", labelKey: "nav.mobile.people", icon: Users },
307 |     { to: "/admin/settings/profile", labelKey: "nav.me", icon: UserCog },
308 |   ],
309 | };
310 |
311 | const presentItems = (
312 |   role: UserRole,
313 |   definitions: PrototypeDesktopItem[]
314 | ): PresentedNavItem[] => {
315 |   const itemsByPath = new Map(
316 |     (navItems[role] ?? []).map((item) => [item.to, item])
317 |   );
318 |   return definitions.flatMap((definition) => {
319 |     const item = itemsByPath.get(definition.to);
320 |     return item
321 |       ? [{ ...item, labelKey: definition.labelKey, emoji: definition.emoji }]
322 |       : [];
323 |   });
324 | };
325 |
326 | export const getPrimaryNavItems = (role: UserRole): PresentedNavItem[] =>
327 |   presentItems(role, desktopPrimaryByRole[role]).map((item) => {
328 |     const prototypeItem = mobileTabsByRole[role].find(
329 |       (candidate) => candidate.to === item.to
330 |     );
331 |     return prototypeItem
332 |       ? {
333 |         ...item,
334 |         labelKey: prototypeItem.labelKey,
335 |         icon: prototypeItem.icon,
336 |       }
337 |       : item;
338 |   });
339 |
340 | export const getMoreNavItems = (role: UserRole): PresentedNavItem[] =>
341 |   presentItems(role, desktopMoreByRole[role]);
342 |
343 | export const getMobileTabItems = (role: UserRole): MobileTabItem[] =>
344 |   mobileTabsByRole[role];
345 |

```

## FILE: src/lib/notificationBatcher.ts

TYPE: .ts | LINES: 1-178 | SHA256: 98852b6bf12fa965814295b99e1e21ff00ea3a89a37f421e6577c9b100a14bde

```

1 | // =====
2 | // NotificationBatcher - Batching, rate limiting, and grouping for notifications
3 | // Validates: Requirements 65.1, 65.2, 65.3
4 | // =====
5 |
6 | export const PEER_MILESTONE_BATCH_WINDOW_MS = 60 * 60 * 1000;
7 | export const PEER_MILESTONE_DAILY_LIMIT = 5;
8 | export const GROUPING_THRESHOLD = 3;
9 |
10 | export interface NotificationRecord {
11 |   id: string;
12 |   user_id: string;
13 |   type: string;
14 |   title: string;
15 |   body: string | null;
16 |   is_read: boolean;
17 |   metadata: Record<string, unknown> | null;
18 |   created_at: string;
19 | }
20 |
21 | export interface BatchedNotification {
22 |   type: string;
23 |   title: string;
24 |   body: string;
25 |   count: number;
26 |   is_grouped: boolean;
27 |   items: NotificationRecord[];
28 | }
29 |
30 | export function countInWindow(
31 |   notifications: NotificationRecord[],
32 |   studentId: string,
33 |   type: string,
34 |   windowMs: number,
35 |   referenceTime: Date = new Date()
36 | ): number {
37 |   const cutoff = referenceTime.getTime() - windowMs;
38 |   return notifications.filter(
39 |     (n) =>
40 |       n.user_id === studentId &&
41 |       n.type === type &&
42 |       new Date(n.created_at).getTime() >= cutoff
43 |   ).length;
44 | }
45 |
46 | export function shouldBatch(
47 |   notifications: NotificationRecord[],
48 |   studentId: string,
49 |   type: string,
50 |   windowMs: number = PEER_MILESTONE_BATCH_WINDOW_MS,
51 |   referenceTime: Date = new Date()
52 | ): boolean {
53 |   if (type !== "peer_milestone") return false;
54 |   return (
55 |     countInWindow(notifications, studentId, type, windowMs, referenceTime) > 0
56 |   );
57 | }
58 |
59 | const TWENTY_FOUR_HOURS_MS = 24 * 60 * 60 * 1000;

```

```

60 |
61 | export function hasReachedDailyLimit(
62 |   notifications: NotificationRecord[],
63 |   studentId: string,
64 |   referenceTime: Date = new Date()
65 | ): boolean {
66 |   const count = countInWindow(
67 |     notifications,
68 |     studentId,
69 |     "peer_milestone",
70 |     TWENTY_FOUR_HOURS_MS,
71 |     referenceTime
72 |   );
73 |   return count >= PEER_MILESTONE_DAILY_LIMIT;
74 | }
75 |
76 | export function canCreatePeerMilestone(
77 |   notifications: NotificationRecord[],
78 |   studentId: string,
79 |   referenceTime: Date = new Date()
80 | ): boolean {
81 |   return !hasReachedDailyLimit(notifications, studentId, referenceTime);
82 | }
83 |
84 | const TYPE_LABELS: Record<string, string> = {
85 |   grade_released: "new grades released",
86 |   new_assignment: "new assignments posted",
87 |   badge_earned: "badges earned",
88 |   streak_at_risk: "streak alerts",
89 |   at_risk_alert: "at-risk alerts",
90 |   peer_milestone: "classmate milestones",
91 |   perfect_day_nudge: "perfect day nudges",
92 |   prerequisite_unlocked: "prerequisites unlocked",
93 | };
94 |
95 | export function groupNotifications(
96 |   notifications: NotificationRecord[]
97 | ): BatchedNotification[] {
98 |   const byType = new Map<string, NotificationRecord[]>();
99 |   for (const n of notifications) {
100 |     const existing = byType.get(n.type) ?? [];
101 |     existing.push(n);
102 |     byType.set(n.type, existing);
103 |   }
104 |
105 |   const result: BatchedNotification[] = [];
106 |
107 |   for (const [type, items] of byType) {
108 |     if (items.length > GROUPING_THRESHOLD) {
109 |       const label = TYPE_LABELS[type] ?? type + " notifications";
110 |       result.push({
111 |         type,
112 |         title: items.length + " " + label,
113 |         body: "You have " + items.length + " " + label,
114 |         count: items.length,
115 |         is_grouped: true,
116 |         items: items.sort(
117 |           (a, b) =>
118 |             new Date(b.created_at).getTime() - new Date(a.created_at).getTime()
119 |         ),
120 |       });
121 |     } else {
122 |       for (const item of items) {
123 |         result.push({
124 |           type: item.type,
125 |           title: item.title,
126 |           body: item.body ?? "",
127 |           count: 1,
128 |           is_grouped: false,
129 |           items: [item],
130 |         });
131 |       }
132 |     }
133 |   }
134 |
135 |   return result.sort((a, b) => {
136 |     const aFirst = a.items[0];
137 |     const bFirst = b.items[0];
138 |     if (!aFirst || !bFirst) return 0;
139 |     return (
140 |       new Date(bFirst.created_at).getTime() -
141 |       new Date(aFirst.created_at).getTime()
142 |     );
143 |   });
144 | }
145 |
146 | export function createBatchedPeerMilestonePayload(
147 |   studentId: string,
148 |   milestoneDescriptions: string[]
149 | ): {
150 |   user_id: string;
151 |   type: string;
152 |   title: string;
153 |   body: string;
154 |   is_read: boolean;
155 |   metadata: Record<string, unknown>;
156 | } {
157 |   const count = milestoneDescriptions.length;
158 |   const title =
159 |     count === 1 ? "Classmate Milestone" : count + " Classmate Milestones";
160 |   const body =
161 |     count === 1
162 |       ? milestoneDescriptions[0] ?? ""
163 |       : count + " classmates achieved milestones recently!";

```

```

164 |
165 |     return {
166 |       user_id: studentId,
167 |       type: "peer_milestone",
168 |       title,
169 |       body,
170 |       is_read: false,
171 |       metadata: {
172 |         is_batched: true,
173 |         milestone_count: count,
174 |         descriptions: milestoneDescriptions,
175 |       },
176 |     };
177 |   }
178 | }

```

FILE: [src/lib/notificationCta.ts](#)

TYPE: .ts | LINES: 1-88 | SHA256: 335363a503a01e2127333f83214997d9e256df6d57235356846b28bc86f96985

```

1 | /**
2 |  * Notification CTA resolver – maps notification type + role to a call-to-action.
3 |  *
4 |  * Pure function with no side effects; safe to call in render paths.
5 |  *
6 |  * Design: ADR-19
7 |  * Requirements: 2.29
8 |  */
9 |
10 | import type { UserRole } from "@types/app";
11 | import type { NotificationType } from "@hooks/useNotifications";
12 |
13 | export interface NotificationCta {
14 |   labelKey: string;
15 |   route: string;
16 | }
17 |
18 | export interface NotificationWithMetadata {
19 |   type: NotificationType;
20 |   metadata: Record<string, unknown> | null;
21 | }
22 |
23 | /**
24 |  * Resolve a CTA for a notification.
25 |  *
26 |  * Returns `null` when no CTA is applicable for the given type.
27 |  *
28 |  * @example
29 |  * resolveNotificationCta({ type: 'grade_released', metadata: { submission_id: 'abc' } }, 'student')
30 |  * // !' { labelKey: 'notifications.cta.viewGrade', route: '/student/grades/abc' }
31 |  */
32 | export function resolveNotificationCta(
33 |   n: NotificationWithMetadata,
34 |   role: UserRole
35 | ): NotificationCta | null {
36 |   const meta = n.metadata ?? {};
37 |
38 |   switch (n.type) {
39 |     case "grade_released":
40 |       return {
41 |         labelKey: "notifications.cta.viewGrade",
42 |         route: `/${role}/grades/${meta.submission_id ?? ""}`,
43 |       };
44 |     case "new_assignment":
45 |       return {
46 |         labelKey: "notifications.cta.openAssignment",
47 |         route: `/${role}/assignments/${meta.assignment_id ?? ""}`,
48 |       };
49 |     case "badge_earned":
50 |       return {
51 |         labelKey: "notifications.cta.viewBadge",
52 |         route: `/student/gamification#badge-${meta.badge_id ?? ""}`,
53 |       };
54 |     case "streak_at_risk":
55 |       return {
56 |         labelKey: "notifications.cta.viewStreak",
57 |         route: "/student/gamification",
58 |       };
59 |     case "at_risk_alert":
60 |       return {
61 |         labelKey: "notifications.cta.viewProgress",
62 |         route: "/student/progress",
63 |       };
64 |     case "peer_milestone":
65 |       return {
66 |         labelKey: "notifications.cta.viewLeaderboard",
67 |         route: "/student/leaderboard",
68 |       };
69 |     case "perfect_day_nudge":
70 |       return {
71 |         labelKey: "notifications.cta.viewHabits",
72 |         route: "/student/habits",
73 |       };
74 |     case "prerequisite_unlocked":
75 |       return {
76 |         labelKey: "notifications.cta.viewAssignment",
77 |         route: `/student/assignments/${meta.assignment_id ?? ""}`,
78 |       };
79 |     case "digest":
80 |       return {
81 |         labelKey: "notifications.cta.viewAll",
82 |         route: `/${role}/notifications`,
83 |       };
84 |     default:
85 |       return null;

```

```
86 | }
87 | }
88 |
```

#### FILE: src/lib/notificationPresentation.ts

TYPE: .ts | LINES: 1-15 | SHA256: ec908c3168bacbc84312724e4f88651e15841e8386a67ed3b129e80c240780c0

```
1 | const humanizeIdentifier = (value: string): string =>
2 |   value
3 |     .replace(/[_-]+/g, " ")
4 |     .replace(/\b\w/g, (character) => character.toUpperCase());
5 |
6 | export const formatNotificationTitle = (title: string): string => {
7 |   const badgePrefix = "Badge earned:";
8 |   if (!title.toLocaleLowerCase().startsWith(badgePrefix.toLocaleLowerCase())) {
9 |     return title;
10 |   }
11 |
12 |   const badgeName = title.slice(badgePrefix.length).trim();
13 |   return badgeName ? `${badgePrefix} ${humanizeIdentifier(badgeName)} ` : title;
14 | };
15 |
```

#### FILE: src/lib/offlineQueue.ts

TYPE: .ts | LINES: 1-93 | SHA256: 736dbd2209021fbedc2d534b4e4a316497a7c039307f2482d4f97900aa1d8244

```
1 | // Task 54: Offline Queue – queue events when offline, flush on reconnect
2 | const QUEUE_KEY = "edevise_offline_queue";
3 | const MAX_RETRIES = 3;
4 |
5 | interface QueuedEvent {
6 |   id: string;
7 |   payload: unknown;
8 |   handler: string;
9 |   retries: number;
10 |   queuedAt: number;
11 | }
12 |
13 | function getQueue(): QueuedEvent[] {
14 |   try {
15 |     const raw = localStorage.getItem(QUEUE_KEY);
16 |     return raw ? JSON.parse(raw) : [];
17 |   } catch {
18 |     return [];
19 |   }
20 | }
21 |
22 | function saveQueue(queue: QueuedEvent[]): void {
23 |   try {
24 |     localStorage.setItem(QUEUE_KEY, JSON.stringify(queue));
25 |   } catch {
26 |     /* ignore */
27 |   }
28 | }
29 |
30 | type FlushHandler = (payload: unknown) => Promise<void>;
31 | const handlers = new Map<string, FlushHandler>();
32 |
33 | export const offlineQueue = {
34 |   registerHandler(name: string, handler: FlushHandler): void {
35 |     handlers.set(name, handler);
36 |   },
37 |
38 |   enqueue(handlerName: string, payload: unknown): void {
39 |     const queue = getQueue();
40 |     queue.push({
41 |       id: `${Date.now()}_${Math.random().toString(36).slice(2, 8)}`,
42 |       payload,
43 |       handler: handlerName,
44 |       retries: 0,
45 |       queuedAt: Date.now(),
46 |     });
47 |     saveQueue(queue);
48 |   },
49 |
50 |   async flush(): Promise<{ flushed: number; failed: number }> {
51 |     const queue = getQueue();
52 |     if (queue.length === 0) return { flushed: 0, failed: 0 };
53 |
54 |     let flushed = 0;
55 |     let failed = 0;
56 |     const remaining: QueuedEvent[] = [];
57 |
58 |     for (const event of queue) {
59 |       const handler = handlers.get(event.handler);
60 |       if (!handler) {
61 |         remaining.push(event);
62 |         failed++;
63 |         continue;
64 |       }
65 |       try {
66 |         await handler(event.payload);
67 |         flushed++;
68 |       } catch {
69 |         event.retries++;
70 |         if (event.retries < MAX_RETRIES) remaining.push(event);
71 |         else failed++;
72 |       }
73 |     }
74 |
75 |     saveQueue(remaining);
76 |     return { flushed, failed };
77 |   },
78 | }
```



```

79 |   init(): () => void {
80 |     const handler = () => {
81 |       offlineQueue
82 |         .flush()
83 |         .catch((err) => console.error("[OfflineQueue] flush failed:", err));
84 |     };
85 |     window.addEventListener("online", handler);
86 |     return () => window.removeEventListener("online", handler);
87 |   },
88 |
89 |   size(): number {
90 |     return getQueue().length;
91 |   },
92 | };
93 |

```

FILE: [src/lib/onboardingChecklist.ts](#)

TYPE: .ts | LINES: 1-94 | SHA256: cd04f9d255731a48747f469201af29f86298959ae6d1ee9b14f5e476e72df68c

```

1 | // =====
2 | // onboardingChecklist – Role-specific Quick Start checklist items
3 | // =====
4 |
5 | import type { UserRole } from "@types/app";
6 |
7 | export interface ChecklistItemDef {
8 |   id: string;
9 |   label: string;
10 |  route: string;
11 | }
12 |
13 | const ADMIN_CHECKLIST: ChecklistItemDef[] = [
14 |   {
15 |     id: "create-ilo",
16 |     label: "Create your first ILO",
17 |     route: "/admin/outcomes",
18 |   },
19 |   { id: "create-program", label: "Create a program", route: "/admin/programs" },
20 |   {
21 |     id: "invite-coordinator",
22 |     label: "Invite a coordinator",
23 |     route: "/admin/users",
24 |   },
25 |   { id: "invite-teacher", label: "Invite a teacher", route: "/admin/users" },
26 | ];
27 |
28 | const COORDINATOR_CHECKLIST: ChecklistItemDef[] = [
29 |   {
30 |     id: "create-plo",
31 |     label: "Create your first PLO",
32 |     route: "/coordinator/plos",
33 |   },
34 |   {
35 |     id: "map-plo-ilo",
36 |     label: "Map a PLO to an ILO",
37 |     route: "/coordinator/plos",
38 |   },
39 |   {
40 |     id: "view-matrix",
41 |     label: "View the curriculum matrix",
42 |     route: "/coordinator/matrix",
43 |   },
44 | ];
45 |
46 | const TEACHER_CHECKLIST: ChecklistItemDef[] = [
47 |   { id: "create-clo", label: "Create your first CLO", route: "/teacher/clos" },
48 |   { id: "build-rubric", label: "Build a rubric", route: "/teacher/rubrics" },
49 |   {
50 |     id: "create-assignment",
51 |     label: "Create an assignment",
52 |     route: "/teacher/assignments",
53 |   },
54 |   {
55 |     id: "grade-submission",
56 |     label: "Grade a submission",
57 |     route: "/teacher/grading",
58 |   },
59 | ];
60 |
61 | const STUDENT_CHECKLIST: ChecklistItemDef[] = [
62 |   {
63 |     id: "view-progress",
64 |     label: "Check your learning progress",
65 |     route: "/student/progress",
66 |   },
67 |   {
68 |     id: "write-journal",
69 |     label: "Write a journal entry",
70 |     route: "/student/journal",
71 |   },
72 |   {
73 |     id: "view-leaderboard",
74 |     label: "View the leaderboard",
75 |     route: "/student/leaderboard",
76 |   },
77 |   {
78 |     id: "view-habits",
79 |     label: "Explore your habit tracker",
80 |     route: "/student/habits",
81 |   },
82 | ];
83 |
84 | const CHECKLIST_MAP: Partial<Record<UserRole, ChecklistItemDef[]>> = {
85 |   admin: ADMIN_CHECKLIST,

```

```

86 |     coordinator: COORDINATOR_CHECKLIST,
87 |     teacher: TEACHER_CHECKLIST,
88 |     student: STUDENT_CHECKLIST,
89 |   };
90 |
91 |   export const getChecklistForRole = (role: UserRole): ChecklistItemDef[] => {
92 |     return CHECKLIST_MAP[role] ?? [];
93 |   };
94 |

```

FILE: src/lib/onboardingConstants.ts

TYPE: .ts | LINES: 1-231 | SHA256: 8501df5abeab660f120e27f69edb4df0abed4c7a6e6f7f9a8d2cfdb38b25f0d6

[illegible]

[illegible]

```

196 |
197 | export const MAX_MICRO_DISMISSALS = 3;
198 |
199 | // % % Goal Difficulty Thresholds % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
200 |
201 | /** Cohort completion rate thresholds: >= 80% = easy, >= 50% = moderate, < 50% = ambitious */
202 | export const GOAL_DIFFICULTY_THRESHOLDS = {
203 |   easy: 80,
204 |   moderate: 50,
205 | } as const;
206 |
207 | // % % Starter Session Duration Tiers % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
208 |
209 | export const STARTER_SESSION_TIERS = {
210 |   low: { sessions: 5, duration: 25 },
211 |   moderate: { sessions: 4, duration: 35 },
212 |   high: { sessions: 3, duration: 45 },
213 | } as const;
214 |
215 | export const SELF_EFFICACY_TIER_THRESHOLDS = {
216 |   low: 40,
217 |   moderate: 70,
218 | } as const; // < 40 = low, 40-70 = moderate, > 70 = high
219 |
220 | // % % Profiling Dimensions % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
221 |
222 | export const PROFILING_DIMENSIONS = [
223 |   "personality",
224 |   "self_efficacy",
225 |   "study_strategy",
226 |   "learning_style",
227 |   "baseline",
228 | ] as const;
229 |
230 | export type ProfilingDimension = (typeof PROFILING_DIMENSIONS)[number];
231 |

```

**FILE: [src/lib/onboardingSchemas.ts](#)**

TYPE: .ts | LINES: 1-129 | SHA256: 618f4f7fa7d7d2718c3f3560b0cf1d956b2d3a7a9cb7eb25eba18ed4be41f04a

```

1 | import { z } from "zod";
2 |
3 | // % % Response Schemas % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
4 |
5 | export const likertResponseSchema = z.object({
6 |   question_id: z.string().uuid(),
7 |   selected_option: z.number().int().min(1).max(5),
8 | });
9 |
10 | export const varkResponseSchema = z.object({
11 |   question_id: z.string().uuid(),
12 |   selected_option: z.number().int().min(0).max(3),
13 | });
14 |
15 | export const baselineResponseSchema = z.object({
16 |   question_id: z.string().uuid(),
17 |   selected_option: z.number().int().min(0).max(3),
18 | });
19 |
20 | export const selfEfficacyResponseSchema = z.object({
21 |   question_id: z.string().uuid(),
22 |   selected_option: z.number().int().min(1).max(5),
23 | });
24 |
25 | export const studyStrategyResponseSchema = z.object({
26 |   question_id: z.string().uuid(),
27 |   selected_option: z.number().int().min(1).max(5),
28 | });
29 |
30 | // % % Edge Function Payload Schemas % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
31 |
32 | export const saveResponsesSchema = z.object({
33 |   student_id: z.string().uuid(),
34 |   assessment_type: z.enum([
35 |     "personality",
36 |     "learning_style",
37 |     "baseline",
38 |     "self_efficacy",
39 |     "study_strategy",
40 |   ]),
41 |   assessment_version: z.number().int().min(1),
42 |   course_id: z.string().uuid().optional(),
43 |   responses: z
44 |     .array(
45 |       z.union([
46 |         likertResponseSchema,
47 |         varkResponseSchema,
48 |         baselineResponseSchema,
49 |       ])
50 |     )
51 |     .min(1),
52 | });
53 |
54 | export const processOnboardingSchema = z.object({
55 |   student_id: z.string().uuid(),
56 |   assessment_version: z.number().int().min(1),
57 |   skipped_sections: z.array(
58 |     z.enum([
59 |       "personality",
60 |       "learning_style",
61 |       "baseline",
62 |       "self_efficacy",
63 |       "study_strategy",
64 |     ])

```

```
65     ),
66     baseline_course_ids: z.array(z.string().uuid()),
67     is_day1: z.boolean().default(false),
68 });
69
70 // %% Baseline Configuration Schemas %%%%%%%%%%%
71
72 export const baselineQuestionSchema = z.object({
73   question_text: z.string().min(10).max(1000),
74   options: z.array(z.string().min(1).max(500)).length(4),
75   correct_option: z.number().int().min(0).max(3),
76   clo_id: z.string().uuid(),
77   difficulty_level: z.enum(["easy", "medium", "hard"]),
78 });
79
80 export const baselineTestConfigSchema = z.object({
81   course_id: z.string().uuid(),
82   time_limit_minutes: z.number().int().min(5).max(60).default(15),
83   is_active: z.boolean(),
84 });
85
86 // %% Starter Week & Goal Schemas %%%%%%%%%%%
87
88 export const starterWeekSessionSchema = z.object({
89   course_id: z.string().uuid().nullable(),
90   session_type: z.enum(["reading", "practice", "review", "exploration"]),
91   suggested_date: z.string().date(),
92   suggested_time_slot: z.enum(["morning", "afternoon", "evening"]),
93   duration_minutes: z.number().int().min(25).max(50),
94   description: z.string().min(10).max(500),
95 });
96
97 export const goalSuggestionSchema = z.object({
98   goal_text: z.string().min(10).max(500),
99   smart_specific: z.string().optional(),
100  smart_measurable: z.string().optional(),
101  smart_achievable: z.string().optional(),
102  smart_relevant: z.string().optional(),
103  smart_timebound: z.string().date().optional(),
104  difficulty: z.enum(["easy", "moderate", "ambitious"]),
105 });
106
107 export const smartGoalTemplateSchema = z.object({
108   specific: z.string().min(5).max(200),
109   measurable: z.string().min(5).max(200),
110   achievable: z.string().min(5).max(200),
111   relevant: z.string().min(1).max(200),
112   timebound: z.string().date(),
113 });
114
115 // %% Inferred Types %%%%%%%%%%%
116
117 export type LikertResponse = z.infer<typeof likertResponseSchema>;
118 export type VARKResponse = z.infer<typeof varkResponseSchema>;
119 export type BaselineResponse = z.infer<typeof baselineResponseSchema>;
120 export type SelfEfficacyResponse = z.infer<typeof selfEfficacyResponseSchema>;
121 export type StudyStrategyResponse = z.infer<typeof studyStrategyResponseSchema>;
122 export type SaveResponsesInput = z.infer<typeof saveResponsesSchema>;
123 export type ProcessOnboardingInput = z.infer<typeof processOnboardingSchema>;
124 export type BaselineQuestion = z.infer<typeof baselineQuestionSchema>;
125 export type BaselineTestConfig = z.infer<typeof baselineTestConfigSchema>;
126 export type StarterWeekSession = z.infer<typeof starterWeekSessionSchema>;
127 export type GoalSuggestion = z.infer<typeof goalSuggestionSchema>;
128 export type SmartGoalTemplate = z.infer<typeof smartGoalTemplateSchema>;
```

FILE: src/lib/outcomeChain.ts

```
TYPE: .ts | LINES: 1-333 | SHA256: bd7b222003b54c7cbd749def420ecda3d7cc8bff5cb52475456542a6c7e1427b
```

```

1 //
2 // outcomeChain – pure assembler for the end-to-end OBE outcome chain
3 // Feature: qa-partner-review-remediation, Req 16
4 // =====
5 //
6 // This module contains the *pure* graph-walking / assembly logic behind
7 // `useOutcomeChain` (src/hooks/useOutcomeChain.ts). Keeping it side-effect free
8 // lets it be unit-tested directly (task 26.3) without mocking Supabase.
9 //
10 // Mapping-direction convention (confirmed against the existing hooks –
11 // useILOs.useDeleteILO, usePLOs.useDeletePLO, useCLOs.useCreateCLO):
12 //   outcome_mappings.source_outcome_id = PARENT (higher level)
13 //   outcome_mappings.target_outcome_id = CHILD (lower level)
14 // so PLOs under an ILO are the `target`s of rows whose `source` is the ILO, and
15 // CLOs under a PLO are the `target`s of rows whose `source` is that PLO.
16 //
17 // The assembled chain is: ILO !' GA !' PLO !' CLO !' Assessment !' Rubric !'
18 // Student !' Attainment (Req 16.1). Graduate Attributes are placed as a level
19 // between the ILO and the PLOs (Req 16.3).
20
21 // % % % Raw input row shapes (column-explicit; mirror the hook's selects) % % % % %
22
23 export interface RawChainOutcome {
24   id: string;
25   title: string;
26   type: "ILO" | "PLO" | "CLO" | "SUB_CLO";
27   blooms_level: string | null;
28   course_id: string | null;
29 }
30
31 export interface RawGraduateAttributeMapping {
32   graduate_attribute_id: string;
33   outcome_id: string;
34   weight: number | null;
35 }

```

[illegible]

[illegible]

```

244 | }
245 |
246 | // %% Assessments grouped by CLO (assignments.clo_weights[.clo_id) % % % % % % %
247 | const assessmentsByClo = new Map<string, AssessmentNode[]>();
248 | for (const a of input.assignments) {
249 |   for (const cw of a.clo_weights) {
250 |     const list = assessmentsByClo.get(cw.clo_id) ?? [];
251 |     list.push({
252 |       id: a.id,
253 |       title: a.title,
254 |       weight: cw.weight,
255 |       rubric: a.rubric_id ? rubricById.get(a.rubric_id) ?? null : null,
256 |     });
257 |     assessmentsByClo.set(cw.clo_id, list);
258 |   }
259 | }
260 |
261 | // %% CLOs grouped by their parent PLO (cloMappings: source = PLO) % % % % % % %
262 | const cloById = new Map(input.clos.map((c) => [c.id, c]));
263 | const closByPlo = new Map<string, CloNode[]>();
264 | for (const m of input.cloMappings) {
265 |   const clo = cloById.get(m.target_outcome_id);
266 |   if (!clo) continue;
267 |   const att = attainmentIndex.get(clo.id);
268 |   const node: CloNode = {
269 |     id: clo.id,
270 |     title: clo.title,
271 |     bloomsLevel: clo.blooms_level,
272 |     attainmentPercent: att?.aggregate ?? null,
273 |     weight: m.weight,
274 |     assessments: assessmentsByClo.get(clo.id) ?? [],
275 |     rubrics: rubricsByClo.get(clo.id) ?? [],
276 |     students: (att?.students ?? []).map((row) => ({
277 |       studentId: row.student_id as string,
278 |       studentName: studentNameById.get(row.student_id as string) ?? "-",
279 |       attainmentPercent: row.attainment_percent,
280 |     })),
281 |   };
282 |   const list = closByPlo.get(m.source_outcome_id) ?? [];
283 |   list.push(node);
284 |   closByPlo.set(m.source_outcome_id, list);
285 | }
286 |
287 | // %% PLOs under the start ILO (ploMappings: source = start ILO) % % % % % % %
288 | const ploById = new Map(input.plos.map((p) => [p.id, p]));
289 | const plos: PloNode[] = [];
290 | for (const m of input.ploMappings) {
291 |   const plo = ploById.get(m.target_outcome_id);
292 |   if (!plo) continue;
293 |   plos.push({
294 |     id: plo.id,
295 |     title: plo.title,
296 |     attainmentPercent: attainmentIndex.get(plo.id)?.aggregate ?? null,
297 |     weight: m.weight,
298 |     clos: closByPlo.get(plo.id) ?? [],
299 |   });
300 | }
301 |
302 | // %% Graduate Attributes mapped to the start ILO (level between ILO/PLO) % % %
303 | const startAttainment = attainmentIndex.get(start.id)?.aggregate ?? null;
304 | const gaById = new Map(input.graduateAttributes.map((g) => [g.id, g]));
305 | const graduateAttributes: GraduateAttributeNode[] = [];
306 | for (const m of input.gaMappings) {
307 |   const ga = gaById.get(m.graduate_attribute_id);
308 |   if (!ga) continue;
309 |   graduateAttributes.push({
310 |     id: ga.id,
311 |     name: ga.name,
312 |     weight: m.weight,
313 |     // Within this chain the GA connects to the start ILO, so the ILO's
314 |     // aggregate attainment is the meaningful figure to surface at the node.
315 |     attainmentPercent: startAttainment,
316 |   });
317 | }
318 |
319 | const isEmpty = graduateAttributes.length === 0 && plos.length === 0;
320 |
321 | return {
322 |   start: {
323 |     id: start.id,
324 |     title: start.title,
325 |     type: start.type,
326 |     attainmentPercent: startAttainment,
327 |   },
328 |   graduateAttributes,
329 |   plos,
330 |   isEmpty,
331 | };
332 | };
333 |

```

## FILE: src/lib/pagination.ts

TYPE: .ts | LINES: 1-115 | SHA256: 2de3968f4f1e5401c60b540a20096ae3dda9e36cb3645c2857074577c4c4672f

```

1 | // Task 6.5: Shared pagination helpers (pure business logic)
2 | // Requirements: 32.2, 33.2, 33.3, 34.3
3 | //
4 | // Single source of pagination math shared by the leaderboard, transaction
5 | // history, marketplace, and discussion surfaces. The low-level range formula
6 | // lives in "@types/pagination" (getPaginationRange, consumed by the list
7 | // hooks); these helpers wrap it with totality guarantees and add the
8 | // "is there another page?" and "how many pages?" predicates the student
9 | // surfaces need for "load more" behavior.
10 |

```



```

11 | import { getPaginationRange } from "@types/pagination";
12 |
13 | /**
14 |  * Inclusive, zero-based row range for a single page.
15 |  *
16 |  * Matches the Supabase PostgREST `.range(from, to)` convention exactly: both
17 |  * bounds are inclusive and zero-based, so a page of `pageSize` rows spans
18 |  * `to - from + 1 === pageSize` indices.
19 |  */
20 | export interface PageRange {
21 |   /** Inclusive zero-based index of the first row on the page. */
22 |   from: number;
23 |   /** Inclusive zero-based index of the last row on the page. */
24 |   to: number;
25 | }
26 |
27 | /**
28 |  * Normalizes a 1-based page number to a safe positive integer.
29 |  * Non-finite, fractional, or sub-1 inputs collapse to the first page so the
30 |  * helpers stay total and never emit a negative `from`.
31 |  */
32 | function normalizePage(page: number): number {
33 |   if (!Number.isFinite(page)) return 1;
34 |   const floored = Math.floor(page);
35 |   return floored < 1 ? 1 : floored;
36 | }
37 |
38 | /**
39 |  * Normalizes a page size to a safe positive integer ("e 1").
40 |  * A page size below 1 would make pagination unable to make progress, so it is
41 |  * clamped to 1.
42 |  */
43 | function normalizePageSize(pageSize: number): number {
44 |   if (!Number.isFinite(pageSize)) return 1;
45 |   const floored = Math.floor(pageSize);
46 |   return floored < 1 ? 1 : floored;
47 | }
48 |
49 | /**
50 |  * Normalizes a total count to a non-negative integer.
51 |  */
52 | function normalizeTotal(total: number): number {
53 |   if (!Number.isFinite(total)) return 0;
54 |   const floored = Math.floor(total);
55 |   return floored < 0 ? 0 : floored;
56 | }
57 |
58 | /**
59 |  * Returns the inclusive zero-based `{ from, to }` row range for a page.
60 |  *
61 |  * Page numbers are 1-based to match the existing list hooks. The range formula
62 |  * is delegated to `getPaginationRange` so there is a single source of truth for
63 |  * the `from = (page - 1) * pageSize` math.
64 |  *
65 |  * @param page - 1-based page number (clamped to "e 1").
66 |  * @param pageSize - rows per page (clamped to "e 1").
67 |  */
68 | export function toRange(page: number, pageSize: number): PageRange {
69 |   const { from, to } = getPaginationRange(
70 |     normalizePage(page),
71 |     normalizePageSize(pageSize)
72 |   );
73 |   return { from, to };
74 | }
75 |
76 | /**
77 |  * Total number of pages required to hold `total` rows at `pageSize` per page.
78 |  *
79 |  * This is the exact mathematical count: `ceil(total / pageSize)`. An empty
80 |  * collection therefore has `0` pages, which keeps Property 4 (completeness)
81 |  * exact – concatenating zero pages reproduces an empty source. Callers that
82 |  * want a "Page 1 of 1" display for empty data should apply their own
83 |  * `Math.max(1, ...)` at the presentation layer.
84 |  *
85 |  * @param total - total row count (clamped to "e 0").
86 |  * @param pageSize - rows per page (clamped to "e 1").
87 |  */
88 | export function pageCount(total: number, pageSize: number): number {
89 |   return Math.ceil(normalizeTotal(total) / normalizePageSize(pageSize));
90 | }
91 |
92 | /**
93 |  * Whether at least one more row exists beyond the given page.
94 |  *
95 |  * Returns `true` exactly when there is a further item past the end of `page`,
96 |  * and `false` on (or past) the final page. This drives "load more" affordances
97 |  * without silently truncating: the consumer keeps paging while `hasMore` is
98 |  * `true`.
99 |  *
100 |  * Equivalent to `page < pageCount(total, pageSize)`.
101 |  *
102 |  * @param page - 1-based page number (clamped to "e 1").
103 |  * @param pageSize - rows per page (clamped to "e 1").
104 |  * @param total - total row count across all pages (clamped to "e 0").
105 |  */
106 | export function hasMore(
107 |   page: number,
108 |   pageSize: number,
109 |   total: number
110 | ): boolean {
111 |   const p = normalizePage(page);
112 |   const ps = normalizePageSize(pageSize);
113 |   return normalizeTotal(total) > p * ps;
114 | }

```

## FILE: src/lib/passwordVisibility.ts

TYPE: .ts | LINES: 1-89 | SHA256: d68edaca0bd95686f14693aab1d5482feaceb697c3ad2da1fe55d0c6edd23ba6

```

1  /**
2   * Pure mutual-exclusion reducer for password-field visibility.
3   *
4   * A form may contain N password fields (e.g. "new password" + "confirm
5   * password"). Requirement 5.5 mandates that at most one password field may
6   * display its characters in plain text at a time: revealing one field masks
7   * every other field. This module models that invariant as a pure reducer so it
8   * can be unit/property tested in isolation and wired into a
9   * `PasswordVisibilityGroup` React context via `useReducer`.
10  */
11  * The state tracks only the id of the single currently-revealed field (or
12  * `null` when every field is masked), which makes the mutual-exclusion
13  * invariant structurally impossible to violate.
14  *
15  * _Requirements: 5.5_
16  */
17
18  /**
19   * Visibility state for a group of password fields.
20   *
21   * `revealed` holds the id of the single field currently shown in plain text,
22   * or `null` when all fields are masked. Representing the revealed field as a
23   * single id (rather than a per-field boolean map) guarantees at most one field
24   * is ever revealed.
25   */
26  export interface PasswordVisibilityState {
27    readonly revealed: string | null;
28  }
29
30  /**
31   * Actions a password field can dispatch.
32   *
33   * - `reveal`: show the field with the given id in plain text (hiding all others).
34   * - `hide`: mask the field with the given id (no-op for any other field).
35   */
36  export type PasswordVisibilityAction =
37    | { readonly type: "reveal"; readonly id: string }
38    | { readonly type: "hide"; readonly id: string };
39
40  /**
41   * The initial state for a freshly-mounted group: every field masked.
42   */
43  export const initialPasswordVisibilityState: PasswordVisibilityState = {
44    revealed: null,
45  };
46
47  /**
48   * Pure reducer enforcing password-visibility mutual exclusion.
49   *
50   * - Revealing a field makes it the sole revealed field, masking any other.
51   * - Hiding clears the revealed field only when it is the one currently shown;
52   *   hiding any other (already-masked) field leaves state unchanged.
53   *
54   * The function never mutates its inputs and is fully deterministic.
55   *
56   * _Requirements: 5.5_
57   */
58  export function passwordVisibilityReducer(
59    state: PasswordVisibilityState,
60    action: PasswordVisibilityAction
61  ): PasswordVisibilityState {
62    switch (action.type) {
63      case "reveal":
64        // Revealing one field hides all others by replacing the revealed id.
65        // Return the same reference when nothing actually changes.
66        return state.revealed === action.id ? state : { revealed: action.id };
67      case "hide":
68        // Hiding only clears the matching field; other fields are already masked.
69        return state.revealed === action.id ? { revealed: null } : state;
70      default: {
71        // Exhaustiveness guard: a new action type would surface here at compile
72        // time. `never` is assignable to the return type, so returning it both
73        // satisfies the type checker and keeps the variable used.
74        const _exhaustive: never = action;
75        return _exhaustive;
76      }
77    }
78  }
79
80  /**
81   * Selector: is the field with the given id currently revealed?
82   */
83  export function isFieldRevealed(
84    state: PasswordVisibilityState,
85    id: string
86  ): boolean {
87    return state.revealed === id;
88  }
89

```

## FILE: src/lib/peerMilestoneNotifier.ts

TYPE: .ts | LINES: 1-32 | SHA256: 3a6c31487c311ae30e8208193aab026a54e3c996a3d4338e1c3036a7c291525a

```

1  // =====
2  // Peer Milestone Notifier – Creates notifications for course peers on milestones
3  // =====
4
5  export type MilestoneType = "level_up" | "rare_badge" | "streak_milestone";
6

```

```

7 | export interface PeerMilestonePayload {
8 |   triggeringStudentId: string;
9 |   milestoneType: MilestoneType;
10 |   milestoneDetail: string;
11 | }
12 |
13 | /**
14 |  * Build notification message based on milestone type.
15 |  */
16 | export function buildMilestoneMessage(
17 |   studentName: string,
18 |   milestoneType: MilestoneType,
19 |   milestoneDetail: string
20 | ): string {
21 |   switch (milestoneType) {
22 |     case "level_up":
23 |       return `Your classmate ${studentName} just hit ${milestoneDetail}!`;
24 |     case "rare_badge":
25 |       return `${studentName} just earned the ${milestoneDetail} badge!`;
26 |     case "streak_milestone":
27 |       return `${studentName} is on a ${milestoneDetail}-day streak!`;
28 |     default:
29 |       return `${studentName} achieved a milestone: ${milestoneDetail}`;
30 |   }
31 | }
32 |

```

## FILE: src/lib/percentileBand.ts

TYPE: .ts | LINES: 1-48 | SHA256: f4899ae2b8bd5abc5e319fdffeb50548e70d5249657720bb51cb83053a4f5428

```

1 | // Task 148.1: Percentile Band utility
2 | // Requirements: 131.1, 131.2, 131.3
3 |
4 | export type PercentileBandLabel =
5 |   | "Top 10%"
6 |   | "Top 25%"
7 |   | "Top 50%"
8 |   | "Bottom 50%";
9 |
10 | export type PercentileBandResult =
11 |   | { type: "exact"; rank: number }
12 |   | { type: "band"; band: PercentileBandLabel };
13 |
14 | /**
15 |  * Calculate percentile band for a student's rank.
16 |  * - Top 10 students always see exact rank (Req 131.1)
17 |  * - Others see percentile band: Top 10%, Top 25%, Top 50%, Bottom 50% (Req 131.2)
18 |  * - Percentile = (rank / totalStudents) * 100 (Req 131.3)
19 |  */
20 | export function calculatePercentileBand(
21 |   rank: number,
22 |   totalStudents: number
23 | ): PercentileBandResult {
24 |   if (totalStudents <= 0 || rank <= 0) {
25 |     return { type: "exact", rank: Math.max(1, rank) };
26 |   }
27 |
28 |   // Top 10 always see exact rank
29 |   if (rank <= 10) {
30 |     return { type: "exact", rank };
31 |   }
32 |
33 |   const percentile = (rank / totalStudents) * 100;
34 |
35 |   if (percentile <= 10) return { type: "band", band: "Top 10%" };
36 |   if (percentile <= 25) return { type: "band", band: "Top 25%" };
37 |   if (percentile <= 50) return { type: "band", band: "Top 50%" };
38 |   return { type: "band", band: "Bottom 50%" };
39 | }
40 |
41 | /**
42 |  * Format a percentile band result for display.
43 |  */
44 | export function formatPercentileBand(result: PercentileBandResult): string {
45 |   if (result.type === "exact") return `#${result.rank}`;
46 |   return result.band;
47 | }
48 |

```

## FILE: src/lib/perfectDay.ts

TYPE: .ts | LINES: 1-111 | SHA256: cc67f8b81174696d1396c037e271e0c135c9ce745e083bb36f831fe347b4743d

```

1 | // =====
2 | // perfectDay – idempotent Perfect Day award business logic
3 | // =====
4 | //
5 | // Domain: a student has a "Perfect Day" when all 4 daily habits
6 | // (login, submit, journal, read) are completed in a single day. Completing the
7 | // 4th habit awards a milestone `perfect_day` 50 XP and fires a badge check.
8 | //
9 | // This lives in `src/lib/` (clean-architecture: business logic, not a hook or
10 | // component) so any engagement flow (login, submit, journal, read) can call it
11 | // after writing its `habit_logs` row.
12 | //
13 | // Idempotency: `award-xp` enforces idempotency on `reference_id`, so the
14 | // deterministic `perfect_day:{student_id}:{date}` reference guarantees the 50 XP
15 | // is paid at most once per student per day – repeated calls (e.g. a student who
16 | // completes a 5th action, or re-triggers a habit) cannot farm XP.
17 |
18 | import { supabase } from "@lib/supabase";
19 |
20 | // The 4 canonical daily academic habits (see domain-knowledge: Daily Habits).
21 | const REQUIRED_HABITS = ["login", "submit", "journal", "read"] as const;
22 |

```

```

23 | export const PERFECT_DAY_XP = 50;
24 |
25 | export interface PerfectDayResult {
26 |   /** True when all 4 habits were present and the award was attempted. */
27 |   awarded: boolean;
28 |   /** The habit types recorded for the student today (deduplicated). */
29 |   completedHabits: string[];
30 |   /** The UTC date (YYYY-MM-DD) the check ran for. */
31 |   date: string;
32 | }
33 |
34 | /** Today's date in UTC (YYYY-MM-DD), matching the midnight-UTC habit reset rule. */
35 | const todayUtc = (): string => new Date().toISOString().split("T")[0] as string;
36 |
37 | /**
38 |  * Reads today's `habit_logs` for the student and, when all 4 daily habits are
39 |  * present, awards the idempotent `perfect_day` 50 XP milestone and fires a
40 |  * `check-badges` pass.
41 |  *
42 |  * Fire-and-forget: errors are caught and logged (never thrown) so this can be
43 |  * safely awaited at the tail of an engagement flow without breaking it. Returns
44 |  * a result so callers can react (e.g. celebrate) if useful.
45 |  */
46 | export const awardPerfectDayIfComplete = async (
47 |   studentId: string
48 | ): Promise<PerfectDayResult> => {
49 |   const date = todayUtc();
50 |
51 |   if (!studentId) {
52 |     return { awarded: false, completedHabits: [], date };
53 |   }
54 |
55 |   try {
56 |     const { data, error } = await supabase
57 |       .from("habit_logs")
58 |       .select("habit_type")
59 |       .eq("student_id", studentId)
60 |       .eq("date", date);
61 |
62 |     if (error) throw error;
63 |
64 |     const completedHabits = [
65 |       ...new Set((data ?? []).map((row) => row.habit_type)),
66 |     ];
67 |
68 |     const allComplete = REQUIRED_HABITS.every((habit) =>
69 |       completedHabits.includes(habit)
70 |     );
71 |
72 |     if (!allComplete) {
73 |       return { awarded: false, completedHabits, date };
74 |     }
75 |
76 |     // All 4 habits present !' award idempotent perfect_day milestone XP.
77 |     try {
78 |       await supabase.functions.invoke("award-xp", {
79 |         body: {
80 |           student_id: studentId,
81 |           xp_amount: PERFECT_DAY_XP,
82 |           source: "perfect_day",
83 |           reference_id: `perfect_day:${studentId}:${date}`,
84 |           is_milestone: true,
85 |         },
86 |       });
87 |     } catch {
88 |       console.error("[awardPerfectDayIfComplete] award-xp invocation failed");
89 |     }
90 |
91 |     // Fire the perfect_day badge check.
92 |     try {
93 |       await supabase.functions.invoke("check-badges", {
94 |         body: {
95 |           student_id: studentId,
96 |           trigger: "perfect_day",
97 |         },
98 |       });
99 |     } catch {
100 |       console.error(
101 |         "[awardPerfectDayIfComplete] check-badges invocation failed"
102 |       );
103 |     }
104 |
105 |     return { awarded: true, completedHabits, date };
106 |   } catch {
107 |     console.error("[awardPerfectDayIfComplete] habit_logs read failed");
108 |     return { awarded: false, completedHabits: [], date };
109 |   }
110 | };
111 |

```

## FILE: src/lib/perfectDayCheck.ts

TYPE: .ts | LINES: 1-14 | SHA256: c3308a43311cdc079d46cd17a0095e097f1e201bc8d789f45a712a4fd83ae4d8

```

1 | export const isPerfectDay = (academicHabits: {
2 |   login: boolean;
3 |   submit: boolean;
4 |   journal: boolean;
5 |   read: boolean;
6 | }): boolean => {
7 |   return (
8 |     academicHabits.login &&
9 |     academicHabits.submit &&
10 |    academicHabits.journal &&
11 |    academicHabits.read

```

```

12 |   );
13 | };
14 |

```

## FILE: src/lib/personalBestLeaderboard.ts

TYPE: .ts | LINES: 1-72 | SHA256: f30e1398ac9b156f9cc4eac8cdeecb9defb56cb32b2801f50be43aaf3e34ccc5

```

1 | // =====
2 | // Personal Best leaderboard utility (Task 147.1)
3 | // Groups xp_transactions by ISO week for the last 8 weeks, identifies personal best.
4 | // Requirements: 129.1, 129.2
5 | // =====
6 |
7 | export interface WeeklyXP {
8 |   weekStart: string; // ISO week start date (Monday)
9 |   weekLabel: string;
10 |   xp: number;
11 |   isCurrentWeek: boolean;
12 |   isPersonalBest: boolean;
13 | }
14 |
15 | /**
16 |  * Get the Monday (ISO week start) for a given date, using UTC.
17 |  */
18 | export function getISOWeekStart(date: Date): Date {
19 |   const d = new Date(date);
20 |   d.setUTCHours(0, 0, 0, 0);
21 |   const day = d.getUTCDay();
22 |   // ISO weeks start on Monday (1). Sunday is 0, so shift it to 7.
23 |   const diff = day === 0 ? 6 : day - 1;
24 |   d.setUTCDate(d.getUTCDate() - diff);
25 |   return d;
26 | }
27 |
28 | /**
29 |  * Compute weekly XP totals from xp_transactions for the last 8 ISO weeks.
30 |  * Each week runs Monday 00:00 to Sunday 23:59:59.
31 |  */
32 | export function computeWeeklyXP(
33 |   transactions: Array<{ xp_amount: number; created_at: string }>,
34 |   referenceDate: Date = new Date()
35 | ): WeeklyXP[] {
36 |   const currentWeekStart = getISOWeekStart(referenceDate);
37 |
38 |   const weeks: WeeklyXP[] = [];
39 |
40 |   for (let i = 7; i >= 0; i--) {
41 |     const weekStart = new Date(currentWeekStart);
42 |     weekStart.setUTCDate(weekStart.getUTCDate() - i * 7);
43 |
44 |     const weekEnd = new Date(weekStart);
45 |     weekEnd.setUTCDate(weekEnd.getUTCDate() + 7);
46 |
47 |     const weekXP = transactions
48 |       .filter((t) => {
49 |         const d = new Date(t.created_at);
50 |         return d >= weekStart && d < weekEnd;
51 |       })
52 |       .reduce((sum, t) => sum + t.xp_amount, 0);
53 |
54 |     weeks.push({
55 |       weekStart: weekStart.toISOString().slice(0, 10),
56 |       weekLabel: `Week ${8 - i}`,
57 |       xp: weekXP,
58 |       isCurrentWeek: i === 0,
59 |       isPersonalBest: false,
60 |     });
61 |   }
62 |
63 |   // Mark personal best – exactly one week with the highest XP (first occurrence)
64 |   const maxXP = Math.max(...weeks.map((w) => w.xp));
65 |   if (maxXP > 0) {
66 |     const bestWeek = weeks.find((w) => w.xp === maxXP);
67 |     if (bestWeek) bestWeek.isPersonalBest = true;
68 |   }
69 |
70 |   return weeks;
71 | }
72 |

```

## FILE: src/lib/plannerUtils.ts

TYPE: .ts | LINES: 1-447 | SHA256: 39029311c5a7d125f8e7dcef1ab9c74d91ea6f1171bc8b7a5f9b65564aeefe6c

```

1 | // =====
2 | // Weekly Planner & Today View – Pure Utility Functions
3 | // =====
4 |
5 | import type {
6 |   GoalProgress,
7 |   PlannerTask,
8 |   PomodoroIntervalType,
9 |   StudySession,
10 |   TaskPriority,
11 |   TimelineItem,
12 |   TimeOfDay,
13 |   WeeklyGoal,
14 |   WeeklyStudyData,
15 | } from "@types/planner";
16 |
17 | // -----
18 | // XP Calculation
19 | // -----
20 |
21 | /**

```

```

22 | * Calculate XP for a completed study session.
23 | * Base 20 + 5 per additional 15-min block, cap 50. Evidence bonus 10.
24 | * Zero XP for sessions < 15 min.
25 | */
26 | export function calculateSessionXP(
27 |   actualDurationMinutes: number,
28 |   hasEvidence: boolean
29 | ): number {
30 |   if (actualDurationMinutes < 15) return 0;
31 |   const blocks = Math.floor(actualDurationMinutes / 15);
32 |   const baseXP = 20 + (blocks - 1) * 5;
33 |   const cappedXP = Math.min(baseXP, 50);
34 |   const evidenceBonus = hasEvidence ? 10 : 0;
35 |   return cappedXP + evidenceBonus;
36 | }
37 |
38 | // -----
39 | // Today View Grouping
40 | // -----
41 |
42 | /** Group timeline items by time of day. Items with null time go to 'todo'. */
43 | export function groupByTimeOfDay(
44 |   items: TimelineItem[]
45 | ): Record<TimeOfDay | "todo", TimelineItem[]> {
46 |   const groups: Record<TimeOfDay | "todo", TimelineItem[]> = {
47 |     morning: [],
48 |     afternoon: [],
49 |     evening: [],
50 |     todo: [],
51 |   };
52 |   for (const item of items) {
53 |     if (!item.time) {
54 |       groups.todo.push(item);
55 |       continue;
56 |     }
57 |     const hour = parseInt(item.time.split(":")[0] ?? "0", 10);
58 |     if (hour < 12) groups.morning.push(item);
59 |     else if (hour < 17) groups.afternoon.push(item);
60 |     else groups.evening.push(item);
61 |   }
62 |   return groups;
63 | }
64 |
65 | // -----
66 | // Task Sorting
67 | // -----
68 |
69 | /** Sort tasks by priority: high !' medium !' low. */
70 | export function sortTasksByPriority(tasks: PlannerTask[]): PlannerTask[] {
71 |   const order: Record<TaskPriority, number> = { high: 0, medium: 1, low: 2 };
72 |   return [...tasks].sort((a, b) => order[a.priority] - order[b.priority]);
73 | }
74 |
75 | // -----
76 | // Deadline Urgency
77 | // -----
78 |
79 | /** Determine deadline urgency based on hours until due. */
80 | export function getDeadlineUrgency(
81 |   dueDate: string,
82 |   now: Date
83 | ): "red" | "yellow" | "green" {
84 |   const due = new Date(dueDate);
85 |   const hoursUntilDue = (due.getTime() - now.getTime()) / (1000 * 60 * 60);
86 |   if (hoursUntilDue <= 24) return "red";
87 |   if (hoursUntilDue <= 72) return "yellow";
88 |   return "green";
89 | }
90 |
91 | // -----
92 | // Session Status
93 | // -----
94 |
95 | /** Check if a session is missed (past scheduled time, not started). */
96 | export function isSessionMissed(session: StudySession, now: Date): boolean {
97 |   if (session.status !== "planned") return false;
98 |   const sessionDateTime = new Date(
99 |     `${session.plannedDate}T${session.plannedStartTime}`
100 |   );
101 |   const endTime = new Date(
102 |     sessionDateTime.getTime() + session.plannedDurationMinutes * 60 * 1000
103 |   );
104 |   return now > endTime;
105 | }
106 |
107 | // -----
108 | // Goal Progress
109 | // -----
110 |
111 | /** Calculate goal progress from sessions and tasks. */
112 | export function calculateGoalProgress(
113 |   goal: WeeklyGoal,
114 |   sessions: StudySession[],
115 |   tasks: PlannerTask[]
116 | ): GoalProgress {
117 |   let currentValue = 0;
118 |   switch (goal.goalType) {
119 |     case "study_hours":
120 |       currentValue =
121 |         sessions
122 |           .filter((s) => s.status === "completed")
123 |           .reduce((sum, s) => sum + (s.actualDurationMinutes ?? 0), 0) / 60;
124 |       break;
125 |     case "sessions_completed":

```

```

126     currentValue = sessions.filter((s) => s.status === "completed").length;
127     break;
128     case "tasks_completed":
129         currentValue = tasks.filter((t) => t.status === "done").length;
130         break;
131     }
132     const percentage =
133     goal.targetValue > 0
134     ? Math.min(Math.round((currentValue / goal.targetValue) * 100), 100)
135     : 0;
136     return {
137         goal,
138         currentValue,
139         percentage,
140         isMet: currentValue >= goal.targetValue,
141     };
142 }
143
144 // -----
145 // Timer Display
146 // -----
147
148 /** Format timer display as MM:SS. */
149 export function formatTimerDisplay(remainingMs: number): string {
150     const totalSeconds = Math.max(0, Math.ceil(remainingMs / 1000));
151     const minutes = Math.floor(totalSeconds / 60);
152     const seconds = totalSeconds % 60;
153     return `${String(minutes).padStart(2, "0")}:${String(seconds).padStart(
154         2,
155         "0"
156     )}`;
157 }
158
159 // -----
160 // Duration Calculation
161 // -----
162
163 /** Calculate actual duration in minutes excluding paused time. */
164 export function calculateActualDuration(
165     startedAt: number,
166     endedAt: number,
167     totalPausedMs: number
168 ): number {
169     const totalMs = endedAt - startedAt - totalPausedMs;
170     return Math.max(0, Math.round(totalMs / 60000));
171 }
172
173 // -----
174 // Week Utilities
175 // -----
176
177 /** Get the Monday of the week containing the given date (YYYY-MM-DD). */
178 export function getWeekStartDate(date: Date): string {
179     const d = new Date(date);
180     const day = d.getDay();
181     const diff = d.getDate() - day + (day === 0 ? -6 : 1);
182     d.setDate(diff);
183     return d.toISOString().split("T")[0] as string;
184 }
185
186 /** Check if a week is in the past (for preventing goal edits). */
187 export function isWeekInPast(weekStartDate: string, today: Date): boolean {
188     const weekEnd = new Date(weekStartDate);
189     weekEnd.setDate(weekEnd.getDate() + 6);
190     weekEnd.setHours(23, 59, 59, 999);
191     return today > weekEnd;
192 }
193
194 // -----
195 // Text Utilities
196 // -----
197
198 /** Count words in a text string. */
199 export function countWords(text: string): number {
200     return text
201         .trim()
202         .split(/\s+/)
203         .filter((w) => w.length > 0).length;
204 }
205
206 // -----
207 // Analytics
208 // -----
209
210 /** Compute weekly study time aggregation for chart data. */
211 export function aggregateWeeklyStudyTime(
212     sessions: StudySession[],
213     weekCount: number,
214     today: Date
215 ): WeeklyStudyData[] {
216     const result: WeeklyStudyData[] = [];
217     for (let i = weekCount - 1; i >= 0; i--) {
218         const weekStart = new Date(today);
219         weekStart.setDate(weekStart.getDate() - weekStart.getDay() + 1 - i * 7);
220         const weekEnd = new Date(weekStart);
221         weekEnd.setDate(weekEnd.getDate() + 6);
222         const weekStartStr = weekStart.toISOString().split("T")[0] as string;
223         const totalMinutes = sessions
224             .filter((s) => {
225                 if (s.status !== "completed") return false;
226                 const d = new Date(s.plannedDate);
227                 return d >= weekStart && d <= weekEnd;
228             })
229             .reduce((sum, s) => sum + (s.actualDurationMinutes ?? 0), 0);

```

```

230     result.push({ weekStartDate: weekStartStr, totalMinutes });
231 }
232 return result;
233 }
234
235 // -----
236 // Pomodoro
237 // -----
238
239 /** Determine Pomodoro interval type based on completed intervals. */
240 export function getPomodoroIntervalType(
241   completedIntervals: number
242 ): PomodoroIntervalType {
243   if (completedIntervals > 0 && completedIntervals % 4 === 0)
244     return "long_break";
245   return completedIntervals % 2 === 0 ? "work" : "break";
246 }
247
248 /** Get Pomodoro interval duration in milliseconds. */
249 export function getPomodoroIntervalDuration(
250   intervalType: PomodoroIntervalType
251 ): number {
252   switch (intervalType) {
253     case "work":
254       return 25 * 60 * 1000;
255     case "break":
256       return 5 * 60 * 1000;
257     case "long_break":
258       return 15 * 60 * 1000;
259   }
260 }
261
262 // -----
263 // Spaced Repetition Review Dates
264 // -----
265
266 import type {
267   SimpleReflectionValues,
268   GibbsReflectionValues,
269   QualityCategory,
270 } from "@types/planner";
271
272 /** Generate 3 review dates at 1, 3, and 7-day intervals from the given date. */
273 export function generateReviewDates(
274   dateStr: string
275 ): Array<{ reviewDate: string; intervalDays: number }> {
276   const intervals = [1, 3, 7];
277   const [year, month, day] = dateStr.split("-").map(Number) as [
278     number,
279     number,
280     number
281   ];
282   return intervals.map((intervalDays) => {
283     const d = new Date(Date.UTC(year, month - 1, day + intervalDays));
284     const y = d.getUTCFullYear();
285     const m = String(d.getUTCMonth() + 1).padStart(2, "0");
286     const dd = String(d.getUTCDate()).padStart(2, "0");
287     return { reviewDate: `${y}-${m}-${dd}`, intervalDays };
288   });
289 }
290
291 /** Check if a review cycle is complete (all 3 intervals completed). */
292 export function isReviewCycleComplete(
293   reviews: Array<{ intervalDays: number; status: string }>
294 ): boolean {
295   const requiredIntervals = [1, 3, 7];
296   if (reviews.length === 0) return false;
297   return requiredIntervals.every((interval) =>
298     reviews.some((r) => r.intervalDays === interval && r.status === "completed")
299   );
300 }
301
302 // -----
303 // Reflection Template Concatenation
304 // -----
305
306 const simpleLabels: Record<keyof SimpleReflectionValues, string> = {
307   whatWentWell: "## What went well?",
308   whatWasChallenging: "## What was challenging?",
309   whatWillChange: "## What will I do differently?",
310 };
311
312 const gibbsLabels: Record<keyof GibbsReflectionValues, string> = {
313   description: "## Description",
314   feelings: "## Feelings",
315   evaluation: "## Evaluation",
316   analysis: "## Analysis",
317   conclusion: "## Conclusion",
318   actionPlan: "## Action Plan",
319 };
320
321 /**
322  * Concatenate reflection template values into a formatted string.
323  * Only includes sections with non-empty content.
324  */
325 export function concatenateReflectionTemplate(
326   type: "simple" | "gibbs",
327   values: SimpleReflectionValues | GibbsReflectionValues
328 ): string {
329   if (type === "simple") {
330     const v = values as SimpleReflectionValues;
331     const sections: string[] = [];
332     for (const key of Object.keys(simpleLabels) as Array<
333       keyof SimpleReflectionValues

```



```

334 |     >) {
335 |         const text = v[key]?.trim();
336 |         if (text) {
337 |             sections.push(`${simpleLabels[key]}\n${text}`);
338 |         }
339 |     }
340 |     return sections.join("\n\n");
341 | }
342 |
343 | const v = values as GibbsReflectionValues;
344 | const sections: string[] = [];
345 | for (const key of Object.keys(gibbsLabels) as Array<
346 |     keyof GibbsReflectionValues
347 | >) {
348 |     const text = v[key]?.trim();
349 |     if (text) {
350 |         sections.push(`${gibbsLabels[key]}\n${text}`);
351 |     }
352 | }
353 | return sections.join("\n\n");
354 | }
355 |
356 | // -----
357 | // Quality Category
358 | // -----
359 |
360 | /** Map a quality score to a category. */
361 | export function getCategory(score: number | null): QualityCategory {
362 |     if (score === null) return "needs_detail";
363 |     if (score >= 80) return "thoughtful";
364 |     if (score >= 30) return "good_effort";
365 |     return "needs_detail";
366 | }
367 |
368 | // -----
369 | // Reflection XP Calculation
370 | // -----
371 |
372 | /**
373 |  * Calculate quality-adjusted reflection XP.
374 |  *
375 |  * - null score !' base XP (capped)
376 |  * - score < 30 !' 5 XP (minimum)
377 |  * - score 30-79 !' base XP (capped)
378 |  * - score >= 80 !' base + 10 bonus (capped)
379 |  *
380 |  * Relevance and depth bonuses add 5 each when >= 70.
381 |  * Session cap: 30, Journal cap: 40.
382 |  */
383 | export function calculateReflectionXP(
384 |     baseXP: number,
385 |     score: number | null,
386 |     relevance: number | null,
387 |     depth: number | null,
388 |     isJournal: boolean
389 | ): number {
390 |     const cap = isJournal ? 40 : 30;
391 |
392 |     // Null score !' return base capped
393 |     if (score === null) {
394 |         return Math.min(baseXP, cap);
395 |     }
396 |
397 |     // Low quality !' minimum 5 XP
398 |     if (score < 30) {
399 |         return 5;
400 |     }
401 |
402 |     let xp = baseXP;
403 |
404 |     // High quality bonus
405 |     if (score >= 80) {
406 |         xp += 10;
407 |     }
408 |
409 |     // Relevance bonus
410 |     if (relevance !== null && relevance >= 70) {
411 |         xp += 5;
412 |     }
413 |
414 |     // Depth bonus
415 |     if (depth !== null && depth >= 70) {
416 |         xp += 5;
417 |     }
418 |
419 |     return Math.min(xp, cap);
420 | }
421 |
422 | // -----
423 | // Timer ARIA Announcement
424 | // -----
425 |
426 | const ANNOUNCEMENT_MINUTES = [1, 5, 10, 15, 20, 25, 30];
427 |
428 | /**
429 |  * Returns an announcement string at specific minute marks, null otherwise.
430 |  * Only announces at exact minute boundaries (no seconds remainder).
431 |  */
432 | export function getTimerAnnouncement(remainingMs: number): string | null {
433 |     if (remainingMs <= 0) return null;
434 |
435 |     const totalSeconds = Math.round(remainingMs / 1000);
436 |     const minutes = Math.floor(totalSeconds / 60);
437 |     const seconds = totalSeconds % 60;

```

```

438 |
439 | // Only announce at exact minute marks (no seconds)
440 | if (seconds !== 0) return null;
441 |
442 | if (!ANNOUNCEMENT_MINUTES.includes(minutes)) return null;
443 |
444 | if (minutes === 1) return "1 minute remaining";
445 | return `${minutes} minutes remaining`;
446 | }
447 |

```

**FILE: src/lib/portfolioAccess.ts**

TYPE: .ts | LINES: 1-127 | SHA256: f9bb55a43aa24282c993023fbb521b95e6b24607826e3b53f1500a6001ba434

```

1 | // Task 6.18: Public-portfolio route status mapping (pure business logic)
2 | // Requirements: 24.3
3 | //
4 | // Mirrors the SECURITY DEFINER `portfolio_public_access(p_student_id)` RPC
5 | // (see design.md !' "Public portfolio RLS") so the `PublicPortfolio` route can
6 | // translate the data-layer authorization discriminator into an HTTP-style
7 | // outcome WITHOUT leaking protected content.
8 | //
9 | // The decisive distinction this enforces (R24.3):
10 | //   - an existing-but-unauthorized portfolio yields 403 (forbidden), never 404
11 | //   - a non-existent portfolio yields 404 (not_found)
12 | //   - an authorized portfolio renders
13 | // so a guessable `portfolio/{userId}` URL cannot be used to tell "private"
14 | // portfolios apart by status code beyond the intended forbidden/not-found split.
15 |
16 | /**
17 |  * Authorization discriminator returned by the `portfolio_public_access` RPC.
18 |  *
19 |  * - `authorized` !' the profile exists, is public, AND sharing is permitted.
20 |  * - `forbidden` !' the profile exists but is not authorized for public view.
21 |  * - `not_found` !' no such profile exists.
22 |  */
23 | export type PortfolioAccessStatus = "authorized" | "forbidden" | "not_found";
24 |
25 | /**
26 |  * Outcome the public-portfolio route should take, derived purely from the
27 |  * authorization discriminator. Modeled as a discriminated union so consumers
28 |  * handle every case exhaustively and never accidentally render protected
29 |  * content for a non-`render` outcome.
30 |  *
31 |  * - `{ kind: "render" }` !' render the public portfolio.
32 |  * - `{ kind: "error"; status: 403 }` !' Forbidden (exists, not authorized).
33 |  * - `{ kind: "error"; status: 404 }` !' Not Found (no such portfolio).
34 |  */
35 | export type PortfolioRouteOutcome =
36 |   | { readonly kind: "render" }
37 |   | { readonly kind: "error"; readonly status: 403 | 404 };
38 |
39 | /**
40 |  * Inputs to the authorization rule, mirroring the columns the RPC inspects.
41 |  * This is a faithful model of the SQL discriminator's decision logic, kept in
42 |  * `src/lib` so the rule can be reasoned about and tested as pure logic.
43 |  */
44 | export interface PortfolioAccessInput {
45 |   /** Whether a `profiles` row exists for the requested student id. */
46 |   readonly profileExists: boolean;
47 |   /** `profiles.portfolio_public`. */
48 |   readonly portfolioPublic: boolean;
49 |   /** `profiles.portfolio_sharing_permitted` (school/admin granted). */
50 |   readonly sharingPermitted: boolean;
51 | }
52 |
53 | /**
54 |  * Pure model of the `portfolio_public_access` discriminator (R24.3).
55 |  *
56 |  * Equivalent to the SQL CASE expression:
57 |  * ```sql
58 |  * WHEN NOT EXISTS (profile) THEN 'not_found'
59 |  * WHEN portfolio_public = true AND portfolio_sharing_permitted = true THEN 'authorized'
60 |  * ELSE 'forbidden'
61 |  * ```
62 |  *
63 |  * The non-existence check is evaluated first so that a missing profile is
64 |  * always `not_found`; an existing profile is `authorized` only when BOTH the
65 |  * public flag AND the sharing permission are set, and `forbidden` in every
66 |  * other existing-profile case.
67 |  *
68 |  * This function is total: it returns a defined `PortfolioAccessStatus` for any
69 |  * combination of boolean inputs.
70 |  */
71 | export function evaluatePortfolioAccess(
72 |   input: PortfolioAccessInput
73 | ): PortfolioAccessStatus {
74 |   if (!input.profileExists) return "not_found";
75 |   if (input.portfolioPublic && input.sharingPermitted) return "authorized";
76 |   return "forbidden";
77 | }
78 |
79 | /**
80 |  * The route status mapping (R24.3): `authorized` !' `render`, `forbidden` !' `403`,
81 |  * `not_found` !' `404`.
82 |  *
83 |  * Total over `PortfolioAccessStatus`; the exhaustive `switch` is compile-time
84 |  * checked via the `never` assertion so adding a new status without updating the
85 |  * mapping is a type error rather than a silent fall-through to rendering.
86 |  */
87 | export function mapPortfolioAccessToRoute(
88 |   status: PortfolioAccessStatus
89 | ): PortfolioRouteOutcome {
90 |   switch (status) {

```

```

91 |     case "authorized":
92 |       return { kind: "render" };
93 |     case "forbidden":
94 |       return { kind: "error", status: 403 };
95 |     case "not_found":
96 |       return { kind: "error", status: 404 };
97 |     default: {
98 |       // Exhaustiveness guard: unreachable while `PortfolioAccessStatus` is
99 |       // fully handled above.
100 |       const _exhaustive: never = status;
101 |       return _exhaustive;
102 |     }
103 |   }
104 | }
105 |
106 | /**
107 |  * Safely narrow the raw `text` returned by the RPC into a known
108 |  * `PortfolioAccessStatus`.
109 |  *
110 |  * The RPC column type is `text`, so the value crossing the network boundary is
111 |  * an arbitrary string. Any unrecognized value is treated conservatively as
112 |  * `forbidden` – which maps to 403 and never renders – so a malformed or
113 |  * unexpected response can never expose protected content.
114 |  */
115 | export function parsePortfolioAccessStatus(
116 |   raw: string | null | undefined
117 | ): PortfolioAccessStatus {
118 |   switch (raw) {
119 |     case "authorized":
120 |     case "forbidden":
121 |     case "not_found":
122 |       return raw;
123 |     default:
124 |       return "forbidden";
125 |   }
126 | }
127 |

```

FILE: [src/lib/predictionValidator.ts](#)

TYPE: .ts | LINES: 1-121 | SHA256: 70aa15387e313ac7ee386a54a215dcb3761634914f0275907f5fd3d3d1dd9341

```

1 | // =====
2 | // predictionValidator – Validates at-risk predictions against actual grades
3 | // Validates: Requirements 49.2
4 | // =====
5 |
6 | import { supabase } from "@lib/supabase";
7 |
8 | /** Threshold below which a student is considered "at risk" (prediction was correct). */
9 | const AT_RISK_THRESHOLD = 70;
10 |
11 | export interface PredictionValidationResult {
12 |   updatedCount: number;
13 |   errors: string[];
14 | }
15 |
16 | /**
17 |  * After a grade is submitted, look up any unvalidated at-risk predictions
18 |  * for the given student + CLO IDs and mark them as 'correct' or 'incorrect'
19 |  * based on the student's current attainment.
20 |  *
21 |  * - correct: attainment < 70% (the at-risk prediction was right)
22 |  * - incorrect: attainment >= 70% (the student is doing fine)
23 |  *
24 |  * This is fire-and-forget – errors are collected but never thrown.
25 |  */
26 | export async function validateAtRiskPredictions(
27 |   studentId: string,
28 |   cloIds: string[]
29 | ): Promise<PredictionValidationResult> {
30 |   const result: PredictionValidationResult = { updatedCount: 0, errors: [] };
31 |
32 |   if (!studentId || cloIds.length === 0) return result;
33 |
34 |   try {
35 |     // 1. Fetch unvalidated at-risk predictions for this student
36 |     const { data: predictions, error: predError } = await supabase
37 |       .from("ai_feedback")
38 |       .select("id, suggestion_data")
39 |       .eq("student_id", studentId)
40 |       .eq("suggestion_type", "at_risk_prediction")
41 |       .is("validated_outcome", null);
42 |
43 |     if (predError) {
44 |       result.errors.push(`Failed to fetch predictions: ${predError.message}`);
45 |       return result;
46 |     }
47 |
48 |     if (!predictions || predictions.length === 0) return result;
49 |
50 |     // 2. Filter predictions whose at_risk_clo_id matches any of the graded CLOs
51 |     const relevantPredictions = predictions.filter((p) => {
52 |       const data = p.suggestion_data as Record<string, unknown> | null;
53 |       const predCloId = data?.at_risk_clo_id as string | undefined;
54 |       return predCloId && cloIds.includes(predCloId);
55 |     });
56 |
57 |     if (relevantPredictions.length === 0) return result;
58 |
59 |     // 3. Fetch current attainment for the student on the relevant CLOs
60 |     const relevantCloIds = [
61 |       ...new Set(
62 |         relevantPredictions.map(
63 |           (p) =>

```

```

64 |         (p.suggestion_data as Record<string, unknown>)
65 |         .at_risk_clo_id as string
66 |     ),
67 | ),
68 | ];
69 |
70 | const { data: attainments, error: attError } = await supabase
71 |     .from("outcome_attainment")
72 |     .select("outcome_id, attainment_percent")
73 |     .eq("student_id", studentId)
74 |     .in("outcome_id", relevantCloIds);
75 |
76 | if (attError) {
77 |     result.errors.push(`Failed to fetch attainment: ${attError.message}`);
78 |     return result;
79 | }
80 |
81 | const attainmentMap = new Map<string, number>(
82 |     (attainments ?? []).map((a) => [
83 |         a.outcome_id as string,
84 |         a.attainment_percent as number,
85 |     ])
86 | );
87 |
88 | // 4. Validate each prediction
89 | for (const prediction of relevantPredictions) {
90 |     const cloId = (prediction.suggestion_data as Record<string, unknown>)
91 |         .at_risk_clo_id as string;
92 |     const attainment = attainmentMap.get(cloId);
93 |
94 |     // If no attainment record exists yet, treat as at-risk (correct prediction)
95 |     const validatedOutcome =
96 |         attainment === undefined || attainment < AT_RISK_THRESHOLD
97 |         ? "correct"
98 |         : "incorrect";
99 |
100 |     const { error: updateError } = await supabase
101 |         .from("ai_feedback")
102 |         .update({ validated_outcome: validatedOutcome })
103 |         .eq("id", prediction.id);
104 |
105 |     if (updateError) {
106 |         result.errors.push(
107 |             `Failed to update prediction ${prediction.id}: ${updateError.message}`
108 |         );
109 |     } else {
110 |         result.updatedCount++;
111 |     }
112 | }
113 | } catch (err) {
114 |     result.errors.push(
115 |         `Unexpected error: ${err instanceof Error ? err.message : String(err)}`
116 |     );
117 | }
118 |
119 | return result;
120 | }
121 |

```

FILE: [src/lib/primaryCtaSelector.ts](#)

TYPE: .ts | LINES: 1-93 | SHA256: f930c687f77292d84b38fb7b3bf5dfeab89d345480f6961e7e4e3b5b6d4bd1fd

```

1 | /**
2 |  * Primary CTA selector – chooses the single dominant dashboard call-to-action
3 |  * from a set of candidate actions, and orders the remaining applicable
4 |  * candidates as subordinate secondary actions.
5 |  *
6 |  * Pure functions with no side effects; safe to call in render paths.
7 |  *
8 |  * Precedence convention: a LOWER `priority` value means HIGHER precedence
9 |  * (priority 0 outranks priority 1). Only candidates marked `applicable` are
10 |  * ever considered; non-applicable candidates are excluded from the comparison
11 |  * so that completing or invalidating the current primary promotes the next
12 |  * applicable candidate.
13 |  *
14 |  * Design: Area K – Dashboard single PrimaryCTA
15 |  * Requirements: 16.1, 16.2, 16.3
16 |  */
17 |
18 | export interface CtaCandidate {
19 |     /** Stable identifier, unique within a candidate set. */
20 |     id: string;
21 |     /** Precedence value; lower wins. */
22 |     priority: number;
23 |     /** Whether the action currently applies to the student. */
24 |     applicable: boolean;
25 | }
26 |
27 | /**
28 |  * Select the single highest-precedence applicable candidate.
29 |  *
30 |  * Returns `null` when no candidate is applicable (R16.1: at most one dominant
31 |  * CTA, none when nothing applies). Non-applicable candidates are excluded from
32 |  * the comparison (R16.2). Ties on `priority` resolve to the first such
33 |  * candidate in input order, keeping selection deterministic.
34 |  *
35 |  * @example
36 |  * selectPrimary([
37 |  *   { id: "profile", priority: 0, applicable: false },
38 |  *   { id: "submit", priority: 1, applicable: true },
39 |  *   { id: "continue", priority: 2, applicable: true },
40 |  * ]);
41 |  * // !' { id: "submit", priority: 1, applicable: true }
42 |  */

```

```

43 | export function selectPrimary<T extends CtaCandidate>(
44 |   candidates: readonly T[]
45 | ): T | null {
46 |   let best: T | null = null;
47 |   for (const candidate of candidates) {
48 |     if (!candidate.applicable) continue;
49 |     if (best === null || candidate.priority < best.priority) {
50 |       best = candidate;
51 |     }
52 |   }
53 |   return best;
54 | }
55 |
56 | /**
57 |  * Order the subordinate (secondary) applicable candidates by ascending
58 |  * precedence, excluding the primary candidate.
59 |  *
60 |  * When `primaryId` is omitted, the primary is computed from the candidate set
61 |  * via {@link selectPrimary}, so callers can request "everything except the
62 |  * dominant action" in one call. Non-applicable candidates are never included.
63 |  * The sort is stable: ties on `priority` preserve input order (R16.3 – the
64 |  * next-highest-precedence applicable candidate is promoted predictably).
65 |  *
66 |  * @example
67 |  * orderSecondary([
68 |  *   { id: "submit", priority: 1, applicable: true },
69 |  *   { id: "continue", priority: 2, applicable: true },
70 |  *   { id: "feedback", priority: 3, applicable: true },
71 |  * ], "submit");
72 |  * // !' [{ id: "continue", ... }, { id: "feedback", ... }]
73 |  */
74 | export function orderSecondary<T extends CtaCandidate>(
75 |   candidates: readonly T[],
76 |   primaryId?: string | null
77 | ): T[] {
78 |   const excludeId =
79 |     primaryId !== undefined && primaryId !== null
80 |       ? primaryId
81 |       : selectPrimary(candidates)?.id ?? null;
82 |
83 |   return candidates
84 |     .filter((candidate) => candidate.applicable && candidate.id !== excludeId)
85 |     .map((candidate, index) => ({ candidate, index }))
86 |     .sort((a, b) =>
87 |       a.candidate.priority === b.candidate.priority
88 |         ? a.index - b.index
89 |         : a.candidate.priority - b.candidate.priority
90 |     )
91 |     .map(({ candidate }) => candidate);
92 | }
93 |

```

## FILE: src/lib/profileCache.ts

TYPE: .ts | LINES: 1-129 | SHA256: 10c48bccbedff72ee337effe0cfb1503bcdacf1bb6fd28a8b7d67fd75f066d12

```

1 | import type { Profile, UserRole } from "@types/app";
2 |
3 | // =====
4 | // Shell-first profile cache (Option J, Phase 0)
5 | // =====
6 | //
7 | // Purpose: let the app shell + route render instantly on reload by hydrating
8 | // the user's profile from localStorage, instead of blocking first paint on a
9 | // `profiles` SELECT (the verified "blank spinner on open" root cause).
10 | //
11 | // SECURITY MODEL – read this before changing anything:
12 | // - This cache is NON-AUTHORITATIVE. It only pre-fills UI state (name, role,
13 | //   institution) so the router can resolve a dashboard immediately. Row Level
14 | //   Security remains the sole source of truth server-side; a stale or tampered
15 | //   cache can at most render the wrong shell for one paint, never grant data
16 | //   access, because every query still authorizes against the live JWT + RLS.
17 | // - It is IDENTITY-GUARDED: a cached profile is only ever returned for the
18 | //   exact user id that owns it (cross-user no-leak). A single storage key is
19 | //   used and overwritten per user, and it is cleared on sign-out.
20 | // - It stores the profile row only – never auth tokens or secrets.
21 | // =====
22 |
23 | const STORAGE_KEY = "edevise.auth.profile.v1";
24 |
25 | /**
26 |  * A cached profile is treated as "fresh" within this window. A reload inside it
27 |  * skips the background revalidation entirely, which is what collapses the
28 |  * just-signed-in double profile fetch (signIn fetches + seeds the cache, then
29 |  * the SIGNED_IN event hydrates from that fresh cache instead of refetching).
30 |  */
31 | export const PROFILE_CACHE_FRESH_MS = 30_000;
32 |
33 | interface CachedProfileEnvelope {
34 |   userId: string;
35 |   profile: Profile;
36 |   cachedAt: number;
37 | }
38 |
39 | const VALID_ROLES: readonly UserRole[] = [
40 |   "admin",
41 |   "coordinator",
42 |   "teacher",
43 |   "student",
44 |   "parent",
45 | ];
46 |
47 | const isBrowser = (): boolean =>
48 |   typeof window !== "undefined" && typeof window.localStorage !== "undefined";
49 |

```

```

50 | /**
51 |  * Validates the stored envelope AND enforces the identity guard: the cached
52 |  * profile must belong to `expectedUserId`. This is the cross-user no-leak
53 |  * invariant – user B can never hydrate user A's cached profile.
54 |  */
55 | const isValidEnvelope = (
56 |   value: unknown,
57 |   expectedUserId: string
58 | ): value is CachedProfileEnvelope => {
59 |   if (typeof value !== "object" || value === null) return false;
60 |   const env = value as Record<string, unknown>;
61 |   if (typeof env.userId !== "string" || typeof env.cachedAt !== "number") {
62 |     return false;
63 |   }
64 |   if (env.userId !== expectedUserId) return false;
65 |
66 |   const profile = env.profile as Partial<Profile> | undefined;
67 |   if (!profile || typeof profile !== "object") return false;
68 |   if (profile.id !== expectedUserId) return false;
69 |   if (!VALID_ROLES.includes(profile.role as UserRole)) return false;
70 |
71 |   return true;
72 | };
73 |
74 | /**
75 |  * Returns the cached profile for `userId` (with its cache timestamp), or null on
76 |  * miss / mismatch / parse error. A foreign or malformed entry is proactively
77 |  * removed so it can never linger on the device.
78 |  */
79 | export const readCachedProfile = (
80 |   userId: string
81 | ): { profile: Profile; cachedAt: number } | null => {
82 |   if (!isBrowser() || !userId) return null;
83 |   try {
84 |     const raw = window.localStorage.getItem(STORAGE_KEY);
85 |     if (!raw) return null;
86 |
87 |     const parsed: unknown = JSON.parse(raw);
88 |     if (!isValidEnvelope(parsed, userId)) {
89 |       window.localStorage.removeItem(STORAGE_KEY);
90 |       return null;
91 |     }
92 |     return { profile: parsed.profile, cachedAt: parsed.cachedAt };
93 |   } catch {
94 |     return null;
95 |   }
96 | };
97 |
98 | /**
99 |  * Persists `profile` for `userId`. No-ops if the profile's id does not match the
100 |  * user id (defensive: never cache a profile under the wrong identity).
101 |  */
102 | export const writeCachedProfile = (userId: string, profile: Profile): void => {
103 |   if (!isBrowser() || !userId || profile.id !== userId) return;
104 |   try {
105 |     const envelope: CachedProfileEnvelope = {
106 |       userId,
107 |       profile,
108 |       cachedAt: Date.now(),
109 |     };
110 |     window.localStorage.setItem(STORAGE_KEY, JSON.stringify(envelope));
111 |   } catch {
112 |     // Non-fatal: a full or blocked localStorage must never break auth.
113 |   }
114 | };
115 |
116 | /** Removes the cached profile. Called on sign-out and on any unauthenticated load. */
117 | export const clearCachedProfile = (): void => {
118 |   if (!isBrowser()) return;
119 |   try {
120 |     window.localStorage.removeItem(STORAGE_KEY);
121 |   } catch {
122 |     // ignore
123 |   }
124 | };
125 |
126 | /** True if a cache entry captured at `cachedAt` is still within the fresh window. */
127 | export const isProfileFresh = (cachedAt: number): boolean =>
128 |   Date.now() - cachedAt < PROFILE_CACHE_FRESH_MS;
129 |

```

## FILE: src/lib/profileCompleteness.ts

TYPE: .ts | LINES: 1-87 | SHA256: b7d0f53b677bc70a0b1ce96a1b11dae02212fadeea369ac41bfeb9fc0b0b3523

```

1 | import {
2 |   calculateProfileCompleteness,
3 |   type ProfileCompletenessInput,
4 | } from "../scoreCalculator";
5 |
6 | export interface ProfileDimensionStatus {
7 |   dimension: string;
8 |   completed: number;
9 |   total: number;
10 |   percentage: number;
11 | }
12 |
13 | export function getProfileDimensions(
14 |   input: ProfileCompletenessInput
15 | ): ProfileDimensionStatus[] {
16 |   return [
17 |     {
18 |       dimension: "personality",
19 |       completed: Math.min(input.personality_items, 25),
20 |       total: 25,

```

```

21 |     percentage: Math.round(Math.min(input.personality_items / 25, 1) * 100),
22 |   },
23 |   {
24 |     dimension: "self_efficacy",
25 |     completed: Math.min(input.self_efficacy_items, 6),
26 |     total: 6,
27 |     percentage: Math.round(Math.min(input.self_efficacy_items / 6, 1) * 100),
28 |   },
29 |   {
30 |     dimension: "study_strategy",
31 |     completed: Math.min(input.study_strategy_items, 8),
32 |     total: 8,
33 |     percentage: Math.round(Math.min(input.study_strategy_items / 8, 1) * 100),
34 |   },
35 |   {
36 |     dimension: "learning_style",
37 |     completed: Math.min(input.learning_style_items, 16),
38 |     total: 16,
39 |     percentage: Math.round(
40 |       Math.min(input.learning_style_items / 16, 1) * 100
41 |     ),
42 |   },
43 |   {
44 |     dimension: "baseline",
45 |     completed: input.baseline_courses > 0 ? 1 : 0,
46 |     total: 1,
47 |     percentage: input.baseline_courses > 0 ? 100 : 0,
48 |   },
49 | ];
50 | }
51 |
52 | export function getRemainingDimensions(
53 |   input: ProfileCompletenessInput
54 | ): ProfileDimensionStatus[] {
55 |   return getProfileDimensions(input).filter((d) => d.percentage < 100);
56 | }
57 |
58 | export function getEstimatedTimeMinutes(
59 |   remaining: ProfileDimensionStatus[]
60 | ): number {
61 |   let minutes = 0;
62 |   for (const d of remaining) {
63 |     const itemsLeft = d.total - d.completed;
64 |     switch (d.dimension) {
65 |       case "personality":
66 |         minutes += Math.ceil(itemsLeft * 0.3); // ~18 sec per question
67 |         break;
68 |       case "self_efficacy":
69 |         minutes += Math.ceil(itemsLeft * 0.3);
70 |         break;
71 |       case "study_strategy":
72 |         minutes += Math.ceil(itemsLeft * 0.3);
73 |         break;
74 |       case "learning_style":
75 |         minutes += Math.ceil(itemsLeft * 0.4); // ~24 sec per question
76 |         break;
77 |       case "baseline":
78 |         minutes += 15; // default baseline time
79 |         break;
80 |     }
81 |   }
82 |   return minutes;
83 | }
84 |
85 | export { calculateProfileCompleteness };
86 | export type { ProfileCompletenessInput };
87 |

```

## FILE: src/lib/purchaseValidator.ts

TYPE: .ts | LINES: 1-91 | SHA256: 753726d115d6f3817893fdbc7c337069b1813ffb54b4a3cd5db7f91591490995

```

1 | // =====
2 | // Purchase Validator – Pure function: validate purchase eligibility
3 | // =====
4 |
5 | import type {
6 |   MarketplaceItemSubCategory,
7 |   StockType,
8 | } from "@lib/marketplaceSchemas";
9 |
10 | export interface PurchaseValidationInput {
11 |   balance: number;
12 |   studentLevel: number;
13 |   item: {
14 |     xp_price: number;
15 |     level_requirement: number;
16 |     stock_type: StockType;
17 |     stock_quantity: number | null;
18 |     is_active: boolean;
19 |     sub_category: MarketplaceItemSubCategory;
20 |   };
21 |   alreadyOwned: boolean;
22 |   currentStreakFreezes: number;
23 |   effectivePrice: number;
24 | }
25 |
26 | export type PurchaseErrorCode =
27 |   | "ITEM_INACTIVE"
28 |   | "LEVEL_REQUIREMENT"
29 |   | "OUT_OF_STOCK"
30 |   | "ALREADY_OWNED"
31 |   | "MAX_INVENTORY"
32 |   | "INSUFFICIENT_BALANCE";
33 |

```

```

34 | export interface PurchaseValidationResult {
35 |   valid: boolean;
36 |   error_code?: PurchaseErrorCode;
37 | }
38 |
39 | /**
40 |  * Validate whether a student is eligible to purchase a marketplace item.
41 |  *
42 |  * Checks are evaluated in priority order:
43 |  * 1. ITEM_INACTIVE – item must be active
44 |  * 2. LEVEL_REQUIREMENT – student level must meet the item's gate
45 |  * 3. OUT_OF_STOCK – limited items must have remaining stock
46 |  * 4. ALREADY_OWNED – one-per-student items cannot be re-purchased
47 |  * 5. MAX_INVENTORY – streak shields capped at 3
48 |  * 6. INSUFFICIENT_BALANCE – student must have enough XP
49 |  */
50 | export function validatePurchase(
51 |   input: PurchaseValidationInput
52 | ): PurchaseValidationResult {
53 |   const {
54 |     balance,
55 |     studentLevel,
56 |     item,
57 |     alreadyOwned,
58 |     currentStreakFreezes,
59 |     effectivePrice,
60 |   } = input;
61 |
62 |   if (!item.is_active) {
63 |     return { valid: false, error_code: "ITEM_INACTIVE" };
64 |   }
65 |
66 |   if (studentLevel < item.level_requirement) {
67 |     return { valid: false, error_code: "LEVEL_REQUIREMENT" };
68 |   }
69 |
70 |   if (
71 |     item.stock_type === "limited" &&
72 |     (item.stock_quantity === null || item.stock_quantity <= 0)
73 |   ) {
74 |     return { valid: false, error_code: "OUT_OF_STOCK" };
75 |   }
76 |
77 |   if (item.stock_type === "one_per_student" && alreadyOwned) {
78 |     return { valid: false, error_code: "ALREADY_OWNED" };
79 |   }
80 |
81 |   if (item.sub_category === "streak_shield" && currentStreakFreezes >= 3) {
82 |     return { valid: false, error_code: "MAX_INVENTORY" };
83 |   }
84 |
85 |   if (balance < effectivePrice) {
86 |     return { valid: false, error_code: "INSUFFICIENT_BALANCE" };
87 |   }
88 |
89 |   return { valid: true };
90 | }
91 |

```

FILE: [src/lib/queryClient.ts](#)

TYPE: .ts | LINES: 1-155 | SHA256: a8d657aa5eba3ec9d7c934881bcc131863d2bb7921dcfdab365d10b480059ed1

```

1 | import { QueryClient, QueryCache, MutationCache } from "@tanstack/react-query";
2 | import * as Sentry from "@sentry/react";
3 | import { toast } from "sonner";
4 |
5 | /**
6 |  * Global query/mutation error safety net (spec: production-bug-fixes, Req 6).
7 |  *
8 |  * Without a cache-level handler, surfacing an error relies on every consumer
9 |  * reading `isError`/`error`; a forgotten branch becomes a silent no-op
10 |  * (violates engineering-guardrails "never swallow errors silently"). These
11 |  * handlers are a defense-in-depth FALLBACK – per-hook handling stays primary.
12 |  *
13 |  * Behaviour:
14 |  * - ALWAYS log (console + Sentry). Sentry is a no-op until consent-gated init,
15 |  *   so this is safe to call unconditionally.
16 |  * - Toast is DEDUP-AWARE: a query/mutation opts out with
17 |  *   `meta: { suppressGlobalError: true }` so hooks that already toast their own
18 |  *   message never double-toast.
19 |  * - Queries get a fallback toast (components commonly render their own UI but
20 |  *   forget to toast). Mutations overwhelmingly toast their own specific
21 |  *   message, so the mutation net only logs + surfaces the shared 429 notice –
22 |  *   this avoids the app-wide double-toast the requirement warns against.
23 |  */
24 |
25 | const RATE_LIMIT_MESSAGE = "Too many requests. Please wait a moment.";
26 |
27 | const is429 = (error: unknown): boolean =>
28 |   !!error &&
29 |   typeof error === "object" &&
30 |   "status" in error &&
31 |   (error as { status: number }).status === 429;
32 |
33 | /**
34 |  * Best-effort extraction of an HTTP status code from an unknown error.
35 |  *
36 |  * Different Supabase error shapes carry status in different places:
37 |  * - `error.status` – Storage / some REST errors, fetch Response-likes
38 |  * - `error.context.status` – Edge Function errors (FunctionsHttpError wraps
39 |  *   the upstream `Response` in `context`)
40 |  * PostgREST errors carry a Postgres `code` (not an HTTP status), so this returns
41 |  * `undefined` for them and the caller falls back to count-based retry.
42 |  */

```



```

43 | export const getErrorStatus = (error: unknown): number | undefined => {
44 |   if (!error || typeof error !== "object") return undefined;
45 |   const e = error as { status?: unknown; context?: { status?: unknown } };
46 |   if (typeof e.status === "number") return e.status;
47 |   if (e.context && typeof e.context.status === "number")
48 |     return e.context.status;
49 |   return undefined;
50 | };
51 |
52 | /**
53 |  * Query retry policy (spec: dashboard-and-ux-performance – reduce request
54 |  * fan-out). TanStack Query's default retries 3x on every failure. That is
55 |  * wasteful for DETERMINISTIC client errors (4xx): a 404/403/400/422 will simply
56 |  * reproduce the same failure on every retry, multiplying the request fan-out
57 |  * (e.g. a single broken call becoming 4 network requests). We therefore:
58 |  * - never retry 429 (respect the rate limit – surfaced as a toast instead),
59 |  * - never retry other 4xx EXCEPT 408 (Request Timeout, which is transient),
60 |  * - retry everything else (network errors, 5xx, and unknown/PostgreSQL errors
61 |  *   with no detectable status) up to 3 times.
62 |  */
63 | export const shouldRetryQuery = (
64 |   failureCount: number,
65 |   error: unknown
66 | ): boolean => {
67 |   if (is429(error)) return false;
68 |   const status = getErrorStatus(error);
69 |   if (status !== undefined && status >= 400 && status < 500 && status !== 408) {
70 |     return false;
71 |   }
72 |   return failureCount < 3;
73 | };
74 |
75 | /** Best-effort human-readable message from an unknown error. */
76 | export const getErrorMessage = (error: unknown): string => {
77 |   if (
78 |     error &&
79 |     typeof error === "object" &&
80 |     "message" in error &&
81 |     typeof (error as { message: unknown }).message === "string" &&
82 |     (error as { message: string }).message.length > 0
83 |   ) {
84 |     return (error as { message: string }).message;
85 |   }
86 |   return "Something went wrong. Please try again.";
87 | };
88 |
89 | const optedOut = (meta: Record<string, unknown> | undefined): boolean =>
90 |   meta?.suppressGlobalError === true;
91 |
92 | /** queryCache.onError – log always; fallback toast unless opted out. */
93 | export const handleGlobalQueryError = (
94 |   error: unknown,
95 |   meta?: Record<string, unknown>
96 | ): void => {
97 |   console.error("[query error]", error);
98 |   Sentry.captureException(error, { tags: { source: "queryCache" } });
99 |   if (optedOut(meta)) return;
100 |   toast.error(is429(error) ? RATE_LIMIT_MESSAGE : getErrorMessage(error));
101 | };
102 |
103 | /**
104 |  * mutationCache.onError – log always; only surface the shared 429 notice
105 |  * (mutations toast their own specific message). Opt-out still suppresses it.
106 |  */
107 | export const handleGlobalMutationError = (
108 |   error: unknown,
109 |   meta?: Record<string, unknown>
110 | ): void => {
111 |   console.error("[mutation error]", error);
112 |   Sentry.captureException(error, { tags: { source: "mutationCache" } });
113 |   if (optedOut(meta)) return;
114 |   if (is429(error)) toast.error(RATE_LIMIT_MESSAGE);
115 | };
116 |
117 | /**
118 |  * Retry backoff with EQUAL JITTER (spec: dashboard-and-ux-performance; AWS
119 |  * "exponential backoff and jitter"). The base doubles per attempt (capped at
120 |  * RETRY_MAX_DELAY_MS), then we wait half the base plus a random portion of the
121 |  * other half. The jitter de-synchronizes retries: without it, many queries that
122 |  * fail together under DB contention retry in lockstep and re-create the same
123 |  * thundering herd – a big contributor to the multi-second (~60s) stalls under
124 |  * load. `random` is injectable for deterministic tests.
125 |  */
126 | export const RETRY_MAX_DELAY_MS = 10_000;
127 |
128 | export const computeRetryDelay = (
129 |   attempt: number,
130 |   random: () => number = Math.random
131 | ): number => {
132 |   const base = Math.min(1000 * 2 ** attempt, RETRY_MAX_DELAY_MS);
133 |   return Math.round(base / 2 + random() * (base / 2));
134 | };
135 |
136 | export const queryClient = new QueryClient({
137 |   queryCache: new QueryCache({
138 |     onError: (error, query) => handleGlobalQueryError(error, query.meta),
139 |   }),
140 |   mutationCache: new MutationCache({
141 |     onError: (error, _vars, _ctx, mutation) =>
142 |       handleGlobalMutationError(error, mutation.options.meta),
143 |   }),
144 |   defaultOptions: {
145 |     queries: {
146 |       staleTime: 5 * 60 * 1000, // 5 minutes

```



[illegible]

[illegible]

```

275 | journal,
276 | habitLogs,
277 | studentHabitLevel,
278 | bonusXPEvents,
279 | streakFreezes,
280 | // Notifications
281 | notifications,
282 | emailPreferences,
283 | // AI Co-Pilot
284 | aiSuggestions,
285 | atRiskPredictions,
286 | feedbackDrafts,
287 | aiFeedback,
288 | aiGovernance,
289 | // Institutional Management
290 | semesters,
291 | departments,
292 | courseSections,
293 | sectionAttainment: createKeys("sectionAttainment"),
294 | surveys,
295 | surveyQuestions,
296 | surveyResponses,
297 | cqiPlans,
298 | institutionSettings,
299 | programAccreditations,
300 | announcements,
301 | announcementAttachments,
302 | announcementReads,
303 | courseModules,
304 | courseMaterials,
305 | discussionThreads,
306 | discussionReplies,
307 | classSessions,
308 | attendanceRecords,
309 | quizzes,
310 | quizQuestions,
311 | quizAttempts,
312 | gradeCategories,
313 | gradebook,
314 | timetableSlots,
315 | academicCalendarEvents,
316 | parentStudentLinks,
317 | feeStructures,
318 | feePayments,
319 | studentFees,
320 | transcripts,
321 | courseFiles,
322 | // Dashboards
323 | adminDashboard,
324 | coordinatorDashboard,
325 | studentDashboard,
326 | parentDashboard,
327 | teacherDashboard,
328 | // Reports
329 | accreditationReports,
330 | // Production
331 | calendarEvents,
332 | globalSearch,
333 | notificationPreferences,
334 | sessions,
335 | auditLogs,
336 | // Marketplace
337 | marketplace,
338 | // Heatmap & Wellness
339 | heatmap,
340 | wellness,
341 | habitAnalytics,
342 | // Adaptive Quiz
343 | questionBank,
344 | questionAnalytics,
345 | quizGeneration,
346 | reviewQueue,
347 | quizCLOCorrelation,
348 | // Practice Mode
349 | practiceMode,
350 | // Explanation Confidence
351 | explanationConfidence,
352 | verifiedExplanations,
353 | explanationReviewQueue,
354 | // Mastery Recovery
355 | masteryRecovery,
356 | // Bloom's Progression
357 | bloomsProgression,
358 | // Onboarding
359 | onboarding,
360 | // OBE Visualizations
361 | graduateAttributes: createKeys("graduateAttributes"),
362 | graduateAttributeMappings: createKeys("graduateAttributeMappings"),
363 | graduateAttributeAttainment: createKeys("graduateAttributeAttainment"),
364 | competencyFrameworks: createKeys("competencyFrameworks"),
365 | competencyItems: createKeys("competencyItems"),
366 | competencyOutcomeMappings: createKeys("competencyOutcomeMappings"),
367 | sankeyData: createKeys("sankeyData"),
368 | gapAnalysisData: createKeys("gapAnalysisData"),
369 | coverageHeatmapData: createKeys("coverageHeatmapData"),
370 | adminPLOHeatmap: createKeys("adminPLOHeatmap"),
371 | semesterTrends: createKeys("semesterTrends"),
372 | cohortComparison: createKeys("cohortComparison"),
373 | historicalEvidence: createKeys("historicalEvidence"),
374 | outcomeChain: createKeys("outcomeChain"),
375 | // Teams
376 | teams: createKeys("teams"),
377 | teamMembers: createKeys("teamMembers"),
378 | teamGamification: createKeys("teamGamification"),

```

```

379 teamLeaderboard: createKeys("teamLeaderboard"),
380 teamBadges: createKeys("teamBadges"),
381 teamInvitations: createKeys("teamInvitations"),
382 // Friends (student social graph)
383 friends: createKeys("friends"),
384 // Challenges
385 challenges: createKeys("challenges"),
386 challengeProgress: createKeys("challengeProgress"),
387 studentChallenges: createKeys("studentChallenges"),
388 socialChallenges: createKeys("socialChallenges"),
389 challengeLeaderboard: createKeys("challengeLeaderboard"),
390 // Contribution Accountability
391 contributionStatus: createKeys("contributionStatus"),
392 replacementVotes: createKeys("replacementVotes"),
393 // Peer Teaching
394 peerTeachingMoments: createKeys("peerTeachingMoments"),
395 teachingMomentRatings: createKeys("teachingMomentRatings"),
396 teachingImpact: createKeys("teachingImpact"),
397 // Team Health
398 teamHealth: createKeys("teamHealth"),
399 teamHealthReport: createKeys("teamHealthReport"),
400 // Adaptive XP
401 xpMultiplier: createKeys("xpMultiplier"),
402 diminishingReturns: createKeys("diminishingReturns"),
403 improvementBonusHistory: createKeys("improvementBonusHistory"),
404 assignmentDifficultyBonus: createKeys("assignmentDifficultyBonus"),
405 // Comeback Challenge
406 comebackChallenge: createKeys("comebackChallenge"),
407 // Leaderboard Enhancements
408 personalBest: createKeys("personalBest"),
409 mostImproved: createKeys("mostImproved"),
410 leagueTier: createKeys("leagueTier"),
411 percentileBand: createKeys("percentileBand"),
412 leagueLeaderboard: createKeys("leagueLeaderboard"),
413 leaderboardCosmetics: createKeys("leaderboardCosmetics"),
414 // Badge Tiers
415 tieredBadges: createKeys("tieredBadges"),
416 badgeDefinitions: createKeys("badgeDefinitions"),
417 badgeSpotlight: createKeys("badgeSpotlight"),
418 badgeSpotlightSchedule: createKeys("badgeSpotlightSchedule"),
419 // Gap Analysis – XP Economy & Creative Expression
420 economist: createKeys("economist"),
421 quests: createKeys("quests"),
422 studentContent: createKeys("studentContent"),
423 donations: createKeys("donations"),
424 dynamicPricing: createKeys("dynamicPricing"),
425 leagues: createKeys("leagues"),
426 // i18n
427 i18n,
428 // Accessibility
429 accessibility,
430 // Weekly Planner
431 studySessions: createKeys("studySessions"),
432 plannerTasks: createKeys("plannerTasks"),
433 weeklyGoals: createKeys("weeklyGoals"),
434 // Session Intent / Flow / Reviews / Reflection Quality
435 sessionIntents: createKeys("sessionIntents"),
436 suggestedIntents: createKeys("suggestedIntents"),
437 flowCheckIns: createKeys("flowCheckIns"),
438 reviewSchedules: createKeys("reviewSchedules"),
439 reflectionQuality: createKeys("reflectionQuality"),
440 reflectionDigests: createKeys("reflectionDigests"),
441 // AI Tutor
442 tutorConversations: {
443   ...createKeys("tutorConversations"),
444   byCourse: (courseId: string) =>
445     ["tutorConversations", "byCourse", courseId] as const,
446 },
447 tutorMessages: {
448   ...createKeys("tutorMessages"),
449   byConversation: (conversationId: string) =>
450     ["tutorMessages", "byConversation", conversationId] as const,
451 },
452 tutorUsage: {
453   all: ["tutorUsage"] as const,
454   daily: (studentId: string, date: string) =>
455     ["tutorUsage", "daily", studentId, date] as const,
456 },
457 tutorAnalytics: {
458   ...createKeys("tutorAnalytics"),
459   byCourse: (courseId: string) =>
460     ["tutorAnalytics", "byCourse", courseId] as const,
461   byCourseAndRange: (courseId: string, startDate: string, endDate: string) =>
462     ["tutorAnalytics", "byCourse", courseId, startDate, endDate] as const,
463 },
464 } as const;
465

```

FILE: src/lib/questionAnalytics.ts

TYPE: .ts | LINES: 1-60 | SHA256: 20cea83a9511963f69a33f4326bc2bbbed7b06d65432b10cc47167ef1e1068b68

[illegible]

```

14 |   for (const a of answers) {
15 |     totals[a.clo_id] = (totals[a.clo_id] ?? 0) + 1;
16 |     if (a.is_correct) {
17 |       corrects[a.clo_id] = (corrects[a.clo_id] ?? 0) + 1;
18 |     }
19 |   }
20 |
21 |   const result: Record<string, number> = {};
22 |   for (const cloId of Object.keys(totals)) {
23 |     result[cloId] = ((corrects[cloId] ?? 0) / totals[cloId]!) * 100;
24 |   }
25 |   return result;
26 | }
27 |
28 | /**
29 |  * Detects a discrepancy between quiz score and CLO attainment.
30 |  * Returns true if the absolute difference exceeds 15 percentage points.
31 |  */
32 | export function detectCLODiscrepancy(
33 |   quizScorePercent: number,
34 |   cloAttainmentPercent: number
35 | ): boolean {
36 |   return Math.abs(quizScorePercent - cloAttainmentPercent) > 15;
37 | }
38 |
39 | /**
40 |  * Computes approval rate for a generation batch.
41 |  * Returns 0 when total is 0 to avoid division by zero.
42 |  */
43 | export function computeApprovalRate(approved: number, total: number): number {
44 |   if (total === 0) return 0;
45 |   return approved / total;
46 | }
47 |
48 | /**
49 |  * Computes bonus XP for correctly answered hard questions (difficulty "e 4.0").
50 |  * Awards 10 XP per qualifying question, capped at 50.
51 |  */
52 | export function computeBonusXP(
53 |   correctQuestions: { difficulty_rating: number }[]
54 | ): number {
55 |   const hardCount = correctQuestions.filter(
56 |     (q) => q.difficulty_rating >= 4.0
57 |   ).length;
58 |   return Math.min(hardCount * 10, 50);
59 | }
60 |

```

## FILE: src/lib/quizCorrectness.ts

TYPE: .ts | LINES: 1-62 | SHA256: 586bc0a40268bcc830e285d7c35291ff231295fb611f45d5642063a526f2dc37

```

1 | // Task 6.11: Adaptive quiz practice correctness derivation (pure business logic)
2 | // Requirements: 2.1, 2.4, 2.6
3 |
4 | /**
5 |  * Inputs for deriving the correctness of a single submitted practice answer.
6 |  *
7 |  * Correctness is determined from real evidence – never a hardcoded `true`
8 |  * (the root cause of R2). The server's authoritative evaluation is preferred;
9 |  * when absent, correctness is derived by comparing the submitted answer to the
10 |  * question's known correct answer.
11 |  */
12 | export interface CorrectnessInput {
13 |   /** The answer the student submitted. */
14 |   selectedAnswer: string;
15 |   /** The question's correct answer, when the question exposes it. */
16 |   correctAnswer?: string | null;
17 |   /** Server-evaluated correctness from the adaptive-selection response. */
18 |   serverEvaluated?: boolean | null;
19 | }
20 |
21 | /**
22 |  * Derives whether a submitted practice answer is correct.
23 |  *
24 |  * Resolution order (R2.1, R2.4, R2.6):
25 |  * 1. WHEN the backend provides an authoritative evaluation
26 |  *    ( `serverEvaluated` is a boolean), use it directly.
27 |  * 2. ELSE WHEN the question exposes its `correctAnswer`, derive correctness by
28 |  *    equality against the submitted answer (trimmed, case-insensitive).
29 |  * 3. ELSE return `false` – the system never silently reports "correct" without
30 |  *    evidence (this is the bug R2 fixes; the previous code hardcoded `true`).
31 |  *
32 |  * The single value returned here drives the feedback UI, the recorded
33 |  * `previous_answer_correct`, and the correct-count increment, guaranteeing the
34 |  * displayed feedback matches the recorded value (R2.4, R2.6).
35 |  *
36 |  * This function is pure and total: it returns a defined boolean for every input.
37 |  *
38 |  * @param input - The submitted answer plus any available evaluation evidence.
39 |  * @returns `true` only when there is evidence the answer is correct.
40 |  */
41 | export function deriveCorrectness(input: CorrectnessInput): boolean {
42 |   const { selectedAnswer, correctAnswer, serverEvaluated } = input;
43 |
44 |   // 1. Server-authoritative evaluation wins when present.
45 |   if (typeof serverEvaluated === "boolean") {
46 |     return serverEvaluated;
47 |   }
48 |
49 |   // 2. Fall back to equality against the known correct answer.
50 |   if (correctAnswer !== null) {
51 |     return normalize(selectedAnswer) === normalize(correctAnswer);
52 |   }
53 |

```

```

54 | // 3. No evidence !ð never report "correct".
55 | return false;
56 | }
57 |
58 | /** Normalizes an answer for tolerant comparison (trim + case-fold). */
59 | function normalize(value: string): string {
60 |   return value.trim().toLowerCase();
61 | }
62 |

```

FILE: [src/lib/quizEvidence.ts](#)

TYPE: .ts | LINES: 1-113 | SHA256: 93c8e038fe1e8dd49b82953d06c94f813995717d763ed5f1efc8a378bccdd2aa

```

1 | // =====
2 | // Quiz Evidence Generator – Wires quiz scores into the CLO attainment pipeline
3 | // =====
4 |
5 | import { supabase } from "@lib/supabase";
6 |
7 | type AttainmentLevel = "excellent" | "satisfactory" | "developing" | "not_yet";
8 |
9 | interface QuizEvidenceInput {
10 |   attemptId: string;
11 |   quizId: string;
12 |   studentId: string;
13 |   scorePercent: number;
14 |   cloIds: string[];
15 | }
16 |
17 | /**
18 |  * Generate evidence records from a quiz attempt score, feeding into the same
19 |  * CLO attainment pipeline used by assignment grades.
20 |  *
21 |  * Creates one evidence record per linked CLO, then cascades the attainment rollup.
22 |  */
23 | export async function generateQuizEvidence(
24 |   input: QuizEvidenceInput
25 | ): Promise<void> {
26 |   const { attemptId, quizId, studentId, scorePercent, cloIds } = input;
27 |
28 |   if (cloIds.length === 0) return;
29 |
30 |   const attainmentLevel = classifyAttainment(scorePercent);
31 |
32 |   // Insert evidence records (one per CLO)
33 |   // Cast through unknown to handle nullable FK columns for quiz-sourced evidence
34 |   const evidenceRows = cloIds.map((cloId) => ({
35 |     student_id: studentId,
36 |     clo_id: cloId,
37 |     score_percent: scorePercent,
38 |     attainment_level: attainmentLevel,
39 |   }));
40 |
41 |   const { error: evidenceErr } = await (
42 |     supabase.from("evidence") as unknown as {
43 |       insert: (v: unknown) => Promise<{ error: { message: string } | null }>;
44 |     }
45 |   ).insert(
46 |     evidenceRows.map((r) => ({
47 |       ...r,
48 |       submission_id: attemptId, // re-use submission_id FK for quiz attempt reference
49 |       grade_id: attemptId, // re-use grade_id FK for quiz attempt reference
50 |       plo_id: null,
51 |       ilo_id: null,
52 |     })))
53 |   );
54 |
55 |   if (evidenceErr) {
56 |     console.error(
57 |       "[QuizEvidence] Failed to insert evidence:",
58 |       evidenceErr.message
59 |     );
60 |   }
61 |
62 |   // Fetch the quiz's course_id for attainment rollup
63 |   const { data: quiz } = await supabase
64 |     .from("quizzes")
65 |     .select("course_id")
66 |     .eq("id", quizId)
67 |     .maybeSingle();
68 |
69 |   if (!quiz) return;
70 |
71 |   // Upsert CLO attainment for each linked CLO
72 |   for (const cloId of cloIds) {
73 |     try {
74 |       const { data: evidenceList } = await supabase
75 |         .from("evidence")
76 |         .select("score_percent")
77 |         .eq("student_id", studentId)
78 |         .eq("clo_id", cloId);
79 |
80 |       if (!evidenceList || evidenceList.length === 0) continue;
81 |
82 |       const avgPercent =
83 |         evidenceList.reduce((sum, e) => sum + (e.score_percent ?? 0), 0) /
84 |         evidenceList.length;
85 |
86 |       await supabase.from("outcome_attainment").upsert(
87 |         {
88 |           outcome_id: cloId,
89 |           student_id: studentId,
90 |           course_id: quiz.course_id,
91 |           scope: "student_course",

```





```
79 |     },  
80 |   });  
81 |  
82 | // % % % Inferred Types % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
83 |  
84 | export type GenerateQuestionsInput = z.infer<typeof generateQuestionsSchema>;  
85 | export type QuestionBankEntry = z.infer<typeof questionBankEntrySchema>;  
86 | export type AdaptiveQuizConfig = z.infer<typeof adaptiveQuizConfigSchema>;  
87 | export type RecoverySession = z.infer<typeof recoverySessionSchema>;  
88 | export type VerifiedExplanation = z.infer<typeof verifiedExplanationSchema>;  
89 | export type PracticeModeConfig = z.infer<typeof practiceModeConfigSchema>;  
90 | export type BloomsClimbState = z.infer<typeof bloomsClimbStateSchema>;  
91 |
```

FILE: src/lib/quizGrader.ts

TYPE: .ts | LINES: 1-180 | SHA256: 5648a206b05db7c1de1bdc06520943b056a6964f970f53912d5e82cad4769cb2

```

1 // =====
2 // Quiz Auto-Grader – Pure functions for client-side quiz grading
3 // =====
4
5 export type QuestionType =
6   | "mcq_single"
7   | "mcq_multi"
8   | "true_false"
9   | "short_answer"
10  | "fill_blank";
11
12 export interface GradableQuestion {
13   id: string;
14   question_type: QuestionType;
15   correct_answer: string | string[];
16   points: number;
17 }
18
19 export interface QuestionGradeResult {
20   questionId: string;
21   score: number;
22   maxPoints: number;
23   autoGradable: boolean;
24   isCorrect: boolean | null; // null for manual-graded questions
25 }
26
27 export interface QuizGradeResult {
28   autoScore: number;
29   totalPoints: number;
30   totalAutoGradablePoints: number;
31   hasManualQuestions: boolean;
32   gradingStatus: "auto_graded" | "pending_manual" | "fully_graded";
33   scorePercent: number | null;
34   perQuestion: QuestionGradeResult[];
35 }
36
37 /**
38  * Grade a single question. Returns score and whether it's auto-gradable.
39  * Short answer questions are flagged for manual teacher grading.
40  */
41 export function gradeQuestion(
42   question: GradableQuestion,
43   studentAnswer: string | string[] | undefined
44 ): QuestionGradeResult {
45   const { id, question_type, correct_answer, points } = question;
46
47   if (!studentAnswer) {
48     return {
49       questionId: id,
50       score: 0,
51       maxPoints: points,
52       autoGradable: question_type !== "short_answer",
53       isCorrect: question_type === "short_answer" ? null : false,
54     };
55   }
56
57   switch (question_type) {
58     case "mcq_single":
59     case "true_false": {
60       const correct =
61         typeof correct_answer === "string"
62           ? correct_answer
63           : correct_answer[0] ?? "";
64       const answer =
65         typeof studentAnswer === "string"
66           ? studentAnswer
67           : studentAnswer[0] ?? "";
68       const isCorrect =
69         answer.trim().toLowerCase() === correct.trim().toLowerCase();
70       return {
71         questionId: id,
72         score: isCorrect ? points : 0,
73         maxPoints: points,
74         autoGradable: true,
75         isCorrect,
76       };
77     }
78
79     case "mcq_multi": {
80       const correctSet = new Set(
81         (Array.isArray(correct_answer) ? correct_answer : [correct_answer]).map(
82           (a) => a.trim().toLowerCase()
83         )
84       );
85       const answerSet = new Set(
86         (Array.isArray(studentAnswer) ? studentAnswer : [studentAnswer]).map(
87           (a) => a.trim().toLowerCase()
88         )
89       );
90       const isCorrect = correctSet.size === answerSet.size;
91       const score = isCorrect ? points : 0;
92       return {
93         questionId: id,
94         score,
95         maxPoints: points,
96         autoGradable: false,
97         isCorrect,
98       };
99     }
100
101     case "fill_blank": {
102       const correct = correct_answer;
103       const answer = studentAnswer;
104       const isCorrect =
105         correct instanceof String
106           ? correct.trim().toLowerCase() === answer.trim().toLowerCase()
107           : correct.some(
108               (c) => c.trim().toLowerCase() === answer.trim().toLowerCase()
109             );
110       return {
111         questionId: id,
112         score: isCorrect ? points : 0,
113         maxPoints: points,
114         autoGradable: false,
115         isCorrect,
116       };
117     }
118   }
119 }

```

```

88 |     )
89 |   );
90 |   const isCorrect =
91 |     correctSet.size === answerSet.size &&
92 |     [...correctSet].every((v) => answerSet.has(v));
93 |   return {
94 |     questionId: id,
95 |     score: isCorrect ? points : 0,
96 |     maxPoints: points,
97 |     autoGradable: true,
98 |     isCorrect,
99 |   };
100 | }
101 |
102 | case "fill_blank": {
103 |   const correct =
104 |     typeof correct_answer === "string"
105 |       ? correct_answer
106 |       : correct_answer[0] ?? "";
107 |   const answer =
108 |     typeof studentAnswer === "string"
109 |       ? studentAnswer
110 |       : studentAnswer[0] ?? "";
111 |   const isCorrect =
112 |     answer.trim().toLowerCase() === correct.trim().toLowerCase();
113 |   return {
114 |     questionId: id,
115 |     score: isCorrect ? points : 0,
116 |     maxPoints: points,
117 |     autoGradable: true,
118 |     isCorrect,
119 |   };
120 | }
121 |
122 | case "short_answer":
123 |   return {
124 |     questionId: id,
125 |     score: 0,
126 |     maxPoints: points,
127 |     autoGradable: false,
128 |     isCorrect: null,
129 |   };
130 |
131 | default:
132 |   return {
133 |     questionId: id,
134 |     score: 0,
135 |     maxPoints: points,
136 |     autoGradable: true,
137 |     isCorrect: false,
138 |   };
139 | }
140 | }
141 |
142 | /**
143 |  * Grade an entire quiz attempt. Auto-grades MCQ, true/false, fill-in-blank.
144 |  * Flags short answer questions for manual teacher grading.
145 |  */
146 | export function gradeQuiz(
147 |   questions: GradableQuestion[],
148 |   answers: Record<string, string | string[]>
149 | ): QuizGradeResult {
150 |   const perQuestion = questions.map((q) => gradeQuestion(q, answers[q.id]));
151 |
152 |   const autoScore = perQuestion
153 |     .filter((r) => r.autoGradable)
154 |     .reduce((sum, r) => sum + r.score, 0);
155 |
156 |   const totalPoints = perQuestion.reduce((sum, r) => sum + r.maxPoints, 0);
157 |
158 |   const totalAutoGradablePoints = perQuestion
159 |     .filter((r) => r.autoGradable)
160 |     .reduce((sum, r) => sum + r.maxPoints, 0);
161 |
162 |   const hasManualQuestions = perQuestion.some((r) => !r.autoGradable);
163 |   const gradingStatus = hasManualQuestions ? "pending_manual" : "fully_graded";
164 |
165 |   const scorePercent =
166 |     !hasManualQuestions && totalPoints > 0
167 |       ? Math.round((autoScore / totalPoints) * 10000) / 100
168 |       : null;
169 |
170 |   return {
171 |     autoScore,
172 |     totalPoints,
173 |     totalAutoGradablePoints,
174 |     hasManualQuestions,
175 |     gradingStatus,
176 |     scorePercent,
177 |     perQuestion,
178 |   };
179 | }
180 |

```

FILE: src/lib/quizScore.ts

TYPE: .ts | LINES: 1-44 | SHA256: d3e497f5a20cfe6b75a2d397ab95968c919feed570f61e6cb3330b973d1ec0e8

```

1 | // Task 6.11: Adaptive quiz score computation (pure business logic)
2 | // Requirements: 3.1, 3.3
3 |
4 | /**
5 |  * Computes the rounded percentage score for an adaptive quiz attempt.
6 |  *
7 |  * The score is derived from the number of correct answers over the number of

```

```

8 | * questions actually presented in the attempt. It is the single source of
9 | * truth for finalization (timer expiry, manual submit, or session completion),
10 | * so the recorded score always equals the score computed from the student's
11 | * most recently submitted answers (R3.1, R3.3).
12 | *
13 | * Rules:
14 | * - Returns a whole-number percentage in the inclusive range [0, 100].
15 | * - WHEN there are no questions (or the count is not a positive finite number),
16 | *   the score is 0 rather than `NaN`/`Infinity` (no division by zero).
17 | * - Non-finite or negative `totalCorrect` is treated conservatively as 0.
18 | * - `totalCorrect` is clamped to `totalQuestions` so a miscount can never
19 | *   produce a score above 100%.
20 | *
21 | * This function is pure and total: it returns a defined number for every input.
22 | *
23 | * @param totalCorrect - Number of questions answered correctly.
24 | * @param totalQuestions - Number of questions presented in the attempt.
25 | * @returns Rounded percentage score in [0, 100].
26 | */
27 | export function computeScore(
28 |   totalCorrect: number,
29 |   totalQuestions: number
30 | ): number {
31 |   // No questions !ò no score; also guards against division by zero and
32 |   // non-positive/non-finite question counts.
33 |   if (!Number.isFinite(totalQuestions) || totalQuestions <= 0) {
34 |     return 0;
35 |   }
36 |
37 |   // Undetermined or negative correct counts contribute nothing.
38 |   const correct = Number.isFinite(totalCorrect)
39 |     ? Math.max(0, Math.min(totalCorrect, totalQuestions))
40 |     : 0;
41 |
42 |   return Math.round((correct / totalQuestions) * 100);
43 | }
44 |

```

#### FILE: src/lib/quizXpAward.ts

TYPE: .ts | LINES: 1-53 | SHA256: a592faf764d1f47cd45f917d0d2e9433b783820cc6efce9ff1317aef24cbf826

```

1 | // =====
2 | // Quiz XP Award – Awards XP on quiz completion (50 on-time, 25 if late)
3 | // =====
4 |
5 | import { supabase } from "@lib/supabase";
6 |
7 | const QUIZ_XP_ON_TIME = 50;
8 | const QUIZ_XP_LATE = 25;
9 |
10 | interface QuizXpInput {
11 |   studentId: string;
12 |   quizId: string;
13 |   attemptId: string;
14 |   dueDate: string;
15 |   submittedAt: string;
16 | }
17 |
18 | /**
19 |  * Award XP for quiz completion using the same schedule as assignments:
20 |  * - 50 XP if submitted on-time (before due date)
21 |  * - 25 XP if submitted late (after due date)
22 |  *
23 |  * Calls the award-xp Edge Function with source='quiz_completion'.
24 |  */
25 | export async function awardQuizXp(input: QuizXpInput): Promise<void> {
26 |   const { studentId, quizId, attemptId, dueDate, submittedAt } = input;
27 |
28 |   const isLate = new Date(submittedAt) > new Date(dueDate);
29 |   const xpAmount = isLate ? QUIZ_XP_LATE : QUIZ_XP_ON_TIME;
30 |
31 |   try {
32 |     const { error } = await supabase.functions.invoke("award-xp", {
33 |       body: {
34 |         student_id: studentId,
35 |         xp_amount: xpAmount,
36 |         source: "quiz_completion",
37 |         reference_id: `quiz:${quizId}:${attemptId}`,
38 |         note: isLate
39 |           ? `Late quiz completion: ${quizId}`
40 |           : `On-time quiz completion: ${quizId}`,
41 |       },
42 |     });
43 |
44 |     if (error) {
45 |       console.error("[QuizXP] Failed to award XP:", error.message);
46 |     }
47 |   } catch (err) {
48 |     console.error("[QuizXP] Unexpected error awarding XP:", err);
49 |   }
50 | }
51 |
52 | export { QUIZ_XP_ON_TIME, QUIZ_XP_LATE };
53 |

```

#### FILE: src/lib/reflectionPrompts.ts

TYPE: .ts | LINES: 1-46 | SHA256: c9544384553a4fa41dacbf1007f7fc4bc0d32e60f495c616044b8bad1e4059b8

```

1 | // =====
2 | // reflectionPrompts – static reflection prompt templates for the journal
3 | // =====
4 | //
5 | // These are the always-available reflection templates surfaced on the journal
6 | // surface a student reaches from navigation (R10.1). They are intentionally

```

```

7 | // CLO-independent so journaling stays possible even when the richer,
8 | // CLO-contextual `generateJournalPrompt` capability is unavailable (R10.3a).
9 | //
10 | // Each template maps to an i18n key under `journal.prompts.*` in the student
11 | // locale so the prompt text is available in both English and Arabic (R10.4).
12 | // This module is pure data – no React, no i18n runtime, no side effects.
13 |
14 | export interface ReflectionPromptTemplate {
15 |   /** Stable identifier used as a React key and in tests. */
16 |   id: string;
17 |   /** i18n key (in the `student` namespace) for the prompt's display + seed text. */
18 |   i18nKey: string;
19 | }
20 |
21 | export const REFLECTION_PROMPT_TEMPLATES: readonly ReflectionPromptTemplate[] =
22 |   [
23 |     { id: "learned", i18nKey: "journal.prompts.learned" },
24 |     { id: "confused", i18nKey: "journal.prompts.confused" },
25 |     { id: "proud", i18nKey: "journal.prompts.proud" },
26 |   ] as const;
27 |
28 | /**
29 |  * Seeds journal content with a selected prompt (R10.2).
30 |  *
31 |  * Pure function: given the current content and a prompt, returns the new
32 |  * content. If the current content is empty the prompt becomes the starting
33 |  * point; otherwise the prompt is appended on a fresh line so existing writing
34 |  * is never lost.
35 |  */
36 | export function seedContentWithPrompt(
37 |   currentContent: string,
38 |   promptText: string
39 | ): string {
40 |   const existing = currentContent.trim();
41 |   if (existing.length === 0) {
42 |     return `${promptText}\n\n`;
43 |   }
44 |   return `${existing}\n\n${promptText}\n\n`;
45 | }
46 |

```

## FILE: src/lib/routePrefetch.ts

TYPE: .ts | LINES: 1-60 | SHA256: a3bdc984d01582850d525632f81a92830c72da2db234a917f366d97b0395d443

```

1 | /**
2 |  * Route-chunk prefetch registry (spec: dashboard-and-ux-performance, Req 9).
3 |  *
4 |  * Maps a sidebar nav path to a thunk that dynamically `import()`s the SAME lazy
5 |  * route module `AppRouter` loads. Calling it on hover/focus warms the route's JS
6 |  * chunk so the click-through navigation has no lazy-load wait. The browser/Vite
7 |  * dedupe the module, so re-importing is cheap and re-running is a no-op.
8 |  *
9 |  * Coverage is the high-traffic routes per role (dashboards + primary list
10 |  * pages); unmapped paths are a safe no-op, so adding more is purely additive.
11 |  * The import specifiers MUST match `AppRouter`'s `lazy(() => import(...))` exactly
12 |  * so the same chunk is warmed (not a second copy).
13 |  *
14 |  * Intentionally only warms the JS chunk: a route's primary *query* prefetch needs
15 |  * per-route keys/filters (and auth context), so that stays the route component's
16 |  * responsibility rather than being guessed here.
17 |  */
18 | const ROUTE_CHUNK_LOADERS: Record<string, () => Promise<unknown>> = {
19 |   // Admin
20 |   "/admin/dashboard": () => import("@/pages/admin/AdminDashboard"),
21 |   "/admin/users": () => import("@/pages/admin/users/UserListPage"),
22 |   "/admin/programs": () => import("@/pages/admin/programs/ProgramListPage"),
23 |   "/admin/courses": () => import("@/pages/admin/courses/CourseListPage"),
24 |   "/admin/outcomes": () => import("@/pages/admin/outcomes/ILOListPage"),
25 |   // Coordinator
26 |   "/coordinator/dashboard": () =>
27 |     import("@/pages/coordinator/CoordinatorDashboard"),
28 |   "/coordinator/plos": () => import("@/pages/coordinator/plos/PLoListPage"),
29 |   // Teacher
30 |   "/teacher/dashboard": () => import("@/pages/teacher/TeacherDashboard"),
31 |   "/teacher/clos": () => import("@/pages/teacher/clos/CLOListPage"),
32 |   "/teacher/assignments": () =>
33 |     import("@/pages/teacher/assignments/AssignmentListPage"),
34 |   "/teacher/grading": () => import("@/pages/teacher/grading/GradingQueuePage"),
35 |   "/teacher/rubrics": () => import("@/pages/teacher/rubrics/RubricListPage"),
36 |   // Student
37 |   "/student/dashboard": () => import("@/pages/student/StudentDashboard"),
38 |   "/student/courses": () =>
39 |     import("@/pages/student/courses/StudentCoursesPage"),
40 |   "/student/assignments": () =>
41 |     import("@/pages/student/assignments/AssignmentListPage"),
42 |   // Parent
43 |   "/parent/dashboard": () => import("@/pages/parent/ParentDashboard"),
44 | };
45 |
46 | /**
47 |  * Warms the route chunk for `path` if it is in the registry. Returns the import
48 |  * promise (so callers may await/catch) or `undefined` for an unmapped path.
49 |  * Never throws synchronously; rejection handling is the caller's concern
50 |  * ( `useIntentPrefetch` swallows it).
51 |  */
52 | export const prefetchRoute = (path: string): Promise<unknown> | undefined => {
53 |   const loader = ROUTE_CHUNK_LOADERS[path];
54 |   return loader ? loader() : undefined;
55 | };
56 |
57 | /** True when a path has a registered chunk loader (used in tests). */
58 | export const hasRoutePrefetch = (path: string): boolean =>
59 |   path in ROUTE_CHUNK_LOADERS;
60 |

```

## FILE: src/lib/salePriceCalculator.ts

TYPE: .ts | LINES: 1-39 | SHA256: e36206272de5e2fda13bb48381d900b3636946719e4e82567e725cc8b0114194

```
1 | // =====
2 | // Sale Price Calculator – Pure functions: apply discounts to item prices
3 | // =====
4 |
5 | export interface ActiveDiscount {
6 |   discount_percentage: number;
7 | }
8 |
9 | /**
10 |  * Return the highest discount percentage from a list of active discounts.
11 |  * Returns 0 when no discounts are active.
12 |  */
13 | export function getHighestDiscount(
14 |   activeDiscounts: readonly ActiveDiscount[]
15 | ): number {
16 |   if (activeDiscounts.length === 0) return 0;
17 |   return Math.max(...activeDiscounts.map((d) => d.discount_percentage));
18 | }
19 |
20 | /**
21 |  * Compute the effective sale price for an item.
22 |  *
23 |  * Formula: GREATEST(1, base_price - floor(base_price * max_discount / 100))
24 |  *
25 |  * Takes the highest active discount and applies it. The minimum price is 1 XP
26 |  * to prevent free items via extreme discounts.
27 |  */
28 | export function computeSalePrice(
29 |   basePrice: number,
30 |   activeDiscounts: readonly ActiveDiscount[]
31 | ): number {
32 |   const maxDiscount = getHighestDiscount(activeDiscounts);
33 |   if (maxDiscount === 0) return basePrice;
34 |
35 |   const discountedPrice =
36 |     basePrice - Math.floor((basePrice * maxDiscount) / 100);
37 |   return Math.max(1, discountedPrice);
38 | }
39 |
```

## FILE: src/lib/sanitizeFilter.ts

TYPE: .ts | LINES: 1-14 | SHA256: e08f040125c7c425f32128779c6b292919a6b33657e9085bc0d40ee6e747e3af

```
1 | /**
2 |  * Escapes PostgREST special characters in user-provided search strings
3 |  * to prevent filter injection when interpolating into `.or()` expressions.
4 |  *
5 |  * Characters escaped: . , ( ) % * \
6 |  *
7 |  * @example
8 |  *   sanitizePostgresValue('admin%') // => 'admin\\%'
9 |  *   sanitizePostgresValue('a.b,c(d)') // => 'a\\b\\c\\(d\\)'
10 |  */
11 | export function sanitizePostgresValue(input: string): string {
12 |   return input.replace(/[.,()%%*]/g, (char) => `\\${char}`);
13 | }
14 |
```

## FILE: src/lib/sankeyTransform.ts

TYPE: .ts | LINES: 1-76 | SHA256: 27fd3b2132a78d735325be575845ab195900733df9727d248845b86480633635

```
1 | // Task 117.1: Sankey data transformation utility
2 |
3 | export interface SankeyNode {
4 |   name: string;
5 |   id: string;
6 |   type: "ILO" | "PLO" | "CLO";
7 |   attainment: number;
8 |   color: string;
9 | }
10 |
11 | export interface SankeyLink {
12 |   source: number;
13 |   target: number;
14 |   value: number;
15 |   weight: number;
16 | }
17 |
18 | interface OutcomeMapping {
19 |   parent_id: string;
20 |   child_id: string;
21 |   weight: number;
22 | }
23 |
24 | interface OutcomeAttainment {
25 |   outcome_id: string;
26 |   score_percent: number;
27 | }
28 |
29 | interface Outcome {
30 |   id: string;
31 |   type: "ILO" | "PLO" | "CLO";
32 |   title: string;
33 | }
34 |
35 | const getAttainmentColor = (score: number): string => {
36 |   if (score >= 85) return "#22c55e"; // green – Excellent
37 |   if (score >= 70) return "#3b82f6"; // blue – Satisfactory
38 |   if (score >= 50) return "#f59e0b"; // yellow – Developing
39 |   if (score > 0) return "#ef4444"; // red – Not Yet

```

```

40 |     return "#94a3b8"; // gray – Unmapped
41 | };
42 |
43 | export const transformToSankey = (
44 |   outcomes: Outcome[],
45 |   mappings: OutcomeMapping[],
46 |   attainments: OutcomeAttainment[]
47 | ): { nodes: SankeyNode[]; links: SankeyLink[] } => {
48 |   const attMap = new Map(
49 |     attainments.map((a) => [a.outcome_id, a.score_percent])
50 |   );
51 |
52 |   const nodes: SankeyNode[] = outcomes.map((o) => {
53 |     const att = attMap.get(o.id) ?? 0;
54 |     return {
55 |       name: o.title,
56 |       id: o.id,
57 |       type: o.type,
58 |       attainment: att,
59 |       color: getAttainmentColor(att),
60 |     };
61 |   });
62 |
63 |   const nodeIndex = new Map(nodes.map((n, i) => [n.id, i]));
64 |
65 |   const links: SankeyLink[] = mappings
66 |     .filter((m) => nodeIndex.has(m.parent_id) && nodeIndex.has(m.child_id))
67 |     .map((m) => ({
68 |       source: nodeIndex.get(m.child_id)!,
69 |       target: nodeIndex.get(m.parent_id)!,
70 |       value: m.weight,
71 |       weight: m.weight,
72 |     }));
73 |
74 |   return { nodes, links };
75 | };
76 |

```

#### FILE: [src/lib/schemas/academicCalendar.ts](#)

TYPE: .ts | LINES: 1-22 | SHA256: 125194b129a523cae23aa78f1ac112ad2e99882af69b79554f9d9a8876906775

```

1 | import { z } from "zod";
2 |
3 | export const academicCalendarEventSchema = z.object({
4 |   semester_id: z.uuid(),
5 |   title: z.string().min(1, "Event title is required").max(255),
6 |   event_type: z.enum([
7 |     "semester_start",
8 |     "semester_end",
9 |     "exam_period",
10 |     "holiday",
11 |     "registration",
12 |     "custom",
13 |   ]),
14 |   start_date: z.iso.date(),
15 |   end_date: z.iso.date(),
16 |   is_recurring: z.boolean().default(false),
17 | });
18 |
19 | export type AcademicCalendarEventFormData = z.infer<
20 |   typeof academicCalendarEventSchema
21 | >;
22 |

```

#### FILE: [src/lib/schemas/adaptiveXP.ts](#)

TYPE: .ts | LINES: 1-20 | SHA256: b2c04c0d11d2ae23aeb050327fa992f7edc535df22ed7eb80c8a40a1adb573a

```

1 | // Task 137.2: Adaptive XP Zod schemas
2 |
3 | import { z } from "zod";
4 |
5 | export const xpMultiplierSchema = z.object({
6 |   level_multiplier: z.number().min(0.1).max(2.0),
7 |   difficulty_multiplier: z.number().min(1.0).max(2.0),
8 |   diminishing_multiplier: z.number().min(0.2).max(1.0),
9 | });
10 |
11 | export type XPMultiplier = z.infer<typeof xpMultiplierSchema>;
12 |
13 | export const diminishingReturnsSchema = z.object({
14 |   action_type: z.string().min(1),
15 |   repeat_count: z.number().int().min(0),
16 |   window_start: z.string().datetime(),
17 | });
18 |
19 | export type DiminishingReturns = z.infer<typeof diminishingReturnsSchema>;
20 |

```

#### FILE: [src/lib/schemas/aiSuggestion.ts](#)

TYPE: .ts | LINES: 1-45 | SHA256: b0ff4540d8d1dd991c1d1116119c58706100c3ee01adb8dc4b0b758d9102291a

```

1 | import { z } from "zod";
2 |
3 | export const moduleSuggestionSchema = z.object({
4 |   student_id: z.uuid(),
5 |   weak_clo_id: z.uuid(),
6 |   weak_clo_title: z.string(),
7 |   weak_clo_attainment: z.number().min(0).max(100),
8 |   prerequisite_clo_id: z.uuid().nullable(),
9 |   prerequisite_clo_title: z.string().nullable(),
10 |   suggestion_text: z.string().min(1),
11 |   social_proof_text: z.string().nullable(),
12 |   feedback: z.enum(["thumbs_up", "thumbs_down"]).nullable(),

```



```

13 | });
14 |
15 | export const atRiskPredictionSchema = z.object({
16 |   student_id: z.uuid(),
17 |   at_risk_clo_id: z.uuid(),
18 |   at_risk_clo_title: z.string(),
19 |   probability_score: z.number().min(0).max(100),
20 |   contributing_signals: z.object({
21 |     login_frequency: z.enum(["low", "medium", "high"]),
22 |     submission_pattern: z.enum(["early", "on_time", "late", "missed"]),
23 |     attainment_trend: z.enum(["improving", "declining", "stagnant"]),
24 |   }),
25 |   prediction_date: z.iso.date(),
26 |   validated_outcome: z.enum(["correct", "incorrect"]).nullable(),
27 | });
28 |
29 | export const feedbackDraftSchema = z.object({
30 |   criterion_id: z.uuid(),
31 |   criterion_name: z.string(),
32 |   draft_comment: z.string(),
33 |   status: z.enum(["pending", "accepted", "edited", "rejected"]),
34 | });
35 |
36 | export const aiFeedbackSchema = z.object({
37 |   feedback_id: z.uuid(),
38 |   feedback: z.enum(["thumbs_up", "thumbs_down"]),
39 | });
40 |
41 | export type ModuleSuggestionData = z.infer<typeof moduleSuggestionSchema>;
42 | export type AtRiskPredictionData = z.infer<typeof atRiskPredictionSchema>;
43 | export type FeedbackDraftData = z.infer<typeof feedbackDraftSchema>;
44 | export type AIFeedbackData = z.infer<typeof aiFeedbackSchema>;
45 |

```

#### FILE: src/lib/schemas/announcement.ts

TYPE: .ts | LINES: 1-13 | SHA256: 5cc9dcece33fbc4e5cd9484a5d95d2dc41964ab96a87f6610c04b5ee18a6c5f9

```

1 | import { z } from "zod";
2 |
3 | export const createAnnouncementSchema = z.object({
4 |   course_id: z.uuid(),
5 |   title: z.string().min(1, "Title is required").max(255),
6 |   content: z.string().min(1, "Content is required"),
7 |   is_pinned: z.boolean().default(false),
8 | });
9 |
10 | export type CreateAnnouncementFormData = z.infer<
11 |   typeof createAnnouncementSchema
12 | >;
13 |

```

#### FILE: src/lib/schemas/announcementAttachment.ts

TYPE: .ts | LINES: 1-45 | SHA256: a78cd372ad64456b92af7a5d2a5897e6a80e994873d6452dfc4372e2885016d1

```

1 | // Feature: qa-partner-review-remediation - Req 15.3 (announcement attachments)
2 | // Client-side file validation for announcement attachments. Mirrors the avatar
3 | // upload schema pattern and is consumed by validateAnnouncementAttachmentFile
4 | // in src/lib/fileUpload.ts so the same allow-list is enforced before any upload.
5 | import { z } from "zod";
6 |
7 | /** Documents are capped at 10MB, consistent with submission uploads. */
8 | export const ANNOUNCEMENT_ATTACHMENT_MAX_SIZE_BYTES = 10 * 1024 * 1024; // 10MB
9 |
10 | /** Allowed MIME types - common documents and images teachers attach. */
11 | export const ANNOUNCEMENT_ATTACHMENT_ALLOWED_TYPES: readonly string[] = [
12 |   "application/pdf",
13 |   "application/msword", // .doc
14 |   "application/vnd.openxmlformats-officedocument.wordprocessingml.document", // .docx
15 |   "application/vnd.ms-powerpoint", // .ppt
16 |   "application/vnd.openxmlformats-officedocument.presentationml.presentation", // .pptx
17 |   "application/vnd.ms-excel", // .xls
18 |   "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet", // .xlsx
19 |   "text/plain",
20 |   "image/png",
21 |   "image/jpeg",
22 | ];
23 |
24 | /** Human-readable list of accepted formats for error/help copy. */
25 | export const ANNOUNCEMENT_ATTACHMENT_ACCEPTED_LABEL =
26 |   "PDF, Word, PowerPoint, Excel, TXT, PNG, or JPG";
27 |
28 | export const announcementAttachmentSchema = z.object({
29 |   file: z
30 |     .instanceof(File)
31 |     .refine(
32 |       (file) => ANNOUNCEMENT_ATTACHMENT_ALLOWED_TYPES.includes(file.type),
33 |       {
34 |         message: `Unsupported file type. Accepted formats: ${ANNOUNCEMENT_ATTACHMENT_ACCEPTED_LABEL}.`,
35 |       }
36 |     )
37 |     .refine((file) => file.size <= ANNOUNCEMENT_ATTACHMENT_MAX_SIZE_BYTES, {
38 |       message: `File exceeds the 10MB limit.`,
39 |     }),
40 | });
41 |
42 | export type AnnouncementAttachmentInput = z.infer<
43 |   typeof announcementAttachmentSchema
44 | >;
45 |

```



### FILE: src/lib/schemas/assignment.ts

TYPE: .ts | LINES: 1-32 | SHA256: f67e6a3c7f392b66938f6322585bcc28eed98a7c46475b8b4a78257dd523e2bd

```
1 | import { z } from "zod";
2 |
3 | export const cloWeightSchema = z.object({
4 |   clo_id: z.uuid(),
5 |   weight: z.number().min(0).max(100),
6 | });
7 |
8 | export const createAssignmentSchema = z.object({
9 |   title: z.string().min(1, "Title is required").max(255),
10 |   description: z.string().min(1, "Description is required"),
11 |   course_id: z.uuid(),
12 |   due_date: z.iso.datetime(),
13 |   total_marks: z.number().int().min(1),
14 |   clo_weights: z
15 |     .array(cloWeightSchema)
16 |     .min(1, "At least 1 CLO required")
17 |     .max(3, "Maximum 3 CLOs allowed"),
18 |   late_window_hours: z.number().int().min(0).default(24),
19 |   prerequisites: z
20 |     .array(
21 |       z.object({
22 |         clo_id: z.uuid(),
23 |         required_attainment: z.number().min(0).max(100),
24 |       })
25 |     )
26 |     .optional(),
27 |   tutor_autonomy_level: z.enum(["L1", "L2", "L3"]).optional(),
28 | });
29 |
30 | export type CLOWeightFormData = z.infer<typeof cloWeightSchema>;
31 | export type CreateAssignmentFormData = z.infer<typeof createAssignmentSchema>;
32 |
```

### FILE: src/lib/schemas/attendance.ts

TYPE: .ts | LINES: 1-22 | SHA256: 29c343d2915f92b6cfa2ebf961533116f7a0bd8a327f26ea69ca1220f3455be7

```
1 | import { z } from "zod";
2 |
3 | export const createSessionSchema = z.object({
4 |   section_id: z.uuid(),
5 |   session_date: z.iso.date(),
6 |   session_type: z.enum(["lecture", "lab", "tutorial"]),
7 |   topic: z.string().min(1, "Topic is required").max(255),
8 | });
9 |
10 | export const attendanceRecordSchema = z.object({
11 |   session_id: z.uuid(),
12 |   records: z.array(
13 |     z.object({
14 |       student_id: z.uuid(),
15 |       status: z.enum(["present", "absent", "late", "excused"]),
16 |     })
17 |   ),
18 | });
19 |
20 | export type CreateSessionFormData = z.infer<typeof createSessionSchema>;
21 | export type AttendanceRecordFormData = z.infer<typeof attendanceRecordSchema>;
22 |
```

### FILE: src/lib/schemas/auth.ts

TYPE: .ts | LINES: 1-25 | SHA256: eee9ee8e4c1babd7774ed1c0ac55463b610abfc8da19e11d9d19fc8838e737c4

```
1 | import { z } from "zod";
2 |
3 | export const loginSchema = z.object({
4 |   email: z.email("Invalid email address"),
5 |   password: z.string().min(8, "Password must be at least 8 characters"),
6 | });
7 |
8 | export const resetPasswordSchema = z.object({
9 |   email: z.email("Invalid email address"),
10 | });
11 |
12 | export const updatePasswordSchema = z
13 |   .object({
14 |     password: z.string().min(8, "Password must be at least 8 characters"),
15 |     confirmPassword: z.string().min(1, "Please confirm your password"),
16 |   })
17 |   .refine((data) => data.password === data.confirmPassword, {
18 |     message: "Passwords do not match",
19 |     path: ["confirmPassword"],
20 |   });
21 |
22 | export type LoginFormData = z.infer<typeof loginSchema>;
23 | export type ResetPasswordFormData = z.infer<typeof resetPasswordSchema>;
24 | export type UpdatePasswordFormData = z.infer<typeof updatePasswordSchema>;
25 |
```

### FILE: src/lib/schemas/avatarUpload.ts

TYPE: .ts | LINES: 1-34 | SHA256: 1ade1b8654704b09b141c74c187b772e555f0bf69796678006615ed6bbc3cd63

```
1 | import { z } from "zod";
2 |
3 | /**
4 |  * Avatar upload validation schema
5 |  *
6 |  * Validates:
7 |  * - File type: png, jpg, jpeg, webp
8 |  * - File size: "d 2 MB
9 |  */
```

```

10 | export const avatarUploadSchema = z.object({
11 |   file: z
12 |     .instanceof(File)
13 |     .refine(
14 |       (file) => {
15 |         const validTypes = ["image/png", "image/jpeg", "image/webp"];
16 |         return validTypes.includes(file.type);
17 |       },
18 |       {
19 |         message: "Invalid file type. Please upload a PNG, JPG, or WebP image.",
20 |       }
21 |     )
22 |     .refine(
23 |       (file) => {
24 |         const maxSize = 2 * 1024 * 1024; // 2 MB
25 |         return file.size <= maxSize;
26 |       },
27 |       {
28 |         message: "File is too large. Maximum size is 2 MB.",
29 |       }
30 |     ),
31 | });
32 |
33 | export type AvatarUploadInput = z.infer<typeof avatarUploadSchema>;
34 |

```

#### FILE: src/lib/schemas/badgeDefinition.ts

TYPE: .ts | LINES: 1-23 | SHA256: 75269aa073698e139dba7640bc47e317bcf165b070cc52dfc84123060c55b7ff

```

1 | import { z } from "zod";
2 |
3 | export const badgeDefinitionSchema = z.object({
4 |   name: z.string().trim().min(1, "Name is required").max(80),
5 |   badge_key: z
6 |     .string()
7 |     .trim()
8 |     .min(1, "Badge key is required")
9 |     .max(80)
10 |    .regex(
11 |      /^[a-z0-9]+(?:_[a-z0-9]+)*$/,
12 |      "Use lowercase letters, numbers, and underscores"
13 |    ),
14 |   description: z.string().trim().max(240),
15 |   emoji: z.string().trim().min(1, "Emoji is required").max(12),
16 |   category: z.string().trim().max(40),
17 |   bronze_condition: z.string().trim().max(120),
18 |   silver_condition: z.string().trim().max(120),
19 |   gold_condition: z.string().trim().max(120),
20 | });
21 |
22 | export type BadgeDefinitionFormValues = z.infer<typeof badgeDefinitionSchema>;
23 |

```

#### FILE: src/lib/schemas/badgeSpotlight.ts

TYPE: .ts | LINES: 1-26 | SHA256: 022b5e8e69131d65f03c53d07b8f5f2c0927b724d32261b42d572174d1aa455a

```

1 | // Task 150.3: Badge Spotlight Zod schema
2 | // Requirements: 134.2
3 |
4 | import { z } from "zod";
5 |
6 | /**
7 |  * Validates that a date string falls on a Monday (ISO day 1).
8 |  */
9 | function isMonday(dateStr: string): boolean {
10 |   const d = new Date(dateStr + "T00:00:00Z");
11 |   return d.getUTCDay() === 1;
12 | }
13 |
14 | export const badgeSpotlightScheduleSchema = z.object({
15 |   week_start: z
16 |     .string()
17 |     .regex(/^\d{4}-\d{2}-\d{2}$/, "Must be YYYY-MM-DD format")
18 |     .refine(isMonday, { message: "week_start must be a Monday" }),
19 |   category: z.string().min(1, "Category is required"),
20 |   is_manual: z.boolean().default(false),
21 | });
22 |
23 | export type BadgeSpotlightSchedule = z.infer<
24 |   typeof badgeSpotlightScheduleSchema
25 | >;
26 |

```

#### FILE: src/lib/schemas/badgeTier.ts

TYPE: .ts | LINES: 1-28 | SHA256: 9ba0db268aaf6b55d9b6b17640c2c59560dc704ff996aec3ea7a98e37400c61f

```

1 | // Task 150.2: Badge Tier Zod schemas
2 |
3 | import { z } from "zod";
4 |
5 | export const badgeTierSchema = z.object({
6 |   tier: z.enum(["bronze", "silver", "gold"]),
7 |   category: z.string().min(1),
8 | });
9 |
10 | export type BadgeTier = z.infer<typeof badgeTierSchema>;
11 |
12 | export const tieredBadgeSchema = z.object({
13 |   category: z.string().min(1),
14 |   current_tier: z.enum(["bronze", "silver", "gold"]).nullable(),
15 |   progress: z.number().min(0).max(100),
16 |   is_pinned: z.boolean().default(false),
17 |   archived_at: z.string().datetime().nullable().default(null),

```

```

18 | });
19 |
20 | export type TieredBadge = z.infer<typeof tieredBadgeSchema>;
21 |
22 | export const badgePinSchema = z.object({
23 |   student_badge_id: z.string().uuid(),
24 |   is_pinned: z.boolean(),
25 | });
26 |
27 | export type BadgePin = z.infer<typeof badgePinSchema>;
28 |

```

#### FILE: src/lib/schemas/bonusXPEvent.ts

TYPE: .ts | LINES: 1-24 | SHA256: f61d6bc2e97c438df1541305fbfe6734b99393de00158d7abf3ac1700db43e9d

```

1 | import { z } from "zod";
2 |
3 | export const bonusXPEventSchema = z.object({
4 |   id: z.uuid(),
5 |   title: z.string().min(1),
6 |   multiplier: z.number().positive(),
7 |   starts_at: z.iso.datetime(),
8 |   ends_at: z.iso.datetime(),
9 |   is_active: z.boolean(),
10 |   created_by: z.uuid(),
11 | });
12 |
13 | export const createBonusEventSchema = z.object({
14 |   name: z.string().min(1, "Name is required").max(255),
15 |   event_type: z.string().min(1, "Event type is required").default("bonus_xp"),
16 |   xp_multiplier: z.number().positive().default(2),
17 |   starts_at: z.iso.datetime(),
18 |   ends_at: z.iso.datetime(),
19 |   is_active: z.boolean().default(true),
20 | });
21 |
22 | export type BonusXPEventData = z.infer<typeof bonusXPEventSchema>;
23 | export type CreateBonusEventFormData = z.infer<typeof createBonusEventSchema>;
24 |

```

#### FILE: src/lib/schemas/bulkGradeExport.ts

TYPE: .ts | LINES: 1-11 | SHA256: bba51fc2d8891426b3c533f42cfd504d97540e3c21bb0637dac374f18d3bfc9a

```

1 | import { z } from "zod";
2 |
3 | export const gradeExportSchema = z.object({
4 |   course_id: z.uuid(),
5 |   semester_id: z.uuid().optional(),
6 |   format: z.enum(["csv", "xlsx"]),
7 |   include_clo_attainment: z.boolean().default(false),
8 | });
9 |
10 | export type GradeExportFormData = z.infer<typeof gradeExportSchema>;
11 |

```

#### FILE: src/lib/schemas/bulkImport.ts

TYPE: .ts | LINES: 1-11 | SHA256: 2fc3e8f503fde82435a3ebf41f85edb639a44a2e29b2a166d403b064daa63e46

```

1 | import { z } from "zod";
2 |
3 | export const csvRowSchema = z.object({
4 |   email: z.email("Invalid email address"),
5 |   full_name: z.string().min(1, "Full name is required"),
6 |   role: z.enum(["admin", "coordinator", "teacher", "student", "parent"]),
7 |   program_id: z.uuid().optional(),
8 | });
9 |
10 | export type CSVRowData = z.infer<typeof csvRowSchema>;
11 |

```

#### FILE: src/lib/schemas/challenge.ts

TYPE: .ts | LINES: 1-107 | SHA256: 7474accd7d3a1752f386603e8e2cef4b0a7814daa6d29059e6a4edc95ec8e2a2

```

1 | // Challenge Zod schemas for Social Challenges (Quests) feature
2 |
3 | import { z } from "zod";
4 |
5 | // % % Legacy schemas (preserved for backward compatibility) % % % % % % % % % % % % % % % % % %
6 |
7 | export const challengeSchema = z
8 |   .object({
9 |     title: z.string().min(1, "Title is required").max(200),
10 |     description: z.string().max(1000).optional(),
11 |     challenge_type: z.enum(["team", "course_wide"]),
12 |     course_id: z.string().uuid(),
13 |     start_date: z.string().datetime(),
14 |     end_date: z.string().datetime(),
15 |     goal_metric: z.enum([
16 |       "total_xp",
17 |       "habits_completed",
18 |       "assignments_submitted",
19 |       "quiz_score_avg",
20 |     ]),
21 |     goal_target: z.number().int().positive(),
22 |     reward_type: z.enum(["xp_bonus", "badge"]),
23 |     reward_value: z.number().int().min(0),
24 |   })
25 |   .refine((data) => new Date(data.end_date) > new Date(data.start_date), {
26 |     message: "End date must be after start date",
27 |     path: ["end_date"],
28 |   });
29 |
30 | export type ChallengeInput = z.infer<typeof challengeSchema>;

```

[illegible]

FILE: src/lib/schemas/clo.ts

TYPE: .ts | LINES: 1-32 | SHA256: c8a292ec86c63e34dd5163e5b7f2fb588be8125457e3118e8759facefc08dbe3

```

1 import { z } from "zod";
2
3 export const bloomsLevelSchema = z.enum([
4   "remembering",
5   "understanding",
6   "applying",
7   "analyzing",
8   "evaluating",
9   "creating",
10 ]);
11
12 export const createCLOSchema = z.object({
13   title: z.string().min(1, "Title is required").max(255),
14   title_ar: z.string().max(255).optional(),
15   description: z.string().optional(),
16   course_id: z.uuid(),
17   blooms_level: bloomsLevelSchema,
18   sort_order: z.number().int().min(0).optional(),
19   plo_mappings: z
20     .array(
21       z.object({
22         plo_id: z.uuid(),
23         weight: z.number().min(0).max(1),

```

```

24 |     })
25 |   )
26 |   .optional(),
27 |   tutor_autonomy_level: z.enum(["L1", "L2", "L3"]).optional(),
28 | });
29 |
30 | export type BloomsLevel = z.infer<typeof bloomsLevelSchema>;
31 | export type CreateCLOFormData = z.infer<typeof createCLOSchema>;
32 |

```

#### FILE: src/lib/schemas/comebackChallenge.ts

TYPE: .ts | LINES: 1-14 | SHA256: 3c3d3c2efae7b15a53926facb2061ca50136bd3a5185d4751c2d0a2de953b221

```

1 | // Task 143.2: Comeback Challenge Zod schemas
2 |
3 | import { z } from "zod";
4 |
5 | export const comebackChallengeStateSchema = z.object({
6 |   is_active: z.boolean(),
7 |   days_completed: z.number().int().min(0).max(3),
8 |   streak_to_restore: z.number().int().min(0),
9 | });
10 |
11 | export type ComebackChallengeState = z.infer<
12 |   typeof comebackChallengeStateSchema
13 | >;
14 |

```

#### FILE: src/lib/schemas/competencyFramework.ts

TYPE: .ts | LINES: 1-33 | SHA256: 757e9520793de4ebcd246215c841f7195a12cd703df98d13777e9ff0dd952442

```

1 | // Task 114.2: Competency Framework Zod schemas
2 |
3 | import { z } from "zod";
4 |
5 | export const competencyFrameworkSchema = z.object({
6 |   name: z.string().min(1, "Name is required").max(200),
7 |   version: z.string().min(1, "Version is required").max(20),
8 |   source: z.string().max(100).optional(),
9 | });
10 |
11 | export const competencyItemSchema = z.object({
12 |   framework_id: z.string().uuid(),
13 |   parent_id: z.string().uuid().nullable(),
14 |   level: z.enum(["domain", "competency", "indicator"]),
15 |   code: z.string().min(1).max(20),
16 |   title: z.string().min(1).max(300),
17 | });
18 |
19 | export const competencyCSVRowSchema = z.object({
20 |   domain_code: z.string().min(1),
21 |   domain_title: z.string().min(1),
22 |   competency_code: z.string().min(1),
23 |   competency_title: z.string().min(1),
24 |   indicator_code: z.string().min(1),
25 |   indicator_title: z.string().min(1),
26 | });
27 |
28 | export type CreateCompetencyFrameworkInput = z.infer<
29 |   typeof competencyFrameworkSchema
30 | >;
31 | export type CompetencyItemInput = z.infer<typeof competencyItemSchema>;
32 | export type CompetencyCSVRow = z.infer<typeof competencyCSVRowSchema>;
33 |

```

#### FILE: src/lib/schemas/cookieConsent.ts

TYPE: .ts | LINES: 1-11 | SHA256: 33e592a913ea68ddaf37d38f5ed681acd70342072bac1a31f5ed33081b831fb6

```

1 | import { z } from "zod";
2 |
3 | export const cookieConsentSchema = z.object({
4 |   essential: z.literal(true),
5 |   analytics: z.boolean().default(false),
6 |   marketing: z.boolean().default(false),
7 |   consented_at: z.iso.datetime(),
8 | });
9 |
10 | export type CookieConsentFormData = z.infer<typeof cookieConsentSchema>;
11 |

```